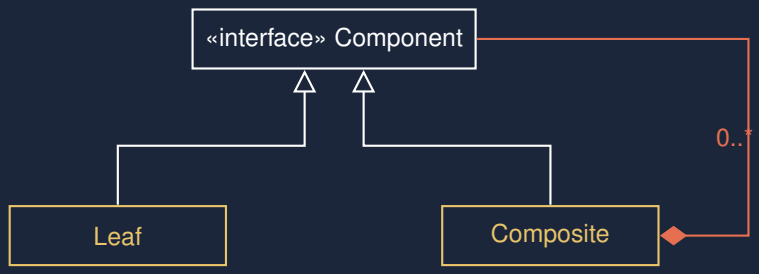
BUET • CSE 213

Software Engineering

Topic-wise term-final questions with complete worked solutions



14	265	132	3
syllabus topics	in-syllabus questions	out-of-syllabus questions	course codes

CSE213 and CSE307 term finals, plus the in-syllabus questions of CSE325 papers (2017–2024), aligned with the lecture slides of IJ Ma'am and AI Sir.

Every question in the topic-wise question bank has been re-typeset and is followed by its solution. Questions outside the 14 syllabus topics are collected, also fully solved, in a separate **Out of Syllabus** part. UML, BPMN, flow-graph and network diagrams are drawn in Ti*k*Z; code is in Java unless the question fixes another language.

Contents

01 Design Principles & Basics of Design Pattern . . . 1

02 Creational Design Patterns . . . 23

03 Structural Design Patterns . . . 52

04 Behavioural Design Patterns . . . 99

05 Class Diagram & State Diagram . . . 137

06 Sequence & Collaboration Diagrams . . . 195

07 Process Models & Agile Development . . . 234

08 Software Testing . . . 281

09 Unit Testing & Code Review . . . 315

10 Code Refactoring . . . 343

11 Software Ethics & Requirement Analysis . . . 369

12 Equivalence Partitioning & Coverage . . . 397

13 Use Case Diagram . . . 420

14 BPMN Diagram . . . 450

PART II • OUT OF SYLLABUS . . . 476

X1 Design Patterns Beyond the Slides . . . 477

X2 Requirements & Information Systems . . . 484

X3 User Interface & Usability . . . 499

X4 DFD, Activity & Decision Modelling . . . 511

X5 Project Planning, Estimation & Risk . . . 526

X6 Security & Dependability . . . 597

X7 Architecture, Performance & DevOps . . . 622

X8 Quality Management & Metrics . . . 658

X9 Other Topics Beyond the Slides . . . 669

How to read this book

2023–2024 • Q2(a) Session • question number | course • exam date • bank ID MARKS 20

A question card. The title line gives the session and the question number in the original paper, then the course code, the exam date and the identifier used in the topic-wise question bank. The coral badge on the right shows the printed marks. Code, tables and figures from the original paper are re-typeset inside the card.

SOLUTION

The solution panel always follows its question. Solutions start from the concept, as defined in the lecture slides, reason step by step to the deliverable (diagram, code, table or calculation) and end with a summary of the key result:

Answer. like this.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

Paper A Q1 MARKS 8

A question that was asked again…

Paper B Q4 MARKS 10

…possibly with slightly different wording, is stacked and solved *once* below.

SOLUTION

One shared solution covers every stacked card.

NOTE Gold boxes flag an issue in the source scan (a cut-off line, a missing value, a misprint) and state which reading or assumption the solution uses.

EXAM TIP Teal boxes give exam tips and common mistakes.

Organisation

- **Part I (chapters 01–14)** follows the syllabus topic list: design principles and patterns, UML modelling, process models, testing, unit testing and code review, refactoring, ethics and requirement analysis, equivalence partitioning and coverage, use cases and BPMN.
- **Part II (chapters X1–X9, plum banners)** holds every question whose central concept is *not* in the syllabus topics or the lecture slides. Examples are design patterns that were not taught (Facade, Proxy, Chain of Responsibility, …), project planning and estimation, security, architecture, DFDs and quality management. These questions are fully solved too.
- **Order inside a chapter:** questions are *not* grouped by sub-topic. They are sorted by session, most

recent first; within the same session the lower course number comes first (CSE213, then CSE307, then CSE325). Practice problems (BPMN) come at the end of their chapter.

- Diagram conventions follow the slides: UML class boxes have three compartments (name, attributes, operations; + public, - private, # protected). A hollow triangle means generalization; a dashed hollow triangle means realization; a hollow diamond means aggregation and a filled diamond means composition.

01

Design Principles & Basics of Design Pattern

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q2(c) CSE307 • 05/05/2024 • D010.1 **MARKS 8**

Show an example of Interface Segregation Principle.

2012–2013 • Q4(b) CSE307 • 17/05/2014 • D034 **MARKS 14 2/3**

What is the primary motivation behind "Interface Segregation Principle"? With a suitable "bad example" explain the "Interface Segregation Principle".

SOLUTION

1. The principle (as stated in the lecture)

Interface Segregation Principle (ISP) — the "I" of SOLID:

> *"Clients shouldn't be forced to depend on methods they do not use."*

Try to make your interfaces *narrow enough* that client classes don't have to implement behaviours they don't need. Instead of one "fat" general-purpose interface, provide several small, *role-specific* interfaces; a class may implement as many of them as it really supports.

2. Primary motivation — why we need ISP

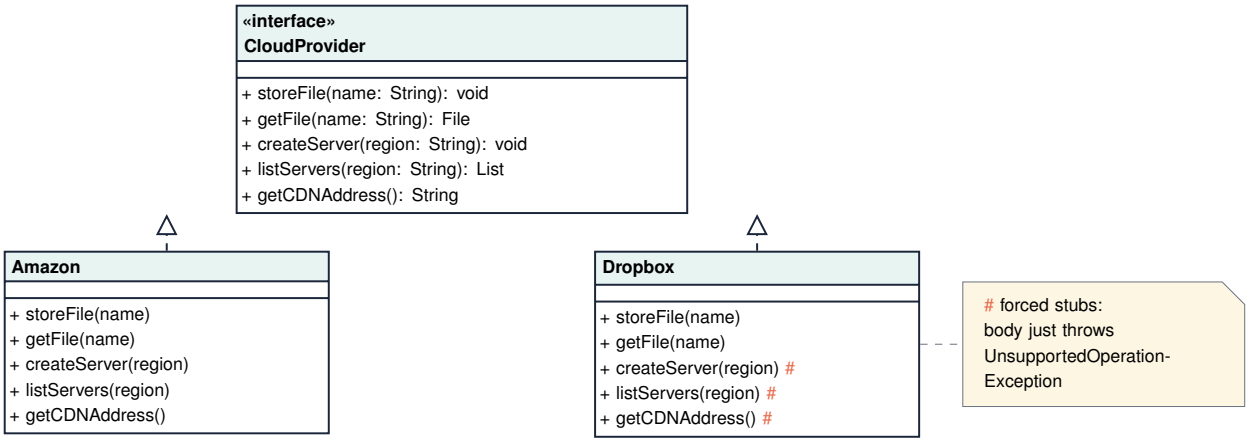
The motivation is to get rid of **fat (polluted) interfaces**, i.e. interfaces that bundle methods needed by different kinds of clients. Such an interface causes:

- **Forced, meaningless implementations.** Every implementer must provide *all* methods, so classes that do not support a feature write empty bodies or `throw new UnsupportedOperationException()`. This is a lie in the type system and usually also breaks the Liskov Substitution Principle (a client calling the method crashes).
- **Unnecessary coupling and ripple effects.** A client depends on the whole interface. When a method it never calls is changed or added, the client (and every implementer) must still be edited, recompiled, retested and redeployed. (Robert C. Martin formulated ISP after exactly this happened at Xerox: one huge `Job` class used by printing, stapling and faxing tasks meant a tiny change forced a rebuild of everything.)
- **Low cohesion of the abstraction.** A fat interface has more than one "reason to change" — ISP is SRP applied to interfaces.

In short, ISP keeps dependencies *minimal*: each client sees only what it actually uses, so changes stay local and implementers are never forced to fake behaviour.

3. A "bad example" — one fat cloud-provider interface

Suppose an application integrates with several cloud providers through one interface. At first only Amazon is supported, so the interface contains *everything* Amazon offers. Later we want to support Dropbox, which only offers file storage.




```java
import java.io.File;
import java.util.List;

interface CloudProvider {                        // FAT interface  (violates ISP)
    void storeFile(String name);
    File getFile(String name);
    void createServer(String region);
    List<String> listServers(String region);
    String getCDNAddress();
}

class Dropbox implements CloudProvider {         // supports storage only ...
    public void storeFile(String name) { System.out.println("Dropbox: upload " + name); }
    public File getFile(String name)   { return new File(name); }
    // ... but is FORCED to implement methods it does not have:
    public void createServer(String region) {
        throw new UnsupportedOperationException();
    }
    public List<String> listServers(String region) {
        throw new UnsupportedOperationException();
    }
    public String getCDNAddress() {
        throw new UnsupportedOperationException();
    }
}
```

What is wrong?

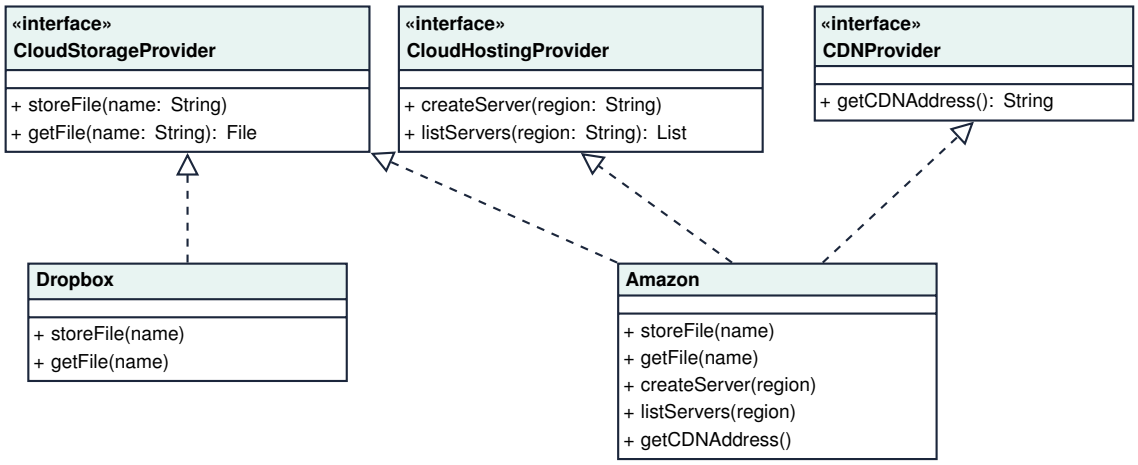
(i) `Dropbox` depends on (and must implement) three methods it does not use — a direct violation of ISP.

(ii) A client holding a `CloudProvider` reference may call `createServer()` on a `Dropbox` object and get a run-time exception, so `Dropbox` is not substitutable for `CloudProvider` (LSP is broken as a side-effect).

(iii) If hosting-related methods change (e.g. a new parameter for `createServer`), `Dropbox` and every storage-only client must be modified and recompiled although they never use hosting.

4. Applying ISP — split the fat interface by client role

Break the interface into narrow, cohesive role interfaces: *storage*, *hosting* and *CDN*. Amazon implements all three; Dropbox implements only the one it really supports.




```
interface CloudStorageProvider {
    void storeFile(String name);
    File getFile(String name);
}
interface CloudHostingProvider {
    void createServer(String region);
    List<String> listServers(String region);
}
interface CDNProvider {
    String getCDNAddress();
}

class Amazon implements CloudStorageProvider, CloudHostingProvider, CDNProvider {
    public void storeFile(String name)             { System.out.println("S3: upload " + name); }
    public File getFile(String name)               { return new File(name); }
    public void createServer(String region)        { System.out.println("EC2 in " + region); }
    public List<String> listServers(String region) { return List.of("ec2-1", "ec2-2"); }
    public String getCDNAddress()                  { return "cdn.amazon.example"; }
}

class Dropbox implements CloudStorageProvider {   // implements ONLY what it supports
    public void storeFile(String name) { System.out.println("Dropbox: upload " + name); }
    public File getFile(String name)   { return new File(name); }
}

class BackupService {                    // a client that needs storage only
    private final CloudStorageProvider storage;
    BackupService(CloudStorageProvider storage) { this.storage = storage; }
    void backup(String file) { storage.storeFile(file); }
}

public class Main {
    public static void main(String[] args) {
        new BackupService(new Dropbox()).backup("report.pdf");  // works, no stubs
        new BackupService(new Amazon()).backup("report.pdf");   // Amazon also fits
        CloudHostingProvider host = new Amazon();               // hosting client
        host.createServer("ap-south-1");
    }
```

```
}
```

5. What we gained

- No class is forced to implement methods it does not use — no dummy/throwing stubs.
- Every client depends only on the narrow role it needs (`BackupService` knows nothing about servers or CDNs), so hosting changes can no longer ripple into storage clients.
- Each object is safely substitutable for every interface it implements (LSP holds again).
- Interfaces are highly cohesive and the coupling between modules is reduced.

EXAM TIP Do not over-apply ISP: splitting interfaces that are already specific just multiplies types. Segregate by *client role* — methods that are always used together stay together.

Answer. ISP: "clients shouldn't be forced to depend on methods they do not use". Motivation: fat interfaces force implementers to write meaningless stubs and couple clients to changes they don't care about. Bad example: a single `CloudProvider` interface forces `Dropbox` to stub out hosting/CDN methods; the fix splits it into `CloudStorageProvider`, `CloudHostingProvider` and `CDNProvider`, with each class implementing only the roles it supports.

2021–2022 • Q1(a) CSE307 • 07/05/2023 • D012 **MARKS 16**

Differentiate the use cases of these following design patterns with necessary examples and/or figures:

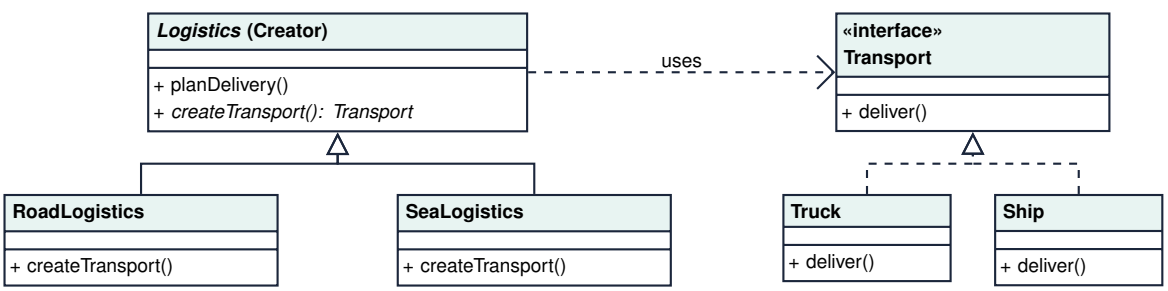
Factory Method Pattern, Abstract Factory Pattern,
Builder Pattern and Bridge Pattern

SOLUTION

Three of these are *creational* patterns (they differ in *what* is created and *how*), while Bridge is a *structural* pattern (it is about how classes are organised, not about creation). Each is discussed below with its intent, when to use it, an example and a figure; a comparison table follows.

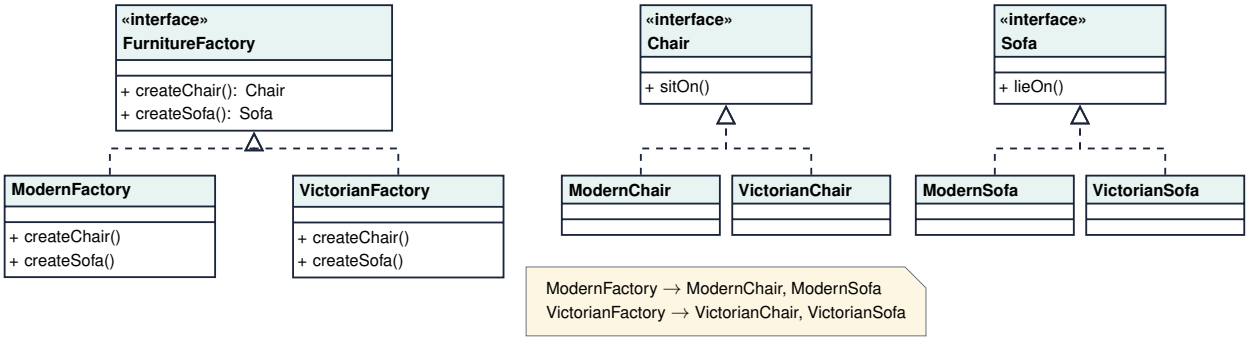
1. Factory Method — one product, the subclass decides which

Intent: "Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses."
Use when: a class cannot anticipate the concrete class of the *single* product it must create; you want users of a framework to extend its internal components; you want to remove `new ConcreteX()` from client code.
Example: a logistics app first handled only trucks; sea transport is added by a new subclass `SeaLogistics` that overrides `createTransport()`.



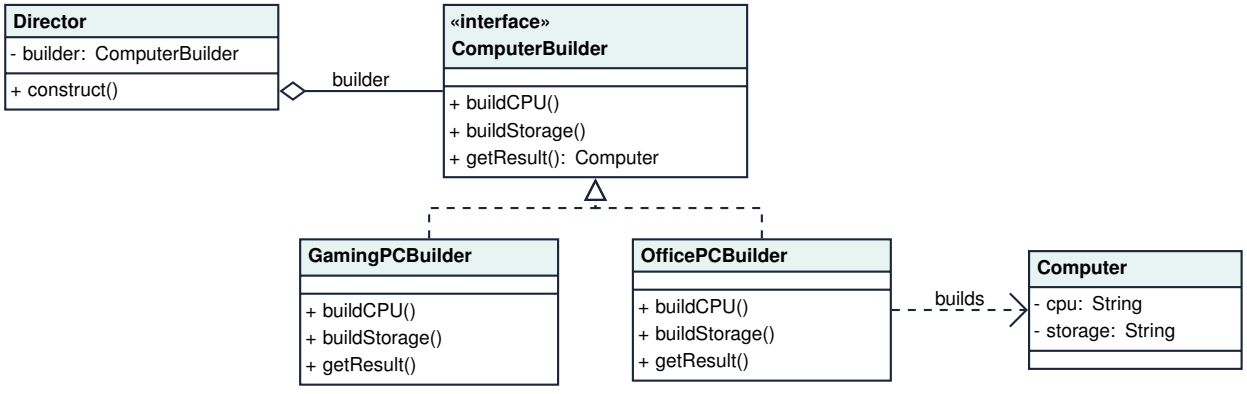
2. Abstract Factory — a family of related products

Intent: "Provide an interface for creating *families* of related or dependent objects without specifying their concrete classes" (a "factory of factories").
Use when: the code must work with several families of related products and the products of one family must be used together (consistency), e.g. multiple look-and-feels, multiple DBMSs, or the furniture shop where a customer must not receive a Modern chair with a Victorian sofa.
Example: `ModernFactory` creates `ModernChair`+`ModernSofa`; `VictorianFactory` creates the Victorian variants. Changing the factory object changes the whole family at once.



3. Builder — one complex product, built step by step

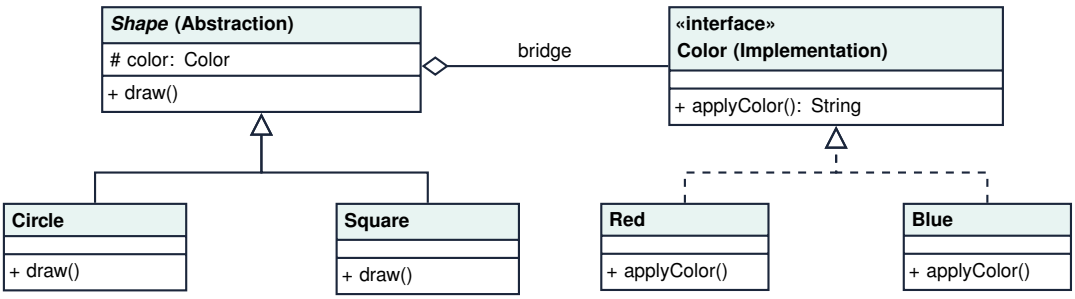
Intent: "Separate the construction of a complex object from its representation so that the same construction process can create different representations."
Use when: an object has many parts/optional settings (avoid "telescoping" constructors), the parts must be assembled in a certain order, or the same construction steps must yield different representations.
Example (lecture): assembling a computer to order — one customer wants a 500 GB disk with an Intel processor, another a 250 GB disk with an AMD processor. The `Director` runs the same steps; each concrete builder produces a different `Computer`.



4. Bridge — two independent dimensions of variation

Intent: "Bridge lets you split a large class or a set of closely related classes into two separate hierarchies, *abstraction* and *implementation*, which can be developed independently of each other."
Use when: a class varies along two orthogonal dimensions and inheritance would make the hierarchy grow exponentially (`RedCircle`, `BlueCircle`, `RedSquare`, ...); you want to switch implementations at run time; e.g. the same GUI (abstraction) running on Linux/Windows/macOS APIs

(implementation).
Example (lecture): `Shape` (form) *has-a* `Color`; m shapes + n colours need $m + n$ classes instead of $m \times n$.



5. Client-side view of the four patterns

```
// Factory Method: subclass decides the ONE product
Logistics lg = new SeaLogistics();
lg.planDelivery();                              // internally: createTransport() -> Ship

// Abstract Factory: one factory object = one consistent FAMILY
FurnitureFactory f = new VictorianFactory();
Chair c = f.createChair();  Sofa s = f.createSofa();   // guaranteed to match

// Builder: same steps, different representation, product returned at the END
Director d = new Director(new GamingPCBuilder());
Computer pc = d.construct();                    // buildCPU(); buildStorage(); getResult()

// Bridge: combine any abstraction with any implementation at run time
Shape shape = new Circle(new Red());
shape.draw();                                   // Circle delegates colouring to Red
```

6. Differentiating the use cases

	Factory Method	**Abstract Factory**	**Builder**	**Bridge**
Category	Creational	Creational	Creational	Structural
Problem	Which concrete class of *one* product to create	Create *families* of related products that must match	Construct a *complex* object step by step	A class that varies in *two* independent dimensions
Mechanism	Inheritance: subclass overrides a creation method	Composition: client holds a factory object with several creation methods	Director drives a builder through ordered steps	Abstraction holds a reference to an implementation object
Result returned	Immediately, one object	Immediately, one object per call (all of one family)	At the end (`getResult()`)	Not about creation
Adding new variant	New creator subclass	New concrete factory (new product type is hard)	New concrete builder	New subclass on either side independently
Lecture example	Logistics / shapes	Furniture, OS-specific UI, Honda car factory	Computer assembly, HTML page	Shape × Color, GUI × OS

EXAM TIP Quick rule: *one* product → Factory Method; a *matching family* → Abstract Factory; a *complex* object assembled in steps → Builder; *two dimensions* of variation of an existing class structure → Bridge. They also combine: an Abstract Factory is often implemented with factory methods, and an Abstract Factory can create the matching implementation object of a Bridge.

Answer. Factory Method lets subclasses choose the class of a single product; Abstract Factory creates consistent families of related products through one factory object; Builder separates the step-by-step construction of a complex object from its representation; Bridge (structural) splits an abstraction from its implementation so both hierarchies vary independently and avoids a class explosion.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2020–2021 • Q5(a) CSE325 • 12/04/2023 • C325-2021-5a **MARKS 15**

Class A has a public method which is needed by class B. If class B inherits class A, class B gets access to that method. On the other hand, passing an instance of class A to class B also enables class B to access that method. How do you decide which technique to use? Explain your reasoning.

2010–2011 • Q3(c) CSE307 • 19/03/2012 • D037 **MARKS 5**

The ordinary way to extend a software system is to find a suitable superclass, and to subclass it. You can also "inherit" behavior from more than one class by copying some of the operations of the second class and delegating calls to an instance of that class. Both of these extension techniques, however, require that you know at compile time what behavior you want to add.

Name these two techniques that we just described. Should one of these techniques require a preferential treatment?

SOLUTION

The two techniques

- **Inheritance** (`class B extends A`) — an *is-a* relationship; B reuses A's code *white-box*, at compile time, and B's objects can be used wherever an A is expected.
- **Composition / delegation** (B holds a reference to an A, passed in the constructor or a method) — a *has-a / uses-a* relationship; B reuses A *black-box* by calling it, and the A can be replaced at run time.

The design-principles lecture (*Dive Into Design Patterns*, ch. 3) gives the rule **Favour composition over inheritance**; inheritance is chosen only when the subclass is a genuine subtype.

Decision criteria

Question to ask	Inheritance if …	Composition if …
Is B *really* a kind of A?	B is-a A in the domain (`SavingsAccount` is an `Account`)	B merely *needs a service* of A (`ReportPrinter` needs a `Formatter`)
Liskov substitution	every B can replace an A without breaking clients	B would have to disable/override parts of A (*Refused Bequest*)
Does B need *all* of A's interface?	yes, the whole public interface of A makes sense for B	no — inheriting would expose unrelated public methods of A through B
Must the behaviour change at run time / be swapped in tests?	no, it is fixed	yes — pass another implementation (or a mock) of A's interface
Coupling	acceptable to depend on A's implementation details (protected members, order of calls)	B should depend only on A's public contract (better encapsulation, no *fragile base class*)
Reuse from several classes	Java allows only one superclass	B can hold references to any number of collaborators

Example – the wrong and the right choice

`Stack` needs `add()` and `remove()` of a list. Inheriting from `ArrayList` gives it the method, but also exposes `add(index, x)`, `get(i)`, ... so a client could insert into the middle of the stack; a stack is *not* a list.

```
// Inheritance (bad here): Stack "is-a" ArrayList -> exposes list operations
class Stack1<T> extends java.util.ArrayList<T> {
    void push(T x) { add(x); }
    T pop()        { return remove(size() - 1); }
}                       // client can still call stack.add(0, x)  -> breaks LIFO

// Composition (good): Stack "has-a" List and uses only what it needs
class Stack2<T> {
    private final java.util.List<T> items;
    Stack2(java.util.List<T> items) { this.items = items; }   // A passed to B
    void push(T x) { items.add(x); }
    T pop()        { return items.remove(items.size() - 1); }
}
```

In contrast, `class SavingsAccount extends Account` is right: every savings account *is* an account and can be used wherever an `Account` is expected, and it reuses `deposit()/getBalance()` unchanged.

Reasoning summary

(i) If the only motive is "B needs a method of A", use **composition** — that is a uses-a relationship.

(ii) Use **inheritance** only if B is-a A, B can substitute A (LSP) and B needs A's whole interface; then you also get polymorphism.

(iii) Prefer passing A as an *interface type* (`B(AInterface a)`): B depends on an abstraction (DIP), A can be replaced at run time or by a test double, and changes inside A do not ripple into B.

Answer. Decide by the relationship: is-a and substitutable (LSP) ⇒ inheritance; otherwise (B only uses A's service) ⇒ pass an instance (composition/delegation). Favour composition: looser coupling, no unwanted inherited interface, behaviour replaceable at run time, no single-inheritance limit, no fragile base class.

Additionally for 2010–2011 Q3(c) (D037): naming the two techniques and which one is preferred

(i) **Inheritance** (subclassing / class inheritance): the new class `extends` a superclass and reuses or overrides its behaviour — "white-box" reuse, fixed at compile time.

(ii) **Composition with delegation** (object composition / aggregation): the class holds a reference to an instance of the second class, offers the same operations and *delegates* the calls to it — "black-box" reuse; this is how "multiple inheritance" of behaviour is simulated in Java.

```
class Logger { void log(String m) { System.out.println(m); } }
class Repository { void save(String r) { /* ... */ } }

class AuditedRepository extends Repository {     // (1) inheritance from Repository
    private final Logger logger = new Logger();   // (2) composition ...
    void log(String m) { logger.log(m); }         //     ... + delegation to a Logger
    @Override void save(String r) { log("saving " + r); super.save(r); }
}
```

Yes, one deserves preferential treatment: **favour object composition (delegation) over class inheritance** ("Dive into Design Patterns", Ch. 3, and the Decorator lecture where "inheritance didn't work well"). Besides the reasons in the table above:

- Extending along several dimensions with inheritance causes a *combinatorial explosion* of subclasses (e.g. every beverage × every condiment), and multiple inheritance brings the *diamond problem*.
- With composition the delegate is accessed through an interface, so it can be *replaced at run time* (inject a different object); this removes the "must know at compile time" limitation mentioned in the question — the idea behind Decorator, Strategy, Bridge and State.

Inheritance is still appropriate for a genuine, stable "is-a" relationship (e.g. in Template Method), but composition should be the first choice.

Answer. The techniques are **inheritance (subclassing)** and **object composition with delegation**. Composition should be preferred ("favour composition over inheritance"): it is loosely coupled, keeps encapsulation and allows behaviour to be combined/changed at run time.

2020–2021 • Q5(b) CSE325 • 12/04/2023 • C325-2021-5b **MARKS 10**

Give an example of "Evil switch" anti-pattern. Incorporation of which design principle is most likely to avoid "Evil switch"?

SOLUTION

What the "evil switch" is

A **switch (or if–else chain) on a type code** that selects behaviour, typically *repeated* in several methods. Every time a new type is added, every such switch must be found and modified — the class is never closed for modification, and forgetting one switch causes a bug (the *Switch Statements* smell in the refactoring lecture).

Example

```
class Employee {
    static final int ENGINEER = 0, MANAGER = 1, SALESMAN = 2;
    int type; double base, bonus, commission;

    double pay() {
        switch (type) {
            case ENGINEER: return base;
            case MANAGER:  return base + bonus;
            case SALESMAN: return base + commission;
            default: throw new IllegalStateException();
        }
    }
    String title() {                // the same switch again ...
        switch (type) {
            case ENGINEER: return "Engineer";
            case MANAGER:  return "Manager";
            default:       return "Salesman";
        }
    }
}
```

Adding an `Intern` type requires editing `pay()`, `title()` and every other switch on `type`.

Principle that avoids it: Open/Closed Principle (via polymorphism)

"Classes should be open for extension but closed for modification." Put the varying behaviour behind an abstraction and let each type implement it (refactoring **Replace Conditional with Polymorphism**); the lecture's `AreaCalculator` example (Rectangle/Circle/Ellipse) is exactly this transformation.

```
abstract class Employee {
    protected double base;
    abstract double pay();
    abstract String title();
}
class Engineer extends Employee {
    double pay()   { return base; }
    String title() { return "Engineer"; }
}
class Manager extends Employee {
    double bonus;
    double pay()   { return base + bonus; }
    String title() { return "Manager"; }
}
```

```java
class Salesman extends Employee {
    double commission;
    double pay()   { return base + commission; }
    String title() { return "Salesman"; }
}
// Adding Intern = one new subclass; no existing class is modified.
```

Answer. Evil switch = a type-code switch that chooses behaviour and is duplicated across methods (e.g. `pay()` and `title()` switching on employee type). The Open/Closed Principle (realised by polymorphism / Replace Conditional with Polymorphism, or the Strategy/State patterns) removes it: new types are added by extension, not by editing switches.

> **NOTE** A single switch inside a *factory* that maps input to the right subclass is acceptable — it is the one place where the type is decided.

2020–2021 • Q6(a) CSE325 • 12/04/2023 • C325-2021-6a **MARKS 25**

State the basic idea of each of five SOLID principles. Explain them with appropriate code examples.

SOLUTION

SOLID

SOLID is a mnemonic for five design principles intended to make software designs more understandable, flexible and maintainable: **S**ingle responsibility, **O**pen/closed, **L**iskov substitution, **I**nterface segregation and **D**ependency inversion.

1. Single Responsibility Principle (SRP)

A class should have just one reason to change; its single responsibility should be entirely encapsulated by the class. Benefits: high cohesion, low coupling, easier testing and debugging.

```java
// Violation: three reasons to change (salary rule, report format, database)
class Employee {
    double calculatePay()  { /* payroll rules  */ return 0; }
    String reportHours()   { /* HR format      */ return ""; }
    void save()            { /* SQL            */ }
}
// SRP: one responsibility per class
class Employee         { String id; double rate; int hours; }
class PayCalculator    { double pay(Employee e) { return e.rate * e.hours; } }
class HourReporter     { String report(Employee e) { return e.id + ": " + e.hours + "h"; } }
class EmployeeRepository { void save(Employee e) { /* SQL */ } }
```

2. Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification: new behaviour is added by new code, not by editing working code. (Lecture example: area calculator.)

```java
// Violation: every new shape forces a change here
double area(Object[] shapes) {
    double a = 0;
    for (Object s : shapes)
        if (s instanceof Rectangle r) a += r.w * r.h;
        else if (s instanceof Circle c) a += Math.PI * c.r * c.r;
    return a;
```

```
}
// OCP: extend by adding a Shape subclass; AreaCalculator never changes
abstract class Shape { abstract double area(); }
class Rectangle extends Shape { double w, h; double area() { return w * h; } }
class Circle    extends Shape { double r;    double area() { return Math.PI * r * r; } }
class Ellipse   extends Shape { double a, b; double area() { return Math.PI * a * b; } }
class AreaCalculator {
    double area(Shape[] shapes) { double t = 0; for (Shape s : shapes) t += s.area(); return t; }

}
```

3. Liskov Substitution Principle (LSP)

Objects of a subclass must be usable in place of objects of the superclass without breaking the client; a subclass should extend the base behaviour, not replace or weaken it (no stronger pre-conditions, no weaker post-conditions, no new exceptions).

```
// Violation: Square changes the meaning of setWidth/setHeight
class Rectangle { protected int w, h;
    void setWidth(int w)  { this.w = w; }
    void setHeight(int h) { this.h = h; }
    int area() { return w * h; } }
class Square extends Rectangle {
    void setWidth(int w)  { this.w = this.h = w; }
    void setHeight(int h) { this.w = this.h = h; } }
void client(Rectangle r) { r.setWidth(5); r.setHeight(4); assert r.area() == 20; } // fails for
    Square

// LSP-compliant: both are Shapes; neither promises independent width/height setters
interface Shape { int area(); }
record Rect(int w, int h) implements Shape { public int area() { return w * h; } }
record Sq(int side)       implements Shape { public int area() { return side * side; } }
```

4. Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use; make interfaces narrow (role interfaces).

```
// Violation: a simple printer must implement scan() and fax()
interface Machine { void print(Doc d); void scan(Doc d); void fax(Doc d); }
class SimplePrinter implements Machine {
    public void print(Doc d) { /* ok */ }
    public void scan(Doc d)  { throw new UnsupportedOperationException(); }
    public void fax(Doc d)   { throw new UnsupportedOperationException(); }
}
// ISP: small interfaces, combined only where needed
interface Printer { void print(Doc d); }
interface Scanner { void scan(Doc d); }
interface Fax     { void fax(Doc d); }
class SimplePrinter2 implements Printer { public void print(Doc d) { } }
class OfficeMachine implements Printer, Scanner, Fax {
    public void print(Doc d) { } public void scan(Doc d) { } public void fax(Doc d) { } }
```

5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions. (Lecture example: switch and light bulb.)

```
// Violation: the high-level switch is welded to the concrete LightBulb
```

```
class LightBulb { void turnOn() { } void turnOff() { } }
class PowerSwitch1 { private final LightBulb bulb = new LightBulb(); /* press() uses bulb */ }

// DIP: both sides depend on the abstraction Switchable
interface Switchable { void turnOn(); void turnOff(); }
class Bulb implements Switchable { public void turnOn() { } public void turnOff() { } }
class Fan  implements Switchable { public void turnOn() { } public void turnOff() { } }
class PowerSwitch {
    private final Switchable client; private boolean on;
    PowerSwitch(Switchable client) { this.client = client; }
    void press() { if (on) client.turnOff(); else client.turnOn(); on = !on; }
}
```

Answer. SRP: one reason to change per class. OCP: extend behaviour without modifying existing classes (abstraction + polymorphism). LSP: subclasses must be substitutable for their base class without breaking clients. ISP: many small client-specific interfaces instead of one fat interface. DIP: high- and low-level modules both depend on abstractions, not on each other's concrete classes.

2020–2021 • Q6(b) CSE325 • 12/04/2023 • C325-2021-6b MARKS 10

What is the difference between dependency inversion and dependency injection? Explain with an appropriate code example where we need to separate an object's creation from its use.

SOLUTION

Definitions

- **Dependency Inversion (DIP)** is a *design principle* about the **direction of dependencies**: high-level policy code and low-level details must both depend on an abstraction (an interface owned by the high-level side). It says *what* to depend on.
- **Dependency Injection (DI)** is a *technique* about **who creates and supplies** a dependency: the object does not `new` its collaborators; they are handed in from outside (constructor, setter or method parameter, often by a factory/"composition root" or a DI container). It says *how* the object gets its dependency.

They are complementary: DI is the usual way to *achieve* DIP at run time, but DI can also inject a concrete class (no inversion), and DIP can be followed with a factory instead of injection.

Example – order service needs a payment gateway

```
// Before: creation mixed with use -> high-level depends on a concrete low-level class
class OrderService {
    private final BkashGateway gateway = new BkashGateway("api-key");   // creates it itself
    void checkout(Order o) { gateway.charge(o.total()); }
}   // cannot switch to card payment or test without calling bKash
```

```
// After: DIP (depend on an abstraction) + DI (dependency passed in)
interface PaymentGateway { void charge(double amount); }              // abstraction

class BkashGateway implements PaymentGateway {                        // detail
    BkashGateway(String key) { }
    public void charge(double amount) { /* call bKash API */ }
}
class CardGateway implements PaymentGateway {
    public void charge(double amount) { /* call card processor */ }
```

```
}
class OrderService {                                                    // high-level policy
    private final PaymentGateway gateway;
    OrderService(PaymentGateway gateway) { this.gateway = gateway; }   // constructor injection
    void checkout(Order o) { gateway.charge(o.total()); }              // only USES it
}
public class App {                                                      // composition root:
    only CREATES
    public static void main(String[] args) {
        PaymentGateway g = args.length > 0 && args[0].equals("card")
                         ? new CardGateway() : new BkashGateway(System.getenv("BKASH_KEY"));
        new OrderService(g).checkout(new Order(1500));
    }
}
// Unit test: new OrderService(amount -> recorded.add(amount)) -- a fake, no network
```

Why creation must be separated from use here

- The concrete gateway depends on configuration (API keys, environment) that `OrderService` should not know.
- The gateway must be *replaceable* (bKash, card, a test fake) without editing `OrderService` (OCP).
- `OrderService` now has one responsibility (checkout logic); construction lives in one place (`App` or a factory).

Answer. DIP is a principle: depend on abstractions, not concretions (direction of the dependency). DI is a technique: supply dependencies from outside instead of creating them inside (who builds the object). Example: `OrderService(PaymentGateway g)` uses an injected abstraction, while `App` (composition root) decides and creates `BkashGateway`/`CardGateway`.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2018–2019 • Q3(c) CSE307 • 06/11/2019 • D021.2 MARKS 8

Discuss cohesion and coupling in brief in the context of software engineering.

2017–2018 • Q7(c) CSE307 • 29/09/2018 • D025.2 MARKS 8

What is *tight cohesion* and *loose coupling*? How does it influence software design?

SOLUTION

1. Cohesion

Cohesion is the degree to which the elements *inside* one module (method, class, package) belong together, i.e. how strongly they are focused on a single, well-defined task.

- **Tight (high) cohesion**: everything in the module contributes to one responsibility — e.g. a `TaxCalculator` class that only computes tax. This is what the *Single Responsibility Principle* asks for ("a class should have just one reason to change"), and the lecture lists "high cohesion" as a direct benefit of SRP.
- **Low cohesion**: unrelated duties are lumped together — e.g. a `Utility` class that formats dates,

sends e-mail and computes salaries.

Levels of cohesion (worst → best): coincidental, logical, temporal, procedural, communicational, sequential, **functional** (the module performs exactly one function).

2. Coupling

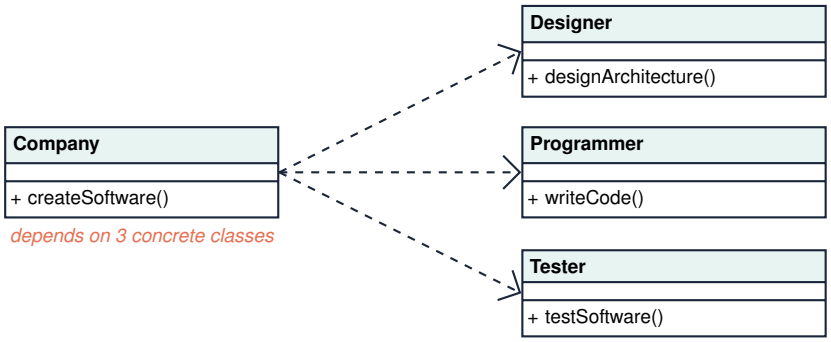
Coupling is the degree of interdependence *between* modules — how much one module must know about, and is affected by, another.

- **Tight coupling**: a class directly creates and uses concrete classes, reads their internals, or shares global data; a change in one forces changes in the other.
- **Loose coupling**: modules interact only through small, stable interfaces (abstractions) and simple data parameters, so each can change independently.

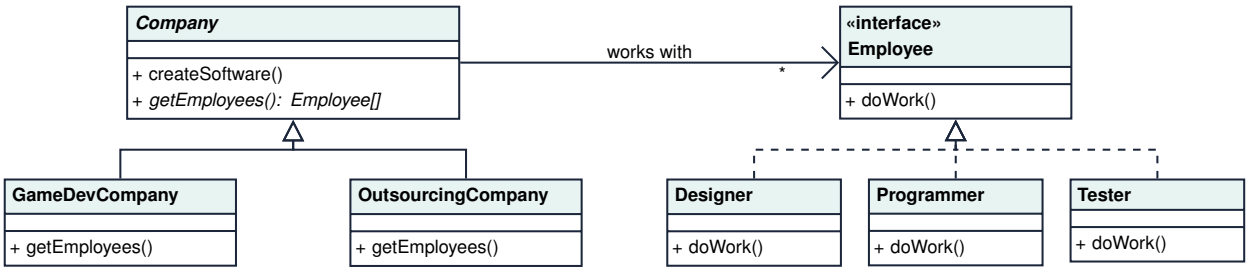
Levels of coupling (worst → best): content, common (global data), external, control, stamp, **data** coupling (only the needed elementary data is passed).

3. Example from the lecture (tight → loose coupling)

Before. `Company.createSoftware()` itself instantiates `Designer`, `Programmer` and `Tester` and calls their specific methods — `Company` is tightly coupled to every concrete employee class.



After. All employees implement a common `Employee` interface; `Company` works only through that abstraction and the choice of concrete employees is pushed into subclasses (`getEmployees()`). "The `Company` class has become independent from various employee classes" — new companies and employees can be added without breaking existing code.




```
interface Employee { void doWork(); }

abstract class Company {
    protected abstract Employee[] getEmployees();   // concrete choice pushed to subclasses
    public void createSoftware() {
        for (Employee e : getEmployees()) e.doWork(); // knows ONLY the interface
    }
}
```

4. How tight cohesion and loose coupling influence software design

Quality	Effect of high cohesion + low coupling
Maintainability	A change is confined to one focused module; no ripple effect through the system.
Understandability	Each module has one clear purpose and few, explicit dependencies.
Reusability	A cohesive, loosely coupled class can be lifted into another system without dragging others along.
Testability	Modules can be unit-tested in isolation; dependencies on interfaces are easily replaced by stubs/mocks.
Extensibility	New behaviour is plugged in behind an interface (supports OCP and DIP).
Team work	Independent modules can be developed and replaced in parallel.

The two goals reinforce each other: when every module does one thing well (high cohesion), it needs fewer collaborators and exposes a narrower interface (low coupling). Design principles from the lecture are tools to reach them: *encapsulate what varies*, *program to an interface, not an implementation*, SRP and ISP (cohesion), DIP (coupling).

Answer. Cohesion = how strongly the parts *within* a module belong to one task; coupling = how strongly modules depend *on each other*. Good design aims at **tight (high) cohesion and loose (low) coupling**, which localises change and makes software easier to understand, test, reuse and extend.

2016–2017 • Q8(d) CSE307 • 15/07/2017 • D029 MARKS $6\frac{2}{3}$

Which class design principle does the code in figure 8(d) violates? Re-write the code to remove the violation of the principle.

```
class GraphicEditor{
    public float getArea(Shape s){
        if(s.m_type == 1)return s.width*s.width;
        else if(s.m_type == 2)return 3.1416*s.r*s.r;
    }
}
class Shape{
    int m_type;
}
class Rectangle extends Shape{
    int width;
    Rectangle(int width){
        super.m_type = 1;
        this.width = width;
    }
}
class Circle extends Shape{
    int r;
    Circle(int radius){
        super.m_type = 1;
        this.r = radius;
    }
}
```

figure for question 8(d)

SOLUTION

1. Principle violated: the Open/Closed Principle (OCP)

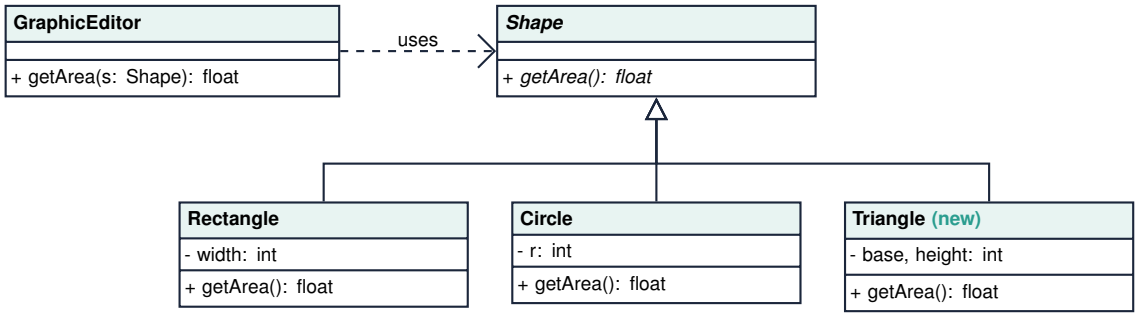
OCP: *"Classes should be open for extension but closed for modification."* Once a class is completed and tested, new or changed features should be added by creating new classes, not by editing the existing one.
`GraphicEditor.getArea()` decides what to do by checking a *type code* (`m_type`) with an `if-else` chain. Therefore:

- Adding a new shape (triangle, ellipse, ...) forces us to **modify** `GraphicEditor` (one more `else if`) and to invent a new type number — the class is not closed for modification.
- `GraphicEditor` must know the internal data of every shape (`width`, `r`), so it is tightly coupled to all subclasses; the area logic is not encapsulated in the class that owns the data (this is exactly the `AreaCalculator` example of the OCP lecture).
- The design is fragile: the figure even sets `m_type = 1` for `Circle`, so a circle would silently be measured as a rectangle — a typical bug of type-code designs.

> **NOTE** As printed, the code would not even compile (`s.width/s.r` are not members of `Shape`, a `float` method returns a `double`, and there is no final `return`). `Circle` is printed with `m_type = 1`; the intended value is clearly 2. These slips do not change the answer — the principle violated is OCP.

2. Re-designed structure

Replace the type code by polymorphism: `Shape` becomes abstract with an abstract `getArea()`, each subclass computes its own area, and `GraphicEditor` simply delegates.



3. Re-written code

```
abstract class Shape {
    public abstract float getArea();          // every shape knows its own area
}

class Rectangle extends Shape {
    private final int width;
    Rectangle(int width) { this.width = width; }
    @Override public float getArea() { return width * width; }   // formula of the original
}

class Circle extends Shape {
    private final int r;
    Circle(int radius) { this.r = radius; }
    @Override public float getArea() { return 3.1416f * r * r; }
}
```

```java
class GraphicEditor {                          // CLOSED for modification
    public float getArea(Shape s) { return s.getArea(); }   // no type checks at all
}

// OPEN for extension: a new shape is a new class; GraphicEditor is untouched
class Triangle extends Shape {
    private final int base, height;
    Triangle(int base, int height) { this.base = base; this.height = height; }
    @Override public float getArea() { return 0.5f * base * height; }
}

public class Demo {
    public static void main(String[] args) {
        GraphicEditor editor = new GraphicEditor();
        Shape[] shapes = { new Rectangle(4), new Circle(2), new Triangle(3, 6) };
        for (Shape s : shapes) System.out.println(editor.getArea(s));
    }
}
```

The type field `m_type` disappears completely; adding `Triangle` (or `Ellipse`, ...) requires *zero* changes in `GraphicEditor` or in the existing shapes.

Answer. The code violates the **Open/Closed Principle**: `GraphicEditor` switches on a type code, so every new shape forces a modification. Fix: make `Shape` abstract with `getArea()`, override it in `Rectangle`/`Circle`, and let `GraphicEditor` just call `s.getArea()`.

2014–2015 • Q1(c) CSE307 • 04/02/2016 • D030 **MARKS 7**

Describe the significance of token, glue token and super glue token to define cohesion and coupling.

SOLUTION

NOTE Token-based cohesion metrics are not in the lecture slides; they come from Bieman and Ott's *functional cohesion* measures (1994), which this question refers to. The solution explains them in terms of the cohesion/coupling concepts of the course.

1. Definitions

To *measure* (not just describe) cohesion, a module is examined at the level of its data:

- **Data token**: any occurrence (definition or use) of a variable or constant in the module. Every occurrence is a separate token (a_1, a_2, ...).
- **Data slice**: for each *output* of the module, the set of tokens that can affect that output (a program slice traced backwards from the output).
- **Glue token**: a token that lies on *more than one* data slice — it "glues" two or more outputs together.
- **Super-glue token**: a token that lies on *every* data slice — it binds all outputs of the module together.

2. Cohesion metrics built from them

With T = all tokens, G = glue tokens, SG = super-glue tokens and k slices:

$$\text{SFC} = \frac{|SG|}{|T|}, \qquad \text{WFC} = \frac{|G|}{|T|}, \qquad \text{A} = \frac{\sum_{t \in G}(\text{number of slices containing } t)}{|T| \cdot k},$$

$$0 \le \text{SFC} \le \text{A} \le \text{WFC} \le 1.$$

SFC = strong functional cohesion, WFC = weak functional cohesion, A = adhesiveness.

3. Worked example

```
1  procedure Calc(a, b : integer; var s, d, p : integer)
2  begin
3     s := a + b;
4     d := a - b;
5     p := a * 2;
6  end
```

Tokens: line 1 $\{a_1, b_1, s_1, d_1, p_1\}$, line 3 $\{s_2, a_2, b_2\}$, line 4 $\{d_2, a_3, b_3\}$, line 5 $\{p_2, a_4, 2\} \Rightarrow |T| = 14$; three outputs $\Rightarrow k = 3$.

Slice	Tokens on the slice
S_s	a_1, b_1, s_1, s_2, a_2, b_2
S_d	a_1, b_1, d_1, d_2, a_3, b_3
S_p	a_1, p_1, p_2, a_4, 2
Glue tokens	a_1 (3 slices), b_1 (2 slices) $\Rightarrow |G| = 2$
Super-glue	a_1 only $\Rightarrow |SG| = 1$

$$\text{SFC} = \tfrac{1}{14} = 0.07, \qquad \text{WFC} = \tfrac{2}{14} = 0.14, \qquad \text{A} = \tfrac{3+2}{14\times 3} = \tfrac{5}{42} = 0.12.$$

The values are very low: the three computations share almost nothing, so `Calc` really holds three independent functions (low cohesion). Splitting it into `sum(a,b)`, `diff(a,b)` and `twice(a)` gives modules with one slice each, where every token is super-glue and SFC = WFC = A = 1 (maximal, *functional* cohesion).

4. Significance

- **Tokens** make cohesion *measurable*: they are the atomic units on which the relationships among a module's computations are counted.
- **Glue tokens** show which outputs share data processing; their share (WFC) and how many slices they connect (adhesiveness) indicate how tightly the module hangs together.
- **Super-glue tokens** show data processing common to *all* outputs; a high SFC means the module performs one single function — the ideal *functional cohesion*. Few or no super-glue tokens signal a module that should be split.
- **Coupling**: the same idea applied *across* module boundaries. Tokens that must flow between modules (parameters, shared/global variables) are the "glue" that ties modules together; the more such shared tokens, the tighter the coupling. Good design therefore wants many glue/super-glue tokens *inside* a module (high cohesion) and few tokens crossing *between* modules (low coupling).

Answer. A data token is one occurrence of a variable/constant; a glue token lies on two or more output slices; a super-glue token lies on all of them. Their proportions give strong/weak functional cohesion and adhesiveness — many super-glue tokens $\Rightarrow$ high (functional) cohesion, while tokens shared between modules indicate coupling.

2014–2015 • Q6(a) CSE307 • 04/02/2016 • D031.1 MARKS 10

What is the difference between Structural and Behavioral patterns? Provide criteria for determining if a new pattern should be considered Structural or Behavioral. The criteria must also explain why the current patterns are classified as they are.

SOLUTION

1. The GoF classification

The 23 Gang-of-Four patterns are classified by *purpose*: **creational** (how objects are instantiated), **structural** (how objects/classes can be combined) and **behavioral** (how objects communicate).

2. Difference between structural and behavioral patterns

Aspect	Structural	Behavioral
Main concern	How classes and objects are *composed* into larger structures	How objects *interact* and how *responsibilities / algorithms* are distributed among them
Viewpoint	Static: the shape of the relationships (wrapping, trees, two hierarchies)	Dynamic: run-time flow of control and messages
Typical question answered	"How do I connect / assemble these parts?"	"Who does what, and how do they talk?"
Typical result	New functionality or interface obtained by combining existing objects	Behaviour that can vary or be changed at run time; decoupled sender and receiver
Examples (taught)	Adapter, Bridge, Composite, Decorator (also Facade, Flyweight, Proxy)	Strategy, Template Method, Observer, Command, Mediator, State (and 5 more)

3. Criteria for classifying a new pattern

Look at the pattern's **intent** (the problem it primarily solves), not at the shape of its class diagram:

(i) **Structural** if its primary aim is to *compose* classes/objects — to adapt an interface, to wrap an object, to build a part–whole hierarchy, to separate two dimensions into linked hierarchies, or to simplify/share structure — and the participants' algorithms stay unchanged.

(ii) **Behavioral** if its primary aim is to *organise communication* or to *encapsulate behaviour* — an algorithm, a request, a state-dependent behaviour, a notification mechanism, a traversal — so that it can vary independently or be selected at run time.

(iii) Secondary check (scope): *class* patterns use inheritance (fixed at compile time), *object* patterns use composition (changeable at run time); this does not decide the purpose but helps describe it.

(iv) If two patterns have the same structure (e.g. Bridge vs State/Strategy), the intent decides: Bridge's goal is to *split a structure* into two independently extendable hierarchies, whereas State/Strategy aim to *vary behaviour*.

4. Why the current patterns are classified as they are

Pattern	Class	Reason (intent)
Adapter	S	converts an interface by wrapping an object — pure composition, no new behaviour
Bridge	S	splits a class into abstraction + implementation hierarchies joined by composition
Composite	S	builds tree (part–whole) structures that are treated uniformly
Decorator	S	wraps objects in wrappers of the same interface (changes the "skin")
Strategy	B	encapsulates interchangeable algorithms, chosen at run time (changes the "guts")
Template Method	B	fixes the skeleton of an algorithm, subclasses redefine steps
Observer	B	one-to-many notification protocol between objects
Command	B	turns a request into an object (queue, log, undo)
Mediator	B	centralises the communication among colleague objects
State	B	object's behaviour changes when its internal state changes

Note Decorator vs Strategy: both use composition, but Decorator is structural because its essence is the *wrapping structure*, while Strategy is behavioral because its essence is the *exchangeable algorithm*.

Answer. Structural patterns deal with how classes/objects are *composed* into larger structures; behavioral patterns deal with how objects *communicate* and how algorithms/responsibilities are assigned. Criterion: classify a new pattern by its primary intent — composing structure ⇒ structural; encapsulating/varying behaviour or interaction ⇒ behavioral.

2011–2012 • Q4(c) CSE307 • 23/09/2013 • D036 **MARKS 8**

What are the four essential elements of a Design Pattern? Discuss them briefly.

SOLUTION

1. What a design pattern is

A design pattern is "a standard solution to a common programming problem" — a description or template for how to solve a problem that can be used in many different situations. To be reusable, every pattern is documented with the same four essential elements.

2. The four essential elements

(i) **Intent / Pattern name.** A short name ("Observer", "Builder") plus the design goal of the pattern in one or two sentences. The name raises the vocabulary of designers: a whole design idea can be communicated with one word. (GoF lists this element as the *pattern name*; the lecture calls it the *intent*.)

(ii) **Problem.** Describes *when* to apply the pattern: the recurring design problem, its context and the forces/conditions that must hold (e.g. "many objects must be notified when one object changes, without tight coupling").

(iii) **Solution.** The elements (classes, interfaces, objects) that make up the design, their responsibilities, relationships and collaborations — an abstract template (usually a class diagram), not a concrete implementation, so it can be adapted to many situations.

(iv) **Consequences.** The results of applying the pattern: its benefits and its trade-offs (flexibil-

ity, extensibility, performance, complexity, ...). They are essential for choosing between alternatives.

3. Example: the four elements of the Observer pattern

Element	Observer
Intent / name	Observer — define a one-to-many dependency so that when one object changes state, all its dependents are notified and updated automatically.
Problem	A blog/news source must inform a changing set of subscribers without knowing their concrete classes.
Solution	`Subject` keeps a list of `Observer` interfaces with `attach()/detach()/notify()`; concrete observers implement `update()`.
Consequences	+ loose coupling between subject and observers, observers added at run time; − unexpected update cascades, notification order not guaranteed.

Answer. Intent (pattern name), Problem (when to apply it and in which context), Solution (the participating elements and their relationships/collaborations) and Consequences (benefits and trade-offs).

02

Creational Design Patterns

2023–2024 • Q1(a) CSE213 • 10/05/2026 • D001 **MARKS 15**

A travel company plans to offer customers a number of travel packages. Each package has its own set of activities, pricing, and a brief description. Currently, the system supports two package types:

- **Adventure Package:** Designed for thrill-seekers, this package includes physically engaging outdoor activities such as mountain hiking, river rafting, etc.
- **Regular Package:** Designed for regular leisure travelers, this package includes city sightseeing, museum visits, etc.

Each package independently manages its own details. Your task is to implement the tour package management system with an appropriate design pattern so that the client code does not directly construct a concrete class, but the appropriate package object is instantiated through a centralized creation component. Moreover, the design must support new package types without modifying any existing code.

SOLUTION

Step 1 – Reading the requirements

- There is a **family of interchangeable products** (`AdventurePackage`, `RegularPackage`, ...) that all answer the same questions: activities, price, description.
- "The client code does not directly construct a concrete class" ⇒ the client must never write `new AdventurePackage();` it must depend only on an abstraction (*code to an interface, not to an implementation*).
- "Instantiated through a centralized creation component" ⇒ object creation is moved out of the client into one factory.
- "Support new package types without modifying any existing code" ⇒ the factory itself must be **closed for modification but open for extension** (OCP). A plain `switch(type)` factory would violate this, because every new package means editing the switch.

Step 2 – Identifying the pattern: Factory Method

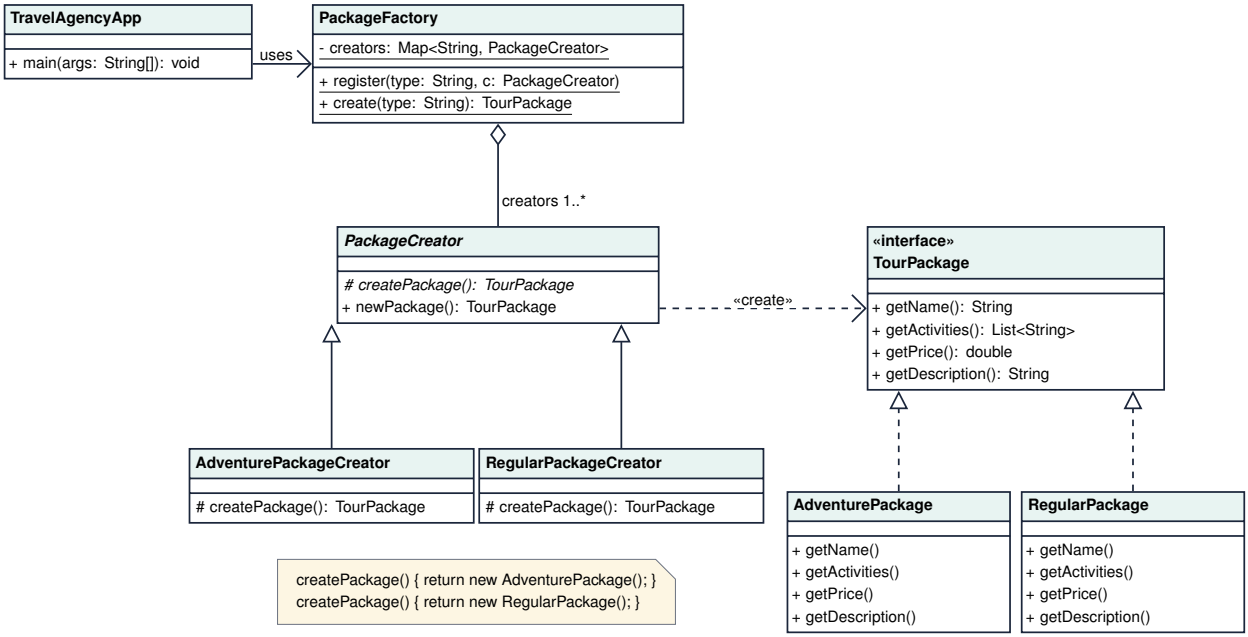
These forces are exactly what the **Factory Method** pattern addresses. Its intent (slides): *"Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses."*

- Every package type gets its own *creator* subclass that overrides the factory method `createPackage()`.
- The creators are kept in a registry inside one central `PackageFactory`, so the client asks for a package *by name* and receives a `TourPackage`.

- A new package type = one new product class + one new creator class + one registration entry in configuration. No existing class (products, creators, factory, client logic) is edited.

Pattern role	Class in our design
Product (interface)	`TourPackage`
Concrete products	`AdventurePackage`, `RegularPackage`
Creator (declares factory method)	`PackageCreator` (abstract, `createPackage()`)
Concrete creators	`AdventurePackageCreator`, `RegularPackageCreator`
Centralized creation component	`PackageFactory` (registry: type name → creator)
Client	`TravelAgencyApp`

Step 3 – UML class diagram



Step 4 – Java implementation

```java
import java.util.*;

// ====================== Product ======================
interface TourPackage {
    String getName();
    List<String> getActivities();
    double getPrice();
    String getDescription();
    default void showDetails() {
        System.out.println(getName() + " | " + getActivities() + " | Tk " + getPrice()
                           + " | " + getDescription());
    }
}

// ================ Concrete products ================
class AdventurePackage implements TourPackage {      // manages its own details
```

```
    public String getName()             { return "Adventure Package"; }
    public List<String> getActivities() { return List.of("Mountain hiking", "River rafting"); }
    public double getPrice()            { return 25000; }
    public String getDescription()      { return "Physically engaging outdoor trip for thrill-
    seekers"; }
}

class RegularPackage implements TourPackage {
    public String getName()             { return "Regular Package"; }
    public List<String> getActivities() { return List.of("City sightseeing", "Museum visits"); }
    public double getPrice()            { return 12000; }
    public String getDescription()      { return "Relaxed leisure trip for regular travelers"; }
}

// ============ Creator with the factory method ============
abstract class PackageCreator {
    protected abstract TourPackage createPackage();   // FACTORY METHOD: subclasses decide

    public TourPackage newPackage() {                 // common, product-independent logic
        TourPackage p = createPackage();
        System.out.println("[log] created " + p.getName());
        return p;
    }
}

class AdventurePackageCreator extends PackageCreator {
    protected TourPackage createPackage() { return new AdventurePackage(); }
}

class RegularPackageCreator extends PackageCreator {
    protected TourPackage createPackage() { return new RegularPackage(); }
}

// ========== Centralized creation component ==========
final class PackageFactory {
    private static final Map<String, PackageCreator> creators = new HashMap<>();
    private PackageFactory() { }

    public static void register(String type, PackageCreator creator) {
        creators.put(type.toLowerCase(), creator);
    }

    public static TourPackage create(String type) {
        PackageCreator c = creators.get(type.toLowerCase());
        if (c == null) throw new IllegalArgumentException("Unknown package type: " + type);
        return c.newPackage();            // no 'new ConcreteClass()' and no switch here
    }
}

// ======================= Client =======================
public class TravelAgencyApp {
    public static void main(String[] args) {
        // configuration / bootstrap (could also be read from a properties file)
        PackageFactory.register("adventure", new AdventurePackageCreator());
        PackageFactory.register("regular",   new RegularPackageCreator());

        TourPackage p1 = PackageFactory.create("adventure");  // client never uses 'new'
        TourPackage p2 = PackageFactory.create("regular");
```

```
        p1.showDetails();
        p2.showDetails();
    }
}
```

Step 5 – Adding a new package type (proof of extensibility)

Suppose a *Luxury Package* is introduced. We only *add* code:

```
class LuxuryPackage implements TourPackage { /* its own activities, price, description */ }
class LuxuryPackageCreator extends PackageCreator {
    protected TourPackage createPackage() { return new LuxuryPackage(); }
}
// one configuration line:  PackageFactory.register("luxury", new LuxuryPackageCreator());
```

`TourPackage`, the existing products and creators, `PackageFactory` and the client's business logic stay untouched, so the Open–Closed Principle is respected. The client depends only on `TourPackage` and `PackageFactory` (Dependency Inversion), and every class has a single responsibility (a product describes itself; a creator only creates; the factory only looks up).

Answer. Use the **Factory Method** pattern: `TourPackage` is the product interface, each package type has a concrete creator overriding `createPackage()`, and a central registry-based `PackageFactory` hands out packages by name. The client never instantiates a concrete class and new package types are added by writing new classes only.

2023–2024 • Q5(a) CSE213 • 09/02/2025 • D007.1 **MARKS 15**

A software system is being developed for a pizza delivery service that allows customers to order pizzas with regional styles. The service operates in two countries, Italy and America, each with distinct preferences for pizza types. For instance, the crust type and toppings differ between regions. The system must ensure that each pizza is created with region-specific ingredients and should be easily extendable in the future to accommodate additional countries. Identify the most suitable design pattern to solve the given problem. Justify your choice and provide the corresponding code implementation.

SOLUTION

Identifying the pattern: Abstract Factory

A pizza is built from *several related objects* – crust, sauce, toppings – and these objects come in *families*: an Italian family and an American family. The requirement "each pizza is created with region-specific ingredients" means we must never mix families (e.g. an Italian thin crust with American toppings). The slide intent of **Abstract Factory** is precisely: *"Provide an interface for creating families of related or dependent objects without specifying their concrete classes."*

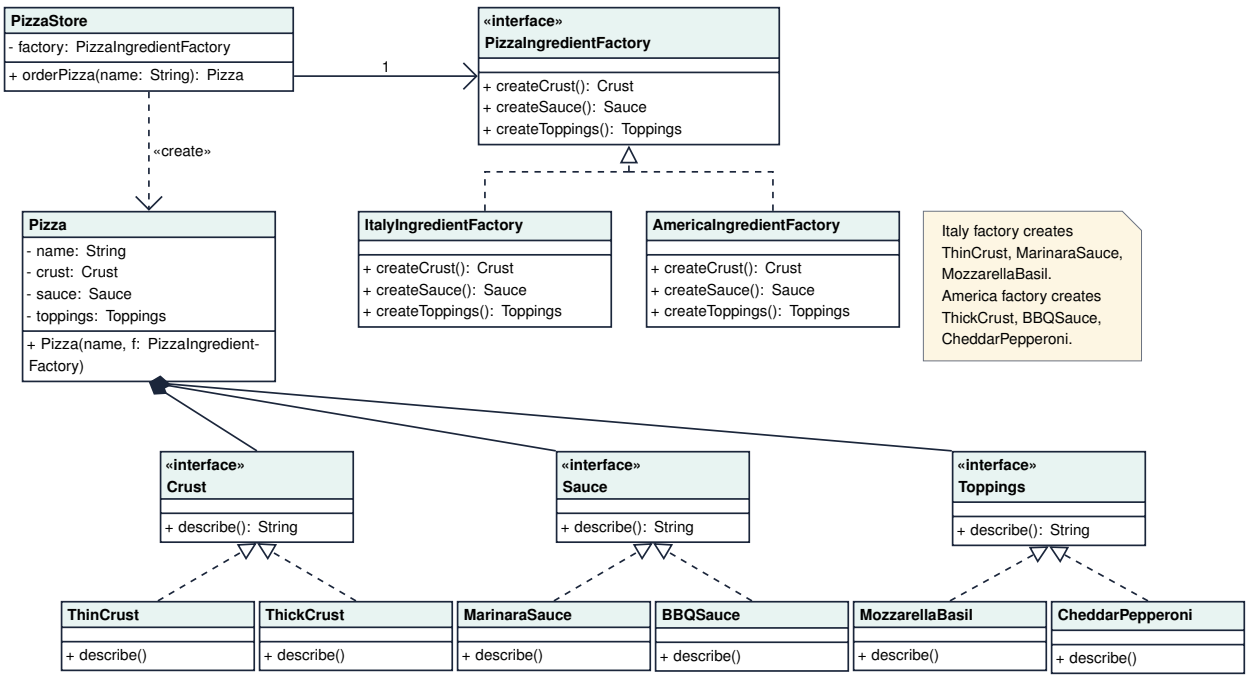
Justification

- **Consistency among products**: a region's factory can create only that region's ingredients, so a pizza can never receive ingredients of two regions (slides: "promotes consistency among products").
- **Isolates concrete classes**: `Pizza` and `PizzaStore` talk only to the interfaces `PizzaIngredientFactory`, `Crust`, `Sauce`, `Toppings`.
- **Exchanging the product family is easy**: the region is chosen by passing a different factory object (one line).

- **Extensible for new countries**: adding Japan = a new `JapanIngredientFactory` plus its ingredient classes; no existing class changes (OCP).
- Factory Method alone would create *one* product; here we need a coordinated *set* of products, hence Abstract Factory ("factory of factories").

Role	Class
AbstractFactory	`PizzaIngredientFactory`
ConcreteFactory	`ItalyIngredientFactory`, `AmericaIngredientFactory`
AbstractProduct	`Crust`, `Sauce`, `Toppings`
ConcreteProduct	`ThinCrust`, `MarinaraSauce`, `MozzarellaBasil` (Italy); `ThickCrust`, `BBQSauce`, `CheddarPepperoni` (America)
Client	`Pizza`, `PizzaStore`

Class diagram



Java implementation

```java
import java.util.*;

// ---------------- Abstract products ----------------
interface Crust    { String describe(); }
interface Sauce    { String describe(); }
interface Toppings { String describe(); }

// ---------------- Italian family ----------------
class ThinCrust implements Crust {
    public String describe() { return "thin wood-fired crust"; }
}
class MarinaraSauce implements Sauce {
    public String describe() { return "marinara sauce"; }
```

```
}
class MozzarellaBasil implements Toppings {
    public String describe() { return "fresh mozzarella & basil"; }
}

// ---------------- American family ----------------
class ThickCrust implements Crust {
    public String describe() { return "thick pan crust"; }
}
class BBQSauce implements Sauce {
    public String describe() { return "smoky BBQ sauce"; }
}
class CheddarPepperoni implements Toppings {
    public String describe() { return "cheddar & pepperoni"; }
}

// ---------------- Abstract factory ----------------
interface PizzaIngredientFactory {
    Crust    createCrust();
    Sauce    createSauce();
    Toppings createToppings();
}

// ---------------- Concrete factories (one per region) ----------------
class ItalyIngredientFactory implements PizzaIngredientFactory {
    public Crust    createCrust()    { return new ThinCrust(); }
    public Sauce    createSauce()    { return new MarinaraSauce(); }
    public Toppings createToppings() { return new MozzarellaBasil(); }
}

class AmericaIngredientFactory implements PizzaIngredientFactory {
    public Crust    createCrust()    { return new ThickCrust(); }
    public Sauce    createSauce()    { return new BBQSauce(); }
    public Toppings createToppings() { return new CheddarPepperoni(); }
}

// ---------------- Client classes ----------------
class Pizza {
    private final String name;
    private final Crust crust;
    private final Sauce sauce;
    private final Toppings toppings;

    Pizza(String name, PizzaIngredientFactory f) {   // ALL ingredients from ONE family
        this.name = name;
        this.crust = f.createCrust();
        this.sauce = f.createSauce();
        this.toppings = f.createToppings();
    }
    public String toString() {
        return name + " [" + crust.describe() + ", " + sauce.describe() + ", "
                    + toppings.describe() + "]";
    }
}

class PizzaStore {
    private final PizzaIngredientFactory factory;    // decided once per region
    PizzaStore(PizzaIngredientFactory factory) { this.factory = factory; }
```

```
    Pizza orderPizza(String name) {
        Pizza p = new Pizza(name, factory);
        System.out.println("Baking " + p);
        return p;
    }
}

public class PizzaDeliveryApp {
    public static void main(String[] args) {
        Map<String, PizzaIngredientFactory> regions = new HashMap<>();   // configuration
        regions.put("Italy",   new ItalyIngredientFactory());
        regions.put("America", new AmericaIngredientFactory());

        new PizzaStore(regions.get("Italy")).orderPizza("Margherita");
        new PizzaStore(regions.get("America")).orderPizza("Margherita");
    }
}
```

Output: `Baking Margherita [thin wood-fired crust, marinara sauce, fresh mozzarella & basil]` followed by the American version with thick crust, BBQ sauce, cheddar & pepperoni.

Extending to a new country

To add, say, Japan: write `JapanIngredientFactory implements PizzaIngredientFactory` and its ingredient classes (e.g. `RiceFlourCrust`, `TeriyakiSauce`, `SeaweedToppings`) and register it. `Pizza`, `PizzaStore` and the existing factories are not modified.

Answer. **Abstract Factory**: `PizzaIngredientFactory` declares `createCrust/Sauce/Toppings`; `ItalyIngredientFactory` and `AmericaIngredientFactory` produce matching ingredient families; `Pizza` receives one factory and therefore always gets region-consistent ingredients, and a new country is just a new concrete factory.

2022–2023 • Q3 CSE307 • 05/05/2024 • D010.2 **MARKS 15**

A Director class `HTMLPageDirector` is given a builder class, `HTMLPageBuilder`. The director tells the builder what the `pageTitle` & `pageHeading` will be, and gives multiple lines of text for the page. The director then has the builder to do a final assembly of the parts, and return the *page*.
Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfills all the requirements.

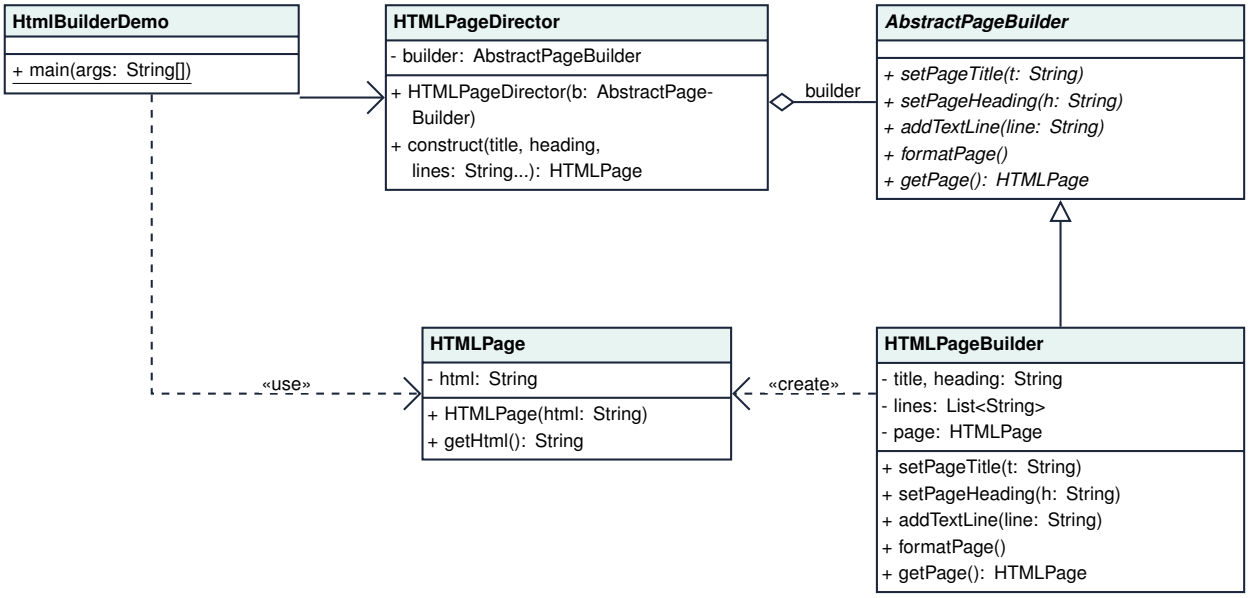
SOLUTION

Identifying the pattern: Builder

The description names the participants of the **Builder** pattern explicitly: a *Director* that knows the *order of construction steps*, and a *Builder* that knows *how to build each part* and finally returns the finished *Product* (the page). Builder's intent (slides): *"Separate the construction of a complex object from its representation so that the same construction process can create different representations."*

Builder role	Class / responsibility
Director	`HTMLPageDirector` – decides title, heading, text lines, then asks for final assembly
Builder (abstract)	`AbstractPageBuilder` – declares the step methods
ConcreteBuilder	`HTMLPageBuilder` – stores the parts and assembles HTML in `formatPage()`
Product	`HTMLPage` – the finished page returned by `getPage()`
Client	`HtmlBuilderDemo`

Class diagram



Java implementation

```java
import java.util.*;

// ======================== Product ========================
class HTMLPage {
    private final String html;
    HTMLPage(String html) { this.html = html; }
    public String getHtml() { return html; }
}

// ======================== Builder ========================
abstract class AbstractPageBuilder {
    public abstract void setPageTitle(String title);
    public abstract void setPageHeading(String heading);
    public abstract void addTextLine(String line);
    public abstract void formatPage();            // final assembly of the parts
    public abstract HTMLPage getPage();
}

// ================== Concrete builder ==================
class HTMLPageBuilder extends AbstractPageBuilder {
    private String title = "", heading = "";
    private final List<String> lines = new ArrayList<>();
```

```
    private HTMLPage page;

    public void setPageTitle(String title)     { this.title = title; }
    public void setPageHeading(String heading) { this.heading = heading; }
    public void addTextLine(String line)       { lines.add(line); }

    public void formatPage() {
        StringBuilder sb = new StringBuilder();
        sb.append("<html>\n<head><title>").append(title).append("</title></head>\n<body>\n");
        sb.append("  <h1>").append(heading).append("</h1>\n");
        for (String l : lines) sb.append("  <p>").append(l).append("</p>\n");
        sb.append("</body>\n</html>");
        page = new HTMLPage(sb.toString());
    }

    public HTMLPage getPage() {
        if (page == null) throw new IllegalStateException("formatPage() has not been called");
        return page;
    }
}

// ======================== Director ========================
class HTMLPageDirector {
    private final AbstractPageBuilder builder;
    HTMLPageDirector(AbstractPageBuilder builder) { this.builder = builder; }

    public HTMLPage construct(String title, String heading, String... lines) {
        builder.setPageTitle(title);          // 1. tell the title
        builder.setPageHeading(heading);      // 2. tell the heading
        for (String l : lines)                // 3. give multiple lines of text
            builder.addTextLine(l);
        builder.formatPage();                 // 4. final assembly
        return builder.getPage();             // 5. return the page
    }
}

// ======================== Client ========================
public class HtmlBuilderDemo {
    public static void main(String[] args) {
        HTMLPageDirector director = new HTMLPageDirector(new HTMLPageBuilder());
        HTMLPage page = director.construct("CSE 213", "Builder Pattern",
                "The director controls the order of steps.",
                "The builder knows how to build each part.",
                "The product is returned only after final assembly.");
        System.out.println(page.getHtml());
    }
}
```

Output:

```
<html>
<head><title>CSE 213</title></head>
<body>
  <h1>Builder Pattern</h1>
  <p>The director controls the order of steps.</p>
  <p>The builder knows how to build each part.</p>
  <p>The product is returned only after final assembly.</p>
</body>
</html>
```

Why this satisfies the requirements

- The director only *tells* the parts (title, heading, lines) and the moment of assembly; it never touches HTML syntax – construction is separated from representation.
- The page is assembled in one step (`formatPage()`) and handed out only afterwards, giving "finer control over the construction process" (slides).
- Another representation (e.g. a `MarkdownPageBuilder` producing `# heading`) can be plugged into the *same* director without changing it.

Answer. **Builder pattern**: `HTMLPageDirector` (Director) calls `setPageTitle`, `setPageHeading`, `addTextLine`$\times n$ and `formatPage` on an `AbstractPageBuilder`; `HTMLPageBuilder` (ConcreteBuilder) assembles the HTML and returns the `HTMLPage` (Product) via `getPage()`.

2021–2022 • Q3(a) CSE307 • 07/05/2023 • D015 **MARKS 20**

Suppose you are working on a restaurant's software development team. Your restaurant provides four kinds of set menus such as: *Veggie Harvest, Chicken Delight, Beef Fiesta, Sea Shanty*. *Veggie Harvest* contains Mushroom Rice, Paneer Masala and Fried Vegetable Pakora. *Chicken Delight* contains Chicken Fried Rice, Butter Chicken and Fried Chicken wings. *Beef Fiesta* contains Beef Fried Rice, Beef Chili Onion and Fried Beef Bacon. *Sea Shanty* contains Rice with Shrimp, Crab Masala and Fried Calamari.
Your restaurant have three different kitchens named as *Rice Station* where all the rice based items are prepared, *Main Mania* where all the main dishes (Paneer Masala, Butter Chicken, Beef Chili Onion and Crab Masala) are prepared and *Fry More* where the fried items are prepared.
Your restaurant takes orders from the user, then assembles the items prepared by the kitchens described above and delivers it.
Your job is to identify the design pattern that can best capture the scenario above and implement it using your preferred language (you can use pseudo syntax). You must provide the UML class diagram of your implementation. ***Be careful, as users are very sensitive to their orders.***

SOLUTION

Step 1 – Analysing the scenario

Arrange the menu items in a grid: rows are the set menus, columns are the kitchens.

Set menu (family)	Rice Station	Main Mania	Fry More
Veggie Harvest	Mushroom Rice	Paneer Masala	Fried Vegetable Pakora
Chicken Delight	Chicken Fried Rice	Butter Chicken	Fried Chicken Wings
Beef Fiesta	Beef Fried Rice	Beef Chili Onion	Fried Beef Bacon
Sea Shanty	Rice with Shrimp	Crab Masala	Fried Calamari

- Each **column** is a *kind of product* (rice item, main dish, fried item) – one kitchen per kind.
- Each **row** is a *family of related products* that must always be delivered together.
- "Users are very sensitive to their orders" $\Rightarrow$ a customer who orders *Sea Shanty* must never get Butter Chicken in the box: the products of one meal must *match one family*. This is the same force as the slides' furniture shop: "customers get quite mad when they receive non-matching furniture".

- New set menus are likely in future and should not require changes to the ordering code.

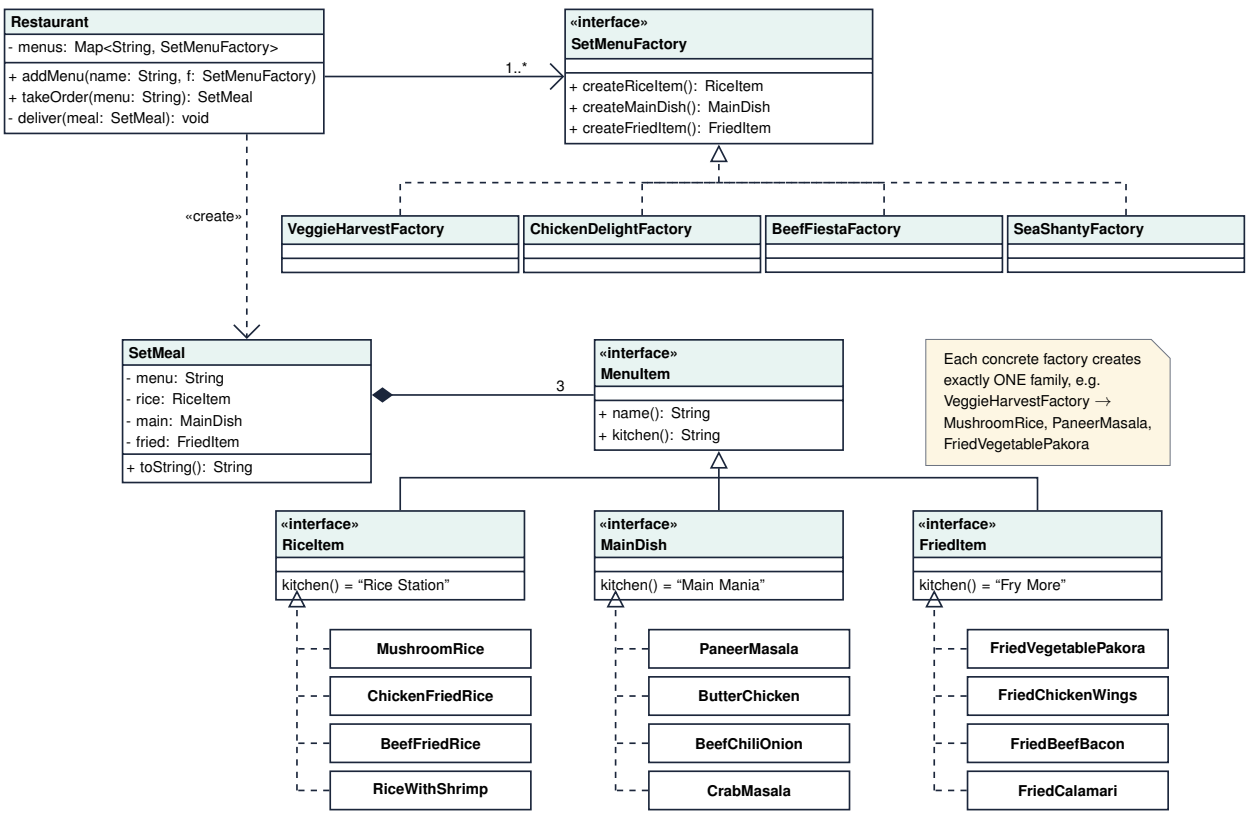
Step 2 – Identifying the pattern: Abstract Factory

Abstract Factory – *"Provide an interface for creating families of related or dependent objects without specifying their concrete classes."* The 4×3 grid maps directly onto it:

- one **abstract product** per kitchen: `RiceItem` (Rice Station), `MainDish` (Main Mania), `FriedItem` (Fry More);
- one **abstract factory** `SetMenuFactory` with a creation method per kitchen;
- one **concrete factory** per set menu (`VeggieHarvestFactory`, ...) that can create *only* the three items of its own menu – so a mismatched meal is impossible by construction;
- the **client** `Restaurant` takes the order, picks the factory of the ordered menu, assembles the three items into a `SetMeal` and delivers it, without knowing any concrete dish class.

> **NOTE** A Builder-based answer (Restaurant = Director that assembles a meal step by step) is also seen in some solutions. However, the decisive requirement here is *consistency of a family of products* ("users are very sensitive to their orders"), which is the stated consequence of Abstract Factory; the assembly step is trivial (always rice + main + fried), so Abstract Factory is the best fit.

Step 3 – UML class diagram



Step 4 – Implementation (Java)

```java
import java.util.*;

// ======== Abstract products: one per KITCHEN ========
```

```java
interface MenuItem  { String name(); String kitchen(); }
interface RiceItem  extends MenuItem { default String kitchen() { return "Rice Station"; } }
interface MainDish  extends MenuItem { default String kitchen() { return "Main Mania"; } }
interface FriedItem extends MenuItem { default String kitchen() { return "Fry More"; } }

// ======== Concrete products ========
// --- prepared at Rice Station ---
class MushroomRice implements RiceItem {
    public String name() { return "Mushroom Rice"; } }
class ChickenFriedRice implements RiceItem {
    public String name() { return "Chicken Fried Rice"; } }
class BeefFriedRice implements RiceItem {
    public String name() { return "Beef Fried Rice"; } }
class RiceWithShrimp implements RiceItem {
    public String name() { return "Rice with Shrimp"; } }

// --- prepared at Main Mania ---
class PaneerMasala implements MainDish {
    public String name() { return "Paneer Masala"; } }
class ButterChicken implements MainDish {
    public String name() { return "Butter Chicken"; } }
class BeefChiliOnion implements MainDish {
    public String name() { return "Beef Chili Onion"; } }
class CrabMasala implements MainDish {
    public String name() { return "Crab Masala"; } }

// --- prepared at Fry More ---
class FriedVegetablePakora implements FriedItem {
    public String name() { return "Fried Vegetable Pakora"; } }
class FriedChickenWings implements FriedItem {
    public String name() { return "Fried Chicken Wings"; } }
class FriedBeefBacon implements FriedItem {
    public String name() { return "Fried Beef Bacon"; } }
class FriedCalamari implements FriedItem {
    public String name() { return "Fried Calamari"; } }

// ======== Abstract factory: one creation method per kitchen ========
interface SetMenuFactory {
    RiceItem  createRiceItem();     // prepared at Rice Station
    MainDish  createMainDish();     // prepared at Main Mania
    FriedItem createFriedItem();    // prepared at Fry More
}

// ======== Concrete factories: one per SET MENU (family) ========
class VeggieHarvestFactory implements SetMenuFactory {
    public RiceItem  createRiceItem()  { return new MushroomRice(); }
    public MainDish  createMainDish()  { return new PaneerMasala(); }
    public FriedItem createFriedItem() { return new FriedVegetablePakora(); }
}
class ChickenDelightFactory implements SetMenuFactory {
    public RiceItem  createRiceItem()  { return new ChickenFriedRice(); }
    public MainDish  createMainDish()  { return new ButterChicken(); }
    public FriedItem createFriedItem() { return new FriedChickenWings(); }
}
class BeefFiestaFactory implements SetMenuFactory {
    public RiceItem  createRiceItem()  { return new BeefFriedRice(); }
    public MainDish  createMainDish()  { return new BeefChiliOnion(); }
    public FriedItem createFriedItem() { return new FriedBeefBacon(); }
```

```
}
class SeaShantyFactory implements SetMenuFactory {
    public RiceItem  createRiceItem()  { return new RiceWithShrimp(); }
    public MainDish  createMainDish()  { return new CrabMasala(); }
    public FriedItem createFriedItem() { return new FriedCalamari(); }
}

// ======== The assembled meal ========
class SetMeal {
    private final String menu;
    private final RiceItem rice;  private final MainDish main;  private final FriedItem fried;
    SetMeal(String menu, RiceItem r, MainDish m, FriedItem f) {
        this.menu = menu; this.rice = r; this.main = m; this.fried = f;
    }
    public String toString() {
        return menu + ": " + rice.name() + " (" + rice.kitchen() + "), "
             + main.name() + " (" + main.kitchen() + "), "
             + fried.name() + " (" + fried.kitchen() + ")";
    }
}

// ======== Client ========
class Restaurant {
    private final Map<String, SetMenuFactory> menus = new LinkedHashMap<>();

    public void addMenu(String name, SetMenuFactory f) { menus.put(name, f); }

    public SetMeal takeOrder(String menuName) {
        SetMenuFactory f = menus.get(menuName);
        if (f == null) throw new IllegalArgumentException("No such set menu: " + menuName);
        // the three kitchens each prepare their item -- all from the SAME family
        SetMeal meal = new SetMeal(menuName, f.createRiceItem(), f.createMainDish(), f.
    createFriedItem());
        deliver(meal);
        return meal;
    }
    private void deliver(SetMeal meal) { System.out.println("Delivering -> " + meal); }
}

public class RestaurantApp {
    public static void main(String[] args) {
        Restaurant r = new Restaurant();
        r.addMenu("Veggie Harvest",  new VeggieHarvestFactory());
        r.addMenu("Chicken Delight", new ChickenDelightFactory());
        r.addMenu("Beef Fiesta",     new BeefFiestaFactory());
        r.addMenu("Sea Shanty",      new SeaShantyFactory());

        r.takeOrder("Sea Shanty");
        r.takeOrder("Veggie Harvest");
    }
}
```

Sample output:

```
Delivering -> Sea Shanty: Rice with Shrimp (Rice Station), Crab Masala (Main Mania), Fried
    Calamari (Fry More)
Delivering -> Veggie Harvest: Mushroom Rice (Rice Station), Paneer Masala (Main Mania), Fried
    Vegetable Pakora (Fry More)
```

Step 5 – Why the design is safe and extensible

- **No mismatched orders:** `Restaurant` obtains all three items from the *same* factory object; e.g. `SeaShantyFactory` simply has no way to return Butter Chicken.
- **Isolation of concrete classes:** `Restaurant` and `SetMeal` depend only on `SetMenuFactory`, `RiceItem`, `MainDish`, `FriedItem`.
- **New set menu** (e.g. "Mutton Feast"): add one concrete factory + its three dishes, then `addMenu(...)`; existing classes are untouched (OCP).
- **Known trade-off** (slides): adding a *new kind* of product – e.g. a fourth kitchen "Sweet Corner" for desserts – requires changing `SetMenuFactory` and every concrete factory.

Answer. The scenario is an **Abstract Factory**: kitchens → abstract products (`RiceItem`, `MainDish`, `FriedItem`); set menus → concrete factories implementing `SetMenuFactory`; `Restaurant` (client) takes the order, uses the chosen menu's factory to get all three items and delivers a `SetMeal`. Because every item comes from one family's factory, a customer can never receive a non-matching order.

Jan 2021 • Q1(a) CSE307 • 04/09/2021 • D019.1 MARKS 15

Consider a web based application that stores the information of users for registration purpose. After successful registration the registration page returns a user identification number (userID) which identifies the users for further references. The registration process also requires identifiers to identify the home district and educational qualification. The web application has the facility to create a home district and educational qualifications as per requirement of the user applying for registration if there is no available home district and educational qualification in the list of the interface. The transactions made by the web based application will also be identified by another identifier (TxID). A particular identifier will never be duplicate and it is expected that the identifiers are created serially starting from 1. These numbers will actually be used as the primary keys in the data storage table for accessing the data. Demonstrate creation of those identifiers in the back end of your application whenever necessary by the user interface. You do not need to demonstrate the details of user interface or front end. It is expected that a suitable design pattern must be used to demonstrate the solution.

SOLUTION

Step 1 – What must be guaranteed?

- Four independent sequences: **userID**, **districtID**, **qualificationID** and **TxID**; each starts at 1 and increases by 1.
- An identifier must **never be duplicated** – they are primary keys.
- Many parts of the back end (registration, district creation, qualification creation, every transaction) and many concurrent web requests need new identifiers.

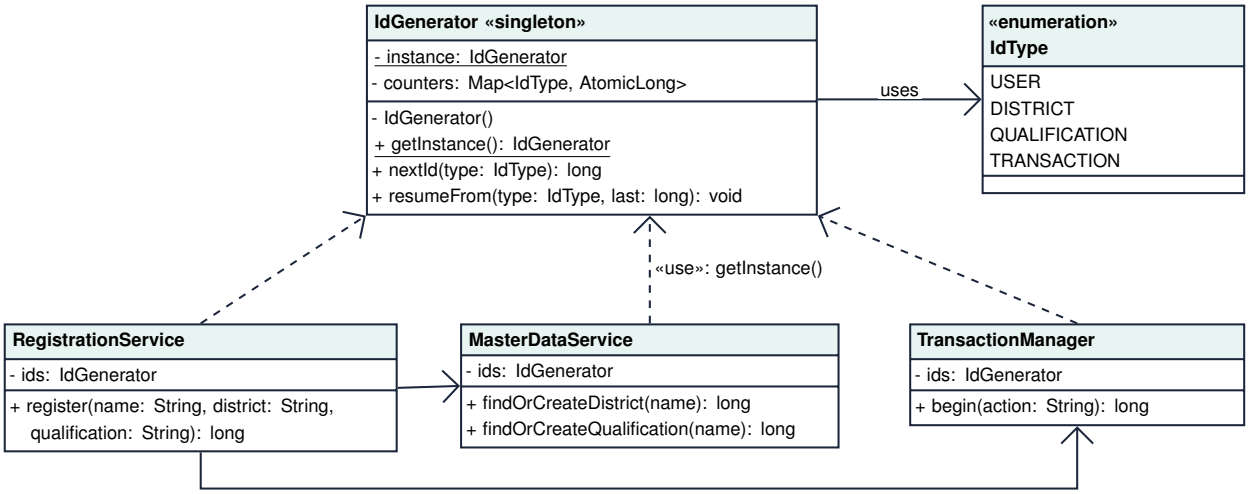
If every service created its own counter object (`new IdGenerator()`), each would start at 1 again and keys would collide. All requests must share *one and only one* counter authority.

Step 2 – Identifying the pattern: Singleton

Singleton – *"Ensure a class has only one instance, and provide a global point of access to it."* An `IdGenerator` singleton owns one counter per identifier type:

- private constructor ⇒ no second generator (and no second set of counters) can exist;
- `getInstance()` gives every service the same object (global access point);
- counters are `AtomicLong` and the lazy creation uses double-checked locking, so concurrent web requests can never receive the same number;
- on server restart the counters are resumed from `MAX(id)` of each table, so numbering continues instead of restarting at 1.

Step 3 – Class diagram



Step 4 – Java back-end code

```
import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

enum IdType { USER, DISTRICT, QUALIFICATION, TRANSACTION }

// ====================== SINGLETON ======================
final class IdGenerator {
    private static volatile IdGenerator instance;                // the sole instance
    private final Map<IdType, AtomicLong> counters = new EnumMap<>(IdType.class);

    private IdGenerator() {                                      // no 'new' from outside
        for (IdType t : IdType.values()) counters.put(t, new AtomicLong(0));
    }

    public static IdGenerator getInstance() {                   // global access point
        if (instance == null) {                                  // lazy + thread-safe (DCL)
            synchronized (IdGenerator.class) {
                if (instance == null) instance = new IdGenerator();
            }
        }
        return instance;
    }

    public long nextId(IdType type) {                            // 1, 2, 3, ... never repeats
        return counters.get(type).incrementAndGet();
    }
```

```
    // called once at start-up with SELECT MAX(id) FROM <table>, so keys survive a restart
    public void resumeFrom(IdType type, long lastUsed) {
        counters.get(type).set(lastUsed);
    }
}

// ======================== BACK-END SERVICES ========================
class TransactionManager {
    private final IdGenerator ids = IdGenerator.getInstance();
    long begin(String action) {
        long txId = ids.nextId(IdType.TRANSACTION);
        System.out.println("TxID " + txId + " : " + action);
        return txId;
    }
}

class MasterDataService {
    private final IdGenerator ids = IdGenerator.getInstance();
    private final Map<String, Long> districts = new HashMap<>();       // stands for DISTRICT
    table
    private final Map<String, Long> qualifications = new HashMap<>();  // stands for
    QUALIFICATION table

    long findOrCreateDistrict(String name) {      // created only if not in the UI list
        return districts.computeIfAbsent(name, n -> {
            long id = ids.nextId(IdType.DISTRICT);
            System.out.println("  new district   '" + n + "' -> districtID " + id);
            return id;
        });
    }
    long findOrCreateQualification(String name) {
        return qualifications.computeIfAbsent(name, n -> {
            long id = ids.nextId(IdType.QUALIFICATION);
            System.out.println("  new qualification '" + n + "' -> qualificationID " + id);
            return id;
        });
    }
}

class RegistrationService {
    private final IdGenerator ids = IdGenerator.getInstance();
    private final MasterDataService master = new MasterDataService();
    private final TransactionManager tx = new TransactionManager();

    long register(String name, String district, String qualification) {
        tx.begin("register " + name);
        long dId = master.findOrCreateDistrict(district);
        long qId = master.findOrCreateQualification(qualification);
        long userId = ids.nextId(IdType.USER);
        // INSERT INTO users(user_id, name, district_id, qual_id) VALUES (userId, name, dId, qId
    )
        System.out.println("  user '" + name + "' registered, userID = " + userId);
        return userId;                    // returned to the registration page
    }
}

public class RegistrationDemo {
```

```
    public static void main(String[] args) {
        RegistrationService reg = new RegistrationService();
        reg.register("Rahim",  "Dhaka",   "HSC");
        reg.register("Karim",  "Khulna",  "BSc");
        reg.register("Fatema", "Dhaka",   "HSC");   // existing district & qualification reused
        System.out.println(IdGenerator.getInstance() == IdGenerator.getInstance());   // true
    }
}
```

Output (abridged): TxID 1 / districtID 1 / qualificationID 1 / userID 1; TxID 2 / districtID 2 / qualificationID 2 / userID 2; TxID 3 / (Dhaka and HSC reused) / userID 3; `true`. Each sequence starts at 1, is serial, and no value is ever handed out twice, even under concurrent requests.

> **EXAM TIP** Mention the three Singleton ingredients (private constructor, static instance, static `getInstance()`) and *thread safety* – a web server serves requests in parallel, so a non-synchronised lazy singleton or a plain `count++` could produce duplicate primary keys.

Answer. Use the **Singleton** pattern: a single `IdGenerator` (private constructor, static `getInstance()`) keeps one atomic counter per identifier type (userID, districtID, qualificationID, TxID). All back-end services obtain the same instance and call `nextId(type)`, which yields serial, duplicate-free primary keys starting from 1.

2018–2019 • Q3(b) CSE307 • 06/11/2019 • D021.1 **MARKS 20**

There is a software application that can be used by the students of a University to provide them with the list of their grades for their exams. This application needs to run in different ways depending on the users that are using it. For example, the admin needs to have some buttons enabled whereas those buttons need to be disabled for the student. The Builder provides the interface for building forms depending on the user login information. The ConcreteBuilders are the specific forms for each type of user. The product is the final form that the application will use in the given case and the Director is the application that, based on the login information, needs a specific form. Draw a class diagram to present Builder design pattern and write necessary codes so that your code fulfills all the requirements.

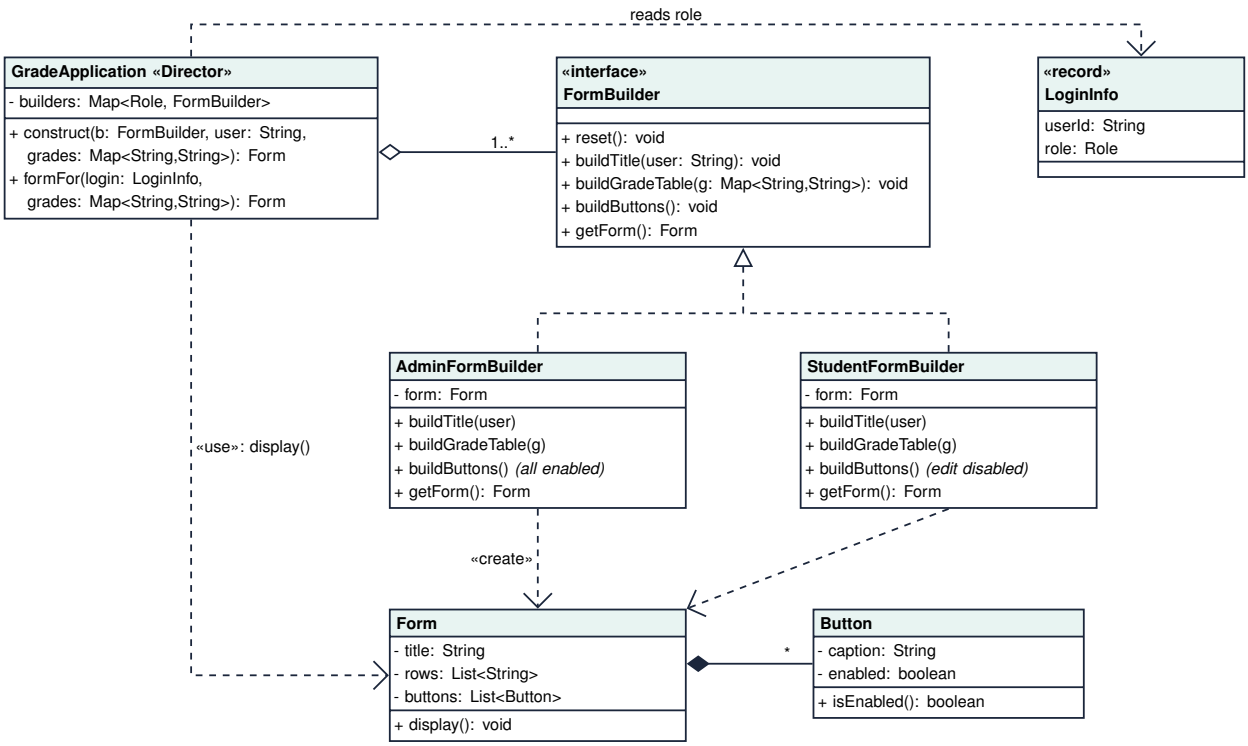
SOLUTION

Identifying the pattern: Builder (named in the question)

Builder: *"Separate the construction of a complex object from its representation so that the same construction process can create different representations."* Here the *construction process* of a grade form is always the same – title, grade table, buttons – but the *representation* differs per user type: an admin form has editing buttons enabled, a student form has them disabled.

Builder role	Class in this solution
Builder	`FormBuilder` (interface): `buildTitle()`, `buildGradeTable()`, `buildButtons()`, `getForm()`
ConcreteBuilders	`AdminFormBuilder`, `StudentFormBuilder` – one specific form per user type
Product	`Form` (with `Button` parts) – the final form the application shows
Director	`GradeApplication` – picks a builder from the login information and runs the steps
Helper	`LoginInfo` (user id + role), `Button` (label + enabled flag)

Class diagram



Java implementation

```java
import java.util.*;

enum Role { ADMIN, STUDENT }
record LoginInfo(String userId, Role role) { }

// ======================== Product parts ========================
class Button {
    private final String caption;
    private final boolean enabled;
    Button(String caption, boolean enabled) { this.caption = caption; this.enabled = enabled; }
    public boolean isEnabled() { return enabled; }
    public String toString() { return "[" + caption + (enabled ? "" : " (disabled)") + "]"; }
}

// ======================== Product ========================
class Form {
    private String title;
    private final List<String> rows = new ArrayList<>();
    private final List<Button> buttons = new ArrayList<>();
    void setTitle(String t)   { title = t; }
    void addRow(String r)     { rows.add(r); }
    void addButton(Button b)  { buttons.add(b); }
    public void display() {
        System.out.println("=== " + title + " ===");
        rows.forEach(r -> System.out.println("  " + r));
        System.out.println("  " + buttons);
    }
}
```

```
// ======================== Builder ========================
interface FormBuilder {
    void reset();                                       // start a new, empty form
    void buildTitle(String user);
    void buildGradeTable(Map<String, String> grades);
    void buildButtons();
    Form getForm();
}

// ======================== Concrete builders ========================
class AdminFormBuilder implements FormBuilder {
    private Form form;
    public void reset() { form = new Form(); }
    public void buildTitle(String user) { form.setTitle("Grade Management (admin: " + user + ")"
    ); }
    public void buildGradeTable(Map<String, String> g) {
        g.forEach((course, grade) -> form.addRow(course + " : " + grade + "   <editable>"));
    }
    public void buildButtons() {                        // everything enabled for the admin
        form.addButton(new Button("View", true));
        form.addButton(new Button("Edit Grade", true));
        form.addButton(new Button("Publish Result", true));
        form.addButton(new Button("Delete Record", true));
    }
    public Form getForm() { return form; }
}

class StudentFormBuilder implements FormBuilder {
    private Form form;
    public void reset() { form = new Form(); }
    public void buildTitle(String user) { form.setTitle("My Grades (student: " + user + ")"); }
    public void buildGradeTable(Map<String, String> g) {
        g.forEach((course, grade) -> form.addRow(course + " : " + grade));   // read-only
    }
    public void buildButtons() {                        // admin buttons disabled
        form.addButton(new Button("View", true));
        form.addButton(new Button("Edit Grade", false));
        form.addButton(new Button("Publish Result", false));
        form.addButton(new Button("Delete Record", false));
    }
    public Form getForm() { return form; }
}

// ======================== Director ========================
class GradeApplication {
    private final Map<Role, FormBuilder> builders = new EnumMap<>(Role.class);

    GradeApplication() {
        builders.put(Role.ADMIN,   new AdminFormBuilder());
        builders.put(Role.STUDENT, new StudentFormBuilder());
    }

    // the fixed construction process -- identical for every type of user
    public Form construct(FormBuilder b, String user, Map<String, String> grades) {
        b.reset();
        b.buildTitle(user);
        b.buildGradeTable(grades);
```

```
        b.buildButtons();
        return b.getForm();
    }

    // chooses the specific form from the login information
    public Form formFor(LoginInfo login, Map<String, String> grades) {
        FormBuilder b = builders.get(login.role());
        if (b == null) throw new IllegalArgumentException("Unknown role " + login.role());
        return construct(b, login.userId(), grades);
    }
}

// ====================== Client ======================
public class GradeAppDemo {
    public static void main(String[] args) {
        Map<String, String> grades = new LinkedHashMap<>();
        grades.put("CSE 213", "A+");
        grades.put("CSE 215", "A");

        GradeApplication app = new GradeApplication();
        app.formFor(new LoginInfo("admin01",  Role.ADMIN),   grades).display();
        app.formFor(new LoginInfo("2305003", Role.STUDENT), grades).display();
    }
}
```

Output:

```
=== Grade Management (admin: admin01) ===
  CSE 213 : A+   <editable>
  CSE 215 : A   <editable>
  [[View], [Edit Grade], [Publish Result], [Delete Record]]
=== My Grades (student: 2305003) ===
  CSE 213 : A+
  CSE 215 : A
  [[View], [Edit Grade (disabled)], [Publish Result (disabled)], [Delete Record (disabled)]]
```

Discussion

- **Same process, different representations**: `construct()` is written once; the forms differ only because a different ConcreteBuilder is passed.
- **Isolates construction and representation**: the director never knows which buttons are enabled; that knowledge lives in the builders.
- **Extensible**: a new user type (e.g. `TEACHER` who may edit but not publish) needs one new `TeacherFormBuilder` and one map entry; `Form` and `construct()` are unchanged.

Answer. Builder: `FormBuilder` (Builder) declares `buildTitle/buildGradeTable/buildButtons/getForm`; `AdminFormBuilder` and `StudentFormBuilder` (ConcreteBuilders) build forms with enabled vs. disabled buttons; `Form` is the Product; `GradeApplication` (Director) chooses the builder from `LoginInfo` and runs the fixed construction steps.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q5(a) CSE307 • 15/07/2017 • D027 **MARKS 5+5+5**

Describe eager and lazy initialization of singleton class. Give an example where lazy initialization should be preferred over eager initialization.

2012–2013 • Q2(a) CSE307 • 17/05/2014 • D033 **MARKS 6+4**

Differentiate between lazy and eager initialization of Singleton pattern. Draw the structure of singleton pattern.

SOLUTION

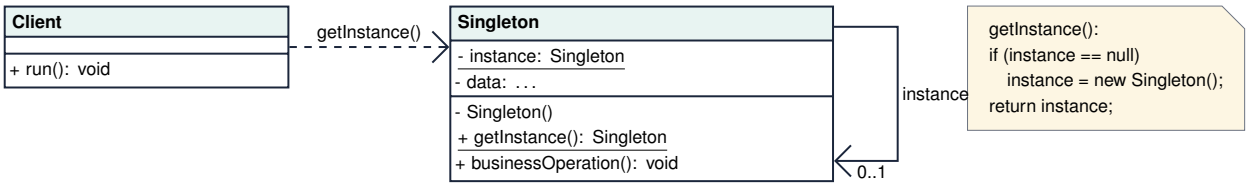
Singleton – intent and structure

Singleton: *"Ensure a class has only one instance, and provide a global point of access to it."* It is used when an application needs one, and only one, instance of an object (a DB connection manager, a logger, a configuration store, …). The three ingredients are:

(i) a **private constructor** – nobody outside can call `new`;

(ii) a **private static field** that holds the sole instance;

(iii) a **public static access method** `getInstance()` that returns that instance.



Static members are underlined; the self-association shows that the class holds (at most) one instance of itself.

Eager initialization

The instance is created *when the class is loaded*, before anybody asks for it.

```
public class EagerSingleton {
    // created once, when the class is loaded by the JVM
    private static final EagerSingleton instance = new EagerSingleton();
    private EagerSingleton() { }
    public static EagerSingleton getInstance() { return instance; }
}
```

- ✓ Very simple and automatically **thread-safe** (the JVM initialises static fields once).
- × The object is created even if it is **never used**; costly constructors (opening a DB connection, reading a large file) slow down start-up and waste memory.
- × No way to handle construction errors or to pass run-time parameters.

Lazy initialization

The instance is created *on the first call* of `getInstance()` ("just-in-time initialisation / initialisation on first use", as in the slides' `DbConnection` example).

```java
public class LazySingleton {
    private static LazySingleton instance;            // null until first use
    private LazySingleton() { }
    public static synchronized LazySingleton getInstance() {
        if (instance == null)                         // first call only
            instance = new LazySingleton();
        return instance;
    }
}
```

Without `synchronized`, two threads could both see `null` and create two objects. Cheaper thread-safe variants are *double-checked locking* (with a `volatile` field) and the *holder idiom*:

```java
public class HolderSingleton {
    private HolderSingleton() { }
    private static class Holder {                         // loaded only when getInstance() runs
        static final HolderSingleton INSTANCE = new HolderSingleton();
    }
    public static HolderSingleton getInstance() { return Holder.INSTANCE; }
}
```

Eager vs. lazy – comparison

Aspect	Eager initialization	Lazy initialization
When created	at class loading time	at first call of `getInstance()`
Unused instance	created anyway (waste)	never created
Start-up cost	higher (all work up front)	lower; cost paid on first use
Thread safety	free (JVM class init)	must be added (`synchronized`, DCL, holder)
Code complexity	minimal	a null check plus synchronisation
Exception handling	hard (static initializer)	easy (inside `getInstance()`)
Best for	small, always-needed objects	heavy or rarely-used resources

Example where lazy initialization should be preferred

A desktop application has a `ReportDatabase` singleton that opens a connection to a remote reporting server and loads several megabytes of metadata. Most users never open the "Reports" screen. With eager initialization every launch would pay the connection time (and would even fail when the server is offline, although the user never wanted a report). With lazy initialization the connection is opened only when the user first clicks "Reports":

```java
public class ReportDatabase {
    private static volatile ReportDatabase instance;
    private final java.sql.Connection conn;
    private ReportDatabase() { conn = openRemoteConnection(); }   // slow + may fail
    public static ReportDatabase getInstance() {
        if (instance == null) {
            synchronized (ReportDatabase.class) {
                if (instance == null) instance = new ReportDatabase();
            }
        }
        return instance;
    }
    private java.sql.Connection openRemoteConnection() { /* DriverManager.getConnection(...) */
    return null; }
```

```
}
```

Similar cases: a logger that writes only when an error occurs, a spell-checker dictionary, a printer spooler. Eager initialization is preferable only when the object is cheap and certainly needed.

Answer. Eager: instance created at class-load time – simple and thread-safe but may waste resources. Lazy: instance created on first `getInstance()` call – saves resources and start-up time but needs synchronisation. Prefer lazy for expensive, possibly unused objects such as a remote DB/reporting connection. Structure: private constructor, private static `instance`, public static `getInstance()`.

2016–2017 • Q5(c) CSE307 • 15/07/2017 • D028.1 **MARKS 3+7**

Assume that you are trying to implement syntax highlighting in a programming language editor. To perform this, you need to be able to parse different kinds of files such as `.c`, `.cpp`, `.java`, `.html` etc. All the parsers implement an interface named `Parser` which contains all the functions required to parse a file. When a file is selected, you will decide which parser to use based on the extension of the file. Now which design pattern should be followed to implement the selection process? Write appropriate java implementation of your solution.
[Hint: You don't need to write the parsers. Clearly mention the assumptions you have made.]

SOLUTION

(i) Identifying the pattern (3 marks)

The editor has to create *one of several classes that share a common interface* (`Parser`), and the concrete class is decided at run time from data (the file extension). The editor should not contain `if (ext.equals("java")) new JavaParser() ...` chains scattered in its code. This is the **Factory** pattern (Factory / Factory Method family): a `ParserFactory` encapsulates the decision "which concrete `Parser` to instantiate" and returns it through the `Parser` interface, exactly like `ShapeFactory` / `GetPayment` in the slides.
Benefits (slides): the client codes to the interface rather than the implementation; instantiation of the concrete classes is removed from client code; the code is less coupled and easy to extend (a new language = a new parser class + one registration).

Assumptions

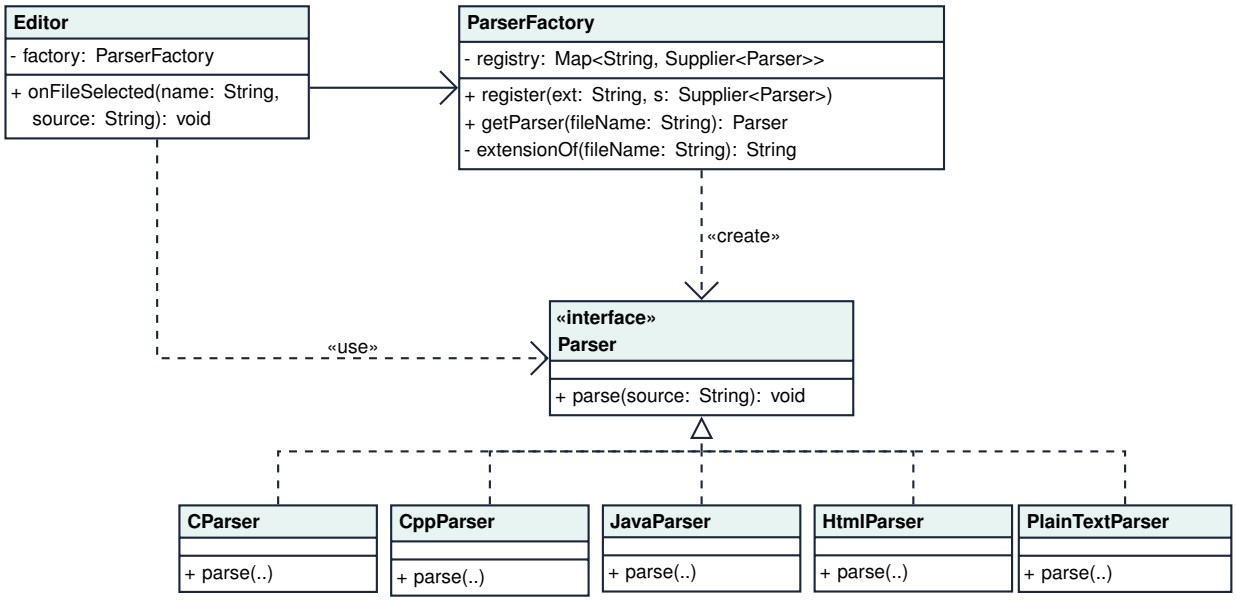
(i) `Parser` is already given; we assume it declares `void parse(String source)` (plus other parsing functions not needed here). Concrete parsers `CParser`, `CppParser`, `JavaParser`, `HtmlParser` exist; their bodies are not written (as the hint allows).

(ii) The extension is the text after the *last* dot of the file name, compared case-insensitively (`Main.JAVA` → `java`). `.h` files are treated as C, `.hpp`/`.cc` as C++, `.htm` as HTML.

(iii) For an unknown extension the editor should still open the file without highlighting, so the factory returns a `PlainTextParser` (a "do-nothing" parser) instead of `null`.

(iv) Parsers are stateless, so a fresh parser is created for each opened file (a `Supplier` is stored, not a shared object).

(ii) Class diagram



(iii) Java implementation (7 marks)

```java
import java.util.*;
import java.util.function.Supplier;

// ---------- given interface (assumed signature) ----------
interface Parser {
    void parse(String source);          // + other parsing functions
}

// ---------- concrete parsers (bodies not required) ----------
class CParser    implements Parser { public void parse(String s) { /* C grammar    */ } }
class CppParser  implements Parser { public void parse(String s) { /* C++ grammar  */ } }
class JavaParser implements Parser { public void parse(String s) { /* Java grammar */ } }
class HtmlParser implements Parser { public void parse(String s) { /* HTML grammar */ } }
class PlainTextParser implements Parser {          // fallback: no highlighting
    public void parse(String s) { }
}

// ---------- the factory: the ONLY place that knows concrete classes ----------
class ParserFactory {
    private final Map<String, Supplier<Parser>> registry = new HashMap<>();

    public ParserFactory() {                       // default mapping (assumption 2)
        register("c",    CParser::new);    register("h",    CParser::new);
        register("cpp",  CppParser::new);  register("cc",   CppParser::new);
        register("hpp",  CppParser::new);
        register("java", JavaParser::new);
        register("html", HtmlParser::new); register("htm",  HtmlParser::new);
    }

    public void register(String ext, Supplier<Parser> creator) {   // extension point
        registry.put(ext.toLowerCase(), creator);
    }

    public Parser getParser(String fileName) {
```

```
        Supplier<Parser> s = registry.get(extensionOf(fileName));
        return (s != null) ? s.get() : new PlainTextParser();       // assumption 3
    }

    private String extensionOf(String fileName) {
        int dot = fileName.lastIndexOf('.');
        return (dot < 0) ? "" : fileName.substring(dot + 1).toLowerCase();
    }
}

// ---------- client ----------
class Editor {
    private final ParserFactory factory = new ParserFactory();

    public void onFileSelected(String fileName, String source) {
        Parser parser = factory.getParser(fileName);   // editor never says 'new JavaParser()'
        System.out.println(fileName + " -> " + parser.getClass().getSimpleName());
        parser.parse(source);                          // then highlight using the parse result
    }
}

public class EditorDemo {
    public static void main(String[] args) {
        Editor editor = new Editor();
        editor.onFileSelected("main.c",     "int main(){return 0;}");   // CParser
        editor.onFileSelected("Shape.java", "class Shape{}");           // JavaParser
        editor.onFileSelected("index.HTML", "<html></html>");           // HtmlParser
        editor.onFileSelected("notes.txt",  "hello");                   // PlainTextParser
    }
}
```

To support Python later, we only add a new class `PythonParser` that implements `Parser` and register it once with `register("py", PythonParser::new)`; the `Editor` is not touched.

Answer. Use the **Factory** pattern: `ParserFactory.getParser(fileName)` extracts the extension, looks it up in a registry of parser constructors and returns the right object through the `Parser` interface (falling back to `PlainTextParser`); the editor depends only on `Parser` and the factory.

2011–2012 • Q1(b) CSE307 • 23/09/2013 • D035.2 MARKS $18\frac{2}{3}$

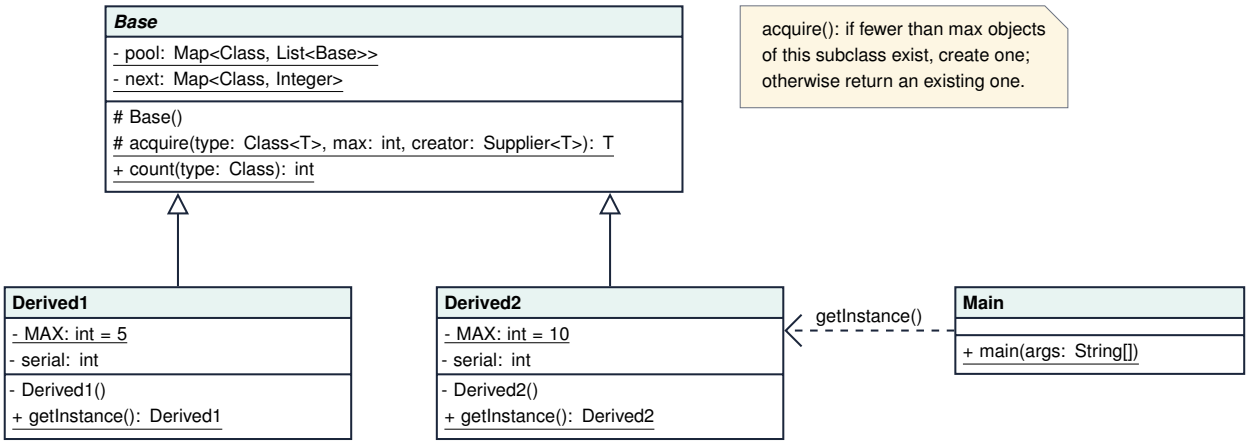
Write a Java code fragment based on Singleton Pattern. There should be class called `Base`, which can be inherited by other classes. Define two classes called `Derived1` and `Derived2` that will inherit `Base`. The maximum number of instances of `Derived1` and `Derived2` that can be created is 5 and 10 respectively. Write a main function where two arrays of `Derived1` and `Derived2` are created respectively and show the Singleton implementation.

SOLUTION

Idea: a Singleton that "permits a variable number of instances"

The classic Singleton allows exactly one instance. The slides list as a consequence of Singleton that it *"permits variable number of instances"*: since all creation is funnelled through one static access method, the same technique can cap the number of objects at any N. Here $N = 5$ for `Derived1` and $N = 10$ for `Derived2`.
Design decisions:

- `Base` is **abstract** with a **protected** constructor, so it can be inherited but not instantiated directly.
- `Base` contains the reusable *instance-limiting* logic in a protected static method `acquire(type, max, creator)`: it keeps, per subclass, the list of objects created so far. While the list is shorter than `max` a new object is created; afterwards an existing object is returned (round-robin). Thus there are *never* more than `max` objects.
- Each subclass has a **private constructor** (no `new Derived1()` from outside) and a **public static** `getInstance()` – the global access point – which passes its own limit.
- `acquire` is `synchronized`, so the limit also holds with several threads.



Java code

```java
import java.util.*;
import java.util.function.Supplier;

// ==================== Base: inheritable, instance-limited ====================
abstract class Base {
    // per concrete subclass: objects created so far, and next index for round-robin reuse
    private static final Map<Class<?>, List<Base>> pool = new HashMap<>();
    private static final Map<Class<?>, Integer>    next = new HashMap<>();

    protected Base() { }                          // only subclasses can call it

    // Singleton logic generalised to "at most max instances"
    protected static synchronized <T extends Base> T acquire(Class<T> type, int max,
                                                             Supplier<T> creator) {
        List<Base> list = pool.computeIfAbsent(type, k -> new ArrayList<>());
        if (list.size() < max) {                  // limit not reached -> create a new object
            T obj = creator.get();
            list.add(obj);
            return obj;
        }
        int i = next.getOrDefault(type, 0);       // limit reached -> hand out an existing one
        next.put(type, (i + 1) % max);
        return type.cast(list.get(i));
    }

    public static synchronized int count(Class<?> type) {
        return pool.getOrDefault(type, Collections.emptyList()).size();
```

```
    }
}

// ====================== Derived1: at most 5 objects ======================
class Derived1 extends Base {
    private static final int MAX = 5;
    private static int created = 0;
    private final int serial;

    private Derived1() { serial = ++created; }    // private: 'new Derived1()' impossible
    outside

    public static Derived1 getInstance() {        // global access point
        return acquire(Derived1.class, MAX, Derived1::new);
    }
    public String toString() { return "D1#" + serial; }
}

// ====================== Derived2: at most 10 objects ======================
class Derived2 extends Base {
    private static final int MAX = 10;
    private static int created = 0;
    private final int serial;

    private Derived2() { serial = ++created; }

    public static Derived2 getInstance() {
        return acquire(Derived2.class, MAX, Derived2::new);
    }
    public String toString() { return "D2#" + serial; }
}

// ====================== main ======================
public class Main {
    public static void main(String[] args) {
        Derived1[] arr1 = new Derived1[7];         // ask for MORE than the limit of 5
        Derived2[] arr2 = new Derived2[12];        // ask for MORE than the limit of 10

        for (int i = 0; i < arr1.length; i++) arr1[i] = Derived1.getInstance();
        for (int i = 0; i < arr2.length; i++) arr2[i] = Derived2.getInstance();

        System.out.println("arr1 = " + Arrays.toString(arr1));
        System.out.println("arr2 = " + Arrays.toString(arr2));
        System.out.println("Derived1 objects created: " + Base.count(Derived1.class));  // 5
        System.out.println("Derived2 objects created: " + Base.count(Derived2.class));  // 10
        System.out.println("arr1[5] == arr1[0] ? " + (arr1[5] == arr1[0]));            // true

        // Derived1 d = new Derived1();   // COMPILE ERROR: constructor is private
    }
}
```

Output:

```
arr1 = [D1#1, D1#2, D1#3, D1#4, D1#5, D1#1, D1#2]
arr2 = [D2#1, D2#2, D2#3, D2#4, D2#5, D2#6, D2#7, D2#8, D2#9, D2#10, D2#1, D2#2]
Derived1 objects created: 5
Derived2 objects created: 10
arr1[5] == arr1[0] ? true
```

How the Singleton idea is shown

- Private constructors + static `getInstance()` = controlled access to the instances (slides: "controlled access to sole instance").
- The limit is enforced in one place (`Base.acquire`); a new subclass, say `Derived3` with a limit of 3, only writes a private constructor and a one-line `getInstance()`.
- With `MAX = 1` the same code degenerates to the ordinary Singleton.

> **NOTE** The question does not say what should happen when the limit is reached. We return an already existing object (round-robin), which keeps the Singleton contract "`getInstance()` always returns a valid object". An equally acceptable alternative is to throw an `IllegalStateException` (or return `null`) once `max` objects exist; then the arrays in `main` would be created with exactly 5 and 10 elements.

Answer. `Base` (abstract, protected constructor) provides a synchronized static `acquire()` that counts instances per subclass; `Derived1`/`Derived2` have private constructors and static `getInstance()` methods with limits 5 and 10. `main` fills two arrays through `getInstance()` and shows that only 5 and 10 distinct objects ever exist.

2010–2011 • Q3(d) + Q4(a) CSE307 • 19/03/2012 • D038 **MARKS 5 + 5**

3(d) Does the following code fragment implement the **Factory design pattern**? Explain in brief.

```
public class XMLReaderFactory {
    // This method returns an instance of a class
    // that implements the XMLReader interface.
    // the specific class it creates and returns is
    // based on a system property.
    public static XMLReader createXMLReader();
}
public interface XMLReader {
    public void setContentHandler(ContentHandler handler);
    public void parse(InputStream is);
}
```

4(a) Which of the following (i) and (ii) is a valid implementation of **Singleton Pattern**? Give specific reason for each of the following implementation.

(i)

```
public class Singleton {
  private static Singleton s_singleton = new
    Singleton();
  private Singleton() { }
  private static Singleton getInstance() {
      return s_singleton;
  }
}
```

(ii)

```
public class Singleton {
  private Singleton() { }
  private static class SingletonHolder {
    private static final Singleton instance =
     new Singleton();
  }
  public static Singleton getInstance() {
      return SingletonHolder.instance;
  }
}
```

SOLUTION

3(d) Is `XMLReaderFactory` a Factory? – Yes

A factory is a component whose job is to *create objects for the client*, returning them through an abstract type so that the client does not know (or name) the concrete class. Check the fragment

against this idea:

- `createXMLReader()` is a **creation method** whose return type is the **interface** `XMLReader` – the client codes to the interface, not to an implementation.
- The **concrete class is chosen inside the factory** (from a system property) – the client never writes `new SomeXMLReaderImpl();` changing the property swaps the implementation without touching client code ("removes the instantiation of actual implementation classes from client code", slides).
- All implementations share the same contract (`setContentHandler`, `parse`), so they are interchangeable products.

Hence it **does implement the Factory pattern**, in its *simple / static factory* form – the same shape as the slides' `ShapeFactory.drawShape(type)` and `GetPayment.getProcessor(type)`. (This is in fact the real SAX API class `org.xml.sax.helpers.XMLReaderFactory`.)
Strictly speaking it is not the GoF *Factory Method*, because there is no creator hierarchy whose *subclasses* decide the product; a static method cannot be overridden. (As printed, the static method also lacks a body; in real code it contains the property lookup and returns the instance.)

```
XMLReader reader = XMLReaderFactory.createXMLReader();   // client knows only the interface
reader.setContentHandler(myHandler);
reader.parse(inputStream);
```

4(a)(i) – Not a valid (usable) Singleton

- ✓ Private constructor and a private static instance created eagerly – only one object exists.
- × But `getInstance()` is declared `private`. No other class can obtain the instance, so the second half of the Singleton intent – "*provide a global point of access to it*" – is violated. The class is useless to clients.
- Fix: `public static Singleton getInstance()` (and preferably make the field `final`). With that fix it is a correct *eager* singleton.

4(a)(ii) – Valid Singleton (initialization-on-demand holder)

- ✓ Private constructor: no outside instantiation.
- ✓ Public static `getInstance()`: global access point.
- ✓ The instance lives in the nested class `SingletonHolder`, which the JVM loads only when `getInstance()` first refers to it ⇒ **lazy** initialization.
- ✓ Class initialisation is guaranteed by the JVM to happen once and atomically, so the creation is **thread-safe without `synchronized`**; the `final` field cannot be replaced.

Answer. 3(d): Yes – a static (simple) factory: `createXMLReader()` hides which concrete class is instantiated and returns it via the `XMLReader` interface. 4(a): (i) is *invalid* because `getInstance()` is private, so there is no global access point; (ii) is a *valid*, lazy and thread-safe singleton (holder idiom).

03

Structural Design Patterns

2023–2024 • Q2(a) CSE213 • 10/05/2026 • D002 **MARKS 20**

FinCorp is a financial consultancy firm that originally operated with one branch in several states across the United States. Every month, the headquarters received state-wise expenditure reports of operational expenses. Recently, FinCorp expanded significantly. It now operates multiple offices across different cities within each state and has also expanded internationally with offices in several European countries. As a result, reporting requirements have become more complex. Different stakeholders require expenditure summaries at multiple levels:

- A State Director needs a consolidated report for all cities within a state
- A Country Head needs a report aggregating all states within a country
- The Global CFO needs a complete worldwide report

Each office (city, state, country) maintains its own expenditure data. Higher-level reports must aggregate data from all subordinate units, perform analysis on the aggregated data, and attach the results to the report. The nature of this analysis and processing may vary across organizational levels.

Besides, some additional sections might be provided with the original report, such as, tax calculation details, investment summaries, and government compliance reports. These additional sections may or may not be included, and a single report may include one or more of them based on instructions, without changing the core reporting structure.

As a member of the development team, you must redesign the reporting system so that it supports (i) the hierarchical structure while keeping the reporting interface consistent for all levels and (ii) the flexible inclusion of additional report sections based on requirements.

Identify the appropriate design pattern(s) to satisfy these requirements, and provide an implementation with the minimum required functionality.

SOLUTION

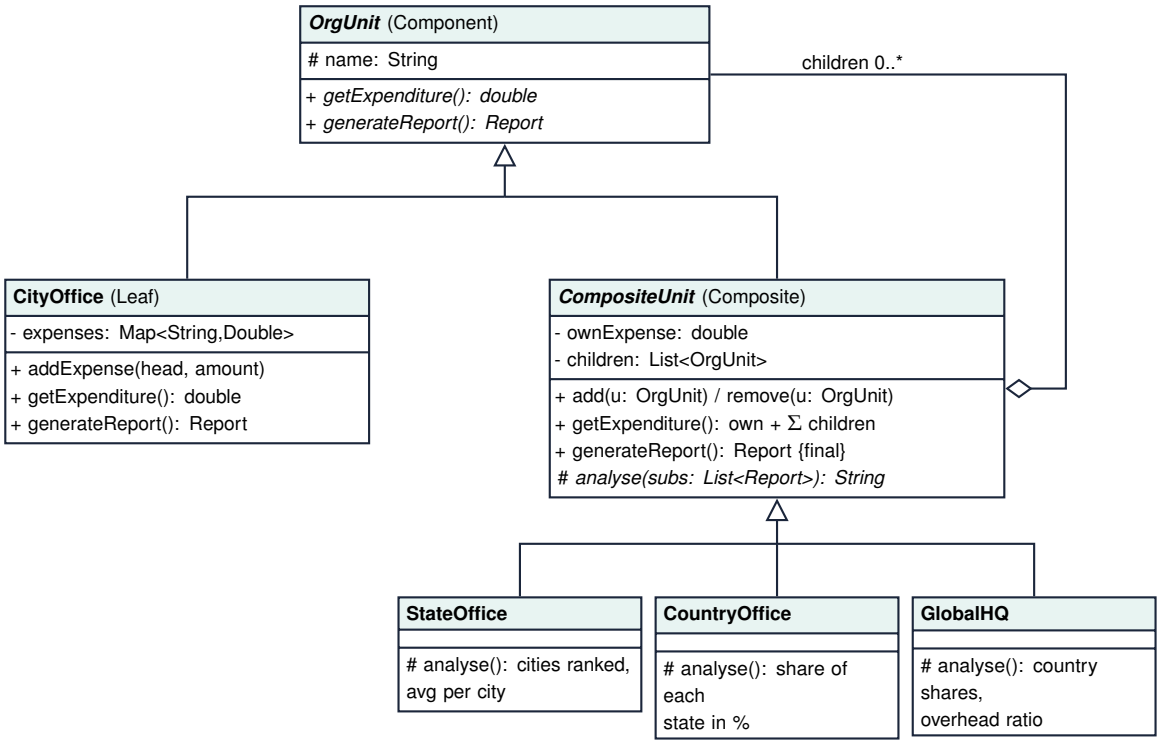
Step 1 – Identifying the patterns

The question names two separate forces, and each one points to a different structural pattern.

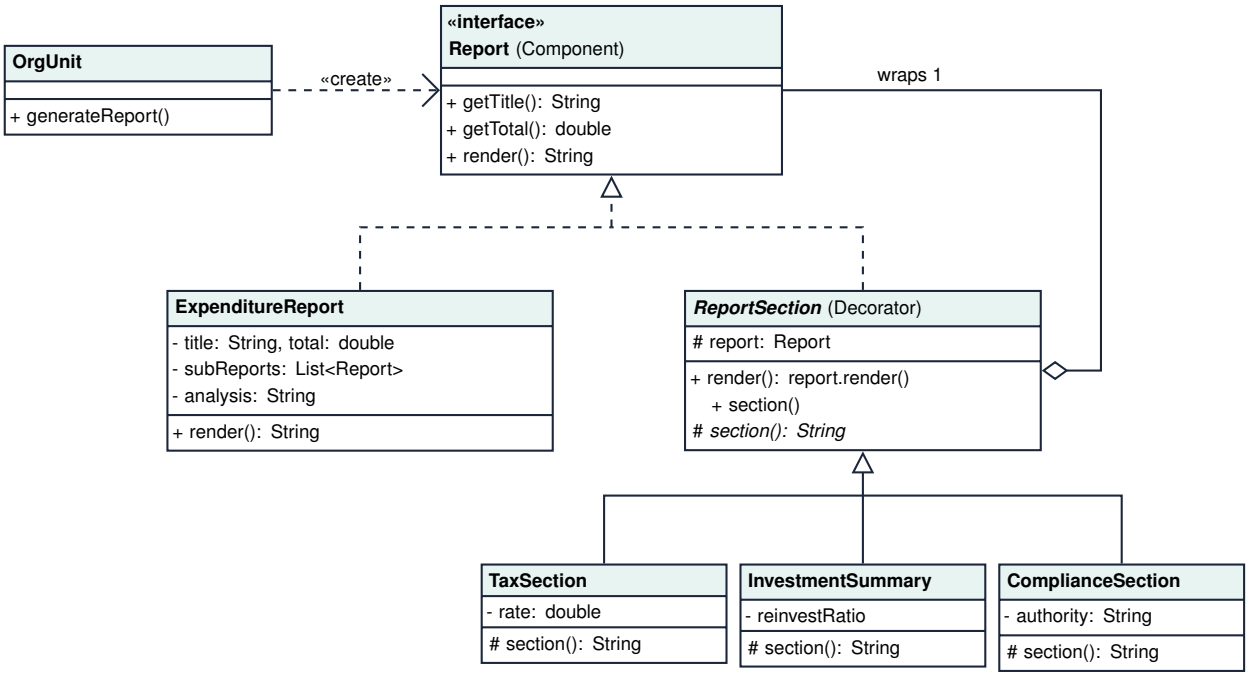
Force in the problem	Why a pattern is needed	Pattern
(i) City ⊂ State ⊂ Country ⊂ Global. Every level must produce a report through the *same* interface, and higher levels aggregate their subordinates.	A part–whole tree in which a single office and a group of offices must be treated uniformly.	**Composite**
(ii) Tax, investment and compliance sections are optional and can be combined in any way, "without changing the core reporting structure".	Responsibilities must be added to one report object at run time without subclassing every combination.	**Decorator**
(iii) "The nature of the analysis may vary across levels."	The aggregation algorithm is fixed, but one step (the analysis) differs per level.	hook method

- **Composite** lets you compose objects into tree structures and then work with these structures as if they were individual objects. The State Director, Country Head and CFO all call the same `generateReport()`, only on different nodes.
- **Decorator** lets you attach new behaviours to objects by placing them inside wrapper objects. Each optional section is a wrapper around the core `Report`.
- The varying analysis is a protected *hook* `analyse()` that each composite level overrides. The aggregation steps stay fixed in the composite (a Template-Method style hook inside the Composite).

Step 2 – Class diagram (a): Composite for the organisation



Step 3 – Class diagram (b): Decorator for the report sections



Step 4 – Java implementation (minimum required functionality)

```
import java.util.*;

// ================== REPORT side : Decorator ==================
interface Report {
    String getTitle();
    double getTotal();
    String render();
}

class ExpenditureReport implements Report {              // concrete component
    private final String title, analysis;
    private final double total;
    private final List<Report> subReports;
    ExpenditureReport(String title, double total, List<Report> subs, String analysis) {
        this.title = title;       this.total = total;
        this.subReports = subs;   this.analysis = analysis;
    }
    public String getTitle() { return title; }
    public double getTotal() { return total; }
    public String render() {
        StringBuilder sb = new StringBuilder();
        sb.append(String.format("== %s : total $%,.0f%n", title, total));
        for (Report r : subReports)
            sb.append(String.format("   - %-22s $%,.0f%n", r.getTitle(), r.getTotal()));
        if (!analysis.isEmpty()) sb.append("   Analysis: ").append(analysis).append('\n');
        return sb.toString();
    }
}

abstract class ReportSection implements Report {         // decorator
```

```
    protected final Report report;
    ReportSection(Report report) { this.report = report; }
    public String getTitle() { return report.getTitle(); }
    public double getTotal() { return report.getTotal(); }
    public String render()   { return report.render() + section(); }
    protected abstract String section();
}
class TaxSection extends ReportSection {
    private final double rate;
    TaxSection(Report r, double rate) { super(r); this.rate = rate; }
    protected String section() {
        return String.format("   [Tax] deductible $%,.0f, saving @%.0f%% = $%,.0f%n",
                             getTotal(), rate * 100, getTotal() * rate);
    }
}
class InvestmentSummary extends ReportSection {
    InvestmentSummary(Report r) { super(r); }
    protected String section() {
        return String.format("   [Investment] reinvest 10%% = $%,.0f%n",
                             getTotal() * 0.10);
    }
}
class ComplianceSection extends ReportSection {
    private final String authority;
    ComplianceSection(Report r, String authority) {
        super(r); this.authority = authority;
    }
    protected String section() {
        return "   [Compliance] report filed with " + authority + "\n";
    }
}

// ================== ORGANISATION side : Composite ==================
abstract class OrgUnit {
    protected final String name;
    OrgUnit(String name) { this.name = name; }
    String getName() { return name; }
    abstract double getExpenditure();
    abstract Report generateReport();                    // SAME interface at all levels
}

class CityOffice extends OrgUnit {                        // leaf
    private final Map<String, Double> expenses = new LinkedHashMap<>();
    CityOffice(String name) { super(name); }
    void addExpense(String head, double amount) {
        expenses.merge(head, amount, Double::sum);
    }
    double getExpenditure() {
        double t = 0;
        for (double a : expenses.values()) t += a;
        return t;
    }
    Report generateReport() {
        List<Report> lines = new ArrayList<>();
        expenses.forEach((h, a) ->
            lines.add(new ExpenditureReport(h, a, List.of(), "")));
        return new ExpenditureReport("City " + name, getExpenditure(), lines, "");
    }
```

```
}

abstract class CompositeUnit extends OrgUnit {            // composite
    private final double ownExpense;                       // the office's own data
    protected final List<OrgUnit> children = new ArrayList<>();
    CompositeUnit(String name, double ownExpense) {
        super(name); this.ownExpense = ownExpense;
    }
    void add(OrgUnit u)    { children.add(u); }
    void remove(OrgUnit u) { children.remove(u); }
    double getExpenditure() {
        double t = ownExpense;
        for (OrgUnit c : children) t += c.getExpenditure();      // recursive aggregation
        return t;
    }
    final Report generateReport() {                          // fixed algorithm
        List<Report> subs = new ArrayList<>();
        for (OrgUnit c : children) subs.add(c.generateReport());  // 1. aggregate
        String result = analyse(subs);                            // 2. level-specific analysis
        return new ExpenditureReport(level() + " " + name,        // 3. attach to report
                                     getExpenditure(), subs, result);
    }
    protected double own() { return ownExpense; }
    protected abstract String level();
    protected abstract String analyse(List<Report> subs);    // varies per level
}

class StateOffice extends CompositeUnit {
    StateOffice(String n, double own) { super(n, own); }
    protected String level() { return "State"; }
    protected String analyse(List<Report> subs) {
        Report max = Collections.max(subs, Comparator.comparingDouble(Report::getTotal));
        double avg = subs.stream().mapToDouble(Report::getTotal).average().orElse(0);
        return String.format("highest-spending city = %s, average per city = $%,.0f",
                             max.getTitle(), avg);
    }
}
class CountryOffice extends CompositeUnit {
    CountryOffice(String n, double own) { super(n, own); }
    protected String level() { return "Country"; }
    protected String analyse(List<Report> subs) {
        StringBuilder sb = new StringBuilder("state shares:");
        for (Report r : subs)
            sb.append(String.format(" %s %.1f%%", r.getTitle(),
                                    100 * r.getTotal() / getExpenditure()));
        return sb.toString();
    }
}
class GlobalHQ extends CompositeUnit {
    GlobalHQ(String n, double own) { super(n, own); }
    protected String level() { return "Global"; }
    protected String analyse(List<Report> subs) {
        Report max = Collections.max(subs, Comparator.comparingDouble(Report::getTotal));
        return String.format("largest market = %s; HQ overhead = %.1f%% of world spend",
                             max.getTitle(), 100 * own() / getExpenditure());
    }
}
```

```java
// ================== Client ==================
public class FinCorpReporting {
    static CityOffice city(String name, double rent, double salary) {
        CityOffice c = new CityOffice(name);
        c.addExpense("Rent", rent);
        c.addExpense("Salary", salary);
        return c;
    }
    public static void main(String[] args) {
        CityOffice la  = city("Los Angeles",   30000,  90000);
        CityOffice sf  = city("San Francisco", 40000,  70000);
        CityOffice nyc = city("New York",      50000, 120000);
        CityOffice mun = city("Munich",        25000,  60000);

        StateOffice ca = new StateOffice("California", 10000); ca.add(la); ca.add(sf);
        StateOffice ny = new StateOffice("New York", 8000);     ny.add(nyc);
        StateOffice by = new StateOffice("Bavaria", 5000);      by.add(mun);
        CountryOffice usa = new CountryOffice("USA", 20000);    usa.add(ca); usa.add(ny);
        CountryOffice ger = new CountryOffice("Germany", 7000); ger.add(by);
        GlobalHQ world = new GlobalHQ("FinCorp", 50000);   world.add(usa); world.add(ger);

        // (i) same interface at every level
        Report stateDirector = ca.generateReport();
        Report countryHead   = new TaxSection(usa.generateReport(), 0.21);
        // (ii) any combination of optional sections, chosen at run time
        Report globalCfo = new ComplianceSection(
                               new InvestmentSummary(
                                   new TaxSection(world.generateReport(), 0.21)),
                               "SEC (US) and BaFin (EU)");

        System.out.println(stateDirector.render());
        System.out.println(countryHead.render());
        System.out.println(globalCfo.render());
    }
}
```

Step 5 – What the stakeholders get (sample totals)

Stakeholder	Call	Total ($)
State Director (California)	`ca.generateReport()`	$10{,}000 + 120{,}000 + 110{,}000 = 240{,}000$
Country Head (USA)	`new TaxSection( usa.generateReport(), 0.21)`	$20{,}000 + 240{,}000 + 178{,}000 = 438{,}000$
Global CFO	3 decorators around `world.generateReport()`	$50{,}000 + 438{,}000 + 97{,}000 = 585{,}000$

Step 6 – How the design meets the requirements

- **Consistent interface at all levels:** every node is an `OrgUnit` with `generateReport()`. Adding a new level, such as a *Region* between Country and Global, is one new `CompositeUnit` subclass. Existing code is unchanged (OCP).
- **Aggregation + analysis + attach** is written once in `CompositeUnit.generateReport()`. Only the analysis step varies, through the `analyse()` hook.

- **Optional sections:** any subset and order of `TaxSection`, `InvestmentSummary` and `ComplianceSection` wraps the core report. The core `ExpenditureReport` never changes, and a new section type (for example an ESG section) is one new decorator.
- The two patterns are independent. The Composite produces a `Report`, and the Decorator enriches whatever `Report` it receives.

Answer. Use **Composite** for the City/State/Country/Global hierarchy (`OrgUnit` is the component, `CityOffice` the leaf and `CompositeUnit` the composite, with per-level `analyse()`), so that every level offers the same `generateReport()`. Use **Decorator** for the optional tax, investment and compliance sections, which wrap the core `Report` in any combination.

2022–2023 • Q1(a) CSE307 • 05/05/2024 • D008 MARKS 15

You have a service which returns the temperature (in celsius) by passing city name as a input value. Now, assume that your client wants to pass zipcode as input and expecting the temperature of the city in return. Develop a class diagram to present appropriate design pattern and write necessary codes so that you code fulfills all the requirements.

SOLUTION

Step 1 – Identifying the pattern

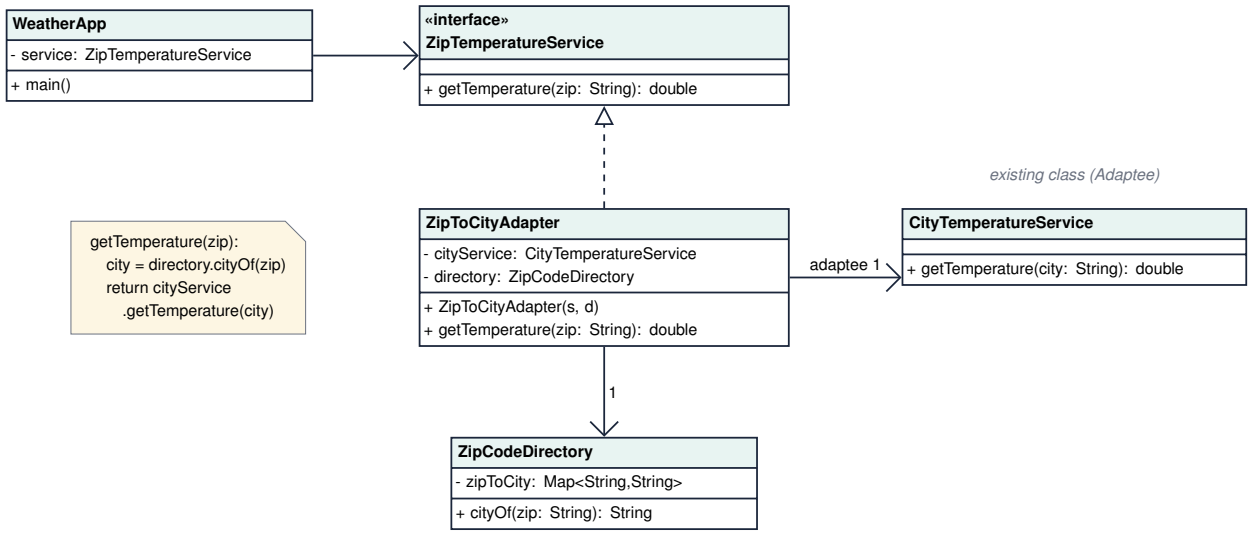
- An **existing service** already does the real work: `getTemperature(cityName)`. We should not rewrite or modify it, because it is tested and may be a third-party component.
- The **client expects a different interface**: `getTemperature(zipCode)`.
- The only gap is an *interface mismatch*. Zip code has to be translated to city before the call is forwarded.

This is exactly the **Adapter** pattern: *convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.* In the lecture's Duck–Turkey example a `TurkeyAdapter` made a Turkey look like a Duck. Here a `ZipToCityAdapter` makes the city-based service look like a zip-based one.

Step 2 – Participants

Adapter role	Class
Target (what the client wants)	`ZipTemperatureService` – `getTemperature(zip)`
Adaptee (existing, unchanged)	`CityTemperatureService` – `getTemperature(city)`
Adapter	`ZipToCityAdapter` – implements Target, wraps Adaptee (object adapter)
Helper	`ZipCodeDirectory` – maps a zip code to a city name
Client	`WeatherApp`

Step 3 – Class diagram



Step 4 – Java implementation

```java
import java.util.HashMap;
import java.util.Map;

// ---------- Adaptee : the existing service (NOT modified) ----------
class CityTemperatureService {
    private final Map<String, Double> table = new HashMap<>();
    CityTemperatureService() {
        table.put("Dhaka", 31.5); table.put("Chattogram", 29.0); table.put("Sylhet", 27.4);
    }
    public double getTemperature(String city) {         // degree Celsius
        Double t = table.get(city);
        if (t == null) throw new IllegalArgumentException("Unknown city: " + city);
        return t;
    }
}

// ---------- Target : what the client wants ----------
interface ZipTemperatureService {
    double getTemperature(String zipCode);
}

// ---------- Helper used by the adapter ----------
class ZipCodeDirectory {
    private final Map<String, String> zipToCity = new HashMap<>();
    ZipCodeDirectory() {
        zipToCity.put("1000", "Dhaka"); zipToCity.put("1205", "Dhaka");
        zipToCity.put("4000", "Chattogram"); zipToCity.put("3100", "Sylhet");
    }
    public String cityOf(String zip) {
        String c = zipToCity.get(zip);
        if (c == null) throw new IllegalArgumentException("Unknown zip: " + zip);
        return c;
    }
}

// ---------- Adapter (object adapter: implements Target, HAS-A Adaptee) ----------
```

```java
class ZipToCityAdapter implements ZipTemperatureService {
    private final CityTemperatureService cityService;
    private final ZipCodeDirectory directory;
    ZipToCityAdapter(CityTemperatureService s, ZipCodeDirectory d) {
        this.cityService = s; this.directory = d;
    }
    @Override
    public double getTemperature(String zipCode) {
        String city = directory.cityOf(zipCode);      // translate the request
        return cityService.getTemperature(city);      // delegate to the adaptee
    }
}

// ---------- Client : knows only the Target interface ----------
public class WeatherApp {
    public static void main(String[] args) {
        ZipTemperatureService service =
            new ZipToCityAdapter(new CityTemperatureService(), new ZipCodeDirectory());
        for (String zip : new String[]{"1000", "4000", "3100"})
            System.out.println("Zip " + zip + " -> " + service.getTemperature(zip) + " C");
    }
}
```

Output: `Zip 1000 -> 31.5 C`, `Zip 4000 -> 29.0 C`, `Zip 3100 -> 27.4 C`.

Step 5 – Discussion

- **Existing code untouched:** `CityTemperatureService` is reused as it is (Open–Closed).
- **Client decoupled:** `WeatherApp` depends only on `ZipTemperatureService`. If a real zip-based weather API appears later, it can implement the same interface directly and the client does not change.
- **Object vs. class adapter:** composition is used (object adapter), so it works in Java and with any subclass of the adaptee. A class adapter would be `class ZipToCityAdapter extends CityTemperatureService implements ZipTemperatureService`.

Answer. Adapter pattern (object adapter). `ZipToCityAdapter` implements the client's target interface `getTemperature(zip)`, converts the zip code to a city through `ZipCodeDirectory`, and delegates to the unchanged `CityTemperatureService.getTemperature(city)`.

2021–2022 • Q2(a) CSE307 • 07/05/2023 • D013 MARKS 20

Let's assume you are building your own computer networking model. Your model has five (5) layers. Such as: Physical Layer, Data-link Layer, Network Layer, Transport Layer and Application Layer. When an user wants to send a message (e.g. "*hello*") using your designed model, it works like this:

- First user creates a *Packet* object. Packet object has two properties: *Message* and *Length*. If user wants to send the message "*hello*", the Packet object would look like this:
  message: hello, length: 5
- Then the *Packet* object is received by the Application layer. Application Layer adds a header to the original *Packet* object. *Packet* object would look like this:
  message: APPHEADER_hello, length: 15

- Then the *Packet* object is received by the Transport layer. Transport Layer adds a header to the received *Packet* object. *Packet* object would look like this.
  message: TRHEADER_APPHEADER_hello, length: 24
- Then the *Packet* object is received by the Network layer. Network Layer adds a header to the received *Packet* object. *Packet* object would look like this.
  message: NETHEADER_TRHEADER_APPHEADER_hello, length: 34
- Then the *Packet* object is received by the Data-link layer. Data-link Layer adds a header and a trailer to the received *Packet* object. *Packet* object would look like this.
  message: DHEADER_NETHEADER_TRHEADER_APPHEADER_hello_DTRAILER, length: 51
- Then the *Packet* object is received by the Physical layer. Physical Layer adds a header to the received *Packet* object. *Packet* object would look like this.
  message:
  PHEADER_DHEADER_NETHEADER_TRHEADER_APPHEADER_hello_DTRAILER,
  length: 59

  And returns this as the final *Packet* object.

Identify the design pattern that can best capture the scenario above and implement it using your preferred language (you can use pseudo syntax). You **must** provide the UML class diagram of your implementation.

SOLUTION

NOTE The bank's context note says the final Physical-layer length is printed as 39, while the scan reads "59". Counting characters settles it: `PHEADER_` has 8 characters, so the length is $51 + 8 = 59$. In the solution the length is never stored. It is always computed as `message.length()`, so the code gives 59 automatically and cannot contain a typed-in error like 39.

Step 1 – What is happening in the scenario?

One *and the same Packet* passes through a stack of layers. Each layer

- accepts *any* packet, whether it is a raw packet or one already processed by another layer,
- adds its own extra part (a header, plus a trailer at the data-link layer) around the existing content, and
- hands on something that is *still a Packet*, so the next layer treats it exactly like the original.

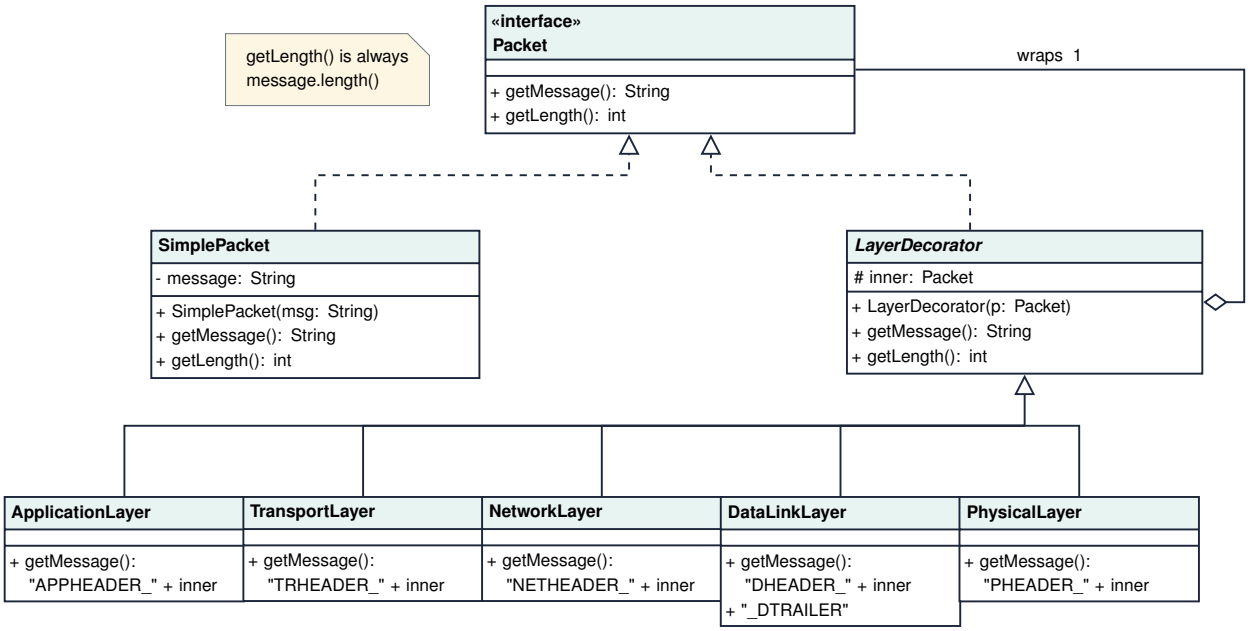
So each layer adds responsibilities to the object while keeping its interface. We do not want subclasses such as `AppPacket`, `TrAppPacket`, `NetTrAppPacket`, ... for every combination. That would cause the *subclass explosion* problem shown in the lecture's Starbuzz-coffee example.

Step 2 – The pattern: Decorator

Decorator lets you attach new behaviours to objects by placing these objects inside special wrapper objects that contain the behaviours. A decorator has the *same supertype* as the object it wraps, holds a reference to it (*HAS-A*) and adds its behaviour before or after delegating to it. Wrappers can be stacked in any number and order at run time, exactly as the five layers wrap the packet one after another.

Decorator role	Class in our model
Component (common interface)	`Packet` – `getMessage()`, `getLength()`
Concrete component	`SimplePacket` – the user's original message
Decorator (abstract wrapper)	`LayerDecorator` – holds the wrapped `Packet`
Concrete decorators	`ApplicationLayer`, `TransportLayer`, `NetworkLayer`, `DataLinkLayer`, `PhysicalLayer`
Client	`NetworkModel` / `main` – builds the wrapper chain

Step 3 – UML class diagram



The hollow-triangle dashed arrows show that both the raw packet and every layer *are* Packets (same type). The aggregation from `LayerDecorator` to `Packet` shows that each layer *has* the packet it wraps.

Step 4 – Java implementation

```
// ---------- Component ----------
interface Packet {
    String getMessage();
    int    getLength();
}

// ---------- Concrete component ----------
class SimplePacket implements Packet {
    private final String message;
    SimplePacket(String message) { this.message = message; }
    public String getMessage() { return message; }
    public int    getLength()  { return message.length(); }
}

// ---------- Abstract decorator ----------
abstract class LayerDecorator implements Packet {
    protected final Packet inner;                // the wrapped packet (HAS-A)
```

```
    LayerDecorator(Packet inner) { this.inner = inner; }
    public abstract String getMessage();          // each layer decides what to add
    public int getLength() { return getMessage().length(); }
}

// ---------- Concrete decorators (the five layers) ----------
class ApplicationLayer extends LayerDecorator {
    ApplicationLayer(Packet p) { super(p); }
    public String getMessage() { return "APPHEADER_" + inner.getMessage(); }
}
class TransportLayer extends LayerDecorator {
    TransportLayer(Packet p) { super(p); }
    public String getMessage() { return "TRHEADER_" + inner.getMessage(); }
}
class NetworkLayer extends LayerDecorator {
    NetworkLayer(Packet p) { super(p); }
    public String getMessage() { return "NETHEADER_" + inner.getMessage(); }
}
class DataLinkLayer extends LayerDecorator {        // header AND trailer
    DataLinkLayer(Packet p) { super(p); }
    public String getMessage() { return "DHEADER_" + inner.getMessage() + "_DTRAILER"; }
}
class PhysicalLayer extends LayerDecorator {
    PhysicalLayer(Packet p) { super(p); }
    public String getMessage() { return "PHEADER_" + inner.getMessage(); }
}

// ---------- Client ----------
public class NetworkModel {
    static void show(String stage, Packet p) {
        System.out.println(stage + " -> message: " + p.getMessage()
                           + ", length: " + p.getLength());
    }
    public static void main(String[] args) {
        Packet p = new SimplePacket("hello");   show("User       ", p);
        p = new ApplicationLayer(p);            show("Application", p);
        p = new TransportLayer(p);              show("Transport  ", p);
        p = new NetworkLayer(p);                show("Network    ", p);
        p = new DataLinkLayer(p);               show("Data-link  ", p);
        p = new PhysicalLayer(p);               show("Physical   ", p);
        // the same chain in one expression:
        // new PhysicalLayer(new DataLinkLayer(new NetworkLayer(
        //     new TransportLayer(new ApplicationLayer(new SimplePacket("hello"))))));
    }
}
```

Step 5 – Trace of the program

After	getMessage()	getLength()
User	`hello`	5
Application	`APPHEADER_hello`	15
Transport	`TRHEADER_APPHEADER_hello`	24
Network	`NETHEADER_TRHEADER_APPHEADER_hello`	34
Data-link	`DHEADER_NETHEADER_..._hello_DTRAILER`	51
Physical	`PHEADER_DHEADER_NETHEADER_..._hello_DTRAILER`	59

Step 6 – Why this design is good

- **Open–Closed Principle:** a new layer (say an encryption or session layer) is just one new decorator class. No existing class changes.
- **Composition over inheritance:** layers are combined at run time. A model that skips a layer, or a test harness that uses only two layers, needs no new subclass.
- **Single Responsibility:** each layer class knows only its own header or trailer.
- **Transparency:** every layer sees a `Packet`. It does not know how many wrappers are inside it.

Answer. Decorator pattern. `Packet` is the component, `SimplePacket` is the concrete component, and the five layers are concrete decorators of `LayerDecorator`. Each one wraps the received packet and adds its header (and the data-link trailer). The length is derived from the message, which gives 5, 15, 24, 34, 51 and 59.

2020–2021 • Q1(a), (b) CSE307 • 19/10/2022 • D017.1 **MARKS 28 + 7**

(a) Consider a burger shop selling burger, appetizer and drinks. The following points describe the services provided to the customer.

(1) Three different types of pizza are being sold: Veggi, Beef and Chicken.

(2) There are provisions to add extra cheese on those pizza.

(3) There are provisions of different types of drinks: Coffee, Water and Coke. The prices of these different types of drinks are different. The customer can select one or more drinks in an order.

(4) Onion Rings and French Fries are two different types of appetizers available.

(5) A customer can order a meal which consists of an appetizer, a pizza with necessary cheese and drinks.

Construct a class indicating the following using three different patterns:

(1) Beef pizza with French fry and cheese

(2) Chicken Pizza with onion rings and Bottle of Water

(3) A combo meal with Veggi pizza, French Fry and two bottles of Coke

(4) A combo meal with Veggi pizza, Onion Rings, Coffee and Water

In each of the cases determine the price of the options shown above as a method of the class to be created.

(b) Discuss the relative advantages and disadvantages of using different patterns in constructing the class in the problem 1(a).

SOLUTION

NOTE No price list is given, and the shop is called a "burger shop" although only pizzas are described. The products are taken to be pizzas, and these sample prices (Tk) are used consistently in all three designs: Veggi 300, Beef 450, Chicken 400, Extra cheese 50, Coffee 120, Water 20, Coke 40, Onion Rings 150, French Fries 120. The paper supplied a sheet of pattern structures, and the three patterns chosen here are **Decorator**, **Builder** and **Composite**.

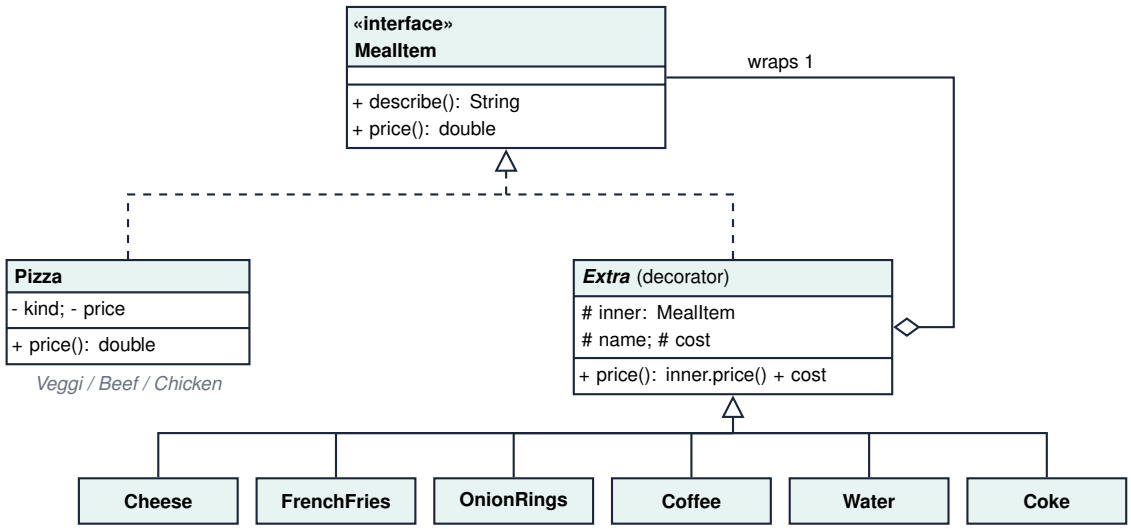
Expected prices (all three designs must reproduce these)

Case	Items	Price (Tk)
1	Beef pizza 450 + French fries 120 + cheese 50	620
2	Chicken pizza 400 + Onion rings 150 + Water 20	570
3	Veggi pizza 300 + French fries 120 + 2 × Coke 40	500
4	Veggi pizza 300 + Onion rings 150 + Coffee 120 + Water 20	590

Step 1 – Identifying suitable patterns

- A meal = a base pizza + optional extras (cheese, appetizer, drinks, even repeated ones). Adding priced extras at run time is the Starbuzz situation, so **Decorator** fits.
- A meal is a *complex object assembled step by step* (pizza → cheese → appetizer → drinks) with some optional parts, and the same steps produce different meals. That is the intent of **Builder** (creational).
- A combo meal is a *whole made of parts*, and a part can itself be a group (a drinks set). Its price is the sum over the tree, so **Composite** fits.

Design 1 – Decorator




```
interface MealItem { String describe(); double price(); }

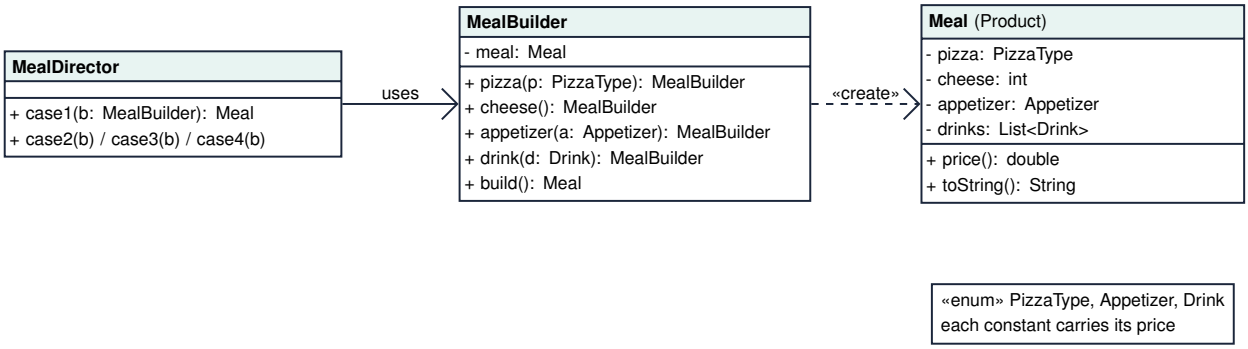
class Pizza implements MealItem {                       // concrete component
    private final String kind; private final double price;
    private Pizza(String k, double p) { kind = k; price = p; }
    static Pizza veggi()   { return new Pizza("Veggi pizza", 300); }
    static Pizza beef()    { return new Pizza("Beef pizza", 450); }
    static Pizza chicken() { return new Pizza("Chicken pizza", 400); }
    public String describe() { return kind; }
    public double price()    { return price; }
}

abstract class Extra implements MealItem {              // decorator
    protected final MealItem inner;
    private final String name;
    private final double cost;
```

```
    Extra(MealItem inner, String name, double cost) {
        this.inner = inner; this.name = name; this.cost = cost;
    }
    public String describe() { return inner.describe() + " + " + name; }
    public double price()    { return inner.price() + cost; }
}
class Cheese      extends Extra { Cheese(MealItem m)      { super(m, "cheese", 50); } }
class FrenchFries extends Extra { FrenchFries(MealItem m) { super(m, "French fries", 120); } }
class OnionRings  extends Extra { OnionRings(MealItem m)  { super(m, "Onion rings", 150); } }
class Coffee      extends Extra { Coffee(MealItem m)      { super(m, "Coffee", 120); } }
class Water       extends Extra { Water(MealItem m)       { super(m, "Water", 20); } }
class Coke        extends Extra { Coke(MealItem m)        { super(m, "Coke", 40); } }

class DecoratorDemo {
    public static void main(String[] a) {
        MealItem c1 = new Cheese(new FrenchFries(Pizza.beef()));                    // 620
        MealItem c2 = new Water(new OnionRings(Pizza.chicken()));                   // 570
        MealItem c3 = new Coke(new Coke(new FrenchFries(Pizza.veggi())));           // 500
        MealItem c4 = new Water(new Coffee(new OnionRings(Pizza.veggi())));         // 590
        for (MealItem m : new MealItem[]{c1, c2, c3, c4})
            System.out.println(m.describe() + " = Tk " + m.price());
    }
}
```

Design 2 – Builder




```
import java.util.*;

enum PizzaType {
    VEGGI(300), BEEF(450), CHICKEN(400);
    final double p; PizzaType(double p) { this.p = p; }
}
enum Appetizer {
    ONION_RINGS(150), FRENCH_FRIES(120);
    final double p; Appetizer(double p) { this.p = p; }
}
enum Drink {
    COFFEE(120), WATER(20), COKE(40);
    final double p; Drink(double p) { this.p = p; }
}

class Meal {                                          // product
    static final double CHEESE = 50;
    PizzaType pizza; int cheese; Appetizer appetizer;
    final List<Drink> drinks = new ArrayList<>();
```

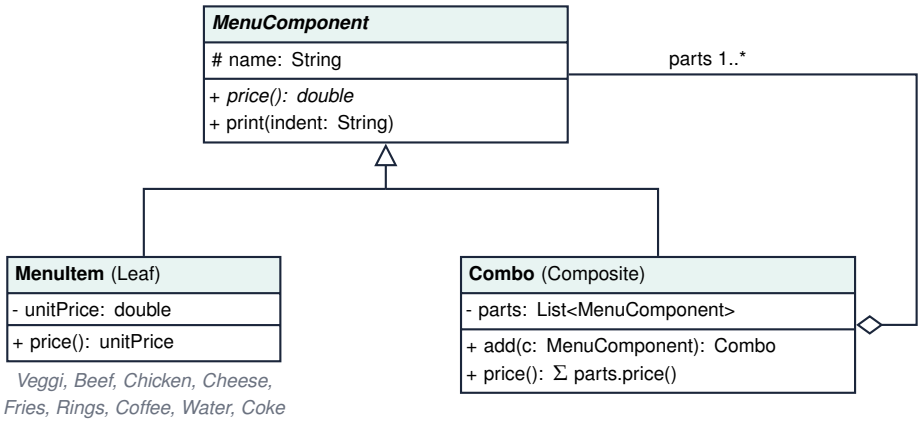
```
    double price() {
        double t = pizza.p + cheese * CHEESE;
        if (appetizer != null) t += appetizer.p;
        for (Drink d : drinks) t += d.p;
        return t;
    }
    public String toString() {
        return pizza + " cheese x" + cheese + " " + appetizer + " " + drinks;
    }
}

class MealBuilder {                                    // builder (fluent)
    private Meal meal = new Meal();
    MealBuilder pizza(PizzaType p)     { meal.pizza = p; return this; }
    MealBuilder cheese()               { meal.cheese++; return this; }
    MealBuilder appetizer(Appetizer a) { meal.appetizer = a; return this; }
    MealBuilder drink(Drink d)         { meal.drinks.add(d); return this; }
    Meal build() {
        if (meal.pizza == null) throw new IllegalStateException("A meal needs a pizza");
        Meal done = meal; meal = new Meal(); return done;
    }
}

class MealDirector {                                   // knows the recipes of the 4 options
    Meal case1(MealBuilder b) {          // beef pizza + French fries + cheese
        return b.pizza(PizzaType.BEEF).cheese()
                .appetizer(Appetizer.FRENCH_FRIES).build();
    }
    Meal case2(MealBuilder b) {          // chicken pizza + onion rings + water
        return b.pizza(PizzaType.CHICKEN).appetizer(Appetizer.ONION_RINGS)
                .drink(Drink.WATER).build();
    }
    Meal case3(MealBuilder b) {          // combo: veggi + fries + 2 coke
        return b.pizza(PizzaType.VEGGI).appetizer(Appetizer.FRENCH_FRIES)
                .drink(Drink.COKE).drink(Drink.COKE).build();
    }
    Meal case4(MealBuilder b) {          // combo: veggi + rings + coffee + water
        return b.pizza(PizzaType.VEGGI).appetizer(Appetizer.ONION_RINGS)
                .drink(Drink.COFFEE).drink(Drink.WATER).build();
    }
}

class BuilderDemo {
    public static void main(String[] a) {
        MealDirector d = new MealDirector(); MealBuilder b = new MealBuilder();
        for (Meal m : List.of(d.case1(b), d.case2(b), d.case3(b), d.case4(b)))
            System.out.println(m + " = Tk " + m.price());      // 620, 570, 500, 590
    }
}
```

Design 3 – Composite




```
import java.util.*;

abstract class MenuComponent {
    protected final String name;
    MenuComponent(String name) { this.name = name; }
    abstract double price();
    void print(String indent) { System.out.printf("%s%-16s Tk %.0f%n", indent, name, price()); }
}
class MenuItem extends MenuComponent {                    // leaf
    private final double unitPrice;
    MenuItem(String n, double p) { super(n); unitPrice = p; }
    double price() { return unitPrice; }
}
class Combo extends MenuComponent {                       // composite
    private final List<MenuComponent> parts = new ArrayList<>();
    Combo(String n) { super(n); }
    Combo add(MenuComponent c) { parts.add(c); return this; }
    double price() {
        double t = 0;
        for (MenuComponent c : parts) t += c.price();     // recursive sum
        return t;
    }
    @Override void print(String indent) {
        super.print(indent);
        for (MenuComponent c : parts) c.print(indent + "   ");
    }
}

class CompositeDemo {
    static MenuItem veggi()   { return new MenuItem("Veggi pizza", 300); }
    static MenuItem beef()    { return new MenuItem("Beef pizza", 450); }
    static MenuItem chicken() { return new MenuItem("Chicken pizza", 400); }
    static MenuItem cheese()  { return new MenuItem("Cheese", 50); }
    static MenuItem fries()   { return new MenuItem("French fries", 120); }
    static MenuItem rings()   { return new MenuItem("Onion rings", 150); }
    static MenuItem coffee()  { return new MenuItem("Coffee", 120); }
    static MenuItem coke()    { return new MenuItem("Coke", 40); }
    static MenuItem water()   { return new MenuItem("Water", 20); }

    public static void main(String[] a) {
```

```
        Combo beefWithCheese = new Combo("Beef + cheese").add(beef()).add(cheese());
        Combo c1 = new Combo("Option 1").add(beefWithCheese).add(fries());      // 620
        Combo c2 = new Combo("Option 2").add(chicken()).add(rings())
                                        .add(water());                          // 570
        Combo drinks3 = new Combo("Drinks").add(coke()).add(coke());
        Combo c3 = new Combo("Combo 3").add(veggi()).add(fries())
                                       .add(drinks3);                           // 500
        Combo drinks4 = new Combo("Drinks").add(coffee()).add(water());
        Combo c4 = new Combo("Combo 4").add(veggi()).add(rings())
                                       .add(drinks4);                           // 590
        for (Combo m : List.of(c1, c2, c3, c4)) m.print("");
    }
}
```

Each of the three programs prints the four prices **620, 570, 500 and 590**. In every design the price is a method of the class that is created (`price()`).

(b) Relative advantages and disadvantages

Pattern	Advantages here	Disadvantages here
Decorator	Extras are added at run time in any number (two Cokes = two wrappers). A new extra is one small class (OCP). No subclass explosion such as `BeefPizzaWithCheeseAndFries`.	Many tiny classes. An appetizer or drink is not really a "decoration" of the pizza, so the semantics are stretched. Rules such as "exactly one appetizer" or "cheese only on pizza" cannot be enforced, and order or identity checks are awkward. Debugging deep wrapper chains is harder.
Builder	Fits the "meal = pizza + cheese + appetizer + drinks" recipe exactly. Construction rules are validated in `build()`. The Director names the standard combos. The client code reads naturally.	Price logic sits in one `Meal.price()`, so a new kind of component changes `Meal` and the builder. It is less flexible for arbitrary nesting. There are extra classes (Builder, Director) for a simple product.
Composite	Natural for combos: a combo is a whole made of parts, and parts can be groups (a drinks set). The price is computed recursively and uniformly. Easy to print an itemised bill. New items are just data (no new class).	It is too general: nothing stops a combo with no pizza or with three appetizers. The type rules of a meal are lost (the "overly general design" consequence). Extra cheese is modelled as a separate item, not as a modification of the pizza.

Answer. Decorator, Builder and Composite each give a `price()` method. Cases 1–4 cost Tk 620, 570, 500 and 590 with the assumed prices. Decorator is best for run-time add-ons, Builder for rule-checked step-wise construction of a fixed meal structure, and Composite for nested combos and itemised bills, at the cost of weaker type constraints.

2020–2021 • Q2(a) CSE307 • 19/10/2022 • D017.2 **MARKS 10**

Consider a web-based system where users will select a mobile phone talk time scheme and the cost associated with the scheme will be displayed to the users. New schemes are added frequently. Demonstrate how you can decouple the programmers of the user interface from the classes implementing the schemes.

SOLUTION

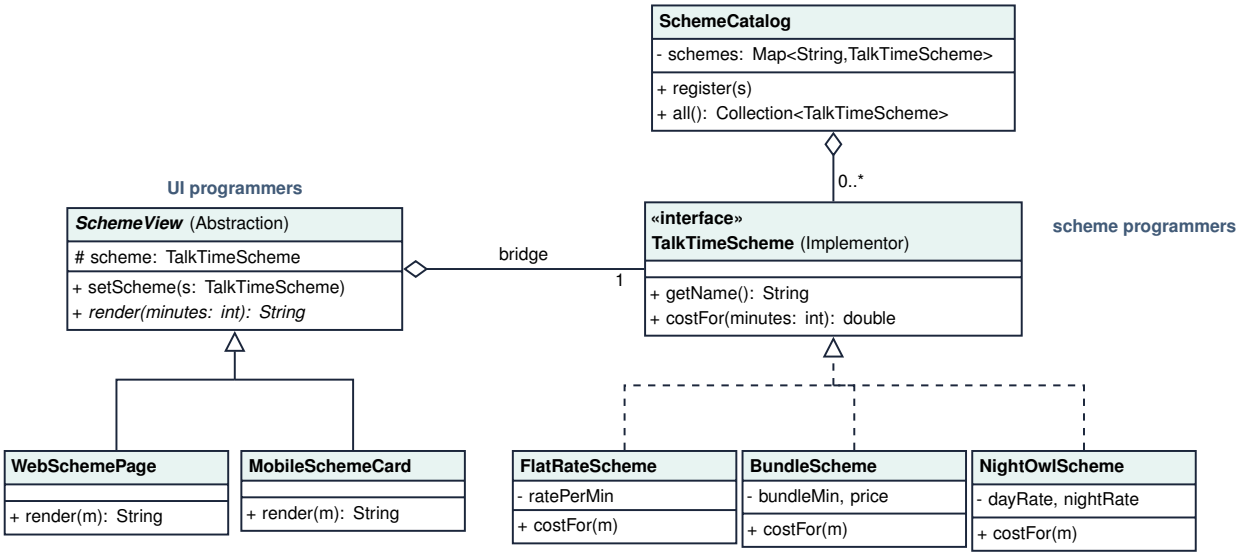
Step 1 – Identifying the pattern

There are two things that change for different reasons and are worked on by different people:

- the **user interface**: how schemes are listed, selected and their cost shown (web page, mobile view, admin view, . . .), and
- the **schemes**: the business rules that compute the talk-time cost. New schemes appear frequently.

If the UI classes referred to concrete scheme classes, every new scheme would force UI changes, and mixing the two dimensions through inheritance would give classes like `WebNightOwlScheme` or `MobileSaverScheme`. This is the same exponential growth as the lecture's "coloured shapes" example. The **Bridge** pattern *splits a large class or a set of closely related classes into two separate hierarchies – abstraction and implementation – which can be developed independently of each other*. Here the UI is the **abstraction** and the scheme classes are the **implementation**. The UI only *delegates* cost work to a `TalkTimeScheme` reference.

Step 2 – Class diagram



Step 3 – Java code

```java
import java.util.*;

// ======== Implementor: the only thing UI programmers need to know ========
interface TalkTimeScheme {
    String getName();
    double costFor(int minutes);
}

// ======== Concrete implementors: written by the scheme team ========
class FlatRateScheme implements TalkTimeScheme {
    public String getName() { return "Flat 1.0 Tk/min"; }
    public double costFor(int m) { return m * 1.0; }
}
class BundleScheme implements TalkTimeScheme {          // 300 min for 250 Tk, then 1.2 Tk/min
    public String getName() { return "Bundle 300"; }
    public double costFor(int m) { return 250 + Math.max(0, m - 300) * 1.2; }
}
```

```java
class NightOwlScheme implements TalkTimeScheme {        // assume 40% of usage at night
    public String getName() { return "Night Owl"; }
    public double costFor(int m) { return m * 0.6 * 1.3 + m * 0.4 * 0.3; }
}

// ======== Registry: new schemes are plugged in here, the UI never names a class ========
class SchemeCatalog {
    private final Map<String, TalkTimeScheme> schemes = new LinkedHashMap<>();
    void register(TalkTimeScheme s) { schemes.put(s.getName(), s); }
    Collection<TalkTimeScheme> all() { return schemes.values(); }
}

// ======== Abstraction: written by the UI team ========
abstract class SchemeView {
    protected TalkTimeScheme scheme;                     // the bridge
    void setScheme(TalkTimeScheme s) { this.scheme = s; }
    abstract String render(int minutes);
}
class WebSchemePage extends SchemeView {
    String render(int minutes) {
        return "<tr><td>" + scheme.getName() + "</td><td>" + minutes + " min</td><td>Tk "
             + String.format("%.2f", scheme.costFor(minutes)) + "</td></tr>";
    }
}
class MobileSchemeCard extends SchemeView {
    String render(int minutes) {
        return "[" + scheme.getName() + "] " + minutes + "min = Tk "
             + String.format("%.2f", scheme.costFor(minutes));
    }
}

// ======== Client ========
public class TalkTimePortal {
    public static void main(String[] args) {
        SchemeCatalog catalog = new SchemeCatalog();
        catalog.register(new FlatRateScheme());
        catalog.register(new BundleScheme());
        catalog.register(new NightOwlScheme());          // tomorrow: one more line here

        SchemeView view = new WebSchemePage();
        for (TalkTimeScheme s : catalog.all()) {          // user browses and selects
            view.setScheme(s);
            System.out.println(view.render(400));
        }
    }
}
```

Step 4 – How the decoupling works

- UI programmers compile only against `SchemeView` and the `TalkTimeScheme` interface. They never see `FlatRateScheme` and the other concrete schemes.
- Scheme programmers add `class StudentScheme implements TalkTimeScheme` and register it. No UI class is recompiled (**OCP**, **DIP**: both sides depend on the abstraction).
- The two hierarchies grow independently: m views + n schemes classes instead of $m \times n$.
- The registry (a simple factory) removes the last coupling point, object creation, from the UI.

Answer. Use the Bridge pattern. The UI hierarchy (`SchemeView` and its subclasses) is the abstraction and holds a reference to the `TalkTimeScheme` interface (the implementor). Each scheme is a concrete implementor, registered in a catalog, so new schemes are added without touching the UI code.

2020–2021 • Q3(a) CSE307 • 19/10/2022 • D018.1 **MARKS 15**

You are to implement the hierarchy of the employees of the company. An employee may be under a boss and he or she might have one or more subordinates. The level of hierarchy is not fixed, and it varies any time. You can add a subordinate employee under an employee or you can remove an already included subordinate employee in the hierarchy. The employees listed in the subordinate list is considered as direct subordinate. The subordinate of subordinates is called indirect subordinate. The list of all subordinates (direct or indirect up to leaf) with detailed information will be available through a method of the class representing an employee. Show necessary implementation of the classes using appropriate design patterns.

SOLUTION

Step 1 – Recognising the structure

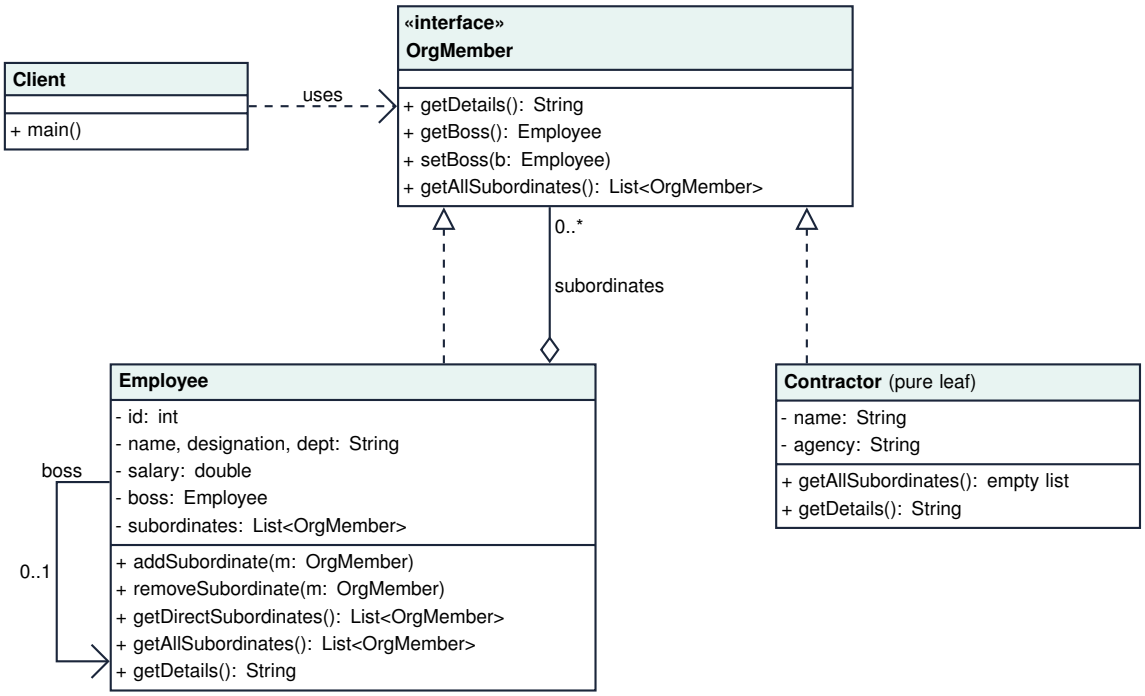
The employees form a **tree**. Every employee may have a boss (parent) and zero or more direct subordinates (children). The depth of the tree is not fixed and changes at run time. The key requirement is that we must be able to ask *any* employee for *all* subordinates, both direct and indirect, "up to leaf". That is a recursive walk over a part–whole hierarchy in which one employee and a whole sub-tree under that employee are handled the same way.
This is the intent of the **Composite** pattern: *compose objects into tree structures and then work with these structures as if they were individual objects*.

Step 2 – Design decision: who is Leaf, who is Composite?

In the textbook Composite, a *Leaf* class has no children and a separate *Composite* class has children. Here, however, "the level of hierarchy is not fixed, and it varies any time". A developer with no subordinates today may get a trainee tomorrow. If `Developer` were a Leaf class, we would have to destroy and re-create the object as a different class. Therefore:

- `OrgMember` is the **Component** interface (uniform operations for all nodes);
- `Employee` is the **Composite**. It holds a list of `OrgMember` children. An employee whose list is empty plays the **Leaf** role, and this role can change at run time.
- A **parent reference** (`boss`) is kept so that "an employee may be under a boss" can be answered and so that an employee is never placed under two bosses. Keeping an explicit parent reference is one of the standard Composite implementation choices.
- A true leaf class (here `Contractor`, who can never manage anyone) can still be added. This shows the pattern's open-ended Component interface.

Step 3 – Class diagram



The aggregation (hollow diamond on `Employee`) to `OrgMember` with multiplicity `0..*` is the recursive Composite link. The self-association `boss 0..1` is the parent pointer.

Step 4 – Java implementation

```java
import java.util.*;

// ================= Component =================
interface OrgMember {
    String getDetails();
    Employee getBoss();
    void setBoss(Employee boss);
    List<OrgMember> getAllSubordinates();          // direct + indirect
}

// ================= Composite (a leaf when it has no subordinates) =================
class Employee implements OrgMember {
    private final int id;
    private final String name, designation, dept;
    private final double salary;
    private Employee boss;                          // parent reference
    private final List<OrgMember> subordinates = new ArrayList<>();

    Employee(int id, String name, String designation, String dept, double salary) {
        this.id = id; this.name = name; this.designation = designation;
        this.dept = dept; this.salary = salary;
    }

    // ---- child management ----
    public void addSubordinate(OrgMember m) {
        if (m == this || isAncestor(m))
            throw new IllegalArgumentException("Cycle in hierarchy");
        if (m.getBoss() != null) m.getBoss().removeSubordinate(m);  // one boss only
```

```
        subordinates.add(m);
        m.setBoss(this);
    }
    public void removeSubordinate(OrgMember m) {
        if (subordinates.remove(m)) m.setBoss(null);  // whole sub-tree is detached
    }
    private boolean isAncestor(OrgMember m) {          // is m above me?
        for (Employee e = boss; e != null; e = e.getBoss()) if (e == m) return true;
        return false;
    }

    public List<OrgMember> getDirectSubordinates() {
        return Collections.unmodifiableList(subordinates);
    }

    // ---- the recursive Composite operation ----
    public List<OrgMember> getAllSubordinates() {
        List<OrgMember> all = new ArrayList<>();
        for (OrgMember m : subordinates) {
            all.add(m);                            // direct subordinate
            all.addAll(m.getAllSubordinates());    // indirect ones (recursion)
        }
        return all;
    }

    public String getDetails() {
        return String.format("#%d %-8s %-18s %-10s Tk%,.0f boss=%s", id, name, designation,
                             dept, salary, boss == null ? "-" : boss.name);
    }
    public Employee getBoss() { return boss; }
    public void setBoss(Employee b) { this.boss = b; }
}

// ================= A pure Leaf (can never have subordinates) =================
class Contractor implements OrgMember {
    private final String name, agency;
    private Employee boss;
    Contractor(String name, String agency) { this.name = name; this.agency = agency; }
    public List<OrgMember> getAllSubordinates() { return Collections.emptyList(); }
    public String getDetails() {
        return "Contractor " + name + " (" + agency + ") boss=" + (boss == null ? "-" : "set");
    }
    public Employee getBoss() { return boss; }
    public void setBoss(Employee b) { this.boss = b; }
}

// ================= Client =================
public class CompanyDemo {
    static void report(Employee e) {
        System.out.println("All subordinates of " + e.getDetails());
        for (OrgMember m : e.getAllSubordinates()) System.out.println("    " + m.getDetails());
    }
    public static void main(String[] args) {
        Employee ceo  = new Employee(1, "Rahim",  "CEO",            "Board", 500000);
        Employee cto  = new Employee(2, "Karim",  "CTO",            "Tech",  300000);
        Employee lead = new Employee(3, "Nusrat", "Team Lead",      "Tech",  180000);
        Employee dev1 = new Employee(4, "Tanvir", "Developer",      "Tech",  120000);
        Employee dev2 = new Employee(5, "Mitu",   "Developer",      "Tech",  110000);
```

```
        Employee cfo  = new Employee(6, "Selina", "CFO",            "Finance",280000);
        Employee acc  = new Employee(7, "Jamal",  "Accountant",     "Finance", 90000);

        ceo.addSubordinate(cto);   ceo.addSubordinate(cfo);
        cto.addSubordinate(lead);  lead.addSubordinate(dev1); lead.addSubordinate(dev2);
        cfo.addSubordinate(acc);
        lead.addSubordinate(new Contractor("Arif", "TechStaffing Ltd"));

        report(ceo);                      // 7 subordinates, direct and indirect
        report(cto);                      // lead, dev1, dev2, contractor

        dev1.addSubordinate(new Employee(8, "Rafi", "Intern", "Tech", 20000)); // level grows
        lead.removeSubordinate(dev2);     // hierarchy shrinks
        report(cto);
    }
}
```

Step 5 – How the requirements are met

Requirement	Where it is satisfied
Employee may be under a boss	`boss` parent reference, kept consistent by `addSubordinate()`
One or more subordinates	`subordinates : List<OrgMember>` (the Composite children)
Level not fixed, changes any time	any `Employee` can gain or lose children at run time; no class change needed
Add / remove subordinate	`addSubordinate()` (with cycle check) and `removeSubordinate()`
Direct subordinates	`getDirectSubordinates()`
All subordinates up to leaf, with details	recursive `getAllSubordinates()` plus `getDetails()` of each node

NOTE The question does not say what happens to the subordinates of a removed employee. The solution assumes the whole sub-tree is detached together. If the subordinates should instead move up to the removed employee's boss, `removeSubordinate()` would first call `addSubordinate(child)` on `this` for each child of `m`.

Answer. Composite pattern. `OrgMember` is the Component and `Employee` is the Composite (it acts as a Leaf while its list is empty, so the depth can change at any time). `Contractor` is a pure Leaf. The recursive `getAllSubordinates()` returns every direct and indirect subordinate with full details.

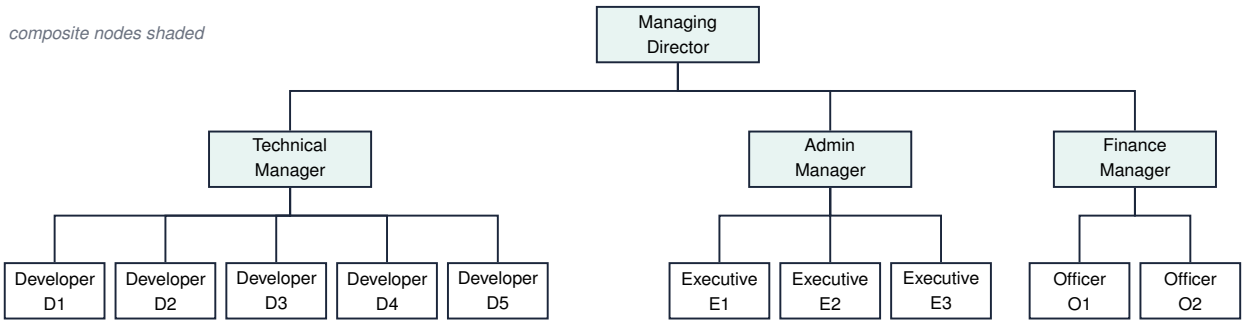
Jan 2021 • Q2(a) CSE307 • 04/09/2021 • D019.3 MARKS 20

A company is headed by a Managing Director. There are three divisions under Managing Director namely Technical, Administration and Finance. Each of these divisions is headed by a Manager. The Technical Division consists of five Developers under the supervision of the Technical manager. The Administration is run by three executives under the supervision of the Admin Manager. There are two officers under Finance Manager. You are to implement the hierarchy of the employees using composite pattern. The composite class must have the provision to show all the employees under any employee. Write all the classes for demonstrating this hierarchy of employees.

SOLUTION

Step 1 – The hierarchy to be built

The organisation is a three-level tree. The Managing Director and the three Managers have people under them, so they are **composite** nodes. Developers, Executives and Officers have nobody under them, so they are **leaf** nodes.

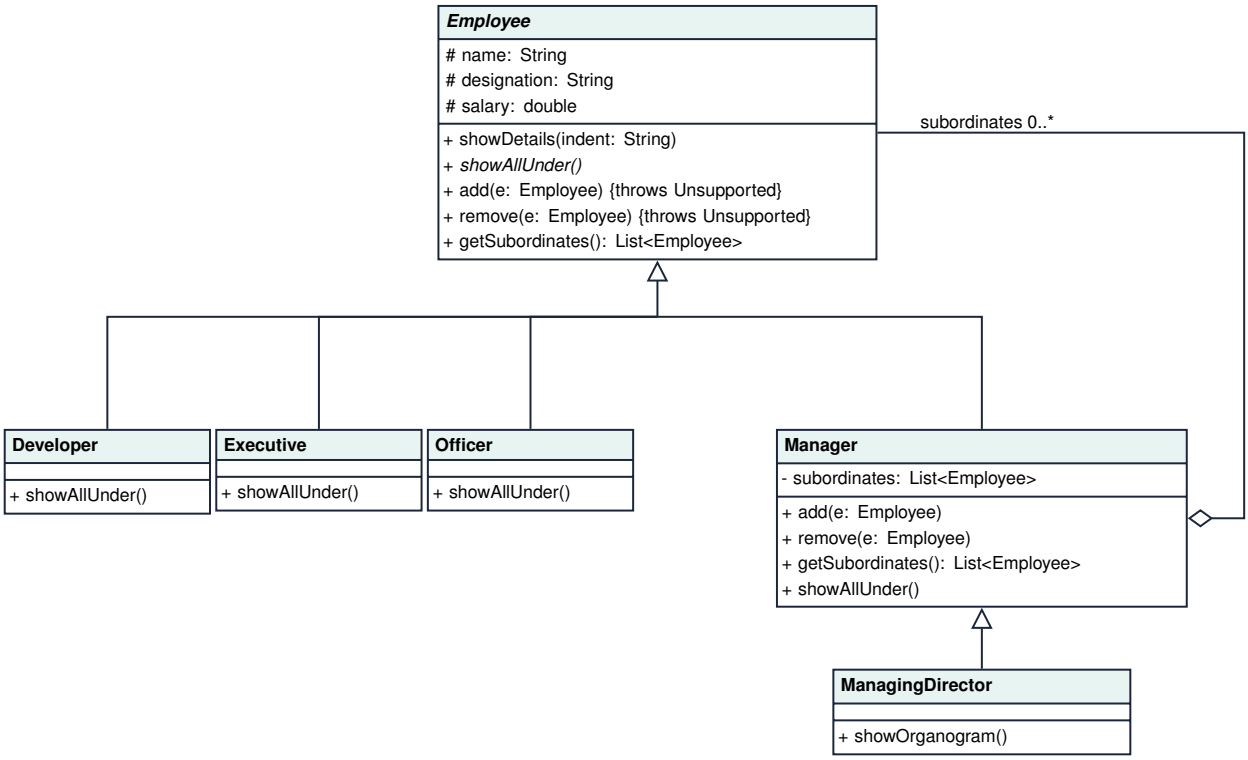


Step 2 – Composite pattern and the participants

Composite lets us compose objects into tree structures and then work with these structures as if they were individual objects. The client calls `showDetails()` or `showAllUnder()` on the MD, a Manager or a Developer in exactly the same way.

Composite role	Class
Component	`Employee` (abstract): name, designation, salary, `showDetails()`, `showAllUnder()`
Leaf	`Developer`, `Executive`, `Officer`
Composite	`Manager` (holds `List<Employee>` and adds/removes subordinates)
Composite (specialised)	`ManagingDirector` extends `Manager`
Client	`CompanyHierarchy.main()`

Step 3 – Class diagram



Step 4 – Java implementation (all classes)

```
import java.util.*;

// ================ Component ================
abstract class Employee {
    protected final String name, designation;
    protected final double salary;
    Employee(String name, String designation, double salary) {
        this.name = name; this.designation = designation; this.salary = salary;
    }
    public void showDetails(String indent) {
        System.out.printf("%s%-22s %-12s Tk %,.0f%n", indent, designation, name, salary);
    }
    // child-management defaults (transparent Composite)
    public void add(Employee e)    { throw new UnsupportedOperationException(); }
    public void remove(Employee e) { throw new UnsupportedOperationException(); }
    public List<Employee> getSubordinates() { return Collections.emptyList(); }
    public abstract void showAllUnder();        // show all employees under this one
}

// ================ Leaves ================
class Developer extends Employee {
    Developer(String n, double s) { super(n, "Developer", s); }
    public void showAllUnder() { System.out.println(name + " has no subordinates."); }
}
class Executive extends Employee {
    Executive(String n, double s) { super(n, "Admin Executive", s); }
    public void showAllUnder() { System.out.println(name + " has no subordinates."); }
}
```

```
class Officer extends Employee {
    Officer(String n, double s) { super(n, "Finance Officer", s); }
    public void showAllUnder() { System.out.println(name + " has no subordinates."); }
}

// ================ Composite ================
class Manager extends Employee {
    private final List<Employee> subordinates = new ArrayList<>();
    Manager(String n, String designation, double s) { super(n, designation, s); }

    @Override public void add(Employee e)    { subordinates.add(e); }
    @Override public void remove(Employee e) { subordinates.remove(e); }
    @Override public List<Employee> getSubordinates() { return subordinates; }

    /** the required provision: show ALL employees under this employee */
    @Override public void showAllUnder() {
        System.out.println("Employees under " + designation + " " + name + ":");
        print(this, "   ");
    }
    private static void print(Employee e, String indent) {   // recursive walk
        for (Employee s : e.getSubordinates()) {
            s.showDetails(indent);
            print(s, indent + "   ");
        }
    }
}
class ManagingDirector extends Manager {
    ManagingDirector(String n, double s) { super(n, "Managing Director", s); }
    public void showOrganogram() { showDetails(""); showAllUnder(); }
}

// ================ Client: build and demonstrate the hierarchy ================
public class CompanyHierarchy {
    public static void main(String[] args) {
        ManagingDirector md = new ManagingDirector("Mr. Haque", 400000);
        Manager tech  = new Manager("Ms. Rina",  "Technical Manager", 200000);
        Manager admin = new Manager("Mr. Kabir", "Admin Manager",     180000);
        Manager fin   = new Manager("Ms. Shila", "Finance Manager",   190000);
        md.add(tech); md.add(admin); md.add(fin);

        for (int i = 1; i <= 5; i++) tech.add(new Developer("Dev-" + i, 100000));
        for (int i = 1; i <= 3; i++) admin.add(new Executive("Exec-" + i, 60000));
        for (int i = 1; i <= 2; i++) fin.add(new Officer("Officer-" + i, 70000));

        md.showOrganogram();        // whole company (13 employees under MD)
        tech.showAllUnder();        // only the 5 developers
        fin.showAllUnder();         // only the 2 officers
        new Developer("X", 1).showAllUnder();   // a leaf answers uniformly
    }
}
```

Step 5 – Sample output (abridged)

```
Managing Director      Mr. Haque    Tk 400,000
Employees under Managing Director Mr. Haque:
   Technical Manager      Ms. Rina     Tk 200,000
      Developer              Dev-1        Tk 100,000
      ...                    (Dev-2 .. Dev-5)
   Admin Manager          Mr. Kabir    Tk 180,000
```

```
      Admin Executive        Exec-1       Tk 60,000   (Exec-2, Exec-3)
   Finance Manager        Ms. Shila    Tk 190,000
      Finance Officer        Officer-1    Tk 70,000   (Officer-2)
Employees under Technical Manager Ms. Rina:
   Developer              Dev-1        Tk 100,000   ... Dev-5
```

Step 6 – Why Composite is the right choice

- The client treats an individual employee and a whole department **uniformly** through `Employee`.
- `showAllUnder()` works for *any* employee and at *any* depth, because the recursion follows the tree.
- A new kind of employee (for example `Intern`, or a `DeputyManager` composite) is just a new subclass. The client code stays the same (**OCP**).

Answer. Component `Employee`, Leaves `Developer`, `Executive` and `Officer`, and Composite `Manager` (with `ManagingDirector` as a specialised composite). The MD holds 3 managers, which hold 5 developers, 3 executives and 2 officers. The recursive `showAllUnder()` lists every employee under any node.

2018–2019 • Q4(b) CSE307 • 06/11/2019 • D022 MARKS 18

Suppose you have a Bird class with fly() and makeSound() methods. And also a ToyDuck class with squeak() method. Let's assume that you are short on ToyDuck objects and you would like to use Bird objects in their place. Birds have some similar functionality but implement a different interface, so we can't use them directly. Draw a class diagram to present Adapter design pattern for the scenario above and write necessary codes so that your code fulfills all the requirements.

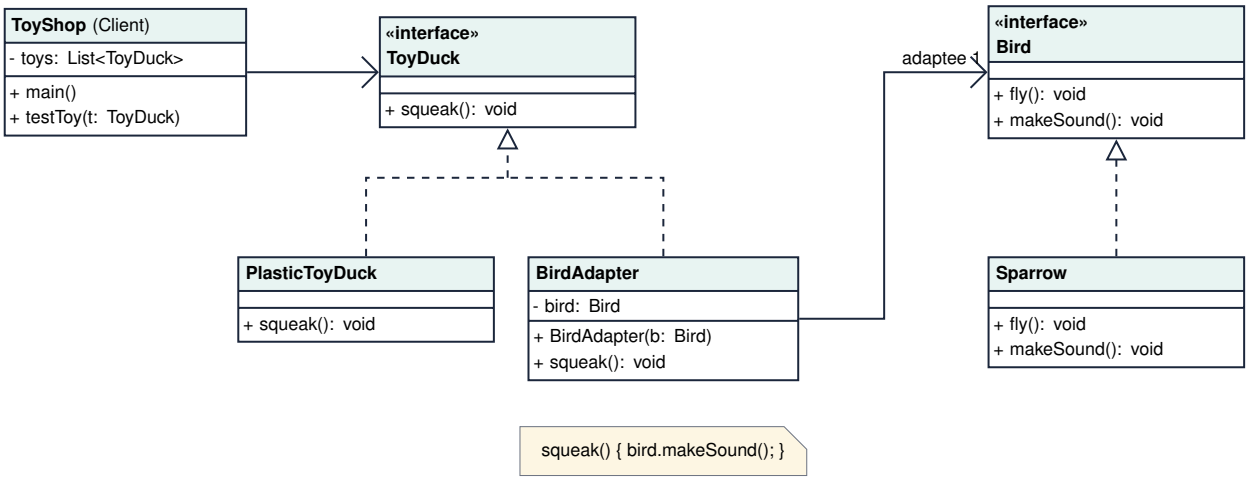
SOLUTION

Step 1 – Identifying the pattern and its roles

The question names the pattern. It is still worth stating why it fits. Client code is written against `ToyDuck` and calls `squeak()`. We have `Bird` objects that can make a sound, but through a *different interface* (`makeSound()`). We want birds to stand in for toy ducks without changing either class. That is **Adapter**: convert the interface of a class into another interface clients expect.

Adapter role	Class
Target	`ToyDuck` (interface) – `squeak()`
Adaptee	`Bird` (interface) with `fly()`, `makeSound()`. The concrete adaptee is `Sparrow`.
Adapter	`BirdAdapter` – implements `ToyDuck`, holds a `Bird`
Client	`ToyShop` – works only with `ToyDuck` references

Step 2 – Class diagram



Step 3 – Java implementation

```java
import java.util.ArrayList;
import java.util.List;

// ---------------- Adaptee side ----------------
interface Bird {
    void fly();
    void makeSound();
}
class Sparrow implements Bird {
    public void fly()       { System.out.println("Sparrow is flying"); }
    public void makeSound() { System.out.println("Chirp chirp"); }
}

// ---------------- Target side ----------------
interface ToyDuck {
    void squeak();
}
class PlasticToyDuck implements ToyDuck {
    public void squeak() { System.out.println("Squeak squeak"); }
}

// ---------------- Adapter ----------------
class BirdAdapter implements ToyDuck {
    private final Bird bird;                // composition: object adapter
    BirdAdapter(Bird bird) { this.bird = bird; }
    @Override
    public void squeak() {                  // translate the request
        bird.makeSound();
    }
}

// ---------------- Client ----------------
public class ToyShop {
    static void testToy(ToyDuck t) { t.squeak(); }    // only knows ToyDuck

    public static void main(String[] args) {
        List<ToyDuck> toys = new ArrayList<>();
        toys.add(new PlasticToyDuck());
```

```
        toys.add(new PlasticToyDuck());
        // short on toy ducks -> use birds in their place
        toys.add(new BirdAdapter(new Sparrow()));

        for (ToyDuck t : toys) testToy(t);
        // Output: Squeak squeak / Squeak squeak / Chirp chirp
    }
}
```

Step 4 – Remarks

- `fly()` has no counterpart in `ToyDuck`, so the adapter simply does not expose it. An adapter maps *only* the operations that the target interface needs.
- Neither `Bird`/`Sparrow` nor `ToyDuck`/`PlasticToyDuck` was modified (OCP). The client cannot tell an adapted bird from a real toy duck.
- Because the adapter holds a `Bird` reference, *every* bird (`Sparrow`, `Parrot`, ...) can be adapted by the same class. That is the advantage of the object adapter. The class-adapter form `class SparrowAdapter extends Sparrow implements ToyDuck` works for one concrete bird only.

Answer. Target `ToyDuck.squeak()`, Adaptee `Bird` (`Sparrow`), Adapter `BirdAdapter implements ToyDuck`, which holds a `Bird` and implements `squeak()` as `bird.makeSound()`. The client uses birds wherever toy ducks are expected.

2017–2018 • Q6(a) CSE307 • 29/09/2018 • D023 **MARKS 7 + 15**

In the food shop at Lalbagh many BUET students visit to have dinners. There, they have option to choose different types of set-meals. Each has a main item like rice or bread. Along with rice, gravy curries are added. For bread, chicken or beef kebabs are there. Drinks/juices need to be ordered additionally out of the set-meal if the customer desires to have. Along with the meal, the bill is served and need to be paid before the start of consumption. Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfills all the requirements.

SOLUTION

Step 1 – Analysing the requirement

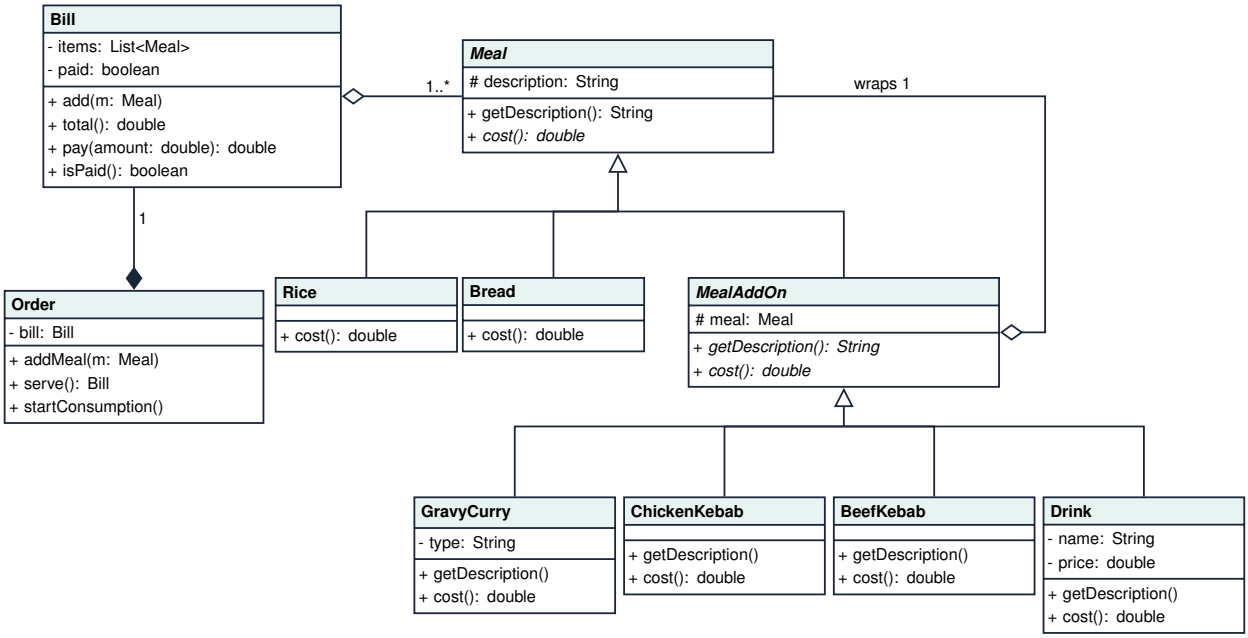
- Every set-meal starts from a **base main item**: *rice* or *bread*.
- Extra items are **added on top** of the base: gravy curry with rice, and chicken or beef kebab with bread. A customer may also take more than one (for example two kebabs).
- **Drinks/juices** are optional add-ons ordered *in addition* to the set-meal.
- The **bill** must total whatever was combined and must be paid *before consumption starts*.

The final price and description depend on a run-time combination of add-ons wrapped around a base item. Writing one class per combination (`RiceWithChickenCurryAndJuice`, ...) would explode, and boolean flags in a base class would break whenever a new item appears. This is the same situation as the Starbuzz-coffee condiments in the lecture, so we use the **Decorator** pattern. Each add-on wraps the meal and adds its own description and cost.

Step 2 – Mapping of participants

Role	Class
Component	`Meal` (abstract): `getDescription()`, `cost()`
Concrete components	`Rice`, `Bread` (main items)
Decorator	`MealAddOn` (abstract, wraps a `Meal`)
Concrete decorators	`GravyCurry`, `ChickenKebab`, `BeefKebab`, `Drink`
Collaborators	`Bill` (computes total, takes payment), `Order` (serves and starts consumption)

Step 3 – Class diagram (7 marks)



Step 4 – Java code (15 marks)

```java
import java.util.ArrayList;
import java.util.List;

// ================= Component =================
abstract class Meal {
    protected String description = "Unknown meal";
    public String getDescription() { return description; }
    public abstract double cost();
}

// ================= Concrete components (main items) =================
class Rice extends Meal {
    Rice()  { description = "Plain rice"; }
    public double cost() { return 40; }
}
class Bread extends Meal {
    Bread() { description = "Naan bread"; }
    public double cost() { return 30; }
}
```

```
// ================ Decorator ================
abstract class MealAddOn extends Meal {
    protected final Meal meal;                    // the meal being decorated
    MealAddOn(Meal meal) { this.meal = meal; }
    public abstract String getDescription();
}

// ================ Concrete decorators ================
class GravyCurry extends MealAddOn {              // goes with rice
    private final String type;                    // "chicken", "beef", "dal" ...
    GravyCurry(Meal m, String type) {
        super(m); this.type = type;
        if (!(base(m) instanceof Rice))
            throw new IllegalArgumentException("Gravy curry is served with rice");
    }
    public String getDescription() { return meal.getDescription() + ", " + type + " curry"; }
    public double cost() { return meal.cost() + 120; }
    static Meal base(Meal m) {                    // find the main item inside the wrappers
        while (m instanceof MealAddOn) m = ((MealAddOn) m).meal;
        return m;
    }
}
class ChickenKebab extends MealAddOn {            // goes with bread
    ChickenKebab(Meal m) { super(m); }
    public String getDescription() { return meal.getDescription() + ", chicken kebab"; }
    public double cost() { return meal.cost() + 110; }
}
class BeefKebab extends MealAddOn {               // goes with bread
    BeefKebab(Meal m) { super(m); }
    public String getDescription() { return meal.getDescription() + ", beef kebab"; }
    public double cost() { return meal.cost() + 140; }
}
class Drink extends MealAddOn {                   // ordered additionally
    private final String name; private final double price;
    Drink(Meal m, String name, double price) { super(m); this.name = name; this.price = price; }
    public String getDescription() { return meal.getDescription() + " + " + name; }
    public double cost() { return meal.cost() + price; }
}

// ================ Bill & Order ================
class Bill {
    private final List<Meal> items = new ArrayList<>();
    private boolean paid = false;
    void add(Meal m) { items.add(m); }
    double total() { double t = 0; for (Meal m : items) t += m.cost(); return t; }
    double pay(double amount) {                   // returns change
        if (amount < total()) throw new IllegalStateException("Insufficient payment");
        paid = true;
        return amount - total();
    }
    boolean isPaid() { return paid; }
    void print() {
        for (Meal m : items) System.out.printf("%-55s %7.2f%n", m.getDescription(), m.cost());
        System.out.printf("%-55s %7.2f%n", "TOTAL", total());
    }
}
class Order {
```

```
    private final Bill bill = new Bill();
    void addMeal(Meal m) { bill.add(m); }
    Bill serve() { System.out.println("Meal served with bill:"); bill.print(); return bill; }
    void startConsumption() {
        if (!bill.isPaid()) throw new IllegalStateException("Pay the bill before eating!");
        System.out.println("Enjoy your dinner.");
    }
}

// ================ Client ================
public class LalbaghFoodShop {
    public static void main(String[] args) {
        Meal m1 = new Drink(new GravyCurry(new Rice(), "chicken"), "Mango juice", 60);
        Meal m2 = new BeefKebab(new ChickenKebab(new Bread()));    // two kebabs, no drink
        Order order = new Order();
        order.addMeal(m1);
        order.addMeal(m2);
        Bill bill = order.serve();
        System.out.println("Change: " + bill.pay(600));
        order.startConsumption();
    }
}
```

Step 5 – Output and discussion

With the sample prices the bill is `Plain rice, chicken curry + Mango juice` $= 40 + 120 + 60 = 220$ and `Naan bread, chicken kebab, beef kebab` $= 30 + 110 + 140 = 280$, so the **total is 500**. Paying 600 returns a change of 100, and only then does `startConsumption()` succeed. Calling it before `pay()` throws an exception. This enforces the rule "paid before the start of consumption".

- A new curry or kebab, or a new drink, is one new decorator class (**OCP**). Base items never change.
- Double portions are just two wrappers, which the boolean-flag design cannot handle.
- The rules "curry with rice" and "kebab with bread" are checked inside the decorators' constructors (shown for curry; kebab is analogous). Drinks are kept as decorators so the bill gets a single cost per set-meal, but they could equally be separate `Meal` items added to the `Bill`.

> **EXAM TIP** Some students answer this with Builder (a "set-meal builder"). That is acceptable if justified, but the wording "main item + items added along with it + extra drinks, then a bill" is the classic cost-accumulation Decorator. Always state the problem force (combinatorial add-ons) before naming the pattern.

Answer. Decorator pattern: `Meal` is the component, `Rice` and `Bread` are concrete components, and `GravyCurry`, `ChickenKebab`, `BeefKebab` and `Drink` are decorators that add description and cost. `Bill` sums `cost()` over the ordered meals, and `Order` lets consumption start only after the bill is paid.

2017–2018 • Q7(b) CSE307 • 29/09/2018 • D024 **MARKS 6 + 15**

"Uber" and "Pathao" are the popular car based ride sharing applications. They calculate cost based on number of passengers and distance using the following formula:

Uber:

Cost = n*d*18*time_factor; where n = number of passengers and d = distance

if it is extremely peak hour, then time_factor = 1.8
if it is moderately peak hour, then time_factor = 1.2
if it is off peak hour, then time_factor = 0.9
Pathao:
Cost = n*d*20*time_factor;
if it is extremely peak hour, then time_factor = 1.5
if it is moderately peak hour, then time_factor = 1.15
if it is off peak hour, then time_factor = 0.9
"Uber Moto" and "Pathao Moto" are the popular bike based ride sharing app. They calculate cost based on the distance to destination using of following formula:

Uber: Cost = 25 + d*12

pathao: Cost = 30 + d*11
The user will input the destination distance, number of passengers and an input t. (t = 1, 2, 3 for extremely peak, moderately peak and off peak hour respectively.) You need to output the cost for each app. Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfils all the requirements.

SOLUTION

NOTE The 6 + 15 marks are read as 6 for the class diagram and 15 for the code. Sub-part 7(c) of the same paper (tight cohesion and loose coupling) is a separate question in the Design Principles chapter.

Step 1 – Identifying the pattern

The fare depends on **two independent dimensions**:

- **the company**: Uber or Pathao. Each has its own rates (18 vs. 20 per passenger-km, its own peak factors, its own bike base fare and per-km rate);
- **the vehicle / ride type**: car (cost $= n \cdot d \cdot$ rate $\cdot$ factor) or bike (cost $=$ base $+ d \cdot$ perKm).

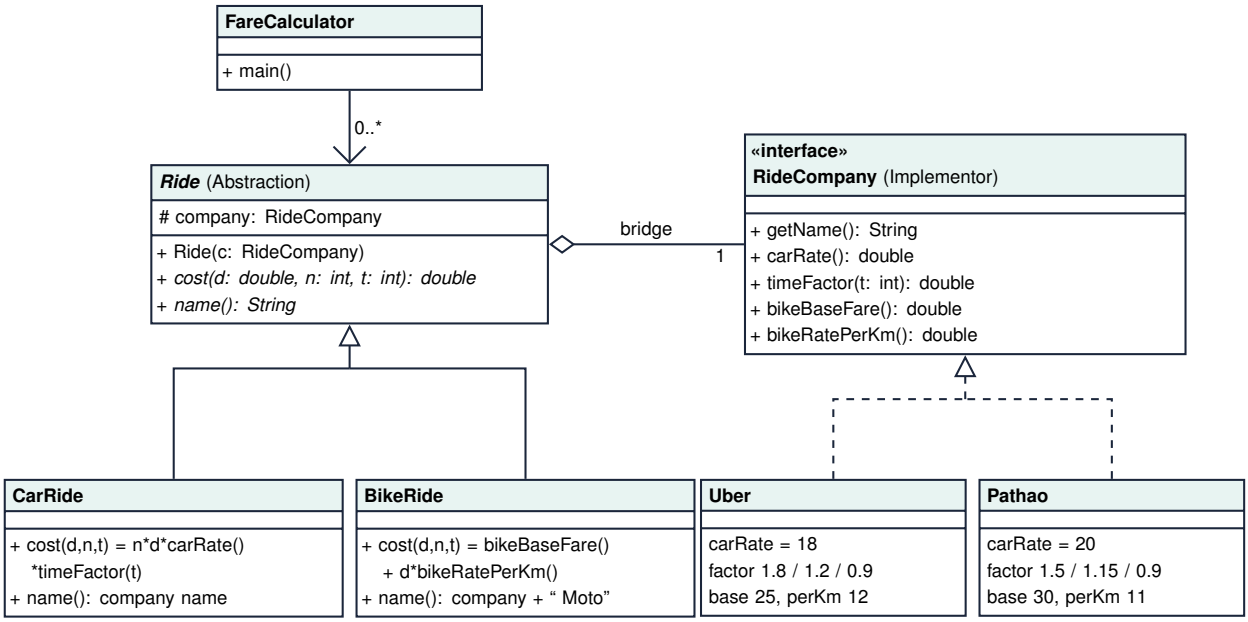
One class per combination (`UberCar`, `UberMoto`, `PathaoCar`, `PathaoMoto`) is the "Circle/Square $\times$ Red/Blue" explosion from the lecture. Adding a third company (e.g. Obhai) or a third vehicle (e.g. CNG) would multiply the classes again. The **Bridge** pattern splits this into two hierarchies joined by composition:

- **Abstraction** `Ride` (`CarRide`, `BikeRide`) holds the formula *shape*, and
- **Implementation** `RideCompany` (`Uber`, `Pathao`) supplies the company's *numbers*.

The ride delegates every company-specific value to the linked company object.

EXAM TIP Strategy (one fare strategy per app) also solves this and is acceptable if justified. Bridge is the better fit here because there are two dimensions that vary independently, and this question sits in the structural-pattern part of the syllabus.

Step 2 – Class diagram (6 marks)



Step 3 – Java code (15 marks)

```
import java.util.*;

// ================ Implementor ================
interface RideCompany {
    String getName();
    double carRate();                 // per passenger per km
    double timeFactor(int t);         // t = 1 extreme peak, 2 moderate peak, 3 off peak
    double bikeBaseFare();
    double bikeRatePerKm();
}

// ================ Concrete implementors ================
class Uber implements RideCompany {
    public String getName()        { return "Uber"; }
    public double carRate()        { return 18; }
    public double timeFactor(int t) {
        switch (t) {
            case 1: return 1.8;
            case 2: return 1.2;
            case 3: return 0.9;
            default: throw new IllegalArgumentException("t must be 1, 2 or 3");
        }
    }
    public double bikeBaseFare()   { return 25; }
    public double bikeRatePerKm()  { return 12; }
}
class Pathao implements RideCompany {
    public String getName()        { return "Pathao"; }
    public double carRate()        { return 20; }
    public double timeFactor(int t) {
        switch (t) {
            case 1: return 1.5;
```

```
            case 2: return 1.15;
            case 3: return 0.9;
            default: throw new IllegalArgumentException("t must be 1, 2 or 3");
        }
    }
    public double bikeBaseFare()   { return 30; }
    public double bikeRatePerKm()  { return 11; }
}

// ================ Abstraction ================
abstract class Ride {
    protected final RideCompany company;            // the bridge
    Ride(RideCompany company) { this.company = company; }
    abstract double cost(double d, int n, int t);
    abstract String name();
}

// ================ Refined abstractions ================
class CarRide extends Ride {
    CarRide(RideCompany c) { super(c); }
    double cost(double d, int n, int t) {
        return n * d * company.carRate() * company.timeFactor(t);
    }
    String name() { return company.getName(); }
}
class BikeRide extends Ride {
    BikeRide(RideCompany c) { super(c); }
    double cost(double d, int n, int t) {           // n and t are not used for bikes
        return company.bikeBaseFare() + d * company.bikeRatePerKm();
    }
    String name() { return company.getName() + " Moto"; }
}

// ================ Client ================
public class FareCalculator {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Distance (km): ");        double d = in.nextDouble();
        System.out.print("Passengers: ");          int n = in.nextInt();
        System.out.print("t (1/2/3): ");           int t = in.nextInt();

        List<RideCompany> companies = List.of(new Uber(), new Pathao());
        List<Ride> rides = new ArrayList<>();
        for (RideCompany c : companies) rides.add(new CarRide(c));
        for (RideCompany c : companies) rides.add(new BikeRide(c));

        for (Ride r : rides)
            System.out.printf("%-12s : Tk %.2f%n", r.name(), r.cost(d, n, t));
    }
}
```

Step 4 – Sample run

Input: $d = 10$ km, $n = 2$, $t = 1$ (extremely peak).

App	Computation	Cost (Tk)
Uber	$2 \times 10 \times 18 \times 1.8$	648.00
Pathao	$2 \times 10 \times 20 \times 1.5$	600.00
Uber Moto	$25 + 10 \times 12$	145.00
Pathao Moto	$30 + 10 \times 11$	140.00

Step 5 – Benefits of the design

- A new company is *one* new implementor class, and every ride type works with it at once.
- A new vehicle type (e.g. `CngRide`) is one new abstraction class that works with every company.
- Classes needed: #rides + #companies instead of #rides × #companies.
- The client never uses `if (app == "Uber")`. It iterates over `Ride` objects polymorphically.

Answer. Bridge pattern. The abstraction `Ride` (`CarRide`, `BikeRide`) holds the fare formula and delegates the company-specific rates and time factor to the implementor `RideCompany` (`Uber`, `Pathao`). For $d = 10, n = 2, t = 1$ the output is Uber 648, Pathao 600, Uber Moto 145 and Pathao Moto 140.

2016–2017 • Q5(d) CSE307 • 15/07/2017 • D028.2 MARKS $3 + 3\frac{2}{3} = 6\frac{2}{3}$

Draw the UML class diagram of "Class Adapter Pattern". Write down the pros and cons of Object Adapter Pattern and Class Adapter Pattern.

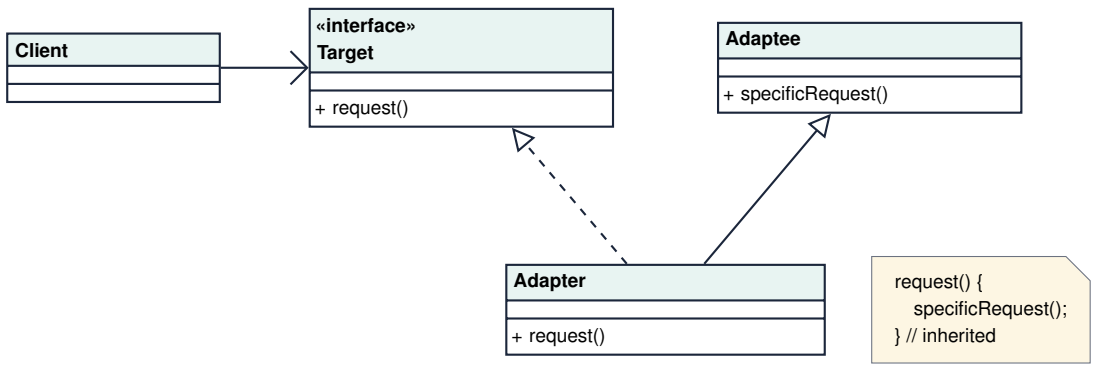
SOLUTION

Background

Adapter converts the interface of a class into another interface clients expect. The lecture shows two ways to build the adapter:

- **Object adapter** uses *composition*. The adapter implements the target interface and *wraps* an adaptee object.
- **Class adapter** uses *inheritance*. The adapter inherits from *both* the target and the adaptee at the same time. This needs multiple inheritance (C++). In Java it is possible only when the target is an interface: `class Adapter extends Adaptee implements Target`.

(i) UML class diagram of the Class Adapter



There is **no association** from Adapter to Adaptee. The adapter *is* an Adaptee (generalisation) and calls the inherited `specificRequest()` on itself. For comparison, the object adapter replaces the inheritance arrow with an association `Adapter` → `Adaptee`.

(ii) Pros and cons

	Object Adapter (composition)	Class Adapter (inheritance)
Pros	✓ A single adapter works with the Adaptee *and all its subclasses*, because any subtype can be passed in. ✓ Works in every OO language. No multiple inheritance needed. ✓ The adaptee can be replaced at run time. Looser coupling (composition over inheritance). ✓ Final classes and interfaces can be adapted.	✓ No extra object or indirection: requests go straight to inherited code, which is slightly faster and needs less code. ✓ Can *override* some of the Adaptee's behaviour, since the adapter is a subclass. ✓ Can use the Adaptee's protected members.
Cons	× Hard to override Adaptee behaviour: you must subclass the Adaptee and adapt that subclass. × One more object and one more level of delegation. × Cannot reach the Adaptee's protected members.	× Commits to *one concrete* Adaptee class, so it cannot adapt that class's subclasses. × Needs multiple inheritance (C++). In Java the target must be an interface, and final adaptees cannot be adapted. × Tighter, compile-time coupling. The Adaptee's whole public interface leaks into the adapter.

Answer. Class adapter: `Adapter` realises `Target` and inherits `Adaptee`, and implements `request()` by calling the inherited `specificRequest()`. The object adapter is flexible (it adapts all subclasses, needs no multiple inheritance, and the adaptee is swappable) but cannot easily override behaviour. The class adapter is direct and can override, but it is tied to one concrete class and needs multiple inheritance.

2014–2015 • Q6(b) CSE307 • 04/02/2016 • D031.2 **MARKS 12**

Give one design pattern that can be reduce dependence on hardware and/or software platforms. Explain how the pattern reduces a program's dependence on hardware and/or software platforms.

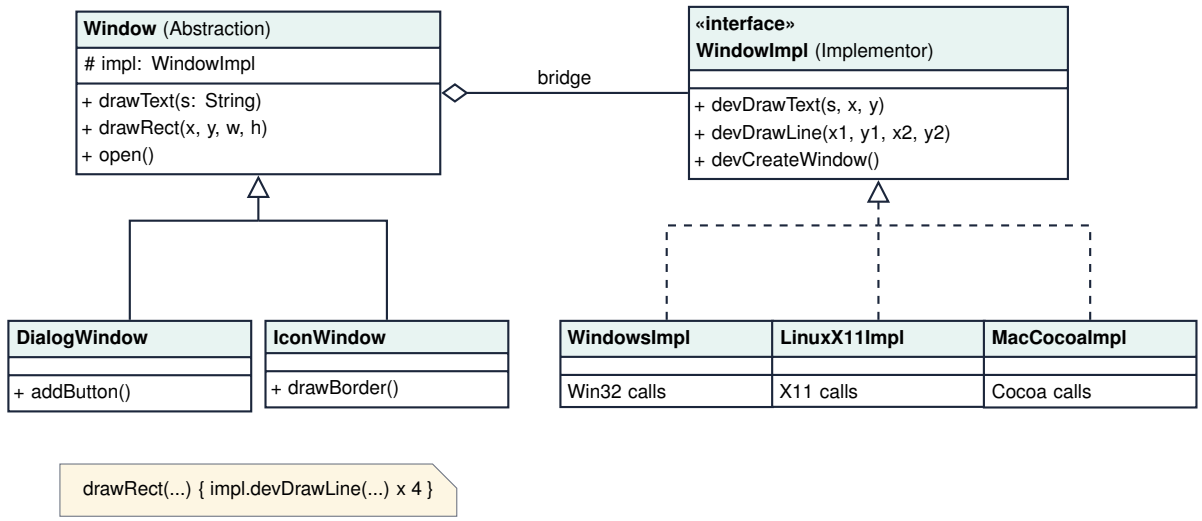
SOLUTION

Step 1 – Identifying the pattern

Platform dependence means that code calls OS or hardware APIs (window systems, file systems, drivers, graphics libraries) directly, so moving to a new platform forces changes everywhere. The remedy is to keep the platform-specific code behind a stable abstraction that the program talks to. The pattern taught for exactly this purpose is **Bridge**. (Abstract Factory, which creates platform-specific families of objects behind abstract interfaces, is the other standard answer.)
Bridge lets you split a large class or a set of closely related classes into two separate hierarchies, *abstraction* and *implementation*, which can be developed independently of each other. In the lecture's terms, the **abstraction** ("interface") is the high-level control layer, such as the GUI used by regular

customers and admins, and it does no real platform work itself. It delegates to the **implementation** ("platform") layer, such as the APIs for Linux, Windows and macOS.

Step 2 – Structure




```
interface WindowImpl {                          // implementor = platform API
    void devCreateWindow();
    void devDrawLine(int x1, int y1, int x2, int y2);
}
class LinuxX11Impl implements WindowImpl {
    public void devCreateWindow() { /* XCreateWindow(...) */ }
    public void devDrawLine(int a, int b, int c, int d) { /* XDrawLine(...) */ }
}
class Window {                                   // abstraction = what the program uses
    protected final WindowImpl impl;
    Window(WindowImpl impl) { this.impl = impl; }
    void open() { impl.devCreateWindow(); }
    void drawRect(int x, int y, int w, int h) {
        impl.devDrawLine(x, y, x + w, y);          impl.devDrawLine(x + w, y, x + w, y + h);
        impl.devDrawLine(x + w, y + h, x, y + h); impl.devDrawLine(x, y + h, x, y);
    }
}
// Client: the only platform decision is made once, e.g. from configuration
//   WindowImpl p = os.equals("linux") ? new LinuxX11Impl() : new WindowsImpl();
//   new Window(p).drawRect(10, 10, 200, 100);
```

Step 3 – How Bridge reduces platform dependence

(i) Platform code is isolated. All OS/hardware calls live only in the concrete implementors (`WindowsImpl`, `LinuxX11Impl`, ...). The rest of the program, the abstraction hierarchy and the client, contains no platform-specific code.

(ii) The program depends on an interface, not a platform. `Window` talks only to `WindowImpl` (Dependency Inversion). Porting to a new platform means writing one new implementor. No application class is modified or even recompiled.

(iii) Implementations are interchangeable at run time. The platform object is chosen once, at start-up or from configuration, and can even be switched. The same GUI therefore works under different operating systems, as the lecture's example states.

(iv) Independent extension, no class explosion. New kinds of windows (abstraction side) and new platforms (implementation side) are added independently, giving $m + n$ classes instead of $m \times n$ (`LinuxDialogWindow`, `MacIconWindow`, ...).

(v) Implementation details are hidden from clients, so a change in a platform API is absorbed by one implementor class.

Answer. The Bridge pattern. It separates the platform-independent abstraction (e.g. `Window`, used by the application) from a platform implementation interface (`WindowImpl`) with one concrete implementor per OS or hardware. The program depends only on the abstraction, and supporting a new platform means adding one implementor class.

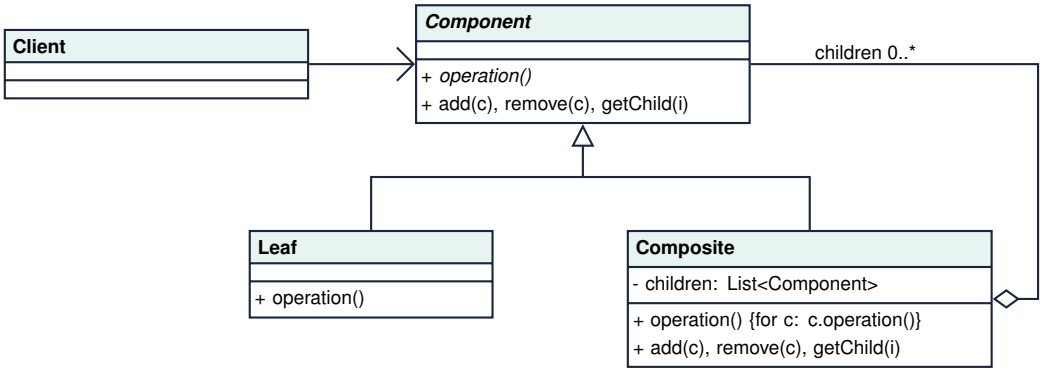
2011–2012 • Q1(a) CSE307 • 23/09/2013 • D035.1 **MARKS 14**

Discuss the different implementation issues and consequences of Composite Pattern.

SOLUTION

Recap – the Composite pattern

Composite lets you compose objects into tree structures and then work with these structures as if they were individual objects. Its participants are the **Component** (common interface), the **Leaf** (primitive object), the **Composite** (stores child Components and implements child-related operations by delegating to the children) and the **Client** (manipulates everything through Component).



(A) Implementation issues

(i) Explicit parent references. Storing a reference from a child to its parent makes it easy to move up the tree and to delete a component. It also supports the Chain-of-Responsibility style of passing requests upward. The parent pointer is usually defined in Component, and it must be kept consistent: set it in `add()` and clear it in `remove()`, and nowhere else.

(ii) Sharing components. Sharing a sub-tree between several parents saves memory, but a child with more than one parent makes parent references ambiguous. Shared leaves usually have to be designed without stored parent state.

(iii) Maximising the Component interface. To make Leaf and Composite truly interchangeable, Component should declare as many common operations as possible, with sensible defaults (for example `getChild()` returns nothing for a Leaf). This sometimes conflicts with the principle that a class should only define operations that are meaningful to its subclasses.

(iv) Where to declare child-management operations – transparency vs. safety. This is the main trade-off.

- *Transparency:* declare `add()`, `remove()` and `getChild()` in Component. The client treats all nodes alike, but it may call `add()` on a Leaf, which must then fail at run time (throw an exception or do nothing).
- *Safety:* declare them only in Composite. Misuse is caught at compile time, but the client must know (or check with `instanceof`/`getComposite()`) whether a node is a Composite, so uniformity is lost.

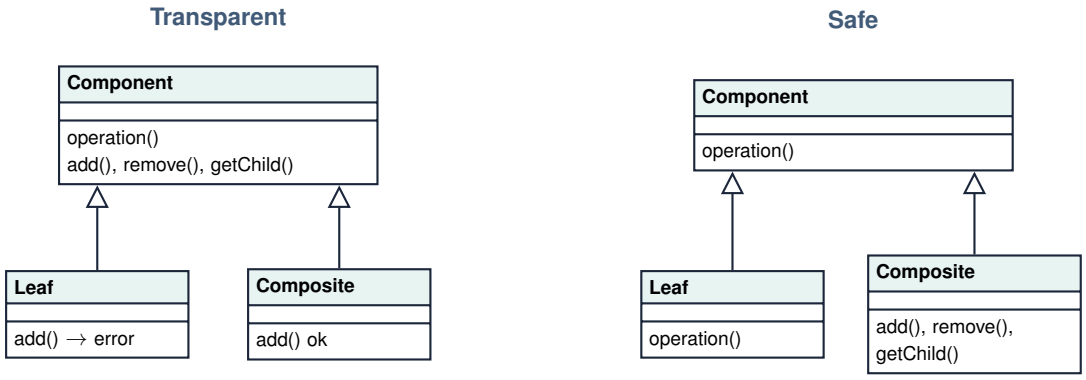
(v) Should Component hold the list of children? Putting the child list in Component wastes space in every Leaf. It belongs in Composite unless there are very few leaves.

(vi) Child ordering. Some domains need ordered children (parse trees, menus, pages of a document). Then the child-access interface must be designed to manage the sequence, for example with an Iterator.

(vii) Caching to improve performance. A Composite may cache results of traversals (total cost, bounding box, number of descendants). Every change to a child must then invalidate the caches of its ancestors, which needs the parent references.

(viii) Who deletes components? In languages without garbage collection, a Composite is normally responsible for deleting its children when it is destroyed (unless leaves are shared).

(ix) Best data structure for storing children. A linked list, array, tree or hash table can be used, chosen for efficiency. A Composite can even keep a separate field per child when the number of children is fixed.



(B) Consequences

Consequence	Explanation
✓ Defines class hierarchies of primitive and composite objects	Primitives can be composed into more complex objects, which can be composed again, recursively. Wherever client code expects a primitive object it can also take a composite.
✓ Makes the client simple	Clients treat composite structures and individual objects uniformly. They do not need `if/instanceof` tests to know whether they are dealing with a leaf or a composite.
✓ Easier to add new kinds of components	New Leaf or Composite subclasses work automatically with existing structures and client code (Open–Closed Principle).
× Can make the design overly general	Because everything is a Component, it is hard to *restrict* what a composite may contain (for example "a menu may hold only menu items, not orders"). The type system cannot enforce such rules, so run-time checks are needed.
× Transparency costs safety	With the transparent variant, meaningless calls such as `leaf.add()` compile and fail only at run time.

Answer. Implementation issues: explicit parent references, sharing components, maximising the Component interface, where to declare child-management operations (transparency vs. safety), whether Component keeps a child list, child ordering, caching, responsibility for deletion, and the data structure for children. Consequences: recursive part–whole hierarchies, a simple and uniform client, easy addition of new component types, but an overly general design that makes restricting components hard.

2010–2011 • Q4(d) CSE307 • 19/03/2012 • D040.2 MARKS $12\frac{2}{3}$

Context – Q4(c) of the same paper (solved in the Behavioural Design Patterns chapter):
(c) In principle a social network service focuses on building online communities of people who share interests and/or activities, or who are interested in exploring the interests and activities of others. Facebook support groups that people can join. Each group has a title, administrative members, a group type (open/ closed), and a list of related groups. Otherwise, a group operates just like an ordinary page. If somebody writes on the wall page of the group, the information is broadcasted to all the members and it is visualized in the news feeds of the members. Which design pattern is the most appropriate to handle this basic functionality of such a Facebook group? Users should be able to join a group as well as leave a group if they get bored. Once a user has joined a group it will automatically receive any updates that are published on the wall. Give a short explanation. In particular, show a complete class diagram(s) and enough code fragments to illustrate your use of the pattern to solve the problem. (12)

Task:
(d) Now let's suppose that you would like to extend the functionality of Facebook group that you have created in **Question. 4(c)** by adding the possibility that besides normal users a group page can also be a member of another group. As an example it should be possible that the group page of the "CSE BUET 08" joins the "BUETIANS" group. As a consequence all the members of the "CSE BUET 08" group become automatically members of the "BUETIANS" group.
Which design pattern is most appropriate to accommodate this change? How the solution proposed in the previous example shall be extended. Please give a short explanation why. Show a complete class diagram(s) and enough code fragments to illustrate your use of the pattern to solve the problem.

SOLUTION

Step 1 – Starting point: the solution of 4(c)

In 4(c) the group wall is the **subject** and users are **observers** (the Observer pattern). A `Group` keeps a list of members. `join()` and `leave()` are *attach* and *detach*, and `postOnWall()` calls `notifyMembers()`, which calls `update(post)` on every member. Each `User` puts the post into its news feed. This behaviour must be kept.

Step 2 – Identifying the pattern for the change

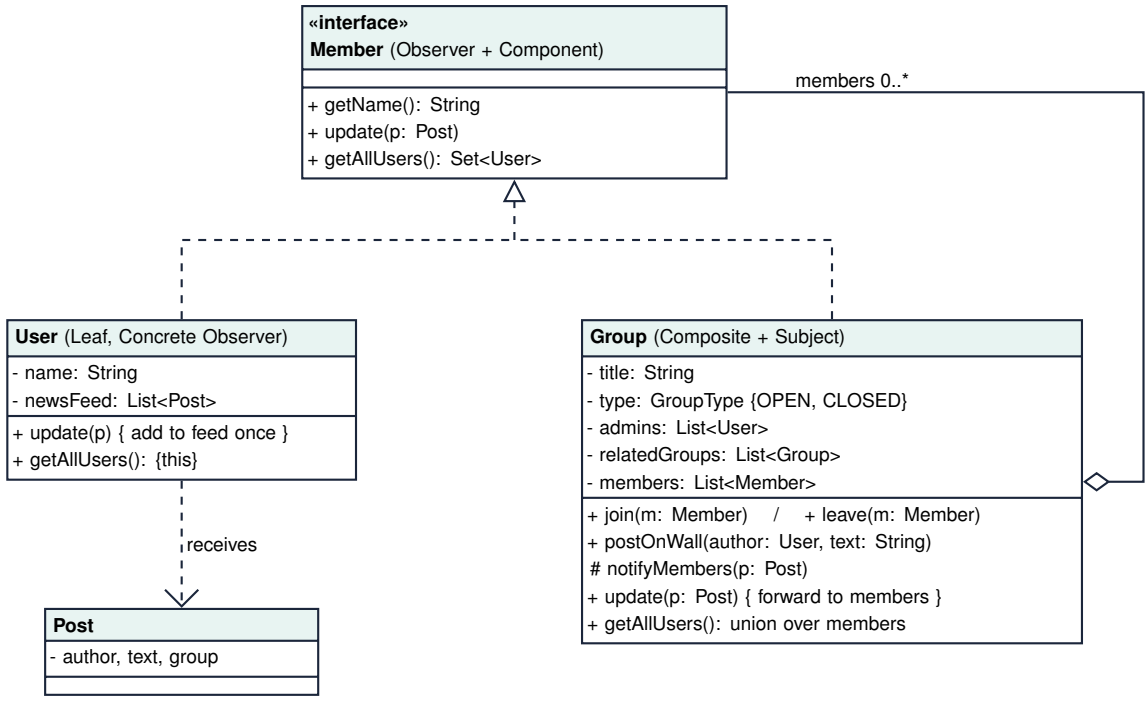
The new requirement is that a *group* can be a member of another group, *just like a user*. Its own members then become members of the outer group automatically. So:

- a member can be either an individual (`User`) or a *collection* of members (`Group`);
- groups can nest to any depth (CSE BUET 08 $\subset$ BUETIANS $\subset \ldots$), which forms a tree;
- the outer group must treat both kinds of member *uniformly*: it just calls `update(post)`.

This is the Composite pattern: compose objects into tree structures and work with them as if they were individual objects.

How 4(c) is extended: the observer interface `Member` becomes the Composite **Component**. `User` is the **Leaf**. `Group` now *also* implements `Member`, so it is the **Composite**, while remaining the **Subject** of its own wall. When a group receives `update(post)` from an outer group, it forwards the post to all of its own members. That is how "all members of CSE BUET 08 become members of BUETIANS" without copying any member list. Users who join CSE BUET 08 later are covered automatically.

Step 3 – Class diagram (Observer + Composite)



Step 4 – Code fragments

```
import java.util.*;

class Post {
    final User author; final String text; final String group;
```

```
    Post(User a, String t, String g) { author = a; text = t; group = g; }
}

// ---------- Component + Observer ----------
interface Member {
    String getName();
    void update(Post p);                    // observer notification
    Set<User> getAllUsers();                // composite query
}

// ---------- Leaf / concrete observer ----------
class User implements Member {
    private final String name;
    private final List<Post> newsFeed = new ArrayList<>();
    User(String name) { this.name = name; }
    public String getName() { return name; }
    public void update(Post p) {            // a user in several nested groups sees a post once
        if (!newsFeed.contains(p)) {
            newsFeed.add(p);
            System.out.println(name + " feed <- [" + p.group + "] " + p.text);
        }
    }
    public Set<User> getAllUsers() { return Set.of(this); }
}

enum GroupType { OPEN, CLOSED }

// ---------- Composite + Subject ----------
class Group implements Member {
    private final String title; private final GroupType type;
    private final List<User> admins = new ArrayList<>();
    private final List<Group> relatedGroups = new ArrayList<>();
    private final List<Member> members = new ArrayList<>();   // users OR groups

    Group(String title, GroupType type, User admin) {
        this.title = title; this.type = type; admins.add(admin); members.add(admin);
    }
    public String getName() { return title; }

    // subject: attach / detach (from 4(c)), now accepting any Member
    public void join(Member m) {
        if (m == this || (m instanceof Group && ((Group) m).contains(this)))
            throw new IllegalArgumentException("A group cannot (indirectly) join itself");
        if (!members.contains(m)) members.add(m);
    }
    public void leave(Member m) { members.remove(m); }

    public void postOnWall(User author, String text) {
        notifyMembers(new Post(author, text, title));
    }
    protected void notifyMembers(Post p) { for (Member m : members) m.update(p); }

    // composite: a group that is itself a member forwards the update to its own members
    public void update(Post p) { notifyMembers(p); }

    public Set<User> getAllUsers() {
        Set<User> all = new LinkedHashSet<>();
        for (Member m : members) all.addAll(m.getAllUsers());     // recursion through nesting
```

```
        return all;
    }
    boolean contains(Member m) {
        for (Member x : members)
            if (x == m || (x instanceof Group && ((Group) x).contains(m))) return true;
        return false;
    }
    void addRelatedGroup(Group g) { relatedGroups.add(g); }
}

// ---------- Client ----------
public class FacebookDemo {
    public static void main(String[] args) {
        User rafi = new User("Rafi"), mitu = new User("Mitu"), sami = new User("Sami");
        Group cse08    = new Group("CSE BUET 08", GroupType.CLOSED, rafi);
        Group buetians = new Group("BUETIANS",   GroupType.OPEN,   sami);
        cse08.join(mitu);

        buetians.join(cse08);          // the group page joins another group
        buetians.postOnWall(sami, "Reunion on Friday!");
        // Sami, Rafi and Mitu all receive it; Rafi and Mitu via CSE BUET 08

        User tuli = new User("Tuli");
        cse08.join(tuli);              // joins only CSE BUET 08 ...
        buetians.postOnWall(sami, "Venue: ECE building");   // ... but gets BUETIANS posts too
        System.out.println(buetians.getAllUsers().size());  // 4
    }
}
```

Step 5 – Why this is the right extension

- **Minimal change to 4(c):** the Observer structure stays. We only (1) let `Group` implement the member/observer interface and (2) let `join()` accept any `Member`. `User` is unchanged.
- **Uniform treatment:** `notifyMembers()` does not care whether a member is a user or a group. This is the Composite's main benefit.
- **Automatic membership:** nesting is by reference, so membership is always current. There is no copying or synchronisation of member lists.
- **Safety concerns handled:** cycle check in `join()`, and duplicate suppression in `User.update()` for users who are members both directly and through a sub-group.

Answer. Composite, combined with the Observer of 4(c). `Member` becomes both the observer interface and the Composite component, `User` is the leaf, and `Group` is the composite (and still the subject). A group can `join()` another group, and when it receives `update(post)` it forwards the post to all its members, so every member of CSE BUET 08 automatically gets BUETIANS updates.

2010–2011 • Q4(e) CSE307 • 19/03/2012 • D040.3 **MARKS 7**

Suppose Alice told you the following story
"At the company for which I work we are facing this problem: We have a large amount of application software written using a particular library. Let's call this library LibX. The provider of this library was XIndustries AB. We wanted to extend the functionality of our application and for

this reason we tried to contact XIndustries AB to implement the necessary library functionality. To our surprise we found out that XIndustries AB has gone out of business. Fortunately we found out that there is another company called YEnterprise AB is producing LibY that provides the same functionality as LibX but many of the classes have different interfaces. We don't have access to the source code of the old library LibX and neither to the source code of LibY. What should we do?"
Which design pattern is most appropriate to accommodate this change? In particular, show an appropriate class diagram to solve the problem.

SOLUTION

Step 1 – Identifying the pattern

- A *large* body of application code is written against LibX's interfaces. Rewriting all of it is costly and risky.
- LibY offers the *same functionality* under *different interfaces*.
- Neither library's source can be changed.

The application expects one interface, and the available implementation offers another. We cannot edit either side, so we need a translator in between. This is the **Adapter** pattern: *convert the interface of a class into another interface clients expect*.

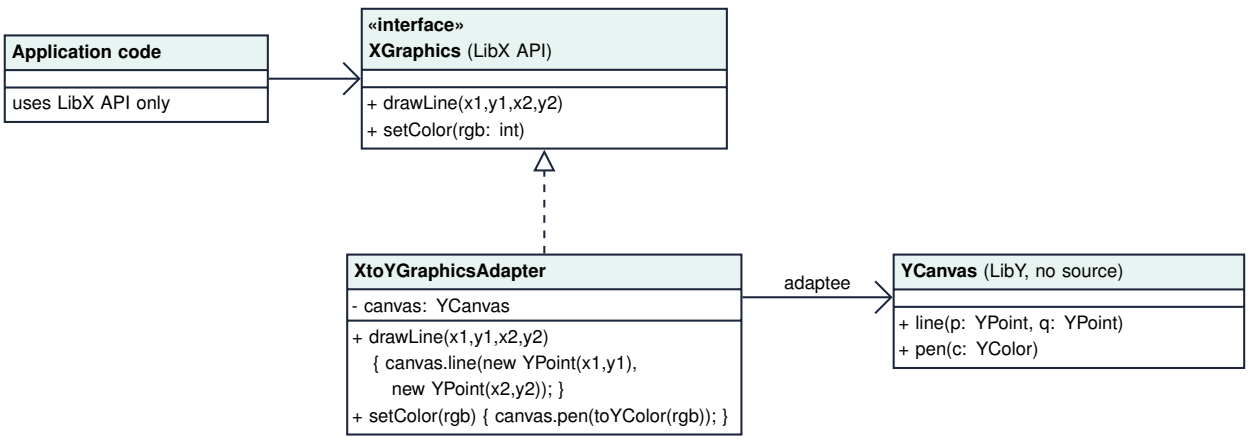
Step 2 – How to apply it

(i) Keep the LibX API that the application uses (its interface or abstract types, which are available in the compiled jar or header files) as the **Target**. If LibX exposes only concrete classes, extract a thin interface `XGraphics` with the same method signatures and change only the creation points (ideally a factory) in our code.

(ii) For each LibX class whose interface differs in LibY, write an **adapter** that implements the LibX-style interface and *delegates* to the corresponding LibY object (**Adaptee**), converting method names, parameter orders, units and return types.

(iii) Use *object* adapters (composition). They need no source code of LibY and also work with LibY subclasses.

Step 3 – Class diagram (one adapted class shown; repeat per class)



EXAM TIP This is the classic "vendor library replaced" situation from the lecture (the new vendor class plays the Turkey, our existing code expects a Duck). The application keeps compiling against the old interface, and only the adapters know about LibY.

Answer. Adapter pattern. Keep LibX's interfaces as the Target, write one object adapter per LibX class that implements that interface and delegates to the matching LibY class (the Adaptee), translating the calls. The application code and both libraries remain unchanged.

04

Behavioural Design Patterns

2023–2024 • Q5(b) CSE213 • 10/05/2026 • D003 **MARKS 10**

A Smart Home Hub coordinates communication among devices such as motion sensors, lights, alarms, and other smart devices.

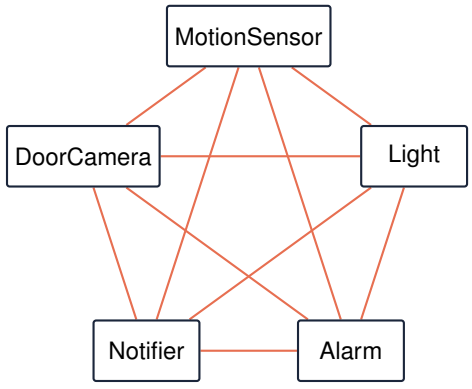
(i) Suppose a motion sensor event may trigger one or more coordinated actions, such as turning on a light, activating an alarm, or notifying the hub. Illustrate how the Mediator pattern is preferable here than allowing the Motion Sensor to communicate with the other device objects directly.

(ii) Suppose they system must support recording, queueing, and later replaying of sensor-triggered actions such as turning on a light, activating an alarm, or sending a notification. Select a suitable behavioral design pattern for this requirement and show how it can be applied.

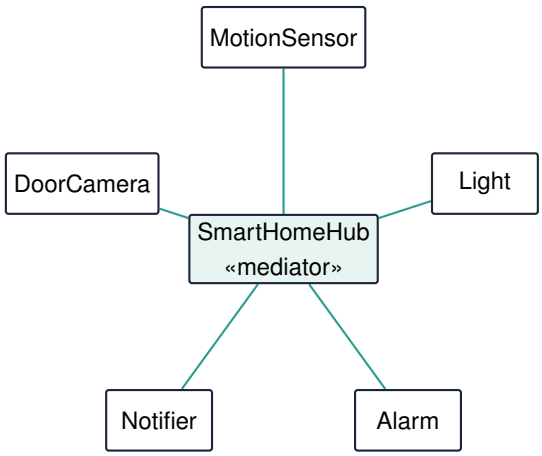
SOLUTION

(i) Why a Mediator is preferable to direct communication

Mediator (slides): *"restricts direct communications between objects and forces them to collaborate only via a mediator. Lets you reduce chaotic dependencies between objects."*
Without a mediator the `MotionSensor` itself must hold references to the `Light`, the `Alarm`, the notification service, … and must contain the coordination rules ("at night switch the light on", "if the house is armed raise the alarm and notify the owner").



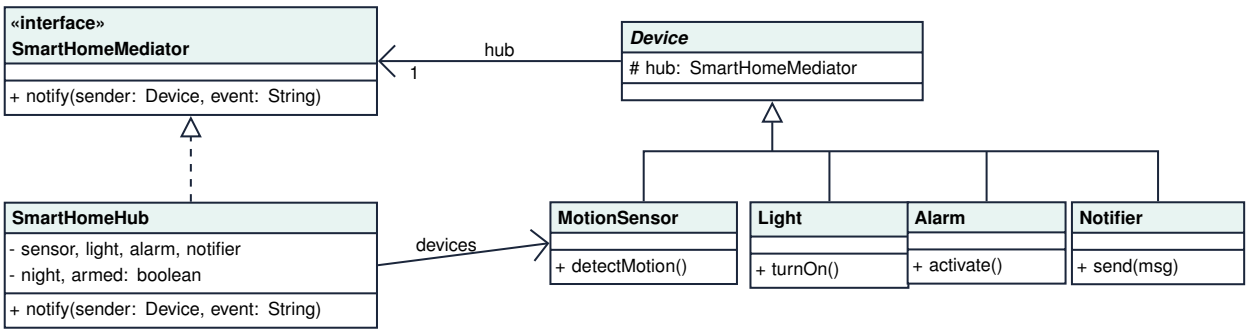
(a) direct: every device may know every other



(b) mediator: each device knows only the hub

Aspect	Sensor talks to devices directly	Via SmartHomeHub (Mediator)
Coupling	sensor depends on every device class; n devices $\Rightarrow$ up to $n(n-1)$ links	every device depends only on the hub interface; n links
Reuse	sensor cannot be reused in another home/layout	sensor is reusable anywhere; it only reports `"MOTION"`
Changing a rule	edit `MotionSensor` (and maybe others)	edit only the hub (rules in one place)
Adding a device	modify existing device classes	register it with the hub; devices unchanged (OCP)

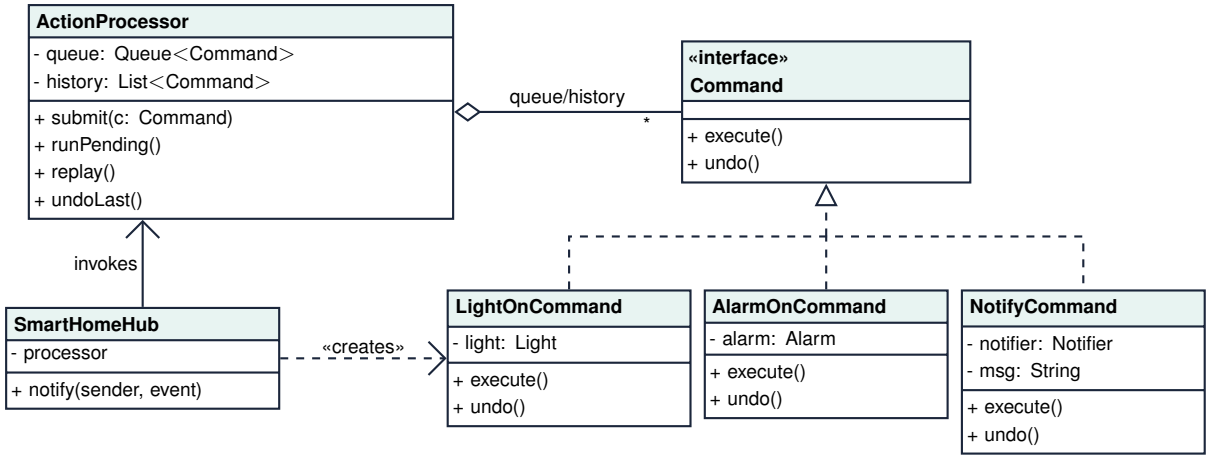
Mapping: **Mediator** = `SmartHomeMediator`; **Concrete mediator** = `SmartHomeHub`; **Components (colleagues)** = `MotionSensor`, `Light`, `Alarm`, `Notifier`, each holding only a reference to the mediator, exactly like the dialog/components example of the slides.



(ii) Recording, queueing and replaying actions: Command pattern

The requirement is to treat each action (*turn on light*, *activate alarm*, *send notification*) as something that can be *stored, put in a queue and executed again later*. That is precisely the intent of the **Command** pattern: *"turns a request into a stand-alone object that contains all information about the request. This transformation lets you parameterize methods with different requests, delay or queue a request's execution, and support undoable operations."*

- **Command:** `Command` interface with `execute()` (and `undo()`).
- **Concrete commands:** `LightOnCommand`, `AlarmOnCommand`, `NotifyCommand` – each pre-configured with its receiver and parameters.
- **Receivers:** `Light`, `Alarm`, `Notifier`.
- **Invoker:** `ActionProcessor` – keeps a *queue* of pending commands and a *history* list (recording); `replay()` re-executes the history.
- **Client:** the hub (mediator) creates the commands when the sensor fires.



Implementation of (i) + (ii) together (Java)

```java
import java.util.*;

// ---------- Mediator part ----------
interface SmartHomeMediator { void notify(Device sender, String event); }

abstract class Device {                                  // colleague
    protected final SmartHomeMediator hub;
    Device(SmartHomeMediator hub) { this.hub = hub; }
}
class MotionSensor extends Device {
    MotionSensor(SmartHomeMediator h) { super(h); }
    void detectMotion() { hub.notify(this, "MOTION"); }   // knows only the hub
}
class Light extends Device {
    Light(SmartHomeMediator h) { super(h); }
    void turnOn()  { System.out.println("Light ON"); }
    void turnOff() { System.out.println("Light OFF"); }
}
class Alarm extends Device {
    Alarm(SmartHomeMediator h) { super(h); }
    void activate()   { System.out.println("Alarm ACTIVATED"); }
    void deactivate() { System.out.println("Alarm off"); }
}
class Notifier extends Device {
    Notifier(SmartHomeMediator h) { super(h); }
    void send(String msg) { System.out.println("Notification: " + msg); }
}

// ---------- Command part ----------
interface Command { void execute(); void undo(); }

class LightOnCommand implements Command {
    private final Light light;
    LightOnCommand(Light l) { light = l; }
    public void execute() { light.turnOn(); }
    public void undo()    { light.turnOff(); }
}
class AlarmOnCommand implements Command {
    private final Alarm alarm;
    AlarmOnCommand(Alarm a) { alarm = a; }
    public void execute() { alarm.activate(); }
```

```
    public void undo()    { alarm.deactivate(); }
}
class NotifyCommand implements Command {
    private final Notifier notifier; private final String msg;
    NotifyCommand(Notifier n, String msg) { notifier = n; this.msg = msg; }
    public void execute() { notifier.send(msg); }
    public void undo()    { /* a sent notification cannot be recalled */ }
}

class ActionProcessor {                                    // invoker
    private final Queue<Command> queue = new ArrayDeque<>();
    private final List<Command> history = new ArrayList<>();
    void submit(Command c) { queue.add(c); }               // queueing
    void runPending() {
        while (!queue.isEmpty()) {
            Command c = queue.poll();
            c.execute();
            history.add(c);                                // recording
        }
    }
    void replay()   { for (Command c : history) c.execute(); }  // later replay
    void undoLast() { if (!history.isEmpty()) history.remove(history.size() - 1).undo(); }
}

// ---------- Concrete mediator = client of the commands ----------
class SmartHomeHub implements SmartHomeMediator {
    MotionSensor sensor; Light light; Alarm alarm; Notifier notifier;
    boolean night = true, armed = true;
    final ActionProcessor processor = new ActionProcessor();

    public void notify(Device sender, String event) {
        if (sender == sensor && event.equals("MOTION")) {   // coordination rules in ONE place
            if (night) processor.submit(new LightOnCommand(light));
            if (armed) {
                processor.submit(new AlarmOnCommand(alarm));
                processor.submit(new NotifyCommand(notifier, "Motion detected at home"));
            }
        }
    }
}

public class SmartHomeDemo {
    public static void main(String[] args) {
        SmartHomeHub hub = new SmartHomeHub();
        hub.sensor = new MotionSensor(hub); hub.light = new Light(hub);
        hub.alarm = new Alarm(hub);         hub.notifier = new Notifier(hub);

        hub.sensor.detectMotion();          // actions are only queued
        hub.processor.runPending();         // executed and recorded
        System.out.println("--- replay ---");
        hub.processor.replay();             // e.g. for testing / audit
    }
}
```

Answer. (i) Mediator: devices talk only to `SmartHomeHub`, which holds all coordination rules, so the motion sensor is decoupled, reusable and the many-to-many links become one-to-many. (ii) Command pattern: each sensor-triggered action is a `Command` object; an invoker (`ActionProcessor`) queues them, records executed ones in a history and can replay (or undo) them.

2023–2024 • Q8(a), (b), (c) CSE213 • 10/05/2026 • D004 **MARKS 15 + 10 + 10**

(a) Consider the following design for a file-parsing framework: **(15)**

```
abstract class DataParser {
    public final void parse(String fileName) {
        open(fileName);
        extract();
        close();
    }
    protected void open(String fileName) {
        System.out.println("Opening file: " + fileName);
    }
    protected abstract void extract();
    protected void close() {
        System.out.println("Closing file");
    }
}
class CsvParser extends DataParser {
    protected void extract() {
        System.out.println("Extracting data from CSV");
    }
}
class JsonParser extends DataParser {
    protected void extract() {
        System.out.println("Extracting data from JSON");
    }
}
```

The design follows the Template pattern, where the overall parsing workflow is fixed as: `open()` → `extract()` → `close()`. Currently, different file formats, such as CSV and JSON, are handled by creating separate subclasses that override the `extract()` method. This approach has a limitation: every new file format requires a new subclass, even though the overall parsing workflow remains the same. Redesign the solution and write the relevant Java code to demonstrate how the Strategy pattern can be used within the Template pattern so that the extraction logic becomes interchangeable at runtime.

(b) A news agency needs a system where multiple displays (Website, Mobile App, and Office Billboard) are updated instantly whenever a new "Breaking News" headline is published. What is the most appropriate Behavioral Design Pattern for this scenario? Justify your answer. **(10)**

(c) Compare State and Strategy patterns. Why are they structurally similar but conceptually different? **(10)**

SOLUTION

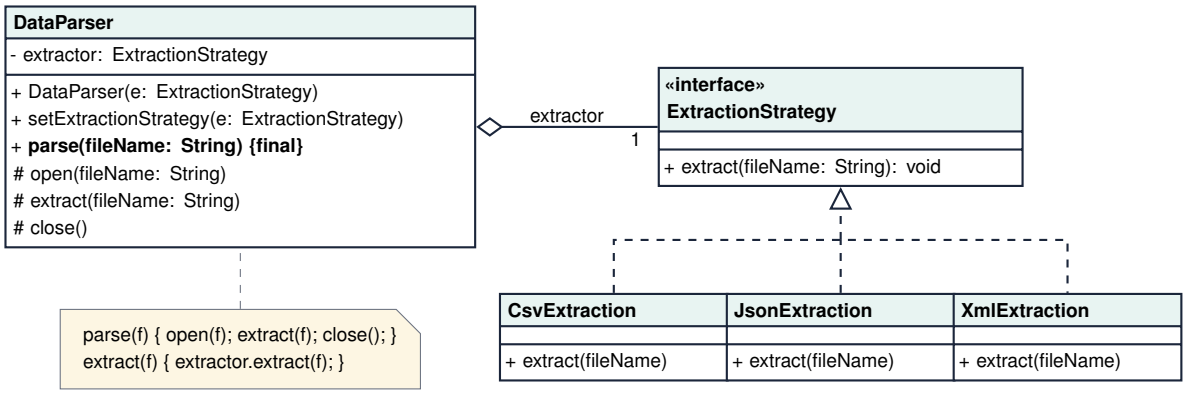
(a) Strategy inside a Template Method

The problem. Template Method *"defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps"*. It is *inheritance*-based and works *at class level* (static): the extraction step is fixed when the object is created (`new CsvParser()`), and each new format needs one more subclass, although only one step varies.

The redesign. Keep the template method `parse()` (the invariant skeleton `open` → `extract` → `close` stays `final` in `DataParser`), but *delegate* the varying step to an object: `extract()` calls an `ExtractionStrategy`. As the slides put it, Strategy is *composition*-based and *"works on the object level, letting you switch behaviors at runtime"*. So:

- Template Method still controls *the order of steps*;

- **Strategy** supplies *how* the extraction step is done, and can be replaced with `setExtractionStrategy()` while the program runs;
- a new format = a new small strategy class; no new parser subclass.




```
interface ExtractionStrategy {                          // Strategy
    void extract(String fileName);
}
class CsvExtraction implements ExtractionStrategy {
    public void extract(String f) { System.out.println("Extracting data from CSV"); }
}
class JsonExtraction implements ExtractionStrategy {
    public void extract(String f) { System.out.println("Extracting data from JSON"); }
}
class XmlExtraction implements ExtractionStrategy {  // new format: NO new parser subclass
    public void extract(String f) { System.out.println("Extracting data from XML"); }
}

class DataParser {                                      // template + context
    private ExtractionStrategy extractor;
    DataParser(ExtractionStrategy extractor) { this.extractor = extractor; }
    public void setExtractionStrategy(ExtractionStrategy e) { this.extractor = e; }

    public final void parse(String fileName) {       // TEMPLATE METHOD (unchanged order)
        open(fileName);
        extract(fileName);
        close();
    }
    protected void open(String fileName) { System.out.println("Opening file: " + fileName); }
    protected void extract(String fileName) { extractor.extract(fileName); }  // delegated step
    protected void close() { System.out.println("Closing file"); }
}

public class ParserDemo {
    public static void main(String[] args) {
        DataParser parser = new DataParser(new CsvExtraction());
        parser.parse("sales.csv");

        parser.setExtractionStrategy(new JsonExtraction());   // switched at run time
        parser.parse("config.json");

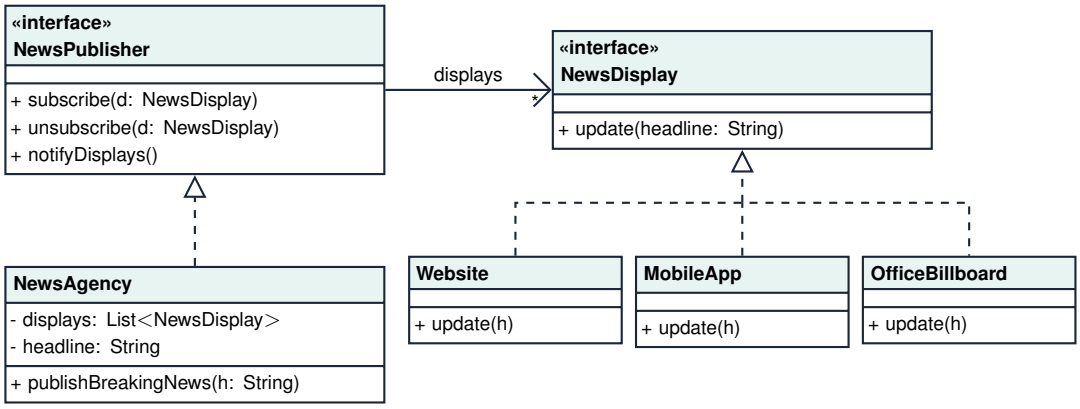
        parser.setExtractionStrategy(new XmlExtraction());
        parser.parse("feed.xml");
    }
}
```

`open()` and `close()` remain `protected` hooks, so a subclass can still customise them (e.g. a `RemoteDataParser` that opens a URL) – the template part is preserved, while the part that *varies by format* has moved to composition (*"favour composition over inheritance"*, OCP).

(b) News agency: Observer pattern

Answer: Observer. Justification from the problem forces:

- *One* publisher (the news agency) and *many* dependents (Website, Mobile App, Office Billboard) – a *one-to-many* dependency.
- The dependents must be updated *automatically and instantly* when the publisher's state changes (a new headline) – exactly *"when one object changes state, all its dependents are notified and updated automatically"*.
- The set of displays can grow or shrink (a new TV ticker, an SMS service): observers subscribe/unsubscribe at run time and the agency, which knows only the `Observer` interface, is never modified (loose coupling, OCP).
- It is not Mediator: there is no many-to-many interaction among the displays to be coordinated; the slides note that Observer *"establishes dynamic one-way one-to-many connection"*, while Mediator *"eliminates mutual dependencies"*.

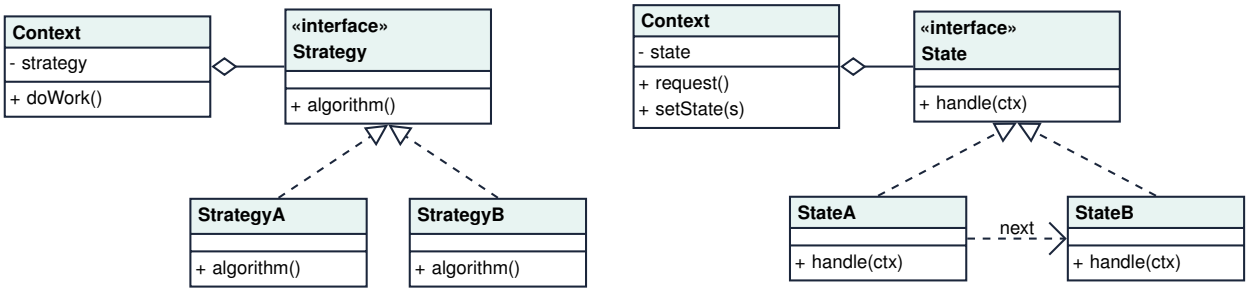



```java
interface NewsDisplay { void update(String headline); }

class NewsAgency {                                          // subject
    private final List<NewsDisplay> displays = new ArrayList<>();
    void subscribe(NewsDisplay d)   { displays.add(d); }
    void unsubscribe(NewsDisplay d) { displays.remove(d); }
    void publishBreakingNews(String headline) {
        for (NewsDisplay d : displays) d.update(headline);   // instant push
    }
}
class Website implements NewsDisplay {
    public void update(String h) { System.out.println("Website banner: " + h); }
}
class MobileApp implements NewsDisplay {
    public void update(String h) { System.out.println("Push notification: " + h); }
}
class OfficeBillboard implements NewsDisplay {
    public void update(String h) { System.out.println("Billboard scroll: " + h); }
}
// main: agency.subscribe(new Website()); ... agency.publishBreakingNews("Election results out");
```

(c) State vs Strategy

Why structurally similar. The slides call them *"twins separated at birth"*: both are based on *composition*; a *Context* holds a reference to an interface, several concrete classes implement that interface, and the context *delegates* work to the current object. Their class diagrams are practically identical:



Why conceptually different.

Aspect	Strategy	State
Intent	"family of algorithms, encapsulate each one, make them interchangeable"	"allow an object to alter its behavior when its internal state changes; the object will appear to change its class"
What the objects represent	alternative *ways of doing the same task* (sorting algorithms, fly behaviours)	*conditions/modes* of the context (NoQuarter, HasQuarter, Sold, . . .)
Who chooses / changes it	normally the *client* chooses and sets the strategy	the *context or the state objects themselves* trigger transitions as events happen
Knowledge of each other	strategies are completely independent and unaware of each other	states may know other states ("knows who's next") and alter the context's state at will
Reference to context	usually not needed	usually needed (`GumballMachine` reference passed in)
Frequency of change	seldom; often fixed for a task	frequently, during the object's life cycle
Replaces	a conditional selecting an algorithm	a large state-dependent `if/switch` in every method

Answer. (a) Keep `parse()` as the final template method in `DataParser` but delegate the `extract` step to an `ExtractionStrategy` field that can be swapped by `setExtractionStrategy()`; new formats are new strategy classes. (b) Observer: the agency is the subject, the three displays are observers notified on every new headline. (c) Both delegate from a context to an interface via composition (same structure), but Strategy encapsulates client-selected, mutually independent algorithms, whereas State encapsulates the context's internal modes, and states drive the transitions themselves.

2023–2024 • Q2(a) CSE213 • 09/02/2025 • D005 MARKS 6 + 16

You are required to build a notification system named *StockMarket*. The system can generate *StockGrabber* objects. Each *StockGrabber* object sends notifications to all of its registered observers,

only when a different price of a stock item is set. If the price is not changed, then setting the same value to any stock item does not generate any notification. When a *StockObserver* object gets registration under a *StockGrabber*, it gets an autogenerated *observerID* (starting from 1) and knows the prices of all stock items known to the associated *StockGrabber*. Assume for starting the system, when a *StockGrabber* is generated, it gets prices of three stocks items – *aplPrice* ($230 per unit), *googPrice* ($200 per unit) and *ibmPrice* ($180 per unit). A part of the program is given below.

```
public interface StockExchnage {
    public void register(StockObserver newObserver);
    public void unregister(StockObserver newObserver);
    public void notifyAllObserver(String company, double
                                  price);
}

public interface Observer {
    public void update(String company, double price);
}

public class StockMarket {
    public static void main(String [] args){
        StockGrabber securityCompany = new
                                 StockGrabber("Shamol");
        Observer observer1 = new
                    StockObserver(securityCompany, "Arif");
        Observer observer2 = new
                    StockObserver(securityCompany, "Kamal");
        securityCompany.setAplPrice(250.0);
        securityCompany.setGoogPrice(200.0);
    }
}
```

(i) Draw the appropriate UML diagram for the above descriptions using appropriate design pattern. **(6)**

(ii) Write down the codes for *StockGrabber* and *StockObserver* class so that no changes in the above code is required using appropriate Design Pattern. Ensure that the classes have minimum number of methods with simplified design and avoid any setters and getters which is unnecessary for the above code segment. **(8×2=16)**

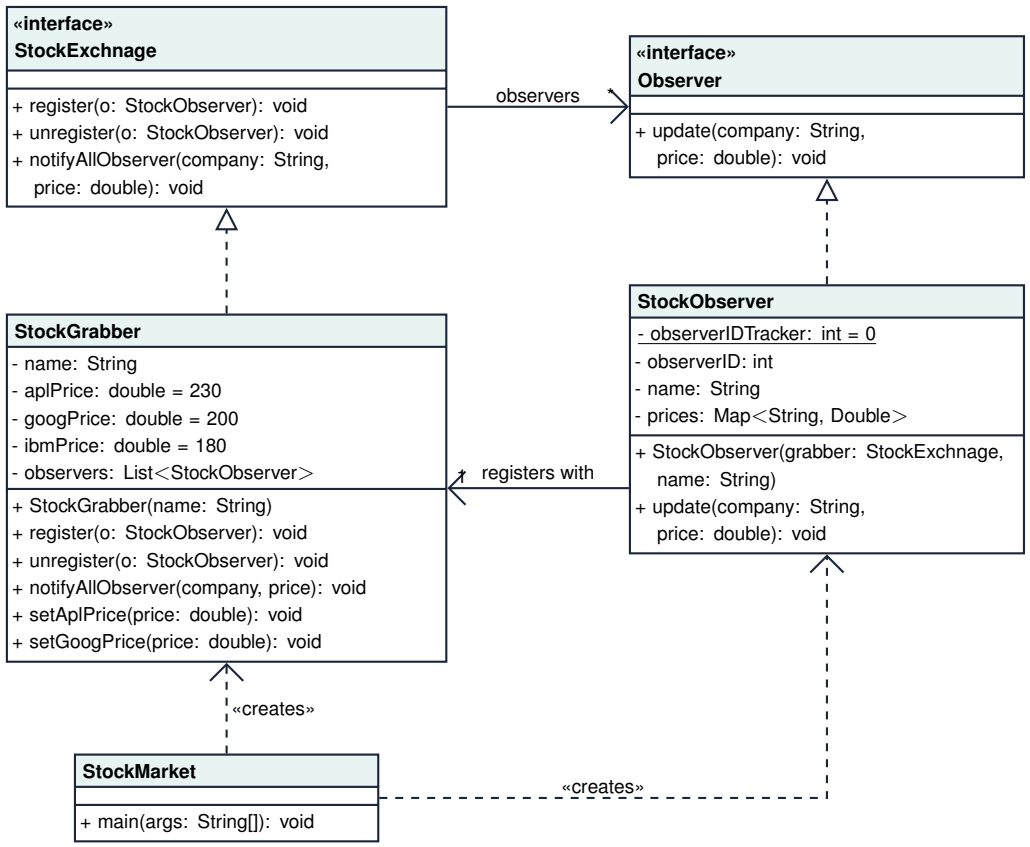
SOLUTION

Identifying the pattern

One `StockGrabber` (publisher) must inform *many* `StockObservers` automatically when its state (a stock price) changes, and observers can register/unregister at any time. This is the **Observer** pattern: *"Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically."* The given interfaces already fix the roles: `StockExchnage` = Subject (publisher) interface, `Observer` = Observer (subscriber) interface.

Observer-pattern role	Class
Subject interface	`StockExchnage` (`register`, `unregister`, `notifyAllObserver`) – name kept as given
Concrete subject	`StockGrabber` – keeps the three prices and the observer list
Observer interface	`Observer` (`update(company, price)`)
Concrete observer	`StockObserver` – has an auto-generated `observerID` and its own copy of all prices
Client	`StockMarket.main()` (must compile unchanged)

(i) UML class diagram



The association `observers` is shown at interface level (as in the slides): the subject knows its observers only through the `Observer` abstraction; `StockObserver` registers itself with the grabber passed to its constructor.

(ii) Code for `StockGrabber` (concrete subject)

```
import java.util.ArrayList;
import java.util.List;

public class StockGrabber implements StockExchnage {
    private final String name;
    private final List<StockObserver> observers = new ArrayList<>();
    private double aplPrice = 230, googPrice = 200, ibmPrice = 180;   // initial prices

    public StockGrabber(String name) { this.name = name; }
```

```
    @Override
    public void register(StockObserver newObserver) {
        observers.add(newObserver);
        // a newly registered observer must know ALL current prices
        newObserver.update("APL",  aplPrice);
        newObserver.update("GOOG", googPrice);
        newObserver.update("IBM",  ibmPrice);
    }

    @Override
    public void unregister(StockObserver observer) { observers.remove(observer); }

    @Override
    public void notifyAllObserver(String company, double price) {
        System.out.println(name + ": " + company + " price changed to " + price);
        for (Observer o : observers) o.update(company, price);
    }

    // only the setters used by main(); notify ONLY when the value really changes
    public void setAplPrice(double price) {
        if (price != aplPrice) { aplPrice = price; notifyAllObserver("APL", price); }
    }
    public void setGoogPrice(double price) {
        if (price != googPrice) { googPrice = price; notifyAllObserver("GOOG", price); }
    }
}
```

(ii) Code for `StockObserver` (concrete observer)

```
import java.util.LinkedHashMap;
import java.util.Map;

public class StockObserver implements Observer {
    private static int observerIDTracker = 0;   // shared counter -> IDs 1, 2, 3, ...
    private final int observerID;
    private final String name;
    private final Map<String, Double> prices = new LinkedHashMap<>(); // all known prices

    public StockObserver(StockExchnage grabber, String name) {
        this.observerID = ++observerIDTracker;  // auto-generated, starts from 1
        this.name = name;
        grabber.register(this);                 // self-registration with the subject
    }

    @Override
    public void update(String company, double price) {
        prices.put(company, price);
        System.out.println("  Observer " + observerID + " (" + name + ") knows " + prices);
    }
}
```

Run of the given `main()` (unchanged)

```
  Observer 1 (Arif) knows {APL=230.0}
  Observer 1 (Arif) knows {APL=230.0, GOOG=200.0}
  Observer 1 (Arif) knows {APL=230.0, GOOG=200.0, IBM=180.0}
  Observer 2 (Kamal) knows {APL=230.0}
  Observer 2 (Kamal) knows {APL=230.0, GOOG=200.0}
  Observer 2 (Kamal) knows {APL=230.0, GOOG=200.0, IBM=180.0}
```

```
Shamol: APL price changed to 250.0
  Observer 1 (Arif) knows {APL=250.0, GOOG=200.0, IBM=180.0}
  Observer 2 (Kamal) knows {APL=250.0, GOOG=200.0, IBM=180.0}
```

`setGoogPrice(200.0)` produces *no* output, because 200 is the current Google price.

Why the design is minimal

- `StockGrabber` has only the constructor, the three interface methods and the two setters that `main()` calls; `setIbmPrice()` and all getters are omitted because the given code never needs them.
- `StockObserver` has only a constructor and `update()`. It learns the initial prices through `update()` calls made in `register()` (push model), so no getter on the subject is needed.
- Adding a new kind of observer (e.g. an SMS alert) needs no change in `StockGrabber`: it is coupled only to the `Observer` interface (loose coupling, OCP).

> **NOTE** The interface name `StockExchnage` (sic) and the parameter type `StockObserver` of `register/unregister` are kept exactly as printed, since the question requires that the given code compiles without change.

Answer. Observer pattern – `StockGrabber` is the concrete subject implementing `StockExchnage`; `StockObserver` is the concrete observer implementing `Observer`, self-registers in its constructor, gets a static-counter ID from 1, and is notified only when a setter changes a price.

2022–2023 • Q2(a) CSE307 • 05/05/2024 • D009 **MARKS 15**

Air traffic controller (ATC) helps in communication between flights and coordinates/controls landing, take-off. Two flights need not interact directly and there is no dependency between them. This dependency is solved by the ATC. Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfills all the requirements.

SOLUTION

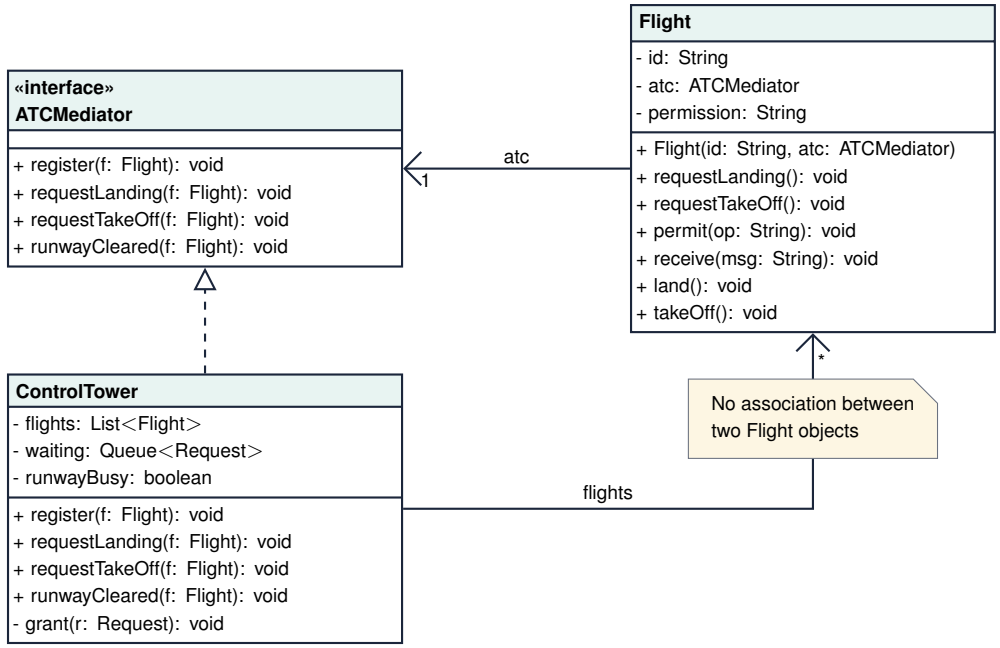
Identifying the pattern

Many flights must coordinate their use of a shared resource (the runway), yet "two flights need not interact directly and there is no dependency between them". If every flight negotiated with every other flight we would get a many-to-many web of dependencies. Moving all the coordination into one object (the ATC tower) through which flights communicate is the **Mediator** pattern – it is the very example used in the slides: *"Aircraft pilots don't talk to each other directly; all communication goes through the control tower."*

Participants

- **Mediator:** `ATCMediator` interface – `register`, `requestLanding`, `requestTakeOff`, `runwayCleared`.
- **Concrete mediator:** `ControlTower` – knows all flights, the runway status and a FIFO queue of waiting requests; decides who may use the runway.
- **Colleagues (components):** `Flight` objects – each knows *only* the mediator.

Class diagram



Implementation (Java)

```java
import java.util.*;

interface ATCMediator {                                    // Mediator
    void register(Flight f);
    void requestLanding(Flight f);
    void requestTakeOff(Flight f);
    void runwayCleared(Flight f);
}

class ControlTower implements ATCMediator {                // Concrete mediator
    private static class Request {
        final Flight flight; final String op;              // op = "LAND" / "TAKE OFF"
        Request(Flight f, String op) { flight = f; this.op = op; }
    }
    private final List<Flight> flights = new ArrayList<>();
    private final Queue<Request> waiting = new ArrayDeque<>();
    private boolean runwayBusy = false;

    public void register(Flight f)       { flights.add(f); }
    public void requestLanding(Flight f) { handle(new Request(f, "LAND")); }
    public void requestTakeOff(Flight f) { handle(new Request(f, "TAKE OFF")); }

    private void handle(Request r) {
        if (runwayBusy) {
            waiting.add(r);
            r.flight.receive("runway busy, hold. Position " + waiting.size());
        } else grant(r);
    }
    private void grant(Request r) {
        runwayBusy = true;
        r.flight.permit(r.op);
        for (Flight other : flights)                       // inform the others
            if (other != r.flight) other.receive(r.flight.getId() + " is using the runway");
    }
    public void runwayCleared(Flight f) {
```

```
        runwayBusy = false;
        if (!waiting.isEmpty()) grant(waiting.poll());   // next in FIFO order
    }
}

class Flight {                                           // Colleague
    private final String id;
    private final ATCMediator atc;                       // the ONLY dependency
    private String permission;                           // null = not cleared

    Flight(String id, ATCMediator atc) { this.id = id; this.atc = atc; atc.register(this); }
    String getId() { return id; }

    void requestLanding() { System.out.println(id + ": requests landing");  atc.requestLanding(
    this); }
    void requestTakeOff() { System.out.println(id + ": requests take-off"); atc.requestTakeOff(
    this); }

    void permit(String op)    { permission = op; receive("cleared to " + op); }
    void receive(String msg)  { System.out.println("   ATC -> " + id + ": " + msg); }

    void land()    { perform("LAND", "landed"); }
    void takeOff() { perform("TAKE OFF", "took off"); }
    private void perform(String op, String done) {
        if (!op.equals(permission)) { System.out.println(id + ": not cleared to " + op); return;
     }
        System.out.println(id + " " + done + ", runway vacated");
        permission = null;
        atc.runwayCleared(this);                          // tell the mediator, not the flights
    }
}

public class AirportDemo {
    public static void main(String[] args) {
        ATCMediator tower = new ControlTower();
        Flight bg = new Flight("BG-084", tower);
        Flight ek = new Flight("EK-585", tower);
        Flight qr = new Flight("QR-638", tower);

        bg.requestLanding();   // runway free -> BG cleared to land
        ek.requestTakeOff();   // busy -> EK holds (1)
        qr.requestLanding();   // busy -> QR holds (2)
        ek.takeOff();          // refused: not yet cleared
        bg.land();             // runway cleared -> EK cleared
        ek.takeOff();          // runway cleared -> QR cleared
        qr.land();
    }
}
```

How the requirements are fulfilled

- `Flight` has no field or parameter of another `Flight`; all communication is `Flight` → `ATCMediator` → `Flight`: the dependency between flights is "solved by the ATC".
- Landing/take-off control (runway lock, FIFO waiting queue) lives in a single class, so a new rule (emergency priority, two runways) changes only `ControlTower`.
- `Flight` is reusable with any other `ATCMediator` implementation (e.g. a different airport's tower).

Answer. Mediator pattern: `ATCMediator` (mediator interface) / `ControlTower` (concrete mediator) coordinate landing and take-off of `Flight` colleagues, each of which only references the mediator; flights never call each other.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q4(a) CSE307 • 05/05/2024 • D011 **MARKS 15**

There is a blog of former BUET students. BUET Alumni register to that blog for receiving update. When a new article is posted in the blog, it will send update to the registered users saying a new article is posted. Then the user will access the blog and read the new article posted. Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfils all the requirements following the pattern.

2017–2018 • Q8(b) CSE307 • 29/09/2018 • D026 **MARKS 5 + 12**

Assume that there is a blog and users register to that blog for update. When a new article is posted in the blog, it will send update to the registered users saying a new article is posted. Then the user will access the blog and read the new article posted. Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfils all the requirements.

SOLUTION

Identifying the pattern

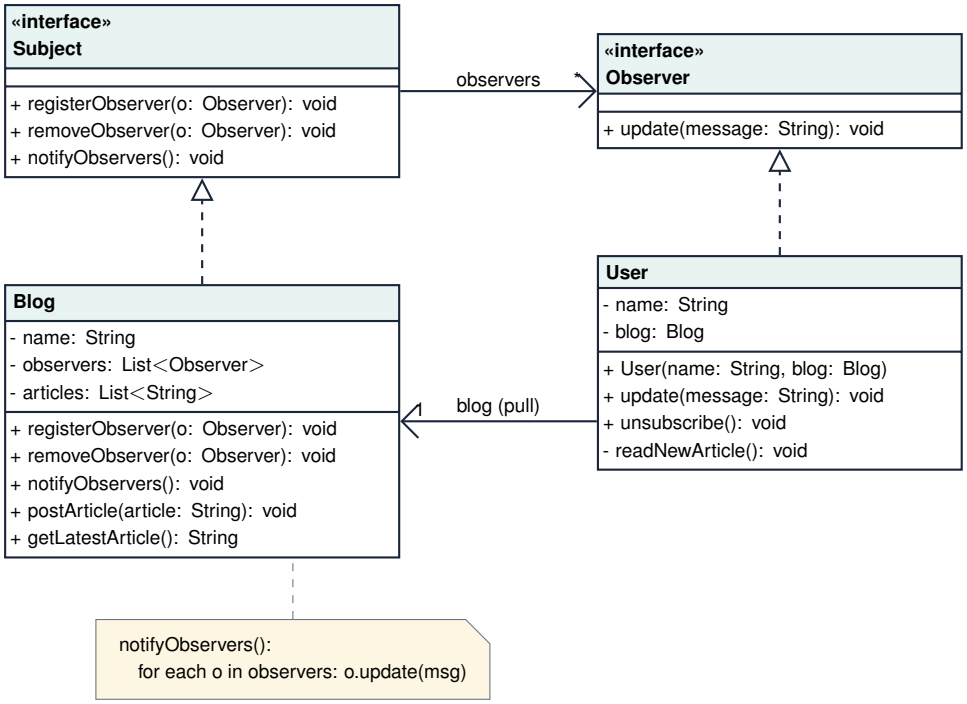
The blog is one object whose state changes (a new article); an open-ended set of users depends on that change and must be told automatically; users can join (register) and, naturally, leave. This is the **Observer** pattern (publisher = subject, subscriber = observer): *"Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically."*

The question describes the **pull** variant of the pattern: the blog only sends the short message "a new article is posted", and *the user then accesses the blog* and reads the article itself. So each user keeps a reference to the blog (as the `WeatherData` reference kept by displays in the slides) and pulls the article with `getLatestArticle()`.

Participants

- **Subject:** `Subject` interface – `registerObserver()`, `removeObserver()`, `notifyObservers()`.
- **Concrete subject:** `Blog` (the BUET alumni blog) – list of articles + list of registered users; `postArticle()` triggers notification.
- **Observer:** `Observer` interface – `update(message)`.
- **Concrete observer:** `User` (an alumnus) – on `update()` it visits the blog and reads the newest article.

Class diagram



Implementation (Java)

```java
import java.util.ArrayList;
import java.util.List;

interface Subject {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}

interface Observer {
    void update(String message);
}

class Blog implements Subject {                        // concrete subject
    private final String name;
    private final List<Observer> observers = new ArrayList<>();
    private final List<String> articles = new ArrayList<>();

    Blog(String name) { this.name = name; }

    public void registerObserver(Observer o) { observers.add(o); }
    public void removeObserver(Observer o)   { observers.remove(o); }
    public void notifyObservers() {
        for (Observer o : observers)
            o.update("A new article is posted on " + name);
    }

    public void postArticle(String article) {          // state change
        articles.add(article);
        notifyObservers();                             // automatic notification
    }
    public String getLatestArticle() {                 // used by observers (pull)
        return articles.isEmpty() ? "" : articles.get(articles.size() - 1);
```

```
    }
}

class User implements Observer {                        // concrete observer
    private final String name;
    private final Blog blog;                            // reference to the subject

    User(String name, Blog blog) {
        this.name = name;
        this.blog = blog;
        blog.registerObserver(this);                    // register for updates
    }
    public void update(String message) {
        System.out.println(name + " got update: " + message);
        readNewArticle();                               // then access the blog
    }
    private void readNewArticle() {
        System.out.println("    " + name + " reads: " + blog.getLatestArticle());
    }
    public void unsubscribe() { blog.removeObserver(this); }
}

public class BlogDemo {
    public static void main(String[] args) {
        Blog alumniBlog = new Blog("BUET Alumni Blog");
        User rafi  = new User("Rafi",  alumniBlog);
        User nabila = new User("Nabila", alumniBlog);

        alumniBlog.postArticle("Reunion 2024: registration open");
        rafi.unsubscribe();                             // Rafi leaves
        alumniBlog.postArticle("Scholarship fund report");  // only Nabila notified
    }
}
```

Output (abridged): both users get the first update and read "Reunion 2024 . . . "; only Nabila gets the second.

Design notes

- `Blog` depends only on the `Observer` interface; new kinds of subscribers (an email digest, a mobile push service) are added without touching `Blog` – **loose coupling**, **OCP**.
- Push vs. pull: we deliberately send only a short message and let the user pull the article, which matches "the user will access the blog and read the new article".

Answer. Observer pattern (pull model): `Blog` is the concrete subject keeping a list of registered `Observers` and calling `update()` on each when `postArticle()` is executed; each `User` (concrete observer) holds a `Blog` reference and on `update()` calls `getLatestArticle()` to read the new post.

2021–2022 • Q4(a) CSE307 • 07/05/2023 • D016 **MARKS 20**

For each of the situations below, identify the most specific pattern used in that situation. You need to write the pattern name with sufficient justification (UML class diagram if necessary).

(i) You are building your own video steaming platform named *NuetFlix*. In the earlier version of *NuetFlix*, it only provides three types of genres: Horror, Thriller and Comedy. A user can

choose any one or more of these genres as their preferred genre. When a movie releases you must recommend the movie to the users who are interested in that particular genre (in which the newly released movie belongs).

(ii) You want to build your own version control system named *jit*. Where you want to provide some basic functionalities such as:
kommit that saves your current updates locally but doesn't sync with the remote version, *pussh* that syncs all the "kommit"ed codes with the remote version and *stashh* that discards all the un"pussh"ed "kommit"s and retrieves the latest remote version to the local.

(iii) You work in a telecommunication company named *BanglaPhone*. *BanglaPhone* have two different implementations for their mobile communication. One of the implementation is more stable, can support a lot of users but it incurs more cost and tends to be slower. The other implementation is fast, cheaper but less stable if user count surpasses a threshold. Your company wants you to switch between the implementations without hampering any other code segments based on the current number of user under a specific cell tower.

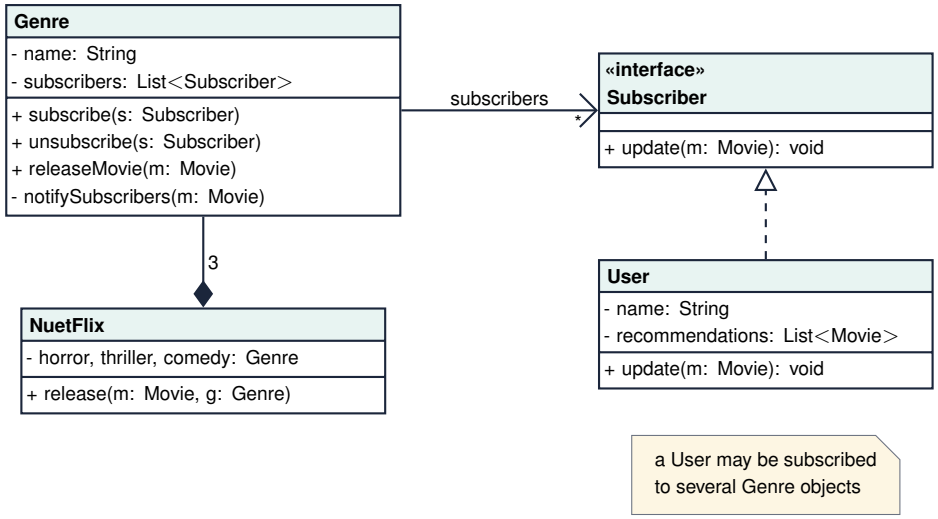
(iv) You work on a software development team of a peer-reviewed journal named *R2-D2*. A peer-reviewed journal works like this: Authors submit their article in the journal. A team of expert receives a number of such articles and express their opinion (known as reviews). These reviews are acknowledged by the authors. Authors try to update their article to address the reviews, and re-submit their article. Finally, those experts receive the modified version of those articles and they decide to accept or reject the article. In all of these communication no author could know the identity of the reviewers and vise-versa. These communication occurs via an internal team formed by the journal committee. You need to build an automated system to facilitate this situation.

SOLUTION

Case	Pattern	Key force
(i) NuetFlix	Observer	users subscribe to genres; a release must be pushed to all subscribers of that genre
(ii) jit	Command	operations stored as objects, queued (*delay*) and later executed or discarded (*undo*)
(iii) BanglaPhone	Strategy	two interchangeable implementations of the same service, selected at run time
(iv) R2-D2	Mediator	authors and reviewers must not know each other; all talk through an internal team

(i) NuetFlix – Observer

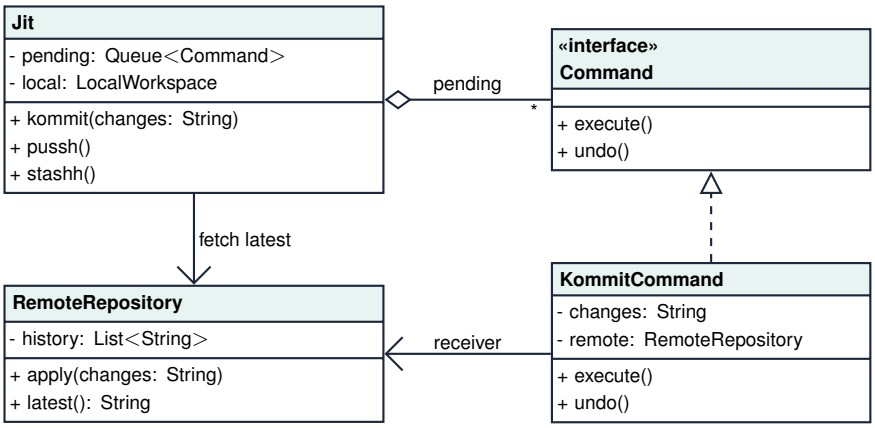
Each genre is a *subject* (publisher) and each user is an *observer* (subscriber). Choosing a preferred genre = `subscribe()`; a user may subscribe to one or more genres, i.e. register with several subjects. When a movie of genre g is released, `g.notifySubscribers(movie)` calls `update(movie)` only on users interested in g – a one-to-many dependency with automatic notification. Adding a fourth genre is just another `Genre` object; users can change preferences at run time.



(ii) jit – Command

Every *kommit* is turned into a stand-alone object that contains all information about the request (the changes, message, author). It is *not executed* on the remote immediately; it is stored in a local queue. *pussh* executes all queued commands on the remote repository in order; *stashh* discards (undoes) every queued, un-pushed command and reloads the remote version. This is exactly what the slides list as the benefits of **Command**: *"delay or queue a request's execution, and support undoable operations"*.

- Command = `Command` interface (`execute()`, `undo()`); concrete command = `KommitCommand`; receiver = `RemoteRepository`; invoker = `Jit` (holds the queue).




```
interface Command { void execute(); void undo(); }

class KommitCommand implements Command {             // concrete command
    private final String changes;
    private final RemoteRepository remote;            // receiver
    KommitCommand(String changes, RemoteRepository remote) {
        this.changes = changes; this.remote = remote;
    }
    public void execute() { remote.apply(changes); }  // sync this kommit
    public void undo()    { System.out.println("discarded: " + changes); }
}

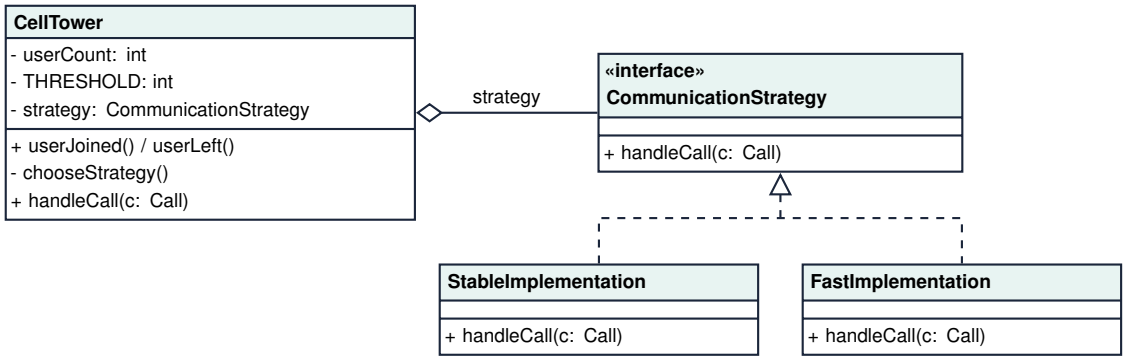
class Jit {                                           // invoker
```

```
    private final Deque<Command> pending = new ArrayDeque<>();
    private final RemoteRepository remote;
    private String local;
    Jit(RemoteRepository remote) { this.remote = remote; local = remote.latest(); }

    void kommit(String changes) {                    // saved locally, NOT executed
        pending.addLast(new KommitCommand(changes, remote));
    }
    void pussh() {                                   // execute queued commands in order
        while (!pending.isEmpty()) pending.pollFirst().execute();
        local = remote.latest();
    }
    void stashh() {
        while (!pending.isEmpty()) pending.pollLast().undo(); // discard un-pusshed
        local = remote.latest();                     // retrieve latest remote version
    }
}
```

(iii) BanglaPhone – Strategy

There are two *interchangeable algorithms* for the same job (mobile communication): "stable but slow and costly" and "fast and cheap but unstable above a threshold". The system must switch between them at run time *without hampering other code*. Encapsulating each implementation behind a common interface and letting the `CellTower` (context) hold and swap the current one is the **Strategy** pattern ("define a family of algorithms, encapsulate each one, make them interchangeable"). The rest of the code calls only `tower.handleCall()` and never knows which implementation runs.




```
interface CommunicationStrategy { void handleCall(Call c); }

class StableImplementation implements CommunicationStrategy {
    public void handleCall(Call c) { /* robust, high capacity, slower */ }
}
class FastImplementation implements CommunicationStrategy {
    public void handleCall(Call c) { /* fast, cheap, for low load */ }
}

class CellTower {                                    // context
    private static final int THRESHOLD = 500;
    private final CommunicationStrategy stable = new StableImplementation();
    private final CommunicationStrategy fast   = new FastImplementation();
    private CommunicationStrategy strategy = fast;
    private int userCount;

    void userJoined() { userCount++; chooseStrategy(); }
    void userLeft()   { userCount--; chooseStrategy(); }
```

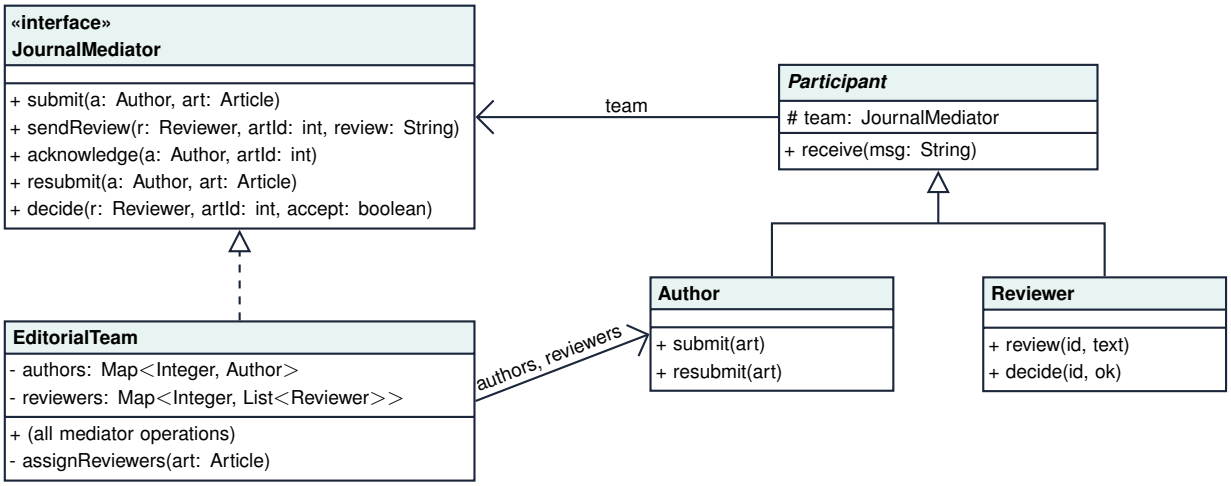
```
    private void chooseStrategy() { strategy = (userCount > THRESHOLD) ? stable : fast; }

    void handleCall(Call c) { strategy.handleCall(c); }  // other code is unaffected
}
```

NOTE Because the switch is driven by an internal condition of the tower (user count), an answer of *State* ("normal load" / "heavy load" states) is also defensible. Strategy is preferred here because the two classes are alternative *implementations of the same algorithm* that are unaware of each other, not modes with state-specific transitions.

(iv) R2-D2 journal – Mediator

Authors and reviewers interact in several rounds (submit → review → acknowledge → re-submit → decision), but *no author may know the identity of a reviewer and vice-versa*; all communication goes *via an internal team*. Forcing objects to collaborate only through a central object, so that they hold no references to each other, is the **Mediator** pattern ("restricts direct communications between objects and forces them to collaborate only via a mediator"). The editorial team (mediator) assigns reviewers, forwards anonymised reviews and revised versions, and relays the final decision.




```
class EditorialTeam implements JournalMediator {
    public void submit(Author a, Article art) {
        authors.put(art.id, a);
        for (Reviewer r : assignReviewers(art))
            r.receive("Please review article #" + art.id);        // author's name hidden
    }
    public void sendReview(Reviewer r, int artId, String review) {
        authors.get(artId).receive("Review for #" + artId + ": " + review); // reviewer hidden
    }
    // acknowledge(), resubmit(), decide() relay messages the same way
}
```

Answer. (i) Observer – genres are subjects, users subscribe to one or more genres and are notified of new releases. (ii) Command – each kommit is a queued command object; pussh executes the queue, stashh undoes/discards it. (iii) Strategy – two interchangeable communication implementations selected at run time by the cell tower according to the user count. (iv) Mediator – an editorial team object relays all author–reviewer communication, keeping both sides anonymous and decoupled.

2020–2021 • Q2(b) CSE307 • 19/10/2022 • D017.3 **MARKS 10**

There are three sorting routines implementing selection, insertion and quicksort algorithm. Demonstrate the use of strategy pattern for using these routines for sorting.

SOLUTION

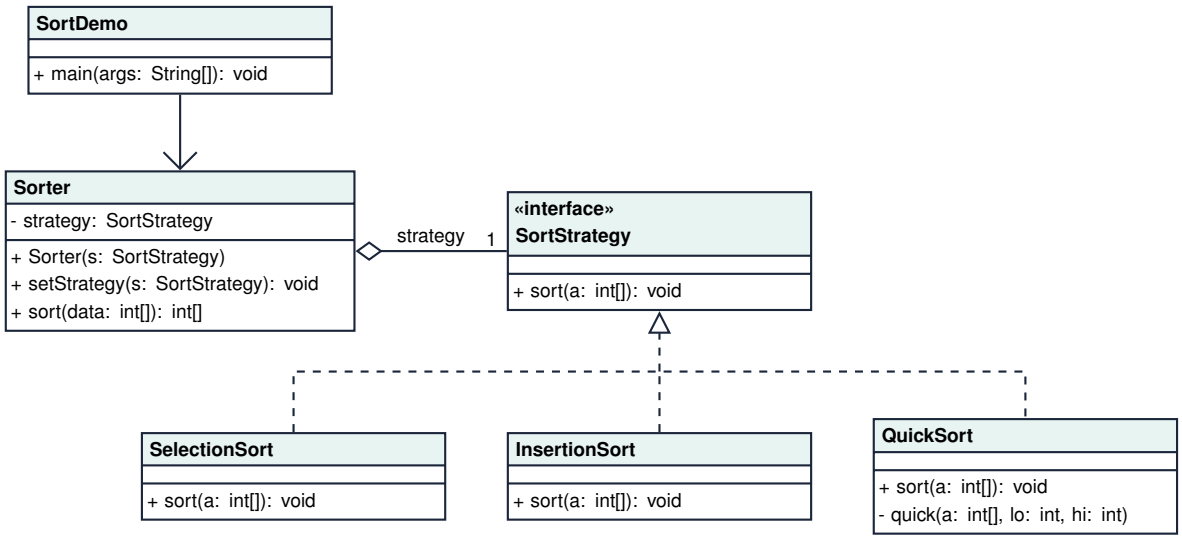
Concept: the Strategy pattern

The **Strategy** pattern (slides): *"Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it."* The three sorting routines are exactly such a family: they solve the same problem (put an array in order) with different algorithms, and the caller should be able to pick or swap any of them at run time without an `if/else` or `switch` on the algorithm name.

Mapping the participants

Pattern role	Class in this design
Strategy (common interface)	`SortStrategy` with the single operation `sort(int[])`
Concrete strategies	`SelectionSort`, `InsertionSort`, `QuickSort`
Context	`Sorter` – *has-a* `SortStrategy` and delegates the sorting to it
Client	`SortDemo` – configures the context with a strategy and may change it later

Class diagram



Implementation (Java)

```java
import java.util.Arrays;

interface SortStrategy {                        // Strategy
    void sort(int[] a);
}

class SelectionSort implements SortStrategy {  // Concrete strategy 1
    public void sort(int[] a) {
        for (int i = 0; i < a.length - 1; i++) {
```

```
            int min = i;
            for (int j = i + 1; j < a.length; j++)
                if (a[j] < a[min]) min = j;
            int t = a[i]; a[i] = a[min]; a[min] = t;
        }
    }
}

class InsertionSort implements SortStrategy {  // Concrete strategy 2
    public void sort(int[] a) {
        for (int i = 1; i < a.length; i++) {
            int key = a[i], j = i - 1;
            while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
            a[j + 1] = key;
        }
    }
}

class QuickSort implements SortStrategy {      // Concrete strategy 3
    public void sort(int[] a) { quick(a, 0, a.length - 1); }
    private void quick(int[] a, int lo, int hi) {
        if (lo >= hi) return;
        int pivot = a[hi], i = lo;
        for (int j = lo; j < hi; j++)
            if (a[j] < pivot) { int t = a[i]; a[i] = a[j]; a[j] = t; i++; }
        int t = a[i]; a[i] = a[hi]; a[hi] = t;
        quick(a, lo, i - 1);
        quick(a, i + 1, hi);
    }
}

class Sorter {                                  // Context
    private SortStrategy strategy;
    Sorter(SortStrategy strategy)          { this.strategy = strategy; }
    void setStrategy(SortStrategy strategy) { this.strategy = strategy; }
    int[] sort(int[] data) {
        int[] copy = data.clone();             // do not disturb caller's array
        strategy.sort(copy);                   // delegation to the strategy
        return copy;
    }
}

public class SortDemo {                        // Client
    public static void main(String[] args) {
        int[] data = {29, 3, 17, 8, 41, 1};
        Sorter sorter = new Sorter(new SelectionSort());
        System.out.println("Selection: " + Arrays.toString(sorter.sort(data)));

        sorter.setStrategy(new InsertionSort());     // swapped at run time
        System.out.println("Insertion: " + Arrays.toString(sorter.sort(data)));

        sorter.setStrategy(new QuickSort());
        System.out.println("Quick    : " + Arrays.toString(sorter.sort(data)));
    }
}
```

All three lines print `[1, 3, 8, 17, 29, 41]`; only the algorithm used differs.

Why this is a good design

- **Open–Closed:** a new routine (e.g. `MergeSort`) is added as one new class; `Sorter` and the client code are not modified.
- **Composition over inheritance / program to an interface:** `Sorter` depends only on `SortStrategy`, never on a concrete algorithm.
- The choice can even be data-driven at run time, e.g. `sorter.setStrategy(n < 20 ? new InsertionSort() : new QuickSort())`.

Answer. `SortStrategy` (strategy interface) is implemented by `SelectionSort`, `InsertionSort` and `QuickSort`; the context `Sorter` holds a `SortStrategy` reference and delegates `sort()` to it, so the client can plug in or switch the algorithm at run time.

2020–2021 • Q2(c) CSE307 • 19/10/2022 • D017.4 **MARKS 15**

There is a list of student IDs with their obtained marks. You are to find the corresponding grades of the students. You must prepare a merit list for the students who get the highest grade. Provide a suitable solution using a design pattern. Write necessary code to demonstrate the solution. You can assume any grading scheme in this problem.

SOLUTION

Analysing the problem

Two independent jobs exist:

(i) converting marks to a grade – this depends on a *grading scheme*, which the question explicitly leaves open ("you can assume any grading scheme"), i.e. it is the part that *varies*;

(ii) preparing the merit list (find the best grade, pick the students having it, order them by marks) – this logic is the same whatever the scheme.

Separating the varying algorithm (grading scheme) from the stable client logic and making schemes interchangeable is the intent of the **Strategy** pattern: *"define a family of algorithms, encapsulate each one, and make them interchangeable"*. The result processor is configured with a `GradingStrategy`; changing from the BUET letter-grade scheme to, say, a pass/fail or a relative scheme needs no change in the merit-list code.

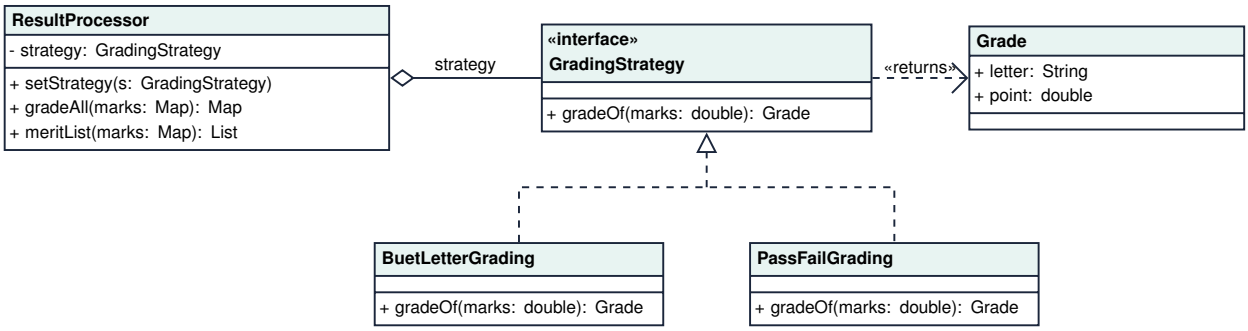
Participants

- **Strategy:** `GradingStrategy` – `gradeOf(marks): Grade`.
- **Concrete strategies:** `BuetLetterGrading` (assumed scheme below) and `PassFailGrading` (to show interchangeability).
- **Context:** `ResultProcessor` – computes grades and the merit list using the current strategy.
- **Value object:** `Grade` (letter + grade point, so grades can be compared).

Assumed grading scheme (BUET style)

Marks	≥ 80	75–79	70–74	65–69	60–64	55–59	50–54	45–49	40–44	< 40
Grade	A+	A	A−	B+	B	B−	C+	C	D	F
GP	4.00	3.75	3.50	3.25	3.00	2.75	2.50	2.25	2.00	0.00

Class diagram



Implementation (Java)

```java
import java.util.*;

final class Grade {                                  // value object
    final String letter; final double point;
    Grade(String letter, double point) { this.letter = letter; this.point = point; }
    public String toString() { return letter + " (" + point + ")"; }
}

interface GradingStrategy {                          // Strategy
    Grade gradeOf(double marks);
}

class BuetLetterGrading implements GradingStrategy {
    private static final double[] CUT   = {80, 75, 70, 65, 60, 55, 50, 45, 40};
    private static final String[] LET   = {"A+","A","A-","B+","B","B-","C+","C","D"};
    private static final double[] POINT = {4.0,3.75,3.5,3.25,3.0,2.75,2.5,2.25,2.0};
    public Grade gradeOf(double m) {
        for (int i = 0; i < CUT.length; i++)
            if (m >= CUT[i]) return new Grade(LET[i], POINT[i]);
        return new Grade("F", 0.0);
    }
}

class PassFailGrading implements GradingStrategy {
    public Grade gradeOf(double m) {
        return m >= 40 ? new Grade("P", 1.0) : new Grade("F", 0.0);
    }
}

class ResultProcessor {                              // Context
    private GradingStrategy strategy;
    ResultProcessor(GradingStrategy s) { strategy = s; }
    void setStrategy(GradingStrategy s) { strategy = s; }

    /** student ID -> grade */
    Map<String, Grade> gradeAll(Map<String, Double> marks) {
        Map<String, Grade> result = new LinkedHashMap<>();
        for (Map.Entry<String, Double> e : marks.entrySet())
            result.put(e.getKey(), strategy.gradeOf(e.getValue()));
        return result;
    }
```

```
    /** IDs of students having the highest grade, best marks first */
    List<String> meritList(Map<String, Double> marks) {
        Map<String, Grade> grades = gradeAll(marks);
        double best = 0;
        for (Grade g : grades.values()) best = Math.max(best, g.point);

        List<String> merit = new ArrayList<>();
        for (Map.Entry<String, Grade> e : grades.entrySet())
            if (e.getValue().point == best) merit.add(e.getKey());
        merit.sort((a, b) -> Double.compare(marks.get(b), marks.get(a)));
        return merit;
    }
}

public class MeritListDemo {                        // Client
    public static void main(String[] args) {
        Map<String, Double> marks = new LinkedHashMap<>();
        marks.put("2005001", 82.5);  marks.put("2005002", 67.0);
        marks.put("2005003", 91.0);  marks.put("2005004", 38.0);
        marks.put("2005005", 80.0);  marks.put("2005006", 74.5);

        ResultProcessor rp = new ResultProcessor(new BuetLetterGrading());
        rp.gradeAll(marks).forEach((id, g) -> System.out.println(id + " : " + g));
        System.out.println("Merit list (A+): " + rp.meritList(marks));
        // -> [2005003, 2005001, 2005005]

        rp.setStrategy(new PassFailGrading());     // another scheme, same client code
        System.out.println("Merit list (P) : " + rp.meritList(marks));
    }
}
```

Discussion

- `ResultProcessor` never contains grade cut-offs; adding a new scheme (relative grading, a different university's table, …) means writing one new `GradingStrategy` class – **OCP** is satisfied and the merit-list logic is reused untouched.
- The grading table is data inside one class, so a change of cut-offs touches only that class (**SRP**).

Answer. Use the Strategy pattern: `GradingStrategy` encapsulates the (assumed) grading scheme, `ResultProcessor` (context) uses it to grade every student and builds the merit list of those with the highest grade point, sorted by marks; the scheme can be replaced at run time.

2020–2021 • Q3(b) CSE307 • 19/10/2022 • D018.2 **MARKS 10**

Consider a dashboard that shows the maximum temperature and average temperature of the last one year. The temperature of the day is entered at the end of day. This triggers updating the temperature in the dashboard. You have to demonstrate the solution using appropriate software design pattern.

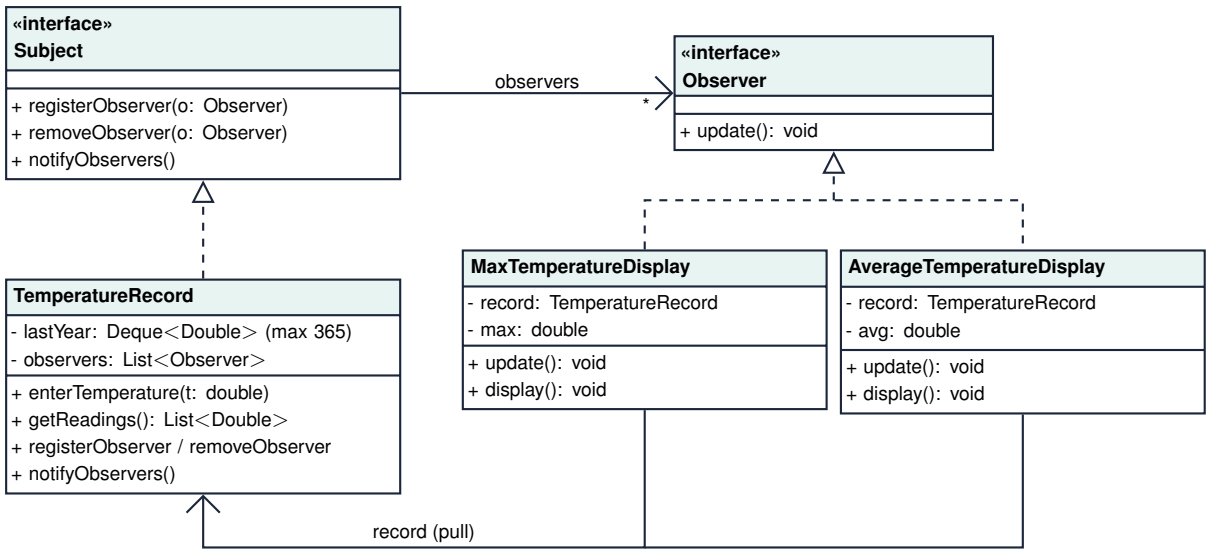
SOLUTION

Identifying the pattern

Entering a day's temperature changes the state of one object (the temperature record), and that change must *trigger* an automatic refresh of the dependent views (maximum and average panels of the dashboard). This is the weather-station example of the slides and the classic use of the **Observer** pattern: *one-to-many dependency, dependents notified and updated automatically*. New panels (minimum, trend chart, …) can be added or removed without changing the record.

Participants and class diagram

- **Subject:** `Subject` interface; **concrete subject:** `TemperatureRecord` – keeps the readings of the last 365 days; `enterTemperature()` is called at the end of each day.
- **Observer:** `Observer` interface; **concrete observers:** `MaxTemperatureDisplay` and `AverageTemperatureDisplay` (the two dashboard panels).



Implementation (Java)

```java
import java.util.*;

interface Subject  { void registerObserver(Observer o); void removeObserver(Observer o);
                     void notifyObservers(); }
interface Observer { void update(); }

class TemperatureRecord implements Subject {
    private static final int DAYS = 365;
    private final Deque<Double> lastYear = new ArrayDeque<>();
    private final List<Observer> observers = new ArrayList<>();

    public void registerObserver(Observer o) { observers.add(o); }
    public void removeObserver(Observer o)   { observers.remove(o); }
    public void notifyObservers() { for (Observer o : observers) o.update(); }

    /** called once at the end of each day */
    public void enterTemperature(double t) {
        if (lastYear.size() == DAYS) lastYear.removeFirst();   // keep a 1-year window
        lastYear.addLast(t);
        notifyObservers();                                      // triggers dashboard update
    }
    public List<Double> getReadings() { return new ArrayList<>(lastYear); }
```

```
}

class MaxTemperatureDisplay implements Observer {
    private final TemperatureRecord record;
    private double max;
    MaxTemperatureDisplay(TemperatureRecord r) { record = r; r.registerObserver(this); }
    public void update() {
        max = Collections.max(record.getReadings());            // pull the data
        display();
    }
    void display() { System.out.printf("Max temperature (last year): %.1f C%n", max); }
}

class AverageTemperatureDisplay implements Observer {
    private final TemperatureRecord record;
    private double avg;
    AverageTemperatureDisplay(TemperatureRecord r) { record = r; r.registerObserver(this); }
    public void update() {
        List<Double> d = record.getReadings();
        double sum = 0; for (double x : d) sum += x;
        avg = sum / d.size();
        display();
    }
    void display() { System.out.printf("Average temperature (last year): %.1f C%n", avg); }
}

public class DashboardDemo {
    public static void main(String[] args) {
        TemperatureRecord record = new TemperatureRecord();
        new MaxTemperatureDisplay(record);
        new AverageTemperatureDisplay(record);
        record.enterTemperature(31.5);   // end of day 1 -> both panels refresh
        record.enterTemperature(34.0);   // end of day 2 -> Max 34.0, Avg 32.8
    }
}
```

Remarks

- The data-entry side never calls the panels directly; it only changes the record, and the notification mechanism updates whatever panels are registered (loose coupling, OCP).
- For a long window, each display could maintain running values instead of recomputing; the pattern structure stays the same.

Answer. Observer pattern: `TemperatureRecord` (subject, 365-day window) calls `notifyObservers()` whenever the daily temperature is entered; the dashboard panels `MaxTemperatureDisplay` and `AverageTemperatureDisplay` (observers) pull the readings in `update()` and redisplay the maximum and the average.

Jan 2021 • Q1(b) CSE307 • 04/09/2021 • D019.2 MARKS 20

There are three servers serving sorting requests using selection, insertion and quicksort algorithms. There is a central broker that receives request of sorting by an algorithm with the data to be sorted. The central broker forwards the request to the appropriate server and returns the result to the class requesting for sorting service after getting the sorted array from the server. The central

broker maintains a queue if more than one request arrives at a time. Demonstrate the use of mediator pattern for this system.

SOLUTION

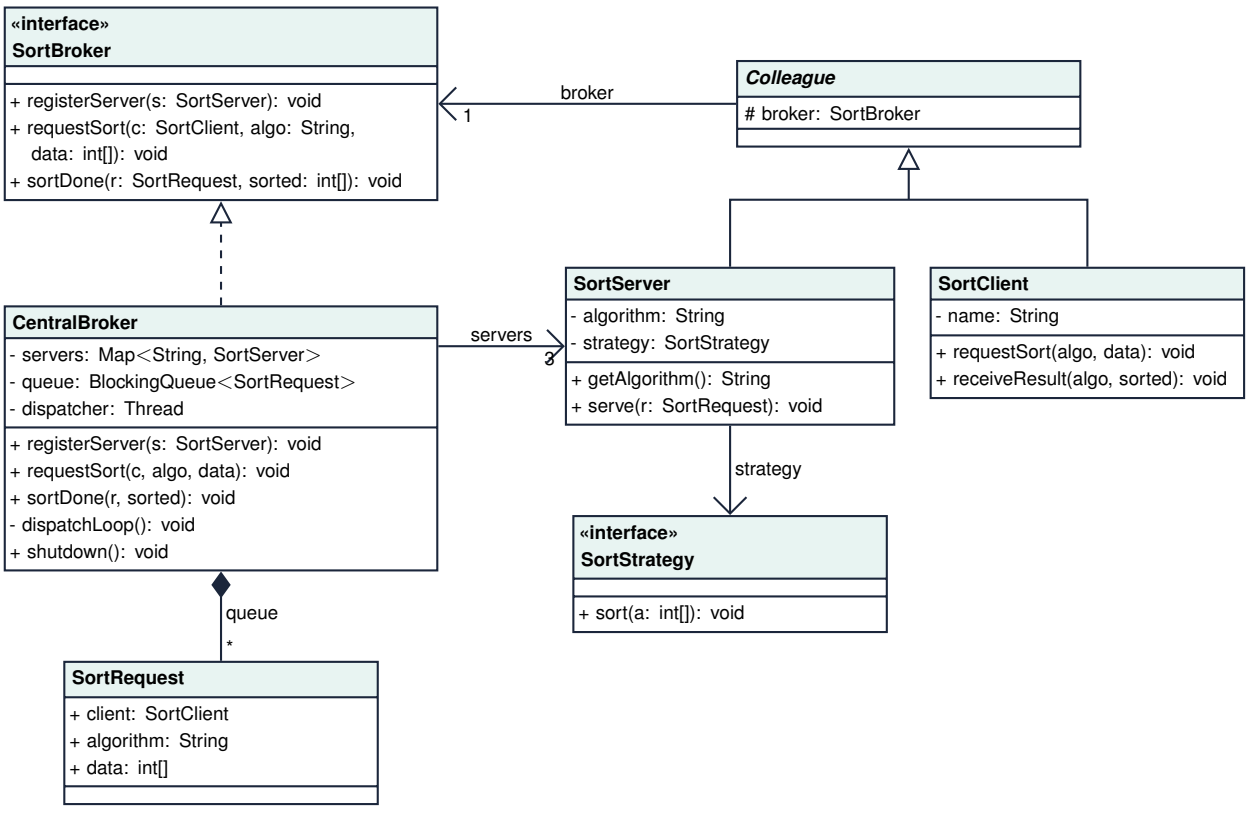
Concept and why Mediator fits

Mediator: *"restricts direct communications between objects and forces them to collaborate only via a mediator."* Here two groups of objects must cooperate – requesting clients and sorting servers – but a client should not know which server object implements which algorithm, where it is, or whether it is busy; and a server should not know who asked. The *central broker* is the mediator: clients and servers are its colleagues, each knowing only the broker. The broker also owns the cross-cutting policy (routing by algorithm name and FIFO queueing of simultaneous requests).

Participants

Role	Class
Mediator	`SortBroker` interface: `registerServer`, `requestSort`, `sortDone`
Concrete mediator	`CentralBroker`: map algorithm → server, a `BlockingQueue` of requests, a dispatcher thread
Colleague (abstract)	`Colleague`: holds the `SortBroker` reference
Concrete colleagues	`SortClient` (requester), `SortServer` (three instances: selection, insertion, quick)
Helper	`SortRequest` (client, algorithm, data); `SortStrategy` implementations used inside the servers

Class diagram



Interaction for one request (client and server never reference each other):

(i) `SortClient.requestSort(algo, data)` calls `broker.requestSort(this, algo, data);`

(ii) the broker wraps it in a `SortRequest` and appends it to its FIFO queue;

(iii) the dispatcher takes the oldest request and calls `serve(r)` on the server registered for `algo`;

(iv) the server sorts a copy and calls `broker.sortDone(r, sorted);`

(v) the broker returns the array with `r.client.receiveResult(algo, sorted)`.

Implementation (Java)

The three algorithm classes `SelectionSort`, `InsertionSort` and `QuickSort` (implementing `SortStrategy{ void sort(int[] a); }`) are exactly those written for 2020–2021 Q2(b) (D017.3), so they are not repeated; each server simply wraps one of them.

```
import java.util.*;
import java.util.concurrent.*;

interface SortBroker {                                      // Mediator
    void registerServer(SortServer s);
    void requestSort(SortClient c, String algorithm, int[] data);
    void sortDone(SortRequest r, int[] sorted);
}

class SortRequest {
    final SortClient client; final String algorithm; final int[] data;
    SortRequest(SortClient c, String a, int[] d) { client = c; algorithm = a; data = d; }
}

abstract class Colleague {                                  // knows only the mediator
    protected final SortBroker broker;
    Colleague(SortBroker broker) { this.broker = broker; }
}

class SortServer extends Colleague {
    private final String algorithm; private final SortStrategy strategy;
    SortServer(SortBroker b, String algorithm, SortStrategy strategy) {
        super(b); this.algorithm = algorithm; this.strategy = strategy;
        b.registerServer(this);
    }
    String getAlgorithm() { return algorithm; }
    void serve(SortRequest r) {
        int[] copy = r.data.clone();
        strategy.sort(copy);
        broker.sortDone(r, copy);                           // result goes back via broker
    }
}

class SortClient extends Colleague {
    private final String name;
    SortClient(SortBroker b, String name) { super(b); this.name = name; }
    String getName() { return name; }
    void requestSort(String algorithm, int[] data) { broker.requestSort(this, algorithm, data); }

    void receiveResult(String algorithm, int[] sorted) {
        System.out.println(name + " got (" + algorithm + "): " + Arrays.toString(sorted));
    }
```

```
}

class CentralBroker implements SortBroker {                  // Concrete mediator
    private static final SortRequest STOP = new SortRequest(null, null, null);
    private final Map<String, SortServer> servers = new HashMap<>();
    private final BlockingQueue<SortRequest> queue = new LinkedBlockingQueue<>();
    private final Thread dispatcher = new Thread(this::dispatchLoop);

    CentralBroker() { dispatcher.start(); }

    public synchronized void registerServer(SortServer s) { servers.put(s.getAlgorithm(), s); }

    public void requestSort(SortClient c, String algorithm, int[] data) {
        queue.add(new SortRequest(c, algorithm, data));      // waits in FIFO queue
        System.out.println("broker: queued " + algorithm + " for " + c.getName());
    }

    private void dispatchLoop() {                            // one request at a time
        try {
            while (true) {
                SortRequest r = queue.take();
                if (r == STOP) return;
                SortServer s;
                synchronized (this) { s = servers.get(r.algorithm); }
                if (s != null) s.serve(r);                   // forward to the right server
                else System.out.println("broker: no server for " + r.algorithm);
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    public void sortDone(SortRequest r, int[] sorted) {
        r.client.receiveResult(r.algorithm, sorted);         // return to the requester
    }

    void shutdown() throws InterruptedException { queue.add(STOP); dispatcher.join(); }
}

public class BrokerDemo {
    public static void main(String[] args) throws InterruptedException {
        CentralBroker broker = new CentralBroker();
        new SortServer(broker, "selection", new SelectionSort());
        new SortServer(broker, "insertion", new InsertionSort());
        new SortServer(broker, "quick",     new QuickSort());

        SortClient a = new SortClient(broker, "ClientA");
        SortClient b = new SortClient(broker, "ClientB");

        // several requests arrive "at a time" -> they are queued
        Thread t1 = new Thread(() -> a.requestSort("quick",     new int[]{9, 4, 7, 1}));
        Thread t2 = new Thread(() -> b.requestSort("selection", new int[]{5, 3, 8}));
        Thread t3 = new Thread(() -> a.requestSort("insertion", new int[]{2, 6, 0}));
        t1.start(); t2.start(); t3.start();
        t1.join();  t2.join();  t3.join();

        broker.shutdown();                                   // drain queue, stop dispatcher
    }
}
```

Why this is the Mediator pattern (and its benefits)

- Clients and servers are **colleagues** that communicate only through `SortBroker`; a client never holds a server reference and vice versa ("reduce chaotic dependencies").
- Routing and queueing policy are centralised in `CentralBroker`: adding a fourth server (e.g. merge sort) is one `new SortServer(broker, "merge", new MergeSort())` – no client changes (OCP).
- The `BlockingQueue` serialises simultaneous requests in arrival (FIFO) order, as required.

Answer. `CentralBroker` is the concrete mediator: it receives `requestSort()` from client colleagues, queues them, forwards each to the `SortServer` colleague registered for the requested algorithm, receives the sorted array in `sortDone()` and returns it to the requesting client – clients and servers never talk directly.

2018–2019 • Q2(a) CSE307 • 06/11/2019 • D020 MARKS 20

An application needs to be developed to simulate and study robots' interaction. You have the following classes:
`IBehaviour` – an interface that defines the behavior of a robot
`AggressiveBehaviour`, `DefensiveBehaviour`, `NormalBehaviour` – each of them defines a specific behavior.
`Robot` – It keeps or gets information such as position, close obstacles, etc., and passes necessary information to the appropriate class.
In order to select an action, the Behaviour classes need information that is passed from robot sensors like position, close obstacles, etc.
In the main section of the application the several robots are created and several different behaviors are created. Each robot has a different behavior assigned: 'Big Robot' is an aggressive one and attacks any other robot found, 'George v.2.1' is really scared and run away in the opposite direction when it encounter another robot and 'R2' is pretty calm and ignore any other robot. At some point the behaviors are changed for each robot.
Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfils all the requirements.

SOLUTION

Identifying the pattern

Every robot does the same thing at a high level – it reads its sensors and moves – but *how* it decides the move is a family of interchangeable algorithms (aggressive, defensive, normal), and the algorithm assigned to a robot *changes at run time* ("at some point the behaviors are changed for each robot"). Encapsulating each behaviour behind a common interface and letting the robot hold a reference to one of them is the **Strategy** pattern – the same idea as the `FlyBehavior`/`QuackBehavior` of the Duck simulator in the slides (*"encapsulate what varies"*, *"favour composition over inheritance"*). If behaviour were coded with subclasses (`AggressiveRobot`, `ScaredRobot`, ...) a robot could never change behaviour without being re-created, and every new behaviour would multiply the class hierarchy.

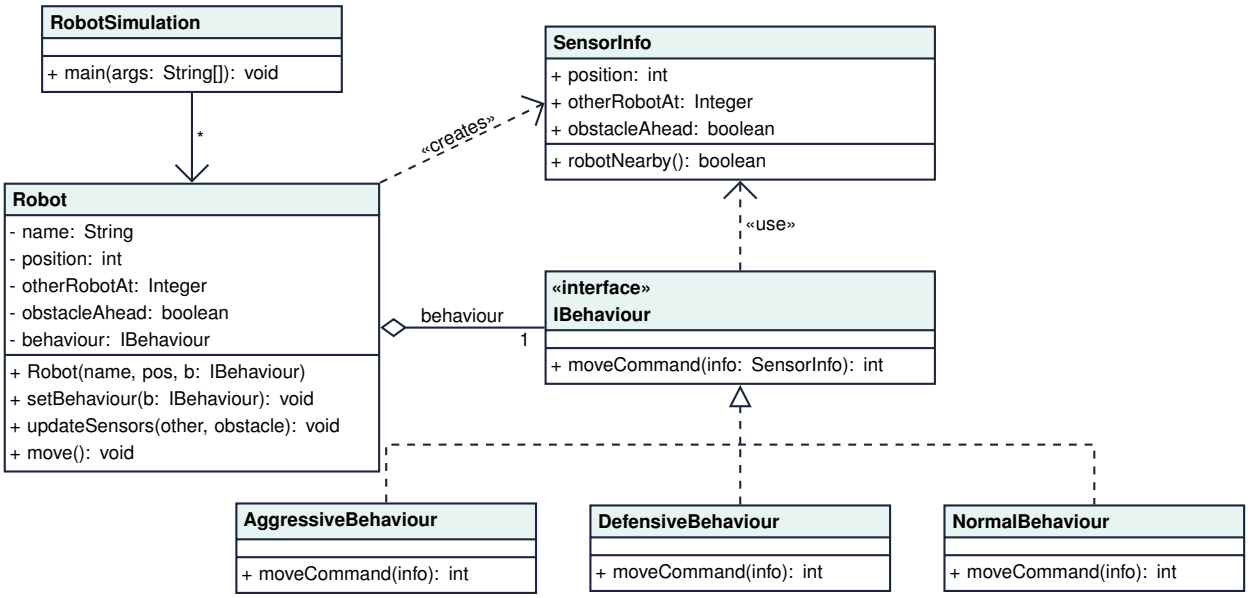
Participants

- **Strategy:** `IBehaviour` – `moveCommand(SensorInfo): int`.
- **Concrete strategies:** `AggressiveBehaviour` (move towards and attack a robot),

`DefensiveBehaviour` (run in the opposite direction), `NormalBehaviour` (ignore robots).

- **Context:** `Robot` – keeps position and sensor readings, packs them into a `SensorInfo` object and passes it to its current behaviour; exposes `setBehaviour()` for run-time change.
- **Client:** `RobotSimulation.main()` – creates robots and behaviours and later swaps them.

Class diagram



Implementation (Java)

```java
/** Data read from the robot's sensors and passed to a behaviour. */
class SensorInfo {
    final int position;          // robot's own position (1-D track for simplicity)
    final Integer otherRobotAt;  // position of the nearest robot, null if none seen
    final boolean obstacleAhead; // a close obstacle in front
    SensorInfo(int position, Integer otherRobotAt, boolean obstacleAhead) {
        this.position = position; this.otherRobotAt = otherRobotAt;
        this.obstacleAhead = obstacleAhead;
    }
    boolean robotNearby() { return otherRobotAt != null; }
}

interface IBehaviour {                  // Strategy
    /** @return displacement: + forward, - backward, 0 stay */
    int moveCommand(SensorInfo info);
}

class AggressiveBehaviour implements IBehaviour {
    public int moveCommand(SensorInfo s) {
        if (s.robotNearby()) {
            System.out.println("   [aggressive] robot found -> ATTACK!");
            return 2 * Integer.signum(s.otherRobotAt - s.position);  // charge at it
        }
        return s.obstacleAhead ? 0 : 1;                              // patrol
    }
}
```

```
class DefensiveBehaviour implements IBehaviour {
    public int moveCommand(SensorInfo s) {
        if (s.robotNearby()) {
            System.out.println("   [defensive] robot found -> run away!");
            return -3 * Integer.signum(s.otherRobotAt - s.position); // opposite direction
        }
        return 0;                                                    // hide and wait
    }
}

class NormalBehaviour implements IBehaviour {
    public int moveCommand(SensorInfo s) {
        System.out.println("   [normal] ignoring other robots");
        return s.obstacleAhead ? 0 : 1;                              // just walk on
    }
}

class Robot {                           // Context
    private final String name;
    private int position;
    private Integer otherRobotAt;       // latest sensor readings
    private boolean obstacleAhead;
    private IBehaviour behaviour;       // has-a strategy

    Robot(String name, int position, IBehaviour behaviour) {
        this.name = name; this.position = position; this.behaviour = behaviour;
    }
    void setBehaviour(IBehaviour behaviour) { this.behaviour = behaviour; }

    void updateSensors(Integer otherRobotAt, boolean obstacleAhead) {
        this.otherRobotAt = otherRobotAt; this.obstacleAhead = obstacleAhead;
    }

    void move() {
        System.out.println(name + " at " + position + ":");
        SensorInfo info = new SensorInfo(position, otherRobotAt, obstacleAhead);
        int step = behaviour.moveCommand(info);   // behaviour decides
        position += step;
        System.out.println("   moved " + step + " -> now at " + position);
    }
}

public class RobotSimulation {          // Client
    public static void main(String[] args) {
        IBehaviour aggressive = new AggressiveBehaviour();
        IBehaviour defensive  = new DefensiveBehaviour();
        IBehaviour normal     = new NormalBehaviour();

        Robot big    = new Robot("Big Robot",    0,  aggressive);
        Robot george = new Robot("George v.2.1", 10, defensive);
        Robot r2     = new Robot("R2",           20, normal);

        // all three robots sense each other
        big.updateSensors(10, false);
        george.updateSensors(0, false);
        r2.updateSensors(10, false);
        big.move(); george.move(); r2.move();
```

```
        System.out.println("--- behaviours are changed ---");
        big.setBehaviour(defensive);
        george.setBehaviour(normal);
        r2.setBehaviour(aggressive);
        big.move(); george.move(); r2.move();
    }
}
```

How the requirements are met

- `Robot` only *gets and passes* sensor information (`SensorInfo`); the decision is made by the behaviour object – single responsibility for each class.
- 'Big Robot', 'George v.2.1' and 'R2' receive different strategies in `main`; the call `setBehaviour()` changes them at run time without re-creating the robots.
- A new behaviour (e.g. `CuriousBehaviour`) is one new class implementing `IBehaviour`; `Robot` is closed for modification (OCP).
- Behaviour objects are stateless, so one instance can be shared by many robots.

Answer. Strategy pattern: `IBehaviour` is the strategy, the three behaviour classes are concrete strategies, `Robot` is the context holding an `IBehaviour` reference and delegating `move()` decisions to it via `moveCommand(SensorInfo)`; `setBehaviour()` swaps the behaviour at run time.

2010–2011 • Q4(c) CSE307 • 19/03/2012 • D039 **MARKS 12**

In principle a social network service focuses on building online communities of people who share interests and/or activities, or who are interested in exploring the interests and activities of others. Facebook support groups that people can join. Each group has a title, administrative members, a group type (open/ closed), and a list of related groups. Otherwise, a group operates just like an ordinary page. If somebody writes on the wall page of the group, the information is broadcasted to all the members and it is visualized in the news feeds of the members.
Which design pattern is the most appropriate to handle this basic functionality of such a Facebook group? Users should be able to join a group as well as leave a group if they get bored. Once a user has joined a group it will automatically receive any updates that are published on the wall. Give a short explanation. In particular, show a complete class diagram(s) and enough code fragments to illustrate your use of the pattern to solve the problem.

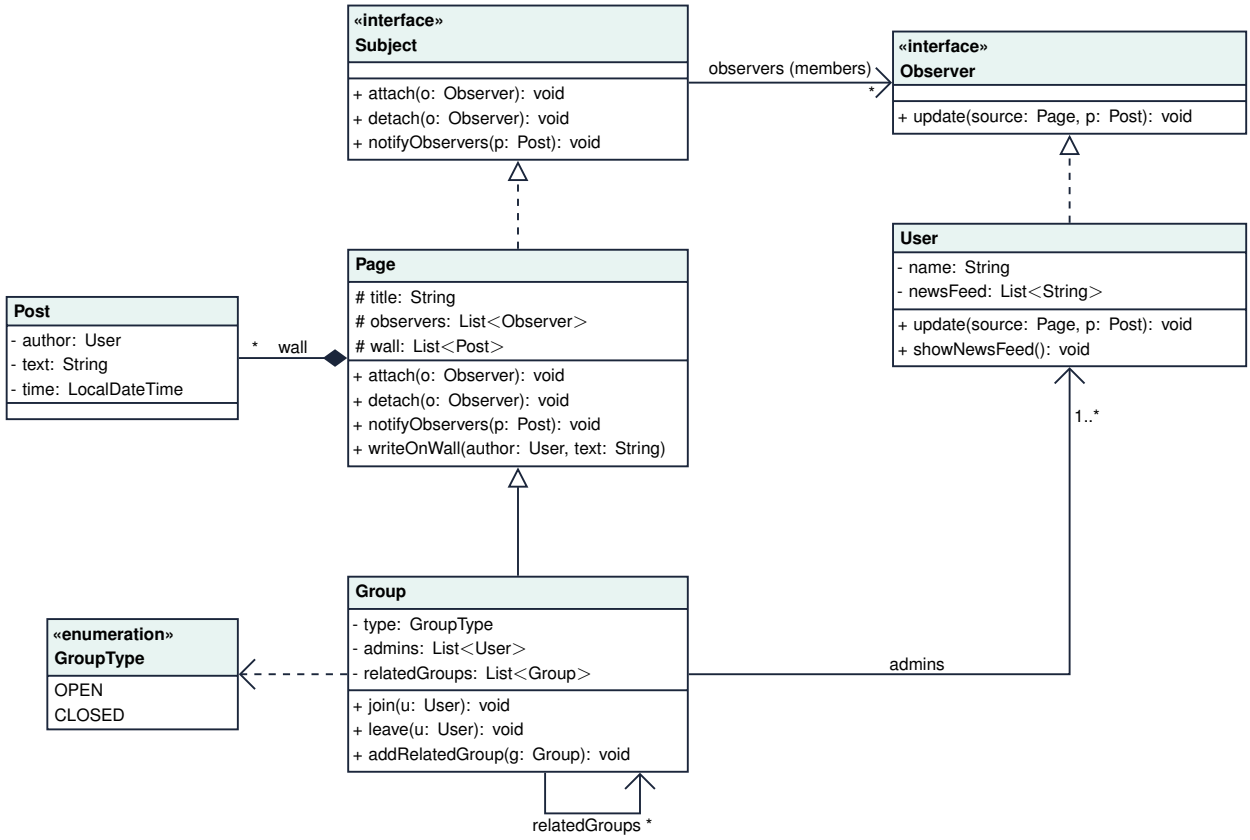
SOLUTION

Pattern and short explanation

The most appropriate pattern is **Observer**. A group (the *subject*/publisher) changes state whenever somebody writes on its wall; every member (*observer*/subscriber) must be informed automatically so that the post appears in his/her news feed. The relationship is one-to-many and *dynamic*: joining a group = `attach()` (subscribe), leaving = `detach()` (unsubscribe). The group does not need to know what a member does with the post (show it in the feed, send an email, ...) – it only calls `update()` on the `Observer` interface, so groups and members stay loosely coupled.
Because "a group operates just like an ordinary page", the subject behaviour (wall + followers + broadcast) is put in a `Page` class and `Group` *inherits* it, adding title, admins, open/closed type and

related groups.

Class diagram



Code fragments (Java)

```
import java.time.LocalDateTime;
import java.util.*;

interface Subject  { void attach(Observer o); void detach(Observer o); void notifyObservers(Post
     p); }
interface Observer { void update(Page source, Post post); }
enum GroupType { OPEN, CLOSED }

class Post {
    final User author; final String text; final LocalDateTime time = LocalDateTime.now();
    Post(User author, String text) { this.author = author; this.text = text; }
}

class Page implements Subject {                        // an ordinary page = subject
    protected final String title;
    protected final List<Observer> observers = new ArrayList<>();
    protected final List<Post> wall = new ArrayList<>();
    Page(String title) { this.title = title; }

    public void attach(Observer o) { if (!observers.contains(o)) observers.add(o); }
    public void detach(Observer o) { observers.remove(o); }
    public void notifyObservers(Post p) { for (Observer o : observers) o.update(this, p); }
```

```
    public void writeOnWall(User author, String text) {
        Post p = new Post(author, text);
        wall.add(p);
        notifyObservers(p);                          // broadcast to all members
    }
    String getTitle() { return title; }
}

class Group extends Page {                           // operates like a page
    private final GroupType type;
    private final List<User> admins = new ArrayList<>();
    private final List<Group> relatedGroups = new ArrayList<>();
    private final Set<User> pending = new LinkedHashSet<>();

    Group(String title, GroupType type, User creator) {
        super(title); this.type = type;
        admins.add(creator); attach(creator);
    }
    public void join(User u) {
        if (type == GroupType.OPEN) attach(u);       // subscribe immediately
        else pending.add(u);                         // CLOSED: needs admin approval
    }
    public void approve(User admin, User u) {
        if (admins.contains(admin) && pending.remove(u)) attach(u);
    }
    public void leave(User u) { detach(u); }         // unsubscribe ("got bored")
    public void addRelatedGroup(Group g) { relatedGroups.add(g); }
}

class User implements Observer {                     // member = observer
    private final String name;
    private final List<String> newsFeed = new ArrayList<>();
    User(String name) { this.name = name; }

    public void update(Page source, Post p) {        // shown in the news feed
        newsFeed.add("[" + source.getTitle() + "] " + p.author.name + ": " + p.text);
    }
    void showNewsFeed() { System.out.println(name + " feed: " + newsFeed); }
}

public class FacebookDemo {
    public static void main(String[] args) {
        User ana = new User("Ana"), ben = new User("Ben"), cat = new User("Cat");
        Group se = new Group("CSE 213 Study Group", GroupType.OPEN, ana);
        se.join(ben); se.join(cat);
        se.writeOnWall(ben, "Observer pattern notes uploaded"); // Ana, Ben, Cat notified
        se.leave(cat);                                          // Cat got bored
        se.writeOnWall(ana, "Quiz on Sunday");                  // only Ana and Ben notified
        ana.showNewsFeed(); cat.showNewsFeed();
    }
}
```

How the requirements map to the pattern

Requirement	Realisation
join a group	`Group.join()` → `attach()` (subscribe); closed groups wait for admin approval
leave a group	`Group.leave()` → `detach()` (unsubscribe)
write on the wall	`Page.writeOnWall()` stores the `Post` and calls `notifyObservers()`
broadcast to members	`notifyObservers()` calls `update()` on every attached observer
visible in news feed	`User.update()` appends the post to the member's `newsFeed`
title, admins, type, related groups	attributes of `Group`; behaviour otherwise inherited from `Page`

Answer. Observer pattern: `Page`/`Group` is the subject (wall + list of observers), `User` is the observer; joining/leaving = attach/detach, and every wall post is pushed by `notifyObservers()` to `update()` of all current members, which put it in their news feeds.

05

Class Diagram & State Diagram

2023–2024 • Q2(a) CSE325 • 17/05/2026 • C325-2324-2a **MARKS 15**

In an **Online Food Delivery Order System**, when an order is placed, the order enters pending state. If payment succeeds, the order is confirmed. If payment fails 3 times, the order is cancelled. After confirmation, the restaurant prepares the food and at the same time, the system assigns a delivery rider. Once both food is ready and rider is assigned, the order is out for delivery. If delivery is not completed within 60 minutes, the system marks the order as escalated. If delivery succeeds, it is recorded.
Design a State Diagram for the scenario.

SOLUTION

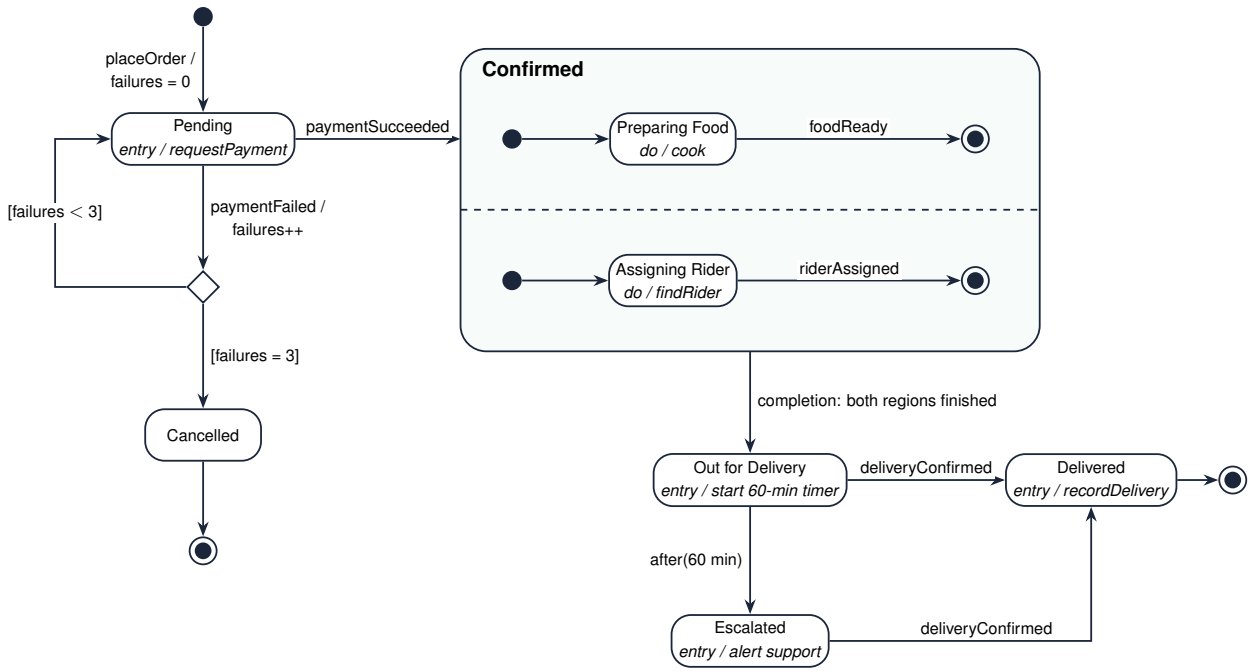
What the state diagram models

A state (machine) diagram describes the life cycle of *one* object – here an `Order`. It shows the **states** the order can be in, the **transitions** between them written as `event [guard] / action`, **entry / do** activities inside states, pseudo-states (initial, choice, final), and – important for this question – **concurrency** through a composite state with *orthogonal regions* (equivalently a fork / join pair).

Step 1 – states and events extracted from the text

Text	Modelling element
order placed → pending	initial transition `placeOrder / failures = 0` into **Pending**
payment succeeds → confirmed	`paymentSucceeded`: Pending → **Confirmed**
payment fails 3 times → cancelled	`paymentFailed / failures++` into a **choice**: `[failures < 3]` back to Pending (retry), `[failures = 3]` → **Cancelled**
restaurant prepares food *and at the same time* rider assigned	composite state Confirmed with two **orthogonal regions**: Preparing Food ‖ Assigning Rider
once both are done → out for delivery	completion transition fires only when *both* regions reach their final states → **Out for Delivery**
not completed within 60 min → escalated	time event `after(60 min)` → **Escalated**
delivery succeeds → recorded	`deliveryConfirmed` → **Delivered** (`entry / recordDelivery`)

Step 2 – the state diagram



Step 3 – modelling decisions explained

- **Counting the payment attempts.** A state diagram has no loop construct, so the attempts are counted by an *action* (`failures++`) and tested by **guards** on a **choice pseudo-state**. Returning to Pending re-runs its `entry / requestPayment`, i.e. the retry. The third failure (`failures = 3`) leads to Cancelled and the life cycle ends.
- **"At the same time" = orthogonal regions.** Confirmed is a composite state split by a dashed line into two regions that are active *simultaneously*: the restaurant's food preparation and the system's rider assignment. Each region has its own initial and final state.
- **"Once both" = completion transition.** The unlabelled (completion) transition leaving Confirmed fires only when *every* region has reached its final state – exactly the synchronisation "food ready *and* rider assigned". (The same could be drawn with a fork bar after Confirmed and a join bar before Out for Delivery.)
- **Time event.** `after(60 min)` is a relative time event measured from entry into Out for Delivery, so the timer is started in the entry action. If `deliveryConfirmed` arrives first, the order goes straight to Delivered and the timer is discarded.
- **Escalated is not a dead end.** An escalated order can still be delivered later, so `deliveryConfirmed` also leads from Escalated to Delivered, where the delivery is recorded (`entry / recordDelivery`).

> **NOTE** The text does not say what happens after escalation; it is assumed that the order is still delivered eventually (support is alerted). If escalation should instead end the order, Escalated would simply get a transition to a final state.

Answer. Initial → Pending; `paymentFailed / failures++` → choice: `[failures<3]` back to Pending, `[failures=3]` → Cancelled → final; `paymentSucceeded` → composite Confirmed with concurrent regions Preparing Food | Assigning Rider; completion (both done) → Out for Delivery; `after(60 min)` → Escalated; `deliveryConfirmed` (from Out for Delivery or Escalated) → Delivered (recorded) → final.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q6(b) CSE325 • 17/05/2026 • C325-2324-6b MARKS 5

In a class diagram, we demonstrate various relationships among classes and interfaces. Briefly show the three relationships with appropriate examples: (1) aggregation, (2) composition, and (3) dependency.

2018–2019 • Q4(c) CSE307 • 06/11/2019 • R009 MARKS 8

Show example class diagram of Aggregation and Composition in the context of a video rental store.

2017–2018 • Q8(c) CSE307 • 29/09/2018 • R013 MARKS 6

Show example diagrams of real-world scenario to present Aggregation and Composition in the context of Class Diagram.

SOLUTION

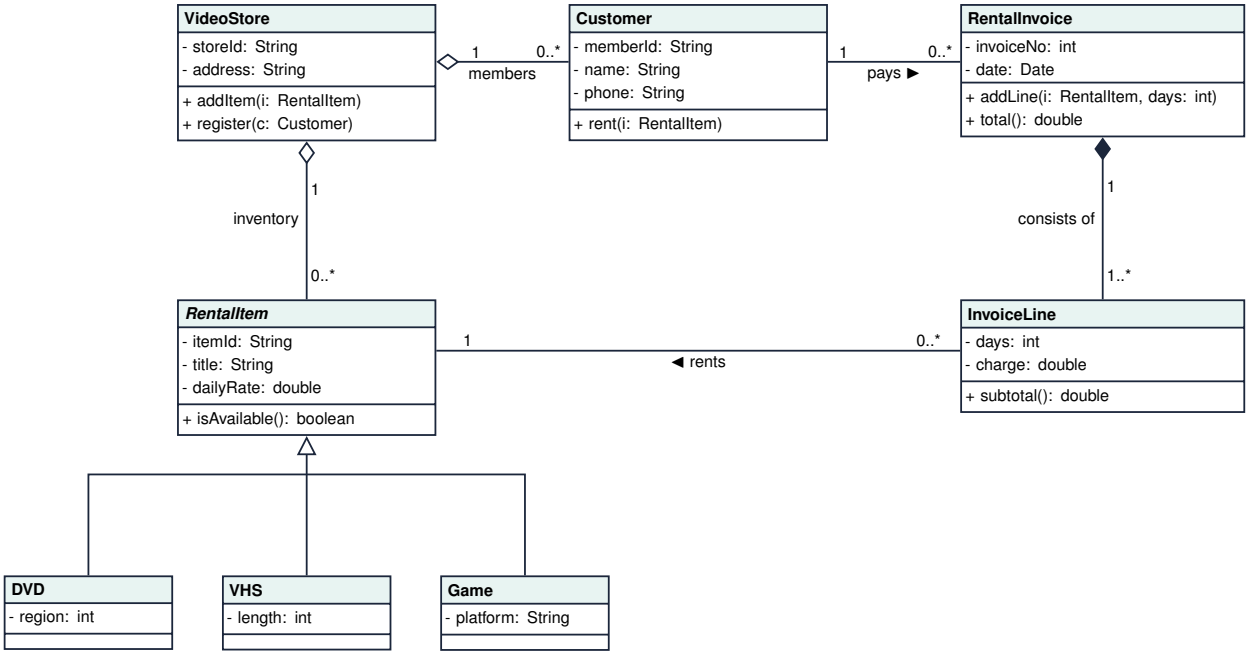
Concept (as defined in the lecture)

Both aggregation and composition are special kinds of **association** that express a *whole–part* relationship.

- **Aggregation** – "*is part of*". Drawn with a **hollow (white) diamond** at the *whole* end. The part can exist independently of the whole and may even be shared by several wholes; destroying the whole does *not* destroy the part.
- **Composition** – "*is entirely made of*". A **stronger** form of aggregation, drawn with a **filled (black) diamond** at the whole end. The parts *live and die with the whole*: a part belongs to exactly one whole at a time (multiplicity at the diamond end is always `1` or `0..1`) and is deleted when the whole is deleted.

Real-world scenario: a video rental store

- A **VideoStore** keeps an inventory of **RentalItem**s (DVDs, VHS tapes, games). If the store branch closes, the DVDs still exist – they can be moved to another branch or sold. ⇒ **aggregation**.
- The store also has registered **Customer**s (members). A customer exists as a person even if the store deletes the membership list ⇒ **aggregation**.
- A **RentalInvoice** is made up of **InvoiceLine**s (one line per item rented: rental days, charge). A line has no meaning outside its invoice and is deleted when the invoice is deleted ⇒ **composition**.
- Every line refers to the physical **RentalItem** rented – an ordinary association (the DVD does not die when an invoice is cancelled).



Reading the diagram

Relationship	Kind	Multiplicity	Why
VideoStore – RentalItem	aggregation (◇)	1 – 0..*	items outlive the store; can be transferred
VideoStore – Customer	aggregation (◇)	1 – 0..*	a customer is a person independent of the store
RentalInvoice – InvoiceLine	composition (⧫)	1 – 1..*	lines are created with and deleted with the invoice
InvoiceLine – RentalItem	association	0..* – 1	a line merely *refers* to the item rented
RentalItem ◁ DVD/VHS/Game	generalization	–	the slide's "abstract class" example

The same difference in code

```
class VideoStore {                          // AGGREGATION: parts are passed in
    private final List<RentalItem> inventory = new ArrayList<>();
    void addItem(RentalItem i) { inventory.add(i); }   // created elsewhere
}                                           // store gone -> items still referenced elsewhere

class RentalInvoice {                       // COMPOSITION: whole creates & owns its parts
    private final List<InvoiceLine> lines = new ArrayList<>();
    void addLine(RentalItem item, int days) {
        lines.add(new InvoiceLine(item, days));          // created INSIDE the whole
    }
}                                           // invoice garbage-collected -> lines die with it
```

EXAM TIP The diamond is always drawn at the *whole* end. In composition the multiplicity at the diamond end can never exceed 1 – a part cannot belong to two wholes.

Other quick textbook examples

Aggregation: a `Team` has `Players` (`Team` ◇– `Player`, 0..1 – 0..*); if the team is dissolved the players still exist and can join another team. Composition: a `House` is composed of `Rooms` (`House` ⧫– `Room`, 1 – 1..*); a room does not exist without its house.

Additionally for 2023–2024 Q6(b) (CSE325): Dependency – "uses", temporary

A dependency means that a change in one class (the supplier) may affect another (the client), because the client *uses* it only temporarily – as a method parameter, local variable or return type – without keeping a reference as an attribute. Notation: **dashed arrow** from client to supplier. *Example:* `OrderService.checkout(PaymentGateway g)` uses a gateway passed as a parameter.




```
class OrderService {                          // dependency
    void checkout(Order o, PaymentGateway g) { g.pay(o.total()); } // used, not stored
}
```

	Aggregation	Composition	Dependency
Symbol	hollow diamond (whole)	filled diamond (whole)	dashed arrow
Part's life cycle	independent	ends with the whole	not a whole–part relation
Sharing	part may be shared	part has one owner	–
Held as attribute?	yes	yes	no (parameter / local)

Answer. Aggregation (hollow diamond): `VideoStore` ◇– `RentalItem/Customer` – parts exist independently. Composition (filled diamond): `RentalInvoice` ◆– `InvoiceLine` – lines live and die with the invoice. Dependency (dashed arrow, 2023–2024 Q6(b)): `OrderService` --› `PaymentGateway` – temporary use through a parameter / local variable.

2022–2023 • Q4(b) CSE307 • 05/05/2024 • R003 **MARKS 12**

Suppose we are designing an Accounting Software. We have a class named Account to represent the accounting heads in the chart of accounts. In each transaction one account is debited and at the same time another account is credited. The transaction is made from a user account authenticated using two-step authentication process (password, OTP). There are classes called Debtor and Creditor which stores the information which account is debited and which account is credited. An account may have children. The account that does not have any child is called leaf account. An account that has any transaction associated with it cannot be deleted. Also an account which has child cannot be deleted. Design a class diagram to implement the scenario described above.

SOLUTION

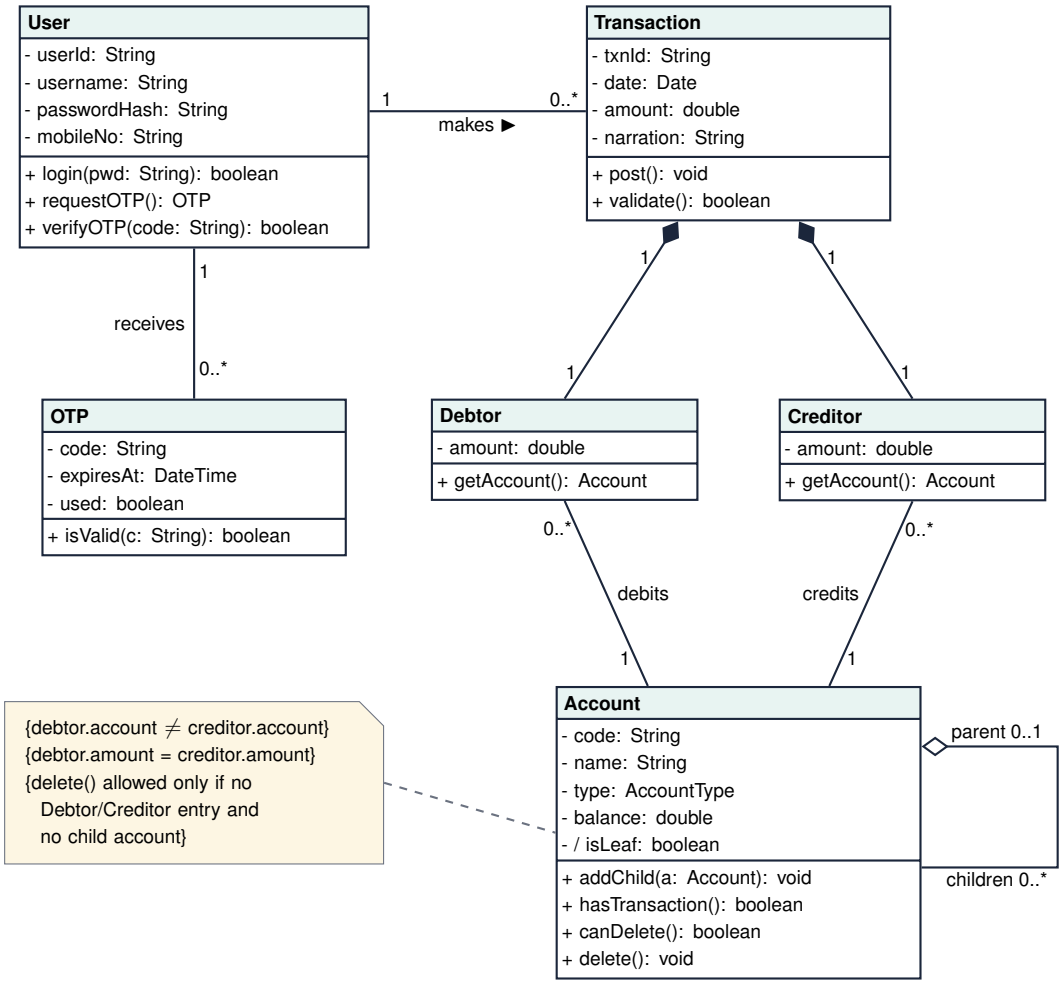
Identifying classes and responsibilities

- **Account** – an accounting head (code, name, type, balance). Accounts form a *tree* (chart of accounts): an account may have child accounts. A **leaf account** is simply an account with no children, so "leaf" is a **derived attribute** `/isLeaf` rather than a separate class (an account can stop being a leaf when a child is added – an object cannot change its class at run time).
- **Transaction** – date, amount, narration. Each transaction has *exactly one* **Debtor** entry and *exactly*

one **Creditor** entry.

- **Debtor / Creditor** – store which account is debited/credited (and the amount). They have no life outside their transaction ⇒ **composition**.
- **User** and **OTP** – the user who makes the transaction is authenticated in two steps: `login(password)` and then `verifyOTP(code)` against an `OTP` object with an expiry time.

Class diagram



Modelling decisions explained

(i) Reflexive aggregation on Account (`parent 0..1 ◇- children 0..*`): a root account has no parent, every other account has one parent, and any account may have many children. This is the Composite idea (part–whole tree) expressed with a single class.

(ii) Transaction ♦ Debtor and ♦ Creditor (1 – 1 each): "in each transaction one account is debited and at the same time another account is credited". Deleting a transaction deletes its two entries.

(iii) Debtor/Creditor 0..* – 1 Account: an entry refers to exactly one account; an account can appear in many entries over time. These links are what `hasTransaction()` checks.

(iv) Deletion rules are business constraints written in the note and implemented by `canDelete()`: `return !hasTransaction() && isLeaf;`

(v) Two-step authentication: `User 1 - 0..* OTP`; a transaction can be `post()`ed only by a user whose password and current OTP were both verified.

```
boolean canDelete() {                // in class Account
    return children.isEmpty()        // an account with a child cannot be deleted
        && entries.isEmpty();        // ...nor one used by any Debtor/Creditor entry
}
```

Answer. Account (with self-aggregation parent 0..1 / children 0..*, derived `/isLeaf`, `canDelete()`); Transaction composed of exactly one Debtor and one Creditor, each linked to one Account; User (password) with OTP for two-step authentication makes 0..* Transactions.

2022–2023 • Q4(a) CSE325 • 30/01/2025 • C325-2223-4a **MARKS 14**

You need to develop a system for a garage that experiences the following events:

- mode-XXX: the operator sets the garage control mode to XXX (off, auto, closed, open).
- exitgate: an exit gate signals that a car has left.
- entrygate: an entry gate signals that a car has entered.
- The garage counts the number of cars it hold (count), and knows its maximum capacity (max).
- The entry gates should be locked in closed mode; unlocked in open mode; are in no particular state in off mode; and in auto mode, should be locked if the garage is full.
- The operator may set any mode at any time.

Design a State diagram for this system.

SOLUTION

Step 1 – Find the states, events, guards and actions

Element	From the scenario
Attributes	`count` (cars inside), `max` (capacity)
Events	`mode-XXX` (XXX ∈ {off, auto, closed, open}), `entrygate`, `exitgate`
Mode states	**Off**, **Open**, **Closed**, **Auto**
Sub-states of Auto	**NotFull** (gates unlocked) and **Full** (gates locked) – the only place where the gate state depends on `count`
Actions	`count := count + 1` on entrygate, `count := count − 1` on exitgate; `lockEntryGates()`, `unlockEntryGates()` as *entry* actions

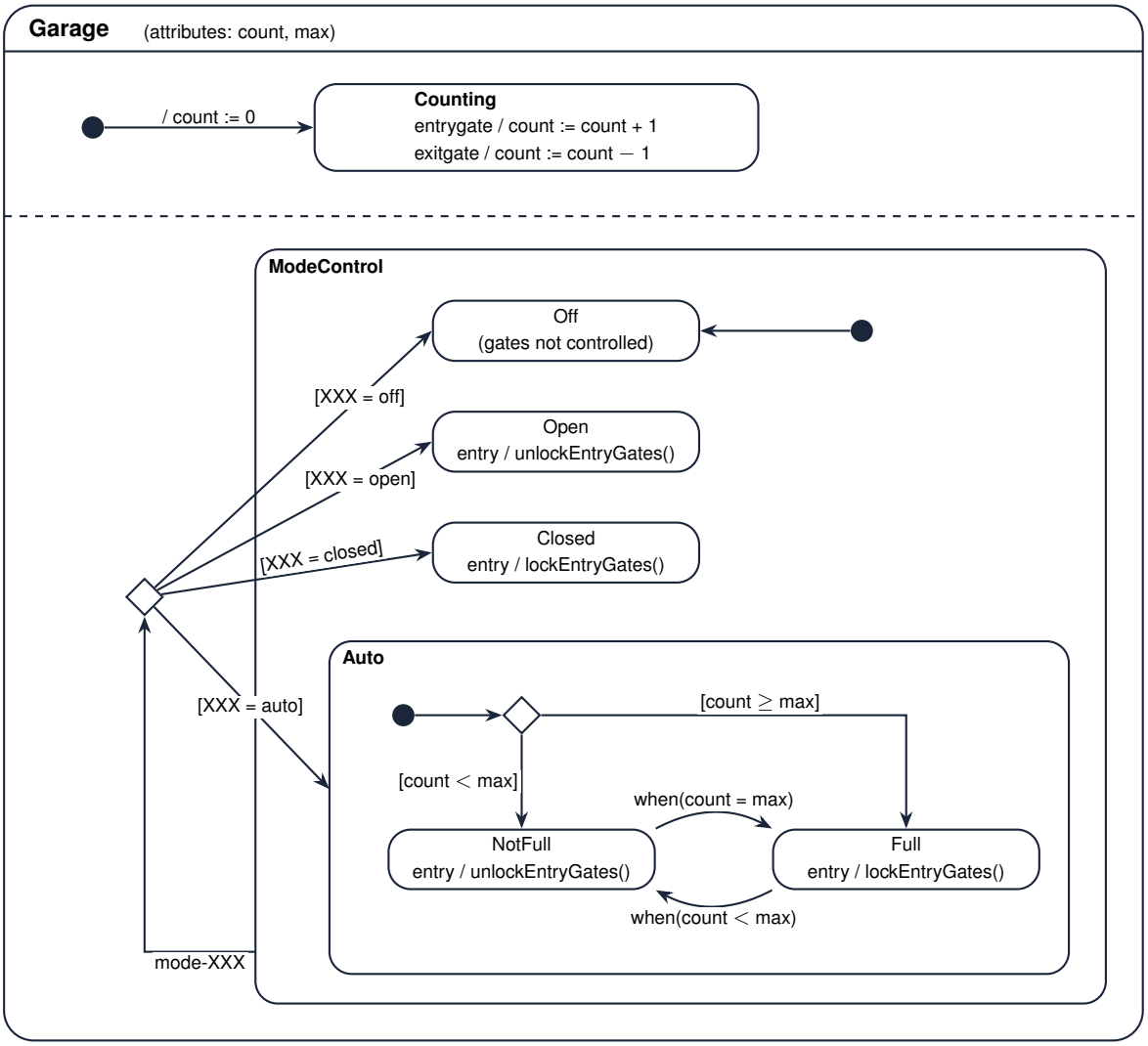
Two observations shape the diagram:

(i) Counting is independent of the mode – cars may leave in every mode (and enter in every mode except Closed), and the count must stay correct when the operator switches modes. So counting is modelled in its own **orthogonal (concurrent) region**, while the mode logic sits in a second region of the composite state *Garage*.

(ii) "Any mode at any time" would need $4 \times 3 = 12$ transitions between the four mode states. Instead, the four modes are grouped in a composite state *ModeControl*; a single transition

`mode-XXX` leaves the composite (from whatever sub-state is active, including Auto's sub-states) and a **choice pseudo-state** routes it to the requested mode by the guard `[XXX = ...]`.

Step 2 – State diagram



Step 3 – How the diagram satisfies every requirement

- **Counting.** In the *Counting* region, `entrygate` and `exitgate` are *internal transitions*: they update `count` without leaving the state, whatever mode is active (events are delivered to both orthogonal regions).
- **Closed / Open / Off.** *Closed* locks the entry gates in its entry action, *Open* unlocks them, *Off* has no gate action ("no particular state").
- **Auto.** On entering Auto, the initial pseudo-state leads to a choice: if the garage is already full the gates are locked (*Full*), otherwise unlocked (*NotFull*). While in Auto, the **change events** `when(count = max)` and `when(count < max)` fire as soon as the counting region changes `count`, so the gates lock exactly when the last free space is taken and unlock as soon as a car leaves.

- **Any mode at any time.** The `mode-XXX` transition is drawn from the *boundary* of *ModeControl*, hence it is enabled in every mode sub-state (and in both Auto sub-states). The choice pseudo-state picks the target mode, whose entry action sets the gates correctly.
- **Initial mode.** At power-on the garage starts with `count = 0` and in *Off* (assumption).

Answer. Composite state *Garage* (count, max) with two orthogonal regions: (1) *Counting* with internal transitions `entrygate / count++` and `exitgate / count-`; (2) *ModeControl* containing *Off*, *Open* (entry/unlock), *Closed* (entry/lock) and composite *Auto* = {*NotFull* (entry/unlock), *Full* (entry/lock)} with `when(count = max)` / `when(count < max)` between them; a single `mode-XXX` transition from ModeControl's border to a choice `[XXX = off/open/closed/auto]` lets the operator set any mode at any time.

NOTE The scenario does not give the initial mode; *Off* with `count = 0` is assumed. An equivalent flat answer draws the four mode states and all 12 `mode-XXX` transitions between them, with `entrygate / count++` and `exitgate / count-` as internal transitions of each state – correct but much more cluttered.

2021–2022 • Q2(b) CSE307 • 07/05/2023 • R004.1 **MARKS 10**

Consider the following simplified description of a university:
A university has multiple departments. Each department can have a number of faculties. A department must have one department head. Courses are offered by the departments. The departments have the full freedom to not to offer any course. Students are also a member of the university. Students attend course(s) which is instructed by the faculties.
Draw a class diagram for this university. Add attributes and methods when necessary. Visibility modifiers (public, private, etc.) are not required.

SOLUTION

Method (ERD to class, as in the lecture)

The lecture builds a class diagram in five steps: **(1)** pick entity/class candidates from the nouns, **(2)** find relationships, **(3)** decide association type and multiplicity, **(4)** draw the basic class diagram, **(5)** add attributes and then operations (behaviours).

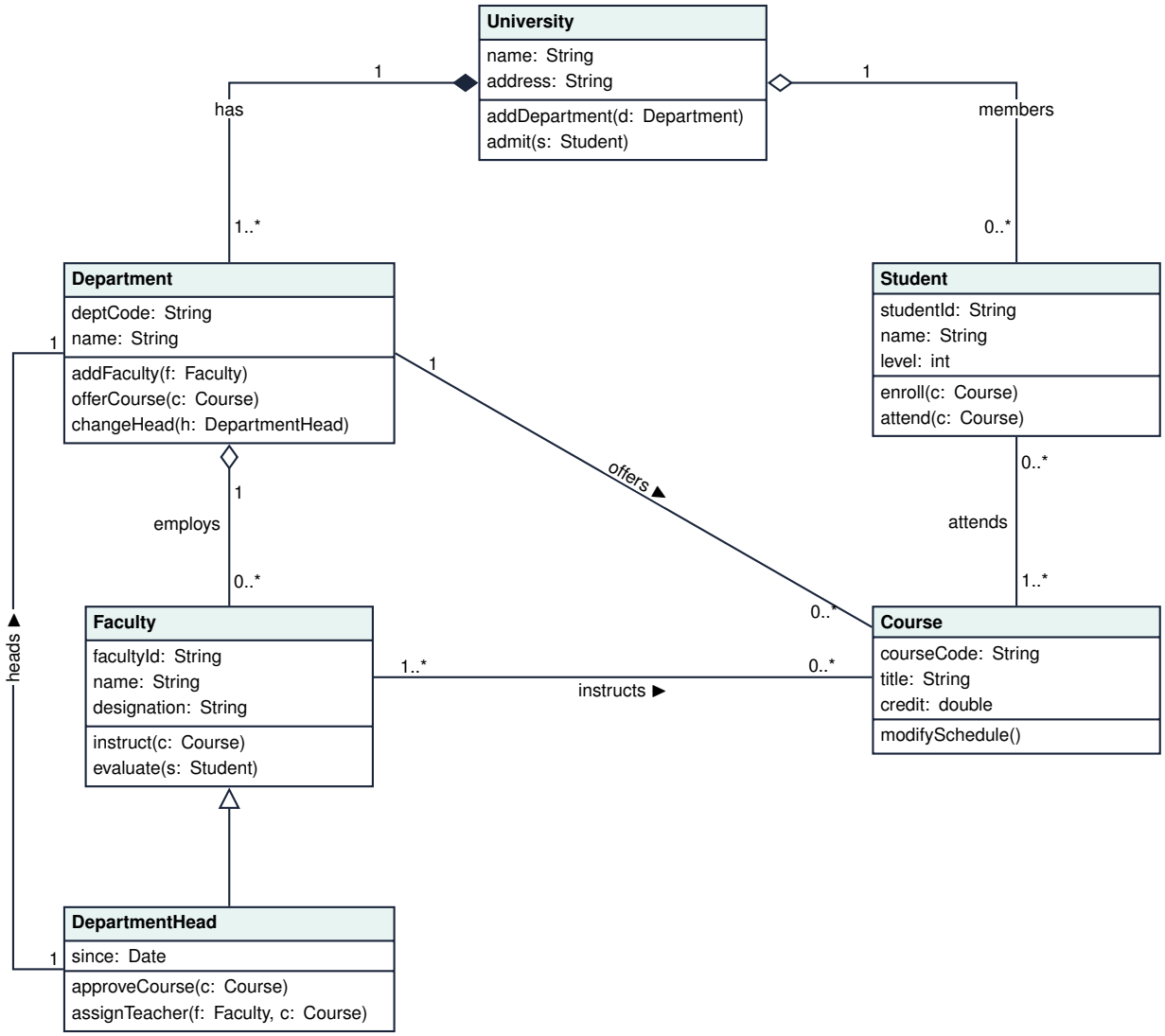
Step 1 – class candidates

University, Department, Faculty (teacher), Department Head, Course, Student. Following the lecture's `Chair/Professor` example, the **department head is a faculty member with extra duties**, so `DepartmentHead` is a *subclass* of `Faculty`.

Steps 2–3 – relationships and multiplicities

Relationship	Type / multiplicity	Reason from the text
University – Department	composition, 1 – 1..*	"has multiple departments"; a department cannot exist without its university
Department – Faculty	aggregation, 1 – 0..*	"can have a number of faculties"; a teacher exists independently
Department – DepartmentHead	association, 1 – 1	"must have *one* department head"
Department – Course	association, 1 – 0..*	"offered by" and "freedom to not offer any course" ⇒ lower bound 0
University – Student	aggregation, 1 – 0..*	"students are also a member of the university"
Student – Course	association, 0..* – 1..*	"students attend course(s)"
Faculty – Course	association, 1..* – 0..*	"course(s) which is instructed by the faculties"

Steps 4–5 – class diagram with attributes and operations



Explanation of the key decisions

- **Composition vs. aggregation.** A department is created and closed by the university (◆); faculties and students are people who exist on their own (◇).

- **Head as generalization + association.** `DepartmentHead` *is a* `Faculty` (inherits name, id, `instruct()`), and the `heads` association with multiplicity `1 - 1` enforces "a department *must* have one head".
- **Optional course offering.** The `0..*` at the Course end of `offers` directly encodes "full freedom to not offer any course".
- Visibility symbols are omitted, as the question allows.

Answer. Classes University, Department, Faculty, DepartmentHead (subclass of Faculty), Course, Student; University ♦ 1..* Department, Department ◇ 0..* Faculty, Department 1–1 head, Department 1 – 0..* Course, Student 0..* – 1..* Course, Faculty 1..* – 0..* Course.

2021–2022 • Q8(a) CSE325 • 07/10/2023 • C325-2122-8a **MARKS 13**

A university department organizes seminars on a number of research topics. These seminars take place in September each year. There is a computer program to help with the administration of these seminars. In the program there is an object representing a seminar, and this `Seminar` class has a number of possible states.
A seminar is first proposed by a professor. Once dates, times and rooms for the seminar have been arranged, it is considered as scheduled. On the second Monday in August the seminar is advertised and it becomes open for enrolment to the students. When all places are taken, the seminar is full and cannot allow any more enrolment. If a student drops out (cancels his/her application) the seminar reverts to the Open For Enrolment position. Two days before the start of the seminar, enrolment will be closed (no more applicants will be considered).
Design a State diagram to show different states of a Seminar in the context discussed above.

SOLUTION

Step 1 – find the states (conditions the Seminar object stays in)

Reading the scenario for "it is …" / "it becomes …" phrases gives the states:

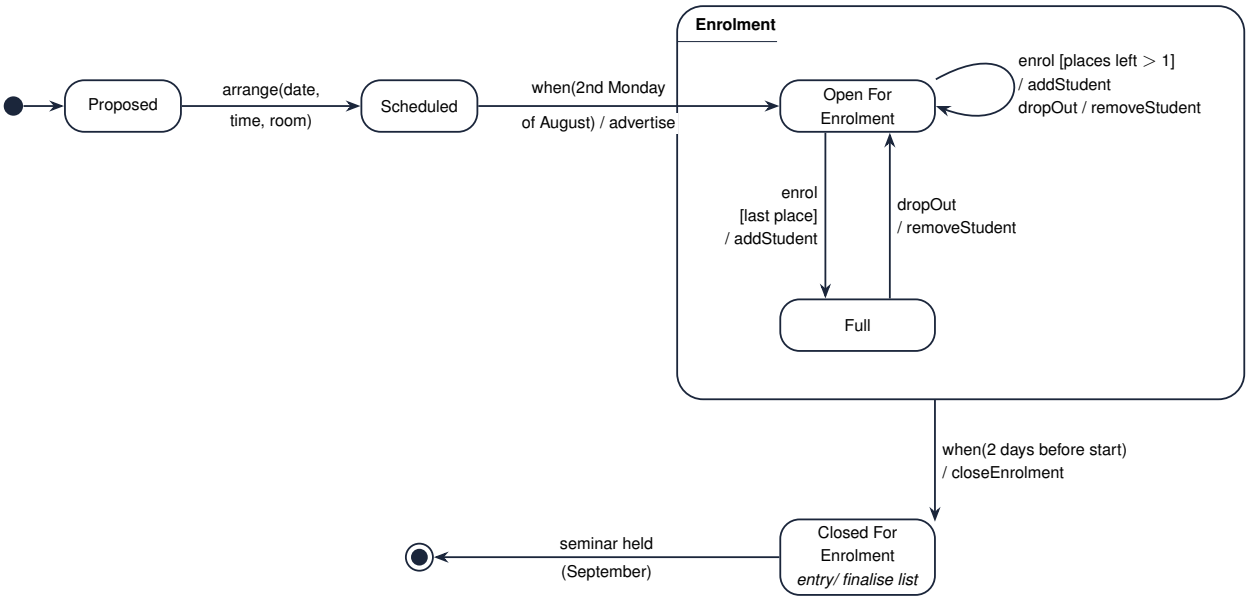
State	Meaning (what is true while the object is in it)
Proposed	a professor has proposed the topic; no date/time/room yet
Scheduled	date, time and room are arranged, but the seminar is not yet advertised
Open For Enrolment	advertised; students may enrol; at least one place is free
Full	all places taken; further enrolment requests are refused
Closed For Enrolment	two days before the start: no more applicants are considered

Step 2 – find the events, guards and actions

- `arrange(date, time, room)` — Proposed → Scheduled.
- `when(second Monday of August) / advertise()` — a **change (time) event**; Scheduled → Open.
- `enrol(student) [places left > 1] / addStudent()` — stays in Open (self-transition); `enrol(student) [last place] / addStudent()` — Open → Full. The **guard** decides which.
- `dropOut(student) / removeStudent()` — Full → Open (a place becomes free); in Open a drop-out is just a self-transition.

- `when(2 days before start) / closeEnrolment()` — happens whether the seminar is Open *or* Full, so both states are grouped into the composite state Enrolment and *one* transition leaves the composite border (this avoids drawing two identical arrows).
- After the seminar has been held in September the object's life ends (final state).

Step 3 – the state diagram



Modelling decisions

- **Composite state.** Open For Enrolment and Full are substates of Enrolment; the closing transition leaves the composite border, so it applies to whichever substate is active.
- **Time events** use `when(...)` (an absolute date) rather than an ordinary call event.
- **Guards** on `enrol` make the choice between staying Open and becoming Full deterministic.
- Enrol requests in Full have no transition, i.e. they are ignored/refused — exactly "cannot allow any more enrolment".

Answer. Proposed → (arrange) Scheduled → (when 2nd Monday of August / advertise) Open For Enrolment ⇄ Full (enrol [last place] / dropOut), both inside composite Enrolment; when(2 days before start) → Closed For Enrolment → final after the seminar is held.

NOTE The scenario does not mention cancellation of a seminar; if wanted, add `cancel` from the Proposed/Scheduled/Enrolment states to a Cancelled final state.

2020–2021 • Q5(a) CSE307 • 19/10/2022 • R006 **MARKS 16**

A media agency is responsible to order advertisements at different newspapers. Each newspaper has a standard rate for unit size (1 column-inch) for each page which may change over time. Different newspaper gives different discounts over this rate to the agency. The agency has a

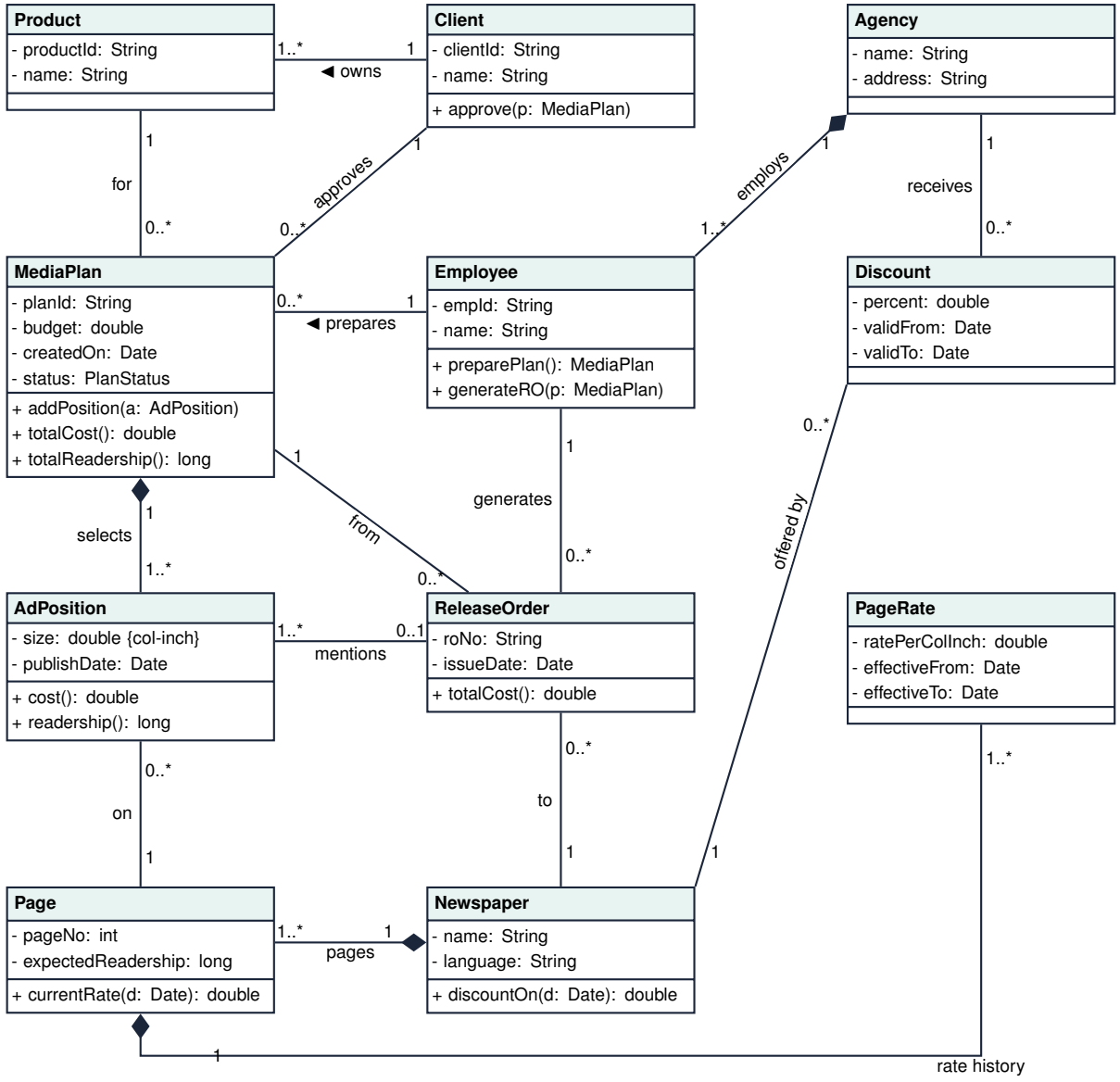
database of expected readership for each page of each newspaper. An employee of the agency prepares a media plan for a product by selecting different positions for the ad (page, size, date and newspaper) to achieve maximum readership for a given budget. This plan is approved by client and then another employee generates Release Order from this plan. The Release Order mentions a number of ads to display with their positions, date to publish and cost.
Design a Class diagram to implement the scenario described above.

SOLUTION

Candidate classes (nouns) and their data

Class	Why it is needed / key attributes
Agency, Employee	the agency and its staff who prepare plans and release orders
Client, Product	a plan is made *for a product* of a client, and the client approves it
Newspaper, Page	readership is stored "for each page of each newspaper" ⇒ Page (pageNo, expectedReadership) is part of Newspaper
PageRate	"standard rate . . . for each page which may change over time" ⇒ a rate *history*: ratePerColInch, effectiveFrom, effectiveTo
Discount	"different newspaper gives different discounts . . . to the agency" ⇒ links Agency and Newspaper: percent, validFrom, validTo
MediaPlan	budget, date, status; holds the chosen positions; computes total cost and readership
AdPosition	one selected slot: page, size (col-inch), publishDate (newspaper is reached through the page)
ReleaseOrder	generated from an approved plan; lists ads with positions, date and cost

Class diagram



NOTE The rate is attached to **Page** because the question says "standard rate for unit size … for each page". It is assumed that one Release Order is issued *per newspaper* (as in practice), hence `ReleaseOrder 0..* - 1 Newspaper` and a plan may produce several ROs.

How the design supports the required behaviour

- **Rates and discounts that change over time** are kept as *history classes* (`PageRate`, `Discount`) with validity dates, instead of a single attribute that would be overwritten; the price on any publish date can then be reconstructed.
- **Cost of an ad** (computed by `AdPosition.cost()`):

$$\text{cost} = \text{size} \times \text{rate}_{\text{page}}(\text{date}) \times \left(1 - \frac{\text{discount}_{\text{newspaper}}(\text{date})}{100}\right),$$

  where size is in column-inches and date is the ad's publish date.
- **Maximum readership for a given budget**: `MediaPlan` accumulates `totalReadership()` (sum of `Page.expectedReadership`) subject to `totalCost()` $\leq$ `budget`.

- **Workflow**: Employee *prepares* plan → Client *approves* (status changes to APPROVED) → another Employee *generates* ReleaseOrder(s) from the plan, which *mention* the chosen AdPositions (position, date and cost).
- **Composition** is used where the part has no meaning alone: Page in Newspaper, AdPosition in MediaPlan, PageRate in Page, Employee in Agency.

Answer. Eleven classes: Agency, Employee, Client, Product, MediaPlan, AdPosition, ReleaseOrder, Newspaper, Page, PageRate, Discount. Time-varying rate/discount are history classes; MediaPlan ♦ AdPosition (on a Page of a Newspaper); ReleaseOrder generated from an approved MediaPlan mentions the ads with date and cost.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

Jan 2021 • Q5(a) CSE307 • 04/09/2021 • R007 MARKS 20

Ward is a division of a hospital or a suite of rooms shared by patients who need a similar kind of care. In a hospital, there are a number of wards, each of which may be empty or have on it one or more patients. Each ward has a unique name. Wards are differentiated by gender of its patients, i.e. male wards and female wards. A ward can only have patients of the gender admitted to it. Every ward has a fixed capacity, which is the maximum number of patients that can be on it at one time (i.e. the capacity is the number of beds in the ward). Different wards may have different capacities. The doctors in the hospital are organized into teams. Each team has a unique name or code (e.g. Orthopedics or Pediatrics) and is headed by a consultant doctor or attending physician. She or he can supervise fellows, residents, and medical students. The rest of the team members are all junior doctors. Each doctor could be a member of no more than one team. Everyday each consultant doctor visits the patients admitted under him/her. During this visit, s/he prescribes treatment or modifies previous prescription. When a patient is ready to be released, s/he advises the release.
Based on the scenario discussed above, draw a Class Diagram.

2017–2018 • Q5(a) CSE307 • 29/09/2018 • R010 MARKS 14

Same hospital-ward scenario as above, with one extra sentence: *"Consultant doctor or attending physician is the senior doctor who has completed all of his or her specialist training, residency and practices medicine in a clinic or hospital, in the specialty learned during residency."* Design a Class diagram to implement the scenario described above.

SOLUTION

Step 1 – classes from the scenario

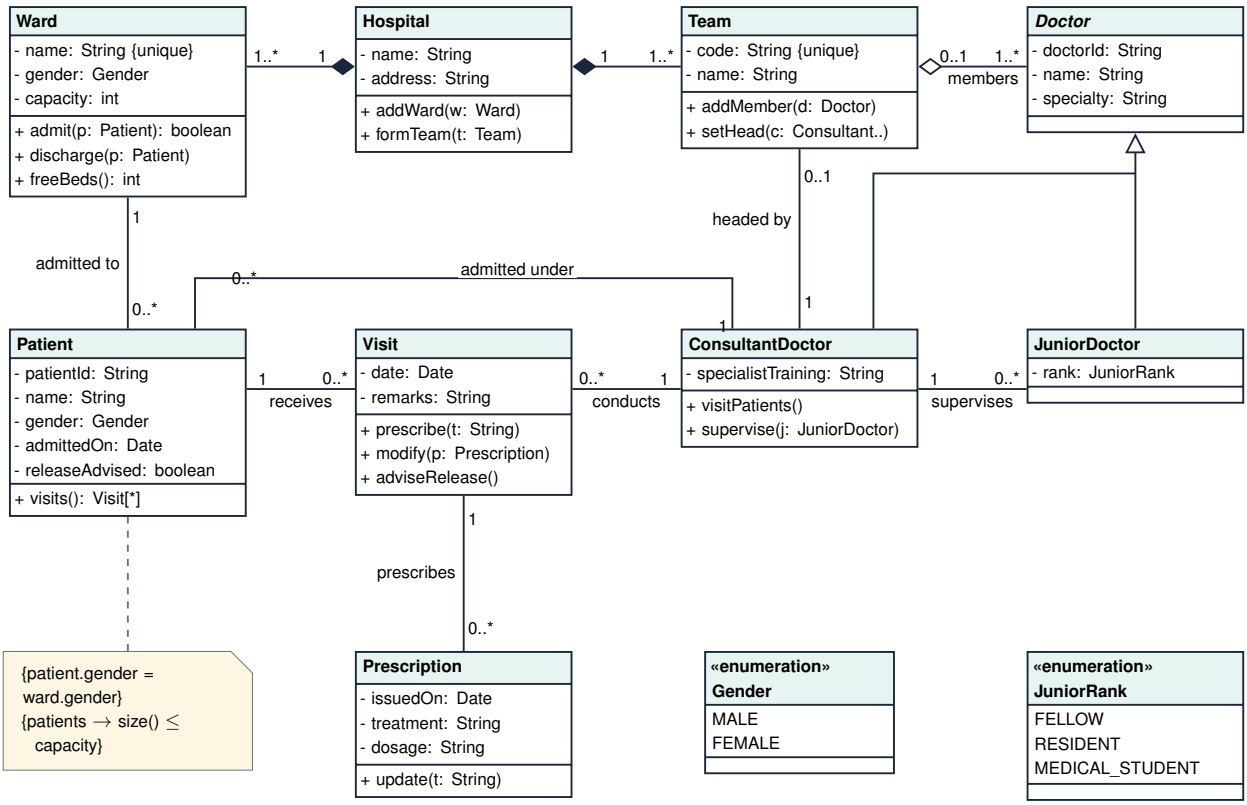
- **Hospital**, **Ward** (unique name, gender, capacity), **Patient**.
- **Team** (unique code/name), **Doctor** (abstract), with two kinds: **ConsultantDoctor** (= attending physician, the senior specialist) and **JuniorDoctor** (fellow, resident or medical student).
- **Visit** – the daily visit of a consultant to a patient; during it a **Prescription** is created or modified and the release may be advised.
- Enumerations **Gender** {MALE, FEMALE} and **JuniorRank** {FELLOW, RESIDENT, MEDI-

CAL_STUDENT}.

Step 2 – relationships and multiplicities

Relationship	Multiplicity	Text that fixes it
Hospital ♦ Ward	1 – 1..*	"a number of wards" (a ward is a division of the hospital)
Ward – Patient (admitted to)	1 – 0..*	"may be empty or have one or more patients"; upper bound is `capacity`
Hospital ♦ Team	1 – 1..*	doctors are organised into teams
Team ◇ Doctor (member)	0..1 – 1..*	"no more than one team" ⇒ 0..1 on the Team side
Team – ConsultantDoctor (headed by)	0..1 – 1	every team has exactly one head
ConsultantDoctor – JuniorDoctor (supervises)	1 – 0..*	"can supervise fellows, residents and students"
ConsultantDoctor – Patient (admitted under)	1 – 0..*	"patients admitted under him/her"
ConsultantDoctor – Visit – Patient	1 – 0..* – 1	a visit *every day*: many visits per pair
Visit – Prescription	1 – 0..*	prescribes new or modifies previous prescription

Step 3 – class diagram



Step 4 – explanation of the modelling decisions

(i) Uniqueness: `Ward.name` and `Team.code` carry the UML constraint `{unique}`.

(ii) Male/female wards: rather than two subclasses with no different behaviour, a `gender: Gender` attribute is used; the rule "a ward can only have patients of that gender" is the OCL-style constraint `{patient.gender = ward.gender}`, checked in `admit()`.

(iii) Capacity: `capacity` is the number of beds; `admit()` refuses a patient when `patients.size() = capacity`, so the `0..*` multiplicity is bounded by the constraint note. `freeBeds() = capacity - patients.size()`.

(iv) Doctor hierarchy: `Doctor` is *abstract* (italic) – every real doctor is either a consultant (head, supervisor, visits patients) or a junior doctor. Fellows, residents and medical students differ only by rank, so a `JuniorRank` enumeration is enough (subclasses would also be acceptable).

(v) Team membership: aggregation `Team 0..1 ◇- 1..* Doctor` expresses "no more than one team"; the head is a separate association `headed by` with multiplicity exactly 1, because being head is a *role* the consultant plays in that team.

(vi) Visit as a class, not an association class: an association class allows only one object per (consultant, patient) pair, but the consultant visits *every day*. So `Visit` is an ordinary class with `date`; it records the prescription changes and the release advice (`adviseRelease()` sets `Patient.releaseAdvised`).

Answer. Hospital ♦ Ward (unique name, gender, capacity) – 0..* Patient; Hospital ♦ Team ◇ Doctor (abstract) with subclasses ConsultantDoctor (heads team, supervises 0..* JuniorDoctor, has patients admitted under him/her) and JuniorDoctor; daily Visit (Consultant 1 – 0..* Visit 0..* – 1 Patient) creates/modifies Prescriptions and advises release.

2020–2021 • Q2(a) CSE325 • 12/04/2023 • C325-2021-2a **MARKS 13**

In an educational institution, the teachers offer different course in different semesters. Some courses are jointly offered by multiple teachers. Students enroll in multiple offerings and receive grades at the end of the course. The grades are computed following some grading rule from the marks of different test items. Draw a Class Diagram to reflect the scenario discussed above. Apart from model classes, show the classes that would implement business logic and store computed values. You are not required to show any Boundary or Controller class.

SOLUTION

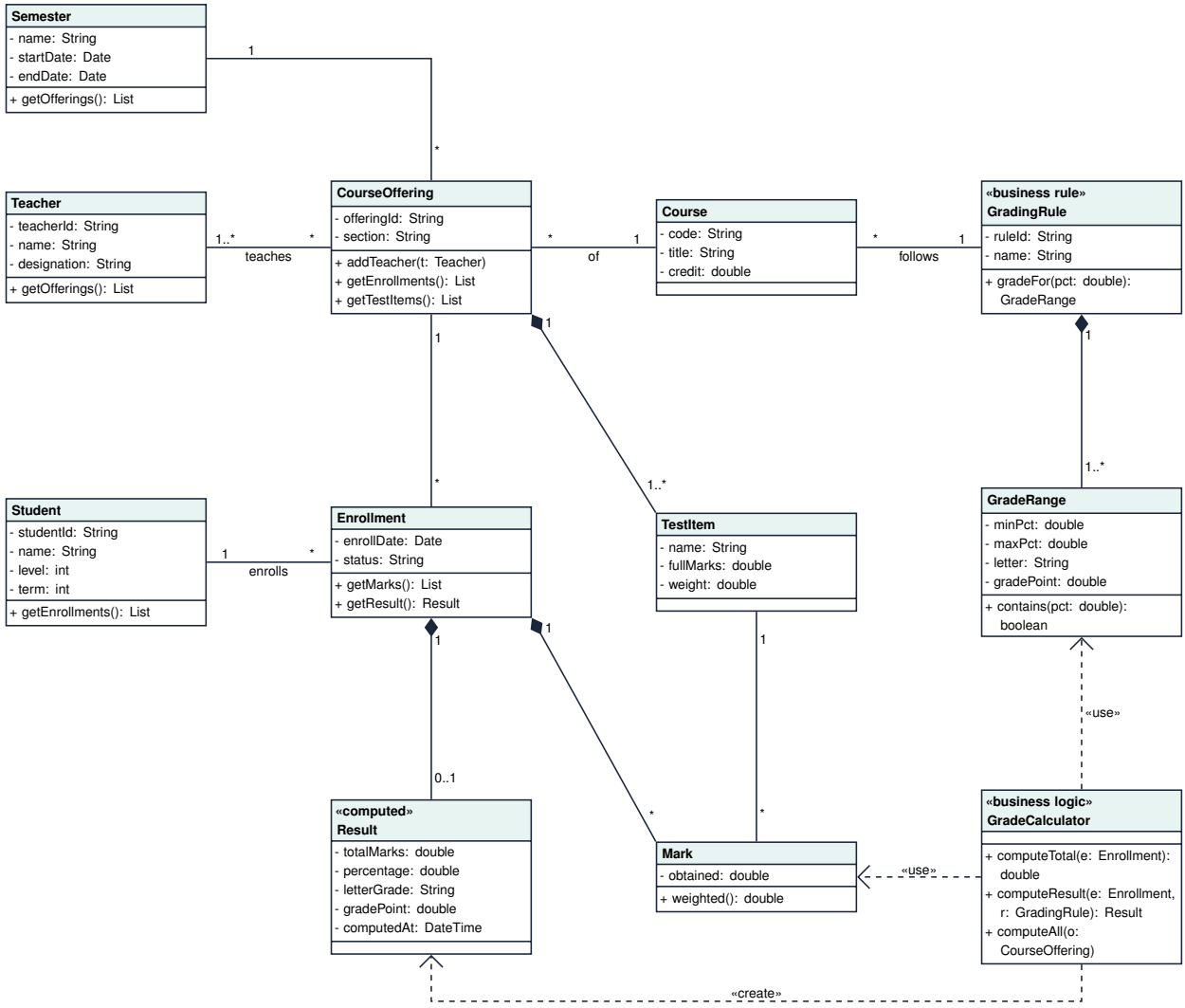
Step 1 – candidate classes from the nouns

Noun in the scenario	Decision
teacher, student, course, semester	entity (model) classes
"course offered in a semester" (offering)	a separate entity `CourseOffering`: a course is offered many times, each offering belongs to one semester and has its own teachers, students and test items
student *enrolls in* offering	many-to-many with its own data (grade) ⇒ entity `Enrollment` (association class promoted to a class)
test items (class tests, mid-term, final . . .)	entity `TestItem`, owned by an offering (weight, full marks)
marks of a student in a test item	entity `Mark` between `Enrollment` and `TestItem`
grading rule	business-rule class `GradingRule` made of `GradeRange` rows (e.g. $\geq$80% $\rightarrow$ A+, 4.00)
"grades are computed"	business-logic (service) class `GradeCalculator`
computed total / grade	stored computed value: `Result` (one per enrollment)

Step 2 – relationships and multiplicities

- A teacher teaches $*$ offerings; an offering is taught by `1..*` teachers ("jointly offered").
- A course has $*$ offerings; each offering is of exactly `1` course and in `1` semester.
- A student has $*$ enrollments; an offering has $*$ enrollments; each enrollment links exactly one student to one offering.
- An offering is *composed* of `1..*` test items (they have no meaning without the offering).
- An enrollment has $*$ marks; each mark is for `1` test item.
- A course (and hence every offering of it) follows `1` grading rule (a rule may be reused by many courses); a rule is composed of `1..*` grade ranges.
- An enrollment owns `0..1` result (absent until grades are computed).
- `GradeCalculator` *uses* enrollments, marks and the grading rule and *creates* results (dependencies, not structural associations).

Step 3 – class diagram



NOTE `Course` *follows* `GradingRule` is drawn from the course; read it as "each offering of the course uses the course's grading rule" ($*$ offerings/courses share 1 rule). If a teacher can choose a rule per offering, attach the association to `CourseOffering` instead — the rest is unchanged.

Step 4 – explanation of the three kinds of classes

- **Model (entity) classes:** `Semester`, `Teacher`, `Course`, `CourseOffering`, `Student`, `Enrollment`, `TestItem`, `Mark` — persistent data of the domain.
- **Business-logic classes:** `GradeCalculator` computes total $= \sum_{\text{items}} \frac{\text{obtained}}{\text{fullMarks}} \times \text{weight}$ for each enrollment and asks `GradingRule.gradeFor(pct)` for the letter grade; `GradingRule`/`GradeRange` encapsulate the rule itself, so a new rule (e.g. relative grading) is a new rule object — the calculator is unchanged (OCP).
- **Classes that store computed values:** `Result` keeps the computed total, percentage, letter grade, grade point and time of computation, so transcripts/grade sheets need not recompute them; it is owned by (composition) the `Enrollment`.

Answer. Entities Teacher (1..*)–(*) CourseOffering (*)–(1) Course, CourseOffering (*)–(1) Semester; Student 1–* Enrollment *–1 CourseOffering; CourseOffering ◇ 1..* TestItem; Enrollment 1–* Mark *–1 TestItem. Business logic: GradeCalculator (uses Mark, GradingRule) and GradingRule ◇ GradeRange. Computed values: Result (0..1 per Enrollment) holding total, letter grade and grade point.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2020–2021 • Q4(a) CSE325 • 12/04/2023 • C325-2021-4a MARKS 12

In an automatic Inventory Management System, an office stocks different products that are expected to be used by the users. When a user needs something, s/he makes a Requisition. If that is in the stock, the requisition is approved by the management and becomes ready to Issue. If the stock level of a product reaches below a threshold, a re-order request is generated. If the product is not consumable, it has to be returned when the employee is retired or transferred. Design a State diagram to show different status of a product in the context discussed above.

2018–2019 • Q4(a) CSE325 • 20/12/2021 • C325-1819-4a MARKS 22

In an automatic Inventory Management System, an office stocks different products that are expected to be used by the users. When a user needs something, s/he makes a Requisition. If that is in the stock, the requisition is approved by the management and becomes ready to Issue. If the stock-level of a product reaches below a threshold, a re-order request is generated. If the product is not consumable, it has to be returned when the employee is retired or transferred. Design a State diagram to show different status of a product in the context discussed above.

SOLUTION

Concept and the object being modelled

A state diagram describes the dynamic behaviour of *one class*: "the current state of an object is determined by the current values of the object's attributes and the links that it has with other objects" (lecture). The word *product* is used in two senses in the scenario, and each has its own status:

(i) a **product item** (one physical unit, e.g. laptop #LT-17, or one ream of paper) that is stocked, requisitioned, issued, and either consumed or returned;

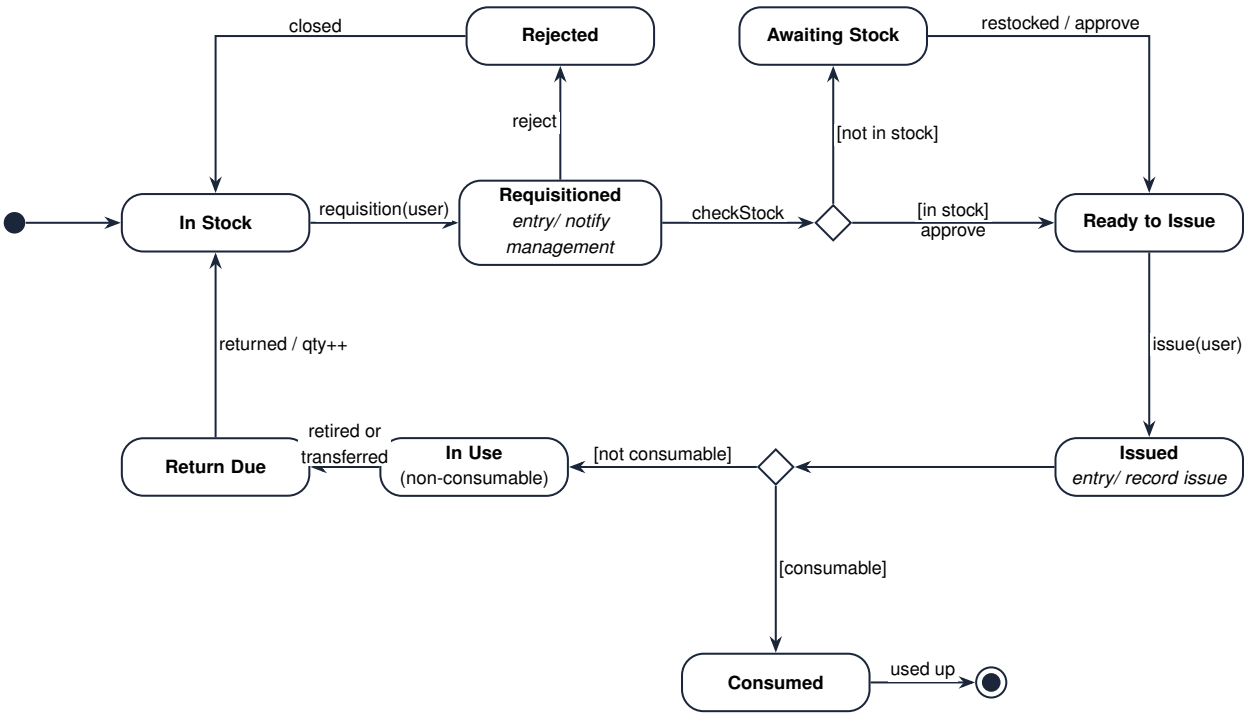
(ii) the **stock of a product** (all units of "A4 paper"), whose status depends on the `quantity` attribute compared with the `threshold` (re-order logic).

A complete answer therefore shows the item life cycle and the stock-level behaviour. Drawing them as two *orthogonal (concurrent) regions* of one `Product` state machine would be hard to read, so they are drawn as two diagrams linked by the events `issue` and `returned`.

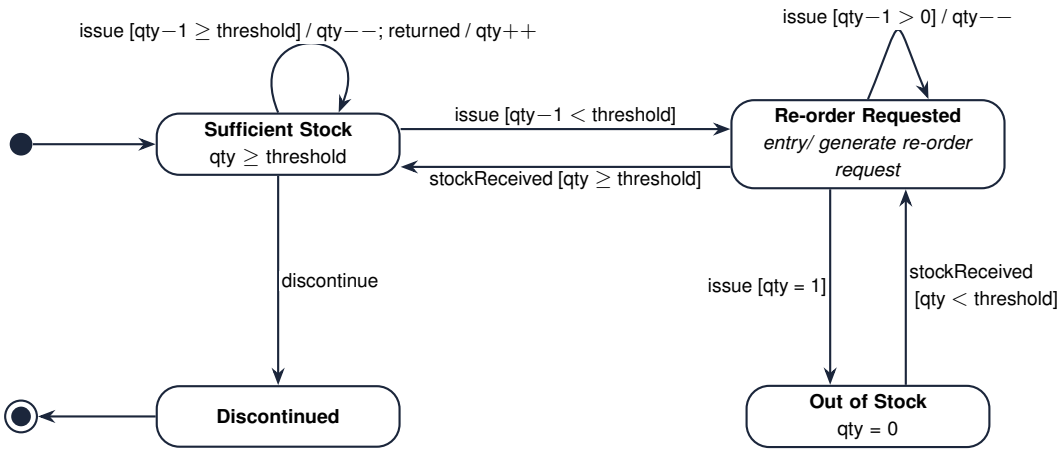
Step 1 – States and events from the scenario

Scenario phrase	Element
office stocks products	state *In Stock* (available)
user makes a Requisition	event `requisition(user)` → *Requisitioned*
if in stock, approved by management	guard `[in stock]`, event `approve` → *Ready to Issue*
(not in stock)	guard `[not in stock]` → *Awaiting Stock* until `restocked`
becomes ready to issue; issued	event `issue` → *Issued*
consumable product	guard `[consumable]` → *Consumed* (final)
not consumable: returned on retirement/transfer	*In Use*; event `retired or transferred` → *Return Due*; `returned` → *In Stock*
stock-level below threshold, re-order request	change event `when(qty < threshold)` / generate re-order request → *Re-order Requested*

Step 2 – (a) State diagram of a product item



Step 3 – (b) State diagram of the stock of a product (re-order logic)



Step 4 – Explanation of the modelling decisions

- **Choice pseudo-state after Requisitioned**: the scenario approves a requisition only *"if that is in the stock"*; the diamond splits on `[in stock]` / `[not in stock]`. A requisition for an unavailable product waits in *Awaiting Stock* and is approved when the re-order arrives.
- **Ready to Issue** is a separate state because approval and physical handover are two different events (store keeper issues later).
- **Consumable vs. non-consumable** is a guard on the second choice after *Issued*: a consumable item ends (final state) once used; a non-consumable one stays *In Use* until the event `retired or transferred`, then it is *Return Due* and goes back to *In Stock* when returned (`qty++`).
- **Re-order is a stock-level behaviour**: the threshold rule is about the *quantity* of a product, so it belongs to diagram (b). Every `issue` in diagram (a) is an event for diagram (b); the guard compares the new quantity with the threshold and the *entry* action of *Re-order Requested* generates the re-order request exactly once when the level drops.
- Self-transitions in (b) show that issuing/returning while staying on the same side of the threshold only updates `qty`; it does not change the state.

Additional points (from the 2020–2021 Q4(a) solution)

- The two machines change independently, so they may equally be drawn as two **orthogonal regions** of one composite `Product` state; separate diagrams are simply easier to read.
- "Returned when the employee is retired or transferred" is a **change event** `when(employee retired or transferred)` – it fires when a condition becomes true, not on a user command. *In Use* can carry the activity *do/ used by employee*.
- On return the item can be inspected before it is restocked: `returned / inspect, restock` (`qty++`).
- The diagrams are linked: while the stock is in *Out of Stock* the guard `[in stock]` of diagram (a) is false, so no requisition can be approved (it waits in *Awaiting Stock*).
- Equivalent naming used in that solution: *Return Pending* = *Return Due*, *Adequate Stock* = *Sufficient Stock*, *Re-order Pending* = *Re-order Requested*. The re-order request may be written either as the transition action `issue(q) [level − q < threshold] / generate re-order request` or as the *entry* action of the state – both generate it exactly once.

> **NOTE** *Rejected* (management refuses), *Discontinued* and the cancel/out-of-stock transitions are not in the question; they are added as common-sense completions. If only one diagram is expected in the exam, diagram (a) with the re-order transition written as an action (`/ generate re-order request [qty < threshold]`) on `issue` is acceptable.

Answer. (a) Item: In Stock → requisition → Requisitioned → [in stock] approve → Ready to Issue (or [not in stock] Awaiting Stock → restocked) → issue → Issued → [consumable] Consumed • / [not consumable] In Use → retired or transferred → Return Due → returned → In Stock. (b) Stock: Sufficient Stock ↔ Re-order Requested (entry/ generate re-order request when qty < threshold) ↔ Out of Stock.

2019–2020 • Q1(a) CSE325 • 02/04/2022 • C325-1920-1a MARKS 10

Describe briefly how Unified Modeling Language is used for visualizing, specifying, constructing and documenting.

SOLUTION

What UML is

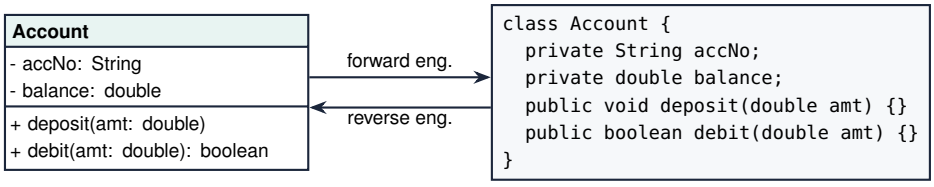
UML (Unified Modeling Language) is "an international industry standard *graphical notation* for describing software analysis and design" (lecture). It gives *pictures or views* of an object-oriented system – use case, class, sequence, collaboration, state, activity, component and deployment diagrams – because "a picture is worth a thousand words". UML is a *language*, not a process: it has a vocabulary (things, relationships, diagrams) and rules for combining them, so a model has one agreed meaning. Booch, Rumbaugh and Jacobson list exactly four uses.

The four uses

Use	What it means	Example
Visualizing	Some structures transcend what can be read in code (class hierarchies, object interactions, distribution). UML gives every one of them a *standard symbol*, so a mental model becomes an explicit picture that any other developer reads the same way – no private, ambiguous "boxes and arrows".	A class diagram showing that `Loan` is associated with `Employee` (many loans to one employee).
Specifying	Specifying means building models that are *precise, unambiguous and complete*. UML specifies all important analysis, design and implementation decisions: which classes exist, their attributes and visibilities, the messages exchanged and their order, the states an object passes through.	A sequence diagram fixes the exact order `verifyPIN()` → `checkBalance()` → `dispense()`; a state diagram fixes legal transitions.
Constructing	UML is not a programming language, but its models map *directly* to Java, C++, C# or to tables in a relational database. This allows **forward engineering** (generate code skeletons from a class diagram) and **reverse engineering** (rebuild a model from code) – round-trip engineering in tools such as StarUML or Rational Rose.	`class Account { private double balance; public void debit(double a) {} }` generated from the class box.
Documenting	A software organisation produces artefacts besides code: requirements, architecture, design, tests, releases. UML diagrams document these at every stage and remain a shared, standard record for maintainers and new team members.	Use case diagrams in the SRS; deployment diagram in the release document.

A tiny illustration

The same class `Account` serves all four uses: drawing it *visualizes* the concept, the typed attributes and visibilities *specify* it, a tool *constructs* the Java skeleton from it, and the diagram is kept in the design document to *document* it.



Answer. UML *visualizes* (standard pictures of structure and behaviour), *specifies* (precise, unambiguous, complete models of every decision), *constructs* (models map to code/database – forward and reverse engineering) and *documents* (requirements, design, tests, releases) a software-intensive system.

2019–2020 • Q2(a) CSE325 • 02/04/2022 • C325-1920-2a **MARKS 7**

Explain the concept for Class and Instance with necessary examples.

SOLUTION

Class

A **class** is a *description (template, blueprint)* of a set of objects that share the same attributes, operations, relationships and semantics. It says *what kind of data* every member has (attributes with types) and *what every member can do* (operations), but holds no particular values itself (except static/class-scope members, which are underlined in UML). In a class diagram it is a box with three compartments: name, attributes, operations.

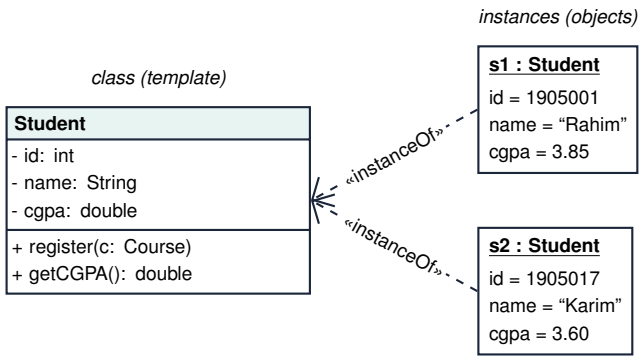
Instance (object)

An **instance** (object) is one concrete member of a class, created at run time (e.g. by `new`). It has

- **identity** – it is distinguishable from every other object even if its values are equal;
- **state** – actual values of the attributes and links to other objects at a moment;
- **behaviour** – it responds to the operations defined by its class.

Many instances can be created from one class; each instance belongs to exactly one class (its type). In UML an instance is drawn as a box whose name is underlined in the form `objectName : ClassName` and whose attributes show *values*.

Example




```
class Student {                          // CLASS: one description
    private int id; private String name; private double cgpa;
    Student(int id, String name, double cgpa) {
```

```
        this.id = id; this.name = name; this.cgpa = cgpa;
    }
    double getCGPA() { return cgpa; }
}
Student s1 = new Student(1905001, "Rahim", 3.85);   // INSTANCE 1
Student s2 = new Student(1905017, "Karim", 3.60);   // INSTANCE 2 (same class, own state)
```

	Class	Instance
Nature	abstraction / type, design-time	concrete thing, run-time
Values	attribute *types* only	attribute *values* (state)
Count	one	zero or many per class
UML notation	bold name, 3 compartments	obj : Class, values shown
Real-life example	"Car" as a model in a catalogue	the car with plate Dhaka-Metro-Ga-12-3456

Answer. A class is the blueprint that defines common attributes and operations (e.g. `Student`); an instance is a particular object created from it with its own identity and state (e.g. `s1 : Student` with `id = 1905001`).

2019–2020 • Q2(b) CSE325 • 02/04/2022 • C325-1920-2b MARKS 20

An account holder of Sonali Bank can withdraw an amount from his or her account through debit card with the help of ATM (Automatic Teller Machine) booth. The account holder must insert the card into the machine and then after authorization through PIN (Personal Identification Number) the amount of money to be withdrawn will be entered through keypad of the ATM machine. The requested amount will be checked against the balance of the bank account and the amount will be dispensed from the machine if there is available balance to satisfy the request. There must be a notification to the registered phone number to the account holder from central server of the bank. Finally, the ATM machine ejects the card after the transactions. Show the steps of identifying the classes from this use case scenario.

SOLUTION

Approach

Classes are found from a use case scenario by **noun-phrase (textual) analysis**: nouns and noun phrases are *candidate classes*, adjectives/possessives hint at attributes, verbs hint at operations and associations. The candidates are then filtered, classified (boundary / control / entity, as in Bennett's analysis model) and connected. The steps below follow that order.

Step 1 – Extract the noun phrases

account holder, *Sonali Bank*, *amount*, *account / bank account*, *debit card*, *ATM (machine, booth)*, *card*, *machine*, *authorization*, *PIN*, *money*, *keypad*, *balance*, *request*, *notification*, *registered phone number*, *central server of the bank*, *transaction*.

Step 2 – Filter the candidates

Each candidate is kept, merged or rejected with one of the standard elimination rules.

Candidate	Decision	Reason
account holder	✓ class	external actor, but the bank must *remember* him/her (name, phone) → entity `AccountHolder`
Sonali Bank	× reject	the whole system/organisation itself (irrelevant as a class)
account, bank account	✓ class	synonyms → one class `BankAccount`
debit card, card	✓ class	synonyms → `DebitCard`
ATM, machine, booth	✓ class	synonyms → `ATM` (booth is only its location → attribute)
keypad	✓ class	hardware interface through which data enter → boundary `Keypad`
(card slot), dispensing	✓ class	implied devices: `CardReader` (insert/eject) and `CashDispenser`
amount, money	× attribute	a value, not an object: `amount` of a withdrawal
balance	× attribute	value of `BankAccount`
PIN	× attribute	value stored with the card (`pinHash`)
registered phone number	× attribute	value of `AccountHolder`
authorization	× operation	a behaviour (`verifyPIN()`) not a thing
request, transaction	✓ class	an event the bank must record (date, amount, status) → entity `Withdrawal` (a kind of `Transaction`)
notification	✓ class	a message with content and delivery status → `SMSNotification`
central server of the bank	✓ class	coordinates authorization, balance check and notification → `BankServer`

Step 3 – Classify the surviving classes

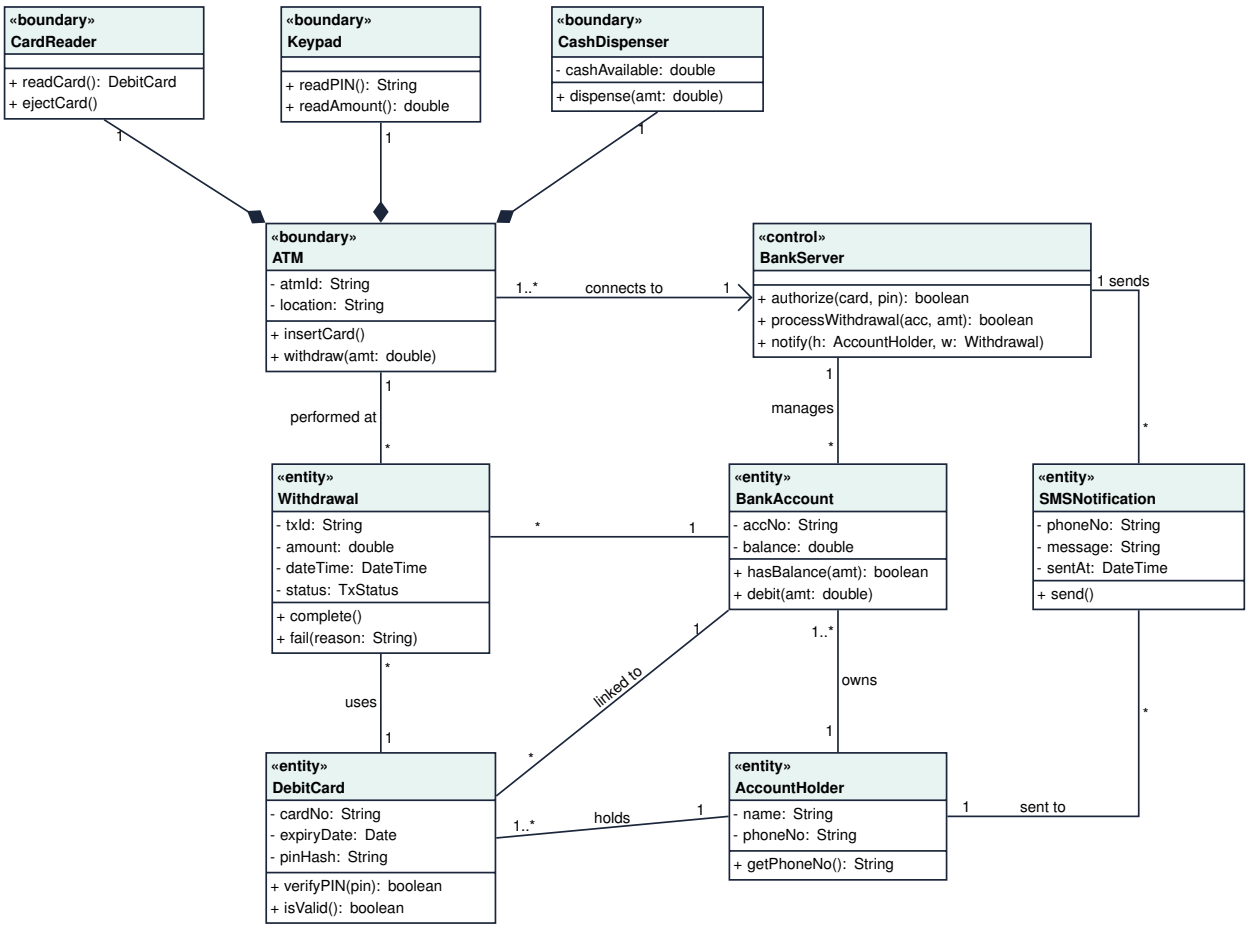
Stereotype	Classes
«boundary» (interface to actors/devices)	`CardReader`, `Keypad`, `CashDispenser`, and the `ATM` that aggregates them
«control» (coordinates the use case)	`BankServer` (authorize, check balance, debit, notify); a `WithdrawalController` could be split out of it
«entity» (persistent information)	`AccountHolder`, `BankAccount`, `DebitCard`, `Withdrawal`, `SMSNotification`

Step 4 – Attributes and operations from adjectives and verbs

- "*insert* the card", "*ejects* the card" → `CardReader.readCard()`, `ejectCard()`.
- "*authorization* through PIN" → `DebitCard.verifyPIN(pin)`, `BankServer.authorize()`.
- "amount *entered* through keypad" → `Keypad.readAmount()`.
- "*checked against the balance*" → `BankAccount.hasBalance(amt)`; on success `debit(amt)`.
- "amount will be *dispensed*" → `CashDispenser.dispense(amt)`.
- "*notification* to the registered phone number" → `BankServer.notify()`, `SMSNotification.send()`; `AccountHolder.phoneNo`.

Step 5 – Relationships and the resulting class diagram

An ATM *is made of* a card reader, a keypad and a cash dispenser (composition: the devices are part of one machine). An account holder owns one or more accounts and holds cards; each card is linked to one account. Every withdrawal is made at one ATM against one account and causes one SMS notification.



Why these decisions?

- **Composition** (filled diamond at `ATM`): the reader, keypad and dispenser exist only as parts of one machine and are replaced with it.
- `Withdrawal` is an *entity* because the bank must keep a record of every transaction (statement, audit, daily limits); it links the ATM, the account and the card used.
- `BankServer` is the *control* class: it is the "central server of the bank" that authorizes the PIN, checks and debits the balance and triggers the SMS – the ATM only collects input and dispenses cash.
- The phone number lives in `AccountHolder` ("*registered* phone number"); the notification copies it at sending time so that the record stays correct even if the number changes later.

Answer. Steps: (1) list noun phrases, (2) filter – merge synonyms, drop the system itself, turn values (amount, balance, PIN, phone no.) into attributes and verbs (authorization) into operations, (3) classify: boundary `ATM`, `CardReader`, `Keypad`, `CashDispenser`; control `BankServer`; entity `AccountHolder`, `BankAccount`, `DebitCard`, `Withdrawal`, `SMSNotification`, (4) add attributes/operations, (5) add associations/compositions and multiplicities.

2019–2020 • Q5(b) CSE325 • 02/04/2022 • C325-1920-5b **MARKS 12**

Scenario of Q5(a): If you want to make a tour with a tourism operator, you have to select a package from their website. Each package presents a combination of site visit, hotels, food, activities. Depending on these components and number of days to spend, the price of the packages varies. A package can be customized for a group of tourists with size larger than 10 and price adjustment is made accordingly. The customer has to book a package with partial advance payment. The remaining payment is made prior to the tour to confirm all the components of the tour. If any activity is not available, the refund is made later.
(b) Draw state diagram for a customer booking considering the scenario described for Q.5a.

SOLUTION

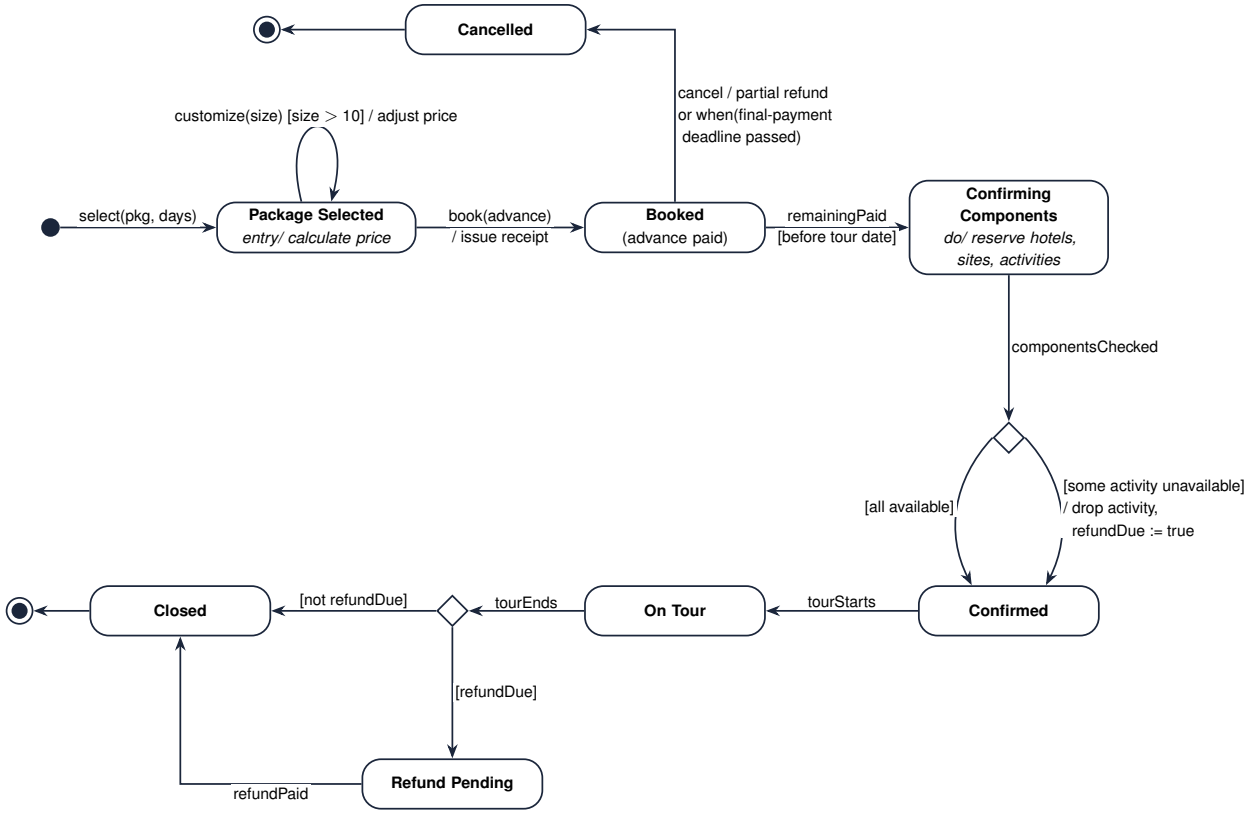
Concept

A **state diagram** models the life of *one object* – here one `Booking` – as the set of **states** it can be in (a condition during which it satisfies some condition or waits for some event), the **events** that move it, the **guards** `[ ]` that must be true and the **actions** `/ ...` performed on the transition. (The BPMN of Q5(a) is in the BPMN chapter.)

Step 1 – Events and conditions found in the scenario

Scenario phrase	State-machine element
select a package; price depends on components and days	initial state *Package Selected*, *entry/ calculate price*
customized for a group larger than 10	event `customize(size)` with guard `[size > 10]` and action `/ adjust price` (self-transition)
book with partial advance payment	event `advancePaid` → *Booked (Advance Paid)*
remaining payment prior to the tour	event `remainingPaid` → *Confirming Components*
confirm all components	choice on availability
activity not available, refund made later	guard `[some activity unavailable]` / mark refund → *Refund Pending* after the tour
(implicit) no payment by deadline, customer cancels	`when(deadline passed)` or `cancel` → *Cancelled*

Step 2 – State diagram of a Booking



Step 3 – Explanation of the decisions

- **Price customization is a self-transition**: customizing does not change the *life-cycle phase* of the booking – it is still only a selected package; only the price attribute changes, so a self-transition with guard `[size > 10]` and action `/ adjust price` is used. For a group of 10 or fewer the event is simply not accepted.
- **Booked** represents "book a package with partial advance payment": the booking exists and holds a reservation, but nothing is guaranteed yet.
- **Confirming Components** has a `do/` activity because the operator must contact hotels, site authorities and activity providers – it takes time. Its completion leads to a **choice pseudo-state**: if an activity is unavailable the tour still goes ahead (the action drops it and sets the refund flag), as the scenario says the refund is made *later*.
- **Refund Pending** is entered only after the tour if `refundDue`; the booking is **Closed** when the refund is paid.
- **Cancelled** uses a *change/time event* `when(final-payment deadline passed)` for a customer who never pays the remaining amount, plus an explicit `cancel` event.

> **NOTE** Deadlines, cancellation and the refund of the advance are not stated in the question; they are reasonable assumptions added so that every state has a way out (no "dead" states).

Answer. Package Selected (self-loop customize [size>10]/adjust price) → book(advance) → Booked → remaining-Paid → Confirming Components → choice → Confirmed (with refundDue if an activity is unavailable) → On Tour → [refundDue] Refund Pending → Closed; cancel/deadline → Cancelled.

2018–2019 • Q1(a) CSE307 • 06/11/2019 • R008 **MARKS 14**

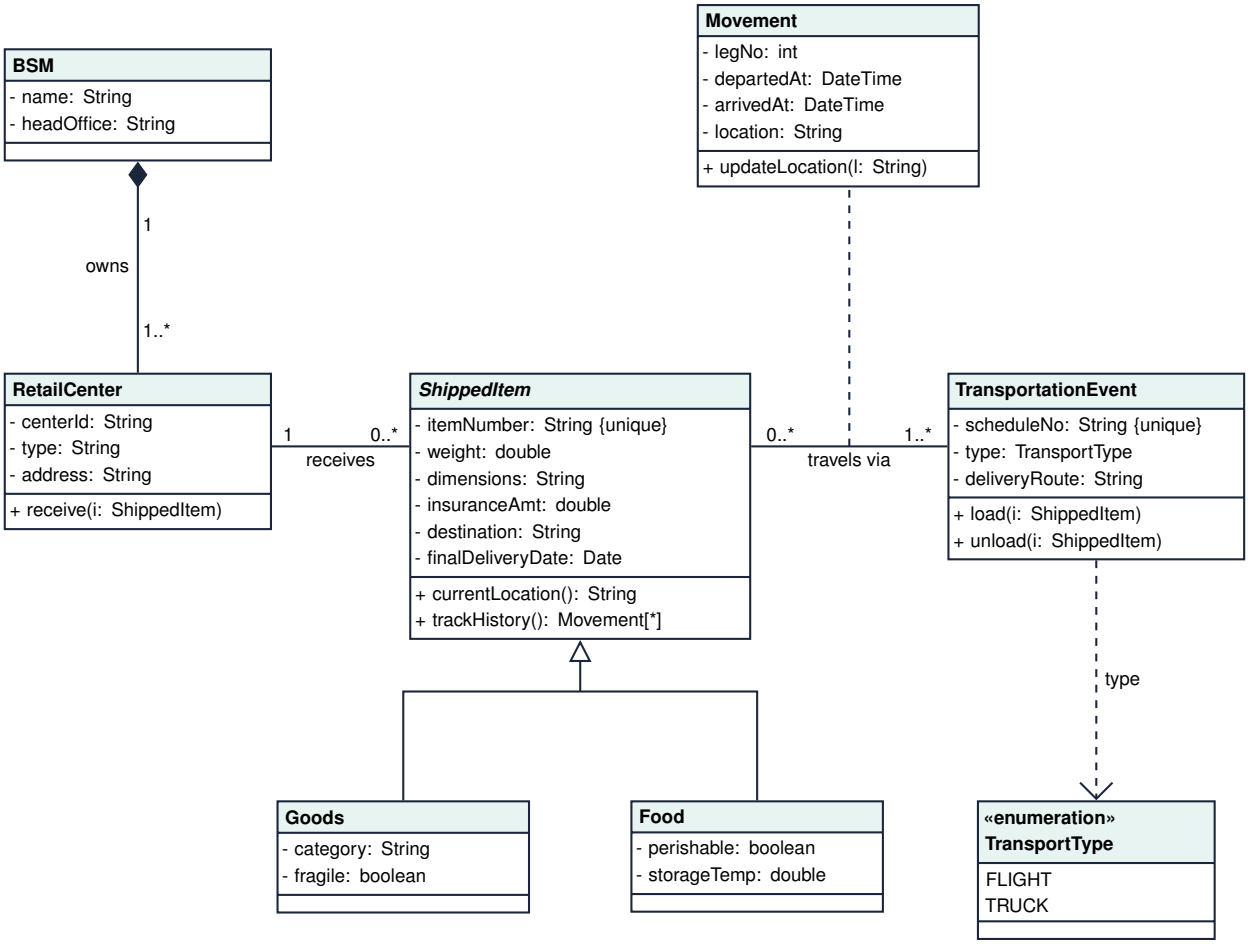
Bangladesh Shipping Management (BSM) prides itself on having up-to-date information on the processing and current locations of each shipped item. To do this, BSM relies on a company-wide information system. Shipped items are the heart of the BSM product tracking information system. Shipped items may be of two types, goods and food. They can be characterized by item number (unique), weight, dimensions, insurance amount, destination, and final delivery date. Shipped items are received into the BSM system at a single retail center. Shipped items make their way to their destination via one or more standard BSM transportation events (i.e. flights, truck deliveries). These transportation events are characterized by a unique schedule Number, a type (e.g., flight, truck), and a delivery Route.
Design a Class diagram to implement the scenario described above.

SOLUTION

Classes and their source in the text

- **ShippedItem** – "the heart of the system"; attributes item number {unique}, weight, dimensions, insurance amount, destination, final delivery date. Since every item is *either* goods *or* food, ShippedItem is **abstract** with subclasses **Goods** and **Food** (**generalization**).
- **RetailCenter** – "received … at a single retail center" ⇒ each item is associated with exactly 1 retail center; a center receives many items.
- **TransportationEvent** – unique schedule number, type, delivery route. Its *type* has a fixed set of values ⇒ enumeration **TransportType** {FLIGHT, TRUCK}.
- **Movement** (association class) – an item travels on "one or more" events and an event carries many items. The facts belonging to one *(item, event)* pair – leg number, departure and arrival time, current location – are what gives BSM "current locations of each shipped item".
- **BSM** – the company that owns its retail centers.

Class diagram



Modelling decisions explained

(i) Generalization (Goods, Food → ShippedItem): common attributes are written once in the abstract superclass; each subtype adds only its own data (e.g. perishable/storage temperature for food).

(ii) **`RetailCenter 1 - 0..* ShippedItem`**: "received at a *single* retail center" fixes the `1`.

(iii) **`ShippedItem 0..* - 1..* TransportationEvent`**: "via *one or more*" events fixes `1..*` on the event side; a flight or truck carries many items (`0..*`).

(iv) Association class Movement: the location/time data belongs neither to the item alone nor to the event alone but to the pair – exactly the lecture's `Job` example. `currentLocation()` returns the location of the latest Movement.

(v) Uniqueness of item number and schedule number is shown with `{unique}`.

Answer. Abstract ShippedItem (unique itemNumber, weight, dimensions, insurance, destination, delivery date) with subclasses Goods and Food; RetailCenter 1 – 0..* ShippedItem; ShippedItem 0..* – 1..* TransportationEvent (unique scheduleNo, type: TransportType, route) with association class Movement holding the current location.

2018–2019 • Q1(a) CSE325 • 20/12/2021 • C325-1819-1a **MARKS 22**

In an office XYZ, the employees can apply for loans which are approved by the management if all the requirements are met. The requirements include age, certain years of experience, clean disciplinary record, etc. The loans can be created in the system by the user specifying relevant parameters such as interest rate, years to repay, maximum allowed amount based on salary of the applicant, etc. These conditions may change over time.
Design a Class diagram to implement the scenario described above.

SOLUTION

Step 1 – Reading the scenario for classes

- **Employee** – applies for loans; age, experience and salary are needed for the requirements.
- **Management** – approves; modelled as **Manager**, a specialised employee.
- **the user** who "creates loans in the system" – an administrative employee; modelled as **LoanOfficer**.
- **disciplinary record** – has its own data (case, date, resolved?) → class **DisciplinaryRecord**.
- "the loans can be created … by specifying parameters" – this is a *loan product*, not an individual's loan → **LoanScheme** (e.g. House-building loan, Motor-cycle loan).
- the employee's request → **LoanApplication**; the sanctioned money → **Loan**, repaid in **Installment**s.
- **requirements** (age, experience, clean record, "etc.") → **EligibilityRule**.

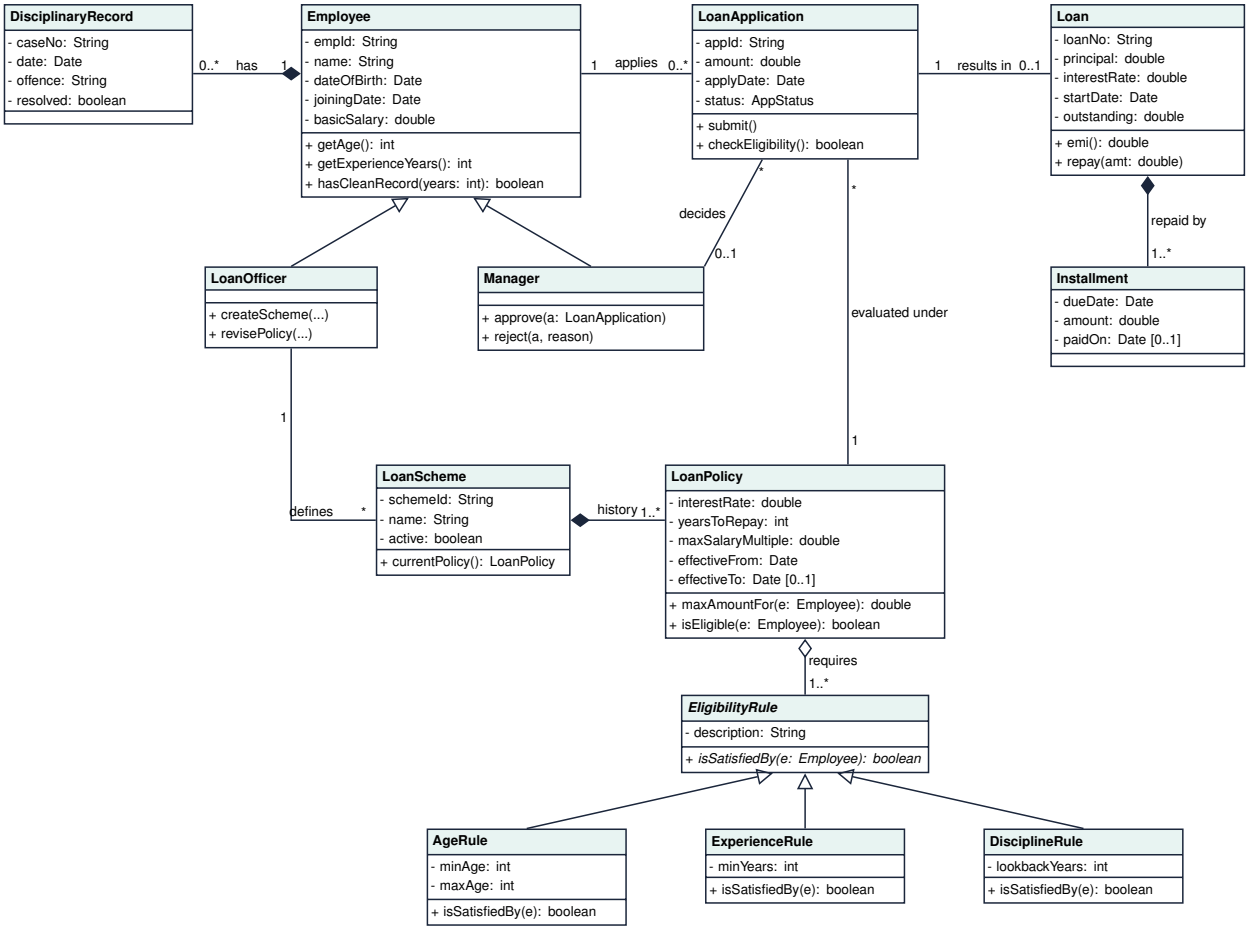
Step 2 – The key design problem: "these conditions may change over time"

Two kinds of change must be absorbed *without changing code or corrupting history*:

(i) **Parameter values change** (interest rate 7% → 9%, max amount 10 → 12 months' salary). If the values were attributes of `LoanScheme`, editing them would silently change the terms of loans already sanctioned. So the values live in a separate, *versioned* class **LoanPolicy** with `effectiveFrom/effectiveTo`; a scheme *owns* a history of policies (composition 1 – 1..*), exactly one of which is current. Each `Loan` copies the rate it was sanctioned under.

(ii) **The set of requirements changes** ("etc." – tomorrow a "no other running loan" rule may be added). The requirements are therefore *objects*, not `if`-statements: an abstract class **EligibilityRule** with `isSatisfiedBy(e: Employee)` and one subclass per kind of rule (`AgeRule`, `ExperienceRule`, `DisciplineRule`). A policy aggregates any number of rules with their own thresholds. Adding a new kind of rule = adding a subclass (Open–Closed Principle); changing a threshold = data entry by the loan officer.

Step 3 – Class diagram



NOTE `AppStatus` is an enumeration {SUBMITTED, ELIGIBLE, REJECTED, APPROVED}. "Management" is assumed to be represented by employees with the `Manager` role; the "user" who creates loans by a `LoanOfficer` (e.g. an accounts officer). Both are employees and may themselves apply for loans – hence generalisation.

Step 4 – Explaining the modelling decisions

- **Composition** `Employee` → `DisciplinaryRecord`: a record has no meaning without its employee. `hasCleanRecord(years)` lets `DisciplineRule` ask the employee instead of reading his records directly (information expert).
- **LoanApplication vs. Loan**: an application may be rejected (no loan, multiplicity `0..1`); only an approved one *results in* a `Loan`, which carries its own `interestRate` copied from the policy at sanction time, so later policy revisions do not affect it.
- **LoanScheme composed of a LoanPolicy history**: the scheme is stable (name, id); every revision of the parameters is a new `LoanPolicy` object with an effective period – "conditions may change over time" is handled by *adding data*, not editing old data.
- **Aggregation** `LoanPolicy` → `EligibilityRule` (hollow diamond): a rule object (e.g. "age 25–55") can be reused by several policies.
- **Abstract class + polymorphism**: `LoanPolicy.isEligible(e)` simply asks every rule `isSatisfiedBy(e)`; it never needs to know which concrete rules exist.
- `maxAmountFor(e)` = `maxSalaryMultiple` × `e.basicSalary` implements "maximum allowed amount based on salary of the applicant".

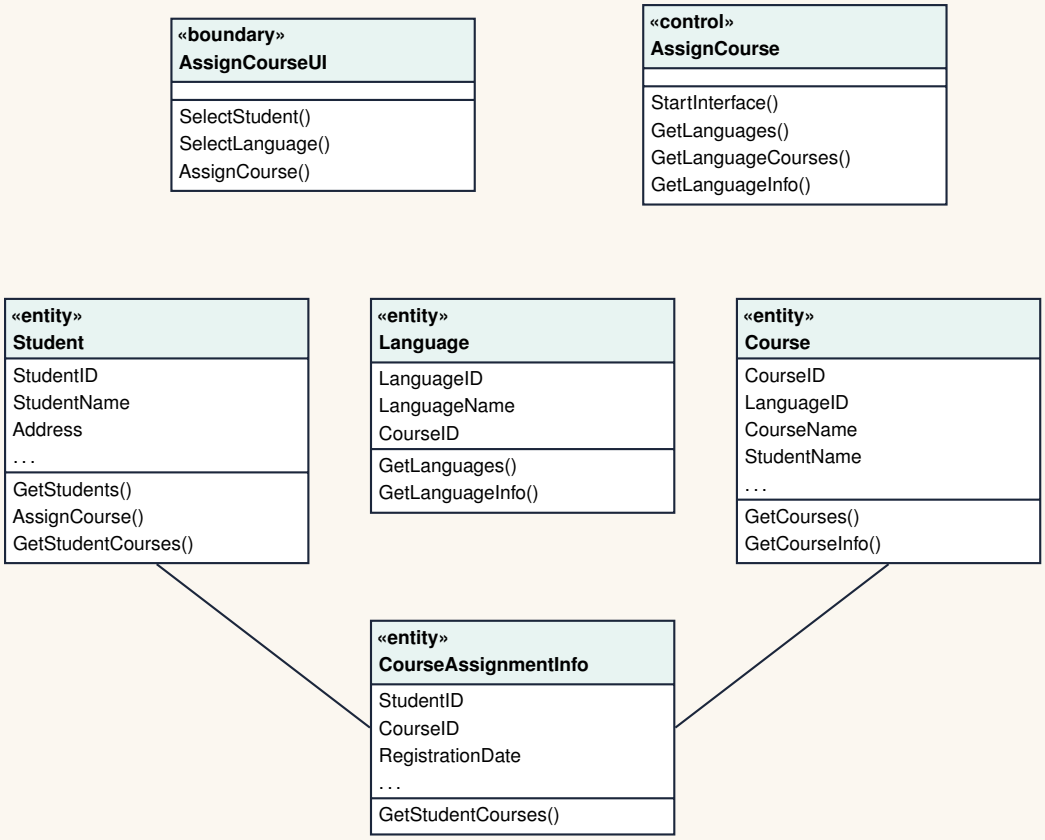
Step 5 – How the classes work together (code sketch)

```
abstract class EligibilityRule {
    protected String description;
    abstract boolean isSatisfiedBy(Employee e);
}
class AgeRule extends EligibilityRule {
    private int minAge, maxAge;
    boolean isSatisfiedBy(Employee e) { return e.getAge() >= minAge && e.getAge() <= maxAge; }
}
class ExperienceRule extends EligibilityRule {
    private int minYears;
    boolean isSatisfiedBy(Employee e) { return e.getExperienceYears() >= minYears; }
}
class DisciplineRule extends EligibilityRule {
    private int lookbackYears;
    boolean isSatisfiedBy(Employee e) { return e.hasCleanRecord(lookbackYears); }
}
class LoanPolicy {
    private double interestRate, maxSalaryMultiple; private int yearsToRepay;
    private List<EligibilityRule> rules = new ArrayList<>();
    double maxAmountFor(Employee e) { return maxSalaryMultiple * e.getBasicSalary(); }
    boolean isEligible(Employee e) {
        for (EligibilityRule r : rules) if (!r.isSatisfiedBy(e)) return false;
        return true;
    }
}
class LoanApplication {
    private Employee applicant; private LoanPolicy policy; private double amount;
    boolean checkEligibility() {
        return policy.isEligible(applicant) && amount <= policy.maxAmountFor(applicant);
    }
}
```

Answer. Classes: `Employee` (with `Manager`, `LoanOfficer` subclasses) composed of `DisciplinaryRecords`; `Employee` 1 – 0..* `LoanApplication` 1 – 0..1 `Loan` composed of `Installments`; `LoanScheme` composed of 1..* versioned `LoanPolicy` (interest rate, years, salary multiple, effective dates) ◇– 1..* abstract `EligibilityRule` ← `AgeRule`, `ExperienceRule`, `DisciplineRule`. Changing conditions = new policy version / new rule objects, no code change.

2017–2018 • Q1(a) CSE325 • 10/04/2019 • C325-1718-1a **MARKS 20**

Consider the following scenario:
A language teacher wants to register a particular student for an online French course. After he logs in, he is shown a list of students. When he selects a particular student, a list of languages is displayed. After he selects a language (say, French), he is shown a list of course on French that the selected student has not taken yet. When he selects a course, the course assignment is complete.
Make a list of the inconsistencies in the class diagram given below. Don't redraw the diagram.



SOLUTION

How to check a class diagram against a scenario

A robustness-style (boundary–control–entity) class diagram is consistent with a scenario when (1) every step of the scenario has an operation that performs it in the right class, (2) every class that has to send a message to another class is *linked* to it by an association, (3) every attribute really belongs to the concept named by its class (no repeated or misplaced data), and (4) the associations carry correct multiplicities. Walking through the scenario step by step: *log in* → *show students* → *select student* → *show languages* → *select language* → *show the language's courses the student has not taken* → *select course* → *record the assignment*.

(A) Inconsistencies in relationships

(i) No association between the boundary and the control class. `AssignCourseUI` has to forward every selection to `AssignCourse`, but the two classes are not linked.

(ii) The control class is not linked to any entity. `AssignCourse` must read students, languages, courses and must create the assignment, yet it has no association with `Student`, `Language`, `Course` or `CourseAssignmentInfo`.

(iii) `Language` and `Course` are unrelated. The scenario says that a language (French) has a list of courses, and `Course` even stores `LanguageID`, but the `Language 1 - 1..* Course` association is missing.

(iv) No multiplicities anywhere. `Student`–`CourseAssignmentInfo` should be `1 - 0..*` and `Course`–`CourseAssignmentInfo` should be `1 - 0..*` (a student takes many courses, a course is taken by many students). Equivalently, `CourseAssignmentInfo` is an *association class* on a `* - *` association between `Student` and `Course`.

(B) Inconsistencies in attributes

(v) **`Language.CourseID` is wrong.** One language has many courses, so a single-valued `CourseID` in `Language` cannot represent it; the link is already given by `Course.LanguageID` (the foreign key belongs on the "many" side).

(vi) **`Course.StudentName` is wrong.** A course is taken by many students and the student's name is a property of `Student`; who took which course is exactly what `CourseAssignmentInfo` records. Keeping it in `Course` is redundant and inconsistent.

(vii) **No class for the teacher/login.** The scenario starts with the teacher logging in, but there is no `Teacher` (user) entity holding `TeacherID`/password, so the login cannot be checked; `CourseAssignmentInfo` also cannot record which teacher made the assignment.

(C) Inconsistencies in operations

(viii) **Login is not supported.** Neither the boundary (`Login()`) nor the control (`ValidateTeacher()`) has an operation for the first step of the scenario.

(ix) **The control class cannot fetch the student list.** After login a list of students must be shown, but `AssignCourse` has no `GetStudents()` (only `StartInterface()`).

(x) **The control class has no `AssignCourse()`.** The boundary has `AssignCourse()`, but there is no control operation that it can delegate to, so the final step (completing the assignment) is not realised.

(xi) **"Courses not taken yet" is not supported.** `GetLanguageCourses()` (control) and `GetCourses()` (`Course`) take no parameters, so they can neither filter by the selected language nor exclude the courses that the selected student has already taken; an operation such as `GetUntakenCourses(studentID, languageID)` is needed.

(xii) **`GetLanguageInfo()` in the control class is not needed** by any step of the scenario (the teacher only needs the list of languages), while needed operations are missing.

(xiii) **Misplaced responsibility: `Student.AssignCourse()`.** Creating an assignment creates a `CourseAssignmentInfo` object (with `RegistrationDate`); that class has no create / `AddAssignment()` operation, while `Student` has one that it cannot implement with its own data.

(xiv) **Duplicated responsibility: `GetStudentCourses()`** appears both in `Student` and in `CourseAssignmentInfo`; only the class that owns the data (`CourseAssignmentInfo`) should have it.

(xv) **Boundary class has no display operations.** The UI has to *show* the students, the languages and the filtered courses (`DisplayStudents()`, `DisplayLanguages()`, `DisplayCourses()`) and confirm the assignment, but only the three select operations are given.

(xvi) **Operations lack parameters.** `GetStudentCourses()`, `GetCourseInfo()`, `GetLanguageInfo()`, `AssignCourse()` etc. need arguments such as `studentID`, `courseID`, `languageID`; without them the operations are ambiguous.

NOTE In the scanned figure the boundary and control classes sit inside a frame and only the two lines `Student`–`CourseAssignmentInfo` and `Course`–`CourseAssignmentInfo` are visible; the frame lines are read as a border, not as associations. If the horizontal line under the two upper classes is meant to be a link between them, item 1 drops out but the other items remain.

Answer. Main inconsistencies: (1) missing associations (UI–control, control–entities, Language–Course) and missing multiplicities (Student/Course 1 – 0..* CourseAssignmentInfo); (2) wrong attributes `Language.CourseID` and `Course.StudentName`, no Teacher/login class; (3) missing operations for login, getting students, getting the untaken courses of the selected language and completing the assignment in the control class; `AssignCourse()` misplaced in `Student`, `GetStudentCourses()` duplicated, unnecessary `GetLanguageInfo()` in the control, no display operations in the UI and no parameters on the operations.

2017–2018 • Q3(b) CSE325 • 10/04/2019 • C325-1718-3b **MARKS 7 + 4**

(i) Name the types of structural things in UML.

(ii) Name four types of relationships in UML class diagram.

SOLUTION

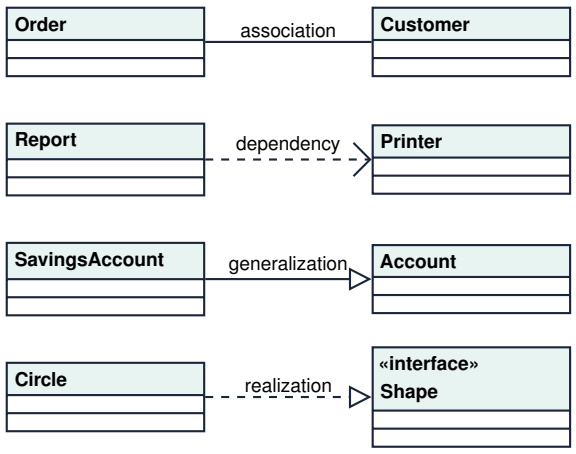
Similar to card R025.2 ("four kinds of things"), but here only the *structural* things are asked together with the class-diagram relationships.

(i) Structural things

Structural things are the "nouns" of a UML model – the mostly static parts that represent conceptual or physical elements. There are seven of them:

Structural thing	Meaning (notation)
Class	set of objects with the same attributes, operations, relationships (3-compartment rectangle)
Interface	named set of operations that specifies a service of a class/component («interface» or lollipop)
Collaboration	society of elements that work together to give cooperative behaviour (dashed ellipse)
Use case	set of action sequences that yields an observable result for an actor (ellipse)
Active class	class whose objects own a thread/process of control (class box with double side lines)
Component	modular, replaceable physical part of a system that realises interfaces (box with component icon)
Node	physical computational resource at run time, e.g. a server (3-D box)

(ii) Four relationships in a class diagram



- **Dependency** – a "uses" relationship: a change in one class may affect the other (dashed arrow).
- **Association** – a structural link between objects (solid line, with multiplicities and roles); its special forms are *aggregation* (open diamond) and *composition* (filled diamond).
- **Generalization** – "is-a" relationship between a subclass and a superclass (hollow triangle).
- **Realization** – a class implements an interface (dashed line with hollow triangle).

Answer. (i) Class, interface, collaboration, use case, active class, component, node. (ii) Dependency, association (incl. aggregation/composition), generalization, realization.

2016–2017 • Q1(a) CSE307 • 15/07/2017 • R014.1 MARKS 12

Discuss some extensibility mechanisms of UML diagram.

SOLUTION

Why UML needs extensibility

UML is a general-purpose language; no fixed notation can express everything every domain needs (e.g. "this class is a database table", "this class was written by Rahim, version 2.1", "balance can never be negative"). Instead of changing the language, UML provides three **extensibility mechanisms** that let us extend it in a controlled way. A coherent set of them for one domain is packaged as a **profile**.

1. Stereotypes – extend the vocabulary

A stereotype creates a new kind of model element from an existing one, with its own meaning (and optionally its own icon). It is written in guillemets above the element's name.

- Examples: «interface», «enumeration», «entity», «boundary», «control», «table», «include»/«extend» on use-case dependencies.
- Effect: a reader immediately knows how the element is to be treated, without new symbols.

2. Tagged values – extend the properties

A tagged value is a `name = value` pair, written in braces, that adds new information to an element which UML itself has no slot for.

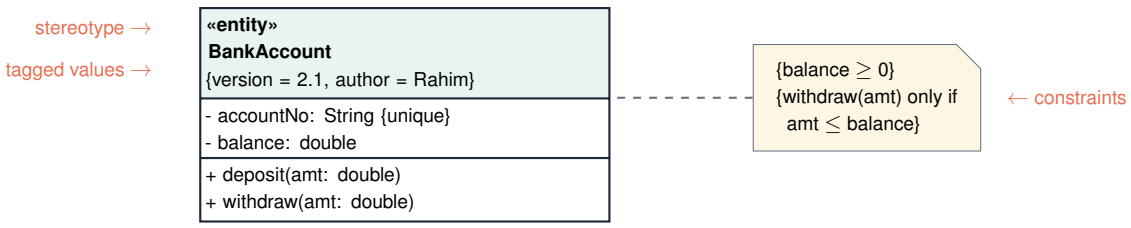
- Examples: `{version = 2.1}`, `{author = Rahim}`, `{persistent = true}`, `{location = server}`.
- Effect: useful for code generation, configuration management or documentation.

3. Constraints – extend the semantics

A constraint is a condition or rule, written in braces (in natural language or OCL), that must always hold for the element. It can be placed next to the element or in an attached note.

- Examples: `{balance >= 0}`, `{ordered}`, `{unique}`, `{xor}` between two associations, `{patient.gender = ward.gender}`.
- Effect: adds rules that the graphical notation cannot show.

All three on one class



Mechanism	What it extends	Notation
Stereotype	vocabulary – new kinds of building blocks	«name»
Tagged value	properties – new information on an element	{tag = value}
Constraint	semantics – new rules / conditions	{condition} or a note

Answer. UML's extensibility mechanisms are **stereotypes** (new kinds of elements, «...»), **tagged values** (new properties, {tag = value}) and **constraints** (new rules, {condition}); a domain-specific set of them forms a profile.

2016–2017 • Q2(d) CSE307 • 15/07/2017 • R016.3 MARKS 13

The following is a narrative description of the business process of a bank:
There are two types of users in a Bank: Normal user and Foreign User. Both users can create account, deposit money and withdraw money. For money withdrawal there are certain rules. A user can make a withdrawal of maximum 3,00,000 taka per day. At each transaction the highest limit of withdrawal is 1,00,000 taka. A user can make a maximum of 5 withdrawals per day. After every three months, interest is added to the account balance. If an account balance reaches 20,00,000 taka, for every deposit to that account 2% bonus is applied. The foreign user has one extra capability. He can convert his taka to foreign currency.
The use cases of this scenario are: (i) Create account (ii) Deposit money (iii) Withdraw money (iv) Update balance (v) Calculate bonus (vi) Calculate interest and (vii) Convert to foreign currency.

(d) List all the entity classes of this system. Draw state chart diagram of the most important entity class in your consideration.

SOLUTION

Background: entity classes and statecharts

An **entity class** represents information that the system must *remember* (it outlives a single use case). A **statechart (state diagram)** models the life of *one* object of a class: **states** (conditions during the object's life in which it satisfies some condition, performs some activity or waits for an event), **transitions** labelled `event [guard] / action`, an initial pseudostate and (optionally) a final state. In the lecture's words, the current state of an object is determined by the current values of its attributes and its links — so we look for the class whose *attribute values change the rules* that apply to it.

Step 1 – Entity classes

Nouns in the narrative that must be stored persistently:

Entity class	Key attributes	Why it is needed
`User` (abstract)	userId, name, address	owner of accounts
`NormalUser`	(inherits User)	ordinary customer
`ForeignUser`	passportNo, nationality	extra capability: currency conversion
`Account`	accountNo, balance, openDate, withdrawCountToday, withdrawnToday, lastInterestDate	the money and the per-day limits
`Transaction` (`Deposit`, `Withdrawal`)	txnId, amount, dateTime, type	history; needed to check daily limits
`CurrencyConversion`	fromAmount, currency, rate, dateTime	record of a conversion
`InterestRate` / `ExchangeRate`	rate, effectiveDate	used by interest / conversion

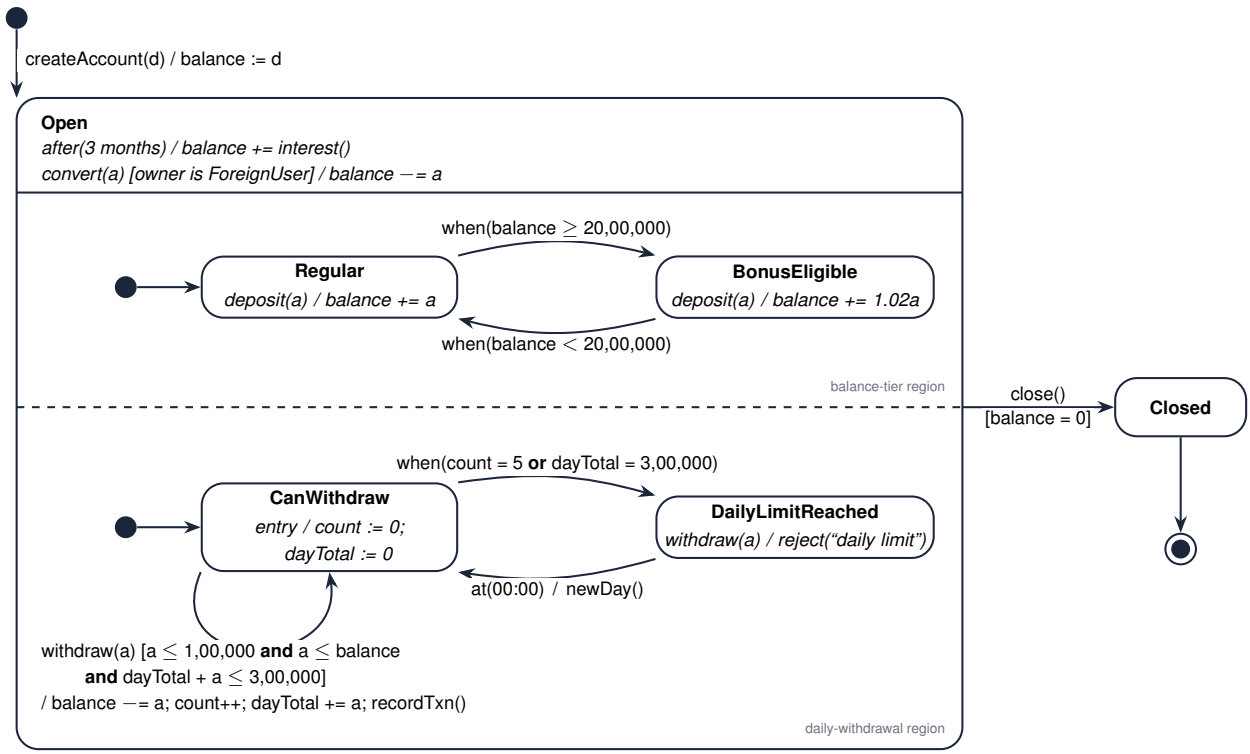
Step 2 – Choosing the most important entity

Almost every use case (deposit, withdraw, update balance, bonus, interest, conversion) reads or changes the **Account**. Moreover, the *rules that apply to it depend on its own attribute values*: a deposit gives 2% bonus only when balance $\geq$ 20,00,000, and a withdrawal is allowed only while today's count $<$ 5 and today's total $<$ 3,00,000. Attribute-dependent behaviour is exactly what a statechart captures, so `Account` is modelled.

Step 3 – Identifying states, events and concurrency

- Two *independent* aspects change over time: the **balance tier** (Regular / Bonus-eligible) and the **daily withdrawal status** (Can withdraw / Daily limit reached). They change independently, so they are modelled as two **concurrent regions** of the composite state Open.
- **Change events** `when(balance >= 20,00,000)` and its opposite move between tiers.
- A **call event** `withdraw(a)` with a **guard** enforcing the per-transaction (1,00,000), daily (3,00,000) and balance limits; a change event fires when 5 withdrawals or 3,00,000 taka is reached.
- An **elapsed-time event** `after(3 months)` adds interest; a time event `at(00:00)` (new day) resets the daily counters.
- `convert(a)` is only accepted for a foreign user (guard on the owner type).

Step 4 – Statechart of `Account`



Explanation of the modelling decisions

- **Composite state Open with two concurrent regions.** While open, the account is *simultaneously* in one tier state and one withdrawal state (e.g. BonusEligible *and* DailyLimitReached). Modelling them as a flat machine would need 2×2 combined states; concurrency keeps the diagram small.
- **Internal transitions** of Open (`after(3 months)`, `convert(a)`) do not change the substates, so they are written inside the state, not as arrows.
- The **2% bonus** is an action inside BonusEligible; the tier changes by *change events* whenever the balance crosses 20,00,000 (after a deposit, interest, withdrawal or conversion).
- The **guard** on the self-transition of CanWithdraw encodes the per-transaction limit, the daily-amount limit and sufficient balance; the **5-withdrawals** rule is enforced by the change event to DailyLimitReached, which leaves only at the time event `at(00:00)`. Re-entering CanWithdraw runs its `entry` action and resets the counters.
- **Closing** leaves the whole composite (both regions) at once; a guard requires a zero balance.

Answer. Entity classes: User (NormalUser, ForeignUser), Account, Transaction (Deposit/Withdrawal), CurrencyConversion, InterestRate/ExchangeRate. The key entity is **Account**; its statechart has a composite state Open with concurrent regions {Regular ↔ BonusEligible} and {CanWithdraw ↔ DailyLimitReached}, interest as `after(3 months)` and conversion as guarded internal transitions, then Closed.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2014–2015 • Q5(c) CSE307 • 04/02/2016 • R020.2 **MARKS 10**

What types of UML diagrams do exits? Name each diagram type and describe its main purpose. Classify each of the UML diagram types as static or dynamic.

2010–2011 • Q1(b) CSE307 • 19/03/2012 • R042.2 **MARKS 8**

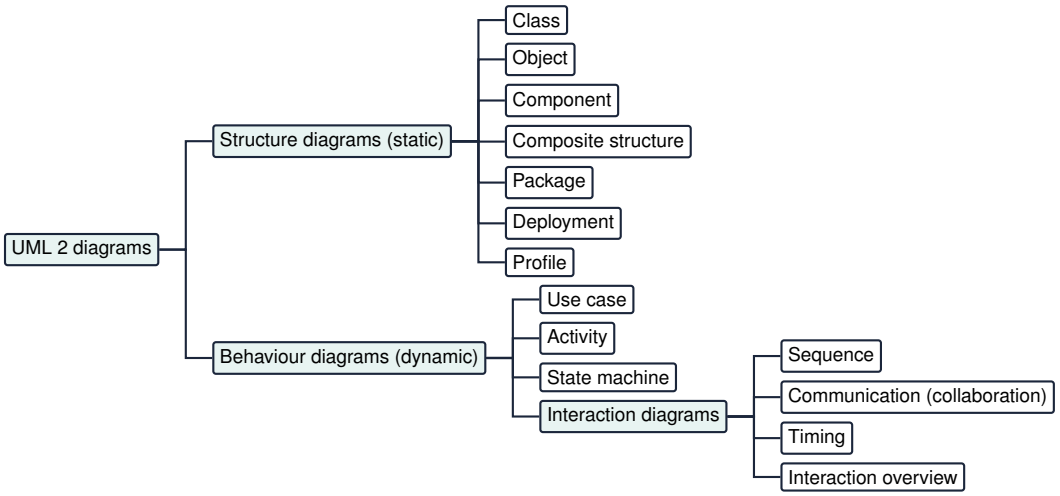
List all behavioral UML diagrams and describe the purpose of each of them.

SOLUTION

Classification

UML 2.x defines **14 diagram types** in two families:

- **Structure diagrams** – **static**: show the elements of the system and how they are related, independent of time.
- **Behaviour diagrams** – **dynamic**: show how the system or its objects behave and change over time. A sub-family, **interaction diagrams**, focuses on messages between objects.



Purpose of each diagram

Diagram	Kind	Main purpose
Class	static	classes, attributes, operations and their relationships – the logical structure of the system
Object	static	a snapshot of objects and links at one moment (an instance of a class diagram)
Component	static	the modular, replaceable parts (components) and their provided/required interfaces
Composite structure	static	internal structure of a class: its parts, ports and connectors
Package	static	organisation of model elements into packages and dependencies between packages
Deployment	static	physical nodes (hardware, execution environments) and the artifacts deployed on them
Profile	static	defines stereotypes, tagged values and constraints to extend UML for a domain
Use case	dynamic	functional requirements: actors, use cases and include/extend relationships
Activity	dynamic	workflow / business process or algorithm as actions, decisions, forks and joins
State machine	dynamic	life cycle of one object: states, events, guards and actions on transitions
Sequence	dynamic	time-ordered messages between objects (lifelines) for one scenario
Communication	dynamic	the same interaction emphasising links between objects; messages are numbered
Timing	dynamic	changes of state/condition of lifelines along a precise time axis
Interaction overview	dynamic	an activity-like flow whose nodes are interactions – the overview of many scenarios

Answer to R042.2 (behavioural diagrams only)

The seven behavioural diagrams are: **use case, activity, state machine**, and the four interaction diagrams **sequence, communication (collaboration), timing, interaction overview**; their purposes are the "dynamic" rows of the table above.

Answer. 7 structure (static) diagrams: class, object, component, composite structure, package, deployment, profile. 7 behaviour (dynamic) diagrams: use case, activity, state machine, sequence, communication, timing, interaction overview.

2014–2015 • Q8 CSE307 • 04/02/2016 • R022.1 **MARKS** $12\frac{2}{3}$ **+ (12+10)**

(a) Describe different types of classes used in a class diagram with example.
(b) Consider, you have a collection of several hundred music CDs. Your friends love to borrow them, often borrowing several at the same time. You don't mind lending the CDs, but it is becoming difficult for you to keep track of which friend borrowed which CD. The difficulty is increased by the fact that you have more than one copy of some CDs. Also you are thinking of introducing a daily fee per CD loaned. For these reasons you would like to build a database, which you could query to find out how many CDs a given friend has; how long the friend has had them and how much he or she owes; who has a particular CD and so on. Draw class and

sequence diagram for the above scenario.

SOLUTION

(a) Types of classes in a class diagram

A class is drawn as a rectangle with three compartments (name, attributes, operations). Depending on its nature, the class is shown with a different notation:

(i) **Concrete (ordinary) class** – can be instantiated; name in plain bold. *e.g.* `Customer`.

(ii) **Abstract class** – cannot be instantiated; it only gathers common attributes/operations for its subclasses. Name (and abstract operations) written in *italics*. *e.g. Shape* with abstract *area()*.

(iii) **Interface** – a set of operation signatures only; written with «interface» above the name. Realised by classes with a dashed line and hollow triangle. *e.g.* «interface» `Drawable`. All its operations must be shown.

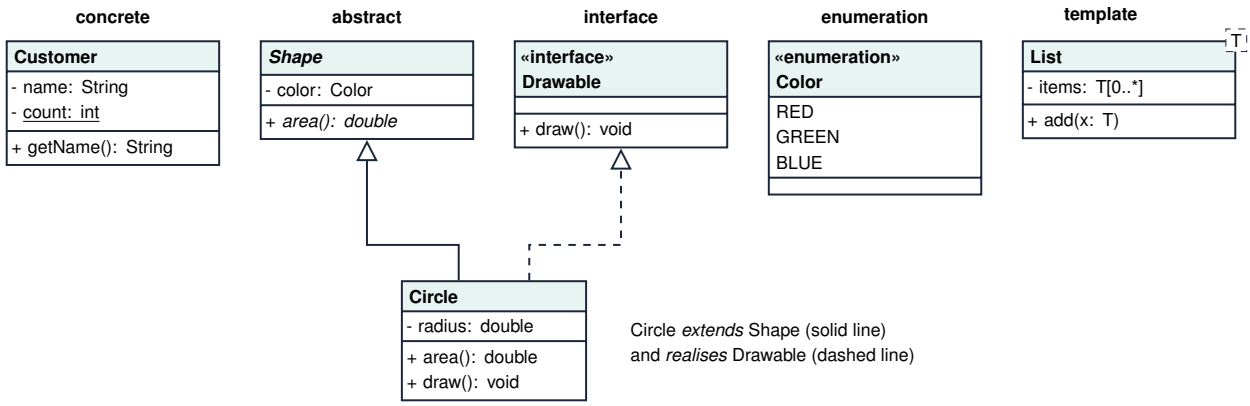
(iv) **Enumeration** – a data type whose values are a fixed list of literals; stereotype «enumeration». *e.g.* `Color {RED, GREEN, BLUE}`.

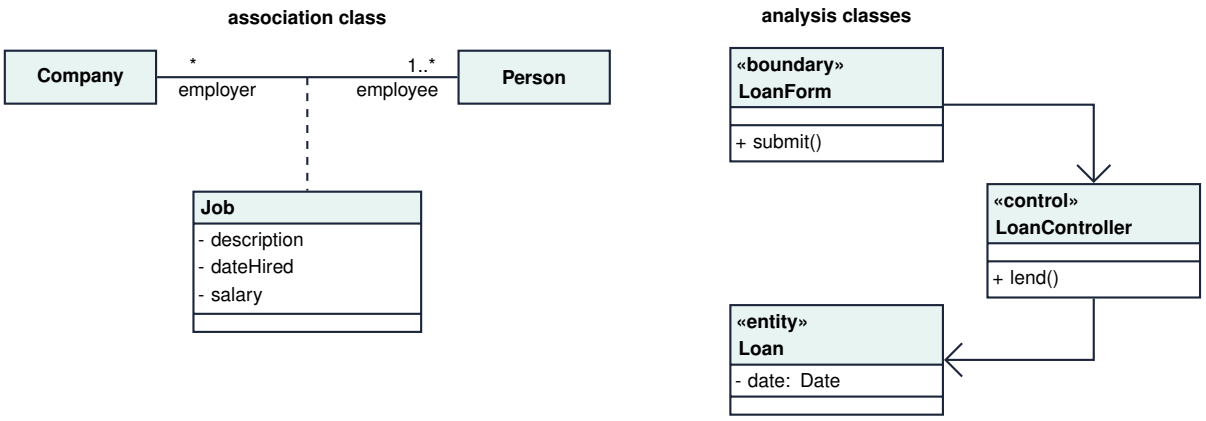
(v) **Association class** – a class that holds properties of an association itself and is attached to the association line with a dashed line (lecture example: `Job` between `Company` and `Person`, holding `salary`, `dateHired`).

(vi) **Parameterised (template / generic) class** – a class with type parameters shown in a dashed box at its top-right corner, *e.g.* `List<T>`.

(vii) **Analysis classes** (stereotypes used when building the first class model from use cases): «entity» classes hold persistent business data (`Loan`), «boundary» classes interact with actors (`LoanForm`), «control» classes coordinate a use case (`LoanController`).

(viii) **Static (utility) class members** – attributes/operations that belong to the class rather than to objects are underlined (*e.g.* `totalStudents`).

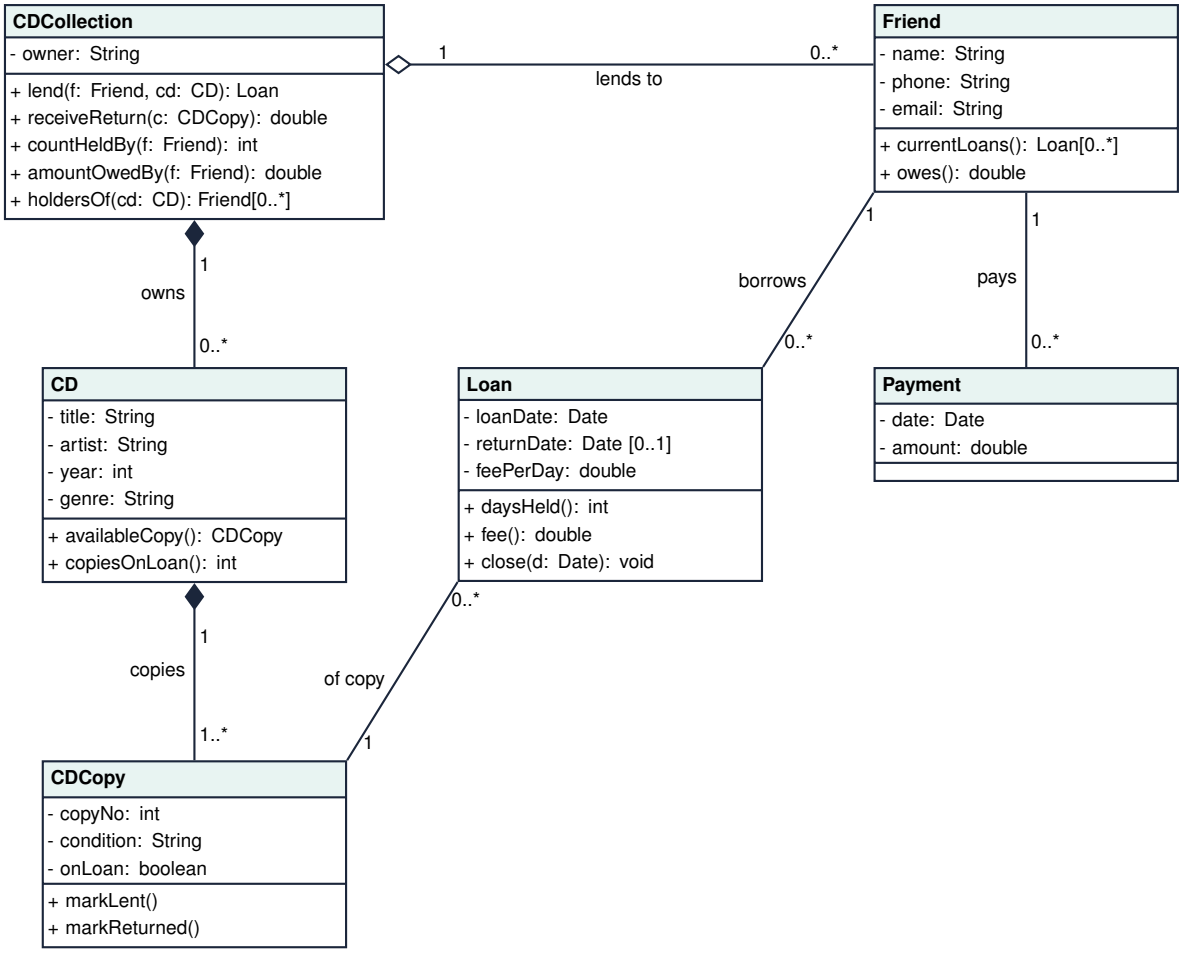




(b) Identifying the classes for the CD-loan database

Nouns in the description and what we must be able to *query* give the candidate classes:

- **CD** – the title (album) with artist, year, genre. It is a *description*.
- **CDCopy** – "more than one copy of some CDs": the physical disc that is actually lent. Separating `CD` (description) from `CDCopy` (physical item) is the key modelling decision: we lend copies, but friends ask for titles.
- **Friend** – the borrower (name, phone).
- **Loan** – one lending of one copy to one friend: `loanDate`, `returnDate`, `feePerDay`. It answers "how long the friend has had them" and "how much he or she owes". Because the same friend may borrow the same copy many times, `Loan` is modelled as a normal class (one object per lending) instead of an association class.
- **Payment** – money the friend pays back, so that *owed* = $\sum$fees $-$ $\sum$payments.
- **CDCollection** – the database itself; it owns the CDs and offers the query operations.

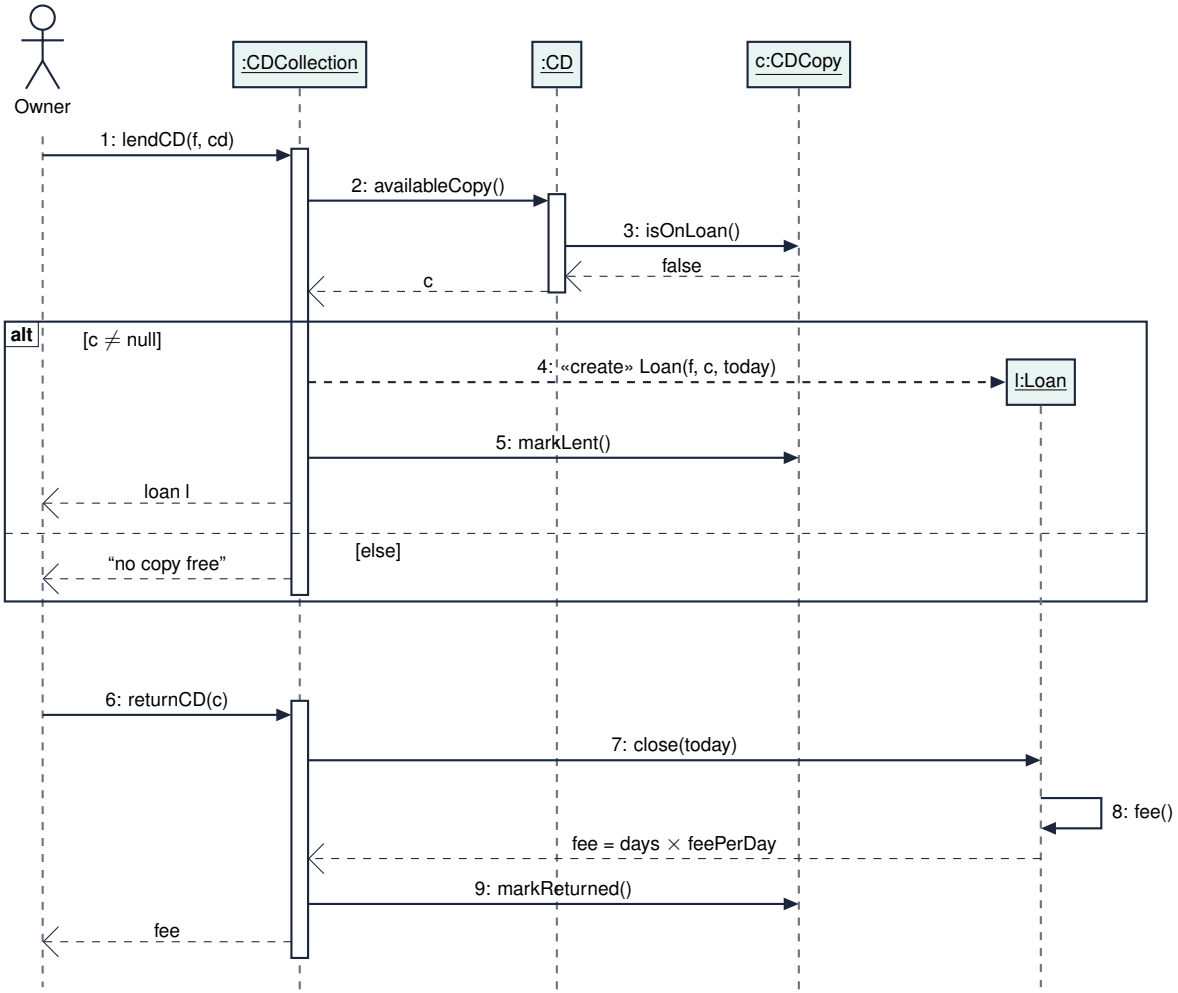


Modelling decisions

- **CDCollection ♦ CD ♦ CDCopy** – composition: a copy record makes no sense without its CD, and removing a CD from the collection removes its copies.
- **Friend 1 – 0..* Loan** – "often borrowing several at the same time"; every loan has exactly one borrower.
- **Loan 0..* – 1 CDCopy** – a copy is lent many times over its life, but at any moment at most one loan is open (`returnDate` empty); this is checked through `onLoan`.
- Queries are realised by navigation: *how many CDs a friend has* = open loans of the friend; *how long* = `Loan.daysHeld()`; *how much owed* = ∑`Loan.fee()` − ∑`Payment.amount`; *who has a CD* = friends of the open loans of the copies of the CD.

(b) Sequence diagram: lending a CD and later returning it

Scenario: the owner lends an available copy of a CD to a friend; later the friend returns it and the fee is computed.



- Message 4 is a **create message** (dashed arrow to the head of the new lifeline): the `Loan` object starts existing only when a copy has been found.
- The `alt` fragment shows the two outcomes of the guard `[c ≠ null]`.
- Message 8 is a self-message: the loan computes its own fee from `daysHeld()` and `feePerDay`.

Answer. (a) Concrete, abstract (italic), interface («interface»), enumeration, association class, template class, and analysis classes «entity»/«boundary»/«control». (b) Classes `CDCollection`, `CD`, `CDCopy`, `Friend`, `Loan`, `Payment`; the key decisions are CD vs. CDCopy and a `Loan` object per lending (holds dates and fee).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2013–2014 • Q5(b) CSE307 • 08/08/2015 • R025.2 **MARKS 12**

What are the four kinds of things used in UML?

2012–2013 • Q7(d) CSE307 • 17/05/2014 • R033.5 **MARKS $4\frac{2}{3}$**

What are the different kinds of *things* in UML?

SOLUTION

Building blocks of UML

The vocabulary of UML has three building blocks: **things** (the abstractions), **relationships** (dependency, association, generalization, realization – which tie things together) and **diagrams** (which group interesting collections of things). There are **four kinds of things**:

1. Structural things – the "nouns" (mostly static parts of a model)

- **Class** – a set of objects with the same attributes, operations and relationships.
- **Interface** – a collection of operations that specify a service of a class/component.
- **Collaboration** – a society of elements that work together to provide some behaviour (dashed ellipse).
- **Use case** – a sequence of actions that yields a result of value to an actor (ellipse).
- **Active class** – a class whose objects own a thread of control (thick border).
- **Component** – a replaceable physical/modular part of the system (e.g. a .jar, DLL).
- **Node** – a physical computational resource at run time (a server, a device; 3-D box).

2. Behavioural things – the "verbs" (dynamic parts)

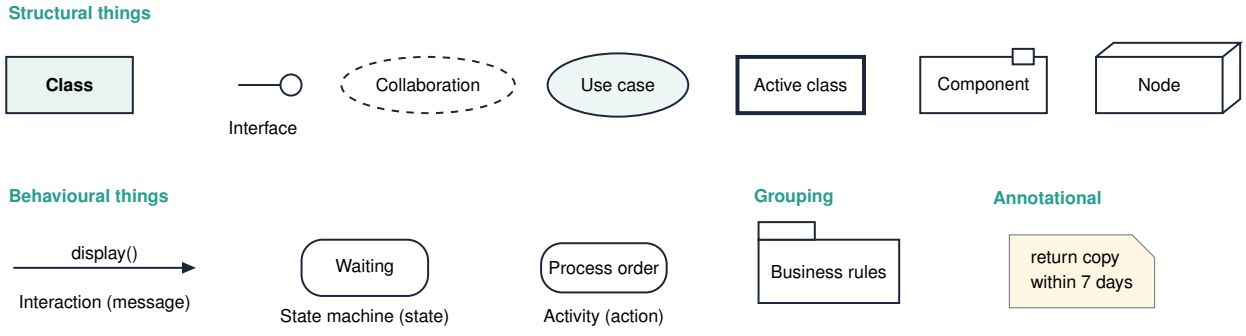
- **Interaction** – a set of messages exchanged among objects to accomplish a purpose (shown in sequence/communication diagrams).
- **State machine** – the sequence of states an object goes through in response to events.
- **Activity** – the sequence of steps (actions) a computational process performs.

3. Grouping things – organisational parts

Package – a general-purpose mechanism (tabbed folder) for organising model elements into groups; it exists only at development time.

4. Annotational things – explanatory parts

Note – a dog-eared rectangle holding comments or constraints attached to any element.



Answer. Structural things (class, interface, collaboration, use case, active class, component, node), behavioural things (interaction, state machine, activity), grouping things (package) and annotational things (note).

2013–2014 • Q7(a) CSE307 • 08/08/2015 • R027.1 MARKS 20

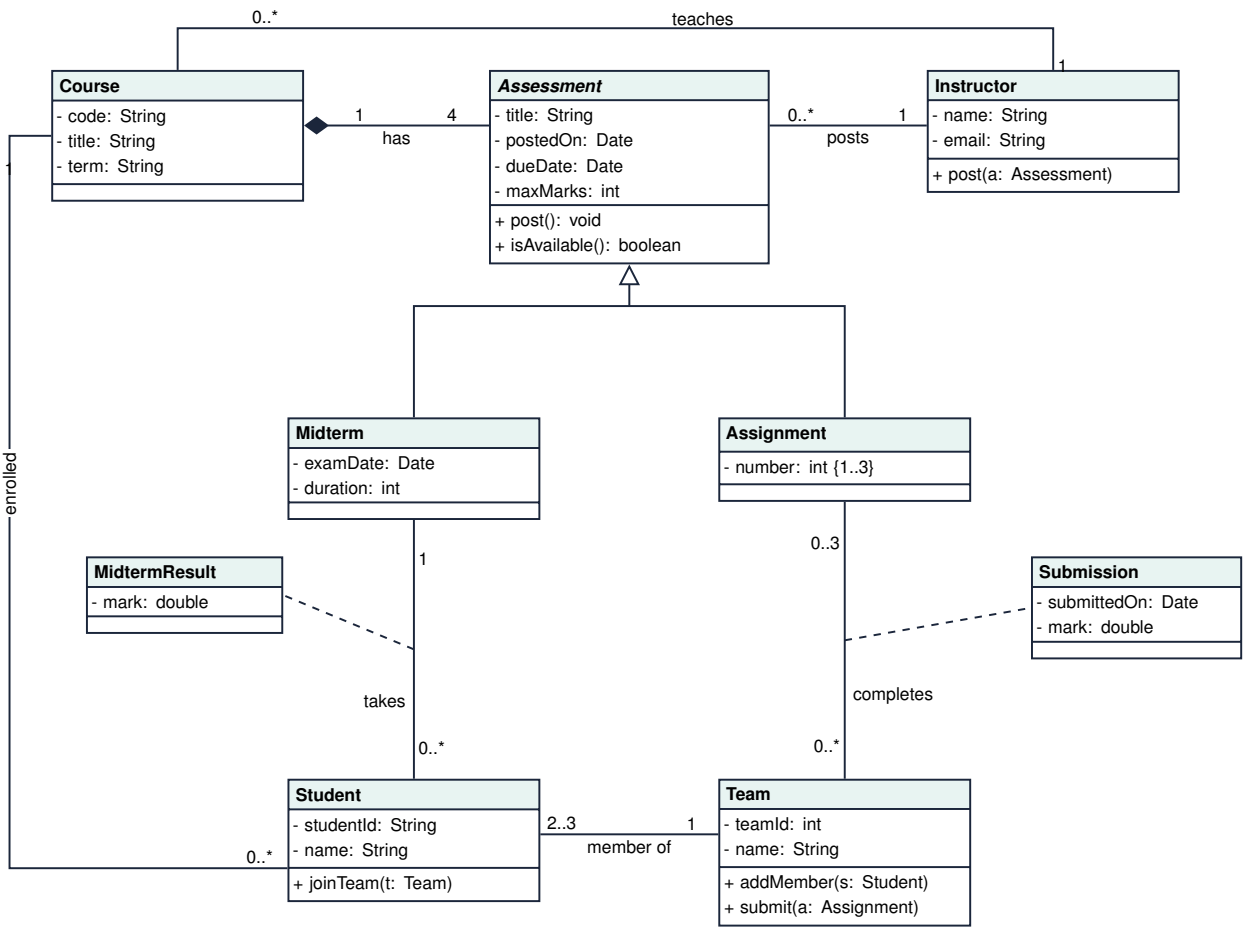
To take CSE 308, a student must go through the following process. When the instructor posts assignment 1, each student must form a team, do the assignment and hand it in. When assignment 2 is posted, each student works on it with her team. When the midterm assignment is made available, each student writes it. After both assignment 2 and the midterm are done with, each student waits for assignment 3, and when it is available, does it with her team. Draw a class diagram (with attributes, where appropriate) that describes entities and relationship relevant to the process. In particular, students are members of teams. Each team has 2 or 3 members. Each team completes 0 to 3 assignments. Each student takes exactly one midterm test. Each assignment and midterm is assigned a mark.

SOLUTION

Entities and the facts the diagram must capture

- **Course** (CSE 308) and its **Instructor**, who *posts* the assessments.
- **Student** – belongs to exactly one **Team**; a team has **2..3** members.
- Two kinds of assessment share title, posting date, due date and maximum marks ⇒ abstract **Assessment** with subclasses **Assignment** (number 1–3, done by teams) and **Midterm** (written individually).
- "Each assignment … is assigned a mark" – the mark belongs to a *(team, assignment)* pair ⇒ association class **Submission** (mark, submittedOn).
- "Each student takes exactly one midterm" and gets a mark – the mark belongs to a *(student, midterm)* pair ⇒ association class **MidtermResult** (mark).

Class diagram



NOTE The question describes *three* assignments and *one* midterm, so a Course has exactly 4 assessments (multiplicity 4); `{1..3}` on `Assignment.number` records the assignment number. The process ordering ("after both assignment 2 and the midterm . . . ") is behaviour and belongs to a state diagram, not the class diagram.

Modelling decisions explained

(i) Student 2..3 – 1 Team (*member of*): a team has 2 or 3 members; each student must form/join exactly one team (from assignment 1 onwards).

(ii) Team 0..* – 0..3 Assignment (*completes*): exactly the "0 to 3 assignments" rule. An assignment is completed by many teams.

(iii) Student 0..* – 1 Midterm (*takes*): "each student takes exactly one midterm test".

(iv) Association classes `Submission` and `MidtermResult` carry the `mark` because a mark is a property of the link (which team, which assignment), not of the team or the assignment alone – the lecture's `Job` pattern.

(v) Generalization: `Assignment` and `Midterm` inherit common data (dates, max marks) and the `post()` operation from the abstract `Assessment`.

(vi) Composition Course ♦ Assessment: assessments exist only within the course offering; **Instructor 1 – 0..* Assessment** records who posts them.

Answer. Student (2..3) member of (1) Team; Team (0..*) completes (0..3) Assignment with association class Submission(mark); Student (0..*) takes (1) Midterm with association class MidtermResult(mark); Assignment and Midterm specialise abstract Assessment, posted by the Instructor of the Course.

2012–2013 • Q7(a) CSE307 • 17/05/2014 • R033.2 **MARKS 15**

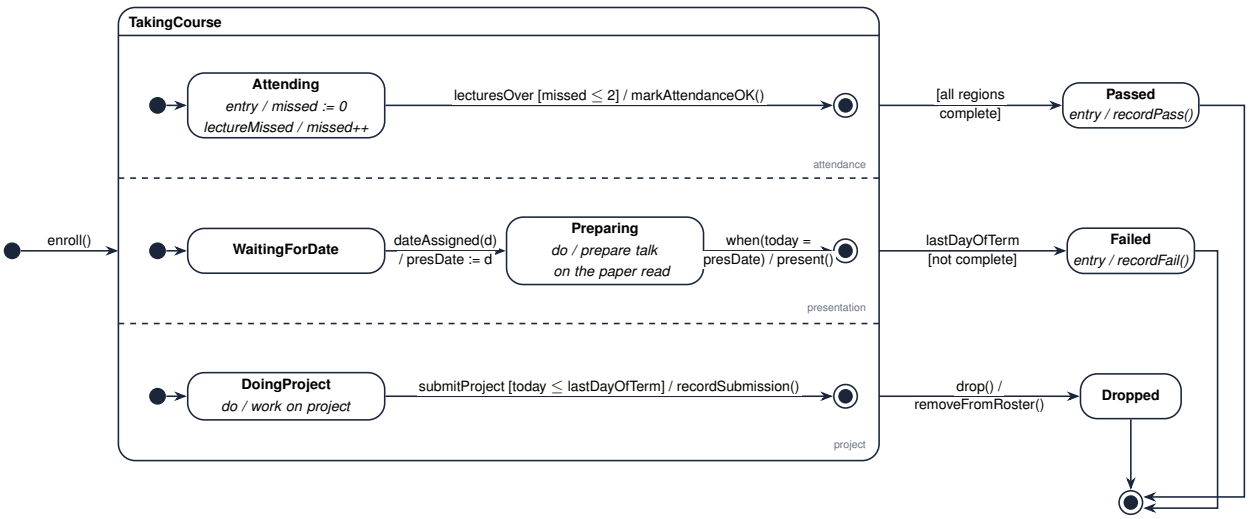
Give a state diagram that describes the process of passing a graduate course as a set of concurrent activities. The process works as follows: To pass, a student has to attend all but two lectures, present to the class a paper she read, and complete a course project, due on the last day of the term. To give her presentations, the student is given a date by the instructor, prepares her presentation, and gives it on the assigned day. At any time, the student can drop the course. Define events, conditions and actions for transitions in your diagram, where appropriate.

SOLUTION

Key idea: concurrent substates

A **composite state** may be split by dashed lines into **concurrent regions**. A transition into the composite state is equivalent to a simultaneous transition to the initial state of *every* region; each region then progresses independently. The composite state is exited normally (**completion transition**) only when *all* regions have reached their final states, while a transition drawn from the composite's border (e.g. `drop()`) leaves all regions at once, no matter how deeply nested.
The three passing requirements — *attendance*, *paper presentation*, *project* — happen in parallel during the term, so each becomes one region of the composite state TakingCourse.

State diagram



Events, conditions (guards) and actions

Transition	Event	Condition	Action
start → TakingCourse	`enroll()`	–	enter all three regions
Attending (internal)	`lectureMissed`	–	`missed++`
Attending → done	`lecturesOver`	`missed <= 2`	`markAttendanceOK()`
WaitingForDate → Preparing	`dateAssigned(d)` (by instructor)	–	`presDate := d`
Preparing → done	change event `when(today = presDate)`	–	`present()`
DoingProject → done	`submitProject`	`today <= lastDayOfTerm`	`recordSubmission()`
TakingCourse → Passed	completion (no event)	all regions in final state	`recordPass()`
TakingCourse → Failed	`lastDayOfTerm` (time event)	some region not finished	`recordFail()`
TakingCourse → Dropped	`drop()`	– ("at any time")	`removeFromRoster()`

Why this design

- "*Set of concurrent activities*" ⇒ one composite state with three **concurrent regions**, each with its own initial and final state.
- "*All but two lectures*" is a counter, not many states: an internal transition increments `missed` and a guard `[missed <= 2]` checks it at the end.
- The presentation has a clear sequence (*given a date → prepares → gives it on the assigned day*), hence two states and a **change event**.
- The project's deadline becomes the **guard** `[today <= lastDayOfTerm]`.
- "*At any time, the student can drop*" is drawn **once**, from the border of the composite state, because a transition out of a composite state applies to all its substates.
- Passed is reached by a **completion transition** only after all three regions complete (an implicit join); otherwise the end-of-term time event leads to Failed.

Answer. A composite state TakingCourse with three concurrent regions (Attendance, Presentation: WaitingForDate → Preparing, Project); completion of all regions → Passed; `lastDayOfTerm [not complete]` → Failed; `drop()` from the composite border → Dropped.

2012–2013 • Q7(c) CSE307 • 17/05/2014 • R033.4 **MARKS 12**

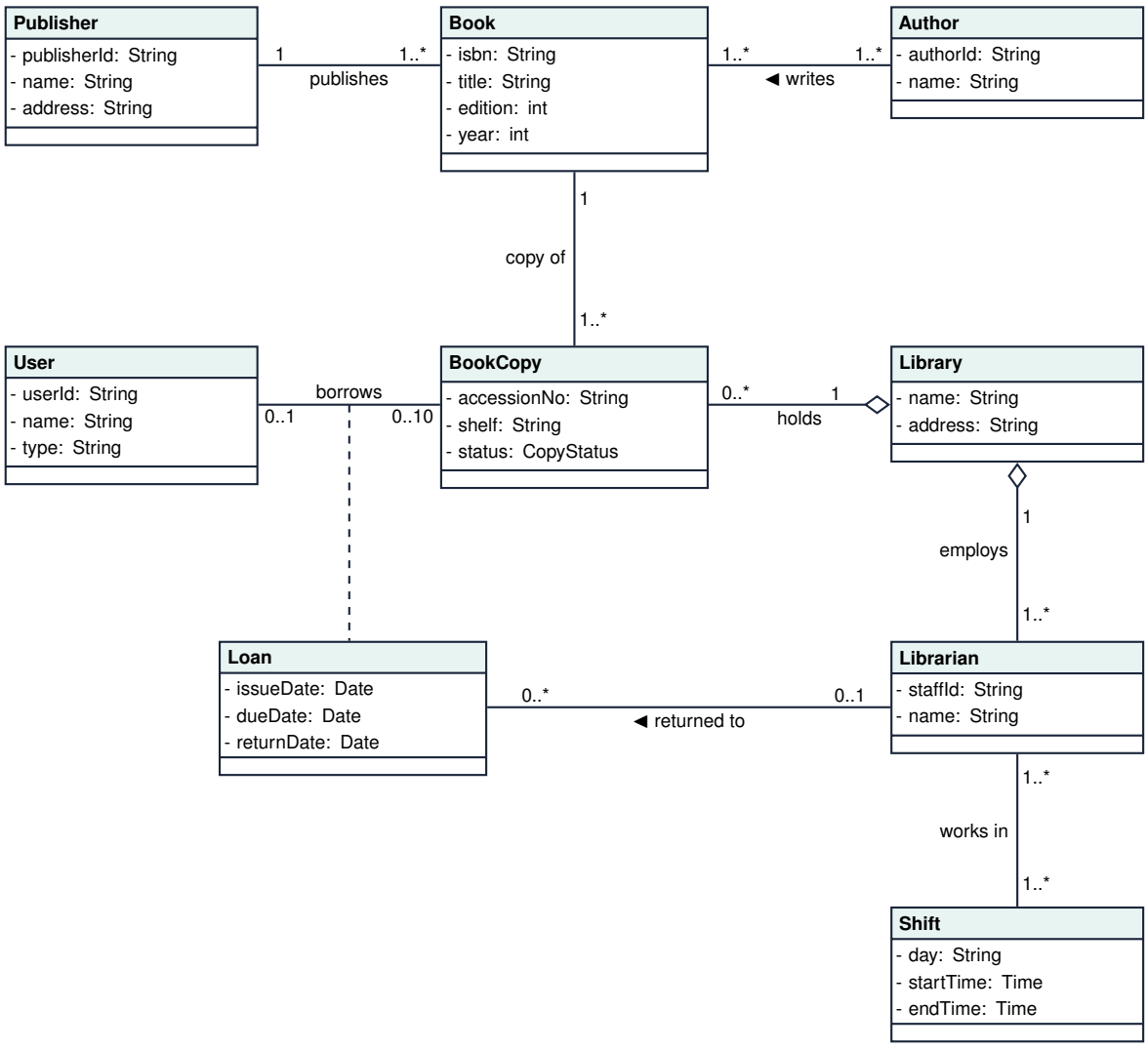
Different librarians work in different shifts in the university central library. The library contains multiple copies of each book. Each book is written by one or more authors and published by a publisher. A user may borrow up to 10 books at a time. User returns a borrowed book to the librarian working in that shift. Now design a class diagram with attributes and associations based upon the above scenario. You don't have to show any methods.

SOLUTION

Identifying classes

- **Library** – the central library; holds copies and employs librarians.
- **Book** (title-level description: ISBN, title, edition) vs. **BookCopy** (the physical copy: accession number, status). "Multiple copies of each book" forces this split – a user borrows a *copy*, while authors and publisher belong to the *book*.
- **Author**, **Publisher**.
- **User** (borrower) and **Loan** – the borrowing of one copy by one user, with issue, due and return dates.
- **Librarian** and **Shift** – librarians work in shifts; a returned copy is received by the librarian on duty.

Class diagram (attributes and associations only)



Modelling decisions explained

(i) **User 0..1 – 0..10 BookCopy (borrows)**: "a user may borrow up to 10 books at a time" becomes the multiplicity `0..10`; a copy is with at most one user at a time (`0..1`).

(ii) **Loan as an association class** on *borrows*: issue/due/return dates belong to the link between a user and a copy, not to either class alone.

(iii) **Loan 0..* – 0..1 Librarian (returned to)**: `0..1` because a loan that is not yet returned has no receiving librarian. The rule "the librarian working in that shift" is the constraint {returnDate lies within one of the librarian's shifts}.

(iv) **Librarian 1..* – 1..* Shift (works in)**: different librarians work in different shifts; each shift is covered by at least one librarian.

(v) **Book 1 – 1..* BookCopy**: a plain association ("description – item"); a copy is not a *kind of* book, so generalization would be wrong.

(vi) **Author 1..* – 1..* Book** ("one or more authors"); **Publisher 1 – 1..* Book**.

(vii) **Library ◇ BookCopy, Librarian**: aggregation – copies and staff exist independently of the library object.

Answer. Book (1..* authors, 1 publisher) has 1..* BookCopy held by the Library; User 0..1 – borrows – 0..10 BookCopy with association class Loan(issue/due/return date); Loan returned to 0..1 Librarian, who works in 1..* Shifts.

2011–2012 • Q3(a), Q3(b) CSE307 • 23/09/2013 • R037 **MARKS 8 + 14 2/3**

(a) What are the different types of events in a state chart diagram?

(b) An insurance company provides life insurance policies to its customer. A customer can purchase a policy worth Tk 3,50,000 for the duration of 5 years. After purchasing a policy the customer must pay monthly installments of Tk 5000 to the company. If a customer fails to pay the installment he becomes a defaulter. A defaulter must pay an extra Tk 500 for each month he did not pay to become a regular client again. If a customer fails to pay 6 consecutive installments then his policy is cancelled by the company. If the customer dies within the given time period then the company pays the nominee Tk 3,50,000. If the customer does not die during the period then the customer is paid Tk 3,50,000 by the company.
Now draw the state chart diagram of a customer for this system.

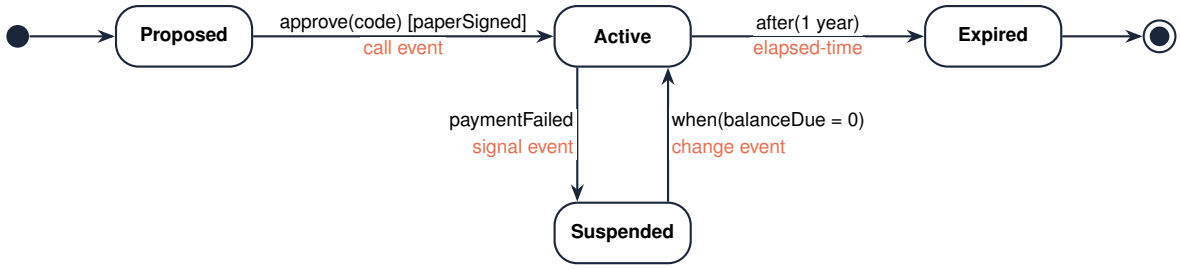
SOLUTION

(a) Types of events

An **event** is an occurrence that has a location in time and space and that may trigger a transition of an object from one state to another. The lecture lists four types:

Event type	Meaning	Notation / example
Change event	occurs when a *condition becomes true* (the object keeps checking a boolean expression)	`when(balanceDue = 0)`; `when [rateStartDate <= currentDate]`
Call event	occurs when an object *receives a call to one of its operations*, from another object or from itself (synchronous); the event must correspond to an operation of the class	`authorized(authorizationCode) [contractSigned] / setCampaignActive()`
Signal event	occurs when an object *receives a signal* — an asynchronous communication; the sender does not wait	`«signal» paymentFailed`, `sensorTripped`
Elapsed-time event	caused by the *passage of a designated period of time* after a specified event (frequently entry to the current state)	`after(1 year)`, `after(30 s)`

All four on one small statechart of a `Subscription` object:



EXAM TIP A transition label has the general form `event(args) [guard] / action`; any part can be omitted. A transition with *no* event is a *completion transition* — it fires as soon as the source state's activity finishes.

(b) Step 1 – Identifying the states of a customer

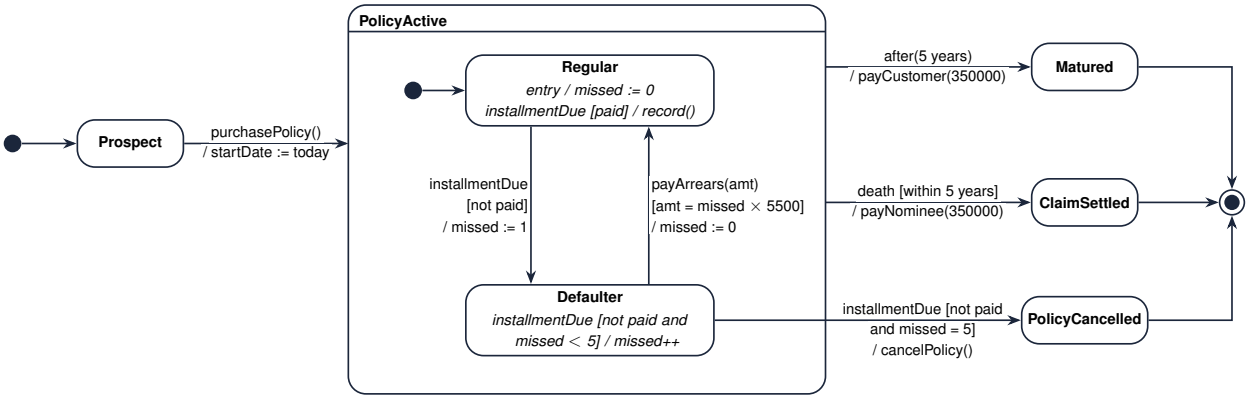
- Prospect – a customer without a policy yet.
- While the policy is in force (5 years) the customer is either a Regular payer or a Defaulter. Both share the exits "dies" and "5 years over", so they are grouped in the composite state PolicyActive.
- Three ways to end: PolicyCancelled (6 consecutive missed installments), ClaimSettled (died within the period, nominee paid), Matured (survived, customer paid).

Step 2 – Events, guards and actions

- `purchasePolicy()` (call event) / `startDate := today`.
- `installmentDue` is a monthly time event. In Regular: if paid, nothing changes (internal transition); `installmentDue [not paid] / missed := 1` makes him a defaulter.
- In Defaulter: `installmentDue [not paid and missed < 5] / missed++` (internal); the 6th consecutive miss `[missed = 5]` cancels the policy.
- `payArrears(amt) [amt = missed × (5000 + 500)] / missed := 0` returns him to Regular: the unpaid installments plus Tk 500 extra for every unpaid month.

- `death [within 5 years] / payNominee(350000)` and the elapsed-time event `after(5 years) / payCustomer(350000)` leave the composite from its border.

Step 3 – State chart diagram of a Customer



Explanation

- **Composite state PolicyActive** groups Regular and Defaulter: the 5-year maturity and the death claim apply in *both* substates, so they are drawn once from the composite border (a transition out of a composite applies to all its substates).
- **Counting consecutive misses with a guard** instead of six separate states keeps the diagram readable; paying arrears resets `missed`, so the next default starts counting again from 1 ("consecutive").
- The **extra Tk 500 per unpaid month** is captured in the guard of `payArrears`: each missed month costs $5000 + 500 = 5500$ taka.
- Cancellation is possible only from Defaulter, since a regular payer has missed nothing.
- `after(5 years)` is an **elapsed-time event** measured from entry into PolicyActive; `death` is a **signal event** received from outside.

> **NOTE** The narrative does not say what happens to a defaulter when the 5 years end; we assume that any arrears are adjusted from the Tk 3,50,000 paid out, so `after(5 years)` applies to the whole composite state.

Answer. (a) Change, call, signal and elapsed-time events. (b) Prospect → PolicyActive {Regular ↔ Defaulter} → Matured / ClaimSettled / PolicyCancelled, with the missed-installment counter in guards and payouts as actions.

2010–2011 • Q2(b) CSE307 • 19/03/2012 • R043.3 **MARKS 12$\frac{2}{3}$**

Give a **State Chart diagram** that describes the lifetime of a video tape in a video store. You can assume that a video tape is purchased, packaged properly (plastic case with identification information on the outside), put in the video store database, and is then put up for rental. Customer who choose to rent it, check it out and return it in 3 days. If a customer fails to return it, the store calls him/her the next day. The call is repeated a second time after 2 more days, and if

the tape is not returned within 2 more days, the store delegates the matter to a collection agency and removes the video tape from its collection. If the tape is damaged on return, it is removed from the collection database also. Finally, if the tape is missing during the annual store inventory, it is removed from the collection database as well.
Make sure to define **events, conditions, actions** for transitions in your diagram, where appropriate.

SOLUTION

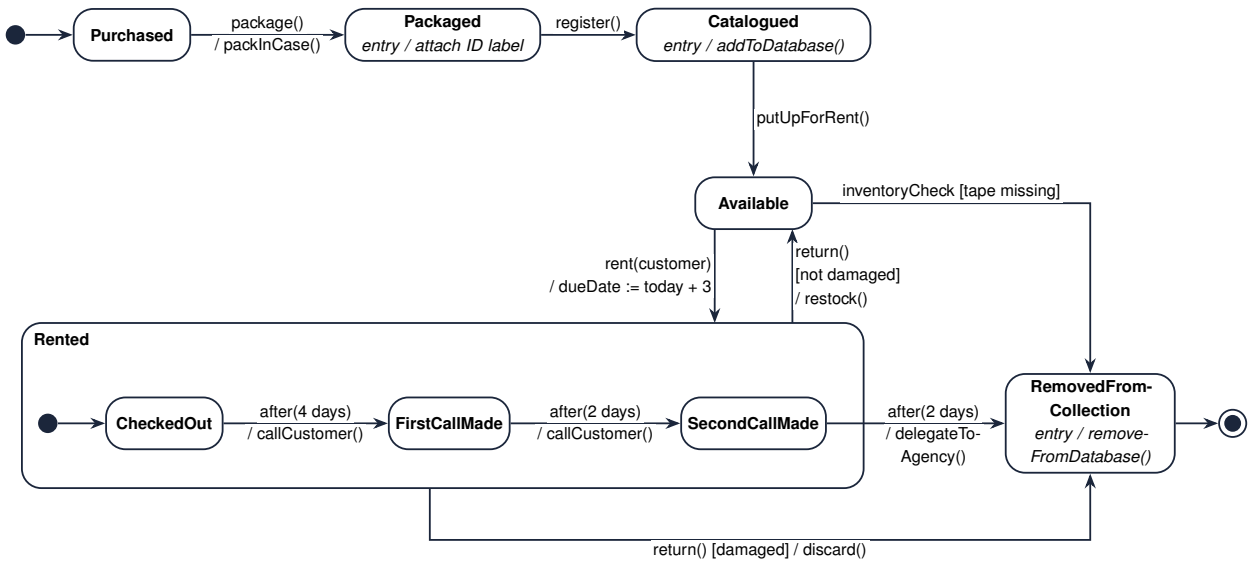
Approach (life-cycle approach)

The lecture gives two ways to prepare a statechart: the *behavioural* approach (from interaction diagrams) and the **life-cycle approach**, which follows one object from creation to destruction. Here the narrative literally describes the lifetime of a `VideoTape` object, so we walk through it and turn each phase into a state, each happening into an event, each "if" into a guard and each thing the store does into an action.

States

- Purchased → Packaged → Catalogued (in database) → Available (up for rental).
- Rented – a composite state with substates CheckedOut, FirstCallMade, SecondCallMade. All of them react the same way to `return()`, so the return transitions are drawn once, from the composite border.
- RemovedFromCollection – reached by damage, by the collection-agency case, or by being missing at the annual inventory; then the object's life ends (final state).

State chart diagram of a VideoTape



Events, conditions and actions

From → To	Event	Condition	Action
Purchased → Packaged	`package()`	–	put in plastic case, attach ID label
Packaged → Catalogued	`register()`	–	`addToDatabase()` (entry)
Catalogued → Available	`putUpForRent()`	–	–
Available → Rented	`rent(customer)`	–	`dueDate := today + 3`
Rented → Available	`return()`	not damaged	`restock()`
Rented → Removed	`return()`	damaged	`discard(); removeFromDatabase()`
CheckedOut → FirstCallMade	`after(4 days)` (3 days + next day)	not returned (implicit)	`callCustomer()`
FirstCallMade → SecondCallMade	`after(2 days)`	not returned	`callCustomer()`
SecondCallMade → Removed	`after(2 days)`	not returned	`delegateToAgency(); removeFromDatabase()`
Available → Removed	`inventoryCheck` (annual)	tape missing	`removeFromDatabase()`

Modelling notes

- The deadlines are **elapsed-time events** (`after(n days)`), measured from entry into the current substate. If `return()` happens first, the composite is left and the pending time event is cancelled — this is why the "not returned" condition needs no explicit guard.
- Using the composite state Rented avoids drawing three copies of each `return()` transition.
- `removeFromDatabase()` is an **entry action** of RemovedFromCollection, because every way into that state must perform it.
- Once removed, the tape object reaches the **final state** (its life in the system ends).

Answer. Purchased → Packaged → Catalogued → Available ⇄ Rented {CheckedOut → FirstCallMade → SecondCallMade} with `after(...)` time events; damaged return, collection agency and missing-at-inventory all lead to RemovedFromCollection (entry / removeFromDatabase) and then the final state.

06

Sequence & Collaboration Diagrams

2023–2024 • Q1(a) CSE325 • 17/05/2026 • C325-2324-1a **MARKS 16**

A Hospital Emergency Management System handles patient admission in the emergency department. When a patient arrives, Reception registers the patient. Then the system verifies insurance details. Next, a nurse assesses the patient's severity level. If the patient is critical, an emergency doctor is assigned immediately. Otherwise, the patient is added to the waiting queue. In both cases admission details are stored in the hospital database and SMS notification is sent to the patient's emergency contact.
Design a Collaboration Diagram for the scenario discussed above.

SOLUTION

What a collaboration (communication) diagram must show

A collaboration diagram is the *structural* view of an interaction. It contains

- the participating **objects** (and actors), drawn as rectangles with underlined names `name:Class` and placed freely on the page;
- **links** between objects that exchange messages (a message can only travel along a link);
- **messages** written beside the links with a small arrow for the direction and a **nested sequence number** (1, 1.1, 1.2, . . .) that replaces the time axis of a sequence diagram;
- **guards** `[condition]` for branching; mutually exclusive alternatives get the same number with letter suffixes (`2.2a`, `2.2b`).

Step 1 – participants (boundary / control / entity view)

Participant	Role in the scenario
Receptionist (actor)	registers the arriving patient – starts the interaction
Nurse (actor)	assesses and enters the severity level
`:AdmissionController` «control»	coordinates the admission workflow (one object that "knows the steps")
`p:Patient` «entity»	the newly created patient record, holds severity level
`:InsuranceService`	verifies the patient's insurance details
`:DoctorRoster`	finds and assigns an available emergency doctor
`:WaitingQueue`	queue of non-critical patients
`:HospitalDatabase`	persistent store of admission details
`:SMSGateway`	sends the SMS to the emergency contact

Putting a single controller in the middle keeps the diagram a "star": every service is linked only to the controller, so each link corresponds to exactly one association in the later class diagram.

Step 2 – message ordering

(i) `1: register(patientInfo)` – Receptionist → controller; the controller then `1.1: «create»` the `p:Patient` object and calls `1.2: verifyInsurance(p)`.

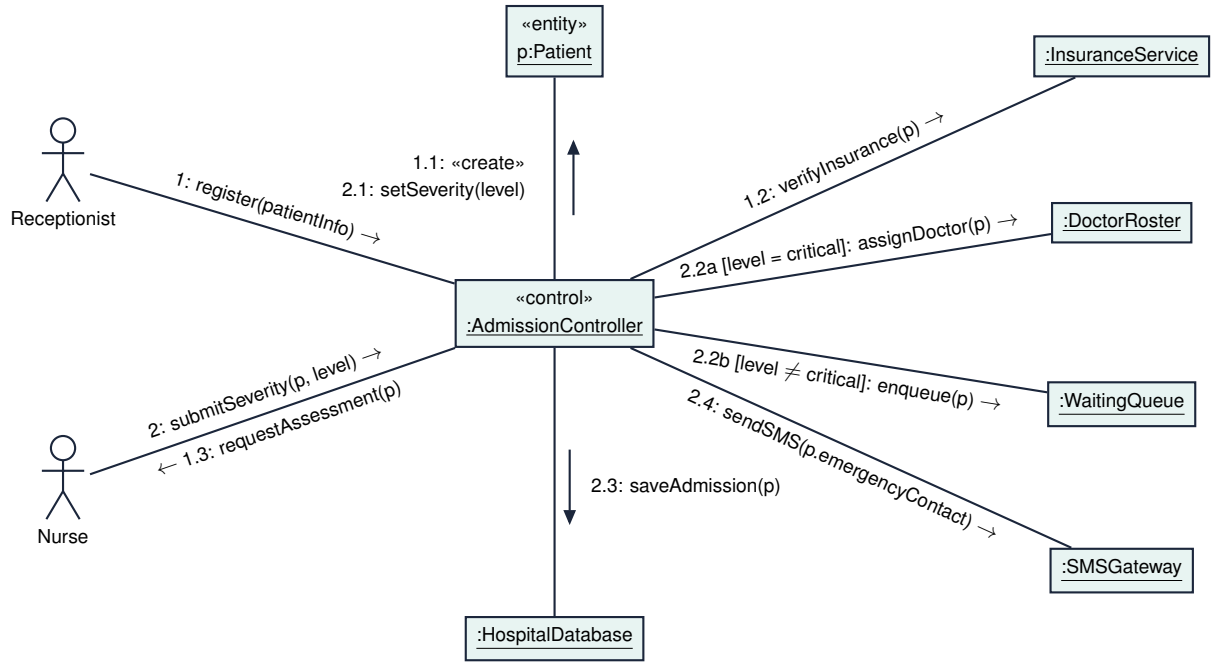
(ii) After verification the controller asks the nurse for triage: `1.3: requestAssessment(p)`.

(iii) The nurse replies with `2: submitSeverity(p, level)`; the controller stores it in the patient (`2.1: setSeverity(level)`).

(iv) Exactly one of the two alternatives fires: `2.2a [level = critical]: assignDoctor(p)` or `2.2b [level ≠ critical]: enqueue(p)`.

(v) "In both cases" – so these have no guard: `2.3: saveAdmission(p)` and `2.4: sendSMS(p.emergencyContact, msg)`.

Step 3 – the collaboration diagram



Step 4 – modelling decisions explained

- **Nested numbering.** Everything the controller does while handling the registration is numbered 1.x; everything triggered by the nurse's input is 2.x. The numbers alone reproduce the exact order of the textual scenario (register → verify insurance → assess → assign/queue → store → SMS).
- **Alternative paths.** `2.2a` and `2.2b` share the number 2.2 and carry complementary guards, which is how a collaboration diagram expresses an `alt`; only one of them is sent.
- **Common tail.** Storing the admission and sending the SMS happen "in both cases", so `2.3` and `2.4` carry no guard and follow either branch.
- **Object creation.** `1.1` is a «create» message – the patient record does not exist before registration.
- **Links = associations.** Each line is a link that must later exist as an association (`AdmissionController` knows `InsuranceService`, `DoctorRoster`, ...); this is the main advantage of the collaboration view over the sequence view.

NOTE The scenario does not say what happens when insurance verification fails; it is assumed that the result is only recorded (emergency care is never refused), so no extra branch is drawn. If required, a guard `[insured]` could be added to message 1.3.

Answer. Objects Receptionist, Nurse, `:AdmissionController`, `p:Patient`, `:InsuranceService`, `:DoctorRoster`, `:WaitingQueue`, `:HospitalDatabase`, `:SMSGateway`; messages 1 register → 1.1 create patient → 1.2 verifyInsurance → 1.3 requestAssessment → 2 submitSeverity → 2.1 setSeverity → 2.2a [critical] assignDoctor / 2.2b [else] enqueue → 2.3 saveAdmission → 2.4 sendSMS.

2023–2024 • Q3(a) CSE325 • 17/05/2026 • C325-2324-3a **MARKS 15**

A **Ride Sharing Application** allows passengers to book rides through a mobile app. When a passenger requests a ride, the app sends the ride request to the backend server. The backend searches for nearby available drivers. A ride offer is sent to drivers one by one until one accepts. When a driver accepts, the passenger is notified and the ride details are stored in the database. If no driver accepts within the timeout period, the passenger is informed that no driver is available. Draw Sequence Diagram to model the scenario above.

SOLUTION

Elements of a sequence diagram used here

- **Lifelines**: one per participant (actor or object), the dashed line represents its life in time; time flows *top to bottom*.
- **Activation bars**: the period during which an object is executing an operation.
- **Messages**: synchronous (filled head), asynchronous (open head), **return** (dashed), and a **self-message** for an object calling its own operation.
- **Combined fragments**: `loop [guard]` for "one by one until one accepts" and `alt [guard] / [else]` for "driver accepted" versus "timeout".

Step 1 – participants

Passenger (actor), `:MobileApp`, `:BackendServer`, `d:Driver` (the driver currently being offered the ride; the same lifeline stands for "the next driver" in every loop iteration) and `:Database`.

Step 2 – message flow

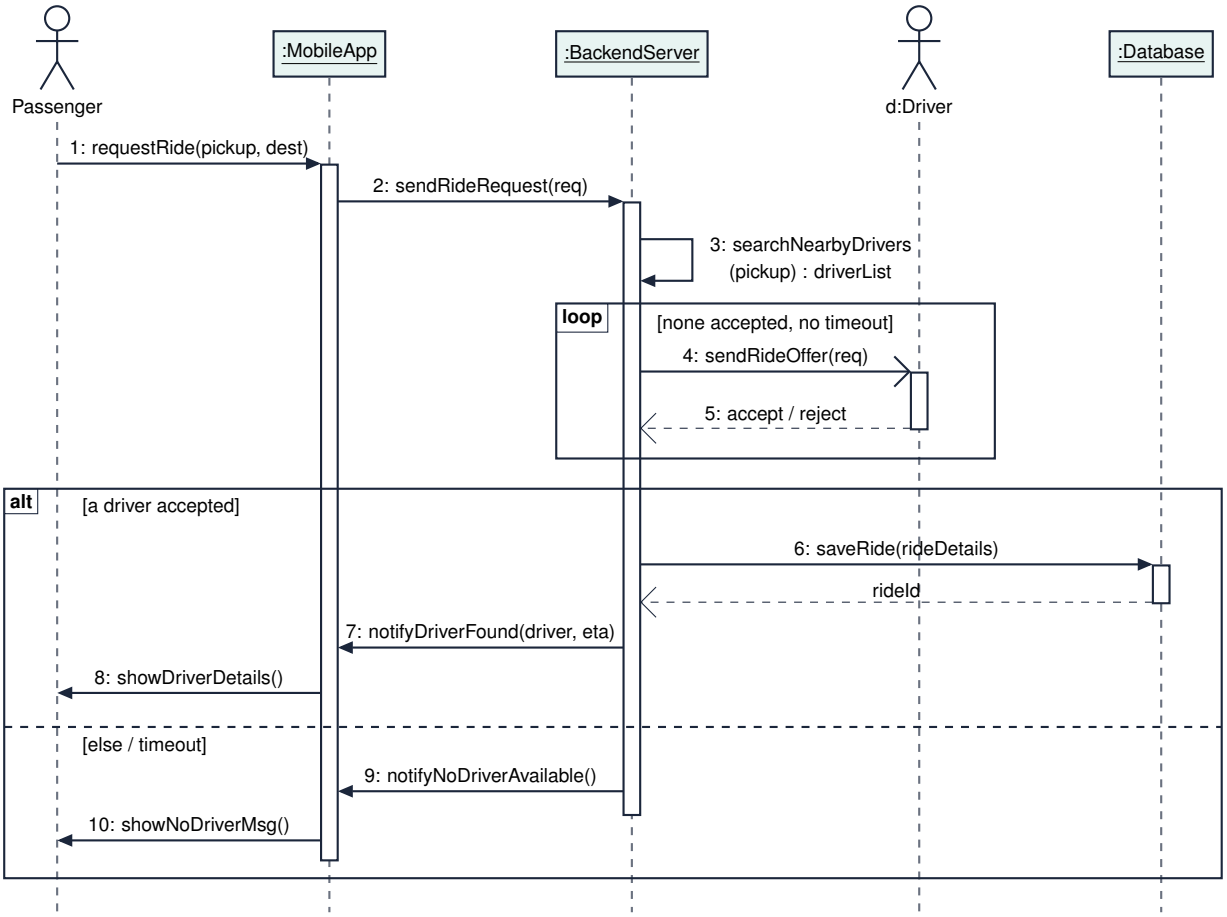
(i) Passenger → App: `requestRide(pickup, destination);` App → Backend: `sendRideRequest(req).`

(ii) Backend self-call `searchNearbyDrivers(pickup)` returns a list of available drivers.

(iii) `loop [no driver accepted and time < timeout and drivers left]`: Backend → Driver `sendRideOffer(req)` (asynchronous – the driver answers later); Driver returns `accept / reject`.

(iv) `alt`: `[driver accepted]` → `saveRide(details)` in the database, then `notifyDriverFound(driver, eta)` to the app, which shows the driver details; `[timeout - no driver accepted]` → `notifyNoDriverAvailable()` and the app shows the message.

Step 3 – the sequence diagram



Step 4 – modelling decisions explained

- **Self-message 3** – the backend itself searches for drivers; the result (`driverList`) drives the loop.
- **`loop` fragment** – "sent to drivers one by one until one accepts" is an iteration whose guard stops when a driver accepts, when the list is exhausted or when the timeout elapses. Only the backend and the current driver take part, so the frame covers only those two lifelines.
- **Asynchronous offer** – `sendRideOffer` uses an open arrowhead: the server does not block on the driver's phone; the reply comes back as a separate (dashed) response.
- **`alt` fragment** – the two mutually exclusive outcomes (accepted / timeout) are the two operands separated by the dashed line. The order inside the first operand follows the text: ride details stored, passenger notified.
- Messages to the Passenger are shown as the app's display operations, since an actor cannot execute code.

> **NOTE** The text says "the passenger is notified and the ride details are stored". The diagram stores first and then notifies, so the passenger never sees a ride that failed to persist; drawing the two in the opposite order is equally acceptable.

Answer. Lifelines Passenger, `:MobileApp`, `:BackendServer`, `d:Driver`, `:Database`; requestRide → sendRideRequest → self searchNearbyDrivers; `loop` [none accepted, not timed out] sendRideOffer / accept-reject; `alt` [accepted] saveRide, notifyDriverFound, show details | [timeout] notifyNoDriverAvailable, show "No driver available".

2022–2023 • Q2(a) CSE325 • 30/01/2025 • C325-2223-2a MARKS 15

In Bangladesh, every school teacher is appointed based on MPO regulation. According to this regulation, each candidate has to meet certain qualification criteria defined for each post (of a particular subject) of a certain salary grade. The qualifications are stated in terms of age, subject of graduation, and result at different levels. Example of such requirement is shown in Table for Q 2(a). This is for a particular type of institution, e.g., school, college, etc. When a candidate applies for a post, his/her qualifications are matched against such requirements. If the qualification criteria satisfy, he/she is eligible to sit for a recruitment test.

Table for Q 2(a) *(printed in Bangla in the paper; English translation)*

Sl.	Post	Educational qualification	Age limit	Salary grade
13	Assistant Teacher (Social Science)	(1) Bachelor (Hons)/equivalent degree with the relevant subject from a recognised university **and** B.Ed degree/equivalent; **or** (2) Bachelor (Hons)/equivalent degree with the relevant subject from a recognised university. Not more than one (01) 3rd division/class in the whole academic life.	At most 35 years (age limit relaxable for index holders of the same post)	(1) Grade-10 (16000–38640/-) (2) Grade-11 (12500–30230/-)
14	Assistant Teacher (Mathematics)	(1) Bachelor (Hons)/equivalent degree in a science department with Mathematics from a recognised university **and** B.Ed degree/equivalent; **or** (2) Bachelor (Hons)/equivalent degree in a science department with Mathematics from a recognised university. Not more than one (01) 3rd division/class in the whole academic life.	At most 35 years (age limit relaxable for index holders of the same post)	(1) Grade-10 (16000–38640/-) (2) Grade-11 (12500–30230/-)

Draw a collaboration diagram to check eligibility of an applicant according to the scenario discussed above.

SOLUTION

Step 1 – What a collaboration diagram must show

A collaboration (communication) diagram is an interaction diagram that emphasises the **objects and the links** between them; time order is shown only by the **decimal sequence numbers** (1, 1.1, 1.1.1, . . .) written on the messages. Iteration is written `*[i := 1..n]`, a condition as a guard `[cond]`. Every message needs a link, so the diagram also tells us which objects must know each other.

Step 2 – Extract the rules from the table

For a post such as *Assistant Teacher (Mathematics)* an applicant is eligible iff

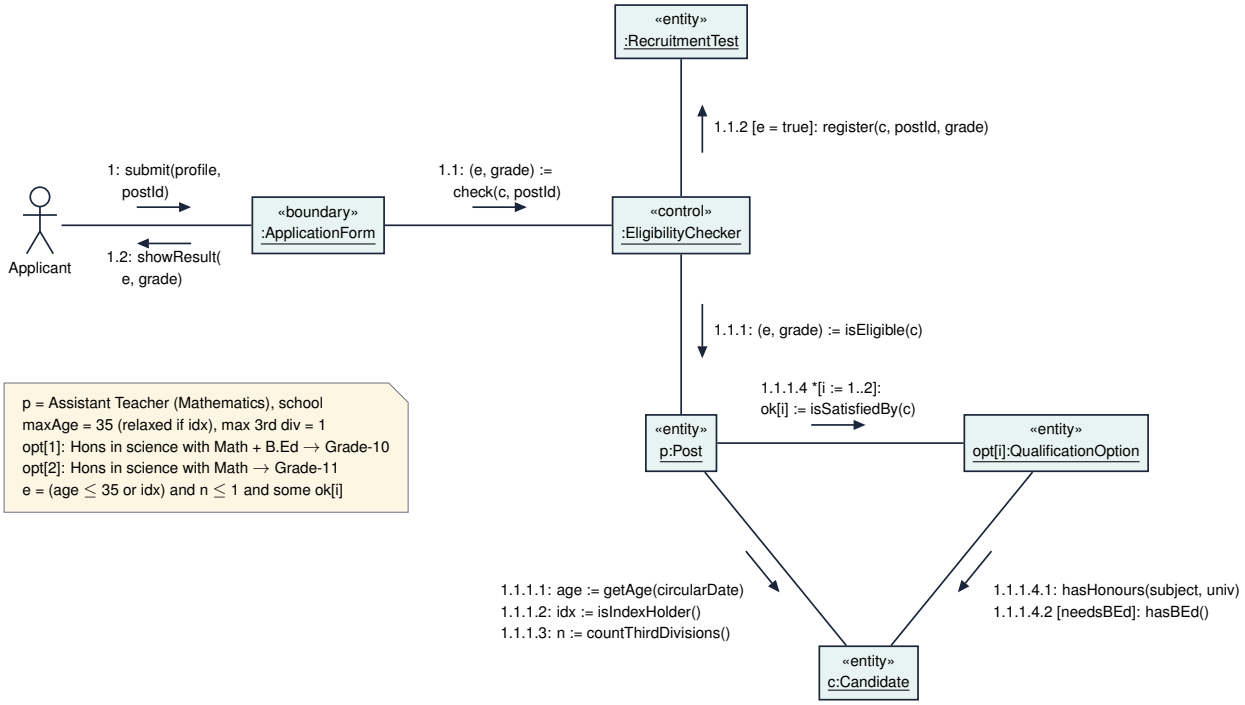
- **Age:** age on the circular date $\leq$ 35, *or* the applicant is an index holder of the same post (relaxation);
- **Results:** at most one 3rd division/class in the whole academic life;
- **Degree:** at least one *qualification option* is satisfied – option 1 = relevant Bachelor (Hons) from a recognised university **+ B.Ed** $\Rightarrow$ Grade-10; option 2 = relevant Bachelor (Hons) from a recognised university (no B.Ed) $\Rightarrow$ Grade-11.

Since the criteria differ by post, subject, grade and institution type, they are held as data in a `Post` object with a list of `QualificationOption` objects, not hard-coded in the checker.

Step 3 – Objects

Object	Kind	Responsibility
Applicant	actor	submits the application
`:ApplicationForm`	boundary	collects the profile and the chosen post, shows the result
`:EligibilityChecker`	control	coordinates the check, registers eligible applicants
`p:Post`	entity	max age, relaxation rule, 3rd-division limit, its qualification options (for one institution type)
`opt[i]:QualificationOption`	entity	required degree/subject, B.Ed needed?, salary grade
`c:Candidate`	entity	date of birth, index-holder flag, degrees with subjects, results
`:RecruitmentTest`	entity	list of applicants allowed to sit for the test

Step 4 – Collaboration diagram



Step 5 – Reading the diagram (message by message)

(i) 1 The applicant submits the profile and the chosen post through the form (boundary object).

(ii) 1.1 The form asks the control object to check eligibility, passing the candidate `c`.

(iii) 1.1.1 The checker delegates to the `Post`, which owns the rules (*information expert*).

- **1.1.1.1–1.1.1.3** The post asks the candidate for age on the circular date, the index-holder status (relaxes the 35-year limit) and the number of 3rd divisions (≤ 1 required).
- **1.1.1.4 *[i := 1..2]** It iterates over its qualification options; each option checks the candidate's Honours degree/subject/university (**1.1.1.4.1**) and, only when the option needs it, the B.Ed degree (**1.1.1.4.2**, guarded by `[needsBEd]`). The first satisfied option gives the salary grade (Grade-10 with B.Ed, else Grade-11).

(iv) 1.1.2 [e = true] Only an eligible applicant is registered for the recruitment test.

(v) 1.2 The form shows the result (eligible / not eligible and the grade) to the applicant.

The links `p-c` and `opt-c` are *parameter* links (the candidate object is passed as argument), which is why the post and the options can send messages to it.

Answer. Objects: Applicant, `:ApplicationForm`, `:EligibilityChecker`, `p:Post`, `opt[i]:QualificationOption`, `c:Candidate`, `:RecruitmentTest`. Messages: `1 submit` → `1.1 check` → `1.1.1 isEligible(c)` → `1.1.1.1–3` age / index-holder / 3rd-division queries → `1.1.1.4 *[i:=1..2] isSatisfiedBy(c)` with 1.1.1.4.1 degree check and 1.1.1.4.2 `[needsBEd] hasBEd()` → `1.1.2 [e] register` → `1.2 showResult`.

NOTE The table in the paper is in Bangla; the translation above is used. It is assumed that option (1) (with B.Ed) maps to Grade-10 and option (2) (without B.Ed) to Grade-11, and that "age limit relaxable" means the 35-year limit is waived for index holders of the same post.

2022–2023 • Q3(a) CSE325 • 30/01/2025 • C325-2223-3a **MARKS 15**

Let's consider a scheduler app. The scenario begins when the user chooses to add a new appointment in the UI. The UI notices which part of the calendar is active and pops up an Add Appointment window for that date and time. The user enters information about the appointment's name, location, start and end times. The UI will prevent the user from entering an appointment that has invalid information, such as an empty name or negative duration. The calendar records the new appointment in the user's list of appointments. Any reminder selected by the user is added to the list of reminders. If the user already has an appointment at that time, the user is shown a message and asked to choose an available time or replace the appointment. If the user enters an appointment with the same name and duration as an existing meeting, the calendar asks the user whether he/she intended to join that meeting instead. If so, the user is added to that meeting's list of participants.
Draw Sequence Diagram to model the scenario above.

SOLUTION

Step 1 – Participants (lifelines)

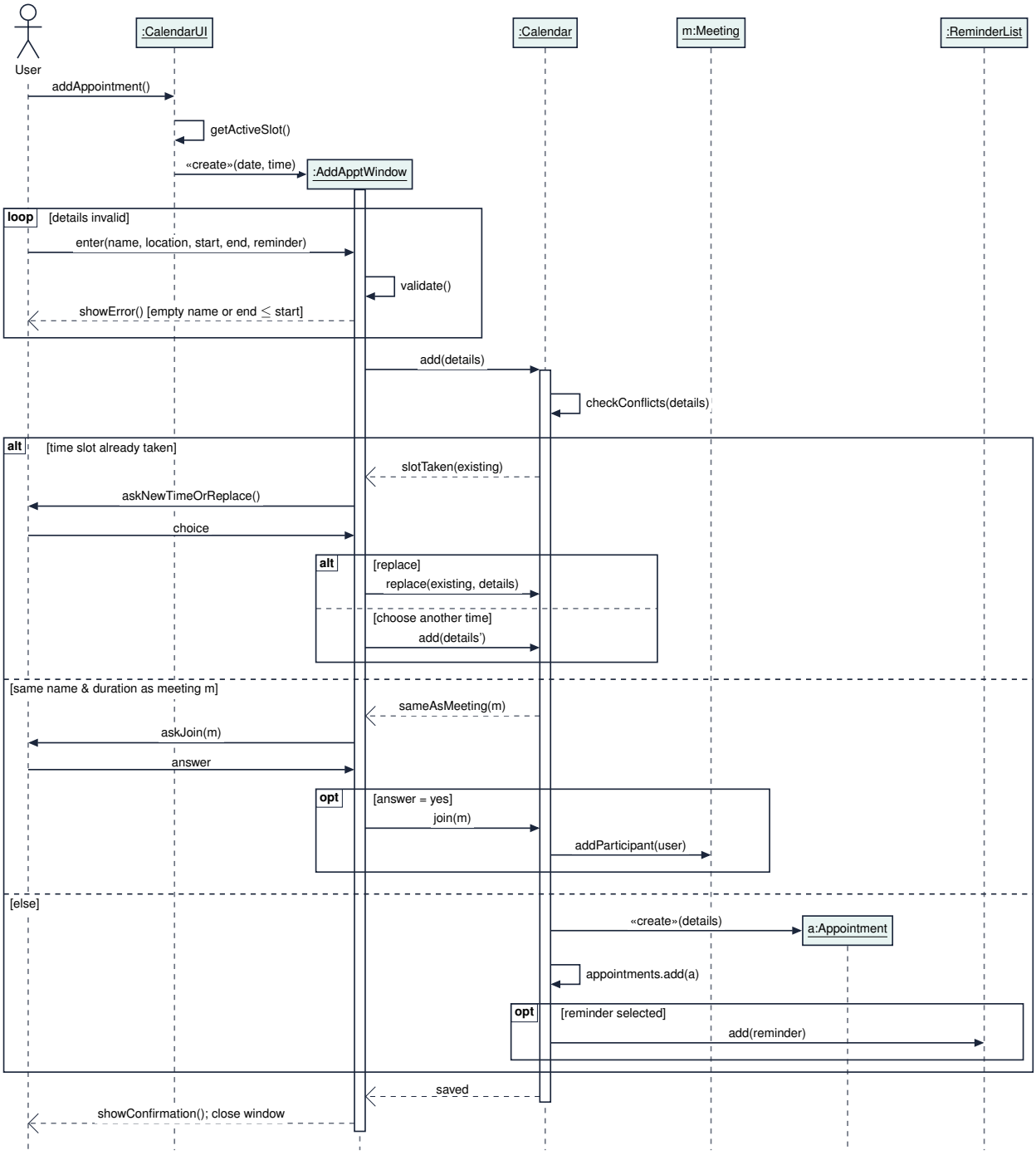
- **User** (actor) – starts the scenario and answers the questions.
- `:CalendarUI` – knows which part of the calendar is active (date and time).
- `:AddApptWindow` – *created* by the UI ("pops up"), collects and validates the input.
- `:Calendar` – owns the user's list of appointments, detects conflicts and matching meetings.
- `m:Meeting` – an existing meeting (keeps its participants list).
- `a:Appointment` – *created* only when the appointment is really recorded.
- `:ReminderList` – the list of reminders.

Step 2 – Control structure of the scenario

- "The UI will *prevent* the user from entering invalid information" ⇒ a **loop** fragment: enter details, `validate()`, show an error while the data are invalid (empty name, end ≤ start).

- Three mutually exclusive outcomes after the calendar checks the new entry ⇒ one **alt** fragment: (1) time already taken → ask user: choose another time *or* replace (nested **alt**); (2) same name & duration as an existing meeting → ask whether to join; **opt**[yes] add participant; (3) `[else]` create and record the appointment.
- "Any reminder selected" ⇒ **opt**[reminder selected] `add(reminder)`.

Step 3 – Sequence diagram



Step 4 – Modelling decisions

- **Object creation.** `:AddApptWindow` and `a:Appointment` are drawn *lower* than the other heads, at the point where the «create» message reaches them – they do not exist before.
- **Validation loop.** The window keeps the user in the loop until `validate()` succeeds; only then is `add(details)` sent to the calendar, so the calendar never receives invalid data.
- **Conflict handling (operand 1).** The calendar returns `slotTaken(existing)`; the nested `alt` shows the two choices offered to the user: replace the existing appointment, or resubmit with another time.
- **Join-meeting handling (operand 2).** The `opt[answer = yes]` fragment sends `join(m)` and the calendar adds the user to `m`'s participants list. (If the answer is no, the user is returned to the form, as in the scenario nothing else is specified.)
- **Normal case (operand `[else]`).** The appointment object is created, added to the user's list, and only if a reminder was chosen it is added to the reminder list (`opt`).

Answer. Lifelines User, `:CalendarUI`, `:AddApptWindow` (created), `:Calendar`, `m:Meeting`, `a:Appointment` (created), `:ReminderList`. `loop[details invalid]` enter/validate/showError; then `add(details)` → `checkConflicts`; `alt` with three operands: [slot taken] ask and nested `alt`[replace]/[other time]; [same name & duration as meeting] askJoin, `opt`[yes] `join(m)` → `addParticipant(user)`; [else] «create» appointment, `appointments.add(a)`, `opt`[reminder selected] `add(reminder)`.

2021–2022 • Q3(c) CSE325 • 07/10/2023 • C325-2122-3c **MARKS 19**

(Context, Q3(a): Consider an Automated Teller Machine (ATM) of a Bank. You can have the following banking operations using the ATM card: (i) Withdrawing money, (ii) Checking Balance from Core Banking System, (iii) Changing PIN, (iv) Depositing money, (v) Transferring money to another account, (vi) Recharging your mobile phone.)

(c) Draw collaboration diagram for the use case for withdrawing money using ATM Card. Show the steps of deriving class diagram from the collaboration diagram.

SOLUTION

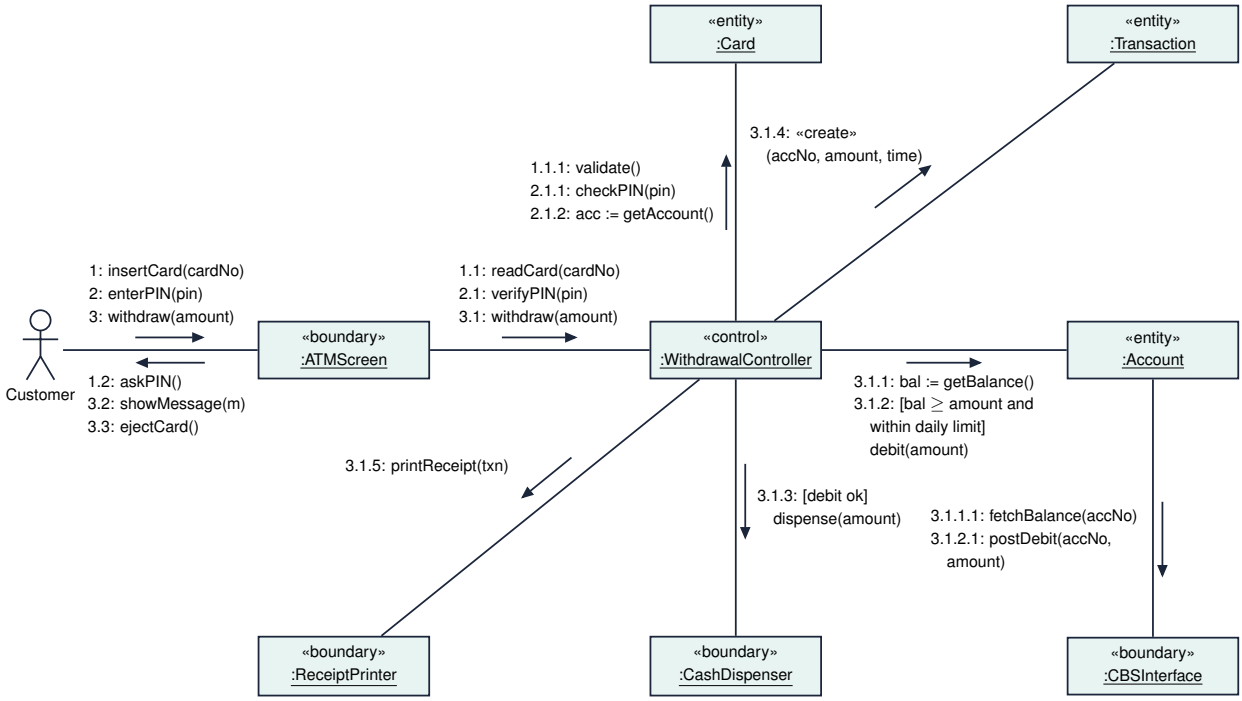
Use case flow that the diagram must realise

Withdraw Money (see the narrative in the Use Case chapter, C325-2122-3b): the customer inserts the card, the card is validated and the PIN checked; the customer enters an amount; the ATM checks the balance (held by the Core Banking System) and the daily limit, debits the account, dispenses cash, records the transaction, prints a receipt and ejects the card.

Objects (Bennett's analysis stereotypes)

- «boundary»: `:ATMScreen` (screen + keypad + card reader the customer talks to), `:CashDispenser`, `:ReceiptPrinter` (devices), `:CBSInterface` (gateway to the external Core Banking System);
- «control»: `:WithdrawalController` — coordinates the use case;
- «entity»: `:Card`, `:Account`, `:Transaction` (created during the use case).

Collaboration (communication) diagram – Withdraw Money



Numbering is nested: `3.1` is sent while handling `3`, `3.1.2.1` while handling `3.1.2`, and so on. Guards (`[bal ≥ amount ...]`) show the conditional messages; if the guard fails the controller sends `3.2: showMessage("insufficient balance")` instead and no cash is dispensed.

Deriving the class diagram – steps

(i) **Every object becomes a class** (the class of the instance), keeping its stereotype: `ATMScreen`, `WithdrawalController`, `Card`, `Account`, `CBSInterface`, `CashDispenser`, `ReceiptPrinter`, `Transaction`.

(ii) **Every link becomes an association** between the two classes; the direction of the messages on it gives the **navigability** (sender → receiver).

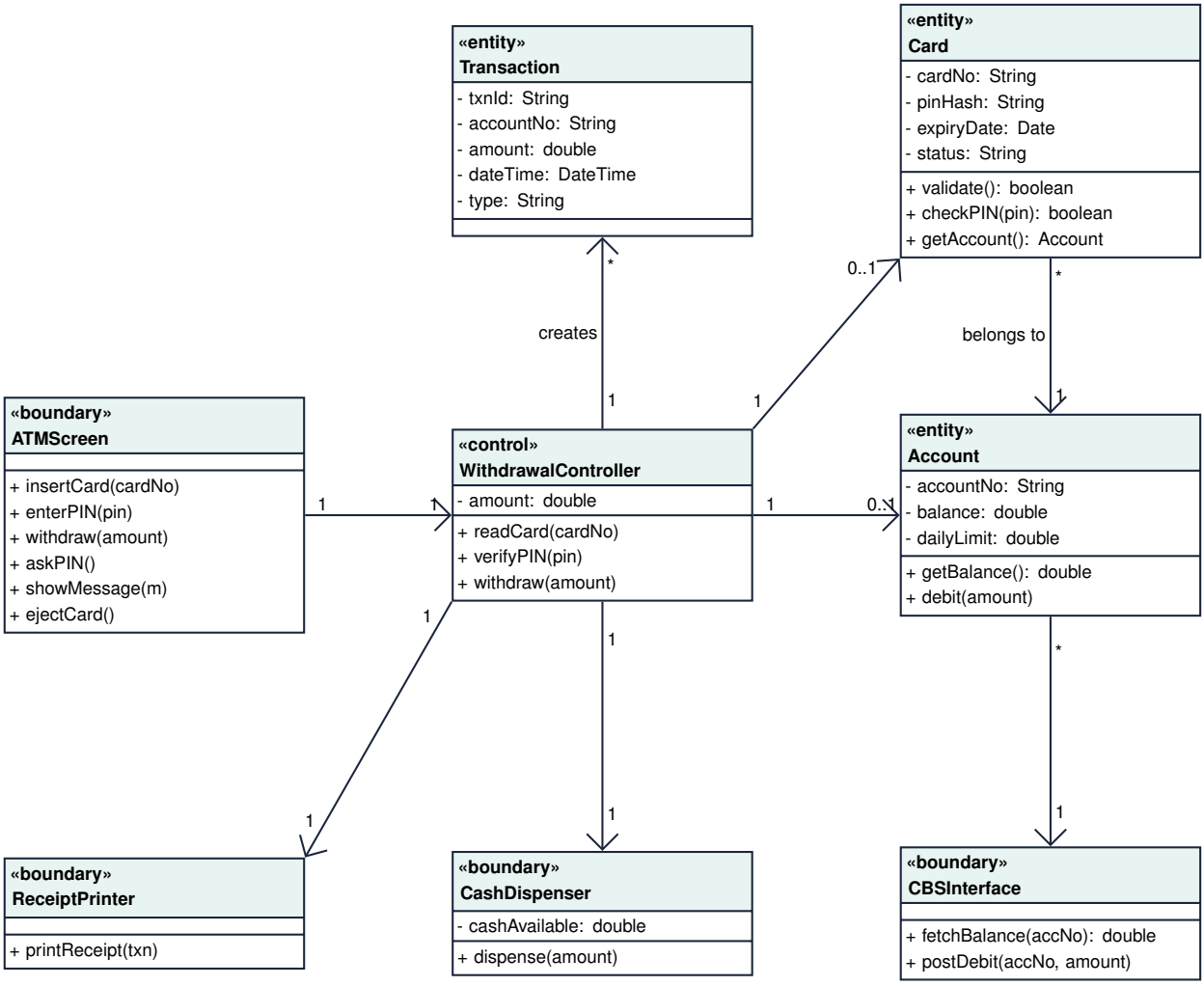
(iii) **Every message received becomes an operation of the receiver's class**, with the parameters and return value of the message (table below).

(iv) **Data carried by messages / needed by operations become attributes** (e.g. `getBalance()` ⇒ `Account.balance`; `checkPIN(pin)` ⇒ `Card.pinHash`).

(v) **Add multiplicities** from the domain (one card belongs to one account, an account may have several cards, a controller session creates many transactions, ...) and merge the classes obtained from the other use cases' diagrams.

Receiver class	Operations obtained from the messages
ATMScreen	`insertCard(cardNo)`, `enterPIN(pin)`, `withdraw(amount)`, `askPIN()`, `showMessage(m)`, `ejectCard()`
WithdrawalController	`readCard(cardNo)`, `verifyPIN(pin)`, `withdraw(amount)`
Card	`validate()`, `checkPIN(pin)`, `getAccount(): Account`
Account	`getBalance(): double`, `debit(amount)`
CBSInterface	`fetchBalance(accNo): double`, `postDebit(accNo, amount)`
CashDispenser	`dispense(amount)`
Transaction	constructor `Transaction(accNo, amount, time)`
ReceiptPrinter	`printReceipt(txn)`

Resulting (analysis) class diagram



Answer. Collaboration: Customer → ATMScreen → WithdrawalController, which talks to Card (validate, checkPIN, getAccount), Account (getBalance, debit → CBSInterface fetchBalance/postDebit), CashDispenser (dispense), Transaction (create) and ReceiptPrinter. Class diagram: objects → classes, links → associations (navigable in message direction), received messages → operations, message data → attributes, then add multiplicities.

2020–2021 • Q2(b) CSE325 • 12/04/2023 • C325-2021-2b **MARKS 14**

(Scenario of Q2(a): In an educational institution, the teachers offer different course in different semesters. Some courses are jointly offered by multiple teachers. Students enroll in multiple offerings and receive grades at the end of the course. The grades are computed following some grading rule from the marks of different test items.)

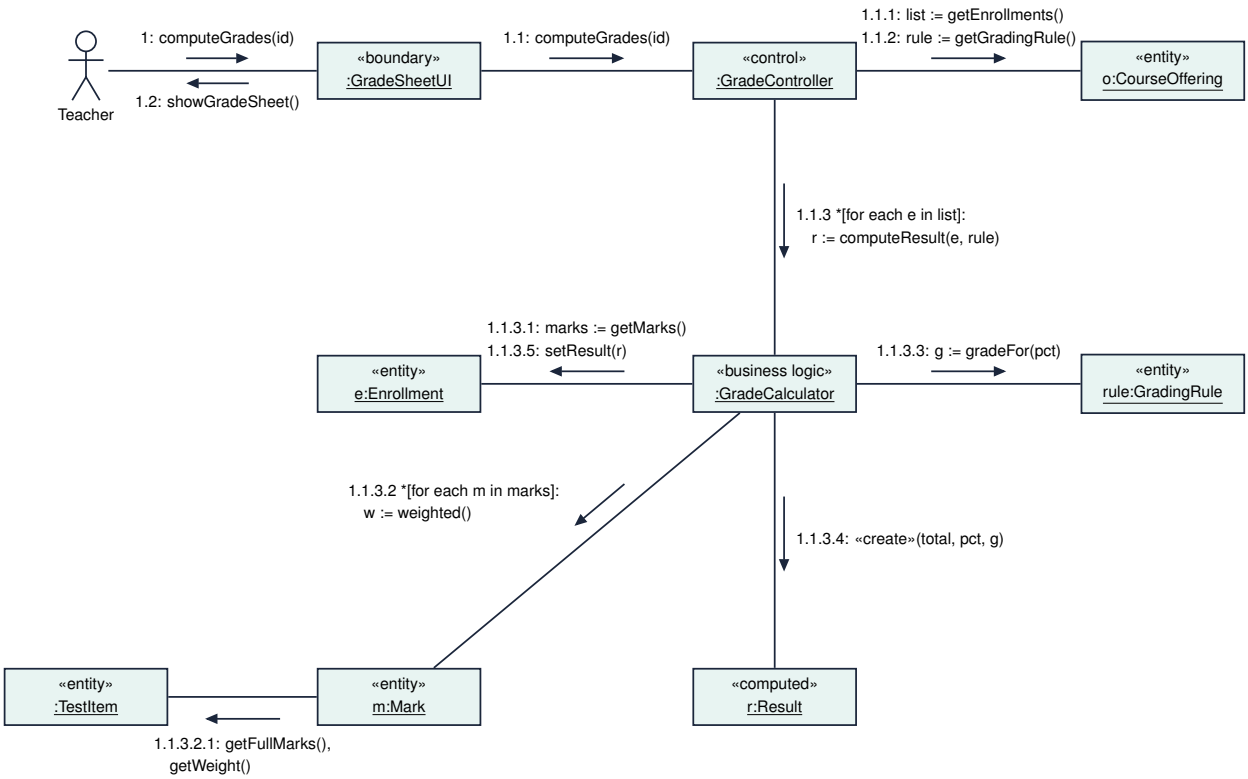
(b) Draw Collaboration diagram for the scenario described in Q 2(a) to show the computation of grade. You are required to show relevant Boundary and Controller classes.

SOLUTION

Objects taking part (consistent with the class diagram of Q2(a), C325-2021-2a)

- Actor **Teacher** triggers "compute grades" for one of his/her course offerings.
- «boundary» `:GradeSheetUI` — the screen where the teacher selects the offering and sees the grade sheet.
- «control» `:GradeController` — coordinates the use case *Compute Grades*.
- «entity» `:CourseOffering`, `:Enrollment`, `:Mark`, `:TestItem`, `:GradingRule`; business logic `:GradeCalculator`; computed value `:Result` (created).

Collaboration diagram – Compute Grades



How the computation proceeds

(i) The teacher asks the boundary object to compute grades (1); it forwards the request to the controller (1.1).

(ii) The controller fetches all enrollments and the grading rule of the offering (1.1.1, 1.1.2) — the offering gets the rule from its course.

(iii) For *each* enrollment (iteration `*[...]`) the controller delegates to the business-logic object `GradeCalculator` (1.1.3), which

- gets the marks of the enrollment (1.1.3.1);
- for each mark computes the weighted contribution $\frac{\text{obtained}}{\text{fullMarks}} \times \text{weight}$, asking the test item for full marks and weight (1.1.3.2, 1.1.3.2.1), and sums them;
- asks the grading rule for the letter grade / grade point of the percentage (1.1.3.3);
- creates a `Result` object storing the computed values (1.1.3.4) and attaches it to the enrollment (1.1.3.5).

(iv) The grade sheet is displayed to the teacher (1.2).

EXAM TIP Only objects joined by a *link* may exchange messages, so every link here must correspond to an association (or a parameter/local dependency) in the class diagram of Q2(a). Keep the controller thin: the calculation itself belongs to the business-logic class, not to the controller or the UI.

Answer. Teacher → GradeSheetUI (boundary) → GradeController (control) → CourseOffering (enrollments, rule); then *[for each enrollment] GradeCalculator.computeResult: getMarks, *[each mark] weighted() (TestItem full marks/weight), GradingRule.gradeFor(pct), create Result, setResult; finally 1.2 showGradeSheet.

2020–2021 • Q3(a) CSE325 • 12/04/2023 • C325-2021-3a **MARKS 15**

If you want to make a tour with a tourism operator, you have to select a package from their website. Each package presents a combination of site visit, hotels, food, activities. Depending on these components and number of days to spend, the price of the packages varies. A package can be customized for a group of tourists with size larger than 10 and price adjustment is made accordingly. The customer has to book a package with partial advance payment. The remaining payment is made prior to the tour to confirm all the components of the tour. If any activity is not available, the refund is made later. Draw a Sequence diagram to model the scenario discussed above.

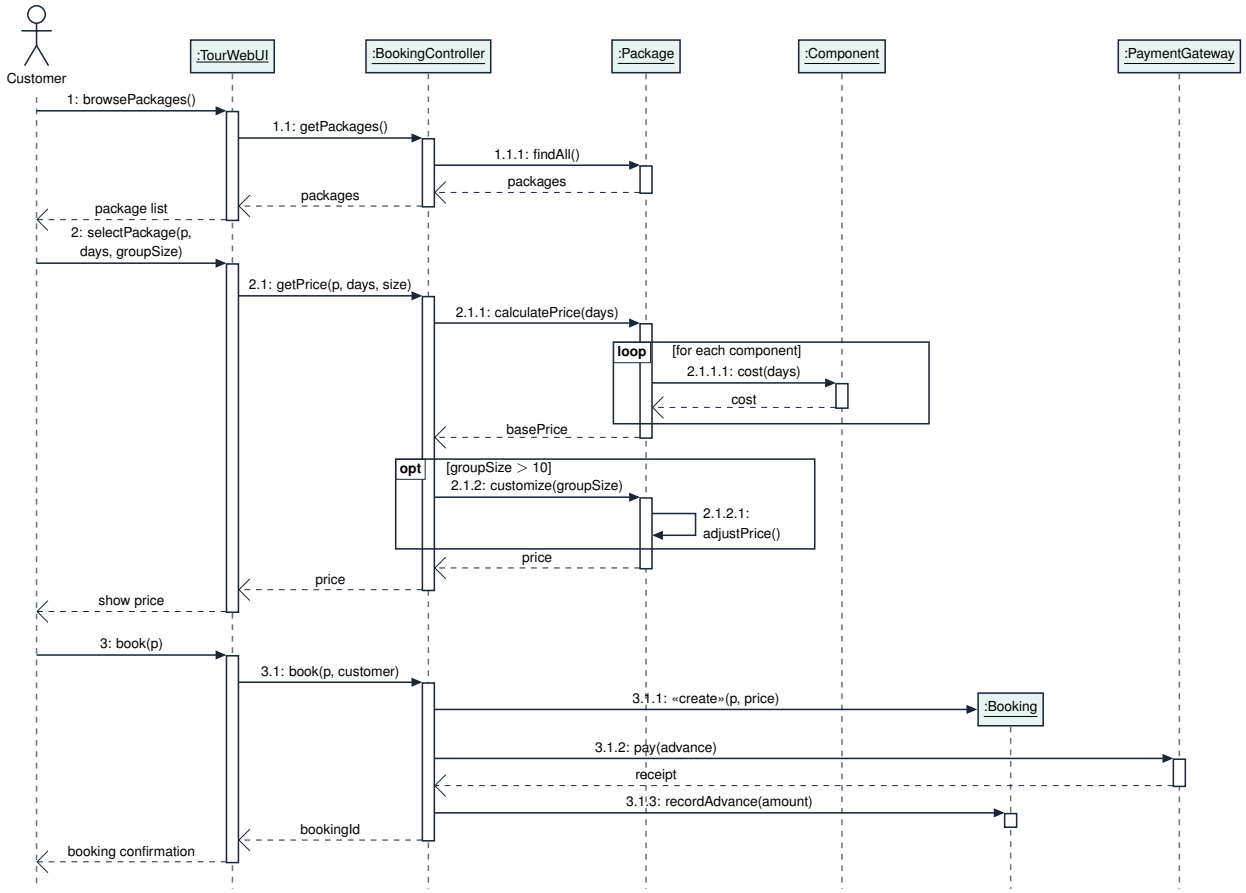
SOLUTION

Participants

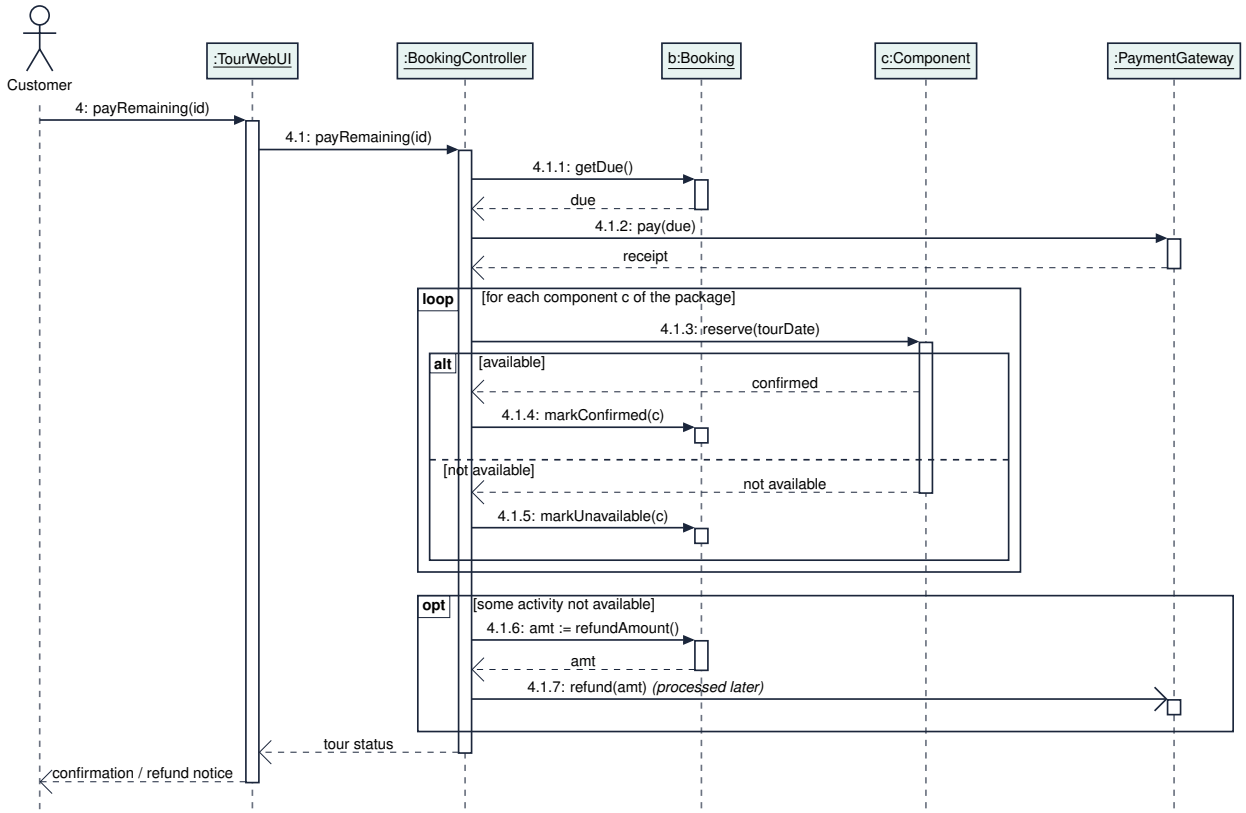
- Actor **Customer**; «boundary» `:TourWebUI` (the operator's website); «control» `:BookingController`;
- «entity» `:Package` (knows its components and computes its price), `:Component` (site visit, hotel, food or activity — each has its own cost and availability), `:Booking` (created when the customer books);
- external `:PaymentGateway` (advance, remaining payment and refund).

The scenario has two separate phases (booking now, final payment *prior to the tour*), so two sequence diagrams are drawn; both belong to the same interaction.

(1) Select, price, customise and book a package (advance payment)



(2) Remaining payment before the tour, confirming components, refund



Modelling decisions

- **Price computation** is the `Package`'s responsibility (information expert): it loops over its components (`loop` fragment) and the result depends on `days`.
- **Customisation** only happens for large groups: `opt [groupSize > 10]` with the price adjustment as a self-message.
- **Booking** is «create»d by the controller (its lifeline starts at the creation message) and records the advance payment received from the external payment gateway.
- **Final payment** confirms each component: a `loop` with a nested `alt` for available / not available; unavailable activities are marked on the booking.
- **Refund** is an `opt` fragment; it is sent as an *asynchronous* message (open arrowhead) because the money is returned later, the controller does not wait for it.

Answer. Customer → TourWebUI → BookingController → Package: calculatePrice (loop over components' cost), opt [group > 10] customize/adjustPrice; book: create Booking, pay(advance) via PaymentGateway, recordAdvance. Later: payRemaining → pay(due), loop over components reserve() with alt [available] markConfirmed / [not available] markUnavailable, opt [some unavailable] refund(amt) asynchronously.

2019–2020 • Q6(b) CSE325 • 02/04/2022 • C325-1920-6b **MARKS 12**

In a Payroll Management System, the salary of every company is managed. The salary comprises several components such as basic, house rent (a certain percentage of basic), medical allowance, etc. and deductible components like PF contribution, insurance subscription, etc. If an employee takes loan, the repayment is adjusted. At the beginning of a financial year, basic of all employees are incremented following a rule which defines the percentage of increment. However, each salary grade has a maximum bound. The increment will be applicable within this bound.
Based on the scenario discussed above, answer to the following questions a) and b):

(a) Draw the ER diagram of this system. (15)

(b) Draw Sequence diagram for the bulk salary increment usecase. (12)

[(a) – ER diagram – is outside the syllabus (ER diagrams as such) and is not solved here.]

SOLUTION

Step 1 – Participants (boundary – control – entity)

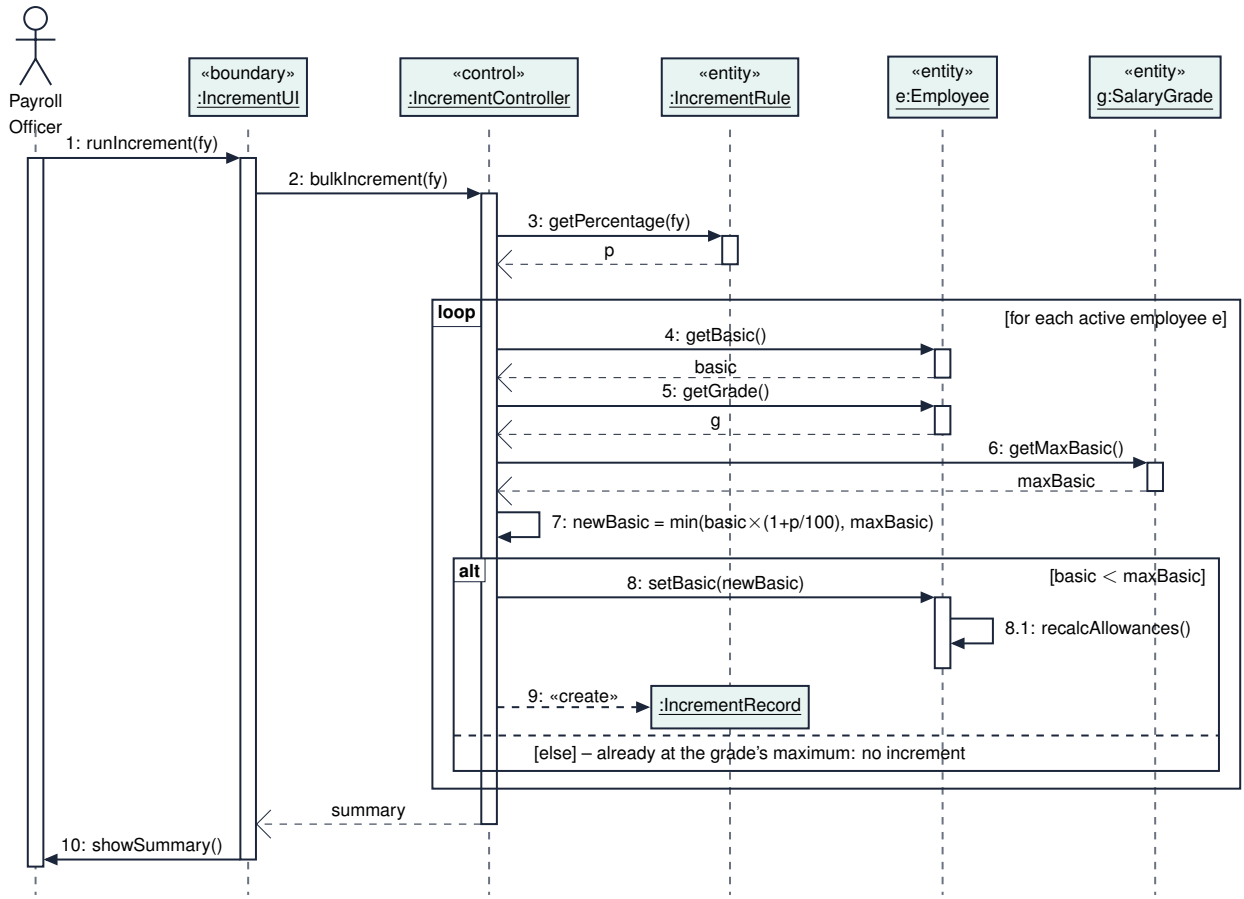
- **Actor** Payroll Officer starts the use case at the beginning of the financial year (it could equally be a *Time* actor; see the note).
- «boundary» :IncrementUI – the screen where the officer selects the financial year.
- «control» :IncrementController – applies the rule to every employee.
- «entity» :IncrementRule (percentage for the year), e:Employee (basic, grade), g:SalaryGrade (maximum bound), and a new :IncrementRecord per employee (audit/history).

Step 2 – Logic to show

For each employee: $\text{newBasic} = \min\big(\text{basic} \times (1 + p/100),\ \text{maxBasic of grade}\big)$. The repetition over *all* employees is a **loop** fragment; "applicable within this bound" is an **alt** fragment (employee

already at the bound gets nothing). Because house rent is a percentage of basic, the employee recalculates its dependent components after `setBasic()` (self-message).

Step 3 – Sequence diagram



Step 4 – Explanation

- The percentage is fetched *once*, before the loop (it is the same rule for everybody that year).
- Inside the **loop** the controller collects the three values needed for the formula (basic, grade, grade maximum) and computes the capped basic with a **self-message** (7).
- The **alt** fragment implements "the increment will be applicable within this bound": if the employee is already at the maximum the [else] operand does nothing; otherwise `min(...)` guarantees the new basic never exceeds the bound.
- 8.1 `recalcAllowances()` – nested numbering shows that e:Employee itself updates house rent (a percentage of basic) and other basic-dependent components.
- A new :IncrementRecord is «create»d (its box is drawn at the creation point) to keep the history of old and new basic for the financial year.

> **NOTE** If the system is expected to run the increment automatically on 1 July, replace the Payroll Officer by a *Time* actor (scheduler); the rest of the diagram is unchanged.

Answer. Officer → IncrementUI → IncrementController: getPercentage(fy) once; loop [each employee]: getBasic, getGrade, getMaxBasic, self-calc newBasic = min(basic(1+p), max); alt [basic < max] setBasic, recalcAllowances, create IncrementRecord / [else] nothing; return summary.

2019–2020 • Q7(a) CSE325 • 02/04/2022 • C325-1920-7a **MARKS 15**

Create a Collaboration diagram for the following scenario descriptions for a Gym Center's customer management system.
When members join the Gym Center, they pay a fee for a certain length of time (in months). The club wants to send SMS to members, asking them to renew their memberships one week before their memberships expire. About one-third of the members do not renew their memberships. These members are sent follow-up surveys to complete about why they decided not to renew so that the Center can learn how to increase retention. If the member did not renew because of cost, a special discount is offered to that customer. Typically, 12% of accounts are reactivated because of this offer.

SOLUTION

Concept

A **collaboration (communication) diagram** shows the *same* interaction as a sequence diagram but emphasises the *links* between objects: objects are placed freely, connected by link lines, and every message is written next to a link with a small arrow showing its direction and a *sequence number* (1, 1.1, 1.2, 2, . . .) that gives the order. Iteration is shown by `*[condition]` and a condition by `[guard]`.

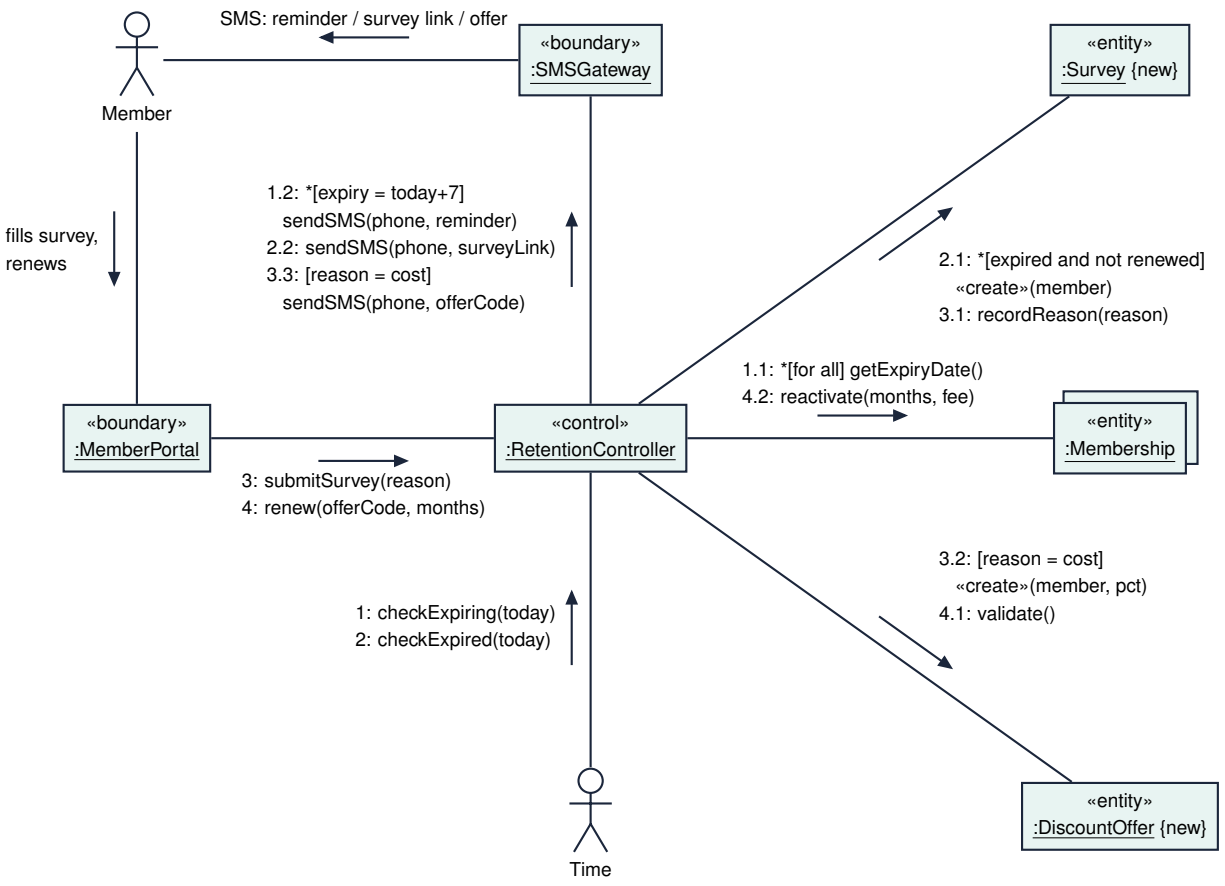
Step 1 – Objects

- **Actors**: Time (a daily scheduler – "one week before" and "do not renew" are time-triggered) and Member.
- «control» :RetentionController – coordinates reminders, surveys and offers.
- «entity» :Membership (multi-object, all memberships with expiry dates), :Survey and :DiscountOffer (created during the interaction).
- «boundary» :SMSGateway (sends SMS) and :MemberPortal (where the member answers the survey or renews).

Step 2 – Messages in order

1. `checkExpiring(today)` – daily trigger. 1.1 `*[for all] getExpiryDate()`; 1.2 `*[expiry = today + 7] sendSMS(phone, renewReminder)`.
2. `checkExpired(today)`. 2.1 `*[expired and not renewed] create(member)` a :Survey; 2.2 `sendSMS(phone, surveyLink)`.
3. Member `submitSurvey(reason)` via the portal; 3.1 `recordReason(reason)`; 3.2 `[reason = cost] create(member, discount%)` a :DiscountOffer; 3.3 `[reason = cost] sendSMS(phone, offerCode)`.
4. Member `renew(offerCode, months)`; 4.1 `validate()` the offer; 4.2 `reactivate(months, discountedFee)` the membership.

Step 3 – Collaboration diagram



Step 4 – Explanation of the decisions

- **Sequence numbers** replace the vertical time axis: 1.x is the reminder run, 2.x the survey run for lapsed members, 3.x the processing of a survey answer and 4.x the renewal with the offer. Nested numbers (1.1, 1.2) are messages sent *while* handling message 1.
- **Iteration** `*[...]` on 1.1/1.2/2.1 – the controller scans *all* memberships, which is why :Membership is drawn as a *multi-object* (stacked boxes).
- **Guards**: "one week before" → `[expiry = today+7]`; "do not renew" → `[expired and not renewed]`; "because of cost" → `[reason = cost]`.
- :Survey and :DiscountOffer are marked {new}: they are «create»d during this collaboration.
- "One-third do not renew" and "12% are reactivated" are *statistics*, not behaviour; they are not messages (they could be produced by a report use case).

Answer. Time → RetentionController: 1 checkExpiring (1.1 *getExpiryDate on :Membership, 1.2 *[today+7] sendSMS via SMSGateway); 2 checkExpired (2.1 *[not renewed] create :Survey, 2.2 sendSMS link); Member via MemberPortal: 3 submitSurvey (3.1 recordReason, 3.2 [cost] create :DiscountOffer, 3.3 sendSMS offer); 4 renew (4.1 validate offer, 4.2 reactivate membership).

2018–2019 • Q3(a) CSE325 • 20/12/2021 • C325-1819-3a **MARKS 21**

Design a Sequence diagram for the scenario of a bank described in Question 2(a).
Scenario of Q2(a): There are two types of users in a Bank: Normal user and Foreign User. Both users can create account, deposit money and withdraw money. For money withdrawal there are certain rules. A user can make a withdrawal of maximum 3,00,000 taka per day. At each transaction the highest limit of withdrawal is 1,00,000 taka. A user can make a maximum of 5 withdrawals per day. After every three months, interest is added to the account balance. If an account balance reaches 20,00,000 taka, for every deposit to that account 2% bonus is applied. The foreign user has one extra capability. He can convert his taka to foreign currency.

SOLUTION

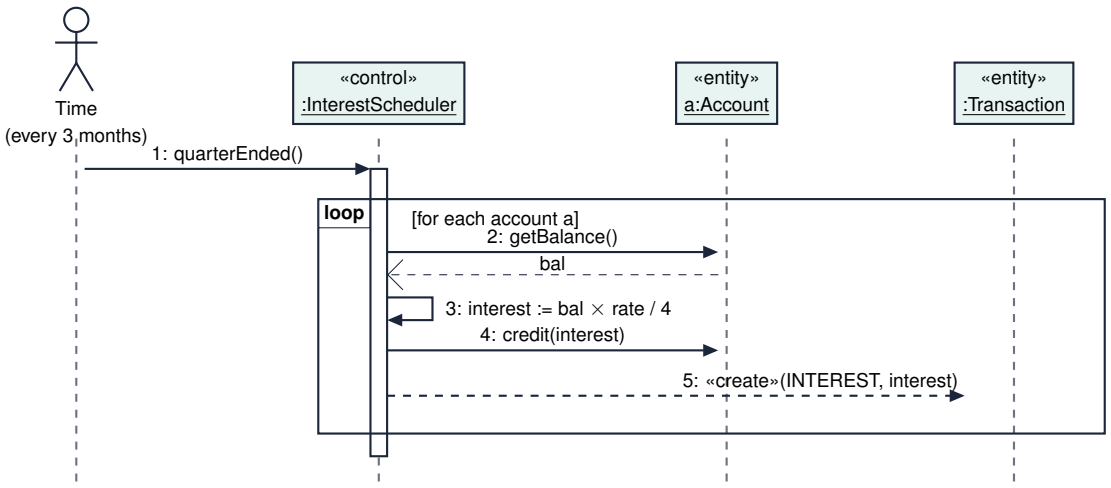
NOTE The same bank narrative appears in CSE307 2016–2017 (R016.2), but there only the use case "withdraw money" is asked. Here the whole scenario is asked, so the answer below covers every behaviour: create account, deposit with bonus, withdraw with the three limits, currency conversion (foreign user only) and quarterly interest.

Step 1 – Participants

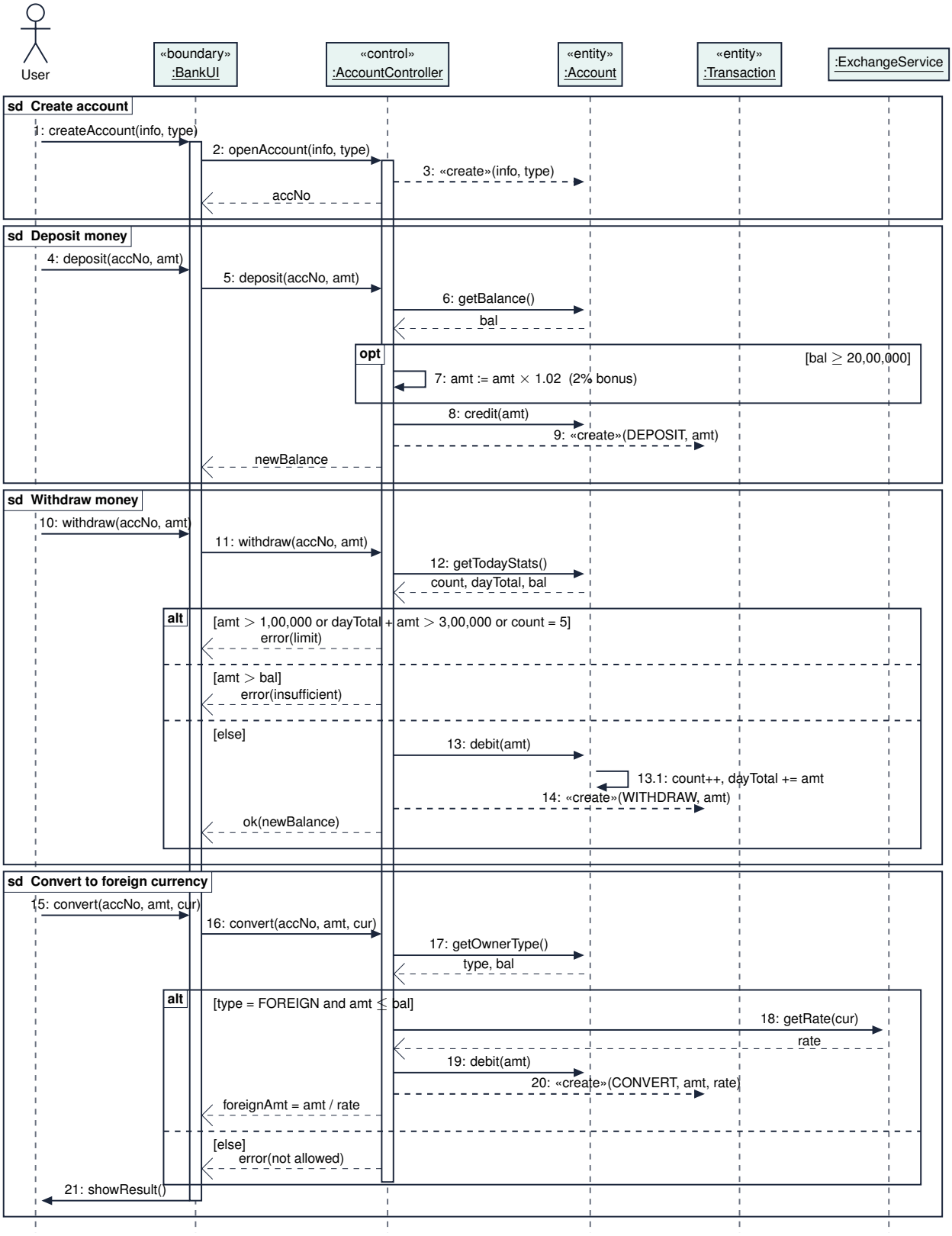
- Actors: User (normal or foreign) and Time (quarterly trigger for interest).
- «boundary» :BankUI; «control» :AccountController; «entity» :Account (balance, today's withdrawal count and total, owner type) and new :Transaction records; external :ExchangeService (currency rates).

Each behaviour of the scenario becomes one block of the interaction, combined fragments carry the rules: `opt` for the 2% bonus and for the foreign-only conversion, `alt` for the withdrawal limits and `loop` for the interest run.

Step 2 – Sequence diagram (1): quarterly interest (time-triggered)



Step 3 – Sequence diagram (2): user-initiated operations



NOTE For compactness the :Account head is drawn at the top; strictly, in the "Create account" frame the new account object begins its life at message 3 («create»). :Transaction heads likewise stand for the records created by messages 9, 14 and 20.

Step 4 – Explanation of the decisions

- **One diagram per triggering actor**: user operations start from the User; interest posting starts from Time, which is an actor because "after every three months" is a time event. Each user operation is framed (`sd`/`ref` labels) so that the long interaction stays readable.
- **Bonus rule** ("if balance reaches 20,00,000, every deposit gets 2% bonus") is an `opt` fragment: executed only when the guard holds, otherwise skipped.
- **Withdrawal rules** are an `alt` with three operands: any limit violated (per-transaction 1,00,000; daily 3,00,000; 5 per day), insufficient balance, and the normal path in which the balance is debited, the daily counters updated (self-message 13.1) and a :Transaction created so that the next withdrawal can be checked.
- **Foreign user's extra capability**: the controller checks the owner type and the conversion operand runs only for `type = FOREIGN`; the exchange rate comes from an external service.
- **Creation messages** (dashed with «create») make every money movement leave an audit record.

Answer. (1) User → BankUI → AccountController → Account: createAccount (create a:Account); deposit with opt [bal ≥ 20 lakh] 2% bonus, credit, create Transaction; withdraw with alt [limit violated] error / [amt > bal] error / [else] debit, update counters, create Transaction; convert with alt [FOREIGN] getRate, debit, create Transaction / [else] error. (2) Time → InterestScheduler: loop [each account] getBalance, compute, credit(interest), create Transaction.

2017–2018 • Q2(a) CSE325 • 10/04/2019 • C325-1718-2a MARKS 13

Refer to the scenario mentioned in 1(a). Now, draw a collaboration diagram which accurately represents the scenario. Note that, solving 1(a) is not mandatory for this step. However, make sure that the function names accurately represent the tasks they perform.

Scenario of 1(a): A language teacher wants to register a particular student for an online French course. After he logs in, he is shown a list of students. When he selects a particular student, a list of languages is displayed. After he selects a language (say, French), he is shown a list of course on French that the selected student has not taken yet. When he selects a course, the course assignment is complete.

SOLUTION

What a collaboration diagram must show

A collaboration (communication) diagram shows the objects that take part in one scenario, the **links** between them (a message can only travel along a link) and the **messages**, each written next to its link with a small arrow for its direction and a **sequence number**. Time is shown only by the numbers: a nested number (1.1, 1.1.1) is a message sent *while* handling the enclosing message.

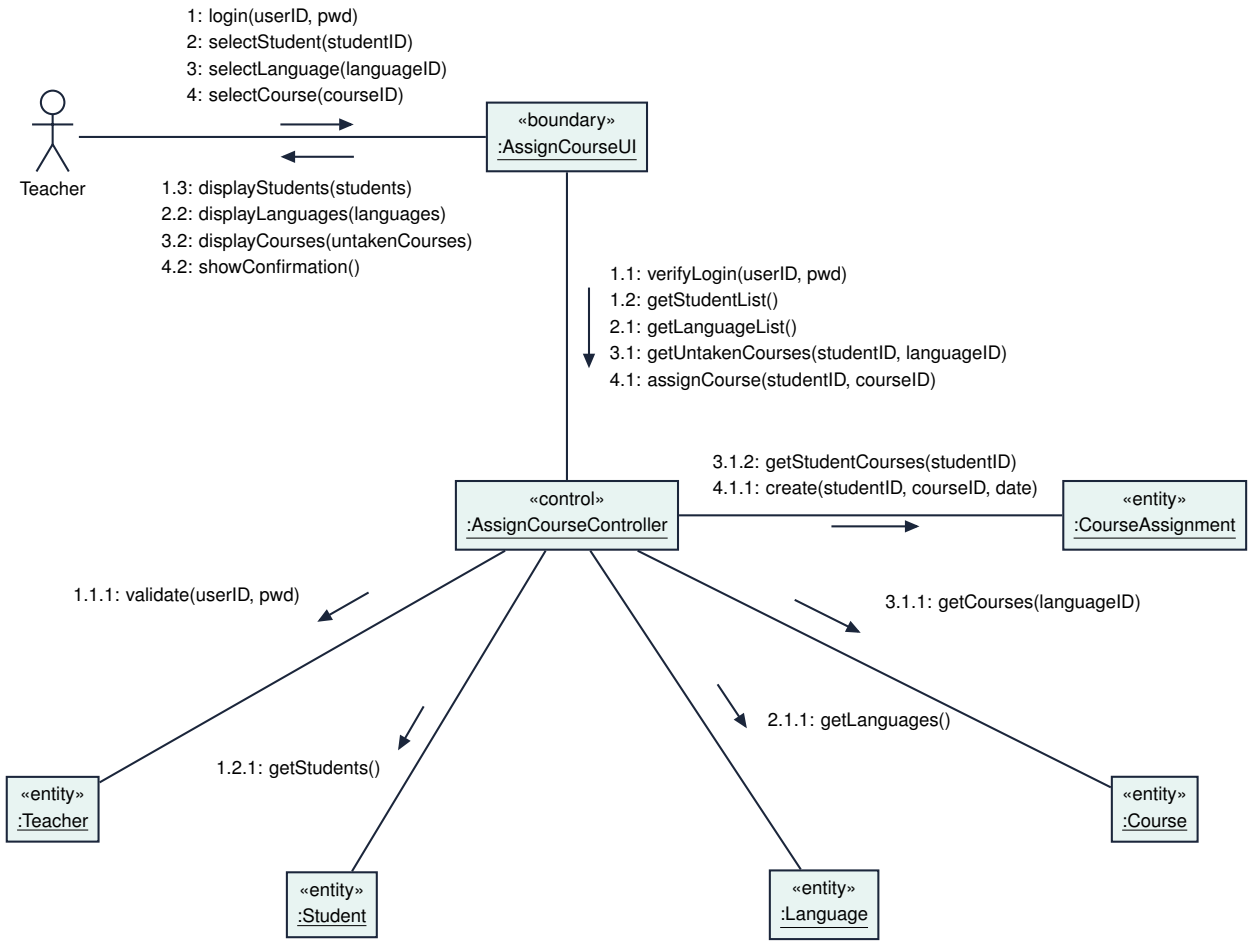
Participating objects (boundary–control–entity)

- Actor **Teacher**.
- «boundary» **:AssignCourseUI** – the screens the teacher uses.
- «control» **:AssignCourseController** – carries out the use case "Assign course".
- «entity» **:Teacher** (login data), **:Student**, **:Language**, **:Course** and **:CourseAssignment** (which student has taken which course, with registration date).

Message list derived from the scenario

Scenario step	Messages (sender → receiver)
Teacher logs in	1: login(userID, pwd) Teacher → UI 1.1: verifyLogin(userID, pwd) UI → Controller 1.1.1: validate(userID, pwd) Controller → :Teacher
List of students is shown	1.2: getStudentList() UI → Controller 1.2.1: getStudents() Controller → :Student 1.3: displayStudents(students) UI → Teacher
Selects a student, languages are shown	2: selectStudent(studentID) Teacher → UI 2.1: getLanguageList() UI → Controller 2.1.1: getLanguages() Controller → :Language 2.2: displayLanguages(languages) UI → Teacher
Selects a language, untaken courses are shown	3: selectLanguage(languageID) Teacher → UI 3.1: getUntakenCourses(studentID, languageID) UI → Controller 3.1.1: getCourses(languageID) Controller → :Course 3.1.2: getStudentCourses(studentID) Controller → :CourseAssignment 3.2: displayCourses(untakenCourses) UI → Teacher
Selects a course, assignment completes	4: selectCourse(courseID) Teacher → UI 4.1: assignCourse(studentID, courseID) UI → Controller 4.1.1: create(studentID, courseID, date) Controller → :CourseAssignment 4.2: showConfirmation() UI → Teacher

Collaboration diagram



Modelling decisions

- **Only linked objects communicate.** The actor talks only to the boundary; the boundary talks only to the controller; only the controller talks to the entities. This keeps the user interface independent of the stored data (and gives the class diagram of 1(a) the associations it was missing).
- **Nested numbering shows causality**: e.g. 3.1 is sent while the UI handles 3 (the language selection), and 3.1.1, 3.1.2 are sent while the controller handles 3.1. The controller then removes the courses returned by 3.1.2 from the list returned by 3.1.1, so 3.2 shows only the *untaken* French courses.
- **Function names describe the work**: `getUntakenCourses(studentID, languageID)` rather than a vague `GetLanguageCourses()`, `assignCourse(studentID, courseID)` that ends in `create(...)` on `:CourseAssignment` (the assignment is a new record, not a change to `Student`).
- The selected `studentID` and `languageID` are remembered by the UI between steps and passed as parameters.

> **NOTE** Login is modelled with a `:Teacher` entity that checks the credentials; a failed login (alternative flow) is not drawn because the scenario describes only the successful path.

Answer. Teacher → :AssignCourseUI → :AssignCourseController → entities. Messages 1 login (1.1 verifyLogin, 1.1.1 validate), 1.2 getStudentList (1.2.1 getStudents), 1.3 displayStudents; 2 selectStudent, 2.1/2.1.1 get languages, 2.2 displayLanguages; 3 selectLanguage, 3.1 getUntakenCourses (3.1.1 getCourses(languageID), 3.1.2 getStudentCourses(studentID)), 3.2 displayCourses; 4 selectCourse, 4.1 assignCourse, 4.1.1 create on :CourseAssignment, 4.2 showConfirmation.

2017–2018 • Q2(c) CSE325 • 10/04/2019 • C325-1718-2c **MARKS 2 × 5 = 10**

Write one/two word answers to the following questions on Sequence Diagrams.

(i) The execution of an operation in a sequence diagram is shown by _____.

(ii) An asynchronous message is drawn with _____.

(iii) Returns are drawn with _____.

(iv) Is focus of control the same as activation?

(v) What is the one major difference between sequence diagrams and collaboration diagrams?

SOLUTION

Answers

(i) **An activation** (activation box / execution occurrence): a thin rectangle drawn on the object's lifeline for as long as the operation is executing (its method is on the stack).

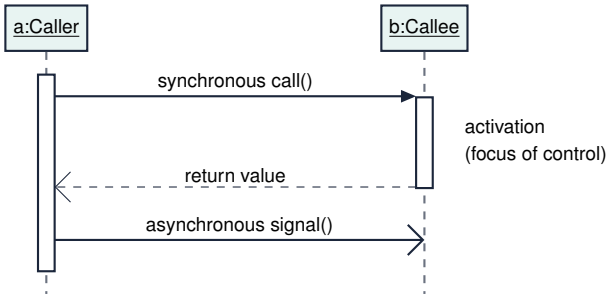
(ii) **A solid line with a stick (open) arrowhead** – the sender does not wait for a reply. (A synchronous call uses a solid line with a filled arrowhead.)

(iii) **A dashed line with a stick (open) arrowhead**, drawn back to the caller.

(iv) **Yes.** "Focus of control" is the older UML name for the activation bar: both mean the period

during which the object is performing an action.

(v) **Emphasis:** a sequence diagram emphasises the *time ordering* of messages (vertical time axis), while a collaboration diagram emphasises the *structural organisation* of the objects (links), showing order only by sequence numbers.



Answer. (i) activation box; (ii) solid line + stick arrowhead; (iii) dashed line + stick arrowhead; (iv) yes; (v) sequence = time ordering, collaboration = structural links (order by numbers).

2016–2017 • Q2(c) CSE307 • 15/07/2017 • R016.2 **MARKS 13**

The following is a narrative description of the business process of a bank:
There are two types of users in a Bank: Normal user and Foreign User. Both users can create account, deposit money and withdraw money. For money withdrawal there are certain rules. A user can make a withdrawal of maximum 3,00,000 taka per day. At each transaction the highest limit of withdrawal is 1,00,000 taka. A user can make a maximum of 5 withdrawals per day. After every three months, interest is added to the account balance. If an account balance reaches 20,00,000 taka, for every deposit to that account 2% bonus is applied. The foreign user has one extra capability. He can convert his taka to foreign currency.
The use cases of this scenario are: (i) Create account (ii) Deposit money (iii) Withdraw money (iv) Update balance (v) Calculate bonus (vi) Calculate interest and (vii) Convert to foreign currency.

(c) Draw sequence diagram for the use case "withdraw money".

SOLUTION

Step 1 – Participants (boundary–control–entity)

Following the lecture (and Bennett's analysis style) one interaction is realised by

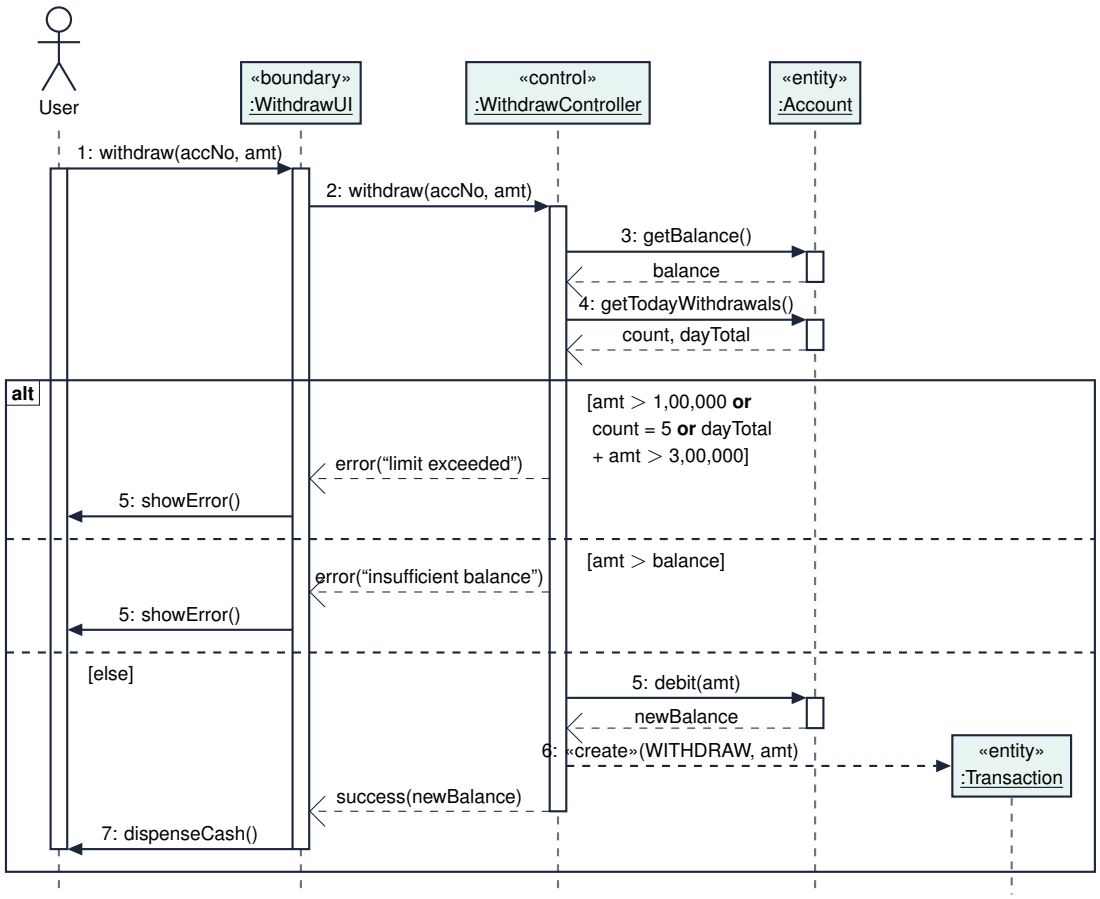
- the **actor** User (normal or foreign – both can withdraw);
- a «boundary» object :WithdrawUI (teller/ATM screen);
- a «control» object :WithdrawController that applies the business rules;
- «entity» objects :Account (balance, today's count and total) and a new :Transaction record (needed to check the daily rules next time).

Step 2 – Business rules to show

Per transaction $\leq$ 1,00,000; per day $\leq$ 3,00,000; at most 5 withdrawals per day; and of course the balance must be sufficient. These conditions select one of three outcomes, so they are modelled

with an **alt** combined fragment. "Update balance" (a use case included by withdraw) appears as the `debit()` message to the account.

Step 3 – Sequence diagram



Explanation of the decisions

- **Why a controller?** The withdrawal rules belong neither to the screen nor to the account alone; a control object coordinates them (high cohesion, the UI stays thin).
- **Data needed for the rules** is fetched first (`getBalance()`, `getTodayWithdrawals()`); the returns are drawn dashed.
- **alt fragment** with three operands separated by dashed lines: limit violation, insufficient balance, and the normal path. Exactly one operand executes; its guard is written at its top.
- In the success operand the account balance is updated (`debit`) and a **new** :Transaction object is created — its box is drawn *lower*, at the moment of creation. This record is what makes "5 per day" and "3,00,000 per day" checkable on the next withdrawal.
- **Activations** show that the UI and controller remain active (waiting) during the whole use case, while the account is active only while serving each call.

Answer. User → «boundary» WithdrawUI → «control» WithdrawController, which queries «entity» Account for balance and today's count/total, then an `alt` fragment: [limit exceeded] error / [amt > balance] error / [else] `debit(amt)`, «create» Transaction, success, dispense cash.

2014–2015 • Q7(c) CSE307 • 04/02/2016 • R021.5 **MARKS 9 2/3**

Explain two different types of interaction diagrams with example.

SOLUTION

What an interaction diagram is

An **interaction diagram** shows how a group of objects collaborate, by exchanging *messages*, to realise one scenario of a use case. UML offers two equivalent forms that hold the *same information* but emphasise different things:

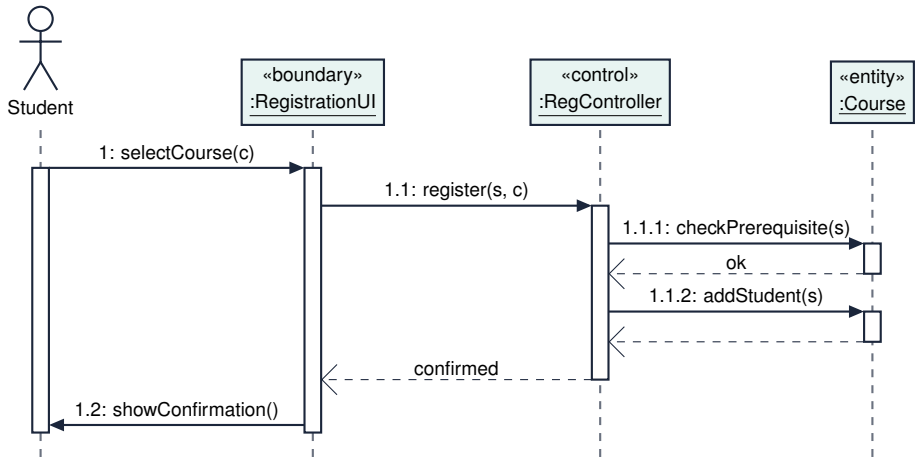
(i) **Sequence diagram** – emphasises the *time ordering* of messages.

(ii) **Collaboration diagram** (called *communication diagram* in UML 2) – emphasises the *structural links* between the participating objects.

Running example (course registration): a student selects a course; the registration controller checks the prerequisite and adds the student to the course; a confirmation is shown.

1. Sequence diagram

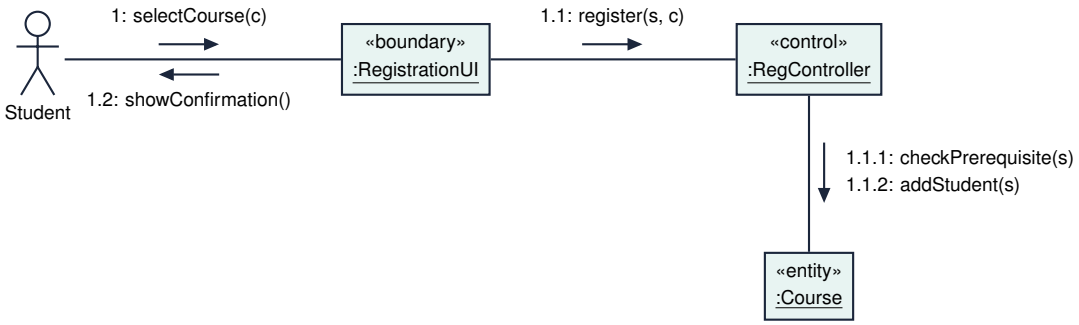
- Participants are placed horizontally at the top (name:Class boxes or actor figures); *time runs downward*.
- Each participant has a dashed **lifeline**; a thin rectangle on it is an **activation** (the object is executing or waiting for a call it made to return).
- Messages are horizontal arrows: solid head = synchronous call, stick head = asynchronous, dashed = return. `alt`, `opt`, `loop` **frames** show branching and repetition.



2. Collaboration (communication) diagram

- The same objects are placed freely on the page and joined by **links** (instances of associations); there is *no time axis*.
- Each message is written next to a link with a small arrow showing its direction; the order is given by **sequence numbers**, written in a nested style (1, 1.1, 1.1.1, . . .) to show which message is sent while handling which.

- Iteration is shown by `*[condition]`, branching by guards `[condition]` (and suffixes such as 14a / 14b in the lecture example).



Comparison

	Sequence diagram	Collaboration diagram
Emphasis	time order of messages	structural organisation (links) of objects
Time	explicit vertical axis	implicit, only via sequence numbers
Links shown?	no	yes – a message needs a link
Lifelines / activations	yes	no
Best for	complex control flow, many messages, `alt`/`loop` frames	seeing which objects must know each other; checking against the class diagram

Answer. The two interaction diagrams are the **sequence diagram** (objects on lifelines, messages ordered top-to-bottom in time, activations and frames) and the **collaboration/communication diagram** (objects joined by links, messages ordered by nested sequence numbers). Both describe the same interaction; one stresses time, the other structure.

2013–2014 • Q7(b) CSE307 • 08/08/2015 • R027.2 MARKS $16\frac{2}{3}$

Draw a sequence diagram to represent the following scenario:
A patient arrives at a Magnetic Resonance Imaging (MRI) center carrying a doctor's referral form. A clerk in the Intake Department reviews the referral form for completeness, checks the Active Doctor file to verify that the doctor is in good standing with the department, and signs the doctor's referral form. The intake department clerk then prepares a 3-part MRI assignment form, which identifies the MRI machine and technician. The third copy of the MRI assignment is mailed to the doctor, the second copy is recorded by date with the referral, and the first copy is forwarded to the Imaging Department. A technician in the imaging department performs the imaging procedure, updates the assignment form to include information about when the procedure was performed.

SOLUTION

Step 1 – Participants

Reading the scenario for *who acts* and *what is acted upon*:

- People (drawn as actors): Patient, Intake Clerk, Doctor (receives mail), Technician (Imaging Department).
- Objects (documents/files): :ReferralForm, :ActiveDoctorFile, :MRIAssignmentForm (created during the scenario, three copies) and :ReferralFile (where copy 2 is recorded by date with the referral).

Step 2 – Messages in time order

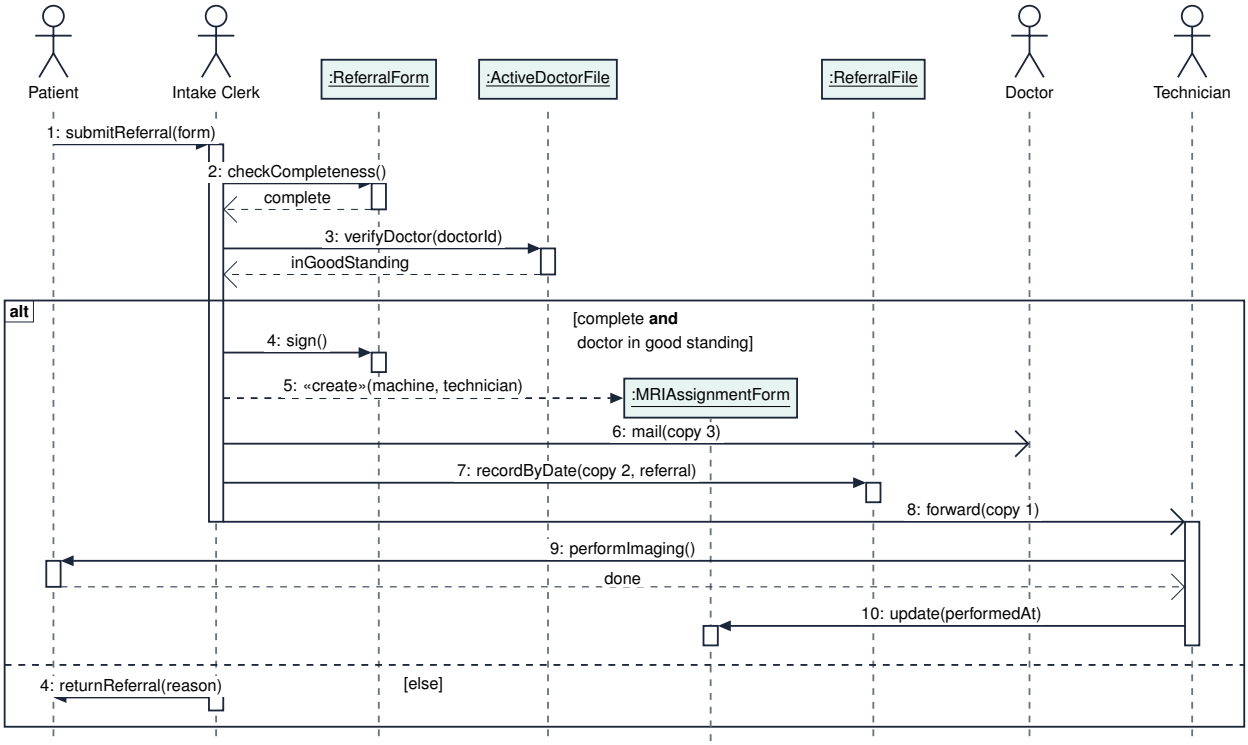
(i) Patient hands over the referral; the clerk checks completeness and the doctor's standing.

(ii) If both are fine (`alt` guard): the clerk signs the form and creates the assignment form.

(iii) The three copies go out: mail to the doctor and forward to imaging are *asynchronous* (the clerk does not wait); recording copy 2 is a synchronous filing operation.

(iv) The technician performs the imaging and updates the assignment form with the time.

Step 3 – Sequence diagram



Explanation

- **Object creation:** the MRI assignment form does not exist at the start; its box is drawn at the level of message 5 («create»). Its lifeline continues because the technician updates it later (message 10).
- **Asynchronous vs synchronous:** mailing a copy to the doctor and forwarding a copy to the Imaging Department are fire-and-forget (stick arrowheads); the clerk continues working. Checking, verifying, signing and recording are synchronous (filled heads, with returns where a result matters).

- **Activations** show when each participant is busy: the clerk from receiving the referral until the copies are dispatched; the technician from receiving copy 1 until the form is updated.
- **alt fragment:** the narrative only describes the success path; an incomplete form or a doctor not in good standing is handled in the `[else]` operand.

> **NOTE** The scenario does not say what happens when the referral is incomplete or the doctor is not in good standing; we assume the referral is returned to the patient (the `[else]` operand). The printed text "A technician is the imaging department" is read as "A technician *in* the imaging department".

Answer. Patient → Clerk (submitReferral); Clerk checks ReferralForm and ActiveDoctorFile; in the `alt [ok]` operand: sign, «create» MRIAssignmentForm, async mail(copy 3) to Doctor, recordByDate(copy 2) in ReferralFile, async forward(copy 1) to Technician, who performs imaging on the patient and updates the assignment form.

2012–2013 • Q5(c) CSE307 • 17/05/2014 • R031.1 **MARKS 9**

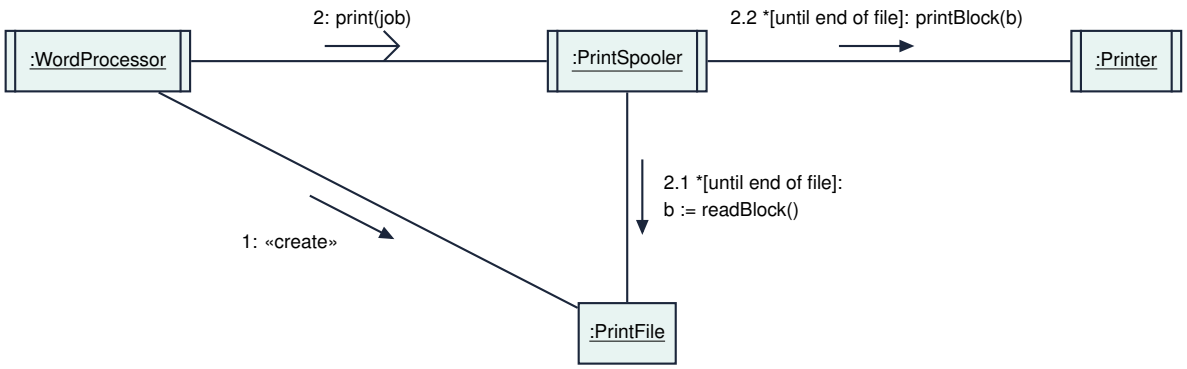
Draw a collaboration diagram to illustrate the following scenario: a word-processor (active object) creates a print file and then asynchronously requests a print spooler (active object) to print the job. The print spooler repeatedly reads a block from the print file and sends the block to the printer (active object) using a procedure call.
Also draw an activity diagram of the *print file* operation.

SOLUTION

Notation used

- An **active object** owns its own thread of control; UML draws it as an object box with *double vertical sides* (older notation: thick border, or the property `{active}`).
- Collaboration-diagram messages carry **sequence numbers**; nested numbers (2.1, 2.2) mean "sent while handling message 2". `*[condition]` marks **iteration**.
- Arrow heads: *stick* head = asynchronous (sender continues), *filled* head = synchronous procedure call (sender waits).

Collaboration diagram

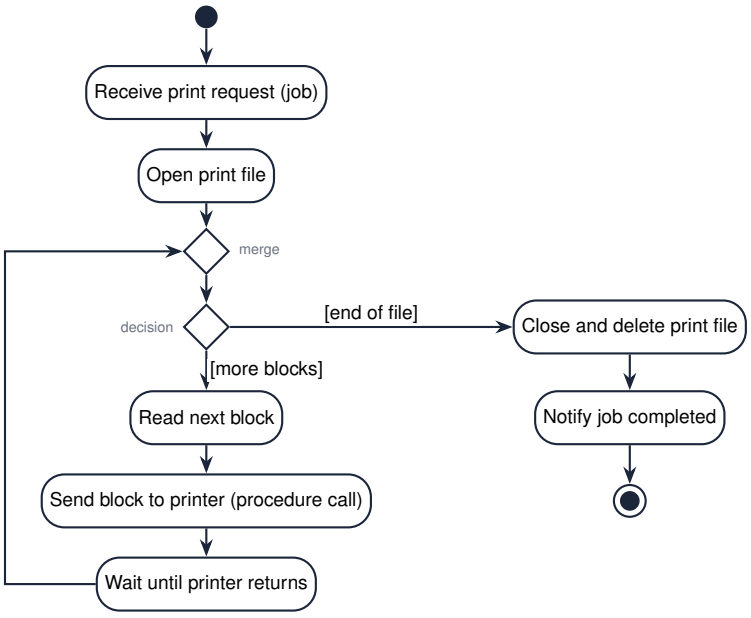


Explanation

- WordProcessor, PrintSpooler and Printer are active (double sides); PrintFile is a passive data object.

- **1: «create»** – the word processor first creates the print file (link to it).
- **2: print(job)** – asynchronous (stick head): the word processor does not wait and the user can go on editing; the spooler runs in its own thread.
- **2.1 and 2.2** are nested under 2 because the spooler sends them while handling the print request; the `*[until end of file]` marks repetition: each iteration reads one block (2.1) and then passes it to the printer with a *procedure call* (2.2, filled head, the spooler waits until the printer accepts the block).

Activity diagram of the *print file* operation (performed by the spooler)

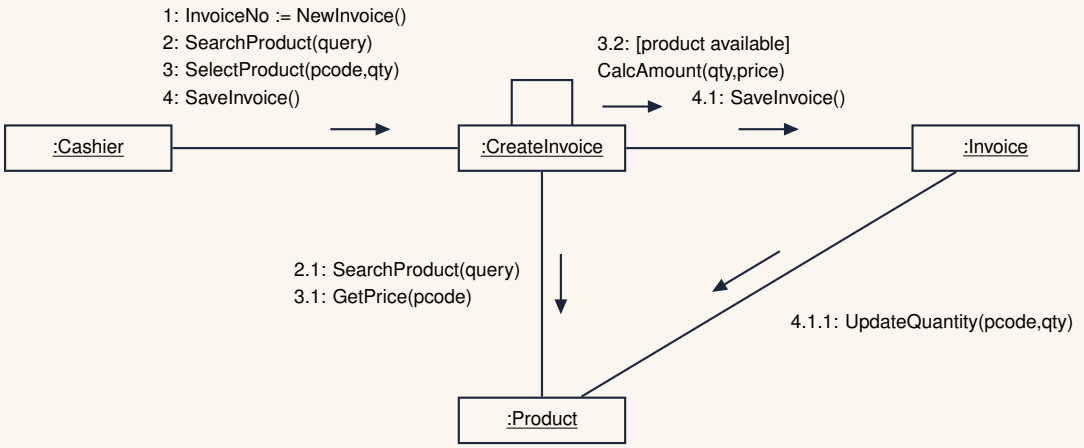


- The **decision** node with guards `[more blocks]` / `[end of file]` and the **merge** node above it form the loop that the collaboration diagram wrote as `*[until end of file]`.
- "Send block to printer" followed by "Wait until printer returns" expresses the synchronous procedure call.

Answer. Collaboration: active :WordProcessor –1: «create»–> :PrintFile; –2: print(job) (async)–> active :PrintSpooler; spooler: 2.1 *[until EOF]: readBlock() on :PrintFile and 2.2 *[until EOF]: printBlock(b) (sync call) on active :Printer. Activity: open file → loop {read block, send, wait} → close/delete file → notify.

2012–2013 • Q6(a) CSE307 • 17/05/2014 • R031.3 **MARKS 10 + 2 + 4**

A grocery store has just implemented a new sales system. The system generates invoice for each sale. Consider the following collaboration diagram for *creating invoice*.



Now answer the following:

(i) Draw the sequence diagram for *creating invoice*.

(ii) Include stereotypes for each classes.

(iii) Find the UML Class for **Invoice**. Include everything that can be determined from the above diagram such as attributes, methods. You do not need to include information which is not contained in the above diagram.

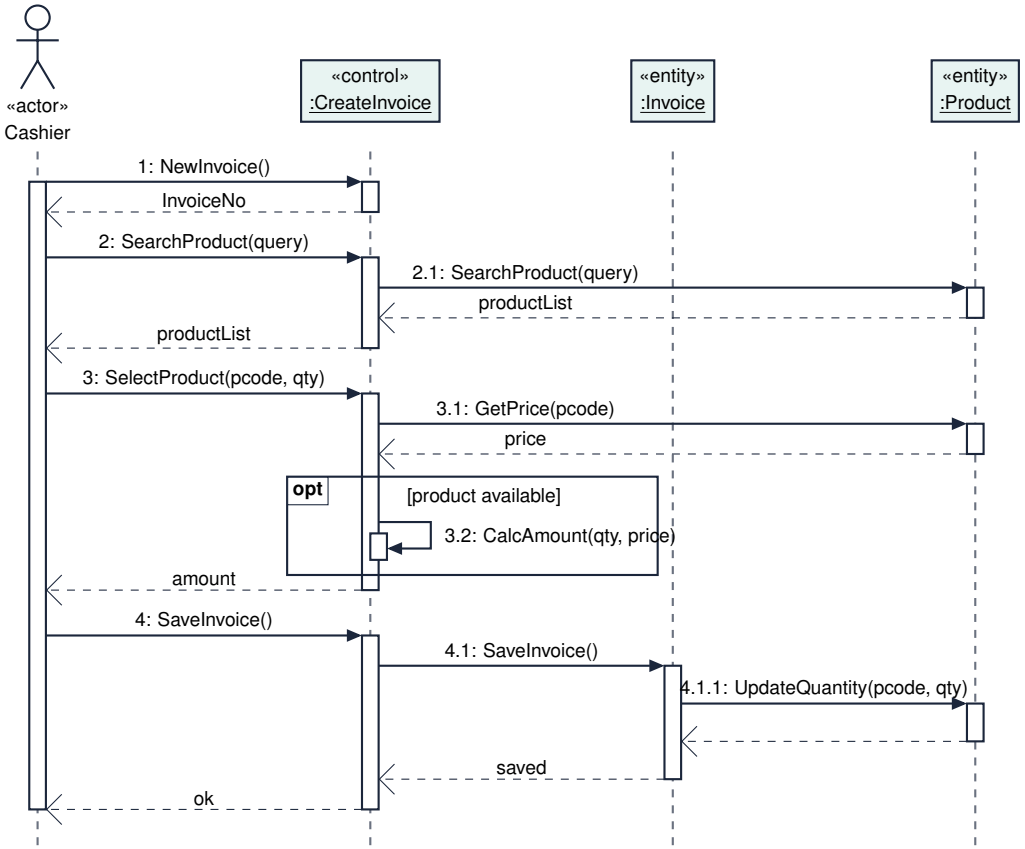
SOLUTION

Reading the collaboration diagram

Sequence numbers fix the order; nested numbers tell *who* sends a message while handling which one: 2.1 is sent by CreateInvoice while handling 2, 4.1.1 is sent by Invoice while handling 4.1, and so on. Converting to a sequence diagram therefore means: one lifeline per object, messages top-to-bottom in number order, nested messages inside the activation of their parent message.

No.	Sender	Receiver	Message	Remark
1	Cashier	CreateInvoice	`NewInvoice()`	returns `InvoiceNo`
2 / 2.1	Cashier / CreateInvoice	CreateInvoice / Product	`SearchProduct(query)`	forwarded to Product
3 / 3.1	Cashier / CreateInvoice	CreateInvoice / Product	`SelectProduct`, `GetPrice(pcode)`	price returned
3.2	CreateInvoice	CreateInvoice	`CalcAmount(qty,price)`	self message, guard
4 / 4.1	Cashier / CreateInvoice	CreateInvoice / Invoice	`SaveInvoice()`	
4.1.1	Invoice	Product	`UpdateQuantity(pcode,qty)`	stock reduced

(i) + (ii) Sequence diagram with stereotypes



(ii) Stereotypes. Using the analysis-class stereotypes of the lecture (Bennett):

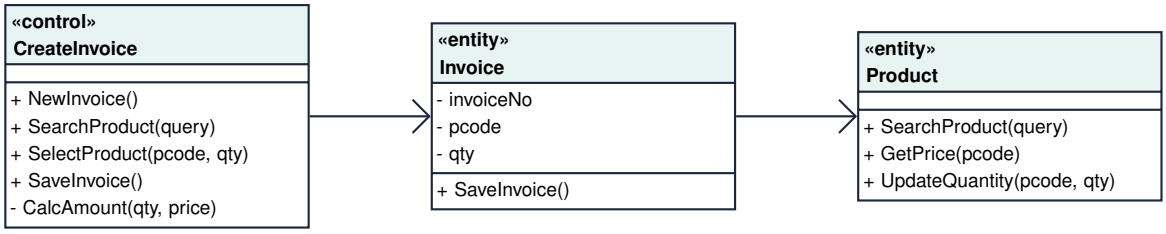


- Cashier – «actor»: external user who starts the interaction.
- CreateInvoice – «control»: it coordinates the use case (it only forwards searches, asks for prices, calculates the amount and orders the save); it holds no permanent data.
- Invoice, Product – «entity»: persistent business information (invoices are saved, product stock is updated).
- No «boundary» object appears in the given diagram; in a full design the cashier would talk to a «boundary» :InvoiceUI placed between Cashier and CreateInvoice.

(iii) UML class for Invoice

Only what the diagram reveals:

- Invoice *receives* `SaveInvoice()` ⇒ operation `+SaveInvoice()`.
- It *sends* `UpdateQuantity(pcode, qty)` to Product ⇒ it must know a `pcode` and a `qty` (attributes) and needs a navigable association to Product.
- `NewInvoice()` returns an `InvoiceNo` ⇒ attribute `invoiceNo`.
- CreateInvoice sends to it ⇒ association CreateInvoice → Invoice.



Answer. (i) Lifelines Cashier, CreateInvoice, Invoice, Product; messages 1, 2 → 2.1, 3 → 3.1 then `opt [product available]` self call 3.2 CalcAmount, 4 → 4.1 → 4.1.1 UpdateQuantity. (ii) Cashier «actor», CreateInvoice «control», Invoice and Product «entity». (iii) «entity» Invoice {invoiceNo, pcode, qty; SaveInvoice()}, navigable association to Product.

2011–2012 • Q2(c) CSE307 • 23/09/2013 • R036.1 MARKS 12

What are the different types of messages in a sequence diagram? Discuss them briefly with examples.

SOLUTION

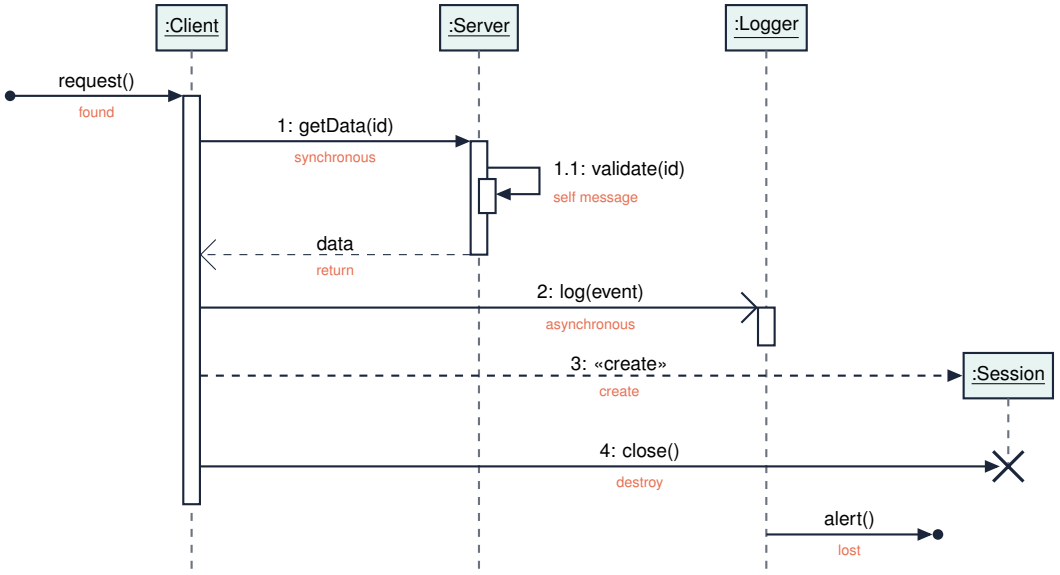
Message = communication between objects

In a sequence diagram a message is drawn as a horizontal arrow from the sender's lifeline to the receiver's lifeline, with the message name and arguments written above it. The *type of arrow* tells the type of message.

Types of messages

Type	Notation	Meaning and example
Synchronous	solid line, solid (filled) head	sender *waits* until the receiver finishes (an ordinary method call). `getData(id)`
Asynchronous	solid line, open stick head	sender continues immediately without waiting (signal, thread, event). `log(event)`
Return (reply)	dashed line, stick head	control and a result coming back to the caller of a synchronous message. `data`
Self (reflexive)	arrow leaving and re-entering the same lifeline, nested activation	an object calls one of its own operations. `validate(id)`
Create	arrow to the *head box* of a new object, labelled `new` or «create»	object created during the scenario; its box appears lower than the others. `new Session()`
Destroy (delete)	message ending at a large **X** on the lifeline	the object's life ends (in Java it simply becomes garbage). `close()`
Found	starts from a filled dot	sender unknown / outside the diagram; a sequence diagram often *starts with an unattached found message*. `request()`
Lost	ends in a filled dot	receiver unknown or message never arrives. `alert()`

All message types in one diagram



Notes

- A synchronous call always opens an **activation** on the receiver; its matching return message closes it (returns may be omitted when obvious).
- With a **self message** the nested activation is drawn slightly offset on the same lifeline.
- An object **created** during the scenario has its box placed at the height of the create message; an object **destroyed** has its lifeline end in an X.

Answer. Synchronous (filled head), asynchronous (stick head), return/reply (dashed), self/reflexive, create (to a lowered object box), destroy (X on the lifeline), found (from a dot) and lost (to a dot) messages.

2010–2011 • Q1(a) CSE307 • 19/03/2012 • R041 MARKS $24\frac{2}{3}$

Read the following Use Case Carefully.
Use Case : Check Out Books
Primary Actor: Worker
Stakeholders and Interests:
-Worker: wants fast, and easy check out of books.
-Patron: wants fast check out, and does not want to be charged for books they did not check out.
-Library: wants fast check out of books, and wants to make sure that all books that leave the library have been checked out. Wants to allocate books fairly.
-Government: wants to protect investment in books and keep costs down. Wants to promote learning and citizen happiness.
Preconditions: The Worker has been authenticated.
Success Guarantee (Post conditions): The System remembers that the Patron has checked out the books.
Main Success Scenario (or Basic Flow):

1. The Worker tells the System the identity of a patron who wishes to check out books.
2. The System confirms that the patron is allowed to check out books, and remembers the patron's

identity.

3. The Worker tells the system the identity of a book this patron is checking out.
4. The System confirms that the book can circulate, calculates the due date based on whether the patron is a faculty member or a student, and records that the patron has checked out this book, which is due on the calculated due date, and makes that information available from the library catalog.
5. The System tells the Worker the due date (which also confirms that the book has been checked out).

The Worker repeats steps 3-5 until indicates done.
Extensions (or Alternative Flows):
2a. if the patron is not allowed to check out books because they have violated some library policy (for example, if the patron has not paid their university bill or library fines):

1. The System tells the Worker that the patron is not allowed to check out books and the reason for this prohibition.
2. The use case ends.

4a. If the book that is being checked out is non-circulating;

1. The System tells the Worker why the book is non-circulating.
2. The use case continues from step 3 in the main success scenario.

Special Requirements
-There are different due dates depending on the kind of patron one is dealing with. For example, faculty can take out books for the whole academic year, whereas students can only take them out for a limited time.
-The System must respond to the Worker, at least giving some progress indication, within 3 seconds, 95% of the time.
-Workers are experts, because they use the system continuously, so the interface should have minimal interaction and should minimize the physical effort involved; for example, workers should not have to be prompted.
-Displays for the worker should be visible from one meter away.
-The system should be quiet.
Technology and Data Variations List:
3a. Barcode scanners are normally used to identify books.
3b. Books without barcodes have to be entered manually.
Frequency of Occurrence: nearly continuous.
Open Issues: How to deal with failures and recovery? How to deal with overnight check out of reserve items? How to deal with library books that are unknown to the circulation system.

Now draw a **Collaboration** and **Class diagram**, which should be relatively complete for the use case given. It should include associations also.

SOLUTION

Step 1 – From the use case to analysis objects

Following the boundary–control–entity approach:

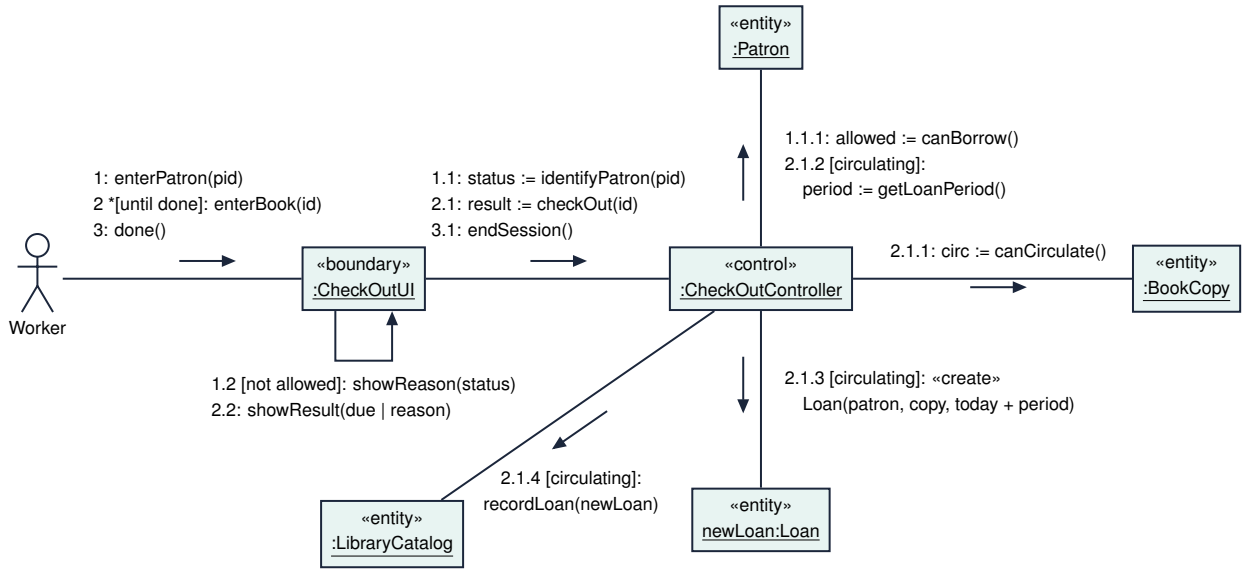
- **Actor:** Worker (authenticated librarian).

- **«boundary»** :CheckOutUI – receives the patron id and book ids (barcode or manual entry, 3a/3b) and displays results; minimal prompting (special requirement).
- **«control»** :CheckOutController – runs the use case, *remembers the patron's identity* for the session (step 2) and applies the rules.
- **«entity»** :Patron (allowed? fines, unpaid bill; kind = faculty/student), :BookCopy (circulating or not), :Loan (new: who, which copy, due date) and :LibraryCatalog (makes the loan information available).

Step 2 – Message design (mapped to the flow)

Use-case step	Messages
1–2 identify patron	1, 1.1 `identifyPatron`, 1.1.1 `canBorrow()`; 1.2 [not allowed] show reason (ext. 2a, use case ends)
3 identify book (repeated)	2 `*[until done]`, 2.1 `checkOut(id)`
4 circulate? due date? record	2.1.1 `canCirculate()` (ext. 4a if false); 2.1.2 `getLoanPeriod()` (faculty vs student); 2.1.3 «create» `Loan`; 2.1.4 `recordLoan` in catalog
5 tell due date	2.2 show result (due date, or non-circulating reason)
done	3, 3.1 `endSession()`

Step 3 – Collaboration diagram (Check Out Books)

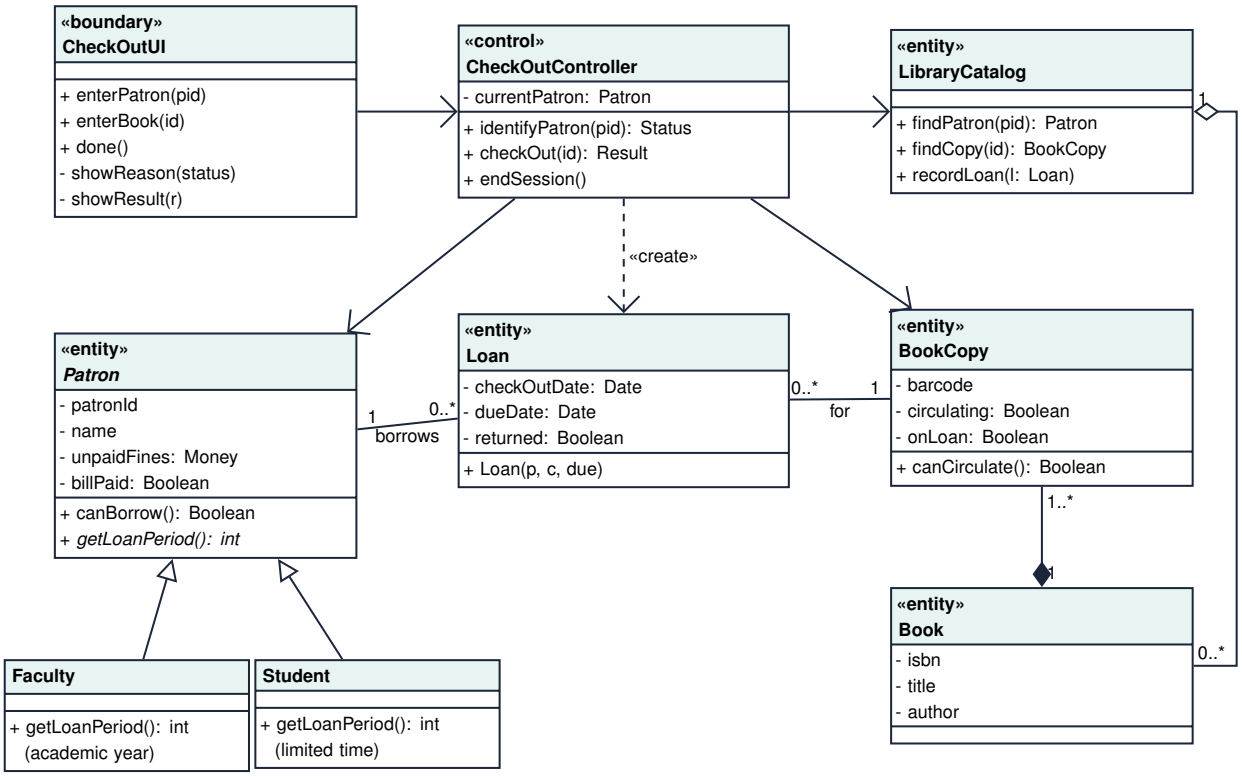


Decisions in the collaboration diagram

- **Sequence numbers** follow the use-case steps; nested numbers show which message is sent while handling which (e.g. 2.1.1–2.1.4 are all sent by the controller while processing `checkOut`).
- **Iteration** "repeat steps 3–5 until done" is the iteration marker `2 *[until done]` on the Worker → UI link.
- **Extensions** appear as guards: `1.2 [not allowed]` (2a, ends the use case) and the `[circulating]` guards on 2.1.2–2.1.4 (4a: if the copy is non-circulating only the reason is shown and the worker continues with the next book).

- **Faculty vs student due dates**: the controller asks the patron for its loan period (`getLoanPeriod()`), which is polymorphic in the class diagram.
- **Model consistency** (lecture): every message travels along a link, and every link below corresponds to an association or dependency in the class diagram.

Step 4 – Class diagram



Decisions in the class diagram

- **Patron is abstract** with subclasses Faculty and Student overriding `getLoanPeriod()` – this realises the special requirement "different due dates depending on the kind of patron" without `if`-statements (polymorphism, open for new patron kinds).
- **Loan** is a class of its own (not an attribute of the book) because it carries its own data (check-out date, due date) and history: Patron 1 – 0..* Loan 0..* – 1 BookCopy.
- **Book vs BookCopy**: the catalog lists titles (Book); a title has 1..* physical copies identified by barcode, each with its own `circulating` flag (composition).
- **LibraryCatalog** aggregates the books and offers `recordLoan()`, so the loan becomes "available from the library catalog"; it also locates patrons and copies by id.
- **Controller** keeps `currentPatron` ("remembers the patron's identity"); it has navigable associations to the entities it messages and a «create» dependency on Loan.
- Worker is the actor (authenticated beforehand), so it is not needed as a class for this use case.

Answer. Collaboration: Worker → «boundary» CheckOutUI → «control» CheckOutController → «entity» Patron (canBorrow, getLoanPeriod), BookCopy (canCirculate), new Loan, LibraryCatalog (recordLoan); numbering 1/1.1/1.1.1/1.2 for the patron, 2 `*[until done]` with 2.1.x for each book. Classes: CheckOutUI, CheckOutController, LibraryCatalog, abstract Patron {Faculty, Student}, Loan, BookCopy, Book with the associations shown.

2010–2011 • Q2(a) CSE307 • 19/03/2012 • R043.2 **MARKS 12**

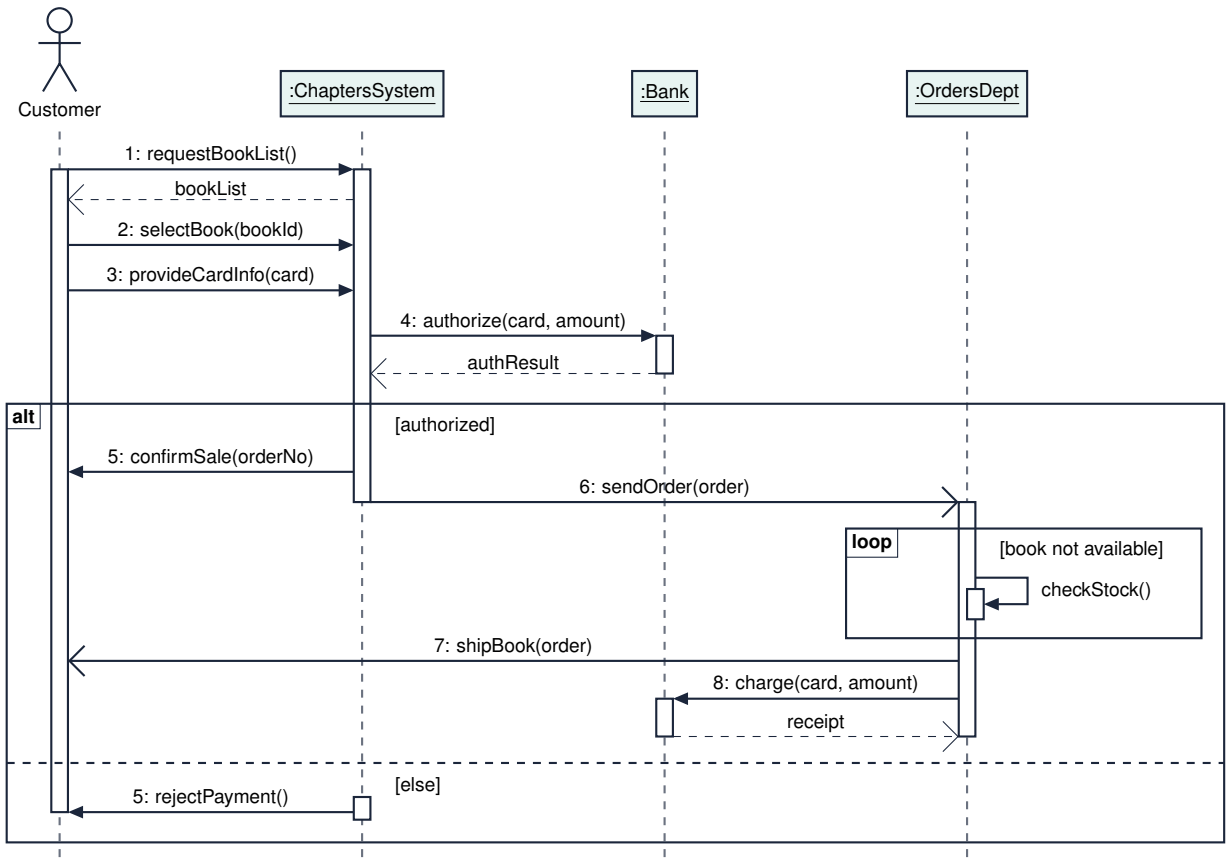
To buy a book electronically from chapters.com, a customer needs to select the book from a list provided by Chapter's eCommerce system, provide credit card information to the system, then the system gets authorization from the bank for the payment, and –if positive– confirms the sale. The order is then sent to the orders department and when the book becomes available, it is shipped to the customer. Also, the order department charges the customer's credit card by informing the bank of the amount.
Draw a **Sequence diagram** that models this process. Make sure to model all relevant actors and the interactions between them. Do show explicitly the time intervals when different actors actively participate in the process you are modeling.

SOLUTION

Participants

Customer (actor), :ChaptersSystem (the eCommerce system), :Bank and :OrdersDept. The phrase "*time intervals when different actors actively participate*" asks for **activation bars** (execution occurrences) on every lifeline.

Sequence diagram



Explanation

- **Activations** make the participation intervals explicit: the customer is involved from browsing until the book is shipped (or the payment is rejected); the system is busy from the list request until it hands the order over; the bank only while authorising and while charging; the orders department from receiving the order until the card is charged.
- **alt [authorized] / [else]** models "–if positive– confirms the sale".
- **sendOrder** and **shipBook** are *asynchronous*: the system does not wait for the book to be shipped, and shipping is a physical delivery, not a call that returns.
- The **loop [book not available]** with the self message `checkStock()` models "when the book becomes available".
- **Charging** is done by the orders department after shipping (as the narrative says), by informing the bank of the amount; the authorization earlier only reserves the amount.

Answer. Lifelines Customer, ChaptersSystem, Bank, OrdersDept with activation bars; requestBookList/selectBook/provideCardInfo → authorize at Bank → `alt`: [authorized] confirmSale, async sendOrder, `loop` until available, async shipBook, charge(card, amount) at Bank; [else] rejectPayment.

07

Process Models & Agile Development

2023–2024 • Q1(c) CSE213 • 10/05/2026 • P001 **MARKS 10**

A startup client approaches a software development team with the idea of a mobile application called "Personalized Mental Health Companion". The application is expected to track users' mental states, provide AI-based mental exercises, and include a virtual companion feature. The client has only provided high-level requirements and left detailed design and implementation decisions to the development team. To manage development, the team chooses an incremental process model. They plan to build the system in small functional parts so that they can get client feedback after each increment. Investigate the suitability of the incremental model for this scenario, with justification, and propose alternative process models if you find another more appropriate.

SOLUTION

Step 1 — Key facts of the scenario

- **Startup client**: limited budget, wants something in users' hands early, business idea may pivot.
- **Only high-level requirements**; details are left to the team ⇒ requirements are *vague and will evolve*.
- **AI-based exercises and a virtual companion**: novel, technically uncertain features (accuracy, appropriate responses); users' reaction is unknown.
- **Mental-health data**: sensitive and safety-critical (privacy, harmful advice, crisis situations) ⇒ significant risk.
- Team wants **feedback after each small functional part**.

Step 2 — Where the incremental model fits

✓ A **core product** (e.g. mood tracking/journal) can be delivered early; the startup gets a working app quickly and the initial cost is lower.

✓ Client feedback after every increment is incorporated; errors are found early; each increment is easy to test.

✓ The app is naturally divisible: tracking → exercises → AI personalisation → companion.

Step 3 — Where the incremental model falls short

× In the incremental model the *requirements are specified beforehand* and only *refined* in each increment, and "a complete planning of the project is required before committing". Here only high-level requirements exist, so that up-front plan would be guesswork.

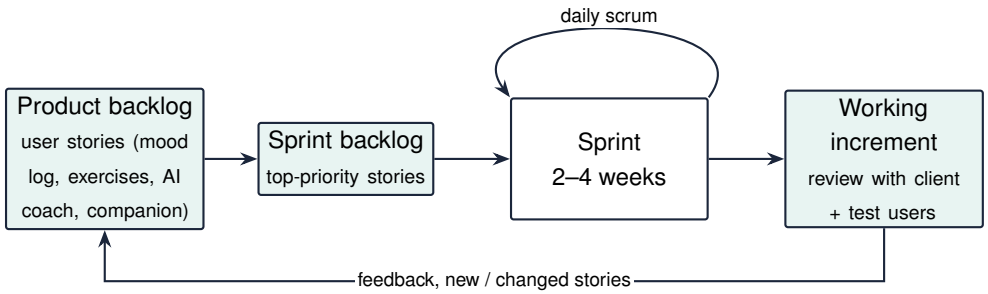
× "Refining requirements in each iteration may affect the software architecture": adding AI personalisation and a conversational companion later may demand a very different architecture (ML pipeline, cloud inference, data model), causing costly redesign.

× Incremental has no explicit mechanism for *risk analysis* — yet the AI features and sensitive data are the biggest risks.

× Increments are usually long; a startup in a fast-changing market needs shorter feedback loops and continuous client involvement.

Verdict: the incremental model is *partially suitable* (good idea of delivering in parts) but not the best choice, because the requirements are evolving, the AI features are uncertain and risky, and the architecture cannot be planned completely up front.

Step 4 — Recommended model: Agile (Scrum) with prototyping spikes



- **Handles vague, evolving requirements**: the high-level ideas become a **product backlog** of user stories that the product owner (startup) re-prioritises every sprint; "we welcome changing requirements, even late".
- **Very short feedback loops**: a working increment every 2–4 weeks, reviewed by the client and real test users — the startup can validate or pivot early.
- **Continuous client involvement** fits a client who left decisions to the team: the client steers by priorities rather than by a fixed specification.
- **Uncertain AI features** are first explored through **prototyping/spike solutions** inside sprints (e.g. a throw-away chatbot prototype) before committing the architecture.
- **Small startup team**: Scrum works best with small teams.
- **Quality and safety**: XP practices (test-first, pair programming, refactoring) and a "definition of done" including privacy/security review keep quality high.

Step 5 — Other alternatives considered

Model	Assessment for this app
Waterfall	Unsuitable: needs complete requirements up front; feedback only at the end; a startup cannot wait.
Prototyping	Very useful for the companion/AI user experience, but alone gives no schedule discipline and prototypes may be scrapped; best used *inside* the agile process.
Spiral	Strong for risk (AI safety, data privacy) and evolving requirements; acceptable alternative if the client is funded for a larger, regulated product — but costly, complicated and needs risk expertise, heavy for a startup.

Answer. Incremental is only partly suitable: it gives early, feedback-driven delivery, but it assumes requirements and a complete plan up front, has no explicit risk handling, and later AI features may break the architecture. A better choice is Agile Scrum (with prototyping spikes for the AI companion); Spiral is the alternative if risk dominates and budget allows.

2023–2024 • Q8(a) CSE213 • 09/02/2025 • P002 **MARKS 8**

Describe a scenario where the Incremental Process Model is more suitable than the Agile Model for software development. Explain the factors that make the Incremental Process Model a better choice for the project, emphasizing the key characteristics of the project and the nature of its requirements.

SOLUTION

Scenario

A government **land-records management system** for all upazila land offices is contracted to a software firm. Requirements come from existing land laws and established procedures, are documented in a tender (fixed price, fixed deadline) and change very little. The ministry wants the *basic* service live quickly, then extra services in later phases:

(i) Increment 1 (core product): digitised record entry and search of khatian/plot information.

(ii) Increment 2: mutation (ownership transfer) workflow with approvals.

(iii) Increment 3: online fee payment and certified-copy requests.

(iv) Increment 4: GIS map integration and reports.

The client officials can meet the team only at formal milestones; a 40-person team works at multiple sites; audits require complete documentation.

Factors that make Incremental the better choice

- **Nature of requirements**: well defined and stable (law-driven). The incremental model expects requirements to be *specified beforehand* and only refined; Agile's strength — coping with rapidly changing requirements — is not needed.
- **Early core functionality**: the client wants basic functionality quickly — exactly the stated use case of the incremental model (core product first). Each increment is a complete, deployable, tested release.
- **Complete up-front planning possible and required**: a fixed-price tender needs an overall plan, budget and architecture before committing; incremental supports this, whereas agile deliberately avoids detailed long-term planning.
- **Documentation and accountability**: government audits need SRS, design and test documents; agile discourages detailed documentation.
- **Limited client availability**: agile needs the client to work with developers regularly (daily/sprint-level); here feedback is possible only after each increment.
- **Large, distributed team**: Scrum usually works well only with small teams; incremental with phase plans coordinates many people better.

- **Divisible system**: functions are naturally separable into increments, avoiding the main incremental weakness.

Summary

Project characteristic	Incremental	Agile
Stable, documented requirements	fits (plan once, refine)	strength unused
Fixed contract, full plan needed	supported	weak (short-lived plans)
Client available only at milestones	fits	needs continuous involvement
Large distributed team, audits	fits (documents)	works best with small teams

Answer. For a large, contract-based government land-records system with stable law-driven requirements, mandatory documentation, limited client availability and a need for a basic service early, the incremental model (planned up front, core product first, further increments) is better than Agile, whose strengths (continuous change, small co-located team, minimal docs) do not match.

2022–2023 • Q6(a) CSE307 • 05/05/2024 • P003 **MARKS 10**

Briefly describe the core activities of a generic process model. What is the Waterfall process model? What are the advantages and disadvantages of the Waterfall process model?

SOLUTION

Core (framework) activities of a generic process model

(i) **Communication** — the project team communicates with the client to gather and clarify requirements.

(ii) **Planning** — the team schedules the development and estimates time and cost.

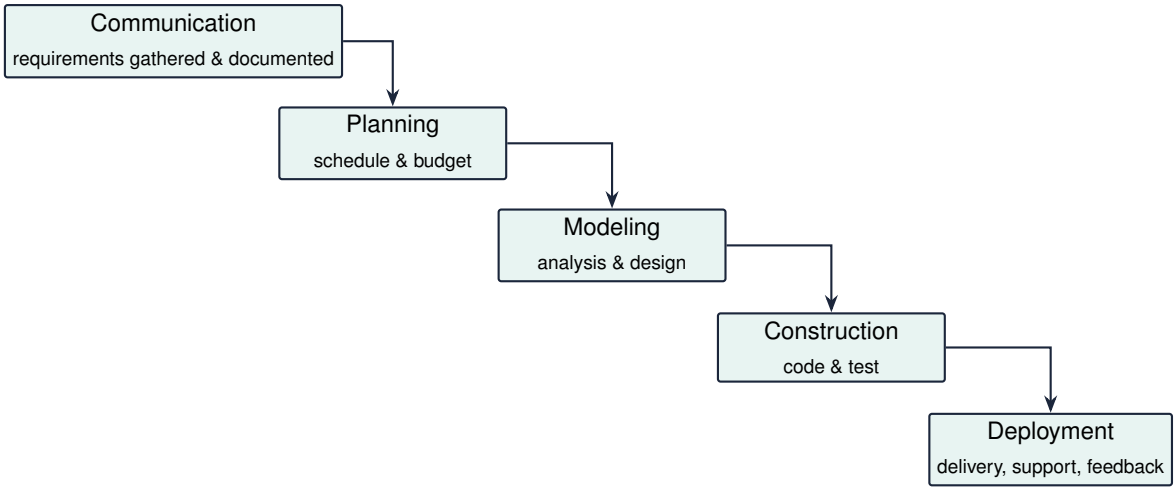
(iii) **Modeling** — the team analyses requirements and designs algorithms, flowcharts and components.

(iv) **Construction** — components are implemented and tested.

(v) **Deployment** — the product is delivered to the client, feedback is taken and the product is improved if required.

The Waterfall process model

Waterfall is a **linear (sequential) process flow**: each framework activity is completed and signed off before the next one starts, and the output of one phase "falls" into the next. It is normally used for (small) projects whose requirements are *well defined and known before the project starts*.



- **Communication**: requirements are collected from the client and documented (SRS).
- **Planning**: time and financial constraints are estimated → a schedule and a budget.
- **Modeling**: the design is crafted from the requirements, keeping constraints in mind.
- **Construction**: code is generated and tested to build the final product.
- **Deployment**: the product is delivered; maintenance is provided based on client feedback.

Advantages and disadvantages

Advantages	Disadvantages
Simple model to use and implement; phases are clearly separated.	It may be hard for the client to state *all* requirements at the beginning.
Easily understandable workflow; clear milestones and documents per phase.	Testing and client evaluation happen only in the last phases ⇒ errors are found late ⇒ high risk.
Easily manageable (planning, tracking, budgeting) because requirements are known up front.	Iteration is not promoted; going back to an earlier phase is costly, so changing requirements are handled badly.
	No working software until late; team members may wait for each other ("blocking states").

Answer. Generic process = Communication, Planning, Modeling, Construction, Deployment. Waterfall executes them once, strictly in sequence; it is simple and easy to manage when requirements are stable, but it needs all requirements early, tests late (high risk) and resists change.

2022–2023 • Q6(b) CSE307 • 05/05/2024 • P003 **MARKS 10**

"Developers can adjust workload per kanban card by analyzing the kanban board to maintain a decent lead time in workflow." Elaborate on this statement.

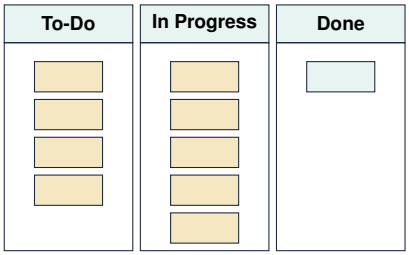
SOLUTION

Terms

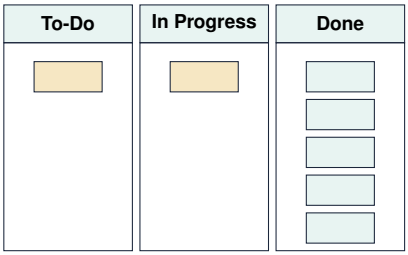
- **Kanban card**: one modular subtask (with its user story) written on a card.
- **Kanban board**: columns To-Do / In Progress / Done (possibly more) showing the state of every card.
- **Lead time**: the time a card takes to move through the workflow, from entering To-Do to reaching Done.
- **Workload per card**: how much work is packed into one card (its size/granularity).

Reading the board

Because the board is visible to everyone, developers can analyse the *flow* of cards:



cards stuck in progress ⇒ long lead time
reduce workload per card



cards flow quickly to Done
workload per card may be increased

Adjusting the workload

- **Cards pile up in "In Progress" and Done grows slowly** ⇒ lead time is too long; work is too big or too much is started at once. Developers *reduce the workload per card* — split large cards into smaller subtasks, limit how many cards each person holds, finish before pulling new ones — to speed up the flow.
- **Cards move to Done very quickly and people are often idle** ⇒ cards are too fine-grained (overhead of many tiny cards). Developers can *increase the workload per card* (merge related subtasks) to finish the process efficiently.
- **One column becomes a bottleneck** (e.g. Testing) ⇒ the team rebalances people or card size at that stage.

Because Kanban is a continuous pull system with no sprint deadline, this self-regulation is the main way the team keeps work flowing at a predictable pace; each developer can make the decision, which reflects Kanban's "leadership from every participant" and adaptability to different team natures.

Example

A card "Implement complete payment module" stays in In Progress for 3 weeks. The team splits it into "card form UI", "gateway call", "receipt e-mail", "refund"; each now finishes in 2–3 days, Done fills steadily and the average lead time falls — problems also surface earlier.

Answer. By watching how cards move across the visible board, developers sense the lead time: if cards stagnate, they reduce the work per card (split tasks, limit WIP); if cards flow very fast, they can increase it. This self-adjustment keeps a steady, decent lead time in Kanban's continuous flow.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q5(b) CSE307 • 07/05/2023 • P004 MARKS 20

Give an overview of the different steps involved in a *sprint* in *scrum* method of software development.

2022–2023 • Q6(c) CSE307 • 05/05/2024 • P003 MARKS 15

What are the activities performed in a sprint of the Scrum method? What are the two types of backlogs of the Scrum method?

SOLUTION

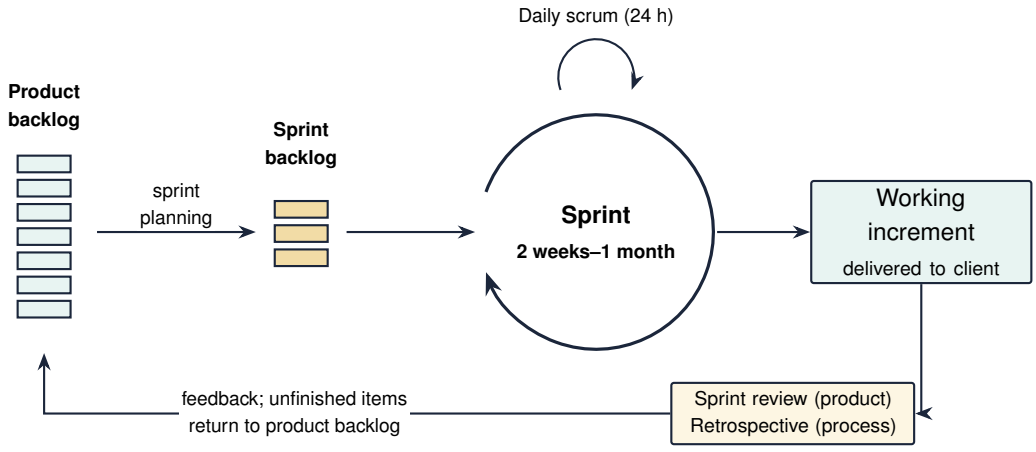
Scrum and the sprint

Scrum is a lightweight, iterative and incremental agile process model. Development is organised into **sprints**: time-boxed iterations, usually **2 weeks to 1 month**, during which the team turns a selected set of requirements into a working, potentially shippable product increment. Roles: **product owner** (client; owns the backlog), **scrum master** (usually the project manager; facilitates and removes impediments) and the self-organising **development team**.

The two types of backlog

- **Product backlog** — the overall, prioritised list of client requirements (features, user stories, fixes) for the whole product. Owned by the product owner; continuously refined and re-prioritised.
- **Sprint backlog** — the subset of product-backlog items selected for one particular sprint, broken down into tasks (tickets) by the team. It is frozen during the sprint.

Sprint cycle



Steps (activities) of a sprint

(i) Backlog grooming / selection. The sprint begins by analysing the *product backlog*; the product owner explains the top-priority items and the team decides which requirements can be implemented in this sprint. The selected items are moved into the *sprint backlog*.

(ii) Sprint planning. The team agrees the sprint goal, breaks each selected item into tasks (tickets), estimates them and puts them on the *scrum board* (To-do / In progress / Done). Team members volunteer for tasks (self-organisation).

(iii) Implementation (development work). The team designs, codes, tests and integrates the sprint-backlog items. The sprint backlog is not changed by outsiders during the sprint, so the team can focus; progress is tracked on the board and a burndown chart.

(iv) Daily scrum. Every day a short (about 15-minute) stand-up meeting is held for synchronisation. Each member answers: *What did I do since the last meeting? What will I do next? What obstacles am I facing?* The scrum master removes impediments.

(v) Product review (sprint review/demo). At the end of the sprint the completed increment is demonstrated to the product owner and stakeholders, who give feedback; the product backlog is updated accordingly.

(vi) Process review (retrospective). The team reflects on *how* it worked — what went well, what did not, what to improve — so fewer mistakes are made in the next sprint.

(vii) Delivery and carry-over. The completed portion of the product is delivered to the client for feedback; unfinished sprint-backlog items go back to the product backlog for a later sprint. The next sprint starts immediately.

In the slides' terms, the work on the sprint backlog has **four phases: planning, implementation, product review and process review (retrospective)**, preceded by backlog selection and followed by delivery.

Answer. Sprint steps: select items from the product backlog → sprint planning (sprint backlog, tasks, board) → implementation with daily scrums → product review (demo, feedback) → process review (retrospective) → deliver increment, unfinished items return to product backlog. Two backlogs: product backlog (all requirements) and sprint backlog (requirements of one sprint).

2022–2023 • Q5(a) CSE325 • 30/01/2025 • C325-2223-5a **MARKS 8**

A hospital is developing a critical patient monitoring system that must continuously track patient vital signs (e.g., heart rate, blood pressure) and alert staff in case of emergencies. The system needs to ensure zero downtime and high reliability. It needs to be extensively tested and frequently updated based on feedback from doctors and nurses. The hospital's development team is small but highly skilled; they work closely with medical staff to understand and refine requirements. Given the requirements, propose an agile process model that is the most suitable for this project and justify you answer.

SOLUTION

Proposed model: Extreme Programming (XP)

Extreme Programming (Kent Beck) takes good engineering practices "to extreme levels, from day one": testing from day one, pair programming, continuous integration, small frequent releases, simple design and constant communication with the client. It values communication, simplicity, feedback and respect and prioritises customer satisfaction.

Mapping the project's forces to XP practices

Project characteristic	XP practice that addresses it
Must be *extensively tested*; zero downtime, high reliability	**Test-first / test-driven development** (unit tests written before code), automated acceptance tests, **continuous integration** (every change is built and the full suite re-run several times a day) – defects in alarm logic are caught immediately
Frequently updated based on feedback from doctors and nurses	**Small releases** and short iterations (1–2 weeks); requirements written as *user stories* that the customer can re-prioritise every iteration
Team works *closely with medical staff*	**On-site customer**: a doctor/nurse representative sits with the team, writes stories and acceptance criteria, and answers questions immediately
Small but highly skilled team	**Pair programming** and collective code ownership suit a small expert team; two people review every line of safety-critical code as it is written
Code must stay changeable without breaking	**Simple design** and continuous **refactoring**, protected by the test suite

Why not the alternatives?

- **Scrum** gives the iteration/backlog framework (sprints, product owner, daily scrum) but prescribes *no engineering practices*; the critical needs here – exhaustive testing and reliability – are exactly what XP adds (TDD, CI, pair programming). (XP practices can be run inside Scrum sprints.)
- **Kanban** suits a continuous flow of independent tickets (e.g. support), not planned feature increments with strong test discipline.
- Plan-driven models (waterfall, V-model) give rigorous testing but cannot absorb the *frequent* feedback changes from clinicians.

Caution

XP's known weaknesses – light documentation and possible neglect of up-front design – matter in a medical system. They are mitigated by keeping the automated tests as living documentation, adding a short architecture spike for the high-availability design (redundant servers, failover) and keeping traceability from user stories to tests for audits.

Answer. Extreme Programming (XP): test-first development, continuous integration and pair programming give the extensive testing and reliability required; small releases and short iterations absorb frequent clinician feedback; an on-site customer (doctor/nurse) matches the close collaboration; and XP works best with a small, highly skilled team. Scrum alone lacks these engineering practices.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q6(a) CSE307 • 07/05/2023 • P005 MARKS 10

Point out how *incremental* and *evolutionary* process models address the shortcomings of *waterfall* process model in a software project.

2017–2018 • Q2(a) CSE307 • 29/09/2018 • P011 **MARKS 7**

What are the problems of waterfall model? Describe how other models solve the problem.

SOLUTION

Step 1 — Shortcomings (problems) of the Waterfall model

(i) All requirements up front. The client must state everything at the beginning, but clients rarely know all requirements before seeing software.

(ii) Late testing and evaluation ⇒ high risk. Errors in requirements or design are found only at the end, when they are most expensive to fix.

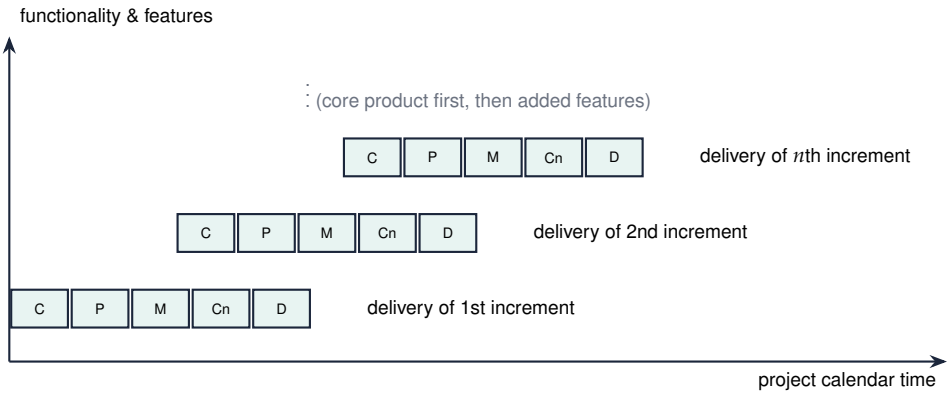
(iii) No iteration. Going back to an earlier phase is not promoted, so changing requirements are handled badly.

(iv) No working product until the end. The client waits a long time and cannot give feedback; the business may change meanwhile.

(v) Blocking states. Team members wait for preceding phases to finish.

(vi) Uncertainty/risk is not analysed explicitly.

Step 2 — How the Incremental model addresses them



C = communication, P = planning, M = modeling, Cn = construction, D = deployment.

- A **core product** with the basic requirements is delivered early → the client has *working software early* (fixes problem 4).
- Client feedback after each increment is incorporated into the next → *requirements can be refined and modified* during the process (problems 1, 3).
- Each increment is tested and evaluated by the client → *errors are found early*, risk falls (problem 2).
- While one increment is deployed, the next can be designed → less waiting (problem 5).
- Lower initial cost: only part of the system is built at first.

Step 3 — How Evolutionary models (Prototyping, Spiral) address them

- **Prototyping**: when requirements are vague, a quick prototype is built from currently known requirements, the client evaluates it and adds requirements; this repeats until the prototype evolves into an acceptable product. It removes the need for complete up-front requirements (problem 1), involves the client continuously and detects errors early (problems 2, 4).
- **Spiral**: development proceeds through loops, each producing a more mature version, and **risk analysis** is performed in every loop. It tackles uncertainty explicitly (problem 6), and changes can be incorporated even at a later stage (problem 3).
- In general, evolutionary models let *requirements themselves evolve*, which Waterfall forbids.

Step 4 — Summary table

Waterfall problem	Incremental remedy	Evolutionary remedy
All requirements needed up front	only core requirements needed first; others refined later	prototype clarifies vague requirements; requirements evolve
Late testing, high risk	every increment tested and evaluated	each prototype/loop evaluated; spiral performs explicit risk analysis
No iteration, change is costly	changes go into next increment	each cycle/loop welcomes changes, even late (spiral)
No product until the end	core product delivered early	early prototype / basic version in first loop

(Agile models push the same ideas further, with very short iterations and continuous client involvement.)

Answer. Waterfall's problems are up-front requirements, late testing/high risk, no iteration and late delivery. Incremental delivers a core product early and adds tested increments with client feedback; evolutionary models (prototyping, spiral) let requirements evolve through repeated evaluated versions, and spiral adds explicit risk analysis in every loop.

2021–2022 • Q7(a) CSE307 • 07/05/2023 • P006 MARKS 10

State the reasons behind recent shift from adopting *traditional* process models to adopting *agile* process models in software projects.

SOLUTION

Background

Traditional (prescriptive) models — Waterfall, Incremental, Prototyping, Spiral — were introduced to bring *order and structure*. But, as the slides put it, they are "slow, extremely slow", while the modern business environment is *fast-paced and ever-changing*. The shift to agile is a response to this mismatch.

Reasons for the shift

(i) Rapidly changing requirements. Requirements change frequently; plans are short-lived (features persist, change or are dropped). Traditional models freeze requirements early; agile welcomes change even late.

(ii) Time-to-market pressure. Businesses need software in weeks, not years. Agile delivers working increments every few weeks.

(iii) Early and continuous value. "Our highest priority is to satisfy the customer through early and continuous delivery of valuable software." Clients get usable functionality (and return on investment) early.

(iv) Lower risk through early feedback. Traditional models test and evaluate late; agile tests from day one and gets client feedback every iteration, so mistakes are cheap to fix.

(v) Lower cost of change. With incremental delivery, continuous testing and refactoring, the cost of making a change stays low throughout the schedule (flattened cost-of-change curve).

(vi) Customer collaboration. Clients want to be involved and to steer the product; agile puts client and developers together regularly.

(vii) Less wasted effort. Heavy documentation and detailed long-term plans often become obsolete; agile values working software over comprehensive documentation and "maximising the work not done".

(viii) People-centric, motivated teams. Self-organising teams with less hierarchy are more creative and motivated; the process adapts to the people rather than vice versa.

(ix) Better tools and practices. Continuous integration, automated testing, cloud deployment and tools such as Jira make frequent releases practical.

(x) Industry success. Scrum and Kanban have become widely used and proven in industry, encouraging further adoption.

Traditional vs agile at a glance

	Traditional	Agile
Philosophy	understand → plan → design → build → deliver	build something useful → deliver → get feedback → adapt → repeat
Change	resisted, costly	welcomed
Delivery	mostly at the end	frequent increments
Client	at start and end	throughout
Best when	future is predictable	change is expected

Answer. Organisations moved to agile because modern markets change quickly: agile copes with volatile requirements, gives faster and continuous delivery, reduces risk and cost of change through early feedback and testing, involves the customer throughout, avoids wasteful documentation, and empowers self-organising teams.

2021–2022 • Q8(b) CSE307 • 07/05/2023 • P007 **MARKS 15**

"*Kanban method focuses on the visualization aspect of a software project with kanban cards and kanban board.*" – elaborate on this statement.

SOLUTION

Kanban: the idea

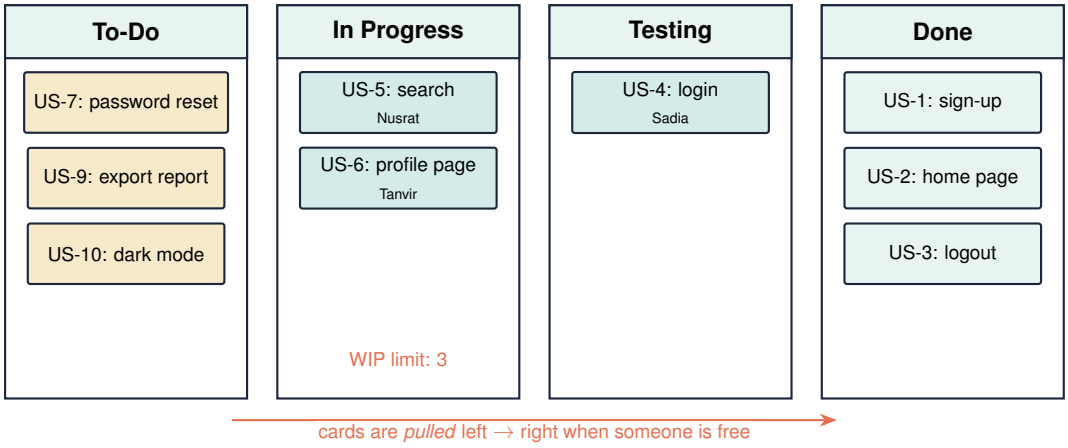
Kanban is a Japanese word meaning *signboard* — something visible to everyone around it. In project management **Kanban** is an agile process model that puts heavy focus on the **visualisation** of the project, so that everyone involved is kept up to date all the time. The two visual instruments are the **kanban card** and the **kanban board**.

Kanban cards

Tasks in the project are divided into modular *subtasks*; each subtask is written on a kanban card containing the details of that subtask and the associated *user story* (client requirement) — typically also the assignee, priority, estimate and dates. One card = one visible unit of work.

Kanban board

The board has a column for each workflow stage; cards move from left to right as work progresses.



How visualisation drives the project

(i) **Transparency**: anyone (developers, agile coach, client) sees at a glance what is not started, in progress and done — no status reports needed.

(ii) **Pull system**: whenever a participant is free, he/she pulls the next card from the left; work is not pushed onto overloaded people.

(iii) **Bottleneck detection**: cards piling up in one column (e.g. Testing) reveal a bottleneck immediately; the team swarms to clear it. Work-in-progress limits per column make this explicit.

(iv) **Workload and lead-time control**: by watching how fast cards flow, developers adjust workload per card to keep a decent lead time.

(v) **Continuous flow**: unlike Scrum there are no sprints; cards flow continuously and new requirements simply become new cards in To-Do, ordered by priority.

(vi) **Collaboration and shared leadership**: the shared board supports daily stand-ups and retrospectives (with an agile coach) and lets everyone take decisions rapidly.

(vii) **Tool support**: physical boards or digital ones such as Jira.

Limitation

Visualisation does not impose a timeframe; without sprints, the project can drift and be delayed.

Answer. Kanban makes all work visible: each subtask/user story is a kanban card, and the kanban board's columns (To-Do, In Progress, Done, . . .) show each card's state. Moving cards by a pull system lets everyone see progress, spot bottlenecks, balance workload and keep lead time low in a continuous flow.

2021–2022 • Q8(c) CSE307 • 07/05/2023 • P007 MARKS 10

Explain, in your own words, the *people* and the *process* components of a typical software project.

SOLUTION

Context: the components of a software project

A software project is managed along four dimensions — **People**, **Product**, **Process** and **Project**. The question asks about the two that most strongly decide success: the *people* who do the work and the *process* they follow.

1. The people component

Software is built by people, and software development is a *social learning process*: scattered, incomplete knowledge has to be gathered from many people and turned into code. The people component covers:

- **Stakeholders** — senior managers (define business goals), project managers (plan, motivate, organise), practitioners (engineers who build it), customers (specify requirements) and end users (use the product).
- **Team leaders** — must motivate, organise ideas and encourage innovation; they solve problems and keep the team focused.
- **Team organisation** — e.g. a hierarchical (closed) team led by a manager versus a democratic, *self-organising* agile team.
- **Human factors** — in agile process the slides list traits people must grow: competent, skilful, knowledgeable, committed, collaborative, decisive, faithful, respectful and self-organised.
- **Coordination and communication** — formal (documents, reviews, meetings) and informal (face-to-face talk, daily stand-ups).

Without the right people, well coordinated, no process can rescue a project.

2. The process component

The process is the *framework* within which the people work — the road map that divides the work into phases:

- **Framework activities**: communication, planning, modeling, construction, deployment.
- **Process flow / process model**: linear (Waterfall), incremental, evolutionary (Prototyping, Spiral) or agile (XP, Scrum, Kanban), chosen to match requirement stability, risk and deadline.
- **Task sets, milestones and work products** for each activity (e.g. SRS after communication, design document after modeling, tested increment after construction).
- **Umbrella activities**: tracking and control, risk management, quality assurance, reviews, configuration management, measurement.

The process gives order and structure; the agile view adds that the process should *reshape according to the people* rather than people being forced to fit the process.

How the two interact

	People	Process
Answers	*Who* does the work and how they cooperate	*How* the work is organised into phases
Key elements	stakeholders, leaders, team structure, skills	framework + umbrella activities, chosen model
Failure symptom	poor communication, low motivation, wrong skills	chaos, missed steps, late testing, uncontrolled change

Answer. People = the stakeholders and the team (their roles, skills, organisation and communication); Process = the framework of activities, the chosen process model and the umbrella activities that structure their work. A project succeeds when a capable, well-coordinated team follows a process suited to the project.

2020–2021 • Q7(a) CSE325 • 12/04/2023 • C325-2021-7a MARKS 15

Assume you are a scrum master and you are in the beginning of a sprint after which you need to deliver an additional database search feature along with UI. Plan a sprint which will last for 4 weeks. Include all the scrum events.

SOLUTION

Scrum recap

A **sprint** is the time-boxed iteration (here 4 weeks = 20 working days) that produces a potentially releasable increment. It starts by selecting items from the *product backlog* into the *sprint backlog* and proceeds through planning, implementation, product review and process review (retrospective); unfinished items go back to the product backlog. The scrum events are: the **Sprint** itself, **Sprint Planning**, **Daily Scrum**, **Sprint Review** and **Sprint Retrospective** (plus ongoing backlog refinement). Roles: Product Owner (PO), Scrum Master (SM, me), Development Team (say 5 developers incl. a tester and a UI developer).

Sprint goal

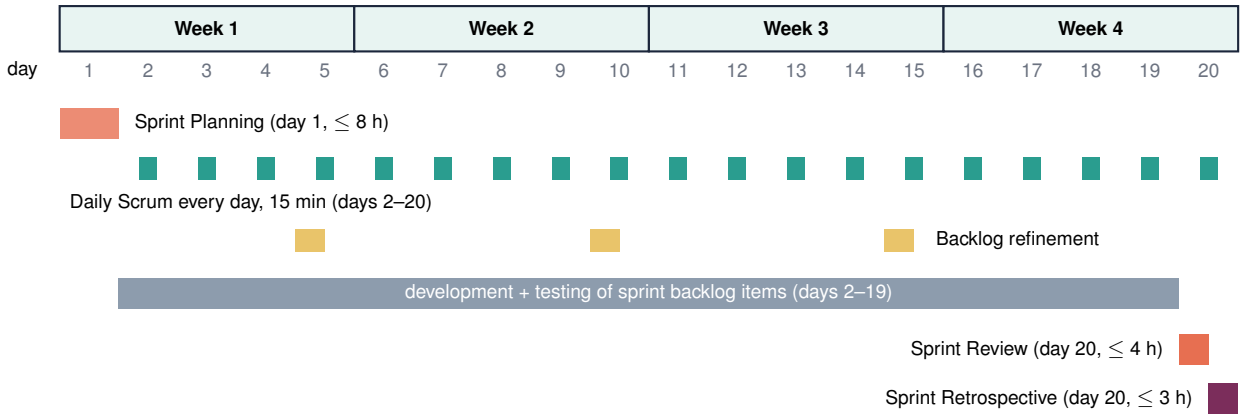
"Users can search records by keyword and filters (date range, category) from a new search page and get paged, sorted results within 2 seconds."

Sprint backlog (selected in Sprint Planning)

#	Item (user story / task)	Tasks	Points
1	Search by keyword	DB full-text index, search query/repository, REST endpoint `GET /search?q=`	8
2	Filter & sort results	date-range/category filters, sort options, input validation	5
3	Paging	limit/offset in query, page metadata in API	3
4	Search UI	search box, filter panel, results table, paging controls, empty/error states	8
5	Performance	query plan review, indexes, response time ≤ 2 s for 1M rows	5
6	Tests & docs	unit + integration tests, UI tests, API docs, user help	5

Definition of Done: code reviewed, unit/integration tests passing in CI, acceptance criteria met, deployed to staging, documentation updated.

Sprint calendar with all events



Event	When / time-box	What happens in this sprint
Sprint Planning	Day 1, max 8 h for a one-month sprint	PO presents the search feature; team agrees the sprint goal, selects items 1–6 by capacity (velocity $\approx$ 34 points), splits them into tasks and estimates hours; sprint backlog and task board created.
Daily Scrum	Every day, 15 min, same time and place	Each member: what I did yesterday, what I will do today, impediments (e.g. index rebuild slow on staging DB). SM removes impediments.
Backlog refinement	About 1 h at the end of weeks 1–3	PO and team clarify upcoming stories (e.g. next sprint's saved searches); not more than 10% of capacity.
Development work	Days 2–19	Week 1: DB index + search query + API skeleton; week 2: filters, sorting, paging + UI layout; week 3: UI integration, performance tuning; week 4: testing, bug fixing, documentation, staging deployment.
Sprint Review	Day 20 morning, max 4 h	Team demonstrates the working search page to PO and stakeholders; feedback is added to the product backlog; PO accepts the items meeting the Definition of Done.
Sprint Retrospective	Day 20 afternoon, max 3 h	Team inspects *how* they worked (e.g. late UI/API integration) and agrees on 1–2 improvements for the next sprint.

Answer. 4-week sprint with goal "keyword + filtered search with UI". Day 1 Sprint Planning ($\leq$ 8 h) selects the search stories into the sprint backlog; Daily Scrums (15 min) every day; weekly backlog refinement; development weeks 1–4 (DB/API $\rightarrow$ filters/paging/UI $\rightarrow$ integration/performance $\rightarrow$ testing/docs); day 20 Sprint Review ($\leq$ 4 h, demo to PO/stakeholders) and Sprint Retrospective ($\leq$ 3 h); unfinished items return to the product backlog.

NOTE The time-boxes are those of the Scrum Guide for a one-month sprint; team size, story points and the weekly breakdown are illustrative assumptions.

2019–2020 • Q2(c) CSE325 • 02/04/2022 • C325-1920-2c **MARKS 8**

What do you mean by Software Process and Software Process Model? How do you describe a process? Why is Software Process so important for software development?

SOLUTION

Software process

A **software process** is the structured set of activities, actions and tasks that are performed to produce a software product. Software is "embodied knowledge that is initially scattered and incomplete" and development is "an iterative social learning process" (lecture), so we need a step-by-step process to bring that knowledge into the software. Every process contains the five **framework (core) activities**: *communication*, *planning*, *modeling*, *construction*, *deployment*, plus umbrella activities (project tracking, risk management, quality assurance, configuration management, reviews).

Software process model

A **process model** is a simplified, abstract representation of a process: "a technique to divide software development work into distinct phases" that "provides a visual road map to organize and improve both software project and software product management" (lecture). It fixes the *process flow* – linear, iterative, evolutionary or parallel – in which the framework activities are executed. Examples: Waterfall (linear), Incremental, Prototyping and Spiral (evolutionary), and agile models such as XP and Scrum.

How a process is described

A process description states, for every activity:

- **Activities/tasks** and their **order** (the process flow), e.g. *write test cases* after *design*.
- **Work products (artefacts)**: inputs and outputs of each activity – SRS, design model, code, test report.
- **Roles**: who is responsible (analyst, architect, programmer, tester, client).
- **Pre- and post-conditions**: what must be true before an activity starts and after it ends (e.g. "requirements signed off by the client" before design starts; "all unit tests pass" after construction).
- **Milestones and quality checkpoints** (reviews, sign-offs) and the tools/methods used.

It can be written as text, a table, or a diagram (activity diagram or BPMN of the process itself).

Why the software process is important

- **Predictability**: a defined process makes schedule and cost estimation possible and lets managers track progress against milestones.

- **Quality**: reviews, testing and validation are built into the process instead of being left to chance, so defects are found early when they are cheap to fix.
- **Coordination and communication**: everybody knows who does what and when, and which artefact to hand over.
- **Repeatability and improvement**: success can be repeated in the next project, and a documented process can be measured and improved.
- **Managing change and risk**: the chosen model (e.g. incremental, spiral, agile) decides how changing requirements and risks are handled.

Answer. Process = set of activities (communication, planning, modeling, construction, deployment) producing software; process model = abstract road map fixing how these activities flow (Waterfall, Incremental, Spiral, XP . . .). A process is described by its activities and order, products, roles, pre/post-conditions and milestones. It is important for predictability, quality, coordination, repeatability/improvement and handling change.

2019–2020 • Q3(a), (b), (d) CSE325 • 02/04/2022 • C325-1920-3abd **MARKS 6 + 7 + 15**

(a) Define Waterfall Model for Software Engineering Process. Explain the problems of Waterfall Model with necessary examples. (6)

(b) Compare the concept of incremental delivery in the process model with the Bohem's spiral model. (7)

(d) Prescribe appropriate process model for the following cases with proper reasoning: (15)

(i) Development of an POS (Point of Sale) ERP for computer accessories shops of Bangladesh.

(ii) A war Simulator for Bangladesh Army.

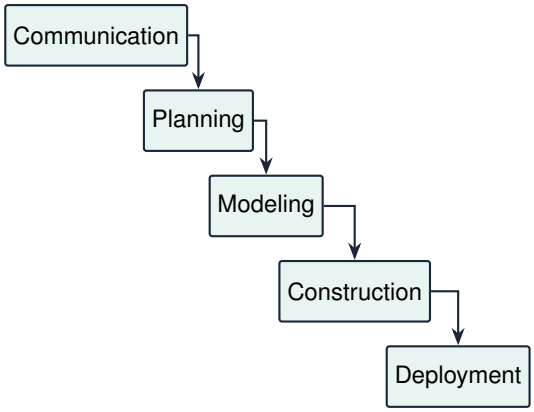
(iii) Development of Insulin Injection System for Intensive Care Unit as an outsourcing project of an US based biomedical engineering company.

[(c) "Rational Unified Process can be configured to Waterfall Model" – RUP is outside the syllabus (as in the rest of this book) and is not solved here.]

SOLUTION

(a) Waterfall model and its problems

The **waterfall model** is the classic *linear (sequential)* prescriptive model: the five framework activities are executed one after another – *communication* (requirements collected and documented), *planning* (schedule, budget), *modeling* (analysis and design), *construction* (code and test), *deployment* (delivery, feedback, maintenance) – and each phase starts only when the previous one is complete and signed off. It suits small projects whose requirements are well defined and known before the project starts.



Problems (with examples).

(i) **All requirements must be known up front** – "it may become tough for the client to provide all the requirements beforehand". *Example:* a startup's food-delivery app – the owners only discover which features users want after launch.

(ii) **Late testing and late client evaluation ⇒ high risk** – the client sees working software only at deployment. *Example:* a university result system whose grading rules were misunderstood is discovered to be wrong only during final acceptance; re-work then touches design and code.

(iii) **No iteration** – going back to an earlier phase is expensive, so changes are resisted. *Example:* a new VAT rule announced during construction of a billing system forces re-opening the signed-off requirements and design.

(iv) **Blocking states / long time to first delivery** – teams wait for the previous phase; nothing usable exists for months.

(b) Incremental delivery vs. Boehm's spiral model

Incremental delivery: the product is delivered as a series of *increments*; a *core product* with the basic requirements comes first, client feedback after each delivery feeds the next increment, until the final version is released. **Spiral model** (Boehm): the product also evolves through a series of versions, but each loop of the spiral is driven by *risk*: determine objectives → identify and resolve risks (prototypes, simulations) → develop and test → plan the next loop; the first loops may produce only a prototype.

Aspect	Incremental delivery	Spiral model
Driving force	delivering useful functionality early	reducing risk / uncertainty
What each cycle yields	an *operational* increment used by the client	early loops: specification, prototype; later loops: more mature versions
Requirements	mostly specified up front, refined per increment	may evolve; resolved through risk analysis
Risk analysis	implicit, not a formal activity	explicit in every loop (core of the model)
Planning	complete plan needed before committing	planning repeated at the end of every loop
Typical use	business systems whose core is clear (e.g. inventory, POS)	large, high-risk, novel systems (defence, new technology)
Cost	lower initial cost, total cost may grow	expensive; needs risk-assessment expertise

Both are *iterative* and both deliver the product through versions; the spiral can be seen as a risk-driven generalisation in which an increment is produced only when its risks are acceptable.

(d) Prescribing process models

(i) POS ERP for computer-accessories shops – Incremental model (or agile/Scrum).

- The domain (sales, purchase, inventory, billing, VAT, accounts) is well understood, so the core requirements can be specified early.
- Shops need a usable core quickly (sales + billing + stock) and can give feedback; later increments add purchase, supplier management, reports, accounting, multi-branch.
- Many small customers with varying needs ⇒ frequent releases and feedback; low initial cost for the vendor. Agile (Scrum sprints) fits equally well if the client is available for regular reviews.

(ii) War simulator for the Bangladesh Army – Spiral model.

- Large, complex, novel system with high technical risk (physics, real-time 3D, AI of opposing forces, hardware integration) and requirements that the users discover gradually.
- Confidential, safety-of-training critical; each loop can build a prototype (e.g. terrain + one weapon system), evaluate it with officers and resolve the biggest risk before the next loop.
- Long budgeted government project ⇒ the cost of formal risk analysis is affordable.

(iii) Insulin injection system for an ICU, outsourced by a US biomedical company – Waterfall / V-model (plan-driven).

- Safety-critical medical device software: a wrong dose can kill. Requirements are fixed by the client and by regulation (FDA / IEC 62304), and must be precisely specified and traced to tests.
- Outsourcing contract with a foreign client ⇒ requirements and deliverables are frozen in the contract; heavy documentation, formal reviews and sign-offs at every phase are mandatory.
- A plan-driven sequential model with verification at each stage (the V-model variant of waterfall) provides that rigour; agile's minimal documentation would not satisfy certification.

Answer. (a) Waterfall = sequential communication → planning → modeling → construction → deployment; problems: all requirements up front, late testing/evaluation (high risk), no iteration, late delivery. (b) Incremental: early operational increments driven by functionality; Spiral: versions driven by explicit risk analysis in every loop. (d) (i) POS ERP → Incremental/agile; (ii) war simulator → Spiral; (iii) insulin injection system → Waterfall/V-model (plan-driven, safety-critical, contract-based).

2019–2020 • Q4(a) CSE325 • 02/04/2022 • C325-1920-4a **MARKS 12**

Write short notes on the following in the context of Extreme Programming: Pair Programming, Testing, Change Adoption.

SOLUTION

Context

Extreme Programming (XP) (Kent Beck) takes beneficial engineering practices "to extreme levels, from day one". It values communication, simplicity, feedback, courage and respect, and "prioritizes customer satisfaction over everything else".

Pair programming

- Two developers work together at *one computer* on the same user story: the *driver* types the code, the *navigator* reviews each line, thinks about design, tests and edge cases; they swap roles frequently and pairs rotate across the team.
- **Benefits**: continuous code review (defects caught as they are typed), better design decisions, shared knowledge of the whole code base (*collective ownership*, no single expert who becomes a bottleneck), faster onboarding of new members.
- **Cost**: two people on one task – justified by fewer defects and less re-work; needs compatible partners.

Testing

- **Test-first development (TDD)**: before writing code for a story, the pair writes automated unit tests (e.g. JUnit); the code is written to make them pass, then refactored.
- **Automated, continuous**: all unit tests must pass before code is integrated; integration happens several times a day, and the whole suite is re-run each time (regression safety net).
- **Customer acceptance tests**: the customer specifies acceptance tests for every user story; a story is "done" only when its acceptance tests pass.
- Effect: "testing codes from day one" gives immediate feedback, makes refactoring safe and keeps the system always releasable.

Change adoption (embracing change)

- XP assumes requirements *will* change and welcomes it (Agile principle: "welcome changing requirements, even late in development").
- **Mechanisms**: short releases and iterations (1–3 weeks) re-planned with the on-site customer (*planning game*: customer reprioritises user stories every iteration); *simple design* – build only what is needed now; continuous *refactoring* keeps the code easy to change; the automated test suite guarantees that a change has not broken anything; collective ownership lets anyone change any code.
- Result: the cost of a late change stays low, unlike plan-driven models where it grows steeply with time.

Answer. Pair programming: two developers, one computer, driver/navigator, continuous review and shared ownership. Testing: test-first automated unit tests, continuous integration with regression runs, customer acceptance tests per story. Change adoption: short iterations re-planned with the on-site customer, simple design, refactoring and the test safety net keep change cheap.

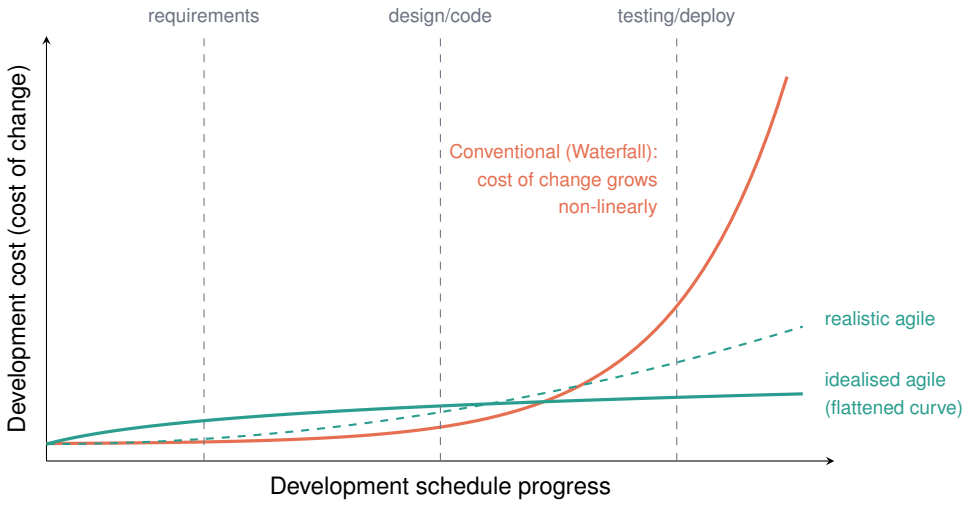
2018–2019 • Q5(b) CSE307 • 06/11/2019 • P008 **MARKS 10**

Draw the graph of Development cost vs. Development schedule progress in the context of Agile Development.

SOLUTION

What the graph shows

The graph plots the **cost of making a change** (development cost) against how far the project has progressed along its schedule. It contrasts conventional (plan-driven) development with agile development.



Explanation

- **Conventional curve**: a change requested during requirements gathering costs little (edit the SRS). The same change during design or coding needs updating requirements, design, code and plans; after testing or deployment it may require rework of many components, regression testing and redeployment. Hence the cost of change rises *non-linearly (roughly exponentially)* with schedule progress.
- **Agile curve**: a well-designed agile process "flattens" the cost-of-change curve. Because the product is built in small increments, tested continuously (test-first, continuous integration), refactored to keep design simple, and reviewed by the client every iteration, a late change only affects the current small increment. The cost still rises somewhat (realistic dashed line), but far more slowly.
- **Implication**: agile teams can accommodate changes late in the project without dramatic cost and time impact — the basis of the principle "welcome changing requirements, even late in development".

Answer. Conventional development: cost of change rises steeply (non-linearly) as the schedule progresses. Agile development: incremental delivery, continuous testing and refactoring keep the curve flat, so late changes remain relatively cheap.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2018–2019 • Q5(c) CSE307 • 06/11/2019 • P008 **MARKS 3**

What does framework activity mean in the context of software processes?

[Second half of Q5(c) — "Name five framework activities encouraged in Personal Software Process (PSP)" (10) — is solved in the out-of-syllabus section.]

2013–2014 • Q4(c) CSE307 • 08/08/2015 • P016 **MARKS 8**

Discuss framework and umbrella activities for software development.

SOLUTION

What a process framework is

A **generic process model** (process framework) gives the definitive description of the work done in *every* software project. It consists of a small set of **framework activities** that produce the product, plus a set of **umbrella activities** that run across the whole project and protect/manage it.

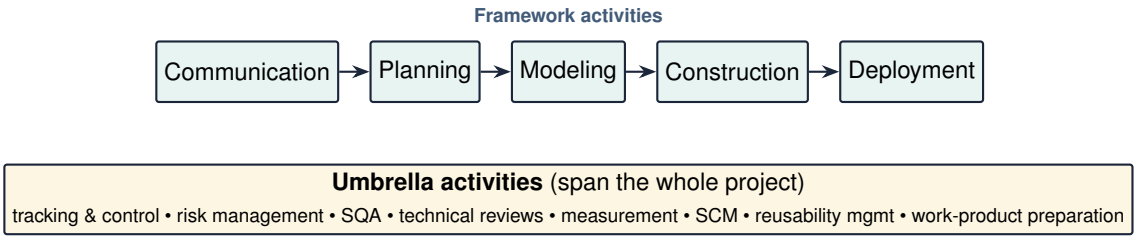
What a framework activity means

A **framework activity** is one of the small set of core activities that the *generic process framework* prescribes for *every* software project, regardless of its size or type. Each framework activity is populated with software-engineering actions and task sets, and produces work products. A process model is then simply a particular *flow* (linear, iterative, evolutionary or parallel) through these activities.

The five framework activities (core activities)

Activity	What the team does
Communication	Talks with the client/stakeholders to understand what they actually want; requirements are gathered and clarified.
Planning	Builds the project "map": schedule, time and cost estimation, resources, work products, progress tracking.
Modeling	Analyses the requirements and designs the software (architecture, algorithms, flowcharts, component design) — *how* it will be built.
Construction	Implements (codes) the designed components and tests them.
Deployment	Delivers the product (or an increment) to the client, receives feedback and improves the product where required.

These five activities can be arranged as a *linear*, *iterative*, *evolutionary* or *parallel* process flow; the arrangement is what distinguishes one process model from another.



Umbrella activities (additionally for 2013–2014 Q4(c))

Umbrella activities are applied *throughout* the project, on top of every framework activity, to help the team manage progress, quality, change and risk:

- **Software project tracking and control** — compare progress with the plan and take corrective action to keep the schedule.
- **Risk management** — identify, assess and plan for risks that could affect the outcome or quality.

- **Software quality assurance (SQA)** — define and carry out activities that ensure the required quality.
- **Technical reviews** — review work products (requirements, design, code) to find errors before they propagate.
- **Measurement** — collect process, project and product metrics that help the team deliver.
- **Software configuration management (SCM)** — control the effects of change on all work products.
- **Reusability management** — define criteria for reusing work products and build reusable components.
- **Work product preparation and production** — create models, documents, logs, forms and lists.

Difference in one line

Framework activities *create* the software step by step; umbrella activities *surround* those steps and keep the project under control from start to finish.

Answer. Framework activities are the generic, always-present building blocks of a software process: Communication, Planning, Modeling, Construction, Deployment; every process model arranges them in its own flow. Umbrella activities: project tracking & control, risk management, SQA, technical reviews, measurement, SCM, reusability management and work-product preparation — applied across all framework activities.

NOTE The CSE213 slides list the five framework activities; umbrella activities are the standard complementary part of the generic process framework (Pressman) and are included for completeness.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2018–2019 • Q7(a) CSE307 • 06/11/2019 • P009 **MARKS 10**

Draw a diagram showing the phases of the Extreme Programming (XP) method.

[First half of Q7(a) — "Name three phases of Adaptive Software Development (ASD)" (6) — is solved in the out-of-syllabus section.]

2011–2012 • Q5(b) CSE307 • 23/09/2013 • P021 **MARKS $18\frac{2}{3}$**

What is agile software development? Briefly describe different stages of extreme programming.

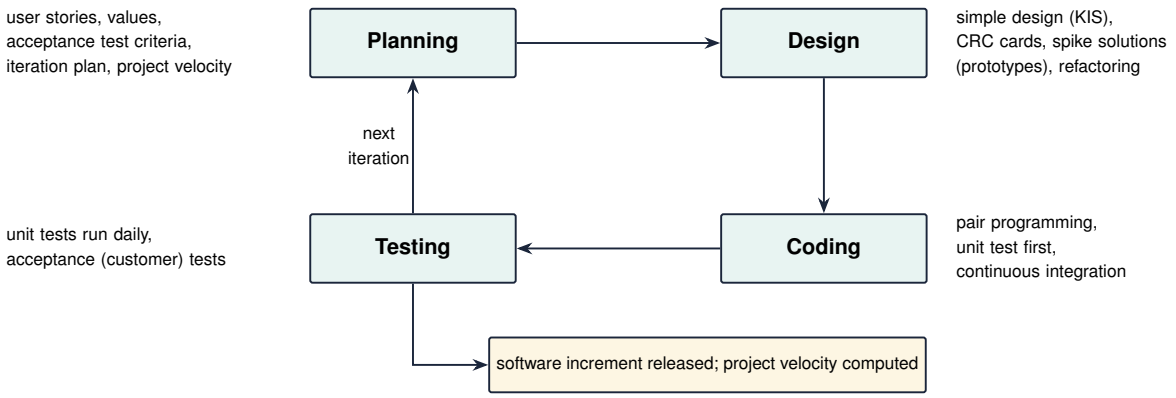
SOLUTION

Additionally for 2011–2012 Q5(b): agile software development

Agile development is a family of lightweight, iterative and incremental process models that focus on **quick, frequent and incremental release** of working software, **strong client involvement**, and **flexibility to change** in both product and team, while discouraging detailed up-front analysis, planning and documentation. It follows the 12 principles of the Agile Manifesto (early and continuous delivery, welcome change, working software as the measure of progress, face-to-face communication, self-organising teams, simplicity, regular reflection, etc.). Examples: XP, Scrum, Kanban.

Extreme Programming (XP) in one paragraph

XP, developed by Kent Beck, takes beneficial engineering practices to *extreme* levels from day one (if testing is good, test continuously; if reviews are good, review continuously via pair programming). It values communication, simplicity, feedback, courage and respect, and prioritises customer satisfaction. XP is organised as repeated iterations of four framework activities (phases) — **planning, design, coding, testing** — each iteration producing a tested software increment, as the diagram shows.



Stage 1 — Planning (the planning game)

- The customer writes **user stories** (short descriptions of required features) on index cards and assigns each a *value* (priority).
- Developers estimate the cost of each story in weeks/story points; stories that are too large are split.
- Customer and team decide which stories go into the next release/iteration: highest value first, or riskiest first.
- After the first release, **project velocity** (stories completed per iteration) is computed and used to plan later releases realistically.
- Acceptance test criteria are agreed for each story.

Stage 2 — Design

- Strictly follows *keep it simple* (KIS): design only for the stories in hand, never for speculative future features.
- CRC (class–responsibility–collaborator) cards organise the object-oriented classes relevant to the increment.
- For a difficult design problem a **spike solution** (operational prototype) is built immediately to reduce risk.
- **Refactoring** — improving the internal structure without changing external behaviour — is continuous, so design happens both before and during coding.

Stage 3 — Coding

- **Test first**: unit tests for each story are written *before* the code; the code is written to make the tests pass.
- **Pair programming**: two developers at one computer (driver and navigator) design, code and test together — real-time problem solving and continuous code review.
- **Continuous integration**: pairs integrate their code frequently (often daily) with the work of others, catching interface errors early.
- Collective code ownership and coding standards.

Stage 4 — Testing

- The automated unit tests are run every time code changes (regression testing), giving immediate feedback.
- **Acceptance tests** (customer tests) are specified by the customer from the user stories and check the externally visible functionality of the increment.
- When the increment passes, it is released; the next iteration starts with planning.

Answer. Agile development = lightweight iterative–incremental development emphasising frequent delivery, client collaboration and adaptation to change. XP stages: Planning (user stories, values, estimates, velocity), Design (simple design, CRC cards, spike solutions, refactoring), Coding (test-first, pair programming, continuous integration) and Testing (automated unit tests, customer acceptance tests) → software increment.

2018–2019 • Q8(b) CSE307 • 06/11/2019 • P010 **MARKS 2 + 8**

(i) When will you use the Incremental model?
(ii) List four problems associated with Evolutionary Process Models.

[Q8(a) — the V model — is solved in the out-of-syllabus section.]

SOLUTION

(i) When to use the Incremental model

Use the incremental model when:

- the **overall requirements are reasonably well defined**, but the full scope is too large to deliver in one linear pass;
- the client **needs a product with primitive/basic functionality quickly** (a core product) rather than waiting for the whole system;
- the system **can be split** into fairly independent functional parts (increments);
- staff or budget are not available for a full implementation by the business deadline, so early increments can be built with fewer people and lower initial cost.

Example: a university result-processing system — increment 1: mark entry and GPA calculation; increment 2: transcripts; increment 3: online notifications.

(ii) Four problems of Evolutionary process models

(i) Project planning is difficult. The number of prototype/spiral cycles needed to reach the final product is unknown, so cost and schedule are hard to estimate; most estimation techniques

assume a linear layout of activities.

(ii) **Speed of evolution is not fixed.** If the product evolves too fast without pauses, the process falls into chaos; if too slowly, productivity suffers. There is no rule for the right pace.

(iii) **Flexibility over quality.** The focus is on flexibility and extensibility, and the client may see an early prototype as the working product; quick implementation compromises (inefficient algorithm, wrong platform) can stay in the final system, and frequent changes increase the system's complexity.

(iv) **Cost, time and expertise.** Repeated prototypes slow down the process and may be scrapped after client dissatisfaction (wasted effort); the spiral model becomes costly and complicated and requires risk-assessment expertise.

Answer. (i) Use incremental when requirements are mostly known but a basic working core is needed early and the system can be split into increments. (ii) Evolutionary models: uncertain planning (unknown number of cycles), undefined speed of evolution, flexibility favoured over quality / growing complexity, and high cost, wasted prototypes and need for risk expertise.

2018–2019 • Q8(a) CSE325 • 20/01/2021 • C325-1819-8a MARKS 15

When should we choose Agile approach over SDLC (Systems Development Life Cycle)? Also, explain why SDLC is sometimes chosen over Agile approach.

SOLUTION

NOTE This question is in Section B of the paper (header date 20/01/2021, "January 2020 Semester" – Section A of the same file is dated 20/12/2021).

The two approaches

- **SDLC (traditional, plan-driven)**: the system passes through planning, analysis, design, implementation and maintenance phases in sequence (waterfall-style); each phase produces complete documents that are signed off before the next phase starts.
- **Agile**: "moving quickly" – rapid, frequent, incremental releases driven by client feedback; working software over comprehensive documentation; changing requirements welcomed (XP, Scrum, Kanban).

When to choose Agile over SDLC

(i) **Requirements are unclear or changing** – market-driven products (mobile apps, e-commerce) where users discover what they want from early versions.

(ii) **Early and frequent delivery has business value** – time-to-market matters; a working core in weeks lets the business start earning or learning.

(iii) **Customer is available and involved** – a product owner can prioritise the backlog and review every sprint.

(iv) **Small, co-located, skilled, self-organising team** – face-to-face communication replaces heavy documentation.

(v) Low criticality – failures cause inconvenience, not loss of life, so minimal documentation is acceptable.

(vi) Innovative/exploratory work where it is cheaper to build and test an idea than to analyse it fully up front.

Example: a ride-sharing startup's app – features and pricing rules change weekly with user feedback.

Why SDLC is sometimes chosen over Agile

(i) Stable, well-understood requirements – e.g. payroll or a regulatory reporting system; nothing is gained from iterations, and a full design avoids architecture churn.

(ii) Fixed-price, fixed-scope contracts and outsourcing – the client needs the scope, cost and schedule agreed up front; government tenders often demand phase deliverables.

(iii) Safety- or mission-critical systems and regulation – medical, avionics, banking core: certification requires complete specifications, design documents, traceability and formal sign-offs that agile does not produce by default.

(iv) Large, distributed teams or many sub-contractors – documentation is the coordination mechanism when face-to-face communication is impossible.

(v) Limited client availability – the client cannot attend reviews every two weeks.

(vi) Predictability for management – budgeting, staffing and milestones are easier to plan and monitor.

Example: an insulin-pump controller or BUET's central accounts system with frozen rules and audit requirements.

Factor	Prefer Agile	Prefer SDLC
Requirements	volatile, emerging	stable, well known
Client involvement	continuous	at milestones only
Criticality	low–medium	high, regulated
Contract	time & material, flexible scope	fixed price, fixed scope
Team	small, co-located	large, distributed
Documentation	minimal, just enough	complete, mandatory

Answer. Choose Agile when requirements are uncertain/changing, early frequent delivery is valuable, the customer is continuously involved, the team is small and co-located and the system is not safety-critical. Choose SDLC when requirements are stable, the contract fixes scope/cost, the system is critical or regulated and needs full documentation, teams are large/distributed, or predictability is paramount.

2017–2018 • Q2(b) CSE307 • 29/09/2018 • P011 **MARKS 7**

Consider a software development of a government organization which is using the paper based system by the employees for the last 40 years. The average age of the employees is more than 50 years. Describe how prototyping helps writing SRS in this situation. Show all the phases of

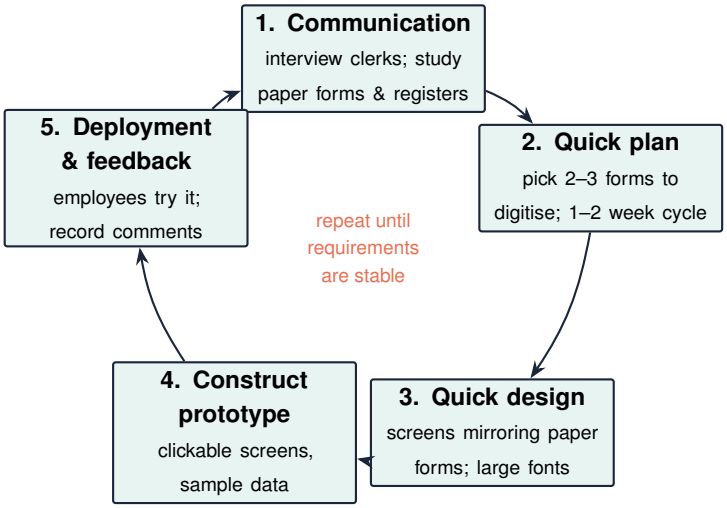
prototyping with respect to this development scenario.

SOLUTION

Why prototyping fits this scenario

- The employees have worked on paper for 40 years and are mostly over 50: they *cannot describe* requirements for a computer system in abstract terms, and requirements are vague.
- They may be anxious about technology; seeing and touching screens builds confidence and acceptance.
- Prototyping is used exactly "when client requirements are vague and likely to change": the prototype becomes the vehicle for discovering requirements, which are then written into the **SRS** precisely.

Phases of prototyping for this project



(i) Communication — meet the officers and clerks, collect sample paper files, registers, approval chains and reports; note what is known (e.g. "file an application, forward it to section head, approve, archive").

(ii) Quick plan — decide which part to prototype first (e.g. application intake form and search of old files) and time-box the iteration.

(iii) Modeling: quick design — design screens that look like the familiar paper forms, with large fonts, Bangla labels and minimal clicks; sketch the workflow.

(iv) Construction of prototype — build a clickable prototype with dummy data (no real database, security or performance tuning).

(v) Deployment, delivery and feedback — demonstrate it to the employees, let them try entering a real application, and record what is missing or confusing ("we also need a remarks column", "print a receipt with the same layout as Form-7"). The feedback starts the next cycle.

How the prototype produces the SRS

After a few cycles the agreed behaviour is written into the SRS: functional requirements (forms, fields, validation, workflow and approvals, reports), non-functional requirements (usability for elderly users: large fonts, simple navigation, Bangla interface, training needs; printing in old formats; data migration of 40 years of records), and acceptance criteria tied to the screens the users approved. Because the users validated the screens, the SRS is complete, correct and agreed, with far fewer misunderstandings than an interview-only SRS.

Answer. Since the elderly, paper-used employees cannot articulate requirements, quick prototypes of familiar-looking screens let them react concretely; through repeated cycles of communication, quick plan, quick design, prototype construction and deployment/feedback, the requirements are discovered, validated and written into a reliable SRS.

2017–2018 • Q2(d) CSE307 • 29/09/2018 • P012 **MARKS 7**

"A pair of programmers working together is more efficient than two programmers working separately" – Justify.

SOLUTION

Concept

Pair programming (XP) = two developers at one computer designing, coding and testing a user story together; the *driver* writes code while the *navigator* continuously reviews, thinks strategically and spots mistakes; roles swap regularly. "Efficiency" here means *useful, correct software delivered per unit of effort*, not raw lines typed.

Justification

(i) Continuous code review ⇒ fewer defects. Mistakes are caught the moment they are typed. Defects found later in testing or production cost many times more to fix, so the pair saves total effort.

(ii) Better design decisions. Two minds discuss alternatives in real time; the navigator keeps the bigger picture, producing simpler, cleaner code that needs less rework.

(iii) Less time stuck. When one person is blocked, the partner usually has an idea; problems are solved faster than two individuals each struggling alone.

(iv) Knowledge sharing and collective ownership. Both know the code; no single point of failure; new members learn quickly; later maintenance by anyone is faster.

(v) Discipline and focus. Pairs are less likely to skip tests, ignore standards or get distracted.

(vi) Less integration effort. Code is already reviewed and consistent, reducing merge and integration problems.

Empirical studies (e.g. Williams and Cockburn) reported that pairs spend only modestly more person-hours than solo programmers on a task, but produce noticeably fewer defects and shorter, better-designed programs — so total project effort, including testing and fixing, is lower.

Caveat

For trivial, routine tasks or badly matched pairs, the benefit shrinks; pairing should be used where quality and learning matter.

Answer. Justified: though two people work on one task, continuous review, better design, faster problem solving, knowledge sharing and fewer downstream defects mean the pair delivers more correct, maintainable software per total effort than two programmers working separately.

2017–2018 • Q2(e) CSE307 • 29/09/2018 • P012 **MARKS 7**

Write down a typical scrum meeting minutes (description of the meeting) for the software development of a digital research library.

SOLUTION

What a scrum meeting is

The **daily scrum** is a short (about 15-minute), daily stand-up meeting of the team, facilitated by the scrum master, for synchronisation. Each member answers three questions: (1) What did you do since the last meeting? (2) What will you do before the next meeting? (3) What obstacles are in your way? Problems are only *noted*; detailed discussion happens afterwards.

Minutes

Daily Scrum — Digital Research Library (DRL) project

Date / time	12 March, 09:30–09:45, Lab-2 (stand-up)
Sprint	Sprint 4 (day 6 of 10). Sprint goal: "Researchers can upload papers and search by keyword/author"
Attendees	Scrum master: Rafi; Team: Nusrat, Tanvir, Mehedi, Sadia; Product owner (Library rep.) observed

Member	Done since last scrum	Plan for today	Impediments
Nusrat	Paper-upload form with PDF validation	Store metadata (title, authors, DOI) in DB	none
Tanvir	Keyword search API (full-text index)	Author search + result paging	index build slow on 20k PDFs
Mehedi	Login via university SSO (LDAP)	Role-based access (student/faculty/admin)	waiting for SSO test credentials from IT
Sadia	Unit tests for upload service	Acceptance tests for search stories	none

Board / progress: 9 of 15 tasks done; burndown on track (2 points behind ideal line).

Impediments and action items

- SSO credentials — *Rafi (scrum master)* to contact IT office today.
- Slow indexing — Tanvir and Nusrat to discuss incremental indexing after the scrum (spike, 2 hours).

Notes: Product owner asked for "citation export (BibTeX)"; recorded as a new item in the *product backlog* for sprint 5 (sprint backlog unchanged). Next daily scrum: 13 March, 09:30.

Answer. Minutes record the sprint and goal, attendees, each member's three answers (done / next / impediments) about DRL tasks such as upload, search and SSO login, burndown status, action items for impediments, and new requests routed to the product backlog.

2017–2018 • Q2(b) CSE325 • 10/04/2019 • C325-1718-2b **MARKS 5 + 7 = 12**

Mention the five stages of the Agile Development life cycle. Draw the Systems Development Life Cycle.

SOLUTION

(i) Five stages of the agile development life cycle

In the systems-analysis view (Kendall & Kendall) an agile project goes through five stages; the middle ones are repeated in short iterations with the customer on site:

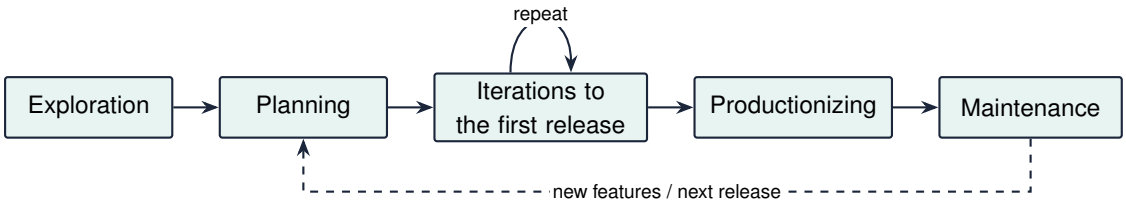
(i) **Exploration** – the team and the customer get to know the problem, the technology and each other; first user stories are written and the feasibility of the approach is checked.

(ii) **Planning** – customer and developers agree on what will be delivered first: user stories are estimated and prioritised and a release date is fixed (the "planning game").

(iii) **Iterations to the first release** – a series of short iterations (typically 1–3 weeks): design, code, test (test-first), integrate continuously and let the customer accept each increment.

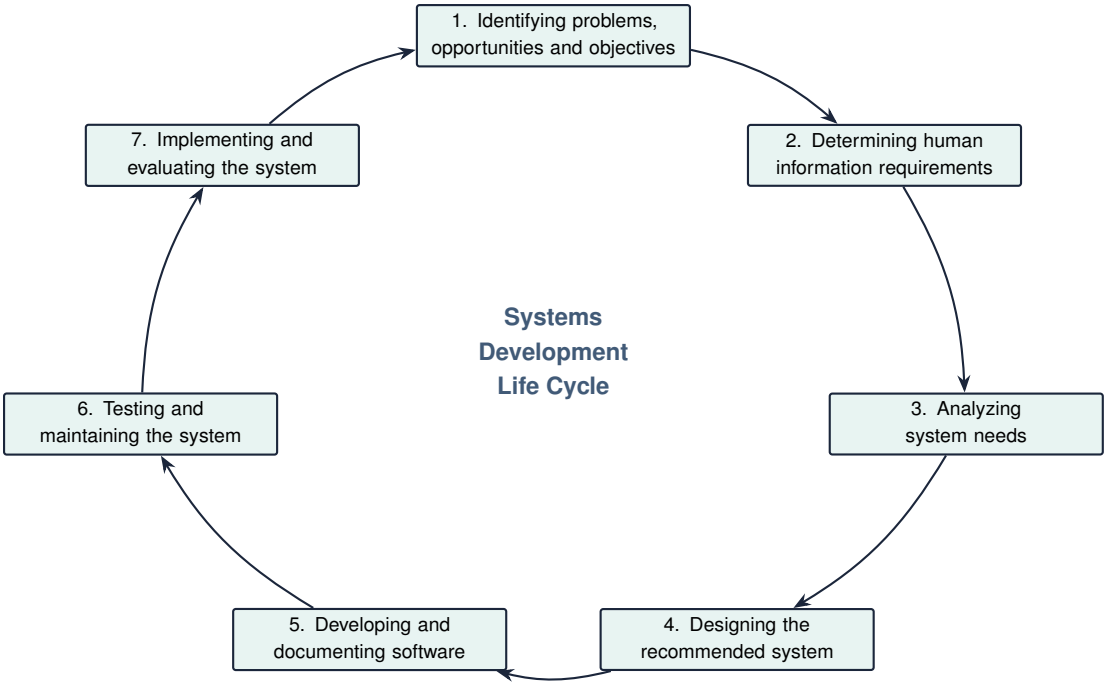
(iv) **Productionizing** – the release is put into production; feedback cycles shorten, performance is tuned and the product is deployed to users.

(v) **Maintenance** – the system is kept running while new features are added in further iterations and releases (the cycle continues until the project is closed).



(ii) Systems Development Life Cycle (SDLC)

The SDLC is the phased approach to analysis and design in which a system is developed through a specific cycle of analyst and user activities. The classic seven phases are:



- **Phases 1–3 (analysis):** define the problem and objectives, find out what users need (interviews, sampling, questionnaires, observation, prototyping) and analyse the needs (DFDs, data dictionary, structured decisions), ending with a system proposal.
- **Phase 4 (design):** logical design of data, input/output, user interface and controls.
- **Phases 5–7 (implementation):** programmers develop and document the software, it is tested and maintained, then installed (users trained, conversion done) and evaluated.
- The arrow from phase 7 back to phase 1 shows the *cycle*: evaluation and maintenance reveal new problems/opportunities, which start a new cycle.

Answer. Agile stages: exploration, planning, iterations to the first release, productionizing, maintenance. SDLC: (1) identify problems, opportunities, objectives → (2) determine human information requirements → (3) analyze system needs → (4) design the recommended system → (5) develop and document software → (6) test and maintain the system → (7) implement and evaluate the system → back to (1).

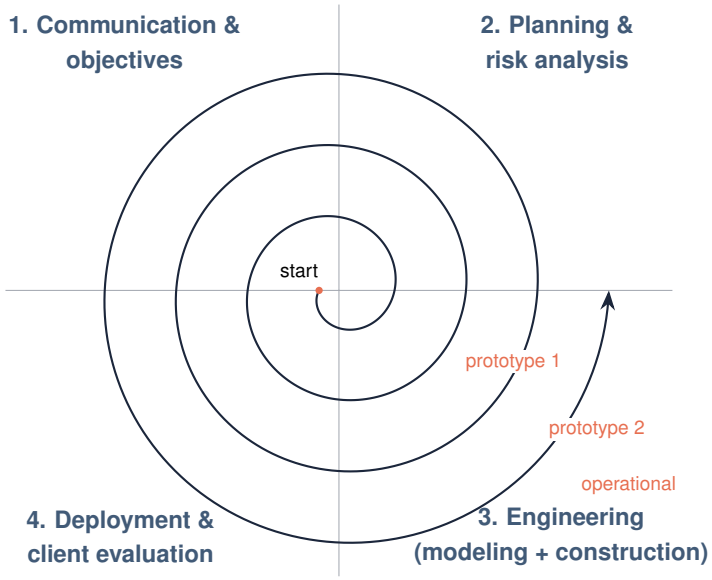
2016–2017 • Q5(b) CSE307 • 15/07/2017 • P013 MARKS 10 + 5

Briefly describe Spiral process model.
Write down the arguments in favor of and in opposition to Spiral process model.

SOLUTION

Concept

The **Spiral model** is an *evolutionary* process model. Like the incremental model it develops the product through a series of increasingly complete versions, but unlike incremental and prototyping models it deals with the uncertainties of the project by **incorporating risk analysis throughout the process**. It combines the iterative nature of prototyping with the controlled, systematic aspects of Waterfall.



How it works

- The process is a spiral of loops; each loop passes through the same sectors: **communication / determine objectives and constraints**, **planning with risk analysis** (identify risks, evaluate alternatives, build risk-reducing prototypes), **engineering** (modeling and construction of this version) and **deployment/evaluation** by the client, which feeds the plan for the next loop.
- The **first loop** may produce only a basic prototype (or even just a concept/requirements spec); **subsequent loops** produce more mature versions.
- Cost and schedule are re-adjusted after every loop based on client feedback; the radius of the spiral represents cumulative cost and the angular progress represents progress within a loop.
- The spiral continues until an acceptable product is built and delivered (and can even continue during maintenance).

Arguments in favour

- **Rigorous risk handling**: risks (technical, schedule, cost) are identified and resolved in every loop before they become problems — the main strength.
- **Any type of change can be incorporated even at a later stage**, because each loop re-plans.
- Early prototypes give the client something to evaluate; requirements become clearer.
- Suitable for large, complex, high-risk projects; combines prototyping with systematic control.
- Cost and schedule estimates become more realistic as the spiral progresses.

Arguments in opposition

- **Costly and complicated**: repeated loops plus risk analysis need significant resources — overkill for small, low-risk projects.
- **Requires risk-assessment expertise**; if risks are not identified correctly the model loses its benefit.

- Hard to convince clients (especially in fixed-price contracts) that the evolutionary approach is controllable; the number of loops and end point are not known in advance.
- More management effort and documentation per loop.

Answer. Spiral = evolutionary, loop-by-loop development where every loop includes explicit risk analysis and produces a more mature version. For: strong risk handling, late changes possible, good for large risky systems. Against: expensive, complex, needs risk expertise, uncertain end-point.

2016–2017 • Q6(a) CSE307 • 15/07/2017 • P014 MARKS 3×5

Write short notes on the following:

(i) Sprint

(ii) Backlog

(iii) Pair Programming

(iv) Code Refactoring

(v) Spike Solution

SOLUTION

(i) Sprint

A **sprint** is the unitary, time-boxed cycle (iteration) of development in **Scrum**, usually lasting **2 weeks to 1 month**. It starts by selecting items from the product backlog into the sprint backlog and proceeds through planning, implementation, product review and process review (retrospective). The sprint backlog is frozen during the sprint; at the end a potentially shippable increment is delivered to the client for feedback, and unfinished items return to the product backlog.

(ii) Backlog

A **backlog** is the prioritised list of client requirements (features, user stories, fixes) for the product. Scrum has two types: the **product backlog** — all requirements of the whole project, owned and prioritised by the product owner and continuously updated — and the **sprint backlog** — the subset selected for a particular sprint, broken into tasks (tickets) by the team.

(iii) Pair programming

An XP practice in which **two developers work together at one computer** to design, code and test a user story. The *driver* types the code; the *navigator* reviews each line, thinks about design, tests and edge cases; they switch roles frequently. Benefits: continuous code review and fewer defects, real-time problem solving, shared knowledge and collective code ownership, better design. Cost: two people on one task — offset by higher quality.

(iv) Code refactoring

Refactoring is the process of changing a software system so that it **does not alter the external behaviour of the code yet improves its internal structure**. In XP it is a continuous design activity: after making tests pass, developers remove duplication, rename, extract methods/classes, simplify conditionals, etc., keeping the design simple and the code easy to change. Unit tests guarantee that behaviour is preserved. Example: three methods each compute a 15% VAT with the same formula → *Extract Method* `vat(amount)` and call it from all three.

(v) Spike solution

When XP developers meet a **difficult design problem** or an uncertain technology/estimate, they immediately create a **spike solution**: a small, focused *operational prototype* of that portion of the design, built only to explore the problem. Its purpose is to lower risk when true implementation starts and to validate the original estimates for the story. The spike is usually thrown away; only the knowledge is kept. Example: before committing to a payment-gateway story, a pair spends a day writing a throw-away client that calls the gateway's sandbox API.

Answer. Sprint = 2–4-week time-boxed Scrum iteration; Backlog = prioritised requirement list (product and sprint backlog); Pair programming = two developers, one computer (driver/navigator); Refactoring = improving internal structure without changing behaviour; Spike solution = quick throw-away prototype to resolve a risky design problem.

2016–2017 • Q6(b) CSE307 • 15/07/2017 • P014 MARKS 15

Explain how "Extreme Programming (XP)" follows the agility principles.

SOLUTION

Approach

The agility principles are the 12 principles of the Agile Manifesto. XP is an agile model because its *practices* directly implement those principles. Each principle is mapped to the XP practice that realises it.

Agile principle	How XP follows it
1. Satisfy the customer by early and continuous delivery of valuable software	XP prioritises customer satisfaction; small releases deliver the most valuable user stories (chosen by customer value) first.
2. Welcome changing requirements, even late	Stories can be added, changed, split or dropped at every planning game; simple design and refactoring keep code changeable.
3. Deliver working software frequently	Short iterations (1–3 weeks) each end with a tested, released increment.
4. Client and developers work together daily	On-site customer writes stories and acceptance tests and is available to answer questions.
5. Build projects around motivated individuals	Whole-team ownership; problems are solved by the whole team; energising environment.
6. Face-to-face conversation	Co-located team, pair programming, stand-up meetings, customer on site instead of documents.
7. Working software is the primary measure of progress	Progress = user stories passing acceptance tests; project velocity counts completed stories.
8. Sustainable pace	XP's 40-hour week / no overtime rule.
9. Technical excellence and good design	Test-first development, continuous integration, refactoring, coding standards, pair programming as continuous review.
10. Simplicity — maximise work not done	"Do the simplest thing that could possibly work" (KIS); no design for speculative future needs.
11. Self-organising teams produce the best designs	Developers estimate and choose tasks; collective code ownership; design emerges through refactoring.
12. Reflect regularly and adjust	Velocity is recomputed after each release and plans adjusted; tests and pairs give feedback from seconds to months.

XP values that support agility

XP's five values — **communication** (with client and in pairs), **simplicity**, **feedback** (unit tests, acceptance tests, frequent releases), **courage** (to refactor and to discard code) and **respect** (between team and client) — are the attitudes behind the agile principles. XP's feedback loops operate at every time scale: seconds (pair), minutes (unit tests), hours (integration), days (stand-up), weeks (iteration) and months (release planning).

Answer. XP follows the agile principles through its practices: small frequent releases and customer on site (continuous delivery, collaboration), planning game with changing stories (welcome change), simple design and refactoring (simplicity, good design), test-first, CI and pair programming (technical excellence), velocity (working software as progress), 40-hour week (sustainable pace) and whole-team ownership (self-organising, motivated team).

2014–2015 • Q4(a) CSE307 • 04/02/2016 • P015 MARKS $6\frac{2}{3}$

Why are framework activities not applicable for any software development project?

SOLUTION

NOTE The wording is ambiguous. The standard textbook point is that the five framework activities *are* applicable to *any* project, but *not in the same way*: they have to be adapted. Both halves of that argument are given below, which answers the question under either reading.

Why the framework activities apply to every project

Every software project, whatever its size, has to do the same five kinds of work: it must **communicate** with the client to find out what to build, **plan** the work, **model** (design) the solution, **construct** (code and test) it and **deploy** it to get feedback. Whether it is a 200-line script or an air-traffic control system, none of these can be skipped completely, so the generic framework is universal.

Why they cannot be applied "as is" to every project

The framework says *what* kinds of work exist, not *how* to do them. A single fixed recipe is unsuitable because projects differ in:

- **Size and complexity** — a small app needs a one-page plan; a large system needs detailed planning, formal modeling and staged deployment.
- **Clarity/stability of requirements** — well-known requirements allow a linear flow (Waterfall); vague or changing requirements need an iterative or evolutionary flow (Prototyping, Spiral, Agile).
- **Risk and criticality** — safety- or finance-critical systems need heavy reviews and testing; a throw-away prototype does not.
- **Team and client** — co-located, self-organising teams with an available client can use agile practices; large distributed teams need more documentation.
- **Time-to-market** — urgent products need parallel or incremental flows.

Therefore each project must *adapt* the framework: choose the process flow (linear, iterative, evolutionary, parallel), the task set inside each activity, the work products, and the degree of rigour. This adaptation is exactly what a **prescriptive** or **agile** process model does.

Answer. The five framework activities are common to all projects, but they cannot be applied identically to every project: size, requirement stability, risk, team and deadline differ, so the order, iteration, depth and task sets of the activities must be tailored by choosing a suitable process model.

2014–2015 • Q4(b) CSE307 • 04/02/2016 • P015 MARKS 10

Compare incremental and spiral model for software development process.

SOLUTION

Common ground

Both are *iterative* models that build the product as a series of versions, deliver something usable before the end and take client feedback between versions. The key difference emphasised in the course: **the spiral model explicitly incorporates risk analysis throughout the process**; the incremental model does not.

Point-by-point comparison

Aspect	Incremental model	Spiral model
Category	prescriptive, incremental	prescriptive, evolutionary
Requirements	mostly specified beforehand; refined in each increment	may evolve; clarified loop by loop
Driver	delivering functionality (core product first)	reducing risk (riskiest items first)
Unit of iteration	increment = linear mini-sequence C–P–M–Cn–D producing a working part	loop through objectives, risk analysis, engineering, evaluation
First output	a usable core product	possibly only a prototype or concept
Risk handling	implicit (early feedback)	explicit risk identification and resolution every loop
Change handling	flexible between increments; big changes may hurt architecture	any change can be incorporated even late
Cost	lower initial cost, higher total cost	costly and complicated overall
Expertise needed	ordinary planning; must split system into increments	risk-assessment expertise
Best suited to	medium projects needing early basic functionality	large, complex, high-risk or novel projects

Illustration

For an online shop with clear requirements that wants to start selling soon, the incremental model delivers catalogue + cart first and adds payment and recommendations later. For a new autonomous-drone control system whose feasibility is uncertain, the spiral model first prototypes the riskiest part (obstacle detection), evaluates it, and only then commits to further loops.

Answer. Both build the system in successive versions; incremental is functionality-driven with mostly known requirements and a core product first, while spiral is risk-driven, lets requirements evolve, performs risk analysis in every loop, and is costlier but safer for large, uncertain projects.

2014–2015 • Q4(d) CSE307 • 04/02/2016 • P015 MARKS 10

Describe the distinguishing features of scrum model.

SOLUTION

Scrum in brief

Scrum (named after the rugby formation in which players pack closely to gain the ball) is a lightweight, iterative and incremental agile process model that prioritises simplicity and develops the product gradually with frequent delivery.

Distinguishing features

(i) **Time-boxed sprints** of 2 weeks to 1 month; each sprint produces a working, potentially shippable increment.

(ii) **Two backlogs**: the *product backlog* (all client requirements, prioritised by the product owner) and the *sprint backlog* (requirements chosen for the current sprint).

(iii) **Fixed sprint scope**: once a sprint starts its backlog is frozen; changes wait for the next sprint — change is welcomed but controlled.

(iv) **Defined roles**: product owner (client), scrum master (project manager / facilitator, removes impediments), self-organising development team.

(v) **Daily scrum**: short daily synchronisation meeting (done yesterday, plan today, impediments).

(vi) **Four sprint phases**: planning, implementation, product review, process review (retrospective).

(vii) **Visibility tools**: tickets (subtasks) on a scrum board; burndown charts.

(viii) **Less hierarchy**: developers have freedom to express themselves and to organise their work.

(ix) **Strong client involvement**: product owner prioritises and reviews every sprint.

(x) **Continuous process improvement** through retrospectives.

Advantages / limitation

Scrum adapts quickly to changing requirements, makes product deliverable in less time, enforces client involvement and reduces repeated mistakes via retrospectives; its main limitation is that it usually works well only with *small project teams*.

Answer. Scrum's distinguishing features: time-boxed sprints delivering increments, product and sprint backlogs, frozen sprint scope, product owner / scrum master / self-organising team roles, daily scrum, sprint review and retrospective, scrum board visibility and minimal hierarchy.

2014–2015 • Q5(b) CSE307 • 04/02/2016 • R020.1 MARKS 10

Discuss the disadvantages of the prototyping technique.

SOLUTION

Recap of prototyping

In the **prototyping model** one or more prototypes are built from the currently known (possibly vague) requirements; the client evaluates each prototype, gives feedback and new requirements, and the cycle repeats until the prototype evolves into an acceptable product.

Disadvantages

(i) **Prototyping can slow down the process.** Every cycle needs building, demonstrating and evaluating; with many cycles the overall delivery is delayed.

(ii) **Frequent changes increase complexity.** Constantly patching the evolving prototype makes its structure complicated and hard to maintain ("spaghetti" growth).

(iii) **Scrapped prototypes waste effort.** Client dissatisfaction may lead to throwing away several prototypes.

(iv) **Client mistakes the prototype for the product.** The client sees something that works and demands it be delivered "with a few fixes", unaware that it was built quickly, without attention to quality, security or maintainability.

(v) **Developer compromises become permanent.** To get the prototype running quickly, developers may use an inappropriate language, OS or inefficient algorithm, then forget why and leave it in the final system.

(vi) **Hard to plan and estimate.** The number of iterations is unknown, so cost and schedule are uncertain; scope can keep growing (scope creep) because the client keeps adding requirements.

(vii) **Documentation and analysis may be neglected**, since emphasis is on building quickly.

(viii) **Heavy dependence on client involvement** — if the client is unavailable, the model stalls.

Mitigation (good practice)

Agree at the start that the prototype is a requirements-discovery mechanism (possibly throw-away); time-box each cycle; and re-engineer the final product with quality in mind.

Answer. Prototyping can slow the process, increases complexity through frequent changes, wastes effort on scrapped prototypes, may lead the client to accept a low-quality prototype as the product, makes planning/estimation difficult and depends heavily on client availability.

2013–2014 • Q4(d) CSE307 • 08/08/2015 • P016 **MARKS 9**

Find a suitable software development process for the following case:

(i) A scientific software development that require lots of innovation and there are unknown issues.

(ii) A quick development of stock brokerage solution for a new instrument.

(iii) A large software development with very unstable users.

Show proper reasoning for each case.

SOLUTION

(i) Scientific software, much innovation, unknown issues → Spiral model

- "Unknown issues" and "innovation" mean high *technical risk*: algorithms may not work, performance may be inadequate, the problem itself is only partly understood.

- The spiral model is the only course model that **incorporates risk analysis throughout the process**: each loop identifies the biggest unknown, builds a prototype/experiment to resolve it, evaluates it with the scientists, and only then plans the next loop.
- Requirements may evolve as scientists learn, and spiral accepts changes even late.
- Why not others: Waterfall needs known requirements; incremental assumes the overall requirements are known; prototyping alone explores requirements but not technical risk systematically. (Exploratory prototyping inside the spiral loops is ideal.)

(ii) Quick stock-brokerage solution for a new instrument → Agile (XP / Scrum)

- The overriding need is *speed*: the instrument must be tradable as soon as possible (competitive advantage). Agile is about "moving quickly": rapid, frequent, incremental delivery of working software.
- Rules of the new instrument may be clarified/changed by the regulator or the business while development is going on; agile welcomes changing requirements.
- The domain team (brokers) can sit with developers; continuous client communication shortens decision time.
- Financial correctness is critical: XP's test-first development, pair programming and continuous integration keep quality high despite speed.
- Why not others: Waterfall and spiral are too slow; incremental would also work if requirements were fully fixed, but agile's shorter cycles deliver the first tradable version sooner.

(iii) Large software with very unstable users → Spiral (risk-driven evolutionary) model

- "Unstable users" = users keep changing their minds / user base and needs are volatile: requirements will *evolve*, so an evolutionary model is required.
- The project is *large*: Scrum "usually works well with small project teams" and agile discourages the documentation and planning a large effort needs; prototyping alone would make a large system's structure increasingly complex.
- Spiral handles both: each loop re-validates requirements with users (customer evaluation), treats requirement volatility as an explicit risk, re-plans cost and schedule, and allows changes even at later stages; it is a realistic approach for large-scale systems.
- Why not others: Waterfall would freeze requirements that will change; incremental assumes requirements are known up front.

Case	Key fact	Model	Deciding reason
(i)	innovation, unknown issues	Spiral	explicit risk analysis + prototypes each loop
(ii)	quick, new instrument	Agile (XP/Scrum)	fastest frequent delivery, embraces change
(iii)	large, unstable users	Spiral	evolving requirements at large scale, re-planning each loop

Answer. (i) Spiral (risk-driven, with prototypes) for innovative scientific software; (ii) Agile XP/Scrum for a quick brokerage solution; (iii) Spiral (evolutionary, scalable, change-tolerant) for a large system with unstable users.

NOTE Other well-argued answers are acceptable: e.g. evolutionary *Prototyping* for (i), or *Incremental*/RAD for (ii). Marks are for matching the scenario facts to model characteristics.

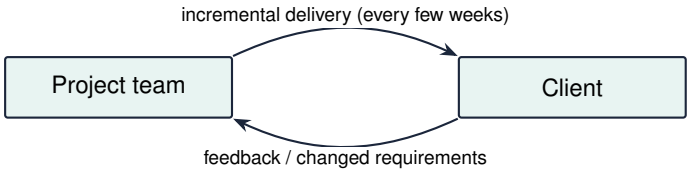
2013–2014 • Q6(a) CSE307 • 08/08/2015 • P017 **MARKS $6\frac{2}{3}$**

What is meant by agile method?

SOLUTION

Definition

Agile means "moving quickly". An **agile method** is a lightweight, iterative and incremental software process in which the focus is on **rapid and incremental delivery** of working software, with **active and continuous communication** between the project team and the client, rather than on extensive up-front analysis, planning and documentation.



Characteristics

- Quick, frequent and incremental releases (a couple of weeks to a couple of months).
- Strong client involvement throughout the project; face-to-face communication.
- Plans are short-lived: requirements change, so the team adapts rather than following a fixed plan; more focus on construction than on planning and modeling.
- Working software is the primary measure of progress; simplicity; sustainable pace.
- Self-organising, motivated teams; regular reflection (retrospectives).
- Flexible to changes in both the product and the project team.

The Agile Manifesto (2001) values individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. Examples: **XP**, **Scrum**, **Kanban**.

Answer. An agile method is a lightweight, iterative–incremental process that delivers working software quickly and frequently, keeps the client continuously involved, and adapts to changing requirements instead of relying on heavy up-front planning and documentation (e.g. XP, Scrum, Kanban).

2012–2013 • Q1(b) CSE307 • 17/05/2014 • P018 **MARKS 7**

"Waterfall Model may lead to blocking state" – Do you agree with the statement? Why or why not?

[Q1(a) of this excerpt — Generic vs Bespoke products — is in the out-of-syllabus section.]

SOLUTION

Verdict

Yes, I agree. Because Waterfall is strictly linear, it naturally creates *blocking states*: situations in which some team members cannot do productive work because they are waiting for other members to finish tasks on which their own work depends.

Why blocking states arise

(i) Phase-gate dependency. Modeling cannot start until the requirements document is complete and approved; coding waits for the complete design; testing waits for the complete code. Programmers and testers therefore sit idle (or do low-value work) during the early phases.

(ii) Single pass, no overlap. Waterfall does not encourage parallel or iterative flow, so a delay in one phase pushes *every* downstream person back.

(iii) Late discovery of problems. If testing finds a requirements or design error, the project must go back to an earlier phase; everybody downstream again waits for the fix.

(iv) Change requests. A client change late in the project freezes construction until documents and design are revised.

Pressman notes that in such projects "the time spent waiting can exceed the time spent on productive work", especially at the start and end of the project.

Small illustration

In a library-system project, three testers are hired at the start. Requirements take 6 weeks and design 8 weeks; the testers cannot test anything for 14 weeks plus the coding time. If the design review finds a flaw in week 13, the coders who were about to start are blocked as well.

How other models avoid it

Incremental and agile models overlap activities: while one increment is being tested the next is being designed, so every role has useful work; parallel process flow also lets independent activities proceed simultaneously.

Answer. Agree — the sequential, dependent phases of Waterfall force team members to wait for others to finish (requirements → design → code → test), and late rework causes further waiting; iterative/incremental and parallel flows reduce such blocking states.

2012–2013 • Q2(d) CSE307 • 17/05/2014 • P020 MARKS $6\frac{2}{3}$

Discuss use of "Project Velocity" with respect to Extreme Programming model. Provide a suitable example.

[Q2(c) — RAD model — is solved in the out-of-syllabus section.]

SOLUTION

Definition

In XP, **project velocity** is the **number of user stories (or story points) implemented and accepted in one iteration/release**. It is measured after the first release and updated after every iteration.

Uses

(i) **Estimate delivery dates and schedule** for subsequent releases: remaining work ÷ velocity = number of iterations still needed.

(ii) **Decide how much to commit** in the next iteration's planning game: the team takes on stories totalling about the last velocity ("yesterday's weather").

(iii) **Detect over-commitment**: if the content of a release exceeds what the velocity allows, the customer must drop or defer stories, or move the date.

(iv) **Monitor progress and estimation accuracy**: a falling velocity signals problems (technical debt, impediments); estimates are calibrated.

Example

An e-library team uses 2-week iterations and story points:

Iteration	Stories accepted	Points	Velocity
1	search, login, view book	5 + 3 + 4	12
2	borrow, return, fines	6 + 4 + 4	14
3	reservation, notifications	8 + 5	13

Average velocity $= (12 + 14 + 13)/3 = 13$ points/iteration. Remaining backlog $= 78$ points $\Rightarrow 78/13 = 6$ more iterations $= 12$ weeks. If the customer wants the release in 8 weeks (4 iterations ≈ 52 points), about 26 points of lower-value stories must be deferred.

Answer. Project velocity = stories (points) completed per iteration; XP uses it to plan how much to commit, forecast release dates (remaining points ÷ velocity) and detect over-commitment, e.g. velocity 13 and 78 remaining points ⇒ 6 more iterations.

2011–2012 • Q5(c) CSE307 • 23/09/2013 • P021 **MARKS 18**

Compare and contrast the Scrum approach to project management with conventional plan-based approaches. The comparisons should be based on the effectiveness of each approach for planning the allocation of people to projects, estimating the cost of projects, maintaining team cohesion, and managing changes in project team membership.

SOLUTION

The two approaches

- **Conventional plan-based** (Waterfall-style): a project manager produces a detailed plan up front — work breakdown, schedule, budget — assigns individuals to tasks and controls progress against the plan; communication is largely through documents.
- **Scrum**: work is organised into time-boxed sprints from a prioritised product backlog; a self-organising team decides how to do the work; the scrum master facilitates; progress is visible daily (daily scrum, board, burndown); the product owner re-prioritises every sprint.

1. Planning the allocation of people to projects

Plan-based: people are allocated to tasks/phases in advance, based on the work breakdown and individual skills. Effective when work is predictable and specialists (analyst, designer, tester)

are needed at known phases; it lets managers share people across several projects. Weak when requirements change, because the allocation plan becomes obsolete.
Scrum: a stable, small, cross-functional team is allocated to the *project* for its duration; inside each sprint members *pull* tasks themselves. Allocation is simple and adapts to change, but it is hard to share specialists between projects, and the team must have all needed skills.

2. Estimating the cost of projects

Plan-based: cost is estimated up front from the complete requirements (e.g. size- or effort-based models), giving a fixed budget useful for contracts. Accuracy is poor when requirements are uncertain, and estimates are rarely revisited until overruns appear.
Scrum: cost per sprint is known (team size $\times$ sprint length); the number of sprints is estimated from backlog size $\div$ velocity and refined every sprint, so estimates become more accurate over time. However, a total fixed price is hard to give at the start; the client may instead fix the budget and let the scope vary.

3. Maintaining team cohesion

Plan-based: individuals work on separately assigned tasks, coordinated by the manager and documents; less day-to-day interaction, so cohesion depends on the manager's effort.
Scrum: strongly supports cohesion — daily scrums, shared sprint goal, collective responsibility, co-location, less hierarchy, and retrospectives where the team improves together. The team becomes self-organising and mutually supportive. Risk: groupthink or dependence on a few strong members.

4. Managing changes in project team membership

Plan-based: detailed documentation (requirements, design, plans) makes it relatively easy for a replacement to come up to speed; individual roles are clearly defined so a person can be swapped.
Scrum: knowledge is largely *tacit* (little documentation), so losing a member can hurt; a newcomer also disturbs the team dynamics and velocity for a few sprints. Mitigations: shared ownership, daily scrums and pair programming spread knowledge, so no single person holds critical information.

Summary

Criterion	Scrum	Plan-based
People allocation	stable team per project; self-assigned tasks; flexible within project	up-front assignment of specialists; good for sharing staff; rigid under change
Cost estimation	per-sprint cost certain; total refined via velocity; weak for fixed price	fixed up-front estimate; good for contracts; inaccurate if requirements unclear
Team cohesion	high (daily scrum, shared goals, retrospectives)	lower; manager-driven
Membership change	risky (tacit knowledge), mitigated by shared ownership	easier (documentation, defined roles)

Answer. Scrum is more effective for flexible task allocation within a stable team, progressively accurate cost estimates and strong team cohesion; plan-based approaches are better for allocating specialists across projects, fixed up-front cost estimates and coping with staff turnover through documentation.

2011–2012 • Q8(a) CSE307 • 23/09/2013 • P022 MARKS 10

You have an impatient, result-oriented client who keeps on changing requirements. Which process model would you adopt to keep the development under control? Why?

[Q8(b) — RUP — is solved in the out-of-syllabus section.]

SOLUTION

Scenario facts

- **Impatient, result-oriented**: wants to see working results soon and often.
- **Keeps changing requirements**: any up-front specification will quickly be outdated.
- Need: **keep development under control** despite the changes.

Choice: Agile — Scrum (with XP engineering practices)

(i) Results quickly and frequently. Each **sprint** (2 weeks–1 month) ends with a working, demonstrable increment; "working software is the primary measure of progress" — exactly what a result-oriented client wants to see.

(ii) Change is absorbed in a controlled way. New or changed requirements are added to the **product backlog** and prioritised by the client (product owner). The **sprint backlog** is fixed for the current sprint, so the team is not disrupted mid-sprint; changes take effect from the next sprint. This is what keeps development *under control*.

(iii) Client involvement. The client takes part in sprint planning and the sprint review, so he sees how each change affects scope and dates; trade-offs become visible and negotiable.

(iv) Visibility. Daily scrum, scrum board and burndown expose progress and impediments every day.

(v) Retrospectives improve the process after each sprint.

(vi) Technical control. XP practices — simple design, test-first, continuous integration, refactoring — keep the code changeable so frequent requirement changes stay cheap.

Why the alternatives fit worse

- **Waterfall**: needs all requirements up front; every change forces rework of documents and design; client sees results only at the end — unacceptable to an impatient client.
- **Incremental**: delivers early, but assumes requirements are known beforehand; continually changing requirements may break the planned architecture.
- **Prototyping**: clarifies requirements but can slow the process; an impatient client may demand the prototype be shipped.
- **Spiral**: handles change and risk but is costly and slow per loop.

Answer. Adopt an agile model — Scrum with XP practices: short time-boxed sprints give the impatient client working results every few weeks, changing requirements go into a client-prioritised product backlog and are taken up only at sprint boundaries (fixed sprint backlog), which keeps the development controlled and transparent.

2010–2011 • Q6(a) CSE307 • 19/03/2012 • P024 **MARKS 3 + 3**

Discuss about waterfall model and iterative model of software development.

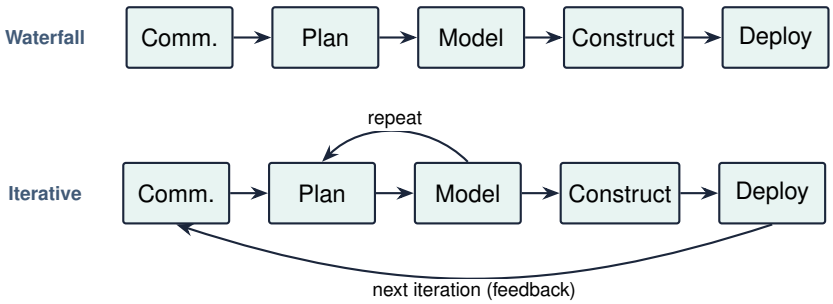
SOLUTION

Waterfall model (linear flow)

The five framework activities — communication, planning, modeling, construction, deployment — are executed *once and strictly in sequence*; a phase starts only after the previous phase is complete. It suits projects whose requirements are well defined and stable. It is simple and easy to manage, but requirements must be known up front, testing and client evaluation come late (high risk), and iteration is not promoted.

Iterative model (iterative flow)

In an iterative process flow, *one or more activities are repeated before proceeding* to the next activity, and in practice the whole cycle is repeated several times: each iteration does some communication, planning, modeling, construction and deployment and produces a better version of the software. Feedback from one iteration (from testing or from the client) is used to refine requirements and design in the next. Incremental, prototyping, spiral and all agile models are built on this idea.



Comparison

Aspect	Waterfall	Iterative
Flow	linear, one pass	repeated cycles
Requirements	fixed up front	refined every iteration
Working software	only at the end	after each iteration
Risk / error discovery	late, expensive	early, cheaper
Best for	small, stable, well-understood projects	changing or unclear requirements

Answer. Waterfall = one sequential pass through the activities (simple, rigid, late feedback); Iterative = the activities are repeated in cycles, each producing an improved version (flexible, early feedback and risk reduction).

08

Software Testing

2023–2024 • Q6(c) CSE213 • 10/05/2026 • T003 **MARKS 10**

A smart temperature system has the following three modules.

- Display Panel: top-level module that shows temperature and warning messages to the user
- Temperature Controller: middle-level module that receives sensor data, processes it, and decides what should be displayed
- Sensor: low-level module that reads the actual temperature from the environment

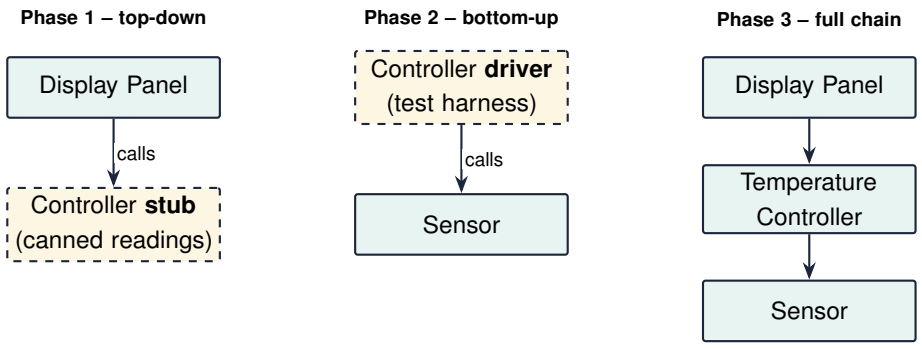
The Display Panel and Sensor are ready first, but the Temperature Controller is still under development. How would you conduct integration testing in this situation?

SOLUTION

Analysis

The missing module is in the *middle* of the call hierarchy: the Display Panel (top) *calls* the Controller, and the Controller *calls* the Sensor (bottom). We do not wait (that would be big-bang integration late in the project); instead we use a **sandwich (hybrid)** strategy:

- test the top module **top-down** with a **stub** replacing the Controller, and
- test the bottom module **bottom-up** with a **driver** replacing the Controller,
- then replace stub and driver with the real Controller and test the full chain.



Step 1 — Display Panel with a Controller stub (top-down)

Agree on the Controller's interface first. A stub implements it and returns pre-set values so that the display logic can be tested: normal temperature → value shown, high temperature → warning shown, sensor-error code → "Sensor fault" message, boundary of the warning threshold.

Step 2 — Sensor with a Controller driver (bottom-up)

A driver (test harness) plays the role of the Controller: it calls the real Sensor, checks that readings arrive in the agreed format/unit (°C, not °F), within the physical range, at the expected rate, and that a disconnected sensor raises the agreed error.

```java
interface TemperatureController { Reading latest(); }      // agreed interface
record Reading(double celsius, boolean warning, boolean sensorError) {}

class ControllerStub implements TemperatureController {     // STUB (phase 1)
    private final Reading canned;
    ControllerStub(Reading canned) { this.canned = canned; }
    public Reading latest() { return canned; }
}

class DisplayPanelTopDownTest {
    @Test
    void showsWarningWhenControllerFlagsHighTemperature() {
        DisplayPanel panel = new DisplayPanel(new ControllerStub(new Reading(48.0, true, false))
    );
        assertEquals("48.0 C - WARNING: too hot", panel.render());
    }
    @Test
    void showsFaultMessageOnSensorError() {
        DisplayPanel panel = new DisplayPanel(new ControllerStub(new Reading(0, false, true)));
        assertEquals("Sensor fault", panel.render());
    }
}

class SensorBottomUpDriverTest {                             // DRIVER (phase 2)
    @Test
    void sensorReturnsReadingInPhysicalRange() {
        Sensor sensor = new Sensor();
        double t = sensor.readCelsius();          // driver calls the real sensor
        assertTrue(t > -40.0 && t < 125.0, "reading out of physical range");
    }
}
```

Step 3 — integrate the real Controller

When the Controller is ready, first test *Controller + Sensor* (driver now calls the real Controller), then *Display + Controller + Sensor*, replacing the stub. Re-run all earlier tests as regression tests and add end-to-end cases: heat the sensor above the threshold → warning appears on the panel; unplug the sensor → fault message.

Answer. Use sandwich (hybrid) integration: test Display Panel top-down with a Controller *stub*, test Sensor bottom-up with a Controller *driver*, both against the agreed Controller interface; when the Controller is finished, replace stub and driver with it and test Controller+Sensor and then the full chain, re-running earlier tests.

2023–2024 • Q7(a) CSE213 • 10/05/2026 • T004.1 **MARKS 15**

You are asked to prepare a performance testing plan for a course-registration system. The system faces the following operational risks:

- a sudden burst to 10,000 students when registration opens
- possible collapse when the load exceeds the system's capacity
- gradual slowdown after 8 hours of continuous operation

For each of the above situations:

(i) Identify the most appropriate performance testing type and briefly state the goal of that test.

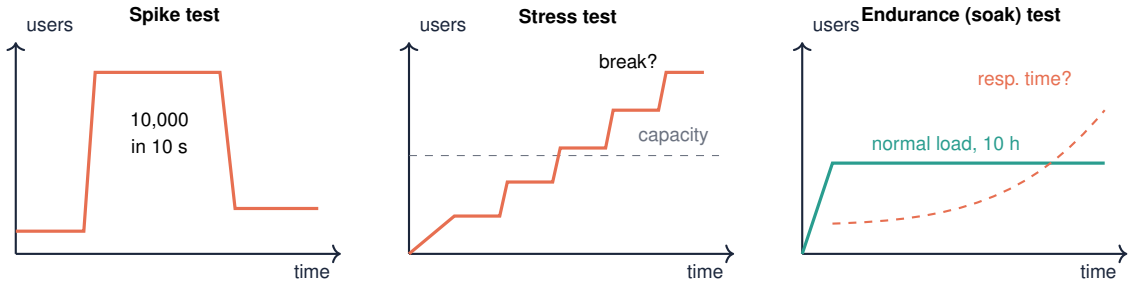
(ii) Propose a simple JMeter testing configuration by specifying Threads, Loop Count, Ramp-Up, and Duration.

SOLUTION

Performance testing types used

Performance testing is non-functional testing of the complete system. Besides plain **load testing** (expected load) and **stress testing** (beyond the specified limit) from the lecture, two specialised variants are relevant: **spike testing** (a stress test in which the load jumps abruptly) and **endurance / soak testing** (a load test held for a long time).

NOTE The question does not state the system's capacity. The plan assumes a designed capacity of about **5,000 concurrent users** and a normal daytime load of about **3,000 users**; the numbers scale proportionally for any other capacity.



(i) Test type and goal for each risk

Risk	Test type	Goal of the test
Sudden burst to 10,000 students at opening	**Spike test** (special stress test)	Verify that the system survives an abrupt jump from almost zero to 10,000 users: acceptable response time/error rate, no crash, no lost or duplicate registrations, and fast return to normal after the spike.
Collapse when load exceeds capacity	**Stress test**	Increase the load step by step beyond the capacity to find the breaking point and the failure mode; check graceful degradation ("server busy" instead of crash), data integrity and automatic recovery after the load drops.
Gradual slowdown after 8 hours	**Endurance / soak test** (long load test)	Run a realistic, normal load for longer than 8 h to expose memory leaks, unclosed DB connections, growing logs/caches and thread exhaustion; response time and memory must stay flat over time.

(ii) JMeter configuration (Thread Group)

In JMeter a *Thread Group* defines the virtual users: **Threads** = number of virtual users; **Ramp-Up** = seconds taken to start all threads; **Loop Count** = how many times each thread repeats the scenario (*Infinite* when a duration is used); **Duration** (scheduler / thread lifetime) = how long the test runs. Each thread executes the scenario: login → view offered courses → register course → view registration slip, with a 2–5 s think-time timer.

Test	Threads	Ramp-Up	Loop Count	Duration
Spike	10,000	10 s	1 (each student registers once)	600 s (10 min)
Stress	15,000 (3× capacity)	1,500 s (10 new users/s)	Infinite	2,400 s (40 min)
Endurance (soak)	3,000 (normal load)	300 s	Infinite	36,000 s (10 h > 8 h)

Explanation of the choices:

- **Spike:** a very short ramp-up (10 s) reproduces the burst at the moment registration opens; loop count 1 because each student registers once; the 10-minute window also observes recovery.
- **Stress:** a slow ramp-up (10 users/s) passes the 5,000-user capacity after about 500 s and keeps climbing, so the exact breaking point and the behaviour beyond it can be read from the results; infinite loops keep every started user active.
- **Endurance:** a moderate, realistic load held for 10 hours (longer than the 8 hours after which the slowdown was reported); infinite loops with a duration limit keep the load constant.

Supporting JMeter elements: CSV Data Set Config (10,000 test student IDs/passwords), HTTP Request samplers, Response Assertions (status 200, "Registration successful"), Constant/Uniform Random Timer for think time, and listeners (Summary/Aggregate Report, response-time-over-time graph); in parallel, monitor server CPU, memory, DB connections and garbage collection.

Answer. (1) Burst of 10,000 → **spike test** (Threads 10,000, Ramp-Up 10 s, Loop 1, Duration 10 min). (2) Collapse beyond capacity → **stress test** (Threads 15,000, Ramp-Up 1,500 s, Loop Infinite, Duration 40 min) to find the breaking point and recovery. (3) Slowdown after 8 h → **endurance/soak test** (Threads 3,000, Ramp-Up 300 s, Loop Infinite, Duration 10 h) to detect leaks and resource exhaustion.

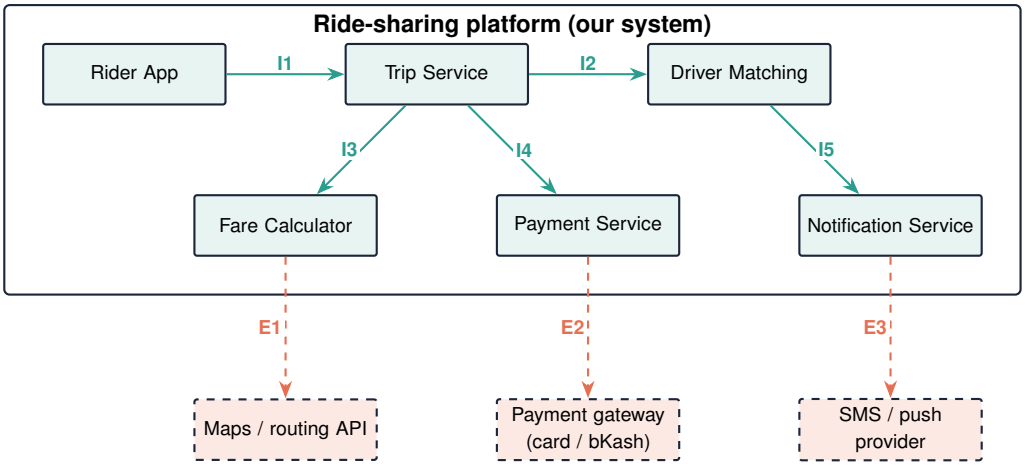
2022–2023 • Q1(c) CSE307 • 05/05/2024 • T009.2 MARKS 10

Construct an example of each of internal and external Integration testing in the context of a ride sharing application.

SOLUTION

Concepts

- **Internal integration testing** — tests the interaction between components *built by the team inside the system boundary* (our own modules/microservices talking through calls, APIs, queues or a shared database).
- **External integration testing** — tests the interaction of our system with *third-party systems outside our control* (payment gateway, maps API, SMS/push service): their contracts, data formats, time-outs, error codes and outages.



Example of internal integration testing — Trip Service ↔ Fare Calculator ↔ Payment Service (I3, I4)

ID	Scenario / input	Expected interaction result
INT-1	Trip completed: 8.4 km, 23 min, surge 1.5	Trip Service passes distance in *km* and time in *minutes*; Fare Calculator returns fare; the same fare (to the paisa) reaches Payment Service and the rider's receipt
INT-2	Rider cancels after driver assigned (I2, I4)	Matching releases the driver; Fare Calculator returns cancellation fee; Payment Service charges only that fee, trip state = CANCELLED in all services
INT-3	Driver accepts (I2, I5)	Notification Service receives driver name, car no. and ETA for the correct trip ID

Typical faults found: units mismatch (metres vs. km), rounding differences, wrong trip ID mapping, inconsistent trip state between services.

Example of external integration testing — Payment Service ↔ payment gateway (E2) and Fare ↔ Maps (E1)

ID	Scenario / input	Expected result
EXT-1	Charge 250.00 BDT with a valid sandbox card	Gateway returns SUCCESS + transaction ID; our system stores the ID and marks trip PAID
EXT-2	Gateway returns "insufficient funds"	Trip marked UNPAID, rider asked for another method, no duplicate charge
EXT-3	Gateway time-out (no reply in 10 s), then retry	Retry uses the same idempotency key; rider is charged exactly once
EXT-4	Maps API returns route for pickup/drop coordinates; or HTTP 503	Distance/ETA parsed correctly; on 503 a fallback (straight-line estimate / cached route) is used and an error is logged

These are run against the providers' sandbox environments; contract tests check that request/response formats still match the providers' documented API.

Answer. Internal integration: our own components interacting, e.g. Trip Service → Fare Calculator → Payment Service must pass consistent distance/fare/trip ID. External integration: our system with third-party services, e.g. Payment Service with the payment gateway (success, decline, time-out without double charge) or Fare Calculator with the Maps API.

2022–2023 • Q2(c) CSE325 • 30/01/2025 • C325-2223-2c **MARKS 9**

Write down functional test cases for the Discount management scenario discussed in Q1(a).

Q1(a) scenario (relevant part): There are different types of discounts for member of Bangla academy, fairs, bulk purchase, etc. which vary over time. The management wants to have real time picture of stock and sell along with financial transactions to ensure transparency.

SOLUTION

Step 1 – What a functional test case is

A **functional (black-box) test case** checks one requirement through the system's interface: it states the *pre-condition*, the *input / steps* and the *expected result*, without looking at the code. Good test sets cover each equivalence class (valid and invalid) and the boundaries – here the boundaries are the *bulk-quantity threshold* and the *validity dates* of each discount ("vary over time").

Step 2 – Requirements assumed for the discount module

- R1 Each discount has a type (*member*, *fair*, *bulk*), a rate (0–100%), and a validity period [from, to]; the management can change them.
- R2 Member discount applies only to a member with a valid (non-expired) membership.
- R3 Bulk discount applies when the quantity in one order $\geq$ the bulk threshold (say 10 copies).
- R4 Fair discount applies only to sales made at a fair stall during that fair's dates.
- R5 If several discounts apply, only the *highest* one is given (no stacking).
- R6 Every sale is posted as a financial transaction with list price, discount and net amount, visible on the dashboard at once.

Step 3 – Test cases

(list price of the test book = Tk 500; member 20%, bulk 15% for $\geq$ 10 copies, fair 25% from 01–28 Feb)

ID	Purpose	Input / pre-condition	Expected result
TC01	no discount	non-member buys 1 copy online	payable = Tk 500, discount = 0
TC02	member discount (R2)	valid member buys 1 copy	discount 20%, payable = Tk 400
TC03	expired membership (R2)	membership expired yesterday, buys 1 copy	no member discount, payable = Tk 500
TC04	bulk boundary below (R3)	non-member buys 9 copies	no bulk discount, payable = Tk 4500
TC05	bulk boundary on (R3)	non-member buys 10 copies	15% bulk discount, payable = Tk 4250
TC06	fair discount (R4)	sale at fair stall on 28 Feb (last day)	25% discount, payable = Tk 375
TC07	fair period expired (R4)	sale at same stall on 1 Mar	fair discount not applied
TC08	fair discount not online (R4)	online order on 10 Feb (during fair)	fair discount not applied
TC09	no stacking (R5)	valid member buys 10 copies	only 20% (highest) applied, payable = Tk 4000
TC10	rate changes over time (R1)	manager changes member rate 20% → 25% effective 1 Apr; sales on 31 Mar and 1 Apr	31 Mar sale uses 20%, 1 Apr sale uses 25%
TC11	invalid rate (R1)	manager enters rate −5% or 120%	rejected with error, old rate unchanged
TC12	invalid period (R1)	valid-to date earlier than valid-from	rejected with error
TC13	invalid quantity	order quantity 0 or negative	order rejected, no transaction posted
TC14	transparency (R6)	complete TC02	ledger has list 500, discount 100, net 400; dashboard sales and stock updated immediately

Answer. Functional test cases cover each discount type (member, bulk, fair), their boundaries (quantity 9/10, first/last/after fair day, membership expiry), the time-varying rate (change effective date), invalid configuration and inputs (negative / >100% rate, inverted period, zero quantity), the no-stacking rule, and the posting of the sale to the financial ledger and dashboard.

NOTE The paper does not define the discount rules; the rates, the bulk threshold and the "highest discount only" policy are assumptions – the test design (equivalence classes + boundaries + invalid inputs + time changes) stays the same for other rules.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q7(d) CSE325 • 30/01/2025 • C325-2223-7d **MARKS 4**

What do you understand by Regression Testing and Smoke Testing?

2020–2021 • Q8(c) CSE307 • 19/10/2022 • T016 **MARKS 8**

What is the difference between Smoke and Regression testing? Discuss with an example.

SOLUTION

Definitions

- **Regression testing** — "*selective retesting* of a system or component to verify that modifications have not caused unintended effects and that the system or component still complies with its specified requirements." It re-runs a *subset of the original test cases*: a representative sample that exercises all existing functions, plus extra tests for the parts most likely affected by the change. It is run after every significant change (bug fix, new feature, refactoring) and is usually automated.
- **Smoke testing** — "a subset of the regression test cases that establish that the system is stable and all major functionality is present and works under 'normal' conditions". It is a quick health check of a new build that decides whether deeper testing is worthwhile.

Aspect	Smoke testing	Regression testing
Purpose	Is the build stable enough to test further?	Did the change break anything that worked before?
Scope	Few, wide-but-shallow tests of major functions	Broad + deep: representative core + change-affected areas
Conditions	"Normal" inputs only (happy path)	Normal, boundary and negative cases
When	On every new build, before other testing	After every bug fix / feature / refactoring
Duration	Minutes	Hours (usually automated)
On failure	Build rejected, returned to developers	Defect reported for the specific broken feature
Relationship	Subset of regression suite	Superset

Example: an online shop — the discount module was changed

Smoke suite (run first, ≈5 min): application starts; home page loads; user can log in; search returns a product; an item can be added to the cart; checkout page opens. If any of these fails, the build is rejected immediately.
Regression suite (run next): all smoke tests *plus* existing cart-total tests, coupon and discount boundary tests (e.g. 10% for orders $\geq$ 1000 Tk: 999, 1000, 1001), VAT, payment, invoice, refund and order-history tests — because the discount change could silently corrupt any of these.

Answer. Smoke testing is a small, quick, happy-path subset of the regression tests that checks that a new build is stable; regression testing is the broader selective re-test that checks that a change did not break existing functionality.

2022–2023 • Q8(b) CSE325 • 30/01/2025 • C325-2223-8b MARKS 8 + 7

Consider two systems: The first one is a Flight Reservation System (FRS) that consists of modules for managing flight schedules, passenger bookings, payment processing, and notifications for booking confirmations and schedule changes. The second one is a Core Banking System (CBS) that includes modules for processing transactions (deposits, withdrawals, transfers), managing financial ledgers, maintaining customer account data, and providing a user interface for customer interactions.

(i) For each system, determine whether Top-Down or Bottom-Up integration testing is more suitable. Justify your choice.

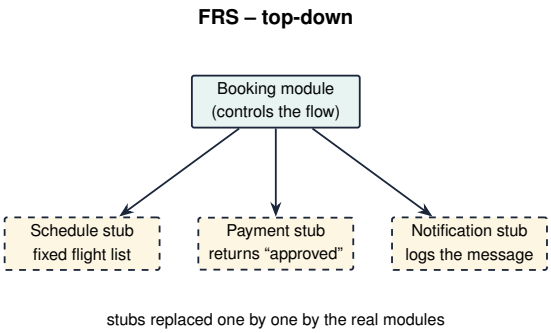
(ii) Suggest where drivers or stubs might be needed and describe their roles in facilitating testing.

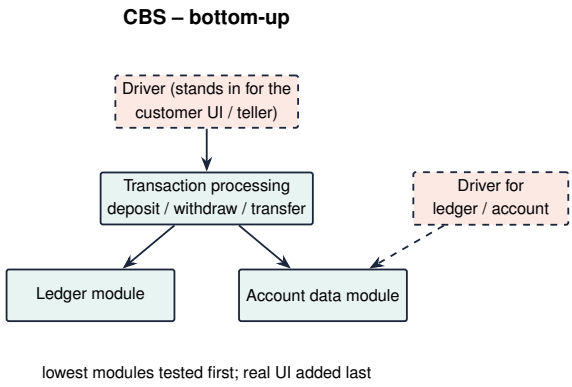
SOLUTION

Background

Integration testing combines units and tests their interaction. In **top-down** integration the top (controlling, user-facing) module is tested first and lower modules that are not ready are replaced by **stubs** – dummy *called* modules returning canned answers. In **bottom-up** integration the lowest modules are tested first; since their callers do not exist yet, **drivers** – dummy *calling* programs that feed inputs and check outputs – are used.

(i) Choice of strategy





FRS ⇒ top-down.

- The system is *workflow-driven*: the booking flow (search flight → book seat → pay → confirm) controls the other modules, so testing from the top validates the main user journey early and gives an early working skeleton/demo to the airline.
- Lower modules depend on *external* services (payment gateway, SMS/e-mail servers) that are often unavailable or costly in test; they are easy to replace with stubs.
- Design errors in the control logic (e.g. seat held but payment failed) are found early.

CBS ⇒ bottom-up.

- The critical, complex logic is at the *bottom*: posting to the ledger, keeping balances and account data consistent. These must be rock-solid (money, regulatory audit) before anything is built on top of them.
- Transactions (deposit/withdraw/transfer) are compositions of ledger and account operations; testing the base first means every higher level is integrated on a verified foundation.
- The user interface is comparatively thin and changes often; it is added last.

(ii) Where stubs and drivers are needed

System	Test double	Role
FRS	**Payment stub**	returns *approved / declined / timeout* on demand so the booking module's handling of each outcome can be tested without a real gateway or real money
FRS	**Notification stub**	records the confirmation / schedule-change message instead of sending SMS or e-mail; the test checks that the right message was produced
FRS	**Schedule stub**	returns a fixed list of flights and seat availability (e.g. a full flight) so booking rules can be tested deterministically
CBS	**Driver for ledger / account modules**	calls `post(debit, credit)`, `getBalance()` with prepared inputs and compares the results (double-entry balanced, balance correct)
CBS	**Driver for transaction processing**	simulates the UI / teller: issues deposits, withdrawals and transfers (including insufficient funds, concurrent transfers) and checks ledger and account states

Thus stubs *simulate the called (lower) modules* that are missing in top-down testing, and drivers *simulate the calling (higher) modules* that are missing in bottom-up testing; both let a module be tested in isolation and make the tests repeatable.

Answer. (i) FRS: top-down – the booking workflow is the controlling, user-visible part and the lower services (payment, notification) are external and easily stubbed. CBS: bottom-up – correctness of the ledger and account data is the critical foundation, so it is verified first and the UI is integrated last. (ii) FRS uses stubs for payment, notification (and schedule) modules; CBS uses drivers that play the role of the UI/transaction layer to call the ledger and account modules, and later the transaction module.

NOTE Related card in this chapter: T028.5 (integration strategies with stubs and drivers). A mixed (*sandwich*) strategy would also be acceptable if justified.

2021–2022 • Q3(b), (c) CSE307 • 07/05/2023 • T011 **MARKS 10 + 5**

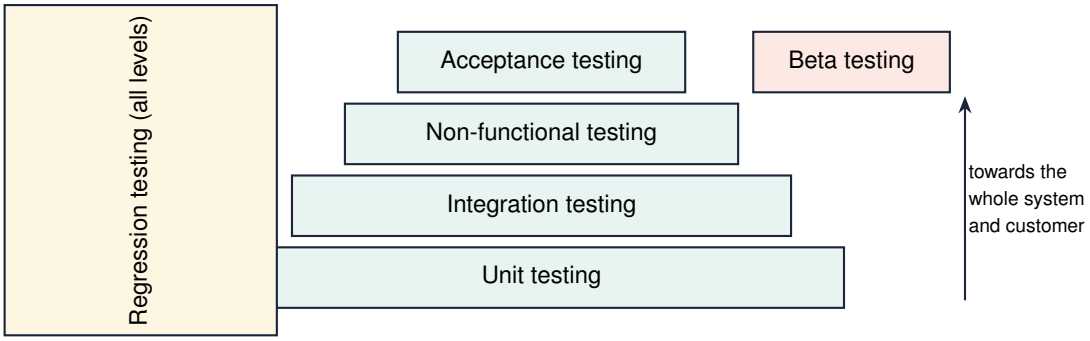
(b) Briefly discuss the six (6) types of testing methods as follows: *Unit testing, Integration testing, Non-Functional testing, Regression testing, Acceptance testing* and *Beta testing*. **(10)**

(c) "It incurs less cost to fix a bug, if we found the bug in the later stages of SDLC" — Do you agree with this statement? Justify your answer. **(5)**

SOLUTION

(b) The six types of testing

The lecture arranges the six types as a **testing hierarchy**: testing grows from small internal pieces to the whole system judged by the customer; regression testing cuts across all levels and beta testing happens near release.



(i) **Unit testing** — testing of individual software units (a method, a class) or groups of related units. Done by the programmer, generally *white box*, verifies that the code does what it is intended to do at a very low structural level; automation (JUnit) is desirable for repeatability. *Example:* test `calculateFare(distance)` alone.

(ii) **Integration testing** — software and/or hardware components are combined and tested to evaluate the *interaction* between them. Done by programmers as they integrate code into the code base; generally white box, maybe some black box. "Just because the components work individually does not mean they all work together." *Example:* registration module sends correct student data to the result module through an API.

(iii) **Non-functional testing** — conducted on the complete, integrated system to evaluate compliance with its quality requirements: *performance/load* (meets specified performance), *stress* (beyond the specified limits), *usability* (ease of learning and use). Mostly black box, preferably by an external test group.

(iv) **Acceptance testing** — formal testing to determine whether the system satisfies its *acceptance criteria* and to let the customer decide whether to accept it. Done by the customer (or representative) in their environment through the GUI; definitely black box.

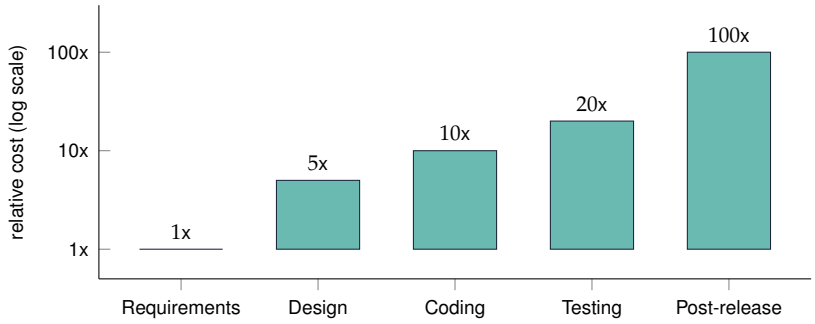
(v) **Regression testing** — selective re-testing after a modification to verify that the change has not caused unintended effects and that the system still meets its requirements. Uses a subset of the original tests: a representative core that exercises all existing functionality plus tests most likely to be affected by the change. A *smoke test* is a small subset of it.

(vi) **Beta testing** — an advance partial or full version is given free to potential users, who use it as they wish and report errors. *Pros:* unexpected errors found, low cost, wide population/environments. *Cons:* not systematic, low-quality error reports.

Type	What is tested	Who	Box
Unit	a single unit in isolation	programmer	white
Integration	interfaces / interaction of units	programmer	mostly white
Non-functional	performance, stress, usability of whole system	external test group	mostly black
Acceptance	acceptance criteria of customer	customer	black
Regression	unchanged behaviour after a change	dev / QA (automated)	both
Beta	real-world use of near-final version	potential users	black

(c) Is a bug cheaper to fix when found late? — No, I disagree

The statement is **false**; the opposite is true. The lecture says software should be tested "throughout the whole development process" and the test plan written early precisely because the *cost of a defect grows with the time it stays undetected*.



Typical (illustrative) relative cost of fixing the same defect by the phase in which it is found.

Justification:

- A requirement defect found during requirement review costs one sentence to fix. Found after coding, the design diagrams, code of several modules, test cases and documentation must all change.
- The later the discovery, the more artefacts depend on the faulty decision (*ripple effect*): integration, data migration, retesting and *regression testing* of everything touched.
- After release the bug becomes a *failure* for users: emergency patches, downtime, support, data repair, lost reputation or even legal/safety consequences.
- Late in the cycle the team is under release pressure — "a very chaotic, hurried environment" — so fixes are rushed and may introduce new faults.

Answer. (b) Unit (single unit, white box, programmer), integration (interaction of combined units), non-functional (performance/load, stress, usability of the whole system), acceptance (customer checks acceptance criteria, black box), regression (re-test after change; smoke test is its subset), beta (real users report errors). (c) Disagree: the cost of fixing a defect rises steeply the later it is found, so test early and throughout the SDLC.

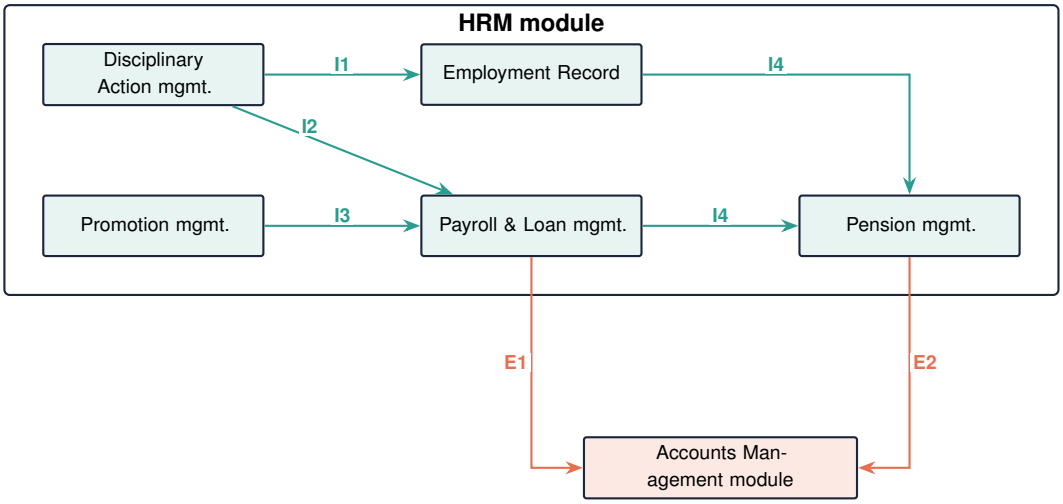
Jan 2021 • Q5(b) CSE307 • 04/09/2021 • T017 **MARKS 15**

In the ERP application developed for an organization, there is a Human Resource Management (HRM) module. It has components of Employment Record, Promotion management, Payroll and Loan management, Disciplinary Action management, and Pension management. All these components need to interact with the Accounts Management module. If there is any allegation against an employee, the employee is suspended and disciplinary investigation is carried out. During suspension, the employee gets half of his/her salary. After completion of the investigation, if s/he is proved innocent, the reduced salary is paid back.
Mention the requirements of 4 inter-component integrations of HRM module and 2 inter-module integrations between HRM and Accounts module that you will need to consider for Integration Testing.

SOLUTION

Approach

Integration testing checks that components which work individually also work *together*: data passed across each interface must be complete, correctly formatted, consistent and timely, and the business rules that span components must hold. First identify the interfaces:



Four inter-component integrations inside HRM

ID	Components	Integration requirement to test
I1	Disciplinary Action → Employment Record	When an allegation is filed and the employee is suspended, the employment status becomes SUSPENDED with start date and case ID; when the investigation ends, the status changes to ACTIVE (innocent) or to the penalty outcome (guilty). Both components must show the same status at all times.
I2	Disciplinary Action → Payroll & Loan	From the suspension date payroll pays exactly **50%** of the salary (pro-rata for a partial month). If proved innocent, payroll computes and pays back the withheld amount (arrears) for the whole suspension period exactly once; if guilty, no arrears. Loan instalments must still be deducted correctly from the reduced salary (or deferred by rule).
I3	Promotion → Payroll & Loan (via Employment Record)	A promotion updates designation/grade in the Employment Record and the new pay scale is used by payroll from the effective date (with arrears if back-dated). A *suspended* employee must not be promoted (Promotion must query the status set by I1); a promotion held back during suspension is restored if found innocent.
I4	Employment Record + Payroll & Loan → Pension	On retirement, Pension receives the length of service (from Employment Record) and the last basic pay (from Payroll); the suspension period counts as service if the employee was proved innocent. Any outstanding loan balance is passed to Pension so that it is adjusted against the gratuity.

Two inter-module integrations between HRM and Accounts

ID	Interface	Integration requirement to test
E1	Payroll & Loan → Accounts	The monthly salary sheet (including half salary of suspended employees), loan disbursements and instalment deductions are posted to Accounts as correct journal entries (salary expense, bank payment, loan receivable). Totals in HRM and in the Accounts ledger must match to the taka; the arrears paid after an acquittal are posted once, in the correct accounting period, with a reference to the disciplinary case.
E2	Pension → Accounts	Pension/gratuity amounts computed in HRM create payment vouchers in Accounts and reduce the pension fund liability; the loan balance adjusted against gratuity closes the loan receivable. Accounts sends back the payment status (paid / failed) which HRM records against the pensioner.

Sample integration test case (I2 + E1)

Employee E-101, basic salary 60,000 Tk, suspended from 1 March to 31 May, proved innocent on 31 May. *Expected:* March–May payroll pays 30,000 Tk each; Accounts shows three salary entries of 30,000 Tk; June payroll pays 60,000 + 90,000 Tk arrears, posted once in June with case reference; Employment Record status ACTIVE from 1 June.

Answer. Inside HRM: (I1) Disciplinary → Employment Record status sync; (I2) Disciplinary → Payroll half salary and one-time arrears on acquittal; (I3) Promotion → Payroll new scale and no promotion while suspended; (I4) Employment Record/Payroll → Pension service length, last pay and loan balance. HRM–Accounts: (E1) payroll and loan postings match the ledger; (E2) pension/gratuity vouchers and payment status.

Jan 2021 • Q8(b) CSE307 • 04/09/2021 • T019 MARKS 15

What is the difference between Load and Stress testing? Discuss in the context of BIIS. Is code-review a Black-box testing? Justify your answer.

SOLUTION

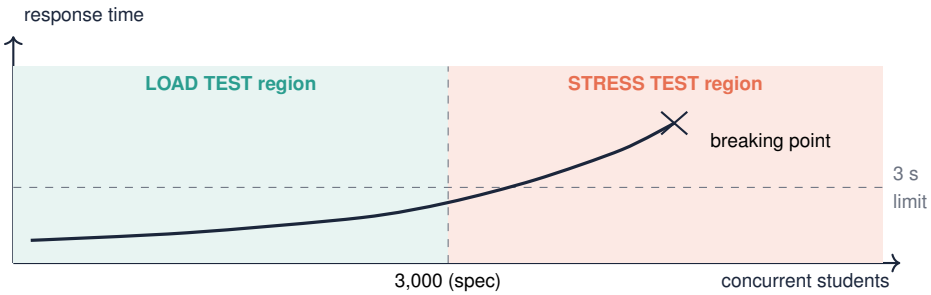
Load vs. stress testing

Both are **non-functional tests** conducted on the complete, integrated system, mostly black box and preferably by an external test group.

- **Load (performance) testing** — evaluates the compliance of the system with its *specified* performance requirements under the expected (peak) workload.
- **Stress testing** — evaluates the system *beyond the limits of its specification* to find where and how it breaks, and whether it recovers.

In the context of BIIS (BUET Institutional Information System)

Assume the requirement: *"During course registration BIIS shall support 3,000 concurrent students with page response* $\leq 3\,s$ *and error rate* $< 1\%$*."*



Aspect	Load testing of BIIS	Stress testing of BIIS
Question	Does BIIS meet its stated performance at the expected peak?	Where does BIIS break and how does it behave/recover?
Workload	Ramp to 3,000 virtual students doing login → view offered courses → register → view schedule, hold for 1 h	4,500, 6,000, 9,000 students; sudden spike when results are published at 6 pm; DB server CPU throttled
Measured	response time, throughput (registrations/min), error rate, CPU/memory/DB usage	breaking point, failure mode, data integrity, error messages, recovery time
Pass criterion	95% of pages ≤ 3 s, errors $< 1\%$	no crash/corruption: no half-saved registrations or duplicate enrolments, extra users get "try later", service recovers automatically
Outcome	confirms the requirement (or not)	reveals capacity limits, guides scaling and degradation design

Is code review a black-box testing? — No

- **Black-box testing** "ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs"; the tester needs *no knowledge of the code*.
- In a **code review** reviewers *read the source code* — logic, conditions, loops, data structures, naming, standards, duplicated code, security flaws. It is therefore impossible without knowledge of the internals; it is the opposite of black box.
- Moreover, code review does not *execute* the program with inputs and compare outputs at all; it is a *static* verification technique (in the lecture it is listed under fault avoidance: "reviews, inspections, walkthroughs"). If it must be placed on the black/white scale it is closest to *white-box* (static white-box) verification: "are we building the product right?".

Answer. Load testing checks that BIIS meets its specified performance at the expected peak (e.g. 3,000 concurrent students, $\leq$3 s); stress testing pushes BIIS beyond that (4,500+ users, spikes) to find the breaking point, failure mode and recovery. Code review is *not* black-box testing: reviewers examine the source code (internal knowledge) and do not execute it — it is static, white-box style verification.

2018–2019 • Q4(a) CSE307 • 06/11/2019 • T022 **MARKS 9**

Discuss Integration, Beta, and Usability testing in brief.

SOLUTION

Integration testing

Integration testing is testing in which software components, hardware components, or both are combined and tested to evaluate the *interaction* between them.

- Done by programmers as they integrate their code into the code base; generally white box with some black-box end-to-end checks; automation desirable.
- Rationale: units that pass their own tests may still fail together because of mismatched data formats, parameter order, units of measure, protocols, timing or error handling.
- Strategies: big-bang, top-down (with *stubs*), bottom-up (with *drivers*), sandwich.
- *Example:* in a university system the registration module calls the result module's API; tests check that IDs and field names match, missing data and API failures are handled.

Beta testing

The organisation offers an advance partial or full version of the software, free, to potential users, who use it as they wish and report any errors back.

- *Advantages:* identifies unexpected errors, low cost, wide population of users, devices and environments.
- *Disadvantages:* lack of systematic testing; low-quality error reports.
- *Example:* a ride-sharing app released to 2,000 volunteers in one city before the nation-wide launch.

Usability testing

A non-functional test that evaluates the extent to which a user can *learn to operate, prepare inputs for, and interpret outputs* of the system.

- Performed with representative (often first-time) users doing realistic tasks, usually observed by an external test group; black box.
- Measures task-completion rate, time per task, number of errors/wrong clicks, requests for help and user satisfaction.
- *Example:* can a new student add a course in BIIS within 2 minutes without help, and does the error message explain a time clash clearly?

Answer. Integration testing checks the interaction of combined components; beta testing gives a pre-release version to real users who report errors (cheap and broad but unsystematic); usability testing measures how easily users learn, operate and interpret the system.

2017–2018 • Q7(a) CSE307 • 29/09/2018 • T024 MARKS 6

The requirement specification of a web-based software application stated that maximum 100 users may access simultaneously. After deployment, the application failed to response properly when 90 users accessed at a time. It was recovered later and then on a certain occasion 150 users tried to access and again it crashed. Which two software testing methods could prevent those two failures respectively?

SOLUTION

Failure 1 — 90 users (within the specification) ⇒ Load (performance) testing

Performance/load testing evaluates the compliance of a system with its *specified* performance requirements. The specification promises 100 simultaneous users, so 90 users is a normal, expected load. A load test that simulated 100 (and therefore also 90) concurrent virtual users before release, and checked response time and error rate, would have exposed the slow/incorrect responses (e.g. too small connection pool, slow queries) so they could be fixed before deployment.

Failure 2 — 150 users (beyond the specification) ⇒ Stress testing

Stress testing evaluates a system *beyond the limits of its specification*. 150 users exceeds the promised 100. The system is not required to serve them all fast, but it must not *crash*: a stress test that pushed the load to 120, 150, 200 users would have revealed the crash and the breaking point, so that graceful degradation could be built in (queue or reject extra users with "server busy, try later", protect data, recover automatically).

	Load test	**Stress test**
Load applied	up to the specified 100 users	120, 150, 200 … users (beyond spec)
Pass criterion	response time / error rate within requirement	no crash, no data loss, graceful degradation, recovery
Prevents	failure at 90 users	crash at 150 users

Answer. The failure at 90 users (inside the 100-user specification) would be prevented by **load/performance testing**; the crash at 150 users (beyond the specification) would be prevented by **stress testing**.

2017–2018 • Q8(a) CSE307 • 29/09/2018 • T025 MARKS 4 + 8

What are the advantages and disadvantages of black-box testing? Discuss performance and regression testing in brief.

SOLUTION

Black-box testing (4 marks)

Black-box testing ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs and execution conditions — the interface is visible, the internals are unknown.

Advantages	Disadvantages
Not influenced by the component being tested — assumptions embodied in the code are not propagated to the test data.	**Misses implementation bugs not covered by the specification**, e.g. control-flow details.
Robust to implementation changes — test data need not change when the code changes.	Cannot target **performance optimisations** hidden in the code.
Allows independent testers — testers need not be familiar with the code.	Cannot see **alternate algorithms for different cases** (e.g. a different sort for small arrays), so some paths stay untested.
Useful for validation (are we building the right software?).	Test design depends on a complete, correct specification.

Performance testing (4 marks)

A non-functional test on the complete, integrated system. **Performance/load testing** evaluates the compliance of a system with its *specified performance requirements*: response time, throughput, number of concurrent users, resource usage (CPU, memory, DB) and error rate under the expected peak load. Its companion **stress testing** evaluates the system *beyond* the limits of its specification to find the breaking point and failure behaviour.

- Mostly black box, done by an external test group, using tools such as JMeter to simulate users.
- *Example:* requirement "500 concurrent students, page response ≤ 3 s"; the load test simulates 500 virtual users and measures response time and errors.

Regression testing (4 marks)

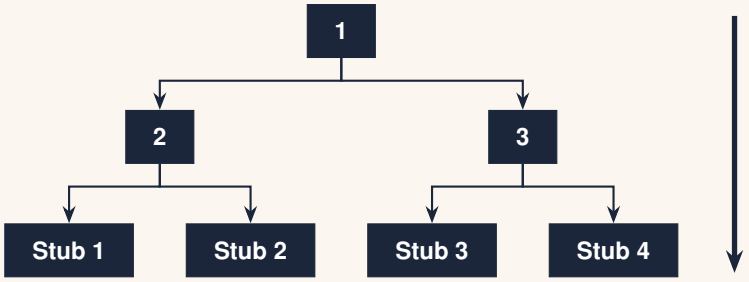
Regression testing is selective re-testing of a system or component to verify that modifications have not caused unintended effects and that it still complies with its specified requirements.

- Uses a *subset* of the original test cases: a core group re-run after any significant change — a representative sample exercising all existing functionality plus tests most likely to be affected by the change.
- Should be automated (JUnit suites, CI pipeline) because it is run very often.
- The *smoke test* is the smallest regression subset that shows the build is stable.
- *Example:* after changing the VAT calculation in a shop, re-run cart total, discount, payment, invoice and refund tests.

Answer. Black box: independent of code, robust to code changes, allows independent testers, good for validation; but misses implementation-specific bugs (control flow, optimisations, alternate algorithms). Performance testing checks specified speed/capacity requirements under expected load; regression testing re-runs a selected subset of tests after a change to catch unintended side effects.

2017–2018 • Q5(a) CSE325 • 10/04/2019 • C325-1718-5a MARKS 4

The following diagram states that Modules 1, 2 and 3 are available for integration, whereas, below modules are still under development that cannot be integrated at this point of time. Hence, Stubs are used to test the modules. Write down the order of integration testing if BFS used.



SOLUTION

Concept

Because the integration proceeds from the top module (1) downwards and the lower-level modules are replaced by **stubs** (dummy *called* modules returning canned answers), this is **top-down incremental integration**. Top-down integration can add modules in two orders:

- **Depth-first (DFS):** integrate all modules on one major control path first (1, 2, Stub 1, Stub 2, 3, . . .).
- **Breadth-first (BFS):** integrate all modules directly subordinate at each level, moving across the structure level by level.

BFS order

Level 0: 1 → level 1: 2, 3 → level 2: Stub 1, Stub 2, Stub 3, Stub 4.

Step	Module added	Configuration under test
1	1	module 1 (main control module)
2	2	1 – 2
3	3	1 – 2 – 3 (level 1 complete)
4	Stub 1	1, 2, 3 + Stub 1
5	Stub 2	1, 2, 3 + Stub 1, Stub 2
6	Stub 3	1, 2, 3 + Stub 1, 2, 3
7	Stub 4	complete structure: 1, 2, 3 + Stub 1–4

After each addition the tests are re-run (regression testing) so that a new fault can be traced to the module just added. When the real lower-level modules become available, each stub is replaced by the corresponding real module in the same order.

Answer. BFS (top-down, breadth-first) integration order: 1 → 2 → 3 → Stub 1 → Stub 2 → Stub 3 → Stub 4. (For comparison, DFS would be 1, 2, Stub 1, Stub 2, 3, Stub 3, Stub 4.)

2014–2015 • Q2(a) CSE307 • 04/02/2016 • T028.2 MARKS 6 $\frac{2}{3}$

Demonstrate an example showing that exhaustive testing is nearly impossible.

SOLUTION

Concept

Exhaustive testing means running the program with *every* possible input (and every possible

execution path). The lecture notes that "there is no way to guarantee that all faults have been detected" and that for white-box testing it is "impossible to thoroughly exercise all paths — exhaustive testing grows without bound". Two small examples show why.

Example 1 — exhaustive *input* testing

```
int add(int a, int b) { return a + b; }   // two 32-bit int parameters
```

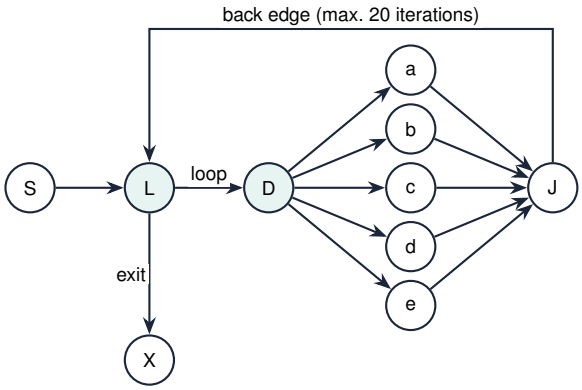
Each `int` has 2^{32} values, so the input space is

$$2^{32} \times 2^{32} = 2^{64} \approx 1.8 \times 10^{19} \text{ test cases.}$$

Even at an (optimistic) 10^9 tests per second: $\frac{1.8 \times 10^{19}}{10^9} = 1.8 \times 10^{10}$ s $\approx$ **585 years** — for a one-line method.

Example 2 — exhaustive *path* testing

A loop may execute up to 20 times; its body contains nested `if`s giving 5 distinct paths per iteration.



Number of distinct paths $= 5^1 + 5^2 + \cdots + 5^{20} \approx 5^{20} \approx 9.5 \times 10^{13}$. At one test per millisecond: 9.5×10^{13} ms $= 9.5 \times 10^{10}$ s $\approx$ **3,000 years**.

Consequence

Testing must *sample* the input/path space intelligently: equivalence partitioning (one value per partition), boundary value analysis, and coverage criteria (statement, branch, condition) instead of exhaustive enumeration — "write as few test cases as possible to cause failures".

Answer. A two-`int` method already has $2^{64} \approx 1.8 \times 10^{19}$ inputs ($\approx$585 years at 10^9 tests/s); a loop of ≤ 20 iterations with 5 paths per iteration has $\approx 10^{14}$ paths ($\approx$3000 years at 1 test/ms). Hence exhaustive testing is practically impossible.

2014–2015 • Q2(e) CSE307 • 04/02/2016 • T028.5 **MARKS 8**

Explain different types of integration testing strategies.

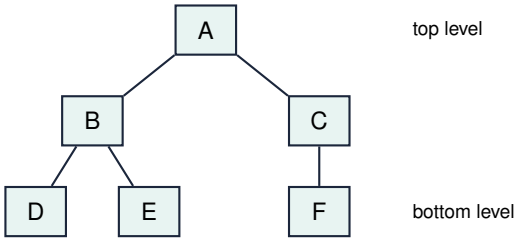
SOLUTION

Background

Integration testing combines components and tests the *interaction* between them. The *strategy* decides in which order the units are combined and what temporary code is needed for units that are not yet integrated:

- **Stub** — a dummy *called* module that replaces a lower-level unit; it returns canned answers so that the caller can be tested.
- **Driver** — a dummy *calling* module (test harness) that invokes a lower-level unit, passes test data and checks the result, replacing a missing higher-level unit.

The strategies are illustrated on the small hierarchy below (A calls B and C, B calls D and E, C calls F).



1. Big-bang integration (non-incremental)

All units are unit-tested separately and then combined *at once*: test {A,B,C,D,E,F} together.
Pro: no stubs/drivers; fine for very small systems. *Con:* when a test fails it is very hard to locate the faulty interface; integration starts late; many interface faults surface together.

2. Top-down integration (incremental)

Start with the top (main control) module, which is driven by the test harness; all its subordinates are replaced by *stubs*. Stubs are then replaced one at a time by real modules, either

- **depth-first**: A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F (one complete control path first), or
- **breadth-first**: A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ F (level by level).

After each replacement the tests (and regression tests) are re-run.
Pro: major control/decision logic and a skeleton working system appear early; easy to demonstrate. *Con:* many stubs to write; low-level (often I/O, data) modules are tested late; stubs may be too simple to test realistic data.

3. Bottom-up integration (incremental)

Start with the lowest-level (atomic) modules, grouped into clusters, each exercised by a *driver*; move upwards replacing drivers by real modules: test D, E with driver d_B and F with driver d_C; then B+D+E and C+F with a driver for A; finally A+B+C+D+E+F.
Pro: no stubs; low-level utilities are tested thoroughly and early; fault localisation is easy. *Con:* drivers needed; there is no working "program as a whole" until the last module is added; top-level design flaws are discovered late.

4. Sandwich (hybrid) integration

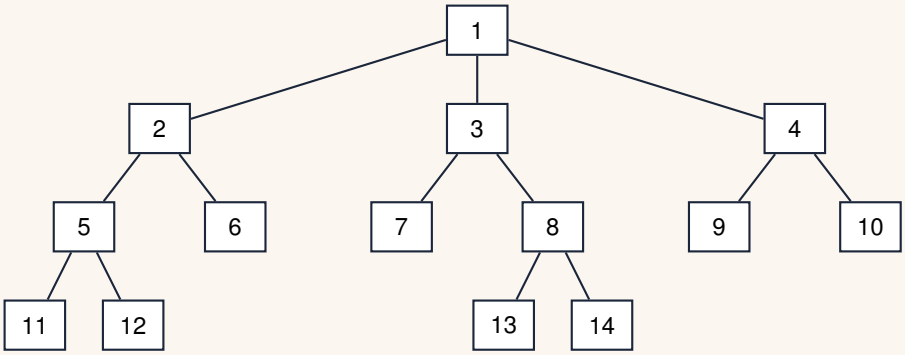
Top layer (A) is integrated top-down with stubs while the bottom layer (D, E, F) is integrated bottom-up with drivers, in parallel; the middle layer (B, C) is where the two meet.
Pro: combines early skeleton with early tested utilities; parallel work. *Con:* middle layer is tested less individually; needs both stubs and drivers.

Strategy	Stubs	Drivers	Main strength / weakness
Big-bang	no	no	simple / fault localisation very hard
Top-down	yes	(only top harness)	early skeleton system / many stubs, low levels late
Bottom-up	no	yes	easy localisation, utilities early / no full system until end
Sandwich	yes	yes	parallel, best of both / more scaffolding, middle tested late

Answer. Integration strategies: big-bang (all at once), incremental top-down (depth- or breadth-first, uses stubs), incremental bottom-up (clusters with drivers) and sandwich/hybrid (top-down for upper layers + bottom-up for lower layers). Incremental strategies make interface faults easy to localise.

2013–2014 • Q3(a), (b) CSE307 • 08/08/2015 • T029.1 MARKS 10 + 8

(a) Consider the following call and return architecture of a subsystem:



Show the sequence of tests done in top down integration mentioning the stubs and drivers. **(10)**
(b) Explain the debugging process with a diagram describing its process. **(8)**

SOLUTION

(a) Top-down integration — idea

In **top-down integration** the main control module (1) is tested first; the test harness acts as the only **driver**, and every subordinate that is not yet integrated is replaced by a **stub** (written S_k for module k) that returns canned values. Stubs are replaced by the real modules one at a time; after each replacement the new tests and the earlier tests (regression) are run. Below the **breadth-first** order is used (here it coincides with the module numbering).

Sequence of tests (breadth-first)

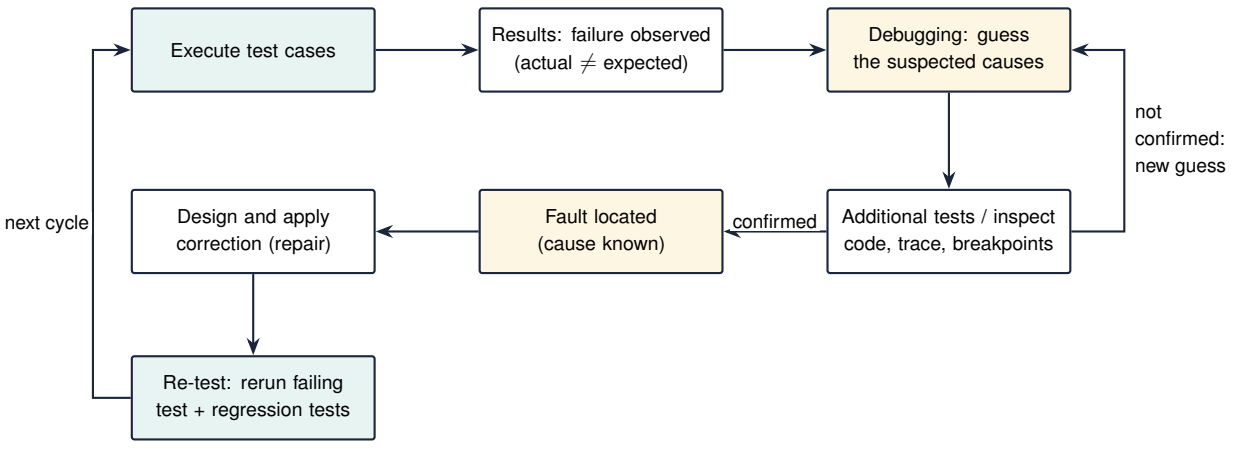
Test	Integrate	Real modules under test	Driver	Stubs in place
T1	1	1	harness	S_2, S_3, S_4
T2	2	1, 2	harness	S_3, S_4, S_5, S_6
T3	3	1–3	harness	S_4, S_5, S_6, S_7, S_8
T4	4	1–4	harness	$S_5, S_6, S_7, S_8, S_9, S_{10}$
T5	5	1–5	harness	$S_6, S_7, S_8, S_9, S_{10}, S_{11}, S_{12}$
T6	6	1–6	harness	$S_7, S_8, S_9, S_{10}, S_{11}, S_{12}$
T7	7	1–7	harness	$S_8, S_9, S_{10}, S_{11}, S_{12}$
T8	8	1–8	harness	$S_9, S_{10}, S_{11}, S_{12}, S_{13}, S_{14}$
T9	9	1–9	harness	$S_{10}, S_{11}, S_{12}, S_{13}, S_{14}$
T10	10	1–10	harness	$S_{11}, S_{12}, S_{13}, S_{14}$
T11	11	1–11	harness	S_{12}, S_{13}, S_{14}
T12	12	1–12	harness	S_{13}, S_{14}
T13	13	1–13	harness	S_{14}
T14	14	1–14 (complete subsystem)	harness	none

Observations:

- **Stubs:** 13 stubs (S_2–S_{14}), one for every module except the root; each is written when its caller is integrated and thrown away when the real module replaces it. E.g. integrating module 5 (T5) removes S_5 but introduces S_{11} and S_{12}.
- **Drivers:** top-down needs no drivers for lower modules; only one driver (the test harness / main program) that calls module 1 and supplies the test inputs.
- Each test Tk re-runs the tests of T1 . . . T$(k-1)$ (regression) so that the new module did not break existing interfaces.
- **Depth-first alternative:** 1, 2, 5, 11, 12, 6, 3, 7, 8, 13, 14, 4, 9, 10 — the same stub rule applies (e.g. after integrating 1, 2, 5 the stubs are $S_{11}, S_{12}, S_6, S_3, S_4$).

(b) The debugging process

Testing reveals a *failure*; **debugging** is the process that starts from that failure, locates the responsible *fault/defect*, corrects it and confirms the correction. It is a cycle:



Steps:

(i) **Execute tests and observe the symptom** — a test fails (or a user reports a failure). Reproduce it reliably and record input, environment and actual vs. expected output.

(ii) **Hypothesise suspected causes** — from the symptom and knowledge of the code, guess where the fault may be (the symptom and the cause can be far apart).

(iii) **Locate the fault** — design *additional tests*, add trace/log statements, use breakpoints and step through, or narrow the search. Classic tactics: *brute force* (memory dumps, print traces), *backtracking* (walk backwards from the symptom), *cause elimination* (binary partitioning of input/code until the cause is isolated). If the hypothesis is not confirmed, form a new one.

(iv) **Design and apply the repair** — fix the fault (and ask: could the same mistake be elsewhere? could the fix break something?).

(v) **Re-test** — re-run the failing test to confirm the fix, then run **regression tests** because a correction can introduce new faults.

Answer. (a) Breadth-first top-down: integrate 1, 2, 3, . . . , 14 one module per step; one test-harness driver calls module 1; stubs S_2–S_{14} stand in for not-yet-integrated modules and are replaced one by one (13 stubs). (b) Debugging: execute tests → failure → hypothesise causes → extra tests to locate the fault → repair → re-test with regression tests (loop until fixed).

2012–2013 • Q4(d) CSE307 • 17/05/2014 • T032.2 **MARKS 6**

Differentiate between:

(i) Verification and Validation

(ii) Defect Testing and Debugging

SOLUTION

(i) Verification vs. Validation

The lecture defines the two activities as follows.

- **Verification** is the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the *start* of that phase — *"Are we building the product right?"*
- **Validation** is the process of evaluating a system or component during or at the end of the development process to determine whether it satisfies the *specified requirements* of the customer — *"Are we building the right product?"*

Aspect	Verification	Validation
Question asked	Building the product *right*?	Building the *right* product?
Reference point	Output of the previous phase (spec, design)	Customer requirements / real needs
Typical techniques	Reviews, inspections, walkthroughs, unit and integration testing (often white box)	Acceptance testing, beta testing, system-level black-box tests, prototypes shown to users
Who does it	Developers, QA, reviewers	QA together with customer / users
When	At the end of every phase	Mainly at the end (also during, with prototypes)
Example	Code enforces "max. 5 courses" exactly as the SRS says	University actually needed students to take 6 courses ⇒ product fails validation

(ii) Defect testing vs. Debugging

Using the lecture's terminology: a *mistake* (human action) introduces a *fault/defect* into the program; when the fault is executed it produces a *failure*.

- **Defect testing** executes the program with test cases deliberately designed to *cause failures*, i.e. to reveal that defects exist ("write test cases to cause failures").
- **Debugging** starts *after* a failure has been observed: it locates the responsible fault in the code, diagnoses why it happens, repairs it and re-tests the program.

Aspect	Defect testing	Debugging
Goal	Discover *that* a defect exists	Find *where/why* it exists and remove it
Input	Specification / code, test cases	A failing test case or bug report
Output	Test report: pass / fail (failure observed)	Corrected code + regression re-test
Nature	Planned, systematic, repeatable, can be automated, can be done by independent testers	Diagnostic, hypothesis-driven, mostly done by the developer who knows the code
Order	Comes first	Follows testing; ends with re-testing

Answer. Verification checks each phase's product against its specification (product built right); validation checks the final product against the customer's needs (right product). Defect testing *reveals* failures; debugging *locates and fixes* the fault that caused a failure and then re-tests.

2011–2012 • Q6(b) CSE307 • 23/09/2013 • F026 MARKS 12

What are the different ways in which software engineers can deal with errors? What are the advantages and disadvantages of these approaches? List and discuss as many ways as you can think of.

SOLUTION

The framework from the lecture

The lecture groups software quality-control techniques into two families:

- **Fault avoidance** — prevents errors *before the system is released*: reviews, inspections, walk-

throughs, development methodologies, testing, verification.

- **Fault tolerance** — enables the system to *recover by itself* from (some classes of) errors at run time: rollbacks, redundancy, mirroring.

Between these sits **fault detection and removal** (testing → debugging), and after release the team can still **fix in the field** (maintenance/patches). Each way is discussed below.

1. Fault avoidance (prevention)

- **Disciplined development methodology** — clear requirements, design reviews, coding standards, design principles (SOLID), modelling (UML) and use of safe languages / libraries.
- **Reviews, inspections and walkthroughs** — peers read requirements, design and code looking for defects *without executing* the program (static verification).
- **Formal specification and verification** — mathematically prove that the design/code satisfies its specification (used in safety-critical systems).

Advantages: defects are removed at the cheapest point (before they propagate into later artefacts); reviews also find standards violations, poor design and maintainability problems that tests cannot see. *Disadvantages:* needs time, skilled people and discipline; can never guarantee zero defects; formal methods are expensive and hard to scale.

2. Fault detection: testing

Execute the software with test cases designed to cause failures — unit, integration, non-functional, acceptance, regression and beta testing; black box and white box.
Advantages: finds real failures in the running program; automatable and repeatable (JUnit); regression suites protect old behaviour. *Disadvantages:* "there is no way to guarantee that all faults have been detected" — exhaustive testing is impossible; writing and maintaining tests costs effort; testing only shows the *presence* of failures.

3. Fault removal: debugging

After a failure is observed, locate the fault, repair it and re-test (with regression tests).
Advantages: actually removes the cause. *Disadvantages:* time-consuming and hard to plan; a fix may introduce new faults (hence regression testing).

4. Fault tolerance (run-time recovery)

- **Exception handling and defensive programming** — validate input, catch exceptions and degrade gracefully instead of crashing.
- **Rollback / checkpoint-restart** — database transactions, restore last consistent state.
- **Redundancy and mirroring** — replicated servers, RAID, N-version programming, fail-over clusters.

Advantages: the system keeps running (or fails safely) even when residual faults or hardware failures occur — essential for banking, medical, aviation software. *Disadvantages:* extra hardware and code cost and complexity; the fault itself is *not* removed; redundant versions may share the same design fault (common-mode failure); recovery code itself may be buggy.

5. Fix after release (field reports, beta testing, patches)

Release, collect error reports from users (beta testers, crash reports) and ship patches.
Advantages: cheap way to find unexpected errors over a wide population/environment. *Disadvan-*

tages: failures hit real users (reputation, financial or safety damage); error reports are often low quality; late fixes are the most expensive.

Summary

Approach	Advantage	Disadvantage
Avoidance (methods, reviews, formal proofs)	Cheapest point to remove defects; improves design quality	Effortful; cannot reach zero defects
Detection (testing)	Finds real failures; automatable, repeatable	Cannot prove absence of faults; costly to maintain
Removal (debugging)	Removes the root cause	Unpredictable effort; fixes may add new faults
Tolerance (exceptions, rollback, redundancy)	System survives residual faults	Extra cost/complexity; fault still present
Post-release fixing (beta, patches)	Wide, cheap exposure	Users suffer failures; most expensive fixes

Answer. Deal with errors by *avoiding* them (methodology, reviews, inspections, formal verification), *detecting* them (testing), *removing* them (debugging + regression), *tolerating* them at run time (exception handling, rollback, redundancy, mirroring) and *fixing in the field* (beta reports, patches). Earlier approaches are cheaper; tolerance is the safety net for faults that inevitably remain.

2011–2012 • Q7(a) CSE307 • 23/09/2013 • T034 MARKS 10

Your company is developing software for an on-line stock-trading service. The software contains the following subsystems: a database of stockholder accounts, a web-based client through which customers place orders and request stock prices, an ordering server which processes stock transactions as requested by clients, and a networking system for communicating with an external stock price server (for up-to-the-minute stock prices).
For each of the following test cases, indicate whether it should be run during unit, integration, or system testing. If you feel a test should be run in more than one testing phase, state the phases and justify your answer.

(i) Assuming the server is up, do stockholders receive price quotes within 3 seconds even if 200 stockholders request quotes simultaneously?

(ii) Does the ordering server accept an order only if the stockholder's account has sufficient funds to cover the order?

(iii) Does the client refuse orders for stocks that do not exist?

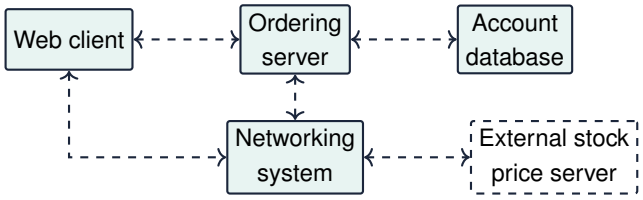
(iv) If the server receives an order to purchase stock and the stockholder has sufficient funds, is the stock eventually purchased?

(v) Once stockholder's account has been removed from the account database, is that stockholder prohibited from placing orders through the software?

SOLUTION

Deciding rule

- **Unit** — the behaviour lives in *one* subsystem/class and can be checked in isolation (other subsystems replaced by stubs/mocks).
- **Integration** — the behaviour depends on data/control passing *between* two or more subsystems.
- **System** — end-to-end or non-functional properties (performance, security) of the complete, integrated system in a realistic environment.

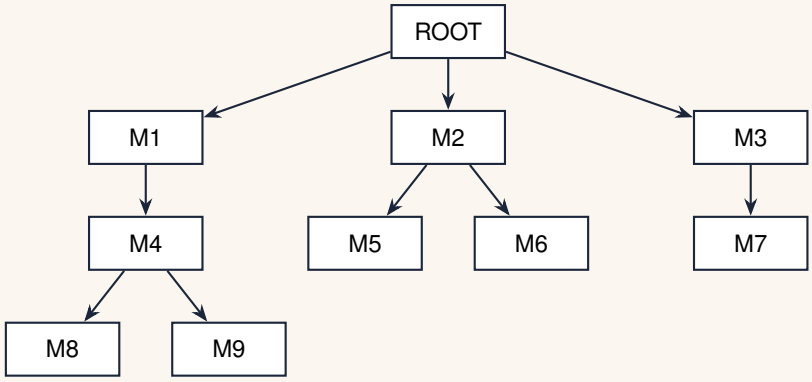


Test	Phase(s)	Justification
(i)	**System** (performance / load)	Response time with 200 simultaneous users depends on client, networking, ordering server, database and the external price server working together under load — a non-functional requirement of the complete system.
(ii)	**Unit** and **Integration**	The "sufficient funds" rule is logic inside the ordering server: unit-test it with a stubbed account repository (balance 999 vs. order 1000). Then integration-test with the real account database to verify the balance is fetched/locked correctly.
(iii)	**Unit** and **Integration**	The client's validation can be unit-tested with a stubbed list of valid symbols. Whether a stock "exists" is known only from the price server via the networking system, so the client–networking–price-server path must also be integration-tested.
(iv)	**Integration** and **System**	"Eventually purchased" requires the ordering server, account database (debit) and networking system/external market to cooperate (integration); the full customer flow from the web client to a confirmed purchase is an end-to-end system test.
(v)	**Integration** and **System**	Requires a change in the account database to be reflected in the ordering server and client (integration); as a security/access-control requirement it must also be confirmed end-to-end (system test, e.g. log in with a deleted account).

Answer. (i) System; (ii) unit + integration; (iii) unit + integration; (iv) integration + system; (v) integration + system — single-subsystem logic is unit-tested, cross-subsystem behaviour is integration-tested, and end-to-end/performance/security properties are system-tested.

2011–2012 • Q8(d) CSE307 • 23/09/2013 • T036 MARKS 9 $\frac{2}{3}$

The structure chart below illustrates the hierarchy of modules in a software sub-system:



Describe the sequence of module tests for integrating the modules using the depth first top-down approach.

SOLUTION

Approach

Top-down integration starts from ROOT, driven by the test harness; every subordinate not yet integrated is simulated by a **stub**. In the **depth-first** variant, all modules on one major control path (one branch of the structure chart, left to right) are integrated before moving to the next branch. Depth-first order:

$$\text{ROOT} \to M1 \to M4 \to M8 \to M9 \to M2 \to M5 \to M6 \to M3 \to M7.$$

Sequence of module tests

Test	Integrate	Real modules under test	Stubs in place
T1	ROOT	ROOT	S-M1, S-M2, S-M3
T2	M1	ROOT, M1	S-M4, S-M2, S-M3
T3	M4	ROOT, M1, M4	S-M8, S-M9, S-M2, S-M3
T4	M8	ROOT, M1, M4, M8	S-M9, S-M2, S-M3
T5	M9	ROOT, M1, M4, M8, M9 *(left branch done)*	S-M2, S-M3
T6	M2	+ M2	S-M5, S-M6, S-M3
T7	M5	+ M5	S-M6, S-M3
T8	M6	+ M6 *(middle branch done)*	S-M3
T9	M3	+ M3	S-M7
T10	M7	all 10 modules	none

- At every step the newly integrated module is tested through its caller, and the earlier tests are re-run as regression tests.
- 9 stubs are written in total (one per non-root module); only one driver (the harness calling ROOT) is needed.
- Depth-first gives an early, complete *vertical slice* (ROOT–M1–M4–M8/M9) that can be demonstrated, at the cost of keeping stubs for M2 and M3 longer.

Answer. Depth-first top-down order: ROOT, M1, M4, M8, M9, M2, M5, M6, M3, M7 — start with ROOT and stubs for M1–M3, then replace one stub per test (adding stubs for the newly exposed subordinates) until no stubs remain.

2010–2011 • Q6(c) CSE307 • 19/03/2012 • T037 **MARKS 2 + 5**

What is the difference between verification and validation? What input/output faults you may check while designing a banking transaction module.

SOLUTION

(a) Verification vs. validation (2 marks)

- **Verification** — evaluating the products of a development phase to determine whether they satisfy the conditions imposed at the start of that phase: *"Are we building the product right?"* (reviews, inspections, unit/integration tests).
- **Validation** — evaluating the system during or at the end of development to determine whether it satisfies the specified customer requirements: *"Are we building the right product?"* (acceptance testing, comparison against requirements).

For the banking module: verification asks whether the `transfer()` code matches the design (e.g. debit and credit in one database transaction); validation asks whether the bank's tellers and customers can actually perform the transfers they need.

(b) Input / output faults for a banking transaction module (5 marks)

A transaction module (deposit, withdraw, transfer) takes an account number, a PIN/authorisation, an amount and possibly a destination account, and produces an updated balance, a receipt and a log entry. Following the equivalence-partitioning and negative-testing ideas, the faults to check are grouped below.

Item	Faults to check	Example test input / expected result
Input faults (does the module reject bad input gracefully?)		
Account no.	wrong length / non-digits, non-existent, closed or frozen account, same source and destination	`"12AB"` → "invalid account"; frozen a/c → rejected
Amount	zero, negative, more than 2 decimal places, non-numeric, above balance, above daily limit, overflow of data type	-500, 0, `12.345`, `1e40` → rejected with clear message
Boundaries	exactly the balance, exactly the daily limit, limit $+0.01$	balance $= 5000$: withdraw 5000 ✓, 5000.01 ×
Authorisation	wrong PIN, PIN retries exceeded, expired session, missing mandatory fields	3 wrong PINs → card blocked
Output faults (is the produced result correct and consistent?)		
Balances	wrong arithmetic, rounding errors, debit done but credit missing (partial update)	sum of all balances unchanged after a transfer
Duplicates	same transaction applied twice on retry / time-out	re-submitting request ID 77 does not debit twice
Receipt / log	wrong amount, date, transaction ID or account shown; missing audit-log entry	receipt and ledger entry match the request
Messages	wrong / misleading error message, sensitive data (full PIN, card no.) displayed	message says "insufficient funds", never shows the PIN

Answer. Verification = built right against the phase specification; validation = right product for the customer. For a banking module check invalid / boundary inputs (account, amount, limits, PIN, missing fields) and output faults (wrong balance or rounding, partial or duplicate updates, wrong receipt / log, misleading or insecure messages).

2010–2011 • Q6(e) CSE307 • 19/03/2012 • T038 MARKS 15

Suppose you are writing a 'Queue' class. Design some test cases to verify your code.

SOLUTION

Step 1 — Fix the specification under test

A test case compares an *actual* result with an *expected* result, so the expected behaviour must be stated first. Assume a bounded FIFO queue:

```
public class BoundedQueue<T> {
    public BoundedQueue(int capacity) // capacity <= 0: IllegalArgumentException
    public void enqueue(T item)       // null: NullPointerException, full: IllegalStateException
    public T dequeue()                // remove+return oldest, empty: NoSuchElementException
    public T peek()                   // return oldest, keep it, empty: NoSuchElementException
    public int size()
    public boolean isEmpty()
    public boolean isFull()
}
```

Step 2 — Identify the partitions / states to cover

- **Queue state partitions:** empty, partially filled, full (capacity reached); plus the boundary transitions empty $\leftrightarrow$ 1 element and capacity$-1 \leftrightarrow$ capacity.
- **Constructor input partitions:** capacity < 0, $= 0$ (invalid), $= 1$ (boundary), normal.
- **Element partitions:** valid object, `null`.
- **Behavioural properties:** FIFO ordering, `peek` does not modify, `size` consistency, re-use after emptying, wrap-around of the internal circular array.

Each property gets a *positive* test (valid use works) and, where relevant, a *negative* test (invalid use is rejected gracefully).

Step 3 — Test cases

ID	Pre-state	Action	Expected result
TC1	–	`new BoundedQueue(3)`	`isEmpty()` true, `size()` 0
TC2	–	`new BoundedQueue(0)`, `(-1)`	IllegalArgumentException
TC3	empty	`enqueue(A)`	size 1, `peek()` = A, not empty
TC4	empty	`enqueue(A,B,C)`; dequeue $\times 3$	returns A, B, C in that order (FIFO)
TC5	[A, B]	`peek()` twice	A both times, size stays 2
TC6	empty	`dequeue()`	NoSuchElementException
TC7	empty	`peek()`	NoSuchElementException

ID	Pre-state	Action	Expected result
TC8	capacity 3, [A, B]	`enqueue(C)` (boundary)	accepted, `isFull()` true
TC9	full [A, B, C]	`enqueue(D)`	IllegalStateException; queue unchanged
TC10	empty	`enqueue(null)`	NullPointerException; size 0
TC11	[A]	`dequeue()` then `isEmpty()`	A; true ($1 \to 0$ boundary)
TC12	capacity 3	enq A,B,C; deq; enq D; deq $\times 3$	B, C, D (wrap-around works)
TC13	capacity 1	enq A; enq B	second call throws (smallest capacity)
TC14	capacity 10^5	enqueue 0..99999 then dequeue all	values in order, final size 0 (volume)

Step 4 — Automated unit tests (JUnit 5)

A *fixture* (`@BeforeEach`, JUnit 4's `@Before`) creates a fresh queue before every test so that the tests are *independent* and *repeatable*; each test checks a *single* behaviour and has a descriptive name.

```
import static org.junit.jupiter.api.Assertions.*;
import java.util.NoSuchElementException;
import org.junit.jupiter.api.*;

class BoundedQueueTest {
    private BoundedQueue<String> q;

    @BeforeEach
    void setUp() { q = new BoundedQueue<>(3); }          // fixture

    @Test
    void newQueue_isEmpty() {                              // TC1
        assertTrue(q.isEmpty());
        assertEquals(0, q.size());
    }

    @Test
    void constructor_rejectsNonPositiveCapacity() {         // TC2
        assertThrows(IllegalArgumentException.class, () -> new BoundedQueue<String>(0));
        assertThrows(IllegalArgumentException.class, () -> new BoundedQueue<String>(-1));
    }

    @Test
    void enqueue_oneItem_sizeOneAndPeekReturnsIt() {        // TC3
        q.enqueue("A");
        assertEquals(1, q.size());
        assertEquals("A", q.peek());
        assertFalse(q.isEmpty());
    }

    @Test
    void dequeue_returnsItemsInFifoOrder() {                // TC4
        q.enqueue("A"); q.enqueue("B"); q.enqueue("C");
        assertEquals("A", q.dequeue());
        assertEquals("B", q.dequeue());
        assertEquals("C", q.dequeue());
    }
```

```
    @Test
    void peek_doesNotRemove() {                              // TC5
        q.enqueue("A"); q.enqueue("B");
        assertEquals("A", q.peek());
        assertEquals("A", q.peek());
        assertEquals(2, q.size());
    }

    @Test
    void dequeueOrPeek_onEmpty_throws() {                    // TC6, TC7
        assertThrows(NoSuchElementException.class, () -> q.dequeue());
        assertThrows(NoSuchElementException.class, () -> q.peek());
    }

    @Test
    void enqueue_upToCapacity_makesQueueFull() {             // TC8
        q.enqueue("A"); q.enqueue("B"); q.enqueue("C");
        assertTrue(q.isFull());
    }

    @Test
    void enqueue_whenFull_throwsAndLeavesQueueUnchanged() {  // TC9
        q.enqueue("A"); q.enqueue("B"); q.enqueue("C");
        assertThrows(IllegalStateException.class, () -> q.enqueue("D"));
        assertEquals(3, q.size());
        assertEquals("A", q.peek());
    }

    @Test
    void enqueue_null_isRejected() {                         // TC10
        assertThrows(NullPointerException.class, () -> q.enqueue(null));
        assertEquals(0, q.size());
    }

    @Test
    void dequeue_lastItem_leavesQueueEmpty() {               // TC11
        q.enqueue("A");
        assertEquals("A", q.dequeue());
        assertTrue(q.isEmpty());
    }

    @Test
    void wrapAround_keepsFifoOrder() {                       // TC12
        q.enqueue("A"); q.enqueue("B"); q.enqueue("C");
        q.dequeue();                        // frees slot 0
        q.enqueue("D");                     // stored at slot 0 (wraps)
        assertEquals("B", q.dequeue());
        assertEquals("C", q.dequeue());
        assertEquals("D", q.dequeue());
    }

    @Test
    void capacityOne_secondEnqueueThrows() {                 // TC13
        BoundedQueue<String> one = new BoundedQueue<>(1);
        one.enqueue("A");
        assertThrows(IllegalStateException.class, () -> one.enqueue("B"));
    }
```

```
    @Test
    void largeVolume_preservesOrder() {                    // TC14
        BoundedQueue<Integer> big = new BoundedQueue<>(100_000);
        for (int i = 0; i < 100_000; i++) big.enqueue(i);
        for (int i = 0; i < 100_000; i++) assertEquals(i, big.dequeue());
        assertTrue(big.isEmpty());
    }
}
```

EXAM TIP Examiners look for: FIFO order, empty and full boundaries (underflow/overflow), invalid inputs (null, bad capacity), `peek` vs. `dequeue`, and wrap-around — each as a separate, independent, named test with clear expected results.

Answer. Specify the queue's behaviour, partition its states (empty / partial / full) and inputs (valid, null, bad capacity), then design positive and negative tests: empty on creation, FIFO order, peek without removal, underflow and overflow exceptions, null rejection, boundary transitions ($0 \leftrightarrow 1$, capacity-1 $\leftrightarrow$capacity), wrap-around and large volume — automated as independent JUnit 5 tests with a `@BeforeEach` fixture.

09

Unit Testing & Code Review

2023–2024 • Q6(a) CSE213 • 10/05/2026 • T002 **MARKS 15**

A Java method `isLeapYear(int year)` is already implemented. It returns true for leap years, false for non-leap years, and throws an `IllegalArgumentException` if year ≤ 0. Write a complete JUnit test class for this method. Your tests should cover all logically distinct input classes relevant to leap-year determination, including invalid input. Use meaningful test method names and appropriate assertions.

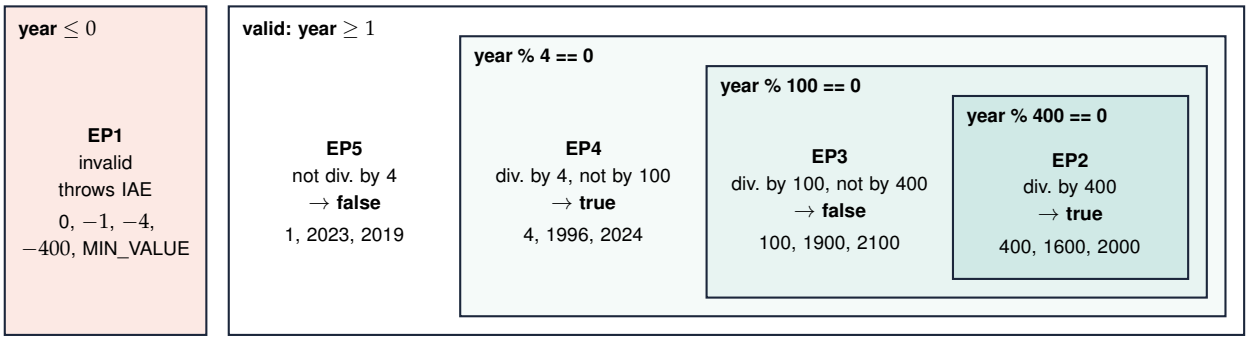
SOLUTION

Step 1 – What a unit test must do here

A **unit test** is a white-box test written by the developer to verify one small unit of functionality (here, one method). A good unit test is *automatic* (no human input), *atomic* (decides pass/fail by itself through assertions), has a *single responsibility* (one behaviour per test), is *independent* (fresh fixture for every test) and *repeatable*. So we first derive the input classes, then write one clearly named test (plus a few extra representative values) per class.

Step 2 – Logically distinct input classes (equivalence partitions)

The Gregorian rule is: a year is a leap year if it is divisible by 4, *except* century years, which are leap only when divisible by 400. Together with the invalid range this gives five classes:



Class	Condition	Representative / boundary values	Expected
EP1	year ≤ 0	0 (boundary), −1, −4, −400, `Integer.MIN_VALUE`	`IllegalArgumentException`
EP2	year % 400 = 0	2000, 400 (smallest), 1600, 2400	`true`
EP3	year % 100 = 0, year % 400 $\neq$ 0	1900, 100 (smallest), 1700, 2100	`false`
EP4	year % 4 = 0, year % 100 $\neq$ 0	2024, 4 (smallest), 1996	`true`
EP5	year % 4 $\neq$ 0	2023, 1 (smallest valid year), `Integer.MAX_VALUE`	`false`

Note the deliberately chosen *negative multiples of 4/400* (-4, -400) in EP1: a buggy implementation that tests divisibility before validating the range would return `true` instead of throwing, and these values expose that ordering bug.

Step 3 – Complete JUnit 5 test class

```
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

@DisplayName("LeapYearChecker.isLeapYear(int)")
class LeapYearCheckerTest {

    private LeapYearChecker checker;           // unit under test

    @BeforeEach
    void setUp() {                             // fixture: fresh object for every test
        checker = new LeapYearChecker();
    }

    // ---------- EP2 : divisible by 400  -> leap year
    @Test
    void isLeapYear_yearDivisibleBy400_returnsTrue() {
        int year = 2000;                                    // Arrange
        boolean actual = checker.isLeapYear(year);          // Act
        assertTrue(actual, "2000 is divisible by 400");     // Assert
    }

    @ParameterizedTest(name = "{0} is divisible by 400 -> leap")
    @ValueSource(ints = {400, 1600, 2400})
    void isLeapYear_otherMultiplesOf400_returnTrue(int year) {
        assertTrue(checker.isLeapYear(year));
    }

    // ---------- EP3 : century year NOT divisible by 400 -> not leap
    @Test
    void isLeapYear_centuryYearNotDivisibleBy400_returnsFalse() {
        assertFalse(checker.isLeapYear(1900), "1900 is a century not divisible by 400");
    }

    @ParameterizedTest(name = "{0} is a century not divisible by 400 -> not leap")
    @ValueSource(ints = {100, 1700, 1800, 2100})
    void isLeapYear_otherNonLeapCenturies_returnFalse(int year) {
        assertFalse(checker.isLeapYear(year));
    }

    // ---------- EP4 : divisible by 4 but not by 100 -> leap year
    @Test
    void isLeapYear_divisibleBy4ButNotBy100_returnsTrue() {
        assertTrue(checker.isLeapYear(2024), "2024 is divisible by 4, not by 100");
    }

    @ParameterizedTest(name = "{0} is divisible by 4 but not by 100 -> leap")
    @ValueSource(ints = {4, 1996, 2004})
    void isLeapYear_otherOrdinaryLeapYears_returnTrue(int year) {
```

```
        assertTrue(checker.isLeapYear(year));
    }

    // ---------- EP5 : not divisible by 4 -> not leap
    @Test
    void isLeapYear_yearNotDivisibleBy4_returnsFalse() {
        assertFalse(checker.isLeapYear(2023), "2023 is not divisible by 4");
    }

    @ParameterizedTest(name = "{0} is not divisible by 4 -> not leap")
    @ValueSource(ints = {2, 3, 2019, 2026, Integer.MAX_VALUE})
    void isLeapYear_otherYearsNotDivisibleBy4_returnFalse(int year) {
        assertFalse(checker.isLeapYear(year));
    }

    // ---------- Boundary of the valid range
    @Test
    void isLeapYear_smallestValidYear1_returnsFalseWithoutException() {
        assertDoesNotThrow(() -> checker.isLeapYear(1));
        assertFalse(checker.isLeapYear(1), "year 1 is valid and not leap");
    }

    // ---------- EP1 : invalid input (year <= 0) -> IllegalArgumentException
    @Test
    void isLeapYear_yearZero_throwsIllegalArgumentException() {
        assertThrows(IllegalArgumentException.class,
                     () -> checker.isLeapYear(0),
                     "0 is the largest invalid year");
    }

    @ParameterizedTest(name = "year {0} is invalid -> exception")
    @ValueSource(ints = {-1, -4, -100, -400, Integer.MIN_VALUE})
    void isLeapYear_negativeYear_throwsIllegalArgumentException(int year) {
        assertThrows(IllegalArgumentException.class, () -> checker.isLeapYear(year));
    }
}
```

Step 4 – Why this test class is complete

- **Every partition** EP1–EP5 has one explicit, descriptively named test (written in Arrange–Act–Assert form) plus a parameterised test with further representatives.
- **Boundaries**: 0 (largest invalid), 1 (smallest valid), 4, 100, 400 (smallest member of each valid class) and the extreme values `Integer.MIN_VALUE/MAX_VALUE`.
- **Assertions** are chosen to match the expected outcome: `assertTrue` / `assertFalse` for the boolean result, `assertThrows` for the exception and `assertDoesNotThrow` for the valid-range boundary; messages explain *what* is checked (JUnit already prints expected/actual, so they are not repeated).
- **Independence**: `@BeforeEach` creates a new `LeapYearChecker` for each test, so no test depends on another and the suite is repeatable.

NOTE The question does not name the class. It is assumed to be an instance method of `LeapYearChecker`; if it is `static`, replace `checker.isLeapYear(y)` by `LeapYearChecker.isLeapYear(y)` and drop the fixture. JUnit 5 (`junit-jupiter-params`) is used; in JUnit 4 the same tests use `@Before` and `@Test(expected=...)`.

Answer. Five input classes: invalid ($\leq 0 \rightarrow$ exception), divisible by 400 ($\rightarrow$ true), century not divisible by 400 ($\rightarrow$ false), divisible by 4 but not 100 ($\rightarrow$ true), not divisible by 4 ($\rightarrow$ false). One named test per class plus boundary/parameterised tests, using `assertTrue/False`, `assertThrows` and a `@BeforeEach` fixture.

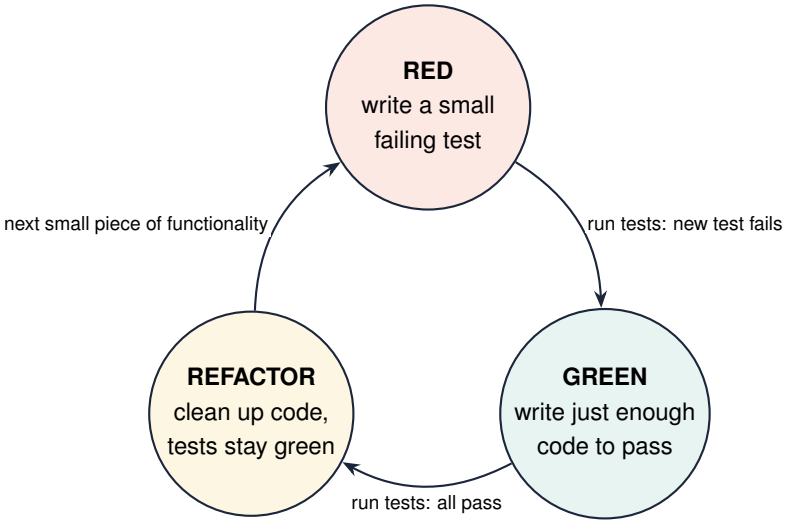
2023–2024 • Q3(a) CSE213 • 09/02/2025 • T006.1 **MARKS 6 + 6**

Briefly describe test-driven development. Write down the benefits of test-driven development.

SOLUTION

(a) Test-driven development (TDD)

Test-driven development is an approach to program development in which *testing and code development are interleaved*: the developer writes an automated (unit) test for a small increment of functionality *before* writing the code for it, and the code is written only to make that test pass. Development proceeds in tiny increments and the next increment is not started until the code passes *all* tests written so far. TDD was introduced as part of Extreme Programming (XP), where it supports "testing from day one".
The process is the short **Red–Green–Refactor** cycle:



(i) Identify a small increment of functionality and **write a test** for it, implemented as an automated test (e.g. a JUnit `@Test`).

(ii) **Run the test** together with all existing tests. The new test *fails* (Red) because the functionality does not exist yet – this proves the test can actually detect a fault.

(iii) **Write the minimum code** that makes the test pass and re-run all tests (Green).

(iv) **Refactor** the code (and tests) to remove duplication and improve design, re-running the tests after every change to be sure behaviour is unchanged.

(v) Repeat with the next increment.

(b) Benefits of TDD

Benefit	Explanation
Code coverage	Every code segment is written to satisfy at least one test, so all code is executed by the test suite from the start.
Regression testing	The automated suite grows with the program and is re-run after every change, so a change that breaks existing behaviour is detected immediately.
Simplified debugging	When a test fails, the fault is almost certainly in the few lines just written, so locating the bug is quick; no long debugging sessions.
System documentation	The tests are executable documentation: they show exactly what each method is supposed to do, with concrete examples, and never become out of date.
Better design	Writing the test first forces the developer to think about the interface and the requirement before the implementation; code becomes modular, loosely coupled and testable.
Confidence to refactor	A green test suite is a safety net, so developers can restructure code (refactoring) without fear.
Fewer defects, lower cost	Defects are caught minutes after they are introduced, when they are cheapest to fix; fewer bugs reach integration/system testing.

Answer. TDD = write an automated failing test first, write just enough code to pass it, refactor, repeat (Red–Green–Refactor). Benefits: full code coverage, a growing regression-test suite, simplified debugging, tests as living documentation, better (testable) design, safe refactoring and fewer defects.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q2(b) CSE307 • 05/05/2024 • F005 MARKS 12

What are the issues commonly identified during code review? Give 3 examples.

2020–2021 • Q6(c) CSE307 • 19/10/2022 • F013 MARKS 8

What are the issues commonly identified during Code review?

SOLUTION

Code review

Code review (peer review, "proofreading" of code) is *the process of analyzing code written by a teammate (a peer) to judge whether it is of sufficient quality to be integrated into the main project codebase*. The author pushes a change to a review tool (Gerrit, GitHub, Review Board, Phabricator, Crucible), reviewers comment, the author edits, and the change is finally approved and merged or abandoned.

Issues commonly identified during code reviews

Issue	What the reviewer notices
Coding style	inconsistent naming, indentation, brace style; violates the team's style guide
Redundant checks	the same condition checked twice, dead code, unnecessary null checks
Project design violation	bypasses the layered architecture, breaks agreed patterns, duplicates an existing utility
Unsafe methods	use of `strcpy`, `gets`, deprecated or thread-unsafe APIs
Critical security defects	buffer overflow, SQL injection, XSS, hard-coded passwords
Malicious code	back doors, hidden logic, data being sent to unknown hosts
Inadequate input validation	no range/null/format checks on user or external input
Misunderstood requirements	code does something different from what the spec/user story asks
Lack of exception handling	ignored return values, empty `catch`, resources not released

Besides these, reviewers also catch plain *silly mistakes* (off-by-one, wrong operator).

Example 1 – Silly mistakes, input validation, coding style (Java)

```java
public double average(int[] marks) {
    int Sum = 0;
    for (int i = 0; i <= marks.length; i++)
        Sum += marks[i];
    return Sum / marks.length;
}
```

Line	Issue	Severity	Review comment / suggested fix
3	Silly mistake	Major	`i <= marks.length` reads past the end → `ArrayIndexOutOfBoundsException`. Use `i < marks.length` (or a for-each loop).
5	Silly mistake	Major	`int / int` truncates (average of 1 and 2 becomes 1.0). Use `(double) sum / marks.length`.
1, 5	Input validation	Major	`null` or empty array → NPE / division by zero. Validate and throw `IllegalArgumentException`.
2	Coding style	Minor	local variable `Sum` should be `sum` (lowerCamelCase).

Example 2 – Unsafe method / critical security defect: buffer overflow (C)

```c
void greet(const char *name) {
    char buf[16];
    strcpy(buf, name);
    printf("Hello %s\n", buf);
}
```

Line 3, Critical: `strcpy` copies without a length check; a name longer than 15 characters overwrites the stack (return address) and can be exploited. **Fix:** `snprintf(buf, sizeof buf, "%s", name);` or validate `strlen(name) < sizeof buf` first.

Example 3 – Critical security defect: XSS (Java servlet)

```java
String q = request.getParameter("q");
out.println("<h2>Results for " + q + "</h2>");
```

Line 2, Critical: user input is written into the HTML page unescaped; `q=<script>...</script>` runs in the victim's browser (cross-site scripting). **Fix:** HTML-encode the output (e.g. `Encode.forHtml(q)` from OWASP Encoder) and validate the length/characters of `q` on line 1.

Answer. Commonly found issues: coding style, redundant checks, project design violations, unsafe methods, critical security defects, malicious code, inadequate input validation, misunderstood requirements and lack of exception handling. Examples: off-by-one loop / integer division (silly mistakes), `strcpy` buffer overflow (unsafe method, security), unescaped output causing XSS (security).

2022–2023 • Q5(c) CSE325 • 30/01/2025 • C325-2223-5c **MARKS 8**

Design different unit test cases for the following function `filterArray()` that filters out all numbers below a specified threshold from an input array.

```
def filterArray (array, threshold):
    filtered = []
    for num in array:
        if num > = threshold:
            filtered.append(num)
    return filtered
```

SOLUTION

Step 1 – Specification under test

The function must return a *new* list containing, *in the original order*, every element `num` with `num >= threshold`; elements below the threshold are dropped. (The `> =` in the paper is read as the operator `>=`.)

Step 2 – Deriving the cases (equivalence classes + boundaries)

- **Array size:** empty / one element / many elements.
- **Element vs threshold:** below (dropped), **equal – the boundary** (kept), above (kept); test the values `threshold-1`, `threshold`, `threshold+1` (and ± 0.01 for floats).
- **Result shape:** none kept, all kept, some kept; order and duplicates preserved.
- **Number kinds:** negative numbers, zero, floats.
- **Invalid input (negative tests):** `None` instead of a list, a non-numeric element.
- **Side effect:** the input array must not be modified.

ID	array	threshold	expected
U1	`[1, 5, 10, 3, 7]` (typical, mixed)	5	`[5, 10, 7]`
U2	`[4, 5, 6]` (boundary $t-1, t, t+1$)	5	`[5, 6]`
U3	`[1, 2, 3]` (all below)	10	`[]`
U4	`[10, 20, 30]` (all at/above)	10	`[10, 20, 30]`
U5	`[]` (empty)	5	`[]`
U6	`[5]` (single, equal)	5	`[5]`
U7	`[-5, -1, 0, 3]` (negatives, zero)	-1	`[-1, 0, 3]`
U8	`[4.99, 5.0, 5.01]` (floats)	5	`[5.0, 5.01]`
U9	`[7, 1, 7, 2, 9]` (duplicates, order)	5	`[7, 7, 9]`
U10	`[3, 8]` (input unchanged)	5	result `[8]`; input still `[3, 8]`
U11	`None` (invalid)	5	raises `TypeError`
U12	`[1, "a", 9]` (non-numeric)	5	raises `TypeError`

Step 3 – Unit tests (pytest, same language as the code)

```python
import pytest
from filters import filterArray

@pytest.mark.parametrize("array, threshold, expected", [
    ([1, 5, 10, 3, 7], 5, [5, 10, 7]),      # U1 typical
    ([4, 5, 6],        5, [5, 6]),          # U2 boundary: equal value is kept
    ([1, 2, 3],       10, []),              # U3 none kept
    ([10, 20, 30],    10, [10, 20, 30]),    # U4 all kept
    ([],               5, []),              # U5 empty input
    ([5],              5, [5]),             # U6 single element on boundary
    ([-5, -1, 0, 3],  -1, [-1, 0, 3]),      # U7 negatives and zero
    ([4.99, 5.0, 5.01], 5, [5.0, 5.01]),    # U8 floats near boundary
    ([7, 1, 7, 2, 9],  5, [7, 7, 9]),       # U9 duplicates, order kept
])
def test_filter_array(array, threshold, expected):
    assert filterArray(array, threshold) == expected

def test_input_not_modified():               # U10
    data = [3, 8]
    assert filterArray(data, 5) == [8]
    assert data == [3, 8]

def test_none_array_raises():                # U11
    with pytest.raises(TypeError):
        filterArray(None, 5)

def test_non_numeric_element_raises():       # U12
    with pytest.raises(TypeError):
```

```
        filterArray([1, "a", 9], 5)
```

Each test follows the *Arrange–Act–Assert* pattern, tests one behaviour, is independent of the others and is automated, so it can be re-run as a regression suite after any change (e.g. if someone accidentally writes > instead of >=, test U2/U6 fails immediately).

Answer. Partition by array size (empty / one / many), by element relative to the threshold (below / equal / above, with boundary values $t-1, t, t+1$), by number kind (negative, zero, float), plus duplicates/order, no side effect and invalid inputs (None, non-numeric). Twelve cases U1–U12 implemented as a parametrised pytest suite.

2021–2022 • Q5(c) CSE307 • 07/05/2023 • F010 **MARKS 5**

Discuss the necessary of writing *short*, *useful*, and *up-to-date* code documents.

SOLUTION

Why code documentation needs these three properties

Code documents (comments, Javadoc/API docs, README, design notes) exist so that *other* people – reviewers, maintainers, new team members – can understand code quickly. Documentation that is long, vague or stale fails this purpose and is even reviewed as a defect during code review.

- **Short.** Developers read documentation under time pressure; a concise note is read, a long one is skipped. Short documents are also cheaper to keep in sync with the code. The code itself (good names, small methods) should explain *what*; comments should only add *why*, constraints and non-obvious decisions.
- **Useful.** Every line must answer a question the reader actually has: purpose of the module, pre/post-conditions, units, thread-safety, exceptions, usage example, reason for an odd workaround. Comments that repeat the code ("`i++; // increment i`") add noise and hide the important information.
- **Up-to-date.** Out-of-date documentation is *worse than none*: it misleads the reader, causes wrong usage and bugs, and destroys trust in all documents. Keep docs next to the code (Javadoc), update them in the same commit, and let reviewers check them in code review.

```
// BAD: long, useless, and stale (the method no longer returns -1)
// This method takes the id and then it loops and searches the list of
// students one by one and returns the index or -1 if not found ...
public Student find(String id) { ... }

// GOOD: short, useful, current
/** Returns the student with this BUET ID.
 *  @throws NoSuchElementException if no such student exists */
public Student find(String id) { ... }
```

Answer. Short docs get read and are cheap to maintain; useful docs explain intent, contracts and non-obvious decisions instead of restating code; up-to-date docs are essential because stale documentation misleads readers and causes defects. Together they reduce maintenance and review effort.

2021–2022 • Q8(a) CSE307 • 07/05/2023 • F012 **MARKS 10**

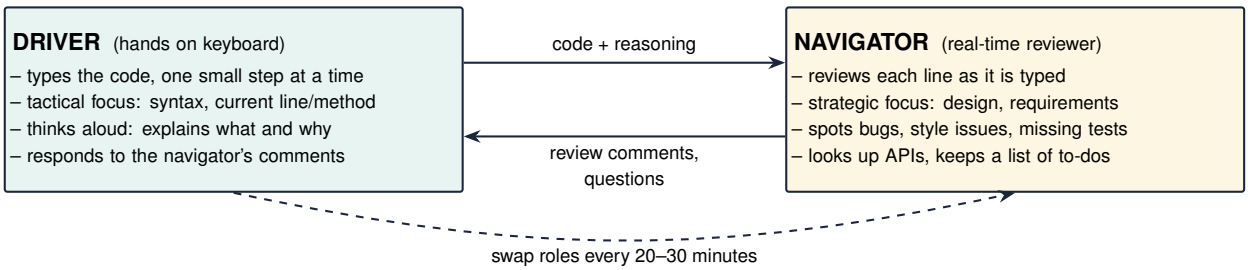
Discuss the role of driver and navigator in the content of code review in pair programming.

SOLUTION

> **NOTE** "content of code review" is read as "context of code review".

Pair programming as continuous code review

In **pair programming** (an XP practice) two developers work together at one computer to design, code and test the same piece of software. One of them is the **driver**, the other the **navigator**, and they swap roles frequently. Because the second person examines every line the moment it is written, pair programming is a form of *continuous, real-time code review*: defects are found immediately instead of days later in a formal review.



Role of the driver

- Controls the keyboard and mouse and actually writes the code and the tests.
- Concentrates on the *tactical* details: correct syntax, the current method, making the current test pass.
- Verbalises intentions ("now I check the null case…") so the navigator can review the reasoning, not just the text.
- Accepts or discusses the navigator's review comments and fixes issues immediately.

Role of the navigator (the reviewer)

- Reviews every line as it is typed: typos, off-by-one errors, wrong conditions, missing null checks, unhandled exceptions – the same issues a formal reviewer would flag.
- Thinks *strategically*: Does the code meet the requirement? Does it fit the design and coding standards? Which edge cases and tests are still missing? Is there duplication to refactor?
- Looks up documentation/APIs and records ideas or problems to handle later, so the driver is not interrupted.
- Gives constructive, specific feedback ("line 12 should be <, not <="), keeping the review culture positive.

Aspect	Driver	Navigator
Activity	writes code (author)	reviews code (reviewer)
Focus	tactical, current line	strategic, big picture
Review timing	receives feedback instantly	gives feedback instantly
Typical catch	–	logic slips, style, design violations, missing tests

Why this matters for code review

- Defects are removed seconds after they are introduced – cheapest point to fix them.
- Knowledge dissemination and collective code ownership: both know every line; regular role switching spreads this further.
- Consistent coding style and continuous design discussion, the same "other benefits" that formal code review gives, but without waiting for a separate review meeting.

Answer. The driver is the author who types and focuses on tactical details while explaining aloud; the navigator is a real-time reviewer who checks each line for defects, watches design, requirements, edge cases and tests, and gives immediate feedback. Roles are swapped often, so pair programming acts as continuous code review.

Jan 2021 • Q3(b) CSE307 • 04/09/2021 • M013 **MARKS 15**

You are a project manager in a software development project where the budget of development is limited. Employing reviewer for system requirement specification, design and code requires a huge budget that may lead towards falling short of budget. You may recruit more testers instead which are not that much costly in terms of salary. What will be your decision considering overall situation? Justify your decision.

SOLUTION

Key idea: where defects are found decides what they cost

A **review** (inspection/peer code review) is the process of analysing an artifact written by a teammate to judge whether it is of sufficient quality to be integrated. Reviews can be applied to the *SRS, design and code*; testing can only be applied once code exists. The cost of fixing a defect grows steeply the later it is found (Boehm/Fagan): roughly $1\times$ in requirements, $5\times$ in design, $10\times$ in coding, 20–$50\times$ in testing and up to $100\times$ after release. Fagan's "snail curve" shows inspected projects spend more effort early but finish sooner and cheaper.

Why "only more testers" is a false saving

- Testers cannot find *requirements and design* defects until the system is built – by then fixing them means redesign and recoding (the most expensive rework).
- Testing shows the *presence* of failures, not their cause; reviews locate the defect directly and also find issues tests rarely reveal: coding-style and maintainability problems, design violations, security defects, unsafe methods, missing exception handling, misunderstood requirements.
- Reviews also spread knowledge in the team and reduce the bus factor.

Illustrative comparison (100 defects injected; relative fix cost: review 1, test 5, field 20 units)

	More testers, no reviews	Light reviews + testers
Found in reviews	0	$60 \times 1 = 60$
Found in testing	$85 \times 5 = 425$	$34 \times 5 = 170$
Escaped to users	$15 \times 20 = 300$	$6 \times 20 = 120$
Total rework cost	**725**	**350**

Even after paying for the review effort, the rework saving (about half) usually exceeds it, and fewer defects reach the customer.

Decision

Do **not** drop reviews in favour of testers; instead make reviews *affordable*:

(i) Use **peer reviews by the existing developers** with a lightweight, tool-based workflow (pull requests in GitHub/Gerrit) instead of hiring expensive dedicated reviewers or heavy Fagan meetings.

(ii) Apply **formal inspection only to high-risk artifacts**: the SRS and the architecture/design, and critical code modules; review code in small chunks (200–400 LOC) with checklists.

(iii) Keep a **balanced number of testers** (and automate unit/regression tests) for the defects reviews cannot find, e.g. integration, performance and usability failures.

(iv) Track review metrics (inspection rate, defect rate, defect density) to show the budget saving.

Answer. Keep reviews – but cheap, risk-focused peer reviews of SRS, design and critical code – combined with a moderate, automated testing effort. Early defect removal costs far less than late testing and field failures, so dropping reviews to hire more testers would increase total cost.

Jan 2021 • Q7(a) CSE307 • 04/09/2021 • T018 MARKS 20

What are *fixtures* in Unit testing? Give 3 examples.
Write down a unit testing method to test values from partitions of a grading system having the following grading policy:

Mark>80: A

Mark>70 and <=80: B

Mark>60 and <=70: C

Consider Boundary value analysis.

SOLUTION

(a) Fixtures

A **test fixture** is *a fixed state of a set of objects used as a baseline for running tests*. Its purpose is to guarantee a well-known, fixed environment in which every test runs, so that results are *repeatable* and tests are *independent* of each other. In JUnit the fixture is built and destroyed by annotated methods:

JUnit 4	JUnit 5	Role
`@Before`	`@BeforeEach`	set-up run before *each* test; each test gets its own fresh objects
`@After`	`@AfterEach`	tear-down after each test (release external resources)
`@BeforeClass`	`@BeforeAll`	expensive one-time set-up for the whole class (e.g. start a DB)
`@AfterClass`	`@AfterAll`	one-time clean-up after all tests

Three examples of fixtures

(i) **A fresh object in a known state** – e.g. creating `new Date(2050, 2, 15)` or a new `GradingSystem` in `@BeforeEach` so every test starts from the same object.

(ii) **Known test data loaded into a database / collection** – e.g. inserting three known student records into an in-memory database before each test and deleting them in `@AfterEach`.

(iii) **External resources and test doubles** – e.g. creating a temporary input file with fixed contents, opening a network/DB connection in `@BeforeAll`, or preparing a stub/mock object that returns canned values; closed or deleted in `@AfterEach`/`@AfterAll`.

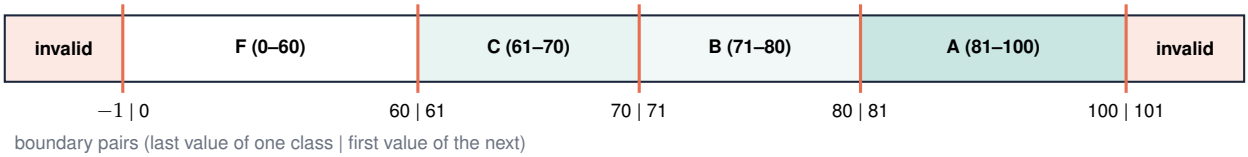
```
class StudentRepositoryTest {
    private static Connection conn;           // shared, expensive fixture
    private StudentRepository repo;           // per-test fixture

    @BeforeAll static void openDb() throws SQLException {
        conn = DriverManager.getConnection("jdbc:h2:mem:test");
    }
    @BeforeEach void loadKnownData() {
        repo = new StudentRepository(conn);
        repo.insert(new Student(1, "Rahim", 85));
        repo.insert(new Student(2, "Karim", 72));
    }
    @AfterEach void clearData()  { repo.deleteAll(); }
    @AfterAll  static void closeDb() throws SQLException { conn.close(); }

    @Test void findById_existingStudent_returnsIt() {
        assertEquals("Rahim", repo.findById(1).getName());
    }
}
```

(b) Partitions and boundary values

The policy only defines A, B and C. To make the partition complete we assume marks are integers in **0–100**, a mark ≤ 60 gets **F**, and a mark outside 0–100 is invalid (`IllegalArgumentException`).



Partition	Range	Test values (boundaries + one nominal)	Expected
Invalid (low)	< 0	-1	exception
F	0–60	0, 30, 60	F
C	61–70	61, 65, 70	C
B	71–80	71, 75, 80	B
A	81–100	81, 90, 100	A
Invalid (high)	> 100	101	exception

The critical boundaries of the given policy are **60/61**, **70/71** and **80/81** ("$>$" versus "$\geq$" mistakes happen exactly there), plus $0/-1$ and $100/101$ of the valid range.

(c) Unit under test (for reference)

```
public class GradingSystem {
    public char grade(int mark) {
```

```
        if (mark < 0 || mark > 100)
            throw new IllegalArgumentException("mark out of range: " + mark);
        if (mark > 80) return 'A';
        if (mark > 70) return 'B';
        if (mark > 60) return 'C';
        return 'F';
    }
}
```

(d) JUnit 5 test class

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;

class GradingSystemTest {

    private GradingSystem system;

    @BeforeEach void setUp()    { system = new GradingSystem(); }   // fixture
    @AfterEach  void tearDown() { system = null; }

    // The requested unit-testing method: one value from each partition + its boundaries
    @Test
    void grade_boundaryAndNominalValuesOfEveryPartition() {
        assertAll("grading policy",
            () -> assertEquals('A', system.grade(100), "upper bound of A"),
            () -> assertEquals('A', system.grade(90),  "nominal A"),
            () -> assertEquals('A', system.grade(81),  "lower bound of A"),
            () -> assertEquals('B', system.grade(80),  "upper bound of B"),
            () -> assertEquals('B', system.grade(75),  "nominal B"),
            () -> assertEquals('B', system.grade(71),  "lower bound of B"),
            () -> assertEquals('C', system.grade(70),  "upper bound of C"),
            () -> assertEquals('C', system.grade(65),  "nominal C"),
            () -> assertEquals('C', system.grade(61),  "lower bound of C"),
            () -> assertEquals('F', system.grade(60),  "just below C"),
            () -> assertEquals('F', system.grade(0),   "lowest valid mark"));
    }

    // Same values, data-driven: each row is reported as a separate test
    @ParameterizedTest(name = "mark {0} -> grade {1}")
    @CsvSource({"81, A", "90, A", "100, A",
                "71, B", "75, B", "80, B",
                "61, C", "65, C", "70, C",
                "0, F",  "30, F", "60, F"})
    void grade_validMark_returnsGradeOfItsPartition(int mark, char expected) {
        char actual = system.grade(mark);                       // Act
        assertEquals(expected, actual, "grade for mark " + mark);  // Assert
    }

    @ParameterizedTest(name = "mark {0} is invalid")
    @ValueSource(ints = {-1, 101})
    void grade_markOutsideZeroToHundred_throwsException(int mark) {
        assertThrows(IllegalArgumentException.class, () -> system.grade(mark));
    }
}
```

NOTE The policy is silent about marks ≤ 60 and about the valid mark range; the solution assumes grade F for 0–60 and an exception outside 0–100. With a different assumption only the "F" and "invalid" rows change – the A/B/C boundary tests (60/61, 70/71, 80/81) stay the same.

Answer. A fixture is a fixed, known baseline state (objects, data, resources) built before each test (`@Before`/`@BeforeEach`) and released after it (`@After`/`@AfterEach`); e.g. a fresh object, preloaded test data, an opened file/DB connection or stub. The grading test checks boundary pairs 60/61, 70/71, 80/81 (plus 0, 100, -1, 101) and a nominal value of every partition.

2018–2019 • Q1(c) CSE307 • 06/11/2019 • F016 **MARKS 9**

What do you understand by defect rate and defect density in Software Engineering? Why are checklists used in code review?

SOLUTION

(a) Defect rate and defect density

One best practice of code review is to *establish quantifiable goals and capture metrics* so the review process can be improved. Three process metrics are collected:

$$\textbf{Inspection rate} = \frac{\text{LOC reviewed}}{\text{review hours}} \qquad \text{(speed of the review)}$$

$$\textbf{Defect rate} = \frac{\text{number of defects found}}{\text{review hours}} \qquad \text{(bugs found per hour of review)}$$

$$\textbf{Defect density} = \frac{\text{number of defects found}}{\text{LOC reviewed}} \qquad \text{(bugs per line, usually per KLOC)}$$

Example. A reviewer spends 2 hours on a 400-line change and logs 12 defects.

- Inspection rate $= 400/2 = 200$ LOC/hour.
- Defect rate $= 12/2 =$ **6 defects/hour** – measures how effective the review effort is.
- Defect density $= 12/400 = 0.03$ defects/LOC $=$ **30 defects/KLOC** – measures the quality of the code itself.

Interpretation: if inspection rate is pushed too high (e.g. > 500 LOC/hour) the defect rate drops because reviewers miss bugs; a module whose defect density is far above the team average needs rework or extra testing. Defect metrics judge the *process and code*, not the people.

(b) Why checklists are used in code review

- **Developers repeat the same mistakes.** A checklist built from past defects (null checks, resource release, input validation, error handling, naming) makes sure each one is looked for every time.
- **Omissions are the hardest defects to find** – it is difficult to review something that is *not there* (a missing `else`, a missing `close()`, an unhandled exception). A checklist item explicitly asks for it.
- Checklists are *the most effective way* to eliminate frequently made errors; they substantially improve results for both authors (who self-check before submitting) and reviewers.
- They make reviews systematic, consistent across reviewers and faster, and the checklist evolves as defect metrics reveal new common faults.

Answer. Defect rate = defects found per hour of review; defect density = defects found per line (or KLOC) of code. Checklists are used because developers repeat the same mistakes and omissions are hard to spot; a checklist systematically reminds reviewers to look for them.

2017–2018 • Q5(c) CSE307 • 29/09/2018 • F018 **MARKS 9**

What indirect benefits are offered by Code Review practice in a software development company? Discuss in brief with examples.

SOLUTION

Direct vs. indirect benefits

The *direct* benefit of code review is finding defects (bugs, security holes, style violations) before the code is merged. Beyond that, the practice gives the company several **indirect ("other") benefits**:

Indirect benefit	Meaning	Example
Project awareness	Reviewers learn what is changing in other parts of the system.	A backend developer reviewing a payment change knows the new refund API exists and does not build a duplicate.
Peer impression formation	Team members get to know each other's skills and reliability.	A lead sees that a junior consistently writes clean, well-tested code and assigns her a critical module.
Consistent coding style	Repeated comments converge the team on one style and idioms.	After a few reviews everyone uses the same naming, logging and exception-handling conventions.
Design discussion	Reviews become a forum to discuss alternative designs.	A reviewer suggests the Strategy pattern instead of a long `switch`; the team adopts it project-wide.
Knowledge dissemination	Knowledge spreads; juniors learn from seniors and vice versa.	A new hire learns the codebase and the company's security practices from review comments.

Further indirect effects for the company.

- *Collective code ownership / lower bus factor:* at least two people know every change, so a developer leaving or on leave does not block the project.
- *Mentoring and onboarding* become part of daily work, reducing training cost.
- *Higher care by authors:* knowing that a peer will read the code, authors write cleaner, documented code (the "ego effect").
- *Lower long-term cost:* consistent, well-understood code is cheaper to maintain and extend.

Answer. Indirect benefits of code review: project awareness, peer impression formation, consistent coding style, design discussion and knowledge dissemination (plus collective ownership, mentoring of new members and more careful authors), each leading to a more capable team and cheaper maintenance.

2016–2017 • Q3(b) CSE307 • 15/07/2017 • F020 **MARKS 15**

What are the checklists for a code reviewer to review the Non-Functional requirements of a project? Explain in short.

SOLUTION

Concept

Non-functional requirements (NFRs) describe *how well* the system works – performance, security, reliability, usability, maintainability, etc. They are easily overlooked in code review, because the code may be functionally correct and still slow, insecure or unmaintainable. A **checklist** reminds the reviewer to check each quality attribute systematically and helps find *omissions* (missing timeouts, missing logging), which are otherwise the hardest defects to see.

Checklist for reviewing non-functional requirements

NFR	Checklist items for the reviewer
Performance / efficiency	Are algorithms and data structures of suitable complexity? Any work inside loops that can be hoisted? N+1 database queries, `SELECT *`, missing indexes or caching? Are large objects created needlessly?
Scalability	Does it work for 10× the data/users (pagination, streaming, no full-table load)? Is it stateless so it can be replicated? Any shared bottleneck or global lock?
Security	Is all external input validated? Queries parameterised (no SQL injection), output encoded (no XSS)? No hard-coded passwords/keys? Authorisation checked? Sensitive data encrypted and not logged? Safe library functions (no buffer overflow)?
Reliability / availability	Are exceptions handled (no empty `catch`)? Resources (files, connections, locks) always released? Timeouts and retries for remote calls? Thread-safety of shared data? Graceful degradation on failure?

NFR (cont.)	Checklist items for the reviewer
Usability	Are error messages clear and helpful to users? Consistent UI texts/labels, accessibility, input formats and localisation handled?
Maintainability	Does it follow the coding standard and design (patterns, layering)? Meaningful names, small methods, no duplication, no magic numbers? Short, useful, up-to-date comments?
Testability	Are unit tests included and meaningful? Can dependencies be replaced by stubs/mocks (dependency injection)? Are edge cases covered?
Portability / compatibility	No OS-specific paths/encodings? Configuration externalised? Backward compatibility of APIs and data formats?
Logging / auditability	Are important events and errors logged at the right level, with enough context but without secrets? Audit trail for critical actions?
Compliance	Licences of third-party code, privacy rules and legal/company standards respected?

Applying the checklist – example review

```
1 public List<Order> findOrders(String customerId) throws SQLException {
2     Connection c = DriverManager.getConnection(URL, "admin", "admin123");
3     Statement st = c.createStatement();
4     ResultSet rs = st.executeQuery(
```

```
5           "SELECT * FROM orders WHERE customer_id = '" + customerId + "'");
6       List<Order> result = new ArrayList<>();
7       while (rs.next()) {
8           Order o = map(rs);
9           log.info("Loaded order, card=" + o.getCardNumber());
10          result.add(o);
11      }
12      return result;
13  }
```

Line	NFR	Severity	Review comment / fix
2	Security	Critical	Hard-coded DB credentials; read them from secured configuration.
4–5	Security	Critical	String concatenation → SQL injection; use a `PreparedStatement` with `?`.
2–3	Reliability	Major	Connection/statement/result set never closed → leak; use try-with-resources.
2	Performance	Major	New connection per call; use the connection pool (`DataSource`).
5	Scalability	Major	`SELECT *` without paging loads all orders; select needed columns, add `LIMIT`/paging.
9	Security, compliance	Major	Card number written to logs (privacy / PCI rules); log only a masked value or the order id.

Answer. An NFR review checklist covers performance, scalability, security, reliability/availability, usability, maintainability, testability, portability/compatibility, logging/auditability and compliance, each as concrete yes/no questions (e.g. “are queries parameterised?”, “are resources always closed?”, “is sensitive data logged?”).

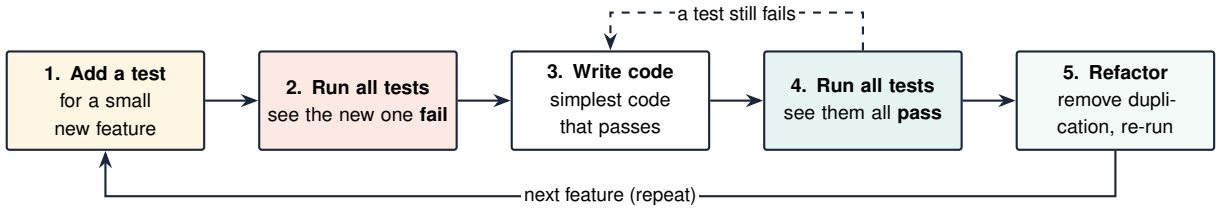
2016–2017 • Q7(a) CSE307 • 15/07/2017 • T026 **MARKS 5 + 10**

Write down the five steps of test-driven development. Demonstrate all the steps sequentially with the help of an example.

SOLUTION

(a) The five steps of TDD

In **test-driven development** the test for a piece of functionality is written *before* the code. One TDD iteration has five steps (Kent Beck):



(i) **Add a test** – choose the next small increment of required behaviour and express it as an automated unit test.

(ii) **Run all tests and see the new one fail** – confirms the test is valid and that the feature is not already present (Red).

(iii) **Write the code** – the simplest code that makes the test pass, nothing more.

(iv) **Run all tests and see them succeed** – new and old tests pass (Green); otherwise go back to step 3.

(v) **Refactor** – improve the structure (remove duplication, rename, extract) while keeping all tests green; then start again with step 1 for the next increment.

(b) Worked example – a `StringCalculator`

Requirement: `add(String numbers)` returns the sum of comma-separated integers; the empty string gives 0. We grow it test-first in three iterations.

Iteration 1 – empty string.

Step 1 (add test):

```
class StringCalculatorTest {
    @Test
    void add_emptyString_returnsZero() {
        StringCalculator calc = new StringCalculator();
        assertEquals(0, calc.add(""));
    }
}
```

Step 2 (run, fails): it does not even compile – `StringCalculator` does not exist (Red).

Step 3 (simplest code):

```
public class StringCalculator {
    public int add(String numbers) { return 0; }
}
```

Step 4 (run, passes): 1/1 green. *Step 5 (refactor):* nothing to clean yet.

Iteration 2 – a single number.

Step 1 (add test):

```
@Test
void add_singleNumber_returnsThatNumber() {
    StringCalculator calc = new StringCalculator();
    assertEquals(7, calc.add("7"));
}
```

Step 2: fails – `expected: <7> but was: <0>` (Red).

Step 3:

```
public int add(String numbers) {
    if (numbers.isEmpty()) return 0;
    return Integer.parseInt(numbers);
}
```

Step 4: 2/2 green. *Step 5:* both tests create `new StringCalculator()` – move it to a `@BeforeEach` fixture (refactoring the test code); re-run: still green.

Iteration 3 – two numbers.

Step 1 (add test):

```
@Test
void add_twoNumbers_returnsSum() {
    assertEquals(3, calc.add("1,2"));     // calc now comes from @BeforeEach
}
```

Step 2: fails with `NumberFormatException: For input string: "1,2"` (Red).
Step 3 (simplest code):

```
public int add(String numbers) {
    if (numbers.isEmpty()) return 0;
    String[] parts = numbers.split(",");
    if (parts.length == 1) return Integer.parseInt(parts[0]);
    return Integer.parseInt(parts[0]) + Integer.parseInt(parts[1]);
}
```

Step 4: 3/3 green.
Step 5 (refactor): the special cases duplicate the parsing logic; generalise to a loop – which also handles any number of values – and re-run all three tests: still green.

```
public class StringCalculator {
    public int add(String numbers) {
        if (numbers.isEmpty()) return 0;
        int sum = 0;
        for (String part : numbers.split(","))
            sum += Integer.parseInt(part.trim());
        return sum;
    }
}

class StringCalculatorTest {                    // final test class
    private StringCalculator calc;
    @BeforeEach void setUp() { calc = new StringCalculator(); }

    @Test void add_emptyString_returnsZero()       { assertEquals(0, calc.add("")); }
    @Test void add_singleNumber_returnsThatNumber() { assertEquals(7, calc.add("7")); }
    @Test void add_twoNumbers_returnsSum()          { assertEquals(3, calc.add("1,2")); }
}
```

Iter.	Step 1–2: new test (Red)	Step 3–4: code (Green)	Step 5: refactor
1	`add("")` = 0; class missing	`return 0;`	–
2	`add("7")` = 7; got 0	empty check + `parseInt`	fixture to `@BeforeEach`
3	`add("1,2")` = 3; exception	split, add two parts	general loop over all parts

Answer. Five steps: (1) add a test, (2) run all tests and watch the new one fail, (3) write the simplest code to pass it, (4) run all tests and see them pass, (5) refactor – then repeat. The `StringCalculator` example grows the method through three Red–Green–Refactor iterations, with the tests always written first.

2014–2015 • Q1(d) CSE307 • 04/02/2016 • F023 MARKS 10

Describe a scenario of reviewing the code for a method implementing quick-sort. Mention the participants and sample of the queries made by the reviewers in the review process.

SOLUTION

Scenario

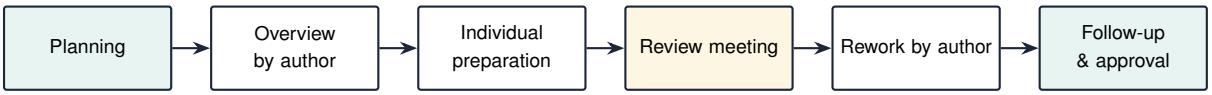
Developer *Rahim* implements `quickSort` for the library's sorting utility and submits it for a formal review (a lightweight Fagan-style inspection). The change is 14 lines, well below the recommended

200–400 LOC per review, so it fits in a single session of under an hour.

Participants

Participant	Role in this review
Author (Rahim)	wrote the code; annotates it before the review, answers questions, later fixes defects
Moderator (team lead)	plans the review, chooses reviewers, runs the meeting, ensures follow-up
Reader	paraphrases the code line by line during the meeting
Reviewers / inspectors (2 peers)	study the code beforehand using a checklist and raise queries
Recorder (scribe)	logs each defect with line, type and severity; collects metrics

In a modern tool-based review (Gerrit/GitHub) the same work is done by the author and two reviewers commenting directly on lines; the tool records the comments.



Code under review

```
public static void quickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int pivot = a[hi];
    int i = lo;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++;
        }
    }
    int t = a[i]; a[i] = a[hi]; a[hi] = t;
    quickSort(a, lo, i);
    quickSort(a, i + 1, hi);
}
```

Sample queries raised by the reviewers

Line	Query (who)	Severity	Outcome / fix
12	"After partition the pivot is final at `i`; why recurse on `lo..i`?" (Reviewer 1)	Critical	If the pivot is the maximum, `i == hi` and the call repeats with the same range → infinite recursion. Fix: `quickSort(a, lo, i - 1)`.
3	"What happens on already-sorted input with the last element as pivot?" (Reviewer 2)	Major	Every partition is maximally unbalanced: $O(n^2)$ time and recursion depth $n \rightarrow$ `StackOverflowError`. Fix: median-of-three or random pivot; recurse on the smaller part first.
6	"How does it behave when all keys are equal?" (Reviewer 2)	Major	All elements go to one side: again $O(n^2)$. Fix: three-way (Dutch-flag) partitioning.
1	"What if `a` is `null` or `lo/hi` are out of range?" (Reviewer 1)	Minor	No validation. Add a public `sort(int[] a)` wrapper that checks input and calls a private recursive method.
7, 11	"The swap is written twice – extract it?" (Reviewer 1)	Minor	Extract `swap(a, i, j)` (removes duplicate code).
–	"Where are the unit tests and Javadoc?" (Moderator)	Minor	Add JUnit tests: empty, single, sorted, reversed, duplicates, negatives; document the in-place contract.

The recorder logs these six items (defect density $\approx 6/14$ LOC for this small change), the author reworks the method, and in follow-up the moderator verifies that each defect is actually fixed and that the new tests pass before the code is approved and merged.

Answer. Participants: author, moderator, reader, two reviewers and a recorder (or author + reviewers in a review tool). Process: planning, overview, preparation, meeting, rework, follow-up. Sample queries: wrong recursion bound on line 12 (infinite recursion), worst-case pivot choice on sorted/equal data, missing input validation, duplicated swap code, missing tests/documentation.

2014–2015 • Q2(c), (d) CSE307 • 04/02/2016 • T028.4 **MARKS 13 + 7**

(c) Consider a module which sorts several floating point data in descending order. Write down the code for automated tester that tests the order as well as originality of the data. Here originality means the unsorted input and sorted output are the same set of data.
(d) What are the components of unit test environment? Explain each of them.

SOLUTION

(c) Idea of the automated tester (test oracle)

An automated tester must decide *by itself* whether the module worked. For a sort, we do not need to know the expected array in advance; two properties together characterise a correct sort:

(i) **Order**: for every i, `out[i] >= out[i+1]` (descending).

(ii) **Originality** (permutation): `out` contains exactly the same values as the input, with the same multiplicities – nothing lost, added or changed (duplicates must survive).

Order alone is not enough: a faulty module returning `{0,0,0}` or one that drops duplicates via a `Set` is still "descending". Originality alone is not enough either (the unsorted input is a permutation of itself). The tester also keeps an untouched copy of the input, since the module may sort in place. Assumed module interface: `double[] Sorter.sortDescending(double[] data)`.

```java
import static org.junit.jupiter.api.Assertions.*;
import java.util.*;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

/** Automated tester for a module that sorts doubles in DESCENDING order. */
class DescendingSortTester {

    private static final Random RND = new Random(42);   // fixed seed -> repeatable

    // ---------------- oracle 1 : order
    static boolean isDescending(double[] a) {
        for (int i = 0; i + 1 < a.length; i++)
            if (Double.compare(a[i], a[i + 1]) < 0) return false;
        return true;
    }

    // ---------------- oracle 2 : originality (same multiset of values)
    static boolean isSameData(double[] input, double[] output) {
        if (input.length != output.length) return false;
        Map<Double, Integer> count = new HashMap<>();
        for (double x : input) count.merge(x, 1, Integer::sum);      // frequency of each value
        for (double y : output) {
            Integer c = count.get(y);
            if (c == null || c == 0) return false;   // value not in input / too many copies
            count.put(y, c - 1);
        }
        return true;                                  // equal length + every value matched
    }

    // ---------------- common check used by every test
    private static void checkSort(double[] input) {
        double[] original = input.clone();                      // untouched copy
        double[] output = Sorter.sortDescending(input.clone()); // call module under test
        assertNotNull(output, "module returned null");
        assertTrue(isDescending(output), "output not in descending order: "
                   + Arrays.toString(output));
        assertTrue(isSameData(original, output), "output is not the same set of data as input");
    }

    // ---------------- hand-picked edge cases
    @Test void emptyArray()       { checkSort(new double[] {}); }
    @Test void singleElement()    { checkSort(new double[] {3.5}); }
    @Test void alreadyDescending(){ checkSort(new double[] {9.0, 5.5, 0.0, -2.5}); }
    @Test void ascendingInput()   { checkSort(new double[] {1.1, 2.2, 3.3, 4.4}); }
    @Test void allEqual()         { checkSort(new double[] {2.0, 2.0, 2.0, 2.0}); }
    @Test void duplicatesAndNegatives() {
        checkSort(new double[] {-1.5, 3.25, -1.5, 0.0, 3.25, 7.0});
    }
    @Test void extremeValues() {
        checkSort(new double[] {Double.MAX_VALUE, -Double.MAX_VALUE,
                                Double.MIN_VALUE, 0.0, 1e-300});
    }

    // ---------------- random data of many sizes (automated, many cases)
    @ParameterizedTest(name = "random array of size {0}")
    @ValueSource(ints = {2, 3, 10, 100, 1000, 100000})
```

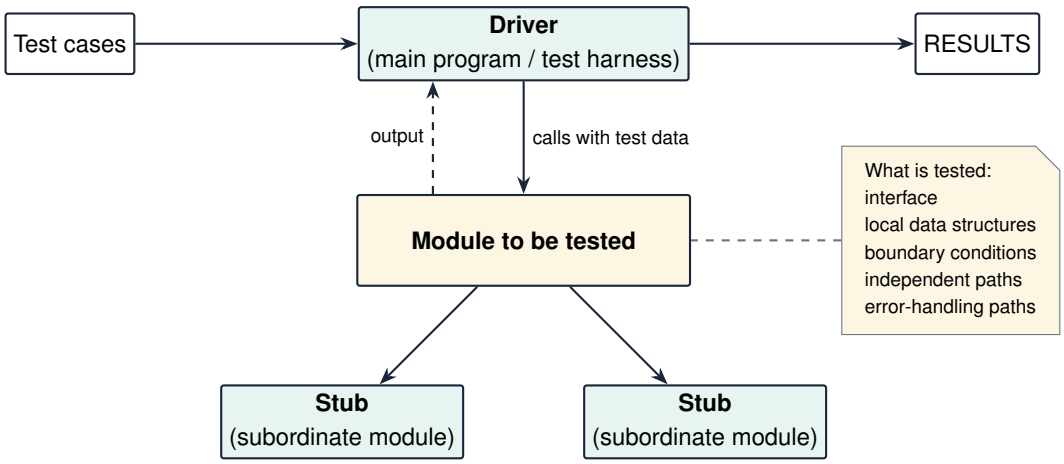
```
    void randomArrays(int n) {
        double[] a = new double[n];
        for (int i = 0; i < n; i++)
            a[i] = Math.round((RND.nextDouble() * 2000 - 1000) * 100) / 100.0; // 2 decimals ->
    duplicates likely
        checkSort(a);
    }

    // ---------------- self-check of the oracles (they must catch wrong sorts)
    @Test void oraclesRejectWrongOutputs() {
        double[] in = {1.0, 3.0, 3.0, 2.0};
        assertFalse(isDescending(new double[] {3.0, 1.0, 3.0, 2.0}));        // wrong order
        assertFalse(isSameData(in, new double[] {3.0, 2.0, 1.0}));           // duplicate lost
        assertFalse(isSameData(in, new double[] {3.0, 3.0, 2.0, 0.0}));      // value changed
        assertTrue (isSameData(in, new double[] {3.0, 3.0, 2.0, 1.0}));
    }
}
```

How it works. `checkSort` is the single automated verdict: it runs the module on a copy, then asserts *order* and *originality*. Originality is checked with a frequency map in $O(n)$, so even the 100 000-element case is fast. The test set covers edge cases (empty, one element, already sorted, reverse sorted, all equal, duplicates, negatives, extreme magnitudes) plus random arrays of growing size; the fixed seed keeps the run repeatable. The last test checks the oracles themselves.

(d) Components of the unit test environment

A module is not a stand-alone program, so to test it in isolation we surround it with extra software. The unit-test environment (Pressman) is:



- **Module to be tested (unit under test)** – the component itself. Tests exercise its interface, local data structures, boundary conditions, independent paths and error-handling paths.
- **Driver** – a "main program" that accepts test-case data, passes it to the module, and prints/compares the relevant results. It replaces the (not yet written or not yet integrated) *caller* of the module. A JUnit test class is a driver; so is `SortDriver` in the next question.
- **Stub (dummy subprogram)** – replaces a module that is *called by* the unit under test. It uses the subordinate module's interface, does minimal data manipulation, may print a trace of its entry, and returns canned values, so the unit can be tested before (or without) the real subordinate. Mocks are stubs that also verify how they were called.

- **Test cases** – the input data and expected outputs (chosen by EP, BVA, path coverage) that the driver feeds to the module.
- **Results / oracle** – actual outputs compared with expected outputs; the driver reports pass/fail so the test is automatic and atomic.

Small example of a driver + stub. `ResultPublisher.publish(id)` (module under test) calls `StudentRepository` (not ready yet), so a stub provides canned data:

```
interface StudentRepository { int marksOf(String id); }

class StudentRepositoryStub implements StudentRepository {   // STUB
    public int marksOf(String id) { return id.equals("2305003") ? 85 : 55; }
}

class ResultPublisherTest {                                   // DRIVER
    @Test void publish_highMarks_givesGradeA() {
        ResultPublisher p = new ResultPublisher(new StudentRepositoryStub());
        assertEquals("2305003: A", p.publish("2305003"));
    }
}
```

Answer. (c) The tester runs the sort on a copy of each input and asserts two oracles: every `out[i] >= out[i+1]` (order) and the output is the same multiset as the input, checked by a frequency map (originality), over edge cases and random arrays of many sizes. (d) Unit-test environment = module under test + driver (calls it with test data, collects results) + stubs (replace modules it calls) + test cases + results/oracle.

2013–2014 • Q3(c) CSE307 • 08/08/2015 • T029.2 **MARKS 12**

Consider a module which shorts a number of floating point data in ascending order. Write down the driver code for generating different sizes of data set. Write down the code for this module.

SOLUTION

NOTE "shorts" in the question is read as "sorts".

Step 1 – Role of a driver

In unit testing, a **driver** is a "main program" that accepts or *generates* test-case data, passes it to the module under test, and checks and prints the results. Here the driver must generate data sets of *different sizes* (0, 1, 2, small, large) and, to be thorough, of different *shapes* (random, already sorted, reverse sorted, all equal, many duplicates), because sorting bugs usually hide in these special cases.

Step 2 – The module under test (ascending sort)

```
/** Module under test: sorts double data in ascending order, in place. */
public class AscendingSorter {

    public static void sort(double[] a) {
        if (a == null)
            throw new IllegalArgumentException("data must not be null");
        // insertion sort: stable, simple, O(n^2) worst case
        for (int i = 1; i < a.length; i++) {
            double key = a[i];
            int j = i - 1;
```

```
            while (j >= 0 && a[j] > key) {      // shift larger elements right
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }
    }
}
```

Step 3 – The driver

```
import java.util.Arrays;
import java.util.Random;

/** Driver: generates data sets of different sizes/shapes, calls the module, checks, reports. */
public class SortDriver {

    private static final int[] SIZES = {0, 1, 2, 5, 10, 100, 1000, 10000};
    private static final Random RND = new Random(2015);      // fixed seed -> repeatable

    enum Shape { RANDOM, SORTED, REVERSED, ALL_EQUAL, FEW_DISTINCT }

    /** generates one test data set of size n and the given shape */
    static double[] generate(int n, Shape shape) {
        double[] a = new double[n];
        for (int i = 0; i < n; i++) {
            switch (shape) {
                case RANDOM:       a[i] = RND.nextDouble() * 2000 - 1000; break; // [-1000,1000)
                case SORTED:       a[i] = i * 0.5;                        break;
                case REVERSED:     a[i] = (n - i) * 0.5;                  break;
                case ALL_EQUAL:    a[i] = 3.14;                           break;
                case FEW_DISTINCT: a[i] = RND.nextInt(3) - 1.0;           break; // -1, 0, 1
            }
        }
        return a;
    }

    public static void main(String[] args) {
        int pass = 0, fail = 0;
        for (int n : SIZES) {
            for (Shape s : Shape.values()) {
                double[] input    = generate(n, s);
                double[] expected = input.clone();
                Arrays.sort(expected);                  // trusted reference (oracle)

                double[] actual = input.clone();
                long t0 = System.nanoTime();
                AscendingSorter.sort(actual);           // call the module under test
                long micros = (System.nanoTime() - t0) / 1000;

                boolean ok = Arrays.equals(expected, actual);
                if (ok) pass++; else fail++;
                System.out.printf("n=%6d  %-12s  %-4s  %9d us%n",
                                  n, s, ok ? "PASS" : "FAIL", micros);
            }
        }
        // invalid input must be rejected
        try {
            AscendingSorter.sort(null);
```

```
            System.out.println("null input       FAIL (no exception)"); fail++;
        } catch (IllegalArgumentException e) {
            System.out.println("null input       PASS"); pass++;
        }
        System.out.println("Summary: " + pass + " passed, " + fail + " failed");
    }
}
```

Sample (abridged) output:

```
n=     0  RANDOM        PASS          3 us
n=     1  SORTED        PASS          1 us
n=  1000  REVERSED      PASS       2140 us
n= 10000  FEW_DISTINCT  PASS      61432 us
null input       PASS
Summary: 41 passed, 0 failed
```

Step 4 – Why this driver is a good one

- **Sizes**: 0 and 1 (degenerate), 2 (smallest real comparison), small, and large sizes (also give a rough timing so the $O(n^2)$ growth becomes visible).
- **Shapes**: sorted / reversed are the best / worst cases of insertion sort; all-equal and few-distinct test the handling of duplicates; random gives general coverage.
- **Automatic verdict**: the result is compared with a trusted oracle (`Arrays.sort`) on a copy of the same data, so order *and* content are both verified; a fixed seed makes the run repeatable; `null` checks the error-handling path.

Answer. The module is an in-place ascending insertion sort that rejects `null`. The driver loops over sizes {0, 1, 2, 5, 10, 100, 1000, 10000} × five data shapes, generates each data set with a seeded `Random`, calls the module on a copy, compares with `Arrays.sort` and prints PASS/FAIL and the time, followed by a summary.

2012–2013 • Q4(a) CSE307 • 17/05/2014 • F024 **MARKS 6**

Suppose a project developed in C programming language. Propose four "Data faults" that should be checked during project inspection.

SOLUTION

Inspection checklist – the "data faults" class

A **program inspection** (Fagan inspection) is a formal code review in which the team examines the code against a **checklist of common errors**, organised by fault class (data faults, control faults, input/output faults, interface faults, storage-management faults, exception-management faults). *Data faults* concern how variables, constants, arrays and strings are declared and used. For a C project, four important data-fault checks are:

(i) Are all variables initialised before their values are used?
C does not initialise local variables. `int sum; for(...) sum += a[i];` uses garbage. Fix: `int sum = 0;`.

(ii) Have all constants been named?
Magic numbers such as `if (n > 86400)` should be `#define SECONDS_PER_DAY 86400` (or `const`), so the meaning is clear and the value changes in one place.

(iii) Should the upper bound of arrays be equal to the size of the array or size − 1?
`int a[10]; for (i = 0; i <= 10; i++) a[i] = 0;` writes `a[10]` – an off-by-one out-of-bounds write. The loop must use `i < 10`.

(iv) If character strings are used, is a delimiter (\0) explicitly assigned, and is there any possibility of buffer overflow?
`char s[5]; strncpy(s, "hello", 5);` leaves `s` without a terminating `\0`; `strcpy`/`gets` into a fixed buffer can overflow. Fix: reserve space for the terminator and use bounded functions such as `snprintf`.

```c
/* data faults an inspector should flag */
int total(const int a[10]) {
    int sum;                          /* (1) not initialised         */
    for (int i = 0; i <= 10; i++)     /* (2) magic 10, (3) <= bound   */
        sum += a[i];
    return sum;
}
char name[5];
strncpy(name, "hello", 5);            /* (4) no '\0' terminator       */
```

Answer. Data faults to check in a C inspection: (1) variables used before initialisation, (2) unnamed "magic" constants, (3) wrong array bounds (size vs. size−1, off-by-one), (4) strings without an explicit `'\0'` delimiter / possible buffer overflow.

10

Code Refactoring

2023–2024 • Q4(a) CSE213 • 10/05/2026 • F001 **MARKS 20**

Identify 5 (five) bad smells present in the following code snippet and perform refactoring for each of them.

```
class Student {
   String name, id;
   int type;           // 1 = UG, 2 = PG
   int year;
   String street, post office, district;
   float[] yearWiseGpa;
   Bool isFulltime;  // For PG

   // constructors ...

   float calculateFee() {
      if (type == 1) {              // UG
         return 50000;
      } else if (type == 2) {       // PG
         float sum = 0;
         for (float gpa : yearWiseGpa) sum += gpa;
         float cgpa = sum / yearWiseGpa.length;
         if (cgpa > 3.5) return 40000;
         else            return 48000;
      }
      return 0;
   }

   void printDetails() {
      // print common details
      System.out.println("Name:" + name);
      System.out.println("Student ID:" + Id);
      System.out.println("Address: " + street + ", " + postOffice + ", " + district);
      ...
      ...
      if (type == 1)
         // Prints UG student-specific details
      else {
         // Prints PG student-specific details
      }
   }
}

class Scholarship {
   String name;
   int   amount;
   float  CgpaThreshold;

   // constructors ...
```

```
    boolean isRegular(Student student) {
       // Returns true if the student has no gaps in yearly GPA records
       return student.yearWiseGpa.length == student.year;
    }

    void checkEligibility(Student student) {
    if (stuent.type == 1)
    {
       float sum = 0;
       for (float gpa : yearWiseGpa) sum += gpa;
       float cgpa = sum / yearWiseGpa.length;

       if (isRegular(student)) {
          if (student.year >= 2) {
             if (cgpa >= CgpaThreshold) {
                System.out.println(student.name + " is awarded scholarship: " + amount);
             } else {
                System.out.println(student.name + " is not eligible..");
             }
          } else {
             System.out.println(student.name + " is not eligible.");
          }
       } else {
          System.out.println(student.name + " is not eligible.");
       }
    }
}
```

SOLUTION

> **NOTE** The code is transcribed exactly as printed, including its slips: `post office` (should be `postOffice`), `Bool (boolean)`, `Id (id)`, `stuent (student)`, `yearWiseGpa` used inside `Scholarship` (should be `student.yearWiseGpa`), and the missing closing brace of `Scholarship`. We treat these as typos and refactor the *intended* program. The two messages `" is not eligible.."` and `" is not eligible."` are treated as the same message (the extra dot is a typo).

Step 1 — Reading the code: what goes wrong?

- A numeric **type code** (`1 = UG, 2 = PG`) is tested with `if` in three different methods (`calculateFee`, `printDetails`, `checkEligibility`). Adding a third kind of student (e.g. PhD) needs edits in all three places.
- The **CGPA formula** is written twice (in `Student.calculateFee` and in `Scholarship.checkEligibility`).
- `Scholarship` reaches into `Student`'s fields (`yearWiseGpa`, `year`, `type`, `name`) more than it uses its own data.
- `street, postOffice, district` always travel together but are three loose `Strings`.
- `checkEligibility` is a long method with a 4-level "arrow" of nested `if`s and three copies of the "not eligible" print.

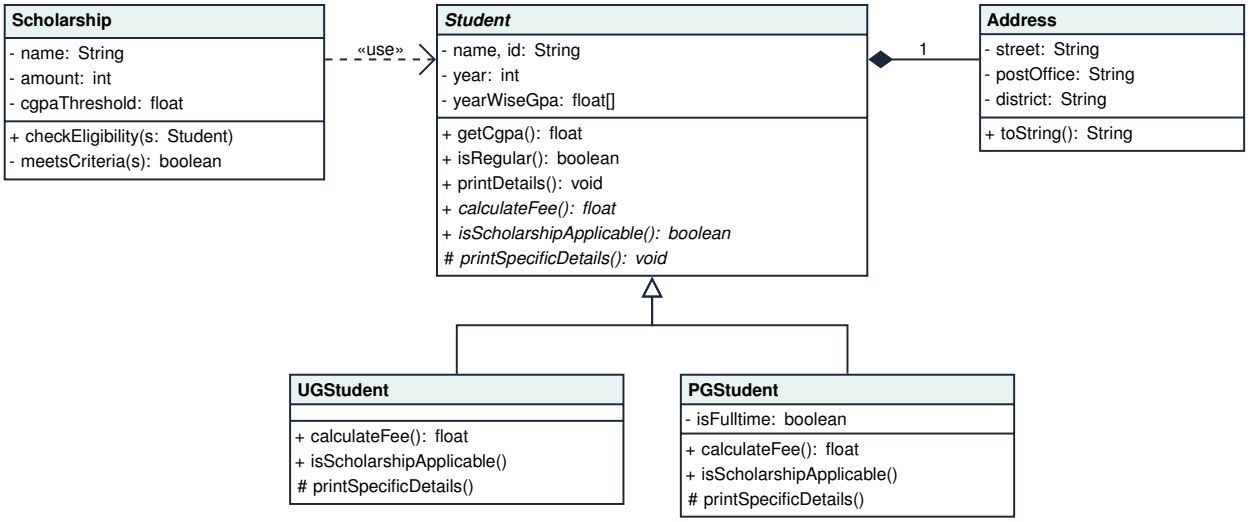
Step 2 — The five smells and their refactorings

Smell (slides)	Location	Why it is a smell	Refactoring applied
1. Switch Statements (type code)	`int type` + `if (type == 1/2)` in `calculateFee`, `printDetails`, `checkEligibility`	Same selection on a type code is duplicated; every new student kind forces changes in many methods (violates OCP); `isFulltime` is meaningful only for PG	**Replace Conditional with Polymorphism** (type code → subclasses `UGStudent`, `PGStudent`); push `isFulltime` down to `PGStudent`
2. Duplicated Code	CGPA loop in `calculateFee` and `checkEligibility`; "not eligible" print ×3	A change to the CGPA rule must be made twice; copies can drift apart	**Extract Method** `getCgpa()`; one "not eligible" branch
3. Feature Envy	`Scholarship.isRegular()` and the CGPA computation in `checkEligibility`	These use only `Student`'s data (`yearWiseGpa`, `year`) and none of `Scholarship`'s	**Move Method**: `student.isRegular()`, `student.getCgpa()`
4. Data Clumps	`String street, postOffice, district`	Three fields always used together (printed together) but not an object; address behaviour has no home	**Extract Class** `Address` (with `toString()`)
5. Long Method with nested conditionals	`Scholarship.checkEligibility()`	Computation, three nested decisions and printing are mixed; the normal path is hidden in an arrow of `if`s	**Extract Method** `meetsCriteria()` + **Replace Nested Conditional with Guard Clauses**

Further smells fixed on the way (worth mentioning for full marks):

- **Magic numbers** `50000, 40000, 48000, 3.5, 2` → **Replace magic number with symbolic constant**.
- **Inappropriate Intimacy**: `Scholarship` reads `Student`'s package-visible fields directly → make fields `private` and expose only intention-revealing methods (Encapsulate Field).
- **Comments** as deodorant (`// 1 = UG, 2 = PG`, `// Returns true if ...`) → the class and method names now say it.

Step 3 — Design after refactoring



Step 4 — Refactored code (complete)

```
// ---- Smell 4: Data Clumps  ->  Extract Class ------------------------------
class Address {
    private final String street, postOffice, district;

    Address(String street, String postOffice, String district) {
        this.street = street;
        this.postOffice = postOffice;
        this.district = district;
    }

    @Override
    public String toString() {                  // same text as before
        return street + ", " + postOffice + ", " + district;
    }
}

// ---- Smell 1: type code -> Replace Conditional with Polymorphism ----------
abstract class Student {
    private final String name, id;
    private final int year;
    private final Address address;
    private final float[] yearWiseGpa;

    protected Student(String name, String id, int year,
                      Address address, float[] yearWiseGpa) {
        this.name = name;
        this.id = id;
        this.year = year;
        this.address = address;
        this.yearWiseGpa = yearWiseGpa;
    }

    String getName() { return name; }
    int    getYear() { return year; }

    // Smell 2 + 3: Extract Method, then Move Method (formula now exists once)
    float getCgpa() {
        float sum = 0;
        for (float gpa : yearWiseGpa) sum += gpa;
```

```
        return sum / yearWiseGpa.length;
    }

    // Smell 3: Move Method (was Scholarship.isRegular(student))
    boolean isRegular() {                       // no gaps in yearly GPA records
        return yearWiseGpa.length == year;
    }

    abstract float   calculateFee();
    abstract boolean isScholarshipApplicable();   // replaces "type == 1"
    protected abstract void printSpecificDetails();

    final void printDetails() {                 // common part written once
        System.out.println("Name:" + name);
        System.out.println("Student ID:" + id);
        System.out.println("Address: " + address);
        // ... other common details ...
        printSpecificDetails();                 // polymorphic, no "if (type == 1)"
    }
}

class UGStudent extends Student {
    private static final float UG_FEE = 50000;

    UGStudent(String name, String id, int year, Address address, float[] gpa) {
        super(name, id, year, address, gpa);
    }
    @Override float   calculateFee()            { return UG_FEE; }
    @Override boolean isScholarshipApplicable() { return true; }
    @Override protected void printSpecificDetails() {
        // prints UG student-specific details
    }
}

class PGStudent extends Student {
    private static final float HIGH_CGPA     = 3.5f;
    private static final float REDUCED_FEE   = 40000;
    private static final float STANDARD_FEE  = 48000;

    private final boolean isFulltime;          // pushed down: PG-only data

    PGStudent(String name, String id, int year, Address address,
              float[] gpa, boolean isFulltime) {
        super(name, id, year, address, gpa);
        this.isFulltime = isFulltime;
    }
    @Override float calculateFee() {
        return getCgpa() > HIGH_CGPA ? REDUCED_FEE : STANDARD_FEE;
    }
    @Override boolean isScholarshipApplicable() { return false; }
    @Override protected void printSpecificDetails() {
        // prints PG student-specific details (e.g. isFulltime)
    }
}

// ---- Smell 5: Long Method / nested conditionals -> guard clauses -----------
class Scholarship {
    private static final int MIN_YEAR = 2;
```

```
    private final String name;
    private final int amount;
    private final float cgpaThreshold;

    Scholarship(String name, int amount, float cgpaThreshold) {
        this.name = name;
        this.amount = amount;
        this.cgpaThreshold = cgpaThreshold;
    }

    void checkEligibility(Student student) {
        if (!student.isScholarshipApplicable()) return;          // guard: PG
        if (!meetsCriteria(student)) {                            // guard: fails
            System.out.println(student.getName() + " is not eligible.");
            return;
        }
        System.out.println(student.getName()
                + " is awarded scholarship: " + amount);          // normal path
    }

    private boolean meetsCriteria(Student student) {              // Extract Method
        return student.isRegular()
            && student.getYear() >= MIN_YEAR
            && student.getCgpa() >= cgpaThreshold;
    }
}
```

Step 5 — Why the observable behaviour is unchanged

- **Fee.** UG → `50000` (same constant). PG → the same CGPA formula (moved, not changed) compared with `3.5` gives `40000` or `48000`. The old "`return 0`" for an invalid type code cannot occur any more: an object is now created as a `UGStudent` or `PGStudent`, so an invalid type is rejected at construction time instead of silently charging 0.
- **Eligibility.** The old nested `if`s award the scholarship only when $regular \wedge year \geq 2 \wedge cgpa \geq threshold$; every other path prints "not eligible". `meetsCriteria()` is exactly that conjunction, so each input still gets the same message. PG students printed nothing before (the outer `if (type == 1)` had no `else`); the first guard clause returns silently for them — same output.
- **Order/side effects.** `getCgpa()` and `isRegular()` are pure queries, so computing them lazily inside `&&` instead of up-front changes nothing visible.
- **Printing.** `Address.toString()` builds the identical `"street, postOffice, district"` string; common lines are printed before the type-specific part, as before.
- **Safety net.** Following the slides, each step (Extract Class, Extract Method, Move Method, subclasses, guard clauses) is done separately and the unit tests (fee for UG, PG high/low CGPA; eligibility for each branch) are re-run after every step.

Answer. Smells: (1) Switch Statements on type code → Replace Conditional with Polymorphism (`UGStudent`/`PGStudent`); (2) Duplicated Code (CGPA loop) → Extract Method `getCgpa()`; (3) Feature Envy (`isRegular`, CGPA in `Scholarship`) → Move Method to `Student`; (4) Data Clumps (street, postOffice, district) → Extract Class `Address`; (5) Long Method with nested ifs → Extract Method + Replace Nested Conditional with Guard Clauses. Also: magic numbers → named constants, public fields → encapsulated.

2023–2024 • Q4(b) CSE213 • 10/05/2026 • F002.2 **MARKS 15**

Define code smell, tech-debt and code refactoring. *[Differentiate between proactive and reactive risk management strategies with examples.]*

SOLUTION

NOTE This excerpt has two independent tasks. The definitions (code smell, technical debt, refactoring) are answered here. The second sentence (proactive vs. reactive *risk management*) belongs to project risk management, which is outside this course's topic list; it is answered in the Out-of-Syllabus chapter under the same ID (F002.2).

1. Code smell

A **code smell** is a characteristic of source code that indicates a possible deeper problem in its design. The code works, but its structure makes it hard to read, extend or modify safely. Typical smells from the lecture: Duplicated Code, Long Method, Large Class, Long Parameter List, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Primitive Obsession, Switch Statements, Lazy Class, Message Chains, Middle Man, Inappropriate Intimacy, Data Class, Refused Bequest and Comments used as "deodorant".

2. Technical debt (tech-debt)

Technical debt is a metaphor (Ward Cunningham) for the *future cost* that a team takes on when it chooses a quick-and-dirty solution today instead of a cleaner one that would take longer.

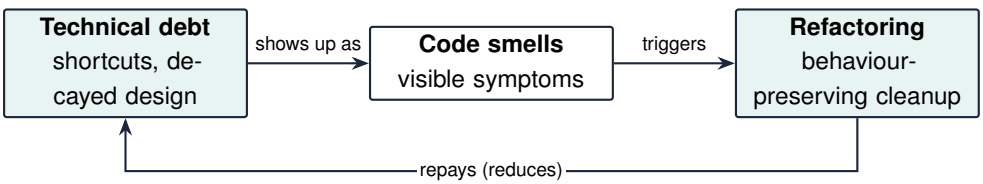
- **Principal** — the work needed to clean the code up (the refactoring that was skipped).
- **Interest** — the extra effort paid on *every* later change because the code is messy: slower feature development, more bugs, longer onboarding.

Debt may be *deliberate* ("ship now, fix next sprint") or *accidental* (design decays as people change the system — the slides' "a design will decay"). Like financial debt it is acceptable in moderation, but if never repaid the interest eventually makes change practically impossible. Code smells are the *visible symptoms* of technical debt.

3. Code refactoring

Fowler's definition (slides): **Refactoring** (noun) is *a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behaviour*; to refactor (verb) is *to restructure software by applying a series of refactorings without changing its observable behaviour* — "improving the design after it has been written". It must be systematic, incremental and safe: use named refactorings (Extract Method, Move Method, Replace Conditional with Polymorphism, …) and re-run the tests after every small step.

How the three ideas relate



	Code smell	Technical debt	Refactoring
Nature	a symptom in code	an accumulated cost/liability	an activity (a fix)
Example	a 120-line `processOrder()`	every change to it takes a day	Extract Method into 5 small methods
Effect on behaviour	none (code still works)	none now; slows future work	none — behaviour preserved

Answer. Code smell = surface symptom of poor design in working code; technical debt = the accumulated future cost of such shortcuts (principal + interest); refactoring = restructuring the code without changing its observable behaviour, which removes smells and pays the debt back.

2023–2024 • Q8(c) CSE213 • 09/02/2025 • F003 MARKS 20

Define bad smells in code and code refactoring. Identify the bad smells present in the provided code below and revise it with appropriate refactoring techniques.

```
class DiscountCalculator {
    public double calculateDiscount(String customerType, String
productType, double price, String customerName, String customerId) {

        System.out.println("Customer Name: " + customerName);
        System.out.println("Customer ID: " + customerId);

        if (customerType.equals("REGULAR") &&
productType.equals("ELECTRONICS")) {
            return price * 0.1; // 10% discount
        } else if (customerType.equals("REGULAR") &&
productType.equals("GROCERY")) {
            return price * 0.05; // 5% discount
        } else if (customerType.equals("PREMIUM") &&
productType.equals("ELECTRONICS")) {
            return price * 0.2; // 20% discount
        } else if (customerType.equals("PREMIUM") &&
productType.equals("GROCERY")) {
            return price * 0.1; // 10% discount
        } else {
            return 0;
        }
    }
}

class OrderProcessor {
    public void processOrder(String customerType, String productType,
double price, String customerName, String customerId) {
        System.out.println("Processing Order for Customer: " +
customerName);
        System.out.println("Customer ID: " + customerId);

        DiscountCalculator discountCalculator = new
DiscountCalculator();
        double discount =
discountCalculator.calculateDiscount(customerType, productType,
```

```
price, customerName, customerId);

        if (customerType.equals("REGULAR")) {
            System.out.println("Regular customer processed");
        } else if (customerType.equals("PREMIUM")) {
            System.out.println("Premium customer processed");
        }

        if (productType.equals("ELECTRONICS")) {
            System.out.println("Electronics item processed");
        } else if (productType.equals("GROCERY")) {
            System.out.println("Grocery item processed");
        } else if (customerType.equals("PREMIUM")) {
            System.out.println("Other item processed");
        }
        System.out.println("Customer Name: " + customerName);
        System.out.println("Customer Type: " + customerType);
        System.out.println("Customer ID: " + customerID);

        System.out.println("Product Type: " + productType);
        System.out.println("Original Price: " + price);
        System.out.println("Discount: " + discount);
        System.out.println("Final Price: " + price - discount);

        if (discount == 0) {
            System.out.println("No discount applied");
        } else if (discount == price * 0.1) {
            System.out.println("10% discount applied");
        } else if (discount == price * 0.15) {
            System.out.println("15% discount applied");
        } else if (discount == price * 0.25) {
            System.out.println("25% discount applied");
        }
    }
}
```

SOLUTION

NOTE As printed, `OrderProcessor` does not compile: `customerID` is undeclared (the parameter is `customerId`) and `"Final Price:  " + price - discount` tries to subtract a `double` from a `String`. We assume the intended meaning (`customerId` and `price - discount` in parentheses). Line breaks inside long statements are from the paper's narrow column.

Definitions

Bad smell in code: a surface indication in working code that usually corresponds to a deeper design problem — e.g. duplicated code, long methods, long parameter lists, type-switching conditionals. It is not a bug, but it makes the code hard to understand and costly to change; smells tell us *where* to refactor.

Code refactoring: "a change made to the internal structure of software to make it easier to understand and cheaper to modify *without changing its observable behaviour*" (Fowler). It is done in small, named, test-protected steps.

Step 1 — Smells found

Smell	Location	Why it is a smell	Refactoring
Long Parameter List	`calculateDiscount()` and `processOrder()`, 5 params each	Hard to read/call; two adjacent `Strings` can be swapped; same chain passed method to method	**Introduce Parameter Object**
Data Clumps	`(customerType, customerName, customerId)` and `(productType, price)`	Always travel together but are not objects	**Extract Class** `Customer`, `Product`
Primitive Obsession	types as raw `Strings`, compared with `equals("REGULAR")`	Typos compile; no place for type-specific data	Replace type code with a class (`enum`)
Switch Statements	5-branch `if/else` on customer × product in `DiscountCalculator`; customer- and product-type chains in `processOrder`	Type selection duplicated; a new customer/product type means editing several chains	**Replace Conditional with Polymorphism** (each enum constant carries its own rates/messages)
Duplicated Code	Name/ID printed in both classes (ID 3 times); rate percentages known in both classes	Copies must be changed together	**Extract Method**; single rate table
Shotgun Surgery	changing a rate needs edits in `calculateDiscount` *and* the `discount == price * x` chain	One business change → many small edits in different classes	**Move Method**/Move Field: rates live in one place
Query with side effect	`calculateDiscount` prints customer info	A "calculate" method secretly does I/O; cannot reuse it silently	**Separate Query for Modifier**
Long Method	`processOrder` (header, discount, logging, summary, message)	Many responsibilities in one block	**Extract Method**
Magic numbers / Comments	`0.1, 0.05, 0.2` with `// 10% discount`	Comments explain cryptic literals	Replace magic number with symbolic constant (named rate table)

Step 2 — Refactored code

```java
import java.util.Map;

// ---- Primitive Obsession -> enum; Switch Statements -> data on each constant ----
enum ProductType {
    ELECTRONICS("Electronics item processed"),
    GROCERY("Grocery item processed"),
    OTHER("Other item processed");

    private final String processedNote;
```

```
    ProductType(String processedNote) { this.processedNote = processedNote; }
    String processedNote() { return processedNote; }

    static ProductType from(String code) {
        for (ProductType t : values())
            if (t != OTHER && t.name().equals(code)) return t;
        return OTHER;
    }
}

enum CustomerType {
    REGULAR("Regular customer processed",
            Map.of(ProductType.ELECTRONICS, 0.10, ProductType.GROCERY, 0.05)),
    PREMIUM("Premium customer processed",
            Map.of(ProductType.ELECTRONICS, 0.20, ProductType.GROCERY, 0.10)),
    OTHER(null, Map.of());                      // unknown type: no note, no discount

    private final String processedNote;
    private final Map<ProductType, Double> discountRates;   // the ONE rate table

    CustomerType(String note, Map<ProductType, Double> rates) {
        this.processedNote = note;
        this.discountRates = rates;
    }
    String processedNote() { return processedNote; }
    double discountRateFor(ProductType p) { return discountRates.getOrDefault(p, 0.0); }

    static CustomerType from(String code) {
        for (CustomerType t : values())
            if (t != OTHER && t.name().equals(code)) return t;
        return OTHER;
    }
}

// ---- Data Clumps -> Extract Class / Introduce Parameter Object -----------------
class Customer {
    private final String name, id, typeCode;
    Customer(String name, String id, String typeCode) {
        this.name = name; this.id = id; this.typeCode = typeCode;
    }
    String getName()       { return name; }
    String getId()         { return id; }
    String getTypeCode()   { return typeCode; }
    CustomerType getType() { return CustomerType.from(typeCode); }
}

class Product {
    private final String typeCode;
    private final double price;
    Product(String typeCode, double price) { this.typeCode = typeCode; this.price = price; }
    String getTypeCode()  { return typeCode; }
    double getPrice()     { return price; }
    ProductType getType() { return ProductType.from(typeCode); }
}

// ---- Separate Query for Modifier: pure calculation, no printing --------------
class DiscountCalculator {
    double discountRate(Customer c, Product p) {
```

```
        return c.getType().discountRateFor(p.getType());
    }
    double calculateDiscount(Customer c, Product p) {
        return p.getPrice() * discountRate(c, p);
    }
}

// ---- Long Method -> Extract Method ------------------------------------------
class OrderProcessor {
    private static final Map<Double, String> DISCOUNT_NOTES = Map.of(
            0.10, "10% discount applied",
            0.15, "15% discount applied",
            0.25, "25% discount applied");

    private final DiscountCalculator discountCalculator = new DiscountCalculator();

    public void processOrder(Customer customer, Product product) {
        printHeader(customer);
        double rate     = discountCalculator.discountRate(customer, product);
        double discount = discountCalculator.calculateDiscount(customer, product);
        printProcessingNotes(customer, product);
        printSummary(customer, product, discount);
        printDiscountNote(discount, rate);
    }

    private void printHeader(Customer c) {
        System.out.println("Processing Order for Customer: " + c.getName());
        System.out.println("Customer ID: " + c.getId());
    }

    private void printProcessingNotes(Customer c, Product p) {
        if (c.getType().processedNote() != null)
            System.out.println(c.getType().processedNote());
        // kept exactly as the original: "Other item" note only for PREMIUM
        if (p.getType() != ProductType.OTHER || c.getType() == CustomerType.PREMIUM)
            System.out.println(p.getType().processedNote());
    }

    private void printSummary(Customer c, Product p, double discount) {
        System.out.println("Customer Name: " + c.getName());
        System.out.println("Customer Type: " + c.getTypeCode());
        System.out.println("Customer ID: " + c.getId());
        System.out.println("Product Type: " + p.getTypeCode());
        System.out.println("Original Price: " + p.getPrice());
        System.out.println("Discount: " + discount);
        System.out.println("Final Price: " + (p.getPrice() - discount));
    }

    private void printDiscountNote(double discount, double rate) {
        if (discount == 0) {
            System.out.println("No discount applied");
            return;                                  // guard clause
        }
        String note = DISCOUNT_NOTES.get(rate);
        if (note != null) System.out.println(note);
    }
}
```

```
// client:  new OrderProcessor().processOrder(
//              new Customer("Rahim", "C-17", "PREMIUM"),
//              new Product("ELECTRONICS", 1000.0));
```

Step 3 — How behaviour is preserved

- **Discount value.** For each (customer, product) pair the rate table holds exactly the old literal (`0.10, 0.05, 0.20, 0.10`); every other combination — including unknown type strings — maps to `OTHER/getOrDefault(..., 0.0)`, i.e. discount 0 as in the old `else return 0`. The arithmetic `price * rate` is the same expression.
- **Type matching.** `from(code)` uses `name().equals(code)`, the same case-sensitive comparison as `customerType.equals("REGULAR")`.
- **Processing notes.** Regular/Premium notes print as before and nothing prints for other customer types (`null` note). The odd original rule "*Other item processed* only when the customer is PREMIUM" is preserved literally.
- **Discount note.** The old test `discount == price * 0.1` is true exactly when the (non-zero) discount was computed with rate `0.1`, so looking up the rate in `DISCOUNT_NOTES` prints the same line; the zero test comes first as before.
- **Summary lines** keep the same text and order; `typeCode` keeps the raw string the user passed, so "Customer Type: ..." prints identically.
- **Only intentional difference:** the two lines "Customer Name/ID" that the old `calculateDiscount` printed as a hidden side effect are gone (Separate Query for Modifier); the same information is printed in the header and the summary. If byte-for-byte identical console output is required, add one call `printCustomerIdentity(customer)` before computing the discount.

EXAM TIP Refactoring must not change behaviour, so suspicious logic is *kept* and fixed later as a separate, tested change: here the notes "15%"/"25% discount applied" can never occur while 5% and 20% discounts get no note, and "Other item processed" depends on the *customer* type. Point such latent bugs out in the exam — it earns credit.

Answer. Smells: Long Parameter List, Data Clumps, Primitive Obsession, Switch Statements, Duplicated Code, Shotgun Surgery, query with side effects, Long Method, magic numbers. Refactorings: Introduce Parameter Object/Extract Class (`Customer`, `Product`), type strings → enums with a single rate table (Replace Conditional with Polymorphism), Separate Query for Modifier, Extract Method, named constants.

2022–2023 • Q5(a)+(b) CSE307 • 05/05/2024 • F006 **MARKS 15 + 20**

(a) Define each of the following code smells with short code examples: (4+4+4+3=15)

(i) Shotgun Surgery

(ii) Parallel Inheritance Hierarchies

(iii) Refused Bequest

(iv) Feature Envy

(b) Identify which refactoring techniques you will apply to refactor the following set of codes.

Construct the refactored codes. (6+7+7=20)

i)

```
void renderBanner() {
  if ((platform.toUpperCase().indexOf("MAC") > -1) &&
     (browser.toUpperCase().indexOf("IE") > -1) &&
      wasInitialized() && resize > 0 )
  {
    // do something
  }
}
```

ii)

```
double getSalary() {
    switch (type) {
      case "Football":
        return baseSalary() + getWeeklyIncrement() * 4;
      case "Cricket":
        return baseSalary() + getMonthlyIncrement();
      case "Badminton":
        return baseSalary();
    }
}
```

iii)

```
double getPayAmount() {
  double result;
  if (isDead) {
    result = deadAmount();
  }
  else {
    if (isSeparated) {
      result = separatedAmount();
    }
    else {
      if (isRetired){
        result = retiredAmount();
      }
      else {
        result = normalPayAmount();
      }
    }
  }
  return result;
}
```

SOLUTION

(a)(i) Shotgun Surgery

Definition (slides): a single change requires lots of little changes in a lot of different classes. The responsibility is scattered, so a change is easy to miss somewhere.

```
class Cart     { boolean canCheckout(double t) { return t >= 500; } }
class Checkout { void validate(double t) {
                     if (t < 500) throw new IllegalStateException("Minimum order is 500"); } }
class Invoice  { String note(double t) { return t < 500 ? "Minimum order is 500" : ""; } }
// Business raises the minimum to 700 -> three classes (and wording) must all be edited.
```

Cure: **Move Method**/Move Field to gather the rule in one class, e.g. `OrderPolicy.MIN_ORDER` and `OrderPolicy.isBelowMinimum(t)`. (Opposite of Divergent Change: there one class changes for many reasons; here one reason changes many classes.)

(a)(ii) Parallel Inheritance Hierarchies

Definition: two class hierarchies mirror each other, so every time you add a subclass to one hierarchy you are forced to add a matching subclass to the other. It is a special case of Shotgun Surgery; the class-name prefixes usually match.

```
abstract class Shape              { }
class Circle  extends Shape       { double r; }
class Square  extends Shape       { double side; }

abstract class ShapeRenderer      { abstract void render(Shape s); }
class CircleRenderer extends ShapeRenderer { void render(Shape s) { /* draw circle */ } }
class SquareRenderer extends ShapeRenderer { void render(Shape s) { /* draw square */ } }
// Adding Triangle forces adding TriangleRenderer as well.
```

Cure: make instances of one hierarchy refer to the other and then **Move Method**/Move Field so one hierarchy disappears, e.g. give `Shape` an abstract `render(Canvas c)` implemented by `Circle` and `Square`.

(a)(iii) Refused Bequest

Definition (slides): a subclass ignores most of the functionality provided by its superclass; it may not pass the IS-A test.

```
class Employee {
    double salary()       { ... }
    double overtimePay()  { ... }
    void   assignTeam()   { ... }
}
class Contractor extends Employee {              // refuses what it inherits
    @Override double overtimePay() { throw new UnsupportedOperationException(); }
    @Override void   assignTeam()  { /* does nothing */ }
}
```

Cure: **Push Down Method** (move `overtimePay`, `assignTeam` to a `PermanentEmployee` subclass) or **Replace Inheritance with Delegation** if `Contractor` only needs a little of `Employee`.

(a)(iv) Feature Envy

Definition (slides): a method requires lots of information from some other class — it is more interested in another class's data than its own. "Move it closer!"

```
class Invoice {
    double calculateDiscount(Customer c) {      // uses only Customer's data
        if (c.getTotalSpent() > 1000 && c.getYearsAsMember() > 2)
            return c.getTotalSpent() * 0.15;
        return 0.0;
    }
}
```

Cure: **Move Method** — put `discount()` in `Customer`; `Invoice` calls `customer.discount()`.

(b)(i) `renderBanner()` — Extract Variable

Smell: a long, cryptic boolean expression; the reader must decode `indexOf(...) > -1` three times to understand the intent ("Mac + Internet Explorer + initialised + resized").
Refactoring: **Extract Variable** (slides: "place the result of the expression or its parts in separate

variables that are self-explanatory"), plus Substitute Algorithm `indexOf(x) > -1` → `contains(x)` (identical semantics).

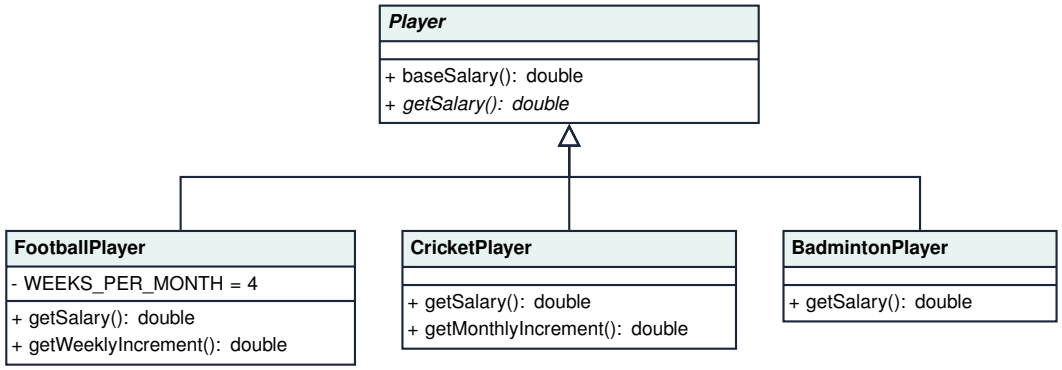
```
void renderBanner() {
    final boolean isMacOs     = platform.toUpperCase().contains("MAC");
    final boolean isIE        = browser.toUpperCase().contains("IE");
    final boolean wasResized  = resize > 0;

    if (isMacOs && isIE && wasInitialized() && wasResized) {
        // do something
    }
}
```

Behaviour. The condition is logically identical. `wasInitialized()` is kept inside the `if` so it is still called only when the first two tests pass (it might have side effects). The string tests are now evaluated eagerly; that is harmless because `toUpperCase()` and `contains()` have no side effects. (If `browser` could be `null` on non-Mac platforms, use **Extract Method** — `isMacOs()`, `isInternetExplorer()` — instead, which keeps short-circuit evaluation.)

(b)(ii) `getSalary()` — Replace Conditional with Polymorphism

Smell: Switch Statements on a type code (`type` string). The same switch tends to be repeated (bonus, schedule, …), and adding "Hockey" means editing every switch. Also a magic number `4` (weeks per month), and the method has no `return`/`default` after the switch (Java rejects it: missing return statement).
Refactoring: **Replace Conditional with Polymorphism** (after replacing the type code with subclasses): move each *leg* of the switch into an overriding method of a subclass and make the original method abstract; **Replace magic number with symbolic constant**.




```
abstract class Player {
    double baseSalary() { /* unchanged */ return 0; }
    abstract double getSalary();

    static Player of(String type) {            // the ONLY remaining switch (creation)
        switch (type) {
            case "Football":  return new FootballPlayer();
            case "Cricket":   return new CricketPlayer();
            case "Badminton": return new BadmintonPlayer();
            default: throw new IllegalArgumentException("Unknown sport: " + type);
        }
    }
}

class FootballPlayer extends Player {
```

```
    private static final int WEEKS_PER_MONTH = 4;
    double getWeeklyIncrement() { /* unchanged */ return 0; }
    @Override double getSalary() {
        return baseSalary() + getWeeklyIncrement() * WEEKS_PER_MONTH;
    }
}

class CricketPlayer extends Player {
    double getMonthlyIncrement() { /* unchanged */ return 0; }
    @Override double getSalary() { return baseSalary() + getMonthlyIncrement(); }
}

class BadmintonPlayer extends Player {
    @Override double getSalary() { return baseSalary(); }
}
```

Behaviour. For each of the three sports the formula is copied unchanged into the corresponding subclass, so `Player.of("Cricket").getSalary()` returns what `getSalary()` returned when `type` was `"Cricket"`. A new sport is now a new subclass (Open–Closed Principle) and no other method changes.

(b)(iii) `getPayAmount()` — Replace Nested Conditional with Guard Clauses

Smell: deeply nested `if/else` ("arrow code") caused by a single-return style; the *normal* case (`normalPayAmount()`) is buried at the deepest level, and dead/separated/retired are really *special cases*, not equal alternatives.
Refactoring: Replace Nested Conditional with Guard Clauses (Fowler: "use guard clauses for all the special cases"), which also removes the temporary `result`.

```
double getPayAmount() {
    if (isDead)      return deadAmount();       // guard clauses: special cases
    if (isSeparated) return separatedAmount();
    if (isRetired)   return retiredAmount();
    return normalPayAmount();                   // normal path, clearly visible
}
```

Behaviour. The flags are tested in the same order (dead → separated → retired) and the same method is called for each combination; e.g. if both `isDead` and `isRetired` are true, both versions return `deadAmount()`. Four returns, yet far easier to read.

Answer. (a) Shotgun Surgery = one change → edits in many classes; Parallel Inheritance Hierarchies = adding a subclass in one hierarchy forces one in another; Refused Bequest = subclass rejects inherited behaviour; Feature Envy = method uses another class's data more than its own. (b)(i) Extract Variable; (ii) Replace Conditional with Polymorphism (+ named constant); (iii) Replace Nested Conditional with Guard Clauses.

2021–2022 • Q1(b) CSE307 • 07/05/2023 • F008 **MARKS 13**

Your younger brother who wanted to build an *Avengers* themed game. For his game he wrote a method which takes a Super Hero's name as input and returns the Super Hero item used by that Super Hero. He wrote the method as follows:

```
class GetSuperHeroItem {
    public SuperHeroItem suit;
    public SuperHeroItem hammer;
    public SuperHeroItem shield;
    public SuperHeroItem arrow;
```

```
    public SuperHeroItem getItem(String heroName) {
        if (heroName.equalsIgnoreCase("IronMan")) {
            return suit;
        }
        else if (heroName.equalsIgnoreCase("Thor")) {
            return hammer;
        }
        else if (heroName.equalsIgnoreCase("CaptainAmerica")) {
            return shield;
        }
        else if (heroName.equalsIgnoreCase("Hawkeye")) {
            return arrow;
        }
        else {
            return null ;
```

You came to know about his little project and saw this piece of code. You know a better way to implement the functionality he wanted. Now, re-write the code above in such a way that would be much easier to read and maintain, also your method should have constant run-time (amortized).

SOLUTION

NOTE The listing in the paper stops after `return null ;` (the closing braces are cut off in the original as well); we assume the obvious `} } }`.

Smells in the given code

Smell	Why it is a smell	Refactoring
Switch Statements (long `if/else-if` chain on a name)	Every new hero = new field + new branch (Shotgun Surgery inside the class); lookup is a linear scan, $O(k)$ comparisons for k heroes	Replace the conditional by a **lookup table** (`HashMap`) — Substitute Algorithm
Inappropriate Intimacy / public fields	`public SuperHeroItem suit, ...` — anyone can overwrite them	Encapsulate Field (private map + `register()`)
Duplicated Code	`heroName.equalsIgnoreCase(...)` repeated 4 times	normalise the key once (Extract Method `key()`)
Poor name	a class named like a method (`GetSuperHeroItem`)	Rename Class `SuperHeroItemRegistry`
Returning `null`	every caller must write `if (item != null)`	(optional) **Introduce Null Object**

Refactored code

```
import java.util.HashMap;
import java.util.Locale;
import java.util.Map;

class SuperHeroItemRegistry {
    private final Map<String, SuperHeroItem> itemsByHero = new HashMap<>();

    SuperHeroItemRegistry(SuperHeroItem suit, SuperHeroItem hammer,
                          SuperHeroItem shield, SuperHeroItem arrow) {
```

```
        register("IronMan",        suit);
        register("Thor",           hammer);
        register("CaptainAmerica", shield);
        register("Hawkeye",        arrow);
    }

    /** New heroes are added with one line - no new field, no new if-branch. */
    public void register(String heroName, SuperHeroItem item) {
        itemsByHero.put(key(heroName), item);
    }

    /** O(1) amortized: one hash computation + one bucket probe. */
    public SuperHeroItem getItem(String heroName) {
        return itemsByHero.get(key(heroName));     // null when hero is unknown (as before)
    }

    private static String key(String heroName) {   // case-insensitive like equalsIgnoreCase
        return heroName.toLowerCase(Locale.ROOT);
    }
}

// usage
SuperHeroItemRegistry registry = new SuperHeroItemRegistry(suit, hammer, shield, arrow);
SuperHeroItem item = registry.getItem("thor");     // -> hammer
```

Optional further step — Introduce Null Object (slides): instead of `null`, return a do-nothing item so callers need no null checks:

```
public SuperHeroItem getItem(String heroName) {
    return itemsByHero.getOrDefault(key(heroName), SuperHeroItem.NONE);
}
```

(This changes what callers receive for unknown heroes, so it is a design change to make together with the callers, not a pure refactoring.)

Why the run time is constant (amortized)

- Old version: up to k string comparisons $\Rightarrow O(k)$ per lookup; it grows with every new hero added to the chain.
- New version: `HashMap.get` computes the key's hash and inspects one bucket. With a good hash function the expected bucket size is $O(1)$, independent of the number of heroes.
- *Amortized*: the map occasionally resizes (rehashes all entries) when it grows, an $O(n)$ operation, but resizing doubles the capacity, so its cost spread over all insertions is $O(1)$ per operation. (String length is a constant here; hashing a name is the same cost as one comparison in the old code.)

Behaviour preserved

- The same four (hero $\rightarrow$ item) pairs are returned, matched case-insensitively (`"ironman"`, `"IRONMAN"` both give `suit`), because both registration and lookup lower-case the key; for ASCII names this is exactly `equalsIgnoreCase`.
- Unknown names still return `null`; a `null heroName` still throws `NullPointerException`, as `heroName.equalsIgnoreCase(...)` did.

Answer. Replace the if/else-if chain by a `HashMap<String, SuperHeroItem>` keyed by the lower-cased hero name (lookup table / Substitute Algorithm), encapsulate the public fields behind `register()`/`getItem()`, rename the class; lookup becomes $O(1)$ amortized and a new hero is a one-line `register` call.

2021–2022 • Q6(c) CSE307 • 07/05/2023 • F011 **MARKS 10**

Define *code smell* in your own words. Provide 3 (three) examples of smell in a program which may degrade the quality of the code.

SOLUTION

What is a code smell?

A **code smell** ("bad smell in code") is a *surface symptom* in source code that hints at a deeper design problem. The program still compiles and produces the right output — a smell is *not* a bug — but the way it is written makes it hard to understand, risky to change and expensive to maintain. In the lecture's words, bad smells are the places in a system that are "ripe" for refactoring: we look for them to decide *where* to refactor, and each smell suggests *which* refactoring to apply. Tools such as Checkstyle, PMD, FindBugs and SonarQube can detect many smells automatically.
In my own words: *a code smell is a pattern in working code that makes a reviewer feel "this will hurt us later" — duplication, oversized units, tangled conditionals, misplaced responsibilities.*

Example 1 — Duplicated Code

The same logic appears in more than one place.

```
class Report {
    double totalWithVat(double net)  { return net + net * 0.15; }
}
class Invoice {
    double grandTotal(double net)    { return net + net * 0.15; }   // same formula again
}
```

Quality degraded: maintainability and correctness. If the VAT rate changes and only one copy is updated, the two classes silently disagree — a bug has been introduced. *Cure:* **Extract Method** into one place (e.g. a `VatPolicy.addVat()`) and call it from both.

Example 2 — Long Method

```
class OrderService {
    public void processOrder(Order order) {
        // validate order        ... 20 lines
        // calculate total price ... 15 lines
        // apply discount        ... 12 lines
        // format receipt        ... 25 lines
        // print receipt         ... 10 lines
    }
}
```

Quality degraded: understandability and testability. A reader must hold 80+ lines in mind; no step can be unit-tested or reused on its own. *Cure:* **Extract Method** for each commented block (`validate()`, `calculateTotal()`, `applyDiscount()`, ...); the comments become method names.

Example 3 — Long Parameter List

```
class PaymentService {
    public void processPayment(String name, String email,
```

```
                                String cardNumber, String expiry, double amount) { ... }
}
```

Quality degraded: readability and reliability. It is hard to remember the parameter order and easy to pass a wrong value (two adjacent `Strings` can be swapped without a compiler error); the same parameter chain is repeated across methods. *Cure:* **Introduce Parameter Object** (`Customer`, `Card`) so the call becomes `processPayment(customer, card, amount)`.

Smell	Quality attribute hurt	Refactoring (slides)
Duplicated Code	maintainability, consistency	Extract Method / Move Method
Long Method	readability, testability, reuse	Extract Method
Long Parameter List	readability, error-proneness	Introduce Parameter Object

Answer. A code smell is a symptom in working code that signals a poor design and makes future change costly. Examples: Duplicated Code, Long Method, Long Parameter List (others: Large Class, Feature Envy, Switch Statements, Data Clumps, Shotgun Surgery).

Jan 2021 • Q6(a) CSE307 • 04/09/2021 • F015 MARKS 20

Write down two program snippets to show examples of code smells Data Clump and Refused Bequest.

SOLUTION

Part 1 — Data Clumps

Definition (slides). **Data Clumps** are attributes (or parameters) that clump together — they are always used together — but are not part of the same class. The test: if you delete one item of the group, do the others still make sense? If not, the group is really one missing object.
Smelly snippet. The three address fields and the two date fields travel together everywhere:

```
class Customer {
    String name;
    String street, city, postCode;          // clump #1 (fields)
}

class ShippingService {
    double shippingCost(String street, String city, String postCode, double weight) { ... }
    void   printLabel  (String name, String street, String city, String postCode)  { ... }
}

class BookingService {
    double price(Date checkIn, Date checkOut, int rooms) { ... }      // clump #2
    boolean isAvailable(Date checkIn, Date checkOut, int rooms) { ... }
    int nights(Date checkIn, Date checkOut) { ... }
}
```

Why it is a smell.

- The same 3 (or 2) values are repeated in many signatures → also a *Long Parameter List*.
- Behaviour that belongs to the group (formatting an address, validating a post code, counting nights, checking that `checkIn` is before `checkOut`) has no home, so it gets duplicated in every service — *Primitive Obsession* too.

- Easy to pass arguments in the wrong order (`city` and `street` are both `String`).

Refactoring. Extract Class for the field clump and Introduce Parameter Object for the parameter clump:

```
class Address {
    private final String street, city, postCode;
    Address(String street, String city, String postCode) {
        this.street = street; this.city = city; this.postCode = postCode;
    }
    String label() { return street + ", " + city + " - " + postCode; }  // behaviour moves in
}

class DateRange {
    private final Date start, end;
    DateRange(Date start, Date end) {
        if (end.before(start)) throw new IllegalArgumentException("end < start");
        this.start = start; this.end = end;
    }
    int nights() { return (int) ((end.getTime() - start.getTime()) / 86_400_000L); }
}

class Customer        { String name; Address address; }
class ShippingService {
    double shippingCost(Address to, double weight)   { ... }
    void   printLabel(String name, Address to)        { ... }
}
class BookingService {
    double  price(DateRange stay, int rooms)          { ... }
    boolean isAvailable(DateRange stay, int rooms)    { ... }
}
```

Part 2 — Refused Bequest

Definition (slides). Refused Bequest: a subclass ignores ("refuses") most of the functionality it inherits from its superclass. Such a subclass usually fails the **IS-A** test, so the inheritance hierarchy is wrong.

Smelly snippet.

```
class Bird {
    void eat()     { System.out.println("Eating"); }
    void layEggs() { System.out.println("Laying eggs"); }
    void fly()     { System.out.println("Flying high"); }
    int  maxAltitude() { return 3000; }
}

class Penguin extends Bird {
    @Override void fly() {                          // refuses the bequest
        throw new UnsupportedOperationException("Penguins cannot fly");
    }
    @Override int maxAltitude() { return 0; }      // meaningless inherited feature
}

void migrate(List<Bird> flock) {
    for (Bird b : flock) b.fly();                   // crashes at run time for a Penguin!
}
```

Why it is a smell. `Penguin` inherits `fly()` and `maxAltitude()` but cannot honour them; it overrides them with exceptions or dummy values. Clients that trust the superclass (`migrate`) break, which

also violates the Liskov Substitution Principle. A penguin IS-A bird, but it is *not* a "flying bird".
Refactoring. Push the refused behaviour down out of the superclass (**Push Down Method**) into a narrower abstraction (**Extract Interface**/**Extract Superclass**); if the subclass used only a few inherited features, **Replace Inheritance with Delegation** would be the alternative.

```
class Bird {                                        // only what ALL birds do
    void eat()     { System.out.println("Eating"); }
    void layEggs() { System.out.println("Laying eggs"); }
}
interface Flyer {
    void fly();
    int  maxAltitude();
}
class Eagle extends Bird implements Flyer {
    public void fly()        { System.out.println("Flying high"); }
    public int  maxAltitude() { return 3000; }
}
class Penguin extends Bird {                        // inherits nothing it refuses
    void swim() { System.out.println("Swimming"); }
}
void migrate(List<? extends Flyer> flock) {         // compiler now rejects penguins
    for (Flyer f : flock) f.fly();
}
```

	Data Clumps	Refused Bequest
Symptom	same group of fields/params repeated	subclass overrides inherited members to do nothing/throw
Root cause	a missing class	a wrong inheritance (fails IS-A)
Refactoring	Extract Class, Introduce Parameter Object	Push Down Method, Extract Interface/Superclass, Replace Inheritance with Delegation

Answer. Data Clump: (street, city, postCode) and (checkIn, checkOut) repeated everywhere → Extract Class `Address`, Introduce Parameter Object `DateRange`. Refused Bequest: `Penguin extends Bird` but refuses `fly()` → push flying into a `Flyer` interface so only real flyers inherit it.

2020–2021 • Q1(b) CSE325 • 12/04/2023 • C325-2021-1b MARKS 16

What do you understand by Feature Envy, Refused Bequest and Cyclomatic Complexity type of Code Smell. Give an example of Inappropriate Intimacy in the context of Code Smell.

SOLUTION

Code smell

A **code smell** ("bad smell in code") is a surface symptom in the code that usually indicates a deeper design problem; it is not a bug, but it makes the code hard to understand and change. Each smell suggests refactorings that remove it (tools such as Checkstyle, PMD, SonarQube detect many of them).

(i) Feature Envy

A method that is more interested in the data of *another* class than in its own — it calls many getters

of another object to compute something. The behaviour is in the wrong place.

```
class InvoicePrinter {
    double lineTotal(OrderLine l) {                        // envies OrderLine
        return l.getQuantity() * l.getUnitPrice() * (1 - l.getDiscount());
    }
}
// Refactoring: Move Method -> the computation lives with the data
class OrderLine {
    private int quantity; private double unitPrice, discount;
    double total() { return quantity * unitPrice * (1 - discount); }
}
```

Fix: **Move Method** (or Extract Method then Move Method).

(ii) Refused Bequest

A subclass inherits methods/data from its parent but does not want them — it ignores them, overrides them to do nothing or throws `UnsupportedOperationException`. The hierarchy is wrong (the subclass is not really a kind of the parent) and it usually violates LSP.

```
class Bird { void eat() { } void fly() { /* ... */ } }
class Penguin extends Bird {
    @Override void fly() { throw new UnsupportedOperationException(); }   // refuses the bequest
}
// Refactoring: Push Down Method / Replace Inheritance with Delegation
class Bird { void eat() { } }
class FlyingBird extends Bird { void fly() { } }
class Penguin extends Bird { void swim() { } }
```

Fix: **Push Down Method/Field** into the subclasses that use them, or **Replace Inheritance with Delegation**.

(iii) Cyclomatic Complexity (as a smell)

Cyclomatic complexity $V(G) = E - N + 2 =$ (number of decisions) $+1$ counts the independent paths through a method. A method with high $V(G)$ (rule of thumb > 10) has deeply nested `if`/`switch`/loops — it is hard to read, needs many test cases (at least $V(G)$ for basis-path coverage) and is error-prone. It usually shows up together with *Long Method* and *Switch Statements*.

```
double shipping(Order o) {                       // 6 decisions -> V(G) = 7
    if (o.isExpress()) {
        if (o.weight() > 10) return 500;
        else if (o.isInternational()) return 400;
        else return 200;
    } else {
        if (o.total() > 5000) return 0;
        if (o.isInternational()) return 300;
        if (o.weight() > 10) return 150;
        return 60;
    }
}
```

Fix: **Decompose Conditional**, **Extract Method**, **Replace Nested Conditional with Guard Clauses**, or **Replace Conditional with Polymorphism** (e.g. `ExpressShipping` and `StandardShipping` strategies), each piece then having a small $V(G)$.

(iv) Example of Inappropriate Intimacy

Inappropriate Intimacy: two classes know too much about each other's *private* parts — they access each other's fields directly or depend on internal details (often bidirectionally), so a change in one breaks the other.

```
class Account {
    double balance;            // package-visible
    List<Txn> history;         // package-visible
    Customer owner;
}
class Customer {
    String name; List<Account> accounts; int loyaltyPoints;

    void transfer(Account from, Account to, double amt) {
        if (from.balance >= amt) {                // digs into Account internals
            from.balance -= amt;  to.balance += amt;
            from.history.add(new Txn(-amt));  to.history.add(new Txn(amt));
            from.owner.loyaltyPoints++;           // and back into Customer through Account
        }
    }
}
```

`Customer` manipulates `Account`'s balance and history directly, and `Account` exposes its owner, so the two classes are tightly bound together; any change to how an account stores its balance breaks `Customer`.

Refactoring: Encapsulate Field (make fields private), Move Method (the transfer logic belongs to `Account`/a `TransferService`), and Change Bidirectional Association to Unidirectional if the back-link is not needed.

```
class Account {
    private double balance; private final List<Txn> history = new ArrayList<>();
    void withdraw(double amt) {
        if (amt > balance) throw new IllegalArgumentException("insufficient");
        balance -= amt; history.add(new Txn(-amt));
    }
    void deposit(double amt) { balance += amt; history.add(new Txn(amt)); }
}
class TransferService {
    void transfer(Account from, Account to, double amt) { from.withdraw(amt); to.deposit(amt); }
}
```

Answer. Feature Envy: a method uses another class's data more than its own (Move Method). Refused Bequest: a subclass does not want/overrides away inherited members (Push Down / Replace Inheritance with Delegation). Cyclomatic Complexity smell: too many decision paths in one method, $V(G) = E - N + 2$ high (Decompose Conditional, Extract Method, polymorphism). Inappropriate Intimacy: `Customer.transfer` directly changes `Account.balance/history` — fix with Encapsulate Field + Move Method.

2017–2018 • Q2(c) CSE307 • 29/09/2018 • F017 MARKS 7

Why is refactoring important in agile development? Give some example of refactoring in software development life cycle.

SOLUTION

Why agile development depends on refactoring

Agile methods (XP, Scrum) deliberately do *not* design everything up front. They deliver small increments, welcome changing requirements and keep the design as simple as possible for today's stories. The slides state the underlying idea: *"it will be challenging to get a design right the first time and, as a program's requirements change, the design may need to change — refactoring provides techniques*

for evolving the design in small incremental steps." Therefore:

- **Evolutionary design.** Design emerges sprint by sprint; refactoring is the mechanism that reshapes yesterday's simple design to fit today's new requirement.
- **Prevents design decay / technical debt.** Frequent small changes by many people make the code decay; continuous refactoring keeps velocity stable across sprints.
- **TDD cycle "Red → Green → Refactor".** Agile writes tests first; after a test passes, the code is cleaned up and the tests are re-run — exactly the slides' "test constantly" safety net.
- **Simple design & collective code ownership.** Anyone may improve any code; readable, well-factored code is what makes that possible.
- **Faster programming and fewer bugs.** "A good design enables progress"; refactoring deepens understanding of the code and helps find bugs.

Examples of refactoring across the life cycle

When (slides' triggers)	Example
During coding (TDD)	After making a test pass, **Extract Method** on a long `checkout()` and **Rename** unclear variables.
Before adding a feature	Before adding a "Bkash" payment option, apply **Replace Conditional with Polymorphism** to the `if (type == ...)` chain so the new option is just a new subclass.
Rule of Three	The third time the same validation is copy-pasted, **Extract Method**/**Extract Class** it into a `Validator`.
While fixing a bug	A bug in deeply nested pay logic is fixed after **Replace Nested Conditional with Guard Clauses** makes the paths visible.
During code review	Reviewer spots `(start, end)` passed to many methods → **Introduce Parameter Object** `DateRange`.
Maintenance / evolution	A "God" `Employee` class with *Divergent Change* is split by **Extract Class** into `Payroll`, `EmployeeReport`, `Notifier`.

Answer. Agile builds the design incrementally and embraces change, so it needs refactoring to evolve the design safely (backed by automated tests) and to keep technical debt low. Refactoring happens during TDD, before adding features, on the rule of three, while fixing bugs, in code reviews and in maintenance.

11

Software Ethics & Requirement Analysis

2023–2024 • Q1(b) CSE213 • 10/05/2026 • E001 **MARKS 10**

A software company developed an online citizen information platform for a government body. After the first release, the corresponding team hired a security engineer to identify and resolve security vulnerabilities within the system.
During the assessment, the engineer discovered several critical flaws that posed high risks of exposing sensitive citizen data. Immediately after the discovery, he disclosed the vulnerabilities to media outlets anonymously. He justified his actions by claiming it was his moral duty to protect citizens who are already using this from potential harm. The resulting media coverage triggered a public scandal that severely damaged the firm's reputation and disrupted the government project. Investigate whether there is any ethical breaches present in the above scenario with proper justification.

SOLUTION

The yardstick: the IEEE-CS/ACM Software Engineering Code (8 principles)

The lecture frames every ethical case with one question: *"Does this system do what it should, for whom it should, without harm?"* and judges conduct against the eight principles of the IEEE-CS/ACM code: **Public** (top priority), **Client & Employer**, **Product**, **Judgment**, **Management**, **Profession**, **Colleagues** and **Self**. Ethical codes are *not* automated rules – they need active assessment of trade-offs and consequences. So we must look at both the *goal* of an action and the *means* used to reach it.

Step 1 – Is the engineer's goal legitimate?

Yes. Principle 1 (**Public**) says a software engineer must act consistently with public health, safety and welfare, and clause 1.04 of the code asks engineers to *disclose to appropriate persons or authorities* any actual or potential danger to the public. Critical flaws that may leak sensitive citizen data are exactly such a danger (dimension 6 of the slides: *security as an ethical duty*; Equifax exposed 147M records by ignoring a known flaw). Staying silent would itself be unethical.

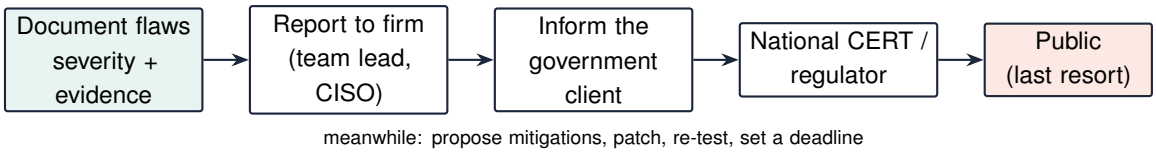
Step 2 – Are the means (anonymous leak to the media, immediately) ethical?

No. The *way* the disclosure was made breaches several principles:

Principle	What the code expects	What the engineer did
1 Public	Disclose dangers to the *appropriate* persons/authorities so that the danger is reduced.	Published unpatched vulnerabilities to the whole world. Attackers learned of the holes *before* a fix existed, so citizens' data became *more* exposed – the action increased the very harm he wanted to prevent.
2 Client & Employer	Act in the best interest of client/employer consistent with the public interest; keep confidential information gained in professional work private (2.05); identify, document and *report problems to the employer/client promptly* (2.06, 2.07).	Never informed the firm or the government client; leaked confidential findings obtained under his professional engagement.
3 Product	Ensure the product meets the highest professional standards – the engineer was hired precisely to *resolve* the flaws.	Abandoned the remediation role; no patch, mitigation or re-test was attempted.
4 Judgment	Maintain integrity and independence; temper technical judgment by human values.	Acted "immediately", without assessing severity, exploitability, or consequences of disclosure.
6 Profession	Advance the integrity and reputation of the profession; express concerns to the people involved first and report to outside authorities only when that is impossible or dangerous (6.12, 6.13).	Anonymous media leak skipped every internal channel and damaged trust in the profession.

Step 3 – What would the ethical course of action have been?

Responsible (coordinated) disclosure: escalate step by step, going outside only if inside channels fail and citizens remain in danger.



- Write a confidential report (flaw, impact, proof, fix) to the company *and* the government client, with a reasonable deadline for patching; propose interim mitigations (take a module offline, rotate keys, add access controls).
- If the firm ignores or hides the risk, escalate to the client and then to the competent authority (e.g. the national CERT / data-protection regulator) – this satisfies Principle 1 without arming attackers.
- Public whistle-blowing is justified only as the *last* resort, when internal and official channels have clearly failed and harm is ongoing.

Step 4 – Breaches by the company itself

The scenario also hints at an ethical failure of the firm: it released a government system handling sensitive citizen data with *critical* flaws, and hired security staff only after release. This violates Principle 3 (Product – adequate testing and review before release) and the lecture's "security

& release QA" standard: *pre-deployment safety gates* rather than deploying known or unchecked vulnerabilities. Accountability (dimension 3) rests with the firm, not "the system".

Answer. Yes, there are ethical breaches. The engineer's *intention* (protecting citizens, Principle 1) is right, but immediate anonymous disclosure to the media violates Client & Employer (confidentiality, prompt reporting), Product (he was hired to fix), Judgment and Profession, and even harms the Public by publicising unpatched holes. He should have followed responsible disclosure: report internally → client → CERT/regulator → public only as a last resort. The firm also breached the Product principle by releasing a citizen-data system without adequate security review.

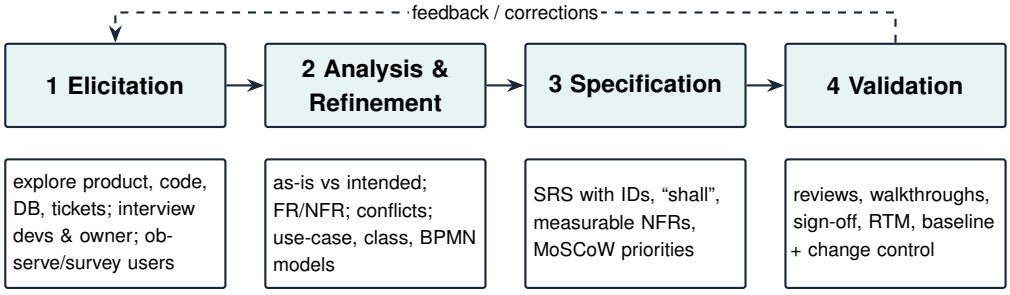
2023–2024 • Q5(c) CSE213 • 10/05/2026 • R001 **MARKS 10**

Suppose you join a startup and find that it has already developed a software product without preparing any formal written specification. The company now wants you to prepare a proper specification so that the system's intended behavior is clearly documented and can serve as an agreed reference for future review and improvement. Explain how you would proceed.

SOLUTION

Key idea

The code shows *what the system does*, not necessarily *what it is intended to do*. I would run the four-stage requirement-analysis pipeline from the lecture – **Elicitation → Analysis & Refinement → Specification → Validation** – using the existing product as one extra source of requirements ("reverse" requirements engineering), and finish with an agreed, baselined document and an RTM.



Step 1 – Elicitation (gather what exists and what is wanted)

- **Study the artefacts**: use the running product screen-by-screen, read the code structure, database schema, API endpoints, existing tests, commit history, README/help pages, bug and support tickets.
- **Interviews** with founders/product owner (intended vision, business rules) and developers (hidden rules, known limitations).
- **Observation** of real users and **surveys** of the user base (how the product is actually used, pain points); a **workshop** with all stakeholders when views conflict.

Step 2 – Analysis & refinement

- Separate *as-built* behaviour from *intended* behaviour – a behaviour that nobody wants is a bug, not a requirement.
- Classify into functional and non-functional requirements; resolve conflicts and duplicates.

- Model the system with the representation techniques of the course: a **use-case diagram** (actors and system boundary), **class diagram** (domain entities), **BPMN** for key business processes.

Step 3 – Specification

Write a structured Software Requirements Specification (introduction & scope, overall description, specific FRs per feature/use case, NFRs, constraints, glossary). Each requirement gets a unique ID, a single need, "shall" wording and measurable acceptance criteria (*unambiguous, verifiable, complete, consistent, measurable, feasible*), and a **MoSCoW** priority (Must/Should/Could/Won't) so that future improvements can be planned.

Step 4 – Validation and baselining

- Review the SRS with stakeholders (walkthroughs); check each requirement against the running system and the business objectives; collect formal sign-off – this makes it the *agreed reference*.
- Build a **Requirements Traceability Matrix**: requirement ↔ source (who asked) ↔ code module ↔ test case. It reveals orphan features (code with no requirement) and untested requirements, and supports impact analysis of future changes.
- Put the SRS under version control with a change-control process so it stays the single source of truth.

Answer. Follow elicitation (product, code, DB, tickets, interviews, observation, surveys) → analysis (as-is vs intended, FR/NFR, conflict resolution, use-case/class/BPMN models) → specification (SRS with IDs, measurable "shall" statements, MoSCoW) → validation (stakeholder reviews, sign-off, RTM linking requirements to code and tests, baseline under change control).

2023–2024 • Q6(a) CSE213 • 09/02/2025 • E002 **MARKS 8**

A start-up company is developing a mobile health app that collects users' personal health data, such as heart rate, physical activity, and dietary habits. The app uses this data to provide critical health suggestions, such as warnings about potential heart risks or medicine recommendations for improving overall health. The project team consists of software and database engineers who have developed an algorithm by studying related articles to generate the suggestions.
The app shares the health records as well as age, profession and address of the users with some healthcare institutions. However, prior to sharing, the app takes user consent to share anonymized health data.
Identify the ethical breaches present in this scenario (if any). Explain why these actions are unethical.

SOLUTION

Approach

We check the scenario against the lecture's key dimensions of software ethics (privacy, bias & fairness, accountability, transparency, autonomy, security) and the IEEE-CS/ACM principles. Two activities raise problems: (1) *how* the health suggestions are produced, and (2) *what* is shared with institutions.

Breach 1 – Critical medical advice from an unqualified, unvalidated algorithm

- Warnings about heart risk and *medicine recommendations* are safety-critical. The algorithm was

built only by software and database engineers "by studying related articles" – no physicians, no clinical validation, no regulatory approval.

- **Principle 2 – Client & Employer** (clause 2.01): provide service only in areas of competence and be honest about limitations. Medicine is outside the team's competence.
- **Principle 3 – Product**: the highest professional standards and rigorous QA are required; a medical recommender needs clinical trials/validation, not article reading.
- **Principle 1 – Public**: a wrong medicine suggestion or a missed heart warning can directly injure users (*Preventing negligence*: rushing defective logic endangers life, cf. Therac-25).
- **Bias & Fairness / Transparency**: rules distilled from a few articles may encode population bias (e.g. data from other demographics) and the user cannot see why a warning was issued.

Breach 2 – Sharing identifiable data under an "anonymized" consent

- The user consented to share *anonymized health data*. What is actually shared is health records *plus age, profession and address*. These are quasi-identifiers: age + profession + address can single out a person, so the data is *re-identifiable*, i.e. not anonymous.
- Consent obtained for one thing and used for another is not **informed consent**; it is deception (dimension 1, Privacy: "consent must be informed, explicit and unburied"; slide contrast: "uninformed harvesting & re-identifying records" is the unethical practice).
- **Data minimisation** is violated: institutions do not need a user's profession or street address to study health trends.
- Codes broken: Principle 1 (Public – privacy, avoid deception, clause 1.06), Principle 3 (Product – respect the privacy of those affected, use data only in properly authorised ways, clauses 3.12, 3.13), Principle 4 (Judgment – integrity). This is the same pattern as the Cambridge Analytica case on the slides.

What would make it ethical?

Unethical practice in the scenario	Ethical engineering standard
Algorithm by non-experts from articles	Involve doctors, validate clinically, obtain regulatory approval, show a disclaimer and "consult a doctor" path
Identifiers shared with health data	True anonymisation (remove/generalise age, profession, address; k-anonymity), privacy-by-design
Consent for "anonymized" data only	Explicit, specific consent describing exactly which fields go to which institution, revocable at any time

Answer. Two breaches: (1) safety-critical medical suggestions produced by engineers without medical competence or validation – violates Public, Client & Employer (competence), Product and Judgment; (2) sharing health data together with age, profession and address while consent was given only for *anonymized* data – the data is re-identifiable, so consent is not informed; violates privacy, data minimisation, honesty (Public, Product, Judgment).

2023–2024 • Q6(b) CSE213 • 09/02/2025 • E003 **MARKS 7**

"Software engineers shall act consistently with the public interest." – Elaborate the implication of this principle with relevant example.

SOLUTION

The principle

This is **Principle 1 – PUBLIC** of the IEEE-CS/ACM Software Engineering Code of Ethics, and the slides label it the *top priority*: act consistently with public health, safety and welfare. Software is never neutral – every design choice affects people who may never use the system directly (patients, citizens, pedestrians). Hence when the interests of the client, employer or the engineer conflict with those of the public, *the public interest wins*.

Implications (what the engineer must actually do)

(i) **Accept full responsibility** for one's own work – "the algorithm did it" is no defence.

(ii) **Moderate** the interests of the engineer, employer, client and users with the public good.

(iii) **Approve software only if** there is a well-founded belief that it is safe, meets specifications, passes appropriate tests, and does not diminish quality of life, privacy or harm the environment.

(iv) **Disclose** to appropriate persons/authorities any actual or potential danger to the user, the public or the environment associated with the software.

(v) **Cooperate** in efforts to address grave public concerns caused by software.

(vi) **Be honest** – avoid deception in all (especially public) statements about software.

(vii) **Consider** disability, economic disadvantage and other factors that affect access to the benefits of software.

In short: deadline or profit pressure is never a reason to ship something known to be unsafe; ethics is a *design constraint*, not an afterthought.

Examples

- **Boeing 737 MAX MCAS** (slides): the risks of MCAS were concealed; 346 people died. Acting in the public interest would have meant disclosing the single-sensor dependency and refusing to approve the design until it was safe.
- **VW Dieselgate**: engineers wrote defeat software that cheated emission tests on 11M cars – the employer's interest was placed above public health and the environment.
- **Everyday example**: you are asked to release a ride-sharing app on Friday although a known bug sometimes shows the wrong pick-up location at night. Acting in the public interest means reporting the bug, documenting the risk and delaying (or disabling the feature) instead of silently shipping – even if the client is unhappy.

Answer. Principle 1 (Public) is the top priority of the code: engineers must protect public health, safety, welfare, privacy and the environment, take responsibility for their work, approve only safe and tested software, disclose dangers to the proper authorities and be honest. When client/employer interests conflict with public welfare, public welfare prevails (e.g. Boeing MCAS and VW Dieselgate show the consequence of ignoring it).

2022–2023 • Q3(c) CSE325 • 30/01/2025 • C325-2223-3c MARKS 9

Give example of Privacy, Accuracy and Usability requirements of a software for the HR Management system of BUET.

SOLUTION

Concept

Privacy, accuracy and usability are **non-functional requirements**: they say *how well* the system must do its job, not *what* it does. Following the lecture's "anatomy of an effective requirement", each example below is written to be *unambiguous, verifiable, measurable and feasible* – not a vague wish such as "the system should be secure and easy to use".

Privacy requirements

(who may see/handle personal data of BUET employees)

(i) PR-1 A teacher or officer shall be able to view only his/her own salary, leave balance, provident-fund and medical records; Heads of departments may view leave records of their own department's staff only.

(ii) PR-2 National ID, bank account and medical information shall be stored encrypted (AES-256) and shown masked (e.g. last 4 digits) on every screen and report.

(iii) PR-3 Every read or change of a personnel record shall be written to an audit log (user, time, record, action) kept for at least 5 years and viewable only by the Registrar's office.

(iv) PR-4 Personal data shall not be exported or shared with any external body (e.g. a bank) without the employee's recorded consent or a written order of the Registrar.

Accuracy requirements

(correctness and precision of HR data and computations)

(i) AR-1 Monthly salary, allowances and deductions (income tax, GPF, house rent) shall be computed exactly according to the national pay scale in force, rounded to the nearest Taka; 100% of the payroll test cases from the Accounts section shall match.

(ii) AR-2 Leave balance shall be updated within one minute of a leave approval and shall never become negative; earned leave shall be credited according to BUET rules on the first working day of each month.

(iii) AR-3 Seniority lists and retirement dates shall be calculated from the joining date and date of birth with zero tolerance (the retirement date must equal the legal age-limit date).

(iv) AR-4 Duplicate employee records shall be prevented: the employee ID and NID shall be unique.

Usability requirements

(ease of learning and use for staff with different IT skills)

(i) UR-1 A first-time user (teacher or office staff) shall be able to apply for leave in at most 3 minutes without training; at least 8 of 10 users in a pilot test shall succeed unaided.

(ii) UR-2 The interface shall be available in both Bangla and English, and every form shall show field-level error messages explaining how to correct the input.

(iii) UR-3 Frequent tasks (download pay-slip, check leave balance, apply for leave) shall be reachable within two clicks from the home page and work on a mobile phone browser.

(iv) UR-4 The system shall meet WCAG 2.1 AA (readable font size, colour contrast, keyboard navigation).

Answer. Privacy: role-based access to own/department records only, encryption and masking of NID/bank/medical data, audit log of every access, no sharing without consent. Accuracy: payroll exactly per pay scale (100% match with reference cases), correct and timely leave balances, exact retirement dates, unique employee records. Usability: leave application in $\leq$ 3 min by 80% of untrained users, Bangla/English UI with clear error messages, frequent tasks within 2 clicks and mobile-friendly, WCAG 2.1 AA.

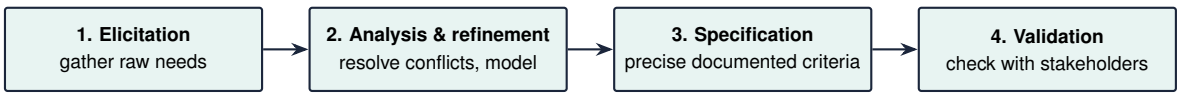
2022–2023 • Q4(b) [2nd] CSE325 • 30/01/2025 • C325-2223-4b2 **MARKS 7**

What are the techniques suggested in Software requirements engineering discipline for proper requirements analysis to design a software?

SOLUTION

The requirement-analysis pipeline

The lecture presents requirement analysis as four stages, each with its own techniques:



1. Elicitation techniques

Technique	Best for	Watch out for
One-to-one interviews	deep needs, domain rules from experts	interviewer bias, does not scale
Surveys / questionnaires	many users, quantifiable opinions	shallow answers, low response
Focus groups	shared priorities, group consensus	group-think
Co-design workshops	conflicting stakeholders, quick consensus	dominant voices, scheduling
Observation / field study	real workflows and workarounds	time-consuming, observer effect

2. Analysis & refinement techniques

- **Classify** requirements into *functional* (what the system must do) and *non-functional* (how well: performance, usability, security, . . .).
- **Model** the system to expose gaps and conflicts: use case diagrams (actors and system boundary), BPMN / data-flow diagrams (process and data flow), UML class diagrams (domain entities and relationships).
- **Negotiate and prioritise**, e.g. the **MoSCoW** method: Must have, Should have, Could have, Won't have (this release).

3. Specification

Turn vague statements into requirements that are *unambiguous, verifiable, complete, consistent, measurable and feasible* (e.g. "fast" → "a balance check completes in ≤ 3 s on 3G for 95% of requests"), recorded in an SRS / user stories with acceptance criteria.

4. Validation and management

Reviews / walkthroughs and prototypes with stakeholders; a **Requirements Traceability Matrix** linking each requirement backward to its origin and forward to design and test cases, used for *impact analysis* when a requirement changes and for verification coverage (no orphan features, no untested requirements).

Answer. Elicitation (interviews, surveys/questionnaires, focus groups, co-design workshops, observation/field study) → analysis & refinement (functional vs non-functional classification, modelling with use case / BPMN / class diagrams, conflict resolution, MoSCoW prioritisation) → specification of unambiguous, measurable, verifiable requirements → validation (reviews, prototypes) with a traceability matrix for impact analysis.

NOTE The paper labels this part "(b)" a second time (after the 14-mark (b) on cache-aside); it is listed here as Q4(b) [2nd].

2021–2022 • Q2(c) CSE307 • 07/05/2023 • R004.2 **MARKS 5**

Briefly describe the differences between *Functional* and *Non-Functional* requirements.

SOLUTION

Definitions

Functional requirement (FR) – *what the system must do*: a service, behaviour or input→output transformation the system must provide. **Non-functional requirement (NFR)** – *how well the system performs*: a quality attribute or constraint on the system's services (performance, usability, reliability, security, portability, . . .).

Differences

Aspect	Functional	Non-functional
Describes	Behaviour / features (*what*)	Qualities / constraints (*how well*)
Scope	Usually a single feature or use case	Often the whole system (emergent property)
Origin	User tasks, business rules	User expectations, budget, laws, standards, platforms
Verification	Functional tests: does the feature give the right output?	Measurement: load tests, usability tests, reliability statistics, audits
Modelled by	Use-case diagrams, BPMN, scenarios	Quality scenarios with metrics (time, %, MB)
If missed	A feature is absent / wrong	System may be unusable as a whole (too slow, insecure)
Example (mobile money, slides)	Send money, check balance, cash-in via agent, view transaction history	Works offline, loads in 3 s on 3G, usable by low-literacy users, audit logging

Answer. FRs state what the system must do (features/behaviour, e.g. "send money"); NFRs state how well it must do it (quality attributes and constraints, e.g. "loads in 3 s on 3G"). FRs are verified by functional tests of features, NFRs by measurement and usually apply to the system as a whole.

2021–2022 • Q4(b) CSE307 • 07/05/2023 • R005 **MARKS 10**

"The software should provide update notification after a fixed time. Users should be able to perform action based on the notification" – assess the quality of this requirement statement based on the different quality measures.

SOLUTION

Quality measures of a good requirement (lecture: "anatomy of an effective requirement")

A requirement should be **unambiguous**, **verifiable** (testable), **complete**, **consistent**, **measurable** and **feasible**. (Many texts add *atomic* – one need per statement – and *traceable*.) The analyst's task is to translate vague user statements into clear, measurable ones.

Assessment of the given statement

Measure	OK?	Observation
Unambiguous	×	"Update" – software version, data, or content? "Fixed time" – what interval, measured from when (install, last check, release)? "Action" – which actions? Different readers will build different features.
Verifiable	×	No observable pass/fail criterion; a tester cannot write a test case for "after a fixed time" or "perform action".
Measurable	×	No numbers: interval, delivery delay, number of retries are all missing.
Complete	×	Missing: trigger condition (only when an update exists?), notification channel (pop-up, push, e-mail), content, which users, behaviour if user ignores/offline, mandatory vs optional updates.
Consistent	?	Not contradictory by itself, but "should" (weak, optional) clashes with the intent of a mandatory feature, and repeated notifications may conflict with other requirements (e.g. do-not-disturb).
Feasible	✓	Technically achievable, but feasibility cannot be estimated precisely until it is clarified.
Atomic	×	Two requirements (notify; act on notification) merged in one sentence.

Verdict: it is a *weak* requirement – feasible but ambiguous, incomplete, unmeasurable and not testable.

Rewritten (testable) version

- **REQ-UPD-1** (Must): When a newer application version is published on the update server, the system *shall* check for it every 24 hours and show an in-app notification containing the version number and release notes within 24 hours of publication.
- **REQ-UPD-2** (Must): The notification *shall* offer exactly three actions: *Update now*, *Remind me in 24 hours* and *Skip this version*; choosing *Update now shall* start the download within 5 seconds.
- **REQ-UPD-3** (Should): If the user chooses *Remind me*, the notification *shall* be shown again after exactly 24 hours; a security update marked *mandatory shall* not offer *Skip*.

- **REQ-UPD-4** (NFR): For 95% of active online users the notification *shall* appear within 1 hour after the 24-hour check.

Each statement now has one need, uses "shall", contains numbers, and can be traced to a test case in the RTM.

Answer. Feasible but poor: ambiguous ("update", "fixed time", "action"), not measurable, not verifiable, incomplete (trigger, channel, actions, user scope missing), weakly worded ("should") and non-atomic. It must be split and rewritten with concrete triggers, intervals and actions, e.g. "check every 24 h; notify within 24 h with Update now / Remind in 24 h / Skip".

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2020–2021 • Q1(c) CSE325 • 12/04/2023 • C325-2021-1c MARKS 5

What do you need to check about requirements collected for a software application?

2019–2020 • Q7(c) CSE325 • 02/04/2022 • C325-1920-7c MARKS 8

How should the requirements be validated?

SOLUTION

What validation is

Requirements validation is the last stage of the analysis pipeline (elicitation → analysis & refinement → specification → **validation**): it "verifies specifications against stakeholder objectives" (lecture), i.e. checks that the documented requirements define the system the customer really wants, *before* design starts – an error found here costs far less than one found after release.

What is checked

Each requirement and the set as a whole are checked for:

- **Validity** – every requirement serves a real stakeholder need/goal.
- **Consistency** – no two requirements contradict (e.g. "log-out after 5 min idle" vs. "stay logged in during an exam").
- **Completeness** – all functions, constraints, error cases and NFRs are present.
- **Unambiguity** – only one interpretation is possible.
- **Realism / feasibility** – achievable with the budget, time and technology.
- **Verifiability (measurable)** – a test can show it is met ("$\leq$ 3 s on 3G for 95% of requests" rather than "fast").
- **Traceability** – each requirement can be traced back to its source and forward to design and tests.
- **Priority** – each requirement is classified (e.g. MoSCoW: Must, Should, Could, Won't).

How (validation techniques)

(i) Requirements reviews / inspections: a team of stakeholders, analysts, designers and testers reads the SRS systematically with a checklist (the properties above) and records problems.

(ii) Walkthroughs with users: the analyst walks the users through use case narratives and scenarios to confirm "is this what you do?".

(iii) Prototyping: an executable or paper prototype (UI mock-up) is shown to users; misunderstandings surface immediately.

(iv) Test-case generation: writing acceptance tests for each requirement; if a test cannot be written the requirement is ambiguous or unverifiable.

(v) Model validation: use case, class, sequence and BPMN models are checked for consistency with each other and with the text.

(vi) Requirements Traceability Matrix (RTM): bi-directional tracing from business goals to requirements to test cases ensures "zero orphan features or missing test plans" and supports impact analysis of changes.

(vii) Formal sign-off of the validated SRS by the client, after which changes go through change control.

Answer. Check that the requirements are valid, consistent, complete, unambiguous, feasible/realistic, verifiable (measurable), traceable and prioritised, using reviews/inspections, user walkthroughs, prototyping, acceptance test-case generation, model consistency checks and an RTM, ending with client sign-off.

2020–2021 • Q2(c) CSE325 • 12/04/2023 • C325-2021-2c **MARKS 8**

Give examples of 4 types of Non-Functional requirements for the example of the application to be developed for the scenario discussed in Q 2(a).

(Scenario of Q2(a): teachers offer courses in semesters, some jointly; students enroll in offerings and receive grades computed by a grading rule from the marks of different test items.)

SOLUTION

Non-functional requirements

Functional requirements state *what the system must do* (enroll, enter marks, compute grades); **non-functional requirements** state *how well* it must do it — qualities and constraints, each written so that it is measurable.

Performance	Grades of an offering with up to 300 students are computed and displayed within 5 seconds; the result page loads within 3 seconds for 95% of requests during result publication.
Security	Only the teacher(s) assigned to an offering can enter or change its marks; students can see only their own grades; all access uses HTTPS and role-based login; every change of a mark is logged (who, when, old/new value).
Availability / reliability	99.5% availability during the mark-entry and result-publication weeks; no mark entered and saved is ever lost (daily backup, recovery within 4 hours).
Usability	A teacher can enter the marks of a 60-student class test using the spreadsheet-like screen in under 10 minutes without training; the student result page works on mobile phones.
Maintainability (extra)	A new grading rule can be added by configuration (rule table) without changing code.

Answer. Performance (grades for 300 students in ≤ 5 s), Security (role-based access, only assigned teachers edit marks, audit log), Availability (99.5% during result week, backups), Usability (mark entry for a class in < 10 min without training; mobile-friendly).

2019–2020 • Q8(b) CSE325 • 02/04/2022 • C325-1920-8b **MARKS 9**

Write down one nonfunctional requirement of each type for the website of a stock exchange in Bangladesh. There are 6 types of requirements.

SOLUTION

Concept

Functional requirements say *what the system must do*; **non-functional requirements (NFRs)** say *how well* it must do it (lecture taxonomy). A good NFR is *unambiguous, verifiable, measurable and feasible*: "the site should be fast" is weak, "95% of quote pages load within 2 s" is testable. The six commonly used NFR types are **performance, security, reliability/availability, usability, scalability (capacity)** and **maintainability/portability**; for a stock exchange **legal/regulatory compliance** is equally important and is given as well.

One NFR of each type for a stock-exchange website (e.g. DSE)

Type	Requirement (measurable)
Performance	During trading hours, live price/index updates shall reach the browser within 1 second of a trade, and 95% of page requests shall be served within 2 seconds.
Security	All traffic shall use HTTPS (TLS 1.2+); broker/investor log-in shall use two-factor authentication, and an account shall be locked for 15 minutes after 5 failed log-in attempts.
Reliability / Availability	The website shall be available 99.9% of the time during trading hours (10:00–14:30, Sunday–Thursday), with recovery from a server failure within 5 minutes and no loss of published market data.
Usability	A first-time investor shall be able to find the latest price and chart of any listed company within 3 clicks / 30 seconds; all pages shall be available in Bangla and English.
Scalability (capacity)	The site shall support at least 100,000 concurrent users (e.g. on IPO-result days) with less than 10% increase in response time, by adding servers without code change.
Maintainability / Portability	A new listed company or a change in circuit-breaker rules shall be configurable by an administrator without redeployment; the site shall render correctly on the latest two versions of Chrome, Firefox, Safari and Edge and on mobile screens ≥ 360 px wide.
Regulatory (extra)	Price-sensitive information shall be published simultaneously to all users as required by BSEC rules, and all disclosures shall be archived for at least 5 years.

NOTE The question does not name the six types; the six most common categories (performance, security, reliability, usability, scalability, maintainability/portability) are assumed. Any recognised taxonomy (e.g. Sommerville's product/organisational/external, or FURPS+) is acceptable if one measurable requirement is given per type.

Answer. Performance (price update ≤ 1 s), security (HTTPS + 2FA + lockout), availability (99.9% in trading hours), usability (any quote in ≤ 3 clicks, Bangla/English), scalability (100k concurrent users), maintainability/ portability (rule changes by configuration; major browsers/mobile).

2018–2019 • Q2(b) CSE325 • 20/12/2021 • C325-1819-2b MARKS 10

Write down examples of 5 types of non-functional requirements for the BIIS application running in BUET.

SOLUTION

Concept

Non-functional requirements describe *how well* BIIS (BUET Integrated Information System – course registration, results, fees, student and teacher portals) performs its functions; each should be stated so that it can be measured and tested.

Five types with BIIS examples

Type	Example for BIIS
Performance	During the course-registration window, the registration page shall respond within 3 seconds for 95% of requests, with up to 5,000 students logged in simultaneously.
Security	Students shall see only their own grades and dues; grade entry shall be possible only for the course teacher during the grade-submission period; passwords stored as salted hashes; session expires after 15 minutes of inactivity; every grade change is logged with user, time and old/new value.
Reliability / Availability	BIIS shall be available 99.5% of the time, and 99.9% during registration and result-publication periods; a daily backup shall allow restoration with at most 24 hours of data loss.
Usability	A first-year student shall be able to complete course registration without help in under 10 minutes; the interface shall work on mobile phones and show clear error messages (e.g. "prerequisite CSE 105 not passed").
Maintainability	A change of academic rules (e.g. credit limits per term, grading scale) shall be applied through configuration tables without changing source code, and a new department shall be added within one working day.

Other valid types: *scalability* (support growth in student numbers), *portability* (runs on any modern browser), *legal/organisational* (complies with BUET academic ordinance and data-privacy rules).

Answer. Performance ($\leq$ 3 s for 95% of requests at 5,000 users), security (role-based access, logging, hashed passwords, timeout), availability (99.5%, 99.9% in peak periods, daily backup), usability (register unaided in $<$ 10 min, mobile-friendly), maintainability (rules changed by configuration).

2017–2018 • Q1(c) CSE325 • 10/04/2019 • C325-1718-1c MARKS 3

Name three types of questions to avoid during an interview. Give one example of each.

SOLUTION

Why some questions must be avoided

Interviews are the main method for eliciting deep needs and domain rules, but they are prone to *interviewer bias*. A badly worded question either pushes the interviewee towards the analyst's own opinion or produces an answer that cannot be interpreted. Three kinds of questions should therefore be avoided:

Type to avoid	Why it is harmful	Example
Leading questions	suggest the "expected" answer, so the interviewee simply agrees and the analyst hears his own bias	"You agree with the other managers that inventory control should be computerised, don't you?"
Double-barrelled questions (two questions in one)	the interviewee answers only one part, or one answer covers both and is ambiguous	"What decisions are made in a typical day and how do you make them?"
Vague / too general (unfocused open-ended) questions	no clear subject, so the answer is long, rambling and useless for requirements	"Tell me everything about how this office works."

Better forms: neutral wording ("How do you feel about computerising inventory control?"), one question at a time, and open questions with a clear focus ("What problems do you face while updating the stock register?").

Answer. Avoid (1) leading questions, (2) double-barrelled (two-in-one) questions and (3) vague/too-general questions – examples as in the table.

2017–2018 • Q1(d) CSE325 • 10/04/2019 • C325-1718-1d MARKS 4

Name two sampling techniques in the context of requirement analysis.

SOLUTION

Sampling in requirement analysis

Sampling is the process of systematically selecting representative elements of a population (documents, forms, records, or people to interview/observe) so that the analyst can learn about the whole population without examining all of it – this reduces cost and time and avoids bias.

Sampling techniques

(i) **Simple random sampling** – every element of the population has an equal chance of being chosen (e.g. pick 50 invoice numbers with a random-number generator).

(ii) **Stratified sampling** (complex random sampling) – the population is first divided into subgroups (strata), and elements are sampled from each stratum in proportion to its size; e.g. sample invoices separately from small, medium and large customers. It gives a representative sample with a smaller sampling error.

Other techniques that can be named: *systematic sampling* (every k-th element, also a complex random sample), *convenience sampling* (whatever is easiest to get – unrestricted, least reliable) and *purposive sampling* (analyst chooses elements by judgement, e.g. only people interested in the system).

Answer. Simple random sampling and stratified sampling (others: systematic, convenience, purposive).

2014–2015 • Q6(c) CSE307 • 04/02/2016 • R021.1 **MARKS 15**

A company has decided to open a training school, which will deal with trainees employed in several departments. It is also intended that the training school will accept students of the local university who will pay a small fee. A new system is required to deal with registration, assigning trainees/students to one of the available classes, allocation of resources, (such as classrooms and tutors from the local university) and attendance. A class is always held in the same classroom. The university tutors will be paid for each hour they teach. They always teach in the same classroom but may teach different classes; however the classroom can be used by different tutors at different times. Briefly describe a fact-finding approach for this system.

SOLUTION

Principle

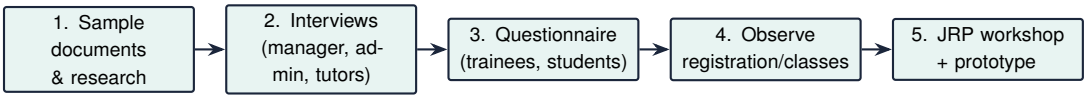
A fact-finding *approach* is a planned sequence of elicitation methods: start with what can be learned without disturbing people (documents, research), then go to people (interviews, surveys, observation), and finish with group confirmation (workshop/prototype). The plan is driven by the stakeholders and by the facts we must discover.

Step 1 – Identify stakeholders and the facts needed

Stakeholder	Facts to find
Training-school manager (sponsor)	Goals, scope, budget, reports needed
Department heads	How trainees are nominated, cost charging per department
Administrative/registration staff	Registration procedure, fee collection from university students
Trainees and university students	Registration channel, timetable needs, attendance reporting
University tutors / tutor coordinator	Availability, hourly payment, classroom assignment
Finance/accounts	Fee receipts, tutor pay calculation (hours × rate)

Business rules already given that must be confirmed: a class always uses one classroom; a tutor always uses the same classroom but may teach several classes; a classroom is shared by tutors at different times; tutors are paid per hour. Open questions to resolve: class capacity, how a student is assigned to a class, attendance thresholds, fee amounts, clash rules for resources.

Step 2 – Sequence of methods



(i) Sampling existing documents – registration forms, class lists, timetables, room booking sheets, attendance registers, tutor time-sheets and payment claims, fee receipts. These give the data items (Trainee, Student, Class, Classroom, Tutor, Attendance, Payment) and volumes. **Research**: look at existing training-management products and the university's own system.

(ii) Interviews – an unstructured interview with the manager (objectives, scope), then structured interviews with registration staff, the tutor coordinator and finance to capture detailed rules (allocation of classrooms and tutors, payment per hour, fee policy).

(iii) Questionnaire to the large group of trainees, department heads and university students: preferred registration channel, time slots, problems with current scheduling.

(iv) Observation of a registration day and a few classes (attendance taking, room usage) to check that documented procedures match reality.

(v) JRP workshop / discovery prototype – bring representatives together to resolve conflicts (e.g. departments vs university students competing for places, room clashes), walk through a prototype of registration and timetable screens, and agree the requirements list (with MoSCoW priorities).

Step 3 – Record and validate

Document facts as FRs (register, assign to class, allocate classroom/tutor, record attendance, compute tutor pay, collect fees) and NFRs (e.g. timetable generated in under a minute); confirm them with the stakeholders and start an RTM.

Answer. Identify stakeholders (manager, departments, admin staff, trainees, university students, tutors, finance) → sample existing forms/timetables/attendance and pay records and research similar systems → interview manager, admin, tutor coordinator and finance → survey trainees/students → observe registration and classes → confirm in a JRP workshop with a prototype; record as FR/NFR and validate.

2014–2015 • Q8(c)(ii) CSE307 • 04/02/2016 • R022.2 **MARKS 6**

ABC Advertising specializes in using large helium balloons to help business with their marketing. They rely on computer processing and databases to manage the distribution of balloons around the country. Customers can hire their advertising balloon by phoning the state branch of ABC Advertising.

Whoever answers the telephone, checks the booking file to see if the requested time by the customer is available. The customer contact details, advertising text, balloon type and install instructions are taken down and entered into the customer file. A total price is negotiated and 10% deposit is requested from the customer. This is written into the accounts file.

When the deposit has been received a receipt is sent to the customer and the accounts file is updated. A designer then uses the advertising details to create a number of design and these are sent to the customer. The customer selects the appropriate design and adds comments for any alteration and returns the design to ABC Advertising. The changes are made and the customer file is updated with the design details that will be used.

The balloon is created and an account is produced and sent to the customer. Installation is then completed at the prearranged time. When the full payment has been received from the customer the account file is updated and a statement showing all payment details is sent to the customer. At the end of the week business transaction report is produced from the customer and account files and this is sent to the head office.

(ii) ABC Advertising is also developing a web site for online booking. They are also considering adding an online payment section for deposits. Describe a fact finding strategy a System Analyst could use to gather information about the sites requirements.

SOLUTION

NOTE Part (i) of this question (centralized vs distributed information system, 6 marks) is not covered by the course slides; it is solved in the out-of-syllabus section under the same ID.

Strategy: from existing knowledge to user confirmation

(i) Sample existing documentation and files – the booking file, customer file, accounts file, receipts, statements and weekly reports. They tell exactly which data the web booking form must capture (contact details, advertising text, balloon type, install instructions, time slot, price, 10% deposit) and the volume of bookings per branch.

(ii) Interviews – with branch telephone staff (how availability is checked, how prices are negotiated – a rule that is hard to put online), the designer (how design proofs could be sent and approved online), accounts staff (deposit, receipts, refunds) and head-office management (goals, budget, reporting).

(iii) Observation – sit with staff while they take phone bookings to see the real sequence and exceptions (double bookings, special requests).

(iv) Questionnaires / online survey – sent to existing customers: would they book online, which payment methods (card, mobile banking), what information they need before booking.

(v) Research and site visits – study competitors' booking sites and payment-gateway requirements (card-security standards such as PCI-DSS, local e-payment regulations).

(vi) Discovery prototyping – a clickable mock-up of the booking and deposit pages reviewed by staff and a few customers.

(vii) JRP / workshop – a short workshop with branch, design, accounts and management representatives to agree requirements and priorities (e.g. Must: online availability check and booking; Should: online deposit; Could: online design approval).

Output: FRs (search availability, book, pay deposit, receive receipt) and NFRs (secure payment, 24/7 availability, response time), prioritised and validated with the stakeholders.

Answer. Sample the booking/customer/accounts files and reports, interview phone staff, designers, accounts and management, observe phone bookings, survey customers about online booking and payment, research competitor sites and payment-security rules, then confirm with a prototype and a JRP-style workshop.

2013–2014 • Q1(a),(b) CSE307 • 08/08/2015 • R023.1 **MARKS 12 + 12**

(a) You are to analyze the requirements for a government office to automate the filing system. Describe how fact finding, questionnaire and interviews play different roles to elicit requirements. (b) Design a typical room JRP (Joint Requirement Planning) session. Mention the roles and responsibilities of different participants.

SOLUTION

(a) Roles of fact finding, questionnaires and interviews

In a government filing office, requirements are hidden in three places: in the *files and procedures* themselves, in the *heads of experienced officers*, and spread across *many clerks and citizens*. Each technique targets one of these.

1. General fact finding (document sampling, research, observation) – "what exists".

- Sample existing files, registers, file-movement registers, forms, notings, circulars and the office instructions/rules that govern filing.
- Observe how a file is opened, numbered, moved between desks, noted and archived; measure volumes and delays.
- Role: produces the *factual baseline* – data items (file no., subject, owner, movement history), document volumes, workflow – and the questions to ask later. It is objective and does not disturb people much.

2. Questionnaires – "breadth".

- Send a structured (mostly fixed-format) questionnaire to the many clerks/section officers across branches, and optionally to citizens who submit applications.
- Role: collects *quantifiable* facts and opinions from a large, scattered population (files handled per day, time to locate a file, common problems, IT literacy) quickly and anonymously, and lets findings be validated statistically. Weakness: shallow answers, low response.

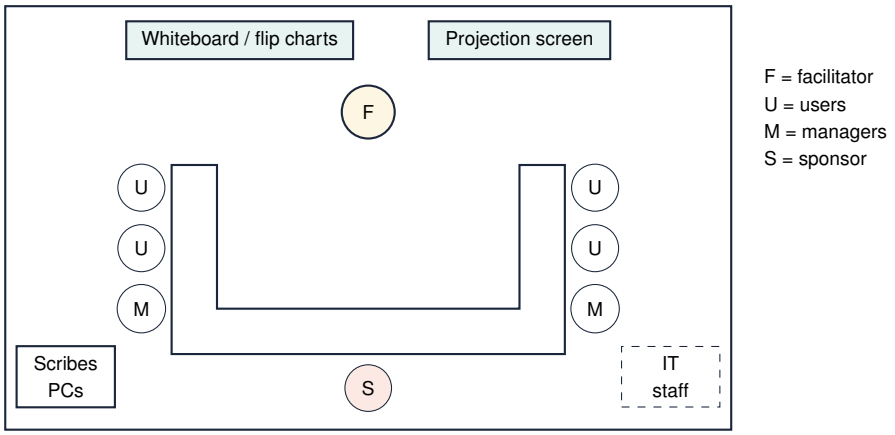
3. Interviews – "depth".

- Interview senior officers, record keepers, the IT cell and decision makers (unstructured first, then structured).
- Role: uncovers *deep needs and domain rules* – approval hierarchy, confidentiality classes, retention/destruction rules, legal validity of e-signatures – and clarifies anomalies found in the questionnaire. Also builds trust and support for the change. Weakness: time, interviewer bias, does not scale.

Together: fact finding gives the *facts*, questionnaires give the *breadth*, interviews give the *depth*.

(b) A typical JRP session

Room layout. An off-site room with a U-shaped table so that everyone can see the facilitator and the displays; scribes at a side table with computers/CASE tools; IT staff at the back as observers.



Participants – roles and responsibilities

Participant	Responsibilities
Sponsor (executive)	Authorises and funds the session, selects participants, opens/closes it, makes final decisions when consensus fails; usually not present throughout.
Facilitator	Plans the agenda, runs the meeting, keeps it on track, enforces ground rules, mediates conflicts, ensures everyone (not just dominant voices) contributes; must be neutral.
Users	Describe business processes, problems and needs; propose and approve requirements – the main source of facts.
Managers	Explain objectives, policies, priorities and constraints; approve requirements at the business level.
Scribe(s)	Record all facts, decisions, open issues and models (often with CASE/prototyping tools); produce the JRP document.
IT staff (analysts, designers)	Listen, answer technical questions, assess feasibility; they do not dominate the discussion.

Agenda of a typical session – opening by sponsor (objectives, scope) → facilitator sets ground rules → review of the current filing process → brainstorm problems and requirements → discuss and prioritise (MoSCoW) → walk through models/prototype → list open issues with owners → summary and closing. Afterwards the scribe circulates the JRP document for validation.

Answer. (a) General fact finding (documents, observation) establishes the objective baseline of files, forms, volumes and workflow; questionnaires gather quantitative facts/opinions from many clerks and citizens; interviews obtain deep rules and clarifications from key officers. (b) An off-site U-shaped room with facilitator and displays at the front; sponsor (authorises, decides), facilitator (runs session), users and managers (supply and approve requirements), scribes (record), IT staff (advise on feasibility).

2013–2014 • Q5(c),(d),(e) CSE307 • 08/08/2015 • R025.3 MARKS 5 + 14 + $10\frac{2}{3}$

(c) What is discovery prototyping?
(d) Briefly describe the fact finding methods.
(e) What is meant by JRP? Describe its advantages and disadvantages.

SOLUTION

(c) Discovery prototyping

Discovery prototyping is the use of a quick, small-scale working model (screen mock-ups, report layouts, a clickable front end) built *during requirements analysis* purely to help users discover and confirm their requirements. Users often say "I'll know it when I see it"; reacting to a prototype is easier than describing needs in the abstract.

- It is built fast with little attention to internal quality and is normally *thrown away* once requirements are understood (the knowledge, not the code, is kept).
- It is the requirements-side use of the *prototyping model*: quick plan → quick design → construct prototype → user feedback, repeated until needs are clear.
- Risk: users may think the system is almost finished, or focus on cosmetics instead of business rules.

(d) Fact-finding (elicitation) methods

Fact-finding is the formal process of using research, meetings, interviews, questionnaires, sampling and other techniques to collect information about systems, requirements and preferences. The classic seven methods (the lecture's elicitation methods are the core of this list):

(i) **Sampling of existing documentation, forms and files** – study organisation charts, forms, reports, manuals, databases, previous system documents; use random or stratified sampling when there are too many documents.

(ii) **Research and site visits** – study the problem domain (literature, trade journals, similar systems, other organisations that solved the same problem).

(iii) **Observation of the work environment** (lecture: *observational/field study*) – watch users do their real work; reveals workarounds and real workflows. Watch out: time-consuming; people behave differently when observed (observer effect).

(iv) **Questionnaires** (lecture: *quantitative surveys*) – structured forms (free-format or fixed-format) sent to many people; cheap, anonymous, statistically analysable. Watch out: shallow answers, low response rate.

(v) **Interviews** (lecture: *1-on-1 interviews*) – structured or unstructured face-to-face conversations; give deep needs and domain rules. Watch out: interviewer bias, does not scale.

(vi) **Discovery prototyping** – see (c).

(vii) **Joint Requirements Planning (JRP)** (lecture: *co-design workshops / focus groups*) – a facilitated group workshop, see (e).

(e) JRP – Joint Requirements Planning

JRP is a highly structured, facilitated group-work session (usually 3–5 days, off-site) in which system owners, users, analysts and designers jointly identify, analyse, define and agree on system requirements. It is a specialisation of JAD (Joint Application Development) and is what the lecture calls a *co-design workshop*. Participants: *sponsor*, *facilitator*, *users and managers*, *scribe(s)* and *IT staff*.

Advantages	Disadvantages
Drastically reduces elapsed time of fact finding (weeks of interviews compressed into a few days)	Requires a skilled, trained facilitator; poor facilitation wastes many expensive people's time
Users and managers actively participate → ownership and commitment to the system	Hard to schedule: busy key people must be away from their jobs together
Conflicts between stakeholders are surfaced and resolved in the room; quick consensus	Dominant voices or senior managers may suppress others' opinions (groupthink)
Requirements are documented and confirmed on the spot (scribe, CASE tools, prototypes)	Costly (venue, travel, preparation); too much detail may be forced into too short a time
Fewer misunderstandings; higher-quality, agreed specification	If the wrong participants attend, wrong requirements are agreed with false confidence

Answer. (c) A throw-away, quickly built model used during analysis so users can react and discover/confirm requirements. (d) Seven fact-finding methods: sampling documents/forms/files, research & site visits, observation, questionnaires, interviews, discovery prototyping, JRP. (e) JRP is a structured, facilitated multi-stakeholder workshop to define requirements; it saves time and builds consensus and ownership, but is costly, hard to schedule and depends on a good facilitator.

2012–2013 • Q5(d) CSE307 • 17/05/2014 • R031.2 **MARKS** $4\frac{2}{3}$

What is JRP? Who are the JRP participants?

SOLUTION

Definition

Joint Requirements Planning (JRP) is a structured, facilitated group workshop in which system owners, users, analysts and designers meet together (typically off-site for a few days) to identify, analyse, define and agree on system requirements. It is a subset of JAD and corresponds to the *co-design workshop* elicitation method of the lecture – best when stakeholders conflict and quick consensus is needed.

Participants

- **Sponsor** – senior executive who authorises and funds the session and has the final say.
- **Facilitator** – neutral leader who plans the agenda, runs the meeting and resolves conflicts.
- **Users and managers** – supply facts, needs and priorities, and approve requirements.
- **Scribe(s)** – record facts, decisions and open issues; produce the session documents.
- **IT staff** – analysts/designers who listen, answer technical questions and judge feasibility.

Answer. JRP is a facilitated multi-stakeholder requirements workshop. Participants: sponsor, facilitator, users, managers, scribes and IT staff.

2012–2013 • Q7(b) CSE307 • 17/05/2014 • R033.3 **MARKS 15**

Assume that you are designing an information system for a health center. Possible users of your system are doctor, nurse, patient, receptionist, pharmacist, attendant and patient. What are the seven fact finding methods? Argue which fact finding method(s) you should select for each user?

SOLUTION

The seven fact-finding methods

(i) Sampling of existing documentation, forms and files.

(ii) Research and site visits.

(iii) Observation of the work environment (field study).

(iv) Questionnaires (surveys).

(v) Interviews.

(vi) Discovery prototyping.

(vii) Joint Requirements Planning (JRP / co-design workshop).

(Each is described in R025.3(d) above. The lecture's guidance: *interviews* for deep needs and domain rules but they do not scale; *observation* for real workflows but time-consuming; *surveys* for many users but shallow; *workshops* for conflicting stakeholders and quick consensus.)

Choosing methods per user – the criteria

For each user group ask: How many are there? How busy are they? Is their knowledge *tacit* (shown in work) or *explicit* (can be told)? Do they produce documents? Is privacy an issue?

User	Methods	Argument
Doctor	Short structured *interviews*; *sampling* of prescriptions/case notes; *JRP* (one representative); *prototype* review	Few, very busy, hold complex clinical rules – a focused interview extracts deep knowledge in little time; documents show real data needs; a prototype lets them confirm screens quickly.
Nurse	*Observation* of ward rounds + *interviews*	Much work is tacit (vitals, medication rounds, hand-over); watching reveals actual workflow and workarounds.
Patient	*Questionnaires/surveys* (and a small focus group)	Very many, varied and available only briefly; surveys give quantifiable views at low cost; consultations cannot be observed for privacy.
Receptionist	*Observation*, *sampling* of appointment registers/forms, *discovery prototyping*	Repetitive, form-based work; documents reveal data fields; a prototype of the registration/appointment screen gives fast feedback on usability.
Pharmacist	*Sampling* of stock ledgers and prescriptions, *interview*, *research* on drug regulations	Well-documented, rule-based work (stock, expiry, dispensing rules, legal limits).
Attendant	*Observation* + informal *interview*	Simple manual tasks; may be less comfortable with written questionnaires; easiest to learn by watching.
All / managers	*Research & site visits* (other hospitals' systems), final *JRP* workshop	Learn from existing solutions; JRP resolves conflicts between groups (e.g. doctor vs pharmacy rules) and confirms requirements.

NOTE The question lists "patient" twice; it is treated as one user group.

Answer. Seven methods: document sampling, research & site visits, observation, questionnaires, interviews, discovery prototyping, JRP. Doctors – short interviews, document sampling, prototype review; nurses and attendants – observation + interviews; patients – questionnaires; receptionist – observation, form sampling, prototyping; pharmacist – document sampling, interview, research; finally a JRP workshop with representatives of all groups.

2011–2012 • Q5(a) CSE307 • 23/09/2013 • R039.2 **MARKS 10**

Classify each of the following statements according to its most appropriate requirement category by indicating 1, 2, 3, 4, 5, 6 or 7, where the numbers denote the following categories:

1 – The sentence is a performance requirement.
2 – The sentence is a portability requirement.
3 – The sentence is a security requirement.
4 – The sentence is a usability requirement.
5 – The sentence is an availability requirement.
6 – The sentence is a maintainability requirement.
7 – The sentence is a domain requirement.

	Statement	Category
i)	The system must authenticate the user at each new session.	
ii)	On average, no more than 2 failures requiring a system restart shall occur within a month of operation.	
iii)	At most 5% of the source code shall be operating system specific.	
iv)	The methods of the system shall have a maximum cyclomatic complexity of 6.	
v)	The system shall be able to run with all functionality enabled within 250 MB of internal memory and 2 GB of external memory.	
vi)	The rocket shall reach a speed of 11.2 km/s in order to evade earth gravity.	
vii)	The system shall be operational on Windows XP, Windows Vista, and Windows 7.	
viii)	The system shall require less than five mouse clicks to add a weekly meeting to a user's calendar.	
ix)	After a 2 hours training period, a regular user shall be able to add a new customer within 3 minutes.	
x)	The Olympics Ticketing System shall be able to process 100 ticket requests per minute at peak load.	

SOLUTION

How to decide

Ask *which quality the statement measures*:

- **Performance** – speed, throughput, response time, resource (memory/CPU) consumption.
- **Portability** – ease of moving to / running on other platforms (OS, hardware).
- **Security** – protection against unauthorised access (authentication, authorisation, encryption).
- **Usability** – effort needed to learn and use (clicks, task time, training time).
- **Availability** – proportion of time the system is up (failures, restarts, MTBF, uptime %).
- **Maintainability** – ease of understanding and changing the code (complexity, modularity).
- **Domain** – a fact/rule of the application domain (physics, law, business), not a software quality.

Classification

#	Cat.	Category	Justification
i	3	Security	Authentication at every session prevents unauthorised use.
ii	5	Availability	Limits failures/restarts per month, i.e. how often the system is down.
iii	2	Portability	Limits OS-specific code so the system can be ported to another OS.
iv	6	Maintainability	Cyclomatic complexity is a code-understandability/changeability metric.
v	1	Performance	Resource (memory/space) efficiency constraint.
vi	7	Domain	Escape velocity is a law of physics from the rocket domain.
vii	2	Portability	Must run on several operating-system platforms.
viii	4	Usability	Ease of use measured in number of clicks.
ix	4	Usability	Learnability: training time and task time for a regular user.
x	1	Performance	Throughput (100 requests/min) at peak load.

NOTE Statement (v) is sometimes argued to be portability (it constrains the target machine), but the standard reading treats memory/space limits as an *efficiency (performance)* requirement; (ii) is a reliability statement used here as the availability category, the closest one offered.

Answer. i–3, ii–5, iii–2, iv–6, v–1, vi–7, vii–2, viii–4, ix–4, x–1.

2010–2011 • Q7(a) CSE307 • 19/03/2012 • E004 MARKS $6\frac{2}{3}$

Discuss about professional and ethical responsibilities of a software engineer.

SOLUTION

Why a software engineer has special responsibilities

"With great power comes great responsibility." Software controls money, health, transport and personal data, so a software engineer's work can cause great harm. Beyond legal obligations, engineers are bound by *professional* norms (good practice) and *ethical* norms (doing what is right).

(1) Classical professional responsibilities

- **Confidentiality** – respect the confidentiality of employers and clients, whether or not a formal agreement is signed.
- **Competence** – do not misrepresent your level of competence or knowingly accept work outside it.
- **Intellectual property rights** – respect patents, copyright and licences (slide dimension 8: fair licensing, e.g. GitHub Copilot lawsuits).
- **Computer misuse** – never use technical skills to misuse others' computers (from playing games on an employer's machine to spreading malware or breaking into systems).

(2) The eight principles of the IEEE-CS/ACM code

Principle	Responsibility
1 Public (top priority)	Act consistently with public health, safety and welfare.
2 Client & Employer	Serve client/employer loyally while remaining compliant with the public good.
3 Product	Ensure products meet the highest professional standards; rigorous QA.
4 Judgment	Maintain integrity and independence in professional judgment.
5 Management	Managers promote an ethical approach to development and maintenance.
6 Profession	Advance the integrity and reputation of the profession.
7 Colleagues	Be fair to and supportive of colleagues.
8 Self	Lifelong learning; promote an ethical approach to practice.

(3) Modern ethical dimensions (lecture)

Data privacy (informed consent, minimal collection), bias & fairness auditing, accountability for system outcomes, transparency/explainability, user autonomy (no dark patterns), security as an ethical duty, environmental sustainability and IP/authorship in AI. Ethics must be treated as a design constraint: at design time ask "Who could this harm, and how would we know?"

Answer. A software engineer must keep client confidentiality, work only within competence, respect intellectual property and never misuse computers, and must follow the eight IEEE-CS/ACM principles – Public first, then Client & Employer, Product, Judgment, Management, Profession, Colleagues and Self – including privacy, fairness, transparency and security as ethical duties.

2010–2011 • Q7(b) CSE307 • 19/03/2012 • R047 MARKS 25

As a part of ensuring eGovernance goal, the Government is going to computerize its land management process that keeps records about land ownership and transfers of ownerships. It is your responsibility to do the requirement analysis for such a project. What fact finding techniques you may use and how?

SOLUTION

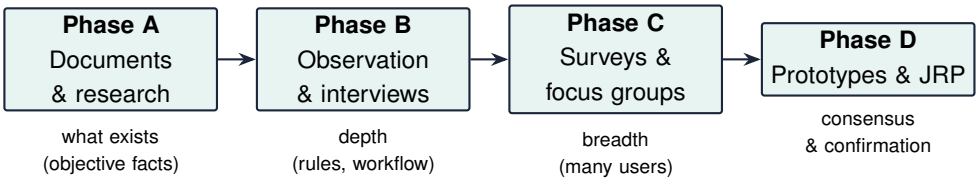
Understanding the problem

Land management is a large, legally sensitive, geographically distributed domain. Records of rights (khatian), mouza maps, deeds and mutation (name-transfer) cases are kept in thousands of land offices, sub-registry offices and survey offices. Requirements therefore come from *laws and documents*, from *expert officials*, and from *millions of citizens*. No single technique is enough – we combine the fact-finding (elicitation) techniques, each for what it does best.

Step 1 – Stakeholder identification

Stakeholder	Interest / knowledge
Ministry of Land (sponsor), policy makers	Goals, scope, legal framework, budget
Land office officials (AC Land, union land officers)	Mutation, land tax, record-keeping rules
Sub-registry offices (registration authority)	Deed registration and transfer of ownership
Survey department	Mouza maps, plot boundaries, survey records
Citizens – landowners, buyers, sellers	Services they need, problems (delay, fraud, harassment)
Deed writers, lawyers, courts, banks	Searches, disputes, mortgages, verification

Step 2 – Techniques and how to apply them



1. Sampling of existing documentation, forms and files. *How:* collect samples of khatians (record of rights), deeds, mutation case files, land-tax receipts, mouza maps, registers and the governing laws and rules (tenancy, registration and mutation rules). Because there are millions of records, use *stratified sampling* – e.g. pick offices from every division, urban and rural, and records of each type and age. *Yields:* data items and formats (plot/dag no., khatian no., owners' shares, area), record volumes, inconsistencies, and the legal rules the system must enforce.

2. Research and site visits. *How:* study land-record systems of other countries, existing government digital services (e.g. online mutation pilots), GIS and digital-signature standards; visit a few land and sub-registry offices. *Yields:* proven solutions, technical options, standards and lessons learned.
3. Observation of the work environment (field study). *How:* spend days in selected land offices following a mutation case from application to approval, a deed registration and a record search; time each step, note hand-offs and workarounds (handwritten notes, informal registers). *Yields:* the real (not the documented) workflow, bottlenecks and points where fraud or delay happens. Observe several offices to reduce the observer effect.
4. Interviews. *How:* unstructured interviews with ministry officials to understand goals and scope; then structured interviews with AC Land officers, sub-registrars, surveyors and record keepers about each business rule (who approves a mutation, how inheritance shares are split, how a dispute freezes a record). Interview banks, courts and lawyers about verification needs. Prepare questions from Phase A, take notes and confirm them with the interviewee. *Yields:* deep domain rules and exceptions.
5. Questionnaires / surveys. *How:* a fixed-format questionnaire to all land offices (staff, record volumes, computers, internet, IT skills) and a citizen survey (paper at offices, online and SMS) about the services needed, time/cost currently spent and preferred channels (web, mobile, union digital centre). *Yields:* quantitative data for capacity planning and prioritisation – far too many people to interview individually.
6. Focus groups. *How:* small facilitated sessions with landowners, farmers and deed writers in a few districts. *Yields:* shared priorities and concerns (fraud, transparency, access for low-literacy users).
7. Discovery prototyping. *How:* build mock-ups of key screens – citizen mutation application, ownership search, officer's approval screen, map view – and let officials and citizens try them. *Yields:* confirmed data fields, usability needs (Bangla interface, few simple steps), hidden requirements.
8. JRP (co-design workshop). *How:* a facilitated multi-day workshop with representatives of the land ministry, registration, survey, IT and citizen bodies, chaired by the sponsor, to integrate the processes of the different agencies (registration → mutation → record update should flow automatically), resolve conflicts and agree priorities. *Yields:* an agreed, prioritised requirement list.

Step 3 – From facts to a specification

- Classify findings: FRs (apply for mutation online, register a transfer, search ownership, pay land tax, issue certified khatian, update maps) and NFRs (legal validity and audit trail of every change, security and role-based access, availability in all upazilas, Bangla UI, response time, data integrity, privacy).
- Model with use-case diagrams (citizen, officer, registrar), a class diagram of Land/Plot/Owner/Deed and BPMN of the transfer process.
- Prioritise with MoSCoW (e.g. Must: digital record of rights and mutation; Could: GIS map integration in phase 2).
- Validate with stakeholders and maintain an RTM linking each requirement to its source (law, interview, survey) and later to design and tests.

EXAM TIP For large "which fact-finding techniques and how" questions, organise the answer as *technique → whom → how → what you get*; listing technique names alone earns few marks.

Answer. Identify stakeholders, then use: sampling of land records, deeds and laws (stratified across offices); research & site visits; observation of land/registry office workflows; structured interviews with officials and experts; questionnaires to all offices and citizens; focus groups of landowners; discovery prototypes of citizen/officer screens; and a JRP workshop across agencies to agree requirements. Record findings as FR/NFR, prioritise (MoSCoW), validate and trace (RTM).

12

Equivalence Partitioning & Coverage

2023–2024 • Q5(a) CSE213 • 10/05/2026 • T001 **MARKS 15**

A university course-registration system decides whether a student may register for a lab section based on the following three independent inputs:

Completed Credits	CGPA	Status
C1: less than 60	**G1:** below 2.50	**S1:** not approved by advisor
C2: 60 to 99	**G2:** 2.50 to 3.49	**S2:** approved by advisor
C3: more than 99	**G3:** 3.50 or above	**S3:** invalid or missing status
C4: invalid credit input	**G4:** invalid CGPA input	

Using Equivalence Class Testing (ECT), perform the following:

(i) Construct a minimal weak ECT test suite for this system.

(ii) Compute the total number of test cases required for strong ECT.

(iii) Illustrate one defect scenario that may remain undiscovered if only the weak ECT suite is used.

SOLUTION

Concepts

In equivalence class testing each input is divided into classes that the program treats the same way, and one representative is chosen per class.

- **Weak ECT** (single-fault assumption): every class of every input must appear in *at least one* test case. Minimum size $=$ max(#classes of any input).
- **Strong ECT** (multiple-fault assumption): *every combination* of classes is tested, i.e. the Cartesian product. Size $=$ $\prod$(#classes of each input).

NOTE The decision rule is not printed. Assumed: registration is *allowed* when credits and CGPA are valid and the status is S2; it is *denied* for S1; any invalid input (C4, G4, S3) gives a validation error. CGPA is on a 0.00–4.00 scale; credits are non-negative integers.

Representative values

Class	Value	Class	Value	Class	Value
C1	45	G1	2.10	S1	NOT_APPROVED
C2	75	G2	3.00	S2	APPROVED
C3	120	G3	3.75	S3	“” (missing)
C4	-10 (or “abc”)	G4	4.70 (or “x”)		

(i) Minimal weak ECT suite

Credits and CGPA have 4 classes each, Status has 3, so at least $\max(4,4,3) = 4$ test cases are needed – and 4 are enough, because each test can carry one class of every input at the same time:

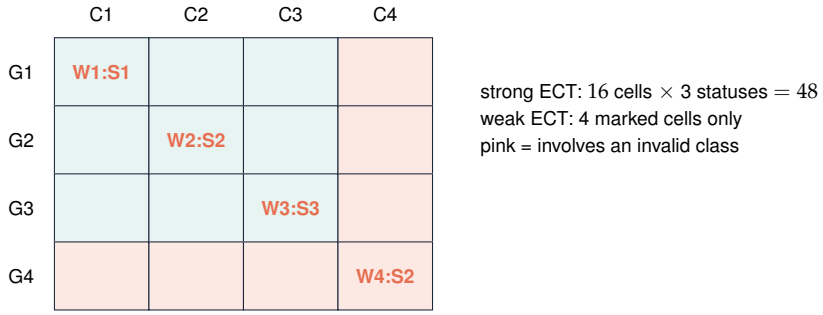
TC	Credits	CGPA	Status	Classes	Expected output
W1	45	2.10	NOT_APPROVED	C1, G1, S1	Registration denied (advisor approval missing)
W2	75	3.00	APPROVED	C2, G2, S2	Registration allowed
W3	120	3.75	(missing)	C3, G3, S3	Error: invalid / missing status
W4	−10	4.70	APPROVED	C4, G4, S2	Error: invalid credit and CGPA input

All $4 + 4 + 3 = 11$ classes appear at least once, so the suite satisfies weak ECT with the minimum number of tests. (S2 is reused in W4 because Status has only three classes.)

(ii) Number of test cases for strong ECT

$$N_{\text{strong}} = |C| \times |G| \times |S| = 4 \times 4 \times 3 = \mathbf{48}.$$

For comparison, if only the *valid* classes were combined (strong *normal* ECT) the count would be $3 \times 3 \times 2 = 18$; the question lists the invalid classes as classes, so the answer is 48. The grid shows the Credits × CGPA plane: strong ECT fills all 16 cells (each for 3 statuses), weak ECT touches only the 4 marked cells.



(iii) A defect that weak ECT can miss

Weak ECT assumes failures are caused by *one* input alone. A fault caused by an *interaction* of inputs can slip through. Suppose the real policy also says *"a student with fewer than 60 credits and CGPA below 2.50 (on probation) may not take this lab, even if the advisor approved"*, and the developer forgot this check:

```
boolean mayRegister(int credits, double cgpa, Status s) {
    if (!valid(credits) || !valid(cgpa) || s == null) throw new InvalidInputException();
    return s == Status.APPROVED;      // BUG: probation rule (C1 and G1) is missing
}
```

- The faulty combination is **(C1, G1, S2)**: e.g. 45 credits, CGPA 2.10, approved ⇒ expected "denied", actual "allowed".
- The weak suite contains C1 and G1 only in **W1**, where the status is S1 – the student is denied anyway (for the advisor reason), so W1 *passes* and the bug stays hidden.
- Strong ECT necessarily contains the cell (C1, G1, S2) and exposes the defect.

A second weakness: W4 contains two invalid classes together; if the program reports only the first error ("invalid credit"), a missing CGPA validation is *masked* by the credit error.

Answer. (i) Weak ECT needs $\max(4,4,3) = 4$ tests: (45, 2.10, S1), (75, 3.00, S2), (120, 3.75, S3), $(-10, 4.70, S2)$. (ii) Strong ECT needs $4 \times 4 \times 3 = 48$ tests. (iii) An interaction fault such as a missing probation rule for (C1, G1, S2) is never executed by the weak suite and goes undetected.

2023–2024 • Q7(b) CSE213 • 10/05/2026 • T004.2 **MARKS 10**

A smart irrigation system activates a water pump based on an integer moisture sensor reading M. The specification is as follows:

- Pump is OFF if $M < 30$
- Pump runs at LOW power if $30 \leq M \leq 70$
- Pump runs at HIGH power if $M > 70$

Derive the complete set of test cases for the LOW-power partition using Boundary Value Analysis.

SOLUTION

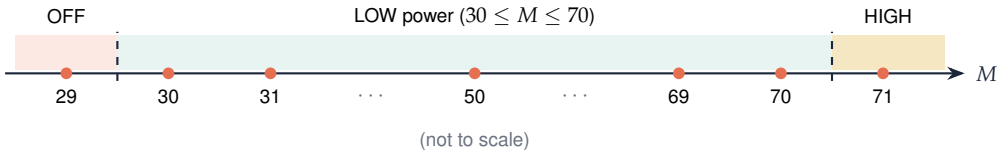
Concept

Boundary value analysis (BVA) tests at the *boundaries between partitions*, because more errors occur at the edges of the input domain (wrong < vs <=, off-by-one) than in its centre. For a closed integer range $[\min, \max]$ the complete (robust) set is

$$\min -1, \quad \min, \quad \min +1, \quad \text{nominal}, \quad \max -1, \quad \max, \quad \max +1.$$

Applying it to the LOW partition $30 \leq M \leq 70$

Since M is an *integer*, the nearest neighbours of a boundary differ by 1: min $= 30$, max $= 70$, nominal value e.g. 50.



TC	Role in BVA	Input M	Expected pump state	Type
B1	min -1 (just below)	29	OFF	just outside LOW
B2	min (on boundary)	30	LOW	valid
B3	min $+1$ (just above)	31	LOW	valid
B4	nominal	50	LOW	valid
B5	max -1 (just below)	69	LOW	valid
B6	max (on boundary)	70	LOW	valid
B7	max $+1$ (just above)	71	HIGH	just outside LOW

What each test catches

- B1/B2 detect a lower bound coded as `M > 30` (30 would wrongly be OFF) or `M >= 29`.
- B6/B7 detect an upper bound coded as `M < 70` (70 would wrongly be HIGH) or `M <= 71`.
- B3, B5 confirm the behaviour just inside the edges; B4 checks a typical value.

If invalid sensor readings are considered (e.g. a moisture percentage outside 0–100), extra negative tests such as $M = -1$ or $M = 101$ belong to the OFF/HIGH partitions' analysis, not to the LOW partition.

Answer. Complete BVA set for LOW power: $M = 29$ (OFF), 30, 31, 50, 69, 70 (LOW), 71 (HIGH) – seven test cases.

2023–2024 • Q1(c) CSE213 • 09/02/2025 • T005 **MARKS 10**

Context (Q1): A mental health clinic is using an information system named Mentcare, which is designed and used to maintain diagnostic and treatment related records for mental patient. Clinic staff log on to the system using a username and password. The system requires passwords to be at least eight letters long but allows any password to be set without further checking. A receptionist may transfer data from the Mentcare system to a national patient record system (PRS) database that is maintained by the health authority of the country. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
A criminal finds out that a well-paid sports star is receiving treatment for mental health problems. He would like to gain illegal access to information in this system so that he can blackmail the star. By posing as a concerned relative and talking with the nurses in the mental health clinic, he discovers how to access the system and personal information about the nurses and their families. By checking name badges, he discovers the names of some of the people allowed access. He then attempts to log on to the system by using these names and systematically guessing possible passwords, such as the names of the nurses' children.
(b) Do you believe that the mental health clinic maintains proper IT security policy? Justify your answer. Design an IT security policy for the use of password for the clinic. (4+10=14)

(c) Design test cases for checking the proposed use of password policy stated in Question 1(b).

SOLUTION

The policy being tested

Part (b) is answered elsewhere; the test design below assumes the following password policy was proposed there (it directly counters the attack in the case: guessable personal names, unlimited guessing).

NOTE **P1** length 10–64 characters. **P2** at least one upper-case letter, one lower-case letter, one digit and one special character. **P3** must not contain the username, the staff member's own name or recorded family names, nor dictionary words such as "password", "mentcare". **P4** none of the last 5 passwords may be reused. **P5** account locked for 30 minutes after 5 consecutive failed log-ins. **P6** password expires after 90 days. **P7** passwords are masked on screen and stored only as salted hashes.

Approach

Each rule becomes an input condition. Using **equivalence partitioning** we choose one valid password (positive test) and one password from each invalid class (negative tests); using **boundary**

value analysis we test the length limits 9/10/11 and 63/64/65 and the lock-out count 4/5. Staff user for the tests: `nurse.rina`, child's name "Tanim".

Test cases

TC	Rule	Input / action	+/–	Expected result
PW1	all	set `Gr8#Clinic24`	+	accepted
PW2	P1	9 chars `Ab3#efghi`	–	rejected "min. 10 characters"
PW3	P1	exactly 10 chars `Ab3#efghij`	+	accepted (lower boundary)
PW4	P1	exactly 64 / 65 chars (valid mix)	+ / –	64 accepted, 65 rejected
PW5	P2	`abcdefgh12#` (no upper case)	–	rejected, rule stated
PW6	P2	`Abcdefghij#` (no digit)	–	rejected
PW7	P2	`Abcdefgh123` (no special)	–	rejected
PW8	P3	`Tanim#2015x` (child's name)	–	rejected "contains personal name"
PW9	P3	`nurse.Rina1#` (contains username)	–	rejected
PW10	P3	`Password#123` (dictionary word)	–	rejected
PW11	P4	reuse the password set 2 changes ago	–	rejected "recently used"
PW12	P5	4 wrong log-ins, then correct password	+	log-in succeeds, counter reset
PW13	P5	5 wrong log-ins, then correct password	–	account locked, log-in refused
PW14	P5	correct password 31 min after lock	+	log-in succeeds
PW15	P6	log in on day 89 / day 90 after change	+ / –	day 89 normal; day 90 forced change
PW16	P7	type password; inspect user table in DB	+	shown as dots; DB holds hash, not text
PW17	P1	empty password field	–	"password required"

PW8–PW10 together with PW13 directly reproduce the attacker's strategy from the case study (guessing children's names repeatedly) and must all be blocked.

Answer. Treat every password rule as an input condition: one valid password (PW1), one invalid class per rule (length, composition, personal/dictionary words, reuse, lock-out, expiry, storage) and boundary values (9/10, 64/65 characters; 4/5 failed attempts; day 89/90) – test cases PW1–PW17.

2023–2024 • Q3(b) CSE213 • 09/02/2025 • T006.2 MARKS 10

Generate test cases for white-box testing from the following code segment:

```
FindMean (FILE ScoreFile){
    float SumOfScores = 0.0, Mean = 0.0, Score;
    int NumberOfScores = 0;

    Read(ScoreFile, Score);
    while (! EOF(ScoreFile) {
        if (Score  > 0.0 ) {
            SumOfScores = SumOfScores + Score;
            NumberOfScores++;
        }
        Read(ScoreFile, Score);
    }

    /* Compute the mean and print the result */
```

```
    if (NumberOfScores > 0) {
        Mean = SumOfScores / NumberOfScores;
        printf(" The mean score is %f\n", Mean);
    } else
        printf ("No scores found in file\n");
}
```

SOLUTION

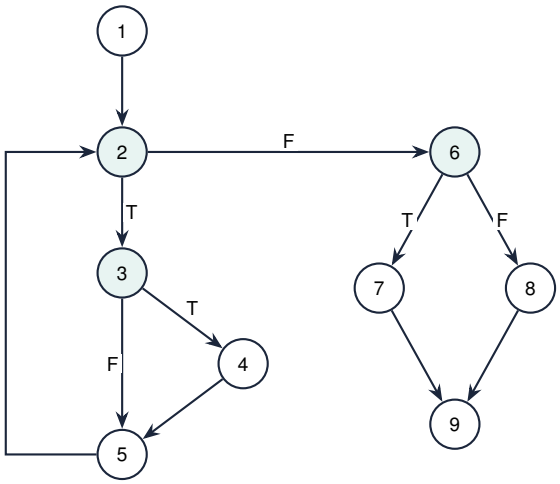
Approach

White-box testing uses the internal structure of the code. We draw the control-flow graph (CFG), list the statements, branches and conditions, and choose inputs (score files) that reach full **statement**, **branch** and **condition coverage**; the independent paths are listed as well so that each logical route is exercised.

> **NOTE** The printed `while` line misses a closing parenthesis; it is read as `while (!EOF(ScoreFile))`. `EOF` is assumed to become true when a `Read` finds no further score (the value read before that is processed).

Control-flow graph

Node	Statements	Node	Statements
1	declarations, `Read(ScoreFile, Score)`	6	`if (NumberOfScores > 0)`
2	`while (!EOF(ScoreFile))`	7	`Mean = ...; printf(mean)`
3	`if (Score > 0.0)`	8	`printf("No scores ...")`
4	`SumOfScores += Score; NumberOfScores++`	9	exit
5	`Read(ScoreFile, Score)`		



Coverage items

- **Statements**: 12 executable statements (2 declarations, 2 `Reads`, `while`, 2 `ifs`, sum, increment, mean, 2 `printfs`).
- **Branches**: 6 outcomes – 2T/2F, 3T/3F, 6T/6F. Each decision has a single atomic condition, so **condition coverage = branch coverage** here.

- **Independent paths** ($V(G) = 3$ decisions $+1 = 4$):
  P1: 1–2–6–8–9 (empty file)
  P2: 1–2–3–4–5–2–6–7–9 (only positive scores)
  P3: 1–2–3–5–2–6–8–9 (only non-positive scores)
  P4: 1–2–3–4–5–2–3–5–2–6–7–9 (positive and non-positive scores)

Test cases

TC	ScoreFile contents	Path / branches covered	Expected output
TC1	empty file	P1: 2F, 6F	"No scores found in file"
TC2	80, −5	P4: 2T, 3T, 3F, 2F, 6T	"The mean score is 80.000000"
TC3	70, 90	P2: 2T, 3T, 2F, 6T	"The mean score is 80.000000"
TC4	0, −3	P3: 2T, 3F, 2F, 6F	"No scores found in file"

Coverage computation

Test set	Statement	Branch / condition	Basis paths
{TC1}	6/12 = 50%	2/6 = 33.3%	1/4
{TC2}	11/12 = 91.7%	5/6 = 83.3%	1/4
{TC1, TC2}	12/12 = **100%**	6/6 = **100%**	2/4
{TC1, TC2, TC3, TC4}	100%	100%	4/4 = **100%**

So the **minimal** test set for 100% statement and branch (= condition) coverage is {TC1, TC2}: TC2 alone misses only the `else printf`, which needs a file with no positive score. TC3 and TC4 complete all independent paths; TC4 is especially valuable because it reaches the "no scores" message through the loop (a file with data but no valid score), which an empty-file test never checks. A further useful test is a file whose *last* score is the only positive one (e.g. $-1, 60$): it reveals an off-by-one fault if `EOF` becomes true immediately after the last value is read.

Answer. CFG with 9 nodes, 6 branch outcomes, $V(G) = 4$. Minimal 100% statement/branch/condition set: empty file ("No scores found") and file {80, −5} (mean 80). Add {70, 90} and {0, −3} to cover all four independent paths.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q3(c) CSE213 • 09/02/2025 • T007 MARKS 8

Explain the basic principle of equivalence partition testing. Why is this testing used in a real-life software?

2013–2014 • Q3(e) CSE307 • 08/08/2015 • T029.4 MARKS $6\frac{2}{3}$

Demonstrate equivalence portioning for reducing the number of test cases with an example.

SOLUTION

Basic principle

Equivalence partitioning (EP) is a black-box technique. It *divides the input data of a software unit into partitions of equivalent data, assuming that all the conditions in one partition will be treated in the same way by the software.* Two consequences follow:

- We need to test **only one condition (one representative value) from each partition**.
- If one value of a partition works, we assume all values of that partition work; if one value fails, we assume none of them will work. So testing more values of the same partition adds cost but no new information.

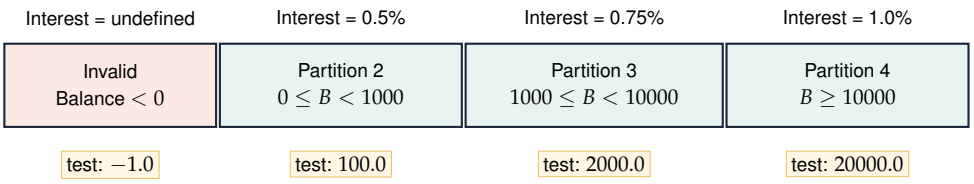
Partitions are formed for *valid* inputs (positive tests) and also for *invalid* inputs (negative tests), because the program must handle both.

Why it is used in real-life software

- **Exhaustive testing is impossible.** A single `float` input already has billions of possible values; real forms have many fields, so trying every value is infeasible.
- **Fewer but smarter tests.** One test per partition gives the same defect-finding power as hundreds of redundant tests, which saves time and cost in every regression run.
- **Systematic coverage of the specification.** Every rule ("0.5% interest if ...") becomes one partition, so no rule and no invalid-input class is forgotten.
- **Combines naturally with boundary value analysis**: once the partitions are known, their edges are exactly where off-by-one errors hide.
- **Independent of the code**: test cases can be written from the requirements before the code exists (useful in TDD and acceptance testing).

Demonstration: savings-account interest (lecture example)

A bank pays interest on a savings account according to the balance: 0.5% for \$0 to \$1000, 0.75% for \$1000 to \$10000 and 1% above \$10000. The function under test is `float calculateSavingsInterest(float balance)`. The input domain splits into four partitions:



How *not* to test. Calling the function with 0, 500, 1000, 2000, ..., 5000 produces many test cases, yet they hit only partitions 2 and 3 – the invalid partition and partition 4 are never exercised.
How to test (EP). One representative per partition is enough – four tests instead of thousands:

```
@Test void negativeBalanceIsRejected() {          // invalid partition
    assertThrows(IllegalArgumentException.class,
                 () -> calculateSavingsInterest(-1.0f));
}
@Test void lowBalance()    { assertEquals(0.50f,  calculateSavingsInterest(100.0f),   0.001); }
@Test void middleBalance() { assertEquals(15.0f,  calculateSavingsInterest(2000.0f),  0.001); }
@Test void highBalance()   { assertEquals(200.0f, calculateSavingsInterest(20000.0f), 0.001); }
```

Adding boundary value analysis gives the values $-0.01; 0, 0.01; 999.99, 1000; 9999.99, 10000$, which still keeps the suite small but also catches off-by-one mistakes such as writing <= for <.

Answer. EP groups inputs that the program treats identically and tests one representative from each valid and invalid group. It is used because exhaustive testing is impossible: e.g. the interest function needs only 4 tests (balance -1, 100, 2000, 20000) instead of thousands, while still exercising every rule of the specification.

2023–2024 • Q5(c) CSE213 • 09/02/2025 • T008 **MARKS 8**

Explain why achiving high testing coverage is important in software development. Mention the potential risks that might arise if coverage is too low.

SOLUTION

What coverage means

Code coverage is a measure used to describe the degree to which the source code of a program is tested by a particular test suite. It is measured with criteria such as **method coverage** (all methods called?), **statement coverage** (all statements executed?), **decision/branch coverage** (every decision taken both true and false?) and **condition coverage** (every atomic condition evaluated both true and false?).

Why high coverage is important

- **More thoroughly tested code**: code that is executed by tests has at least been run once under a check; high coverage means a lower chance of containing hidden bugs.
- **A measure of how complete the test cases are**: it gives an objective number to track and to set as a quality gate in CI.
- **Finds paths you missed**: the coverage report highlights unexecuted lines/branches (e.g. an `else` branch or an error handler) so new test cases can be designed for them.
- **Safe change and refactoring**: well-covered code can be modified with confidence, because regressions are caught by the tests.

Risks when coverage is too low

- **Undetected defects in untested code**: rarely taken branches (error handling, boundary cases, exception paths) fail for the first time in production.
- **Regressions go unnoticed**: changes break untested features silently.
- **Security holes**: unexecuted input-validation code may let malicious input through.
- **Dead or unreachable code** stays hidden and increases maintenance cost.
- **False confidence**: "all tests pass" means little if they touch only 30% of the code.

Example (lecture triangle program): testing only `getTriangle(2,2,2)` executes the "Equilateral" path; the "Isosceles" and "Scalene" returns are never run, so a bug in them would ship unnoticed.

EXAM TIP Also state the limitation: high coverage does *not* guarantee correctness – coverage tools only tell you that what is *there* was executed; missing requirements are not detected. Do not write tests only to make the coverage tool happy.

Answer. High coverage means the code is more thoroughly exercised, measures test completeness and reveals missed paths; low coverage risks hidden defects in untested branches, unnoticed regressions, security holes, dead code and false confidence.

2022–2023 • Q1(b) CSE307 • 05/05/2024 • T009.1 **MARKS 10**

Construct the testcases for the income tax calculation application where people have to pay tax based on different slabs of income, i.e., 0% for income less than 4 lac, 10% for 4–10 lac, 20% for more than 10 lac.

SOLUTION

Step 1: understand the specification

Income I is given in Taka (1 lac = 100,000 Tk). The rule has three *valid* ranges and, as for any numeric input, *invalid* classes as well.

NOTE "Less than 4 lac" $\Rightarrow$ 4 lac itself belongs to the 10% slab; "4–10 lac" is taken as 4 lac $\leq I \leq$ 10 lac; "more than 10 lac" $\Rightarrow I >$ 10 lac. Tax is computed **slab-wise** (as in a real income-tax system: first 4 lac free, next 6 lac at 10%, the rest at 20%). The last column of the table also gives the expected value if the whole income were taxed at its slab rate; the partitions and inputs are the same in both readings.

Step 2: equivalence partitions

ID	Partition	Meaning	Type
EP1	$I < 0$	negative income – impossible	invalid
EP2	$0 \leq I < 400{,}000$	0% slab	valid
EP3	$400{,}000 \leq I \leq 1{,}000{,}000$	10% slab	valid
EP4	$I > 1{,}000{,}000$	20% slab	valid
EP5	non-numeric / empty	e.g. "abc", blank field	invalid



red = EP representatives, black = boundary values

Step 3: test cases (EP representatives + boundary values)

TC	Income (Tk)	Class	Expected tax (slab-wise)	flat reading
Equivalence-partition representatives				
T1	−50,000	EP1	error "income cannot be negative"	error
T2	200,000	EP2	0	0
T3	700,000	EP3	$10\% \times 300{,}000 = 30{,}000$	70,000
T4	1,500,000	EP4	$60{,}000 + 20\% \times 500{,}000 = 160{,}000$	300,000
T5	"abc" / blank	EP5	error "enter a valid amount"	error
Boundary values				
T6	−1	EP1	error	error
T7	0	EP2	0	0
T8	399,999	EP2	0	0
T9	400,000	EP3	0	40,000
T10	400,001	EP3	0.10	40,000.10
T11	1,000,000	EP3	60,000	100,000
T12	1,000,001	EP4	60,000.20	200,000.20

T1, T5 and T6 are **negative tests** (the program must reject the input gracefully); all others are **positive tests**. T2–T4 alone prove each slab rate; T7–T12 check that the comparison operators (< versus <=) are placed exactly at 4 lac and 10 lac.

Answer. Partitions: $I < 0$ (invalid), $[0, 4\text{ lac})$, $[4, 10]$ lac, > 10 lac, non-numeric (invalid). Minimal EP suite: −50,000, 200,000, 700,000, 1,500,000, "abc"; add boundary tests $-1, 0, 399{,}999, 400{,}000, 400{,}001, 1{,}000{,}000, 1{,}000{,}001$.

2022–2023 • Q4(c) CSE307 • 05/05/2024 • T010 **MARKS 8**

Consider the following code snippet:

```
something(int x)
{
    if (x < 0 ||  y < x)
    {
        y := -y;
        x := -x;
    }
    z := x;
}
```

Show a minimal length dataset that achieves condition coverage.

SOLUTION

NOTE y and z are not declared in the snippet; they are taken to be variables in the enclosing scope (e.g. fields), so a test datum is a pair (x, y) with y set before the call. The operator || is kept as printed (short-circuit OR).

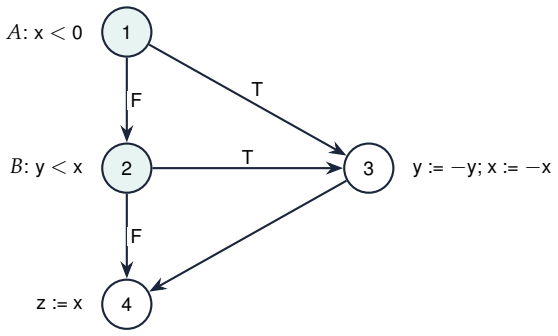
Concept

Condition coverage requires every atomic condition of every decision to evaluate to *true* at least

once and to *false* at least once. Here the single decision has two conditions:

$$A: \; x < 0 \qquad\qquad B: \; y < x.$$

Control-flow graph



Reasoning

With short-circuit OR, B is evaluated *only when A is false* (edge $1 \to 2$). Therefore

- $B =$ T and $B =$ F both need $A =$ F, which already requires **two different** tests with $x \geq 0$;
- $A =$ T needs a third test with $x < 0$ (then B is skipped).

No two tests can do the job, so the minimal dataset has **three** tests:

TC	x	y	A	B	Decision	Path	Expected after call
T1	-3	4	T	–	T	1–3–4	$y = -4, x = 3, z = 3$
T2	5	2	F	T	T	1–2–3–4	$y = -2, x = -5, z = -5$
T3	5	8	F	F	F	1–2–4	$y = 8, x = 5, z = 5$

Condition coverage $= 4/4$ outcomes $= 100\%$; as a bonus the decision is also true (T1, T2) and false (T3), so branch coverage is 100% too.

If both conditions were always evaluated

In the textbook (non-short-circuit) view two tests suffice, e.g. $(x = -1, y = 0)$ giving A=T, B=F and $(x = 3, y = 1)$ giving A=F, B=T. But both make the decision *true*, showing that condition coverage does not imply branch coverage – and with the `||` operator as written, B would not actually be evaluated in the first test.

Answer. Minimal dataset (short-circuit ||): $(x, y) = (-3, 4), (5, 2), (5, 8)$ – three tests that make $x < 0$ true/false and $y < x$ true/false. (Two tests suffice only if both conditions are always evaluated.)

2021–2022 • Q4(c) CSE307 • 07/05/2023 • T012 **MARKS 5**

Describe with example five (5) negative testing ideas that you will use to test a web application.

SOLUTION

Negative testing

Negative testing feeds *invalid* data or unexpected behaviour to the system and checks that it

handles them gracefully. Five typical scenarios from the lecture, applied to a web sign-up/login site:

Idea	Example test	Expected behaviour
1. Required fields empty	submit sign-up form with the *email* field blank	form not submitted, "email is required"
2. Wrong data type	type `xyZ#53()032` into "phone number"	"enter digits only"
3. Character limit	300-character name where max is 50	input blocked / truncated with message
4. Data bounds / limits	day $= -1$ or 32, month $= 13$ in date of birth; age 200	"invalid date / age"
5. Web session testing	log out, then paste a URL recorded while logged in (e.g. `/account/orders`)	redirected to login page; no data shown

Further ideas: values exceeding the data type range (quantity 2,147,483,648 for a 4-byte int) and unreasonable data (a 300-character name, human age 200).

Answer. Empty required fields, wrong data type, exceeding character limits, values outside allowed bounds, and accessing pages after logout (session testing) – each must be rejected with a clear message and without crashing or leaking data.

2020–2021 • Q5(b) CSE307 • 19/10/2022 • T013 MARKS 12

Context – Q5(a) (16 marks, solved in the Class Diagram chapter): A media agency is responsible to order advertisements at different newspapers. Each newspaper has a standard rate for unit size (1 column-inch) for each page which may change over time. Different newspaper gives different discounts over this rate to the agency. The agency has a database of expected readership for each page of each newspaper. An employee of the agency prepares a media plan for a product by selecting different positions for the ad (page, size, date and newspaper) to achieve maximum readership for a given budget. This plan is approved by client and then another employee generates Release Order from this plan. The Release Order mentions a number of ads to display with their positions, date to publish and cost. Design a Class diagram to implement the scenario described above.

(b) What do you understand by Equivalence Partitioning in the context of test data generation? Create test input set to test if the cost of Release Order is computed properly considering the scenario of Q 5(a).

SOLUTION

Equivalence partitioning for test-data generation

Equivalence partitioning divides the input data of a unit into partitions whose members the software treats in the same way. For test-data generation it means: identify every input of the computation, split each into *valid* and *invalid* partitions, then generate *one representative value per partition* (plus boundary values). This yields a small test input set that still exercises every rule of the specification and every kind of invalid data.

The computation under test

From the scenario, for an ad j placed in newspaper n, page p, on date d with size s_j column-inches:

$$\text{cost}_j = s_j \times \text{rate}(n, p, d) \times \left(1 - \frac{\text{discount}(n, d)}{100}\right), \qquad \text{RO cost} = \sum_j \text{cost}_j.$$

NOTE Assumed test data in the database: **Prothom Alo (PA)** has 24 pages; page 1 rate is 2,000 Tk/col-inch up to 31/12/2022 and 2,500 Tk from 01/01/2023; PA gives the agency 10% discount. **Daily Star (DS)** page 3 rate is 1,200 Tk/col-inch with 0% discount. Today's date is 01/12/2022. Valid discounts are $0 \le d < 100$.

Partitions of every input

Input	Valid partitions	Invalid partitions
Newspaper	V1 registered newspaper with a rate card	I1 unknown newspaper
Page	V2 existing page of that newspaper	I2 page ≤ 0 or beyond last page
Ad size s (col-inch)	V3 $s > 0$ and fits the page	I3 $s \le 0$; I4 non-numeric; I5 larger than page
Publish date d	V4a future date before a rate change; V4b future date after a rate change	I6 past date; I7 non-existent date (31/02)
Discount	V5 $d = 0$; V6 $0 < d < 100$	I8 $d < 0$ or $d \ge 100$
Ads in the RO	V7 exactly one ad; V8 several ads, several newspapers	I9 zero ads
Plan status	V9 approved by client	I10 not approved

Test input set

TC	Input (newspaper, page, size, date, …)	Covers	Expected result
Positive tests (cost must be correct)			
T1	PA, p.1, 10 col-in, 15/12/2022; plan approved	V1 V2 V3 V4a V6 V7 V9	$10 \times 2000 \times 0.9 = 18{,}000$ Tk
T2	T1's ad + DS, p.3, 8 col-in, 15/12/2022	V5 V8	$18{,}000 + 8 \times 1200 = 27{,}600$ Tk
T3	PA, p.1, 10 col-in, **10/01/2023**	V4b	new rate: $10 \times 2500 \times 0.9 = 22{,}500$ Tk
T4	PA, p.1, 1 col-in, 15/12/2022 (smallest size)	V3 boundary	$1 \times 2000 \times 0.9 = 1{,}800$ Tk
Negative tests (RO must not be produced, clear error shown)			
T5	"Daily XYZ", p.1, 10 col-in	I1	error: unknown newspaper
T6	PA, p.40, 10 col-in	I2	error: page does not exist
T7	PA, p.1, size 0 / size -5	I3	error: size must be positive
T8	PA, p.1, size "ten"	I4	error: numeric size required
T9	PA, p.1, size larger than the page area	I5	error: ad does not fit page
T10	PA, p.1, 10 col-in, 01/06/2022	I6	error: date already past
T11	PA, p.1, 10 col-in, 31/02/2023	I7	error: invalid date
T12	discount for PA stored as 120% (or -5%)	I8	error / rate card rejected, no negative cost
T13	Release Order with no ads	I9	error: RO must contain at least one ad
T14	T1's ad, plan *not* approved by client	I10	RO generation refused

T1–T4 verify the formula (rate $\times$ size, discount, summation over ads, and that the rate valid *on the publish date* is used); T5–T14 verify that every invalid partition is rejected.

Answer. EP picks one representative per valid/invalid class of each input. For the Release Order cost: positive data T1–T4 (expected 18,000, 27,600, 22,500, 1,800 Tk) and negative data T5–T14 (unknown paper, bad page, non-positive/non-numeric/oversize size, past/invalid date, illegal discount, empty RO, unapproved plan).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2020–2021 • Q6(b) CSE307 • 19/10/2022 • T014 MARKS 15

Give an example test execution ensuring method, branch and condition coverage. What are the characteristics of good unit testing?

2018–2019 • Q2(b) CSE307 • 06/11/2019 • T021.1 MARKS 15

Give an example test execution ensuring method, branch and condition coverage. What are the characteristics of good unit testing?

SOLUTION

> **NOTE** In the bank crop of T014 the visible mark "(12)" belongs to the preceding part 6(a); the original paper shows 6(b) carries 15 marks, the same as T021.1.

Coverage criteria (lecture)

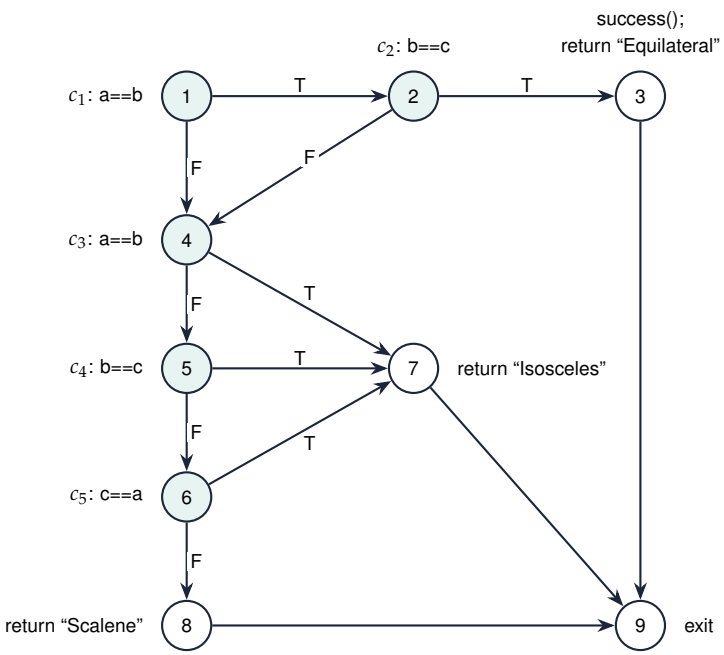
- **Method coverage**: have all methods been called?
- **Branch (decision) coverage**: has every decision been executed in both its true and false path?
- **Condition coverage**: has every atomic condition inside a decision been evaluated both true and false?
- **Statement coverage**: has every statement been executed?

Program under test (lecture example)

```
public class Triangle {
    public String getTriangle(int a, int b, int c) {
        if (a == b && b == c) {                 // decision D1 : c1 && c2
            success();
            return "Equilateral";
        }
        if (a == b || b == c || c == a) {       // decision D2 : c3 || c4 || c5
            return "Isosceles";
        }
        return "Scalene";
    }
    void success() { System.out.println("Got it!"); }
}
```

There are **2 methods**, **4 branch outcomes** (D1 T/F, D2 T/F) and **5 atomic conditions** c_1: $a{==}b$, c_2: $b{==}c$ (in D1) and c_3: $a{==}b$, c_4: $b{==}c$, c_5: $c{==}a$ (in D2), i.e. 10 condition outcomes. Java evaluates && and || with *short-circuit*, so a later condition is evaluated only when needed.

Control-flow graph (condition level)



Nodes 1–2 form decision D1 and nodes 4–6 form decision D2; D1 is true only along $1 \to 2 \to 3$ and D2 is false only along $4 \to 5 \to 6 \to 8$.

Example test execution and the outcome of every condition

Call	c_1	c_2	D1	c_3	c_4	c_5	D2	Expected output
`getTriangle(2,2,3)`	T	F	F	T	–	–	T	"Isosceles"
`getTriangle(2,2,2)`	T	T	T	–	–	–	–	"Equilateral" (prints "Got it!")
`getTriangle(1,2,2)`	F	–	F	F	T	–	T	"Isosceles"
`getTriangle(2,1,2)`	F	–	F	F	F	T	T	"Isosceles"
`getTriangle(2,3,4)`	F	–	F	F	F	F	F	"Scalene"

("–" = not evaluated because of short-circuit.) Every column $c_1 \ldots c_5$ shows both T and F, and both D1 and D2 show both T and F.

Coverage achieved as the tests are run

After test	**Method** (2)	**Statement** (7)	**Branch** (4)	**Condition** (10)
1: (2,2,3)	1/2 = 50%	3/7 = 43%	2/4 = 50%	3/10 = 30%
2: (2,2,2)	2/2 = **100%**	6/7 = 86%	3/4 = 75%	4/10 = 40%
3: (1,2,2)	100%	6/7 = 86%	3/4 = 75%	7/10 = 70%
4: (2,1,2)	100%	6/7 = 86%	3/4 = 75%	9/10 = 90%
5: (2,3,4)	100%	7/7 = **100%**	4/4 = **100%**	10/10 = **100%**

Statements counted: the two `if`s, `success()`, three `return`s and the `println` in `success()`.

- **Method coverage** is reached after the first two calls (`(1,1,2)` and `(2,2,2)` in the lecture also suffice), because `success()` is called only for an equilateral triangle.
- **Condition coverage** needs the first four calls plus c_5=F, which only `(2,3,4)` gives.
- **Branch coverage** needs D2 false, i.e. a scalene triangle – again `(2,3,4)`.

Under short-circuit evaluation all five calls are necessary: c_2=F needs $a{=}b \neq c$, c_4=T needs $a \neq b{=}c$, c_5=T and c_5=F need $a \neq b \neq c$ with $c{=}a$ and $c \neq a$.

The same execution as a JUnit 5 test class

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class TriangleCoverageTest {
    private final Triangle t = new Triangle();

    @Test void firstPairEqual()   { assertEquals("Isosceles",   t.getTriangle(2, 2, 3)); }
    @Test void allEqual()         { assertEquals("Equilateral", t.getTriangle(2, 2, 2)); }
    @Test void lastPairEqual()    { assertEquals("Isosceles",   t.getTriangle(1, 2, 2)); }
    @Test void outerPairEqual()   { assertEquals("Isosceles",   t.getTriangle(2, 1, 2)); }
    @Test void noneEqual()        { assertEquals("Scalene",     t.getTriangle(2, 3, 4)); }
}
```

Characteristics of good unit testing (lecture)

Property	Meaning (with the example above)
Automatic	runs completely by itself without human input (JUnit runs all five tests with one click / in CI).
Atomic	decides by itself whether the function passed or failed – `assertEquals` gives a pass/fail verdict; nobody has to read printed output.
Single responsibility	each test checks exactly one feature (one test per triangle kind / condition).
Independent	runs in isolation from other tests; no shared mutable state, any order gives the same result.
Repeatable	multiple invocations always return the same result (no dependence on time, random numbers, network).

In addition, good unit tests are fast, readable (descriptive names such as `noneEqual`), written by the developer concurrently with the code, and maintained together with it.

Answer. Calls (2,2,3), (2,2,2), (1,2,2), (2,1,2), (2,3,4) give 100% method, statement, branch and condition coverage of `getTriangle`. Good unit tests are Automatic, Atomic, Single-responsibility, Independent and Repeatable.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2020–2021 • Q7(a) CSE307 • 19/10/2022 • T015 MARKS 10

A general manager of an office is entitled to have a laptop and two mobile phones per year. The laptop requisition process is implemented in a software application. Give examples of positive and negative testing in the context of testing the software process.

2018–2019 • Q3(a) CSE307 • 06/11/2019 • T021.2 MARKS 7

A general manager of an office is entitled to have a laptop per year. The laptop requisition process is implemented in a software application. Give examples of positive and negative testing in the context of testing the software process.

SOLUTION

Definitions (lecture)

- **Positive testing** validates that the application works when given *valid* input data – it verifies that the application does *not* show an error when it is not supposed to.
- **Negative testing** gives the system *invalid* data or unexpected user behaviour and checks that it behaves as expected, i.e. handles them gracefully (rejects, shows a clear message, does not crash, does not grant anything).

The requisition process

A requisition form has: employee ID (designation fetched from HR data), item type, quantity, requisition date, justification. Business rule: a General Manager (GM) may receive **1 laptop** and **2 mobile phones** per year (T021.2: laptop only). "Year" is assumed to be the calendar year; entitlement resets on 1 January.

Positive test cases

TC	Input / situation	Expected result
P1	GM, no laptop received in 2022, requests 1 laptop, all fields filled	accepted, sent for approval; laptop balance becomes 0
P2	GM requests 1 mobile phone (first of the year)	accepted; phone balance 1
P3	GM requests a second phone in the same year (boundary = 2)	accepted; phone balance 0
P4	GM who got a laptop in 2022 requests one on 01/01/2023	accepted (entitlement renewed)
P5	GM checks "my entitlement" page	shows correct remaining items

Negative test cases

TC	Input / situation	Expected result
N1	GM requests a **second laptop** in the same year	rejected: "yearly laptop entitlement used"
N2	GM requests a **third phone** in the same year	rejected: phone entitlement exceeded
N3	quantity = 2 laptops in one request; quantity 0 or −1	rejected: invalid quantity
N4	an Assistant Manager (not GM) requests a laptop	rejected: not entitled
N5	required field (item type / justification) left empty	form not submitted, field highlighted
N6	employee ID typed as `ab#12` (wrong data type)	validation error
N7	unknown item "Tablet" via tampered form	rejected
N8	requisition date in the past year or far future	rejected: invalid date
N9	GM approves his own requisition	blocked: approver must be different
N10	open the requisition URL after logging out	redirected to login, nothing saved

For the one-laptop-only version (T021.2), P2/P3 turn into *negative* tests: a phone requisition by the GM must be rejected because phones are not part of his entitlement.

Answer. Positive: a GM's first laptop (and first/second phone) of the year is accepted, entitlement renews in a new year. Negative: second laptop, third phone, wrong quantity, non-GM user, empty or mistyped fields, unknown item, invalid date, self-approval and access without login are all rejected gracefully.

2018–2019 • Q1(b) CSE307 • 06/11/2019 • T020 MARKS 12

What do you understand by Equivalence Partitioning? Create test input set considering standard practices to test different user roles of BIIS.

SOLUTION

Equivalence partitioning

EP divides the input data of a unit into partitions of equivalent data, assuming all values in a partition are treated in the same way by the software. Only one representative from each partition is tested: if it passes, the whole partition is assumed to pass; if it fails, the whole partition is assumed to fail. Valid partitions give *positive* tests, invalid ones give *negative* tests.

Inputs of a role-based system

BIIS (BUET Institutional Information System) serves several kinds of users. For role testing, the inputs that determine behaviour are: **(1)** the login credentials, **(2)** the user's role, **(3)** the requested operation, and **(4)** whether the target data belongs to that user (own course, own advisee, own result).

> **NOTE** Assumed roles: Student, Course Teacher, Adviser, Department Head, Admin (Registrar's office). Standard practices applied: least privilege (each role sees only what it needs), test both allowed and forbidden operations, test ownership of data, and test sessions/authentication.

Partitions

Input	Valid partitions	Invalid partitions
Credentials	V1 registered ID + correct password	I1 wrong password; I2 unregistered ID; I3 empty field; I4 deactivated (graduated) account
Role × operation	V2 operation permitted for the role	I5 operation of another role (privilege escalation)
Data ownership	V3 own data (own result, own course, own advisee)	I6 someone else's data
Session	V4 active session	I7 no session / logged out / expired

Test input set

TC	User / role	Class	Action	Expected
R1	Student 1805017	V1 V2 V3	view own grade sheet	grade sheet shown
R2	Student 1805017	V2 V3	submit course registration in registration period	sent to adviser
R3	Student 1805017	I6	change URL to view 1805018's result	access denied (403)
R4	Student 1805017	I5	open "enter marks" page	access denied
R5	Course teacher of CSE213	V2 V3	enter marks for CSE213	marks saved
R6	Course teacher of CSE213	I6	enter marks for CSE215 (not assigned)	denied
R7	Adviser	V2 V3	approve registration of own advisee	approved, forwarded to Head
R8	Adviser	I6	approve a non-advisee's registration	denied
R9	Department Head	V2	approve departmental registrations / results	approved
R10	Head	I5	create user accounts (admin function)	denied
R11	Admin	V2	create user, assign role "Teacher"	account created
R12	any user	I1	correct ID, wrong password	"invalid credentials", no session
R13	"0000000"	I2	unregistered ID	"invalid credentials"
R14	—	I3	ID or password left blank	"field required"
R15	graduated student	I4	login	"account inactive"
R16	student after logout	I7	reopen a bookmarked result URL	redirected to login page

Every valid and invalid partition is represented once per role; R1, R2, R5, R7, R9, R11 are positive tests, the rest are negative tests that check that BIIS never shows data or functions a role is not entitled to.

Answer. EP = one representative per class of equivalent inputs. For BIIS, partition credentials, role-versus-operation, data ownership and session state into valid and invalid classes, then test each role on a permitted operation (positive) and on another role's operation, another user's data, bad credentials and expired sessions (negative) – 16 test inputs R1–R16.

2017–2018 • Q8(d) CSE325 • 10/04/2019 • C325-1718-8d MARKS 6 + 6 = 12

Consider the following specifications.
Input: length of three sides of a triangle: a, b, c
Output: true if each side is a positive number less or equal to 20 and the triangle is Isosceles (2 sides are equal), false otherwise.
Precondition: the sides form a triangle (You do not need to consider cases like $a + b \leq c$)

(i) First, write down all valid and invalid equivalence classes for this problem. (For example, V1: $0 < a, b, c \leq 20$; V2: $a = b$ and $a \neq c$ are two valid equivalent classes. On the other hand, X1: $a > 20$, X2: $a = b = c$ are two invalid equivalent classes.)

(ii) Generate test cases covering all classes. Use the following table structure. Try to minimize the number of tests. (First few entries have been shown for your convenience)

Test no	Test data	Expected outcome	Class covered
1	5, 5, 2	T	V1, V2
2	22, 14, 14	F	X1
3	7, 7, 7	F	X2

SOLUTION

Method

Equivalence partitioning divides the input domain into classes whose members the program should treat the same way, so one test per class is enough. *Valid* classes lead to output **true**; each *invalid* class violates exactly one condition and leads to **false**. To minimise tests: one test may cover *many valid* classes, but each test covers *only one invalid* class (otherwise one fault could mask another).

Two conditions are in the specification: (1) the range $0 < \text{side} \leq 20$ for each of a, b, c and (2) "exactly two sides equal" (the example X2 shows that equilateral is *not* accepted).

(i) Equivalence classes

Valid classes		Invalid classes	
V1	$0 < a \leq 20, 0 < b \leq 20, 0 < c \leq 20$	X1	$a > 20$
V2	$a = b, a \neq c$	X2	$a = b = c$ (equilateral)
V3	$a = c, a \neq b$	X3	$b > 20$
V4	$b = c, b \neq a$	X4	$c > 20$
		X5	$a \leq 0$
		X6	$b \leq 0$
		X7	$c \leq 0$
		X8	$a \neq b, b \neq c, a \neq c$ (scalene, no two sides equal)

V2, V3, V4 are three different classes because a program could compare only one pair of sides (e.g. only `a == b`); each pair must be exercised. Similarly the range condition is checked for each side separately on the invalid side (X1/X3/X4 and X5/X6/X7).

(ii) Minimised test cases

Test no	Test data (a, b, c)	Expected outcome	Class covered
1	5, 5, 2	T	V1, V2
2	22, 14, 14	F	X1
3	7, 7, 7	F	X2
4	5, 2, 5	T	V1, V3
5	2, 5, 5	T	V1, V4
6	14, 22, 14	F	X3
7	14, 14, 22	F	X4
8	0, 5, 5	F	X5
9	5, 0, 5	F	X6
10	5, 5, 0	F	X7
11	3, 4, 5	F	X8

- All 4 valid classes are covered by 3 tests (1, 4, 5); V1 is covered by each of them. They cannot be merged further because V2, V3 and V4 are mutually exclusive.
- Each of the 8 invalid classes needs its own test (2, 3, 6–11), so the minimum is $3 + 8 = 11$ tests.
- In tests 2, 6, 7 the out-of-range side is chosen so that the other two sides are equal: the only reason for **F** is then the range violation.

NOTE Zero is used for X5–X7 (it is also a boundary value of "positive"); such inputs do not form a real triangle, but the precondition says the triangle inequality need not be considered. If non-numeric input is also to be considered, one more invalid class ("a side is not a number") and test (e.g. `x, 5, 5` $\to$ F) can be added.

Answer. Valid: V1 ($0 < a, b, c \leq 20$), V2 ($a = b \neq c$), V3 ($a = c \neq b$), V4 ($b = c \neq a$). Invalid: X1 $a > 20$, X2 $a = b = c$, X3 $b > 20$, X4 $c > 20$, X5 $a \leq 0$, X6 $b \leq 0$, X7 $c \leq 0$, X8 all sides different. 11 tests: (5,5,2)T, (22,14,14)F, (7,7,7)F, (5,2,5)T, (2,5,5)T, (14,22,14)F, (14,14,22)F, (0,5,5)F, (5,0,5)F, (5,5,0)F, (3,4,5)F.

13

Use Case Diagram

EXAM TIP Exam recipe (from the lecture notes): **(1)** draw the system boundary, **(2)** find the actors (*who?* – people, external systems, devices, time), **(3)** find the use cases (*what?* – one meaningful goal each, verb + noun), **(4)** connect each actor only to the use cases it really takes part in, **(5)** «include» for behaviour that *must* happen as part of another use case (arrow from base to included), **(6)** «extend» for *optional / conditional* behaviour (arrow from the extension to the base), **(7)** generalization (hollow triangle, child → parent) for specialised actors or use cases, **(8)** never join two use cases just because one usually happens before the other.

2023–2024 • Q1(a) CSE213 • 09/02/2025 • R002 **MARKS 7**

Read the following case study carefully and then answer the questions given below.
A mental health clinic is using an information system named, Mentcare system, which is designed and used to maintain diagnostic and treatment related records for mental patient. Clinic staff log on to the system using a username and password. The system requires passwords to be at least eight letters long but allows any password to be set without further checking. A receptionist may transfer data from the Mentcare system to a national patient record system (PRS) database that is maintained by the health authority of the country. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
A criminal finds out that a well-paid sports star is receiving treatment for mental health problems. He would like to gain illegal access to information in this system so that he can blackmail the star. By posing as a concerned relative and talking with the nurses in the mental health clinic, he discovers how to access the system and personal information about the nurses and their families. By checking name badges, he discovers the names of some of the people allowed access. He then attempts to log on to the system by using these names and systematically guessing possible passwords, such as the names of the nurses' children.
(a) Draw a use case diagram and show the processes where the criminal can be engaged to have access of the system.

SOLUTION

Concepts used

A **use case diagram** shows *what* a system does from the standpoint of external actors: actors (stick figures) outside a system boundary, use cases (ovals) inside it, and associations between them. To show the *criminal's* processes we extend the ordinary diagram with **misuse cases**: a misuse case is a use case seen from the side of a hostile actor (a *mis-actor*); it is drawn as a dark oval and is linked to the legitimate use case it attacks by a «threatens» dependency.

Step 1 – normal actors and use cases

- **Clinic Staff** (nurses, doctors): *Log On*, *Maintain Diagnosis & Treatment Records*.
- **Receptionist**: a specialised clinic staff member (actor generalization) who additionally performs *Transfer Data to PRS*.

- **PRS Database**: an external system actor that receives the transferred data.
- Every log on must check the password rule ("at least eight letters"), so *Log On* «include» *Validate Password*.

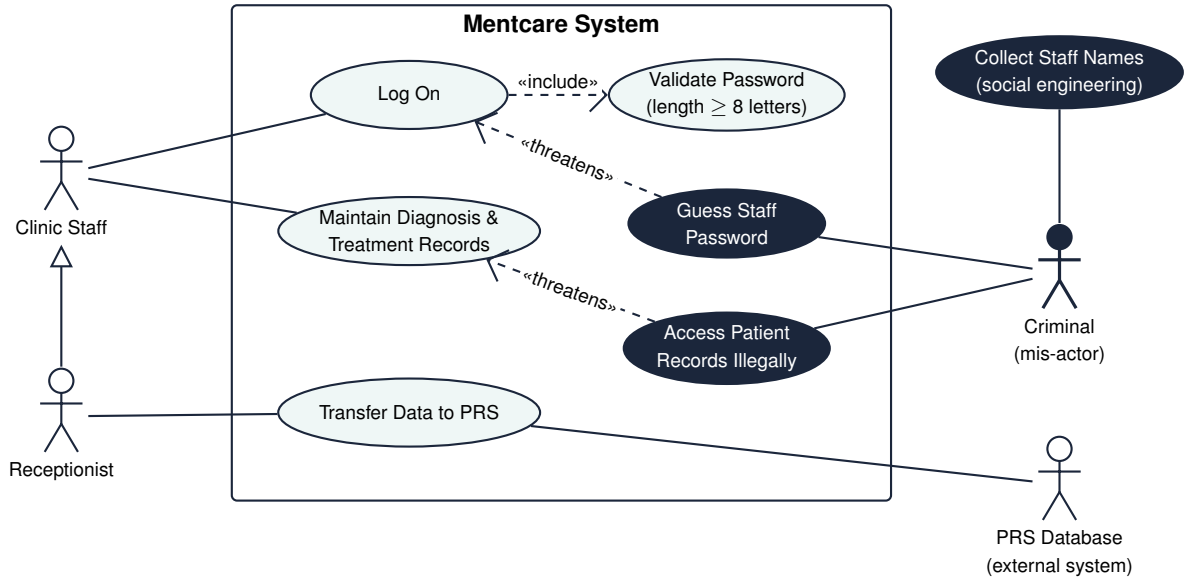
Step 2 – the criminal (mis-actor) and his misuse cases

(i) Collect Staff Names (social engineering) – posing as a relative, chatting with nurses and reading name badges. This happens *outside* the computer system, so it is drawn outside the boundary.

(ii) Guess Staff Password – systematic guessing (children's names, etc.) through the log-on screen; it «threatens» *Log On*. It succeeds because *Validate Password* only checks the length.

(iii) Access Patient Records Illegally – once logged on with a stolen identity he reads the sports star's diagnosis; it «threatens» *Maintain Diagnosis & Treatment Records*.

Use case diagram (with misuse cases)



Explaining the processes where the criminal is engaged

Misuse case	What the criminal does	Weakness exploited / counter-measure
Collect Staff Names	Poses as a relative, talks to nurses, reads name badges	People, not software; staff training, no names on badges
Guess Staff Password	Tries valid user names with guessed passwords	Only a length rule; add password-strength check, lock-out after n failures
Access Records Illegally	Reads the star's diagnosis after logging on	Single-factor login; audit log, two-factor authentication

NOTE Misuse cases and the «threatens» stereotype are the standard way (Sommerville, security requirements) of putting an attacker on a use case diagram; the lecture notation (actors, ovals, boundary, include, generalization) is used for everything else. Drawing the criminal as an ordinary actor associated with *Log On* and with a "view records" use case is an acceptable simpler answer.

Answer. Actors: Clinic Staff, Receptionist (specialises Clinic Staff), PRS Database, Criminal (mis-actor). Normal use cases: Log On «include» Validate Password, Maintain Diagnosis & Treatment Records, Transfer Data to PRS. Criminal's processes: Collect Staff Names (outside the system), Guess Staff Password «threatens» Log On, and Access Patient Records Illegally «threatens» Maintain Records.

2023–2024 • Q6(a) CSE325 • 17/05/2026 • C325-2324-6a **MARKS 15**

Playdom, Inc. develops social games for social network platforms, such as Facebook and MySpace. They are developing a new *Fantasy Adventure Game Management System* for an online role-playing game. The management system needs to handle the following situations: (1) players can create accounts, join quests, view their achievements, and purchase in-game items, (2) game masters can view player profiles, assign quests, and update mission outcomes, (3) healers can update player health status and view active missions, (4) item merchants can view purchase requests and deliver virtual items, (5) lobby managers can register new players and schedule tournament matches, (6) the system must verify payment eligibility through an external payment gateway system before confirming premium purchases, (7) all users must log in through a centralized Authentication System, and (8) the reward system automatically generates bonus points after each completed quest.
Draw a use case diagram for the **Quest & Player Interaction Subsystem** that handles player activities related to quests, including joining, completing, and interacting with other roles and services. Your use case diagram must include the following: (1) all actors (primary and secondary), (2) at least SIX use cases, (3) at least TWO examples of «include» relationships, and (4) at least ONE example of «extend» relationship.

SOLUTION

Use case diagram building blocks

- **Actor** – a role played by a person or an external system that interacts with the system. A *primary* actor initiates a use case to achieve a goal; a *secondary* (supporting) actor is an external system/person the use case needs (it never starts the interaction). Secondary system actors are usually drawn on the right with «system».
- **Use case** – a goal-level piece of functionality (verb phrase) inside the **system boundary**.
- **«include»** – behaviour that the base use case *always* performs (arrow from the base to the included use case).
- **«extend»** – optional / conditional behaviour inserted into a base use case at an extension point (arrow from the extending use case *to* the base).

Step 1 – scoping the subsystem

Only the *Quest & Player Interaction* subsystem is asked for, so the use cases are those about joining / completing quests and the interactions of the player with the other roles and services.

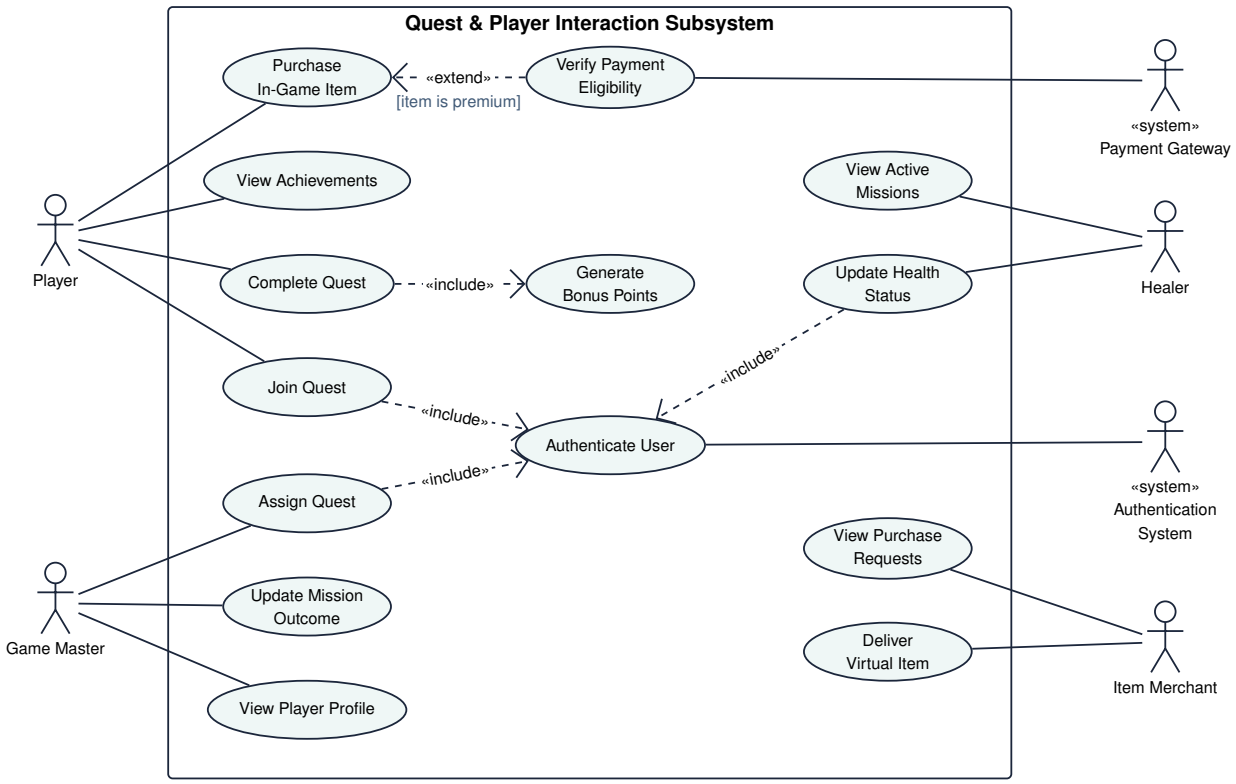
- **Primary actors:** Player, Game Master, Healer, Item Merchant.
- **Secondary actors:** Authentication System (requirement 7), Payment Gateway (requirement 6).
- Lobby Manager is *outside* this subsystem (player registration and tournament scheduling are lobby/administration functions), and account creation belongs to the account subsystem, so

neither is drawn. The reward system of requirement (8) is treated as an internal component, hence modelled as the use case *Generate Bonus Points*.

Step 2 – relationships

Base use case	Relation	Why
Join Quest, Assign Quest, Update Health Status	«include» Authenticate User	every user must log in through the central Authentication System – mandatory, reused behaviour
Complete Quest	«include» Generate Bonus Points	bonus points are generated *after each* completed quest – always
Purchase In-Game Item	← «extend» Verify Payment Eligibility	only *premium* purchases need the payment gateway – conditional (`[item is premium]`)

Step 3 – the use case diagram



Step 4 – checking the requirements

Requirement	Where it is satisfied
All actors (primary + secondary)	primary: Player, Game Master, Healer, Item Merchant; secondary: Payment Gateway, Authentication System
$\geq$ 6 use cases	14 use cases inside the boundary
$\geq$ 2 «include»	Join Quest / Assign Quest / Update Health Status → Authenticate User; Complete Quest → Generate Bonus Points
$\geq$ 1 «extend»	Verify Payment Eligibility → Purchase In-Game Item, condition [item is premium]

NOTE Authentication is included in all use cases started by a logged-in user; for readability the «include» is drawn for three representative ones. Equivalent alternatives: Reward System drawn as a secondary «system» actor of *Generate Bonus Points*; Purchase In-Game Item placed in the item/shop subsystem instead.

Answer. Boundary "Quest & Player Interaction Subsystem"; primary actors Player, Game Master, Healer, Item Merchant; secondary actors Authentication System and Payment Gateway; use cases Join/Complete Quest, View Achievements, Purchase In-Game Item, Assign Quest, Update Mission Outcome, View Player Profile, Update Health Status, View Active Missions, View Purchase Requests, Deliver Virtual Item; «include» Authenticate User and Generate Bonus Points; «extend» Verify Payment Eligibility [premium].

2023–2024 • Q7(c) CSE325 • 17/05/2026 • C325-2324-7c **MARKS 15**

You are a senior software architect at **SiFive**. You are currently leading three projects which are at different stages of systems development life cycle. You need to guide your team in preparing diagrams to illustrate different needs for each project.

- **Project 1:** The intern on your team has been tasked with explaining to investors what users will be able to do with your proposed fitness tracking app. Since the investors are non-technical, the explanation must clearly and quickly convey the app's functionality.
- **Project 2:** The business analyst on your team needs to demonstrate to stakeholders how different user roles (e.g., admin, manager, employee) interact with the system and how access and permissions differ across different roles.
- **Project 3:** your development team is gearing to develop a CRM (Customer Relationship Management) software for Salesforce. Your team needs to understand how data flows through the customer order fulfillment process, including which departments are responsible for which tasks, and where bottlenecks might occur.

For each project, you must choose the **most** appropriate diagram (e.g., BPMN, Use Case, Class). You need to justify why the selected diagram is best suited for the project. You need to mention one key limitation of using the selected diagram for the project.

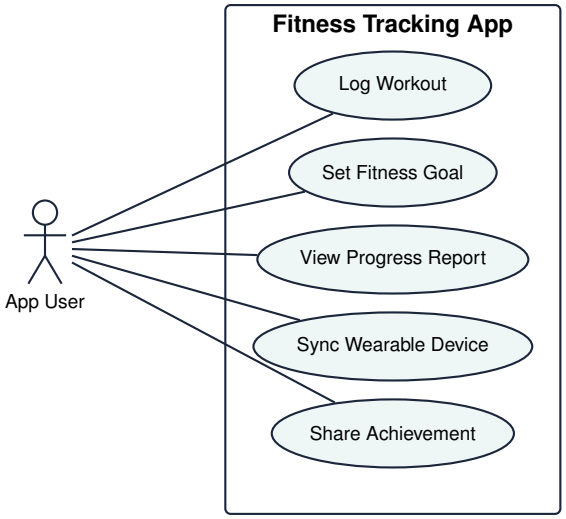
SOLUTION

Selection principle

Each diagram answers a different question: a **use case diagram** answers *"who can do what with the system"* (external, functional view); a **class diagram** answers *"what data/objects exist and how they are related"* (static internal structure); a **BPMN** diagram answers *"how does the business process flow, in which order, and who performs each step"*. The audience and the question to be answered decide the diagram.

Project 1 – Use Case Diagram

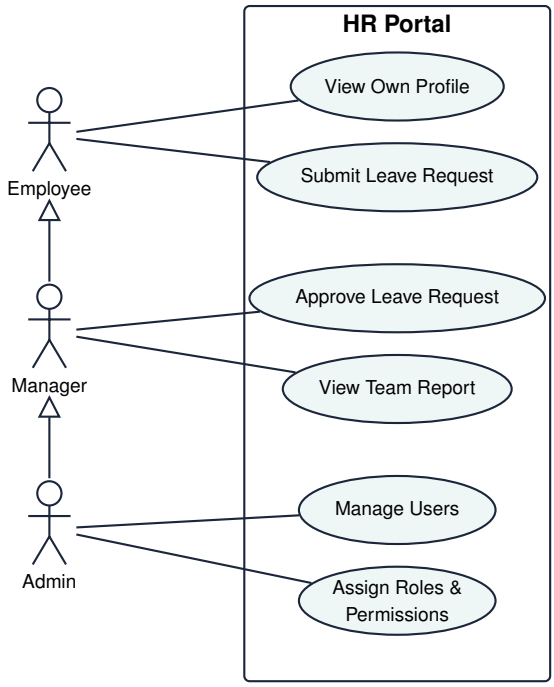
Why. Investors want to know *what users can do*, not how it is built. A use case diagram shows exactly that – one actor (User) and goal-level ellipses – with almost no notation to learn, so it can be understood in seconds. Class diagrams or sequence diagrams expose implementation details that are irrelevant and confusing to a non-technical audience.



Key limitation. It shows only the *names* of the functions; it says nothing about the order of steps, the user interface, business rules or non-functional qualities (performance, battery use, privacy) that investors may also care about – these need narratives or a prototype.

Project 2 – Use Case Diagram with actor generalisation

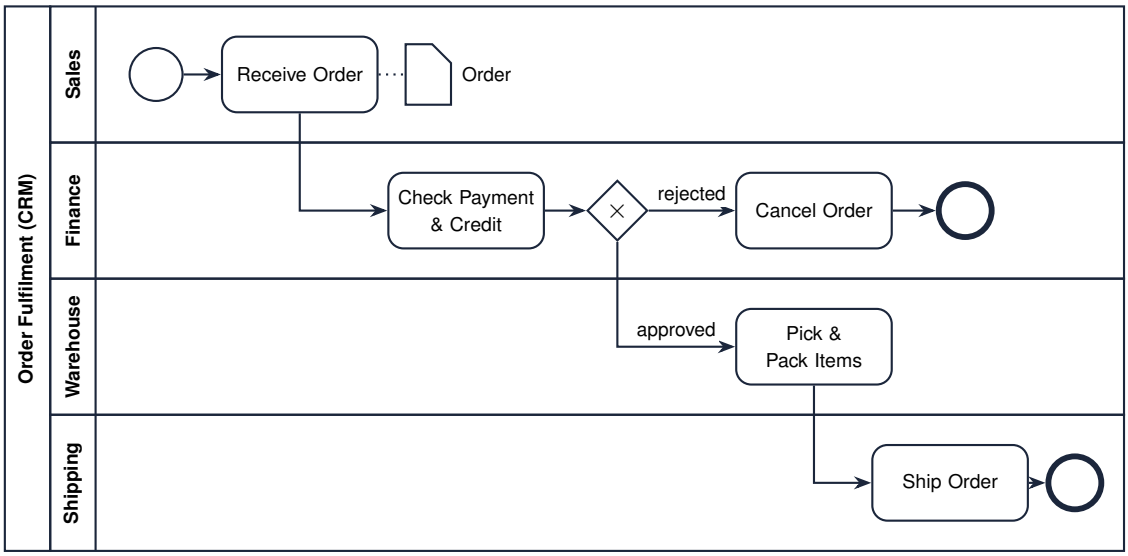
Why. The question is "how different *roles* interact and how their *access* differs". In a use case diagram each role is an actor, and the associations show exactly which functions each role may use. **Actor generalisation** (Manager → Employee, Admin → Manager) expresses that a higher role inherits all permissions of the lower one and adds its own – a direct picture of the permission hierarchy that stakeholders can verify.



Key limitation. A use case diagram shows *which* functions a role can reach but not fine-grained, data-level rules (e.g. "a manager may approve only requests of *his own* team", read vs. write rights on a field). Those must be added in use case descriptions or an access-control matrix. (A class diagram `User-Role-Permission` could model the permission *data*, but it would not show the interaction of the roles with the system, which is what the stakeholders asked for.)

Project 3 – BPMN diagram with pools and lanes

Why. The team must see the *flow* of the order-fulfilment process, *which department* does each task and *where work waits*. BPMN provides exactly these: a **pool** for the process, **swimlanes** for departments (Sales, Finance, Warehouse, Shipping), **activities** and **sequence flows** for the order of work, **gateways** for decisions and **data objects** for the data passed along. Hand-offs between lanes and queues of tasks make bottlenecks visible.



Key limitation. BPMN models the *process* (control flow and responsibilities), not the structure of the data itself; despite the word "data flows", it cannot show the CRM's data model or how data is stored, and it shows no timing/volume figures – bottlenecks are only *suggested* by the structure and must be confirmed with process metrics or simulation. Large processes also become cluttered.

Summary

Project	Diagram	Key strength	Key limitation
1 Investors	Use case	functionality at a glance, minimal notation	no flow, UI, rules or NFRs
2 Roles	Use case + actor generalisation	who may do what; permission inheritance	no fine-grained / data-level access rules
3 CRM fulfilment	BPMN with lanes	order of tasks, department ownership, hand-offs	no data structure, no timing data

Answer. Project 1: Use Case diagram (simple, functional view for non-technical investors; limitation: no sequence/UI/NFR detail). Project 2: Use Case diagram with actor generalisation (roles as actors, associations = permitted functions; limitation: no fine-grained data-level permissions). Project 3: BPMN with pool and swimlanes (flow, responsibilities, hand-offs/bottlenecks; limitation: no data model or quantitative timing).

2021–2022 • Q3(a) CSE325 • 07/10/2023 • C325-2122-3a MARKS 8

Consider an Automated Teller Machine (ATM) of a Bank. You can have the following banking operations using the ATM card:

(i) Withdrawing money

(ii) Checking Balance from Core Banking System

(iii) Changing PIN (Personal Identification Number)

(iv) Depositing money

(v) Transferring money to another account

(vi) Recharging your mobile phone

Draw a use case diagram showing the actors and use cases.

SOLUTION

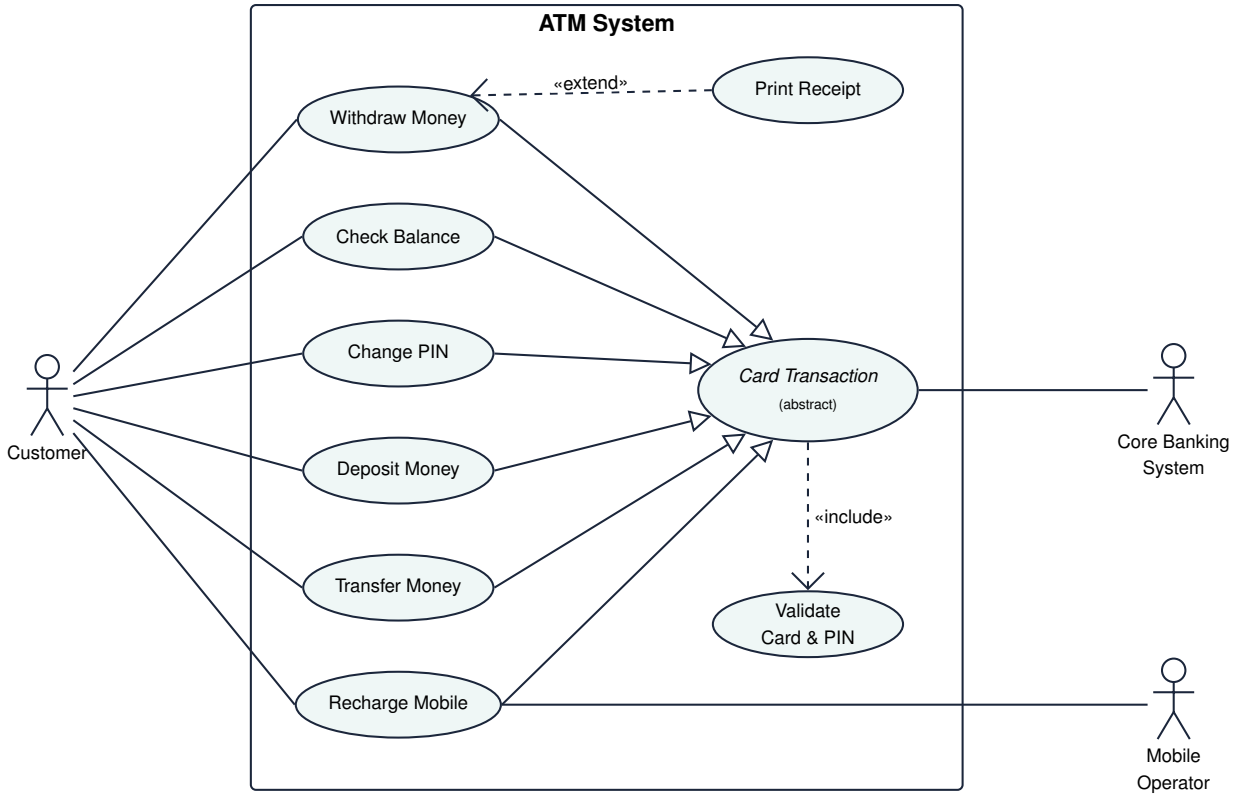
Step 1 – actors

- **Customer** (card holder) — *primary* actor; starts every use case.
- **Core Banking System (CBS)** — *secondary* external system actor; holds accounts and balances and performs every debit/credit and PIN change.
- **Mobile Operator** — secondary system actor that tops up the phone in *Recharge Mobile*.

Step 2 – use cases and relationships

The six operations become six use cases. Every one of them starts with inserting the card and entering the PIN, which the bank must verify, and every one of them is carried out by the Core Banking System. To show this *common* part once instead of drawing six «include» arrows and six CBS lines that would cross each other, the six use cases are made specialisations (**use case generalization**) of an abstract use case *Card Transaction*:

- *Card Transaction* «include»s Validate Card & PIN (mandatory common behaviour, arrow from base to included) and is associated with the CBS; the children inherit both.
- Printing a receipt is optional and happens only if the customer asks for it, so Print Receipt «extend»s Withdraw Money (arrow from the extension to the base).
- Recharge Mobile is additionally associated with the Mobile Operator.



Explanation

- Associations (plain lines) connect actors to the use cases they take part in. The Customer initiates each of the six concrete use cases; the CBS takes part in all of them through the abstract parent (balance enquiry, debit/credit, PIN change, PIN verification); the Mobile Operator only in Recharge Mobile.
- Generalization (hollow triangle, child → parent): each concrete transaction "is a" card transaction and inherits its «include» of Validate Card & PIN and its link to the CBS.
- «include»: every transaction *always* needs card/PIN validation.
- «extend»: Print Receipt is optional (shown for Withdraw Money; the same extension can be attached to Deposit and Transfer).

Answer. Actors: Customer (primary), Core Banking System and Mobile Operator (secondary). Use cases: Withdraw Money, Check Balance, Change PIN, Deposit Money, Transfer Money, Recharge Mobile (all specialise the abstract Card Transaction, which «include»s Validate Card & PIN and is linked to the CBS); Recharge Mobile – Mobile Operator; Print Receipt «extend»s Withdraw Money.

NOTE Equivalent flat version (also accepted): draw an «include» arrow from *each* of the six use cases to Validate Card & PIN and a line from the CBS to each of them. The generalization version shows the same information with far fewer crossing lines.

2021–2022 • Q3(b) CSE325 • 07/10/2023 • C325-2122-3b **MARKS 8**

(For the ATM of Q3(a).) (b) Write down the use case for withdrawing money and changing PIN.

SOLUTION

Form used

A use case is written as a **use case narrative (description)**: name, actors, description, pre-condition, trigger, the *typical course of events* (actor action / system response), *alternate courses*, post-condition and business rules.

(i) Use case: Withdraw Money

Use case name	Withdraw Money	ID: UC-ATM-01	Priority: High
Primary actor	Customer (card holder)		
Secondary actor	Core Banking System (CBS)		
Description	The customer takes cash from his/her account using the ATM card.		
Pre-condition	ATM is online, has cash and paper; customer has a valid, unblocked card.		
Trigger	Customer inserts the card and selects *Withdraw*.		
Includes / extends	includes *Validate Card & PIN*; extended by *Print Receipt*		

Actor action	System response
1. Customer inserts the card.	2. ATM reads the card and asks for the PIN.
3. Customer enters the PIN.	4. ATM validates card and PIN with the CBS («include» Validate Card & PIN) and shows the menu.
5. Customer selects *Withdraw* and enters / chooses an amount.	6. ATM checks that the amount is a multiple of the note size, within the daily limit and that the ATM has enough cash.
	7. ATM asks the CBS for the balance and requests a debit; CBS debits the account and returns a transaction ID.
	8. ATM dispenses the cash and ejects the card.
9. Customer takes card and cash.	10. ATM records the transaction; prints a receipt if asked («extend» Print Receipt).

Alternate courses	**4a** wrong PIN: ask again; after 3 wrong attempts the card is retained and the use case ends. **6a** amount invalid / over the daily limit / ATM short of cash: show message, go back to step 5. **7a** insufficient balance: CBS refuses the debit; show "insufficient funds"; back to step 5 or end. **8a** dispense fault: ATM reverses the debit with the CBS and shows an error. **9a** cash not taken within 30 s: ATM retracts the cash and reverses the debit.
Post-condition	Account debited by the amount, cash delivered, transaction logged.
Business rules	Daily withdrawal limit; amount in multiples of the smallest note; maximum 3 PIN attempts.

(ii) Use case: Change PIN

Use case name	Change PIN	ID: UC-ATM-03	Priority: Medium
Primary actor	Customer (card holder)		
Secondary actor	Core Banking System (stores the PIN of the card)		
Description	The customer replaces the current PIN of the card with a new PIN.		
Pre-condition	ATM online; card valid and not blocked.		
Trigger	Customer selects *Change PIN* from the menu.		

Actor action	**System response**
1. Customer inserts the card and enters the current PIN.	2. ATM validates card and PIN («include» Validate Card & PIN) and shows the menu.
3. Customer selects *Change PIN*.	4. ATM asks for the new PIN.
5. Customer enters the new PIN.	6. ATM checks the PIN format (4 digits, not equal to the old PIN, not trivial such as 1111) and asks to re-enter it.
7. Customer re-enters the new PIN.	8. ATM checks that both entries match and sends the (encrypted) new PIN to the CBS.
	9. CBS updates the PIN; ATM shows "PIN changed successfully" and ejects the card.
10. Customer takes the card.	

Alternate courses	**2a** wrong current PIN (3 times): card retained, use case ends. **6a** new PIN violates the format rule: show the rule, back to step 4. **8a** the two entries differ: show "PINs do not match", back to step 4. **9a** CBS not reachable: old PIN stays valid; show "try again later".
Post-condition	The new PIN is active for the card; the old PIN no longer works.

Answer. Both narratives list actors (Customer; CBS), pre-condition, trigger, the actor/system dialogue (insert card, validate PIN, . . .), alternate courses (wrong PIN, insufficient balance / mismatched new PIN, CBS failure) and the post-condition (account debited and cash delivered / new PIN active).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2020–2021 • Q3(c) CSE325 • 12/04/2023 • C325-2021-3c MARKS 8

Show an example of "extends" in a Use Case diagram.

2018–2019 • Q1(b) CSE325 • 20/12/2021 • C325-1819-1b MARKS 8

Show an example of "extends" in a Use Case diagram.

SOLUTION

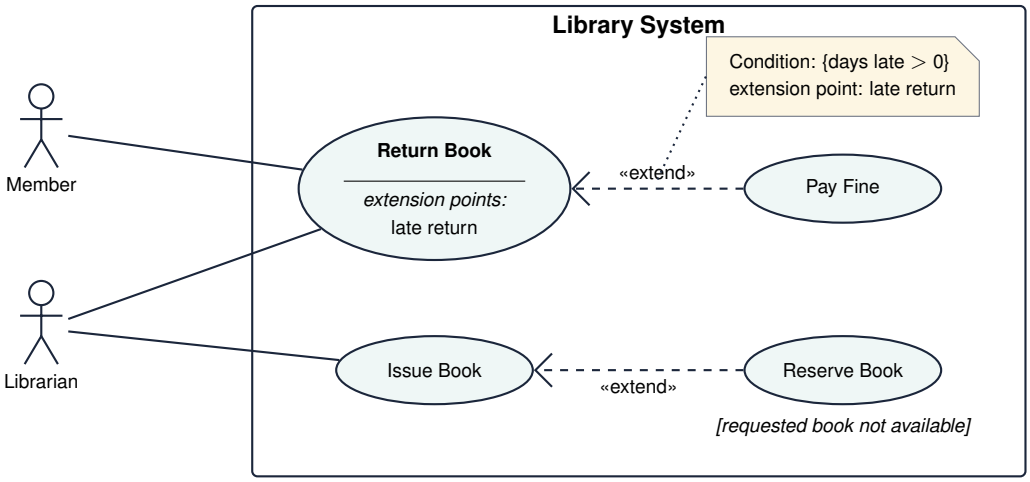
What «extend» means

The **extend** relationship "represents the extension of the use case to include *optional* functionality"

(lecture). The extending use case adds behaviour to a base use case only *under a condition*, at a named **extension point** of the base; the base use case is complete and meaningful without it and does not know about the extension. Notation: a dashed arrow labelled «extend», drawn *from the extension use case to the base use case* (opposite to «include», which points from the base to the always-needed part).

Example: library system

Returning a book is complete on its own; a fine is paid *only if* the book is returned late.



Reading the example

- **Return Book** (base) is complete: the member returns, the system records it. Its compartment lists the extension point *late return*. It works on its own for every on-time return.
- **Pay Fine** «extend»s Return Book: it is inserted at the extension point *only when* the condition {days late > 0} is true. The arrow points from *Pay Fine* to *Return Book* – the base does not know about the extension. The note attached to the «extend» arrow states the condition and the extension point.
- A second example: **Reserve Book** «extend»s *Issue Book* when the requested copy is not available.
- **Contrast with «include»**: an included use case is *always* executed and the arrow goes from base to included (e.g. Issue Book «include»s Validate Membership).

Other typical examples: Print Receipt «extend»s Withdraw Money (only if the customer wants a receipt); Apply Discount Coupon «extend»s Place Order.

Answer. «extend» = optional, conditional behaviour inserted at an extension point; dashed arrow from extension to base, e.g. Pay Fine –«extend»→ Return Book [days late > 0].

2019–2020 • Q1(b) CSE325 • 02/04/2022 • C325-1920-1b MARKS 6 + 12 + 7

Consider a microwave oven that uses microwave technology for the purpose of heating foods. If you want to heat food items you must enter the food item in the heating chamber and then set the timer using the key pad in the front panel located on the door of micro wave oven. You have to press the start button to start heating. The oven will give a beep signal when the heating is complete. You can stop heating by pressing the Stop button any time. Do the following for the

micro wave oven system:

(i) Draw the use case diagram by identifying the actors and use cases for this micro wave oven system.

(ii) Write down the detailed use case scenarios related to setting the timer and heating.

(iii) Write down the acceptance test cases for these use cases as well.

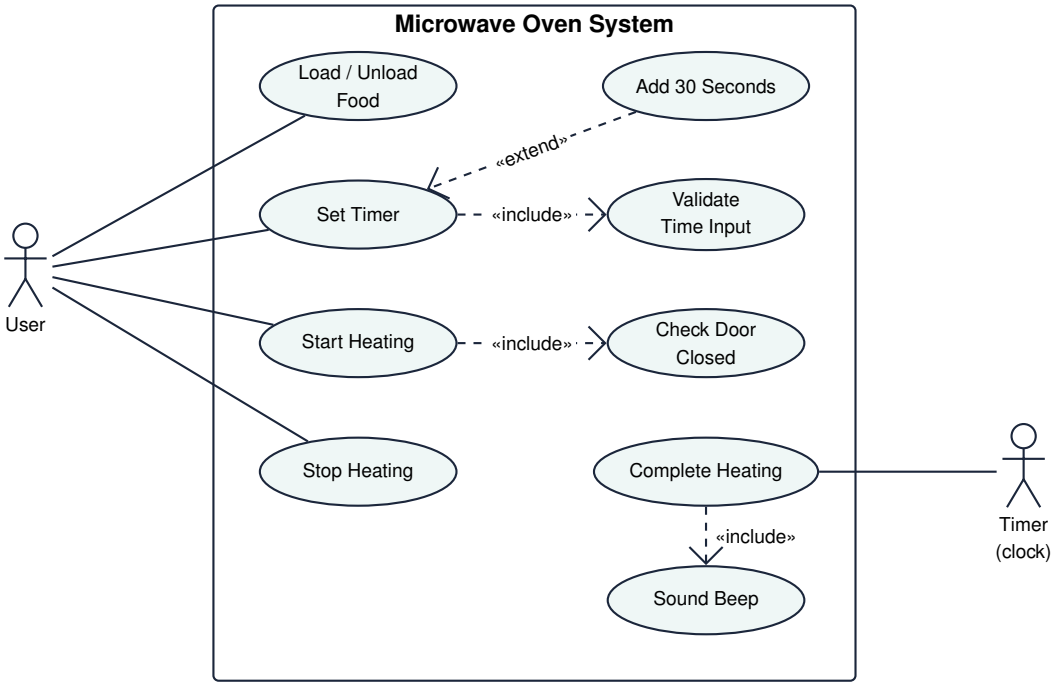
SOLUTION

(i) Use case diagram

Actors.

- **User** (primary actor) – puts food in, sets the timer, presses Start/Stop.
- **Timer / Clock** (time actor) – when the set time runs out it ends the heating. The lecture allows "time or time-based event" as an actor; it is what makes "beep when heating is complete" happen without any user action.

Use cases (each is one goal of an actor): *Load / Unload Food* (open door, place or take food, close door), *Set Timer*, *Start Heating*, *Stop Heating*, *Complete Heating*. Shared behaviour: both *Start Heating* and *Set Timer* need the door to be closed and a valid time, and heating completion always beeps.



Decisions: «include» is used for behaviour that is *always* part of the base (time is always validated, the door is always checked, completion always beeps); «extend» for the optional quick-start key "+30 s" that extends timer setting only when the user presses it. *Stop Heating* is a separate goal of the user ("stop heating by pressing Stop any time"). The *Timer* actor sits outside the boundary because time is external to the oven software.

> **NOTE** *Add 30 Seconds* is not in the question; it is shown only as an illustration of «extend» and may be omitted.

(ii) Detailed use case scenarios

UC-1: Set Timer

Primary actor	User
Goal	Set the heating duration.
Preconditions	Oven is powered on and idle (not heating); food is placed in the chamber.
Trigger	User presses the *Time* key on the front-panel keypad.
Main success scenario	1. User presses *Time*. 2. System shows "00:00" and waits for digits. 3. User enters minutes and seconds (e.g. 0-1-3-0). 4. System «include»s *Validate Time Input*: $0 <$ time $\leq$ 99:59, seconds < 60. 5. System stores the time and displays "01:30".
Alternate flows	**4a.** Invalid time (e.g. 00:00 or 01:75): system beeps once, shows "Err", clears the entry; resume at 2. **3a.** User presses *Stop/Clear* while entering: entry discarded, display returns to clock. **3b.** User presses *+30 s* (extension): time increased by 30 s.
Postcondition	A valid cooking time is stored and displayed; oven is ready to start.

UC-2: Heat Food (Start Heating, Complete / Stop Heating)

Primary actor	User; supporting actor: Timer
Goal	Heat the food for the set time.
Preconditions	Food is in the chamber; a valid time is set (UC-1).
Trigger	User presses *Start*.
Main success scenario	1. User presses *Start*. 2. System «include»s *Check Door Closed*: door sensor reports closed. 3. System switches on the magnetron, lamp and turntable and starts the countdown. 4. System displays the remaining time every second. 5. Timer reaches 00:00 (*Complete Heating*). 6. System switches off the magnetron, lamp and turntable. 7. System «include»s *Sound Beep* (3 beeps) and shows "End".
Alternate flows	**2a.** Door open: system does not heat, shows "Door", waits for the door to be closed; resume at 1. **1a.** No time set: *Start* is ignored (or starts a default 30 s quick-start). **4a.** User presses *Stop* (UC Stop Heating): magnetron off, countdown paused, remaining time kept; pressing *Start* resumes at 3, pressing *Stop* again clears the time. **4b.** Door opened during heating: heating stops immediately (safety), as in 4a.
Postcondition	Food heated for the set time, oven idle, completion signalled by beep.

(iii) Acceptance test cases

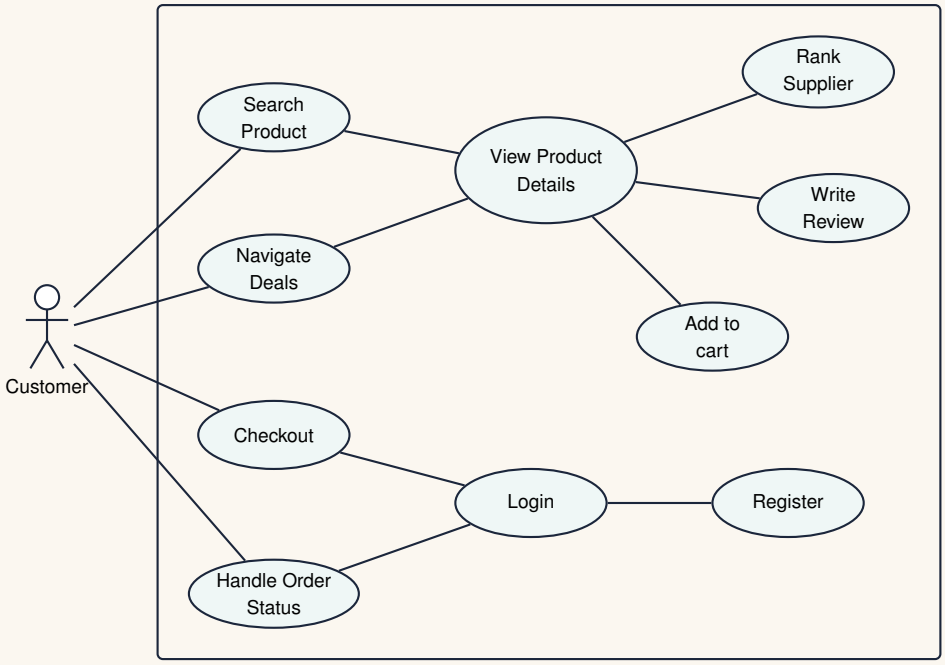
Acceptance tests check, from the user's point of view, that each scenario of the use cases is satisfied (one test per main/alternate flow). Pass = actual result equals expected result.

ID	Precondition	Action / input	Expected result
AT-1	idle, food inside	Time, 0-1-3-0	display “01:30”, time stored
AT-2	idle	Time, 0-0-0-0	“Err”, short beep, time not stored
AT-3	idle	Time, 0-1-7-5 (75 s field)	“Err”, entry cleared
AT-4	entering time	press Stop/Clear	entry discarded, clock shown
AT-5	time 01:30 set, door closed	press Start	heating on, countdown 01:29, 01:28, …
AT-6	time set, door open	press Start	no heating, “Door” shown
AT-7	heating, 00:10 left	wait 10 s	heating off at 00:00, 3 beeps, “End”
AT-8	heating, 00:40 left	press Stop	heating off, “00:40” kept; Start resumes from 00:40
AT-9	heating	open the door	heating stops immediately
AT-10	paused (after Stop)	press Stop again	time cleared, oven idle
AT-11	no time set	press Start	no heating (or 30 s quick start, per requirement)

Answer. (i) Actors User and Timer; use cases Load/Unload Food, Set Timer («include» Validate Time Input), Start Heating («include» Check Door Closed), Stop Heating, Complete Heating (Timer; «include» Sound Beep). (ii) Narratives for Set Timer and Heat Food with alternate flows (invalid time, door open, stop/resume). (iii) One acceptance test per flow: AT-1 … AT-11.

2017–2018 • Q3(a) CSE325 • 10/04/2019 • C325-1718-3a MARKS 24

Consider the following scenario:
Suppose, you want to design an information system for a multi-seller online shopping site. Anyone can search for products, or view some exciting sale offers. In both cases, you will be shown a product list. After you click on a product from the list, you can view the details. Also, in that page, there has to be options for adding the product to the cart (provided that it is available), ranking the supplier of that particular product, and writing a review. After a user has added something to his cart, he can proceed to checkout. In this phase, the system needs to check whether the user is logged in. If the user is an unregistered one, he needs to be requested to sign in. When the user has successfully ordered his product, and is logged into the system, he can view or handle his order status.
Now, complete the Use-case Diagram given below. In case of any extends use-case, clearly mention the extension point. In addition to labeling the existing relationships, you can also add new ones. However, you must not rename the existing use cases. Note that, you can add new use-cases to the diagram, but it is not strictly required.



SOLUTION

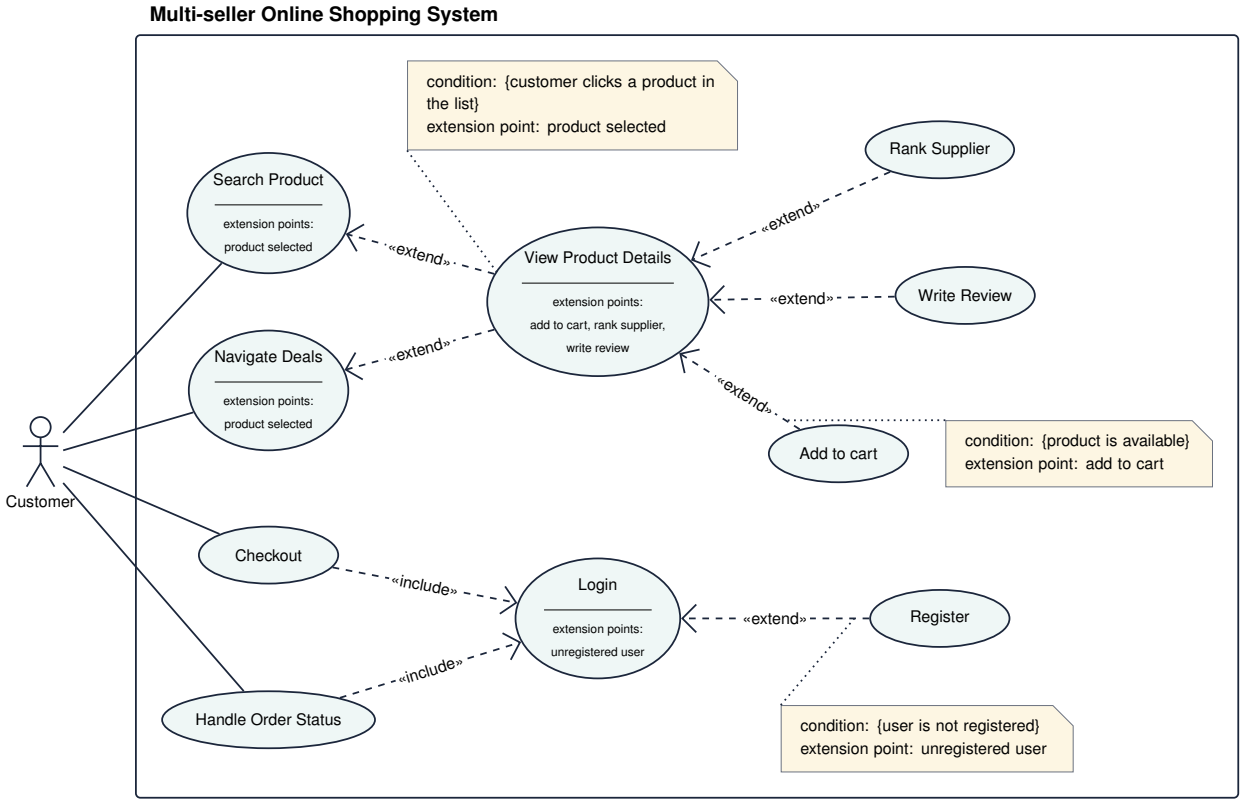
Reading the scenario into relationships

- **«include»** is used when the base use case *always* needs the behaviour of another use case; the arrow goes from the base to the included use case.
- **«extend»** is used when a use case adds *optional/conditional* behaviour to a base use case at a named **extension point**; the arrow goes from the extending use case to the base, and the base lists its extension points in its ellipse (with the condition in a note).

Applying this to every unlabelled line of the given diagram:

Existing line	Label	Reason from the scenario
Customer – Search Product, Navigate Deals, Checkout, Handle Order Status	association	these are the goals the customer starts directly
View Product Details → Search Product	«extend»	details are shown only *if* the customer clicks a product in the result list (extension point *product selected*)
View Product Details → Navigate Deals	«extend»	same: the sale-offer list also leads to details only when a product is clicked
Add to cart → View Product Details	«extend»	optional, and only when {product is available} (extension point *add to cart*)
Rank Supplier → View Product Details	«extend»	optional option on the details page (extension point *rank supplier*)
Write Review → View Product Details	«extend»	optional option on the details page (extension point *write review*)
Checkout → Login	«include»	checkout *always* checks that the user is logged in
Handle Order Status → Login	«include»	order status can only be handled by a logged-in user
Register → Login	«extend»	only an unregistered user has to sign up first (extension point *unregistered user*)

Completed use case diagram



Explanation of the main decisions

- **Two entry points, one details use case.** Both searching and browsing deals end in a product list; viewing the details of a product is an optional continuation of either, so `View Product Details` *extends* both base use cases at the same extension point *product selected*. (It is not «include»: searching is complete even if no product is clicked.)
- **Three optional actions on the details page.** Add to cart, Rank Supplier and Write Review are each optional, so they extend `View Product Details`; the base declares the three extension points. Add to cart carries the condition {product is available} as the question demands.
- **Login is mandatory for checkout and for handling orders**, hence «include» from both. Reusing one `Login` use case avoids describing the check twice.
- **Registration is conditional**: only an unregistered user is asked to sign up before logging in, so `Register` extends `Login` at extension point *unregistered user*.
- **"After adding something to the cart he can proceed to checkout"** is a *precondition* of Checkout (cart not empty), not a relationship between use cases; it belongs to the Checkout use case description. Use case diagrams do not show ordering.
- No existing use case was renamed; no new use case was necessary.

NOTE Some instructors model the login check with `Login` «extend»ing `Checkout` ("only if not yet logged in"). Because the scenario says the system *always* has to check the login state at checkout, «include» is used here, and the conditional part (sign-up for unregistered users) is modelled by the `Register` extension.

Answer. Associations: Customer – Search Product, Navigate Deals, Checkout, Handle Order Status. «extend»: View Product Details → Search Product and → Navigate Deals (ext. point *product selected*); Add to cart [product available], Rank Supplier, Write Review → View Product Details (ext. points *add to cart, rank supplier, write review*); Register → Login (ext. point *unregistered user*). «include»: Checkout → Login, Handle Order Status → Login.

2016–2017 • Q2(a), (b) CSE307 • 15/07/2017 • R016.1 **MARKS 13 + $7\frac{2}{3}$**

The following is a narrative description of the business process of a bank:
There are two types of uses in a Bank: Normal user and Foreign User. Both users can create account, deposit money and withdraw money. For money withdrawal there are certain rules. A user can make a withdrawal of maximum 3,00,000 taka per day. At each transaction the highest limit of withdrawal is 1,00,000 taka. A user can make a maximum of 5 withdrawals per day. After every three months, interest is added to the account balance. If an account balance reaches 20,00,000 taka, for every deposit to that account 2% bonus is applied. The foreign user has one extra capability. He can convert his taka to foreign currency.
The use cases of this scenario are: (i) Create account (ii) Deposit money (iii) Withdraw money (iv) Update balance (v) Calculate bonus (vi) Calculate interest and (vii) Convert to foreign currency.
Now answer the following questions based on the above scenario.

(a) Draw the use case diagram of this system. (13)

(b) Write use case narrative for the use case "withdraw money". ($7\frac{2}{3}$)

SOLUTION

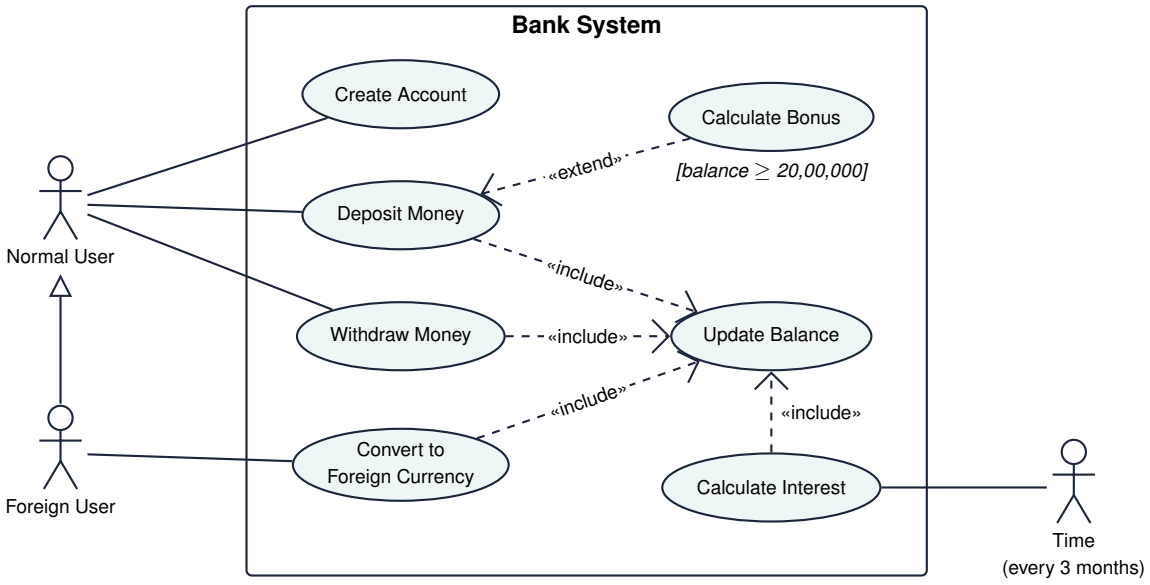
(a) Building the use case diagram

System boundary: the Bank (account) system.
Actors.

- **Normal User** – creates accounts, deposits and withdraws.
- **Foreign User** – can do everything a normal user can *plus* convert taka to foreign currency, so it is an **actor generalization**: Foreign User → Normal User (hollow triangle at the parent).
- **Time** (a quarterly clock) – "after every three months" interest is added. A time-based event is a legitimate actor (slides: an actor may be "time or time-based event").

Relationships between the given use cases.

- Every deposit, withdrawal, interest posting and currency conversion changes the balance, so each of them «include»s *Update Balance* (mandatory, reused behaviour).
- *Calculate Bonus* happens only *when the balance has reached 20,00,000 taka*; it is conditional extra behaviour of a deposit ⇒ *Calculate Bonus* «extend»s *Deposit Money* (extension point: *balance* ≥ *20,00,000*).
- *Calculate Interest* is started by the Time actor and «include»s *Update Balance*.
- *Convert to Foreign Currency* is associated only with Foreign User.



Modelling decisions.

- The withdrawal limits (1,00,000 per transaction, 3,00,000 and 5 withdrawals per day) are *business rules* of *Withdraw Money*; they belong in its narrative, not as separate ovals.
- Generalization means Foreign User inherits the three associations of Normal User, so they are not drawn twice. (An equally correct variant uses an abstract actor *User* with two children.)
- Arrow directions follow the slides: «include» from base to included use case; «extend» from the extension (*Calculate Bonus*) to the base (*Deposit Money*).

(b) Use case narrative: "Withdraw Money"

Use case name	Withdraw Money	ID: UC-03	Priority: High
Primary actor	Normal User (or Foreign User, by inheritance)		
Stakeholders	Account holder (wants cash quickly and safely); Bank (wants limits enforced, correct balance)		
Description	The user withdraws cash from his/her account; the system checks the withdrawal rules and updates the balance.		
Precondition	The user has an active account and is identified (card/PIN or signature verified).		
Trigger	The user requests a withdrawal of amount A.		

Actor action	**System response**
1. User selects *Withdraw* and enters account number and amount A.	2. System retrieves the account and today's withdrawal history (count n, total T).
	3. System checks the rules: $A \leq 1{,}00{,}000$; $T + A \leq 3{,}00{,}000$; $n < 5$; $A \leq$ balance.
	4. System performs *Update Balance* («include»): balance $\leftarrow$ balance $- A$; records the transaction; $n \leftarrow n + 1$, $T \leftarrow T + A$.
5. User collects the cash.	6. System prints/sends a receipt showing amount and new balance.

Alternate courses	**3a.** $A > 1{,}00{,}000$: system rejects, shows the per-transaction limit, user may enter a smaller amount (back to step 1). **3b.** $T + A > 3{,}00{,}000$: system rejects and shows the remaining daily limit $3{,}00{,}000 - T$. **3c.** $n = 5$: system rejects – daily count of 5 withdrawals reached. **3d.** Insufficient balance: system rejects the request.
Postcondition	Success: balance reduced by A, transaction logged, daily counters updated. Failure: account unchanged.
Business rules	Max 1,00,000 Tk per transaction; max 3,00,000 Tk and max 5 withdrawals per day.
Assumptions	Daily counters are reset at midnight.

Answer. (a) Actors Normal User, Foreign User ($\rightarrow$ Normal User), Time; Deposit/Withdraw/Convert/Calculate Interest «include» Update Balance; Calculate Bonus «extend»s Deposit Money [balance $\geq$ 20 lakh]; Convert to Foreign Currency only for Foreign User. (b) Narrative with the three limit checks and their alternate courses.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2014–2015 • Q6(d) CSE307 • 04/02/2016 • R021.2 MARKS $9\frac{2}{3}$

A book is written by an author, published by a publisher, sold by a book store, and read by a reader. Moreover, for a reader to read a book, she must buy it from a book store that is selling it. Draw a use case diagram for this scenario, showing relationships between different use cases.

2010–2011 • Q2(c) CSE307 • 19/03/2012 • R043.4 **MARKS 8**

A book is written by an author, published by a publisher, sold by a book store, and read by a reader. Moreover, for a reader to read a book, she must buy it from a book store that is selling it. Draw a **Use Case diagram** for this scenario, showing relationships between different use cases.

SOLUTION

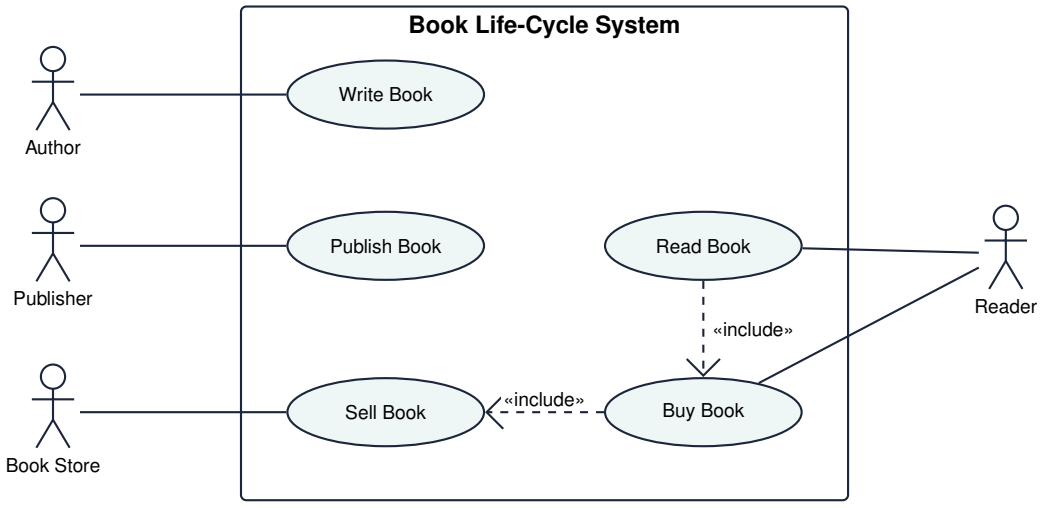
Step 1 – actors and use cases (WHO? / WHAT?)

Actor	Use case (goal)
Author	Write Book
Publisher	Publish Book
Book Store	Sell Book
Reader	Buy Book, Read Book

Step 2 – relationships (HOW RELATED?)

- "For a reader to read a book, she **must** buy it" ⇒ buying is a *mandatory* part of reading ⇒ *Read Book* «include» *Buy Book*.
- "…buy it from a book store **that is selling it**" ⇒ a purchase can only be completed through the store's sale transaction (check the book is on sale, take payment, hand over the book). That sale behaviour is reused by every purchase ⇒ *Buy Book* «include» *Sell Book*.
- Writing → publishing → selling is a chronological order of *different* actors' goals. A mere "happens before" is not an include/extend (lecture notes), so Write Book and Publish Book are left unconnected; the order is a precondition ("book has been published") in their narratives.

Use case diagram



Reading the diagram

- Four actors, one per role in the sentence; each is linked only to its own goal.

- The two «include» arrows encode the only two "must" constraints of the scenario: Read Book always executes Buy Book, and Buy Book always executes the store's Sell Book.
- Reader is associated with both Read Book and Buy Book because she directly initiates both.

Answer. Actors: Author, Publisher, Book Store, Reader. Use cases: Write Book, Publish Book, Sell Book, Buy Book, Read Book. Relationships: Read Book «include» Buy Book; Buy Book «include» Sell Book.

2013–2014 • Q1(c), (d) CSE307 • 08/08/2015 • R023.2 MARKS $7\frac{2}{3}$ + 15

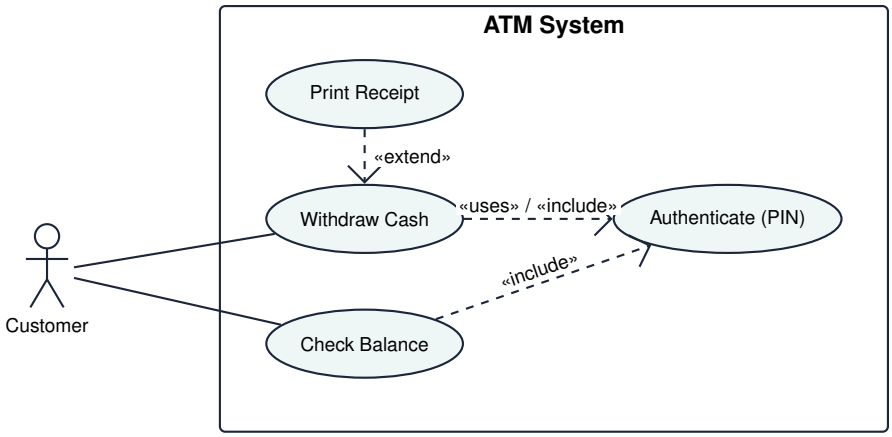
(c) Define *extends* and *uses* relationship in *use case diagrams* with proper real life examples. ($7\frac{2}{3}$)

(d) Consider a teller of a bank serving clients to deposit examination fees in a queue. Show the extended version of *use case* for depositing examination fees by a student. (15)

SOLUTION

(c) «uses» and «extends» relationships

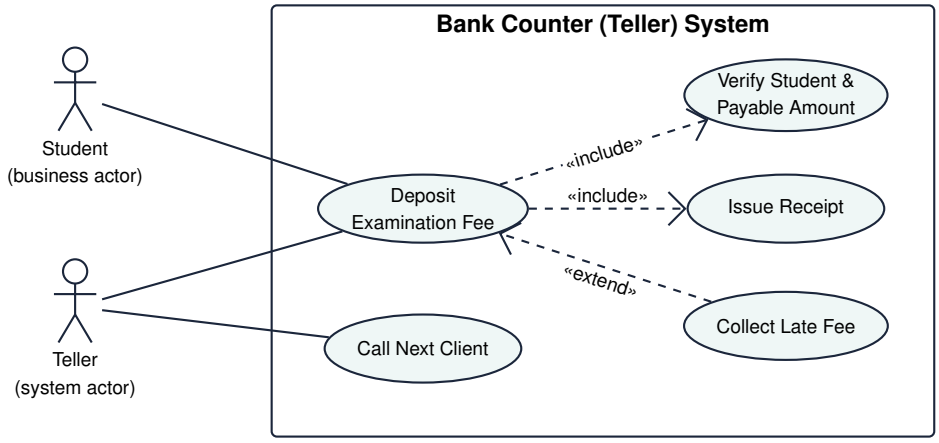
Uses (= «include» in current UML). Represents the *inclusion of the functionality of one use case within another*. The base use case *always* executes the used use case; the used use case is usually a common step shared by several use cases. Arrow (dashed, open head) is drawn **from the base use case to the used use case**, labelled «uses» / «include».
Real-life example: at an ATM, *Withdraw Cash*, *Check Balance* and *Transfer Money* all «use» *Authenticate (PIN)* – none can proceed without it.
Extends (= «extend»). Represents the *extension of a use case to include optional functionality*. The extending use case runs only under a condition at an *extension point* of the base; the base is complete without it. Arrow drawn **from the extension use case to the base use case**, labelled «extend».
Real-life example: Print Receipt «extend»s *Withdraw Cash* – only if the customer asks for a receipt. (Slides: *Update patient information* «extend»s *Make appointment*.)



	«uses» (include)	«extends»
Execution	always (mandatory)	only when a condition holds (optional)
Arrow	base → used use case	extension → base use case
Base complete alone?	No – needs the used use case	Yes
Purpose	factor out common behaviour	add exceptional / optional behaviour

(d) Extended (expanded) use case: "Deposit Examination Fee"

An **extended / expanded use case** (fully documented use case narrative) goes beyond the one-paragraph high-level description: it names the actors, pre- and post-conditions, trigger, the *typical course of events* as an actor-action / system-response dialogue, and the *alternate courses*. First the context diagram:



Use case name	Deposit Examination Fee	ID: UC-01	Priority: High
Primary business actor	Student (owner of the goal "fee paid")		
Primary system actor	Bank Teller (operates the system)		
Other actors	University fee account (receives the money)		
Stakeholders	Student, Teller, Bank, University exam controller		
Description	A student waiting in the queue pays his/her examination fee at the teller's counter; the teller verifies the student, records the payment and gives a receipt.		
Precondition	Teller is logged in; counter is open; student has a filled-in deposit slip (student ID, exam, amount) and the money.		
Trigger	The student reaches the front of the queue and hands over the slip and money.		

Actor action	**System response**
1. Teller calls the next student in the queue; the student hands over deposit slip and cash.	
2. Teller enters the student ID and exam code.	3. System retrieves the student record and the payable exam fee («include» Verify Student & Payable Amount).
4. Teller counts the cash and enters the amount received.	5. System checks amount = payable fee, records the deposit, credits the university fee account and generates a transaction number.
	6. System prints the receipt in two copies («include» Issue Receipt).
7. Teller stamps and signs, returns the student copy and keeps the bank copy.	8. System marks the student's exam fee as *paid*.
9. Teller calls the next client (back to step 1).	

Alternate courses	**3a.** Student ID not found / not eligible: system shows error; teller returns the slip; use case ends. **3b.** Fee already paid: system shows the earlier transaction; no new deposit. **5a.** Amount less than payable: teller asks for the balance or returns the money. **5b.** Amount more than payable: system computes change; teller returns it. **5c.** Deposit after the deadline: *Collect Late Fee* («extend») adds the fine to the payable amount. **6a.** Printer failure: teller writes a manual receipt quoting the transaction number.
Conclusion	Ends when the student receives the stamped receipt.
Postcondition	Payment recorded and linked to the student ID; university account credited; receipt issued.
Business rules	Only the exact exam fee (plus late fee if applicable) is accepted; one receipt per transaction.
Assumptions	Clients are served first-come-first-served from a single queue.

Answer. (c) «uses»/«include»: mandatory reused behaviour, base → used (Withdraw Cash uses Authenticate); «extend»: optional conditional behaviour, extension → base (Print Receipt extends Withdraw Cash). (d) Expanded narrative of *Deposit Examination Fee* with actors, pre/post-conditions, trigger, typical course (steps 1–9) and alternate courses (invalid ID, wrong amount, late fee, printer failure).

2013–2014 • Q6(d) CSE307 • 08/08/2015 • R026.2 **MARKS 22**

Consider the following problem description:
Consider an online reservation system for a bus company. The bus company includes several buses and realizes trips to different cities. Each bus is identified by its plate number and a separately assigned bus number. The trips are based on a predefined schedule and stop at predefined bus stations. Each bus can have only one trip per day. Each bus includes a driver and one hostess. For long trips, the bus will have a breaks at service and rest areas. There are two types of trips: normal trips and express trips. Express trips do not stop at intermediate stations and get faster at the destination.
Seats can be reserved by customers on the web site of the bus company. The customer has the option to directly pay for the seat through the website. In that case, the seat cannot be cancelled (neither by the customer nor by the company). If the customer has not paid for the seat, the bus company can cancel the seat if the customer does not show up one hour before the trip. When the reservation is cancelled, the seat will become free and can be sold to another customer. Both the customer and the company staff must authenticate themselves for performing operations with the system.
Draw a use case diagram for describing the functional requirements of the above system.

SOLUTION

Step 1 – system and actors

System: Online Bus Reservation System. **Actors:** *Customer* (reserves, pays, cancels own unpaid seat) and *Company Staff* (maintains buses, crews and trips; cancels no-show seats). Buses, drivers, hostesses, stations are *data* the system stores (nouns that do not interact), so they are **not** actors.

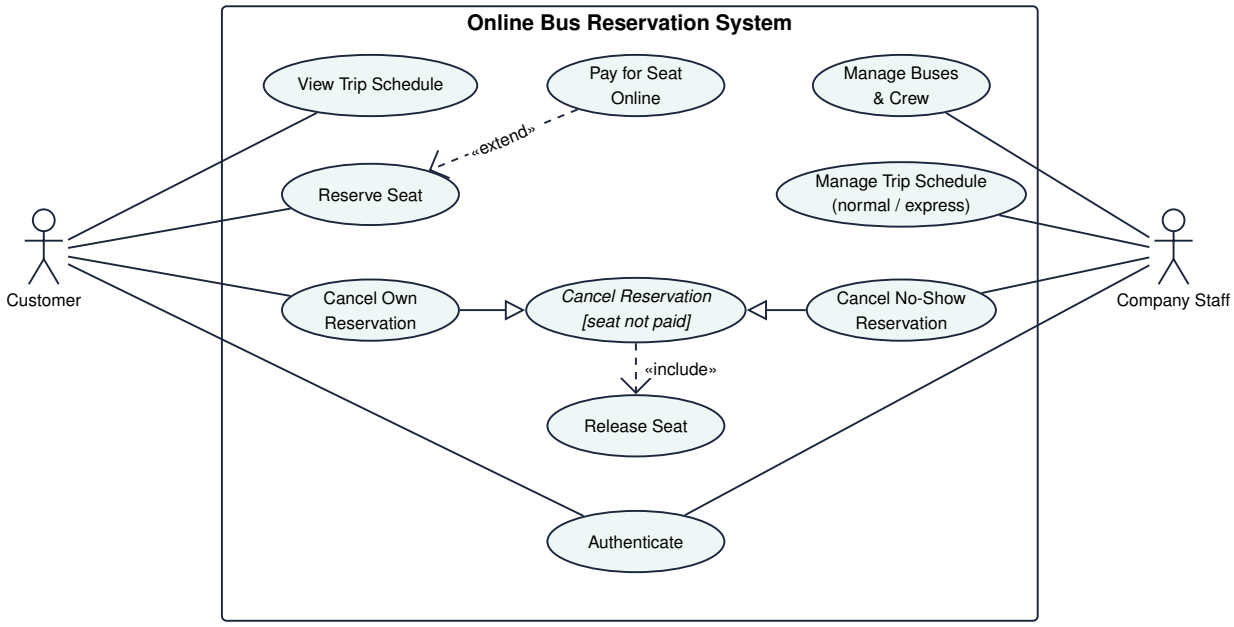
Step 2 – use cases

Use case	Actor	Source sentence / remark
View Trip Schedule	Customer	trips on a predefined schedule and stations
Reserve Seat	Customer	"Seats can be reserved by customers on the web site"
Pay for Seat Online	Customer	"option to directly pay" – optional ⇒ «extend»
Cancel Own Reservation	Customer	allowed only if the seat is unpaid
Cancel No-Show Reservation	Staff	unpaid seat, customer absent 1 hour before the trip
Cancel Reservation (abstract)	–	common parent of the two cancellations
Release Seat	–	"the seat will become free and can be sold" – always after cancellation
Manage Buses & Crew	Staff	plate no., bus no., one driver + one hostess, one trip/day
Manage Trip Schedule	Staff	normal/express trips, stations, rest-area breaks
Authenticate	both	"Both . . . must authenticate themselves"

Step 3 – relationships

- *Pay for Seat Online* «extend»s *Reserve Seat* – only if the customer chooses to pay now.
- *Cancel Own Reservation* and *Cancel No-Show Reservation* are two specialisations of *Cancel Reservation* (use-case generalization); both apply the guard *[seat not paid]*.
- *Cancel Reservation* «include»s *Release Seat*: freeing the seat is mandatory after any cancellation, and the freed seat is then available to *Reserve Seat* for another customer.
- *Authenticate* is associated with both actors (each operation requires a logged-in actor).

Use case diagram



Modelling decisions worth stating in the exam

- **Paid seats cannot be cancelled**: shown by the guard *[seat not paid]* on the abstract *Cancel Reservation*; it is a precondition inherited by both specialised cancellations.
- **Normal vs express trip** and **rest-area breaks** are attributes/variants handled inside *Manage Trip Schedule*; they do not change what the actors want to achieve, so no separate ovals (they would appear in the class diagram as a `Trip` hierarchy).
- **"Seat can be sold to another customer"** needs no new use case: it is simply *Reserve Seat* performed by a different customer once *Release Seat* has freed it.
- **Authentication**: drawn once and associated with both actors. An equivalent answer draws «include» from every operational use case to *Authenticate*; that is correct but cluttered.
- Cancelling a no-show is time-dependent (1 hour before departure); if desired, a *Time* actor can be attached to *Cancel No-Show Reservation* to show that a timer triggers the check.

Answer. Two actors (Customer, Company Staff). Customer: View Trip Schedule, Reserve Seat (extended by Pay for Seat Online), Cancel Own Reservation. Staff: Manage Buses & Crew, Manage Trip Schedule, Cancel No-Show Reservation. Both cancellations specialise *Cancel Reservation [seat not paid]*, which «include»s Release Seat; both actors use Authenticate.

2012–2013 • Q8(a) CSE307 • 17/05/2014 • R033.6 **MARKS 15 + 5**

The Northeast Hotel's website allows potential guests to make a room reservation, specifying the dates and type of room. If they have registered with the website previously, their stored details are used to speed up the process; otherwise they are required to register as a new customer. Each reservation is given a unique reservation code. Before the date of their stay, they may enter this reservation code into the website to amend or cancel the reservation. Amendments can include altering the dates, changing the room type or the number of guests in each room. When the guests arrive at the hotel the reservation id is used by the receptionist to quickly find the reservation to check them in. At the end of their stay, the receptionist checks the guests out. At this point, the hotel system validates their payment through the card payment system. A printed invoice may be requested by the guests. The hotel has many room types available, each with a room-type name, number of guests and additional facility information. Each room in the hotel has a room number and is of one specific type. Monthly reports are prepared by the system which may be viewed on request by the Hotel Manager.

(i) Produce a System Use Case diagram for the above scenario.

(ii) Also give the Use Case Dependency diagram.

SOLUTION

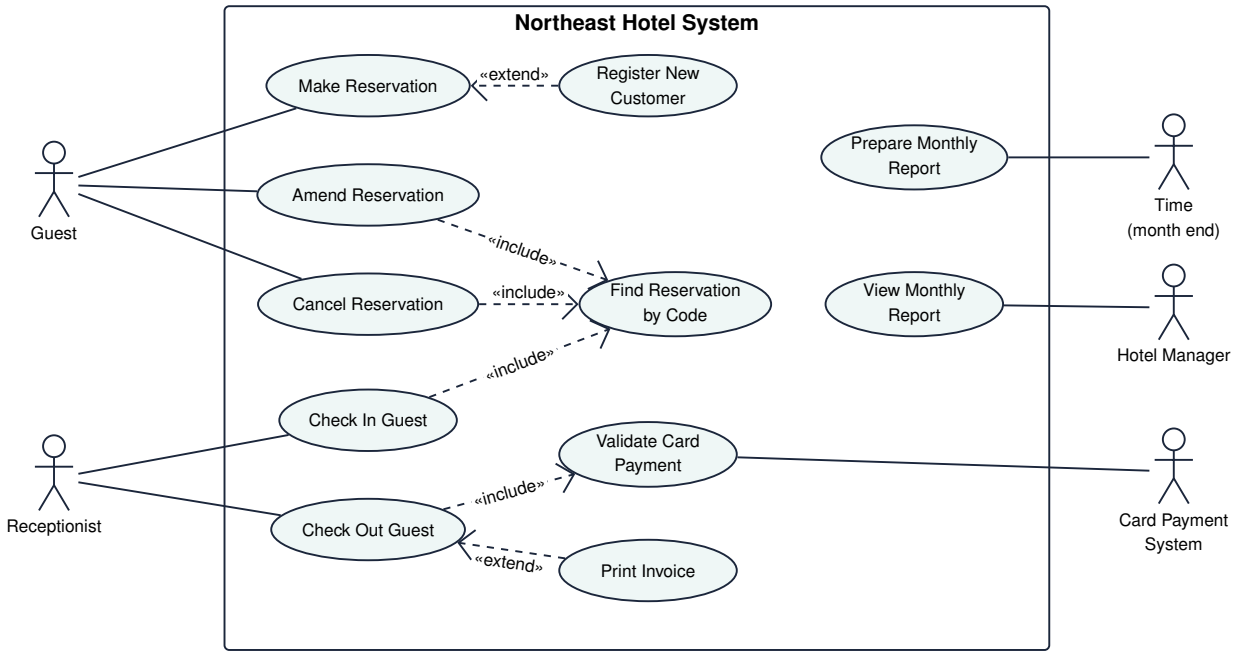
(i) System use case diagram – analysis

Actors. *Guest* (uses the website), *Receptionist* (checks guests in/out), *Card Payment System* (external system that validates payment), *Hotel Manager* (views reports), *Time* (end of month – triggers report preparation). Rooms and room types are stored data, not actors.

Use cases and relationships.

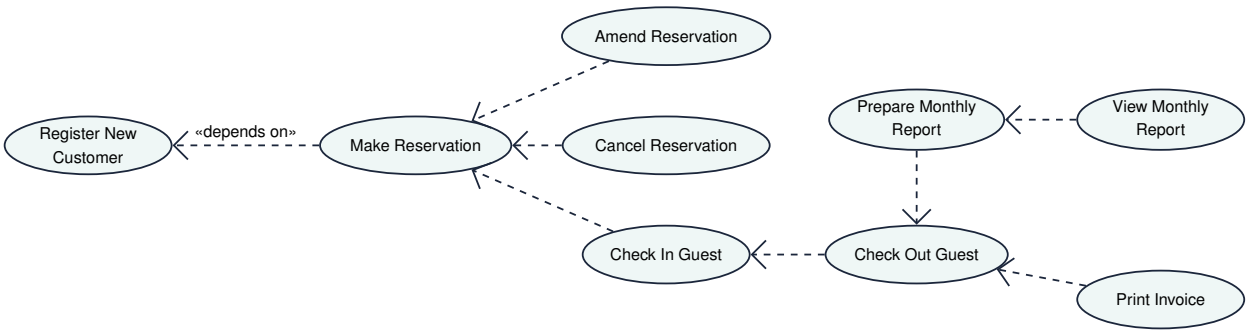
- *Make Reservation* (Guest). "If not registered they are required to register" – conditional ⇒ *Register New Customer* «extend»s *Make Reservation* [guest not registered].

- *Amend Reservation*, *Cancel Reservation* (Guest) and *Check In Guest* (Receptionist) all start by finding the reservation from its code ⇒ each «include»s *Find Reservation by Code* (reused, mandatory).
- *Check Out Guest* (Receptionist) always validates payment ⇒ «include» *Validate Card Payment*, which is associated with the external *Card Payment System*.
- "A printed invoice *may* be requested" ⇒ *Print Invoice* «extend»s *Check Out Guest*.
- *Prepare Monthly Report* (Time) and *View Monthly Report* (Hotel Manager).



(ii) Use case dependency diagram

A **use case dependency diagram** shows the *order* in which business use cases must be carried out: use case *A depends on B* when *B* must have happened before *A* can start (e.g. you cannot amend a reservation that was never made). Unlike the system use case diagram it ignores actors and helps decide the *implementation / release order* of use cases. Here a dashed arrow $A \dashrightarrow B$ means "*A* «depends on» *B*".



- *Make Reservation* depends on the customer being registered (*Register New Customer*).
- *Amend*, *Cancel* and *Check In* all need an existing reservation code.

- *Check Out* needs a checked-in guest; *Print Invoice* needs a completed check-out (validated payment).
- *Prepare Monthly Report* uses the month's stays/payments; *View Monthly Report* needs a prepared report.

Reading it left to right gives a sensible build order: registration → reservation → amend/cancel/check-in → check-out → invoice and reports. (*Validate Card Payment* and *Find Reservation by Code* are included sub-use-cases executed inside their bases, so they are not separate nodes here.)

Answer. (i) Actors Guest, Receptionist, Hotel Manager, Card Payment System, Time; Register New Customer «extend»s Make Reservation; Amend/Cancel/Check In «include» Find Reservation by Code; Check Out «include»s Validate Card Payment and is «extend»ed by Print Invoice; Prepare and View Monthly Report. (ii) Dependency chain: Register ← Make Reservation ← {Amend, Cancel, Check In}; Check In ← Check Out ← {Print Invoice, Prepare Report ← View Report}.

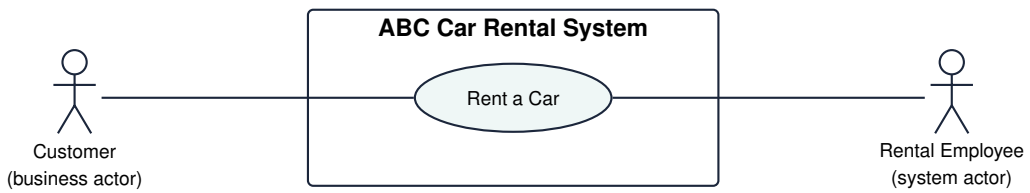
2011–2012 • Q2(a) CSE307 • 23/09/2013 • R035 **MARKS 18 2/3**

Read the following scenario carefully:
A car rental company called "ABC Car Rentals" provides rent-a-car services to its customers. To rent a car from the company a customer must first visit its office. He then provides the time and duration to a company employee at the office. The employee checks the system and shows the customer available car models. The customer chooses a car model. The employee checks if that model is available for the given time. If not then the customer is asked to choose again. After the car model is fixed the employee asks the customer for his national id number and other necessary information. The information is entered into the system by the employee. The specific car is also booked for that particular time given by the customer. The employee then informs the customer about the rental cost and asks him to pay half the cost in advance. If the customer is unable to pay the cost then the booking is cancelled.
Now give the use case narrative for the use-case "Rent a Car" given this scenario.

SOLUTION

What a use case narrative contains

A **use case narrative** is the textual description of one use case. The fully documented (expanded) form records: name/ID, actors (the *primary business actor* who benefits and the *primary system actor* who operates the system), stakeholders, description, *precondition*, *trigger*, the *typical course of events* written as an actor-action / system-response dialogue, *alternate courses* (what happens when a step does not go the normal way), conclusion, *postcondition*, business rules and assumptions.
Here the customer never touches the system: the **employee** is the system actor and the **customer** is the business actor.



Use case narrative: "Rent a Car"

Use case name	Rent a Car	ID: UC-RAC-01	Priority: High
Primary business actor	Customer		
Primary system actor	Company employee (rental clerk)		
Stakeholders	Customer (wants a suitable car for the period); ABC Car Rentals (wants confirmed, half-paid bookings without double-booking)		
Description	A customer at the office books a car of an available model for a given time and duration and pays half of the rental cost in advance.		
Precondition	Customer is physically at the ABC office; employee is logged into the system; car and model data are up to date.		
Trigger	Customer asks the employee to rent a car and states the start time and duration.		

Typical course of events

Actor action	System response
1. Customer gives the rental time and duration; employee enters them.	2. System displays the car models that have cars free in that period.
3. Employee shows the list; customer chooses a model; employee selects it.	4. System checks availability of that model for the given time and confirms it is available.
5. Employee asks for the national ID number and other necessary information (name, address, phone, licence) and enters it.	6. System validates the data and creates/updates the customer record.
	7. System books a specific car of the chosen model for that time, computes the rental cost and the 50% advance, and displays both.
8. Employee informs the customer of the cost; customer pays half in advance; employee records the payment.	9. System records the advance, marks the booking *confirmed* and prints the booking slip/receipt.
10. Employee hands the slip to the customer.	

Alternate courses	**Alt-Step 2:** no model is free in the requested period – employee informs the customer, who may give a different time/duration (back to step 1) or leave (use case ends). **Alt-Step 4:** chosen model not available for the given time – system reports it; customer is asked to choose again (back to step 3). **Alt-Step 6:** NID or other data invalid/incomplete – system highlights the fields; employee re-enters (back to step 5). **Alt-Step 8:** customer is unable to pay the advance – employee cancels; system cancels the booking and releases the car for that time; use case ends unsuccessfully.
Conclusion	Ends when the customer receives the booking slip (or the booking is cancelled).
Postcondition	Success: a specific car is reserved for the customer for the period; customer record and 50% advance stored. Failure: no booking exists and the car remains free.
Business rules	Advance = 50% of total rental cost; a car cannot be booked for overlapping periods; a booking requires the customer's NID.
Assumptions / open issues	Cost is computed from model rate $\times$ duration; how the remaining 50% is paid is outside this use case.

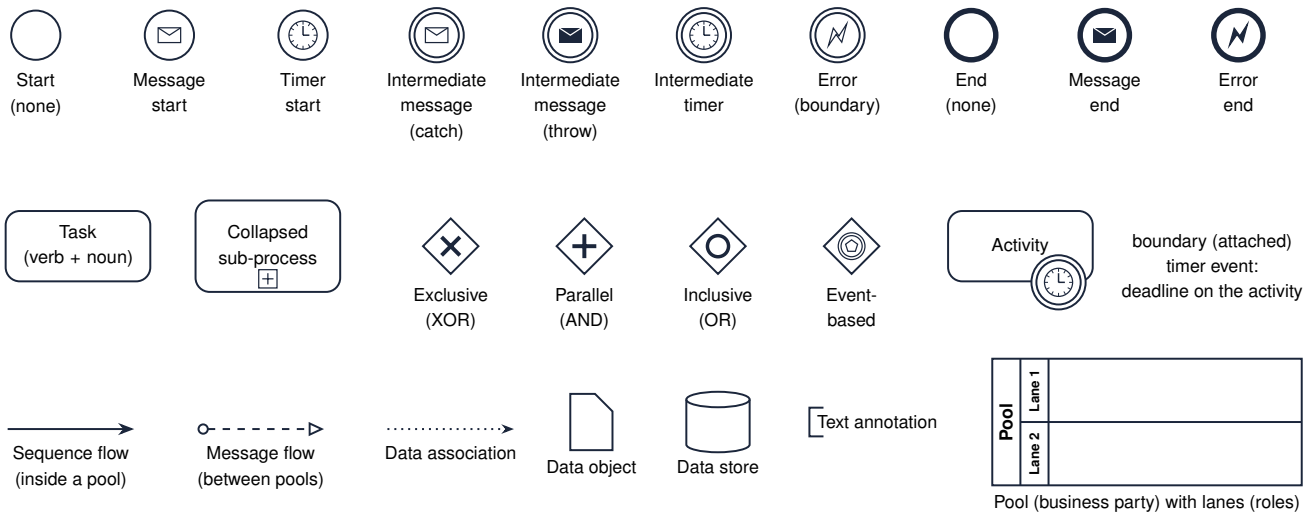
Answer. Narrative of *Rent a Car*: business actor Customer, system actor Employee; trigger = customer requests a car with time and duration; typical course 1–10 (enter time → list models → choose → check → enter NID data → book car → compute cost → pay 50% → confirm); alternate courses for model unavailable (choose again) and advance not paid (booking cancelled).

14

BPMN Diagram

NOTE No CSE213/CSE307 paper in the question bank contains a BPMN question; the BPMN questions below come from the CSE325 papers (most recent first). The chapter ends with **five exam-style practice problems** (not from a past paper), written in the style of recent diagram questions: a paragraph-long, realistic scenario with several participants, decisions, waiting and messages to outside parties, followed by "Draw a BPMN diagram …" and a short second part. Every problem is fully solved.

BPMN notation at a glance (as used in the lecture)



EXAM TIP A safe five-step recipe for any "draw a BPMN diagram" question: **(1)** list the parties – independent organisations become *pools*, departments/roles/systems inside one organisation become *lanes*; **(2)** find the trigger (start event: none, message or timer) and every outcome (one end event per outcome); **(3)** underline the verbs – each unit of work is a task named *verb* + *noun*; **(4)** translate the connectives: "if/otherwise" → XOR, "meanwhile/in parallel/both" → AND, "one or more of" → OR, "whichever happens first" → event-based gateway, "waits until/for" → intermediate event, "within n days" → timer; **(5)** connect: sequence flows inside a pool, message flows between pools, dotted associations to data objects/stores. Every split you open must be closed by a join of the *same* type.

2023–2024 • Q7(b) CSE325 • 17/05/2026 • C325-2324-7b **MARKS 10**

There are three types of flow objects used in BPMN (Business Process Model and Notation) diagram, such as event, activity, and gateway. Illustrate the use of these flow objects with examples.

SOLUTION

Flow objects – the core elements of BPMN

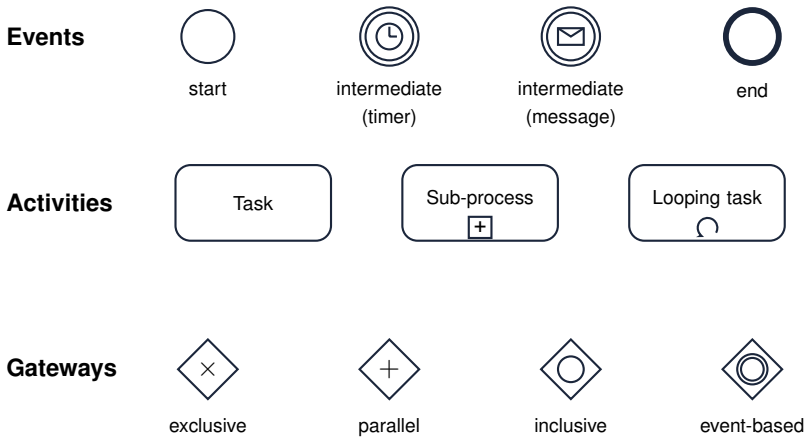
Flow objects are the nodes connected by **sequence flows** (solid arrows) inside a pool/lane. There are exactly three kinds:

(i) **Event** (circle) – *something that happens* and affects the flow. By position: **start** (thin border, begins the process), **intermediate** (double border, occurs during the process – e.g. wait for a message or a time), **end** (thick border, finishes a path). The icon inside gives the trigger: message (envelope), timer (clock), error, etc.

(ii) **Activity** (rounded rectangle) – *work that is performed*. A **task** is atomic; a **sub-process** (marked [+]) contains further detail; markers show loops or multiple instances.

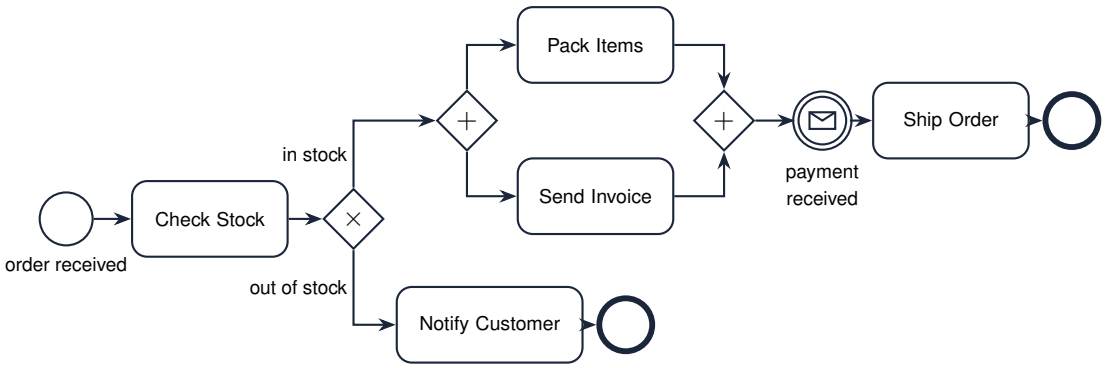
(iii) **Gateway** (diamond) – *controls divergence and convergence* of the flow. **Exclusive (XOR, ×)**: exactly one outgoing path by condition; **parallel (AND, +)**: all paths in parallel (split) / wait for all (join); **inclusive (OR, ○)**: one or more paths; **event-based**: path chosen by the event that occurs first.

Notation summary



Example – online order handling using all three flow objects

Scenario. A shop receives an order. If the items are in stock, packing and invoicing are done in parallel; after both are finished the shop waits for the payment message and then ships the order. If the items are out of stock, the customer is notified and the process ends.



Where each flow object is used in the example

Flow object	Element	Purpose in the process
Event	start event "order received"	triggers a new process instance
	intermediate message event	the flow waits until the payment message arrives
	two end events	each path finishes (shipped / customer notified)
Activity	Check Stock, Pack Items, Send Invoice, Ship Order, Notify Customer	units of work done by the shop
Gateway	exclusive ($\times$)	chooses exactly one path: in stock vs. out of stock
	parallel split ($+$)	Pack Items and Send Invoice run at the same time
	parallel join ($+$)	waits until *both* parallel tasks are complete

Answer. Events (circles: start, intermediate, end, with triggers such as message/timer) mark what *happens*; activities (rounded rectangles: task, sub-process) are the *work*; gateways (diamonds: XOR $\times$, AND $+$, OR $\circ$, event-based) *split and merge* the flow – e.g. an order process with a start event, Check Stock task, XOR on stock, AND split/join for packing and invoicing, a message event for payment and end events.

2022–2023 • Q1(a) CSE325 • 30/01/2025 • C325-2223-1a MARKS 15

In Bangla Academy, published books are kept in production stock. When there is order from their facebook page, they deliver books from here. Copies of books are also brought to their physical book outlet for onsite sell. They also send number of books to different book fairs demand and unsold books are returned to stock after the fair ends.
There are different types of discounts for member of Bangla academy, fairs, bulk purchase, etc. which vary over time. The management wants to have real time picture of stock and sell along with financial transactions to ensure transparency.
Design a BPMN for the stock management of books at Bangla Academy considering stock reconciliation after book fairs.

SOLUTION

Step 1 – Read the scenario as a process: participants, triggers, flows

BPMN (Business Process Model and Notation) shows *who* does *what* and in *which order*: the organisation is a **pool**, its departments are **lanes**, work items are rounded **tasks**, decisions are **gateways**, and anything that "happens" (an order arrives, a fair ends) is an **event**. Sequence flows (solid) stay inside a pool; communication with an outside party is a **message flow** (dashed, open circle at the source).

BPMN element	Taken from the scenario
Pool	*Bangla Academy* (white box); external *Customer* and *Book fair organiser* (black boxes)
Lanes	Sales desk (Facebook page), Production stock (store), Book outlet, Fair team, Accounts
Start events	message: "order from their facebook page"; conditional: outlet runs low (copies are *brought* to the outlet); message: "book fairs demand"
Timer event	"after the fair ends" ⇒ interrupting timer boundary event on the selling task
Gateways	in stock? (XOR); issued = sold + returned? (XOR, the reconciliation check)
Data store	*Stock & sales DB* – every stock movement and sale is written here, which is what gives the management its "real-time picture"
Business rule	"discounts … vary over time" ⇒ the price is computed by a rule task (*apply current discount*) instead of being hard-wired

Because the three channels (online, outlet, fair) start independently, the model uses **two diagrams of the same pool**: (A) the regular stock movements (online orders and the outlet), and (B) the book-fair cycle that ends with the **stock reconciliation** the question stresses.

Diagram A – online orders and outlet sales

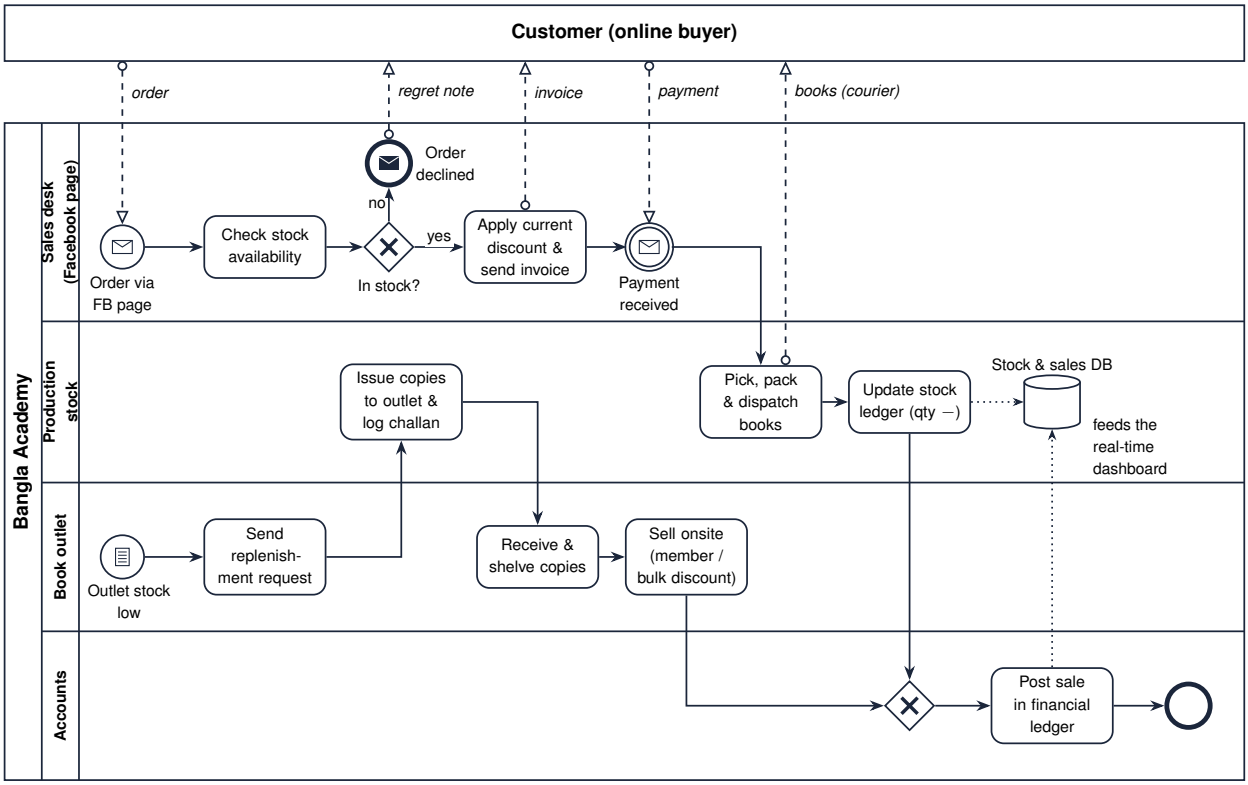
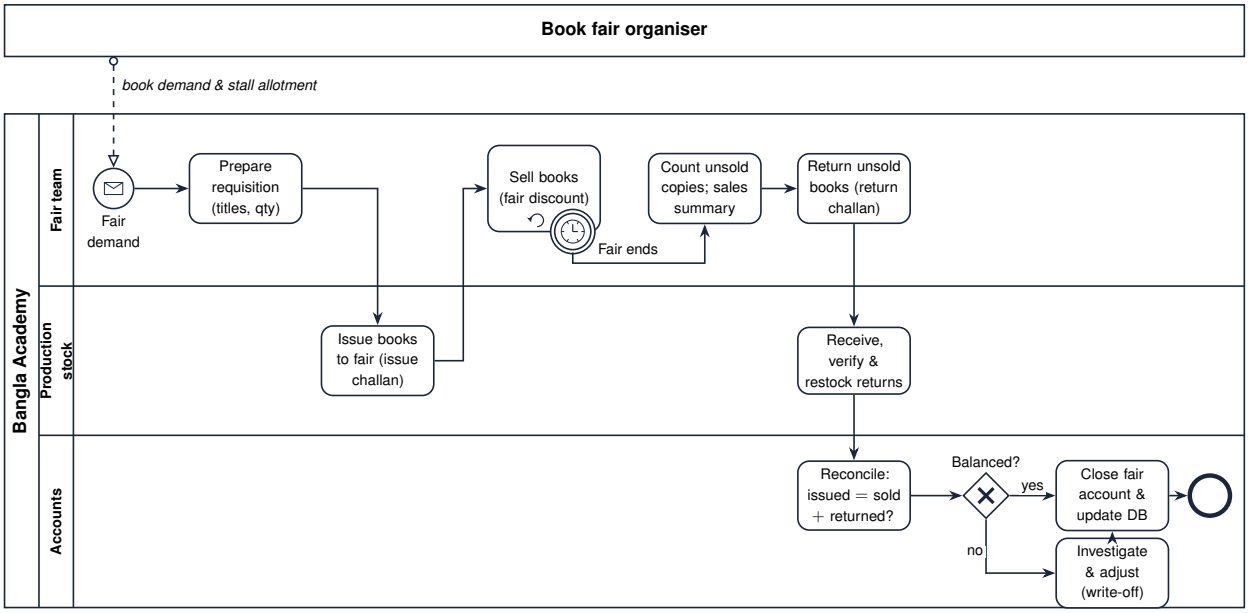


Diagram B – book-fair supply and stock reconciliation



Step 2 – Walking through the model

Online order (Diagram A, top). The process is started by a *message start event* (the Facebook order). The sales desk checks availability against the stock database; the XOR gateway *In stock?* either ends the instance with a *message end event* (regret note) or continues. The price is produced by the task *Apply current discount* – the discount type (member / bulk / none) and the *current* rate are looked up at run time, so changing a discount over time changes data, not the process. After the *payment received* intermediate message event, the production stock picks and dispatches the books and **immediately** posts the quantity decrease to the stock ledger.

Outlet (Diagram A, middle). A *conditional start event* ("outlet stock below reorder level") triggers a replenishment request; the store issues copies against a challan (stock moves from *production stock* to *outlet stock*), the outlet shelves them and sells onsite, applying the member/bulk discount at the counter.

Accounts. Both channels meet at an XOR merge and every sale is posted to the financial ledger. Because each stock movement (*update stock ledger*, *issue to outlet*, *post sale*) writes to the shared *Stock & sales DB*, the management dashboard can show stock, sales and money in real time.

Book fair and reconciliation (Diagram B). A fair's demand (message from the organiser) makes the fair team prepare a requisition; the store issues the books with an *issue challan* (these copies are now "at fair", not in stock). Selling is a *loop task* (one iteration per sale, fair discount applied); it is interrupted by the *timer boundary event Fair ends*. The team counts the unsold copies, returns them with a *return challan*, and the store verifies and restocks them. Accounts then performs the **stock reconciliation**:

$$\begin{aligned} \text{copies issued} &= \text{copies sold} + \text{copies returned}, \\ \text{cash deposited} &= \textstyle\sum_{\text{sales}} (\text{list price} - \text{fair discount}). \end{aligned}$$

If both balance, the fair account is closed; otherwise the discrepancy (lost / damaged copies, cash shortfall) is investigated and an adjustment (write-off or recovery) is posted before closing – so the stock figure in the database always matches the physical stock.

Answer. Pool *Bangla Academy* with lanes Sales desk, Production stock, Book outlet, Fair team and Accounts; external black-box pools Customer and Book fair organiser. Online orders: message start → check stock → XOR(in stock) → apply current discount & invoice → payment event → pick & dispatch → update stock ledger → post in accounts. Outlet: conditional start → request → issue copies → shelve → sell. Fair: demand → requisition → issue → sell (loop, interrupted by timer *fair ends*) → count & return unsold → restock → reconcile (issued = sold + returned) → XOR balanced? → close / investigate & adjust. All movements write to one Stock & sales DB for the real-time dashboard.

NOTE The scenario does not say who runs the fair stall or the outlet; separate lanes *Fair team* and *Book outlet* are assumed. The outlet's walk-in buyers are not drawn as a separate pool to keep Diagram A readable. Two incoming flows into *Close fair account* act as an exclusive merge (only one of the branches is ever taken).

2021–2022 • Q5(a) CSE325 • 07/10/2023 • C325-2122-5a **MARKS 13**

In Bangladesh, every school teacher is appointed based on MPO regulation. According to this regulation, each candidate has to meet certain qualification criteria defined for each post (of a particular subject) of a certain salary grade. The qualifications are stated in terms of age, subject of graduation, and result at different levels. Example of such requirement is shown below. This is for a particular type of institution, e.g., school, college, etc.

Table for Q 5(a) *(printed in Bangla in the paper; English rendering)*

Sl.	Post	Educational qualification	Age	Salary grade
13	Assistant Teacher (Social Science)	(1) Bachelor (Pass)/Honours degree with the relevant subject from a recognised university *and* B.Ed degree/equivalent; **or** (2) Bachelor (Pass)/Honours degree with the relevant subject from a recognised university. Not more than one (01) third division/class/equivalent in the whole academic life is acceptable.	Maximum 35 years (age limit relaxable for indexed holders of the same post)	(1) Grade-10 (16000–38640/-) (2) Grade-11 (12500–30230/-)
14	Assistant Teacher (Mathematics)	(1) Bachelor (Pass)/Honours degree in the science group with Mathematics from a recognised university *and* B.Ed degree/equivalent; **or** (2) Bachelor (Pass)/Honours degree in the science group with Mathematics. Not more than one (01) third division/class/equivalent in the whole academic life is acceptable.	Maximum 35 years (age limit relaxable for indexed holders of the same post)	(1) Grade-10 (16000–38640/-) (2) Grade-11 (12500–30230/-)

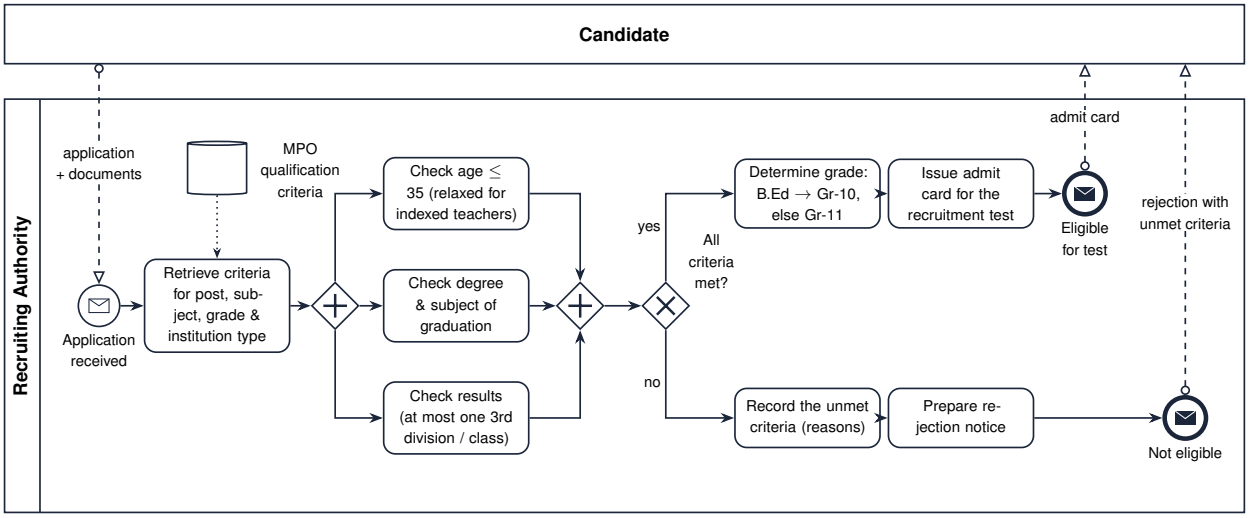
When a candidate applies for a post, his/her qualifications are matched against such requirements. If the qualification criteria satisfy, he/she is eligible to sit for a recruitment test. Design a BPMN diagram to design the scenario discussed above.

SOLUTION

Step 1 – participants, events and decisions

Scenario text	BPMN element
candidate applies for a post	separate participant **Candidate** (black-box pool); the application is a *message flow* that triggers a *message start event* in the authority's pool
criteria defined per post, subject, salary grade and institution type	task *Retrieve criteria*, reading the *data store* "MPO qualification criteria"
qualifications stated in terms of age, subject of graduation, results	three independent checks ⇒ *parallel gateway* (AND split / join)
"if the qualification criteria satisfy"	*exclusive gateway* All criteria met?
B.Ed ⇒ Grade-10, otherwise Grade-11 (from the table)	task *Determine salary grade*
eligible to sit for the recruitment test / not eligible	two *message end events* that inform the candidate

Step 2 – BPMN diagram



Explanation of the design

- **Two pools**: the candidate is a separate participant, so communication with him/her uses *message flows* (dashed); inside the authority's pool only *sequence flows* are used. The candidate pool is drawn as a black box because its internal process is not part of the scenario.
- **Data store**: the criteria table (per institution type, post/subject and grade) is read by *Retrieve criteria*.
- **Parallel gateway** (+): age, subject and result checks are independent and all must be done; the joining + waits for all three before the decision.
- **Exclusive gateway** (×): only if *all* criteria are met does the candidate get a grade (Grade-10 with B.Ed, Grade-11 without) and an admit card; otherwise the unmet criteria are recorded and sent back as the reason.

- **Message end events** (filled envelope) show that the process ends by sending a message to the candidate.

Answer. Candidate (pool) –application–> message start; Retrieve criteria (data store MPO criteria) → AND-split: check age / check subject / check results → AND-join → XOR all met? yes: determine grade (B.Ed Gr-10 else Gr-11) → issue admit card → message end (eligible); no: record unmet criteria → rejection notice → message end (not eligible).

NOTE The table in the paper is printed in Bangla; the English rendering above is used. Parts (b) (ERD) and (c) (mock dashboard) of this question are outside the CSE 213 syllabus and are not included.

2020–2021 • Q1(a) CSE325 • 12/04/2023 • C325-2021-1a **MARKS 14**

You are designing software for a Life Insurance company. The requirements are as follows. If the client has a stable job with salary more than 50,000/= and his/her age is below 40 years, s/he is considered a safe customer and offered a Gold rate for premium payment. If any of these conditions are not made, the premium is 20% higher than the Gold rate. Clients may opt to pay premium installment monthly or yearly and via cheque or mobile banking. If a client fails to make a payment, a penalty of 10% of original premium is applied on the next premium. Before due date of premium, the client receives SMS mentioning the amount to pay. If s/he fails to pay two successive premiums, his/her policy is cancelled.
Draw a BPMN diagram to design the premium calculation scenario discussed above.

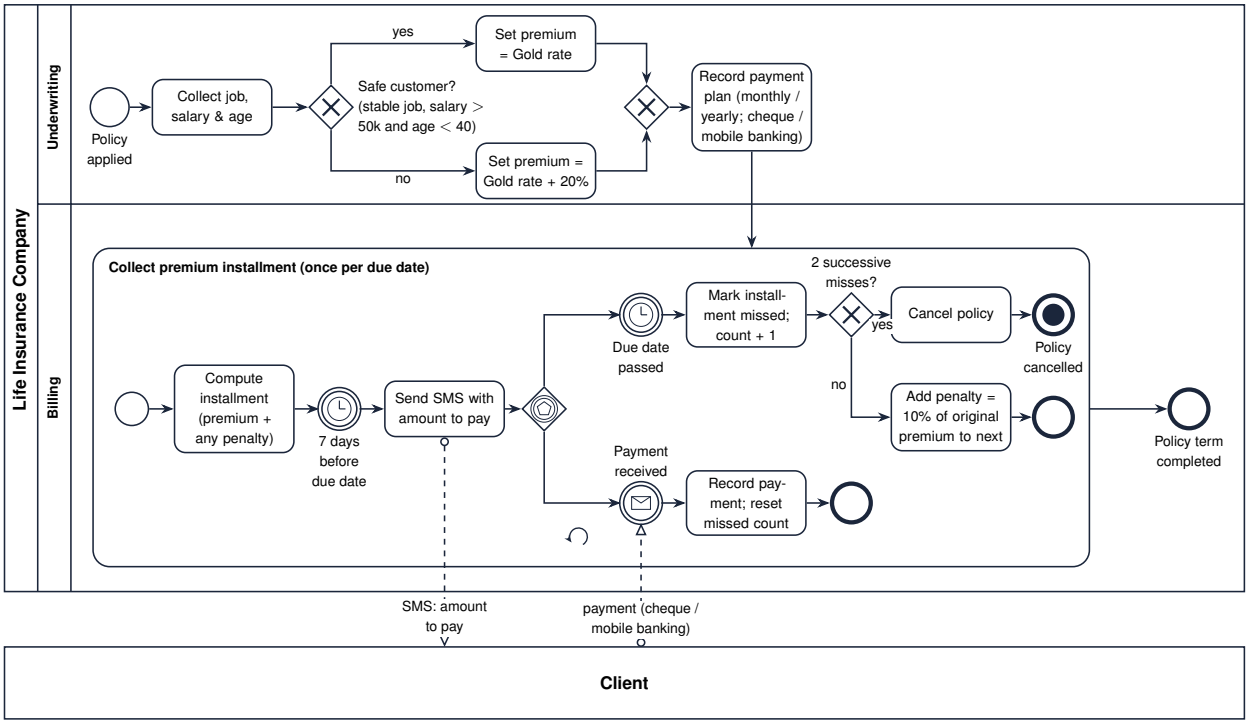
SOLUTION

Step 1 – mapping the requirements to BPMN elements

Requirement	BPMN element
stable job *and* salary $>$ 50,000 *and* age $<$ 40	exclusive gateway *Safe customer?* → Gold rate / Gold rate + 20%
monthly or yearly, cheque or mobile banking	task *Record payment plan* (choice stored as data)
the same steps are repeated for every installment	*loop sub-process Collect premium installment* (loop marker)
SMS before the due date	*timer intermediate event* then task *Send SMS*; message flow to the client
either the client pays or the due date passes	*event-based gateway* followed by a *message* catch event and a *timer* catch event (whichever happens first)
penalty 10% of the original premium on the next premium	task *Add penalty*
two successive misses ⇒ cancellation	exclusive gateway, task *Cancel policy*, *terminate end event* (stops the loop)

Participants: the insurance company is one pool with two lanes, *Underwriting* (computes the premium rate) and *Billing* (collects installments); the client is a separate (black-box) pool.

Step 2 – BPMN diagram



Step 3 – explanation

- **Premium rate (Underwriting lane).** The three conditions are combined in one exclusive gateway: all true → Gold rate; otherwise Gold rate × 1.2. Both paths merge, and the chosen payment frequency (monthly/yearly) and method (cheque/mobile banking) are recorded.
- **Installment cycle (Billing lane).** The expanded *loop sub-process* (loop marker at the bottom) runs once per due date until the policy term is completed. Each iteration computes the amount (premium plus any penalty carried from the last miss), waits on a *timer* until 7 days before the due date and sends the SMS (message flow to the client).
- **Event-based gateway.** After the SMS, the process waits for whichever happens first: the client's payment (*message* catch event) or the due date passing (*timer* catch event).
- **Missed payment.** The miss counter is increased. On the second successive miss the policy is cancelled and a *terminate end event* stops the whole process (the loop does not continue); otherwise a penalty of 10% of the *original* premium is added to the next installment.
- **Payment received** resets the counter, so only *successive* misses lead to cancellation.

Answer. Underwriting: XOR [stable job & salary > 50k & age < 40] → Gold rate / Gold + 20% → record plan. Billing: loop sub-process per installment: compute amount → timer (before due date) → send SMS → event-based gateway: payment received → record & reset count; due date passed → count+1 → XOR 2 successive misses? yes: cancel policy (terminate end); no: add 10% penalty to next premium.

NOTE "7 days before the due date" is an assumed lead time for the SMS (the paper only says "before due date").

2019–2020 • Q5(a) CSE325 • 02/04/2022 • C325-1920-5a **MARKS 15**

If you want to make a tour with a tourism operator, you have to select a package from their website. Each package presents a combination of site visit, hotels, food, activities. Depending on these components and number of days to spend, the price of the packages varies. A package can be customized for a group of tourists with size larger than 10 and price adjustment is made accordingly. The customer has to book a package with partial advance payment. The remaining payment is made prior to the tour to confirm all the components of the tour. If any activity is not available, the refund is made later.
Draw a BPMN diagram to design the portal of the tourism operator.

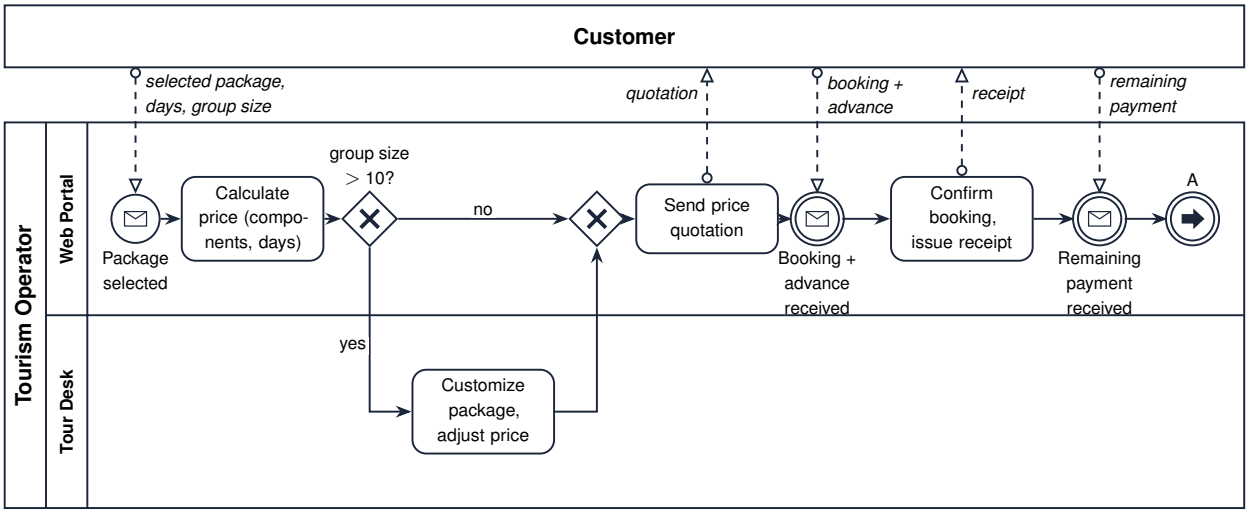
SOLUTION

Step 1 – Participants, lanes and events

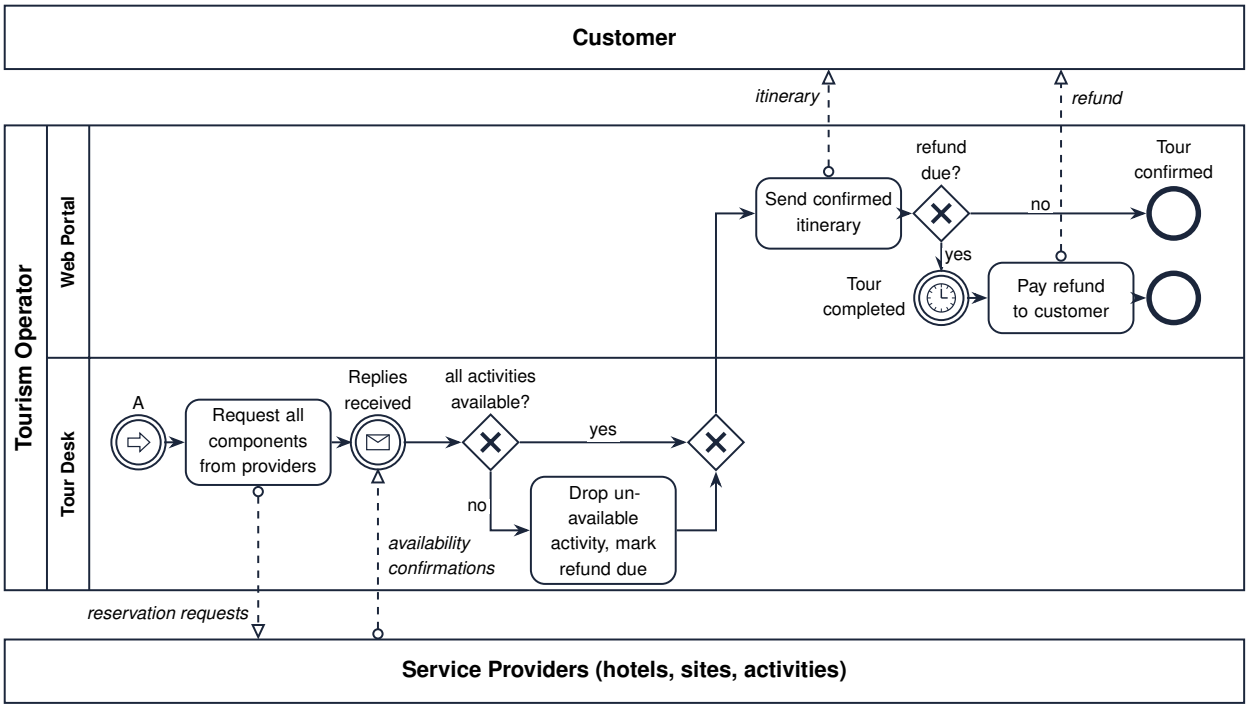
- **Pools**: *Customer* and *Service Providers* (hotels, sites, activity organisers) are independent parties whose internal work we do not know → **black-box pools**; only the *Tourism Operator* process is modelled.
- **Lanes** of the operator: *Web Portal* (automated system: pricing, quotation, payments, confirmations) and *Tour Desk* (staff who customize group packages and contact providers).
- **Events**: the process starts with a *message start event* (package selection from the website); advance and remaining payments are *intermediate message catch events* (the process waits for them); refund "later" is a *timer* event after the tour.
- **Gateways**: XOR "group size > 10?" (customization), XOR "all activities available?" and XOR "refund due?".

The diagram is wide, so it is drawn in two parts joined by a **link event** A (black arrow = throw, white arrow = catch), which is standard BPMN for continuing a diagram.

Step 2 – BPMN diagram, part 1: selection, pricing and booking



Step 3 – BPMN diagram, part 2: confirmation of components and refund



Step 4 – Explanation of the key choices

- **XOR gateway "group size $>$ 10?"**: only one path is taken – either the standard price is quoted or the Tour Desk customizes the package and adjusts the price; the paths join in a merging XOR before the quotation.
- **Intermediate message catch events** model waiting: the token stops until the customer's booking with advance arrives, and again until the remaining payment arrives "prior to the tour".
- **Confirming components** needs the external providers $\rightarrow$ message flows to a black-box pool; the answers arrive at a catch event before the availability decision.
- **"Refund is made later"**: an unavailable activity does not cancel the tour; the activity is dropped and a refund flag recorded. After the itinerary is sent, the XOR "refund due?" leads to a **timer intermediate event** (tour completed) and then *Pay refund*.
- **Link events** only split the drawing; semantically part 2 continues part 1 directly.

> **NOTE** Cancellation when the remaining payment is not received before the deadline is not in the question; it could be added with an *event-based gateway* after *Confirm booking* (message "remaining payment" vs. timer "deadline" $\rightarrow$ *Cancel booking*).

Answer. Black-box pools Customer and Service Providers; operator pool with lanes Web Portal / Tour Desk: message start (package selected) $\rightarrow$ calculate price $\rightarrow$ XOR [group $>$ 10] customize/adjust price $\rightarrow$ send quotation $\rightarrow$ catch booking + advance $\rightarrow$ confirm booking $\rightarrow$ catch remaining payment $\rightarrow$ request components from providers $\rightarrow$ catch replies $\rightarrow$ XOR [activity unavailable] drop + mark refund $\rightarrow$ send itinerary $\rightarrow$ XOR [refund due] timer (tour completed) $\rightarrow$ pay refund $\rightarrow$ end.

2018–2019 • Q2(a) CSE325 • 20/12/2021 • C325-1819-2a **MARKS 20**

The following is a narrative description of the business process of a bank:
There are two types of users in a Bank: Normal user and Foreign User. Both users can create account, deposit money and withdraw money. For money withdrawal there are certain rules. A user can make a withdrawal of maximum 3,00,000 taka per day. At each transaction the highest limit of withdrawal is 1,00,000 taka. A user can make a maximum of 5 withdrawals per day. After every three months, interest is added to the account balance. If an account balance reaches 20,00,000 taka, for every deposit to that account 2% bonus is applied. The foreign user has one extra capability. He can convert his taka to foreign currency.
Design a BPMN diagram to implement the scenario described above.

SOLUTION

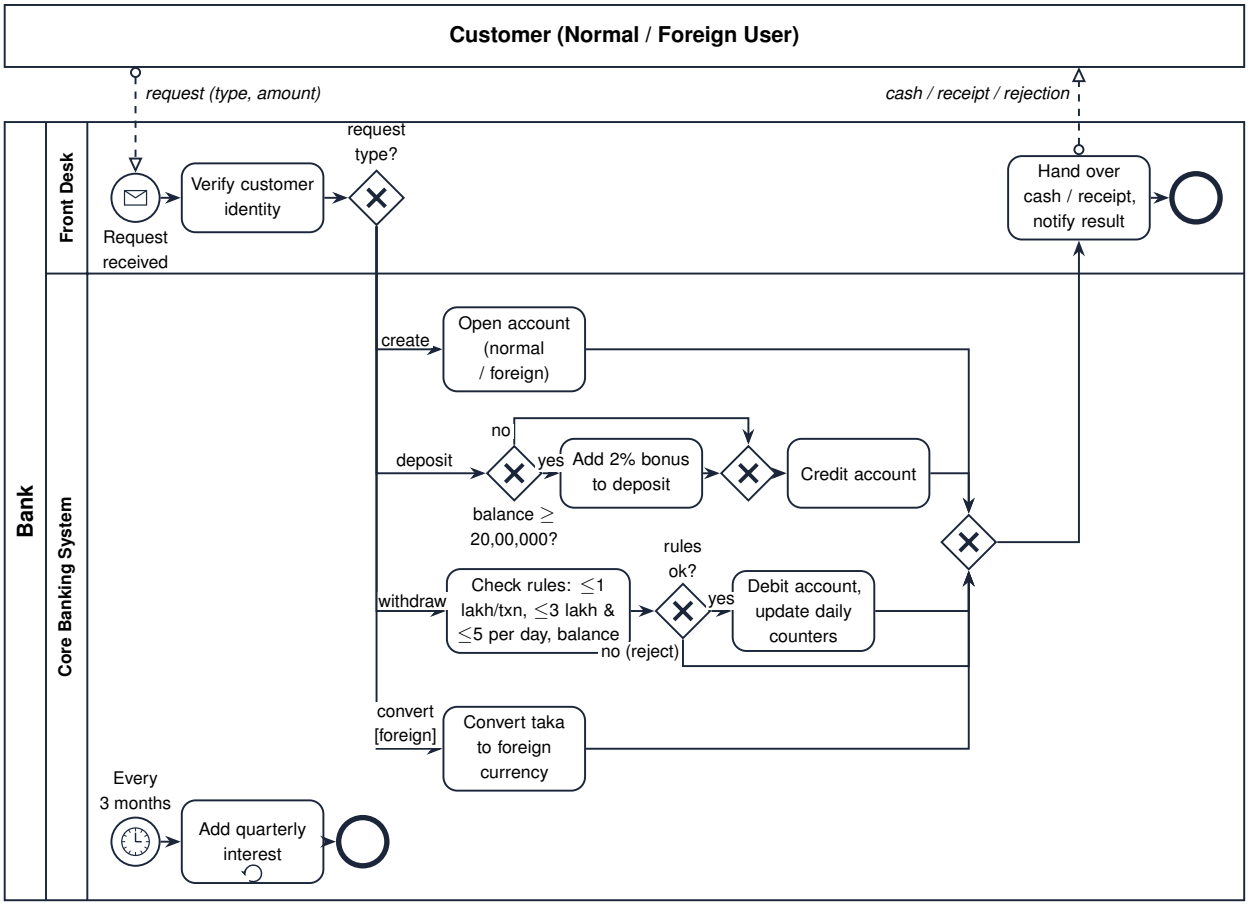
NOTE The same narrative is used in CSE307 2016–2017 (R016.1–R016.3) for a use case diagram, a sequence diagram and a state chart; the BPMN diagram asked here is new.

Step 1 – From the narrative to BPMN elements

Narrative	BPMN element
users (normal / foreign) make requests	black-box pool *Customer*; *message start event* "request received"
create account, deposit, withdraw, convert	XOR gateway on the request type, one branch each
foreign user only can convert	condition on the convert branch: [convert and foreign user]
2% bonus if balance $\geq$ 20,00,000	XOR gateway on the deposit branch + task *Add 2% bonus*
withdrawal limits (1 lakh/txn, 3 lakh/day, 5/day)	task *Check withdrawal rules* + XOR "rules satisfied?"
interest every three months	a second process in the same pool started by a *timer start event*; task with a *loop* marker (for every account)

Lanes: *Front Desk* (teller / ATM interface that talks to the customer) and *Core Banking System* (applies rules and updates balances).

Step 2 – BPMN diagram



Step 3 – Explanation of the key choices

- **One request-driven process**: every customer interaction starts with a *message start event*; the exclusive gateway *request type?* routes the token to exactly one branch. The condition *[foreign]* on the convert branch expresses the foreign user's extra capability.
- **Deposit bonus**: XOR *balance* $\geq$ *20,00,000?* – the yes path adds 2% before crediting, the no path bypasses it; a merging XOR joins them before *Credit account*.
- **Withdrawal rules**: all three limits and the balance are checked in one task; if any is violated the request is rejected (no path), otherwise the account is debited and the daily counters (amount and number of withdrawals) are updated – they are what makes "per day" checkable.
- **All branches merge** (XOR) and the Front Desk returns the result to the customer through a message flow (cash, receipt or rejection notice).
- **Interest** is not triggered by the customer, so it is a separate process in the bank pool with a *timer start event* (every 3 months); the *loop* marker on the task shows it is repeated for every account.

Answer. Customer (black box) → message start in Bank pool → Verify identity → XOR request type: create → Open account | deposit → XOR [bal $\geq$ 20 lakh] Add 2% bonus → Credit account | withdraw → Check rules → XOR [ok] Debit + update counters / [not ok] reject | [foreign] convert → Convert currency; XOR merge → hand over cash/receipt (message to customer) → end. Separate timer-start process: every 3 months → Add interest (loop over accounts) → end.

Practice 1 • BPMN Practice problem (modelled on recent CSE213 papers) MARKS 12 + 3

Out-patient department of a hospital. *Shastho Seba General Hospital*, Mirpur, wants to model the work-flow of its out-patient department (OPD) before automating it. The OPD works as follows. When a patient arrives, the receptionist searches the Hospital Information System (HIS) database with the patient's mobile number to verify whether the patient is already registered. A new patient is registered and the record is stored in the HIS database; an already registered patient skips this step. The patient is then sent to the doctor, who examines the patient and decides whether diagnostic tests are required. If no test is required, the doctor directly writes a prescription. Otherwise the patient goes to the diagnostic lab, where the tests are conducted: a technician first collects the samples; then the pathology unit runs a CBC blood test while, at the same time, the radiology unit takes a chest X-ray; when both results are available the lab prepares a combined test report. The patient then returns to the doctor, who writes the prescription after studying the test report. The consultation is over when the patient receives the prescription.

(i) Draw a BPMN diagram of the OPD process using **one pool with a lane for each participant**. Show the lab work as a *collapsed sub-process*, and show the HIS database, the test report and the prescription. (12)

(ii) Expand the sub-process "Conduct diagnostic tests". (3)

SOLUTION

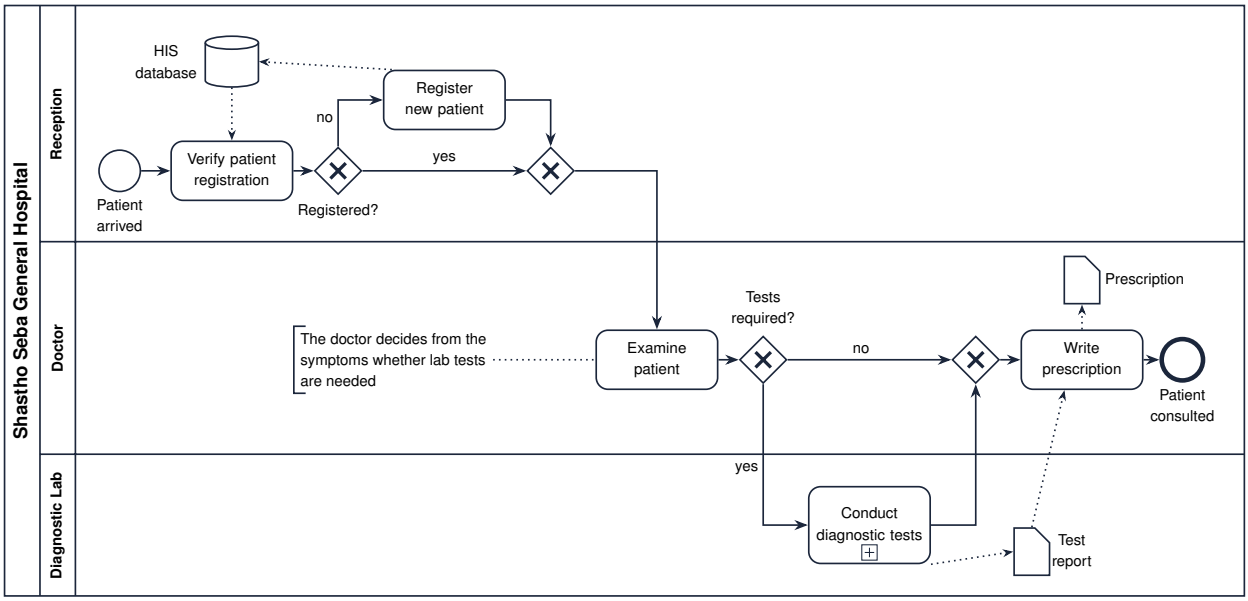
Step 1 – participants: one pool, three lanes

Everything happens inside one organisation, the hospital, so there is **one pool** ("Shastho Seba General Hospital"). The three roles that perform work become **lanes**: *Reception*, *Doctor* and *Diagnostic Lab*. The patient does not perform any modelled task (he/she is the "token" that moves through the process), so the patient is *not* a lane. Because all participants are in the same pool, every hand-over is a *sequence flow* that simply crosses a lane boundary – no message flows are needed.

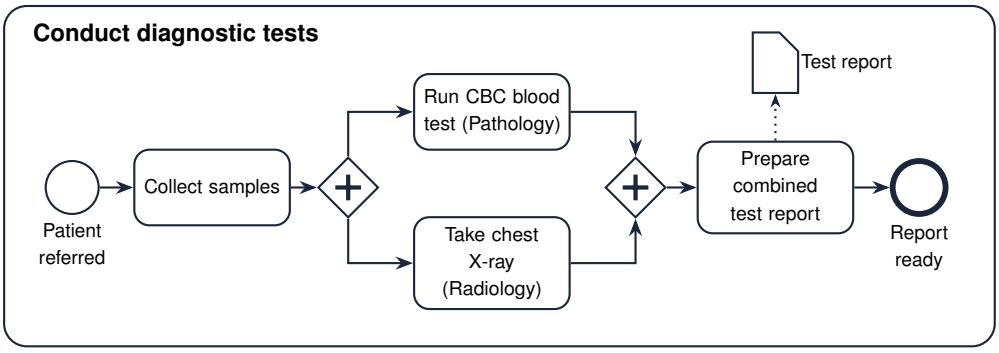
Step 2 – reading the scenario sentence by sentence

Scenario phrase	BPMN element
"When a patient arrives"	none start event *Patient arrived* (Reception)
"searches the HIS database … to verify whether … registered"	task *Verify patient registration* reading the data store *HIS database*
"A new patient is registered …; a registered patient skips this step"	XOR-split *Registered?*, task *Register new patient* (writes to HIS), XOR-join
"the doctor examines … and decides whether tests are required"	task *Examine patient*, XOR-split *Tests required?*
"the tests are conducted" (several steps, by the lab)	collapsed sub-process *Conduct diagnostic tests* (Lab lane); output data object *Test report*
"writes the prescription" (with or without tests)	XOR-join, then task *Write prescription*; output data object *Prescription*
"The consultation is over"	end event *Patient consulted*

(i) BPMN diagram



(ii) Expanded sub-process "Conduct diagnostic tests"



Why these elements?

- **XOR-split + XOR-join around *Register new patient*.** Exactly one of "registered" / "not registered" is true, and both paths continue to the doctor, so the XOR-join simply lets whichever branch arrives pass. The same pattern ("tests" / "no tests") appears in the doctor's lane.
- **Collapsed sub-process.** The lab work is itself a small process (several tasks and a parallel section). Hiding it behind one rounded box with the ⊞ marker keeps the main model readable; part (ii) shows its inside. The sub-process starts with a plain start event and ends with a plain end event – when it ends, the token continues in the parent process (to the XOR-join).
- **AND-split / AND-join inside the sub-process.** "While, at the same time" and "when both results are available" are the textbook signals for parallel branches and for synchronisation.
- **Data store vs. data object.** The HIS database persists *beyond* one patient visit ⇒ data store (cylinder). The test report and the prescription are documents produced *within* this process instance ⇒ data objects. Dotted associations with arrows show input (store → task) and output (task → object/store).
- **Text annotation.** The bracket note explains the decision rule; it does not affect the token flow.

EXAM TIP Common mistakes: (a) making "Patient" a lane – only those who *perform* tasks get lanes; (b) drawing message flows between Reception and Doctor – they are lanes of the *same* pool, so use sequence flows; (c) letting the "no tests" branch go straight into *Write prescription* without an XOR-join (examiners expect explicit merges); (d) naming the event "Arrive" – events are named *noun + past participle* ("Patient arrived").

Answer. One pool, lanes Reception / Doctor / Diagnostic Lab. Patient arrived → Verify patient registration (reads HIS) → XOR[registered?] (no: Register new patient, writes HIS) → XOR-join → Examine patient → XOR[tests required?]: yes → sub-process *Conduct diagnostic tests* (Collect samples → AND{CBC ‖ X-ray} → Prepare report) → XOR-join; no → XOR-join → Write prescription → Patient consulted.

Practice 2 • BPMN Practice problem (modelled on recent CSE213 papers) **MARKS 12 + 3**

Food delivery collaboration. *KhaiDai* is a food-delivery platform operating in Dhaka. Its order handling works as follows.
The process starts when the KhaiDai order system receives an order placed by a customer through the mobile app. The order system immediately sends a payment request to the *bKash* payment gateway and waits until bKash confirms the payment. After the payment is confirmed, the order system forwards the order to the partner restaurant. From then on two things happen in parallel: the order system assigns a nearby delivery rider, and it waits for the restaurant's "food ready" notice. As soon as a rider has been assigned *and* the food is ready, the rider picks up the food parcel from the restaurant and delivers it to the customer. Finally, an e-receipt confirming the delivery is sent to the customer, which completes the process. The internal work of the customer, of bKash and of the restaurant is not of interest to KhaiDai.

(i) Draw a BPMN **collaboration diagram** for the above scenario. Label every message flow. (12)

(ii) Why is the rider modelled as a lane while the restaurant is modelled as a pool? State the rules on sequence and message flows that your diagram obeys. (3)

SOLUTION

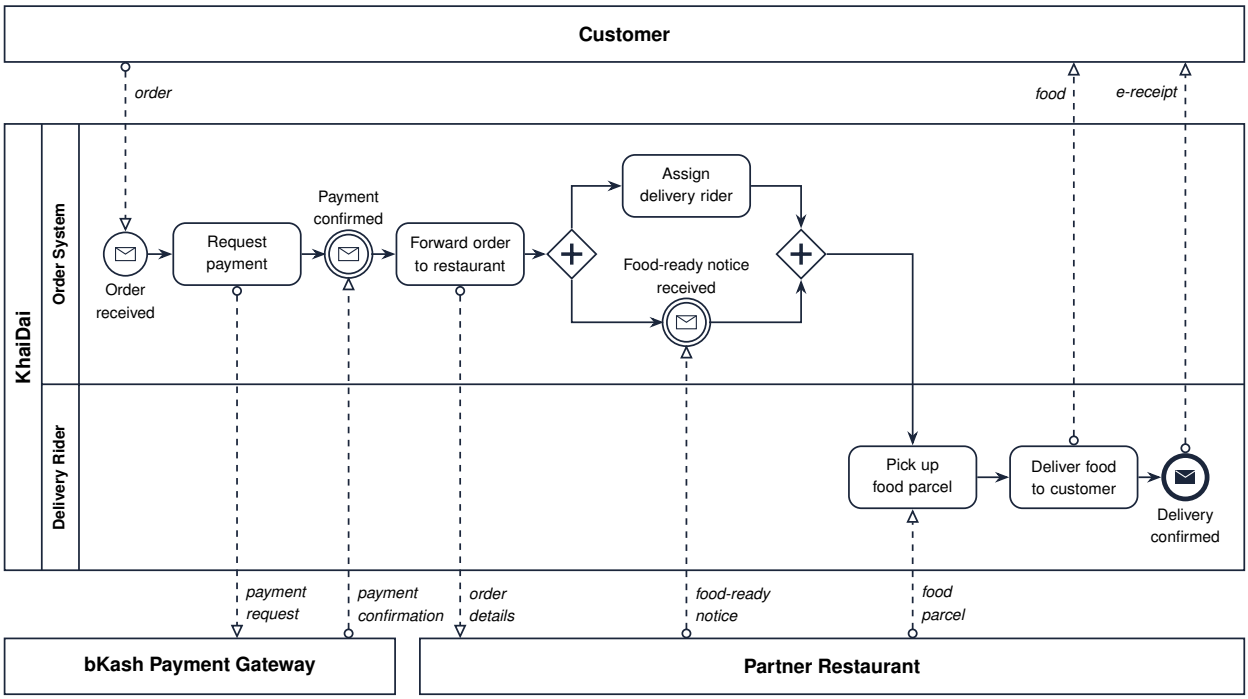
Step 1 – pools and lanes

There are four independent business parties: *Customer*, *KhaiDai*, *bKash* and the *Restaurant*. Each becomes a **pool**. Only KhaiDai's process is required, so the other three are drawn as **black-box pools** (empty rectangles); their messages attach to the pool border, exactly like the black-box customer in the lecture's order-to-cash example. KhaiDai's pool has two **lanes**: *Order System* (a software system – the lecture explicitly allows lanes for systems such as an ERP) and *Delivery Rider*.

Step 2 – messages and events

Scenario phrase	BPMN element
"starts when the order system receives an order"	*message start event Order received* (message flow *order* from Customer)
"sends a payment request to bKash"	task *Request payment* with outgoing message flow
"waits until bKash confirms the payment"	*intermediate message catch event Payment confirmed*
"forwards the order to the partner restaurant"	task *Forward order to restaurant* (message *order details*)
"two things happen in parallel … as soon as … *and* …"	AND-split and AND-join
"waits for the restaurant's food-ready notice"	intermediate message catch event *Food-ready notice received* – an event may sit on a parallel branch
"picks up the food parcel … delivers it"	tasks *Pick up food parcel*, *Deliver food to customer* (Rider lane)
"an e-receipt … is sent, which completes the process"	*message end event Delivery confirmed* (sends the e-receipt)

(i) BPMN collaboration diagram



Explaining the key choices

- **Message start event** (thin circle, white envelope): the process instance is *triggered* by a message from another party, not started spontaneously.
- **Intermediate message catch events** (double circle, white envelope): the token *waits* at *Payment confirmed* until the bKash message arrives; likewise for the food-ready notice. "Waits until/for …" in a scenario is the signal for a catching intermediate event.
- **AND-split / AND-join**: assigning a rider and waiting for the food run concurrently; the AND-join fires only when *both* tokens arrive, which is exactly "as soon as a rider has been assigned

and the food is ready".

- **Message end event** (thick circle, black envelope): ends the process *and* throws the e-receipt message in one symbol (black = throw, white = catch).
- **Black-box pools**: we do not know how bKash or the restaurant work internally, so their message flows end at the pool border.

(ii) Rider lane vs. restaurant pool; flow rules

The rider works *for* KhaiDai and is coordinated by KhaiDai's own process (KhaiDai assigns him/her and decides what he/she does next), so the rider is a *role inside* the KhaiDai pool, i.e. a **lane**. The restaurant is an *independent business party* with its own process and its own control; KhaiDai can only communicate with it, never direct its work, so it is a separate **pool**. Rules respected by the diagram (lecture slide "Pools, lanes and flows: syntactic rules"):

- a sequence flow never crosses a pool boundary – the hand-over from the Order System to the rider is a sequence flow *inside* the KhaiDai pool;
- every interaction with Customer, bKash and the Restaurant is a message flow, and both sequence and message flows may cross lanes;
- no message flow connects two elements of the same pool.

> **EXAM TIP** The classic error in collaboration diagrams is drawing a solid sequence-flow arrow from *Forward order to restaurant* straight into the restaurant (or into a "Cook food" task inside KhaiDai's pool). KhaiDai cannot pass its token to another organisation – it can only send a message and then *wait* for an answer with a catching event.

Answer. Pools: Customer, bKash, Restaurant (black boxes) and KhaiDai (lanes Order System, Delivery Rider). Message start *Order received* → Request payment → catch *Payment confirmed* → Forward order to restaurant → AND{Assign delivery rider ‖ catch *Food-ready notice received*} → Pick up food parcel → Deliver food to customer → message end *Delivery confirmed*. Eight labelled message flows.

Practice 3 • BPMN Practice problem (modelled on recent CSE213 papers) **MARKS 12 + 3**

Course registration through BIIS. At BUET, students register for the courses of a new term through the BUET Integrated Information System (BIIS). The registration process is described below.

The process starts automatically on the date on which the Registrar's office opens the registration period. A student selects the courses for the term in BIIS and submits a registration request; this creates a registration form in the state *submitted*. BIIS then automatically checks the prerequisites and the credit-hour limit. If any constraint is violated, the request is returned to the student, who has to select the courses again. Otherwise the request is forwarded to the student's course adviser, who reviews the selection. If the adviser asks for changes, the student again has to select courses and resubmit (and the new request is again checked by BIIS). When the adviser approves the request, BIIS generates a fee payslip and the student pays the registration fee using the payslip. The process ends when the student has registered for the term.

(i) Draw a BPMN diagram for the registration process. (12)

(ii) Identify the lanes of your diagram and justify the type of each gateway you used. (3)

SOLUTION

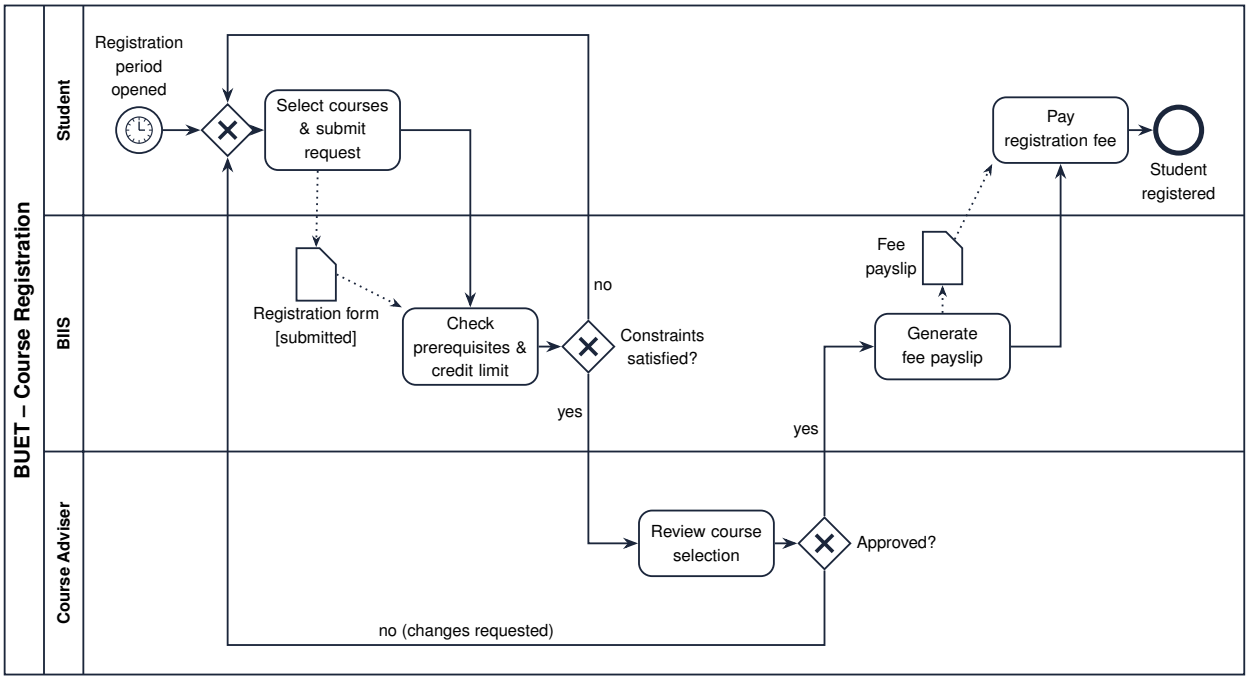
Step 1 – participants

The whole process runs inside BUET, so one pool *BUET – Course Registration* is used. The performers are the *Student*, the information system *BIIS* (an automated performer: a software system can be a lane) and the *Course Adviser* ⇒ three lanes. The Registrar's office performs no task here; it only fixes the date, which is captured by the timer start event.

Step 2 – scenario → BPMN mapping

Scenario phrase	BPMN element
"starts automatically on the date … opens the registration period"	*timer start event Registration period opened*
"selects the courses … and submits a request; this creates a registration form"	task *Select courses & submit request*; output data object *Registration form [submitted]*
"BIIS automatically checks prerequisites and credit-hour limit"	task *Check prerequisites & credit limit* (BIIS lane)
"If any constraint is violated … select the courses again"	XOR-split *Constraints satisfied?* with a *loop back* to an XOR-join placed before *Select courses*
"adviser reviews … asks for changes … select again"	task *Review course selection*, XOR-split *Approved?*, second loop back to the same XOR-join
"BIIS generates a fee payslip"	task *Generate fee payslip*; output data object *Fee payslip*
"the student pays the registration fee"	task *Pay registration fee*
"ends when the student has registered"	end event *Student registered*

(i) BPMN diagram



(ii) Lanes and gateway justification

Lanes: Student, BIIS and Course Adviser – the three performers inside the single BUET pool.

Hand-overs between them (e.g. BIIS → adviser) are sequence flows because they belong to the same organisation.

Gateway	Type	Justification
Merge before *Select courses*	XOR-join	Three flows enter (first entry, "constraints violated", "changes requested"), but in any run exactly *one* of them brings the token. An AND-join would wait for all three and deadlock.
Constraints satisfied?	XOR-split	Exactly one outcome: either all constraints hold (continue) or not (loop back).
Approved?	XOR-split	The adviser either approves or asks for changes – mutually exclusive.

The two backward flows create **loops** (structured repetition): the process repeats *Select courses* . . . until both BIIS and the adviser are satisfied. The **timer start event** is used instead of a none start event because the trigger is a calendar date, not a person's action. The data object is annotated with its *state* (*[submitted]*) exactly like the "purchase order [approved/rejected]" objects in the lecture.

> **EXAM TIP** For a loop always place an **XOR-join** *before* the repeated task and an XOR-split *after* the check; never feed the backward flow straight into the task (implicit merge) and never use a + gateway for the merge. Name the backward branch with its condition ("no", "changes requested").

Answer. Pool BUET with lanes Student / BIIS / Course Adviser. Timer start *Registration period opened* → XOR-join → Select courses & submit request (form [submitted]) → Check prerequisites & credit limit → XOR[satisfied?] no → loop; yes → Review course selection → XOR[approved?] no → loop; yes → Generate fee payslip → Pay registration fee → end *Student registered*.

Practice 4 • BPMN Practice problem (modelled on recent CSE213 papers) **MARKS 14 + 6**

SME loan approval at a bank. *Padma Bank Ltd.* handles its small-and-medium-enterprise (SME) loan applications as follows.
The process starts when a loan application arrives from an applicant. A loan officer verifies the documents attached to the application. If any document is missing, the application is returned to the applicant with a notice and the process ends. Otherwise, a credit analyst assesses the credit risk of the applicant. This assessment is a sub-process: the analyst sends a CIB inquiry to the Credit Information Bureau (CIB) of Bangladesh Bank, waits for the CIB report and, if the applicant is not a defaulter, computes a credit score and prepares a risk report. If the CIB report shows that the applicant is a loan defaulter, the assessment is aborted immediately and the application is rejected. A successfully assessed application goes to the branch manager, who decides on the loan. The manager must take the decision within 3 working days; otherwise the file is escalated to the regional office and the process ends at the branch. If the manager rejects the loan, the loan officer sends a rejection letter to the applicant. If the manager approves it, the loan officer sends the sanction letter and disburses the loan through the bank's core banking system.

(i) Draw a BPMN collaboration diagram for the loan process of Padma Bank. Model the credit-risk assessment as a collapsed sub-process. (14)

(ii) Expand the sub-process "Assess credit risk" and explain how the defaulter case is passed from the sub-process to the parent process. (6)

SOLUTION

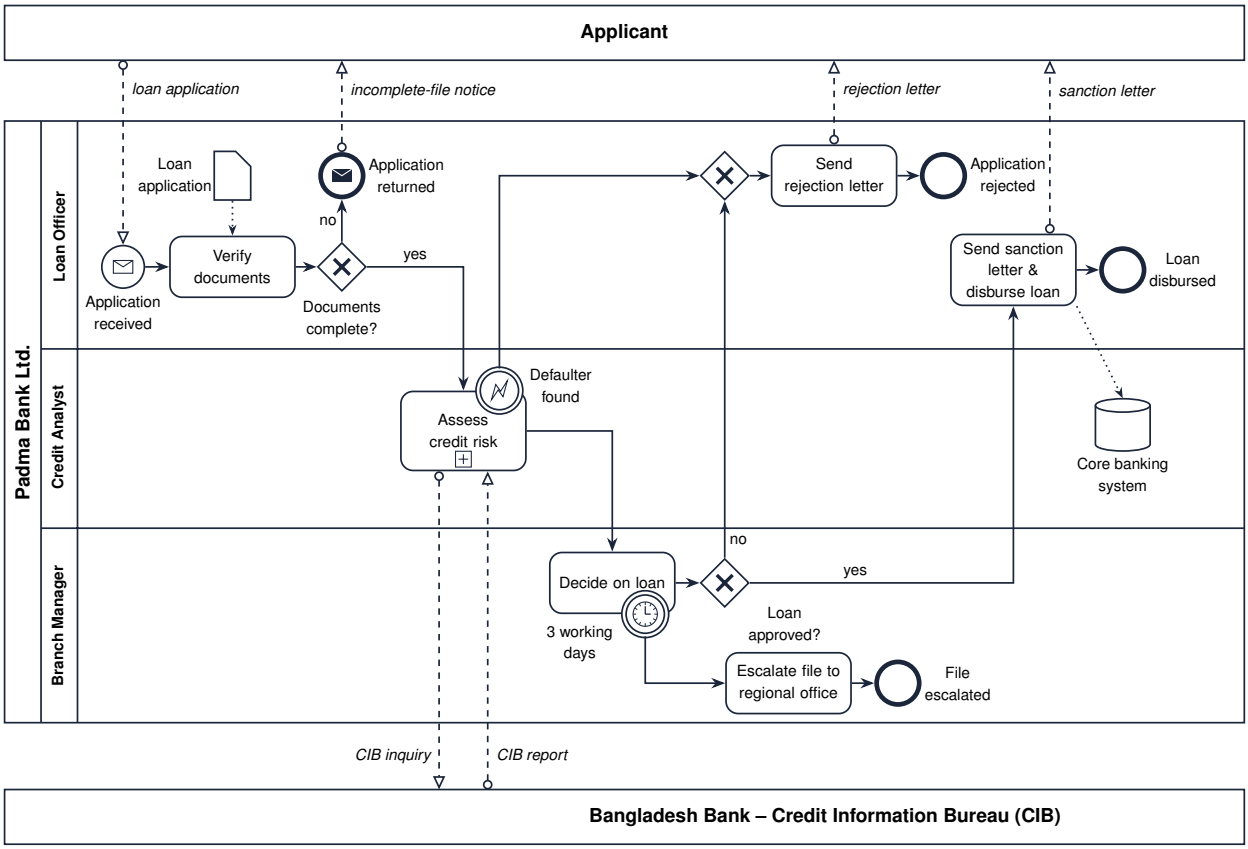
Step 1 – pools and lanes

- **Applicant** and **Bangladesh Bank (CIB)** are independent parties whose internal work is not described ⇒ two *black-box pools*.
- **Padma Bank Ltd.** is the pool we model, with lanes *Loan Officer*, *Credit Analyst* and *Branch Manager*. The regional office only receives the escalated file; it is outside the branch process, so it is not a lane (the escalation is the last task of the branch).

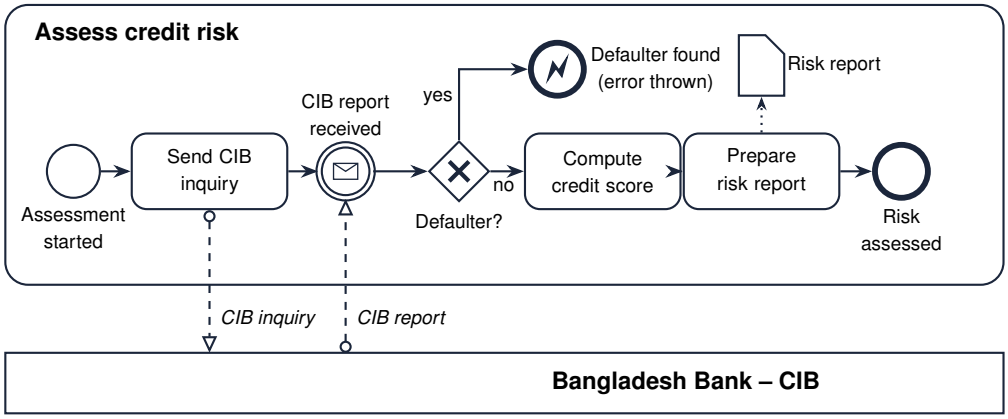
Step 2 – scenario → BPMN mapping

Scenario phrase	BPMN element
"starts when a loan application arrives"	message start event *Application received*; data object *Loan application*
"If any document is missing, … returned … and the process ends"	XOR-split *Documents complete?*; message end event *Application returned* (sends the notice)
"assesses the credit risk … is a sub-process"	collapsed sub-process *Assess credit risk*; message flows *CIB inquiry* / *CIB report* attach to its border
"aborted immediately and the application is rejected"	*error boundary event Defaulter found* on the sub-process (interrupting) → rejection path
"must take the decision within 3 working days; otherwise … escalated"	*timer boundary event 3 working days* on *Decide on loan* → Escalate file to regional office → end *File escalated*
"If the manager rejects … If the manager approves"	XOR-split *Loan approved?*
"application is rejected" (two different causes)	XOR-join, task *Send rejection letter*, end *Application rejected*
"sends the sanction letter and disburses the loan through the core banking system"	task *Send sanction letter & disburse loan*, data store *Core banking system*, end *Loan disbursed*

(i) BPMN collaboration diagram



(ii) Expanded sub-process "Assess credit risk"



How the defaulter case reaches the parent process. Inside the sub-process the "yes" branch ends with an **error end event** *Defaulter found* (thick circle, *black* lightning = *throw*). Throwing the error terminates the sub-process at once (the remaining tasks are not executed). In the parent diagram the same error is **caught** by the **error boundary event** (double circle, *white* lightning) attached to the border of *Assess credit risk*; the token therefore leaves the activity through the boundary event instead of the normal outgoing flow and follows the rejection path. An error boundary event is always *interrupting* – the activity cannot continue after it.

Explaining the other events and gateways

- **Timer boundary event** on *Decide on loan*: it starts counting when the task starts. If the manager

finishes within 3 working days the normal flow continues to *Loan approved?*; if the time runs out first the task is interrupted and the escalation path is taken. "Within n days, otherwise …" is the typical wording for this element.

- **XOR-join before *Send rejection letter***: rejection can come from the error path *or* from the manager – never both for one application – so an XOR-join avoids duplicating the rejection task.
- **Four end events** for four different outcomes (returned, rejected, escalated, disbursed); the lecture requires each outcome to be recorded by its own end event.
- **Message flows on a collapsed sub-process**: the CIB messages attach to the border of *Assess credit risk*; in the expanded view (ii) they attach to the actual send task and catch event.

EXAM TIP Do not model "within 3 working days" as an XOR-split with the condition "decided in time?" – a gateway can only test data that already exists; it cannot wait. A deadline on an activity is a *timer boundary event*; a wait *between* activities is an *intermediate timer event*.

Answer. Pools Applicant and CIB (black boxes), Padma Bank (Loan Officer / Credit Analyst / Branch Manager). Message start → Verify documents → XOR: incomplete → message end *Application returned*; complete → sub-process *Assess credit risk* (CIB inquiry/report; error boundary *Defaulter found* → rejection) → Decide on loan (timer boundary 3 days → Escalate → end) → XOR[approved?]: no → XOR-join → Send rejection letter → end; yes → Send sanction letter & disburse loan (core banking store) → end. Inside the sub-process an error end event throws *Defaulter found*.

Practice 5 • BPMN Practice problem (modelled on recent CSE213 papers) MARKS 15 + 5

Ride-sharing trip. *Chalo* is a ride-sharing app (similar to Pathao or Uber) that operates in Dhaka and Chattogram. Its trip-handling process is as follows.
The process starts when a passenger's ride request reaches the Chalo dispatch system. The dispatch system finds the nearest available driver and sends the ride request to that driver's app. Then it waits for whichever of the following happens first: (a) the driver accepts the request, (b) the driver declines it, or (c) 15 seconds pass without any response. In cases (b) and (c) the dispatch system searches for the next nearest driver and the request is sent again. Once a driver accepts, the passenger is notified with the driver's details. The dispatch system then waits until the driver's app reports that the trip has been completed. After that, the billing system calculates the fare and performs the closing activities: it *always* sends an e-receipt to the passenger; *in addition*, if the passenger has rated the driver, the driver's rating is updated, and if a promo code was used, the discount is recorded in the promotions ledger. The trip is closed only after all applicable closing activities have finished.

(i) Draw a BPMN collaboration diagram for this process (show the passenger and the driver as black-box pools). (15)

(ii) Justify the use of an event-based gateway and of an inclusive (OR) gateway in your model. Why would XOR gateways be wrong in these two places? (5)

SOLUTION

Step 1 – pools and lanes

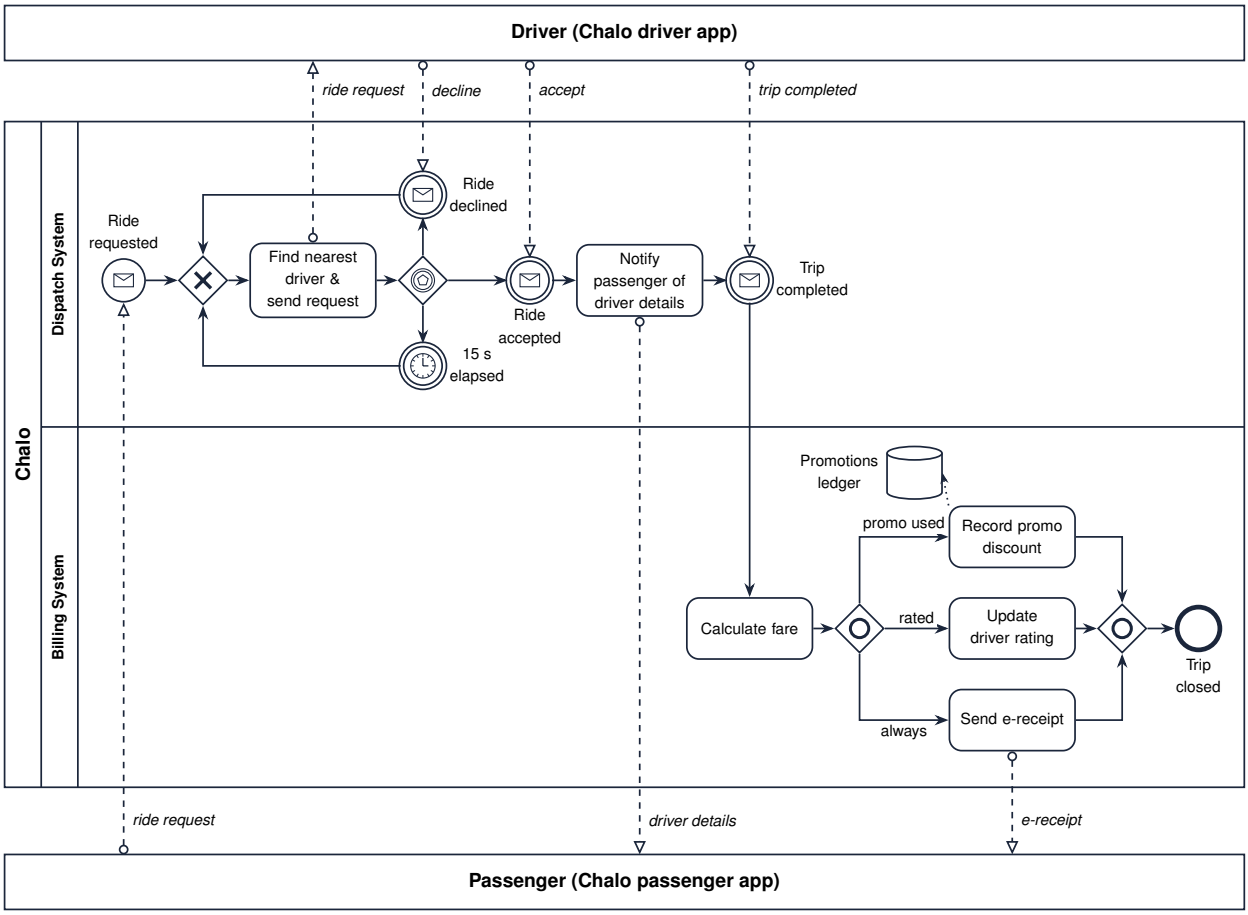
The passenger and the driver are separate parties that interact with Chalo through their apps ⇒ two black-box pools (the question asks for this). Chalo's pool is split into two system lanes:

Dispatch System (matching drivers) and *Billing System* (fare and closing activities).

Step 2 – scenario → BPMN mapping

Scenario phrase	BPMN element
"starts when a passenger's ride request reaches …"	message start event *Ride requested*
"finds the nearest available driver and sends the ride request"	task *Find nearest driver & send request* (message *ride request* to Driver)
"waits for whichever … happens first"	*event-based gateway* followed by three catching events
(a) accepts / (b) declines / (c) 15 seconds pass	message catch *Ride accepted*, message catch *Ride declined*, timer catch *15 s elapsed*
"searches for the next nearest driver and the request is sent again"	loop back to an XOR-join before the *Find …* task
"the passenger is notified with the driver's details"	task *Notify passenger of driver details*
"waits until the driver's app reports … completed"	message catch *Trip completed*
"always …; in addition, if …, and if …"	*inclusive (OR) split* with three conditional branches
"closed only after all applicable activities have finished"	*OR-join*, end event *Trip closed*
"recorded in the promotions ledger"	data store *Promotions ledger*

(i) BPMN collaboration diagram



(ii) Justifying the two special gateways

Event-based gateway. After the request is sent, the *process* has no data on which to decide – the choice is made by *what happens next in the outside world*: an *accept* message, a *decline* message or the expiry of 15 seconds. An event-based gateway is followed only by catching events (or receive tasks); the token waits at the gateway, the *first* event that occurs wins and the other alternatives are discarded (a race). An XOR gateway would be wrong because an XOR-split evaluates conditions *immediately* on existing data and cannot wait; "driver accepted?" is unknown at the moment the token reaches the gateway. (The timer also makes the loop terminate in finite time for each driver.)
Inclusive (OR) gateway. The closing activities are *not* mutually exclusive: the e-receipt is always sent, and zero, one or both of the other two activities are added depending on the data. An OR-split takes *one or more* branches whose conditions are true – exactly the lecture's definition (and the same pattern as the "Always" branch in the hardware-retailer example). The matching **OR-join** waits for all *active* branches only, so the trip is closed after every applicable activity has finished. An XOR-split would run only one activity (e.g. the receipt *or* the rating update), and an XOR-join after a multi-branch split would let the process continue more than once; an AND-split would wrongly force all three activities every time.

EXAM TIP Two checks examiners like: (1) the elements directly after an event-based gateway must be *catching* events – never tasks or another gateway; (2) an OR-split must be closed by an OR-join (lecture: "OR-join proceeds when all *active* incoming branches have completed"). Also notice that the three dispatch outcomes are three different catching events, so each gets its own name (noun + past participle).

Answer. Pools Passenger, Driver (black boxes) and Chalo (Dispatch / Billing). Message start *Ride requested* → XOR-join → Find nearest driver & send request → event-based gateway → {*Ride declined* | *15 s elapsed*} → loop back; *Ride accepted* → Notify passenger of driver details → catch *Trip completed* → Calculate fare → OR-split {always: Send e-receipt; rated: Update driver rating; promo used: Record promo discount (promotions ledger)} → OR-join → end *Trip closed*.

Coverage of the five practice problems

Problem	BPMN elements exercised
1 Hospital OPD	one pool + 3 lanes, none start/end, XOR split/join, collapsed + expanded sub-process, AND split/join, data object, data store, text annotation
2 Food delivery	collaboration with 3 black-box pools, message start, intermediate message catch (on a parallel branch), AND split/join, message end, labelled message flows
3 BIIS registration	system lane, timer start, XOR-join loops (two backward flows), data objects with state
4 Bank loan	message start/end, collapsed sub-process with message flows, error end + error boundary event, timer boundary event, multi-input XOR-join, data store, four end events
5 Ride sharing	event-based gateway with message and timer catch events, loop, intermediate message catch, inclusive (OR) split/join, data store

PART II

Out of Syllabus

Questions from past CSE213/CSE307/CSE325 papers whose central concept is not one of the 14 syllabus topics or is not covered by the lecture slides. This includes design patterns that were not taught, project management, security, architecture, DFD and activity modelling, UI and quality management. Each one is still fully solved, using standard textbook material.

X1

Design Patterns Beyond the Slides

OUT OF SYLLABUS

2023–2024 • Q2(b) CSE213 • 09/02/2025 • D005 **MARKS 8**

Read the following program descriptions carefully and draw the appropriate UML diagram using appropriate design pattern to represent them.
The *HelpHandler* class defines the interface for handling help requests. It maintains a help topic (which is empty by default) and keeps a reference to its successor on the chain of help handlers. The key operation is *HandleHelp*, which subclasses override. *HasHelp* is a convenience operation for checking whether there is an associated help topic. All widgets (like *Dialog*, *Button*) are subclasses of the *Widget* abstract class. *Widget* is a subclass of *HelpHandler*, since all user interface elements can have help associated with them. *Button's* version of *HandleHelp* first tests to see if there is a help topic for buttons. If the developer hasn't defined one, then the request gets forwarded to the successor using the operation *HandleHelp* in *HelpHandler*. If there is a help topic, then the button displays it and the search ends. *Dialog* implements a similar scheme, except that its successor is not a widget but any help handler. This successor is an instance of *Application*. The Application is not a widget, so *Application* is subclassed directly from *HelpHandler*. When a help request propagates to this level, the application can supply information in general or it can offer a list of different help topics.

SOLUTION

> **NOTE** Part (a) of this question (the StockMarket notification system) is in-syllabus and is solved in the Behavioural Design Patterns chapter. This part needs Chain of Responsibility, which is not among the patterns taught in CSE 213.

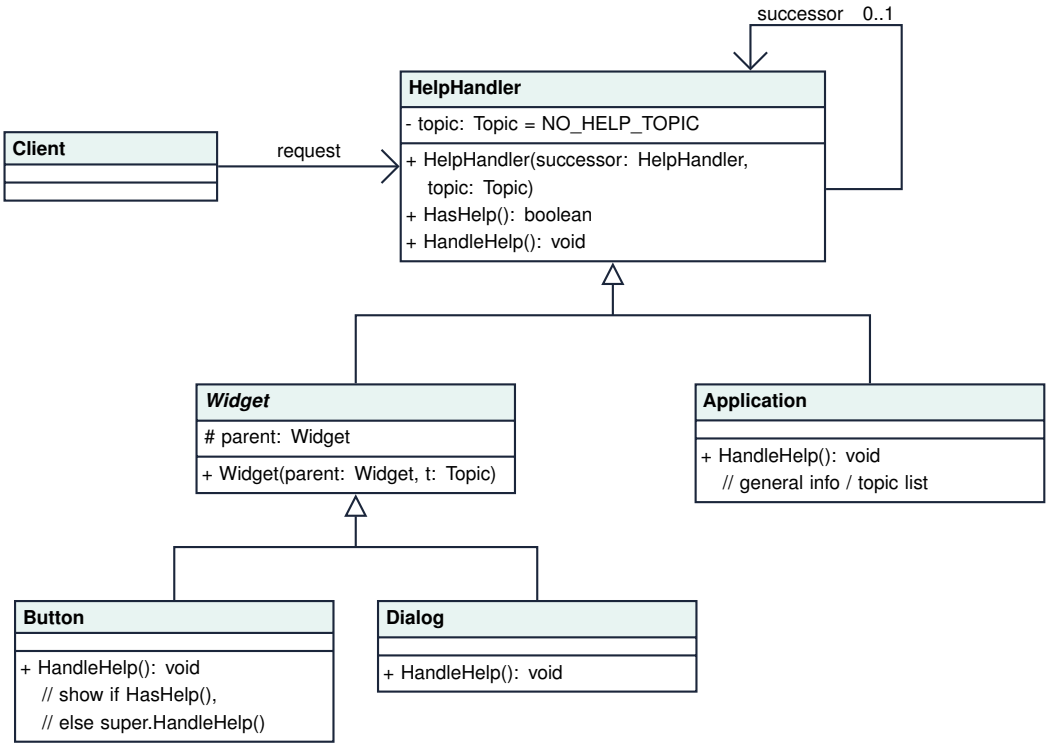
Identifying the pattern

The description has the hallmarks of the **Chain of Responsibility** pattern (GoF): *"Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it."*

- every handler keeps "a reference to its *successor* on the chain";
- a handler either handles the request ("displays it and the search ends") or *forwards* it to the successor (the default `HandleHelp()` in `HelpHandler`);
- the sender (the UI element that was clicked) does not know which object will finally answer.

Roles: **Handler** = `HelpHandler`; **Concrete handlers** = `Button`, `Dialog`, `Application`; `Widget` is an abstract intermediate class for all UI elements.

UML class diagram



The self-association `successor` (0..1) on `HelpHandler` is what links the handlers into a chain. A typical object chain at run time:



Code sketch (Java)

```
class Topic { final String text; Topic(String t) { text = t; } }

class HelpHandler {                                    // Handler
    static final Topic NO_HELP_TOPIC = null;
    private final HelpHandler successor;
    private final Topic topic;
    HelpHandler(HelpHandler successor, Topic topic) { this.successor = successor; this.topic =
    topic; }
    boolean hasHelp() { return topic != NO_HELP_TOPIC; }
    void handleHelp() { if (successor != null) successor.handleHelp(); }   // forward
    protected void show() { System.out.println("Help: " + topic.text); }
}

abstract class Widget extends HelpHandler {
    protected Widget(Widget parent, Topic t) { super(parent, t); }   // successor = parent
    widget
}

class Button extends Widget {
    Button(Widget parent, Topic t) { super(parent, t); }
    void handleHelp() { if (hasHelp()) show(); else super.handleHelp(); }
}

class Dialog extends Widget {
    private final HelpHandler handler;                // successor may be ANY help handler
```

```
    Dialog(HelpHandler h, Topic t) { super(null, t); handler = h; }
    void handleHelp() { if (hasHelp()) show(); else if (handler != null) handler.handleHelp(); }
}

class Application extends HelpHandler {
    Application(Topic t) { super(null, t); }
    void handleHelp() { System.out.println("General help / list of help topics"); }
}
// Application app = new Application(null);
// Dialog dlg = new Dialog(app, new Topic("Printing options"));
// Button btn = new Button(dlg, HelpHandler.NO_HELP_TOPIC);
// btn.handleHelp();   // button has no topic -> dialog shows "Printing options"
```

Answer. Chain of Responsibility: `HelpHandler` (handler with a `successor` self-association, `HasHelp()`, `HandleHelp()`) is specialised by abstract `Widget` (with subclasses `Button`, `Dialog`) and by `Application`; a help request travels Button → Dialog → Application until some handler has a topic to show.

2023–2024 • Q5(b) CSE213 • 09/02/2025 • D007.2 **MARKS 12**

Illustrate how a single class can serve as both an Adapter and a Facade with an example scenario. Design a class diagram to represent the scenario and analyze the functionality of the class in fulfilling the dual role.

SOLUTION

NOTE Out of syllabus. The Adapter half is taught, but the question cannot be answered without the **Facade** pattern, which is not in the CSE 213 slides. A short definition of Facade is given below so that the solution is self-contained.

Step 1 – The two intents

- **Adapter** (taught): *convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.* Its goal is **compatibility**.
- **Facade** (not taught): *provide a unified, higher-level interface to a set of interfaces in a subsystem, which makes the subsystem easier to use.* Its goal is **simplification**. The client calls one method instead of coordinating many subsystem objects.

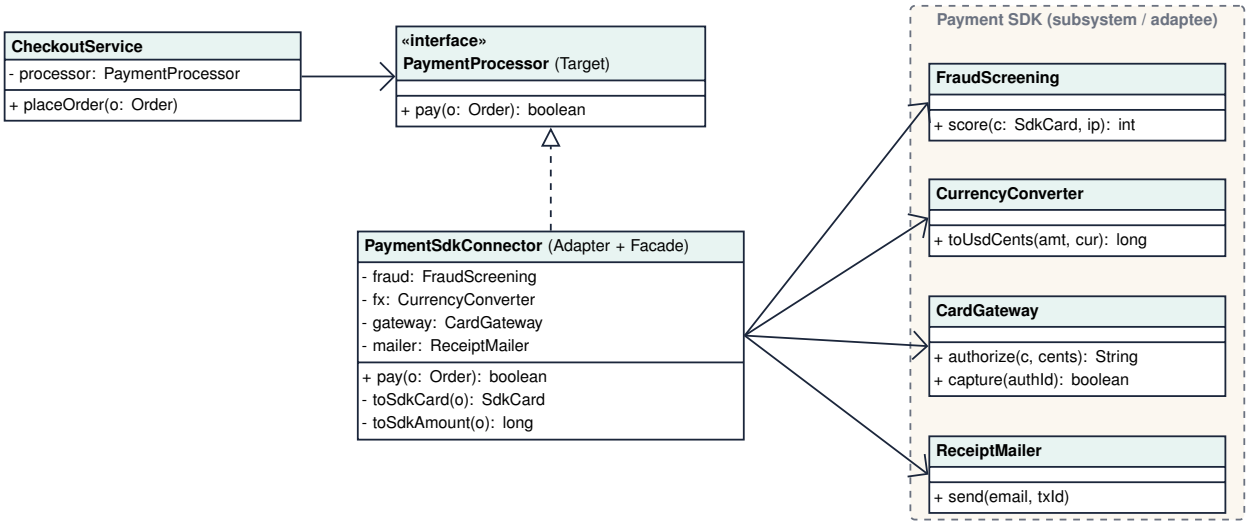
One class plays both roles when (1) it implements an interface that the client *already expects* (Adapter), and (2) behind that interface it *coordinates several subsystem classes* through a single simple call (Facade).

Step 2 – Example scenario

An online shop's checkout module was written against its own interface `PaymentProcessor.pay(Order): boolean`. The company now has to use a third-party payment SDK. The SDK has no `pay(Order)`. Paying through it needs four separate classes with their own data types: `FraudScreening.score(...)`, `CurrencyConverter.toUsdCents(...)`, `CardGateway.authorize(...)/capture(...)`, and `ReceiptMailer.send(...)`.
The class `PaymentSdkConnector`:

- **implements** `PaymentProcessor`, so the checkout code works unchanged → **Adapter** (Target = `PaymentProcessor`, Adaptee = the SDK);
- **hides** the four-step SDK workflow behind one call → **Facade** (subsystem = SDK classes).

Step 3 – Class diagram



Step 4 – Code sketch

```
interface PaymentProcessor { boolean pay(Order o); }     // Target (used by checkout)

class PaymentSdkConnector implements PaymentProcessor {  // Adapter + Facade
    private final FraudScreening fraud = new FraudScreening();
    private final CurrencyConverter fx = new CurrencyConverter();
    private final CardGateway gateway = new CardGateway();
    private final ReceiptMailer mailer = new ReceiptMailer();

    @Override
    public boolean pay(Order o) {
        // ADAPTER role: translate our data types into the SDK's types
        SdkCard card = toSdkCard(o);
        long cents   = fx.toUsdCents(o.getAmount(), o.getCurrency());
        // FACADE role: coordinate the subsystem in the right order
        if (fraud.score(card, o.getIp()) > 70) return false;
        String auth = gateway.authorize(card, cents);
        if (auth == null || !gateway.capture(auth)) return false;
        mailer.send(o.getEmail(), auth);
        return true;                        // ADAPTER role: SDK result -> boolean
    }
    private SdkCard toSdkCard(Order o) {
        return new SdkCard(o.getCardNo(), o.getExpiry(), o.getCvv());
    }
}

class CheckoutService {                     // client: unchanged
    private final PaymentProcessor processor;
    CheckoutService(PaymentProcessor p) { processor = p; }
    void placeOrder(Order o) {
        System.out.println(processor.pay(o) ? "Order confirmed" : "Payment failed");
    }
}
// new CheckoutService(new PaymentSdkConnector()).placeOrder(order);
```

Step 5 – Analysis of the dual role

	As an Adapter	As a Facade
Problem solved	Interface mismatch: the checkout expects `pay(Order)`, and the SDK speaks `SdkCard`, cents and auth IDs.	Complexity: four SDK classes must be called in the right order with error handling.
What the class does	Implements the existing Target interface. Converts `Order` to `SdkCard` and cents, and the SDK result to `boolean`.	Offers one coarse-grained operation and hides the sequence score → convert → authorise → capture → mail.
Who benefits	Existing client code, which needs no change (OCP).	Every client, which no longer needs to know the SDK (loose coupling, least knowledge).
Number of wrapped objects	usually one adaptee	many subsystem objects

- **Why one class can do both:** both patterns are *wrappers*. The interface the class exposes is dictated by the client (Adapter), and that interface is simpler than the subsystem behind it (Facade).
- **Benefits:** the SDK can be replaced by another provider by writing a new connector. The client and the Target interface do not change.
- **Caution:** combining the roles can grow into a "god class" that breaks the Single Responsibility Principle. If the SDK workflow becomes large, split it into a pure `PaymentFacade` (simplification) and a thin `PaymentFacadeAdapter` (interface conversion).

Answer. `PaymentSdkConnector` implements the client's existing `PaymentProcessor` interface and translates the data types (the Adapter role). Behind its single `pay()` method it orchestrates the fraud, currency, gateway and mailer classes of the SDK (the Facade role). The Adapter role provides compatibility and the Facade role provides simplicity.

2012–2013 • Q1(c) CSE307 • 17/05/2014 • D032 MARKS 4 + 10 = 14

Suppose you have to develop an application that takes a string of the format **A operator1 B operator2 C** as input and provides the result of the corresponding operation. Here, A, B and C are number string and the operator1 and operator2 are any one of $+$, $-$, $*$, / operators. Which design pattern will you chose to design a solution to the problem and why? Draw the appropriate class diagram for your proposed solution approach.

SOLUTION

Choice of pattern and why (4)

The input is a sentence of a tiny *language* with a simple grammar:

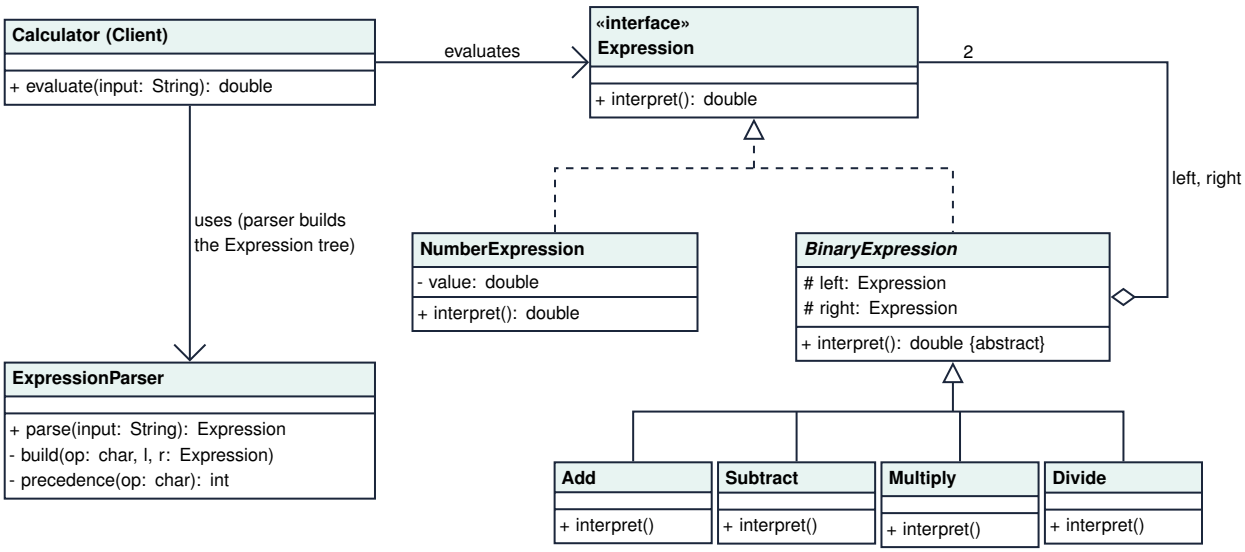
$$expr ::= number\ op\ number\ op\ number, \qquad op ::= + \mid - \mid * \mid /$$

The **Interpreter** pattern (GoF) is designed for exactly this: *"Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language."*

- Each grammar rule becomes a class: a *terminal* expression (`NumberExpression`) and *non-terminal* expressions (`Add`, `Subtract`, `Multiply`, `Divide`), all implementing `Expression.interpret()`.
- A parser turns "A op1 B op2 C" into an abstract syntax tree that respects precedence ($*$, / bind tighter than $+$, $-$): if op2 has higher precedence than op1 the tree is `op1(A, op2(B, C))`, otherwise `op2(op1(A, B), C)`.

- Calling `interpret()` on the root evaluates the tree recursively. New operators (`%`, `^`) are new classes; the evaluation code is not modified (OCP).

Class diagram (10)



Note that the abstract-syntax tree itself is a *Composite* (`BinaryExpression` contains two `Expressions`), which is how Interpreter is normally structured.

Supporting code (Java)

```
interface Expression { double interpret(); }

class NumberExpression implements Expression {          // terminal
    private final double value;
    NumberExpression(String s) { value = Double.parseDouble(s); }
    public double interpret() { return value; }
}

abstract class BinaryExpression implements Expression { // non-terminal
    protected final Expression left, right;
    BinaryExpression(Expression l, Expression r) { left = l; right = r; }
}
class Add      extends BinaryExpression { Add(Expression l, Expression r) { super(l, r); }
    public double interpret() { return left.interpret() + right.interpret(); } }
class Subtract extends BinaryExpression { Subtract(Expression l, Expression r) { super(l, r); }
    public double interpret() { return left.interpret() - right.interpret(); } }
class Multiply extends BinaryExpression { Multiply(Expression l, Expression r) { super(l, r); }
    public double interpret() { return left.interpret() * right.interpret(); } }
class Divide   extends BinaryExpression { Divide(Expression l, Expression r) { super(l, r); }
    public double interpret() { return left.interpret() / right.interpret(); } }

class ExpressionParser {
    Expression parse(String input) {                    // "A op1 B op2 C"
        String[] t = input.trim().split("\\s+");
        Expression a = new NumberExpression(t[0]), b = new NumberExpression(t[2]),
                   c = new NumberExpression(t[4]);
        char op1 = t[1].charAt(0), op2 = t[3].charAt(0);
        if (precedence(op2) > precedence(op1))
            return build(op1, a, build(op2, b, c));    // A + (B * C)
```

```
        return build(op2, build(op1, a, b), c);          // (A op1 B) op2 C
    }
    private int precedence(char op) { return (op == '*' || op == '/') ? 2 : 1; }
    private Expression build(char op, Expression l, Expression r) {
        switch (op) {
            case '+': return new Add(l, r);
            case '-': return new Subtract(l, r);
            case '*': return new Multiply(l, r);
            case '/': return new Divide(l, r);
            default:  throw new IllegalArgumentException("bad operator " + op);
        }
    }
}
// new ExpressionParser().parse("8 + 6 / 2").interpret()  ->  11.0
```

NOTE Interpreter is not among the CSE 213 behavioural patterns. A solution using only taught patterns is also acceptable for this open question: encapsulate each operator as a *Strategy* (`Operation.apply(x, y)` with `Addition`, `Subtraction`, …) selected from a map by the operator symbol, and let a calculator context apply the two operations in precedence order.

Answer. Interpreter pattern: the grammar "number op number op number" is represented by an `Expression` hierarchy (terminal `NumberExpression`; non-terminals `Add`, `Subtract`, `Multiply`, `Divide` under `BinaryExpression`); a parser builds the precedence-correct expression tree and `interpret()` evaluates it.

X2

Requirements & Information Systems

OUT OF SYLLABUS

2021–2022 • Q8(c) CSE325 • 07/10/2023 • C325-2122-8c MARKS 10

Discuss Interactor, Indirect and Domain viewpoints for the requirement analysis of a Stock Price Prediction system. Mention three non-functional requirements for such a system.

SOLUTION

Viewpoint-oriented requirement analysis

Different stakeholders see a system differently; collecting requirements from several **viewpoints** makes the elicitation complete and exposes conflicts. Three classes of viewpoint (Sommerville, VORD):

- **Interactor viewpoints** — people or other systems that *interact directly* with the system; they give detailed functional requirements and interface requirements.
- **Indirect viewpoints** — stakeholders who do *not use* the system themselves but influence its requirements (management, regulators, sponsors, security/compliance staff); they give high-level organisational requirements and constraints.
- **Domain viewpoints** — characteristics and constraints of the *application domain* (standards, laws, domain rules) that the system must respect.

Applied to a Stock Price Prediction system

Viewpoint	Who / what	Typical requirements obtained
Interactor	Investors/traders, financial analysts, system administrator; the stock-exchange market-data feed (e.g. DSE/CSE), news-feed API, brokerage trading platform	view predicted price and confidence band for a stock and horizon; compare prediction with actual; set alerts; analysts tune/retrain models and see accuracy metrics; feed delivers ticks every few seconds in a defined format; admin manages users and data sources
Indirect	Company management/sponsor, regulator (BSEC), compliance and legal team, data-science lead, marketing	business goals (subscription tiers), mandatory disclaimer that predictions are not investment advice, audit trail of published predictions, no use of insider/non-public data, model explainability for compliance
Domain	Stock-market domain: trading hours and holidays, circuit breakers, corporate actions (splits, dividends), time-series nature of prices, financial data standards	prices must be adjusted for splits/dividends; no predictions computed on non-trading days; handle halted stocks; time stamps in exchange time zone; train/test split must respect time order (no look-ahead)

Three non-functional requirements

(i) Performance: a prediction request for any listed stock is answered within 2 s for 95% of requests; predictions are refreshed within 1 minute of new market data.

(ii) Availability / reliability: 99.9% availability during trading hours (10:00–14:30); a feed failure is detected within 30 s and stale predictions are flagged.

(iii) Security & privacy: all access over HTTPS with authenticated accounts; user portfolios encrypted at rest; every published prediction logged for audit for 5 years.

(Others: scalability to 10,000 concurrent users during market open; usability for non-expert investors; accuracy reporting, e.g. MAPE shown per model.)

Answer. Interactor = direct users/systems (traders, analysts, admin, market-data feed); Indirect = stakeholders who influence but do not use it (management, regulator BSEC, compliance); Domain = market rules and data constraints (trading days, corporate-action adjustment, no look-ahead). NFRs: response $\leq$ 2 s, 99.9% availability in trading hours, secure/audited access.

2016–2017 • Q1(b) CSE307 • 15/07/2017 • R014.2 MARKS 12

What are the different types of output document of a system? Explain with examples.

SOLUTION

Classification by distribution and audience

1. Internal outputs – intended for system owners and users *inside* the organisation; they support management and operations. Three kinds of internal reports:

- **Detailed report** – presents information with little or no filtering, one line per transaction/record. *Example:* list of all sales of the day with invoice no., customer, amount.
- **Summary report** – categorises information for managers who do not want to wade through details; totals, averages, charts. *Example:* monthly sales per region with totals.
- **Exception report** – filters data and reports only records meeting some (abnormal) condition. *Example:* list of items below reorder level; overdue customer accounts.

2. External outputs – leave the organisation and go to customers, suppliers, government: *examples* – invoices, account statements, paycheques, course schedules, airline tickets, receipts. Usually printed on pre-printed forms or sent electronically; they reflect the organisation's image.
3. Turnaround documents – external outputs that eventually re-enter the system as *inputs*. *Example:* the payment stub of a utility (electricity) bill that the customer returns with the payment, or a pre-printed exam registration form sent back by a student; data on it is captured by scanning.

Classification by medium / format

Medium	Example
Paper (printer)	Invoices, cheques, statements
Screen / web page	Account balance inquiry, dashboards
Point-of-sale terminal	Receipt and price display at a shop counter
Multimedia / audio	Telephone banking voice response
E-mail / SMS	Payment confirmation SMS, e-statement
Hyperlinks / files	Downloadable PDF reports, CSV exports

Answer. Internal outputs (detailed, summary and exception reports), external outputs (invoices, statements, paycheques, tickets) and turnaround documents (external outputs returned as inputs, e.g. bill payment stubs); outputs can also be classified by medium – printed, screen, POS, multimedia, e-mail, files.

2014–2015 • Q5(a) CSE307 • 04/02/2016 • R019 **MARKS 20**

A magazine company's customer sends an order coupon to the company's order entry department. The order entry clerk approves the order and prepares a four-part sales order form. The clerk then enters the order into the sales journal and accounts receivable subsidiary ledger. The first three copies of the sale orders are forwarded to the fulfillment department; the fourth is filed by date along with the order coupon. The fulfillment department determines which reward will be offered to the customer and updates the other to add reward information. The first copy of the order is mailed to the customer, the second is sent to the company's distribution center, and the third is filed numerically. Draw a prototype of the information system of the magazine company.

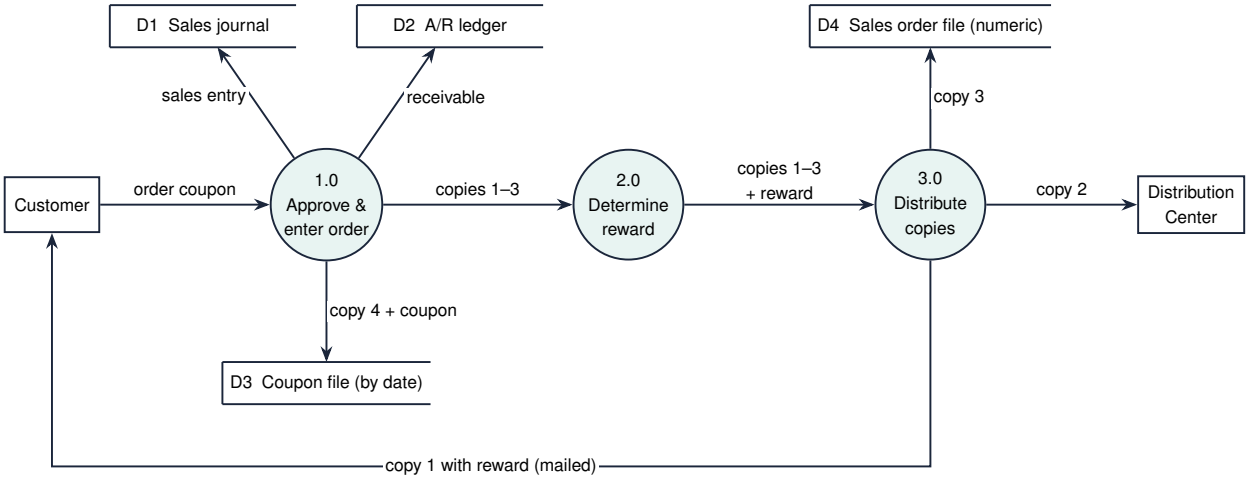
SOLUTION

NOTE "Prototype of the information system" is interpreted as (1) a model of the system's processes, data stores and document flows (a DFD of the order-processing system) and (2) a screen prototype of the main input form. "Updates the other to add reward information" is read as "updates the other copies".

Step 1 – Identify the elements of the narrative

- External entities: **Customer**, **Distribution Center**.
- Processes: **1.0 Approve & enter order** (order entry clerk), **2.0 Determine reward** (fulfilment), **3.0 Distribute order copies** (fulfilment).
- Data stores: **D1 Sales journal**, **D2 Accounts receivable ledger**, **D3 Order coupon file (by date)**, **D4 Sales order file (numeric)**.
- Documents: order coupon; four-part sales order (copies 1–3 to fulfilment, copy 4 filed).

Step 2 – Process model (DFD) of the order-processing system



Step 3 – Screen prototype of the computerised order entry

Magazine Co. – Order Entry

Coupon no.	C-20391
Order date	12-03-2016
Customer ID	CUS-0042 [search]
Name	Rahim Uddin
Mailing address	12 Lake Rd, Dhaka
Magazine & term	Tech Monthly – 12 issues
Amount (Tk.)	1,200

Fulfilment panel
Reward: [Tote bag ▽]
Status: Approved
Copies: 1 Customer
2 Distribution
3 Numeric file
4 Date file

[Approve] [Save & post (journal, A/R)] [Print 4-part order] [Cancel]

Step 4 – How the prototype supports the business

- *Approve* validates the coupon and customer; *Save* posts automatically to the sales journal (D1) and A/R ledger (D2) – no double manual entry.
- The fulfilment panel lets the fulfilment department assign the reward, which is added to all three working copies at once.

- *Print 4-part order* produces copy 1 (mailed to customer), copy 2 (distribution centre), copy 3 (numeric file) and copy 4 (filed by date with the coupon); in a fully electronic version the two "files" become indexed tables queried by date or order number.

Answer. Model the system as Customer → 1.0 Approve & enter order (posts to Sales journal and A/R ledger, files copy 4 + coupon by date) → 2.0 Determine reward → 3.0 Distribute copies (copy 1 to customer, copy 2 to distribution centre, copy 3 to numeric file), and prototype an order-entry screen with approve, post, reward and print-4-copies functions.

2014–2015 • Q7(a) CSE307 • 04/02/2016 • R021.3 **MARKS 15**

Explain different kinds of feasibility analyses used in information system design.

SOLUTION

Feasibility analysis

Feasibility is the measure of how beneficial or practical the development of an information system will be to an organisation; **feasibility analysis** is the process by which it is measured. It is done repeatedly ("creeping commitment"): at the end of the problem-analysis, requirements and design phases, so that a project can be cancelled or revised early.

Kinds of feasibility

1. Operational feasibility – *Will the solution solve the problem and will it be used?*

- Checks urgency of the problem and acceptability of the solution, often with the *PIECES* framework (Performance, Information, Economics, Control, Efficiency, Service).
- *Cultural / political feasibility*: how end-users and managers feel about the system (fear of job loss, resistance to change).

2. Technical feasibility – *Is the solution technically practical, and do we have the expertise?* Maturity of the technology, scale (can it handle volumes/users), compatibility with existing infrastructure, availability of skilled staff.

3. Schedule feasibility – *Can it be built within acceptable time?* Are mandatory deadlines (laws, contracts) realistic; what is the cost of delay; can the deadline be met with available resources?

4. Economic feasibility (cost–benefit analysis) – *Is it cost-effective?*

- Costs: development (one-time) and operating (recurring); benefits: tangible (cost reductions, increased revenue) and intangible (better decisions, customer goodwill).
- Techniques: *payback analysis* (time to recover the investment), *return on investment* $\text{ROI} = \dfrac{\text{lifetime benefits} - \text{lifetime costs}}{\text{lifetime costs}}$, and *net present value* using discounted cash flows, $PV = \dfrac{\text{amount}}{(1+i)^n}$.

5. Legal (and contractual) feasibility – conformity with laws and regulations (data protection, copyright, licences, contractual obligations).

Feasibility	Key question
Operational (incl. cultural/political)	Will it work in the organisation and will people use it?
Technical	Can we build it with available technology and skills?
Schedule	Can we build it in time?
Economic	Do benefits exceed costs (payback, ROI, NPV)?
Legal	Is it lawful and contract-compliant?

Answer. Operational (does it solve the problem / will it be accepted – PIECES, cultural/political), technical (is the technology and expertise practical), schedule (can it be done in time), economic (cost–benefit: payback, ROI, NPV) and legal feasibility.

2014–2015 • Q8(c)(i) CSE307 • 04/02/2016 • R022.2 **MARKS 6**

ABC Advertising specializes in using large helium balloons to help business with their marketing. They rely on computer processing and databases to manage the distribution of balloons around the country. Customers can hire their advertising balloon by phoning the state branch of ABC Advertising.

Whoever answers the telephone, checks the booking file to see if the requested time by the customer is available. The customer contact details, advertising text, balloon type and install instructions are taken down and entered into the customer file. A total price is negotiated and 10% deposit is requested from the customer. This is written into the accounts file.

When the deposit has been received a receipt is sent to the customer and the accounts file is updated. A designer then uses the advertising details to create a number of design and these are sent to the customer. The customer selects the appropriate design and adds comments for any alteration and returns the design to ABC Advertising. The changes are made and the customer file is updated with the design details that will be used.

The balloon is created and an account is produced and sent to the customer. Installation is then completed at the prearranged time. When the full payment has been received from the customer the account file is updated and a statement showing all payment details is sent to the customer. At the end of the week business transaction report is produced from the customer and account files and this is sent to the head office.

(i) Suggest whether a centralized or distributed information system would be used in the above situation. Give two reasons for your answer.

SOLUTION

> **NOTE** Part (ii) (fact-finding strategy) is in-syllabus and solved in the Requirement Analysis chapter under the same ID.

Centralised vs distributed

In a **centralised** IS all data and processing reside at one site, accessed remotely; in a **distributed** IS data and/or processing are spread over several sites connected by a network, each site handling its local work while sharing/consolidating data with others.

Recommendation: a distributed information system

(i) Business is geographically distributed and branch-autonomous. Bookings are taken by

state branches; each branch checks its own booking file, keeps its customers, designs and accounts and performs local installations. Keeping these files at the branch gives fast local response and lets a branch keep working even if the link to head office fails.

(ii) Head office needs only periodic, summarised data. Each branch sends a *weekly* business-transaction report to head office; detailed transactions need not travel continuously to a central computer, reducing communication cost and central load. Head office consolidates the branch reports (or replicated summaries) for company-wide decisions.

(A central database could work only if all branches had reliable, cheap connectivity and needed a single shared booking pool – which is not indicated.)

Answer. Distributed: branches operate autonomously with their own booking, customer and account files (fast, resilient local processing), while head office only receives weekly consolidated reports, so continuous central processing is unnecessary.

2013–2014 • Q5(a) CSE307 • 08/08/2015 • R025.1 MARKS 5

What is a model in the context of information system design?

SOLUTION

Definition

A **model** is a simplified (usually graphical) representation of some aspect of reality – an existing or proposed system – that abstracts away irrelevant detail so that analysts, designers and users can understand, communicate and validate the system before (or instead of) building it. "A picture is worth a thousand words."

Kinds of models in IS design

- **Logical (essential) models** show *what* a system is or must do, independent of technology – e.g. data model (ERD), process model (DFD), use-case model, class diagram.
- **Physical (implementation) models** show *how* it is or will be implemented – e.g. database schema, structure charts, deployment diagrams, network architecture.
- By viewpoint: *data* models (ERD/class), *process* models (DFD/BPMN/activity), *interface* models (screen prototypes), *behaviour* models (state, sequence diagrams).

Why models are used

They reveal ambiguity and missing requirements early, separate concerns, let non-programmers review the design, and serve as a blueprint for construction and documentation.

Answer. A model is an abstract, usually pictorial representation of a system (data, process, interface or behaviour) used to understand, communicate and validate requirements and design; logical models show *what*, physical models show *how*.

2012–2013 • Q6(b) CSE307 • 17/05/2014 • R032 MARKS 4 + 10

Assume that a chain shopping store is going to implement an information system. Every customer is associated with a branch. If a customer purchases products worth more than Tk. 50,000 in a

month, he/she becomes a super customer for that month. Super customers get special offer for purchases from any branch in the specified month. Every branch has several employees and one manager. The manager of the branch defines the offer for that branch. So, offer of one branch is not related to the offer of another branch. Now answer the following based on the given scenario.
(i) What database tables do you need for the store?
(ii) For each database tables, find whether you should replicate or partition the data. Justify your answer.

SOLUTION

(i) Tables

Table	Main columns (PK underlined)
Branch	branch_id, name, address, manager_id
Employee	emp_id, branch_id, name, designation (manager / staff)
Customer	customer_id, home_branch_id, name, phone, address
Product	product_id, name, unit_price
Purchase (sale)	purchase_id, customer_id, branch_id, date, total_amount
PurchaseItem	purchase_id, product_id, quantity, price
Offer	offer_id, branch_id, defined_by (manager), description, discount, month
SuperCustomer	customer_id, month, total_purchase (> Tk. 50,000)

(ii) Replicate or partition?

Rules of thumb: **replicate** data that is small, read at many sites and rarely updated; **partition** (horizontally, by branch) data that is large, updated frequently and used mainly at one site.

Table	Decision	Justification
Branch	Replicate	Small, almost static, needed by every branch (e.g. to show where an offer applies).
Product	Replicate	Same catalogue sold at all branches; read-heavy, updates rare and centrally made.
Employee	Partition by branch	Every employee works in one branch; HR/updates are local.
Customer	Partition by home branch (+ replicate super customers)	Each customer is associated with one branch where most activity happens; but see SuperCustomer.
Purchase, PurchaseItem	Partition by branch where sale occurred	Highest volume, insert-heavy, local transactions; monthly totals can be consolidated centrally.
Offer	Partition by branch	Each manager defines offers for his branch only, and an offer of one branch is unrelated to others – purely local data.
SuperCustomer	Replicate	A super customer may shop at *any* branch, so every branch must recognise him instantly; the list is small and changes only once a month.

Answer. (i) Branch, Employee, Customer, Product, Purchase, PurchaseItem, Offer, SuperCustomer. (ii) Replicate small, read-everywhere data (Branch, Product, SuperCustomer); partition by branch the local, high-volume or branch-owned data (Employee, Customer, Purchase/PurchaseItem, Offer).

2012–2013 • Q6(d) CSE307 • 17/05/2014 • R033.1 MARKS $6\frac{2}{3}$

Document management system (DMS) is used to track and store electronic documents. It is usually capable of history tracking i.e., keeping track of the different versions modified by different users. Now discuss the intangible benefits of a DMS.

SOLUTION

Intangible benefits

Intangible benefits are real advantages that are difficult or impossible to measure in money (contrast tangible benefits such as reduced paper, storage and staff cost). For a DMS with version history:

- **Better collaboration** – many users work on the same documents without overwriting each other; everybody sees the latest version.
- **Improved decision making** – accurate, up-to-date information is found quickly.
- **Accountability and transparency** – history shows who changed what and when, which discourages careless or malicious edits.
- **Regulatory compliance and audit confidence** – version trails support audits and legal disputes.
- **Knowledge retention** – organisational knowledge is preserved when staff leave; old versions can be restored.
- **Security and peace of mind** – controlled access and backup reduce risk of loss or leaks.
- **Customer satisfaction and corporate image** – faster, more reliable responses to clients.
- **Employee morale** – less frustration searching for files or redoing lost work.

Answer. Intangible benefits of a DMS: better collaboration, faster and better decisions, accountability via version history, easier compliance/audit, knowledge retention, improved security, customer satisfaction and staff morale – benefits that are real but hard to quantify in money.

2012–2013 • Q8(d) CSE307 • 17/05/2014 • R034.4 MARKS 9

Define (any three):

(i) Stratification of Sampling Method

(ii) Batch Input Processing

(iii) Stub Test

(iv) POS Terminal

(v) Reverse Engineering

SOLUTION

All five are defined; any three suffice in the exam.

(i) Stratification (stratified sampling)

A sampling method used during fact finding in which the population of documents/people is first divided into subgroups (strata) – e.g. by department, transaction type or value – and a sample

is drawn from each stratum (often proportionally). It ensures that every important subgroup is represented and reduces sampling error. *Example:* sampling invoices separately from small, medium and large customers.

(ii) Batch input processing

Input data (source documents) are collected over a period, grouped into batches, and then entered and processed together at a scheduled time instead of one at a time. Batch totals/control counts are used for validation. *Example:* monthly payroll, end-of-day processing of cheques.

(iii) Stub test

A test of a high-level module performed in top-down integration while lower-level modules are replaced by *stubs* – dummy modules that simulate the called module (return fixed values, print a message). It lets a program's control logic be tested before all subordinate modules are written.

(iv) POS terminal

A Point-Of-Sale terminal is a device at a sales counter (cash register with bar-code scanner, card reader, receipt printer) that captures sales data at its source and updates inventory and sales records, often online. *Example:* supermarket checkout.

(v) Reverse engineering

The process of analysing an existing system (code, database) to recover its design and specification at a higher level of abstraction – e.g. generating a data model or class diagram from an existing database or code – typically for maintenance, re-engineering or documentation.

Answer. Stratified sampling = sample from each subgroup; batch input = collect and process inputs in groups at scheduled times; stub test = test higher module with dummy lower modules; POS terminal = sales-counter device capturing data at the source; reverse engineering = recovering design/specification from an existing implementation.

2011–2012 • Q4(a) CSE307 • 23/09/2013 • R038 **MARKS** $18\frac{2}{3}$

What are the different types of System Design Approaches? Describe the different types of Model Driven Approaches.

SOLUTION

System design approaches

Following Whitten & Bentley, the major approaches to system design are:

(i) **Model-driven approaches** – design is expressed with system models (pictures) that are then transformed into a working system. Four flavours: modern structured design, information engineering, prototyping, object-oriented design (below).

(ii) **Rapid Application Development (RAD)** – iterative construction of a series of evolving prototypes with heavy user involvement and time-boxing, so that a working system is delivered quickly; it blends the prototyping approach with JAD/JRP-style workshops.

(iii) **Commercial off-the-shelf (COTS) / package integration** – buy and configure a software package (ERP, accounting); design focuses on fitting the package to the business, gap analysis and integration with existing systems.

(iv) In practice organisations use **hybrid approaches** (e.g. FAST in Whitten), mixing the above.

Model-driven approaches

(a) Modern Structured Design (MSD). A *process-oriented*, top-down technique: the system is decomposed into a hierarchy of modules shown in *structure charts*, derived from the DFDs of analysis. Design quality is judged by *coupling* (low) and *cohesion* (high). Strength: simple, disciplined, good for process-heavy batch systems. Weakness: data is a by-product, so data redundancy and maintenance problems arise; poor fit for event-driven GUIs.
(b) Information Engineering (IE). A *data-centred* but process-sensitive approach: the enterprise is planned and analysed strategically first (ERDs, process models, business areas); the shared database is designed as the core and applications are built around it, often with CASE tools. Strength: integrated, non-redundant enterprise data; strong planning. Weakness: long and expensive planning; needs strong management commitment.
(c) Prototyping. Small-scale working models of inputs, outputs, screens and reports are built quickly, reviewed by users and refined iteratively; design "emerges" from user feedback (cf. the prototyping process model). Strength: users see and react, requirements become clear, fewer misunderstandings. Weakness: may encourage skipping analysis, poor documentation and scalability, users may mistake a prototype for the finished product.
(d) Object-Oriented Design (OOD). The system is designed as a set of collaborating *objects* that encapsulate both data and behaviour, modelled with UML (use-case, class, sequence, state diagrams), and implemented in OO languages. Strength: reuse, encapsulation, natural fit for GUI and distributed systems, consistent models from analysis to code. Weakness: learning curve, requires mature OO tools/skills.

Approach	Centred on	Main models	Typical use
MSD	Processes	DFD → structure chart	Process-intensive systems
IE	Data (enterprise)	ERD, process dependency, matrices	Integrated enterprise databases
Prototyping	User interfaces	Working mock-ups	Unclear requirements
OOD	Objects (data + behaviour)	UML diagrams	GUI, distributed, reusable systems

Answer. Design approaches: model-driven, rapid application development (RAD), COTS/package integration (and hybrids). Model-driven approaches: Modern Structured Design (top-down structure charts, coupling/cohesion), Information Engineering (data-centred, enterprise ERD-driven), Prototyping (iteratively refined working models) and Object-Oriented Design (objects and UML).

2010–2011 • Q3(b) CSE307 • 19/03/2012 • R044.2 **MARKS 24**

Define each of the following with suitable examples:

(i) Detailed Report
(ii) Summary Report
(iii) Exception Report
(iv) Turnaround Document
(v) Two-Tier Architecture
(vi) Three-Tier Architecture
(vii) File-Server Architecture
(viii) Agile Method

SOLUTION

NOTE Item (viii) belongs to the in-syllabus topic *Process Models + Agile*, but the question as a whole (reports and architectures) is outside the course slides, so it is kept here unsplit.

(i) Detailed report

Presents information with little or no filtering – typically one line per record or transaction – used by operational staff to check and control daily work. *Example:* daily list of all orders received with order no., customer, items and amount.

(ii) Summary report

Groups and totals the detail for managers who need trends rather than individual transactions. *Example:* total sales per product line per month, with grand totals and a bar chart.

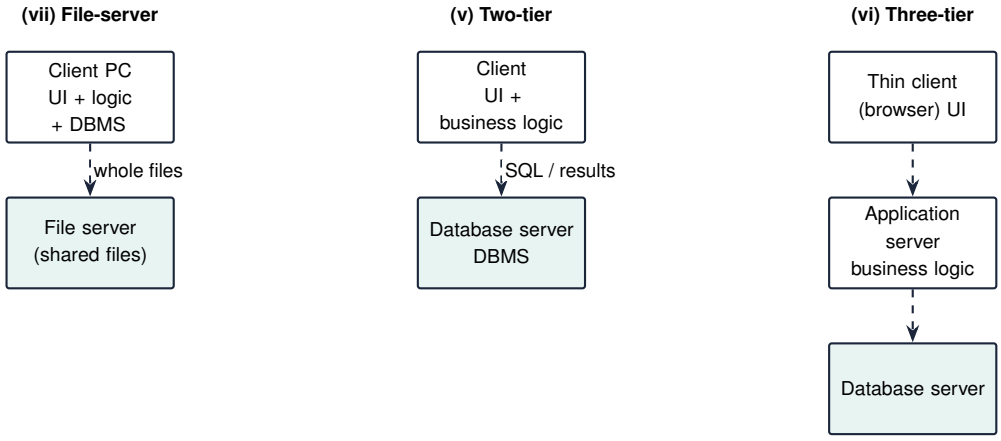
(iii) Exception report

Filters data and shows only records that meet an exceptional condition requiring attention. *Example:* customers whose credit balance exceeds their limit; stock items below reorder level.

(iv) Turnaround document

An external output that the recipient later returns so that it becomes an input to the system. *Example:* electricity bill with a tear-off payment slip returned with the payment; pre-filled renewal form returned by a member.

(v)–(vii) Architectures



(v) Two-tier (client/server) architecture – the client runs the user interface *and* the business logic ("fat client"); a database server runs the DBMS and returns only the requested records. *Example:* a VB/Java desktop accounting program querying an Oracle server on the LAN. Drawbacks: logic must be installed and updated on every client; does not scale to many users.
(vi) Three-tier architecture – presentation, business logic and data are separated onto client, application server and database server. *Example:* web banking – browser → application server → database. Advantages: thin clients, central logic updates, scalability and security.
(vii) File-server architecture – a LAN file server only stores shared files; the entire DBMS and application run on each client, so whole files are transferred over the network and the client does all processing. *Example:* several PCs sharing an MS-Access database file on a network drive. Drawbacks: heavy network traffic, poor concurrency control and integrity.

(viii) Agile method

An iterative and incremental approach to software development that values individuals and interactions, working software, customer collaboration and responding to change (Agile Manifesto). Work is delivered in short iterations (1–4 weeks) with continuous customer feedback, minimal documentation and self-organising teams. *Examples:* Scrum (sprints, product backlog, daily stand-up), Extreme Programming (pair programming, TDD, continuous integration).

Answer. Detailed = unfiltered transaction listing; summary = totals for managers; exception = only abnormal records; turnaround = output returned as input (bill stub). Two-tier = fat client + DB server; three-tier = client / application server / DB server; file-server = clients do all processing on files from a shared server. Agile = iterative, incremental, customer-collaborative development (Scrum, XP).

2010–2011 • Q4(b) CSE307 • 19/03/2012 • R045 MARKS 5 + 5

Write down the guideline for input design and output design of a system.

SOLUTION

Input design guidelines

(i) **Capture only variable data** – do not enter data that can be looked up or computed (e.g. enter product code, not its price or name).

(ii) **Do not re-enter data already stored** – retrieve existing customer details by ID.

(iii) **Use codes and defaults** to reduce keystrokes; provide pick lists, check boxes, calendars.

(iv) **Capture data at its source**, as close to the point of origin as possible (POS scanners, online forms) to avoid transcription errors; prefer automatic data capture (bar code, OCR, RFID).

(v) **Validate all input** – existence, data type, range/limit, format, consistency checks and check digits; give clear error messages and allow easy correction.

(vi) **Make the input screen match the source document** layout and follow a logical top-to-bottom, left-to-right flow; keep it simple and consistent.

Output design guidelines

(i) **Outputs should be simple to read and interpret** – title, date/time, headings, legends, page numbers, consistent formats, meaningful labels.

(ii) **Timeliness** – deliver outputs when the recipient needs them for decisions.

(iii) **Distribution** – deliver to the right people (and only them, for confidential data) in sufficient quantity.

(iv) **Acceptability** – the format/medium must suit the user: detail for operations, summaries and exceptions for managers, charts for trends.

(v) **Minimise volume** – avoid information overload; provide drill-down rather than huge printouts; prefer exception reporting.

(vi) **Choose the right medium** (screen, print, e-mail, POS) and prototype outputs with users before finalising.

Answer. Input: capture only variable data, never re-enter stored data, use codes/defaults and source data automation, validate everything, keep screens simple and matching source documents. Output: readable and clearly labelled, timely, properly distributed, acceptable in format/medium to its users, minimal volume (summary/exception), prototyped with users.

2010–2011 • Q6(d) CSE307 • 19/03/2012 • R046 MARKS $6\frac{2}{3}$

Describe the communication practices a software engineer should adopt.

SOLUTION

Communication practice (Pressman's principles)

Communication with customers and colleagues is the first framework activity; requirements are only as good as the communication that produced them. The core principles:

(i) **Listen** – focus on the speaker's words, ask for clarification, do not interrupt or jump to conclusions.

(ii) **Prepare before you communicate** – understand the problem and domain; prepare an agenda.

(iii) **Someone should facilitate** the activity – a leader keeps the meeting productive and mediates conflicts.

(iv) **Face-to-face communication is best**, supported by a visual (drawing, document) of the relevant information.

(v) **Take notes and document decisions** – a recorder captures points and action items.

(vi) **Strive for collaboration** – build consensus; use the team's collective knowledge.

(vii) **Stay focused; modularise the discussion** – discuss one topic at a time.

(viii) **If something is unclear, draw a picture.**

(ix) **Move on** once something is agreed, or if it cannot be agreed/clarified at the moment.

(x) **Negotiation is not a contest** – aim for a win-win result through compromise.

Answer. Listen, prepare, have a facilitator, prefer face-to-face communication, take notes and document decisions, collaborate, stay focused and modularise, draw pictures when unclear, move on when agreed or stuck, and negotiate for win-win.

2010–2011 • Q8(a) CSE307 • 19/03/2012 • P025 MARKS 5

What is the difference between software engineering and system engineering?

SOLUTION

Definitions

System engineering is concerned with *all* aspects of developing and evolving a complex computer-based system – hardware, software, people, processes and the environment. It covers system requirements definition, architectural design, allocation of functions to hardware/software/humans, integration, installation and decommissioning. **Software engineering** is the engineering discipline

concerned with all aspects of *software* production – specification, design, implementation, testing and evolution of programs and their documentation.

Differences

Aspect	System engineering	Software engineering
Scope	Whole system: hardware, software, people, processes	Software part only
Relation	Parent discipline	Part of system engineering
Typical activities	System requirements, partitioning of functions, procurement, integration of subsystems	Software requirements, design, coding, testing, maintenance
Practitioners	System engineers from several disciplines (electrical, mechanical, software)	Software engineers
Example (aircraft)	Engines, avionics, crew procedures, flight-control software as one system	The flight-control software

Answer. System engineering deals with the whole computer-based system (hardware, software, people, processes) and allocates functions among them; software engineering is the part of it concerned only with the specification, development and evolution of the software.

X3

User Interface & Usability

OUT OF SYLLABUS

2017–2018 • Q5(b) CSE307 • 29/09/2018 • R011 MARKS 12

What do you understand by the usability of software application? How is it measured? Discuss two dimensions of usability with example.

SOLUTION

1. What usability means

Usability is the extent to which a product can be used *by specified users* to achieve *specified goals* with **effectiveness**, **efficiency** and **satisfaction** in a *specified context of use* (ISO 9241-11). In plain words: how easily, quickly and pleasantly the intended users can get their real work done with the software, and how few mistakes they make while doing it. Usability is a *non-functional* (quality) requirement: two systems with identical functions can differ hugely in usability.
Jakob Nielsen breaks usability into five measurable *dimensions* (quality components):

Dimension	Question it answers
Learnability	How easily can first-time users accomplish basic tasks?
Efficiency	Once users have learned the design, how quickly can they perform tasks?
Memorability	After a period of not using it, how easily can users re-establish proficiency?
Errors	How many errors do users make, how severe are they, how easily can they recover?
Satisfaction	How pleasant / subjectively satisfying is the design to use?

2. How usability is measured

Usability is measured empirically, by observing *representative users* performing *representative tasks*, plus expert inspection methods.

- **Usability testing (user testing).** 5–8 real users perform scripted tasks ("buy a book and pay by card") while an observer records what happens (think-aloud, screen/video recording, eye tracking). From the sessions we compute *metrics*:

Metric	How it is computed	Dimension
Task success rate	tasks completed correctly / tasks attempted $\times 100\%$	effectiveness
Time on task	mean seconds to complete a task	efficiency
Error rate	errors per task (wrong clicks, invalid input, dead ends)	errors
Learning curve	time on task in trial 1, 2, 3, …	learnability
Retention	time on task after 2 weeks' break	memorability
SUS score	10-item System Usability Scale questionnaire, 0–100 (> 68 = above average)	satisfaction

- **Questionnaires and interviews** (SUS, QUIS, Likert scales) for subjective satisfaction.
- **Inspection methods** (no users needed): *heuristic evaluation* against Nielsen's 10 heuristics or Shneiderman's 8 golden rules, *cognitive walkthrough* of each step of a task.
- **Field data**: analytics/log files (click paths, abandonment rate, help-desk calls), A/B testing two designs with live users.

Measured values are compared with *usability requirements* written in the SRS, e.g. "a new cashier shall be able to complete a sale within 60 s after 30 min of training, with at most one error per 20 sales". This makes usability *testable*.

3. Two dimensions explained with examples

(a) Learnability. A system is learnable when a novice can become productive quickly, mostly by recognising rather than recalling. *Example:* a bank ATM is used by people with no training at all; it therefore shows one simple question per screen ("Select account type"), big labelled buttons next to the screen and familiar words. It is measured by giving first-time users a task (withdraw Tk 2000) and recording success rate and time on the first attempt.
(b) Efficiency of use. Once learned, the expert should be able to work fast with minimum keystrokes. *Example:* a supermarket POS cashier processes hundreds of sales a day; the interface gives barcode scanning, keyboard shortcuts (F2 = payment, F9 = void item), default quantity 1 and auto-completion of product codes. Measured as average time per sale and keystrokes per sale for experienced cashiers; a redesign that reduces mean sale time from 75 s to 50 s is a 33% efficiency gain.
Other dimensions can be argued similarly, e.g. *errors*: an online form that validates a mobile number as you type and offers "Undo" for deletions reduces both error frequency and severity.

Answer. Usability = effectiveness, efficiency and satisfaction with which specified users reach specified goals in a specified context. It is measured by usability tests on real users (success rate, time on task, error rate, learning curve, SUS questionnaire) and by expert inspections (heuristic evaluation, cognitive walkthrough). Dimensions: learnability (ATM for novices), efficiency (shortcuts for POS cashiers), plus memorability, errors, satisfaction.

2017–2018 • Q6(b) CSE307 • 29/09/2018 • R012 **MARKS 3 + 10**

What do you understand by "Parameterization" in the context of software design? Design a user interface to parameterize the price of product that may vary for different brands, sizes, etc. for a point-of-sales software.

SOLUTION

1. Parameterization (3 marks)

Parameterization means designing software so that values and rules that are expected to *vary or change* (prices, tax rates, discount slabs, units, thresholds) are *not hard-coded* in the program but are kept as *parameters*: data in parameter/configuration tables that authorised users can maintain through a UI. The program reads the parameters at run time, so

- a business change (new VAT rate, new size of a product) needs **no code change, recompile or redeployment**;
- the same software can serve different shops/branches (different parameter values);
- changes can be dated (effective from/to), audited and rolled back.

Example: instead of `if (size == 200) price = 95;` the POS looks up `PRICE(product, brand, size, outlet, date)` in a price table maintained by the store manager.

2. Analysis of what must be parameterised

For a POS the selling price of one product depends on several *dimensions*: brand, pack size (and unit), sometimes outlet/branch, customer type (retail/wholesale) and date (offers, price changes). Therefore the screen must let the manager (a) choose which dimensions apply to a product, (b) fill a price for every combination (a *price matrix*), (c) give each price an effective period, and (d) add VAT/discount rules, with validation and history. The underlying parameter table is `ProductPrice(productId, brand, size, unit, outlet, basePrice, vatPct, discountPct, validFrom, validTo)`.

3. The user interface

POS · Product Price Parameters — User: Store Manager

Product Toothpaste ▼ **Category** Personal care **Outlet** All outlets ▼

Price varies by: ☑ Brand ☑ Size ☐ Customer type ☐ Outlet ☐ Colour + Add dimension

Sel	Brand	Size	Unit	Base price	VAT %	Disc. %	Valid from	Valid to	POS price
☐	Colgate	100	g	85.00	15	0	01/07/2018	–	97.75
☐	Colgate	200	g	155.00	15	5	01/07/2018	–	169.34
☐	Pepsodent	100	g	80.00	15	0	01/07/2018	–	92.00
☐	Pepsodent	150	g	115.00	15	10	15/09/2018	30/09/2018	119.03
☐	Close-up	145	g	120.00	15	0	01/07/2018	–	138.00
+ click to add a new brand/size row . . .									

! Row 4 is a time-limited offer: from 01/10/2018 the regular Pepsodent 150 g price (128.00) applies again.

Add row | Copy row | Delete row | Import from CSV | Bulk change % | Price history / audit | Preview receipt

Last changed by Karim, 01/07/2018 10:42 POS price = base × (1 − disc.) × (1 + VAT) Cancel **Save prices**

4. Design decisions explained

(i) Dimensions are themselves parameters. The check-boxes "Price varies by" (and "+ Add dimension") decide which columns appear in the grid, so a product priced only by size, or by brand × size × outlet, uses the same screen; new pricing factors need no programming.

(ii) Price matrix grid. One row per combination of the selected dimensions (brand × size); fast spreadsheet-like editing, copy row for a new size, bulk change ("+5% on all Colgate") and CSV import for hundreds of SKUs.

(iii) Computed POS price column (read-only) shows the effect of VAT and discount immediately (informative feedback); *Preview receipt* shows exactly what the cashier will see.

(iv) Effective dates allow future price changes and time-limited offers without editing the price on the day; the system picks the row valid on the sale date.

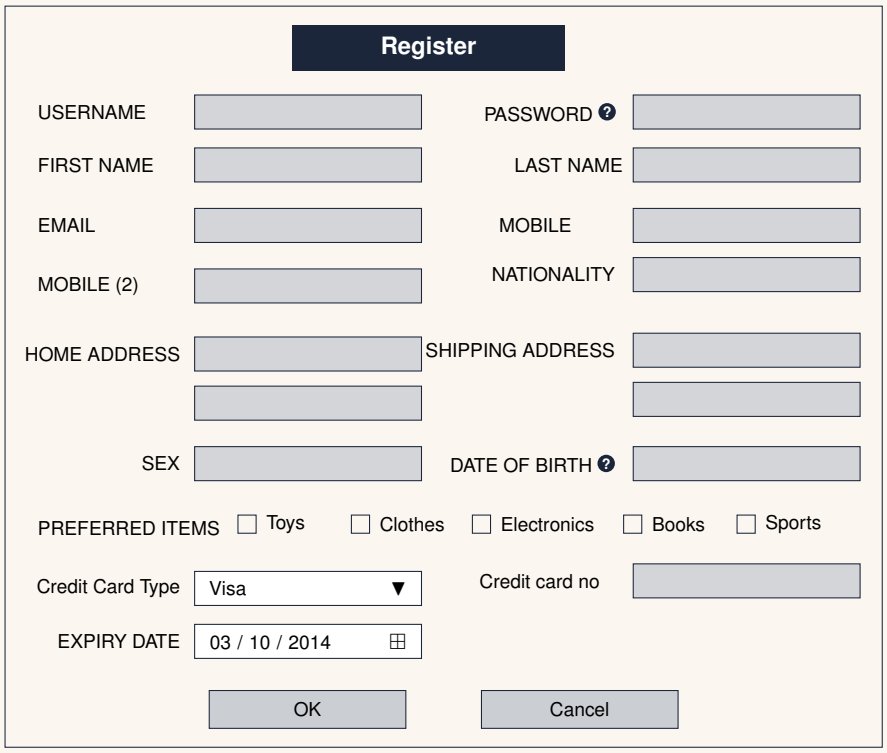
(v) Validation: price > 0, VAT and discount within 0–100%, no two rows with the same dimension values and overlapping dates; errors are shown inline next to the row.

(vi) Control and safety: changes take effect only after *Save*; *Cancel* discards; every change is logged (who/when/old/new) in *Price history*, and only the Store Manager role can open the screen.

Answer. Parameterization = keeping variable business values/rules (prices, taxes, discounts) as data maintained through a UI instead of hard-coding them. The POS price-parameter screen lets the manager choose pricing dimensions (brand, size, ...), edit a brand × size price matrix with VAT, discount and validity dates, preview the computed POS price, validate and save with an audit trail.

2016–2017 • Q4(a) CSE307 • 15/07/2017 • R017 **MARKS 16**

Donald Trump is a rookie UI designer. He designed the following registration page of an e-commerce site (e.g. amazon.com) where the user can browse though various items and buy items. Point out all the problems associated with this page in details.



SOLUTION

Approach

A UI critique is done like a *heuristic evaluation*: walk through the page as the target user (a shopper who only wants to start browsing/buying) and check every element against accepted principles (Shneiderman's golden rules, Nielsen's heuristics, form-design guidelines). The problems found are grouped below; the rule violated is given in brackets.

A. Wrong scope: the form asks for far too much

(i) Registration should be minimal. Browsing needs no account and buying needs only name, e-mail/mobile and password; nationality, sex, date of birth, a second mobile, preferred items are *not needed* to register. Every extra field increases abandonment and privacy concerns

(minimalist design, reduce user effort).

(ii) **Credit-card details at registration** are premature and a security/trust problem; payment data belong in the checkout step, entered over a clearly secured connection, and optionally saved.

(iii) **Both Username and Email** are requested; the e-mail can serve as the login id.

(iv) **Shipping address** at registration is premature as well and, if kept, has no "same as home address" option, forcing double typing.

B. Layout and grouping

(v) **Two-column zig-zag layout.** Fields are read left-right-left-right; the tab order and the logical order (personal → contact → address → payment) are unclear. A single column is faster to fill and to scan.

(vi) **Related fields are scattered**: MOBILE is on the right but MOBILE (2) on the next row on the left; Username and Password are not grouped as "login details"; payment fields are mixed with personal data. No section headings.

(vii) **Inconsistent label alignment and spacing**: some labels are left-aligned, others (HOME ADDRESS, SEX, Credit Card Type, EXPIRY DATE) right-aligned, rows have uneven gaps (consistency).

(viii) **Inconsistent capitalisation and wording**: ALL CAPS labels ("USERNAME") beside mixed case ("Credit card no"); abbreviation "no"; ALL CAPS is harder to read.

(ix) **All fields have the same width** although expected lengths differ greatly (sex vs. address); field size should hint at the expected input.

C. Wrong controls for the data

(x) **SEX as a free-text box**: should be radio buttons (or a drop-down) with fixed options; free text yields "M", "male", "boy" … (prevent errors).

(xi) **DATE OF BIRTH as plain text** with only a "?" icon: no format shown (dd/mm/yyyy?); should be a date picker or three drop-downs.

(xii) **NATIONALITY as free text**: should be a searchable drop-down list of countries.

(xiii) **Address as two unstructured lines**: no separate city, postcode, country; cannot be validated.

(xiv) **Expiry date as a full calendar date (03/10/2014)**: card expiry is only *month/year*; the pre-filled date is already in the past, an invalid default.

(xv) **Credit Card Type pre-set to "Visa"**: a careless user keeps a wrong default; the type can be detected from the card number anyway. CVV and name-on-card fields are missing.

(xvi) **Preferred items** are a fixed list of five categories with no "none/other".

D. Missing feedback, guidance and error prevention

(xvii) **No mandatory/optional indication** (no asterisks), so users do not know what can be skipped.

(xviii) **No "Confirm password"** field and no visible password rules (only a cryptic "?"); no show/hide option and no strength meter.

(xix) **No inline validation or error messages** (e-mail format, mobile digits, username taken); errors would only appear after OK, if at all (informative feedback).

(xx) **No help text / examples** inside fields (e.g. "01XXXXXXXX" for mobile).

(xxi) **No terms-of-service / privacy-policy consent**, no CAPTCHA against bots, no link to "Already registered? Sign in".

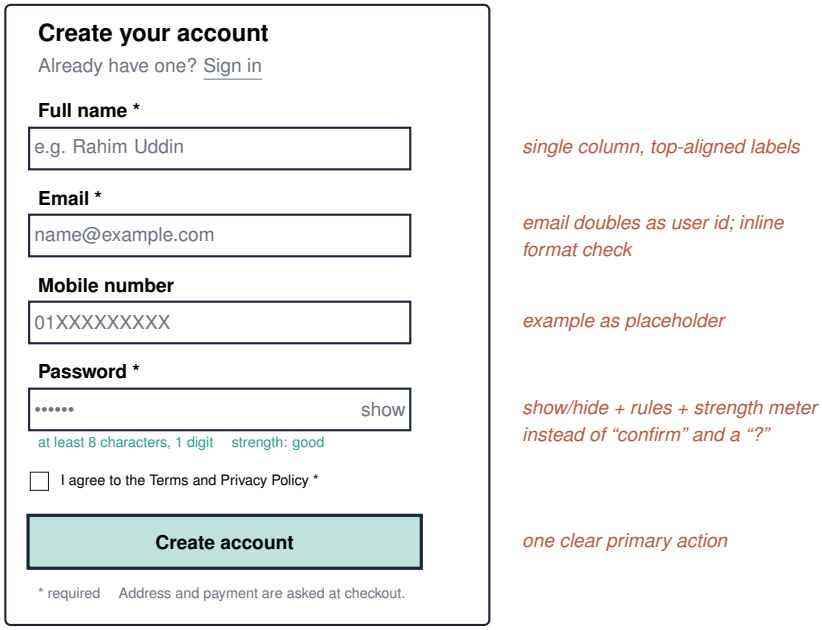
E. Visual design and actions

(xxii) **Dark hatched field backgrounds** give very low contrast with typed text and look disabled; input boxes should be white.

(xxiii) **Vague button labels**: "OK" does not say what happens; use "Create account". The primary and the destructive "Cancel" buttons have identical size and colour, so the main action is not emphasised and a mis-click discards everything without confirmation (closure, reversal).

(xxiv) **No explanation of the benefit** of registering and no progress indication for such a long form; the page is overwhelming for a first-time visitor.

F. Improved design (sketch)

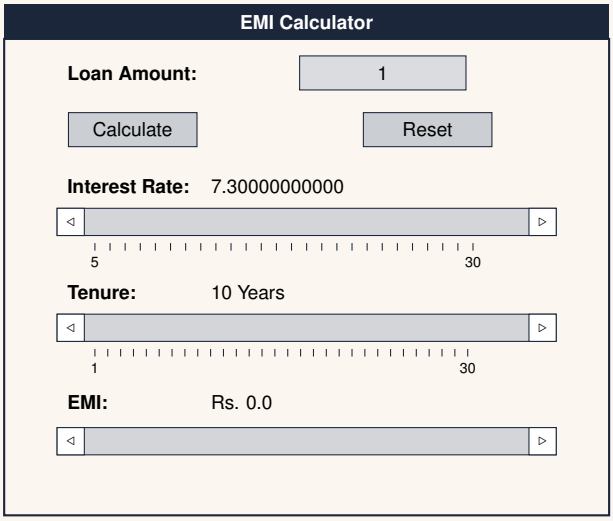


Answer. Main problems: asks far too much (DOB, sex, nationality, preferred items, second mobile, credit card) at registration; confusing two-column zig-zag layout with scattered related fields and inconsistent alignment/capitalisation; wrong controls (free text for sex, DOB, nationality; full date and past default for card expiry; pre-selected "Visa"); no required-field marks, confirm-password/password rules, inline validation, help, consent; low-contrast field backgrounds; vague "OK" and equally weighted "Cancel". Fix: short single-column form, right controls, inline feedback, payment at checkout.

2016–2017 • Q4(c), (d) CSE307 • 15/07/2017 • R018 MARKS 12 + 10

(c) Write down Sehneiderman's 8 Golden Rules for UI design in short. **(12)**

(d) Consider the following design of an EMI Calculator. In this system, the user inputs his loan amount, interest rate and tenure. The system calculates the EMI. Point out the problems associated with this design and draw a paper prototype of this page to solve these problems.



SOLUTION

NOTE The bank prints the mark **(12)** beside part (c) only; in the original paper part (d) carries no printed mark. Q4 totals $46\frac{2}{3}$ ($16 + 8\frac{2}{3} + 12 + ?$), so (d) is worth about 10 marks.

(c) Shneiderman's eight golden rules of interface design

(i) Strive for consistency. Same terminology, layout, colours, fonts and action sequences in similar situations ("Save" is always Ctrl+S and always top-left).

(ii) Seek universal usability (enable frequent users to use shortcuts). Cater for novices and experts: menus for novices, keyboard shortcuts, macros and abbreviations for experts; accessibility for diverse users.

(iii) Offer informative feedback. Every user action gets a system response proportional to its importance: a click highlights, a long upload shows a progress bar, a save shows "Saved".

(iv) Design dialogs to yield closure. Group actions into sequences with a beginning, middle and end, and confirm completion ("Order #1023 placed").

(v) Prevent errors (offer simple error handling). Grey out illegal commands, use pick lists and input masks; if an error occurs, give a specific, constructive way to fix it.

(vi) Permit easy reversal of actions. Undo, Back, Cancel relieve anxiety and encourage exploration.

(vii) Keep users in control (support internal locus of control). Users initiate actions; avoid surprising system behaviour, forced sequences and hard-to-get information.

(viii) Reduce short-term memory load. "Seven plus or minus two": keep displays simple, show information instead of making users remember it, multiple-page displays consolidated,

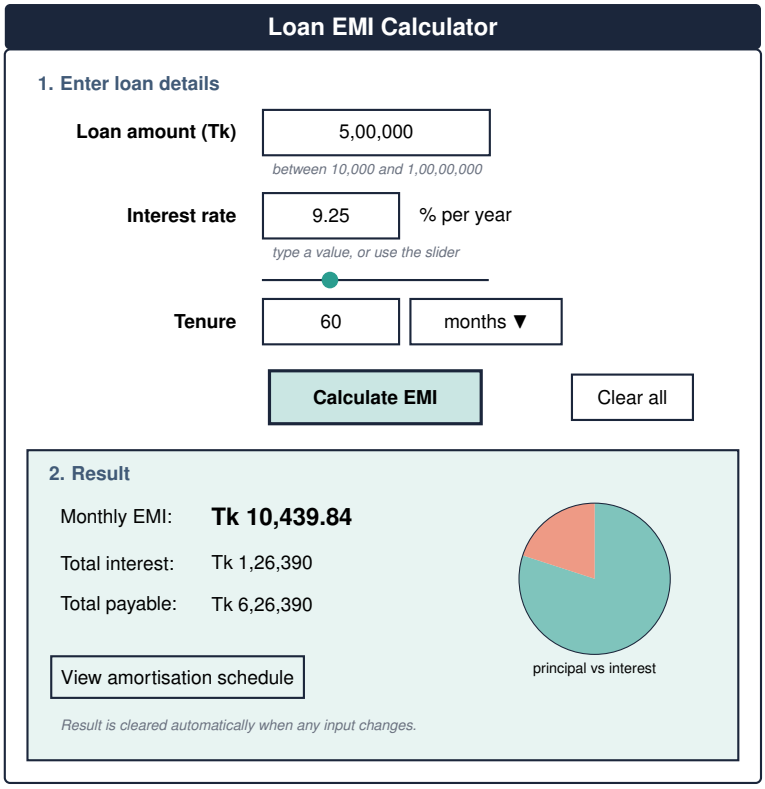
recognition rather than recall.

(d) Problems in the given EMI calculator

#	Problem	Why it is a problem (rule violated)
1	*Calculate* and *Reset* placed after the first input	Natural top-to-bottom flow is broken: the user meets the action buttons before entering rate and tenure, and may press Calculate early (closure, error prevention).
2	Reset right beside Calculate, same size and look	A slip on the destructive button wipes all inputs; no confirmation or undo (prevent errors, reversal).
3	Interest rate shown as `7.30000000000`	Meaningless precision, no "%" and "p.a."; hard to read and verify (reduce memory load, informative display).
4	Sliders as the only way to enter rate and tenure	Sliders are imprecise for exact values such as 9.25%, need many arrow clicks; unreadable tick marks; range 5–30% is arbitrary.
5	Tenure only in whole years	Many loans are in months (e.g. 18 months); no unit choice.
6	EMI output drawn as a scroll bar / slider	An *output* looks like an editable *input*; the user may try to drag it (consistency, users' expectations).
7	"Rs. 0.0" shown before calculation; no currency on Loan Amount	Misleading default result; loan field has no unit, no thousand separators; default value "1" is unrealistic.
8	No validation or error messages	Negative/blank amounts or letters are not caught (prevent errors, informative feedback).
9	Result gives only EMI	No total interest, total payable or amortisation schedule, which users need to decide (closure).
10	Dense hatched backgrounds, uneven alignment, cramped labels	Low contrast and poor grouping make the screen hard to scan (consistency, aesthetic and minimalist design).

(d) Paper prototype of the improved page

A **paper prototype** is a quick, low-fidelity hand-drawn sketch of the screen used to try a design with users before coding it. The redesigned page groups *inputs* (top), *actions* (middle) and *results* (bottom) in reading order:



The EMI shown is computed with the standard formula

$$\text{EMI} = \frac{P\,r\,(1+r)^n}{(1+r)^n - 1}, \qquad r = \frac{\text{annual rate}}{12 \times 100},\ n = \text{months},$$

e.g. $P = 5{,}00{,}000$, rate 9.25% ($r = 0.0077083$), $n = 60$ gives EMI $\approx$ Tk 10,440.

Answer. (c) Consistency; universal usability/shortcuts; informative feedback; dialogs that yield closure; error prevention and simple handling; easy reversal; user in control; reduce short-term memory load. (d) The EMI page mis-orders inputs and buttons, puts Reset next to Calculate, uses imprecise sliders and absurd precision, shows the output as a slider, lacks units/validation/breakdown; the paper prototype orders inputs → action → read-only result with units, validation and a payment breakdown.

2013–2014 • Q6(c) CSE307 • 08/08/2015 • R026.1 **MARKS 8**

Briefly describe human engineering guidelines for user interface design.

SOLUTION

Concept

Human engineering (ergonomics) of user interfaces means fitting the computer dialogue to the capabilities and limitations of people: short-term memory, perception, reaction to errors and expectations. Systems-analysis texts (Whitten & Bentley) summarise it in a set of *human-engineering guidelines* that every input/output screen should satisfy.

The guidelines (with an example for each)

(i) The user should always be aware of what to do next. Every screen tells the user what the system expects: a prompt such as "Type the customer number and press Enter" or a

highlighted default button. Long operations show progress ("Saving 3 of 10 records…").

(ii) **Screens are formatted consistently.** Titles, instructions, data area, error messages and command buttons always appear in the same place on every screen, so the user knows where to look.

(iii) **Messages stay long enough to be read**, and the user dismisses important messages explicitly rather than having them flash past.

(iv) **Use display attributes sparingly.** Colour, blinking, bold and reverse video are reserved for what needs attention (an error field in red), otherwise they distract.

(v) **Provide default values** for fields and answers wherever possible (today's date, quantity 1, "No" for a destructive question) to save keystrokes and errors.

(vi) **Anticipate the errors users might make** and prevent them: drop-down lists instead of free text, input masks for dates, disabling buttons that are not valid now.

(vii) **Do not let the user proceed with an error uncorrected**; validate field-by-field, put the cursor on the wrong field and explain how to fix it.

(viii) **Guard against catastrophic actions.** Deleting a file or closing an unpaid bill asks for confirmation and, ideally, is reversible (undo).

(ix) **Minimise memory load and keystrokes.** Use menus, pick lists and recognisable icons instead of codes that must be remembered; allow shortcuts for experts.

(x) **Speak the user's language and provide help.** Plain, polite terms instead of jargon; context-sensitive help (F1) available on every screen.

EXAM TIP These guidelines overlap with Shneiderman's eight golden rules (consistency, feedback, error prevention, reversal of actions, reduced memory load); quoting either list with one example per point earns full credit.

Answer. Human-engineering guidelines: tell the user what to do next; consistent screen layout; messages readable long enough; sparing display attributes; defaults; anticipate and prevent errors; do not proceed with uncorrected errors; confirm catastrophic actions; minimise memory load; plain language and help.

2012–2013 • Q8(c) CSE307 • 17/05/2014 • R034.3 **MARKS 5**

A fellow designer has asked you to review the dialogue to be used in several screens for a new application. Give your opinion about the following messages based on the guidelines of tone and terminology.

(i) **An error message that says** DISCHARGE, DATE MUST BE ON OR AFTER ADMISSION DATE.

(ii) **An instruction that says** ENTER THE CLIENTS NAME NOW.

(iii) **An error message that says** DATA IS SO FAR OUT OF RANGE IT HAS LEFT THE SOLAR SYSTEM.

(iv) **A question that asks** DO YOU WANT TO RDF THE ACR BEFORE "UCI'ING" CMIS?

(v) **An error message that says** DON'T WORRY-NOT EVERONE GETS IT RIGHT THE FIRST

TIME.

SOLUTION

Guidelines used

Tone: use simple, grammatically correct sentences; be polite but not familiar; do not be funny, cute or sarcastic; do not be condescending or patronising; do not threaten or alarm; avoid ALL CAPITALS, which reads as shouting. **Terminology**: no computer jargon; avoid abbreviations and acronyms the user may not know; use simple, specific words; be consistent; say *what* is wrong and *how* to correct it.

Review of each message

	Opinion (guideline violated)	Better wording
(i)	Content is acceptable (it states the rule), but it is in capitals, the comma makes "DISCHARGE," look like a command, and it does not show the values involved.	"The discharge date (02/03/2014) is earlier than the admission date (05/03/2014). Please type a discharge date on or after the admission date."
(ii)	Bossy, impatient tone ("NOW"), grammatical error (CLIENTS → client's), capitals; "enter" is vaguer than "type".	"Please type the client's name."
(iii)	Attempt at humour/sarcasm; unprofessional, gives no information about which field or what the valid range is.	"Quantity must be between 1 and 500. Please type a value in this range."
(iv)	Pure jargon and unexplained acronyms (RDF, ACR, UCI, CMIS) and a made-up verb "UCI'ing"; the user cannot answer it.	"Do you want to save the account changes before updating the customer records? [Save] [Don't save]"
(v)	Condescending/patronising, spelling error (EVERONE), capitals; says nothing about what went wrong or how to fix it.	"The password you typed is not correct. Please type it again (2 attempts left)."

Answer. (i) acceptable content but shouting caps and no specifics; (ii) bossy, ungrammatical; (iii) cute/ sarcastic, uninformative; (iv) jargon and abbreviations; (v) condescending, misspelt, uninformative. Good dialogue is polite, plain, specific and tells the user how to recover.

2011–2012 • Q4(d) CSE307 • 23/09/2013 • R039.1 MARKS 8

What are the difference between the usage of bar charts and column charts? When do we use radar charts and scatter charts?

SOLUTION

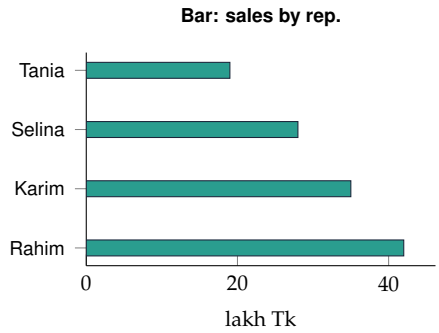
Background

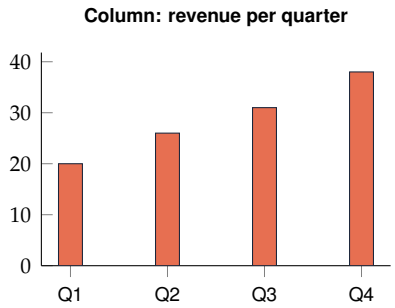
In output design, *graphic* (chart) output is chosen when the reader must see a pattern, trend or comparison at a glance rather than read exact figures. The chart type must match the *message*: comparison among items, change over time, profile over many variables, or relationship between two variables.

Bar charts vs. column charts

Both encode values as rectangle lengths; the difference is orientation and therefore typical usage.

	Bar chart (horizontal bars)	**Column chart (vertical columns)**
Category axis	vertical (y)	horizontal (x)
Best for	comparing *many items/categories* at one point in time; ranking (top 10 products)	showing *change over time* for a few periods (monthly sales) or comparing a few categories
Labels	long category names fit easily, read left to right	labels must be short (months, years)
Example	sales by each of 15 sales representatives	quarterly revenue 2010–2013



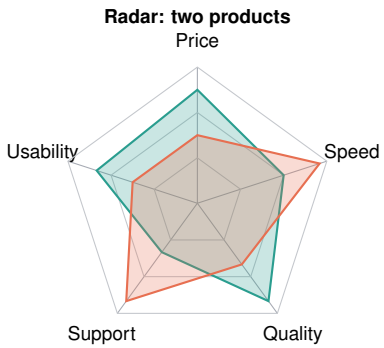


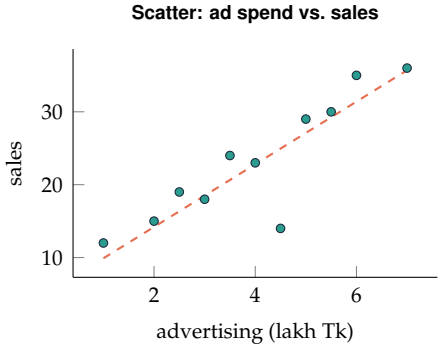
Radar charts

A **radar (spider) chart** has one axis per variable radiating from a centre; the values of one item are joined into a polygon. Use it to compare *several items across many (3–10) variables* at once, i.e. to show a *profile* or balance: performance appraisal of two employees on skills, comparison of two products on price, speed, quality, support, usability. It is also used for cyclic data such as sales in each month around a circle (seasonality). Not suitable for many items or exact reading.

Scatter charts

A **scatter (XY) chart** plots each observation as a point with two numeric co-ordinates. Use it to show the *relationship / correlation* between two variables, clusters and outliers: advertising spend vs. sales, project size (KLOC) vs. effort, defects vs. code complexity. A trend line can be added; points are *not* joined because they are independent observations.





Answer. Bar charts (horizontal) compare many items/categories at one time and suit long labels or rankings; column charts (vertical) show changes over time periods or compare few categories. Radar charts compare items across several variables (profiles, cyclic data); scatter charts show the relationship or correlation between two numeric variables, including clusters and outliers.

X4

DFD, Activity & Decision Modelling

OUT OF SYLLABUS

2014–2015 • Q7(b) CSE307 • 04/02/2016 • R021.4 **MARKS 10 + 12**

In a Video Rental System a *Customer* makes a *Video request* to the *Rent Video* process. The Rent *Video* process also receives *Video info.* from the *Video Library* data store. The process produces a *Bill* to the *Customer*, and stores the *Rental info.* into the *Rental* data store.
A *Customer* can *Return Video* by providing *Video* and *Rental info*. The process stores the *Video info.* into the *Video Library* data store and *Rental info.* into the *Rental* data store. As a result, *Return receipt* is delivered to the *Customer*. Note that Data Flow Diagram does not answer in what way and in what order the information is being used throughout a system. If this information is important and worth mentioning, consider to model it with diagrams. Finally, a *Manager* can receive *Rental report* from the *Generate Rental Report* process and the information involved is provided by the Rental data store.
Draw a context diagram and data flow diagram of the Video Rental System.

SOLUTION

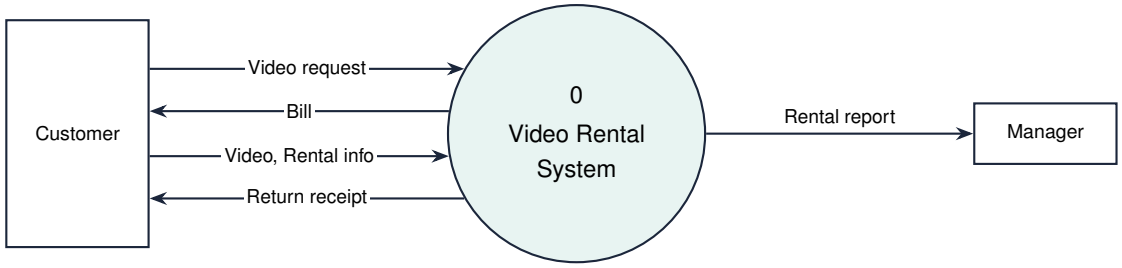
DFD notation used (Yourdon/DeMarco)

A **data flow diagram** shows how data move through a system: where they come from, which processes transform them, where they are stored and where they go. It shows *no* control logic, sequence or timing.

Element	Symbol	Meaning / rules
External entity	rectangle	Person/organisation/system outside the boundary that is a source or sink of data (Customer, Manager).
Process	circle (bubble), numbered	Transforms input data into output data; named *verb + object* ("1.0 Rent Video"); must have at least one input and one output.
Data store	open-ended box with id (D1)	Data at rest (file, table); only processes may read/write it.
Data flow	named arrow	Data in motion; every flow starts or ends at a process (no entity→entity, entity→store, store→store).

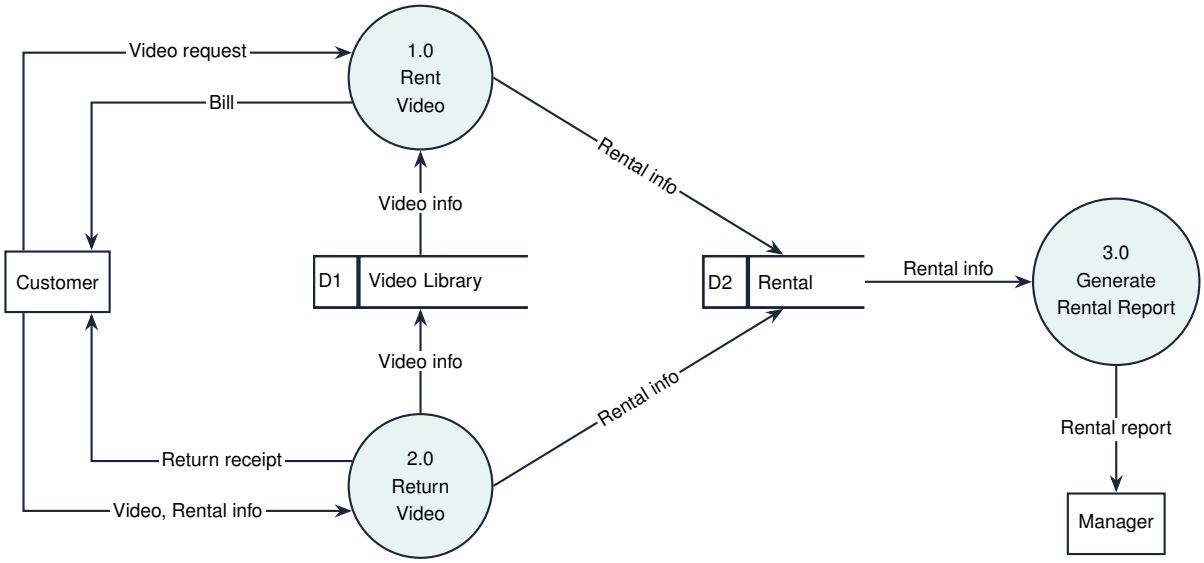
The **context diagram** (level 0) shows the whole system as one process with all external entities and the flows crossing the boundary; data stores are hidden. The **level-1 DFD** explodes that process into its major sub-processes and reveals the data stores. Both must be *balanced*: the level-1 diagram has exactly the same external inputs and outputs as the context diagram.

(a) Context diagram (10 marks)



Only the two external entities and the five boundary-crossing flows appear. The Video Library and Rental data stores are *inside* the system and therefore not shown at this level.

(b) Level-1 data flow diagram (12 marks)

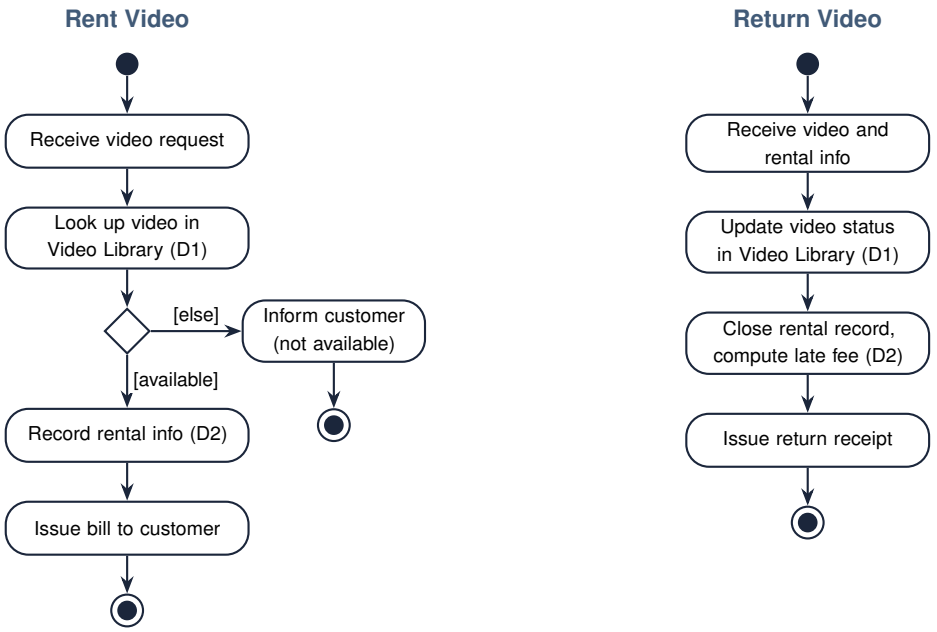


Explanation of the elements.

- **External entities**: *Customer* (source of requests and returned videos, sink of bills and receipts) and *Manager* (sink of the rental report).
- **Processes**: *1.0 Rent Video* reads the video details from D1 and records the rental in D2; *2.0 Return Video* updates both stores (video available again, rental closed); *3.0 Generate Rental Report* only reads D2.
- **Data stores**: *D1 Video Library* (title, copy no., status) and *D2 Rental* (customer, video, rent date, due date, return date, charge).
- **Balancing**: inputs Video request and Video/Rental info, outputs Bill, Return receipt and Rental report are exactly the flows of the context diagram.

(c) Modelling the order of processing

As the question notes, a DFD does not say *in what order* the data are used. When the sequence matters it is modelled with a behavioural diagram, e.g. a UML activity diagram for each process:



Answer. Context diagram: one process "0 Video Rental System" with Customer (Video request, Video+Rental info in; Bill, Return receipt out) and Manager (Rental report out). Level-1 DFD: 1.0 Rent Video, 2.0 Return Video, 3.0 Generate Rental Report with stores D1 Video Library and D2 Rental, balanced with the context diagram; activity diagrams add the processing order that a DFD cannot show.

2013–2014 • Q4(a) CSE307 • 08/08/2015 • R024 **MARKS** $17\frac{2}{3}$

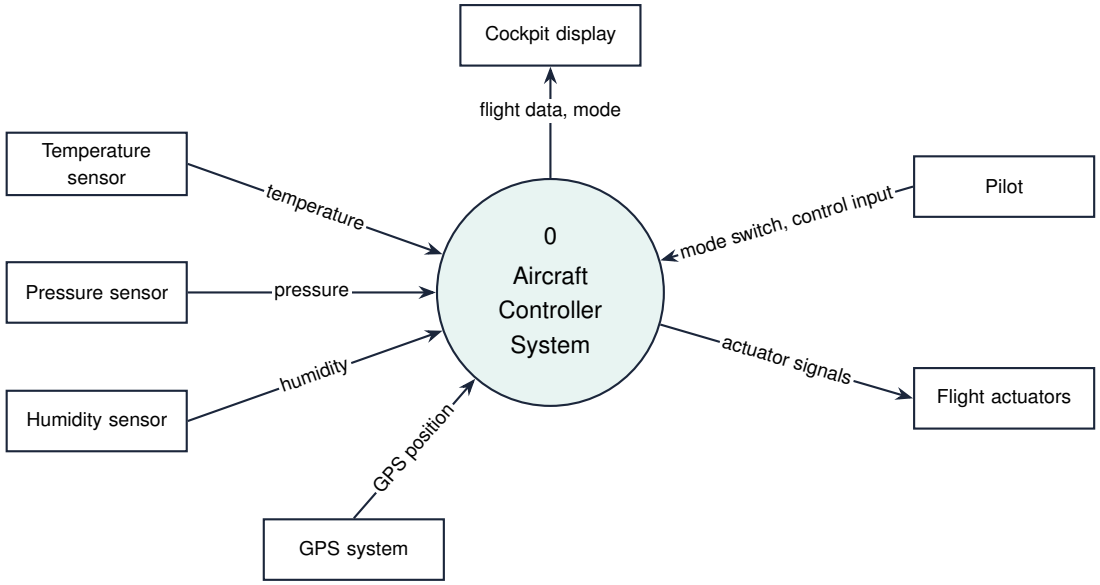
Consider an aircraft controller with the provision of automatic and manual control. These could be switched whenever pilot wants. There are temperature, pressure and humidity sensors in the aircraft and these information are displayed in the cockpit. The automatic controller decides movement based on the sensor data and GPS system output. But whenever the aircraft is under manual control the system moves based on the pilots input. Write a DFD of the system and generate the architecture of the aircraft controller system.

SOLUTION

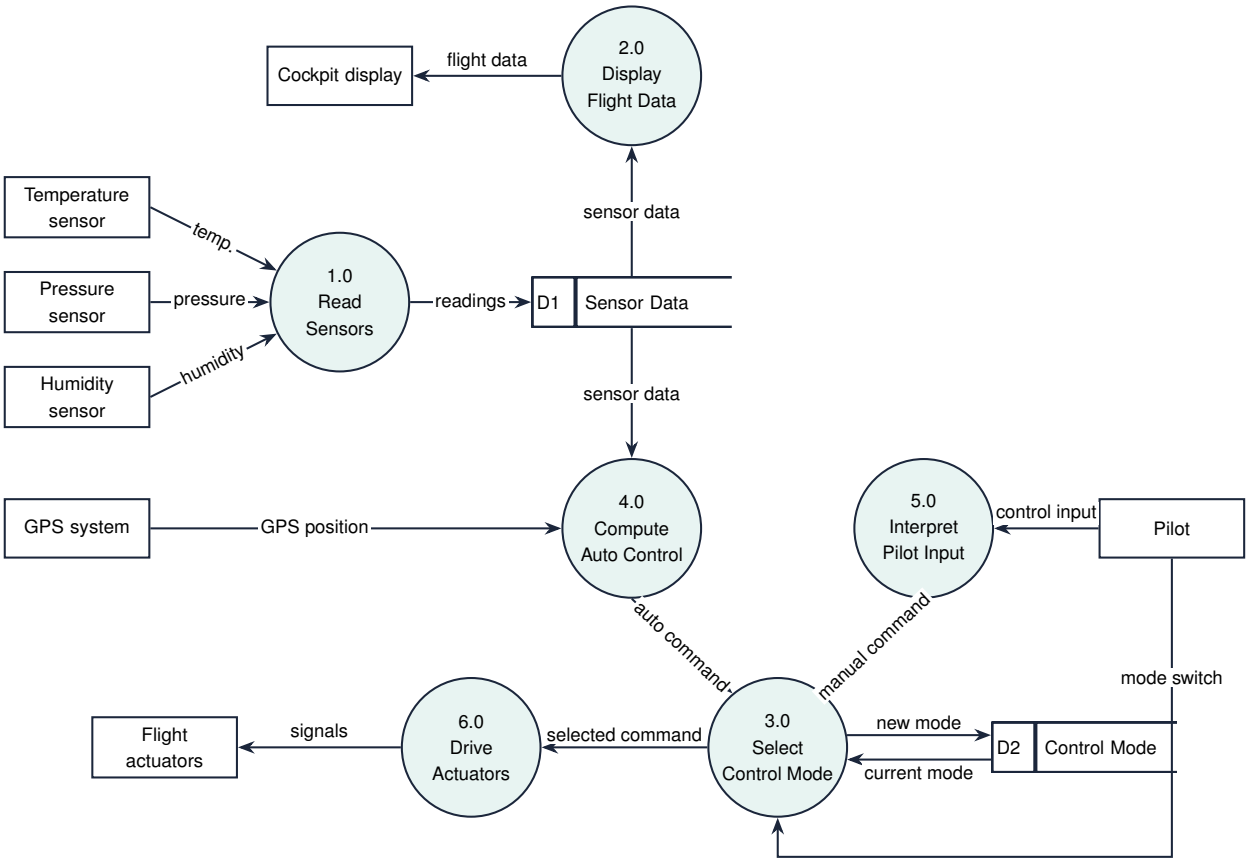
1. Identifying the DFD elements

- **External entities**: Temperature sensor, Pressure sensor, Humidity sensor, GPS system (sources); Pilot (source of mode switch and manual control input); Cockpit display and Flight actuators (control surfaces/engines) (sinks).
- **Processes**: read sensors, display flight data, compute automatic control, interpret manual input, select the active control mode, drive the actuators.
- **Data stores**: D1 Sensor Data (latest validated readings), D2 Control Mode (AUTO/MANUAL).

2. Context diagram



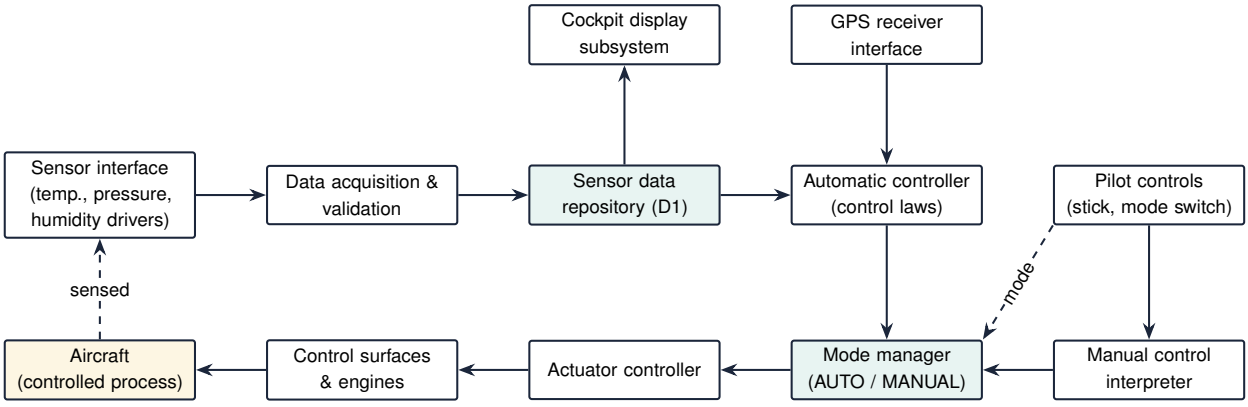
3. Level-1 DFD



How the DFD works. Process 1.0 samples the three sensors and stores validated readings in D1; 2.0 continuously shows them on the cockpit display. 4.0 computes the automatic movement command from D1 and the GPS position, while 5.0 converts the pilot's stick/throttle input into a manual command. 3.0 is the switch: it records the mode chosen by the pilot in D2 and forwards *only* the command of the active mode to 6.0, which produces the actuator signals. The pilot can switch at any time; manual mode always overrides.

4. Architecture of the aircraft controller

The system is a real-time **process-control (feedback-loop) architecture**: sensors measure the controlled process (the aircraft), a controller computes corrective commands, actuators apply them, and the effect is sensed again. Inside the controller a *shared sensor-data repository* decouples the producers (sensor subsystem) from the consumers (display, automatic controller), and a *mode manager* selects between the two control strategies.



- **Sensor subsystem** (interface + acquisition): periodic sampling, range checks, redundancy voting.
- **Repository**: single, time-stamped copy of current flight data used by display and controller.
- **Automatic controller**: control laws using sensor data and GPS (navigation).
- **Manual control interpreter**: converts pilot input into movement commands.
- **Mode manager**: implements the pilot's switch; safety rule "manual input overrides".
- **Actuator controller**: converts the selected command into actuator signals; the aircraft's response is sensed again (dashed feedback loop).

Answer. DFD: external entities = three sensors, GPS, Pilot, Cockpit display, Flight actuators; processes 1.0 Read Sensors, 2.0 Display Flight Data, 3.0 Select Control Mode, 4.0 Compute Auto Control, 5.0 Interpret Pilot Input, 6.0 Drive Actuators; stores D1 Sensor Data, D2 Control Mode. Architecture: real-time process-control (feedback) loop with sensor repository, automatic controller, manual interpreter, mode manager and actuator controller.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2013–2014 • Q8(c) CSE307 • 08/08/2015 • R028 **MARKS** $31\frac{2}{3}$

Draw context and data flow diagram for the case described below. Data flow diagrams which are impossible to interpret because of all the scratching out and changes are less likely to be treated generously.

The Blood Bank Testing Unit. This is one unit within the College Street Red Cross Blood Donor Centre. On the day following a blood donation, the Blood Bank unit tests all blood for blood type and potential viral agents. They send the results of these tests to the Processing Office (another

unit of the Centre). For each tested blood unit, they fill out a form which lists the blood unit number, the blood type, the date and the results of the test. If the tests indicate that the blood may be contaminated with a viral agent, the blood unit is destroyed. This is indicated on the test form. Blood units have a limited shelf life.
The Blood Bank receives a list every day of those units which have exceeded their shelf life. These are discarded and the list sent back to the Processing Office with a signed indication of the disposal of the units. The Blood also distributes blood to various hospitals requesting blood. Requests usually come in for specific blood types. The Blood Bank prepares refrigerated containers of these unit and distributes them to the hospital vans when they arrive to pick up their supply. The Blood Bank receives a listing for each hospital and the specific units of blood to supply to the hospital from the Processing Office. The order is printed in triplicate. When the order is filled, the lab technician signs the order and returns a copy to the Processing Office. A copy of it travels with the blood to the requesting hospital. The final copy is kept in the Blood Bank records but discarded after one year.

2010–2011 • Q3(a) CSE307 • 19/03/2012 • R044.1 **MARKS** $12\frac{2}{3}$

The Blood Bank Testing Unit. This is one unit within the College Street Red Cross Blood Donor Centre. On the day following a blood donation, the Blood Bank unit tests all blood for blood type and potential viral agents. They send the results of these tests to the Processing Office (another unit of the Centre). For each tested blood unit, they fill out a form which lists the blood unit number, the blood type, the date and the results of the test. If the tests indicate that the blood may be contaminated with a viral agent, the blood unit is destroyed. This is indicated on the test form. Blood units have a limited shelf life. The Blood Bank receives a list every day of those units which have exceeded their shelf life. These are discarded and the list sent back to the Processing Office with a signed indication of the disposal of the units.
The Blood Bank also distributes blood to various hospitals requesting blood. Requests usually come in for specific blood types. The Blood Bank prepares refrigerated containers of these units and distributes them to the hospital vans when they arrive to pick up their supply. The Blood Bank receives a listing for each hospital and the specific units of blood to supply to the hospital from the Processing Office. The order is printed in triplicate. When the order is filled, the lab technician signs the order and returns a copy to the Processing Office. A copy of it travels with the blood to the requesting hospital. The final copy is kept in the Blood Bank records but discarded after one year.
Draw a **data flow diagram** for the Blood Bank Testing Unit System.

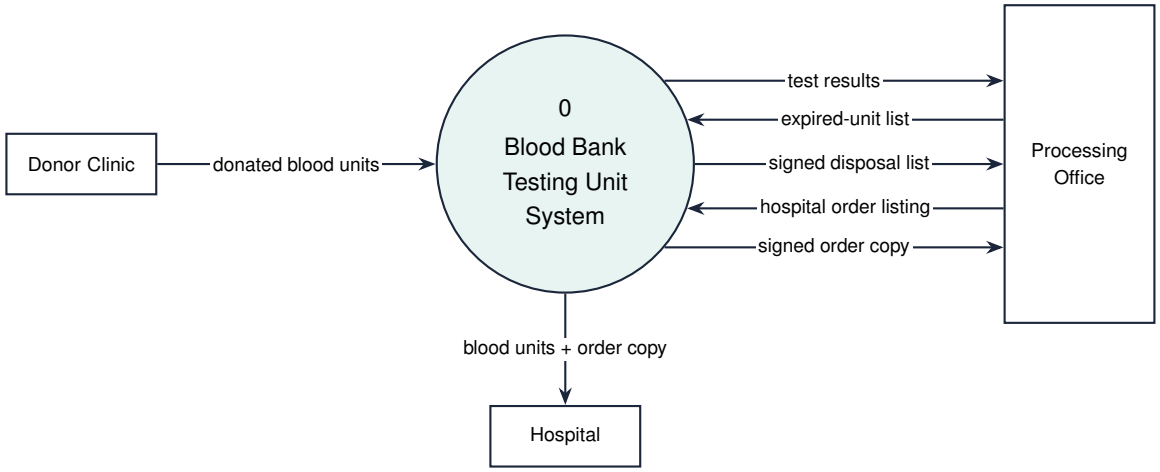
SOLUTION

NOTE The 2015 version (R028) asks for a context diagram *and* a DFD; the 2012 version (R044.1) asks only for the DFD (parts 3–4 below). Assumption: hospitals send their requests to the Processing Office, which allocates units and sends the Blood Bank a per-hospital listing (the text says the listing comes from the Processing Office); the blood donations reach the Blood Bank from the Centre's donor clinic.

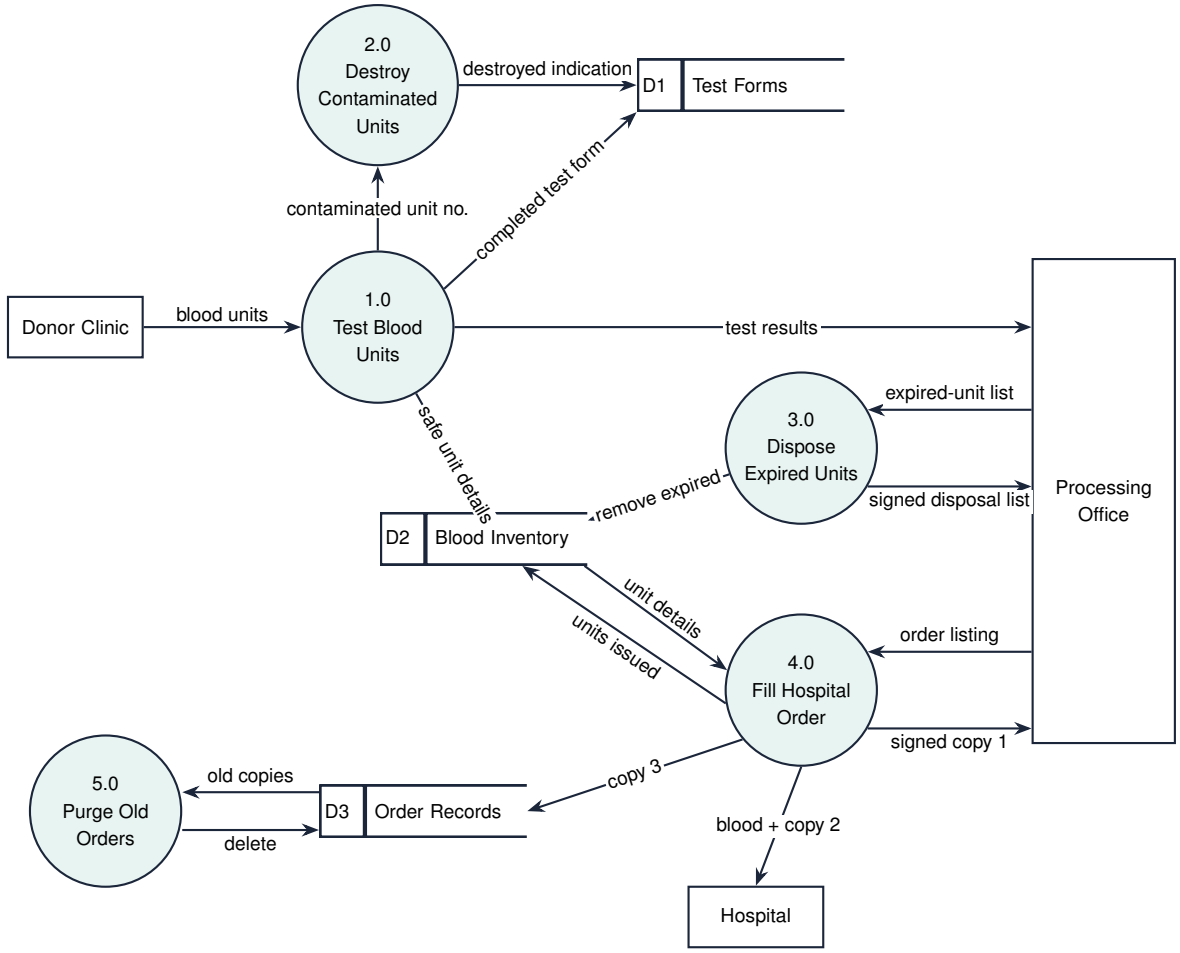
1. Analysis of the case

DFD element	Found in the narrative
External entities	Donor Clinic (supplies donated blood units), Processing Office (receives test results, sends expired-unit list and hospital order listings, receives signed lists/copies), Hospital (receives blood with its copy of the order).
Processes	test blood units; destroy contaminated units; dispose of expired units; fill hospital orders; purge order copies older than one year.
Data stores	D1 Test Forms (unit no., type, date, results, destroyed?), D2 Blood Inventory (units in stock), D3 Order Records (Blood Bank's copy of each order).

2. Context diagram



3. Level-1 (level-0 exploded) data flow diagram



4. Explanation of the processes

(i) 1.0 Test Blood Units – the day after donation each unit is typed and screened; a test form (unit number, type, date, results) is written to D1, the results go to the Processing Office, safe units are added to D2, and contaminated unit numbers are passed to 2.0.

(ii) 2.0 Destroy Contaminated Units – destroys the unit and records "destroyed" on its test form in D1.

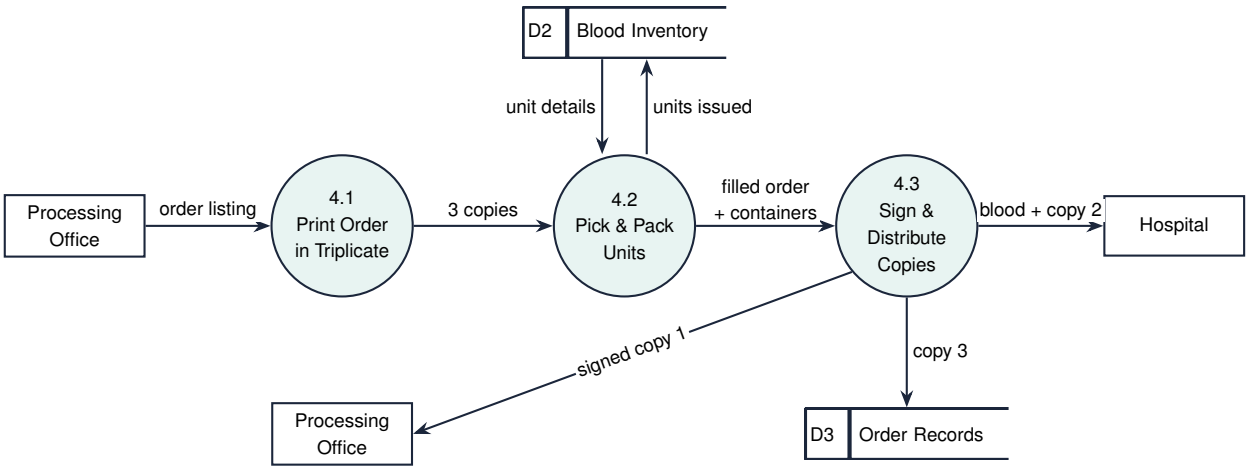
(iii) 3.0 Dispose Expired Units – triggered by the daily expired-unit list from the Processing Office; removes those units from D2 and returns the list signed as disposed.

(iv) 4.0 Fill Hospital Order – uses the per-hospital listing; prints the order in triplicate, picks the listed units from D2 into refrigerated containers, marks them issued, gives the blood with copy 2 to the hospital van, returns signed copy 1 to the Processing Office and files copy 3 in D3.

(v) 5.0 Purge Old Orders – periodically reads D3 and deletes copies older than one year.

Balancing check: inputs (blood units, expired-unit list, order listing) and outputs (test results, signed disposal list, signed copy, blood + copy to hospital) are identical to the context diagram. No flow connects two entities or two stores directly, and every process has both inputs and outputs.

5. Level-2 explosion of process 4.0 (Fill Hospital Order)



The Processing Office is drawn twice only to avoid crossing lines (a duplicated entity). The level-2 diagram is balanced with process 4.0: same inputs (order listing, D2) and outputs (signed copy, blood + copy, D3 copy, D2 update).

EXAM TIP Neatness is marked: plan the layout first, put external entities on the edges, keep stores between the processes that use them, number every process and store, name every flow, and never let lines cross where avoidable.

Answer. Context: one process "0 Blood Bank Testing Unit System" with Donor Clinic, Processing Office and Hospital. Level-1 DFD: 1.0 Test Blood Units, 2.0 Destroy Contaminated Units, 3.0 Dispose Expired Units, 4.0 Fill Hospital Order, 5.0 Purge Old Orders; stores D1 Test Forms, D2 Blood Inventory, D3 Order Records; balanced with the context diagram (process 4.0 further exploded into 4.1–4.3).

2012–2013 • Q5(a) CSE307 • 17/05/2014 • R030 **MARKS 18**

The "Sweet Treat" company is a small, independent business that sells sweets and cakes to the public. The proprietor is very keen on baking and specializes in making homemade sweets and cakes for sale in the shop. Besides, making a lot of the confectionary sold in the shop, the proprietor also buys sweets and some cakes from suppliers to increase the range of products for sale.

The proprietor keeps records of the quantities of stock he has on hand. The stock includes raw ingredients for his baking, and also the sweets and cakes he buys from suppliers. Once a week proprietor checks the stock to dispose of anything that is past its 'sell by' date. He also checks to see if any raw ingredients or any pre-made sweets and cakes need to be ordered from the suppliers. The proprietor orders supplies on a 'Cash on Delivery' basis, so all deliveries are paid for immediately after they arrive.

At the end of each day, the proprietor checks how many of each homemade item have been sold. He keeps a record of these sales, and uses this to decide how many of each cake or sweet to make for the following day.

Produce a top level data flow diagram of the "Sweet Treat" company.

SOLUTION

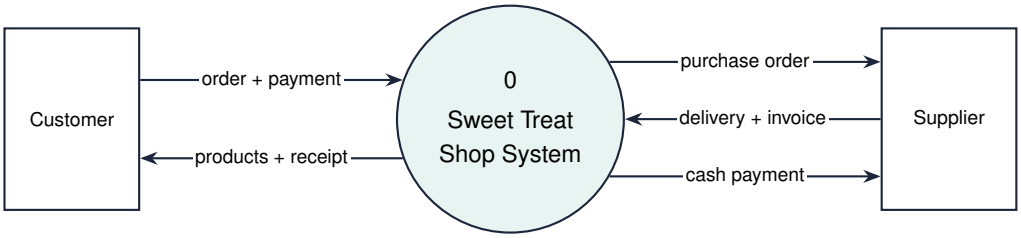
1. Identifying the elements

- **External entities**: *Customer* (buys sweets/cakes) and *Supplier* (supplies ingredients and ready-

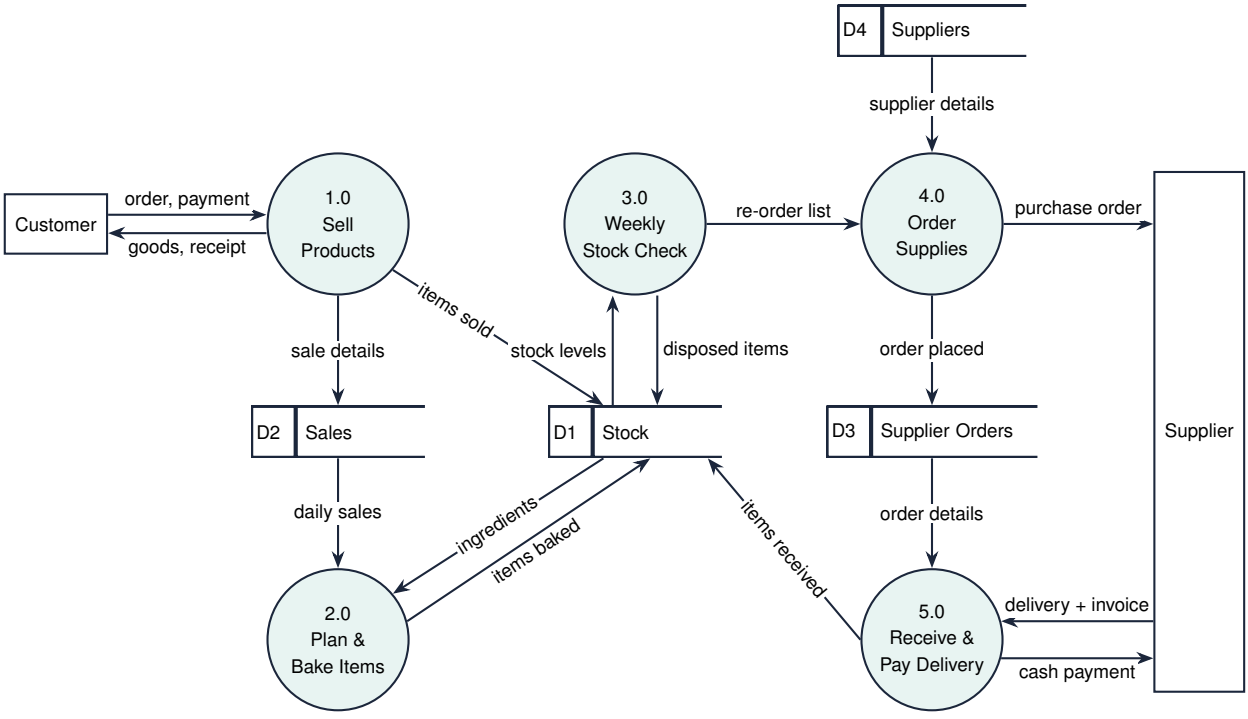
made items, receives orders and cash). The proprietor is *inside* the system: he performs its processes.

- **Processes** (one per business activity): sell products; record daily sales and plan/bake tomorrow's homemade items; weekly stock check (dispose of out-of-date items, decide re-orders); order supplies; receive and pay for deliveries (cash on delivery).
- **Data stores**: D1 Stock (ingredients, homemade and bought items, quantity, sell-by date), D2 Sales (daily sales per homemade item), D3 Supplier Orders, D4 Suppliers.

2. Context diagram (for orientation)



3. Top-level (level-1) data flow diagram



4. Explanation

(i) 1.0 Sell Products: takes the customer's order and payment, hands over goods and receipt, decreases stock (D1) and records each sale of homemade items (D2).

(ii) 2.0 Plan & Bake Items: at the end of each day reads the sales totals from D2 to decide how many of each item to make tomorrow; consumes ingredients from D1 and adds the baked items to D1.

(iii) **3.0 Weekly Stock Check**: reads stock levels and sell-by dates, removes out-of-date items, and produces a re-order list for ingredients and bought-in sweets/cakes.

(iv) **4.0 Order Supplies**: uses supplier details (D4) to send purchase orders and records them in D3.

(v) **5.0 Receive & Pay Delivery**: checks the delivery against the order (D3), pays cash on delivery, and adds the items to stock (D1).

The diagram balances with the context diagram (customer: order/payment in, goods/receipt out; supplier: purchase order and cash out, delivery/invoice in).

Answer. Top-level DFD: entities Customer and Supplier; processes 1.0 Sell Products, 2.0 Plan & Bake Items, 3.0 Weekly Stock Check, 4.0 Order Supplies, 5.0 Receive & Pay Delivery; stores D1 Stock, D2 Sales, D3 Supplier Orders, D4 Suppliers.

2012–2013 • Q8(e) CSE307 • 17/05/2014 • R034.5 MARKS $7\frac{2}{3}$

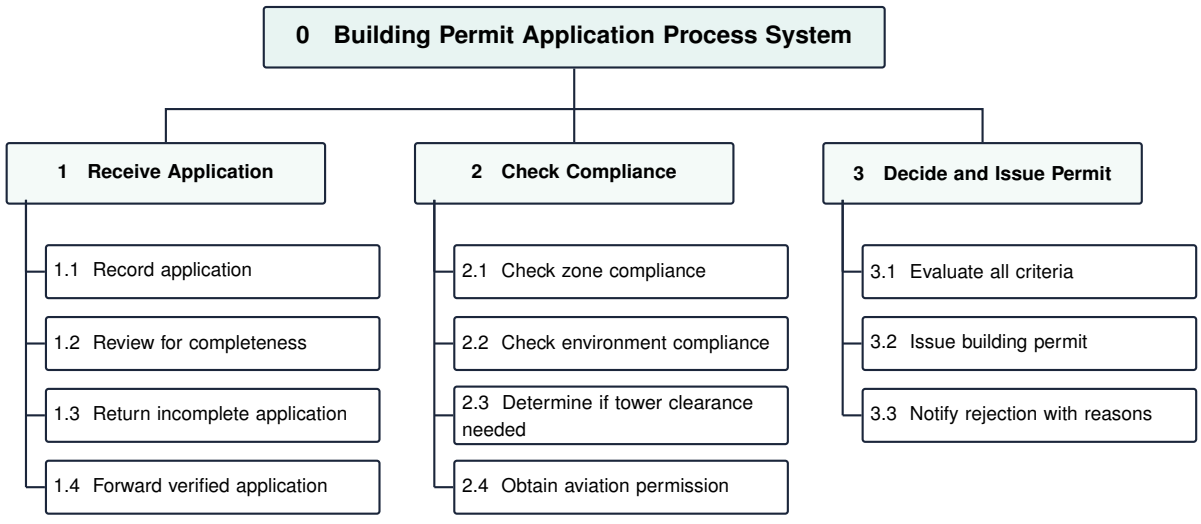
Suppose you have applied for a building permit. The authority, after receiving the application, reviews it for completeness. From this stage, the verified application is forwarded for compliance checking. The authority has to review the application for zone compliance and environment compliance. Further permission or registration for towers may be needed from aviation authorities. If all criteria are fulfilled, the authority will issue you the building permit. Now give the decomposition diagram of the "Building Permit Application Process System".

SOLUTION

What a decomposition diagram is

A **decomposition diagram** (functional hierarchy chart) shows the top-down breakdown of a system into functions and sub-functions. It is drawn as a tree: the root is the whole system, each parent is divided into two or more children, and the numbering (1, 1.1, 1.2, . . .) is the same as that later used for the DFD processes. It shows *structure only*: no data flows, no order, no conditions. Names are *verb + object*.

Decomposition diagram



- **1 Receive Application**: the authority logs the application, checks it for completeness, returns incomplete ones and forwards verified ones.
- **2 Check Compliance**: zone and environment compliance are reviewed; for towers the need for aviation-authority permission/registration is determined and obtained.
- **3 Decide and Issue Permit**: if all criteria are fulfilled the permit is issued, otherwise the applicant is notified.

Answer. Root "0 Building Permit Application Process System" decomposed into 1 Receive Application (record, review completeness, return/forward), 2 Check Compliance (zone, environment, tower check, aviation permission) and 3 Decide and Issue Permit (evaluate, issue, notify rejection).

2010–2011 • Q1(c) CSE307 • 19/03/2012 • R042.3 **MARKS 10**

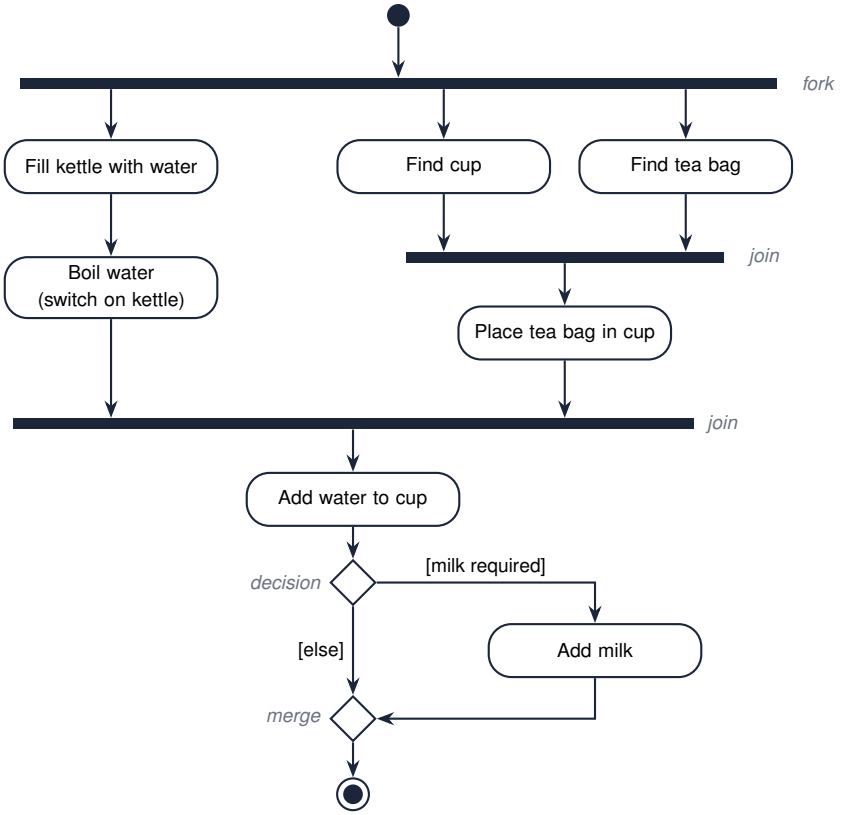
Draw an **Activity diagram** that represents the making of a cup of tea. The initial three activities are Fill kettle with water, Find cup and Find tea bag and they may be performed in parallel. When the Find cup and Find tea bag are completed the activity Place tea bag in cup can start. The kettle must have boiled and the tea bag must have been placed in the cup before the activity Add water to cup can begin. If milk is required then activity Add milk should be performed.

SOLUTION

Activity-diagram elements used

- **Initial node** (filled circle) and **final node** (bull's-eye): start and end of the flow.
- **Action** (rounded rectangle): one step of work.
- **Fork** (thick bar, one in / many out): following actions may run *in parallel*.
- **Join** (thick bar, many in / one out): waits until *all* incoming flows have finished (synchronisation).
- **Decision** (diamond with guarded outgoing flows) and **merge** (diamond with several incoming flows): optional behaviour.

Activity diagram



Modelling decisions

(i) The first **fork** starts the three independent activities *Fill kettle*, *Find cup* and *Find tea bag* concurrently.

(ii) "The kettle must have boiled" is modelled as the action *Boil water* following *Fill kettle*, so the boiling is an explicit completion that can be synchronised.

(iii) The first **join** waits for both *Find cup* and *Find tea bag*; only then can *Place tea bag in cup* start.

(iv) The second **join** waits for *Boil water* and *Place tea bag in cup*; only then *Add water to cup* is enabled.

(v) A **decision** with guards `[milk required]` / `[else]` makes *Add milk* optional; a **merge** brings the two paths together before the final node.

Answer. Initial → fork {Fill kettle → Boil water ‖ Find cup ‖ Find tea bag}; join(cup, tea bag) → Place tea bag in cup; join(boiled, tea bag placed) → Add water to cup → decision [milk required] Add milk / [else] → merge → final.

2010–2011 • Q1(d) CSE307 • 19/03/2012 • R042.4 **MARKS 4**

Assume that A, B, C and Boolean variables, X, Y, Z are outcomes. Propose a decision table for the following procedure:

```
If A then if B then X
     else if C then Y else Z endIf endIf
else if C then X else Z endIf
endIf
```

Make sure your final table is in simplified form, i.e., does not have redundant rows or columns.

SOLUTION

Concept

A **decision table** lists *conditions* (upper part) and *actions* (lower part); each column is a *rule*: one combination of condition values and the action(s) taken. With n Boolean conditions the complete table has 2^n rules; it is then *simplified* by merging rules that differ in one condition but lead to the same action (that condition becomes "–", *don't care*).

Step 1 – reading the procedure

- A true, B true $\Rightarrow$ X (C irrelevant).
- A true, B false: C true $\Rightarrow$ Y, C false $\Rightarrow$ Z.
- A false (B irrelevant): C true $\Rightarrow$ X, C false $\Rightarrow$ Z.

Step 2 – complete table ($2^3 = 8$ rules)

Rule	1	2	3	4	5	6	7	8
A	Y	Y	Y	Y	N	N	N	N
B	Y	Y	N	N	Y	Y	N	N
C	Y	N	Y	N	Y	N	Y	N
X	✓	✓			✓		✓	
Y			✓					
Z				✓		✓		✓

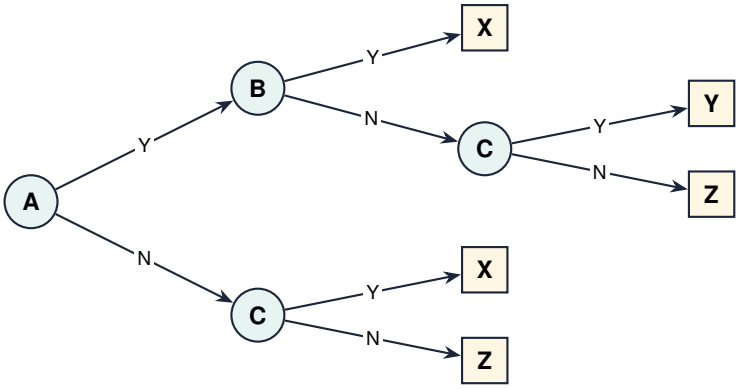
Rules 1 and 2 differ only in C and give X $\Rightarrow$ merge (C = –). Rules 5 and 7 differ only in B and give X; rules 6 and 8 differ only in B and give Z $\Rightarrow$ merge (B = –).

Step 3 – simplified table (5 rules)

Rule	1	2	3	4	5
A	Y	Y	Y	N	N
B	Y	N	N	–	–
C	–	Y	N	Y	N
X	✓			✓	
Y		✓			
Z			✓		✓

Check: the rules are mutually exclusive and cover all 8 combinations ($2+1+1+2+2=8$). No further merging is possible: rules 3 and 5 (both Z) differ in A *and* B, and merging them would also cover A=Y, B=Y, C=N, which must give X. Likewise rules 1 and 4 (both X) differ in A, B and C. No row is redundant: every condition decides the outcome in some rule, and each action occurs.

Equivalent decision tree



Answer. Simplified decision table (5 rules): (A=Y,B=Y,C=–)→X; (Y,N,Y)→Y; (Y,N,N)→Z; (N,–,Y)→X; (N,–,N)→Z.

X5

Project Planning, Estimation & Risk

OUT OF SYLLABUS

2023–2024 • Q2(b) CSE213 • 10/05/2026 • M001 MARKS 15

You and your friend are co-founders of an online education platform. After initial success, the platform attracted several investors. With this funding, you planned to introduce a major AI-based feature that users have been demanding for a long time, especially as some competitors are already moving in the same direction. The feature is expected to significantly boost user engagement and growth.
However, due to political and economic unrest, you anticipate a potential financial crisis. Therefore, you want to cancel or postpone the feature to conserve resources. On the other hand, your partner believes that launching the feature could attract more users and generate revenue to offset crisis.
Identify the risks associated with both strategies. Categorize these risks based on their nature and analyze their potential impact on the platform's future.

SOLUTION

Concepts used

A **risk** is a potential problem that has two characteristics: *uncertainty* (it may or may not happen) and *loss* (if it happens, there are unwanted consequences). Pressman classifies risks by their *nature* into three broad categories:

- **Project risks** – threaten the project plan: budget, schedule, staffing, resources, stakeholders.
- **Technical risks** – threaten the quality and timeliness of the product: design, implementation, technology, maintenance problems, technical uncertainty.
- **Business risks** – threaten the viability of the product/company. Sub-types: *market risk* (nobody wants it / competitors win), *strategic risk* (no longer fits the business strategy), *sales risk* (sales force cannot sell it), *management risk* (loss of senior management / investor support), *budget risk* (loss of budget or personnel commitment).

Each risk is then analysed by its *probability* and *impact* (Pressman scale: 1 = catastrophic, 2 = critical, 3 = marginal, 4 = negligible); risk exposure $RE = P \times C$.

Strategy A – cancel or postpone the AI feature (your view)

Risk	Category	Prob.	Impact	Potential impact on the platform
Competitors release AI features first; users migrate	Business (market)	High	1–2	Permanent loss of market share; hard to win users back later.
Long-demanded feature never arrives; engagement and renewals fall	Business (sales)	High	2	Revenue drops even while costs are cut – the crisis may hurt more.
Investors who funded the AI roadmap lose confidence	Business (management / strategic)	Medium	2	Future funding rounds and valuation suffer; investors may demand control.
AI engineers leave because their work is shelved	Project (staff)	Medium	3	Knowledge loss; re-hiring later is slow and costly.
Feature becomes more expensive later (data not collected, technology moves on, rework)	Technical / Project (budget)	Medium	3	Delayed launch costs more and still arrives late.

Strategy B – launch the AI feature now (partner's view)

Risk	Category	Prob.	Impact	Potential impact on the platform
Development + cloud/GPU running costs drain the cash reserve during the crisis	Business (budget) / Project (cost)	High	1	Cash runway ends; the platform itself may shut down – catastrophic.
Crisis lowers users' ability to pay; projected revenue does not appear	Business (sales / market)	Medium–High	2	Investment is not recovered; losses widen.
AI feature is complex: schedule slips, cost overruns	Project (schedule, budget)	Medium	2	Late, over-budget launch at the worst time.
Poor model quality (wrong answers, bias), scalability problems	Technical	Medium	2	Reputation damage; users lose trust in the whole platform.
Data-privacy / regulatory problems with student data	Business (legal) / Technical	Low–Medium	2	Fines, forced withdrawal of the feature.
Currency fluctuation raises the price of foreign AI APIs / cloud services	Project (budget)	Medium	3	Operating cost becomes unpredictable.

Analysis of the impact on the platform's future

- **Strategy A** has *low short-term exposure* (cash is preserved, survival during the crisis is likely) but *high long-term strategic exposure*: market and sales risks are almost certain because competitors are already moving; the platform may survive the crisis but emerge irrelevant.
- **Strategy B** has *high short-term exposure*: a single catastrophic budget risk (impact 1) can end the company, while its benefit (new revenue) is itself uncertain during a recession. Its long-term strategic position, if it survives, is strong.

- Both strategies therefore trade a *business (market/strategic) risk* against a *business (budget) + project risk*. The decision is not "feature or no feature" but how to cap the exposure.

Recommended course of action (risk mitigation)

(i) **Phased / MVP launch**: release a small, high-value AI feature first (e.g. AI doubt-solving for one course) using pay-per-use APIs, so cost scales with usage; defer the expensive parts.

(ii) **Ring-fence a contingency reserve** (e.g. 12 months of operating cost) that the feature may not touch.

(iii) **Monitor triggers** (RMMM): monthly burn rate, paid conversions, API cost per active user; if a trigger crosses the threshold, freeze further AI work (contingency plan).

(iv) **Monetise the feature** (premium tier) so that it pays for itself, and keep investors informed to retain their support.

Answer. Postponing mainly creates *business* risks (market, sales, strategic/investor, staff) whose damage is long-term; launching mainly creates *budget/project* and *technical* risks whose damage is immediate and possibly catastrophic. A phased, monitored MVP launch with a protected cash reserve keeps both exposures acceptable.

2023–2024 • Q3(a) CSE213 • 10/05/2026 • M002 **MARKS 20**

Suppose, you are managing a software project named "SmartHome" that consists of three major tasks. The planned progress and current status of all tasks are shown in Figure 1. At the end of Week 3, you are analyzing the schedule and budget of the project using Earned Value Management. You are impressed that the second task has already been completed by Week 3, i.e., ahead of the planned schedule. However, the status of the third task remains unknown as it was assigned to a freelancer who has not provided any clear updates yet and this made you anxious.

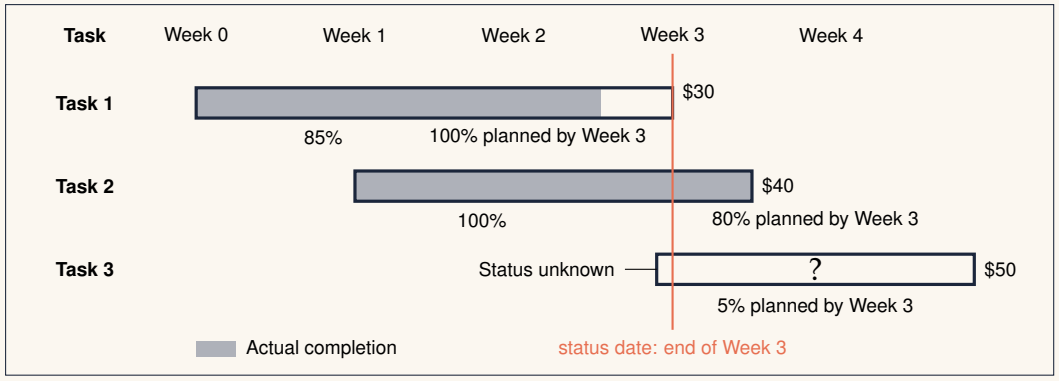


Figure 1: Gantt Chart of SmartHome

To calm you down, your project officer claims: "Whatever the progress of the third task is, the project is ahead of schedule." However, you could not relax as after a bit of investigation about actual cost, you find out that: "No matter what the progress of the third task is, the project will exceed the total budget."

(i) Verify whether the project officer's claim about the schedule is correct.

(ii) Based on your findings, find the lower bound of actual cost (AC).

SOLUTION

EVM quantities used

- **BAC** (budget at completion) = total planned budget of all tasks.
- **PV** (planned value, BCWS) = budgeted cost of the work *scheduled* up to the status date = $\sum$(planned % by now $\times$ task budget).
- **EV** (earned value, BCWP) = budgeted cost of the work *actually performed* = $\sum$(actual % complete $\times$ task budget).
- **AC** (actual cost, ACWP) = money actually spent up to the status date.
- $SV = EV - PV$, $SPI = EV/PV$ (schedule: > 0 / > 1 means ahead); $CV = EV - AC$, $CPI = EV/AC$ (cost: < 0 / < 1 means over budget); $EAC = BAC/CPI$.

Let $p \in [0,1]$ be the unknown fraction of Task 3 that is complete.

Reading the Gantt chart (status date = end of Week 3)

Task	Budget	Planned %	Actual %	PV	EV
Task 1	\$30	100%	85%	$1.00 \times 30 = \$30$	$0.85 \times 30 = \$25.5$
Task 2	\$40	80%	100%	$0.80 \times 40 = \$32$	$1.00 \times 40 = \$40$
Task 3	\$50	5%	p (unknown)	$0.05 \times 50 = \$2.5$	$50p$
Total	BAC = \$120			PV = \$64.5	EV = $\$(65.5 + 50p)$

(i) Is the project ahead of schedule whatever Task 3's progress?

$$SV = EV - PV = (65.5 + 50p) - 64.5 = 1 + 50p.$$

Since $p \geq 0$, the smallest possible value (Task 3 not even started, $p = 0$) is

$$SV_{\min} = 1 + 0 = +\$1 > 0, \qquad SPI_{\min} = \frac{65.5}{64.5} = 1.0155 > 1.$$

For any $p \in [0,1]$: $SV \in [\$1,\ \$51]$ and $SPI \in [1.016,\ 1.791]$ – always positive / above 1. The \$8 that Task 2 is ahead $(40 - 32)$ more than compensates for the \$4.5 that Task 1 is behind $(25.5 - 30)$ and for the \$2.5 of Task 3 that was planned by Week 3.

Answer. The project officer is **correct**: even in the worst case ($p = 0$), $SV = +\$1$ and $SPI \approx 1.02 > 1$, so the project is ahead of schedule regardless of Task 3.

(ii) Lower bound of the actual cost

The project "will exceed the total budget" means the forecast cost at completion is larger than the budget:

$$EAC = \frac{BAC}{CPI} = BAC \cdot \frac{AC}{EV} > BAC \iff \frac{AC}{EV} > 1 \iff AC > EV \quad (CPI < 1).$$

This must hold *no matter what* p is, so AC must exceed the *largest* possible EV, which occurs when Task 3 is fully complete ($p = 1$):

$$EV_{\max} = 65.5 + 50(1) = \$115.5.$$

Hence

$$AC > EV_{\max} = \$115.5.$$

Check: with AC just above \$115.5, even the best case $p = 1$ gives $CPI = 115.5/AC < 1$, so $EAC > \$120$; for any smaller p, CPI is lower still. If $AC \leq 115.5$, then for $p = 1$ the project would be on/under budget, contradicting the finding.

NOTE "Exceed the total budget" is read as the *forecast* (EAC > BAC), which is the standard EVM interpretation. The same bound results from the alternative forecast $EAC = AC + (BAC - EV) > BAC \iff AC > EV$. (If it were read as "AC has already exceeded BAC", the bound would simply be $AC > \$120$, independent of the Gantt chart – which would make the question trivial.)

Answer. $AC > \$115.5$ (the lower bound of the actual cost is \$115.5, exclusive), because the cost overrun ($CPI < 1$) must hold even when Task 3 is 100% complete and $EV = \$115.5$.

2023–2024 • Q4(b) CSE213 • 10/05/2026 • F002.2 **MARKS 15**

[Define code smell, tech-debt and code refactoring.] Differentiate between proactive and reactive risk management strategies with examples.

SOLUTION

NOTE This is the second half of Q4(b). The first half (definitions of code smell, technical debt and refactoring) is solved in the Refactoring chapter under the same ID; the 15 marks cover both halves (roughly 7 + 8).

Background: what a risk strategy is

A **risk** is an uncertain future event that, if it occurs, causes a loss to the project (late delivery, cost overrun, poor quality). Every risk has two properties: *uncertainty* (probability P) and *loss* (impact L); **risk exposure** $RE = P \times L$. A risk management *strategy* is the team's overall attitude to such events — wait for them, or plan for them.

Reactive risk strategy

The team does nothing about a risk until it actually becomes a problem; then it reacts. Pressman calls this the "Indiana Jones school of risk management" — or, more commonly, *fire-fighting* mode.

- At best, the team *monitors* the project for likely risks and sets aside resources to handle them if they occur; at worst it does nothing until something goes wrong.
- Resources are then moved in a hurry to fix the problem; if the fix fails, the project is in real jeopardy (crisis management).

Example. Two senior developers resign in the middle of a sprint. Only then does the manager start hiring and asking others to read their undocumented code; the release slips by a month.

Proactive risk strategy

Risk management starts *before* technical work begins and continues throughout the project.

(i) **Identify** potential risks (project, technical, business risks; checklists).

(ii) **Analyse**: estimate probability and impact, compute risk exposure.

(iii) **Prioritise** (risk table, cut-off line) to focus on the most dangerous risks.

(iv) **Plan** for each: *avoidance/mitigation*, *monitoring*, and a *contingency* plan — the RMMM plan (Risk Mitigation, Monitoring and Management).

(v) **Monitor and review** continuously; the primary objective is to *avoid* risk, and to have a contingency plan ready for risks that cannot be avoided.

Example. At the start of the project the risk "high staff turnover" is rated $P = 0.7$, high impact. Mitigation: pair programming and code reviews so knowledge is shared, mandatory documentation, a backup person assigned to every critical module. Monitoring: watch staff morale and market offers. Contingency: when a developer resigns, the backup takes over immediately with little schedule impact.

Comparison

Aspect	Reactive	Proactive
When action starts	after the risk turns into a problem	before work begins; continuous
Approach	fire-fighting, crisis management	identify → analyse → prioritise → plan (RMMM) → monitor
Main goal	recover from damage	avoid the risk; minimise impact if it occurs
Planning artefact	none (at most a reserve budget)	risk table, RMMM plan, contingency plans
Cost	low up front, very high when risks hit	moderate up front, much lower overall
Effect on schedule	surprises, slips, possible project failure	predictable, controlled
Example	server crashes before demo → scramble to rebuild it	daily backups and a standby server planned in advance

Answer. Reactive strategy = wait until a risk becomes a problem, then fire-fight (e.g. hire only after key staff leave). Proactive strategy = identify, assess and prioritise risks early and prepare mitigation, monitoring and contingency plans (RMMM) (e.g. knowledge-sharing and backups planned for the staff-turnover risk). Proactive is the recommended approach.

2023–2024 • Q6(c), Q7(a) CSE213 • 09/02/2025 • M004 MARKS 20 + 15

6(c) A project manager reviews the progress of a project and discovers that for every dollar spent, he is getting 20% more of its worth work being done. The total project budget is $1,000. Encouraged by this efficiency, the manager is considering adding new features to the project. He has two options: Feature A would cost an additional $200, while Feature B would cost $100. Based on this information, determine which feature the manager can afford.
Additionally, the manager finds that the actual cost incurred up to this point is $500. However, he did not review the planned value i.e., the value that was targeted to achieve at this stage, confidently assuming that the project is ahead of schedule. Verify whether his assumption is correct, providing the necessary calculations and reasoning.

7(a) A global travel agency is planning to introduce a new AI-based feature aimed at enhancing customer experience. The feature will analyze customers' browsing and social media history to tailor recommendations. The agency is investing a significant portion of its budget into the development and rollout of this feature. It expects that it could substantially increase its customer

base and boost sales. However, there are concerns about privacy, as many customers may be reluctant to share their browsing and social media data, which could lead to lower adoption rates. In addition to this, the agency is facing uncertainty due to the onset of a new epidemic that threatens their operations in several countries. If the epidemic becomes severe, government-imposed travel restrictions could force the agency to halt services in affected regions, resulting in a loss of revenue.
What types of risks can you identify in the scenario? For each risk, outline the appropriate risk management strategies that should be adopted and classify them as either proactive or reactive.

SOLUTION

6(c) Earned value analysis

Step 1 – Translate "20% more work per dollar" into CPI

The cost performance index is the value of work earned per dollar actually spent:

$$CPI = \frac{EV}{AC} = \frac{\$1.20 \text{ of work}}{\$1.00 \text{ spent}} = 1.2.$$

Step 2 – Forecast the cost at completion

$$EAC = \frac{BAC}{CPI} = \frac{\$1{,}000}{1.2} = \$833.33, \qquad VAC = BAC - EAC = \$1{,}000 - \$833.33 = \$166.67.$$

If the current efficiency continues, the original scope will be finished for \$833.33, leaving \$166.67 of the \$1,000 budget unspent – this is all the money available for new features.

Step 3 – Which feature fits?

Option	Extra cost	New total forecast	Within \$1,000?
Feature A	\$200	$833.33 + 200 = \$1{,}033.33$	× exceeds budget by \$33.33
Feature B	\$100	$833.33 + 100 = \$933.33$	✓ \$66.67 still in reserve

NOTE If the \$200/\$100 are read as *budgeted* (planned) values that will also be produced at $CPI = 1.2$, Feature A would cost $200/1.2 = \$166.67$, giving $EAC = 1{,}200/1.2 = \$1{,}000$ – exactly the budget, with zero contingency. Even in this reading Feature B is the only option that leaves a safety margin.

Answer. The manager can afford **Feature B** only (forecast \$933.33 $\leq$ \$1,000); Feature A would push the forecast to \$1,033.33.

Step 4 – Is the project ahead of schedule?

With $AC = \$500$ and $CPI = 1.2$:

$$EV = CPI \times AC = 1.2 \times \$500 = \$600 \quad (= 60\% \text{ of BAC earned}).$$

Schedule status needs the planned value: $SV = EV - PV = 600 - PV$, $SPI = EV/PV = 600/PV$.

If PV were…	SV	SPI	Status
\$550	+\$50	1.09	ahead of schedule
\$600	\$0	1.00	exactly on schedule
\$700	−\$100	0.86	behind schedule

The CPI measures *cost efficiency* only (how much work per dollar); it says nothing about *how much work should have been done by now*. A project can be cheap per unit of work and still be late.

Answer. The assumption is **not justified**: $EV = \$600$, so the project is ahead only if $PV < \$600$ (i.e. less than 60% of the work was planned by now). Without reviewing PV, the schedule status cannot be concluded.

7(a) Risks of the AI recommendation feature and the epidemic

Proactive vs. reactive strategies

Reactive strategies wait until a risk becomes a problem and then respond ("fire-fighting", crisis management, contingency resources). **Proactive** strategies identify and assess risks *before* work begins, rank them, and avoid, mitigate, transfer or monitor them (RMMM plan). Standard responses: *avoid*, *mitigate (reduce)*, *transfer* (insurance, contracts) and *accept* (with a contingency plan).

Risks identified and their management

Risk	Type	Management strategy	Class
Privacy breach / non-compliance with data-protection laws (GDPR etc.)	Business (legal), technical	Privacy-by-design, explicit opt-in consent, anonymisation, encryption, legal review before launch	Proactive (avoid / mitigate)
Low adoption: customers refuse to share browsing/social data	Business (market / sales)	Make sharing optional with clear benefits, fall back to non-personal recommendations, pilot / A-B test in one market first	Proactive (mitigate)
Large share of budget sunk into an uncertain feature	Business (budget / strategic), project	Phased investment with go/no-go gates, MVP first, track ROI	Proactive (mitigate)
Recommendation quality poor, social-media API changes, integration delays	Technical, project (schedule)	Prototype early, use proven ML libraries, abstraction layer over third-party APIs, schedule buffer	Proactive (mitigate)
Epidemic forces travel restrictions; services halted, revenue lost	Business (external / market)	Business-continuity plan, diversify (domestic trips, flexible bookings, travel insurance products), insurance cover	Proactive (plan, transfer)
Sudden halt in affected regions after restrictions are announced	Business (operational)	Activate contingency: suspend sales there, refunds/rebooking, cut variable cost, redeploy staff remotely	Reactive (contingency)
Staff fall ill / offices close during epidemic	Project (staff)	Remote-work infrastructure ready in advance (proactive); temporary staffing when it happens	Proactive + reactive

Answer. Risks: privacy/legal, market-adoption, budget/strategic, technical/schedule, and external epidemic (revenue loss, staff loss). Privacy, adoption, budget and technical risks should be handled *proactively* (avoid/mitigate before launch); the epidemic needs a proactive continuity/insurance plan plus a *reactive* contingency response once restrictions occur.

2023–2024 • Q8(b) CSE213 • 09/02/2025 • M005 **MARKS 7**

Compare the roles of a manager and a leader. Provide a scenario where it is more beneficial to use a management approach rather than a leadership approach.

SOLUTION

Manager vs. leader

Aspect	Manager	Leader
Focus	plans, budgets, schedules, controls – "doing things right"	vision and direction – "doing the right things"
Source of power	formal authority of the position	influence, trust and credibility (can exist without a title)
Approach to people	assigns tasks, monitors performance, enforces process	motivates, inspires, coaches, builds commitment
Attitude to change	maintains stability and order, minimises risk	initiates change, takes calculated risks, innovates
Time horizon	short/medium term: deliver this project on time	long term: where the team and product should go
Measures	schedule, cost, quality metrics (e.g. EVM, defects)	team engagement, growth, innovation

Effective software project managers need both, but the emphasis should depend on the situation.

Scenario favouring a management approach

A bank must update its core-banking software to meet a *central-bank regulatory change* with a fixed legal deadline. Requirements are fully specified by the regulator, the team is large and includes many junior and outsourced developers, and every change must pass audit. Here success depends on *control*: a detailed plan and work-breakdown structure, strict schedule and budget tracking (critical path, EVM), formal change control, documented procedures and compliance checks. Inspiration and visionary change add little – deviation from the plan is itself a risk – so a management approach is more beneficial. (Other examples: crisis recovery of a production outage, maintenance of a safety-critical system.)

Answer. Managers plan, organise and control with positional authority to achieve stability; leaders set vision, inspire and drive change through influence. In a fixed-scope, deadline-driven, highly regulated project (e.g. regulatory banking update), a management approach is better.

2022–2023 • Q7(a)–(c) CSE307 • 05/05/2024 • M006 **MARKS 10 + 10 + 15**

(a) What are the two strategies for dealing with risks in a project? Briefly describe the Risk Table and Risk Information Sheet.
(b) Imagine that you are a project manager organizing and leading different projects in a software company. Some information on one such project, which has been running for the past one month, is provided in the following table.

Task	Total PV	PV after one month	EV after one month
A	$70	$50	$40
B	$40	$40	$35
C	$30	$20	$30
D	$60	$35	$35
E	$50	$0	$0

Here, PV and EV stand for planned value and earned value, respectively. The company expended $140 on this project during the past month. Now, given the information, compute the schedule performance index (SPI), cost performance index (CPI), estimated cost at completion (EAC), schedule variance (SV), and cost variance (CV) of this project after one month.

(c) What do you mean by critical path and slack in project scheduling? Suppose you have been assigned to analyze the schedule of the software project you are currently working on by the project manager. The relevant information about the project is provided in the following table. Now, determine the strict deadline (in days), slack of different tasks (in days), and critical path of this project by applying the Critical Path Method (CPM).

Task	Predecessor(s)	Duration (in weeks)
A	–	14
B	A	5
C	A	5
D	A	10
E	B, C, D	31
F	E	7
G	–	14
H	G	21
I	–	14
J	–	9
K	F, H	21

SOLUTION

(a) Reactive vs. proactive risk strategies

- **Reactive strategy** ("Indiana Jones school"): the team does nothing about a risk until it becomes a real problem, then rushes to fix it (fire-fighting, crisis management). At best it only plans extra resources in anticipation; often the project is already in trouble when action starts.
- **Proactive strategy**: begins long before technical work. Potential risks are *identified*, their probability and impact *assessed*, they are *ranked*, and a plan is made to *avoid* them, or to *monitor* them and have a *contingency plan* ready (RMMM: Risk Mitigation, Monitoring and Management).

Risk Table (risk projection tool). A table with one row per identified risk and columns

Risks	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	ref. to plan
Customer will change requirements	PS	80%	2	
Staff turnover will be high	ST	60%	2	
Funding will be lost	CU	40%	1	
Lack of training on tools	DE	80%	3	

Category codes: PS project size, BU business impact, CU customer characteristics, PR process, TE technology, DE development environment, ST staff size/experience. Impact: 1 catastrophic, 2 critical, 3 marginal, 4 negligible. The table is *sorted by probability and then impact*; a horizontal **cut-off line** is drawn – only risks above the line receive further management attention, and the last column points to their RMMM plan.

Risk Information Sheet (RIS). One form (or database record) per important risk, used instead of a long RMMM document. Fields: Risk ID, date, probability, impact; *description*; *refinement/context* (sub-conditions); *mitigation/monitoring* steps; *management/contingency plan* and *trigger*; current status; originator; person assigned. It is updated throughout the project as the risk evolves.

(b) Earned value analysis after one month

$$BAC = 70 + 40 + 30 + 60 + 50 = \$250$$
$$PV = 50 + 40 + 20 + 35 + 0 = \$145$$
$$EV = 40 + 35 + 30 + 35 + 0 = \$140$$
$$AC = \$140 \text{ (amount expended)}$$

Metric	Formula	Value	Meaning
SPI	$EV/PV = 140/145$	0.966	96.6% of planned work done: slightly behind schedule
CPI	$EV/AC = 140/140$	1.00	exactly \$1 of work per \$1 spent: on budget
EAC	$BAC/CPI = 250/1.00$	\$250	forecast total cost equals the budget
SV	$EV - PV = 140 - 145$	$-\$5$	\$5 worth of work behind schedule (tasks A, B)
CV	$EV - AC = 140 - 140$	\$0	no cost overrun

Answer. SPI $= 0.966$, CPI $= 1.00$, EAC $= \$250$, SV $= -\$5$, CV $= \$0$: on budget but slightly behind schedule.

(c) Definitions

- **Critical path**: the longest path (in total duration) through the activity network from start to finish. Its length is the shortest possible project duration; every activity on it has zero slack, so any delay on it delays the project.
- **Slack (float)**: the amount of time an activity can be delayed without delaying the project completion: Slack $= LS - ES = LF - EF$.

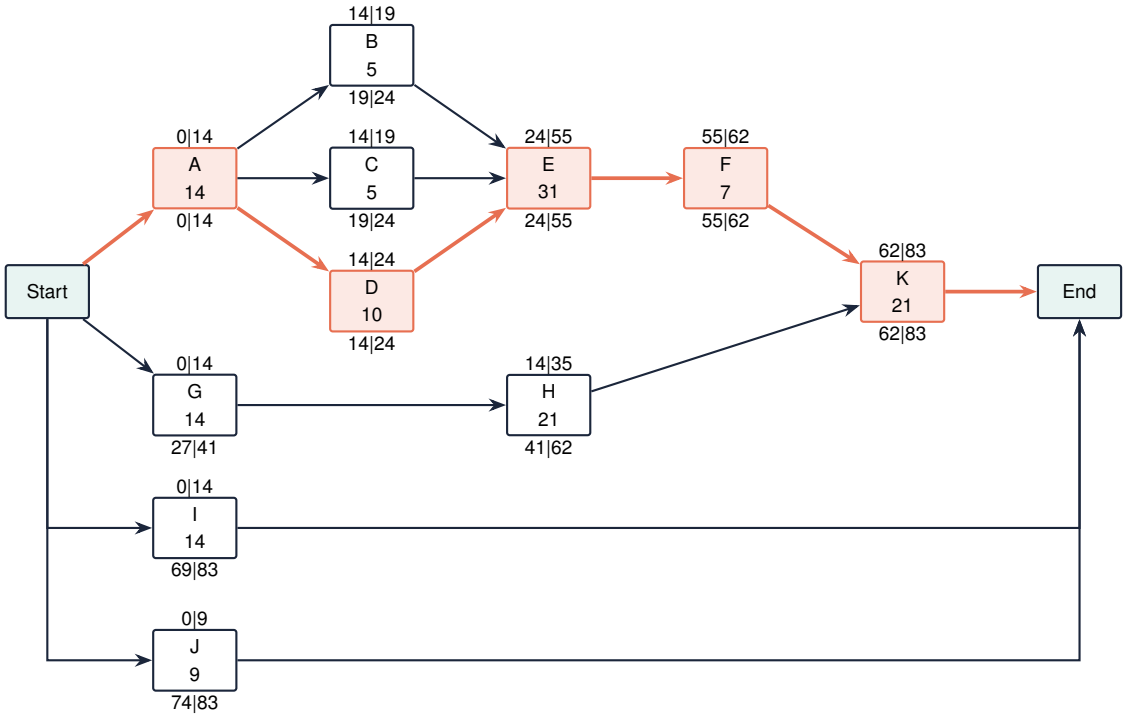
Formulas: forward pass $ES = \max(EF_{\text{pred}})$, $EF = ES + d$; backward pass $LF = \min(LS_{\text{succ}})$, $LS = LF - d$.

NOTE The question asks for the deadline "in days" but the table gives durations in *weeks*. All computations below are in weeks; multiply by 7 for calendar days (or by 5 for working days).

Forward and backward pass

Task	Pred.	d	ES	EF	LS	LF	Slack
A	–	14	0	14	0	14	**0**
B	A	5	14	19	19	24	5
C	A	5	14	19	19	24	5
D	A	10	14	24	14	24	**0**
E	B,C,D	31	$\max(19,19,24)=24$	55	24	55	**0**
F	E	7	55	62	55	62	**0**
G	–	14	0	14	27	41	27
H	G	21	14	35	41	62	27
I	–	14	0	14	69	83	69
J	–	9	0	9	74	83	74
K	F,H	21	$\max(62,35)=62$	83	62	83	**0**

Backward-pass details: project end $=\max(EF_K, EF_I, EF_J)=\max(83,14,9)=83$. $LF_K = LF_I = LF_J = 83$; $LF_F = LF_H = LS_K = 62$; $LF_E = LS_F = 55$; $LF_B = LF_C = LF_D = LS_E = 24$; $LF_G = LS_H = 41$; $LF_A = \min(LS_B, LS_C, LS_D)=\min(19,19,14)=14$.



Activity-on-node network (above each node: $ES \mid EF$; below: $LS \mid LF$; critical path in coral).

Answer. Strict deadline $=83$ weeks ($=581$ days). Slack (weeks): A 0, B 5, C 5, D 0, E 0, F 0, G 27, H 27, I 69, J 74, K 0. Critical path: **A** $\rightarrow$ **D** $\rightarrow$ **E** $\rightarrow$ **F** $\rightarrow$ **K** $(14+10+31+7+21=83)$.

2021–2022 • Q5(a) CSE307 • 07/05/2023 • M007 MARKS 10

Describe, with an appropriate example, the *uncertainty* and *loss* associated with a particular risk in a typical software project.

SOLUTION

The two characteristics of every risk

- **Uncertainty** – the event *may or may not* happen; there are no 100%-probable risks (a certain event is a *constraint/problem*, not a risk). Uncertainty is quantified by the *probability* P of occurrence (e.g. 10%, 50%, 80%).
- **Loss** – *if* the risk becomes a reality, unwanted consequences follow: extra cost, schedule slip, lower quality, lost customers or reputation. It is quantified by the *cost/impact* C.

Together they give the **risk exposure**

$$RE = P \times C.$$

The greater the uncertainty and the greater the loss, the more attention (mitigation, monitoring and contingency planning) the risk deserves.

Worked example – planned reuse falls short

Risk: only 70% of the software components scheduled for reuse will actually be integrated; the remaining functionality will have to be custom-developed.

- **Uncertainty:** the team has never used this component library; the reviewers estimate the probability of the risk as $P = 80\% = 0.80$. It is uncertain because the components might still fit with minor adaptation.
- **Loss:** 60 components were planned for reuse. If only 70% are usable, $30\% \times 60 = 18$ components must be built from scratch. With an average component size of 100 LOC and a local cost of \$14 per LOC:

$$C = 18 \times 100 \text{ LOC} \times \$14/\text{LOC} = \$25{,}200.$$

- **Exposure:** $RE = 0.80 \times \$25{,}200 = \$20{,}160$. This amount is compared with the cost of mitigation (e.g. an early prototype that tests the components, costing far less) and is added to the project's contingency reserve.

Second illustration (staff risk)

Key developer leaves mid-project. Uncertainty: industry turnover in the team is high, $P \approx 0.6$. Loss: about 2 months to recruit and bring a replacement up to speed, i.e. schedule slip plus about 2 person-months of cost; knowledge of the module may be lost. Mitigation: pair programming and documentation so that knowledge is shared; contingency: a trained backup member.

Answer. Uncertainty = the risk may or may not occur (probability $P < 1$); loss = the damage it causes if it occurs (cost C). E.g. reuse shortfall: $P = 0.8$, $C = \$25{,}200$, so $RE = \$20{,}160$.

2021–2022 • Q6(b) CSE307 • 07/05/2023 • M008 MARKS 15

Imagine that you are project manager organizing and leading different projects in a software company. Some information on one such project, which has been running for the past one month, is provided below in Table for Question 6(b).

Objective	Total PV	PV after One Month	EV after One Month
A	\$70	\$50	\$40
B	\$40	\$40	\$35
C	\$30	\$20	\$30
D	\$60	\$35	\$35
E	\$50	\$0	\$0

Table for Question 6(b)

Here, PV and EV stand for *planned value* and *earned value*, respectively. The company expended \$130 in this project during the past month. Now, given the information, compute *schedule performance index (SPI)*, *cost performance index (CPI)*, and *estimated cost at completion (EAC)* of this project after one month. Put some remarks on the progress of this project based on the obtained values.

SOLUTION

Step 1 – Aggregate the values

$$BAC = \sum \text{Total PV} = 70 + 40 + 30 + 60 + 50 = \$250$$
$$PV = 50 + 40 + 20 + 35 + 0 = \$145, \qquad EV = 40 + 35 + 30 + 35 + 0 = \$140, \qquad AC = \$130$$

Step 2 – Indices and forecast

$$SPI = \frac{EV}{PV} = \frac{140}{145} = 0.966$$
$$CPI = \frac{EV}{AC} = \frac{140}{130} = 1.077$$
$$EAC = \frac{BAC}{CPI} = \frac{250}{1.077} = \$232.14 \quad \left(= BAC \cdot \tfrac{AC}{EV} = 250 \times \tfrac{130}{140} \right)$$

Supporting variances: $SV = EV - PV = -\$5$, $CV = EV - AC = +\$10$, $VAC = BAC - EAC = +\$17.86$.

Step 3 – Remarks on progress

Obj.	PV	EV	SV	Comment
A	50	40	−10	behind plan – main cause of the delay
B	40	35	−5	behind plan (should already be complete)
C	20	30	+10	ahead of plan
D	35	35	0	on plan
E	0	0	0	not yet scheduled

- $SPI = 0.966 < 1$: the project is **slightly behind schedule** (96.6% of the planned work is done); objectives A and B need attention (e.g. move people from the ahead-of-plan objective C).
- $CPI = 1.077 > 1$: the project is **under budget** – \$1.077 of work is earned for every \$1 spent.

- $EAC = \$232.14 < BAC = \$250$: if the current cost efficiency continues, the project will finish about \$17.86 under budget.

Answer. SPI $= 0.966$, CPI ≈ 1.08, EAC $\approx \$232.14$: cost-efficient (under budget) but slightly behind schedule.

2021–2022 • Q7(b), (c) CSE307 • 07/05/2023 • M009 **MARKS 20 + 5**

(b) Suppose that you have been assigned to analyze the schedule of the software project, in which you are currently working, by your project manager. Consider the following information about this project in Table for Question 7(b).

Task	Predecessor(s)	Duration (in Week)
A	–	3
B	A	1.5
C	A	2
D	–	3.5
E	D	4
F	C, E	2.5
G	B	1
H	C, F, G	0.5

Table for Question 7(b)

Now, determine *strict deadline*, *slack* of different tasks, and *critical path* of this project by applying *critical path method (CPM)*.
(c) *"Adding manpower to a late software project makes it later."* – justify this statement.

SOLUTION

(b) Method

CPM on the activity-on-node network: forward pass $ES = \max(EF_{\text{pred}})$, $EF = ES + d$; the largest EF is the **strict deadline**. Backward pass from the deadline: $LF = \min(LS_{\text{succ}})$, $LS = LF - d$. **Slack** $= LS - ES$; zero-slack tasks form the **critical path**.

Step 1 – Forward pass

- A: $0 \to 3$; D: $0 \to 3.5$; B: $ES = EF_A = 3$, $EF = 4.5$; C: $ES = 3$, $EF = 5$; E: $ES = EF_D = 3.5$, $EF = 7.5$.
- F: $ES = \max(EF_C, EF_E) = \max(5, 7.5) = 7.5$, $EF = 10$; G: $ES = EF_B = 4.5$, $EF = 5.5$.
- H: $ES = \max(EF_C, EF_F, EF_G) = \max(5, 10, 5.5) = 10$, $EF = 10.5$.

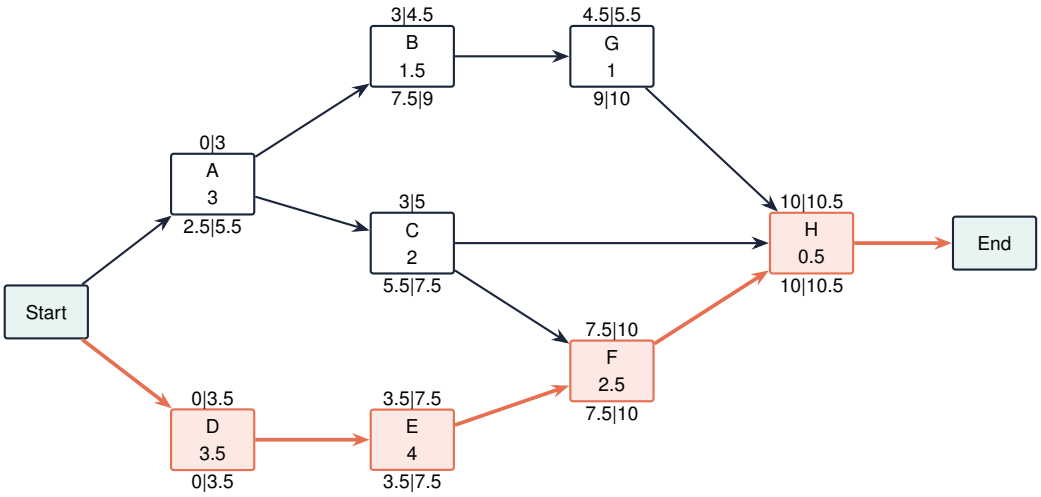
Strict deadline $= EF_H = 10.5$ weeks.

Step 2 – Backward pass (from 10.5)

- H: $LF = 10.5$, $LS = 10$; F: $LF = LS_H = 10$, $LS = 7.5$; G: $LF = LS_H = 10$, $LS = 9$.
- E: $LF = LS_F = 7.5$, $LS = 3.5$; D: $LF = LS_E = 3.5$, $LS = 0$; B: $LF = LS_G = 9$, $LS = 7.5$.
- C: $LF = \min(LS_F, LS_H) = \min(7.5, 10) = 7.5$, $LS = 5.5$; A: $LF = \min(LS_B, LS_C) = \min(7.5, 5.5) = 5.5$, $LS = 2.5$.

Step 3 – Summary table

Task	Pred.	d	ES	EF	LS	LF	**Slack** $= LS - ES$
A	–	3	0	3	2.5	5.5	2.5
B	A	1.5	3	4.5	7.5	9	4.5
C	A	2	3	5	5.5	7.5	2.5
D	–	3.5	0	3.5	0	3.5	**0**
E	D	4	3.5	7.5	3.5	7.5	**0**
F	C,E	2.5	7.5	10	7.5	10	**0**
G	B	1	4.5	5.5	9	10	4.5
H	C,F,G	0.5	10	10.5	10	10.5	**0**



Above each node: $ES \mid EF$; below: $LS \mid LF$ (weeks). Critical path in coral.

Path check: D–E–F–H $= 3.5 + 4 + 2.5 + 0.5 = 10.5$; A–C–F–H $= 3 + 2 + 2.5 + 0.5 = 8$; A–B–G–H $= 3 + 1.5 + 1 + 0.5 = 6$; A–C–H $= 5.5$. The longest is D–E–F–H, confirming the result.

Answer. Strict deadline $= 10.5$ weeks. Slack: A 2.5, B 4.5, C 2.5, D 0, E 0, F 0, G 4.5, H 0 (weeks). Critical path: **D** $\rightarrow$ **E** $\rightarrow$ **F** $\rightarrow$ **H**.

(c) "Adding manpower to a late software project makes it later" (Brooks's law)

- **Ramp-up (learning) time:** newcomers cannot be productive immediately; they must learn the domain, architecture and code. The people who teach them are the experienced developers of the late project, whose own productivity drops.
- **Communication overhead:** with n people there are $n(n-1)/2$ communication paths. Going from 4 to 8 developers raises the paths from $\frac{4\cdot3}{2} = 6$ to $\frac{8\cdot7}{2} = 28$; meetings, coordination and integration errors grow faster than the added capacity.
- **Limited divisibility of work:** many tasks are sequential (a design must exist before coding, a module before its integration); repartitioning the work also costs time. ("Nine women cannot make a baby in one month.")
- **Illustration:** 4 developers have 6 person-months of work left. Add 4 newcomers: for the first month the newcomers produce little and consume, say, 30% of the veterans' time for training,

and every person loses extra time to coordination – the remaining work may take longer than if the original 4 had simply continued.

Effort–time trade-off models agree: Putnam's software equation $E \propto 1/t^4$ shows that compressing the schedule increases effort enormously. People can help only if added *early*, to *independent* well-specified tasks, and if they are already experienced in the system.

Answer. Because newcomers need training from the busy team, communication paths grow as $n(n-1)/2$, and software tasks are only partly divisible, adding people late reduces net productivity and delays the project further.

2020–2021 • Q3(c), Q4(a)–(c) CSE307 • 19/10/2022 • M011 **MARKS 10 + 10 + 15 + 10**

3(c) Describe COCOMO II model for estimation with necessary examples.
4(a) Consider a project for result processing system which is scheduled to prepare the results of SSC 2022. The software is being developed for education boards of the country. The project manager has to consider issues related to natural calamities like flood. Identify five risks while planning this software development project. Discuss the impact and exposure of the risks with proper justification. Briefly describe the risk mitigation plans for the identified risks.
4(b) What are the factors in selecting the team members for developing a Bangla Spell Checker? Explain in details.
4(c) Titas Gas Transmission and Distribution Company is trying to have a new billing software. There are several alternatives to implement that. The first one is to acquire a ready made billing software from a software development company with the cost of Tk. 1 crore. It might require 20 additional changes/reports to customize the software for the use of the company. The cost of each easy, moderately difficult or extremely difficult change/report is Tk. 50K, Tk. 70K or Tk. 100K. The probability of the changes/reports are easy and moderately difficult are 0.30 or 0.25 respectively. The other option is to reuse an open source billing software by an in house team of 10 members. The average salary of the in house team members are Tk. 75K. It will take 12 months to develop the system if the development is straight forward with the probability of 0.60. Otherwise it will take 18 months to develop the system if the development is complex. Which one would be best strategy for developing the billing software?

SOLUTION

3(c) The COCOMO II model

COCOMO II (Constructive Cost Model, Boehm 2000) is an *algorithmic* cost model: effort is computed from a size estimate and a set of project, product, process and people attributes,

$$PM = A \times \text{Size}^{B} \times M,$$

where $A = 2.94$ (calibration constant), Size is in KSLOC, B (1.1–1.24) is the *scale exponent* reflecting diseconomies of scale, and M is the product of *effort multipliers* (cost drivers). It recognises that more information is available as a project proceeds and therefore offers **four sub-models**:

(i) Application-composition model – for prototyping and systems built from reusable components/scripting, early in the project. Size in *application (object) points*: $PM = NAP \times (1 - \%\text{reuse}/100)$ / $PROD$.
Example: 120 application points, 20% reuse, nominal productivity $PROD = 13$ points/PM: $PM = 120 \times 0.8/13 = 7.4$ person-months.

(ii) Early-design model – after requirements, when only a rough architecture exists. Size from function points converted to KSLOC; B from five scale factors; $M = PERS \times RCPX \times RUSE \times PDIF \times PREX \times FCIL \times SCED$ (7 combined cost drivers).
Example: 50 KSLOC, $B = 1.17$, $M = 1$: $PM = 2.94 \times 50^{1.17} = 2.94 \times 97.2 \approx 286$ person-months.

(iii) Reuse model – estimates the effort of integrating reused or generated code by converting it to equivalent new code: $ESLOC = ASLOC \times (1 - AT/100) \times AAM$ (AT = % automatically translated, AAM = adaptation adjustment multiplier for design/code/integration changes, understanding and unfamiliarity).
Example: 20,000 reused lines, $AT = 25\%$, $AAM = 0.4$: $ESLOC = 20{,}000 \times 0.75 \times 0.4 = 6{,}000$ lines are added to Size.

(iv) Post-architecture model – once the architecture is designed; same formula with a more accurate size (new + equivalent reused code, adjusted for requirements volatility) and 17 individual cost drivers (e.g. RELY, CPLX, ACAP, PCAP, TOOL, SCED).

Scale exponent: $B = 1.01 + 0.01 \sum SF_i$ with five scale factors – precedentedness, development flexibility, architecture/risk resolution, team cohesion, process maturity – each rated 5 (very low) to 0 (extra high). Schedule: $TDEV = 3 \times PM^{0.33+0.2(B-1.01)}$ months. (A full numeric example is solved in M020(d).)

4(a) Flood-related risks of the SSC 2022 result processing system

$RE = P \times C$; impact on Pressman's scale (1 catastrophic … 4 negligible); losses C are estimated in Tk lakh.

Risk	P	Imp.	C / RE	Justification
R1 Servers / data-entry offices flooded: loss of marks data	0.15	1	15 / **2.25**	Physical damage to board offices or data centre; re-entry of marks, results delayed nationally.
R2 Power and internet outages in flood-hit regions	0.4	2	4 / **1.6**	Marks upload from regional boards and centres stops; work stalls.
R3 Exams postponed in affected areas; separate/late results needed (requirement change)	0.5	2	3 / **1.5**	System must support partial result publication and re-scheduled subjects.
R4 Late arrival of marks from flood-affected boards (schedule)	0.5	3	2 / **1.0**	Processing window shrinks before the fixed publication date.
R5 Team members unavailable (homes flooded, commuting, waterborne disease)	0.4	3	2.5 / **1.0**	Loss of person-weeks during the critical period.

Mitigation plans: R1 – off-site/cloud backups, disaster-recovery site in a flood-free location, servers above ground floor; R2 – UPS/generators, redundant ISPs, offline data-entry tool that syncs later; R3 – design configurable result batches and supplementary-result module early; R4 – schedule buffer, prioritise unaffected boards, courier backup for data; R5 – remote work, cross-training, health support. Highest exposure (R1) gets the strongest plan.

4(b) Factors in selecting team members for a Bangla Spell Checker

Sommerville lists the factors governing staff selection; for this product they mean:

- **Application-domain experience** – knowledge of Bangla linguistics: spelling rules (Bangla Academy standard), conjuncts (juktakkhor), kar/fola signs, inflection and sandhi, *sadhu/cholito* forms. At least one linguist or domain expert is essential.
- **Platform experience** – where it runs: MS Word/LibreOffice add-in, browser extension, Android keyboard; Unicode Bangla text handling, rendering and input methods (Avro, Bijoy).
- **Programming-language / technique experience** – string algorithms (edit distance, tries, n-grams), NLP (stemming, morphological analysis, language models), the chosen language (Java/Python/C++).
- **Problem-solving ability** – suggestion ranking and handling of ambiguous words need creative algorithm design.
- **Educational background** – CS/NLP background for developers; linguistics background for the language expert.
- **Communication ability** – developers and linguists must understand each other; writing clear documentation.
- **Adaptability** – ability to learn new NLP tools and changing spelling standards.
- **Attitude** – willingness to learn and care for quality (a spell checker with false alarms is useless).
- **Personality** – a balanced team of task-oriented, self-oriented and interaction-oriented people works best; members must be compatible to form a cohesive group.

Also consider *availability/cost* and the *team mix*: e.g. a lead with NLP experience, 2–3 developers, a linguist, and native-speaker testers who build the test corpus.

4(c) Titas Gas billing software – decision by expected cost

> **NOTE** The salary of Tk. 75K is taken as *per member per month*. The probability of an extremely difficult change is the remainder: $1 - 0.30 - 0.25 = 0.45$.

Option 1 – buy the ready-made software and customise

$$E[\text{cost per change}] = 0.30(50\text{K}) + 0.25(70\text{K}) + 0.45(100\text{K}) = 15 + 17.5 + 45 = \text{Tk } 77.5\text{K}$$

$$\text{Customisation} = 20 \times 77.5\text{K} = \text{Tk } 1{,}550\text{K} = \text{Tk } 15.5 \text{ lakh}$$

$$\text{Expected total} = 100 \text{ lakh} + 15.5 \text{ lakh} = \textbf{Tk 115.5 lakh } (= 1.155 \text{ crore})$$

Option 2 – reuse open-source software with the in-house team

$$\text{Team cost per month} = 10 \times 75\text{K} = \text{Tk } 7.5 \text{ lakh}$$

$$\text{Straightforward } (0.60) : 12 \times 7.5 = \text{Tk } 90 \text{ lakh}; \qquad \text{Complex } (0.40) : 18 \times 7.5 = \text{Tk } 135 \text{ lakh}$$

$$\text{Expected total} = 0.60(90) + 0.40(135) = 54 + 54 = \textbf{Tk 108 lakh } (= 1.08 \text{ crore})$$

(Expected duration $= 0.6 \times 12 + 0.4 \times 18 = 14.4$ months.)

Decision

Option 2 has the lower expected cost by $115.5 - 108 = $ Tk 7.5 lakh. Qualitatively, the in-house team also gains ownership of the source code and knowledge for future maintenance. However, there is a 40% chance it costs Tk 135 lakh and takes 18 months; if the company needs the system quickly or is risk-averse, buying (known price, faster deployment) is a defensible alternative.

Answer. Expected cost: buy + customise Tk 1.155 crore; reuse open source in-house Tk 1.08 crore. On expected cost the best strategy is to **reuse the open-source billing software with the in-house team**.

Jan 2021 • Q2(b) CSE307 • 04/09/2021 • M012 **MARKS 15**

The estimated line of codes for a IoT based application is approximately 50 KLOC. The project must be finished within a year. Calculate the effort using dynamic multivariate model.

SOLUTION

The dynamic multivariable model = Putnam's software equation

Pressman calls the **software equation** a *dynamic multivariable model*: it assumes a specific distribution of effort over the life of a project (Putnam–Norden–Rayleigh curve) and was derived from productivity data of about 4,000 projects:

$$E = \left[\frac{LOC \times B^{0.333}}{P}\right]^3 \times \frac{1}{t^4}$$

- E – effort in person-years (or person-months); t – project duration in years (or months);
- B – *special skills factor*: about 0.16 for small programs (5–15 KLOC), rising slowly with size to 0.39 for programs above 70 KLOC;
- P – *productivity parameter* of the organisation: about 2,000 for real-time embedded software, 10,000 for telecommunication and systems software, 12,000 for scientific software, 28,000 for business applications.

Minimum development time: $t_{\min} = 8.14\,(LOC/P)^{0.43}$ months, and for $t = t_{\min}$ the effort simplifies to $E = 180\,B\,t^3$ person-months (t in years).

NOTE The question does not give B and P, so standard textbook (Pressman) values are assumed: $B = 0.34$ (50 KLOC lies between Pressman's 0.28 for a ~33 KLOC system and 0.39 for >70 KLOC) and $P = 10{,}000$ (an IoT application is systems/communication software running on devices and gateways). $t = 1$ year as stated.

Step 1 – Substitute

$$B^{0.333} = 0.34^{1/3} = 0.698$$

$$\frac{LOC \times B^{0.333}}{P} = \frac{50{,}000 \times 0.698}{10{,}000} = 3.490$$

$$E = (3.490)^3 \times \frac{1}{1^4} = 42.5 \text{ person-years} = 42.5 \times 12 = \mathbf{510} \text{ person-months}$$

That is an average staff of 42.5 people working for the whole year.

Step 2 – Is one year realistic? (minimum time check)

$$t_{\min} = 8.14\left(\frac{50{,}000}{10{,}000}\right)^{0.43} = 8.14 \times 5^{0.43} = 8.14 \times 1.997 = 16.3 \text{ months} \approx 1.36 \text{ years}.$$

The required 12 months is *shorter* than the minimum time, which is why the effort is so large ($E \propto 1/t^4$). If the deadline could be relaxed to $t_{\min}$: $E = 42.5/1.355^4 = 42.5/3.37 = 12.6$ person-years ≈ 151 person-months (check: $180 \times 0.34 \times 1.355^3 \approx 152$ person-months).

Sensitivity to the productivity parameter ($t = 1$ year, $B = 0.34$)

Application class	P	E (person-months)	$t_{\min}$ (months)
Real-time embedded	2,000	63,750 (infeasible)	32.5
Systems / telecom (assumed)	10,000	510	16.3
Scientific	12,000	295	15.0
Business application	28,000	23.2	10.4

Answer. Software equation with $B = 0.34$, $P = 10{,}000$, $t = 1$ yr: $E = [50{,}000 \times 0.34^{1/3}/10{,}000]^3/1^4 \approx 42.5$ person-years ≈ 510 person-months. Because 1 year $< t_{\min} \approx 16$ months, the deadline is very costly; relaxing it to 16 months cuts effort to ≈ 151 person-months.

Jan 2021 • Q4(a), (b) CSE307 • 04/09/2021 • M013 MARKS 20 + 15

4(a) Consider a project to develop result processing system which is scheduled to deliver by the end of Year 2021. The software is being developed for Dhaka Education Board. The project manager has to consider issues related to the pandemic. Identify five risks while planning this software development project. Discuss the impact and exposure of the risks with proper justification. Briefly describe the risk mitigation plans for the identified risks.

4(b) Consider the following table where all the duration estimates of various activities of a project are listed and the network proceeds from Node 1 to Node 9.

Activity	Initial Node	Final Node	Estimated Duration (days)
A	1	2	2
B	2	3	2
C	2	4	3
D	2	5	4
E	3	6	2
F	4	6	3
G	5	7	6
H	6	8	2
I	6	7	5
J	7	8	1
K	8	9	2

Draw an Activity on Arrow Diagram representing the project. Identify all paths in the network diagram. What is the critical path of the project? Discuss the significance of the critical path.

(Q3(b) of the same excerpt – reviewers vs. testers – is solved in the Unit Testing & Code Review chapter.)

SOLUTION

4(a) Pandemic-related risks of the result processing system

Approach

Each risk is described by its *probability* P and its *impact/loss* C; the **risk exposure** $RE = P \times C$ ranks them. Impact is also given on Pressman's scale (1 catastrophic, 2 critical, 3 marginal, 4 negligible). Losses below are *estimates* expressed in Tk lakh (cost of extra effort, overtime or rework) for a team of about 8 developers on a 10-month schedule; what matters is the justification and the ranking.

Risk identification, impact and exposure

Risk (category)	P	Impact	C / RE (lakh)	Justification
R1 Requirements change: exams postponed, shortened syllabus, new grading/"subject mapping" rules (business / requirements)	0.7	2	5 / **3.5**	Pandemic decisions by the ministry are frequent and late; grading logic and report formats must be rebuilt.
R2 Team members infected or in quarantine (staff)	0.5	2	6 / **3.0**	Each case removes 2–3 weeks; key developers may be lost near the deadline.
R3 Productivity loss in remote work: poor internet, power cuts, weak coordination (project / environment)	0.6	3	3 / **1.8**	Work-from-home is mandatory during lockdowns; communication overhead rises.
R4 Confidential result data exposed through home networks/remote access (technical / security)	0.2	1	10 / **2.0**	Results are highly sensitive; a leak is catastrophic for the board's credibility.
R5 Delays in hardware procurement and in board officials' availability for sign-off and acceptance testing (project / customer)	0.4	3	3 / **1.2**	Lockdowns disrupt supply chains and office hours; testing with real data is delayed.

Ranking by exposure: R1 (3.5) > R2 (3.0) > R4 (2.0) > R3 (1.8) > R5 (1.2). The total exposure ($\approx$ 11.5 lakh) indicates how large a contingency reserve the plan should hold.

Mitigation (and contingency) plans

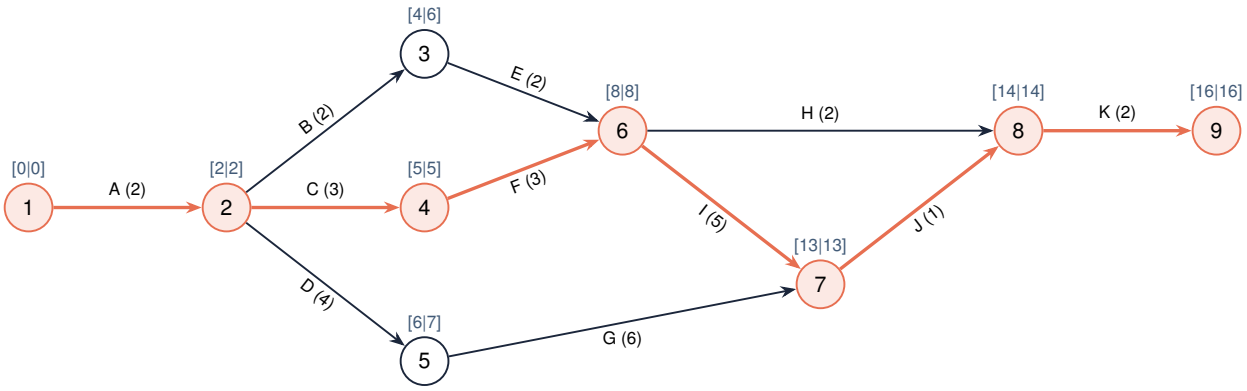
- **R1:** keep grading rules configurable (rule tables, not hard-coded); iterative delivery with frequent board reviews; change-control procedure; contingency: schedule buffer before the result-publication date.
- **R2:** cross-training and pair programming so that every module has two knowledgeable people; documented code; vaccination support, health insurance; contingency: backup/temporary developers.

- **R3:** provide internet allowance, UPS, VPN, collaboration tools (Git, issue tracker, daily online stand-ups); monitor velocity weekly.
- **R4:** VPN + two-factor authentication, no production data on personal machines (use masked test data), encryption, access logs; contingency: incident-response plan.
- **R5:** order hardware early or use cloud/hosted servers; agree online review and sign-off procedures with the board in advance.

4(b) Activity-on-arrow network

Drawing the AOA diagram

In an **activity-on-arrow (AOA)** diagram each *node* is an event (start/finish of activities) and each *arrow* is an activity with its duration. Event times: earliest $T_E(j) = \max_i\{T_E(i) + d_{ij}\}$ and latest $T_L(i) = \min_j\{T_L(j) - d_{ij}\}$.



Event labels $[T_E \mid T_L]$ in days; critical activities in coral.

Event times

Forward: $T_E(1) = 0$, $T_E(2) = 2$, $T_E(3) = 4$, $T_E(4) = 5$, $T_E(5) = 6$, $T_E(6) = \max(4+2,\, 5+3) = 8$, $T_E(7) = \max(6+6,\, 8+5) = 13$, $T_E(8) = \max(8+2,\, 13+1) = 14$, $T_E(9) = 16$.
Backward: $T_L(9) = 16$, $T_L(8) = 14$, $T_L(7) = 13$, $T_L(6) = \min(14-2,\, 13-5) = 8$, $T_L(5) = 13-6 = 7$, $T_L(4) = 8-3 = 5$, $T_L(3) = 8-2 = 6$, $T_L(2) = \min(6-2,\, 5-3,\, 7-4) = 2$, $T_L(1) = 0$.

All paths from node 1 to node 9

#	Path (activities)	Length (days)	Slack vs. 16
P1	A–B–E–H–K	$2+2+2+2+2 = 10$	6
P2	A–B–E–I–J–K	$2+2+2+5+1+2 = 14$	2
P3	A–C–F–H–K	$2+3+3+2+2 = 12$	4
P4	**A–C–F–I–J–K**	$2+3+3+5+1+2 = \mathbf{16}$	**0**
P5	A–D–G–J–K	$2+4+6+1+2 = 15$	1

Activity total floats ($T_L(\text{end}) - T_E(\text{start}) - d$): A 0, B 2, C 0, D 1, E 2, F 0, G 1, H 4, I 0, J 0, K 0.

Significance of the critical path

- Its length (16 days) is the **minimum project duration**; the project cannot finish earlier unless critical activities are shortened (crashing / fast-tracking).
- Every critical activity has **zero float**: a delay of one day in A, C, F, I, J or K delays the whole project by a day. The manager therefore monitors these activities most closely and assigns the best resources to them.
- Non-critical activities have float (e.g. H: 4 days), so their resources can be shifted to critical work and they can be scheduled flexibly. Note P5 (15 days) is *near-critical* – a 1-day slip in D, G turns it critical.

Answer. Critical path: **1–2–4–6–7–8–9, i.e. A–C–F–I–J–K = 16 days**. Other paths: 10, 14, 12 and 15 days.

2018–2019 • Q5(a) CSE307 • 06/11/2019 • M014 **MARKS 4 + 8**

(i) Write down the formula for calculation risk exposure/risk impact and define the variables used.

(ii) Spectrum Electronics Ltd. produces 10,000 ICs every month, and each of those ICs cost \$5. The probability of an IC being faulty is 10%. However, Spectrum Electronics Ltd. replaces every faulty IC with a new one, completely free of charge. Calculate the overall risk exposure for this situation.

SOLUTION

(i) Formula for risk exposure

$$RE = P \times C$$

- RE – **risk exposure** (expected loss, also called risk impact), in money (or time) units;
- P – **probability** that the risk occurs ($0 < P < 1$), i.e. its uncertainty;
- C – **cost/consequence** to the project (or company) if the risk occurs, i.e. its loss.

For several independent risks (or many items exposed to the same risk) the overall exposure is the sum $RE_{\text{total}} = \sum_i P_i C_i$.

(ii) Risk exposure of Spectrum Electronics

The risk event is "an IC is faulty". When it occurs, the company loses the cost of one replacement IC (\$5), because the replacement is free of charge for the customer.

- Per IC: $P = 10\% = 0.10$, $C = \$5 \Rightarrow RE_{\text{IC}} = 0.10 \times \$5 = \$0.50$.
- Per month (10,000 ICs):

$$RE = P \times C = 0.10 \times (10{,}000 \times \$5) = 0.10 \times \$50{,}000 = \$5{,}000 \text{ per month.}$$

- Equivalent view: expected faulty ICs $= 0.10 \times 10{,}000 = 1{,}000$; each costs \$5 to replace $\Rightarrow$ $1{,}000 \times \$5 = \$5{,}000$.
- Per year: $12 \times \$5{,}000 = \$60{,}000$.

NOTE Only the direct cost of the replacement IC is counted, as given. If handling/shipping of replacements had a cost, it would be added to C (and a replacement IC can itself be faulty – with that second-order effect the monthly figure becomes $\$5{,}000/(1-0.1) \approx \$5{,}556$).

Answer. $RE = P \times C$. For Spectrum Electronics: $RE = 0.10 \times 10{,}000 \times \$5 = \$5{,}000$ per month ($60,000 per year).

2018–2019 • Q6(a)–(c) CSE307 • 06/11/2019 • M015 **MARKS 4 + 20 + 11**

(a) Name the four P's of Project Management.
(b) From the Fig. for Question 6(b) compute the expected cost for each path and suggest the best possible course of action.

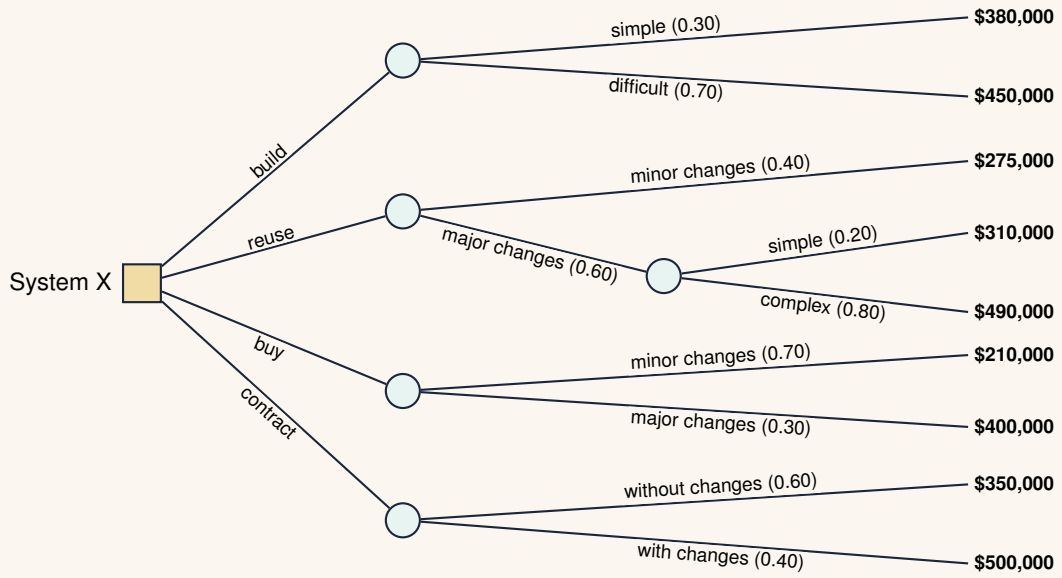


Fig. for Question 6(b)

(c) ABC Software Ltd. has been hired to deliver a business application within a year. The software engineers and developers at ABC Software Ltd. have estimated that the software will probably require 71 KLOCs of code. Using the software equation, compute the estimated effort in person-months.

SOLUTION

(a) The four P's of project management

(i) **People** – the most important element: stakeholders, team leaders, software team; how they are recruited, organised, motivated and coordinated.

(ii) **Product** – the software to be built: its scope, objectives, alternatives and constraints must be defined before estimates can be made.

(iii) **Process** – the framework activities and task set (process model) used to build the product.

(iv) **Project** – all the planned and controlled work (estimation, scheduling, risk and tracking)

that makes the product a reality.

(b) Decision tree – expected cost of each path

At each chance node the **expected cost** is $\sum_i (\text{path probability}_i \times \text{estimated cost}_i)$; the tree is evaluated from the leaves back to the decision node (roll-back), and the option with the *lowest* expected cost is chosen.

$$\begin{aligned}
\text{Build:} \quad & 0.30(\$380\text{K}) + 0.70(\$450\text{K}) = \$114\text{K} + \$315\text{K} = \mathbf{\$429{,}000} \\
\text{Reuse, “major changes” node:} \quad & 0.20(\$310\text{K}) + 0.80(\$490\text{K}) = \$62\text{K} + \$392\text{K} = \$454\text{K} \\
\text{Reuse:} \quad & 0.40(\$275\text{K}) + 0.60(\$454\text{K}) = \$110\text{K} + \$272.4\text{K} = \mathbf{\$382{,}400} \\
\text{Buy:} \quad & 0.70(\$210\text{K}) + 0.30(\$400\text{K}) = \$147\text{K} + \$120\text{K} = \mathbf{\$267{,}000} \\
\text{Contract:} \quad & 0.60(\$350\text{K}) + 0.40(\$500\text{K}) = \$210\text{K} + \$200\text{K} = \mathbf{\$410{,}000}
\end{aligned}$$

Option	Expected cost	Rank
Buy	\$267,000	1 (lowest)
Reuse	\$382,400	2
Contract	\$410,000	3
Build	\$429,000	4

Besides expected cost, the final decision should also consider delivery time, reliability of the acquired package, vendor support, and how well it fits future change; but on cost alone *buy* is clearly best (\$115K cheaper than reuse).

Answer. Expected costs: build \$429,000, reuse \$382,400, buy \$267,000, contract \$410,000. Best course of action: **buy** the system.

(c) Software equation for ABC Software Ltd.

$$E = \left[\frac{LOC \times B^{0.333}}{P}\right]^3 \times \frac{1}{t^4}$$

NOTE Parameters not given in the question; standard Pressman values are used: $B = 0.39$ (program larger than 70 KLOC), $P = 28{,}000$ (business systems application), $t = 1$ year.

$$\begin{aligned}
B^{0.333} &= 0.39^{1/3} = 0.7306 \\
\frac{LOC \times B^{0.333}}{P} &= \frac{71{,}000 \times 0.7306}{28{,}000} = \frac{51{,}873}{28{,}000} = 1.8526 \\
E &= (1.8526)^3 \times \frac{1}{1^4} = 6.36 \text{ person-years} \\
E &= 6.36 \times 12 \approx \mathbf{76.3} \text{ person-months}
\end{aligned}$$

Plausibility check: $t_{\min} = 8.14(71{,}000/28{,}000)^{0.43} = 8.14 \times 1.49 = 12.1$ months ≈ 1 year, so the one-year deadline is just achievable; the simplified form gives $E = 180\,B\,t^3 = 180 \times 0.39 \times 1^3 = 70.2$ person-months, consistent with the result. Average staff $\approx 76/12 \approx$ 6–7 people.

Answer. $E \approx 6.36$ person-years ≈ 76 person-months (about 70 person-months with $E = 180Bt^3$).

2018–2019 • Q7(c) CSE307 • 06/11/2019 • M016.2 MARKS 6 × 2 = 12

X Software Ltd. has budgeted a total of \$200,000 to buy, setup, test, and maintain a new system. The planed values for each task are: Buy–\$60,000, Setup–\$75,000, Test–\$40,000, Maintain–\$25,000. Among these phases, only the first phase has been completed and the actual cost of buying was \$70,000.
Using the above information, compute the following:

(i) Schedule performance index (SPI)
(ii) Cost performance index (CPI)
(iii) Budgeted cost at completion (BAC)
(iv) Estimated cost at completion (EAC)
(v) Schedule Variance (SV)
(vi) Cost Variance (CV)

SOLUTION

Identifying PV, EV and AC

- **EV**: only *Buy* is complete, so the earned value is its budgeted value: $EV = \$60{,}000$.
- **AC**: the money actually spent on buying: $AC = \$70{,}000$.
- **PV**: the work scheduled up to now. The status point is the end of the Buy phase ("only the first phase has been completed"), so the planned work to date is the Buy phase: $PV = \$60{,}000$.

NOTE The question does not state the status date separately; the standard reading (used here) is that the measurement is taken when the Buy phase was planned to finish, so $PV = EV = \$60{,}000$. If some Setup work had also been scheduled by now, PV would be larger and SPI < 1.

Computations

Metric	Formula / substitution	Value	Interpretation
(i) SPI	$EV/PV = 60{,}000/60{,}000$	1.00	on schedule
(ii) CPI	$EV/AC = 60{,}000/70{,}000$	0.857	only \$0.86 of work per \$1 spent: over budget
(iii) BAC	$60{,}000 + 75{,}000 + 40{,}000 + 25{,}000$	\$200,000	total budget
(iv) EAC	$BAC/CPI = 200{,}000/0.857$	\$233,333	forecast cost if inefficiency continues
(v) SV	$EV - PV = 60{,}000 - 60{,}000$	\$0	no schedule deviation
(vi) CV	$EV - AC = 60{,}000 - 70{,}000$	$-\$10{,}000$	\$10,000 cost overrun so far

Exact EAC: $200{,}000 \times \frac{70{,}000}{60{,}000} = \$233{,}333.33$, i.e. a forecast overrun of $VAC = 200{,}000 - 233{,}333 = -\$33{,}333$. If the Buy overrun is treated as a one-off (atypical) variance, the alternative forecast is $EAC = AC + (BAC - EV) = 70{,}000 + 140{,}000 = \$210{,}000$.

Answer. SPI $= 1.00$, CPI $= 0.857$, BAC $= \$200{,}000$, EAC $= \$233{,}333$, SV $= \$0$, CV $= -\$10{,}000$ (on schedule, over budget).

2018–2019 • Q8(c) CSE307 • 06/11/2019 • M018 **MARKS 4 + 10 + 4 = 18**

Using the information given in the Table for Q 8(c) do the following

(i) Draw a network diagram.

(ii) Compute the Early Start (ES) and Late Start (LS) for each node.

(iii) Clearly mention the critical path/paths.

Note that, you have to show the computation steps required for determining ES and LS for all nodes.

Activity	Predecessor	Duration
A	(None)	5
B	(None)	1
C	A	2
D	A, B	3
E	A	2
F	C, E	3
G	D, H	4
H	B, E	2
I	H	1
J	F, G, I	1

Table for Q. 8(c)

SOLUTION

(ii) Forward pass – Early Start (ES) and Early Finish ($EF = ES + d$)

Rule: $ES = 0$ for activities without predecessors, otherwise $ES = \max(EF \text{ of predecessors})$.

- A: $ES = 0$, $EF = 0 + 5 = 5$; B: $ES = 0$, $EF = 0 + 1 = 1$.
- C: $ES = EF_A = 5$, $EF = 7$; E: $ES = EF_A = 5$, $EF = 7$.
- D: $ES = \max(EF_A, EF_B) = \max(5, 1) = 5$, $EF = 8$.
- F: $ES = \max(EF_C, EF_E) = \max(7, 7) = 7$, $EF = 10$.
- H: $ES = \max(EF_B, EF_E) = \max(1, 7) = 7$, $EF = 9$.
- G: $ES = \max(EF_D, EF_H) = \max(8, 9) = 9$, $EF = 13$; I: $ES = EF_H = 9$, $EF = 10$.
- J: $ES = \max(EF_F, EF_G, EF_I) = \max(10, 13, 10) = 13$, $EF = 14 \Rightarrow$ project duration $= 14$.

(ii) Backward pass – Late Finish (LF) and Late Start ($LS = LF - d$)

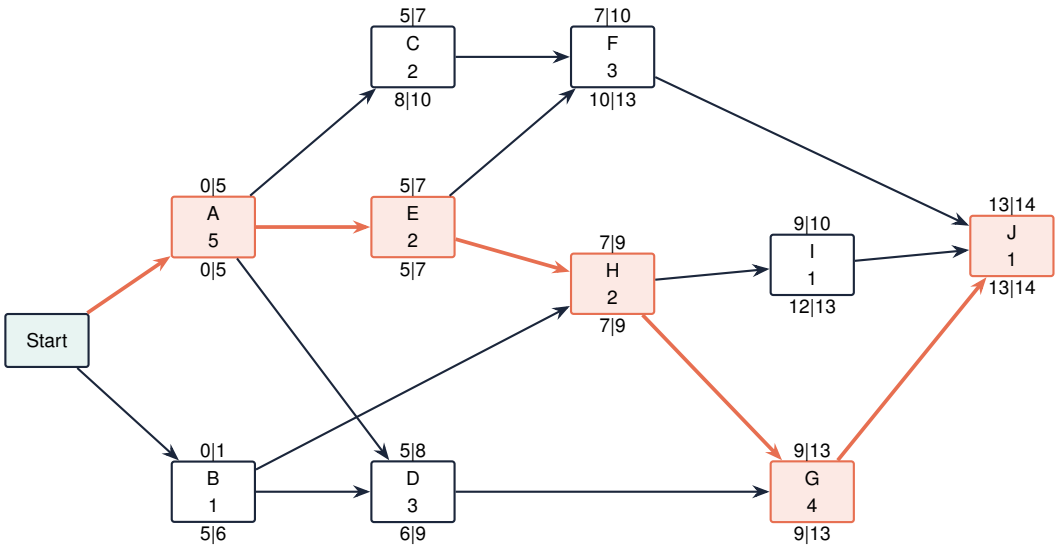
Rule: $LF =$ project end for the last activity, otherwise $LF = \min(LS \text{ of successors})$.

- J: $LF = 14$, $LS = 13$.
- F: $LF = LS_J = 13$, $LS = 10$; G: $LF = 13$, $LS = 9$; I: $LF = 13$, $LS = 12$.
- H: successors G, I: $LF = \min(LS_G, LS_I) = \min(9, 12) = 9$, $LS = 7$.
- D: $LF = LS_G = 9$, $LS = 6$; C: $LF = LS_F = 10$, $LS = 8$.

- E: successors F, H: $LF = \min(10, 7) = 7$, $LS = 5$.
- B: successors D, H: $LF = \min(LS_D, LS_H) = \min(6, 7) = 6$, $LS = 5$.
- A: successors C, D, E: $LF = \min(8, 6, 5) = 5$, $LS = 0$.

Activity	Pred.	d	ES	EF	LS	LF	Slack $= LS - ES$
A	–	5	0	5	0	5	**0**
B	–	1	0	1	5	6	5
C	A	2	5	7	8	10	3
D	A,B	3	5	8	6	9	1
E	A	2	5	7	5	7	**0**
F	C,E	3	7	10	10	13	3
G	D,H	4	9	13	9	13	**0**
H	B,E	2	7	9	7	9	**0**
I	H	1	9	10	12	13	3
J	F,G,I	1	13	14	13	14	**0**

(i) Network diagram (activity on node)



Above each node: $ES \mid EF$; below: $LS \mid LF$. Critical activities in coral.

(iii) Critical path

Zero-slack activities: A, E, H, G, J. They form one continuous chain:

$$\text{A} \to \text{E} \to \text{H} \to \text{G} \to \text{J}: \quad 5 + 2 + 2 + 4 + 1 = 14.$$

Check of the other start-to-end paths: A–C–F–J $= 11$, A–E–F–J $= 11$, A–D–G–J $= 13$, A–E–H–I–J $= 11$, B–D–G–J $= 9$, B–H–G–J $= 8$, B–H–I–J $= 5$ – all shorter than 14, so there is a *single* critical path.

Answer. Project duration $= 14$. ES/LS: A 0/0, B 0/5, C 5/8, D 5/6, E 5/5, F 7/10, G 9/9, H 7/7, I 9/12, J 13/13. Critical path: **A** $\to$ **E** $\to$ **H** $\to$ **G** $\to$ **J** (the only one).

2017–2018 • Q1(a), (b) CSE307 • 29/09/2018 • M019 **MARKS 10 + 8**

(a) There are several options of ERP implementation for organization which has 250 users. These are described as follows:
SAP Customization: ERP for any organization can be implemented by customizing modules of SAP. The cost for per user licensing fee of the SAP is USD 2000.00. It might require complex customization with the effort of 30 Man Month by the highly experienced SAP consultants and programmers. It might require easy customization with the effort of 20 Man Month by the moderately experienced SAP consultants and programmers. The probability of requiring complex customization would be half of the probability of requiring easy customization. The cost of Man Month for highly and moderately experienced SAP consultants or programmers are USD 20000.00 and USD 10000.00 respectively.
Customized ERP Development by own development team: It might require 5 analysts, 20 developers, 19 testers and 5 system engineers to develop and implement ERP in 20 months. The estimated monthly salary of analyst, developer, tester and system engineer are USD 4000.00, USD 3000.00, USD 2500.00 and USD 3500.00 respectively. There is a 60% chance to reuse the previously developed components to reduce the development time by 6 months.
Customized ERP Development through outsourcing: The cost ERP developed by the vendor id USD 400000.00. It might require 2 analysts, 2 developers, 5 testers and 2 system engineers to supervise and monitor ERP development for 8 months. The estimated monthly salary of analyst, developer, tester and system engineer are USD 5000.00, USD 2500.00, USD 2500.00 and USD 3500.00 respectively.
Analyze all the options mentioned above and find the optimal decision.
(b) Discuss the pricing decisions in the following situations with proper reasoning:

(i) The software development company is trying to get a contract for automation of an organization where the maximum job duration of the employees in the client's organization is 2 years.

(ii) The software development company is in the process of getting a new contract which will continue for the next 5 years.

(Situations (iii) and (iv) continue in M020.)

SOLUTION

(a) Option 1 – SAP customisation

Let $P(\text{easy}) = p$; then $P(\text{complex}) = p/2$ and $p + p/2 = 1 \Rightarrow p = 2/3$, $P(\text{complex}) = 1/3$.

$$\begin{aligned}
\text{Licensing} &= 250 \times \$2{,}000 = \$500{,}000 \\
\text{Complex customisation} &= 30\ \text{MM} \times \$20{,}000 = \$600{,}000 \\
\text{Easy customisation} &= 20\ \text{MM} \times \$10{,}000 = \$200{,}000 \\
E[\text{customisation}] &= \tfrac{1}{3}(600{,}000) + \tfrac{2}{3}(200{,}000) \\
&= 200{,}000 + 133{,}333 = \$333{,}333 \\
\text{Expected total} &= 500{,}000 + 333{,}333 = \$\mathbf{833{,}333}
\end{aligned}$$

Option 2 – own development team

$$\begin{aligned}\text{Monthly payroll} &= 5(4{,}000) + 20(3{,}000) + 19(2{,}500) + 5(3{,}500)\\ &= 20{,}000 + 60{,}000 + 47{,}500 + 17{,}500 = \$145{,}000 \text{ per month}\\ E[\text{duration}] &= 0.60 \times (20-6) + 0.40 \times 20 = 8.4 + 8.0 = 16.4 \text{ months}\\ \text{Expected total} &= 145{,}000 \times 16.4 = \$\mathbf{2{,}378{,}000}\end{aligned}$$

(14 months: \$2,030,000 with $P = 0.6$; 20 months: \$2,900,000 with $P = 0.4$.)

Option 3 – outsourcing

$$\begin{aligned}\text{Monitoring payroll} &= 2(5{,}000) + 2(2{,}500) + 5(2{,}500) + 2(3{,}500)\\ &= 10{,}000 + 5{,}000 + 12{,}500 + 7{,}000 = \$34{,}500/\text{month}\\ \text{Monitoring for 8 months} &= 8 \times 34{,}500 = \$276{,}000\\ \text{Total} &= 400{,}000 + 276{,}000 = \$\mathbf{676{,}000}\end{aligned}$$

Option	Expected cost (USD)	Rank
Outsourcing	676,000	1
SAP customisation	833,333	2
Own development	2,378,000	3

Answer. Outsourcing is optimal (\$676,000), cheaper than SAP customisation (\$833,333) and far cheaper than in-house development (\$2,378,000).

(b) Pricing decisions

Software price ≠ cost + fixed profit; Sommerville lists factors that move the price: *market opportunity*, *cost-estimate uncertainty*, *contractual terms* (e.g. who owns the source code), *requirements volatility* and the developer's *financial health*. "Pricing to win" sets the price at what the customer will pay.

(i) Client staff stay at most 2 years: the people who specify and accept the system will change during or soon after the project, so requirements are *volatile* and much training/hand-over support will be needed. Pricing: keep the base contract price competitive (low) to win, but price change requests, training and support separately (or add a contingency for requirement churn), and insist on written, signed-off requirements so that changes demanded by new staff are billable.

(ii) Contract continues for 5 years: a long-term, assured revenue stream and a strong reference – a *market opportunity*. The company can offer a lower annual price (volume/long-term discount, costs spread over years), but must include escalation clauses for inflation and salary rises, and a contingency for the larger cost uncertainty over five years.

2017–2018 • Q1(b) contd., (c), (d) CSE307 • 29/09/2018 • M020 MARKS 5 + 12

(b) contd. (Discuss the pricing decisions in the following situations with proper reasoning:)

(iii) The software development company has just started.

(iv) The new VAT rule may require new software and a multinational company wants to develop that software by the software development company.

(c) What is algorithmic cost model? What are the factors of accuracy in software cost estimation?
(d) After requirement analysis, the experienced system analyst estimated that the size of the code for a training simulator for fertilizer factory might be 230 KLOC. The software development company is a 5 year old company with CMMI level 3. The top management is committed to do formal risk management for smooth software development. Estimate the effort in person month, development time and team size using COCOMO 2 model.

SOLUTION

(b)(iii) The company has just started

A new company has no reputation or references, so winning the first contracts (*market opportunity*) matters more than immediate profit: it may price low – even near cost ("pricing to win") – to enter the market and gain a reference client. But its *financial health* is weak (little cash reserve), so it cannot afford a loss-making contract: it should price at least to cover costs, prefer milestone payments, and avoid fixed-price contracts with high cost-estimate uncertainty.

(b)(iv) New VAT software for a multinational company

A legal change creates an *urgent, fixed-deadline* need, and the same VAT software will be needed by many other companies (*market opportunity*). Two pricing levers:

- If the developer *retains ownership* of the code (contractual terms), it can quote a lower price to the multinational and resell the product to other companies.
- If the multinational wants exclusive ownership or a very short deadline, a higher price is justified (urgency premium, multinational's ability to pay, and *requirements volatility* as VAT rules are clarified – include a change-request clause).

(c) Algorithmic cost model

An **algorithmic cost model** estimates project effort as a mathematical function of product, project and process attributes whose values are estimated by the project manager. The generic form is

$$\text{Effort} = A \times \text{Size}^{B} \times M$$

A is an organisation-dependent constant, Size is usually in KLOC or function points, B ($\approx$1–1.5) captures disproportionate effort for large projects (diseconomy of scale), and M combines multipliers for process, product and development attributes (reliability, complexity, team experience…). The constants are derived from historical project data. COCOMO II and Putnam's software equation are examples.

Factors affecting the accuracy of the estimate:

- **Accuracy of the size estimate** – size is hard to estimate early (before code exists); estimates can be off by a factor of 4 at the start (cone of uncertainty) and improve as the project proceeds.
- **Subjective values of B and M** – ratings of attributes differ between estimators.
- **Calibration** – constants must be calibrated with the organisation's own historical data; using published values reduces accuracy.
- **Technology and process factors** – programming language, reuse, tools, application domain, team experience.

- **Requirements stability** – changing requirements invalidate estimates; also estimates tend to become self-fulfilling (work expands to fill the estimate).

(d) COCOMO II estimate for the training simulator

Model (Sommerville's COCOMO II):

$$PM = A \times \text{Size}^{B} \times M, \quad A = 2.94, \quad B = 1.01 + 0.01 \sum_{i=1}^{5} SF_i, \qquad TDEV = 3 \times PM^{\,0.33+0.2(B-1.01)}$$

with scale factors rated 5 (very low) … 0 (extra high).

Scale factor	Rating	Reason (from the scenario)
Precedentedness	Low = 4	a 5-year-old company is unlikely to have built a fertilizer-factory training simulator before
Development flexibility	Nominal = 3	no information – nominal
Architecture / risk resolution	Very high = 1	top management committed to *formal* risk management
Team cohesion	Nominal = 3	no information – nominal
Process maturity	$5-3=2$	CMMI level 3 (rating = 5− maturity level)
$\sum SF$	13	$B = 1.01 + 0.13 = 1.14$

NOTE The question gives no cost-driver values, so all effort multipliers are taken as nominal ($M = 1$); the ratings above are justified assumptions from the scenario. (With the COCOMO II.2000 calibration $PM = 2.94 \times \text{Size}^{0.91+0.01\sum SF}$, the same ratings give about 990 PM – the method is what is examined.)

Effort

$$PM = 2.94 \times 230^{1.14} \times 1 = 2.94 \times 492.5 = \mathbf{1{,}448} \text{ person-months}$$

Development time

$$\text{exponent} = 0.33 + 0.2(1.14 - 1.01) = 0.33 + 0.026 = 0.356$$
$$TDEV = 3 \times 1{,}448^{0.356} = 3 \times 13.34 = \mathbf{40.0} \text{ months}$$

Team size

$$\text{Average staff} = \frac{PM}{TDEV} = \frac{1{,}448}{40.0} \approx \mathbf{36} \text{ people}$$

Answer. With $B = 1.14$ and $M = 1$: effort $\approx 1{,}448$ person-months, development time ≈ 40 months, average team size ≈ 36 persons.

2017–2018 • Q4(c) CSE307 • 29/09/2018 • M021 **MARKS 10**

A software development schedule is fixed from Oct 2018 to March 2019 in an offshore development center in the Gulshan area of Dhaka city. The top management is thinking of health risks of the software development team during the development period. Identify, analyze and plan mitigation strategies for health related risks in the above mentioned context.

SOLUTION

Approach (risk management steps)

Identify the health hazards specific to the place (Gulshan, Dhaka – dense, polluted, high-rise offices) and the season (October = end of monsoon; December–February = dry winter), **analyse** each by probability and impact (Pressman impact scale 1 = catastrophic … 4 = negligible), then **plan** mitigation (reduce probability), monitoring (watch triggers) and contingency (what to do if it happens).

Identification and analysis

Health risk	Prob.	Impact	Why / effect on the project
R1 Dengue / chikungunya (Aedes mosquito)	High (Oct–Nov)	2	Post-monsoon outbreak season in Dhaka; 1–3 weeks of absence per case, possible hospitalisation.
R2 Respiratory illness from air pollution and winter flu	High (Dec–Feb)	3	Dhaka's air quality is worst in dry winter; cold/flu spreads in air-conditioned open offices; frequent short absences.
R3 Food- and water-borne disease (typhoid, diarrhoea, hepatitis)	Medium	3	Outside food and unsafe water around offices; several days of absence.
R4 Stress, burnout, sleep loss	Medium–High	2	Fixed 6-month offshore deadline, overtime and time-zone overlap with the client; errors and turnover rise.
R5 Ergonomic problems (back/neck pain, RSI, eye strain)	Medium	3	Long hours at screens; productivity falls gradually.
R6 Commuting accidents / fire in high-rise office	Low	1	Gulshan traffic and building-safety issues; rare but catastrophic for the people involved.

Risks with high probability *and* impact 1–2 (R1, R4, R2) are above the *cut-off line* and get full RMMM plans; the rest are handled by general policies.

Mitigation, monitoring and contingency plans

- **R1 Dengue:** mitigation – regular larvicide spraying and removal of standing water in and around the office, mosquito nets/repellent, screens on windows, awareness sessions; monitoring – DGHS dengue bulletins, staff sick-leave log; contingency – tie-up with a nearby hospital (e.g. in Gulshan), health insurance, cross-trained backup developers so no module depends on one person.
- **R2 Respiratory / pollution:** air purifiers and closed windows on high-AQI days, distribute masks, annual flu vaccination, sick-leave + work-from-home policy so infected staff do not spread illness.
- **R3 Food/water:** in-house canteen with hygienic food, filtered drinking water, typhoid/hepatitis vaccination.
- **R4 Stress:** realistic schedule with buffer, no sustained overtime (sustainable pace), flexible hours, breaks/recreation, counselling access; monitor overtime hours and team-mood surveys.
- **R5 Ergonomics:** ergonomic chairs, monitor arms, the 20-20-20 eye rule, regular breaks.
- **R6 Safety:** transport service for late-night staff, fire drills, extinguishers and clear exits, first-aid trained staff.

EXAM TIP Always tie the risks to the *context* given (place + period): marks are for dengue season, winter air pollution, workload of a fixed offshore deadline – not a generic list.

Answer. Main health risks: dengue (Oct–Nov), pollution/winter flu (Dec–Feb), food/water-borne disease, stress/burnout, ergonomic strain, commuting/fire accidents. Mitigate with vector control, clean air/water/food, vaccination, sustainable workload, ergonomic setup and safety measures; keep backups, insurance and hospital tie-ups as contingency.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q(b) CSE307 • 15/07/2017 • M024 **MARKS 15**

Draw a PERT chart using the durations and dependencies of the tasks shown in the following table. Report the critical path and estimated project completion time.

Task	Dependency	Duration
A	–	1
B	–	3
C	A	3
D	B, C	5
E	C	1
F	D	4
G	E, F	3
H	G	2
I	G	2
J	H, I	5
K	J	6

2012–2013 • Q2(b) CSE307 • 17/05/2014 • M032 **MARKS 14**

Draw the corresponding PERT chart of Table-1 and find out the critical path(s).

Task	A	B	C	D	E	F	G	H	I	J	K
Dependency	–	–	A	B,C	C	D	E,F	G	G	H,I	J
Duration	1	3	3	5	1	4	3	2	2	5	6

Table-1 for Q. No. 2(b)

SOLUTION

Concepts used

A **PERT chart** (activity network) shows every task as a node and every dependency as an arrow. With single (expected) durations it is analysed by the **Critical Path Method**:

- **Forward pass** – earliest start $ES = \max(EF \text{ of all predecessors})$ (0 for a task without predecessors), earliest finish $EF = ES + d$.

- **Backward pass** – latest finish $LF = \min(LS \text{ of all successors})$ (project end for a task without successors), latest start $LS = LF - d$.
- **Slack (float)** $= LS - ES = LF - EF$. Tasks with zero slack are **critical**: any delay in them delays the whole project. A **critical path** is a start-to-end chain of critical tasks; its length is the minimum project duration.

Durations are in the (unstated) time unit of the table, e.g. weeks.

Step 1 – Forward pass (earliest times)

- A: $ES = 0,\ EF = 0 + 1 = 1$; B: $ES = 0,\ EF = 0 + 3 = 3$; C: $ES = EF_A = 1,\ EF = 1 + 3 = 4$.
- D: $ES = \max(EF_B, EF_C) = \max(3, 4) = 4,\ EF = 4 + 5 = 9$; E: $ES = EF_C = 4,\ EF = 5$.
- F: $ES = EF_D = 9,\ EF = 13$; G: $ES = \max(EF_E, EF_F) = \max(5, 13) = 13,\ EF = 16$.
- H: $ES = 16,\ EF = 18$; I: $ES = 16,\ EF = 18$; J: $ES = \max(18, 18) = 18,\ EF = 23$; K: $ES = 23,\ EF = 29$.

The project therefore cannot finish before $EF_K = 29$.

Step 2 – Backward pass (latest times) with project end $= 29$

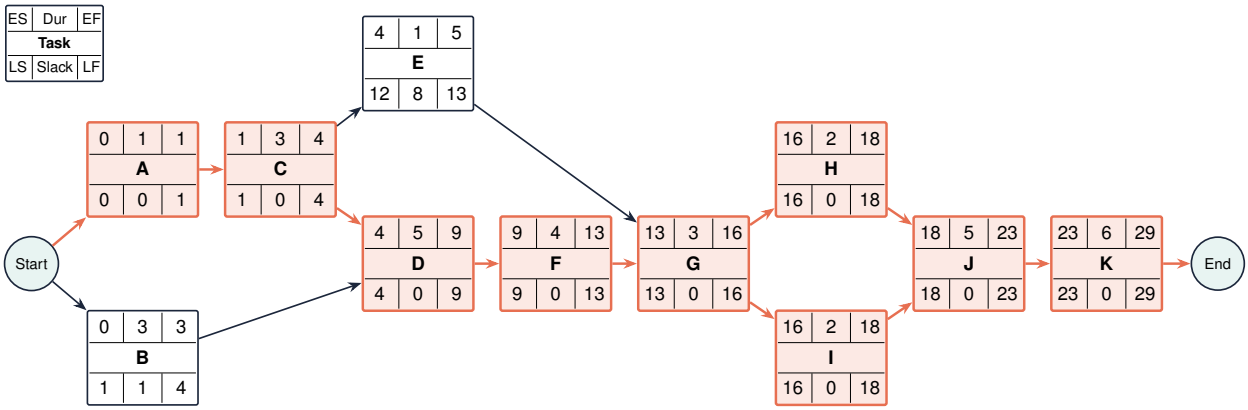
- K: $LF = 29,\ LS = 23$; J: $LF = LS_K = 23,\ LS = 18$; H: $LF = 18,\ LS = 16$; I: $LF = 18,\ LS = 16$.
- G: $LF = \min(LS_H, LS_I) = 16,\ LS = 13$; F: $LF = LS_G = 13,\ LS = 9$; E: $LF = LS_G = 13,\ LS = 12$.
- D: $LF = LS_F = 9,\ LS = 4$; C: $LF = \min(LS_D, LS_E) = \min(4, 12) = 4,\ LS = 1$; B: $LF = LS_D = 4,\ LS = 1$; A: $LF = LS_C = 1,\ LS = 0$.

Step 3 – Schedule table

Task	Pred.	d	ES	EF	LS	LF	Slack	Critical?
A	–	1	0	1	0	1	0	✓
B	–	3	0	3	1	4	1	×
C	A	3	1	4	1	4	0	✓
D	B, C	5	4	9	4	9	0	✓
E	C	1	4	5	12	13	8	×
F	D	4	9	13	9	13	0	✓
G	E, F	3	13	16	13	16	0	✓
H	G	2	16	18	16	18	0	✓
I	G	2	16	18	16	18	0	✓
J	H, I	5	18	23	18	23	0	✓
K	J	6	23	29	23	29	0	✓

Step 4 – PERT chart (activity-on-node)

Each node shows **ES** | **duration** | **EF** on top and **LS** | **slack** | **LF** at the bottom; critical tasks and arrows are drawn in coral.



Step 5 – Critical path(s) and completion time

Every task except B (slack 1) and E (slack 8) has zero slack. Because H and I are parallel and both have duration 2, there are **two** critical paths of equal length:

$$\text{A} \to \text{C} \to \text{D} \to \text{F} \to \text{G} \to \text{H} \to \text{J} \to \text{K}: \quad 1+3+5+4+3+2+5+6 = 29$$

$$\text{A} \to \text{C} \to \text{D} \to \text{F} \to \text{G} \to \text{I} \to \text{J} \to \text{K}: \quad 1+3+5+4+3+2+5+6 = 29$$

Check against the other paths: B–D–F–G–H–J–K $= 3+5+4+3+2+5+6 = 28$ (slack 1), A–C–E–G–H–J–K $= 1+3+1+3+2+5+6 = 21$ (slack 8).

Answer. Estimated completion time = 29 time units. Critical paths: A–C–D–F–G–H–J–K and A–C–D–F–G–I–J–K (both 29). B has 1 unit of slack and E has 8; all other tasks are critical.

> **EXAM TIP** When two parallel tasks have identical durations and the same predecessor/successor (H and I here), both are critical – say "critical path**s**" and list both.

2016–2017 • Q(c) CSE307 • 15/07/2017 • M025 **MARKS 10**

Perform function-point analysis using the data in the table given below and estimate the total cost of the project. Assuming that the project adjustment factor is 17, programmers in the organization can complete 18 function point per month, and each programmer charges $5,200 per month for this service.

Functions	Count per Complexity			Weight by Complexity		
	Simple	Average	Complex	Simple	Average	Complex
Inputs	3	4	1	2	4	6
Outputs	0	6	2	4	5	7
Files	0	0	5	2	5	7
Inquires	0	8	0	10	15	20
Interfaces	0	3	4	4	7	10

SOLUTION

Function-point method (Albrecht / IFPUG)

(i) Unadjusted FP: $UFP = \sum_{\text{type}} \sum_{\text{complexity}} \text{count} \times \text{weight}$.

(ii) Value (complexity) adjustment factor: $VAF = 0.65 + 0.01 \times \sum F_i$, where $\sum F_i$ (0–70) is the total of the 14 general-system-characteristic ratings – here the "project adjustment factor" $\sum F_i = 17$.

(iii) Adjusted FP: $FP = UFP \times VAF$.

(iv) Effort $= FP/\text{productivity}$; **Cost** $=$ Effort $\times$ cost per person-month.

NOTE The weights in this paper are the examiner's own (e.g. inquiries 10/15/20), not the standard IFPUG weights; they are used exactly as given. "Project adjustment factor = 17" is read as $\sum F_i = 17$.

Step 1 – Unadjusted function points

Function	Calculation (S + A + C)		Total
Inputs	$3 \times 2 + 4 \times 4 + 1 \times 6$	$= 6 + 16 + 6$	28
Outputs	$0 \times 4 + 6 \times 5 + 2 \times 7$	$= 0 + 30 + 14$	44
Files	$0 \times 2 + 0 \times 5 + 5 \times 7$	$= 0 + 0 + 35$	35
Inquiries	$0 \times 10 + 8 \times 15 + 0 \times 20$	$= 0 + 120 + 0$	120
Interfaces	$0 \times 4 + 3 \times 7 + 4 \times 10$	$= 0 + 21 + 40$	61
Unadjusted function points, UFP			**288**

Step 2 – Adjusted function points

$$VAF = 0.65 + 0.01 \times 17 = 0.82, \qquad FP = 288 \times 0.82 = 236.16 \text{ FP}.$$

Step 3 – Effort and cost

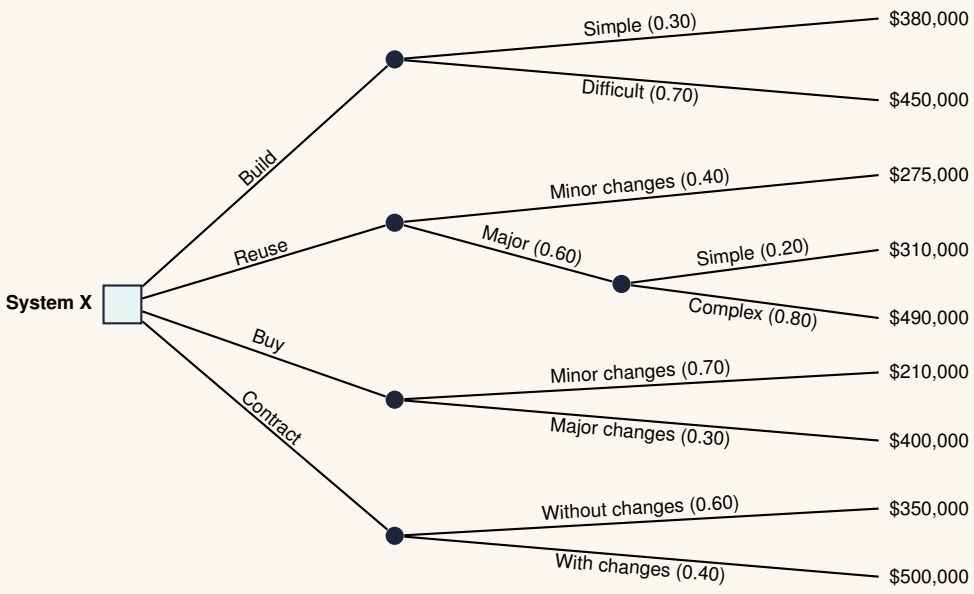
$$\text{Effort} = \frac{236.16 \text{ FP}}{18 \text{ FP/person-month}} = 13.12 \text{ person-months}$$

$$\text{Cost} = 13.12 \text{ PM} \times \$5{,}200/\text{PM} = \$68{,}224 \qquad \left(\text{cost per FP} = \tfrac{5200}{18} = \$288.89\right)$$

Answer. UFP = 288, VAF = 0.82, FP = 236.16; effort ≈ 13.12 person-months; total cost ≈ **$68,224**.

2016–2017 • Q(c)–(d) CSE307 • 15/07/2017 • M023 MARKS 10 + $6\frac{2}{3}$

(c) Using the following decision tree find out the decision that a project manager should take in order to complete "System X".



(d) Describe Pareto Principle.

SOLUTION

(c) Decision-tree (make/buy) analysis

The square is the **decision node** (the manager chooses), the filled circles are **chance nodes** (nature chooses with the given probabilities) and the leaves are costs. The expected cost of a path is

$$\text{expected cost} = \sum_{\text{paths}} (\text{path probability}) \times (\text{estimated path cost}),$$

evaluated from right to left ("roll back"); at the decision node we pick the minimum.

Option	Computation	Expected cost
Build	$0.30 \times 380\text{K} + 0.70 \times 450\text{K} = 114\text{K} + 315\text{K}$	\$429,000
Reuse	major-change node: $0.20 \times 310\text{K} + 0.80 \times 490\text{K} = 62\text{K} + 392\text{K} = 454\text{K}$	
	$0.40 \times 275\text{K} + 0.60 \times 454\text{K} = 110\text{K} + 272.4\text{K}$	\$382,400
Buy	$0.70 \times 210\text{K} + 0.30 \times 400\text{K} = 147\text{K} + 120\text{K}$	**\$267,000**
Contract	$0.60 \times 350\text{K} + 0.40 \times 500\text{K} = 210\text{K} + 200\text{K}$	\$410,000



Answer. Buy is cheapest (expected cost \$267,000 vs Reuse \$382,400, Contract \$410,000, Build \$429,000), so the project manager should **buy** (acquire the off-the-shelf system and adapt it). Cost is not the only criterion: schedule, vendor support and complexity should also be considered.

(d) Pareto principle

The **Pareto principle** (80/20 rule, named after economist Vilfredo Pareto) states that roughly **80% of the effects come from 20% of the causes**. In software engineering it appears as:

- **Testing:** about 80% of the errors uncovered during testing are traceable to 20% of the program modules – one of the basic testing principles (Pressman). Testers should identify these error-prone modules and test them most thoroughly.
- **Quality assurance (SQA):** 80% of defects can be traced to 20% of all possible causes; statistical SQA collects defect data, categorises causes and attacks the "vital few" first.
- **Usage:** about 80% of the time users use only 20% of the features – so those features deserve the most attention in design, optimisation and testing.
- **Performance:** most execution time is spent in a small part of the code (hot spots), so profiling and optimising that part gives the largest benefit.

Use: it is a prioritisation tool – concentrate limited effort on the "vital few" causes rather than the "trivial many". It is an empirical rule of thumb; the exact ratio varies.

2016–2017 • Q(c) CSE307 • 15/07/2017 • M022 **MARKS 15**

Let's assume that, your company developed a system for which the development cost is $50,000. Your system analyst tells you that the operating cost for the next year is $5,000 and in each year the operating cost will increase by $1000. The expected cash inflow is given in the following table:

	Year 1	Year 2	Year 3	Year 4	Year 5	Year 6
Cash inflow	$18,000	$22,000	$26,000	$33,000	$41,000	$50,000

Considering discount rate of 11% calculate the payback period. Also calculate its ROI (Return on Investment) for these 6 years.

SOLUTION

Formulas

- **Present value** of an amount F received in year n at discount rate i: $PV = \dfrac{F}{(1+i)^n}$, discount factor $DF_n = 1/(1.11)^n$.
- **Net cash flow** of year n = cash inflow − operating cost; **NPV** = $\sum PV(\text{net}) -$ development cost.
- **Payback period** = time at which the *cumulative* discounted net cash flow becomes non-negative; within the crossing year we interpolate: payback $= (n-1) + \dfrac{|\text{cumulative at end of year } n-1|}{PV(\text{net in year } n)}$.
- **ROI** $= \dfrac{\text{PV of total benefits} - \text{PV of total costs}}{\text{PV of total costs}}$.

Operating costs: $5,000, 6,000, 7,000, 8,000, 9,000, 10,000 for years 1–6.

Discounted cash-flow table (amounts in $)

Year	Inflow	Op. cost	Net flow	$DF = 1/1.11^n$	PV of net	Cumulative PV
0	–	50,000	−50,000	1.0000	−50,000.00	−50,000.00
1	18,000	5,000	13,000	0.9009	11,711.71	−38,288.29
2	22,000	6,000	16,000	0.8116	12,985.96	−25,302.33
3	26,000	7,000	19,000	0.7312	13,892.64	−11,409.69
4	33,000	8,000	25,000	0.6587	16,468.27	**5,058.58**
5	41,000	9,000	32,000	0.5935	18,990.44	24,049.02
6	50,000	10,000	40,000	0.5346	21,385.63	45,434.66

Present values of the totals:

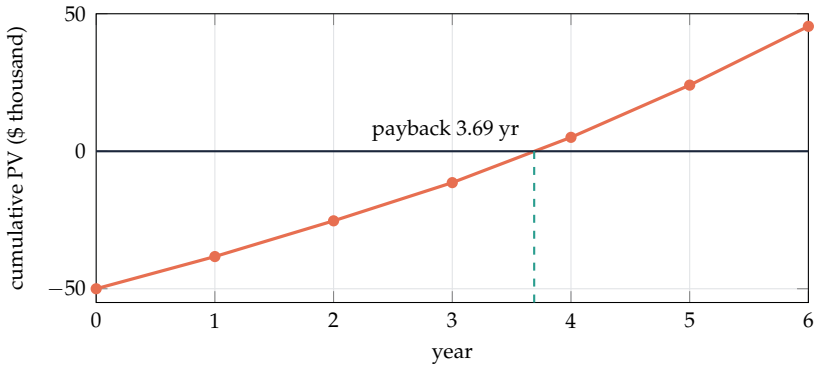
$$\begin{aligned} PV_{\text{benefits}} &= 16{,}216.22 + 17{,}855.69 + 19{,}010.98 + 21{,}738.12 + 24{,}331.50 + 26{,}732.04 = \$125{,}884.55 \\ PV_{\text{costs}} &= 50{,}000 + (4{,}504.50 + 4{,}869.73 + 5{,}118.34 + 5{,}269.85 + 5{,}341.06 + 5{,}346.41) \\ &= 50{,}000 + 30{,}449.90 = \$80{,}449.90 \end{aligned}$$

$$NPV = 125{,}884.55 - 80{,}449.90 = \$45{,}434.66 \ (> 0).$$

Payback period

The cumulative PV changes sign between year 3 (−11,409.69) and year 4 (+5,058.58):

$$\text{Payback} = 3 + \frac{11{,}409.69}{16{,}468.27} = 3 + 0.69 = \mathbf{3.69\ years}\ (\approx 3 \text{ years } 8 \text{ months}).$$



ROI over 6 years

$$ROI = \frac{125{,}884.55 - 80{,}449.90}{80{,}449.90} = \frac{45{,}434.66}{80{,}449.90} = 0.5648 = \mathbf{56.48\%}$$

$$\text{annual ROI} = 56.48\%/6 = 9.41\%.$$

NOTE Undiscounted figures for comparison: payback $= 3 + 2{,}000/25{,}000 = 3.08$ years; ROI $= (190{,}000 - 95{,}000)/95{,}000 = 100\%$. Since the question asks to consider the 11% discount rate, the discounted values above are the answer.

Answer. Discounted payback ≈ **3.69 years**; NPV = $45,434.66; ROI over 6 years = **56.48%** (about 9.4% per year above the 11% discount rate).

2014–2015 • Q3(a)–(e) CSE307 • 04/02/2016 • M026 **MARKS 10 + 7 + $6\frac{2}{3}$ + 8 + 15**

(a) In estimating a software development effort in function point method the following data are collected:

	Counts			
	Pessimistic	Most likely	Optimistic	Weight
Number of user inputs	40	30	20	5
Number of user outputs	10	8	7	5
Number of user inquiries	30	20	15	4
Number of files	8	5	4	6
Number external Interfaces	10	8	6	8
Algorithms	20	15	10	6

If the complexity multiplier is 0.75 and a person can implement eight function points in a week, find the estimated budget. Average salary of a developer is Tk. 50,000 per month.
(b) Write down the guide lines for estimating the software project.
(c) Write down the reasons behind the failure of software projects.
(d) What do you mean by project risk? Explain with example.
(e) Consider the following risks shown below for a Banking software development project:

Risk	**Probability**
Employee turnover	30%
Discontinuation of Development platform support	50%
Change of customer requirement	20%
Reduction number of customers in the market	40%
Missing delivery deadline	60%
Regulatory Constraints	70%
Change of technology	55%

Write down the impact of these risks with proper reasoning. Do formal risk analysis and show mitigation plan for the first five risks.

SOLUTION

(a) Function-point estimate with three-point counts

Because each count is given as pessimistic (p), most likely (m) and optimistic (o) values, we first take the **PERT (beta) expected value** of every count:

$$E = \frac{o + 4m + p}{6}.$$

Then $UFP = \sum E \times \text{weight}$, $FP = UFP \times \text{complexity multiplier}$, effort $= FP/\text{productivity}$.

Parameter	Expected count $E = (o + 4m + p)/6$	Weight	$E \times w$
User inputs	$(20 + 120 + 40)/6 = 30.00$	5	150.00
User outputs	$(7 + 32 + 10)/6 = 8.17$	5	40.83
User inquiries	$(15 + 80 + 30)/6 = 20.83$	4	83.33
Files	$(4 + 20 + 8)/6 = 5.33$	6	32.00
External interfaces	$(6 + 32 + 10)/6 = 8.00$	8	64.00
Algorithms	$(10 + 60 + 20)/6 = 15.00$	6	90.00
Count total (UFP)			**460.17**

$$FP = 460.17 \times 0.75 = 345.13 \text{ FP}$$

$$\text{Effort} = \frac{345.13 \text{ FP}}{8 \text{ FP/person-week}} = 43.14 \text{ person-weeks} = \frac{43.14}{4} = 10.79 \text{ person-months}$$

$$\text{Budget} = 10.79 \text{ PM} \times \text{Tk } 50{,}000 = \text{Tk } 539{,}258 \approx \text{Tk } 5.39 \text{ lakh.}$$

NOTE A month is taken as 4 working weeks (the usual exam convention). With $52/12 = 4.33$ weeks per month the effort is 9.96 PM and the budget Tk 497,800. "Algorithms" as a sixth parameter is Jones' *feature-point* extension of FP.

(b) Guidelines for estimating a software project

(i) **Fix the scope first** – estimate only after the functions, performance, constraints and interfaces are bounded.

(ii) **Decompose** the problem (by function/feature) and the process (by framework activity); estimate the small pieces and sum them.

(iii) **Use historical baseline data** (LOC/PM, FP/PM, cost/FP) from similar past projects of the organisation.

(iv) **Use at least two independent techniques** (e.g. FP-based, process-based, COCOMO, expert judgement) and reconcile them; a large gap means scope is not understood.

(v) **Use three-point estimates** (o, m, p) to capture uncertainty, and add contingency for identified risks.

(vi) **Involve the people who will do the work** and record the assumptions behind every number.

(vii) **Estimate size first**, then effort, then duration and cost; do not compress the schedule below what the people/effort relationship allows (Brooks' law).

(viii) **Re-estimate continuously** as the project proceeds – uncertainty shrinks as requirements and design mature (cone of uncertainty).

(c) Reasons behind the failure of software projects

- Unrealistic deadline imposed from outside the team; honest under-estimation of effort or resources.
- Unclear, incomplete or frequently changing customer requirements (scope creep) without change control.
- Poor planning, monitoring and tracking – slips are not noticed or not acted upon.

- Predictable and unpredictable risks that were never identified or managed.
- Technical difficulties that could not be foreseen; use of immature technology.
- Miscommunication among team members, management and customer; lack of user involvement.
- Lack of skilled people, staff turnover, weak management support and inadequate quality assurance.

(d) Project risk

A **risk** is a potential problem: an uncertain future event that, *if* it occurs, causes loss to the project. It has two characteristics – **uncertainty** (probability $< 100\%$) and **loss** (cost, schedule or quality impact). **Project risks** threaten the project plan: if they occur the schedule slips and costs rise (budget, schedule, personnel, resource, stakeholder and requirements problems). They are distinguished from *technical risks* (threaten quality/implementability) and *business risks* (threaten viability of the product).
Example: the only developer who knows the payment-gateway API leaves in the middle of the project (probability 30%). The impact is a delay of about 2 months while a replacement learns the system. Mitigation: pair programming and documentation so that knowledge is shared; contingency: a backup developer is already familiar with it.

(e) Formal risk analysis for the banking project

Following the **risk table** approach, each risk is categorised, its probability is taken from the table, its impact is assessed (1 = catastrophic, 2 = critical, 3 = marginal, 4 = negligible) and a priority score $= P \times$ severity weight (catastrophic 4, critical 3, marginal 2, negligible 1) is used to sort the table.

Risk	Cat.	P	Impact	Reasoning for impact	Score
Regulatory constraints	BU	0.70	1 (catastr.)	Central-bank rules (KYC, AML, data security) are mandatory; a non-compliant bank system cannot go live.	2.80
Missing delivery deadline	PS	0.60	2 (critical)	Penalty clauses, branch roll-out and user training are tied to the date; credibility lost.	1.80
Discontinuation of platform support	TE	0.50	2 (critical)	No security patches for a banking system is unacceptable; forced migration costs months.	1.50
Reduction of customers in market	BU	0.40	2 (critical)	Fewer buyers/users reduce revenue and the ROI of the product.	1.20
Change of technology	TE	0.55	3 (marg.)	New technology can usually be adopted in a later release; current system still works.	1.10
Employee turnover	ST	0.30	2 (critical)	Domain and code knowledge leaves with people; replacement and training delay the work.	0.90
Change of customer requirement	PS	0.20	3 (marg.)	Iterative development and change control can absorb most changes, at some rework cost.	0.40

Categories: BU business, PS project size/schedule, TE technology, ST staff. A **cut-off line** is drawn at score ≈ 1.0: the first five risks of the sorted table get active management, the last two are monitored only. Risk exposure for any risk can be quantified as $RE = P \times C$; e.g. if missing the deadline would cost Tk 20 lakh in penalties, $RE = 0.60 \times 20 =$ Tk 12 lakh, which is the amount of contingency budget that risk justifies.

Mitigation, monitoring and management (RMMM) plan for the first five risks in the question:

Risk	Mitigation (avoid / reduce)	Monitoring (indicators)	Management (contingency)
Employee turnover (30%)	Find causes of turnover; good pay/environment; pair programming, code reviews and documentation so knowledge is shared; assign a backup per critical module.	Staff morale, interpersonal relations, job-market signals, resignations.	Hand over to the backup; hire/contract replacements; re-plan schedule.
Platform support discontinued (50%)	Choose platforms with long-term vendor/community support; isolate platform-specific code behind abstraction layers.	Vendor road-maps, end-of-life notices, frequency of patches.	Port the isolated layer to an alternative platform; buy extended support meanwhile.
Change of requirements (20%)	Thorough elicitation with bank staff, prototyping, sign-off; formal change-control board.	Number and size of change requests per iteration.	Impact analysis; re-prioritise backlog; renegotiate time/cost.
Reduction of customers (40%)	Market study before investing; modular product that can be sold to other financial institutions.	Sales pipeline, market share, competitor launches.	Re-scope to core features; target new segments; reduce cost.
Missing deadline (60%)	Realistic estimation with several methods; buffer on the critical path; incremental delivery of core functions first.	Earned-value/progress tracking against milestones, slack consumption.	Add experienced staff early, overtime, deliver in increments, negotiate scope or date.

Answer. (a) UFP $= 460.17$, FP $= 345.13$, effort $= 43.14$ person-weeks ≈ 10.79 PM, budget $\approx$ Tk 5.39 lakh. (e) Sorted by risk score: regulatory $>$ deadline $>$ platform $>$ market $>$ technology $>$ turnover $>$ requirements; RMMM plans above for the first five risks.

2014–2015 • Q(d) CSE307 • 04/02/2016 • M028 MARKS $6\frac{2}{3}$

What is the value of the money after 5 years whose present value is \$100, assuming a 5% discount rate?

SOLUTION

Concept

Money has a **time value**: \$1 today can be invested to earn interest, so it is worth more than \$1 received later. The **future value** after n periods of a present amount P at rate i is

$$F = P\,(1+i)^n, \qquad \text{and conversely} \qquad P = \frac{F}{(1+i)^n}.$$

Calculation

$$F = 100 \times (1 + 0.05)^5 = 100 \times 1.27628 = \$127.63.$$

Year by year: $100 \to 105.00 \to 110.25 \to 115.76 \to 121.55 \to 127.63$.

NOTE If the question is read the other way round ("what is \$100 received after 5 years worth today?"), the answer is $P = 100/1.05^5 = \$78.35$.

Answer. \$100 today is worth **\$127.63** after 5 years at 5%.

2013–2014 • Q2(a)–(d) CSE307 • 08/08/2015 • M029 **MARKS 8 + 8 + $10\frac{2}{3}$ + 6**

(a) Why are the following factors considered in selecting software teams: (i) Degree of modularization of the problem; (ii) Rigidity of the delivery date; (iii) Size of the system to be developed; (iv) Quality of the programmers.
(b) Write down the general form of empirical estimation model. How are the equations developed?
(c) You are a CTO of a bank. Your management has asked you to establish mobile banking for the customers of the bank. You have studied the mobile banking solution market and found the following alternatives:

(i) Building the system by the in house programmer might require 100 Man-Month if it is simple and it might require 150 Man-Month if it is difficult. It is assumed that the probability that the system is different to build is 0.65.

(ii) Reusing the previously developed system in another good option. The estimated effort for minor and major change requirements to modify the system for mobile banking are 80 Man-Month and 120 Man-Month respectively.

(iii) Purchasing an off the shelf mobile banking system Tk. 50 lacs only. But the estimated effort for major and minor customizations are 40 Man-Month and 60 Man-Month respectively with equal probability.

If the cost of average Man Month is Tk. 80 thousand then which alternate should be the wisest decision for the CTO?
(d) "Reactive Risk Management is not a Risk Management" – Justify with appropriate example and reasoning.

SOLUTION

(a) Factors in selecting a software team structure

The team organisation (closed/democratic, centralised/chief-programmer, open, synchronous) should fit the characteristics of the project:

- **(i) Degree of modularisation:** if the problem decomposes into well-defined, loosely coupled modules, work can be split among sub-teams with little communication – a *centralised/hierarchical* structure works well. A problem that cannot be modularised needs heavy interaction, favouring a *decentralised (democratic)* team where everyone communicates.
- **(ii) Rigidity of the delivery date:** a tight, fixed deadline needs fast decisions and clear responsibility, so a *centralised* team with a strong leader (chief programmer) is preferred; democratic teams spend more time reaching consensus. Flexible dates allow decentralised, innovative teams.

- **(iii) Size of the system:** a large system (many KLOC/FP) needs many people; communication paths grow as $n(n-1)/2$, so the work must be structured hierarchically into sub-teams. Small systems can be done by one small democratic team.
- **(iv) Quality of the programmers:** highly skilled, experienced engineers work well in self-organising (agile/democratic) teams; a team with juniors needs a structure in which an experienced chief programmer designs and reviews, and the others implement under guidance.

Other factors (Pressman): difficulty of the problem, required quality and reliability, team lifetime and degree of sociability required.

(b) General form of an empirical estimation model

$$E = A + B \times (e_v)^C$$

where E is effort in person-months, e_v is the estimation variable (LOC/KLOC or FP), and A, B, C are empirically derived constants. Examples:

LOC-oriented		FP-oriented	
Walston–Felix	$E = 5.2\,(\text{KLOC})^{0.91}$	Albrecht–Gaffney	$E = -91.4 + 0.355\,FP$
Bailey–Basili	$E = 5.5 + 0.73\,(\text{KLOC})^{1.16}$	Kemerer	$E = -37 + 0.96\,FP$
Boehm (simple)	$E = 3.2\,(\text{KLOC})^{1.05}$	Matson et al.	$E = 585.7 + 15.12\,FP$

How the equations are developed:

(i) Collect size (LOC or FP) and actual effort data from a sample of completed projects.

(ii) Plot effort against size and fit a curve by **regression analysis** (least squares, usually on log-transformed data for the exponent C).

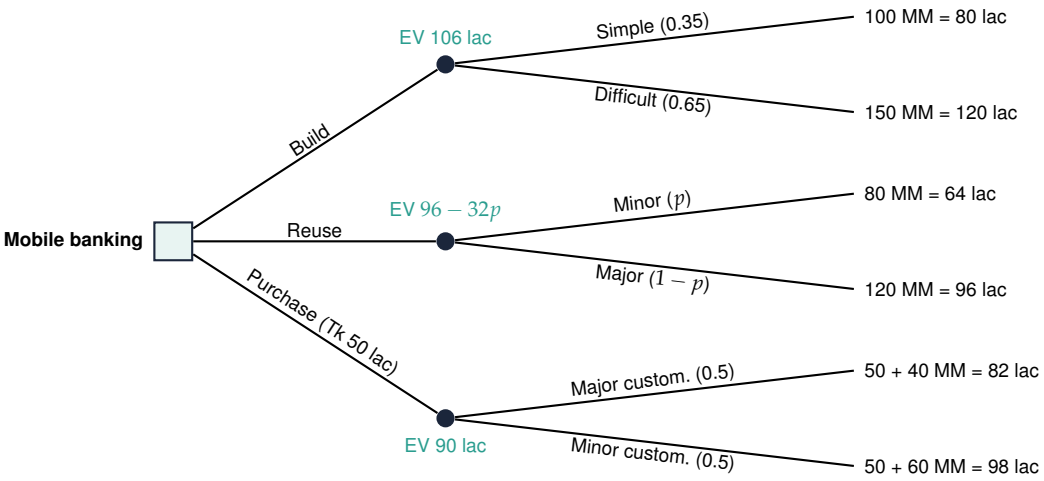
(iii) The fitted coefficients become A, B, C; the model is validated on other projects.

(iv) Because the sample is limited, the model must be **calibrated** to the local environment (re-fit with the organisation's own historical data) before use.

(c) Decision tree for the mobile-banking alternatives

NOTE The scan reads "probability that the system is *different* to build is 0.65", read as *difficult* (so simple = 0.35). The bank's own flag: the **reuse option gives no probabilities** for minor/major changes. Let $p = P(\text{minor change})$; the answer is derived as a function of p, and $p = 0.5$ (equal likelihood, as for the purchase option) is used for the final recommendation. Note also that, as printed, major customisation (40 MM) is cheaper than minor (60 MM); with equal probabilities this does not affect the expected value.

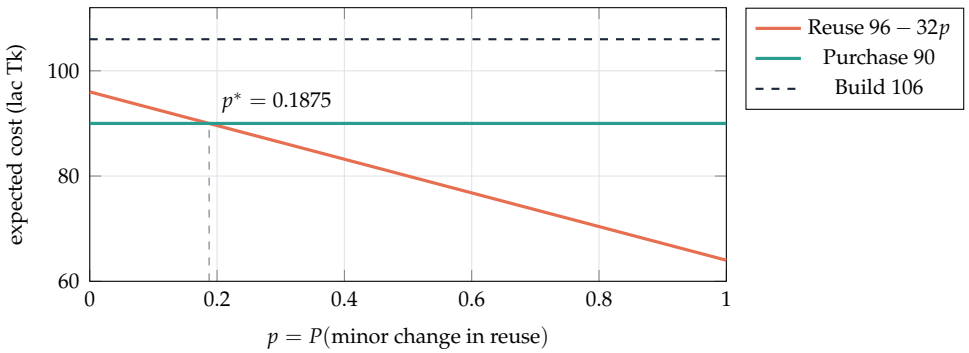
1 man-month costs Tk 80,000 = Tk 0.8 lac.



Alternative	Expected effort / cost	Expected cost
Build	$0.35 \times 100 + 0.65 \times 150 = 35 + 97.5 = 132.5$ MM; 132.5×0.8	Tk 106 lac
Reuse	$80p + 120(1-p) = 120 - 40p$ MM; $\times 0.8 = 96 - 32p$ lac	64–96 lac
	at $p = 0.5$: 100 MM $\times 0.8$	**Tk 80 lac**
Purchase	$50 + 0.8\,(0.5 \times 40 + 0.5 \times 60) = 50 + 0.8 \times 50 = 50 + 40$	Tk 90 lac

Sensitivity to the missing reuse probability

- Reuse vs Build: $96 - 32p < 106$ for every $p \in [0, 1]$ – reuse always beats building.
- Reuse vs Purchase: $96 - 32p < 90 \iff p > 0.1875$.



Answer. Expected costs: Build Tk 106 lac, Purchase Tk 90 lac, Reuse Tk $(96 - 32p)$ lac. Unless minor changes are very unlikely ($p < 0.1875$), **Reuse** is the wisest choice; with equal probability ($p = 0.5$) Reuse costs Tk 80 lac, saving Tk 10 lac over purchasing and Tk 26 lac over building.

(d) "Reactive risk management is not risk management"

Reactive risk management (the "Indiana Jones school") does nothing about risks until they become real problems; then the team rushes to fix them (*fire-fighting*/crisis management). At best it monitors the project for likely risks and sets resources aside, but no attempt is made to prevent them.

Risk management by definition deals with *uncertainty before it becomes a loss*: identify risks, analyse probability and impact, plan avoidance/mitigation, monitor and manage contingencies (RMMM).

Reactive handling skips all of these; it only manages *problems*, not *risks*. Therefore it is not really risk management.
Example: in a banking project the key DBA may resign. Reactive approach: when he resigns, the team suddenly discovers that nobody knows the schema, the release slips for two months and costs rise. Proactive approach: the risk (probability 30%, critical impact) was identified at planning time; knowledge was shared through documentation and pair work, and a backup DBA was trained – when the resignation happened, work continued with little delay. Proactive management is cheaper because avoiding a problem costs much less than recovering from it.

2013–2014 • Q2(e)–(f) CSE307 • 08/08/2015 • M030 MARKS 6 + 8

(e) Explain the risks associated with the following factors: (i) Reasonableness of delivery date; (ii) Sophistication of the customer; (iii) Deviation of the product size.
(f) Explain risk mitigation technique using the risk table.

SOLUTION

(e) Risks from the risk-item checklist

These factors come from Pressman's **risk item checklist**, used to identify generic risks:

- **(i) Reasonableness of delivery date** *(business-impact risk)*. If the deadline is imposed by marketing or the customer without a realistic estimate, the team must compress the schedule: quality drops (testing is cut), staff burn out, and the project is likely to be late or over budget. The risk is high when the date is fixed by external events and no slack exists; it is assessed by comparing the deadline with the estimated duration.
- **(ii) Sophistication of the customer** *(customer-characteristics risk)*. An unsophisticated customer may not know what he wants, cannot express requirements, does not understand the software process and changes requirements late; an over-confident "sophisticated" customer may dictate technical solutions or keep adding features. Both increase requirements ambiguity, rework and communication overhead. Questions to ask: has the customer worked with software engineers before? Will he take part in reviews? Does he understand the product's limitations?
- **(iii) Deviation of the product size** *(product-size risk)*. Effort, cost and schedule are all derived from estimated size (LOC/FP). If the actual size deviates from the estimate – because of scope creep, poor understanding or optimistic estimation – effort grows non-linearly (diseconomy of scale), so budget and deadline are exceeded. The larger the deviation from the average of past projects, the higher the risk.

(f) Risk mitigation using the risk table

A **risk table** lists every identified risk with its category, probability, impact and a reference to its RMMM plan:

Risk	Category	Probability	Impact	RMMM
Customer will change requirements	PS	80%	2	ref
High staff turnover	ST	70%	2	ref
Size estimate may be significantly low	PS	60%	2	ref
Delivery deadline will be tightened	BU	50%	2	ref
— cut-off line: risks below receive no further management attention —				
Less reuse than planned	PS	70%	3	
Staff inexperienced	ST	30%	2	

Impact: 1 catastrophic, 2 critical, 3 marginal, 4 negligible.

How mitigation is done with the table:

(i) **Build the table:** list risks (from the checklist), estimate probability and impact for each.

(ii) **Sort** by probability and then by impact, so high-probability/high-impact risks rise to the top.

(iii) **Draw a cut-off line**: risks above it are managed actively, those below are only re-evaluated later.

(iv) **Quantify exposure** for the risks above the line: $RE = P \times C$ (e.g. $P = 0.70$, cost of the risk $C = \$25{,}200 \Rightarrow RE = \$17{,}640$), which tells how much effort/budget mitigation is worth.

(v) **Write an RMMM plan** (referenced in the last column) for each such risk: **mitigation** – steps to avoid or reduce it in advance (e.g. for staff turnover: improve conditions, cross-train, document, assign backups); **monitoring** – indicators tracked to see whether it is becoming more likely; **management/contingency** – what to do if it actually occurs.

(vi) **Revisit** the table regularly; probabilities change and new risks appear.

Answer. (e) Unrealistic delivery date → schedule/quality risk; customer sophistication → requirements and communication risk; size deviation → effort/cost/schedule overrun. (f) Sort the risk table by probability and impact, draw a cut-off line, compute $RE = P \times C$ and attach mitigation–monitoring–management plans to risks above the line.

2013–2014 • Q8(a)–(b) CSE307 • 08/08/2015 • M031 **MARKS 5 + 10**

(a) What is the present value of \$1,000 one year from now, assuming a 10% discount rate?
(b) What is payback analysis? Explain briefly how it works.

SOLUTION

(a) Present value

$$PV = \frac{F}{(1+i)^n} = \frac{1000}{(1+0.10)^1} = \frac{1000}{1.10} = \$909.09$$

i.e. \$909.09 invested today at 10% grows to \$1,000 in one year.

(b) Payback analysis

Payback analysis is a cost–benefit (economic feasibility) technique that determines *how long it takes*

for the cumulative benefits of a system to repay its development and operating costs. The point at which this happens is the **payback period** (break-even point). Shorter payback means quicker recovery of the investment and less risk.

How it works:

(i) List the one-time development cost (year 0) and, for each year of the system's life, the operating costs and the expected benefits.

(ii) Convert each year's amount to present value with $DF_n = 1/(1+i)^n$ (discounted payback; simple payback skips this).

(iii) Compute the net flow per year (benefits − costs) and the *cumulative* net flow starting from −(development cost).

(iv) The payback period is the year in which the cumulative flow becomes non-negative; interpolate inside that year: $\text{payback} = (n-1) + \dfrac{|\text{cumulative}_{n-1}|}{\text{net}_n}$.

(v) Compare with the organisation's acceptable payback (e.g. ≤ 3 years) or with other alternatives.

Year	Net flow	PV at 10%	Cumulative
0	−10,000	−10,000	−10,000
1	4,000	3,636	−6,364
2	4,000	3,306	−3,058
3	4,000	3,005	−53
4	4,000	2,732	2,679

Example: payback $= 3 + 53/2{,}732 \approx 3.02$ years. *Limitation:* it ignores benefits after the payback point, so it should be used together with NPV/ROI.

Answer. (a) $PV = \$909.09$. (b) Payback analysis finds the time at which cumulative (discounted) benefits equal cumulative costs; it is computed with a cumulative cash-flow table.

2012–2013 • Q3(a)–(b) CSE307 • 17/05/2014 • M033 MARKS $6\frac{2}{3}$ + 15

(a) "To use LOC and FP for estimation, a historical baseline of information must be established" – explain.

(b) Suppose company A has taken up project S. Table 2 and 3 show the frequency and weight of the Function Points of project S respectively. External factors that affect project S are ranked from 0 to 5 and shown in Table 4. From historical data it can be found that the A's average productivity is 24 FP/pm and labor rate is $48000 per year. Calculate the estimated project cost and effort.

Table-2 (Weight assignment of Function Points)

FPs	Simple	Average	Complex
Inputs	2	4	6
Outputs	3	5	7
Files	5	10	15
Inquires	2	4	6
Interfaces	4	7	10

Table-3 (Frequency of Function Points)

FPs	Simple	Average	Complex
Inputs	–	4	2
Outputs	2	2	4
Files	2	5	1
Inquires	1	–	5
Interfaces	10	3	–

Table-4 (External Factors)

Factor	Rank
Complex internal processing	4
Distributed processing	5
High performance	4
Multiple sites	5
Code to be reusable	3
Data Communication	5
Online Data Entry	4
Data Backup and Recovery	5

SOLUTION

(a) Why a historical baseline is needed

LOC and FP are *size* measures. Size by itself is not effort or cost: to convert an estimated size into person-months and money we need **productivity metrics** of the organisation – LOC/PM or FP/PM, \$/LOC or \$/FP – and these can only come from **past projects** measured in the same way. That collection of measurements is the **historical baseline**:

$$\text{Effort} = \frac{\text{estimated size (LOC or FP)}}{\text{baseline productivity (LOC/PM or FP/PM)}}, \qquad \text{Cost} = \text{size} \times \text{baseline cost per unit}.$$

For the baseline to be useful its data must be

- **reasonably accurate** – vague guesses about past projects should be avoided;
- **collected from as many projects as possible** so averages are statistically meaningful;
- **consistent** – LOC must be counted the same way (comments, blank lines, language) and FP with the same rules;
- **from applications similar** to the one being estimated (domain, size, team, technology) – a productivity figure from embedded projects is meaningless for a web project.

Without such a baseline, LOC/FP estimates cannot be translated into effort, schedule or cost, and the estimate degrades to guesswork.

(b) Step 1 – Unadjusted function points (frequency × weight)

A dash in Table 3 means a frequency of 0.

Function	Simple + Average + Complex		Total
Inputs	$0\times2+4\times4+2\times6$	$=0+16+12$	28
Outputs	$2\times3+2\times5+4\times7$	$=6+10+28$	44
Files	$2\times5+5\times10+1\times15$	$=10+50+15$	75
Inquiries	$1\times2+0\times4+5\times6$	$=2+0+30$	32
Interfaces	$10\times4+3\times7+0\times10$	$=40+21+0$	61
UFP			**240**

Step 2 – Value adjustment factor

$$\sum F_i = 4+5+4+5+3+5+4+5 = 35, \qquad VAF = 0.65 + 0.01\times 35 = 1.00$$

NOTE Only 8 of the 14 general system characteristics are listed; the remaining six are assumed to be rated 0 (no influence).

$$FP = UFP \times VAF = 240 \times 1.00 = 240 \text{ FP}$$

Step 3 – Effort and cost

$$\text{Effort} = \frac{240 \text{ FP}}{24 \text{ FP/PM}} = 10 \text{ person-months}$$

$$\text{Labour rate} = \frac{\$48{,}000/\text{year}}{12} = \$4{,}000/\text{PM}, \qquad \text{Cost} = 10\times\$4{,}000 = \$40{,}000 \quad (\$166.67 \text{ per FP}).$$

Answer. UFP = 240, VAF = 1.00, FP = 240; estimated effort **10 person-months**; estimated cost **$40,000**.

2012–2013 • Q3(c)–(d) CSE307 • 17/05/2014 • M034 **MARKS 12 + (3+10=13)**

Context: Question 3(b) of this paper is the function-point problem of project S solved just above (M033): UFP from Tables 2–3, external factors of Table 4, productivity 24 FP/pm, labour rate $48,000 per year.
(c) Consider the decision tree shown in Figure 1 for project S. Difficult implementation of project S costs the same as deduced in question 3(b). Simple implementation of project S costs 75% of the difficult implementation. Given company A's priority is cost minimization, what will be your decision regarding the implementation of project S?

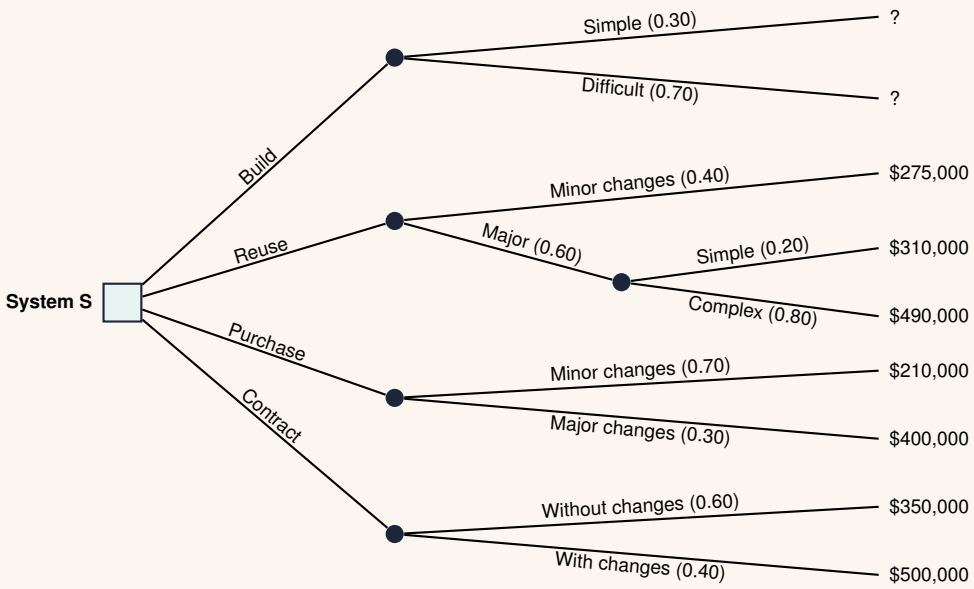


Figure-1 (Decision Tree) for Q. 3(c)

(d) Suppose your ISD lab group has been assigned to develop a "Library Management System" for BUET central library. (i) State one possible risk that can occur for each of the following risk types: technology, tools, and requirements. (ii) What is the probability and affect of each of the possible risks you identified? Justify your assessment.

SOLUTION

NOTE In the scanned figure the second Reuse branch is printed as "Minor changes (0.60)"; since the two Reuse probabilities must add to 1 and the first branch is already "Minor changes (0.40)", it is read as **Major changes (0.60)** (as in the standard Pressman tree). The question marks are the Build costs to be derived.

(c) Step 1 – Build costs from Q3(b)

From M033: $FP = 240$, effort $= 240/24 = 10$ PM, rate $= \$48{,}000/12 = \$4{,}000/\text{PM}$, so

$$C_{\text{difficult}} = 10 \times 4{,}000 = \$40{,}000, \qquad C_{\text{simple}} = 0.75 \times 40{,}000 = \$30{,}000.$$

Step 2 – Expected cost of every option

A **decision tree** is evaluated from the leaves back to the root: at each chance node the expected value is $EV = \sum_i P_i \times C_i$; at the decision node the option with the best (here lowest) expected cost is chosen.

Option	Expected cost computation	Expected cost
Build	$0.30 \times 30{,}000 + 0.70 \times 40{,}000 = 9{,}000 + 28{,}000$	**\$37,000**
Reuse	$0.40 \times 275{,}000 + 0.60\,[0.20 \times 310{,}000 + 0.80 \times 490{,}000]$	
	$= 110{,}000 + 0.60 \times (62{,}000 + 392{,}000) = 110{,}000 + 272{,}400$	\$382,400
Purchase	$0.70 \times 210{,}000 + 0.30 \times 400{,}000 = 147{,}000 + 120{,}000$	\$267,000
Contract	$0.60 \times 350{,}000 + 0.40 \times 500{,}000 = 210{,}000 + 200{,}000$	\$410,000

Step 3 – Decision

$\min(37{,}000;\ 382{,}400;\ 267{,}000;\ 410{,}000) = \$37{,}000$ for **Build**. Since company A's priority is cost minimisation, it should **build project S in-house**. Even in the worst case (difficult, \$40,000)

building is far cheaper than the expected cost of any other option, so the decision is robust to the probabilities.

> **EXAM TIP** The Build leaves come out much cheaper than the other leaves because project S is small (240 FP); the decision follows the data. Always show the roll-back computation, not only the chosen branch.

(d)(i) One risk of each type for the Library Management System

- **Technology risk:** the chosen database/server cannot handle the peak load of many students searching and issuing books at the same time (or the barcode/RFID reader hardware does not integrate with the software).
- **Tools risk:** the CASE/framework tools chosen by the group (e.g. an unfamiliar web framework or code generator) are inefficient or the group cannot use them effectively, so code is produced slowly.
- **Requirements risk:** the library staff change requirements during development (e.g. new fine rules, inter-library loans, e-book access), causing major rework.

(d)(ii) Probability and effect, with justification

Scales (Sommerville): probability – very low (<10%), low (10–25%), moderate (25–50%), high (50–75%), very high (>75%); effect – catastrophic, serious, tolerable, insignificant.

Risk	Probability	Effect	Justification
Technology: DB cannot handle load	Low (10–25%)	Serious	BUET library load (a few hundred concurrent users) is modest for a modern DBMS, so unlikely; but if it happens, the system is unusable at peak time and needs redesign/hardware.
Tools: tools inefficient / unfamiliar	Moderate (25–50%)	Tolerable	Student groups often pick new frameworks with a learning curve, so it is fairly likely; the effect is a delay that can be absorbed by extra hours or switching to a familiar tool.
Requirements: staff change requirements	High (50–75%)	Serious	Librarians see the system only when it runs and usually ask for changes; lab projects have fixed deadlines, so large rework may cause missing features or late submission.

Answer. (c) Expected costs: Build \$37,000, Reuse \$382,400, Purchase \$267,000, Contract \$410,000 ⇒ **Build**. (d) Technology: DB load (low, serious); tools: inefficient tools (moderate, tolerable); requirements: changing requirements (high, serious).

2012–2013 • Q5(b) CSE307 • 17/05/2014 • M035 MARKS 15

Consider the following cash flow description for an online auction management system which is ready to develop. Amounts given in dollar value. Assuming an 11% discount rate, what would be the payback period? Calculate annual ROI. Would be a good investment?

Year	Development Cost
0	588,040

Year	Estimated operation and maintenance cost	Estimated benefit
1	15,500	150,500
2	17,880	185,000
3	16,500	200,000
4	19,000	215,000
5	18,140	240,000
6	20,000	270,500

SOLUTION

Method

Discount every future amount with $DF_n = 1/(1+0.11)^n$, form the net benefit per year (benefit − O&M cost), and accumulate starting from the year-0 development cost. Payback is where the cumulative discounted flow turns positive;

$$ROI = \frac{PV_{\text{benefits}} - PV_{\text{costs}}}{PV_{\text{costs}}}, \qquad \text{annual ROI} = \frac{ROI}{\text{number of years}}.$$

Discounted cash-flow table (amounts in $)

Year	Benefit	Cost	Net	DF	PV of net	Cumulative PV
0	–	588,040	−588,040	1.0000	−588,040.00	−588,040.00
1	150,500	15,500	135,000	0.9009	121,621.62	−466,418.38
2	185,000	17,880	167,120	0.8116	135,638.34	−330,780.04
3	200,000	16,500	183,500	0.7312	134,173.62	−196,606.42
4	215,000	19,000	196,000	0.6587	129,111.27	−67,495.15
5	240,000	18,140	221,860	0.5935	131,663.11	**64,167.96**
6	270,500	20,000	250,500	0.5346	133,927.53	198,095.49

$$\begin{aligned} PV_{\text{benefits}} &= 135{,}585.59 + 150{,}150.15 + 146{,}238.28 + 141{,}627.16 + 142{,}428.32 + 144{,}620.35 \\ &= \$860{,}649.84 \\ PV_{\text{costs}} &= 588{,}040 + (13{,}963.96 + 14{,}511.81 + 12{,}064.66 + 12{,}515.89 + 10{,}765.21 + 10{,}692.82) \\ &= 588{,}040 + 74{,}514.35 = \$662{,}554.34 \end{aligned}$$

$$NPV = 860{,}649.84 - 662{,}554.34 = \$198{,}095.49.$$

Payback period

$$\text{Payback} = 4 + \frac{67{,}495.15}{131{,}663.11} = 4 + 0.51 = \textbf{4.51 years}\ (\approx 4 \text{ years } 6 \text{ months}).$$

ROI

$$ROI_{6\,\text{yr}} = \frac{198{,}095.49}{662{,}554.34} = 29.90\%, \qquad \text{annual ROI} = \frac{29.90\%}{6} = \textbf{4.98}\%.$$

Is it a good investment?

- **Yes, financially.** NPV is positive ($198,095) at an 11% discount rate, i.e. the project returns more than the 11% cost of capital; the 4.98% annual ROI is *in addition* to that 11% because it is computed on discounted values (the internal rate of return is about 20.9%).
- Payback (4.5 years) occurs inside the 6-year horizon, leaving 1.5 years of pure gain.
- **Caution:** the payback is fairly long, so the result is sensitive to the benefit estimates for years 5–6; if benefits are uncertain (auction market competition), a risk assessment or sensitivity analysis should accompany the decision.

Answer. Discounted payback $\approx$ **4.51 years**; NPV $= \$198{,}095$; ROI over 6 years $= 29.9\%$, annual ROI $\approx$ **4.98%**. With positive NPV and payback within the life time it is a good (if not spectacular) investment.

2011–2012 • Q6(a) CSE307 • 23/09/2013 • M037 **MARKS** $16\frac{2}{3}$

CSE department of BUET has got a very exciting project of modifying an existing banking solution. The solution is now working in different branches of many banks. But it has one major disadvantage that it does not use any centralized database. Now the task is to do a complete requirement analysis, design and redevelopment of the banking solution so that it can support online (any branch) banking. The project may need a complete technological up gradation. Also, CSE department has the problem of losing skilled personnel as teachers frequently leave for higher studies. Identify the risks associated in this project and describe how you plan to cope up with these.

SOLUTION

Approach

Risk management = **identify** $\rightarrow$ **analyse** (probability, impact) $\rightarrow$ **plan** (avoid, minimise, contingency) $\rightarrow$ **monitor**. Using the risk types (people, technology, requirements, tools, organisational, estimation, business) the risks of this project are:

Risk identification and analysis

Risk	Type	Prob.	Effect	Why
R1 Skilled personnel (teachers) leave for higher studies	People	High	Serious	Stated in the problem; knowledge of design leaves with them.
R2 New technology (central DB, online platform) unfamiliar or immature	Technology	High	Serious	"Complete technological upgradation" is needed.
R3 Migration of existing branch data to the central DB fails / corrupts data	Technology	Moderate	Catastrophic	Many branches, many banks, inconsistent local databases.
R4 Central DB/network cannot meet performance and availability for all branches	Technology	Moderate	Serious	Single point of failure; WAN links to remote branches.
R5 Security breach of centralised online banking	Technology	Moderate	Catastrophic	Online access increases attack surface; money involved.
R6 Requirements of different banks conflict or change	Require-ments	High	Serious	Many banks, each with its own procedures and regulations.
R7 Size/effort underestimated (complete redevelopment)	Estimation	High	Serious	Hidden business rules in legacy code.
R8 Users/banks resist change, live operations disrupted during cut-over	Organisa-tional	Moderate	Serious	System already running in many branches.
R9 Regulatory (central bank) compliance changes	Business	Low	Serious	Banking rules change periodically.

Risk planning – how to cope

- **R1 Staff turnover:** pair design/programming, mandatory documentation and code reviews so knowledge is shared; involve graduate students and permanent staff as backups for every critical module; plan handovers before known departures (leave dates are known in advance); keep the architecture modular so newcomers can take over a part.
- **R2 New technology:** conduct a technology spike/prototype early; train the team; select mature, well-supported DBMS and middleware; consult experts.
- **R3 Data migration:** analyse and clean legacy data, write and test migration scripts on copies, migrate bank by bank/branch by branch with reconciliation reports and a rollback plan.
- **R4 Performance/availability:** capacity planning and load testing; replicated/clustered database with fail-over; offline branch mode that synchronises when the link returns.
- **R5 Security:** security requirements from the start, encryption, strong authentication, audit trails, penetration testing.
- **R6 Requirements:** thorough elicitation with all banks, prototypes, a configurable core with bank-specific parameters, formal change-control board.
- **R7 Estimation:** reverse-engineer the legacy system first, estimate with several methods, add contingency, deliver incrementally and re-estimate.
- **R8 Cut-over:** run old and new systems in parallel at pilot branches, train users, phased roll-out.
- **R9 Regulation:** keep business rules configurable, liaise with the banks' compliance officers.

Risk monitoring

Review the risk list at each milestone: staff departure notices, progress against plan, number of change requests, defect and performance-test results, migration reconciliation errors. Update probabilities and plans accordingly.

Answer. Main risks: staff turnover, new/immature technology, data migration, performance/availability, security, conflicting/changing requirements, underestimation, user resistance and regulation. Cope by knowledge sharing and backups, early prototyping and training, staged migration with rollback, load/security testing, change control, incremental delivery and continuous risk monitoring.

2011–2012 • Q6(c) CSE307 • 23/09/2013 • M038 **MARKS 18**

The following Table sets out a number of tasks, their durations, and their dependencies. Draw a bar chart showing the project schedule.

Task	Duration	Dependencies	Task	Duration	Dependencies
T1	10	–	T9	15	T6
T2	15	T1	T10	5	T5, T9
T3	10	T1, T2	T11	10	T9
T4	20	–	T12	20	T10
T5	10	–	T13	35	T3, T4
T6	15	T3, T4	T14	10	T8, T9
T7	20	T3	T15	20	T2, T14
T8	35	T7	T16	10	T15

SOLUTION

What a bar chart is

A **bar chart** (Gantt chart) is a calendar-based schedule: time runs along the horizontal axis and each task is a horizontal bar from its start date to its end date. To draw it we first need every task's start date, which we get from a **forward pass** over the dependencies (a task starts when all its predecessors have finished). A **backward pass** then gives each task's **float/slack**, which is shown as a thin extension after the bar. Durations are taken in days.

Step 1 – Forward and backward pass

$ES = \max(EF_{\text{pred}})$, $EF = ES + d$; project end $= \max EF$; $LF = \min(LS_{\text{succ}})$ (or project end), $LS = LF - d$; slack $= LS - ES$.

Task	d	Pred.	ES	EF	LS	LF	Slack
T1	10	–	0	10	0	10	**0**
T2	15	T1	10	25	10	25	**0**
T3	10	T1, T2	$\max(10,25)=25$	35	25	35	**0**
T4	20	–	0	20	40	60	40
T5	10	–	0	10	95	105	95
T6	15	T3, T4	$\max(35,20)=35$	50	60	75	25
T7	20	T3	35	55	35	55	**0**
T8	35	T7	55	90	55	90	**0**
T9	15	T6	50	65	75	90	25
T10	5	T5, T9	$\max(10,65)=65$	70	105	110	40
T11	10	T9	65	75	120	130	55
T12	20	T10	70	90	110	130	40
T13	35	T3, T4	$\max(35,20)=35$	70	95	130	60
T14	10	T8, T9	$\max(90,65)=90$	100	90	100	**0**
T15	20	T2, T14	$\max(25,100)=100$	120	100	120	**0**
T16	10	T15	120	130	120	130	**0**

Key backward-pass steps: T11, T12, T13 and T16 have no successors, so their $LF = 130$. $LF_{T9} = \min(LS_{T10}, LS_{T11}, LS_{T14}) = \min(105, 120, 90) = 90$; $LF_{T4} = \min(LS_{T6}, LS_{T13}) = \min(60, 95) = 60$; $LF_{T3} = \min(LS_{T6}, LS_{T7}, LS_{T13}) = \min(60, 35, 95) = 35$; $LF_{T2} = \min(LS_{T3}, LS_{T15}) = \min(25, 100) = 25$; $LF_{T1} = \min(LS_{T2}, LS_{T3}) = 10$.

Step 2 – Critical path and project duration

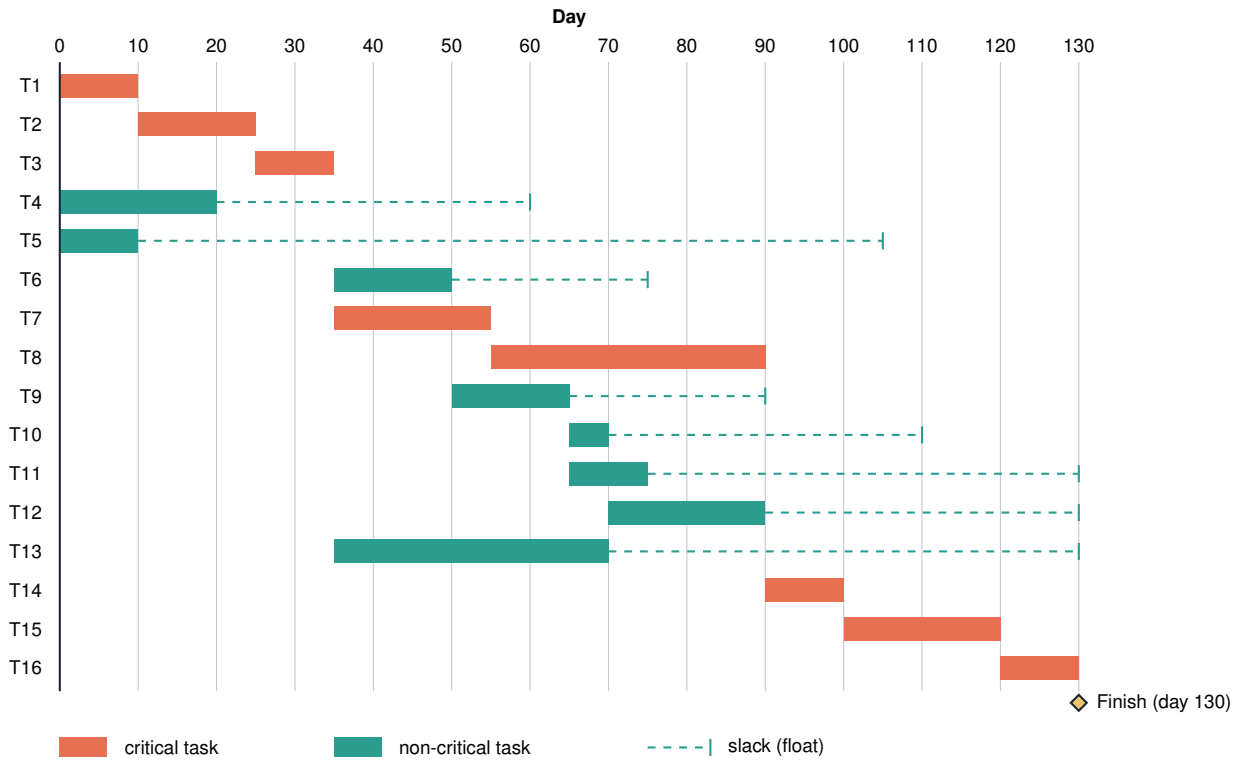
Zero-slack tasks form the critical path

$$\text{T1} \to \text{T2} \to \text{T3} \to \text{T7} \to \text{T8} \to \text{T14} \to \text{T15} \to \text{T16}$$

$$10 + 15 + 10 + 20 + 35 + 10 + 20 + 10 = 130 \text{ days.}$$

Step 3 – Bar chart

Solid coral bars are critical tasks; teal bars are non-critical tasks and the dashed line after a teal bar is its slack (how far it can slip without delaying the project).



Reading the chart

- T1, T4 and T5 start together on day 0; T5 has a huge float (95 days) so it can be scheduled whenever staff are free.
- T15 needs both T2 (finished day 25) and T14 (finished day 100), so it waits until day 100 – the dependency on T14 dominates.
- The schedule needs 130 days; any slip in a coral task moves the finish date, while e.g. T13 may slip up to 60 days.

Answer. Project duration = 130 days; critical path T1–T2–T3–T7–T8–T14–T15–T16. Start days: T1 0, T2 10, T3 25, T4 0, T5 0, T6 35, T7 35, T8 55, T9 50, T10 65, T11 65, T12 70, T13 35, T14 90, T15 100, T16 120.

2011–2012 • Q(c) CSE307 • 23/09/2013 • M039 **MARKS** $11\frac{2}{3}$

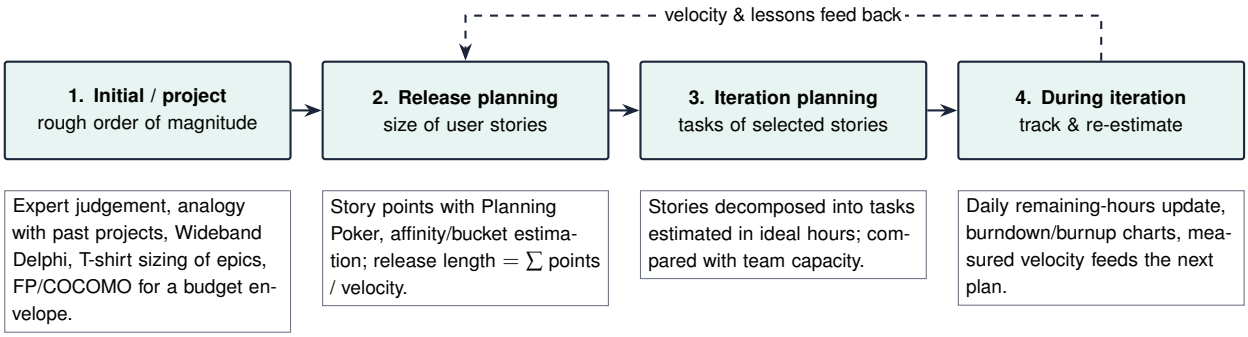
Briefly describe the techniques of estimating different stages of agile development.

SOLUTION

NOTE Agile *process* concepts are in the CSE 213 syllabus, but the course slides do not cover estimation techniques, so this question is kept with the project-estimation material.

Idea

Agile projects do not produce one big up-front estimate. Estimation is done *repeatedly*, at different levels of detail, and is corrected with measured data (velocity) after every iteration. The techniques used at each stage:



1. Initial (pre-project) estimation

Before the backlog is detailed, a coarse estimate is needed for the budget decision: **expert judgement**, **estimation by analogy** with completed projects, **Wideband Delphi** (anonymous rounds of expert estimates until they converge) or relative **T-shirt sizes** (S/M/L/XL) for epics.

2. Release planning – story points and Planning Poker

Each user story is sized in **story points**, a relative unit combining effort, complexity and uncertainty (often on a Fibonacci-like scale 1, 2, 3, 5, 8, 13, …). In **Planning Poker** every team member privately chooses a card; the highest and lowest estimators explain their reasoning and the team re-votes until consensus. With the team's **velocity** (points completed per iteration):

$$\text{number of iterations} = \frac{\sum \text{story points in release}}{\text{velocity}}, \qquad \text{cost} \approx \text{iterations} \times \text{cost per iteration}.$$

E.g. 120 points at a velocity of 20 points/sprint $\Rightarrow$ 6 two-week sprints $\approx$ 12 weeks.

3. Iteration (sprint) planning – task-level estimation

For the stories selected into the iteration, each story is broken into engineering tasks (design, code, test, integrate), and each task is estimated in **ideal hours** by the developer who will do it. Pressman describes exactly this: each user story is estimated separately (by decomposition into tasks, or by its size in LOC/FP/use-case points), the story estimates are summed to give the increment estimate, and this is compared with the time available for the increment.

4. During the iteration – tracking and re-estimation

The remaining hours of each task are re-estimated every day (daily stand-up) and plotted on a **burndown chart**. At the end of the iteration the actual **velocity** is measured and used for the next iteration and to re-forecast the release date. Thus estimates become more accurate as the project proceeds.

Answer. Initial: expert judgement/analogy/Delphi/T-shirt sizing; release: story points with Planning Poker and velocity; iteration: task decomposition in ideal hours vs capacity; execution: burndown and measured velocity for re-estimation.

2011–2012 • Q(c) CSE307 • 23/09/2013 • M040 **MARKS 15**

How COCOMO II model for cost estimation can be used at various stages of a project life cycle?

SOLUTION

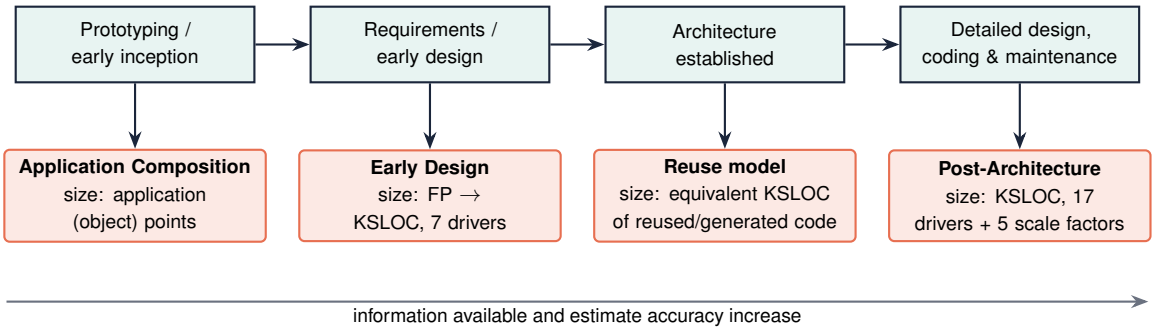
COCOMO II in one line

COCOMO II (Boehm et al., 2000) is an empirical, algorithmic cost model. Instead of one formula

it provides a *hierarchy of sub-models*: the earlier the life-cycle stage, the less information is available, so the model uses a coarser size measure and fewer cost drivers. Their common form is

$$PM = A \times \text{Size}^{B} \times \prod_{i} EM_i,$$

where PM is effort in person-months, Size is in KSLOC, $A \approx 2.94$ is a calibration constant, B is the scale exponent and EM_i are effort multipliers (cost drivers).



Stage 1 – Application Composition model (prototyping)

Used for projects built from reusable components/GUI builders and for early prototyping. Size is measured in **application (object) points**: the weighted number of screens, reports and 3GL modules.

$$PM = \frac{NAP \times (1 - \%\text{reuse}/100)}{PROD}$$

NAP = number of application points, $PROD$ = developer productivity (from 4 for very low to 50 for very high experience/tool maturity).

Stage 2 – Early Design model (after requirements)

Used when requirements are agreed but the architecture is not. Size is estimated in **function points**, converted to KSLOC with language tables. $PM = A \times \text{Size}^{B} \times M$ with $A = 2.94$, $B \approx 1.1$–1.24 and M the product of only **7 coarse multipliers**: product reliability & complexity (RCPX), required reuse (RUSE), platform difficulty (PDIF), personnel capability (PERS), personnel experience (PREX), schedule (SCED) and support facilities (FCIL). It is used to compare options and set the budget.

Stage 3 – Reuse model (integration of reused and generated code)

Computes the effort to integrate reused or automatically generated components. Reused code is converted into **equivalent new lines**:

$$ESLOC = ASLOC \times (1 - AT/100) \times AAM, \qquad PM_{\text{auto}} = \frac{ASLOC \times AT/100}{ATPROD}$$

($ASLOC$ = adapted lines, AT = % automatically generated, AAM = adaptation adjustment multiplier that reflects design/code/integration changes, software understanding and unfamiliarity). ESLOC is then added to the new code size.

Stage 4 – Post-Architecture model (design, coding, maintenance)

Once the architecture is designed, size can be estimated in KSLOC (new + equivalent reused + allowance for requirements volatility). The full model is applied:

$$PM = A \times \text{Size}^{B} \times \prod_{i=1}^{17} EM_i, \qquad B = 0.91 + 0.01 \sum_{j=1}^{5} SF_j, \qquad TDEV = 3.67 \times PM^{0.28+0.2(B-0.91)}$$

with **17 cost drivers** in four groups – product (RELY, DATA, CPLX, RUSE, DOCU), platform (TIME, STOR, PVOL), personnel (ACAP, PCAP, PCON, APEX, PLEX, LTEX), project (TOOL, SITE, SCED) – and **5 scale factors** (precedentedness, development flexibility, architecture/risk resolution, team cohesion, process maturity). (Sommerville's textbook writes the exponent as $1.01 + \sum SF/100$; the idea is identical.) The same model is used during **maintenance**, with size = changed lines $\times$ maintenance adjustment.

Why this staging is useful

- It matches the *cone of uncertainty*: an estimate at inception may be off by a factor of 4, after architecture by about $\pm 25\%$; COCOMO II asks only for data that exists at that stage.
- Estimates are refined at each milestone, enabling re-planning and early detection of overruns.
- It explicitly supports modern practice: prototyping, reuse/COTS, code generation and iterative development.

Summary of the stages

Stage	Sub-model	Size measure / formula	Adjustments
Prototyping, early inception	Application Composition	$PM = NAP(1 - \%\text{reuse}/100)/PROD$	Productivity 4–50 NAP/month
After requirements	Early Design	FP $\rightarrow$ KSLOC; $PM = 2.94\,\text{Size}^B M$	7 coarse multipliers
Reuse and integration	Reuse model	$ESLOC = ASLOC(1 - AT/100)AAM$	Understanding, unfamiliarity, adaptation
After architecture, coding, maintenance	Post-Architecture	$PM = 2.94\,\text{Size}^B \prod EM_i$, $B = 0.91 + 0.01 \sum SF$	17 cost drivers, 5 scale factors

Each successive sub-model uses the more detailed information available at that stage, so the estimate is refined as the project moves through the life cycle.

Answer. COCOMO II uses Application Composition (object points) for prototyping, Early Design (FP, 7 drivers) after requirements, the Reuse model for reused/generated code, and Post-Architecture (KSLOC, 17 cost drivers, 5 scale factors) once the architecture exists and through maintenance – giving progressively more accurate estimates.

2010–2011 • Q5(a) CSE307 • 19/03/2012 • M042 **MARKS 5 + 10 + 15**

What do you know about COCOMO model? Describe the basic COCOMO model with its limitations. How COCOMO II model for cost estimation can be used at various stages of a project lifecycle?

SOLUTION

(i) COCOMO model

COCOMO (*COnstructive COst MOdel*, Barry Boehm, 1981) is an **empirical/algorithmic estimation model**: its equations were derived by regression analysis on data from 63 completed projects. It predicts *effort*, *development time* and *staff size* from the estimated size of the product (KLOC). It

exists in three increasingly detailed forms:

- **Basic COCOMO** – effort from size and project mode only;
- **Intermediate COCOMO** – basic effort multiplied by an effort adjustment factor (EAF) from 15 cost drivers (product, hardware, personnel and project attributes);
- **Detailed COCOMO** – cost drivers applied phase by phase.

It was later replaced by **COCOMO II** (2000) to cover reuse, COTS, prototyping and iterative processes.

(ii) Basic COCOMO

A project is classified into one of three **modes**:

- **Organic** – small team, familiar, in-house environment, flexible requirements (e.g. payroll system);
- **Semi-detached** – medium size, mixed experience, some rigid requirements (e.g. DBMS, compiler);
- **Embedded** – tight hardware/software/operational constraints (e.g. avionics, ATM network control).

$$E = a\,(\text{KLOC})^{b} \text{ person-months}, \qquad D = c\,(E)^{d} \text{ months}, \qquad \text{Staff} = E/D \text{ persons}.$$

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Worked example (32 KLOC):

Mode	E (PM)	D (months)	Staff
Organic	$2.4 \times 32^{1.05} = 91.3$	$2.5 \times 91.3^{0.38} = 13.9$	6.6
Semi-detached	$3.0 \times 32^{1.12} = 145.5$	$2.5 \times 145.5^{0.35} = 14.3$	10.2
Embedded	$3.6 \times 32^{1.20} = 230.4$	$2.5 \times 230.4^{0.32} = 14.3$	16.2

Since $b > 1$, effort grows faster than size (diseconomy of scale), while duration grows slowly.

Limitations of basic COCOMO

- Requires KLOC as input, which is hardest to know early in the project when the estimate is most needed; LOC is also language dependent.
- Ignores cost drivers such as personnel capability, tool support, reliability and hardware constraints – only the mode adjusts the estimate, so accuracy is low (roughly within a factor of 2 only).
- Mode selection is subjective, and real projects often fall between modes.
- Calibrated on 1970s waterfall projects: it does not model reuse, COTS, object-oriented/4GL development, prototyping, incremental or agile processes.
- Needs re-calibration with local historical data; does not account for requirements volatility or documentation.

(iii) COCOMO II at various stages of the life cycle

This part is the same question as M040 above (see its full solution and life-cycle figure). In summary:

Stage	Sub-model	Size measure / formula	Adjustments
Prototyping, early inception	Application Composition	$PM = NAP(1 - \%\text{reuse}/100)/PROD$	Productivity 4–50 NAP/month
After requirements	Early Design	FP $\to$ KSLOC; $PM = 2.94\,\text{Size}^{B} M$	7 coarse multipliers
Reuse and integration	Reuse model	$ESLOC = ASLOC(1 - AT/100)AAM$	Understanding, unfamiliarity, adaptation
After architecture, coding, maintenance	Post-Architecture	$PM = 2.94\,\text{Size}^{B} \prod EM_i$, $B = 0.91 + 0.01 \sum SF$	17 cost drivers, 5 scale factors

Each successive sub-model uses the more detailed information available at that stage, so the estimate is refined as the project moves through the life cycle.

Answer. COCOMO is Boehm's regression-based model; basic COCOMO: $E = a\,\text{KLOC}^{b}$, $D = c\,E^{d}$ with organic/semi-detached/embedded constants; limited by early-KLOC dependence, no cost drivers and outdated data. COCOMO II applies Application Composition, Early Design, Reuse and Post-Architecture sub-models at successive life-cycle stages.

2010–2011 • Q8(b)–(d) CSE307 • 19/03/2012 • M044 MARKS $15\frac{2}{3}$ + (3+3) + 20

(b) Suppose you are the project manager of a newly founded software firm and you are estimating the cost of a customized VOIP software to quote in a tender process. The market is very competitive, but you have the advantage of having some skilled persons in your firm. What pricing factors you should consider while making that cost estimation and why?
(c) Describe the following software cost estimation techniques: (i) Parkinson's Law. (ii) Pricing to win.
(d) Find the critical path of a project using PERT chart base on the following table describing task dependencies and the expected duration of each task.

Task	A	B	C	D	E	F	G	H	I	J	K
Prerequisite	–	–	A	B,C	C	D	E,F	G	G	H,I	J
Duration	2	3	3	5	1	4	3	2	3	5	6

SOLUTION

NOTE The table is split over two source pages (A, B on the first; C–K on the second); they are joined here. Note that in this paper **A takes 2** and **I takes 3** time units, so the answer differs from M024/M032.

(b) Pricing factors for the VOIP tender

Software **pricing** is not simply "cost + profit". The price quoted is influenced by organisational, economic, political and business considerations (Sommerville). For our situation the relevant factors are:

Factor	Meaning	Why it matters for this VOIP tender
Market opportunity	A firm may quote a low price to enter a new market segment; profit comes from later projects.	We are a *new* firm in a *very competitive* market: winning a first reference customer may justify a lower margin.
Cost-estimate uncertainty	If the estimate is uncertain, a contingency is added to the price.	Customised VOIP (codecs, QoS, NAT traversal, telecom regulation) is technically risky; add contingency for unknowns.
Contractual terms	Who owns the source code, maintenance obligations, payment schedule.	If we keep the IPR and can reuse the VOIP core, we can charge less; a fixed-price contract needs more contingency.
Requirements volatility	If requirements are likely to change, a low initial price can be recovered from change requests.	Customised software usually attracts change requests; price the baseline competitively, price changes separately.
Financial health	A firm in difficulty may lower its price to win any contract.	A newly founded firm needs cash flow, but must not quote below cost so far that the project bankrupts it.
Skilled staff (productivity)	Higher productivity means lower effort and therefore lower cost.	Our skilled persons reduce effort and risk – a genuine competitive advantage that allows a lower but still profitable bid.

In addition the estimate itself should be built bottom-up (size $\rightarrow$ effort $\rightarrow$ cost: salaries, hardware/licences, travel, overheads) and then the *price* adjusted with the above business factors.

(c)(i) Parkinson's Law

Parkinson's Law states that "*work expands to fill the time available*". As an estimation technique, the cost is determined by the *available resources* rather than by an objective assessment of the work:

$$\text{Effort} = (\text{people available}) \times (\text{time available})$$

e.g. if the software must be delivered in 12 months and 5 people are available, the effort is estimated as $5 \times 12 = 60$ person-months. It is simple, but it is not a true estimate: it may grossly over- or under-estimate the real work and gives no information about whether the product can actually be finished.

(c)(ii) Pricing to win

The software cost is estimated to be *whatever the customer has available to spend* (or whatever the competitor's price is). The estimated effort depends on the customer's budget, not on the software functionality.

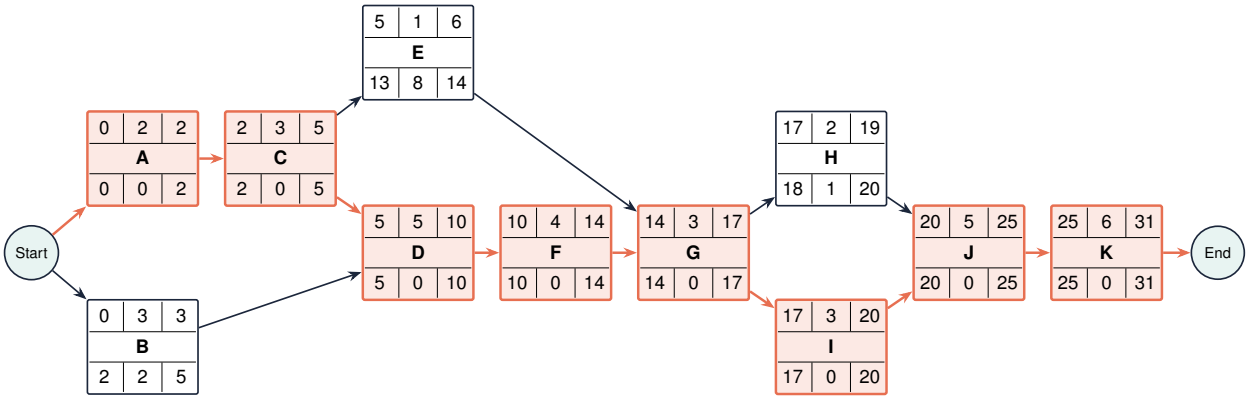
- **Advantage:** the contract is likely to be won.
- **Disadvantage:** the probability that the customer gets the system he wants is small; costs may not accurately reflect the work, leading to losses or a reduced-functionality product. It is usable when detailed requirements are not yet available and the functionality is later negotiated to fit the agreed price.

(d) PERT/CPM analysis

The durations given are the *expected* durations t_e (if three estimates were given we would first compute $t_e = (a + 4m + b)/6$). Forward pass: $ES = \max(EF_{\text{pred}})$, $EF = ES + d$; backward pass: $LF = \min(LS_{\text{succ}})$, $LS = LF - d$; slack $= LS - ES$.

Task	Prereq.	d	ES	EF	LS	LF	Slack	Critical?
A	–	2	0	2	0	2	0	✓
B	–	3	0	3	2	5	2	×
C	A	3	2	5	2	5	0	✓
D	B, C	5	$\max(3,5) = 5$	10	5	10	0	✓
E	C	1	5	6	13	14	8	×
F	D	4	10	14	10	14	0	✓
G	E, F	3	$\max(6,14) = 14$	17	14	17	0	✓
H	G	2	17	19	18	20	1	×
I	G	3	17	20	17	20	0	✓
J	H, I	5	$\max(19,20) = 20$	25	20	25	0	✓
K	J	6	25	31	25	31	0	✓

Backward pass details: $LF_K = 31$; $LF_J = LS_K = 25$; $LF_H = LF_I = LS_J = 20$ so $LS_H = 18$, $LS_I = 17$; $LF_G = \min(18, 17) = 17$, $LS_G = 14$; $LF_E = LF_F = 14$; $LF_D = LS_F = 10$, $LS_D = 5$; $LF_C = \min(LS_D, LS_E) = \min(5, 13) = 5$; $LF_B = LS_D = 5$, $LS_B = 2$; $LF_A = LS_C = 2$, $LS_A = 0$.
PERT chart (node: ES | Dur | EF over LS | Slack | LF; critical path in coral):



Path lengths: A–C–D–F–G–I–J–K $= 2+3+5+4+3+3+5+6 = 31$; A–C–D–F–G–H–J–K $= 30$; B–D–F–G–I–J–K $= 29$; A–C–E–G–I–J–K $= 23$. The longest one is the critical path.

Answer. (b) Market opportunity, estimate uncertainty (contingency), contractual terms/IPR, requirements volatility, financial health, and our skilled staff's productivity. (c) Parkinson: effort = available people × available time; pricing to win: cost = what the customer will pay. (d) Critical path **A–C–D–F–G–I–J–K**, project duration **31** time units; slack: B = 2, E = 8, H = 1.

2010–2011 • Q(d) CSE307 • 19/03/2012 • M041 **MARKS 4 + 10**

(i) What is the present value of \$1,000 one year from now, assuming a 10% discount rate?
(ii) Explain the difference between **return on investment** and **payback period**. Why might you need to know both in order to decide which of two different development alternatives represents

a better investment for a particular organization?

SOLUTION

(i) Present value

$$PV = \frac{1000}{(1+0.10)^1} = \$909.09$$

(Same computation as M031(a).)

(ii) ROI versus payback period

	Return on investment (ROI)	**Payback period**
Question answered	*How much* do we gain relative to what we spend?	*How soon* do we get our money back?
Formula	$\frac{\text{total benefits} - \text{total costs}}{\text{total costs}}$ (over the whole life, usually PVs)	Time at which cumulative benefits = cumulative costs
Unit	percentage (profitability)	years/months (liquidity, risk)
Considers	all benefits over the life of the system	only flows up to the break-even point
Weakness	ignores *timing* – a large return late in life looks as good as an early one	ignores everything *after* payback, hence total profitability

Why both are needed

The two measures can rank alternatives differently. Example:

- Alternative A: cost \$100K, benefits \$60K in each of years 1–2 and nothing after. Payback ≈ 1.7 years, ROI $= 20\%$.
- Alternative B: cost \$100K, benefits \$20K/year for 10 years. Payback $= 5$ years, ROI $= 100\%$.

B is more profitable (higher ROI) but A recovers the money much sooner. Which is "better" depends on the organisation: a firm short of cash, or in a fast-changing market where long-range benefit estimates are uncertain, values quick payback; a stable, well-funded organisation values the higher ROI. Knowing both shows profitability *and* risk/liquidity, so the decision matches the organisation's financial situation and risk tolerance (NPV can be added to account for the time value of money).

Answer. (i) \$909.09. (ii) ROI measures overall profitability; payback measures speed of recovery. Alternatives can rank differently on the two, so both are needed to weigh profit against risk and cash-flow needs.

2010–2011 • Q(c) CSE307 • 19/03/2012 • M043 **MARKS 15**

Your company has previously made the very successful game BUET Premier League 2011 (BPL 2011). But new 3D technology has arrived and the introduction of Kinect sensor requires you to upgrade the game to a newer version. Now, the overall market condition is not good due to economic recession, and this game may be a key to success for your company over the other game developing firms in the market. Identify the associated risks in this project and describe how you plan to cope up with these.

SOLUTION

Risk identification and analysis

Risk	Type	Prob.	Effect	Why
R1 Team lacks 3D and Kinect (motion-sensing) skills	People / technology	High	Serious	Both technologies are new to the team that built BPL 2011.
R2 Kinect SDK / 3D engine immature, buggy or changes	Technology / tools	Moderate	Serious	Newly introduced hardware and APIs.
R3 Existing 2D code cannot be reused; architecture must be rebuilt	Technology / estimation	High	Serious	3D rendering and gesture input change the core loop.
R4 Recession: funding cut, budget overrun not affordable	Business / organisational	Moderate	Catastrophic	Market condition is poor.
R5 Competitors release a similar 3D/Kinect cricket game first	Business	Moderate	Serious	Many firms adopt the new technology.
R6 Low sales: customers cannot afford Kinect/new hardware	Business	Moderate	Serious	Recession reduces spending.
R7 Performance problems (frame rate, gesture latency)	Technology	Moderate	Serious	3D + real-time sensing are demanding.
R8 Schedule underestimated; key staff leave	Estimation / people	Moderate	Serious	New technology makes estimates uncertain; competitors may hire skilled staff.

Coping strategies (avoidance, minimisation, contingency)

- **R1 Skills:** train staff early on the 3D engine and Kinect SDK, hire one or two experienced 3D/Kinect developers or consultants, pair them with existing developers.
- **R2 Immature technology:** build a technical prototype (gesture-controlled batting/bowling) in the first iteration; use a mature commercial 3D engine; isolate Kinect access behind an input-abstraction layer so the game also works with a gamepad/keyboard and SDK changes affect one module only.
- **R3 Reuse:** reuse game rules, team/player data, AI and assets from BPL 2011 where possible; re-architect only the rendering and input layers; estimate after the prototype.
- **R4 Funding:** incremental development with a minimum viable product, strict cost tracking, phased budget approval; seek publisher/sponsor (e.g. cricket board) support.
- **R5 Competition:** time-to-market focus – release a core version early, then updates/DLC; use the strong BPL brand and existing fan base in marketing.
- **R6 Market:** support non-Kinect play so the game sells to players without the sensor; flexible pricing; market research before full investment.
- **R7 Performance:** define frame-rate/latency targets, profile continuously, test on target hardware early.
- **R8 Schedule/staff:** buffers on critical path, re-estimate each iteration, knowledge sharing and retention incentives.

Monitoring

Track prototype results, velocity versus plan, budget consumption, SDK release notes, competitor announcements and market reports; re-assess the risk table at every milestone, and invoke contingency plans when indicators cross thresholds.

Answer. Key risks: new 3D/Kinect skills and immature SDKs, re-architecture effort, recession-driven funding and sales risks, competitors' time-to-market, performance and schedule/staff risks. Cope through early prototyping, training/hiring, abstraction of the input layer, reuse of BPL 2011 assets, incremental release of an MVP, non-Kinect support, cost control and continuous risk monitoring.

X6

Security & Dependability

OUT OF SYLLABUS

2023–2024 • Q6(b) CSE213 • 10/05/2026 • S001 MARKS 10

During online registration night, the university portal becomes unavailable. One engineer argues that the failure is due to ordinary overload caused by many legitimate student requests at the same time. Another argues that it may be a Denial-of-Service (DoS) attack.

(i) Explain how these two possibilities can be distinguished in practice.

(ii) Briefly discuss two defenses that could help mitigate the problem.

SOLUTION

Concepts

Both cases attack **availability**: the portal cannot serve requests in time.

- **Legitimate overload** (a "flash crowd") — genuine users, each behaving normally, collectively exceed the capacity of the servers, database or network.
- **Denial-of-Service (DoS)** — a *deliberate* attempt to make the service unavailable by flooding it with useless traffic or exhausting a resource (bandwidth, connections, CPU, memory). A *Distributed* DoS (DDoS) uses many machines (a botnet).

The symptom ("site is down") is identical; the difference lies in *who* sends the traffic, *what* it looks like and *how* it behaves over time.

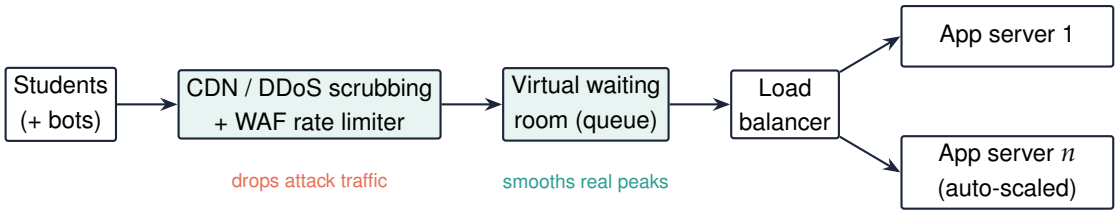
(i) Distinguishing the two in practice

The engineers should inspect web-server and load-balancer logs, network-flow data, monitoring dashboards and IDS/WAF alerts, and compare against the baseline of previous registration nights:

Indicator	Legitimate overload	DoS / DDoS attack
Who sends requests	Authenticated students; campus and local ISP IP ranges	Unknown, spoofed or foreign IPs; botnet; or a few IPs at extreme rates
Number of clients	Bounded by the number of eligible students	Far exceeds the student population
Request content	Normal workflow: login → course list → submit, with valid session cookies	Repeated identical/random URLs, malformed packets, half-open TCP (SYN flood), no sessions
Per-client rate	Human pace (a few requests per minute, plus refreshes)	Hundreds or thousands per second from one source
Timing	Starts exactly when registration opens and decays as students finish	Can start any time, sustained or pulsed, restarts after mitigation
Bottleneck	Application/DB CPU, DB locks on registration tables	Network link or connection table saturated before requests reach the app
Reaction to a waiting room / CAPTCHA	Load falls smoothly; users wait their turn	Traffic does not drop; bots ignore or fail challenges

In short: if traffic correlates with the registration schedule, comes from real student accounts and follows the normal workflow, it is overload; if it comes from unexpected sources with abnormal patterns, it is an attack. (Both can coexist: attackers often pick busy nights.)

(ii) Two defences



(i) Traffic filtering and rate limiting. Put the portal behind a CDN/DDoS-scrubbing service and a web-application firewall. Limit requests per IP and per student account (e.g. 10 requests/s), drop malformed packets, use SYN cookies, block or challenge (CAPTCHA) suspicious sources and geographies. This removes attack traffic before it reaches the servers, and also stops a single over-eager user from hogging capacity.

(ii) Capacity and load management. Add load balancing with auto-scaling of application servers, cache static pages, and — most effective for registration — a *virtual waiting room*: requests beyond capacity are queued and admitted in order, or registration is *staggered* by batch/department. This handles genuine peaks gracefully instead of crashing, and degrades service rather than losing it.

Other useful measures: monitoring with alerts on traffic anomalies, ISP-level black-holing during large attacks, and graceful degradation (disable non-essential features during peaks).

Answer. Distinguish by traffic source, client count vs. students, request pattern, per-client rate, timing and bottleneck (overload = real students following the schedule; DoS = abnormal sources and patterns). Mitigate with (1) CDN/WAF filtering and rate limiting and (2) load balancing, auto-scaling and a virtual waiting room / staggered registration.

2023–2024 • Q7(c) CSE213 • 10/05/2026 • S002 **MARKS 10**

Consider the following login program that checks a password and sets an authentication flag accordingly.

```c
#include <stdio.h>
#include <string.h>

struct LoginData {
    char password[8];
    int isAuthenticated;
};

void check_login(const char *input) {
    struct LoginData data;
    data.isAuthenticated = 0;

    strcpy(data.password, input);

    if (strcmp(data.password, "pass") == 0) {
        data.isAuthenticated = 1;
    }
    if (data.isAuthenticated) {
        printf("Login Successful. Welcome!!!\n");
    } else {
        printf("Wrong Password! Login denied\n");
    }
}
int main() {
    char input[8];
    scanf("%s", input);
    check_login(input);
    return 0;
}
```

(i) Identify the security vulnerabilities present in the program.

(ii) Give a practical attack input idea and explain how it may exploit the program.

(iii) Suggest safer alternatives to prevent the problem.

SOLUTION

Background: what makes a program "insecure"

Sommerville (*Security engineering*) notes that most successful attacks exploit a small number of **programming errors**: unchecked input, unchecked buffer bounds, and trusting data that came from outside the program. C is especially exposed because its arrays carry no length information and library functions such as `strcpy` and `scanf("%s")` copy *until a terminating NUL is found*, not until the destination is full.

(i) Vulnerabilities present

(i) Unbounded read into a fixed buffer (main). `scanf("%s", input)` reads an arbitrarily long word into `char input[8]`. Any word of 8 or more characters (7 + terminating NUL is the maximum that fits) writes beyond the array — a **stack buffer overflow** in `main`.

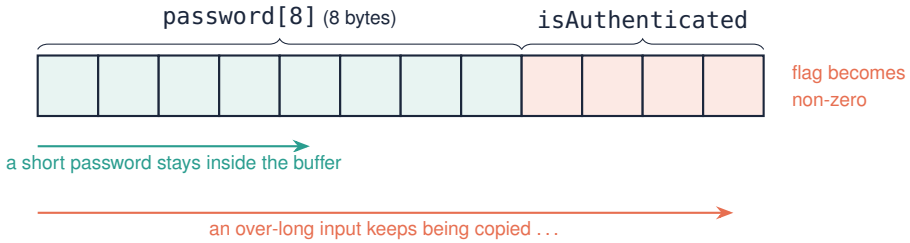
(ii) Unbounded copy (check_login). `strcpy(data.password, input)` copies the whole string

into an 8-byte field without checking its length — a second buffer overflow.

(iii) **Security decision stored next to attacker-controlled memory.** The flag `isAuthenticated` is declared *immediately after* `password[8]` in the same structure, and the flag is initialised to 0 *before* the unsafe copy. Bytes that spill out of `password` therefore land in the authentication flag, and the later test `if (data.isAuthenticated)` treats any non-zero value as "logged in".

(iv) **Hard-coded plaintext credential.** The password `"pass"` is embedded in the executable; anyone with the binary can read it (e.g. with a strings utility), and it cannot be changed without recompiling. It is also extremely weak.

(v) **Weak authentication design.** No hashing, no attempt limiting/lock-out, no logging of failed attempts, and the return value of `scanf` is not checked (empty input/EOF leaves `input` uninitialised).



Conceptual layout of `struct LoginData`: the copy does not stop at the end of `password`.

(ii) Attack idea (conceptual)

The attacker does not need to know the password at all. He simply types a "password" that is *longer than the 8-byte field* — for instance a dozen arbitrary letters.

- `strcpy` copies every character; the first eight fill `password`, the rest overwrite the bytes of `isAuthenticated` that follow it.
- `strcmp(data.password, "pass")` fails (the string is not `"pass"`), so the program never sets the flag to 1 — but it does not need to: the overwritten flag is already non-zero.
- `if (data.isAuthenticated)` is therefore true and the program prints *"Login Successful"*: an **authentication bypass**.

With longer inputs the overflow in `main` can also reach saved control data on the stack, which in general can crash the program (denial of service) or, in the worst case, let an attacker redirect control flow. That is why buffer overflows are treated as critical.

> **NOTE** As the bank itself flags, the printed program has *undefined behaviour*: writing past an array is not defined by the C standard, so the exact effect depends on the compiler, optimisation level, padding and the platform's protections (a stack-protector build may simply abort with a "stack smashing detected" message). The answer above is the standard textbook reading — the over-long input corrupts the adjacent flag — not a guaranteed outcome on every machine.

(iii) Safer alternatives

1. Bound every input and every copy.

- Read with `fgets(buf, sizeof buf, stdin)` (or at least a width: `scanf("%7s", input)`).

- Validate the length *before* use and reject over-long input instead of truncating silently.
- Copy with `snprintf(dst, sizeof dst, "%s", src)`, or `strncpy` followed by explicit termination `dst[sizeof dst - 1] = '\0'`. Never use `gets`.

2. Do not keep the security decision in memory reachable by the input. Return the result of the check as a value instead of a flag sitting next to a buffer.
3. Do not hard-code secrets. Store a salted, slow hash (bcrypt/Argon2/PBKDF2) of the password in protected storage, compare hashes in constant time, limit failed attempts and log them.
4. Use compiler/OS mitigations as defence in depth: stack canaries (`-fstack-protector-strong`), `-D_FORTIFY_SOURCE=2` (checked versions of `strcpy` etc.), ASLR, non-executable stack (NX/DEP), and static-analysis/code-review tools that flag unsafe functions (code review is exactly where the slides' "Buffer Overflow!" example is caught).
Corrected program

```c
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define PW_MAX 64                       /* longest password we accept      */

/* Real system: fetch a salted hash from protected storage and compare
   hashes in constant time.  A literal is kept here only to mirror the
   original exercise.                                                    */
static bool check_login(const char *input) {
    static const char expected[] = "pass";
    return strcmp(input, expected) == 0;  /* result returned, no flag in RAM */
}

int main(void) {
    char input[PW_MAX + 2];             /* room for '\n' and '\0'          */

    if (fgets(input, sizeof input, stdin) == NULL) {   /* bounded read     */
        fprintf(stderr, "No input\n");
        return 1;
    }
    input[strcspn(input, "\n")] = '\0';   /* strip the newline             */

    if (strlen(input) > PW_MAX) {         /* validate length, do not truncate */
        fprintf(stderr, "Password too long\n");
        return 1;
    }

    if (check_login(input))
        printf("Login Successful. Welcome!!!\n");
    else
        printf("Wrong Password! Login denied\n");
    return 0;
}
```

If the structure must be kept, the minimal patch is a bounded read plus a bounded, terminated copy:

```c
if (scanf("%7s", input) != 1) return 1;                /* width = size - 1 */
snprintf(data.password, sizeof data.password, "%s", input);  /* never overflows */
```

Answer. (i) Two stack buffer overflows (`scanf("%s")` into `input[8]` and `strcpy` into `password[8]`), an authentication flag adjacent to the overflowing buffer, and a hard-coded plaintext password. (ii) An input longer than 8 characters spills into `isAuthenticated`, making it non-zero ⇒ "Login Successful" without the password (behaviour formally undefined). (iii) Bounded input (`fgets`, `%7s`), length validation, `snprintf`/terminated `strncpy`, returning the decision instead of storing it, hashed credentials, plus canaries/ASLR/NX.

2023–2024 • Q1(b) CSE213 • 09/02/2025 • S004 **MARKS 4 + 10**

Read the following case study carefully and then answer the questions given below.

A mental health clinic is using an information system named, Mentcare system, which is designed and used to maintain diagnostic and treatment related records for mental patient. Clinic staff log on to the system using a username and password. The system requires passwords to be at least eight letters long but allows any password to be set without further checking. A receptionist may transfer data from the Mentcare system to a national patient record system (PRS) database that is maintained by the health authority of the country. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.

A criminal finds out that a well-paid sports star is receiving treatment for mental health problems. He would like to gain illegal access to information in this system so that he can blackmail the star. By posing as a concerned relative and talking with the nurses in the mental health clinic, he discovers how to access the system and personal information about the nurses and their families. By checking name badges, he discovers the names of some of the people allowed access. He then attempts to log on to the system by using these names and systematically guessing possible passwords, such as the names of the nurses' children.

(b) Do you believe that the mental health clinic maintains proper IT security policy? Justify your answer. Design an IT security policy for the use of password for the clinic.

SOLUTION

Background: what a security policy is

A **security policy** is a high-level statement, approved by management, of *what is and is not allowed* in an organisation's use of its information systems. Following Sommerville, a good policy states (1) the *assets* that must be protected, (2) the *level of protection* each needs, (3) the *responsibilities* of individual users, managers and the organisation, and (4) the *procedures and technologies* that must be used and maintained. A policy is "proper" only if it is complete, enforced by the system, known to staff and periodically reviewed.

(b-i) Does the clinic have a proper IT security policy? — No

The case shows that the clinic's policy is, at best, a single technical rule ("at least eight letters"), which is far from adequate:

- **No password quality checking.** "Any password" of eight letters is accepted, so names of children, pets and dictionary words are allowed — exactly what the criminal guessed.
- **No limit on failed log-ins.** The attacker could *systematically* try many passwords; there is no lock-out, delay, alert or monitoring of repeated failures.
- **Predictable user names.** User names can be inferred from the names on staff badges, so half of the credential is public.

- **No security awareness / social-engineering rules.** Nurses revealed how to access the system and personal family details to a stranger posing as a relative; the policy does not say what staff may disclose or how callers are verified.
- **No second factor, no auditing.** Highly sensitive mental-health records (and transfers to the national PRS by a receptionist) are protected by one guessable secret, with no logging or review of who accessed which record.

Because the stated assets (confidential patient records) are not matched by appropriate protection and staff responsibilities, the clinic **does not** maintain a proper IT security policy.

(b-ii) Password policy for the clinic

Section	Policy statement
1. Purpose & scope	Protect the confidentiality and integrity of patient records in Mentcare and data sent to the PRS. Applies to every doctor, nurse, receptionist, administrator and contractor who has a Mentcare account.
2. Account identity	One personal account per person; no shared or generic accounts. User IDs are system-generated (e.g. `mc4721`) and are *not* printed on badges or derived from the person's name.
3. Password creation	Minimum 12 characters (passphrases encouraged); must mix at least three of upper case, lower case, digits and symbols. The system rejects passwords that contain the user's name or user ID, dictionary words, common/breached passwords, and names of family members or pets. A strength meter is shown on creation.
4. Password use	Passwords are never shared, written down, sent by e-mail/phone, or entered on behalf of another person. Staff must lock or log off unattended terminals; sessions auto-lock after 5 minutes idle.
5. Change & reuse	Change immediately on any suspicion of disclosure and at least every 90 days for privileged accounts; the last 10 passwords may not be reused. Initial/reset passwords are one-time and must be changed at first log-in.
6. Failed log-ins	After 5 consecutive failures the account is locked for 30 minutes and the security officer is alerted; repeated lock-outs require identity verification in person.
7. Storage & transmission	The system stores only salted, slow hashes (e.g. bcrypt/Argon2) of passwords; all log-ins and PRS transfers use encrypted channels (TLS).
8. Additional factor	Access from outside the clinic, privileged accounts and PRS data transfer require a second factor (one-time code or smart card) in addition to the password.
9. Reset procedure	Passwords are reset only after verifying identity in person or by a registered second factor — never on a phone call request.
10. Awareness	All staff receive training on social engineering: never disclose system access details or personal information to callers/visitors; report suspicious enquiries.
11. Monitoring & audit	All log-ins, failures and record accesses are logged; logs are reviewed weekly and unusual access (e.g. to a celebrity's record) is flagged.
12. Responsibility & enforcement	The IT security officer maintains and reviews this policy annually; managers ensure compliance; violation leads to disciplinary action.

How the policy stops the attack in the case

Rule 3 rejects "children's names" passwords; rule 6 defeats systematic guessing; rule 2 hides user names; rule 10 prevents nurses from leaking information; rules 8 and 11 make even a correct guess insufficient and detectable.

Answer. No — an 8-letter minimum with no other checks, no lock-out, guessable user names and no staff awareness is not a proper policy. A proper password policy covers scope, unique non-guessable IDs, strong checked passwords, no sharing, change/reuse rules, lock-out, hashed storage, a second factor for sensitive actions, secure reset, training, auditing and enforcement.

2023–2024 • Q1(d), (e) CSE213 • 09/02/2025 • S005 MARKS 6 + 4 + 4

Context (same Mentcare case study as Q1(b) above). A mental health clinic uses the Mentcare system to maintain diagnostic and treatment records of mental patients. Staff log on with a username and password; the system requires passwords to be at least eight letters long but allows any password to be set without further checking. A receptionist may transfer updated personal information or a summary of diagnosis and treatment to the national patient record system (PRS) database. A criminal, wanting to blackmail a sports star who is a patient, poses as a concerned relative, learns from nurses how to access the system and personal details of the nurses' families, learns staff names from their badges, and then tries to log on using these names and systematically guessing passwords such as the names of the nurses' children.

(d) Find out assets, exposures and vulnerabilities for the Mentcare system stated above and show them in a table. **(6)**
(e) Write down the weaknesses of password-based authentication. Suggest appropriate 2FA and 3FA processes for the above case study for enhancing the access security. **(4+4)**

SOLUTION

Definitions (Sommerville's security terminology)

- **Asset** — something of value that has to be protected (data, system, reputation).
- **Exposure** — the possible loss or harm that could result from a successful attack (loss of data, money, time, reputation, legal penalties).
- **Vulnerability** — a weakness in the computer-based system (or its use) that may be exploited to cause loss or harm.

Related terms: the *attack* here is impersonation of an authorised user; the *threat* is an unauthorised person gaining access by guessing credentials; a *control* would be a password checker that rejects names and dictionary words.

(d) Assets, exposures and vulnerabilities

Asset	Exposure (possible loss/harm)	Vulnerability exploited
Patient records (diagnosis, treatment) — e.g. the sports star's file	Disclosure used for blackmail; patient distress; legal action and compensation claims against the clinic; fines for breaching medical-privacy law	Weak password rule: any 8-letter password (names of children) accepted
Patients' personal information (address, phone)	Identity theft, stalking or harassment of patients and families	No lock-out after repeated failed log-ins, so systematic guessing is possible
Staff accounts / credentials	Attacker acts with a nurse's privileges; actions are wrongly attributed to the nurse	User names guessable from name badges; staff personal details known to outsiders
Data transferred to the national PRS	Corrupted or leaked national records; loss of trust of the health authority	Receptionist can transfer data with no second check, encryption or audit stated
Clinic reputation and Mentcare service	Future patients avoid treatment because they no longer trust the clinic; financial loss; time and effort to recover	Staff susceptible to social engineering (no awareness training or caller verification)

(e-i) Weaknesses of password-based authentication

(i) Guessable passwords. Users choose memorable words — names, birthdays, children's names — which can be guessed with personal knowledge (as in this case).

(ii) Brute-force / dictionary attacks. Automated tools try millions of combinations if the system does not limit attempts.

(iii) Social engineering and phishing. Users can be tricked into revealing passwords or reset details; the password is "something you know", so knowing is enough.

(iv) Reuse and sharing. The same password is reused across sites (a breach elsewhere exposes Mentcare) or shared among colleagues.

(v) Capture and theft. Shoulder surfing, notes stuck on monitors, key-loggers, and theft of a poorly hashed password file.

(vi) No proof of presence. A correct password proves only knowledge of a secret, not the identity of the person typing it.

(e-ii) Suggested 2FA and 3FA processes

Multi-factor authentication combines *independent* factor types: **knowledge** (something you know), **possession** (something you have) and **inherence** (something you are).
2FA (password + possession) — for normal clinic log-in:

(i) The nurse enters the (system-generated) user ID and a strong password.

(ii) On success, Mentcare sends a one-time code (valid for 60 s) to an authenticator app / SMS on the nurse's registered phone, *or* the nurse taps a staff smart card on the terminal reader.

(iii) Access is granted only if the code/card is correct; failures are counted and logged.

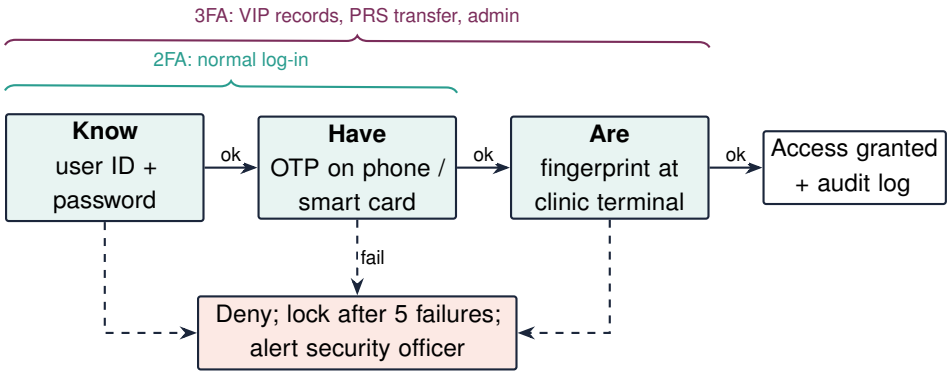
Even if the criminal guesses "Rafi2015", he does not hold the nurse's phone or card.

3FA (password + possession + biometric) — for highly sensitive actions such as opening a VIP/flagged patient record, privileged administration, or transferring data to the PRS:

(i) Steps 1–2 of 2FA.

(ii) The user additionally scans a fingerprint (or face) on the clinic workstation reader; the template is matched against the enrolled staff biometric.

(iii) The action is executed and an audit record (who, what, when, where) is written.



Answer. Assets: patient records, personal data, staff accounts, PRS data, clinic reputation. Exposures: blackmail/privacy loss, legal and financial loss, loss of trust. Vulnerabilities: weak unchecked passwords, no lock-out, guessable user names, social engineering, unchecked PRS transfer. Passwords are guessable, brute-forceable, phishable and shareable; use 2FA = password + OTP/smart card, and 3FA = password + OTP/card + fingerprint for the most sensitive operations.

2023–2024 • Q4(a)–(d) CSE213 • 09/02/2025 • S006 **MARKS 7 + 8 + 7 + 8**

(a) With appropriate diagram and examples show how security threats are arisen on instructions and data based on CIA Triad. (7)

(b) Explain encryption policy with appropriate diagram. Differentiate between symmetric encryption and asymmetric encryption. (8)

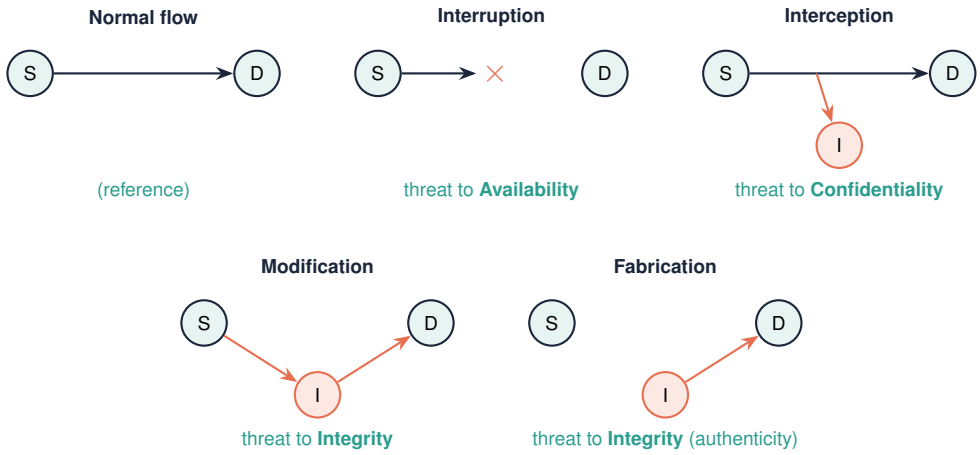
(c) An asset has an estimated value of $200,000 with an exposure factor of 20%. It is also estimated that frequency of attack on such an asset is once in every five-year time period. The actual cost of safeguards for the asset is $10,000 which reduces the exposure factor to 5%. Find out the residual risk that remains after taking the safeguard measures. Justify whether taking such a safeguard measure is worthy or not. (7)

(d) Explain buffer overflow attacks with appropriate example. (8)

SOLUTION

(a) Threats to instructions and data, classified by the CIA triad

The **CIA triad** lists the three properties security must protect: **Confidentiality** (only authorised parties can read), **Integrity** (only authorised parties can change, and changes are detectable) and **Availability** (authorised users get access when needed). A computing system holds two kinds of assets that can be attacked: *instructions* (programs, executable code) and *data*. Threats arise when information flowing from a source to a destination is disturbed in one of four ways (Pfleeger's classification):



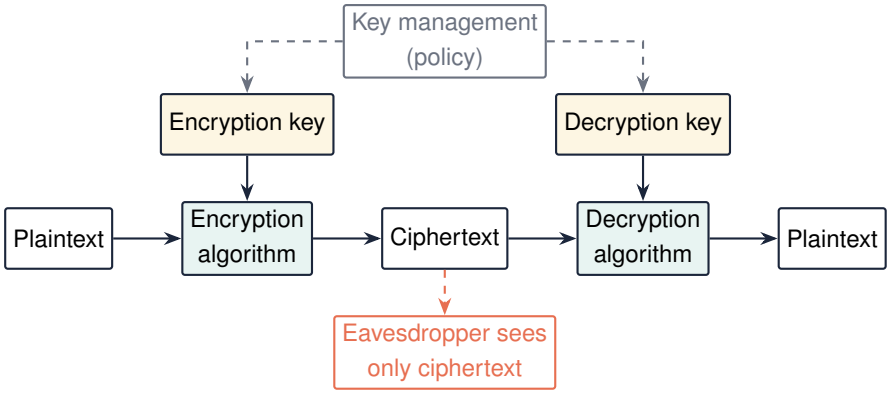
S = source, D = destination, I = intruder.

Property / threat	On instructions (programs)	On data
Confidentiality (interception)	copying/stealing proprietary code; reverse-engineering a binary to read an embedded password	eavesdropping on network traffic; an outsider reading patient or exam records
Integrity (modification, fabrication)	virus/trojan altering an executable; code injection (buffer overflow, SQL/script injection) making the program run attacker logic	tampering with grades or bank balances; inserting forged transactions or fake log entries
Availability (interruption)	deleting or corrupting programs; crashing a service via an overflow; DoS flooding	ransomware encrypting files; deleting a database or its backups

(b) Encryption policy; symmetric vs asymmetric encryption

Encryption transforms readable *plaintext* into unreadable *ciphertext* using an algorithm and a *key*; only a holder of the right key can reverse it. An **encryption policy** is the organisational rule-set that states *what* must be encrypted, *when*, *how*, and *who* manages the keys:

- **Scope / classification:** which data classes must be encrypted (e.g. personal, medical, financial data, credentials).
- **Data in transit** (TLS for all external and internal service traffic, VPN for remote access) and **data at rest** (disk, database, backups, laptops, removable media).
- **Approved algorithms and key lengths** (e.g. AES-256, RSA $\geq$ 3072 bits or elliptic curves; deprecated ones such as DES/MD5 forbidden).
- **Key management life-cycle:** generation, distribution, secure storage (HSM/key vault), rotation, revocation and destruction; keys never stored next to the data they protect.
- **Roles, responsibilities, exceptions and audit** of compliance.



Symmetric: both keys are the same secret key. Asymmetric: encrypt with the receiver's public key, decrypt with the receiver's private key.

Aspect	Symmetric (secret-key)	Asymmetric (public-key)
Keys	one shared secret key for both directions	key pair: public key (shared) + private key (kept secret)
Speed	fast; suits bulk data	100–1000× slower; suits small data, keys, signatures
Key distribution	hard: the secret must be exchanged securely beforehand	easy: public key can be published (via certificates/PKI)
Number of keys (n users)	$n(n-1)/2$	$2n$
Services	confidentiality (and MACs)	confidentiality, digital signatures, non-repudiation, key exchange
Examples	AES, ChaCha20, (DES, 3DES)	RSA, ECC/ECDSA, Diffie–Hellman

In practice they are combined (*hybrid* encryption, e.g. TLS): asymmetric cryptography agrees on a random session key, then symmetric AES encrypts the actual traffic.

(c) Risk calculation

Quantitative risk analysis uses:

$$\text{SLE} = \text{Asset value (AV)} \times \text{Exposure factor (EF)}, \qquad \text{ALE} = \text{SLE} \times \text{ARO},$$

where SLE = single loss expectancy, ARO = annualised rate of occurrence, ALE = annualised loss expectancy. One attack every five years gives ARO $= 1/5 = 0.2$ per year.

	Without safeguard	With safeguard
EF	20%	5%
SLE = AV × EF	200,000 × 0.20 = \$40,000	200,000 × 0.05 = \$10,000
ARO	0.2 / year	0.2 / year
ALE = SLE × ARO	40,000 × 0.2 = \$8,000 / year	10,000 × 0.2 = \$2,000 / year

Residual risk (risk remaining after the safeguard) $= \text{ALE}_{\text{after}} =$ **\$2,000 per year** (equivalently an expected loss of \$10,000 per incident). The safeguard reduces the expected annual loss by $8{,}000 - 2{,}000 =$ \$6,000 per year.

Is the safeguard worthwhile? The standard cost–benefit test is

$$\text{Value of safeguard} = \text{ALE}_{\text{before}} - \text{ALE}_{\text{after}} - \text{annual cost of safeguard.}$$

> **NOTE** The question does not say whether the \$10,000 is a *yearly* cost or a *one-off* cost, so both readings are given.
>
> - **Yearly cost (\$10,000/year)** — the usual reading of the formula: $8{,}000 - 2{,}000 - 10{,}000 = -\$4{,}000$ per year. The safeguard costs more than the loss it prevents, so it is **not worthwhile** on purely financial grounds.
> - **One-off cost (\$10,000 once):** it saves \$6,000/year, so it pays for itself in $10{,}000/6{,}000 \approx 1.7$ years; over the five-year period it saves $5 \times 6{,}000 = \$30{,}000$ for \$10,000, a net benefit of \$20,000 — **worthwhile**.

Non-financial factors (legal duty, reputation, safety) may justify a safeguard even when the numbers are marginal, but the question asks for the quantitative justification.

(d) Buffer overflow attacks

A **buffer overflow** happens when a program writes more bytes into a fixed-size buffer than it can hold, so the extra bytes overwrite neighbouring memory. In C/C++ arrays are not bounds-checked and functions such as `gets`, `strcpy`, `strcat`, `sprintf` and `scanf("%s")` copy until a terminating NUL, regardless of the destination size. On the stack, a local buffer sits next to other local variables and to the function's saved control data (frame pointer, return address) — see the stack-frame figure in S012.

Example.

```c
int check_pin(void) {
    int granted = 0;
    char pin[8];
    gets(pin);                          /* VULNERABLE: no limit on input length */
    if (strcmp(pin, "4711") == 0) granted = 1;
    return granted;
}
```

- A short input behaves normally.
- An input longer than the buffer overwrites whatever is adjacent — possibly `granted` (access without the PIN), and with still longer input the saved return address.
- Effects range from a **crash** (availability), through **corrupted data** (integrity), to **control-flow hijack**: the attacker makes the function "return" into code of his choosing and runs it with the program's privileges. Historic examples are the Morris worm (1988, `fingerd`) and Code Red (2001, IIS).

Prevention.

- Bounded functions: `fgets(pin, sizeof pin, stdin)`, `snprintf`, `strncpy` + explicit NUL; never `gets` (removed from C11).
- Validate the length of every external input; prefer memory-safe languages where possible.
- Compiler/OS defences: stack canaries, `_FORTIFY_SOURCE`, ASLR, non-executable stack (NX/DEP).
- Static analysis, code review and fuzz testing; least privilege for the running process.

```c
if (fgets(pin, sizeof pin, stdin) == NULL) return 0;   /* SAFE: bounded read */
pin[strcspn(pin, "\n")] = '\0';
return strcmp(pin, "4711") == 0;                       /* no flag to corrupt */
```

Answer. (a) Interruption → availability, interception → confidentiality, modification/fabrication → integrity, on both programs and data. (b) Policy = what/when/how to encrypt + key management; symmetric = one shared key, fast, key-distribution problem; asymmetric = public/private pair, slower, enables signatures. (c) $\text{ALE}_{\text{before}} = \$8{,}000/\text{yr}$, residual risk $\text{ALE}_{\text{after}} = \$2{,}000/\text{yr}$; if \$10,000 is yearly, net $-\$4{,}000/\text{yr} \Rightarrow$ not worthy; if one-off, payback ≈ 1.7 years $\Rightarrow$ worthy. (d) Unchecked copy into a fixed buffer overwrites adjacent data/return address; fix with bounded APIs, validation, canaries/ASLR/NX.

2022–2023 • Q8(a),(b) CSE307 • 05/05/2024 • S007 **MARKS 10 + 10**

(a) Briefly describe how the following works: (3+3+4=10)

(i) SQL poisoning attack

(ii) Cross-site scripting attack

(iii) Federated identity

(b) How do symmetric and asymmetric encryption techniques work? How can asymmetric encryption be used for authentication? (10)

SOLUTION

(a)(i) SQL poisoning (SQL injection) attack

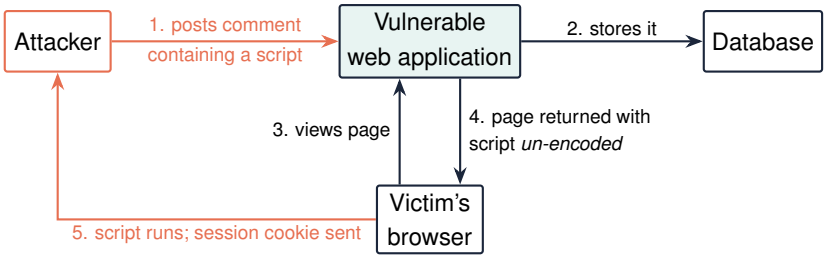
"SQL poisoning" is Sommerville's name for **SQL injection**. A web application takes input from a form field and *concatenates* it into an SQL command. Because the input is pasted into the command text, a crafted input containing SQL syntax (quotes, `OR`, comment markers) changes the structure of the query.

- **How it works:** the query `"... WHERE name='" + name + "' AND password='" + pwd + "'"` receives, as password, the textbook string `' OR '1'='1`; the condition becomes always true and the attacker logs in without a password. Other crafted inputs can read whole tables or alter data.
- **Defence:** **parameterized queries/prepared statements** (values bound as data, never as SQL), allow-list input validation, least-privilege database accounts, no raw DB error messages.

```c
PreparedStatement ps = conn.prepareStatement(
        "SELECT id, pwd_hash FROM users WHERE name = ?");
ps.setString(1, name);            // input can never alter the SQL structure
```

(a)(ii) Cross-site scripting (XSS) attack

XSS occurs when a web application includes *untrusted input* in a page it sends to other users *without encoding it*. The victim's browser cannot tell the injected script from the site's own scripts, so it runs it with the site's privileges (same origin): it can read the session cookie, perform actions as the victim, or alter the page (fake login forms).



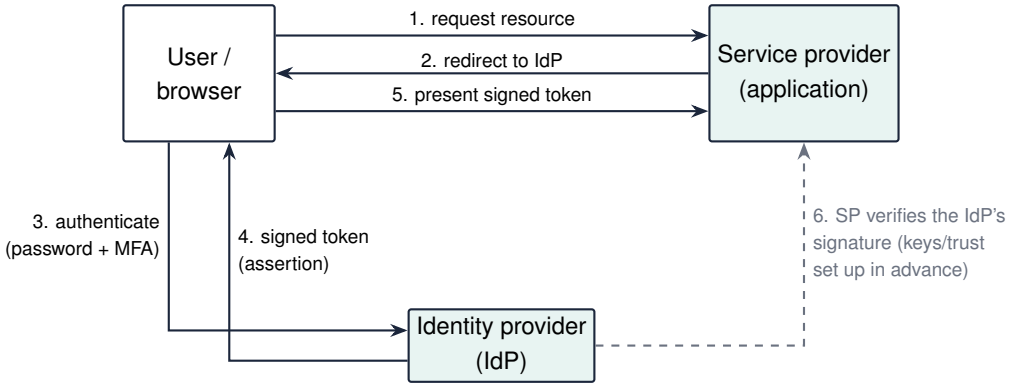
Stored XSS. In reflected *XSS the script travels in a crafted link and is echoed back immediately; in* DOM-based *XSS client-side JavaScript inserts it.*

Defence: context-aware **output encoding** (HTML-escape <, >, ", ', &), auto-escaping template engines, allow-list input validation, a **Content Security Policy** header that forbids inline/unknown scripts, and `HttpOnly`/`SameSite` cookies so scripts cannot read the session.

```
// OWASP Java Encoder: the comment is shown as text, never executed
out.println("<p>" + Encode.forHtml(comment) + "</p>");
response.setHeader("Content-Security-Policy", "script-src 'self'");
```

(a)(iii) Federated identity

In **federated identity** a user's identity is managed and verified by a trusted external **identity provider** (IdP) — e.g. a university SSO, Google, or a government ID service — instead of every application keeping its own user names and passwords. Applications (*service providers*, SPs) establish a trust relationship with the IdP in advance (exchanging certificates) and accept its signed assertions. Standards: SAML 2.0, OpenID Connect / OAuth 2.0.



The browser carries the token (steps 4–5) to the SP, which checks the IdP's signature (step 6) and opens a session; the SP never sees the password.

Benefits: single sign-on (one strong credential instead of many weak ones), applications never see the password, central enforcement of password/MFA policy and central de-provisioning. **Risks:** the IdP becomes a high-value target and a single point of failure; trust configuration and token validation must be done correctly.

(b) How symmetric and asymmetric encryption work

Symmetric (secret-key) encryption. Sender and receiver share one secret key K: $C = E_K(P)$ and $P = D_K(C)$. Algorithms such as AES apply many rounds of substitution and permutation keyed by K; they are fast and suit bulk data. The weakness is *key distribution*: K must reach the partner securely, and n users need $n(n-1)/2$ keys.

Asymmetric (public-key) encryption. Each party owns a mathematically linked **key pair**: a public key KU (published, e.g. in a certificate) and a private key KR (never disclosed). What one key encrypts only the other can decrypt, and KR cannot feasibly be derived from KU (RSA relies on the difficulty of factoring large numbers, ECC on the discrete-logarithm problem on elliptic curves).

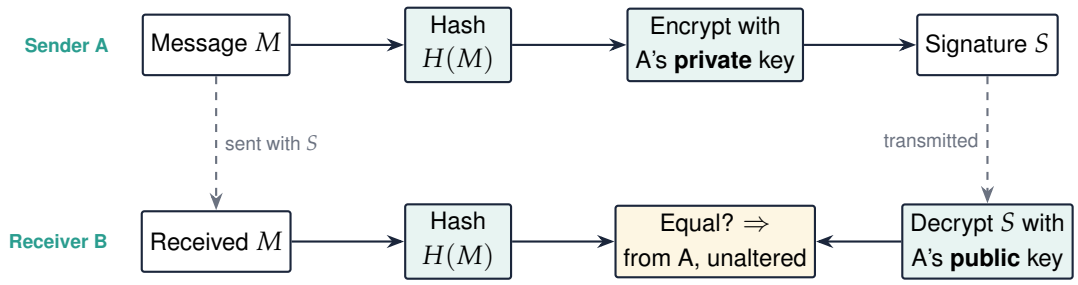
- **Confidentiality:** encrypt with the *receiver's public* key, $C = E_{KU_B}(P)$; only B's private key can decrypt.
- **Authentication:** encrypt (sign) with the *sender's private* key; anyone can verify with the sender's public key.

Because it is slow, real systems use *hybrid* schemes (TLS): public-key methods establish a random symmetric session key, and AES protects the data.

(b) Using asymmetric encryption for authentication

Only the owner holds the private key, so being able to produce something that the owner's public key verifies proves the owner's identity.

1. Digital signature (message/data-origin authentication).



The sender hashes the message and encrypts the hash with his private key; the receiver decrypts the signature with the sender's public key and compares it with a fresh hash of the message. A match proves *who* sent it (authentication), that it was *not changed* (integrity) and that the sender cannot later deny it (*non-repudiation*).

2. Challenge–response (entity authentication, e.g. SSH keys, smart cards, TLS client certificates).

(i) The server sends the client a fresh random number (nonce) N.

(ii) The client returns $E_{KR_A}(N)$, i.e. N signed with its private key.

(iii) The server verifies it with the client's public key KU_A; if it yields N, the client must possess KR_A and is authenticated. The fresh nonce prevents replay of an old response.

In both cases the receiver must be sure the public key really belongs to A; this binding is provided by a **digital certificate** issued by a trusted Certificate Authority (PKI).

Answer. (a) SQL poisoning: input concatenated into SQL changes the query ⇒ use prepared statements. XSS: un-encoded user input is served as script and runs in victims' browsers ⇒ output encoding + CSP + HttpOnly cookies. Federated identity: a trusted IdP authenticates the user and issues a signed token that service providers accept (SSO). (b) Symmetric: one shared key, fast, key-distribution problem; asymmetric: public/private pair, slower, solves key distribution. Authentication: sign with the private key (digital signature or signed nonce), verify with the certificate-bound public key.

2020–2021 • Q5(c) CSE307 • 19/10/2022 • S008 **MARKS 7**

Context — scenario of Q5(a): A media agency is responsible to order advertisements at different newspapers. Each newspaper has a standard rate for unit size (1 column-inch) for each page which may change over time. Different newspaper gives different discounts over this rate to the agency. The agency has a database of expected readership for each page of each newspaper. An employee of the agency prepares a media plan for a product by selecting different positions for the ad (page, size, date and newspaper) to achieve maximum readership for a given budget. This plan is approved by client and then another employee generates Release Order from this plan. The Release Order mentions a number of ads to display with their positions, date to publish and cost. *(Q5(a) asked for a class diagram of this scenario.)*

(c) What do you understand by Non-repudiation? Give an example considering the scenario of Q 5(a).

SOLUTION

Non-repudiation

Non-repudiation is the security property that provides undeniable proof of an action, so that a party who performed it (sent a message, approved a document, received an order) *cannot later deny* having done so. It gives two kinds of evidence:

- **proof of origin** — the sender/approver cannot deny sending or approving;
- **proof of delivery/receipt** — the receiver cannot deny receiving.

It is achieved with **digital signatures** (only the holder of the private key could have produced the signature), trusted **time-stamps**, signed acknowledgements and tamper-evident **audit logs**. Passwords or shared secrets alone are not enough, because the other party also knows the secret.

Example in the media-agency scenario

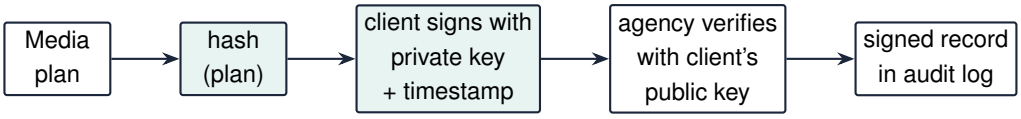
Client approval of the media plan. The client approves a plan worth, say, Tk 20 lakh; the ads are published; later the client claims, "We never approved those positions/dates" and refuses to pay. With non-repudiation the agency system:

(i) computes a hash of the exact media plan (pages, sizes, dates, newspapers, cost);

(ii) asks the client to sign the hash with the client's *private key* (digital signature), attaching a trusted time-stamp;

(iii) verifies the signature with the client's public key and stores plan + signature + time-stamp in an append-only audit log.

Later, anyone can verify that this exact plan was approved by this client at that time, so the client cannot repudiate the approval.



Similar cases in the same scenario: the *newspaper* cannot deny receiving the Release Order (signed acknowledgement of receipt), and the *employee* who generated a Release Order cannot deny having issued it (signed, logged action).

Answer. Non-repudiation = verifiable proof that a party performed an action, so it cannot deny it later. Example: the client digitally signs (with a time-stamp) its approval of the media plan; if it later denies approving and refuses payment, the stored signature proves the approval.

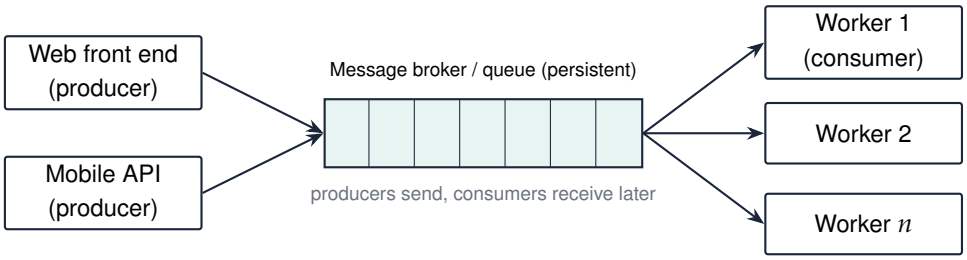
2020–2021 • Q8(a) CSE307 • 19/10/2022 • S009 **MARKS 15**

What are the benefits of using Message Queue? What are the good practices recommended to ensure Confidentiality in software application?

SOLUTION

Message queue

A **message queue** is middleware that lets software components communicate *asynchronously*: a **producer** puts a message on a queue managed by a broker (e.g. RabbitMQ, Kafka, Amazon SQS) and returns immediately; one or more **consumers** take messages off the queue and process them when they are ready.



Benefits of using a message queue

(i) Decoupling. Producers and consumers do not need to know each other's location, technology or availability; each can be changed, deployed and scaled independently.

(ii) Asynchronous processing and responsiveness. The user request returns quickly ("order received") while slow work (e-mail, payment, report generation) is done in the background.

(iii) Load levelling / buffering of peaks. Bursts of requests are stored in the queue and consumed at the rate the back end can sustain, preventing overload and crashes.

(iv) Scalability. More consumers can be added (competing-consumer pattern) to increase throughput horizontally.

(v) Reliability and fault tolerance. Persistent messages survive crashes; if a consumer fails, the message is redelivered (acknowledgements, retries, dead-letter queues); a temporarily unavailable service does not lose requests.

(vi) Guaranteed / ordered delivery. At-least-once or exactly-once semantics and FIFO ordering where needed.

(vii) Integration of heterogeneous systems and support for publish/subscribe (one event, many interested subscribers).

Confidentiality

Confidentiality means information is disclosed only to authorised persons or systems. Good practices recommended to ensure it:

(i) **Classify data** (public, internal, confidential, secret) and apply protection according to sensitivity; collect and keep only what is needed (data minimisation).

(ii) **Strong authentication** — strong password rules, multi-factor authentication, secure session management (random session IDs, timeouts, secure/HttpOnly cookies).

(iii) **Access control with least privilege** — role-based authorisation checked on the server for every request; separation of duties; deny by default.

(iv) **Encrypt data in transit** — TLS/HTTPS for all traffic, including internal service and message-queue connections.

(v) **Encrypt data at rest** — databases, backups, files and queued messages (AES); hash passwords with salted slow hashes (bcrypt/Argon2), never store them in plain text.

(vi) **Key and secret management** — no hard-coded passwords/API keys in source code; use a vault, rotate keys, restrict who can read them.

(vii) **Avoid information leakage** — generic error messages (no stack traces), no sensitive data in logs, URLs or caches; mask/tokenise data such as card numbers on screen.

(viii) **Secure development and testing** — threat modelling, code reviews, static analysis, penetration testing for injection/XSS flaws that leak data; keep libraries patched.

(ix) **Monitoring and auditing** — log access to confidential data and review it; detect unusual access.

(x) **People and policy** — security awareness training against social engineering, NDAs, secure disposal of media and data at end of retention.

Answer. Message queues give decoupling, asynchronous processing, load levelling, scalability, reliable (persistent, retried) and ordered delivery and easy integration. Confidentiality practices: classify/minimise data, strong (multi-factor) authentication, least-privilege access control, encryption in transit and at rest, proper key/secret management, no leakage via errors or logs, secure coding and testing, auditing, and staff awareness.

Jan 2021 • Q7(b) CSE307 • 04/09/2021 • S010 MARKS 15

What do you understand by Non-repudiation. Give 2 examples of Non-repudiation in the context of software application. What are the practices you should follow to ensure Integrity while developing a web application?

SOLUTION

Non-repudiation

Non-repudiation assures that the originator of an action or message cannot later deny having performed it, and the recipient cannot deny having received it. It needs evidence that is *bound to one party only* and *tamper-evident*: digital signatures with the party's private key (backed by a PKI certificate), trusted time-stamps, signed receipts and secure audit trails. It differs from authentication: authentication proves identity *now* to the system; non-repudiation proves an action *later* to a third party (e.g. a court).

Two examples in software applications

(i) **Online banking fund transfer.** The customer authorises a transfer by signing the transaction

details (amount, account, time) with a key held on a device or via a transaction-signing OTP. The bank stores the signed transaction log. If the customer later says "I never sent this money", the bank presents the signature that only the customer's device could have produced.

(ii) Digitally signed e-mail / e-document (e.g. e-contract, e-tender). A supplier submits a tender or signs a contract through an e-signature service (S/MIME, PDF signature). The signature and timestamp prove that the supplier sent *exactly* this document at that time; the system also returns a signed receipt, so the buyer cannot deny receiving it.

Other examples: e-commerce order placement with a logged, signed order; code signing of software updates; git commits signed with GPG keys.

Integrity

Integrity means data and system resources are protected from *unauthorised or accidental modification*: data are accurate, complete and changed only by authorised users in authorised ways, and any tampering is detectable.

Practices to ensure integrity in a web application

Practice	What to do / why
Server-side input validation	Validate type, length, range and format of every input on the server (never trust client-side checks); reject or sanitise invalid data.
Prevent injection	Use parameterised queries / ORM to stop SQL injection; encode output to stop XSS that could alter pages or data.
Access control on every change	Enforce authentication and authorisation (least privilege, RBAC) on each update/delete; check object ownership to avoid IDOR; use CSRF tokens on state-changing requests.
Protect data in transit	HTTPS/TLS everywhere (HSTS) so data cannot be modified in transit; sign tokens (e.g. JWT with HMAC/RSA) and verify signatures.
Hashes, MACs, signatures	Store checksums/HMACs for critical records and files; verify downloads and uploads; use digital signatures for important documents.
Database integrity	Constraints (keys, foreign keys, checks), ACID transactions, optimistic locking for concurrent updates.
Audit trail & versioning	Log who changed what and when in an append-only log; keep history so unauthorised changes can be detected and rolled back.
Secure supply chain	Verify third-party libraries (checksums, signed packages, Subresource Integrity for CDN scripts); keep dependencies patched.
Development process	Code reviews, static analysis, security testing (penetration tests), controlled deployment (CI/CD with signed builds).
Backup and recovery	Regular, tested, protected backups to restore correct data after corruption.

Answer. Non-repudiation = a party cannot deny an action it performed (proof of origin/receipt via digital signatures, timestamps, audit logs). Examples: a signed online bank transfer; a digitally signed e-mail/e-contract with signed receipt. Integrity practices: server-side validation, parameterised queries/output encoding, authorisation and CSRF protection on every change, TLS, hashes/MACs/signatures, DB constraints and transactions, audit trails, verified dependencies, code review/testing, backups.

2017–2018 • Q4(d) CSE307 • 29/09/2018 • S011 MARKS 5

"Increasing reliability does not necessarily mean increasing safety" — Justify mentioning two different scenarios of using software.

SOLUTION

Definitions

- **Reliability** — the probability of failure-free operation *according to the specification* over a given time in a given environment.
- **Safety** — the ability of the system to operate without causing *harm* (injury, death, damage to the environment), whether or not it meets its specification.

Reliability is about *conformance to specification*; safety is about *avoiding hazards*. They are related but not the same, because (Sommerville): (1) the specification may be incomplete or wrong and not describe behaviour in critical situations; (2) hardware malfunctions may put the software in an unanticipated environment; (3) inputs that are individually correct may combine to cause a hazard; and (4) operators may make errors. Moreover, a system that keeps working "at all costs" can be less safe than one that stops.

Scenario 1 — reliable but unsafe: insulin pump

The pump's software calculates and delivers insulin exactly as the specification says, with no failures in millions of doses — highly reliable. But the specification omitted a maximum-dose limit for an abnormal sensor reading (e.g. a faulty glucose sensor reporting very high sugar). The software "correctly" delivers a massive dose and the patient suffers a hypoglycaemic coma. Making the software even more reliable does nothing for safety; safety needs a hazard analysis and a safety requirement ("never exceed the maximum single dose").

Scenario 2 — safer but less reliable: railway signalling / train protection

A train-protection system applies the emergency brake and sets signals to red whenever it is uncertain (lost communication, conflicting sensor data). This *fail-safe* behaviour causes frequent unnecessary stops — the service is less reliable and available — yet the system is *safer*. Conversely, tuning the software to never stop the train (higher reliability of the service) could let it run past a real hazard.

Answer. Reliability measures conformance to the specification; safety measures freedom from harm. A reliable insulin pump can still kill a patient if its specification is wrong, and a fail-safe railway signalling system is safer precisely by being less reliable/available — so increasing reliability does not necessarily increase safety.

2016–2017 • Q3(a) CSE307 • 15/07/2017 • S012 MARKS 15

Write short notes on the following software vulnerabilities. Also provide example and way around on these vulnerabilities.

(i) Buffer Overflow Attack

(ii) SQL Injection

SOLUTION

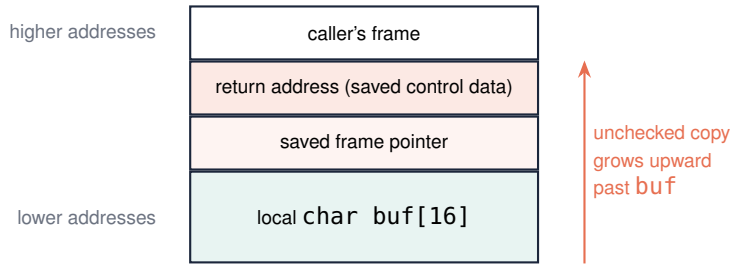
Both are **input-validation vulnerabilities**: the program lets data supplied by an outsider be treated

as something more powerful than data — as memory it may overwrite, or as code it may execute. They are perennial entries in the OWASP Top 10 / CWE Top 25 lists.

(i) Buffer overflow attack

What it is. A buffer overflow occurs when a program writes more data into a fixed-size memory area (buffer) than it can hold, so the excess overwrites adjacent memory. It is typical of C/C++, where arrays are not bounds-checked and functions such as `gets`, `strcpy`, `strcat`, `sprintf` and `scanf("%s")` copy until a NUL terminator rather than until the buffer is full.
Why it is dangerous. Local buffers live on the stack next to other variables and to saved control data (the return address of the function). Overwriting them can:

- change the value of neighbouring variables (e.g. an "is authenticated" flag — see S002);
- crash the program (denial of service);
- in the worst case let an attacker alter where the function returns, i.e. hijack control flow and run code with the program's privileges.



Simplified stack frame: writing past `buf` reaches saved control data.

Example.

```c
void greet(const char *name) {
    char buf[16];
    strcpy(buf, name);          /* VULNERABLE: no length check            */
    printf("Hello %s\n", buf);
}
```

A name longer than 15 characters overruns `buf`.
Way around (prevention).

- **Bounds-checked APIs:** `fgets` instead of `gets`; `snprintf` instead of `sprintf`/`strcpy`; `strncpy` only with explicit NUL termination.
- **Validate length** of all external input and reject what does not fit.
- **Memory-safe languages** (Java, C#, Python, Rust) perform bounds checks automatically.
- **Compiler/OS mitigations:** stack canaries, `_FORTIFY_SOURCE`, ASLR, NX/DEP stack, control-flow integrity.
- **Process:** code review, static analysis and fuzz testing; run with least privilege so a successful overflow gains little.

```c
void greet(const char *name) {
    char buf[16];
    snprintf(buf, sizeof buf, "%s", name);   /* SAFE: never exceeds buf */
    printf("Hello %s\n", buf);
}
```

(ii) SQL injection

What it is. SQL injection occurs when an application builds an SQL statement by *concatenating* user input into the query text. Characters such as a quote in the input end the intended string literal, and the rest of the input is parsed by the database as SQL. The attacker thereby changes the *meaning* of the query: bypassing authentication, reading other users' data, or modifying and deleting records.

Example (vulnerable Java).

```
String sql = "SELECT id FROM users WHERE name = '" + name +
             "' AND password = '" + pwd + "'";
ResultSet rs = conn.createStatement().executeQuery(sql);   // VULNERABLE
boolean ok = rs.next();
```

If the password typed is the classic textbook string `' OR '1'='1`, the `WHERE` clause becomes "`... AND password = '' OR '1'='1'`", which is always true, so the login succeeds without a valid password.

Way around (prevention).

- **Parameterized queries / prepared statements** (primary fix): the SQL text is fixed and compiled first; user values are sent separately as *data* and can never change the query structure.
- **Stored procedures** that themselves use parameters (not dynamic SQL).
- **Input validation by allow-list** (e.g. user name matches `[A-Za-z0-9_]{3,20}`) as an extra layer — never as the only defence.
- **Least privilege:** the application's DB account gets only the rights it needs (no DROP, no access to unrelated tables).
- Do not show raw database errors to users; store *hashed* passwords, never plaintext.

```
String sql = "SELECT id, pwd_hash FROM users WHERE name = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, name);                    // bound as data, not SQL
    try (ResultSet rs = ps.executeQuery()) {
        return rs.next() && PasswordHasher.verify(pwd, rs.getString("pwd_hash"));
    }
}
```

	Buffer overflow	**SQL injection**
Root cause	unchecked length of input copied into fixed memory	input concatenated into a query string
Layer	native code (C/C++), memory	application ↔ database
Impact	crash, data corruption, control-flow hijack	auth bypass, data theft/modification
Main fix	bounds-checked APIs + length validation; canaries, ASLR, NX	prepared statements; allow-list validation; least privilege

Answer. Buffer overflow: writing beyond a fixed buffer corrupts adjacent memory (e.g. `strcpy` into `char buf[16]`); prevent with bounded APIs, length checks, safe languages and canaries/ASLR/NX. SQL injection: concatenated input changes the query (e.g. `' OR '1'='1`); prevent with `PreparedStatement` parameters, allow-list validation and a least-privilege DB account.

2012–2013 • Q2(e) CSE307 • 17/05/2014 • S013 **MARKS 4**

A Web Application and its support environment have not been fully fortified against attack. Web engineers estimate that the likelihood of repelling attack is only 30 percent. The system does not contain sensitive or controversial information, so that threat probability is 25 percent. What is the integrity of the application?

SOLUTION

Integrity as a quality metric (Pressman)

Pressman measures **integrity** — a system's ability to withstand attacks on its security — using two attributes:

- **threat**: probability that an attack of a specific type will occur in a given time;
- **security**: probability that an attack of that type will be *repelled*.

$$\text{Integrity} = \sum_{\text{attack types}} \left[1 - \text{threat} \times (1 - \text{security})\right]$$

(with a single attack type the sum has one term). threat $\times$ $(1 - \text{security})$ is the probability that an attack occurs *and* succeeds.

Substitution

Given: threat $= 0.25$, security $= 0.30$.

$$\text{Integrity} = 1 - 0.25 \times (1 - 0.30) = 1 - 0.25 \times 0.70 = 1 - 0.175 = \mathbf{0.825}$$

Interpretation

There is a 17.5% chance that an attack occurs and succeeds; the application's integrity is about 0.83 (82.5%). Raising security (better fortification) would raise integrity: e.g. with security $= 0.90$, integrity $= 1 - 0.25 \times 0.10 = 0.975$.

Answer. Integrity $= 1 - 0.25 \times (1 - 0.30) = 0.825$ (about 83%).

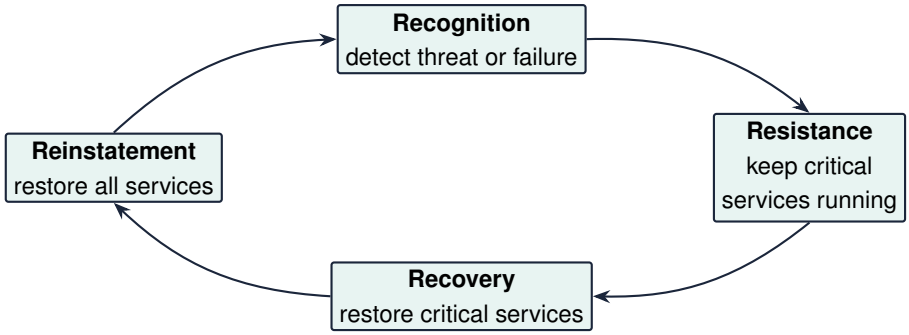
2011–2012 • Q2(b) CSE307 • 23/09/2013 • S014 **MARKS 6**

Define resilient and non-resilient systems.

SOLUTION

Resilient system

A **resilient system** is one that can *maintain the continuity of its critical services* in the presence of disruptive events — equipment failures, software faults, operator errors or cyber-attacks — and can *recover* quickly to normal operation afterwards. Resilience is achieved through four activities (Sommerville's "4 Rs"):



- **Recognition** — detect symptoms of a problem (monitoring, intrusion detection).
- **Resistance** — resist the event and keep critical services running, perhaps in a degraded mode (redundancy, failover, isolation of the damaged part).
- **Recovery** — quickly restore critical services (backups, restart, standby site).
- **Reinstatement** — restore all remaining services to full normal operation.

Example: an online banking system with replicated data centres; when one site fails or is attacked, traffic fails over, balance enquiry and transfers continue, and the failed site is restored from backups.

Non-resilient system

A **non-resilient (brittle) system** cannot tolerate disruption: a single failure, unexpected input or attack causes its critical services to fail, often completely and possibly with data loss, and recovery requires lengthy manual intervention. Typical causes are single points of failure, no redundancy or backups, no monitoring, and no degraded mode of operation. Example: a shop's billing system running on one PC with no backup — a disk crash halts all sales until the machine is rebuilt and data are re-entered.

Aspect	Resilient	Non-resilient
Effect of a failure/attack	Critical services continue (maybe degraded)	Services stop; may fail catastrophically
Design	Redundancy, diversity, monitoring, failover	Single points of failure
Recovery	Fast, largely automatic	Slow, manual; possible data loss

Answer. Resilient system: maintains its critical services despite disruptive events and recovers quickly (recognition, resistance, recovery, reinstatement). Non-resilient system: a disruption leads to failure of critical services and slow/manual recovery.

X7

Architecture, Performance & DevOps

OUT OF SYLLABUS

2023–2024 • Q3(b) CSE213 • 10/05/2026 • A001 **MARKS 15**

An online library platform provides access to a large collection of digital resources, including books, journals, research papers, and multimedia content. Users can search the catalog, view details, read content, and interact through features such as bookmarking and reviews. As the platform grows in popularity, it experiences a high volume of read requests (e.g., searches and content access), while write operations remain relatively low. At the same time, the continuously expanding database is creating challenges in scalability and performance.
To address these issues, the company is considering implementation of server-side caching and database sharding. Additionally, an investor has suggested that, given the platform's expected global reach, adopting a Content Delivery Network (CDN) in the near future would be beneficial. Discuss whether and how these approaches would effectively address the platform's challenges and improve performance.

SOLUTION

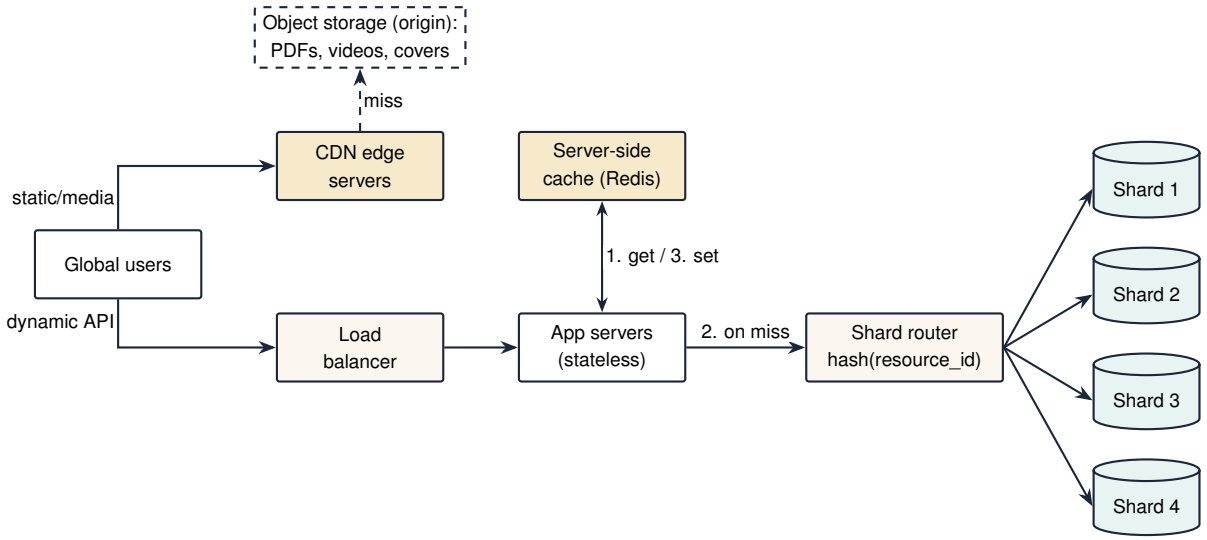
Diagnosing the workload

The platform has two *different* problems:

(i) a **read-heavy load** (searches, catalogue pages, content access ≫ writes) that overloads the database with repetitive queries, and

(ii) a **data-volume problem** – a single database keeps growing, so indexes, backups and queries get slower and one machine eventually cannot store or serve it.

A third, future problem is **geographic latency** for global users. Each proposed technique targets exactly one of these, so all three are justified – but for different reasons.



1. Server-side caching – solves the read-heavy load

- Keep frequently-read, rarely-changed data (catalogue entries, book details, popular search results, review summaries) in an in-memory cache using the **cache-aside / read-through** policy: read from cache; on a miss read the DB and populate the cache with a TTL.
- Because writes are rare, cached data seldom becomes stale; on a write (new review, edit) the corresponding key is *invalidated*.

Effect (illustrative numbers). Let the platform receive $R = 10\,000$ reads/s, cache latency $t_c = 1$ ms, DB latency $t_d = 20$ ms and hit ratio $h = 0.9$:

$$\text{DB reads/s} = (1-h)R = 0.1 \times 10\,000 = 1\,000 \quad (\text{a } 90\% \text{ reduction})$$

$$\bar{t} = h\,t_c + (1-h)(t_c + t_d) = 0.9(1) + 0.1(21) = 0.9 + 2.1 = 3.0 \text{ ms}$$

versus 20 ms without the cache, i.e. about $20/3 \approx 6.7\times$ faster on average. Caching is therefore **highly effective** for this read-dominated workload. It does *not* solve the growing-data problem (the DB still stores everything).

2. Database sharding – solves the growing-data problem

- **Sharding** = horizontal partitioning of the data across several independent DB servers by a *shard key*; each shard holds a subset of rows (e.g. `hash(resource_id)` for resources, `hash(user_id)` for bookmarks/reviews so a user's data lives together).
- Each shard has a smaller data set (smaller indexes, faster queries and backups), and storage/write capacity grows by adding shards.

Effect. With 8 TB of data and 4 shards each shard stores $8/4 = 2$ TB, and the 200 writes/s are spread as $200/4 = 50$ writes/s per shard. **Care needed.** (a) Choose **consistent hashing**: with plain `hash mod N`, growing from 4 to 5 shards keeps a key on the same shard only when $k \bmod 4 = k \bmod 5$, i.e. for 4 of every 20 residues – so 80% of the data must move; with consistent hashing only about $1/5 = 20\%$ moves. (b) Full-text search across shards should be served by a separate search index (e.g. Elasticsearch) rather than scatter–gather queries. (c) Cross-shard joins and transactions become harder. Sharding is **effective and necessary** for long-term growth, but it is complex, so it should come after caching and read replicas.

3. Content Delivery Network – solves global latency and bandwidth

- A CDN replicates *static and multimedia content* (PDFs, e-book pages, images, videos, JS/CSS) on edge servers close to users; requests are served by the nearest edge.
- Example: a reader in Canada fetching from an origin in Dhaka sees an RTT of ≈ 250 ms; from a CDN edge in Toronto ≈ 20 ms – for a page needing 10 sequential round trips that is 2.5 s vs. 0.2 s. With an edge hit ratio of 95%, origin egress drops from, say, 2 TB/day to $0.05 \times 2 = 0.1$ TB/day.
- A CDN does *not* help personalised, dynamic requests (bookmarks, search) – those still need the app tier, caching and sharding.

Verdict

Technique	Problem addressed	Limitation
Server-side cache	read-heavy DB load, latency	staleness/invalidation; not storage
Sharding	growing data size, write/storage scaling	complexity, cross-shard queries, resharding
CDN	global latency, bandwidth for static/media	useless for dynamic, personalised data

Answer. Yes – the three are complementary: caching (cache-aside, invalidate on write) removes most repetitive DB reads ($\approx 90\%$ fewer reads, 20 ms $\rightarrow$ 3 ms); sharding by a hashed key spreads the ever-growing data over several servers; a CDN serves static/media content from edges near global users. Together they address both the read load and the data growth.

2023–2024 • Q7(b), (c) CSE213 • 09/02/2025 • A002 MARKS 10 + 10

(b) In a highly reputed university, when exam results are published online, thousands of students simultaneously attempt to access the results portal. This surge in traffic often causes the portal to crash or become extremely slow. Despite increasing the number of servers, the issue persists. Analyze the possible underlying problem and propose an appropriate solution.
(c) Describe the purpose and benefits of software profiling. Discuss the importance of monitoring resource utilization metrics and assessing the scalability of software during the development process.

SOLUTION

(b) Analysis – why more servers did not help

Adding web/application servers only helps if they are the bottleneck. Every one of them still depends on a **shared resource**, so the true bottleneck is elsewhere, most likely:

(i) **The single database.** Every request queries the same DB for the same kind of data; more app servers only send *more* concurrent queries and exhaust its connection pool.

(ii) **No caching** although results are *read-only and identical* for everyone after publication.

(iii) Other shared limits: network bandwidth of the campus link, a shared session store, a mis-configured load balancer (sticky sessions to one node) or synchronous blocking code.

Illustrative calculation. Suppose 60 000 students try within the first minute: $\lambda = 60\,000/60 = 1\,000$ requests/s. Each result page runs 3 queries $\Rightarrow$ 3 000 queries/s. If the DB server can execute 2 000 queries/s,

$$\text{max throughput} = \frac{2\,000}{3} \approx 667 \text{ req/s, regardless of the number of app servers.}$$

Excess $= 1\,000 - 667 = 333$ req/s pile up; after 30 s about $333 \times 30 \approx 10\,000$ requests are waiting $\Rightarrow$ timeouts, exhausted connections, crash. Bandwidth check: with 200 KB pages, $1\,000 \times 200$ KB $= 200$ MB/s $= 1.6$ Gbit/s – more than a 1 Gbit/s link.

(b) Proposed solution

(i) **Pre-compute and cache results.** Results do not change after publication: generate them once (static JSON/HTML per student) and serve from an in-memory cache (Redis) or as static files. With a 99% hit ratio DB load becomes $3\,000 \times 0.01 = 30$ queries/s.

(ii) **CDN / static hosting and compression** for pages and assets: cuts the 1.6 Gbit/s bandwidth need (gzip of a 200 KB page to ≈ 40 KB gives 0.32 Gbit/s).

(iii) **Read replicas** of the DB and **connection pooling** for whatever still reaches the DB; add proper **indexes** (on student ID).

(iv) **Load balancing + auto-scaling** of stateless app servers (sessions in a shared store).

(v) **Traffic shaping:** a virtual waiting room / rate limiting, or staggered publication (department-wise), or pushing results by SMS/e-mail.

(vi) **Load testing** before result day to find the next bottleneck.

(c) Software profiling – purpose and benefits

Profiling is dynamic analysis that measures *where* a program spends its resources while it runs – CPU time per method, call counts, memory allocation and leaks, I/O waits, lock contention, garbage collection. It is done by *sampling* (periodically recording the call stack) or by *instrumentation* (inserting timers/counters), using tools such as VisualVM, JProfiler, perf, py-spy.

- finds the real **hot spots** (typically 10–20% of code consumes 80–90% of the time), so optimisation effort is targeted instead of guessed;
- detects **memory leaks**, excessive object creation, inefficient algorithms (e.g. an $O(n^2)$ loop) and N+1 database queries;
- verifies that an optimisation really helped (before/after comparison) and prevents performance regressions;
- reduces infrastructure cost and improves user experience.

(c) Monitoring resource utilisation and assessing scalability during development

- **Metrics to monitor:** CPU %, memory/heap and GC pauses, disk I/O, network throughput, thread-pool and DB-connection usage, queue lengths; user-visible **latency percentiles** (p50/p95/p99), throughput (req/s) and error rate.
- **Why during development:** bottlenecks found late are expensive (architectural changes); trends show leaks and saturation before production; the data drives *capacity planning* and auto-scaling thresholds; performance requirements (e.g. "95% of requests < 300 ms") become testable.
- **Assessing scalability:** run load and stress tests with increasing users and check whether throughput grows proportionally to added resources. Example: 1 server = 500 req/s, 4 servers = 1 600 req/s $\Rightarrow$ speed-up $= 1600/500 = 3.2$, efficiency $= 3.2/4 = 80\%$. **Amdahl's law** explains the limit: with a serial (non-parallelisable) fraction $s = 0.1$,

$$S(N) = \frac{1}{s + (1-s)/N}, \qquad S(4) = \frac{1}{0.1 + 0.9/4} = \frac{1}{0.325} \approx 3.08, \qquad S(\infty) = \frac{1}{0.1} = 10.$$

This is exactly the results-portal lesson: the shared serial part (the DB) caps scalability.

Answer. (b) The bottleneck is a shared resource – chiefly an uncached single database (and bandwidth) – so extra app servers cannot help (max ≈ 667 req/s in the example). Pre-compute and cache results, use CDN/static hosting, read replicas, pooling, auto-scaling and a waiting room. (c) Profiling locates CPU/memory/I/O hot spots so optimisation is evidence-based; monitoring utilisation and testing scalability early exposes bottlenecks, supports capacity planning and keeps performance requirements verifiable.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q3(b), (c) CSE307 • 05/05/2024 • A003 MARKS 12 + 8

(b) You have chosen write-through/read-through when you actually should be using write-around/read-through to implement a cache in your system. Discuss an example scenario where such mistake may happen. What will be the consequence?
(c) Discuss CI/CD in brief in the context of DevOps engineering.

2020–2021 • Q7(b), (c); Q8(a), (b) CSE307 • 19/10/2022 • A007 MARKS 16 + 9 + 15 + 12

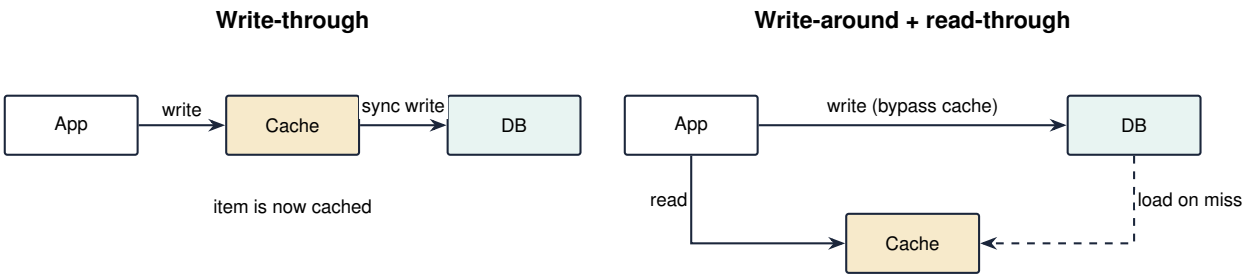
7(b) You have chosen write-through/read-through when you actually should be using write-around/read-through to implement cache in your system. Discuss an example scenario where such mistake may happen. What will be the consequence?
7(c) Give three example scenarios where Data-Centered software architecture is appropriate.
8(a) What are the benefits of using Message Queue? What are the good practices recommended to ensure Confidentiality in software application?
8(b) Discuss Continuous Integration and Containerization in brief in the context of DevOps engineering.

SOLUTION

NOTE The two excerpts share the caching question (A003(b) = A007 Q7(b)); it is solved once below. The remaining parts are answered in turn: A003(c) and A007 Q8(b) (CI/CD, containerization), A007 Q7(c) (data-centred architecture) and A007 Q8(a) (message queue, confidentiality). The A007 marks were read from the original paper (All Past Papers, p. 21) because the crop is cut at the top.

A003(b) / A007 7(b) – Cache write policies

- **Read-through:** the application reads only from the cache; on a miss the cache loads the item from the DB, stores it and returns it.
- **Write-through:** every write goes to the cache *and* synchronously to the DB; the written item is now in the cache. Good when *recently written data is read again soon*.
- **Write-around:** writes go *directly to the DB, bypassing the cache*; the item enters the cache only if/when it is read (via read-through). Good when *written data is rarely or not immediately re-read*.



Example scenario where the mistake happens. An e-commerce site caches its hot *product catalogue* (8 000 products, 5 000 reads/s). The same cache layer is reused for a new *audit-log / click-stream* feature that writes 2 000 new records per second which are almost never read back (similar cases:

IoT sensor readings, chat archives, bulk import of historical data, nightly batch updates). The developer picks write-through for everything, although these write-heavy, read-rarely records should use write-around.

Consequences.

(i) **Cache pollution / thrashing.** With a 10 000-entry LRU cache, the log records fill it in $10\,000/2\,000 = 5$ s and keep evicting the hot product entries.

(ii) **Hit ratio collapses ⇒ latency and DB load explode.** With $t_c = 1$ ms, $t_d = 20$ ms:

$$h = 0.95: \ \bar{t} = 0.95(1) + 0.05(21) = 2.0\text{ ms}; \qquad h = 0.30: \ \bar{t} = 0.30(1) + 0.70(21) = 15.0\text{ ms}$$

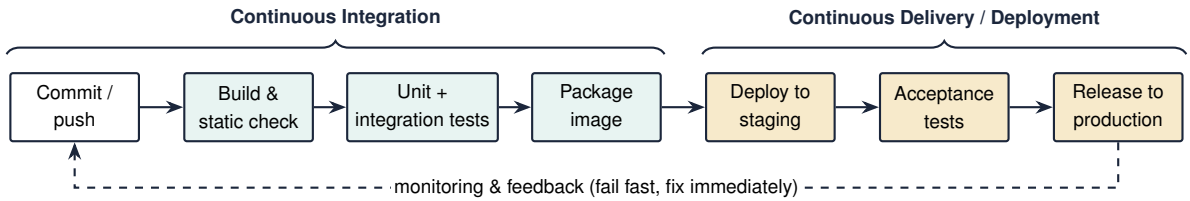
and DB reads rise from $0.05 \times 5\,000 = 250/\text{s}$ to $0.70 \times 5\,000 = 3\,500/\text{s}$ (14×).

(iii) **Higher write latency**: each write waits for both cache and DB.

(iv) **Wasted memory and cost** (and network traffic) to store data nobody reads; a bigger cache would be bought to fix a policy error.

A003(c) / A007 8(b) – CI/CD and containerization in DevOps

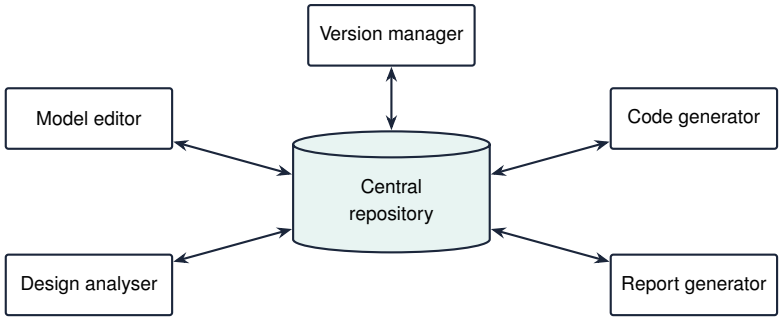
DevOps combines development and operations through culture, automation and continuous feedback (plan → code → build → test → release → deploy → operate → monitor → plan …). CI/CD is the automated pipeline at its core.



- **Continuous Integration (CI):** developers merge small changes into the shared mainline several times a day; every push triggers an automated build, static analysis and test suite. A broken build is fixed immediately. Benefit: integration problems are found within minutes, the mainline is always working.
- **Continuous Delivery (CD):** every build that passes CI is automatically deployed to staging and acceptance-tested, so the software is *always releasable*; release to production is a one-click (business) decision.
- **Continuous Deployment:** goes one step further – every change that passes all stages is released to production automatically (with feature flags, canary/blue-green releases, rollback).
- **Containerization:** the application and all its dependencies (runtime, libraries, configuration) are packaged into a lightweight *container image* (Docker) that shares the host OS kernel. The same image built by CI is promoted unchanged through test, staging and production ("build once, run anywhere"), which removes "works on my machine" problems; containers start in seconds, isolate services from each other, use fewer resources than VMs and are scaled and self-healed by orchestrators (Kubernetes).

A007 7(c) – Data-centred (repository) architecture: three scenarios

In a **data-centred architecture** a central data store (repository or blackboard) is accessed and modified by independent client components that communicate *only through the shared data*.



(i) **IDE / CASE toolset:** editors, analysers, code generators, debuggers all read and write a shared project repository (models, source, symbol tables).

(ii) **Hospital / banking information system:** admission, pharmacy, lab, billing modules all work on one central patient (or account) database; each module is independent but must see consistent, shared records.

(iii) **Blackboard systems:** speech/image recognition or military sensor fusion, where specialised knowledge sources post partial solutions on a shared blackboard and react to changes until a solution emerges. (Others: university records system, configuration management repository, compiler symbol table.)

It is appropriate when large volumes of data are shared and long-lived, and components should be added without knowing about one another.

A007 8(a) – Benefits of a message queue

A **message queue** lets producers send messages to a broker that stores them until consumers process them (asynchronous, point-to-point or publish/subscribe).

(i) **Decoupling** – producer and consumer need not know each other's location, technology or availability.

(ii) **Asynchronous processing** – the producer responds immediately; slow work happens later.

(iii) **Load levelling / buffering** – bursts are absorbed by the queue; consumers work at their own rate.

(iv) **Scalability** – add more competing consumers to increase throughput.

(v) **Reliability** – persistent messages, acknowledgements and retries/dead-letter queues mean no request is lost if a consumer crashes.

(vi) **Fault isolation and ordering guarantees** where required.

A007 8(a) – Good practices for confidentiality

Confidentiality = information is disclosed only to authorised people or systems.

(i) **Strong authentication** (MFA, secure password hashing with salt) and **authorisation** using least privilege / role-based access control.

(ii) **Encryption in transit** (TLS/HTTPS everywhere, including between services and to the queue) and **at rest** (database/disk encryption, encrypted backups).

(iii) **Secret management** – no hard-coded keys or passwords; use a vault; rotate keys.

(iv) **Data minimisation and classification** – collect only what is needed; mask or tokenise sensitive fields; never write passwords, tokens or personal data to logs.

(v) **Input validation and secure coding** to prevent injection/XSS that leak data; regular code review, security testing and patching.

(vi) **Auditing and monitoring** of access to sensitive data; secure session management (timeouts, secure cookies).

Answer. Write-through on write-heavy, rarely-read data (e.g. logs/telemetry) pollutes the cache, evicts hot items, cuts the hit ratio (latency 2 ms $\rightarrow$ 15 ms, DB reads $14\times$) and wastes memory – write-around should be used. CI = frequent merges with automated build/test; CD = always releasable, automated deployment pipeline; containers package app + dependencies for consistent, fast, isolated deployment. Data-centred: IDE/CASE repository, hospital/bank central DB, blackboard systems. Message queues give decoupling, asynchrony, load levelling, scalability and reliability; confidentiality needs least-privilege access, encryption in transit/at rest, secret management, data minimisation, secure coding and auditing.

2022–2023 • Q8(c) CSE307 • 05/05/2024 • A004 **MARKS 15**

Your application is suffering from the following performance bottlenecks:

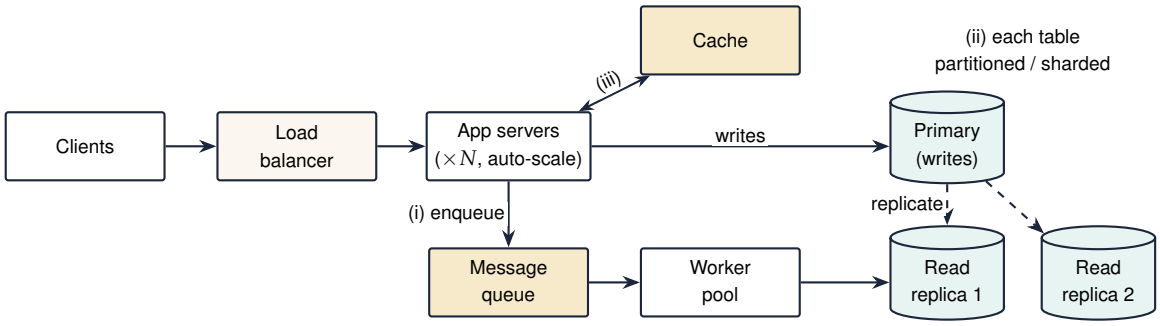
(i) Your application server is overwhelmed with a large number of synchronous requests.

(ii) Your database tables are filled up with billions of rows.

(iii) Your database server is overwhelmed with a large number of read queries.

Describe in detail how you solve each of the above bottlenecks.

SOLUTION



(i) App server overwhelmed by synchronous requests

Why it happens. In a synchronous (blocking) model each request holds a thread until all the work finishes. By **Little's law** $L = \lambda W$: with $\lambda = 2\,000$ req/s and $W = 0.5$ s of work per request, $L = 2\,000 \times 0.5 = 1\,000$ threads must be busy at once. A pool of 200 threads can serve only $200/0.5 = 400$ req/s; the rest queue up and time out.

Solutions.

(i) **Make long work asynchronous:** the server validates the request, puts a job on a **message queue** (RabbitMQ/Kafka/SQS) and immediately returns `202 Accepted` with a job ID; a pool of **workers** consumes jobs at a sustainable rate; the client polls or is notified (WebSocket/callback). If accepting takes $W = 5$ ms, only $2\,000 \times 0.005 = 10$ threads are needed. The queue also *levels bursts*.

(ii) **Non-blocking I/O** (event loop, reactive frameworks) so threads are not idle while waiting for the DB or remote services.

(iii) **Horizontal scaling** of stateless servers behind a **load balancer** with auto-scaling (e.g. add an instance when CPU $> 70\%$).

(iv) **Rate limiting / back-pressure**, timeouts and circuit breakers so overload degrades gracefully instead of crashing.

(ii) Tables with billions of rows

(i) **Indexing** the columns used in `WHERE/JOIN/ORDER BY`. A B^+-tree with fan-out 200 over 10^9 rows has height $\lceil \log_{200} 10^9 \rceil = \lceil 9/2.301 \rceil = \lceil 3.91 \rceil = 4$, so a lookup reads ≈ 4 pages instead of scanning $10^9/100 = 10^7$ pages (at 100 rows/page).

(ii) **Partitioning** (within one server) by range/list/hash, e.g. monthly partitions of an orders table: 5 years = 60 partitions, so a "last month" query touches $1/60$ of the data (*partition pruning*); old partitions can be dropped or archived cheaply.

(iii) **Sharding** (across servers) by a shard key such as `customer_id` when one machine can no longer store or write the data.

(iv) **Archiving cold data** to cheaper storage / data warehouse, and summary (materialised) tables for reports; paginate queries instead of loading millions of rows.

(iii) DB server overwhelmed by read queries

(i) **Caching** (cache-aside with Redis/Memcached, HTTP caching, CDN for static data): with an 80% hit ratio, 30 000 reads/s become $0.2 \times 30\,000 = 6\,000$ reads/s at the DB.

(ii) **Read replicas** (primary–replica replication): writes go to the primary, reads are spread over replicas. If one node sustains 8 000 reads/s, the original load needs $\lceil 30\,000/8\,000 \rceil = 4$ nodes; after caching, $\lceil 6\,000/8\,000 \rceil = 1$ node (keep one extra replica for availability). Mind *replication lag* (read-your-own-writes from the primary).

(iii) **Query optimisation**: proper indexes, avoid `SELECT *` and N+1 queries, use connection pooling, materialised views for expensive aggregates.

Answer. (i) Offload work to a message queue + workers (async, 202 Accepted), non-blocking I/O, load balancing and auto-scaling, rate limiting; (ii) index, partition, shard and archive the huge tables; (iii) add caching and read replicas, and optimise queries.

2021–2022 • Q1(c) CSE307 • 07/05/2023 • A005 **MARKS 6**

Discuss briefly the following **System Implementation** methods. Also, state a real-world use case for each of the implementation methods: *Parallel implementation, Plunge implementation, Pilot implementation* and *Phased implementation.*

SOLUTION

The four methods (see also the timeline figure in A023)

Method	How it works (risk / cost)	Real-world use case
Parallel	Old and new systems run side by side on the same live data for a period; results are compared and the old system is retired once the new one is trusted. *Lowest risk, highest cost* (double work).	A bank moving its *payroll* or interest-calculation system: both compute salaries for 2–3 months and outputs are reconciled before the old one is switched off.
Plunge (direct cut-over)	The old system is stopped and the new one starts at a fixed moment. *Cheapest, fastest, highest risk* – no fallback.	A small shop replacing its POS software over a weekend; or a new online exam-registration site that has no previous system to run in parallel.
Pilot	The complete new system is first installed in one location/department (the pilot); after successful use it is rolled out to the others. *Risk confined to one site.*	A supermarket chain (e.g. Shwapno) deploying a new inventory system in one outlet first; a university trying a new result system in one department.
Phased	The system is introduced in stages – module by module – each stage replacing part of the old system. *Gradual, moderate risk, longer duration.*	An organisation adopting an ERP: finance module first, then HR/payroll, then inventory and procurement.

Answer. Parallel = run old and new together (payroll in a bank); plunge = switch over at once (small shop POS); pilot = one site first then roll-out (one branch of a retail chain); phased = module by module (ERP finance → HR → inventory).

2020–2021 • Q6(a) CSE307 • 19/10/2022 • A006 **MARKS 12**

To purchase books, the sites like Amazon.com or Rokomari.com are widely used. Draw an architecture to design such a system following Microservice architecture.

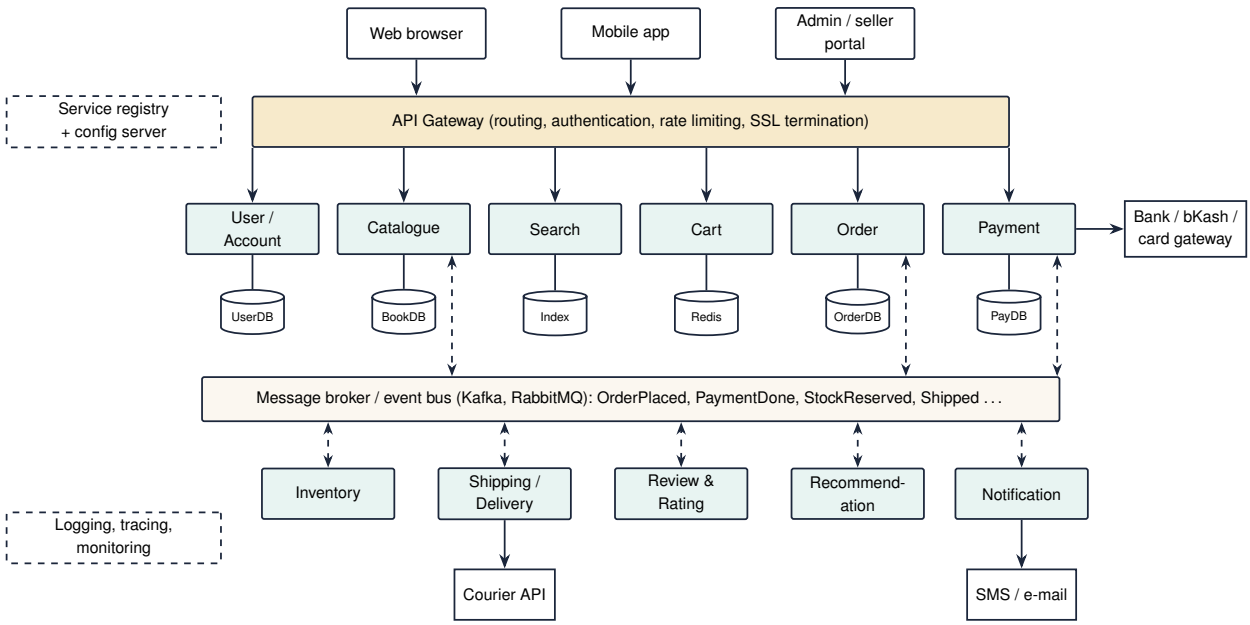
SOLUTION

NOTE The bank's crop of this question is cut at the bottom; the full text and the marks (12) were taken from the original paper (All Past Papers, p. 21).

Identifying the services (decompose by business capability)

An online bookstore has these business capabilities, each of which becomes an independently deployable **microservice** with its **own database**: User/Account, Catalogue (books, authors, publishers), Search, Inventory, Cart, Order, Payment, Shipping/Delivery, Review & Rating, Recommendation and Notification.

Architecture diagram



How it works

(i) Every client request enters through the **API Gateway**, which authenticates the user (token from the User service), routes the call to the right service and aggregates responses.

(ii) **Database per service**: Catalogue owns book data, Order owns orders, Cart uses an in-memory store, Search uses a full-text index – no service touches another's database.

(iii) **Synchronous calls** (REST/gRPC) are used when an immediate answer is needed (search, view book, add to cart). **Asynchronous events** through the broker are used for the order workflow (a *saga*): *OrderPlaced* → Inventory reserves stock → Payment charges the customer → *PaymentDone* → Shipping books the courier → Notification sends SMS/e-mail; if payment fails a compensating event releases the stock.

(iv) **Service registry/discovery**, central configuration, and logging/tracing/monitoring support many independently running instances (containers orchestrated by Kubernetes).

Why microservices for a bookstore

Traffic is uneven (Search and Catalogue get 100× more requests than Payment), features change frequently (recommendation, offers) and the business must stay up even if one part fails; services can be scaled, deployed and repaired independently.

Answer. Clients → API gateway → independent services (User, Catalogue, Search, Cart, Order, Payment, Inventory, Shipping, Review, Recommendation, Notification), each with its own DB, cooperating via REST and an event bus, plus registry/config and monitoring infrastructure.

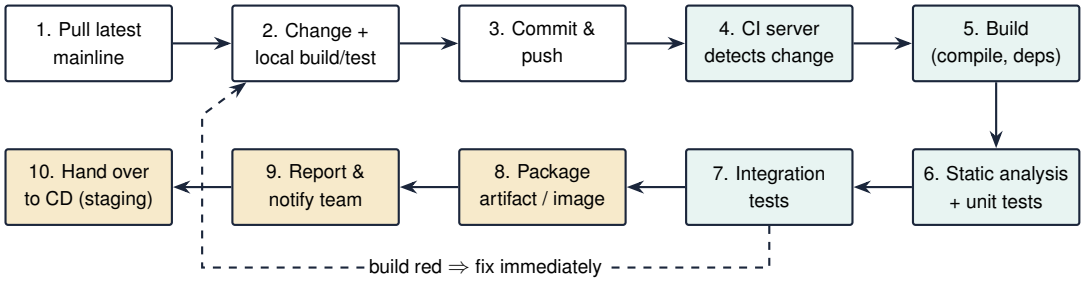
Jan 2021 • Q6(b) CSE307 • 04/09/2021 • A008 **MARKS 15**

Write down the steps of Continuous Integration in DevOps life-cycle. What are the benefits of Containerization?

SOLUTION

Continuous Integration in the DevOps life-cycle

Continuous Integration (CI) is the DevOps practice in which every developer integrates (merges) small changes into a shared mainline frequently – at least daily – and every integration is verified by an *automated build and test* so that errors are detected within minutes. It sits between the *code* and *release* stages of the DevOps loop (plan → code → **build** → **test** → release → deploy → operate → monitor).



(i) **Pull the latest mainline** from the shared repository into the working copy.

(ii) **Make a small change** and build/run unit tests locally (private build).

(iii) **Commit and push** to the shared repository (mainline or short-lived branch + pull request).

(iv) The **CI server** (Jenkins, GitHub Actions, GitLab CI) detects the commit via webhook/polling and checks out the code in a clean environment.

(v) **Automated build**: resolve dependencies, compile.

(vi) **Static analysis and unit tests** (code style, bugs, coverage thresholds).

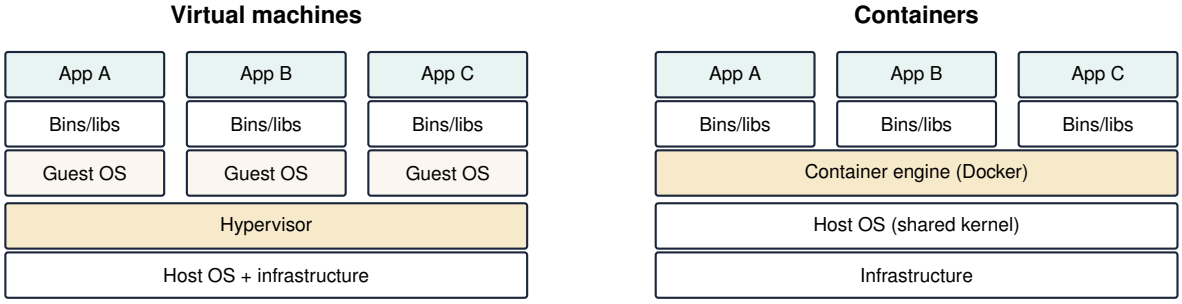
(vii) **Integration tests** (DB, services, APIs) – often inside containers.

(viii) **Package** the build into a versioned artifact/container image and store it in a registry.

(ix) **Report/notify**: the whole team sees green/red status; a broken build is the team's top priority and must be fixed (or reverted) immediately.

(x) The verified artifact is **handed to Continuous Delivery** for deployment to staging and acceptance tests.

Benefits of containerization



A **container** packages an application with all its dependencies into an isolated, portable unit that shares the host OS kernel.

(i) **Consistency / portability** – the same image runs identically on a laptop, CI server, staging and production ("works on my machine" disappears).

(ii) **Lightweight and fast** – no guest OS: images are megabytes, containers start in seconds, so many more fit on one host than VMs.

(iii) **Isolation** – each service has its own file system, libraries and resource limits; conflicting library versions can coexist.

(iv) **Scalability and resilience** – orchestrators (Kubernetes) start more replicas on demand, restart failed containers and perform rolling updates.

(v) **Immutable, versioned deployments** – the image built and tested by CI is exactly what is deployed; rollback = run the previous image tag.

(vi) **Fits microservices and CI/CD** – each service is built, tested and deployed independently; test environments (DB, broker) are spun up on demand.

(vii) **Better resource utilisation and lower cost**.

Answer. CI steps: pull mainline → small change + local test → commit/push → CI server detects → automated build → static analysis + unit tests → integration tests → package artifact → notify (fix broken builds at once) → hand to CD. Containerization gives portability/consistency, lightweight fast start-up, isolation, easy scaling and self-healing, immutable versioned deployments and efficient resource use.

Jan 2021 • Q8(a) CSE307 • 04/09/2021 • A009 MARKS 20

You are trying to log-in to your BIIS account. Discuss all the steps of communications that will take place inside the application developed following Model-View-Controller architecture. Discuss the advantages of Microservice architecture.

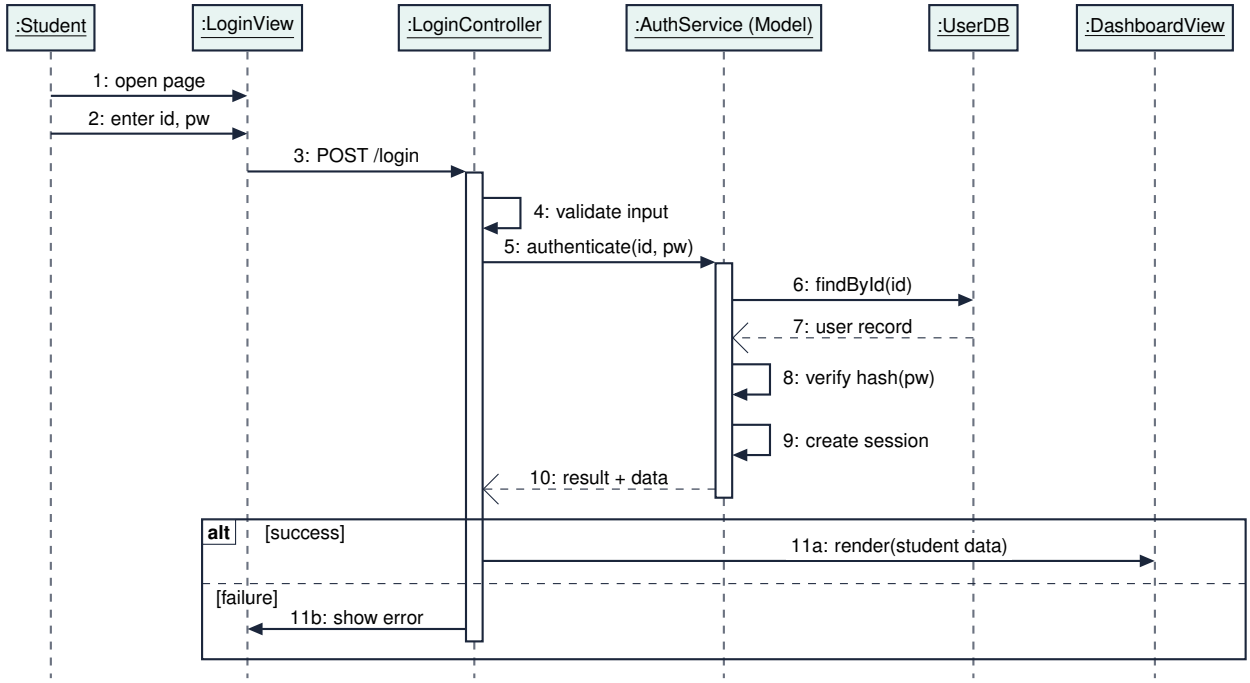
SOLUTION

MVC in brief

Model–View–Controller separates an interactive application into three roles:

- **Model** – application data and business rules (here `User`, `UserRepository`, `AuthService`, the session); it knows nothing about screens.
- **View** – renders the model to the user (login page, dashboard, error message).
- **Controller** – receives user input (HTTP request), invokes model operations and selects the view to display.

Communication steps for "log in to BIIS"



(i) The student opens the BIIS URL; the browser shows the **View** (login form).

(ii) The student types ID and password and presses *Login*.

(iii) The View forwards the input (HTTP POST) to the **Controller** – the View itself performs no business processing.

(iv) The `LoginController` validates the input format (non-empty, 7-digit ID, CAPTCHA . . .).

(v) The Controller calls the **Model**: `authService.authenticate(id, pw)`.

(vi) The Model's repository queries the database for the student record.

(vii) The DB returns the stored record (salted password hash, status, role).

(viii) The Model hashes the supplied password and compares; it also applies business rules (account locked? number of failed attempts?).

(ix) On success the Model creates a session/token and loads the student's data (name, term, results).

(x) The Model returns the outcome to the Controller.

(xi) The Controller *selects the view*: on success (11a) it redirects to the `DashboardView`, which reads the model data and renders the home page; on failure (11b) it returns the login view with an error message. (In classic MVC the Model may also *notify* registered views of changes – the Observer relationship.)

Advantages of microservice architecture

A **microservice** architecture builds an application as a suite of small, independently deployable services, each owning one business capability and its own data, communicating through lightweight APIs/messages.

(i) **Independent deployment** – one service (e.g. Result service) can be updated without redeploying the whole BIIS; enables continuous delivery.

(ii) **Independent scaling** – only the hot service is replicated (Result service on result day), saving resources.

(iii) **Fault isolation** – a crash in the Payment service does not bring down Login or Course registration.

(iv) **Technology heterogeneity** – each team chooses the best language/database for its service.

(v) **Small, understandable code bases** – easier for new developers; faster builds and tests.

(vi) **Team autonomy / organisational alignment** – small teams own services end-to-end.

(vii) **Reusability** – services (authentication, notification) are reused by other systems.

Answer. Login in MVC: View captures credentials → Controller validates and calls Model → Model queries DB, verifies hash, creates session → Controller picks Dashboard or error View, which renders model data. Microservices give independent deployment and scaling, fault isolation, technology freedom, small code bases and team autonomy.

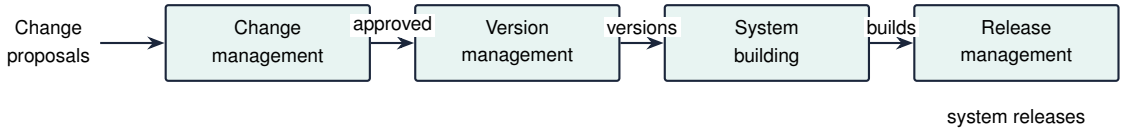
2017–2018 • Q3(a)–(e) CSE307 • 29/09/2018 • A010 **MARKS 7 × 5**

(a) What do you mean by *configuration* in Software Configuration Management? What are the activities of Software Configuration Management? Explain with brief description.
(b) Explain baseline, codeline and mainline with proper examples.
(c) Consider a university records management system. The university will allow the students of any department to take audit courses from other departments. In this case the grades will not be reflected in the cumulative grade point average. Show the content of the change request form after approving the change.
(d) Why is merging required after branching of the codelines. Explain with necessary figures after defining branching and merging.
(e) Explain how the source code checksum helps minimize recompilation in system building.

SOLUTION

(a) Configuration and SCM activities

Software Configuration Management (SCM) is concerned with the policies, processes and tools for managing changing software systems. A **software configuration item (SCI)** is any work product placed under control – source files, designs, requirements, test data, build scripts, documents. A **configuration** is a *specific, named set of versions of the configuration items* that together make up one version of the system (e.g. "BIIS 2.3 = Login.java v7 + Result.java v12 + schema v5 + config v3"). Because many versions of every item exist, the configuration identifies *which* versions belong together.



(i) **Change management** – keeping track of change requests from customers and developers, analysing their cost and impact, deciding (CCB) which changes to implement and tracking them.

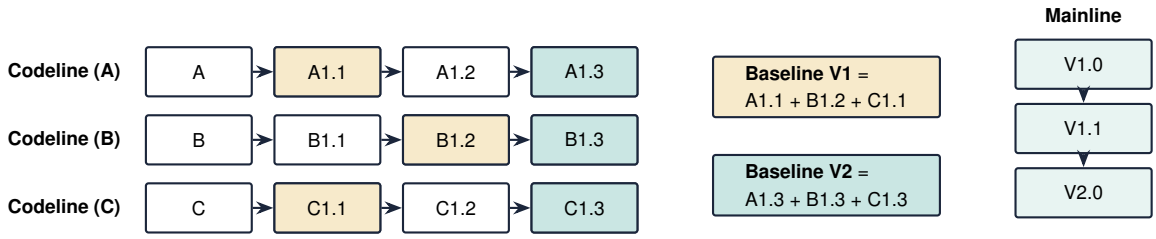
(ii) **Version management** – keeping track of the multiple versions of system components (codelines, baselines) and ensuring that changes made by different developers do not interfere (check-in/check-out, branching, merging).

(iii) **System building** – assembling program components, data and libraries and compiling and linking them into an executable system (build scripts, automated builds).

(iv) **Release management** – preparing software for external release and keeping track of the versions released to customers (release notes, installers, configuration files).

(Pressman lists the same ideas as: identification of SCIs, version control, change control, configuration auditing and status reporting.)

(b) Baseline, codeline and mainline



- **Codeline** – a sequence of versions of source code, with later versions derived from earlier ones; normally the versions of one component. *Example:* `Result.java`: 1.0 → 1.1 (bug fix) → 1.2 (new grading rule).
- **Baseline** – a definition (snapshot) of a specific system: the set of component versions that are included, plus the libraries, configuration files etc. It is *controlled*: it can be recreated at any time and its components cannot be changed without a formal change process. *Example:* "BIIS Release 2.0 baseline = Login 1.3, Result 1.2, Registration 2.1, schema 5, JDK 17."
- **Mainline** – a sequence of baselines representing the different versions of the whole system. *Example:* BIIS V1.0 → V1.1 → V2.0 → V2.1.

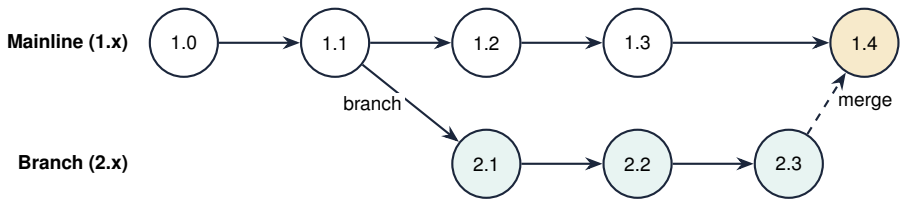
(c) Change request form after approval

Change Request Form

Project	University Records Management System (URMS), version 3.2
Change request number	URMS/CR/2018/017
Change requester	Controller of Examinations, on behalf of Academic Council
Date requested	05/03/2018
Requested change	Allow students of any department to register *audit courses* offered by other departments. Audit-course grades must appear on the transcript, marked "Audit", but must *not* be included in term GPA or CGPA computation.
Change analyser	A. Rahman (system analyst) Analysis date: 12/03/2018
Components affected	CourseRegistration, GradeEntry, GPACalculator, TranscriptGenerator, Registration DB schema (new `isAudit` flag), registration UI
Associated components	Fee module (audit-course fee), Prerequisite checker, Reports
Change assessment	Moderate: new attribute + change of GPA rule; no impact on existing grades; regression testing of GPA/CGPA required
Change priority	Medium (needed before next term's registration)
Change implementation	Add course-type flag; exclude flag = audit in GPA/CGPA query; show "AU" grade on transcript; enforce credit limit
Estimated effort	12 person-days (design 2, code 5, test 4, docs 1)
Date to CCB	15/03/2018 **CCB decision date:** 20/03/2018
CCB decision	**Approved** – implement in release 3.3
Change implementor	S. Karim (developer) Date of change: 02/04/2018
Date submitted to QA	10/04/2018 **QA decision:** passed
Date submitted to CM	15/04/2018
Comments	Notify all departments; update academic rules document.

(d) Branching and merging

- **Branching** – the creation of a new codeline from a version in an existing codeline. The new and existing codelines may then develop independently (e.g. a release-maintenance branch, a feature branch, a customer-specific variant).
- **Merging** – the creation of a new version of a component by combining separate versions from different codelines (with conflict resolution where both changed the same lines).



Why merging is required. After branching, both codelines evolve: the mainline may receive bug fixes (1.2, 1.3) while the branch adds a feature or fixes a released version (2.1–2.3). If the codelines are never merged:

(i) the product *diverges* – fixes made on one line are missing from the other (a bug fixed in the release branch reappears in the next release);

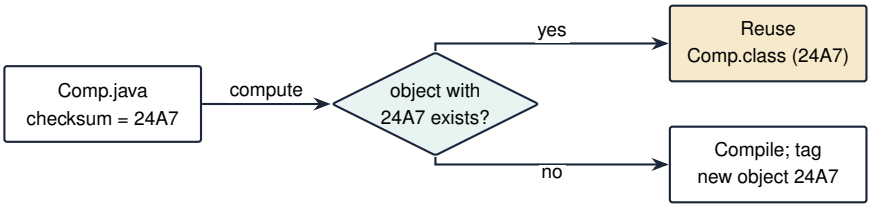
(ii) effort is duplicated, because every change must be re-implemented on each line;

(iii) finally the codelines become so different that integration is very costly.

Merging (here into 1.4) integrates both sets of changes into one version so the next release contains all of them; it is done frequently to keep conflicts small.

(e) Source-code checksums and minimum recompilation

A build system must decide which components need recompiling. With *time-stamps* it recompiles whenever a source file is newer than its object file – which also happens when the file is merely touched, checked out again, or edited and reverted, causing unnecessary recompilation.
With **checksums**: when a source file is compiled, the build system computes a checksum (hash) of its *contents* and stores it with the generated object code.



(i) At the next build the checksum of the current source is recomputed. If an object file with the *same checksum* already exists, the source is unchanged in content and the object is reused – *no recompilation*, even if the time-stamp changed.

(ii) Only files whose content really changed (different checksum) are recompiled.

(iii) Because each object file is linked to the checksum of its source, **several versions of the object code can be kept at the same time**; switching between versions (e.g. building release 1.2 then 1.3 again) reuses the stored objects instead of recompiling.

Answer. (a) A configuration is the specific set of versions of all SCIs that make up a system version; SCM activities: change management, version management, system building, release management. (b) Codeline = versions of a component; baseline = controlled set of component versions forming a system; mainline = sequence of baselines. (c) CR form above (approved, release 3.3). (d) Branching creates parallel codelines; merging recombines them so fixes/features are not lost and the lines do not diverge. (e) Content checksums identify unchanged sources so their stored object code is reused.

2017–2018 • Q4(a), (b) CSE307 • 29/09/2018 • A011 **MARKS 10 + 10**

(a) How can you define software architecture? Show the architecture of a residential student hall management system with the following features:

(i) Seat allocation, change and cancellation

(ii) Food charge receiving through credit card and bank payment with automatic interfacing

(iii) Issuing and returning books from hall library

(iv) Clearance certificate issuing

(v) Loan issue and repayment of the loan.

(b) Represent a library management system using layered architectural pattern and client server

architectural pattern with necessary justifications.

SOLUTION

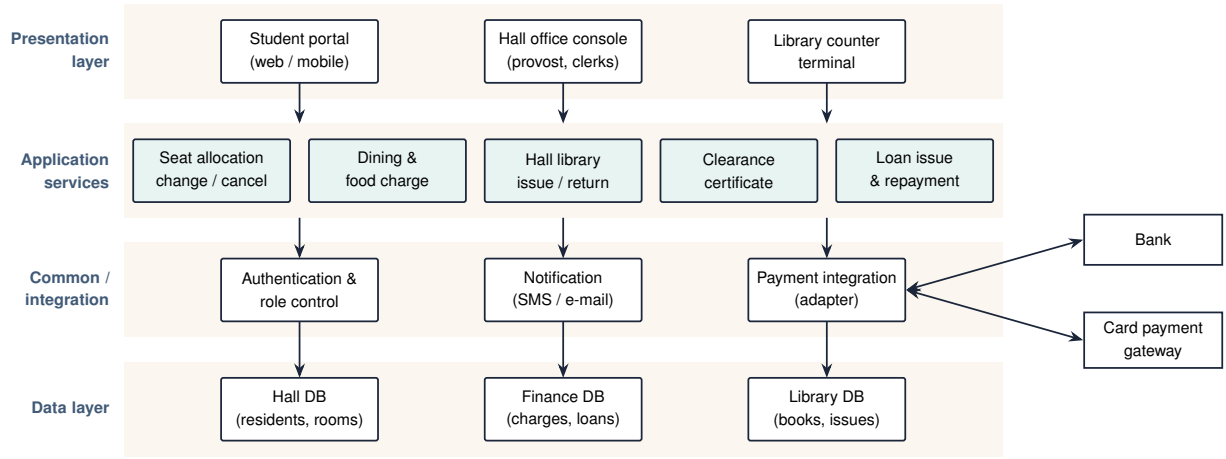
(a) Definition of software architecture

- **Bass, Clements & Kazman:** "The software architecture of a system is the *set of structures* needed to reason about the system, which comprise *software elements*, the *relations* among them, and the *properties* of both."
- **Sommerville:** architectural design is concerned with understanding how a system should be organised and designing its overall structure; its output is an *architectural model* that describes the system as a set of *communicating components*.

So an architecture answers three questions: *what are the major components?*, *how do they interact (connectors / interfaces)?* and *what externally visible properties (performance, security, availability, maintainability) does that organisation give?* It is the first design decision, it is hard to change later, and it is the main vehicle for meeting non-functional requirements.

(a) Architecture of the residential hall management system

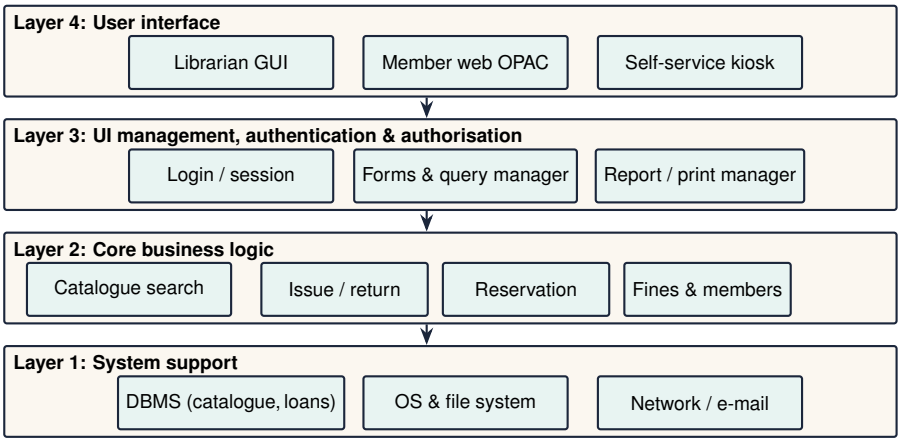
A **layered (3-tier) architecture** with one application service per feature is a natural fit: every feature is used by a different group of users (students, provost office, librarian, accounts), but they all share the same resident and finance data. Payment uses an *integration (adapter) component* that talks to the bank and card gateway automatically.



- **Presentation layer** – separate front ends for students (apply for / change / cancel a seat, view dues, request clearance), the hall office (approve seats, issue loans and clearances) and the library counter.
- **Application services** – one cohesive module per feature (i)–(v). *Clearance* is a composite service: it asks Seat (room handed over?), Dining (no dues?), Library (all books returned?) and Loan (loan repaid?) before issuing the certificate.
- **Common / integration services** – authentication and role-based access, notification, and the *payment integration* component that wraps the bank and card-gateway APIs, posts received payments automatically to the Finance DB (feature ii) and reconciles bank statements.

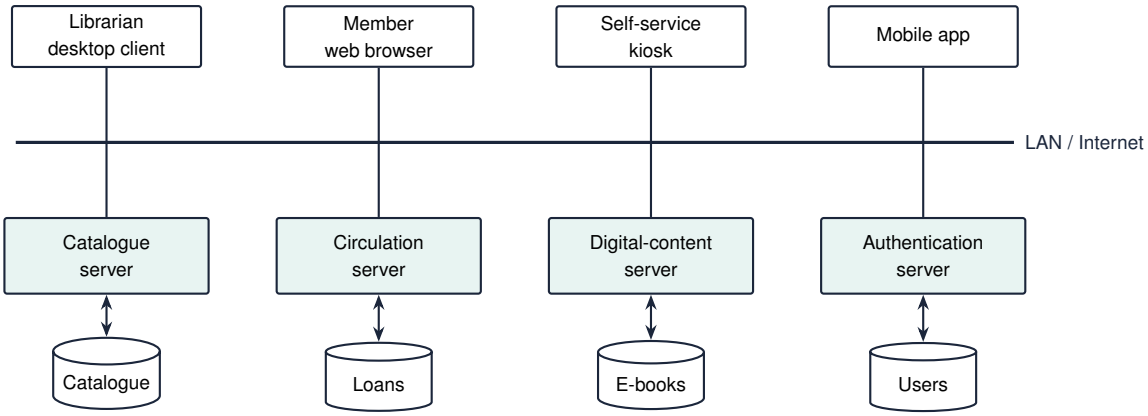
- **Data layer** – resident/room data, finance data (food charges, loans, repayments) and library data, accessed only through the services above.

(b) Library management system – layered pattern



Justification. Each layer offers services only to the layer above and uses only the layer below. Hence (1) the UI can be replaced (desktop → web → mobile) without touching borrowing rules; (2) the DBMS can be changed by modifying one layer; (3) security (authentication) is enforced in one place before any business function is reached; (4) layers can be developed and tested incrementally by different teams. The cost is some performance overhead because a request passes through every layer.

(b) Library management system – client–server pattern



Justification. The library's functionality is naturally a set of *services* (catalogue, circulation, digital content, authentication) used by many *distributed clients* at the counter, in reading rooms and at home. Client–server lets (1) the shared data live on servers so every client sees one consistent catalogue and loan record; (2) new clients (e.g. a mobile app) be added without changing servers; (3) busy servers (catalogue search) be replicated independently. Its weaknesses – each server is a potential single point of failure and performance depends on the network – are handled by replication and caching.

Answer. (a) Architecture = components + their relationships + their externally visible properties; the hall system is a 4-layer design with one service per feature, a payment-adapter to bank/card gateway and shared data stores. (b) Layered: UI / UI-management & security / business logic / system support; client–server: many clients ↔ network ↔ catalogue, circulation, content and authentication servers.

2016–2017 • Q3(c) CSE307 • 15/07/2017 • A012 MARKS 16 $\frac{2}{3}$

The following code takes a user choice from the user. If the choice is 1, it increments the value of x which is initialized as 5. If the choice is anything else, value for x is decremented. Now rewrite this code so that the code follows the MVC principle.

```
package withoutmvc;
import java.util.Scanner;

public class WithoutMVC {

    public static void main(String[] args) {
        int choice;
        int x=5;
        System.out.println("Enter your choice: ");
        Scanner sc=new Scanner(System.in);
        choice=sc.nextInt();
        if(choice==1) x = incx(x);
        else x = decx(x);
        System.out.println("New value: "+x);
    }
    public static int incx(int x) {
        x++;
        return x;
    }
    public static int decx(int x) {
        x--;
        return x;
    }
}
```

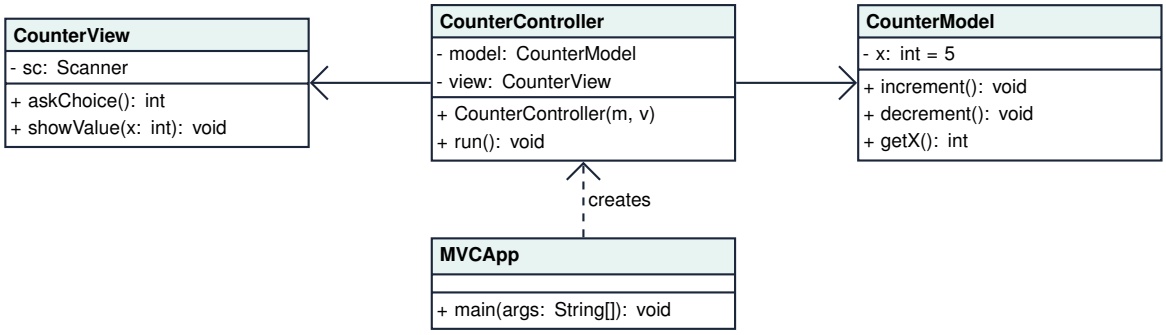
SOLUTION

What is wrong with the original code

`main()` mixes three responsibilities: it *stores state* (`x`), *reads/writes the console* (`Scanner`, `println`) and *decides what to do* (`if (choice==1)`). MVC separates them:

- **Model** (`CounterModel`) – holds `x` (initially 5) and the operations `increment()`/`decrement()`; no I/O.
- **View** (`CounterView`) – all console input/output: prompt, read the choice, show the new value; no logic.
- **Controller** (`CounterController`) – takes the user's choice from the view, applies the rule "1 ⇒ increment, otherwise decrement" to the model, and asks the view to display the result.

Class diagram



Rewritten code (MVC)

```
package withmvc;
import java.util.Scanner;

/* ---------- MODEL: data + business operations, no I/O ---------- */
class CounterModel {
    private int x = 5;
    public void increment() { x++; }
    public void decrement() { x--; }
    public int getX()       { return x; }
}

/* ---------- VIEW: only input/output, no decisions ---------- */
class CounterView {
    private final Scanner sc = new Scanner(System.in);
    public int askChoice() {
        System.out.println("Enter your choice: ");
        return sc.nextInt();
    }
    public void showValue(int x) {
        System.out.println("New value: " + x);
    }
}

/* ---------- CONTROLLER: connects view input to model updates ---------- */
class CounterController {
    private final CounterModel model;
    private final CounterView view;
    public CounterController(CounterModel model, CounterView view) {
        this.model = model;
        this.view = view;
    }
    public void run() {
        int choice = view.askChoice();          // user input via view
        if (choice == 1) model.increment();     // update model
        else             model.decrement();
        view.showValue(model.getX());           // refresh view
    }
}

/* ---------- Application entry point ---------- */
public class MVCApp {
    public static void main(String[] args) {
        CounterModel model = new CounterModel();
        CounterView view = new CounterView();
        new CounterController(model, view).run();
```

```
    }
}
```

Sample run: input `1` → "New value: 6"; input `3` → "New value: 4" – identical behaviour to the original program.

Why this is better

- **Single responsibility** – each class has one reason to change: a GUI view (Swing/JavaFX) can replace `CounterView` without touching model or rule.
- **Testability** – `CounterModel` can be unit-tested without a console.
- **Reuse** – the same model can be shown by several views.

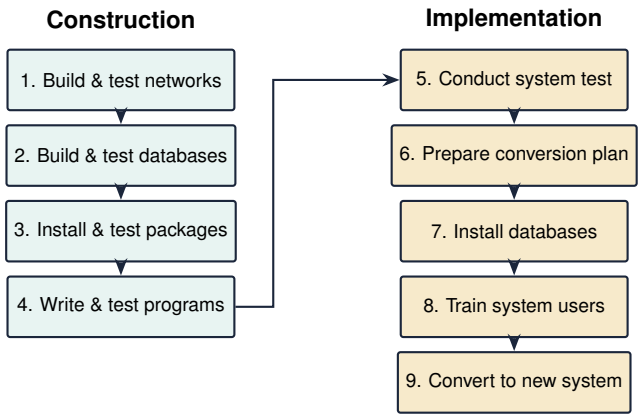
Answer. Split the program into `CounterModel` (x = 5, increment/decrement), `CounterView` (prompt, read choice, print value) and `CounterController` (choice 1 → increment, else decrement, then refresh view); `main` only wires them together.

2016–2017 • Q4(b) CSE307 • 15/07/2017 • A013 MARKS 8 $\frac{2}{3}$

What are the steps of system construction and implementation? Mention only the steps. You do not need to explain them.

SOLUTION

Following Whitten & Bentley (*Systems Analysis and Design Methods*), once the physical design is approved the system goes through a **construction phase** and an **implementation (delivery) phase**:



(i) Build and test networks (if necessary)

(ii) Build and test databases

(iii) Install and test new software packages (if necessary)

(iv) Write and test new programs

(v) Conduct system test

(vi) Prepare conversion plan

(vii) Install databases

(viii) Train system users

(ix) Convert to the new system

Answer. Construction: build/test networks, build/test databases, install/test packages, write/test programs. Implementation: system test, conversion plan, install databases, train users, convert to the new system.

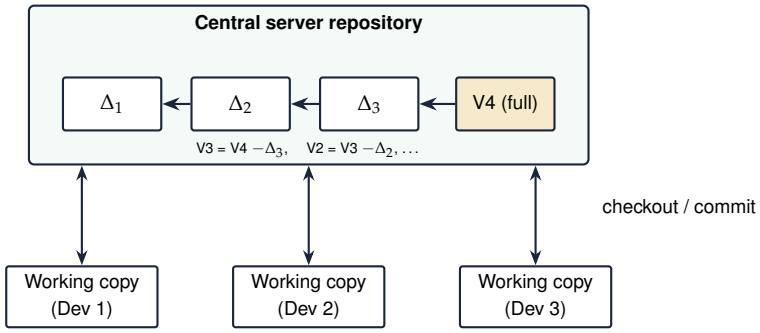
2016–2017 • Q7(c) CSE307 • 15/07/2017 • A014 **MARKS 10**

Describe how distributed VCS and centralized VCS store all the different versions of a repository? Draw necessary diagrams to describe your answer.

SOLUTION

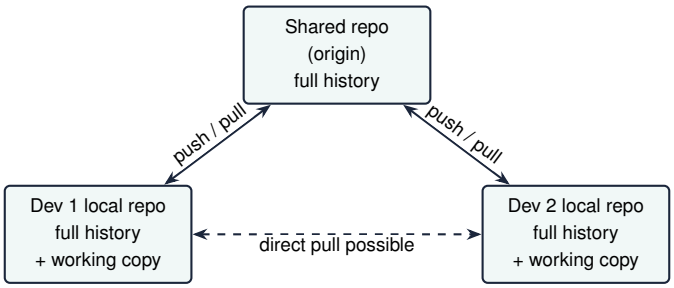
Centralised VCS (CVS, Subversion)

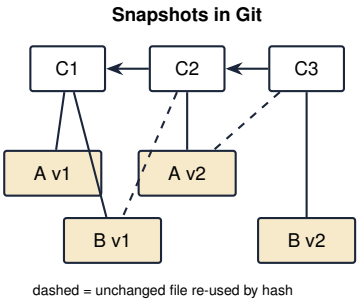
- There is **one master repository on a server** that stores every version of every file. Developers have only a *working copy* of one version; to see history or commit they must contact the server.
- To save space the repository stores versions as **deltas**: the *latest* version is stored in full and earlier versions as *reverse deltas* (lists of differences). An old version is reconstructed by applying the deltas backwards from the newest version (the most frequently requested version is the cheapest to retrieve).



Distributed VCS (Git, Mercurial)

- Every developer **clones the complete repository** – all versions and the full history – to the local machine. Commits, history, diffs and branching are local operations; repositories synchronise by **push** and **pull** (usually through an agreed shared repository such as GitHub, but any copy can serve).
- Git stores versions as **snapshots**, not per-file deltas: each commit points to a *tree* (directory listing) whose entries point to *blobs* (file contents), all named by the SHA hash of their content. An unchanged file is simply referenced again (same hash), so it is not duplicated; packfiles later compress similar objects with deltas.





Comparison

	Centralised VCS	Distributed VCS
Where history lives	only on the central server	in every clone
Storage form	latest version + reverse deltas per file	snapshots of the whole tree (content-addressed, compressed)
Offline work	no commits/history offline	full functionality offline
Single point of failure	yes (server)	no – every clone is a backup
Speed of history ops	network-bound	local, fast

Answer. A CVCS keeps one server repository storing the newest version fully and older versions as reverse deltas, while clients hold only a working copy; a DVCS gives every developer a full clone of the repository (Git: content-addressed commit snapshots sharing unchanged blobs), synchronised by push/pull.

2016–2017 • Q8(a), (b) CSE307 • 15/07/2017 • A015 MARKS (3×5=15) + (4+4+7=15)

(a) Differentiate between the following pairs:

(i) fork vs git clone

(ii) git pull vs git fetch

(iii) git branch vs git checkout

(iv) master vs HEAD

(v) Centralized VCS vs Distributed VCS

(b) Draw two diagrams depicting the monolithitic and microservice architectural design of a taxi-hailing application project. (Make appropriate assumptions about the functionalities and features of the applications.) Describe which drawbacks of the monolithic architecture are solved by the microservice architecture.

SOLUTION

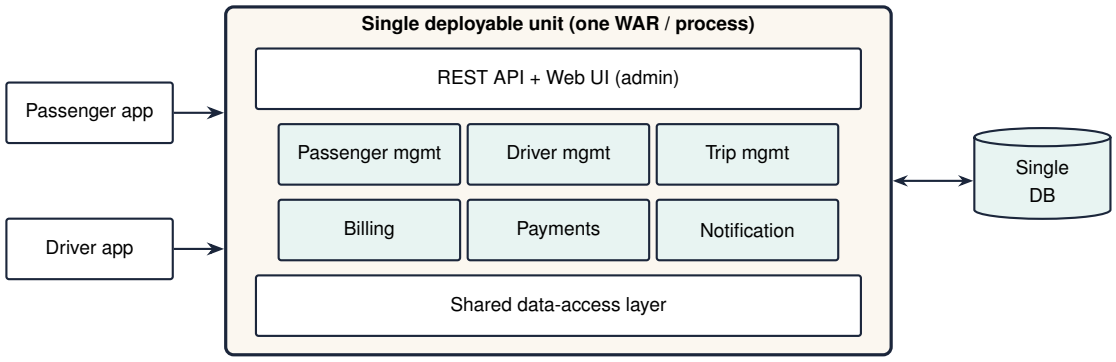
(a) Git / VCS pairs

Pair	First	Second
(i) fork vs clone	**Fork**: a server-side copy of someone else's repository under *your* account (GitHub/GitLab feature, not a git command); used to contribute via pull requests without write access to the original.	**`git clone`**: copies a repository (any remote, incl. your fork) to your *local machine* with full history and sets `origin`.
(ii) pull vs fetch	**`git pull`** = `fetch` + `merge` (or rebase): downloads new commits *and* integrates them into the current branch and working copy.	**`git fetch`**: only downloads new commits/refs into remote-tracking branches (`origin/main`); working copy unchanged – safe to inspect first.
(iii) branch vs checkout	**`git branch name`**: creates (lists/deletes) a branch pointer at the current commit; you stay on the old branch.	**`git checkout name`**: switches HEAD and the working copy to that branch/commit (`checkout -b` creates and switches).
(iv) master vs HEAD	**master** (main): the conventional name of the default branch – a movable pointer to the latest commit of that line.	**HEAD**: a pointer to *what is currently checked out* (usually a branch name, e.g. HEAD → master; "detached" if pointing to a commit).
(v) CVCS vs DVCS	**Centralised**: one server repository; clients have working copies; commits need the network; server is a single point of failure (SVN).	**Distributed**: each clone is a full repository; local commits, offline work, push/pull between peers; fast and resilient (Git).

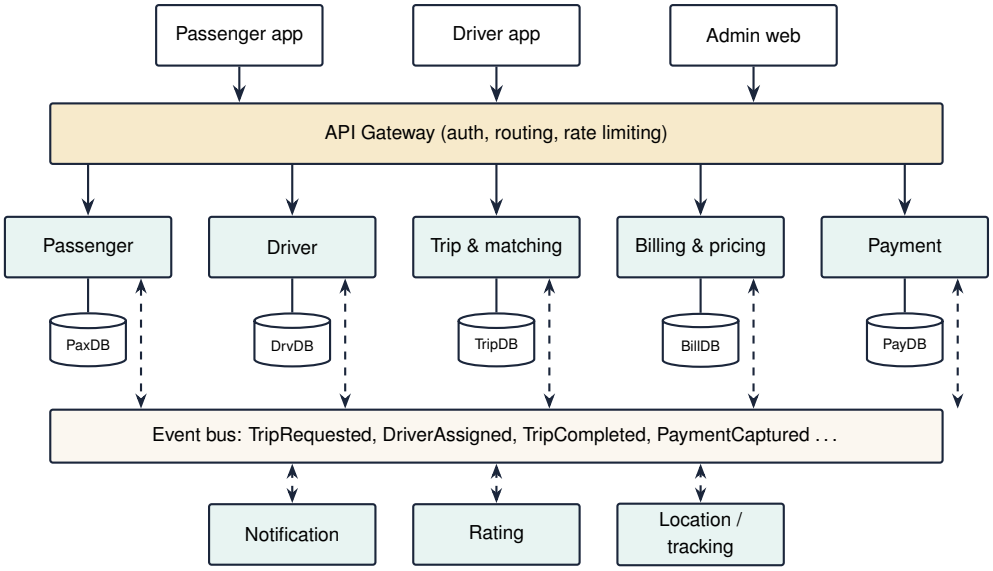
(b) Assumed features of the taxi-hailing app

Passenger management (sign-up, profile), driver management (documents, availability), trip management (request, matching nearest driver, live tracking), pricing/billing (fare, surge), payments (card, mobile wallet), notifications (SMS/push/e-mail), ratings, and an admin dashboard.

(b) Monolithic architecture



(b) Microservice architecture



(b) Monolith drawbacks solved by microservices

Monolith drawback	How microservices solve it
Huge, tangled code base hard to understand	small services, one business capability each
Whole app redeployed for any change; slow, risky releases	each service deployed independently, often several times a day
Can only scale the whole app (e.g. matching needs 20 instances, billing 2)	scale only the hot services (trip matching, tracking)
One bug/memory leak can crash everything	fault isolation; other services keep working, failures contained by timeouts/circuit breakers
Locked into one technology stack	each service may use the best language/DB (e.g. geo-DB for tracking)
Large teams block each other; long build/start-up times	small autonomous teams own services; fast builds and start-up

Answer. (a) Fork = server-side copy under your account vs clone = local full copy; pull = fetch + merge vs fetch = download only; branch creates a pointer vs checkout switches HEAD/working copy; master = default branch pointer vs HEAD = currently checked-out ref; CVCS = single server repo vs DVCS = full repo in every clone. (b) Monolith = one deployable with all modules and one DB; microservices = gateway + independent services with own DBs and an event bus, solving complexity, deployment, scaling, reliability, technology lock-in and team-coordination problems.

2014–2015 • Q4(e) CSE307 • 04/02/2016 • A016 **MARKS 10**

You are the architect for the software system that controls operation of a microwave oven. Which type of architecture will be most suitable? Explain with proper reasoning.

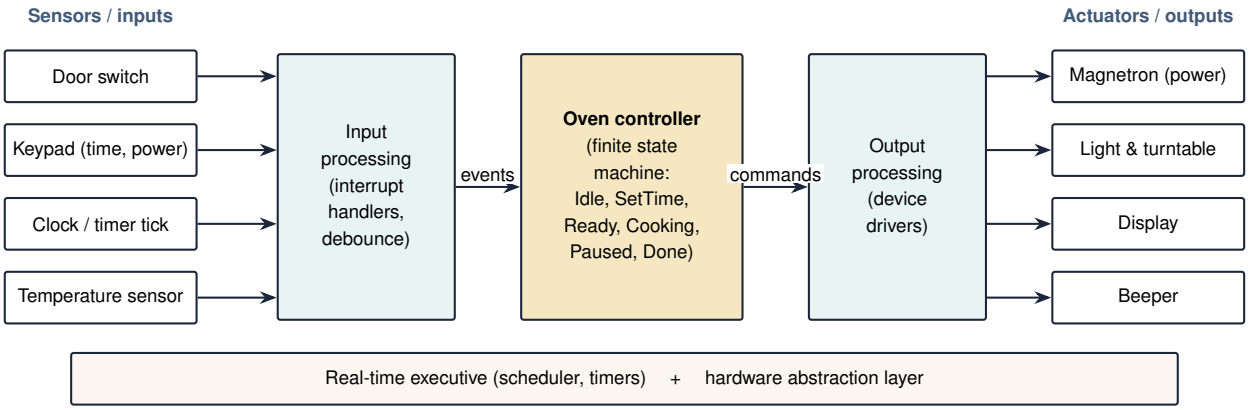
SOLUTION

Nature of the system

A microwave controller is an **embedded, real-time, reactive** system: it does not process large volumes of data, it *reacts to events* (door opened/closed, keypad presses, timer expiry, over-temperature) and drives actuators (magnetron, light, turntable motor, display, beeper) within strict time limits, and it has hard *safety* requirements (the magnetron must never be on while the door is open).

Most suitable architecture

The best fit is an **event-driven, state-machine based process-control architecture** (Sommerville's *environmental/process control* or *sense–compute–control* pattern), running on a small *real-time executive*, with a thin *hardware-abstraction layer* below it:



Reasoning

(i) **Reactive behaviour ⇒ state machine.** The oven's behaviour depends on its current mode (the same "Start" key does different things in *Idle* and *Paused*); a finite state machine makes every legal transition explicit and testable.

(ii) **Real-time constraints.** Sensing and actuation are periodic/interrupt driven; a sense–compute–control loop with a real-time executive guarantees bounded response time (e.g. magnetron off within a few milliseconds of the door opening).

(iii) **Safety.** The safety interlock (door open ⇒ power off) is localised in the controller and can be duplicated in hardware; the architecture keeps critical logic small and verifiable.

(iv) **Portability & maintainability.** The HAL isolates the controller from the specific chips, so the same software runs on different oven models.

(v) **Why not others?** Client–server, repository or pipe-and-filter add communication or data-sharing overhead that a small single-processor device does not need, and they do not model event-driven control naturally.

Answer. Use an event-driven real-time *process-control (sense–compute–control)* architecture whose core is a finite-state-machine controller over a real-time executive and a hardware abstraction layer – it matches the reactive, time-critical and safety-critical nature of the oven.

2013–2014 • Q4(b) CSE307 • 08/08/2015 • A018 **MARKS 12**

Explain the architecture of software system for the following case:

(i) A billing software for Titas Gas Transmission and Distribution Company Ltd

(ii) An embedded system software that controls a microwave woven.

(iii) Software for Network Interface Card

(iv) MATLAB software.

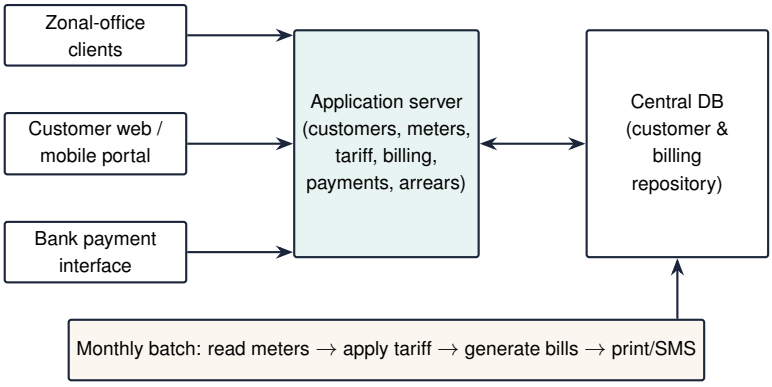
SOLUTION

Approach

For each case we identify the dominant quality requirements and choose the architectural style (pattern) that best satisfies them.

(i) Gas billing software – multi-tier client–server with a central repository and batch processing

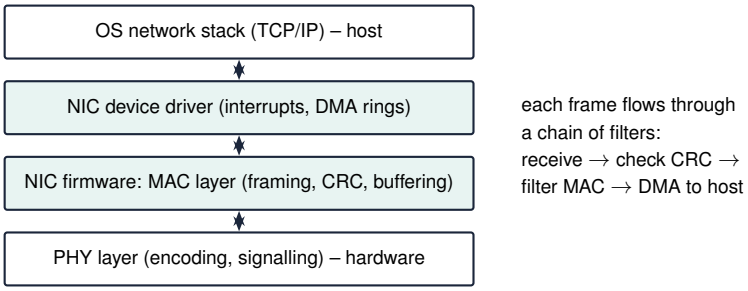
Titas Gas has lakhs of customers, monthly bill generation, payments through many banks, and many offices that need the same customer data. Hence a **3-tier client–server** system around a **central database (repository)**; bill generation is a **batch (pipe-and-filter) job** run each month.



(ii) Embedded microwave-oven controller – real-time process-control (state machine)

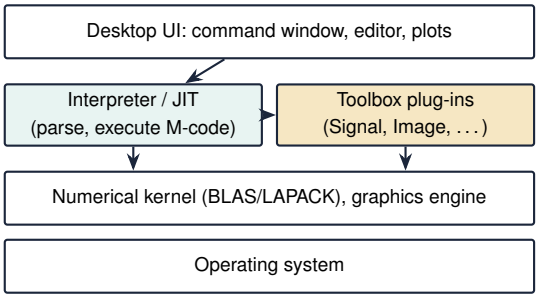
Reactive, time-critical and safety-critical: sensors (door, keypad, timer) → input handlers → **state-machine controller** → actuator drivers (magnetron, light, turntable, display), over a real-time executive (see the diagram in A016). No database, no network; small footprint.

(iii) Network Interface Card software – layered protocol stack (with pipe-and-filter data path)



Each layer hides the hardware details from the layer above, exactly as in the OSI/TCP-IP model; the per-frame processing inside firmware is a **pipe-and-filter** chain, and the driver is **interrupt (event) driven** for low latency.

(iv) MATLAB – interpreter (virtual machine) with plug-in toolboxes over a layered core



MATLAB is an interactive **interpreter** (users type commands/scripts executed by a virtual machine), the toolboxes follow a **plug-in (microkernel)** style – they extend the core without changing it – and the whole is **layered** over highly optimised numerical libraries.

System	Architecture	Key driver
Gas billing	3-tier client–server + repository + batch	many users, shared data, monthly volume
Microwave	real-time control / state machine	reactivity, timing, safety
NIC software	layered stack + pipe-and-filter + interrupts	hardware abstraction, throughput
MATLAB	interpreter + plug-ins over layers	interactivity, extensibility, numeric speed

Answer. (i) multi-tier client–server around a central billing repository with batch bill generation; (ii) real-time sense–compute–control state machine; (iii) layered protocol stack with a pipe-and-filter frame path; (iv) interpreter/virtual-machine with plug-in toolboxes on layered numerical libraries.

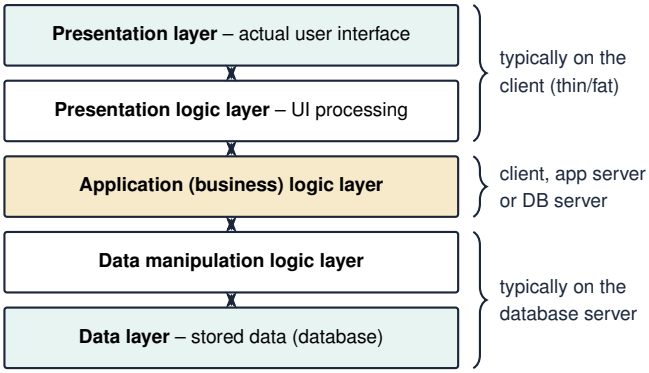
2013–2014 • Q6(b) CSE307 • 08/08/2015 • A019 **MARKS 10**

Briefly describe different computing layers with respect to application architecture.

SOLUTION

Idea

Every information-system application, whatever its physical deployment, can be split into **five logical computing layers** (Whitten & Bentley). Application architecture then decides *on which machine* (client or server) each layer runs.



The five layers

(i) **Presentation layer** – the actual user interface: screens, forms, reports shown to the user and the input devices used to capture data (e.g. the BIIS web page in the browser).

(ii) **Presentation logic layer** – processing that *controls* the interface: navigation between screens, input editing/validation ("roll number must have 7 digits"), formatting of output.

(iii) **Application logic layer** – the business rules and policies: calculating CGPA, checking prerequisites, computing a bill. It is the heart of the application.

(iv) **Data manipulation logic layer** – commands and logic that store and retrieve data: SQL statements, stored procedures, transaction control, integrity checks.

(v) **Data layer** – the stored data itself in files and databases.

How the layers are distributed (architecture styles)

Architecture	Placement of layers
Centralised (mainframe)	all five layers on one host; terminals only display
File-server	presentation + presentation logic + application logic + data manipulation on PCs; only the data file on server
2-tier client–server (fat client)	presentation, presentation logic and application logic on client; data manipulation + data on DB server
3-tier / n-tier	presentation (+ logic) on thin client, application logic on application/web server, data manipulation + data on DB server

Answer. Presentation, presentation logic, application (business) logic, data manipulation logic and data layers; architecture styles differ only in which tier/machine hosts each layer.

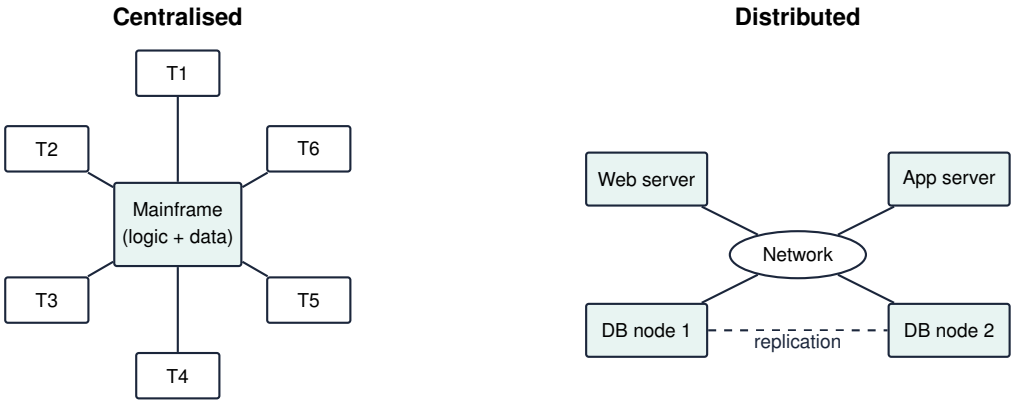
2013–2014 • Q7(c) CSE307 • 08/08/2015 • A020 MARKS 10

Distinguish between distributed and centralized architecture. What are the advantages of distributed architecture?

SOLUTION

Definitions

- **Centralised architecture:** all processing (and usually all data) resides on *one* computer (mainframe or single server); users access it through terminals or thin clients.
- **Distributed architecture:** the system is a collection of *independent computers* connected by a network that appears to users as a single coherent system; processing and/or data are spread over several nodes that cooperate by exchanging messages (Sommerville).



Distinction

Aspect	Centralised	Distributed
Location of processing/data	one host	many networked nodes
Failure behaviour	single point of failure	partial failure; others continue
Scalability	vertical only (bigger machine)	horizontal (add nodes)
Consistency & control	simple, one copy of data	harder: replication, concurrency
Security	one perimeter to protect	many nodes and network links to protect
Communication	local calls, fast	network messages, latency
Cost	expensive host	commodity machines; more management
Typical example	legacy bank mainframe	internet banking, BUET BIIS, e-commerce

Advantages of distributed architecture (Sommerville)

(i) **Resource sharing** – hardware, software and data on different machines can be shared.

(ii) **Openness** – built from standard protocols, so equipment and software from different vendors can be combined.

(iii) **Concurrency** – several processes run at the same time on different nodes, increasing throughput.

(iv) **Scalability** – capacity is increased by adding new resources (scale-out).

(v) **Fault tolerance / availability** – information and processing can be replicated, so the failure of one node does not stop the whole system.

(vi) **Geographic proximity** – processing can be placed near the users (lower latency).

EXAM TIP Also mention the price: complexity, security, manageability and unpredictable network response time are the disadvantages of distributed systems.

Answer. Centralised = one host does everything; distributed = cooperating networked nodes appearing as one system. Advantages of distribution: resource sharing, openness, concurrency, scalability and fault tolerance.

2012–2013 • Q6(c) CSE307 • 17/05/2014 • A022 **MARKS 10**

What are the drawbacks of two-tier client-server architecture? How can the problem be solved?

SOLUTION

Two-tier client–server

In a **two-tier** (fat-client) model the client runs the user interface *and* the application (business) logic; it talks directly to a database server, which holds the data and data-manipulation logic (often stored procedures).

Drawbacks

(i) **Poor scalability** – every client keeps its own connection to the DBMS; with hundreds or thousands of users the database is overloaded by connections and sessions.

(ii) **Hard deployment and maintenance** – business logic lives in the client, so every rule change means reinstalling/upgrading software on every PC ("fat client" problem).

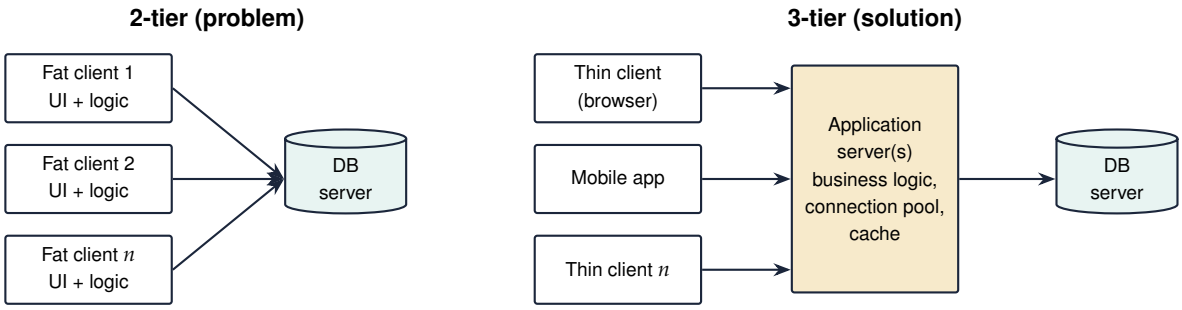
(iii) **Business logic duplicated or locked in the DB** – either copied into many clients (risk of inconsistency) or written as vendor-specific stored procedures (vendor lock-in, hard to test).

(iv) **High network traffic** – raw rows travel to the client to be processed there.

(v) **Security** – the database is exposed directly to clients; database credentials sit on every client machine.

(vi) **Limited reuse & heterogeneity** – different client types (web, mobile) cannot reuse the same logic; no place for load balancing or caching.

Solution: three-tier / n-tier architecture



- Move the business logic into a **middle tier** (application server / web services). Clients become *thin*: only presentation, so a rule change is deployed once on the server.
- The middle tier uses a **connection pool** – thousands of clients share a few dozen DB connections – and can be **replicated behind a load balancer** and use **caching**.

- The DB is no longer exposed: only the application server is allowed to connect (**better security**); logic is written once and reused by web, mobile and other systems.
- For very large systems extend to ***n*-tier** (web tier, application tier, integration tier, data tier) or service-oriented / microservice architectures; middleware (TP monitors, message queues) can be added between tiers.

Answer. Two-tier systems suffer from poor scalability, fat-client deployment/maintenance problems, duplicated or DB-locked logic, high traffic and weak security; they are solved by introducing a middle application tier (3-tier/n-tier) with thin clients, connection pooling, load balancing and caching.

2011–2012 • Q3(c), (d) CSE307 • 23/09/2013 • A023 MARKS 16 + 8

(c) What are the different phases of constructing a new system? Describe them.
(d) What are the different installation strategies while implementing a new system?

SOLUTION

NOTE "Phases of constructing a new system" is interpreted, as in the course text of that period (Whitten & Bentley), as the *construction* and *implementation* phases with their activities (listed in A013); each activity is described below.

(c) Construction phase – build and test the system's components

(i) **Build and test networks (if necessary).** Network engineers install/upgrade LAN/WAN, servers and connectivity specified in the design, and test them (throughput, security) because databases and programs depend on them.

(ii) **Build and test databases.** The database administrator implements the physical schema (tables, keys, indexes, views, access rights) and loads it with *test data*; system builders test it for integrity and performance.

(iii) **Install and test new software packages (if necessary).** Purchased packages (e.g. accounting, ERP modules) are installed, configured and tested to check that they meet the requirements and interface with the rest of the system.

(iv) **Write and test new programs.** Programmers code the custom programs according to the design, then perform *unit testing* (each module alone) and *integration testing* (modules working together); code is reviewed and documented.

(c) Implementation (delivery) phase – deliver the system into operation

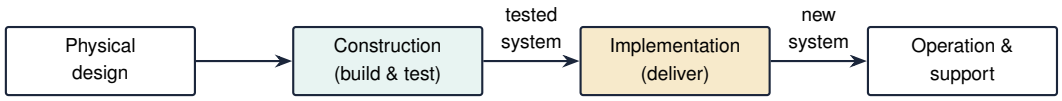
(v) **Conduct system test.** The whole system (networks, databases, packages, programs) is tested together against requirements: functional, performance, stress, security, recovery tests; ends with user *acceptance testing*.

(vi) **Prepare conversion plan.** Decide the installation strategy (see (d)), schedule, data conversion, responsibilities, and acceptance criteria; get management approval.

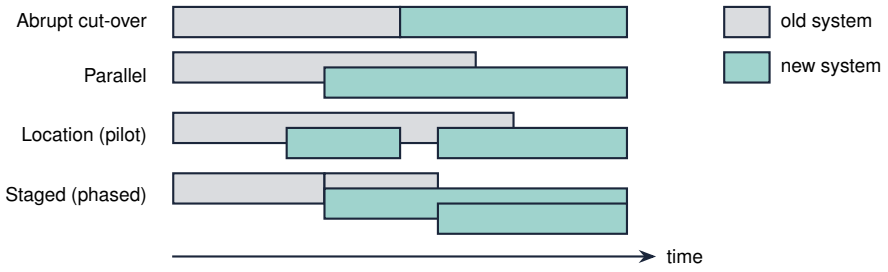
(vii) **Install databases.** Populate the production database with real data – converting and cleansing data from the old system's files/databases.

(viii) **Train system users.** Provide manuals, hands-on training and help-desk support for end-users, managers and operators.

(ix) **Convert to the new system.** Execute the conversion plan, hand over operation and documentation, and conduct a post-implementation review.



(d) Installation (conversion) strategies



(i) **Abrupt cut-over (direct / plunge):** on a chosen date the old system is switched off and the new one used immediately. Cheapest and simplest, but highest risk – no fallback.

(ii) **Parallel conversion:** old and new systems run simultaneously for a period; outputs are compared until the new system is trusted. Lowest risk, highest cost/effort (double data entry).

(iii) **Location conversion (pilot):** the system is installed at one site/branch first; after it works there it is rolled out to other locations. Limits risk to one site and the pilot users help others.

(iv) **Staged conversion (phased):** the new system is introduced module by module (or version by version); each stage is a smaller abrupt or parallel conversion. Gradual learning and lower risk, but longer, and old and new parts must interface temporarily.

Answer. (c) Construction – build/test networks, databases, packages and programs; Implementation – system test, conversion plan, install production databases, train users, convert. (d) Abrupt cut-over, parallel, location (pilot) and staged (phased) conversion.

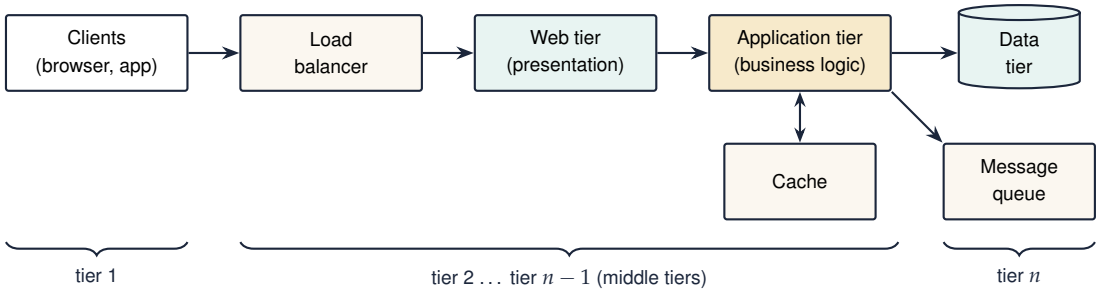
2011–2012 • Q4(b) CSE307 • 23/09/2013 • A024 **MARKS 12**

What is three-tiered or n-tiered client/server computing? Discuss its various aspects.

SOLUTION

Definition

Three-tiered client/server computing separates an application into three logically and usually physically separate tiers – *presentation tier* (client), *application/business logic tier* (application server) and *data tier* (database server) – that communicate over a network through well-defined interfaces. When the middle tier is further split (web server, application server, integration/messaging server, cache, several DB servers . . .) the architecture is called ***n*-tiered**.



Various aspects

(i) **Separation of concerns / maintainability.** UI, business rules and data can be changed independently; a business rule is modified in one place (the middle tier).

(ii) **Thin clients & easy deployment.** Clients need only a browser or a small app; no per-PC installation of business logic.

(iii) **Scalability & performance.** Each tier is scaled separately: add web/app servers behind a load balancer, add read replicas for the DB; connection pooling and caching in the middle tier reduce DB load. The cost is extra network hops (latency) per request.

(iv) **Availability / reliability.** Redundant servers in each tier remove single points of failure; failed nodes are bypassed by the load balancer.

(v) **Security.** Tiers sit in separate network zones (DMZ for the web tier, private network for DB); clients never touch the database directly.

(vi) **Reuse & interoperability.** The middle tier exposes services (APIs) used by web, mobile and partner systems; legacy systems can be wrapped.

(vii) **Technology independence.** Each tier can use the most suitable platform (e.g. React UI, Java services, PostgreSQL) and be replaced without affecting the others.

(viii) **Cost and complexity.** More servers, middleware, monitoring and skills are required; distributed transactions, consistency and debugging become harder.

Answer. 3-tier/n-tier computing splits an application into presentation, business-logic and data tiers (with possibly several middle tiers) on separate servers; it brings maintainability, thin clients, independent scalability, availability, security and reuse, at the price of more complexity, cost and network latency.

X8

Quality Management & Metrics

OUT OF SYLLABUS

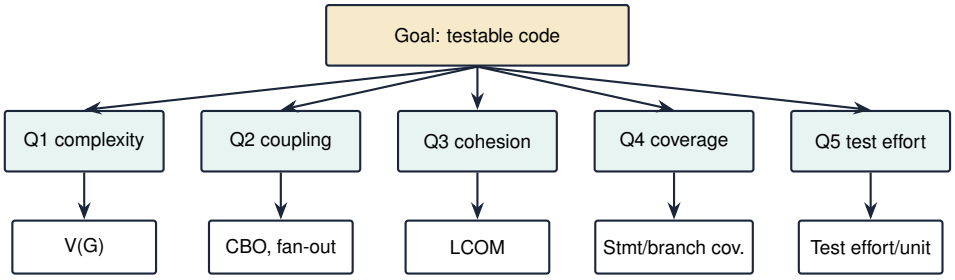
Jan 2021 • Q3(a) CSE307 • 04/09/2021 • F014 MARKS 20

The main goal of a software engineer is to make the developed code testable. Define this goal using goal definition template. Write down the questions that must be answered in order to achieve this goal. Identify the metrics that help answer these questions with formula of measurement and necessary data collections. Discuss two of the metrics identified based on the general attributes of the metrics.

SOLUTION

Goal–Question–Metric (GQM) approach

GQM (Basili) derives measurements top-down: (1) state a measurable **goal**, (2) refine it into **questions** whose answers tell whether the goal is met, (3) define **metrics** that answer each question.



1. Goal definition template

Analyze	the source code (classes and methods) produced by the development team
for the purpose of	evaluating and improving
with respect to	*testability* – ease with which units can be isolated, controlled, observed and fully exercised by tests
from the viewpoint of	the software engineer (developers and testers)
in the context of	the ongoing development of the project (each sprint/build)

2. Questions

Q1. How complex is the control flow of each method, i.e. how many test cases are needed to test it?

Q2. How strongly does each class depend on other classes – can it be tested in isolation?

Q3. How cohesive is each class (does it do one thing that can be tested as a unit)?

Q4. How much of the code is actually exercised by the automated tests?

Q5. How much effort does it take to write and run tests for a unit?

3. Metrics, formulas and data collection

Metric (question)	Formula	Data to collect
Cyclomatic complexity (Q1)	$V(G) = E - N + 2P$ = (number of binary decisions) $+1$; target $V(G) \leq 10$	control-flow graph of every method (static analyser)
Coupling between objects, fan-out (Q2)	CBO(c) = number of other classes c uses or is used by; fan-out = number of called modules	dependency/call graph from the code
Lack of cohesion LCOM (Q3)	LCOM $= \lvert P\rvert - \lvert Q\rvert$ if positive, else 0 (P: method pairs sharing no attribute, Q: pairs sharing one)	which attributes each method reads/writes
Statement / branch coverage (Q4)	statement cov. = (executed statements / total statements) $\times$ 100%; branch cov. = (executed branches / total branches) $\times$ 100%	coverage tool (JaCoCo) output on each CI test run
Test effort per unit (Q5)	person-hours writing tests / units tested; test-to-code ratio = test LOC / production LOC; mocks needed per test	time sheets / issue tracker, LOC counter, test code inspection

Worked example. A method containing 3 `if` statements and 1 `while` loop has $V(G) = 4 + 1 = 5$, so at least 5 independent paths (test cases) are needed for basis-path coverage. If the tests run 170 of its 200 statements and 34 of 40 branches: statement coverage $= 170/200 = 85\%$, branch coverage $= 34/40 = 85\%$.

4. Discussing two metrics against the general attributes of effective metrics

Pressman lists the attributes of an effective software metric: *simple and computable*; *empirically and intuitively persuasive*; *consistent and objective*; *consistent in its use of units and dimensions*; *programming-language independent*; *an effective mechanism for quality feedback*.

Attribute	Cyclomatic complexity $V(G)$	Branch coverage
Simple & computable	yes – count decisions, fully automated	yes – produced by coverage tools on every run
Empirically / intuitively persuasive	more paths $\Rightarrow$ more tests and more defects; matches intuition	untested branches obviously cannot reveal faults
Consistent & objective	same code gives same value for everyone	deterministic for a given test suite
Units / dimensions	a dimensionless count of paths	a percentage (0–100%)
Language independent	defined on the flow graph, so applies to any procedural/OO language	applies to any language with a coverage tool
Quality feedback	flags methods with $V(G) > 10$ for refactoring *before* testing	shows exactly which branches need new tests; can gate CI builds (e.g. $\geq 80\%$)

Both satisfy the attributes well; their limitation is that $V(G)$ ignores data complexity and coverage measures *execution*, not the *quality of assertions*, so they are used together with coupling and test-effort metrics.

Answer. Goal: analyse the source code for the purpose of improving it with respect to testability from the developer's viewpoint in the project context. Questions: complexity, coupling, cohesion, test coverage, test effort. Metrics: $V(G) = E - N + 2P$, CBO/fan-out, LCOM, statement/branch coverage, test effort per unit. $V(G)$ and branch coverage are simple, objective, language-independent and give direct quality feedback.

2018–2019 • Q7(b) CSE307 • 06/11/2019 • M016.1 **MARKS 2 + 5 = 7**

Define Scope Creep. Compute the overall Defect Removal Efficiency of a software team if 300 defects were found and rectified before the delivery and 50 more defects surfaced after the delivery.

SOLUTION

Scope creep

Scope creep is the uncontrolled, gradual growth of a project's scope – new features and requirement changes are added after the scope was agreed, *without* corresponding adjustment of schedule, budget and resources (and often without formal change control). It is a major cause of late, over-budget projects; it is controlled by a clear baseline scope, a change-control board and prioritisation.

Defect Removal Efficiency

$$\text{DRE} = \frac{E}{E + D}$$

E = errors found before delivery, D = defects found after delivery. Here $E = 300$, $D = 50$:

$$\text{DRE} = \frac{300}{300 + 50} = \frac{300}{350} = 0.857 \approx 85.7\%.$$

Answer. Scope creep = uncontrolled growth of scope without matching time/cost changes. DRE $= 300/350 = 0.857$, i.e. the team removed about 85.7% of all defects before delivery.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q1(d) CSE307 • 15/07/2017 • F019 **MARKS 7 $\frac{2}{3}$**

Write short note on "Ishikawa Diagram".

2010–2011 • Q5(b) CSE307 • 19/03/2012 • F027 **MARKS 16 $\frac{2}{3}$**

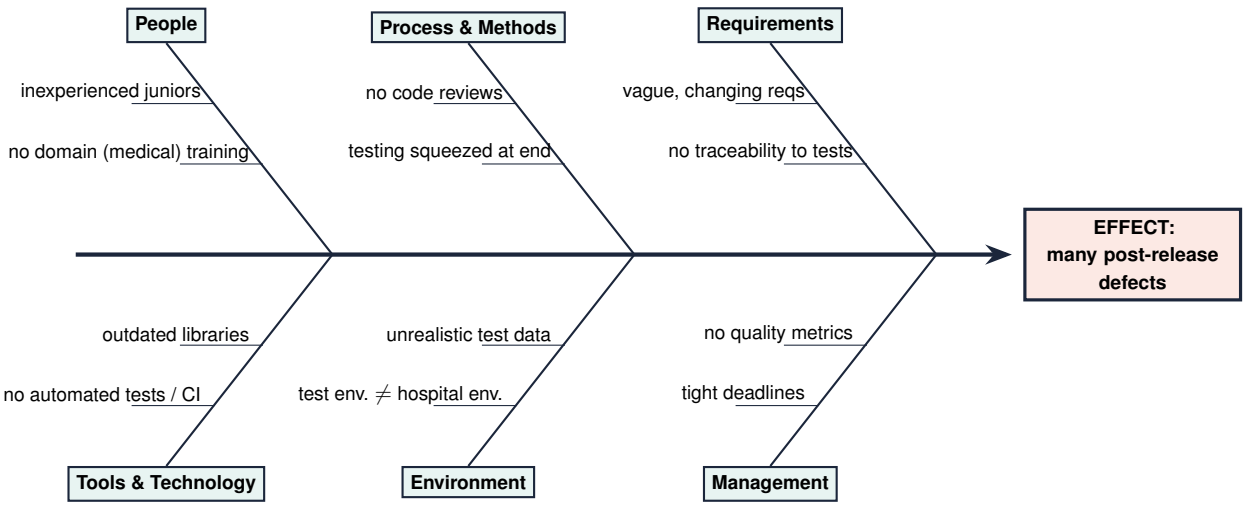
How Ishikawa diagram is used to identify, explore and depict problems, and the causes and the effects of those problems? Describe with an appropriate example.

SOLUTION

What it is

An **Ishikawa diagram** (also *fishbone* or *cause-and-effect* diagram), introduced by Kaoru Ishikawa, is a quality-management tool that graphically relates a **problem (effect)** – written at the "head" of the fish – to all its possible **causes**, grouped into major categories that form the "bones"; sub-causes branch from each bone. It is used in root-cause analysis, defect prevention and process improvement (e.g. in reviews of defect data, CMMI causal analysis).

Example: "Too many defects reported after release" (hospital management software)



How it is used

(i) **Identify the problem (effect).** State the problem precisely and measurably at the head – e.g. "45 defects reported in the first month after release (target ≤ 10)".

(ii) **Choose the major cause categories** (the main bones). Manufacturing uses the 6 M's (Man, Machine, Method, Material, Measurement, Mother-nature); in software typical categories are People, Process/Methods, Tools/Technology, Requirements, Environment and Management.

(iii) **Explore causes – brainstorm.** The team asks "*why does this happen?*" for each category and attaches causes as branches; asking "why?" again (the *5 Whys*) produces sub-causes, until root causes are reached (e.g. defects ← no code reviews ← reviews not planned in schedule ← tight deadlines).

(iv) **Depict** – the completed diagram shows all suspected causes, their grouping and their relationship to the effect on one page, which makes the discussion structured and visible to everyone.

(v) **Analyse and verify** – mark the most likely root causes (team vote), verify them with data (defect logs, Pareto chart of defect origins), and plan corrective actions (introduce code reviews, CI with automated tests, requirement traceability, realistic test environment).

(vi) **Monitor the effect** after the actions to confirm that the problem decreases.

Advantages and limitations

✓ Structured brainstorming that focuses on causes rather than symptoms or blame; easy to understand; encourages team participation; documents the analysis.

× Shows only *possible* causes – they must be verified with data; complex interacting causes are hard to depict; the diagram can become cluttered.

Answer. An Ishikawa (fishbone) diagram puts the effect at the head and groups possible causes on bones (people, process, tools, requirements, environment, management); the team identifies the problem, explores causes by repeated "why?", depicts them in one picture, then verifies root causes with data and acts on them – e.g. "many post-release defects" traced to no reviews, weak testing and unstable requirements.

2016–2017 • Q7(d) CSE307 • 15/07/2017 • T027 MARKS 3 + $3\frac{2}{3} = 6\frac{2}{3}$

Team A found 342 errors during the software engineering process prior to release. Whereas team B found 184 errors. What additional measures would have to be made for projects A and B to determine which of the teams eliminated errors more efficiently? What metrics would you propose to help in making the determination?

SOLUTION

Why the raw counts are not enough

342 vs. 184 tells nothing by itself: project A may be much larger, may have had many more errors in total, or may have spent far more effort. Efficiency of error *elimination* must be judged relative to the errors that *escaped* and to project size and effort.

Additional measures to collect

(i) D – number of **defects found after release** (e.g. during the first 3–6 months of use) for each project.

(ii) **Size** of each product (KLOC or function points).

(iii) **Effort** spent on reviews and testing (person-hours), and the phase in which each error was found.

(iv) **Severity** of the errors and defects.

Proposed metrics

- **Defect Removal Efficiency** DRE $= E/(E+D)$ – the primary measure.
- **Errors per KLOC/FP** before release and **defects per KLOC/FP** after release (normalised quality).
- **Errors found per person-hour** of review/testing (cost-efficiency).
- Phase-wise DRE and severity-weighted defect counts.

Illustration (assumed data). A: 50 KLOC, $D_A = 38$; B: 20 KLOC, $D_B = 8$.

$$\text{DRE}_A = \frac{342}{342+38} = \frac{342}{380} = 0.90, \qquad \text{DRE}_B = \frac{184}{184+8} = \frac{184}{192} = 0.958$$

Post-release defect density: A $= 38/50 = 0.76$/KLOC, B $= 8/20 = 0.40$/KLOC. So team B, despite finding fewer errors, eliminated errors more efficiently.

Answer. Collect post-release defects, product size, review/test effort and severity; compare with DRE $= E/(E+D)$, errors/defects per KLOC (or FP) and errors found per person-hour.

2014–2015 • Q1(a), (b) CSE307 • 04/02/2016 • F022 **MARKS 6 $\frac{2}{3}$ + 8**

(a) What are the quality attributes for software product revision? Define three metrics to measure one of these attributes.
(b) Your goal is to reduce the number of errors in coding. Define three matrices for this goal using goal oriented software measurement.

SOLUTION

(a) Product-revision quality attributes (McCall's quality triangle)

McCall groups quality factors into *product operation*, *product transition* and **product revision**. The product-revision factors describe the ability to *change* the product:

- **Maintainability** – effort required to locate and fix an error in a program.
- **Flexibility** – effort required to modify an operational program.
- **Testability** – effort required to test a program to ensure it performs its intended function.

Three metrics for maintainability:

(i) **Mean Time To Change (MTTC)** – the average time to analyse a change request, design the modification, implement, test it and distribute it to users: MTTC $= \dfrac{\sum \text{time per change}}{\text{number of changes}}$. E.g. 5 changes taking 6, 10, 4, 8, 7 hours give MTTC $= 35/5 = 7$ h. Lower = more maintainable.

(ii) **Software Maturity Index (SMI)** – stability of a product as it is revised:

$$\text{SMI} = \frac{M_T - (F_a + F_c + F_d)}{M_T}$$

M_T = modules in current release, $F_a/F_c/F_d$ = modules added/changed/deleted. With $M_T = 940$, $F_a = 40$, $F_c = 90$, $F_d = 12$: SMI $= (940 - 142)/940 = 0.849$. SMI $\to 1$ means the product is stabilising.

(iii) **Cyclomatic complexity** $V(G) = E - N + 2P$ per module (or the derived *maintainability index* MI $= 171 - 5.2 \ln V - 0.23\, V(G) - 16.2 \ln \text{LOC}$, V = Halstead volume); modules with $V(G) > 10$ are hard to understand and change.

(b) Goal-oriented (GQM) measurement for "reduce coding errors"

Goal: *Analyze* the coding activity *for the purpose of* improvement *with respect to* the number of errors injected in code *from the viewpoint of* the project manager and developers *in the context of* the current project's releases.
Questions: Q1 – How many errors are introduced per unit of code? Q2 – How effectively are coding errors removed before they escape to later phases? Q3 – Which modules/developers inject most errors, and is the trend decreasing?
Three metrics:

(i) **Code defect density** (Q1, Q3):

$$\text{defect density} = \frac{\text{errors traced to coding (found in reviews, unit tests or later)}}{\text{KLOC}}$$

E.g. 96 errors in 24 KLOC ⇒ 4 errors/KLOC; goal: fall below 2 in two releases.

(ii) **Coding-phase DRE** (Q2) $= \dfrac{E}{E+D}$, E = coding errors found during coding (reviews, unit tests), D = coding errors found later. E.g. $E = 80$, $D = 16$: DRE $= 80/96 = 0.83$.

(iii) **Review yield / error-injection rate** (Q2, Q3) – errors found per inspection hour, or errors injected per person-hour of coding $= \dfrac{\text{coding errors}}{\text{coding effort (person-hours)}}$, tracked per module and developer to target training and refactoring.

Answer. (a) Product revision = maintainability, flexibility, testability; maintainability metrics: MTTC, SMI $= [M_T - (F_a + F_c + F_d)]/M_T$, cyclomatic complexity/maintainability index. (b) Goal stated with the GQM template; metrics: defect density per KLOC, coding-phase DRE $= E/(E+D)$, error-injection rate / review yield.

2014–2015 • Q1(e) CSE307 • 04/02/2016 • T028.1 **MARKS 15**

A software development is being done in seven phases. Each of the first four phases generate 25 errors. In the 2nd, 3rd and 4th phase, 40% errors of the previous phase are passed through without amplification and 60% errors of the previous phase are passed with an amplification by the phase number. The final three phases are testing phases without generating any error. The DRE of these three phases are 60% each. In the first four phases, the DRE of the review is defined by 20%×x, where x is the phase number. Find the remaining errors at the end of software development.

SOLUTION

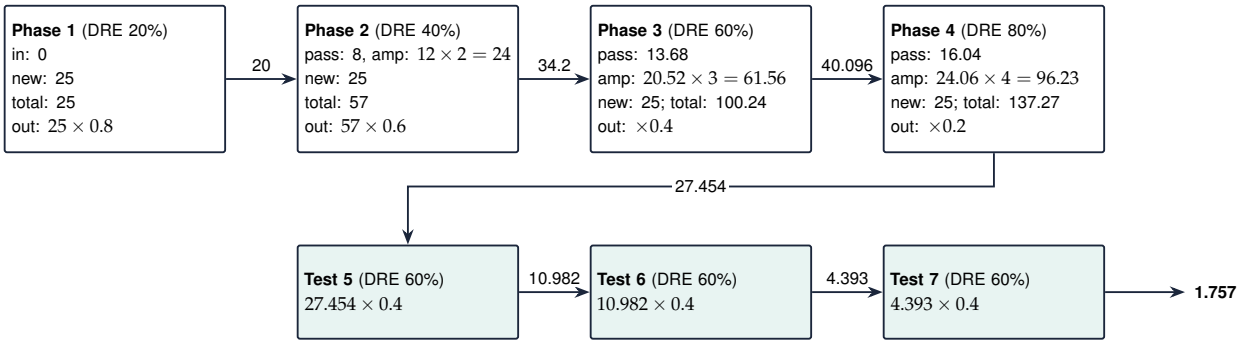
Defect amplification model (Pressman)

For each phase: errors arriving from the previous phase are partly *passed through* unchanged and partly *amplified* (each such error gives rise to x errors); newly generated errors are added; then the phase's review/test removes a fraction equal to its detection efficiency (DRE):

$$\text{Total}_x = 0.4\,O_{x-1} + 0.6\,O_{x-1}\cdot x + N_x, \qquad O_x = \text{Total}_x\,(1 - \text{DRE}_x)$$

with $N_x = 25$ and $\text{DRE}_x = 0.2x$ for $x = 1..4$; $N_x = 0$ and $\text{DRE}_x = 0.6$ for $x = 5..7$.

NOTE "Amplification by the phase number" is taken as amplification factor $1:x$ (each amplified error becomes x errors in phase x). Decimal error counts are kept to the end and only the final answer is rounded.



Step-by-step calculation

x	Passed (40%)	Amplified (60% $\times x$)	New	Total	DRE	Out = Total$(1-$DRE$)$
1	–	–	25	25	20%	$25 \times 0.8 = 20$
2	$0.4(20) = 8$	$0.6(20) \times 2 = 24$	25	57	40%	$57 \times 0.6 = 34.2$
3	$0.4(34.2) = 13.68$	$0.6(34.2) \times 3 = 61.56$	25	100.24	60%	$100.24 \times 0.4 = 40.096$
4	$0.4(40.096) = 16.038$	$0.6(40.096) \times 4 = 96.230$	25	137.269	80%	$137.269 \times 0.2 = 27.454$
5	27.454	–	0	27.454	60%	$27.454 \times 0.4 = 10.982$
6	10.982	–	0	10.982	60%	$10.982 \times 0.4 = 4.393$
7	4.393	–	0	4.393	60%	$4.393 \times 0.4 = 1.757$

Check for the testing phases: $27.454 \times 0.4^3 = 27.454 \times 0.064 = 1.757$.

Interpretation

Only ≈ 1.76 latent errors remain, out of $4 \times 25 = 100$ generated errors plus the many errors created by amplification. The table also shows *why* early reviews matter: the errors that escape phases 2 and 3 are multiplied by 3 and 4 in the following phases.

Answer. Remaining errors at the end of development $\approx 27.454 \times 0.4^3 = 1.757 \approx 2$ errors.

2012–2013 • Q1(e) CSE307 • 17/05/2014 • T030 MARKS 5

At the conclusion of a project that used the Unified Process, it has been determined that 30 errors were found during the elaboration phase and 12 errors were found during construction phase. The error found in the construction phase were traceable to errors that were not discovered in the elaboration phase. But no errors were found in the transition phased. What is the DRE of the elaboration phase?

SOLUTION

Phase DRE

For a phase i, $\text{DRE}_i = \dfrac{E_i}{E_i + E_{i+1}}$, where E_i = errors found in phase i and E_{i+1} = errors found in the next phase that are *traceable* to errors not discovered in phase i.

$$E_{\text{elab}} = 30, \quad E_{\text{constr (traced)}} = 12 \quad \Rightarrow \quad \text{DRE}_{\text{elab}} = \frac{30}{30+12} = \frac{30}{42} = 0.714 \approx 71.4\%.$$

(The transition phase found no errors, so nothing more is traced back.)

Answer. DRE of the elaboration phase $= 30/42 \approx 0.71$ (71.4%).

2011–2012 • Q1(c), (b) CSE307 • 23/09/2013 • F025 MARKS 6 + 8

(c) Discuss the different types of document standards with respect to quality management.
(b) What is process based quality? Describe process based quality briefly.

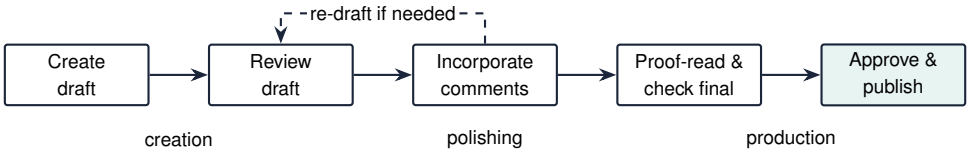
SOLUTION

> **NOTE** The second part is printed as "(b)" after part (c) in the original paper (it is most likely part (d)); the printed labels are kept.

(c) Document standards

Documents are the only tangible representation of software and its process, so quality management defines standards for them (Sommerville). There are three types:

(i) **Documentation process standards** – define the process to be followed for producing a document: how it is drafted, reviewed, revised, approved and published (e.g. every design document must pass a peer review and be signed off by the project manager).



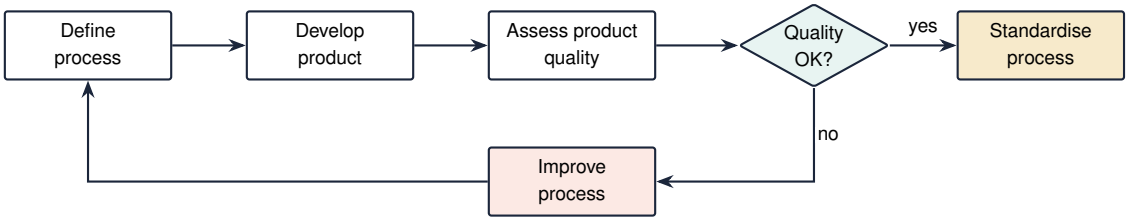
(ii) **Document standards** – govern the documents themselves:

- *identification standards* – how every document is uniquely identified (project code, document type, version, e.g. `URMS/DES/SDD/v2.1`);
- *structure standards* – the standard structure/sections (title page, change history, contents, introduction, . . .);
- *presentation standards* – fonts, styles, logos, headings, use of colour;
- *update standards* – how changes from earlier versions are reflected (change bars, revision history).

(iii) **Document interchange standards** – ensure that electronic documents can be exchanged and reused across tools and organisations: agreed file formats (PDF, XML, Markdown), templates and style sheets, conventions for macros and embedded drawings.

(b) Process-based quality

Process-based quality is the view (from manufacturing) that *the quality of a product is largely determined by the quality of the process used to develop it*. Therefore, instead of only inspecting the final product, an organisation defines, standardises and continuously improves its development process.



- The cycle: define the process → develop the product → assess product quality → if not good enough, *improve the process* and repeat; when quality is satisfactory, *standardise* the process so it is followed in future projects.

- It underlies process standards and models such as **ISO 9001**, **CMMI** (process maturity levels) and organisational process assets; quality assurance checks that the defined process and standards are actually followed (audits, reviews).
- **In software the link is weaker** than in manufacturing: software is designed, not manufactured; individual skill and experience strongly affect quality; and the process must be adapted to each project. Hence process quality is *necessary but not sufficient* – good people, appropriate methods and product reviews are still required, and excessive process bureaucracy can hurt agility.

Answer. (c) Documentation process standards (how documents are produced and approved), document standards (identification, structure, presentation, update) and document interchange standards (formats for exchanging documents). (b) Process-based quality: product quality follows from process quality, so the process is defined, assessed through product quality, improved and then standardised (ISO 9001, CMMI).

2010–2011 • Q6(b) CSE307 • 19/03/2012 • F028 MARKS 4 + 8

What are the challenges that arise in quality management of a software development process? Suppose your team is developing customized hospital management software. Identify the quality criteria for such a project.

SOLUTION

Challenges in software quality management

(i) **Quality is hard to specify** – many quality attributes (usability, maintainability) are subjective and requirements are incomplete or ambiguous; customers and developers may interpret "quality" differently.

(ii) **Conflicting attributes** – e.g. security vs. usability, performance vs. maintainability; trade-offs must be negotiated.

(iii) **Software is intangible and complex** – quality cannot be inspected visually; exhaustive testing is impossible; many quality attributes can only be measured after delivery.

(iv) **Changing requirements** during development break plans, tests and standards.

(v) **Schedule and cost pressure** – QA activities (reviews, testing) are the first to be cut.

(vi) **People and culture** – quality depends heavily on individual skill; resistance to standards/bureaucracy; distributed teams and third-party components are hard to control.

(vii) **Measurement difficulty** – choosing meaningful metrics and collecting reliable data.

Quality criteria for a customised hospital management system

Criterion	Why it matters in a hospital	Example measurable target
Correctness / safety	wrong dosage, patient or lab result can harm patients	zero critical defects; drug-interaction check on every prescription
Reliability & availability	wards, emergency and pharmacy run 24×7	availability $\geq$ 99.9%; MTBF $\geq 2\,000$ h
Security & confidentiality	medical records are highly sensitive (privacy law, ethics)	role-based access, encryption, full audit trail of record access
Data integrity & recoverability	records must never be lost or corrupted	daily backups; recovery time ≤ 1 h; no data loss
Performance	doctors and emergency staff cannot wait	95% of queries < 2 s at peak load
Usability	busy, non-technical staff; errors under stress	new staff trained in ≤ 1 day; few input errors
Interoperability	lab machines, imaging, billing, insurance, national health systems	standard interfaces (HL7/FHIR), import/export
Maintainability & configurability	a *customised* product must adapt to each hospital's departments and rules	MTTC ≤ 2 days; configurable workflows
Scalability	more beds, branches and patients over time	handle $2\times$ users without redesign
Compliance / auditability	legal and accreditation requirements	reports for regulators; traceable changes

Answer. Challenges: hard-to-specify and conflicting quality attributes, intangible complex software, changing requirements, schedule/cost pressure, people/culture and measurement difficulties. Hospital criteria: safety/correctness, reliability & availability, security & confidentiality, data integrity & recoverability, performance, usability, interoperability, maintainability/configurability, scalability and regulatory compliance.

X9

Other Topics Beyond the Slides

OUT OF SYLLABUS

2020–2021 • Q5(c) CSE325 • 12/04/2023 • C325-2021-5c MARKS 10

Both KISS and YAGNI encourage simpler code. Explain their differences with a code example.

SOLUTION

The two principles

- **KISS** — *Keep It Simple, Stupid*: for a requirement you *do* have, choose the simplest solution that works; avoid clever, convoluted or over-engineered code. It is about **how** you implement.
- **YAGNI** — *You Aren't Gonna Need It*: do not implement features, options, hooks or generality for requirements you *do not have yet*. It is about **whether** to implement something at all.

KISS example – same requirement, simpler code

Requirement: check whether a year is a leap year.

```
// Violates KISS: correct but needlessly clever and hard to read
boolean isLeap(int y) {
    return ((y & 3) == 0 && ((y % 25) != 0 || (y & 15) == 0));
}
// KISS: states the rule directly
boolean isLeap(int y) {
    if (y % 400 == 0) return true;
    if (y % 100 == 0) return false;
    return y % 4 == 0;
}
```

YAGNI example – a feature nobody asked for

Requirement: export the report as CSV.

```
// Violates YAGNI: plug-in framework for formats "we might need some day"
interface ExportFormat { String extension(); byte[] encode(Report r); }
class ExporterRegistry {
    private final java.util.Map<String, ExportFormat> formats = new java.util.HashMap<>();
    void register(ExportFormat f) { formats.put(f.extension(), f); }
    byte[] export(Report r, String ext) { return formats.get(ext).encode(r); }
}   // + XmlFormat, PdfFormat stubs, configuration file, ...

// YAGNI: implement exactly what is required now
class CsvExporter {
    String export(Report r) {
        StringBuilder sb = new StringBuilder("name,amount\n");
        for (Row row : r.rows()) sb.append(row.name()).append(',').append(row.amount()).append('
    \n');
        return sb.toString();
    }
```

```
}
```

The registry may be perfectly simple code (it does not necessarily violate KISS) — it violates YAGNI because the need does not exist. Conversely, the clever leap-year expression implements a real need, so it is a KISS (not YAGNI) problem.

Differences

	KISS	YAGNI
Question	Is this the simplest way to do it?	Do we need this at all now?
Applies to	existing, required functionality	speculative, future functionality
Typical violation	over-complicated algorithm, deep abstraction layers, clever one-liners	unused parameters/options, generic frameworks, "just in case" features
Remedy	simplify / refactor the implementation	delete it; add it when a real requirement arrives (refactor then)

Answer. KISS keeps the *implementation* of a needed feature simple (e.g. readable leap-year rule instead of bit tricks); YAGNI forbids *building* features or generality that are not yet required (e.g. a plug-in export framework when only CSV is needed). Both reduce code, but KISS is about how, YAGNI about whether.

2018–2019 • Q5(c) CSE307 • 06/11/2019 • P008 MARKS 10

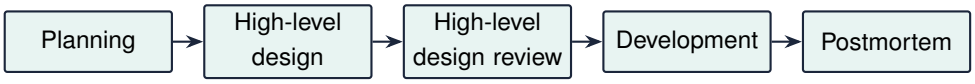
Name five framework activities encouraged in Personal Software Process (PSP).

[First half of Q5(c) — meaning of framework activity (3) — is solved in the in-syllabus Process Models chapter.]

SOLUTION

PSP

The **Personal Software Process** (Watts Humphrey) is a process for an *individual* engineer: it emphasises personal measurement of the work product and its quality, and makes the practitioner responsible for project planning and for controlling quality. It defines five framework activities:



(i) **Planning** — isolate requirements, estimate size and resources, predict defects; record metrics in worksheets/templates; identify development tasks and create a schedule.

(ii) **High-level design** — develop external specifications for each component and create the component design; build prototypes when uncertainty exists; record all issues.

(iii) **High-level design review** — apply formal verification methods to uncover errors in the design; maintain metrics for all important tasks and work results.

(iv) **Development** — refine and review the component-level design, generate code, review it, compile and test; maintain metrics.

(v) **Postmortem** — use the collected measures and metrics to determine the effectiveness of the process and to modify the process to improve it.

Answer. PSP framework activities: Planning, High-level design, High-level design review, Development, Postmortem.

2018–2019 • Q7(a) CSE307 • 06/11/2019 • P009 **MARKS 6**

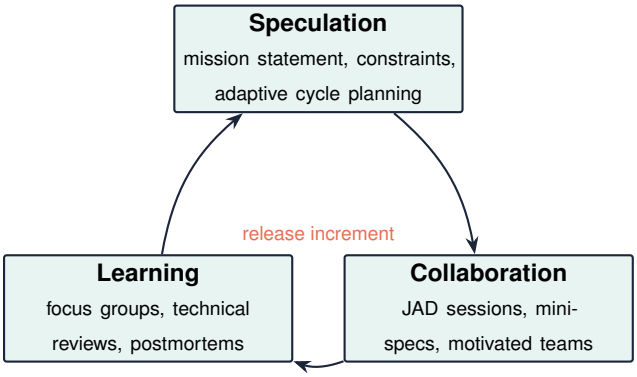
Name three phases of Adaptive Software Development (ASD).

[Second half of Q7(a) — XP phase diagram (10) — is solved in the in-syllabus Process Models chapter.]

SOLUTION

ASD

Adaptive Software Development (Jim Highsmith) is an agile method for complex systems that focuses on human collaboration and team self-organisation. Its life cycle has three phases:



(i) **Speculation** — the project is initiated and *adaptive cycle planning* is done using the mission statement, constraints and basic requirements to define the set of release cycles (increments).

(ii) **Collaboration** — motivated people work together, trusting and communicating, to multiply their talent (e.g. joint application development sessions, mini-specs).

(iii) **Learning** — the team learns from focus groups (customer feedback), formal technical reviews and postmortems, and the knowledge is used to adapt the next cycle.

Answer. ASD phases: Speculation, Collaboration, Learning.

2018–2019 • Q8(a) CSE307 • 06/11/2019 • P010 **MARKS 7**

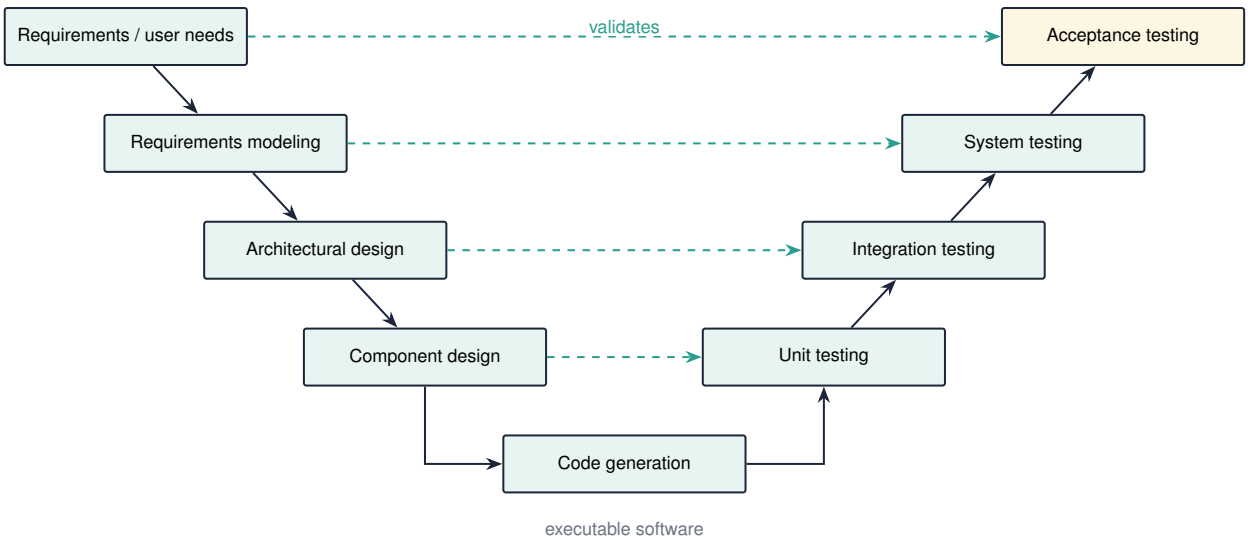
Draw the V Model in the context of Prescriptive Process Models.

[Q8(b) of this excerpt is solved in the in-syllabus Process Models chapter.]

SOLUTION

Concept

The **V-model** is a variation of the Waterfall model that makes the relationship between development actions and *quality-assurance (testing)* actions explicit. Going down the left arm, requirements are refined into progressively more detailed technical representations; at the bottom code is generated; going up the right arm, a series of tests validates each left-side model.



Explanation

- **Left side (verification/definition)**: requirements modeling → architectural design → component design; test plans for each level are prepared as each model is produced.
- **Bottom**: code generation.
- **Right side (validation)**: unit tests check component design; integration tests check the architecture; system tests check the requirements model; acceptance tests check that user needs are met.
- There is no fundamental difference from the classic life cycle; the V-model simply visualises how verification and validation actions are applied to earlier engineering work. It shares Waterfall's weaknesses (stable requirements needed, late executable).

Answer. V-model = Waterfall bent into a V: requirements, architectural and component design on the left, code at the bottom, and unit, integration, system and acceptance testing on the right, each test level validating its matching design level.

2014–2015 • Q2(b) CSE307 • 04/02/2016 • T028.3 **MARKS 12**

Consider a module that finds the maximum number in an array of n numbers. Identify the test cases for loop testing.

SOLUTION

Concept: loop testing

Loop testing is a white-box technique that concentrates on the validity of loop constructs. For a **simple loop** that may execute at most N times, the classic test set is:

(i) skip the loop entirely (0 passes);

(ii) only one pass through the loop;

(iii) two passes;

(iv) m passes, where $2 < m < N - 1$ (a typical value);

(v) $N-1$, N and $N+1$ passes (around the upper bound).

Most loop bugs are initialisation errors (seen at 0/1 pass), wrong update of the loop variable (seen at 2 passes) and off-by-one termination (seen at $N-1$, N, $N+1$).

The module

```
static final int N = 100;                // capacity of the array (assumed)

static int findMax(int[] a, int n) {
    if (n <= 0 || n > N) throw new IllegalArgumentException("n out of range");
    int max = Integer.MIN_VALUE;
    for (int i = 0; i < n; i++)          // the simple loop under test: n passes
        if (a[i] > max) max = a[i];
    return max;
}
```

NOTE The question gives no code or array limit. Assumed: the loop visits each of the n numbers once (so the number of passes is n) and the array can hold at most $N = 100$ numbers.

Loop test cases

TC	Loop rule	n	Input array	Expected output
L1	skip the loop	0	{}	error "empty array" (must *not* return `MIN_VALUE`)
L2	one pass	1	$\{-7\}$	-7 (checks initialisation with a negative value)
L3	two passes	2	$\{3, 9\}$ and $\{9, 3\}$	9 in both cases (max last / max first)
L4	m passes ($m = 50$)	50	50 values, the largest (999) at index 25	999
L5	$N-1$ passes	99	99 values, largest at the last index 98	that value
L6	N passes	100	100 values, largest at the last index 99	that value
L7	$N+1$ passes	101	101 values	error "n out of range" (no index overflow)

Useful data variations inside the same loop counts

- all elements equal (e.g. five 4's) $\Rightarrow$ 4;
- all elements negative (e.g. $-8, -3, -11$) $\Rightarrow -3$ (catches `max = 0` initialisation);
- maximum occurring twice $\Rightarrow$ that value once.

L5 and L6 place the maximum at the *last* position, so a loop written as `i < n-1` (one pass too few) is detected.

Answer. Loop tests with $n = 0$ (skip), 1, 2, a typical m (50), $N-1 = 99$, $N = 100$ and $N+1 = 101$ (rejected); place the maximum at the first and last positions and include all-negative data.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2012–2013 • Q1(d) CSE307 • 17/05/2014 • P019 **MARKS 15**

Describe the phases of "Unified Process" in brief.

2011–2012 • Q8(b) CSE307 • 23/09/2013 • P022 **MARKS 12**

Describe the four phases of the Rational Unified Process (RUP). Mention four ways in which the RUP differs from the Waterfall process model.

2014–2015 • Q4(c) CSE307 • 04/02/2016 • P015 **MARKS 10**

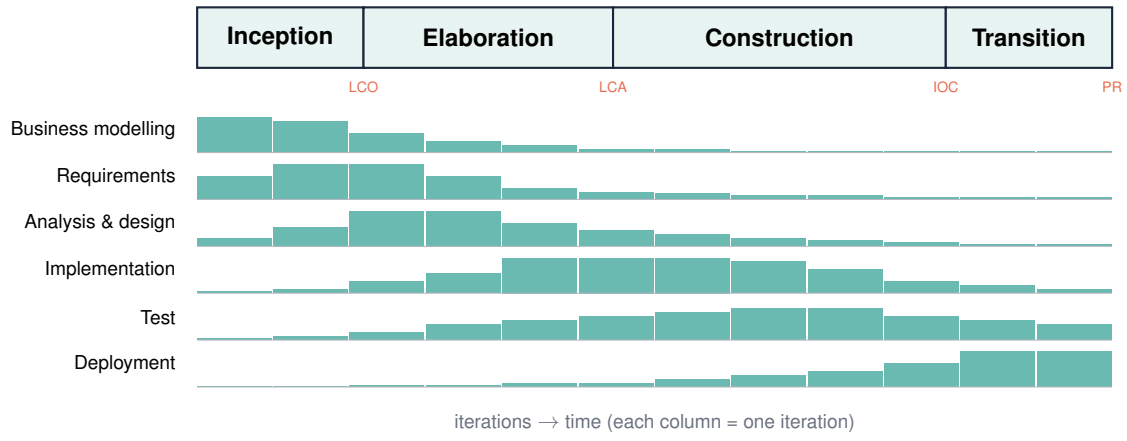
Describe unified process for software development. How does the concept of concurrency and evolution applied in unified process?

SOLUTION

NOTE The Unified Process is not covered in the CSE213 slides (which teach Waterfall, Incremental, Prototyping, Spiral, XP, Scrum and Kanban). The three questions overlap heavily, so one solution is given; the extra asks of each paper are answered in separate sub-sections.

What the Unified Process is

The **Unified Process (UP)**, and its commercial refinement the **Rational Unified Process (RUP)** by Jacobson, Booch and Rumbaugh, is a *use-case driven, architecture-centric, iterative and incremental* software process that uses UML as its modelling language. The life cycle is divided into **four phases** (the dynamic view); each phase consists of one or more **iterations**, and within every iteration several **disciplines/workflows** (business modelling, requirements, analysis & design, implementation, test, deployment) are carried out in varying proportions (the static view).



LCO = life-cycle objectives, LCA = life-cycle architecture, IOC = initial operational capability, PR = product release (milestones).

The four phases (P019, P022(b), P015(c))

(i) Inception — establish the *business case*: identify actors and the key use cases, define scope, estimate rough cost and schedule, identify major risks, sketch a candidate architecture. Milestone: life-cycle objectives — decide whether the project is worth pursuing.

(ii) Elaboration — understand the problem domain in depth: expand the use-case model (most use cases detailed), establish and *validate an executable architectural baseline*, resolve the highest risks, and produce a reliable project plan for construction. Milestone: life-cycle architecture.

(iii) Construction — design, implement and test the remaining components, iteration by iteration, building on the architecture; each iteration yields an executable increment; user documentation is prepared. Milestone: initial operational capability (beta release).

(iv) Transition — deliver the system to its user community: beta testing, user feedback, defect fixing, training, installation and support material. Milestone: product release. (Pressman also mentions a fifth *production* phase for ongoing use, monitoring and support.)

Concurrency and evolution in UP (P015(c))

- **Concurrency**: the disciplines are *not* sequential phases as in Waterfall. In any iteration, requirements, analysis & design, implementation, test and deployment work are all performed *concurrently*, only with different emphasis (the "humps" in the figure above): e.g. in elaboration some code is already written and tested to prove the architecture, while in construction requirements are still being refined. Phases may also overlap (staggered concurrency): work on the next increment's construction may begin while the previous one is in transition.
- **Evolution**: UP is iterative and incremental. Each phase consists of iterations, each of which produces an executable release that is evaluated; feedback refines requirements and design in the next iteration. The architecture is created early (elaboration) and then *evolves*; the system grows from an architectural baseline into the full product through successive increments. Risks are attacked first, so the plan also evolves as uncertainty decreases.

Four ways RUP differs from Waterfall (P022(b))

	RUP	**Waterfall**
1. Flow	iterative and incremental; each phase has several iterations producing executable releases	single linear pass; executable software only near the end
2. Activities vs phases	disciplines (requirements, design, code, test) run concurrently across all phases	each activity is a separate phase done once, in sequence
3. Risk	risk-driven: highest risks addressed early (elaboration, executable architecture)	risks discovered late, during testing/integration
4. Requirements change	changes absorbed in later iterations; use cases refined continuously	requirements frozen early; change is costly
5. Testing & integration	continuous, every iteration	big-bang at the end
6. Driver	use-case driven, architecture-centric, UML models	document driven

Answer. UP/RUP is a use-case driven, architecture-centric, iterative–incremental process with four phases: Inception (business case, scope), Elaboration (architecture baseline, major risks), Construction (build increments) and Transition (deploy to users). Concurrency: all disciplines run in every iteration with varying intensity; evolution: iterations successively grow and refine the system. Unlike Waterfall it is iterative, concurrent, risk-driven, change-tolerant and tests continuously.

2013–2014 • Q3(d) CSE307 • 08/08/2015 • T029.3 MARKS 10

Consider a module that sums up n numbers. Identify the test cases for loop testing.

SOLUTION

Concept

The summation is a **simple loop**; loop testing exercises it with 0, 1, 2, m (typical), $N-1$, N and $N+1$ passes, where N is the maximum number of iterations allowed.

The module

```
static final int N = 100;                // maximum count accepted (assumed)

static long sum(int[] a, int n) {
    if (n < 0 || n > N) throw new IllegalArgumentException("n out of range");
    long s = 0;
    for (int i = 0; i < n; i++)          // simple loop: n passes
        s += a[i];
    return s;
}
```

NOTE Assumed: at most $N = 100$ numbers; the sum of zero numbers is 0 (a valid result, unlike the maximum of zero numbers).

Loop test cases

TC	Loop rule	n	Input	Expected sum
S1	skip the loop	0	{}	0 (checks `s` is initialised to 0)
S2	one pass	1	{5}	5
S3	two passes	2	{5, −2}	3 (checks accumulation, not overwrite)
S4	m passes	50	$1, 2, \ldots, 50$	1275
S5	$N-1$ passes	99	$1, 2, \ldots, 99$	4950
S6	N passes	100	$1, 2, \ldots, 100$	5050
S7	$N+1$ passes	101	101 numbers	error “n out of range”
S8	large values	2	{2147483647, 1}	2147483648 (no `int` overflow)

Using the series $1..n$ makes the expected result easy to verify with $n(n+1)/2$; S5/S6 reveal a loop that stops one element early (`i < n-1`) because the result would differ by the last value.

Answer. Test the summation with $n = 0$ (expect 0), 1, 2, a typical 50 (1275), $N-1 = 99$ (4950), $N = 100$ (5050) and $N+1 = 101$ (rejected), plus an overflow case.

2012–2013 • Q1(a) CSE307 • 17/05/2014 • P018 MARKS $5\frac{2}{3}$

Differentiate between “Generic” and “Bespoke” software products with suitable examples.

[Q1(b) of this excerpt is solved in the in-syllabus Process Models chapter.]

SOLUTION

Definitions

- **Generic products** — stand-alone systems produced by a development organisation and sold on the open market to *any* customer who wants them.
- **Bespoke (customised) products** — systems commissioned by and developed for a *particular* customer to meet that customer's specific needs.

Differences

Aspect	Generic	Bespoke
Specification owned by	the developer (decides features, changes, releases)	the customer (decides requirements and changes)
Customers	many, anonymous market	one specific organisation
Requirements source	market research, competitor analysis	the customer's business processes
Cost to a customer	low (cost spread over many buyers)	high (customer pays the full development)
Fit to needs	general, may need adaptation by the user	exact fit to the customer's processes
Examples	MS Word, Photoshop, Android OS, accounting packages such as QuickBooks	BUET course registration system, a bank's core-banking customisation, air-traffic control system

Note: the line is blurring — generic ERP products (e.g. SAP) are often configured/customised for a specific client.

Answer. Generic products are built by the developer for the open market and their specification is controlled by the developer (e.g. MS Word); bespoke products are built for one customer, who owns the specification (e.g. a university's own registration system).

2012–2013 • Q2(c) CSE307 • 17/05/2014 • P020 MARKS 7 + 5

Draw the schematic diagram of RAD model and explain why it is a high speed adaptation of "Waterfall Model".

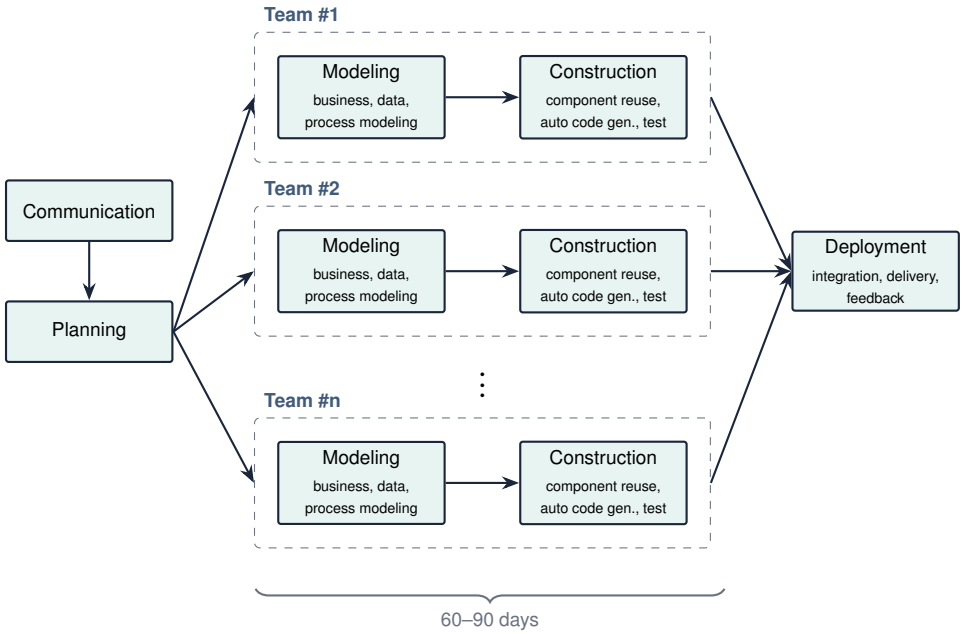
[Q2(d) of this excerpt is solved in the in-syllabus Process Models chapter.]

SOLUTION

Concept

Rapid Application Development (RAD) is an incremental model that emphasises a very short development cycle (typically 60–90 days). It is a "high-speed" adaptation of the linear sequential (Waterfall) model in which rapid development is achieved by **component-based construction** and **parallel teams**.

Schematic diagram



Phases

- **Communication** — understand the business problem and the information characteristics the software must accommodate.
- **Planning** — essential because multiple teams work in parallel on different system functions.
- **Modeling** (per team) — business modeling, data modeling and process modeling establish design representations.
- **Construction** (per team) — emphasises reuse of existing components and automatic code generation; testing of new components.
- **Deployment** — integrate the team outputs, deliver, collect feedback.

Why it is a high-speed adaptation of Waterfall

(i) It keeps the **same linear sequence** of framework activities (communication → planning → modeling → construction → deployment) — hence an adaptation of Waterfall.

(ii) The system is **split into modules developed in parallel** by separate teams, so modeling and construction of different functions proceed simultaneously instead of one after the other.

(iii) **Component reuse and automated tools/code generators** cut construction time drastically.

(iv) A strict **time box** (60–90 days) with well-understood, constrained scope forces speed.

Limitations: needs enough people for several teams, committed client and developers, a system that can be modularised, and is unsuitable for high-performance or high-technical-risk systems.

Answer. RAD: communication and planning, then several teams in parallel doing modeling (business, data, process) and construction (reuse, automatic code generation), then integrated deployment within 60–90 days. It is a high-speed Waterfall because it keeps the linear activity sequence but compresses time via parallel teams, component reuse and time-boxing.

2012–2013 • Q4(c) CSE307 • 17/05/2014 • T031 **MARKS 20**

Consider the following code of "Counting Sort" algorithm that sorts an integer array ar, whose elements are drawn from the range $[0, k)$. Here, n is the length of ar ($n > 0$). Now, draw the corresponding flow graph of "Counting Sort" algorithm and identify the independent paths. For each independent path, design suitable test cases.

```
/** Sort the n elements in ar, drawn from the values [0, k). */
int* countingSort (int *ar, int n, int k) {
1.  int i, idx = 0
2.  If(n > 1){
3.      int *B = calloc (k, sizeof (int));
4.      for (i = 0; i < n; i++) {
5.          B[ar[i]]++;
        }
6.      for (i = 0; i < k; i++) {
7.          while (B[i]-- > 0) {
8.              ar[idx++] = i;
            }
        }
9.      free(B);
    }
10. return ar;
}
```

SOLUTION

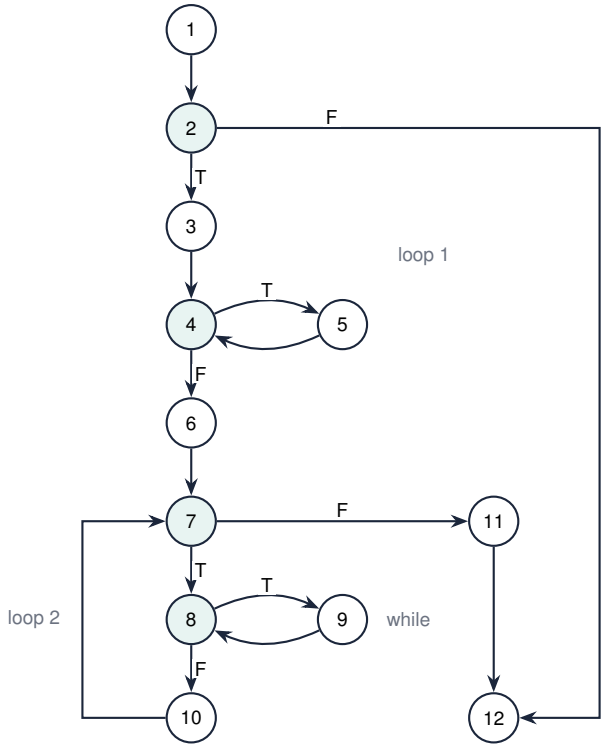
Concepts

A **flow graph** shows each statement block as a node and each transfer of control as an edge. An **independent path** is a path through the program that introduces at least one new edge (new statements or a new condition outcome). The number of independent paths in a basis set equals the **cyclomatic complexity**

$$V(G) = E - N + 2 \;=\; P + 1 \;=\; \text{number of regions.}$$

Flow-graph nodes

Node	Code	Node	Code
1	`int i, idx = 0`	7	`i < k` (second `for`)
2	`if (n > 1)`	8	`B[i]-- > 0` (`while`)
3	`B = calloc(...); i = 0`	9	`ar[idx++] = i`
4	`i < n` (first `for`)	10	`i++` (second `for`)
5	`B[ar[i]]++; i++`	11	`free(B)`
6	`i = 0` (second `for`)	12	`return ar`



Cyclomatic complexity

- Edges: 1-2, 2-3, 2-12, 3-4, 4-5, 5-4, 4-6, 6-7, 7-8, 8-9, 9-8, 8-10, 10-7, 7-11, 11-12 $\Rightarrow E = 15$; nodes $N = 12$.
- $V(G) = E - N + 2 = 15 - 12 + 2 = \mathbf{5}$.
- Check: predicate nodes $P = 4$ (nodes 2, 4, 7, 8) $\Rightarrow V(G) = P + 1 = 5$.

Basis set of independent paths

Path	Node sequence	New edge(s)
P1	1-2-12	2-12 ($n \leq 1$)
P2	1-2-3-4-6-7-11-12	2-3, 4-6, 7-11 (both `for` loops skipped)
P3	1-2-3-4-5-4-6-7-11-12	4-5, 5-4 (loop 1 runs, loop 2 skipped)
P4	1-2-3-4-5-4-6-7-8-10-7-11-12	7-8, 8-10, 10-7 (`while` false at once)
P5	1-2-3-4-5-4-6-7-8-9-8-10-7-11-12	8-9, 9-8 (`while` body runs)

Feasibility and test cases

Because of the precondition ($n > 0$, every element in $[0, k)$), some paths cannot be executed exactly as written; as in standard basis-path testing, such a path is either reported as infeasible or exercised as part of a longer execution, and robustness tests deliberately violate the precondition.

Path	Input (ar, n, k)	Feasible?	Expected result
P1	ar={5}, n=1, k=6	yes	{5} returned unchanged (a single element is sorted)
P2	—	no: $n > 1$ makes $i < n$ true at $i = 0$, so loop 1 always runs	not testable; noted as infeasible
P3	ar={0, 0}, n=2, **k=0** (robustness)	only by violating $k > 0$	function should reject k=0; the code instead writes `B[0]` of a 0-size block (out-of-bounds) ⇒ defect found
P4	ar={2, 2}, n=2, k=3	yes (values 0, 1 absent)	{2, 2}; for $i = 0, 1$ the `while` is false immediately (edge 8-10)
P5	ar={2, 0, 1}, n=3, k=3	yes	{0, 1, 2}; `while` body executes for every i

An extra (non-basis) test with duplicates and reverse order, ar={3, 1, 3, 0, 1}, n=5, k=4 ⇒ {0, 1, 1, 3, 3}, checks that `idx` keeps advancing across different values.

EXAM TIP Observations worth writing in the exam: the code never checks the result of `calloc` (NULL when memory is short) and never validates that `ar[i]` lies in $[0, k)$ – the P3 robustness test exposes the second problem.

Answer. Flow graph with 12 nodes and 15 edges, $V(G) = 5$. Independent paths P1–P5 (above); test cases: P1 n=1; P2 infeasible; P3 only via invalid k=0 (reveals missing validation); P4 ar={2,2}, k=3; P5 ar={2,0,1}, k=3 ⇒ {0,1,2}.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2012–2013 • Q8(b) CSE307 • 17/05/2014 • R034.2 **MARKS 5**

What does the abbreviation CRC stand for? Name two benefits derived from the use of CRC when used for class modeling?

2011–2012 • Q2(d) CSE307 • 23/09/2013 • R036.2 **MARKS 10**

What is a CRC card? Discuss its usages.

SOLUTION

Definition

CRC stands for **Class – Responsibility – Collaborator**. A CRC card is an index card (traditionally 4″×6″) used in object-oriented analysis to discover and document classes. Each card describes one class:

- **Class name** – written at the top;
- **Responsibilities** (left side) – what the class *knows* (its data) and what it *does* (its services);
- **Collaborators** (right side) – other classes whose help it needs to fulfil a responsibility (i.e. the classes it sends messages to).

Class: Loan	
Responsibilities	**Collaborators**
know the borrowed copy	BookCopy
know the borrower	User
know issue and due date	Librarian
calculate the fine for late return	
close itself when the copy is returned	

How CRC cards are used (CRC modelling session)

(i) A small group (users, analysts, developers) **brainstorms candidate classes** from the problem description and writes one card per class.

(ii) The team **role-plays scenarios / use cases**: a member "becomes" a class, and as the scenario proceeds, responsibilities are added to the card that must perform them and collaborators are noted when help is needed.

(iii) Cards that end with no responsibilities are thrown away; overloaded cards are split; cards are arranged on the table so that collaborating classes lie close together.

(iv) The resulting cards are the input for the **class diagram** (classes, operations and associations) and for interaction (sequence/communication) diagrams.

Benefits of CRC for class modelling

- **Simple, cheap and informal** – no tool or notation to learn; non-technical users can take part, which improves communication between users and developers.
- **Responsibility-driven design** – it forces thinking about *behaviour* and distribution of responsibilities, giving cohesive classes with low coupling.
- **Early validation of the model** – role-playing scenarios quickly exposes missing classes, missing responsibilities or wrong collaborations before any diagram or code is produced.
- **Easy to change** – cards can be torn up, rewritten and rearranged, encouraging iteration.
- **Smooth transition to UML** – class name → class, responsibilities → attributes/operations, collaborators → associations/dependencies.

Answer. CRC = Class–Responsibility–Collaborator. A CRC card records a class's name, its responsibilities and its collaborators; cards are used in group role-playing sessions to discover classes. Benefits: simple and user-friendly, responsibility-driven (cohesive design), early validation, and a direct path to the class diagram.

2011–2012 • Q7(b) CSE307 • 23/09/2013 • T035 **MARKS 25**

Draw the control flow graph for the following function. Then determine the cyclomatic complexity and the basis set of independent paths of the CFG. Finally prepare test cases that force execution of each path in the basis set.

```
public double calculate(int amount)
{
```

```
    double rushCharge = 0;
    if (nextday.equals("yes"))
    {
        rushCharge = 14.50;
    }
    double tax = amount * .0725;
    if (amount >= 1000)
    {
        shipcharge = amount * .06 + rushCharge;
    }
    else if (amount >= 200)
    {
        shipcharge = amount * .08 + rushCharge;
    }
    else if (amount >= 100)
    {
        shipcharge = 13.25 + rushCharge;
    }
    else if (amount >= 50)
    {
        shipcharge = 9.95 + rushCharge;
    }
    else if (amount >= 25)
    {
        shipcharge = 7.25 + rushCharge;
    }
    else
    {
        shipcharge = 5.25 + rushCharge;
    }
    total = amount + tax + shipcharge;
    return total;
} //end calculate
```

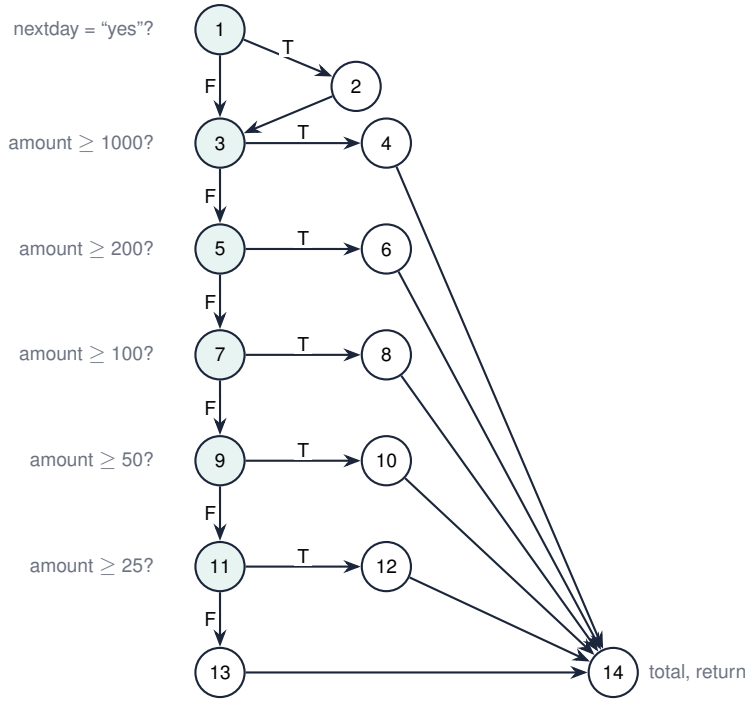
SOLUTION

NOTE `nextday`, `shipcharge` and `total` are not declared in the function; they are taken to be fields of the enclosing class (`nextday` is a `String`).

Step 1: number the nodes

Node	Code	Node	Code
1	`rushCharge = 0; if (nextday.equals("yes"))`	8	`shipcharge = 13.25 + rush`
2	`rushCharge = 14.50`	9	`if (amount >= 50)`
3	`tax = amount*.0725; if (amount >= 1000)`	10	`shipcharge = 9.95 + rush`
4	`shipcharge = amount*.06 + rush`	11	`if (amount >= 25)`
5	`if (amount >= 200)`	12	`shipcharge = 7.25 + rush`
6	`shipcharge = amount*.08 + rush`	13	`shipcharge = 5.25 + rush`
7	`if (amount >= 100)`	14	`total = ...; return total`

Step 2: control flow graph



Step 3: cyclomatic complexity

- Nodes $N = 14$. Edges: 1-2, 1-3, 2-3, 3-4, 3-5, 4-14, 5-6, 5-7, 6-14, 7-8, 7-9, 8-14, 9-10, 9-11, 10-14, 11-12, 11-13, 12-14, 13-14 $\Rightarrow E = 19$.
- $V(G) = E - N + 2 = 19 - 14 + 2 = \mathbf{7}$.
- Predicate nodes: 1, 3, 5, 7, 9, 11 $\Rightarrow P = 6$, $V(G) = P + 1 = 7$; the graph also has 7 regions (6 enclosed + the outer one).

Step 4: basis set of independent paths

Path	Nodes	What is new
P1	1-3-4-14	no rush, amount $\geq$ 1000 (baseline)
P2	1-2-3-4-14	rush charge branch (1-2, 2-3)
P3	1-3-5-6-14	200 $\leq$ amount $<$ 1000
P4	1-3-5-7-8-14	100 $\leq$ amount $<$ 200
P5	1-3-5-7-9-10-14	50 $\leq$ amount $<$ 100
P6	1-3-5-7-9-11-12-14	25 $\leq$ amount $<$ 50
P7	1-3-5-7-9-11-13-14	amount $<$ 25

Step 5: test cases forcing each path

tax $= 0.0725 \times$ amount, total $=$ amount $+$ tax $+$ shipcharge.

Path	nextday	amount	tax ; shipcharge	Expected total
P1	“no”	1500	$108.75 ; 1500 \times 0.06 = 90.00$	1698.75
P2	“yes”	1500	$108.75 ; 90.00 + 14.50 = 104.50$	1713.25
P3	“no”	500	$36.25 ; 500 \times 0.08 = 40.00$	576.25
P4	“no”	150	10.875 ; 13.25	174.125
P5	“no”	75	5.4375 ; 9.95	90.3875
P6	“no”	30	2.175 ; 7.25	39.425
P7	“no”	10	0.725 ; 5.25	15.975

Every edge of the CFG is executed by these seven tests. A JUnit check for P2, for instance:

```
@Test void rushOrderOver1000() {
    Order o = new Order(); o.nextday = "yes";
    assertEquals(1713.25, o.calculate(1500), 1e-9);
}
```

EXAM TIP Basis-path tests use one value inside each range; adding boundary values (999/1000, 199/200, 99/100, 49/50, 24/25) catches >= vs > mistakes that the path tests alone would miss.

Answer. CFG with 14 nodes and 19 edges; $V(G) = 19 - 14 + 2 = 7$ (= 6 predicates + 1). Basis paths P1–P7; tests: (no, 1500)→1698.75, (yes, 1500)→1713.25, (no, 500)→576.25, (no, 150)→174.125, (no, 75)→90.3875, (no, 30)→39.425, (no, 10)→15.975.